

FitlyzerC

Generated by Doxygen 1.17.0

1 FitlyzerC	1
1.1 Download	1
1.2 What is FitlyzerC?	1
1.3 Features	1
1.3.1 Activity Analysis	1
1.3.2 Interactive Charts	2
1.3.3 Mapping	2
1.3.4 Athlete Management	2
1.3.5 Video Export	3
1.3.6 Cross Platform	3
1.4 Screenshots	3
1.5 Current Development Status	3
1.5.1 Planned Areas	3
1.6 Technology Stack	4
1.7 Project Structure	4
1.8 Build Requirements	4
1.8.1 Required Software	4
1.9 Building	4
1.9.1 macOS	4
1.9.1.1 Dependencies	4
1.9.1.2 Configure	5
1.9.1.3 Build	5
1.9.1.4 Create DMG	5
1.9.2 Windows	5
1.9.2.1 Configure	5
1.9.2.2 Build	5
1.9.2.3 Create Installer	5
1.9.3 Linux	5
1.9.3.1 Arch Linux	5
1.9.3.2 Configure	5
1.9.3.3 Build	5
1.9.3.4 AppImage	5
1.9.4 One-Command Packaging	6
1.9.5 Garmin FIT SDK	6
1.9.6 FFmpeg Bundling	6
1.9.7 Running	6
1.9.8 Contributing	7
1.9.9 License	7
2 Namespace Index	9
2.1 Namespace List	9
3 Hierarchical Index	11

3.1 Class Hierarchy	11
4 Class Index	15
4.1 Class List	15
5 File Index	19
5.1 File List	19
6 Namespace Documentation	23
6.1 ActivityAnalysisFlags Namespace Reference	23
6.1.1 Detailed Description	23
6.1.2 Variable Documentation	23
6.1.2.1 HasClimbs	23
6.1.2.2 HasHistogram	23
6.1.2.3 HasIntervals	24
6.1.2.4 HasPowerCurve	24
6.1.2.5 HasVideo	24
6.1.2.6 None	24
6.2 AnalysisVersions Namespace Reference	24
6.2.1 Detailed Description	24
6.2.2 Variable Documentation	24
6.2.2.1 ClimbDetection	24
6.2.2.2 IntervalDetection	24
6.3 anonymous_namespace{ActivityBrowser.cpp} Namespace Reference	25
6.3.1 Variable Documentation	25
6.3.1.1 kIsoDateRole	25
6.3.1.2 kNotesRole	25
6.3.1.3 kWeatherRole	25
6.4 anonymous_namespace{AnalysisWorker.cpp} Namespace Reference	25
6.4.1 Function Documentation	25
6.4.1.1 applySqlitePragmas()	25
6.5 anonymous_namespace{AthleteDialog.cpp} Namespace Reference	25
6.5.1 Function Documentation	26
6.5.1.1 displayDateOrRaw()	26
6.5.1.2 toIsoOrRaw()	26
6.5.2 Variable Documentation	26
6.5.2.1 kEntryIdRole	26
6.6 anonymous_namespace{ClimbDetector.cpp} Namespace Reference	26
6.6.1 Function Documentation	26
6.6.1.1 buildFilteredAltitudeMeters()	26
6.6.1.2 clampValue()	26
6.6.1.3 estimateSpeedKmhAt()	26
6.6.1.4 isFinite()	27

6.6.1.5 segmentMeters()	27
6.7 anonymous_namespace{ClimbMetrics.cpp} Namespace Reference	27
6.7.1 Function Documentation	27
6.7.1.1 classifyClimbCategory()	27
6.7.1.2 classifyClimbShape()	27
6.7.1.3 computeLinearRegressionSlope()	27
6.7.1.4 computeNormalizedPower()	28
6.7.1.5 computeSliceStats()	28
6.7.1.6 isFinite()	28
6.8 anonymous_namespace{ClimbRepository.cpp} Namespace Reference	28
6.8.1 Function Documentation	28
6.8.1.1 recordFromQuery()	28
6.8.2 Variable Documentation	28
6.8.2.1 kSelectCols	28
6.9 anonymous_namespace{ColorProvider.cpp} Namespace Reference	29
6.9.1 Function Documentation	29
6.9.1.1 interpolateColor()	29
6.10 anonymous_namespace{DateFormatter.cpp} Namespace Reference	29
6.10.1 Function Documentation	29
6.10.1.1 datePatternForFormat()	29
6.11 anonymous_namespace{FfmpegPath.cpp} Namespace Reference	29
6.11.1 Function Documentation	29
6.11.1.1 absoluteExecutablePath()	29
6.12 anonymous_namespace{FitParser.cpp} Namespace Reference	29
6.12.1 Function Documentation	30
6.12.1.1 fitTimestampToIsoUtc()	30
6.13 anonymous_namespace{ImportWorker.cpp} Namespace Reference	30
6.13.1 Function Documentation	30
6.13.1.1 applySqlitePragmas()	30
6.14 anonymous_namespace{MapFitMath.cpp} Namespace Reference	30
6.14.1 Function Documentation	30
6.14.1.1 latToTileY()	30
6.14.1.2 normalizeLongitude()	30
6.14.1.3 tileYToLat()	30
6.14.2 Variable Documentation	31
6.14.2.1 kMaxMercatorLat	31
6.14.2.2 kMinLatSpan	31
6.14.2.3 kMinLonSpan	31
6.14.2.4 kPi	31
6.14.2.5 kTilePx	31
6.15 anonymous_namespace{RideChartWidget.cpp} Namespace Reference	31
6.15.1 Function Documentation	31

6.15.1.1 hitTestClimbHandles()	31
6.15.2 Variable Documentation	32
6.15.2.1 kClimbHandleCenterOffsetYPx	32
6.15.2.2 kClimbHandleGrabRadiusPx	32
6.15.2.3 kClimbHandleRadiusPx	32
6.16 anonymous_namespace{TileCache.cpp} Namespace Reference	32
6.16.1 Function Documentation	32
6.16.1.1 defaultDiskCacheRoot()	32
6.17 anonymous_namespace{VideoExportDialog.cpp} Namespace Reference	32
6.17.1 Function Documentation	32
6.17.1.1 formatVideoLength()	32
6.17.1.2 makeDefaultFilename()	32
6.17.1.3 mapStyleName()	32
6.18 anonymous_namespace{VideoRenderJob.cpp} Namespace Reference	33
6.18.1 Function Documentation	33
6.18.1.1 computeTileViewport()	33
6.18.1.2 fmtDuration()	33
6.18.1.3 formatLegendValue()	33
6.18.1.4 interpolateGpsAtTime()	34
6.18.1.5 latLonToTile()	34
6.18.1.6 nearestRecordIndex()	34
6.18.1.7 routeColor()	34
6.18.1.8 tilesForViewport()	34
6.18.1.9 tileToScreen()	34
6.18.1.10 zoneLegendText()	34
6.18.2 Variable Documentation	35
6.18.2.1 kPi	35
6.18.2.2 kTilePx	35
6.18.2.3 kVideoExportTileCache	35
6.19 anonymous_namespace{VideoTileProvider.cpp} Namespace Reference	35
6.19.1 Function Documentation	35
6.19.1.1 defaultPrimaryTileCacheRoot()	35
6.19.1.2 defaultVideoExportCacheRoot()	35
6.20 anonymous_namespace{WorkoutController.cpp} Namespace Reference	35
6.20.1 Function Documentation	35
6.20.1.1 activityDateForFtp()	35
6.20.1.2 resolveAthleteFtpForDate()	36
6.21 anonymous_namespace{ZoneCalculator.cpp} Namespace Reference	36
6.21.1 Function Documentation	36
6.21.1.1 sampleAvailable()	36
6.21.1.2 sampleDurationSeconds()	36
6.21.1.3 valueForMetric()	36

6.22 Platform Namespace Reference	36
6.22.1 Detailed Description	36
6.22.2 Function Documentation	37
6.22.2.1 applicationDataDirectory()	37
6.22.2.2 defaultBackupPath()	37
6.22.2.3 defaultDatabaseDirectory()	37
6.22.2.4 defaultDatabasePath()	37
6.23 SqlQueryHelper Namespace Reference	37
6.23.1 Detailed Description	37
6.23.2 Function Documentation	38
6.23.2.1 bindBoolValue()	38
6.23.2.2 selectExistsEver()	38
6.23.2.3 selectExistsLocked()	39
6.23.2.4 selectExistsNonDeleted()	39
6.23.3 Variable Documentation	40
6.23.3.1 kWhereDeleted	40
6.23.3.2 kWhereLocked	40
6.23.3.3 kWhereNotDeleted	40
7 Class Documentation	41
7.1 AboutDialog Class Reference	41
7.1.1 Detailed Description	42
7.1.2 Constructor & Destructor Documentation	42
7.1.2.1 AboutDialog()	42
7.2 Activity Struct Reference	42
7.2.1 Detailed Description	43
7.2.2 Member Function Documentation	44
7.2.2.1 isValid()	44
7.2.3 Member Data Documentation	44
7.2.3.1 analysisFlags	44
7.2.3.2 analysisVersion	44
7.2.3.3 athleteId	44
7.2.3.4 avgCadence	44
7.2.3.5 avgHr	44
7.2.3.6 avgPower	44
7.2.3.7 avgSpeed	44
7.2.3.8 bike	45
7.2.3.9 distanceM	45
7.2.3.10 durationSec	45
7.2.3.11 elevationGain	45
7.2.3.12 endTime	45
7.2.3.13 equipment	45

7.2.3.14 fatigue	45
7.2.3.15 fileName	45
7.2.3.16 fingerprint	45
7.2.3.17 fitHash	46
7.2.3.18 id	46
7.2.3.19 importedAt	46
7.2.3.20 importSource	46
7.2.3.21 maxHr	46
7.2.3.22 maxPower	46
7.2.3.23 normalizedPower	46
7.2.3.24 notes	46
7.2.3.25 rpe	47
7.2.3.26 sleepHours	47
7.2.3.27 sport	47
7.2.3.28 startTime	47
7.2.3.29 temperature	47
7.2.3.30 weather	47
7.2.3.31 weatherNotes	47
7.2.3.32 weightKg	47
7.2.3.33 wind	47
7.3 ActivityAnalysisCacheRepository Class Reference	48
7.3.1 Detailed Description	48
7.3.2 Constructor & Destructor Documentation	48
7.3.2.1 ActivityAnalysisCacheRepository()	48
7.3.3 Member Function Documentation	48
7.3.3.1 getCache()	48
7.3.3.2 hasCache()	49
7.3.3.3 invalidateCache()	49
7.3.3.4 invalidateKey()	49
7.3.3.5 setCache()	50
7.3.4 Member Data Documentation	50
7.3.4.1 m_db	50
7.4 ActivityAnalysisSettings Struct Reference	50
7.4.1 Detailed Description	51
7.4.2 Member Function Documentation	51
7.4.2.1 isValid()	51
7.4.3 Member Data Documentation	51
7.4.3.1 activityId	51
7.4.3.2 climbMinGain	51
7.4.3.3 climbMinGradient	51
7.4.3.4 climbMinLength	51
7.4.3.5 intervalRestThreshold	52

7.4.3.6 intervalWorkThreshold	52
7.5 ActivityAnalysisSettingsRepository Class Reference	52
7.5.1 Detailed Description	52
7.5.2 Constructor & Destructor Documentation	52
7.5.2.1 ActivityAnalysisSettingsRepository()	52
7.5.3 Member Function Documentation	53
7.5.3.1 insertOrUpdate()	53
7.5.3.2 settingsForActivity()	53
7.5.4 Member Data Documentation	53
7.5.4.1 m_db	53
7.6 ActivityBrowser Class Reference	53
7.6.1 Detailed Description	55
7.6.2 Constructor & Destructor Documentation	55
7.6.2.1 ActivityBrowser()	55
7.6.3 Member Function Documentation	55
7.6.3.1 applyFilters()	55
7.6.3.2 deleteActivity()	55
7.6.3.3 dragEnterEvent()	55
7.6.3.4 dropEvent()	55
7.6.3.5 editNotes()	55
7.6.3.6 focusSearch()	55
7.6.3.7 refresh()	56
7.6.3.8 showContextMenu()	56
7.6.4 Member Data Documentation	56
7.6.4.1 m_athleteId	56
7.6.4.2 m_dbManager	56
7.6.4.3 m_emptyStateLabel	56
7.6.4.4 m_rangeFilter	56
7.6.4.5 m_searchEdit	56
7.6.4.6 m_table	56
7.7 anonymous_namespace{ActivityBrowser.cpp}::ActivityDataTableItem Class Reference	57
7.7.1 Detailed Description	57
7.7.2 Member Function Documentation	57
7.7.2.1 operator<()	57
7.8 ActivityRepository Class Reference	58
7.8.1 Detailed Description	58
7.8.2 Constructor & Destructor Documentation	58
7.8.2.1 ActivityRepository()	58
7.8.3 Member Function Documentation	59
7.8.3.1 deleteActivity()	59
7.8.3.2 getActivity()	59
7.8.3.3 hasFingerprintMatch()	59

7.8.3.4 hasFitHash()	59
7.8.3.5 insertActivity()	60
7.8.3.6 listActivities()	61
7.8.3.7 setImportSource()	61
7.8.3.8 updateActivityProperties()	61
7.8.3.9 updateAnalysisFlags()	62
7.8.3.10 updateFingerprint()	62
7.8.3.11 updateNotes()	63
7.8.4 Member Data Documentation	63
7.8.4.1 m_db	63
7.9 AnalysisQueue Class Reference	63
7.9.1 Detailed Description	65
7.9.2 Constructor & Destructor Documentation	65
7.9.2.1 AnalysisQueue()	65
7.9.2.2 ~AnalysisQueue()	66
7.9.3 Member Function Documentation	66
7.9.3.1 enqueue()	66
7.9.3.2 isBusy()	66
7.9.3.3 onTaskCompleted	66
7.9.3.4 onTaskFailed	66
7.9.3.5 pendingCount()	66
7.9.3.6 pendingCountChanged	67
7.9.3.7 processNext()	67
7.9.3.8 setDatabasePath()	67
7.9.3.9 taskCompleted	67
7.9.3.10 taskFailed	68
7.9.4 Member Data Documentation	68
7.9.4.1 m_busy	68
7.9.4.2 m_dbPath	68
7.9.4.3 m_queue	68
7.9.4.4 m_thread	68
7.9.4.5 m_worker	68
7.10 AnalysisWorker Class Reference	69
7.10.1 Detailed Description	70
7.10.2 Constructor & Destructor Documentation	70
7.10.2.1 AnalysisWorker()	70
7.10.3 Member Function Documentation	70
7.10.3.1 processTask	70
7.10.3.2 setDatabasePath()	71
7.10.3.3 taskCompleted	71
7.10.3.4 taskFailed	72
7.10.4 Member Data Documentation	72

7.10.4.1 m_dbPath	72
7.11 AppSettings Class Reference	73
7.11.1 Detailed Description	73
7.11.2 Member Function Documentation	73
7.11.2.1 dateFormat()	73
7.11.2.2 dateFormatKey()	73
7.11.2.3 instance()	73
7.11.2.4 sanitizeDateFormat()	74
7.11.2.5 setDateFormat()	74
7.11.2.6 setTileCacheSize()	74
7.11.2.7 tileCacheSize()	74
7.11.2.8 tileCacheSizeKey()	75
7.12 Athlete Struct Reference	75
7.12.1 Detailed Description	75
7.12.2 Member Function Documentation	75
7.12.2.1 fullName()	75
7.12.2.2 isValid()	76
7.12.3 Member Data Documentation	76
7.12.3.1 createdAt	76
7.12.3.2 dateOfBirth	76
7.12.3.3 email	76
7.12.3.4 firstName	76
7.12.3.5 ftpWatts	76
7.12.3.6 heightCm	77
7.12.3.7 id	77
7.12.3.8 lastName	77
7.13 AthleteDialog Class Reference	77
7.13.1 Detailed Description	78
7.13.2 Constructor & Destructor Documentation	79
7.13.2.1 AthleteDialog()	79
7.13.3 Member Function Documentation	79
7.13.3.1 addFtpRow	79
7.13.3.2 addWeightRow	79
7.13.3.3 deleteFtpRow	79
7.13.3.4 deleteWeightRow	79
7.13.3.5 loadAthlete()	79
7.13.3.6 onAccepted	80
7.13.3.7 populateFtpTable()	80
7.13.3.8 populateWeightTable()	80
7.13.3.9 savedAthleteld()	80
7.13.4 Member Data Documentation	80
7.13.4.1 m_athleteld	80

7.13.4.2 m_dob	80
7.13.4.3 m_email	80
7.13.4.4 m_firstName	80
7.13.4.5 mftp	80
7.13.4.6 mftpTable	80
7.13.4.7 m_height	81
7.13.4.8 m_lastName	81
7.13.4.9 m_repo	81
7.13.4.10 m_savedId	81
7.13.4.11 m_weightTable	81
7.14 AthleteHeaderWidget Class Reference	81
7.14.1 Detailed Description	82
7.14.2 Constructor & Destructor Documentation	82
7.14.2.1 AthleteHeaderWidget()	82
7.14.3 Member Function Documentation	82
7.14.3.1 setRideMetrics()	82
7.14.3.2 setSummary()	82
7.14.4 Member Data Documentation	83
7.14.4.1 m_metricsLabel	83
7.14.4.2 m_nameLabel	83
7.14.4.3 m_statsLabel	83
7.15 AthleteListDialog Class Reference	83
7.15.1 Detailed Description	84
7.15.2 Constructor & Destructor Documentation	84
7.15.2.1 AthleteListDialog()	84
7.15.3 Member Function Documentation	84
7.15.3.1 deleteAthlete	84
7.15.3.2 editAthlete	84
7.15.3.3 newAthlete	84
7.15.3.4 refresh	84
7.15.3.5 selectAthlete	85
7.15.3.6 selectedAthleteld()	85
7.15.4 Member Data Documentation	85
7.15.4.1 m_deleteBtn	85
7.15.4.2 m_editBtn	85
7.15.4.3 m_repo	85
7.15.4.4 m_selectBtn	85
7.15.4.5 m_selectedId	85
7.15.4.6 m_table	85
7.16 AthleteRepository Class Reference	85
7.16.1 Detailed Description	86
7.16.2 Constructor & Destructor Documentation	86

7.16.2.1 AthleteRepository()	86
7.16.3 Member Function Documentation	86
7.16.3.1 addFtpEntry()	86
7.16.3.2 addWeightEntry()	87
7.16.3.3 deleteAthlete()	87
7.16.3.4 deleteFtpEntry()	87
7.16.3.5 deleteWeightEntry()	88
7.16.3.6 getAthlete()	88
7.16.3.7 getFtpForDate()	89
7.16.3.8 getFtpHistory()	89
7.16.3.9 getWeightHistory()	89
7.16.3.10 insertAthlete()	90
7.16.3.11 listAthletes()	90
7.16.3.12 updateAthlete()	90
7.16.4 Member Data Documentation	90
7.16.4.1 m_db	90
7.17 CalendarWidget Class Reference	91
7.17.1 Detailed Description	92
7.17.2 Constructor & Destructor Documentation	92
7.17.2.1 CalendarWidget()	92
7.17.3 Member Function Documentation	92
7.17.3.1 addPlannedWorkout	92
7.17.3.2 deleteSelectedWorkout	92
7.17.3.3 editSelectedWorkout	92
7.17.3.4 onDateChanged	92
7.17.3.5 recolorMonth	92
7.17.3.6 refresh()	92
7.17.3.7 reloadDayList()	93
7.17.3.8 setAthleteld()	93
7.17.3.9 setDatabaseManager()	93
7.17.3.10 updateMonthFormatting()	93
7.17.4 Member Data Documentation	93
7.17.4.1 m_addButton	93
7.17.4.2 m_athleteld	93
7.17.4.3 m_calendar	93
7.17.4.4 m_colorByCombo	93
7.17.4.5 m_dayList	93
7.17.4.6 m_dbManager	93
7.17.4.7 m_deleteButton	93
7.17.4.8 m_editButton	94
7.18 Climb Struct Reference	94
7.18.1 Detailed Description	95

7.18.2 Member Data Documentation	95
7.18.2.1 aerobicDecouplingPct	95
7.18.2.2 averageCadence	95
7.18.2.3 averageGradient	96
7.18.2.4 averageHeartRate	96
7.18.2.5 averagePower	96
7.18.2.6 averageSpeed	96
7.18.2.7 averageWattsPerKg	96
7.18.2.8 category	96
7.18.2.9 climbCategory	96
7.18.2.10 climbScore	96
7.18.2.11 difficultyScore	96
7.18.2.12 durationSeconds	97
7.18.2.13 elevationGainM	97
7.18.2.14 endDistanceKm	97
7.18.2.15 endElevationM	97
7.18.2.16 endSeconds	97
7.18.2.17 gradePercentile	97
7.18.2.18 hrDriftPct	97
7.18.2.19 id	97
7.18.2.20 lengthKm	98
7.18.2.21 maximumGradient	98
7.18.2.22 name	98
7.18.2.23 normalizedPower	98
7.18.2.24 powerFadePct	98
7.18.2.25 powerFirstHalf	98
7.18.2.26 powerSecondHalf	98
7.18.2.27 quarterMetrics	98
7.18.2.28 shapeClass	98
7.18.2.29 startDistanceKm	99
7.18.2.30 startElevationM	99
7.18.2.31 startSeconds	99
7.18.2.32 vam	99
7.18.2.33 wattsPerKgRank	99
7.19 ClimbDetector Class Reference	99
7.19.1 Detailed Description	100
7.19.2 Member Typedef Documentation	100
7.19.2.1 Config	100
7.19.3 Member Function Documentation	100
7.19.3.1 buildCumulativeDistanceMeters()	100
7.19.3.2 categorizeClimb()	101
7.19.3.3 computeAdaptiveSmoothingWindowsMeters()	101

7.19.3.4 computeClimbScore()	102
7.19.3.5 computeLocalGradientPct()	102
7.19.3.6 computeVAM()	103
7.19.3.7 detect()	103
7.19.3.8 mergeFragmentedClimbs()	105
7.19.3.9 smoothAltitudeByDistance()	105
7.20 ClimbDetectorConfig Struct Reference	106
7.20.1 Detailed Description	108
7.20.2 Member Data Documentation	108
7.20.2.1 adaptiveSmoothingMaxMeters	108
7.20.2.2 adaptiveSmoothingMinMeters	108
7.20.2.3 altitudeSpikeVerticalSpeedMps	108
7.20.2.4 continueGradient	108
7.20.2.5 durationScoreWeight	108
7.20.2.6 elevationSmoothingMeters	108
7.20.2.7 enableClimbMerging	108
7.20.2.8 endGradient	109
7.20.2.9 gainScoreWeight	109
7.20.2.10 gradePersistenceMinMeters	109
7.20.2.11 gradePersistenceThreshold	109
7.20.2.12 gradeScoreWeight	109
7.20.2.13 gradientWindowHalfMeters	109
7.20.2.14 maxDipDistanceMeters	109
7.20.2.15 maxDipMeters	109
7.20.2.16 maxDipPctOfGain	109
7.20.2.17 maxPlateauDistanceMeters	110
7.20.2.18 maxSmallDipsCount	110
7.20.2.19 maxStartInterruptionMeters	110
7.20.2.20 mergeMaxGapElevationM	110
7.20.2.21 mergeMaxGapMeters	110
7.20.2.22 minAverageGradient	110
7.20.2.23 minClimbScore	110
7.20.2.24 minElevationGainM	110
7.20.2.25 minLengthKm	110
7.20.2.26 minStartContinuousDistanceMeters	111
7.20.2.27 plateauMinGradient	111
7.20.2.28 requireGradePersistence	111
7.20.2.29 smallDipThresholdMeters	111
7.20.2.30 startGradient	111
7.20.2.31 useAdaptiveSmoothing	111
7.20.2.32 useScoringDetection	111
7.20.2.33 vamScoreBoost	111

7.21 ClimbEditCommand Class Reference	112
7.21.1 Detailed Description	113
7.21.2 Constructor & Destructor Documentation	113
7.21.2.1 ClimbEditCommand()	113
7.21.3 Member Function Documentation	113
7.21.3.1 apply()	113
7.21.3.2 description()	113
7.21.3.3 redo()	113
7.21.3.4 undo()	114
7.21.4 Member Data Documentation	114
7.21.4.1 m_after	114
7.21.4.2 m_before	114
7.21.4.3 m_connName	114
7.21.4.4 m_refresh	114
7.22 anonymous_namespace{RideChartWidget.cpp}::ClimbHandleHitResult Struct Reference	114
7.22.1 Detailed Description	114
7.22.2 Member Data Documentation	114
7.22.2.1 hit	114
7.22.2.2 startHandle	114
7.23 ClimbMetrics Class Reference	115
7.23.1 Detailed Description	115
7.23.2 Member Function Documentation	115
7.23.2.1 build()	115
7.24 ClimbNameTableItem Class Reference	116
7.24.1 Detailed Description	117
7.24.2 Constructor & Destructor Documentation	117
7.24.2.1 ClimbNameTableItem()	117
7.24.3 Member Function Documentation	117
7.24.3.1 operator<()	117
7.24.3.2 trailingNumber()	117
7.25 ClimbQuarterMetrics Struct Reference	117
7.25.1 Detailed Description	118
7.25.2 Member Data Documentation	118
7.25.2.1 averageCadence	118
7.25.2.2 averageHeartRate	118
7.25.2.3 averagePower	118
7.26 ClimbRecord Struct Reference	118
7.26.1 Detailed Description	119
7.26.2 Member Data Documentation	119
7.26.2.1 activityId	119
7.26.2.2 algorithmVersion	120
7.26.2.3 averageGradient	120

7.26.2.4 avgCadence	120
7.26.2.5 avgHr	120
7.26.2.6 avgPower	120
7.26.2.7 deleted	120
7.26.2.8 elevationGainM	120
7.26.2.9 endSeconds	120
7.26.2.10 favorite	120
7.26.2.11 id	121
7.26.2.12 lengthKm	121
7.26.2.13 locked	121
7.26.2.14 name	121
7.26.2.15 notes	121
7.26.2.16 np	121
7.26.2.17 originalEndSeconds	121
7.26.2.18 originalStartSeconds	121
7.26.2.19 rating	122
7.26.2.20 source	122
7.26.2.21 startSeconds	122
7.26.2.22 uuid	122
7.26.2.23 vam	122
7.27 ClimbRepository Class Reference	122
7.27.1 Detailed Description	123
7.27.2 Constructor & Destructor Documentation	123
7.27.2.1 ClimbRepository()	123
7.27.3 Member Function Documentation	123
7.27.3.1 climbsForActivity()	123
7.27.3.2 deleteClimbsForActivity()	124
7.27.3.3 deletedClimbsForActivity()	124
7.27.3.4 getClimb()	124
7.27.3.5 hardDeleteClimb()	125
7.27.3.6 hasClimbs()	125
7.27.3.7 hasClimbsEver()	125
7.27.3.8 hasLockedClimbs()	126
7.27.3.9 insertClimb()	127
7.27.3.10 softDeleteClimb()	128
7.27.3.11 updateClimb()	128
7.27.4 Member Data Documentation	129
7.27.4.1 m_db	129
7.28 ClimbSortKeyTableItem Class Reference	129
7.28.1 Detailed Description	130
7.28.2 Constructor & Destructor Documentation	130
7.28.2.1 ClimbSortKeyTableItem()	130

7.28.3 Member Function Documentation	131
7.28.3.1 operator<()	131
7.28.4 Member Data Documentation	131
7.28.4.1 m_sortKey	131
7.29 ColorContext Struct Reference	131
7.29.1 Detailed Description	132
7.29.2 Member Data Documentation	132
7.29.2.1 altitudeMax	132
7.29.2.2 altitudeMin	132
7.29.2.3 ftp	132
7.29.2.4 hasAltitudeRange	132
7.29.2.5 heartRateMax	132
7.29.2.6 heartRateRest	133
7.29.2.7 thresholdHeartRate	133
7.30 ColorProvider Class Reference	133
7.30.1 Detailed Description	133
7.30.2 Member Function Documentation	133
7.30.2.1 colorForAltitude()	133
7.30.2.2 colorForCadence()	133
7.30.2.3 colorForHeartRate()	134
7.30.2.4 colorForMetric()	134
7.30.2.5 colorForPower()	134
7.30.2.6 colorForSpeed()	134
7.30.2.7 colorForValue()	134
7.31 CreateDatabaseDialog Class Reference	134
7.31.1 Detailed Description	136
7.31.2 Constructor & Destructor Documentation	136
7.31.2.1 CreateDatabaseDialog()	136
7.31.3 Member Function Documentation	136
7.31.3.1 accept	136
7.31.3.2 browseLocation	136
7.31.3.3 databasePath()	136
7.31.3.4 defaultAthleteFirstName()	136
7.31.3.5 defaultAthleteLastName()	136
7.31.3.6 normalizedDatabaseName()	136
7.31.3.7 shouldCreateDefaultAthlete()	136
7.31.3.8 updateAthleteFields	136
7.31.4 Member Data Documentation	137
7.31.4.1 m_browseButton	137
7.31.4.2 m_buttonBox	137
7.31.4.3 m_createAthleteCheck	137
7.31.4.4 m_firstNameEdit	137

7.31.4.5 m_lastNameEdit	137
7.31.4.6 m_locationEdit	137
7.31.4.7 m_nameEdit	137
7.32 DailyLoadPoint Struct Reference	137
7.32.1 Detailed Description	137
7.32.2 Member Data Documentation	138
7.32.2.1 tss	138
7.33 DatabaseConnectionGuard Class Reference	138
7.33.1 Detailed Description	138
7.33.2 Constructor & Destructor Documentation	138
7.33.2.1 DatabaseConnectionGuard()	138
7.33.2.2 ~DatabaseConnectionGuard()	138
7.33.3 Member Function Documentation	139
7.33.3.1 database()	139
7.33.4 Member Data Documentation	139
7.33.4.1 m_connectionName	139
7.34 DatabaseManager Class Reference	139
7.34.1 Detailed Description	140
7.34.2 Constructor & Destructor Documentation	140
7.34.2.1 ~DatabaseManager()	140
7.34.3 Member Function Documentation	140
7.34.3.1 applySchema()	140
7.34.3.2 cleanupFailedOpen()	140
7.34.3.3 close()	140
7.34.3.4 create()	141
7.34.3.5 database()	141
7.34.3.6 isOpen()	141
7.34.3.7 open()	141
7.34.3.8 path()	142
7.34.3.9 upgradeSchema()	143
7.34.4 Member Data Documentation	143
7.34.4.1 m_connectionName	143
7.34.4.2 m_db	143
7.34.4.3 m_path	143
7.35 DateFormatter Class Reference	143
7.35.1 Detailed Description	143
7.35.2 Member Function Documentation	144
7.35.2.1 formatDate()	144
7.35.2.2 formatDateTime()	144
7.35.2.3 parseDate()	145
7.35.2.4 qtDatePattern()	145
7.35.2.5 toIsoDate()	146

7.35.2.6 tolsoDateTime()	146
7.36 DuplicateActivityInfo Struct Reference	147
7.36.1 Detailed Description	147
7.36.2 Member Data Documentation	147
7.36.2.1 existingActivityId	147
7.36.2.2 existingDurationSeconds	147
7.36.2.3 existingStartUtc	147
7.36.2.4 newDurationSeconds	147
7.36.2.5 newStartUtc	147
7.37 EditActivityDialog Class Reference	147
7.37.1 Detailed Description	148
7.37.2 Constructor & Destructor Documentation	149
7.37.2.1 EditActivityDialog()	149
7.37.3 Member Function Documentation	149
7.37.3.1 updatedActivity()	149
7.37.4 Member Data Documentation	149
7.37.4.1 m_bikeEdit	149
7.37.4.2 m_equipmentEdit	149
7.37.4.3 m_fatigueSpin	149
7.37.4.4 m_initial	149
7.37.4.5 m_notesEdit	149
7.37.4.6 m_rpeSpin	149
7.37.4.7 m_sleepSpin	149
7.37.4.8 m_temperatureSpin	149
7.37.4.9 m_weatherEdit	149
7.37.4.10 m_weightSpin	150
7.37.4.11 m_windEdit	150
7.38 EditNotesDialog Class Reference	150
7.38.1 Detailed Description	151
7.38.2 Constructor & Destructor Documentation	151
7.38.2.1 EditNotesDialog()	151
7.38.3 Member Function Documentation	151
7.38.3.1 notes()	151
7.38.3.2 weatherNotes()	151
7.38.4 Member Data Documentation	151
7.38.4.1 m_notes	151
7.38.4.2 m_weather	151
7.39 anonymous_namespace{VideoRenderJob.cpp}::ExportTileCleanupGuard Class Reference	152
7.39.1 Detailed Description	152
7.39.2 Constructor & Destructor Documentation	152
7.39.2.1 ExportTileCleanupGuard()	152
7.39.2.2 ~ExportTileCleanupGuard()	152

7.39.3 Member Data Documentation	153
7.39.3.1 m_provider	153
7.40 anonymous_namespace{FitParser.cpp}::FitFileIdListener Class Reference	153
7.40.1 Detailed Description	154
7.40.2 Member Function Documentation	154
7.40.2.1 OnMesg()	154
7.40.2.2 timeCreated()	154
7.40.3 Member Data Documentation	154
7.40.3.1 m_timeCreated	154
7.41 FitnessChartWidget Class Reference	154
7.41.1 Detailed Description	155
7.41.2 Constructor & Destructor Documentation	155
7.41.2.1 FitnessChartWidget()	155
7.41.3 Member Function Documentation	156
7.41.3.1 clearChart()	156
7.41.3.2 setTimeline()	156
7.41.4 Member Data Documentation	156
7.41.4.1 m_atlSeries	156
7.41.4.2 m_axisX	156
7.41.4.3 m_axisY	156
7.41.4.4 m_ctlSeries	156
7.41.4.5 m_tsbSeries	156
7.42 FitnessMetricsPoint Struct Reference	156
7.42.1 Detailed Description	157
7.42.2 Member Data Documentation	157
7.42.2.1 atl	157
7.42.2.2 ctl	157
7.42.2.3 tsb	157
7.43 FitParser Class Reference	157
7.43.1 Detailed Description	158
7.43.2 Member Function Documentation	158
7.43.2.1 load()	158
7.44 FitRecordListener Class Reference	159
7.44.1 Detailed Description	160
7.44.2 Constructor & Destructor Documentation	160
7.44.2.1 FitRecordListener()	160
7.44.3 Member Function Documentation	160
7.44.3.1 firstTimestamp()	160
7.44.3.2 lastTimestamp()	160
7.44.3.3 OnMesg()	161
7.44.4 Member Data Documentation	161
7.44.4.1 m_firstTimestamp	161

7.44.4.2 m_lastTimestamp	161
7.44.4.3 m_rideData	161
7.45 MapFitMath::FitResult Struct Reference	161
7.45.1 Detailed Description	162
7.45.2 Member Data Documentation	162
7.45.2.1 centerLat	162
7.45.2.2 centerLon	162
7.45.2.3 lonSpanDegrees	162
7.45.2.4 maxLat	162
7.45.2.5 maxLon	162
7.45.2.6 minLat	163
7.45.2.7 minLon	163
7.45.2.8 zoom	163
7.46 anonymous_namespace{FitParser.cpp}::FitSessionListener Class Reference	163
7.46.1 Detailed Description	164
7.46.2 Member Function Documentation	164
7.46.2.1 OnMesg()	164
7.46.2.2 startTime()	164
7.46.3 Member Data Documentation	164
7.46.3.1 m_startTime	164
7.47 FtpEntry Struct Reference	164
7.47.1 Detailed Description	165
7.47.2 Member Data Documentation	165
7.47.2.1 athleteId	165
7.47.2.2 effectiveFrom	165
7.47.2.3 ftpWatts	165
7.47.2.4 id	165
7.47.2.5 notes	165
7.48 FTPZones Class Reference	166
7.48.1 Detailed Description	166
7.48.2 Member Typedef Documentation	166
7.48.2.1 ZoneArray	166
7.48.3 Member Function Documentation	166
7.48.3.1 compute()	166
7.48.3.2 defaultZones()	167
7.48.4 Member Data Documentation	167
7.48.4.1 kZoneCount	167
7.49 MainWindow::ImportBatchSummary Struct Reference	167
7.49.1 Detailed Description	168
7.49.2 Member Data Documentation	168
7.49.2.1 duplicates	168
7.49.2.2 errors	168

7.49.2.3 failed	168
7.49.2.4 imported	168
7.49.2.5 queued	168
7.49.2.6 showResultDialog	168
7.49.2.7 sourceLabel	168
7.50 ImportJob Struct Reference	168
7.50.1 Detailed Description	169
7.50.2 Member Enumeration Documentation	169
7.50.2.1 State	169
7.50.3 Member Data Documentation	169
7.50.3.1 athleteId	169
7.50.3.2 batchId	169
7.50.3.3 errorMessage	170
7.50.3.4 filePath	170
7.50.3.5 importSource	170
7.50.3.6 jobId	170
7.50.3.7 progress	170
7.50.3.8 state	170
7.51 ImportProgressModel Class Reference	170
7.51.1 Detailed Description	172
7.51.2 Constructor & Destructor Documentation	172
7.51.2.1 ImportProgressModel()	172
7.51.3 Member Function Documentation	172
7.51.3.1 activeJobs()	172
7.51.3.2 attachQueue()	172
7.51.3.3 completedJobs()	173
7.51.3.4 currentFileName()	173
7.51.3.5 failedJobs()	173
7.51.3.6 progress()	173
7.51.3.7 statusText()	173
7.51.4 Member Data Documentation	173
7.51.4.1 m_activeJobs	173
7.51.4.2 m_completedJobs	173
7.51.4.3 m_currentFileName	173
7.51.4.4 m_failedJobs	173
7.51.4.5 m_progress	173
7.51.4.6 m_statusText	173
7.52 ImportQueue Class Reference	174
7.52.1 Detailed Description	175
7.52.2 Constructor & Destructor Documentation	175
7.52.2.1 ImportQueue()	175
7.52.2.2 ~ImportQueue()	176

7.52.3 Member Function Documentation	176
7.52.3.1 activeCountUnsafe()	176
7.52.3.2 enqueue()	176
7.52.3.3 isBusy()	176
7.52.3.4 jobFailed	176
7.52.3.5 onWorkerCompleted	177
7.52.3.6 onWorkerFailed	177
7.52.3.7 onWorkerProgress	177
7.52.3.8 pendingCount()	177
7.52.3.9 processNext()	178
7.52.3.10 setDatabasePath()	178
7.52.4 Member Data Documentation	178
7.52.4.1 m_busy	178
7.52.4.2 m_dbPath	178
7.52.4.3 m_queue	178
7.52.4.4 m_runningJob	178
7.52.4.5 m_thread	178
7.52.4.6 m_worker	178
7.53 ImportRecord Struct Reference	179
7.53.1 Detailed Description	179
7.53.2 Member Data Documentation	179
7.53.2.1 activityId	179
7.53.2.2 errorMessage	179
7.53.2.3 fileName	179
7.53.2.4 id	179
7.53.2.5 importedAt	180
7.53.2.6 status	180
7.54 ImportRepository Class Reference	180
7.54.1 Detailed Description	180
7.54.2 Constructor & Destructor Documentation	180
7.54.2.1 ImportRepository()	180
7.54.3 Member Function Documentation	181
7.54.3.1 getRecentImports()	181
7.54.3.2 logImport()	181
7.54.4 Member Data Documentation	181
7.54.4.1 m_db	181
7.55 ImportStatusWidget Class Reference	182
7.55.1 Detailed Description	183
7.55.2 Constructor & Destructor Documentation	183
7.55.2.1 ImportStatusWidget()	183
7.55.3 Member Function Documentation	183
7.55.3.1 setModel()	183

7.55.3.2 updateFromModel()	183
7.55.4 Member Data Documentation	183
7.55.4.1 m_detailLabel	183
7.55.4.2 m_model	183
7.55.4.3 m_progressBar	183
7.55.4.4 m_titleLabel	184
7.56 ImportWorker Class Reference	184
7.56.1 Detailed Description	185
7.56.2 Constructor & Destructor Documentation	185
7.56.2.1 ImportWorker()	185
7.56.3 Member Function Documentation	185
7.56.3.1 failed	185
7.56.3.2 processJob	186
7.56.3.3 setDatabasePath()	186
7.56.4 Member Data Documentation	187
7.56.4.1 m_dbPath	187
7.57 Interval Struct Reference	187
7.57.1 Detailed Description	187
7.57.2 Member Data Documentation	187
7.57.2.1 averageCadence	187
7.57.2.2 averageHeartRate	188
7.57.2.3 averagePower	188
7.57.2.4 averageSpeed	188
7.57.2.5 durationSeconds	188
7.57.2.6 endSeconds	188
7.57.2.7 label	188
7.57.2.8 maximumPower	188
7.57.2.9 normalizedPower	188
7.57.2.10 startSeconds	188
7.58 IntervalDetector Class Reference	189
7.58.1 Detailed Description	189
7.58.2 Member Typedef Documentation	189
7.58.2.1 Config	189
7.58.3 Member Function Documentation	189
7.58.3.1 buildInterval()	189
7.58.3.2 detect()	189
7.58.3.3 removeOverlappingAndDuplicates()	190
7.58.3.4 smoothPower()	190
7.59 IntervalDetectorConfig Struct Reference	191
7.59.1 Detailed Description	191
7.59.2 Member Data Documentation	191
7.59.2.1 ftp	191

7.59.2.2 minRecoverySeconds	191
7.59.2.3 minWorkSeconds	191
7.59.2.4 smoothingWindow	191
7.59.2.5 workThresholdPct	192
7.60 IntervalEditorDialog Class Reference	192
7.60.1 Detailed Description	193
7.60.2 Constructor & Destructor Documentation	193
7.60.2.1 IntervalEditorDialog()	193
7.60.3 Member Function Documentation	193
7.60.3.1 intervalName()	193
7.60.3.2 intervalNotes()	193
7.60.3.3 intervalType()	193
7.60.4 Member Data Documentation	194
7.60.4.1 m_endSpin	194
7.60.4.2 m_nameEdit	194
7.60.4.3 m_notesEdit	194
7.60.4.4 m_startSpin	194
7.60.4.5 m_typeEdit	194
7.61 IntervalRecord Struct Reference	194
7.61.1 Detailed Description	195
7.61.2 Member Data Documentation	195
7.61.2.1 activityId	195
7.61.2.2 algorithmVersion	195
7.61.2.3 avgCadence	195
7.61.2.4 avgHr	195
7.61.2.5 avgPower	196
7.61.2.6 deleted	196
7.61.2.7 endSample	196
7.61.2.8 id	196
7.61.2.9 locked	196
7.61.2.10 name	196
7.61.2.11 notes	196
7.61.2.12 np	196
7.61.2.13 source	197
7.61.2.14 startSample	197
7.61.2.15 type	197
7.61.2.16 uuid	197
7.62 IntervalRepository Class Reference	197
7.62.1 Detailed Description	198
7.62.2 Constructor & Destructor Documentation	198
7.62.2.1 IntervalRepository()	198
7.62.3 Member Function Documentation	198

7.62.3.1 deleteInterval()	198
7.62.3.2 deleteIntervalsForActivity()	198
7.62.3.3 hasLockedIntervals()	198
7.62.3.4 insertInterval()	199
7.62.3.5 listIntervalsForActivity()	200
7.62.3.6 softDeleteInterval()	200
7.62.3.7 updateInterval()	200
7.62.4 Member Data Documentation	201
7.62.4.1 m_db	201
7.63 MainWindow Class Reference	201
7.63.1 Detailed Description	207
7.63.2 Constructor & Destructor Documentation	207
7.63.2.1 MainWindow()	207
7.63.3 Member Function Documentation	208
7.63.3.1 activityCount()	208
7.63.3.2 addRecentDatabase()	208
7.63.3.3 applyChartHeight()	208
7.63.3.4 applyChartPreset()	208
7.63.3.5 applyChartSmoothing()	208
7.63.3.6 applyEstimatedFtpForCurrentAthlete()	208
7.63.3.7 applyWeightMetricsToClimbs()	208
7.63.3.8 assignClimbWeightMetrics()	208
7.63.3.9 backupDatabase	208
7.63.3.10 buildColorContext()	208
7.63.3.11 buildStatusBar()	209
7.63.3.12 buildToolBar()	209
7.63.3.13 buildUI()	209
7.63.3.14 canImportActivities()	209
7.63.3.15 clearIntervalSummary()	209
7.63.3.16 closeEvent()	209
7.63.3.17 configureFolderWatcher()	209
7.63.3.18 createDatabase	209
7.63.3.19 createOrOpenDefaultDatabase()	209
7.63.3.20 createVideo	210
7.63.3.21 currentColorMetric()	210
7.63.3.22 decreaseChartHeight()	210
7.63.3.23 defaultDatabasePathForAutoCreate()	210
7.63.3.24 detectClimbsAndRefresh()	210
7.63.3.25 dragEnterEvent()	210
7.63.3.26 dropEvent()	210
7.63.3.27 editCurrentActivityProperties()	211
7.63.3.28 editSelectedClimbBoundaries()	211

7.63.3.29 ensureAthlete()	211
7.63.3.30 ensureDatabase()	211
7.63.3.31 ensureImportReady()	211
7.63.3.32 estimatedFtpFromCurrentRide()	211
7.63.3.33 eventFilter()	211
7.63.3.34 finalizeImportBatches()	212
7.63.3.35 fitFilesFromMimeData()	212
7.63.3.36 hideWelcomeScreen()	212
7.63.3.37 importActivities	212
7.63.3.38 importFiles()	212
7.63.3.39 importFilesInternal()	213
7.63.3.40 increaseChartHeight()	214
7.63.3.41 joinSelectedClimbs()	214
7.63.3.42 loadSettings()	214
7.63.3.43 manageAthletes	214
7.63.3.44 monitoredDirectories()	214
7.63.3.45 navigateToInterval()	214
7.63.3.46 onActivityImported	215
7.63.3.47 onAthleteSelectionChanged	215
7.63.3.48 onClimbBoundaryEdited()	215
7.63.3.49 onClimbRowDoubleClicked()	215
7.63.3.50 onClimbSelectionChanged()	215
7.63.3.51 onClimbTableContextMenuRequested()	215
7.63.3.52 onIntervalRowDoubleClicked()	215
7.63.3.53 onIntervalSelectionChanged()	215
7.63.3.54 onNewClimbRequested()	215
7.63.3.55 onWatchDirectoryChanged()	215
7.63.3.56 onWorkoutLoaded	216
7.63.3.57 openDatabase	216
7.63.3.58 openDatabasePath()	216
7.63.3.59 openRecentDatabase	216
7.63.3.60 openSettingsDialog	216
7.63.3.61 previewFitFile	216
7.63.3.62 refreshAthleteSelector()	216
7.63.3.63 refreshClimbViews()	216
7.63.3.64 removeSelectedClimbs()	216
7.63.3.65 resetAllZoom()	216
7.63.3.66 resolveActiveActivityWeightKg()	216
7.63.3.67 revertSelectedClimbToAuto	217
7.63.3.68 saveSettings()	217
7.63.3.69 scanWatchDirectories()	217
7.63.3.70 scheduleActivityBrowserRefresh()	217

7.63.3.71 selectedClimbIndicesFromTable()	217
7.63.3.72 selectIntervalRowOffset()	217
7.63.3.73 setupShortcuts()	217
7.63.3.74 showAbout	217
7.63.3.75 showWelcomeScreen()	217
7.63.3.76 triggerDetectClimbs	217
7.63.3.77 triggerDetectIntervals	217
7.63.3.78 updateAnalysisEmptyStates()	217
7.63.3.79 updateAthleteLabel()	218
7.63.3.80 updateCharts()	218
7.63.3.81 updateClimbingTab()	218
7.63.3.82 updateClimbQuarterCharts()	218
7.63.3.83 updateClimbRowStyles()	218
7.63.3.84 updateColorLegend()	218
7.63.3.85 updateFitnessChart()	218
7.63.3.86 updateHistogram()	218
7.63.3.87 updateImportAvailability()	218
7.63.3.88 updateIntervalRowStyles()	219
7.63.3.89 updateIntervals()	219
7.63.3.90 updateIntervalSummary()	219
7.63.3.91 updatePowerCurve()	219
7.63.3.92 updateRecentDatabaseMenu()	219
7.63.3.93 updateStatsLabel()	219
7.63.3.94 updateStatusBarInfo()	219
7.63.3.95 updateWelcomeScreenVisibility()	219
7.63.3.96 updateZoneAvailability()	219
7.63.3.97 updateZonesTab()	219
7.63.4 Member Data Documentation	220
7.63.4.1 kAnalysisTabCalendar	220
7.63.4.2 kAnalysisTabCharts	220
7.63.4.3 kAnalysisTabClimbing	220
7.63.4.4 kAnalysisTabFitness	220
7.63.4.5 kAnalysisTabHistogram	220
7.63.4.6 kAnalysisTabPDC	220
7.63.4.7 kAnalysisTabZones	220
7.63.4.8 kTabActivities	220
7.63.4.9 kTabAnalysis	220
7.63.4.10 m_activityBrowser	220
7.63.4.11 m_activityCountLabel	220
7.63.4.12 m_activityNotesView	220
7.63.4.13 m_activityTabStack	221
7.63.4.14 m_altitudeChart	221

7.63.4.15 m_analysisQueue	221
7.63.4.16 m_analysisTabWidget	221
7.63.4.17 m_athleteCombo	221
7.63.4.18 m_athleteHeader	221
7.63.4.19 m_athleteLabelAct	221
7.63.4.20 m_athletesMenu	221
7.63.4.21 m_athleteStatusLabel	221
7.63.4.22 m_autoSmoothingCheck	221
7.63.4.23 m_cadenceChart	221
7.63.4.24 m_calendarTabStack	221
7.63.4.25 m_calendarWidget	222
7.63.4.26 m_chartHeight	222
7.63.4.27 m_chartHeightDecreaseButton	222
7.63.4.28 m_chartHeightIncreaseButton	222
7.63.4.29 m_chartMapSplit	222
7.63.4.30 m_chartPresetCombo	222
7.63.4.31 m_chartsContainer	222
7.63.4.32 m_chartScroll	222
7.63.4.33 m_chartsLayout	222
7.63.4.34 m_climbAltitudeChart	222
7.63.4.35 m_climbCadenceChart	222
7.63.4.36 m_climbDipDistanceSpin	222
7.63.4.37 m_climbDipMetersSpin	223
7.63.4.38 m_climbHrChart	223
7.63.4.39 m_climbingTabStack	223
7.63.4.40 m_climbMinGainSpin	223
7.63.4.41 m_climbMinGradientSpin	223
7.63.4.42 m_climbMinLengthSpin	223
7.63.4.43 m_climbOverlayEnabledCheck	223
7.63.4.44 m_climbOverlayMetricCombo	223
7.63.4.45 m_climbPowerChart	223
7.63.4.46 m_climbQuarterCadenceChart	223
7.63.4.47 m_climbQuarterHrChart	223
7.63.4.48 m_climbQuarterPowerChart	223
7.63.4.49 m_climbSmoothingSpin	224
7.63.4.50 m_climbSpeedChart	224
7.63.4.51 m_climbsTable	224
7.63.4.52 m_climbStartGradientSpin	224
7.63.4.53 m_climbSummaryLabel	224
7.63.4.54 m_colorLegendLayout	224
7.63.4.55 m_colorLegendWidget	224
7.63.4.56 m_colorMetricCombo	224

7.63.4.57 m_controller	224
7.63.4.58 m_currentAthleteld	224
7.63.4.59 m_currentAthleteName	224
7.63.4.60 m_dbManager	224
7.63.4.61 m_dbStatusLabel	225
7.63.4.62 m_detectedClimbs	225
7.63.4.63 m_firstLaunchCompleted	225
7.63.4.64 m_fitChartsButton	225
7.63.4.65 m_fitnessChart	225
7.63.4.66 m_fitnessTabStack	225
7.63.4.67 m_followIntervalSelectionCheck	225
7.63.4.68 m_histogram	225
7.63.4.69 m_histogramTabStack	225
7.63.4.70 m_hrChart	225
7.63.4.71 m_importAct	225
7.63.4.72 m_importBatchSummaries	225
7.63.4.73 m_importProgressModel	226
7.63.4.74 m_importQueue	226
7.63.4.75 m_importRefreshTimer	226
7.63.4.76 m_importStatusWidget	226
7.63.4.77 m_intervalsTable	226
7.63.4.78 m_intervalSummaryLabel	226
7.63.4.79 m_knownWatchedFitFiles	226
7.63.4.80 m_lapsTable	226
7.63.4.81 m_lastOpenDir	226
7.63.4.82 m_mapAutoContrastCheck	226
7.63.4.83 m_mapRenderer	226
7.63.4.84 m_mapStyleCombo	226
7.63.4.85 m_pdcWidget	227
7.63.4.86 m_powerChart	227
7.63.4.87 m_powerCurveTabStack	227
7.63.4.88 m_powerSmoothingCombo	227
7.63.4.89 m_previewAct	227
7.63.4.90 m_recentDbMenu	227
7.63.4.91 m_showAltitude	227
7.63.4.92 m_showCadence	227
7.63.4.93 m_showHR	227
7.63.4.94 m_showPower	227
7.63.4.95 m_showSpeed	227
7.63.4.96 m_speedChart	227
7.63.4.97 m_suppressClimbAutoZoom	228
7.63.4.98 m_tabWidget	228

7.63.4.99 m_toolbarImportAct	228
7.63.4.100 m_toolbarSearchAct	228
7.63.4.101 m_undoManager	228
7.63.4.102 m_useEstimatedFtpButton	228
7.63.4.103 m_watcher	228
7.63.4.104 m_watchFolderEnabled	228
7.63.4.105 m_watchFolderPath	228
7.63.4.106 m_watchRescanTimer	228
7.63.4.107 m_welcomeWidget	228
7.63.4.108 m_zonesTable	228
7.63.4.109 m_zonesTabStack	229
7.64 MapRenderer Class Reference	229
7.64.1 Detailed Description	231
7.64.2 Member Enumeration Documentation	231
7.64.2.1 DragHandle	231
7.64.3 Constructor & Destructor Documentation	231
7.64.3.1 MapRenderer()	231
7.64.4 Member Function Documentation	231
7.64.4.1 adjustRouteColorForStyle()	231
7.64.4.2 autoRouteContrast()	231
7.64.4.3 fitToBounds()	232
7.64.4.4 fitToTrack	232
7.64.4.5 fitToVisibleRange	232
7.64.4.6 gpsRecordAtTime()	232
7.64.4.7 latLonToTile()	232
7.64.4.8 mapStyle()	232
7.64.4.9 mouseDoubleClickEvent()	232
7.64.4.10 mouseMoveEvent()	232
7.64.4.11 mousePressEvent()	232
7.64.4.12 mouseReleaseEvent()	233
7.64.4.13 nearestGpsRecord()	233
7.64.4.14 paintEvent()	233
7.64.4.15 rebuildGpsCache()	233
7.64.4.16 rebuildSegmentColors()	233
7.64.4.17 setAutoRouteContrast()	233
7.64.4.18 setCurrentTime()	233
7.64.4.19 setHighlightedElapsedSeconds	233
7.64.4.20 setMapStyle()	233
7.64.4.21 setRideData()	233
7.64.4.22 setRouteColorMode()	234
7.64.4.23 setTileCacheSize()	234
7.64.4.24 setVisibleTimeRange()	234

7.64.4.25 tileToScreen()	234
7.64.4.26 wheelEvent()	234
7.64.4.27 zoomIn	234
7.64.4.28 zoomOut	234
7.64.5 Member Data Documentation	234
7.64.5.1 m_autoFitVisibleRange	234
7.64.5.2 m_autoRouteContrast	234
7.64.5.3 m_centerLat	234
7.64.5.4 m_centerLon	235
7.64.5.5 m_colorContext	235
7.64.5.6 m_currentTimeSeconds	235
7.64.5.7 m_draggingSelection	235
7.64.5.8 m_dragHandle	235
7.64.5.9 m_endMarkerRect	235
7.64.5.10 m_firstGpsRecord	235
7.64.5.11 m_gpsRecords	235
7.64.5.12 m_hasGps	235
7.64.5.13 m_highlightElapsedSeconds	235
7.64.5.14 m_lastGpsRecord	235
7.64.5.15 m_maxLat	235
7.64.5.16 m_maxLon	236
7.64.5.17 m_minLat	236
7.64.5.18 m_minLon	236
7.64.5.19 m_minZoom	236
7.64.5.20 m_panning	236
7.64.5.21 m_panStart	236
7.64.5.22 m_rideData	236
7.64.5.23 m_routeColorMode	236
7.64.5.24 m_segmentColors	236
7.64.5.25 m_startMarkerRect	236
7.64.5.26 m_tileCache	236
7.64.5.27 m_tileRepaintTimer	236
7.64.5.28 m_userMovedMap	237
7.64.5.29 m_visibleEndSeconds	237
7.64.5.30 m_visibleStartSeconds	237
7.64.5.31 m_zoom	237
7.65 NavigationSidebar Class Reference	237
7.65.1 Detailed Description	238
7.65.2 Member Enumeration Documentation	238
7.65.2.1 Page	238
7.65.3 Constructor & Destructor Documentation	238
7.65.3.1 NavigationSidebar()	238

7.65.4 Member Function Documentation	239
7.65.4.1 currentPage()	239
7.65.4.2 setCurrentPage()	239
7.65.5 Member Data Documentation	239
7.65.5.1 m_list	239
7.66 PlannedWorkoutDialog Class Reference	239
7.66.1 Detailed Description	240
7.66.2 Constructor & Destructor Documentation	240
7.66.2.1 PlannedWorkoutDialog() [1/2]	240
7.66.2.2 PlannedWorkoutDialog() [2/2]	240
7.66.3 Member Function Documentation	240
7.66.3.1 draft()	240
7.66.4 Member Data Documentation	241
7.66.4.1 m_dateEdit	241
7.66.4.2 m_durationSpin	241
7.66.4.3 m_id	241
7.66.4.4 m_notesEdit	241
7.66.4.5 m_titleEdit	241
7.66.4.6 m_tssSpin	241
7.66.4.7 m_typeEdit	241
7.67 PlannedWorkoutDraft Struct Reference	241
7.67.1 Detailed Description	241
7.67.2 Member Data Documentation	242
7.67.2.1 date	242
7.67.2.2 durationSeconds	242
7.67.2.3 id	242
7.67.2.4 notes	242
7.67.2.5 targetTss	242
7.67.2.6 title	242
7.67.2.7 type	242
7.68 PowerCurve Class Reference	242
7.68.1 Detailed Description	242
7.68.2 Member Function Documentation	243
7.68.2.1 bestMeanPower()	243
7.68.2.2 computeStandard()	243
7.68.2.3 standardDurations()	244
7.69 PowerCurvePoint Struct Reference	244
7.69.1 Detailed Description	245
7.69.2 Member Data Documentation	245
7.69.2.1 bestPower	245
7.69.2.2 durationSeconds	245
7.70 PowerCurveWidget Class Reference	245

7.70.1 Detailed Description	246
7.70.2 Constructor & Destructor Documentation	246
7.70.2.1 PowerCurveWidget()	246
7.70.3 Member Function Documentation	246
7.70.3.1 clearChart()	246
7.70.3.2 setRideData()	247
7.70.3.3 setRideDataWithComparisons()	247
7.71 PowerHistogramWidget Class Reference	248
7.71.1 Detailed Description	249
7.71.2 Constructor & Destructor Documentation	249
7.71.2.1 PowerHistogramWidget()	249
7.71.3 Member Function Documentation	249
7.71.3.1 clearChart()	249
7.71.3.2 setRideData()	250
7.72 PowerZone Struct Reference	250
7.72.1 Detailed Description	251
7.72.2 Member Data Documentation	251
7.72.2.1 lowerPct	251
7.72.2.2 name	251
7.72.2.3 number	251
7.72.2.4 seconds	251
7.72.2.5 upperPct	251
7.73 RideChartWidget Class Reference	251
7.73.1 Detailed Description	255
7.73.2 Member Enumeration Documentation	255
7.73.2.1 ClimbHandleDrag	255
7.73.3 Constructor & Destructor Documentation	255
7.73.3.1 RideChartWidget()	255
7.73.4 Member Function Documentation	256
7.73.4.1 addClimbBackgroundBands()	256
7.73.4.2 addIntervalBackgroundBands()	256
7.73.4.3 addMetricOverlayBands()	256
7.73.4.4 addZoneBackgroundBands()	256
7.73.4.5 buildSampleIndices()	256
7.73.4.6 clearBackgroundSeries()	256
7.73.4.7 clearChart()	257
7.73.4.8 climbBoundaryEdited	257
7.73.4.9 climbClicked	257
7.73.4.10 computeSmoothedPowerSeries()	258
7.73.4.11 computeSmoothedSeries()	258
7.73.4.12 crosshairMoved	258
7.73.4.13 drawForeground()	258

7.73.4.14 fitToData	258
7.73.4.15 hasData()	259
7.73.4.16 intervalSelectionRequested	259
7.73.4.17 leaveEvent()	259
7.73.4.18 mouseDoubleClickEvent()	260
7.73.4.19 mouseMoveEvent()	260
7.73.4.20 mousePressEvent()	261
7.73.4.21 mouseReleaseEvent()	261
7.73.4.22 newClimbRequested	261
7.73.4.23 rebuildChart()	262
7.73.4.24 resetZoom	262
7.73.4.25 resizeEvent()	262
7.73.4.26 setAutoSmoothingEnabled()	263
7.73.4.27 setClimbEditingEnabled()	263
7.73.4.28 setClimbs()	263
7.73.4.29 setCrosshair	263
7.73.4.30 setIntervals()	263
7.73.4.31 setMetricOverlay()	264
7.73.4.32 setPowerSmoothingSeconds()	264
7.73.4.33 setRideData()	264
7.73.4.34 setSelectedClimbIndex()	264
7.73.4.35 setXRange	265
7.73.4.36 updateXTicks()	265
7.73.4.37 updateYAxis()	265
7.73.4.38 updateYAxisForVisibleRange()	265
7.73.4.39 visibleRangeChanged	265
7.73.4.40 visibleRangeEndMinutes()	266
7.73.4.41 visibleRangeStartMinutes()	266
7.73.4.42 wheelEvent()	266
7.73.4.43 xRangeChanged	267
7.73.5 Member Data Documentation	267
7.73.5.1 cachedAxisMax	267
7.73.5.2 cachedAxisMin	267
7.73.5.3 cachedMaxX	267
7.73.5.4 cachedMinX	267
7.73.5.5 m_autoSmoothing	267
7.73.5.6 m_axisX	267
7.73.5.7 m_axisXDisplay	268
7.73.5.8 m_axisY	268
7.73.5.9 m_axisYOverlay	268
7.73.5.10 m_backgroundSeries	268
7.73.5.11 m_climbEditingEnabled	268

7.73.5.12 m_climbHandleDrag	268
7.73.5.13 m_climbs	268
7.73.5.14 m_colorContext	268
7.73.5.15 m_colorMetric	268
7.73.5.16 m_colorSeries	268
7.73.5.17 m_crosshairX	268
7.73.5.18 m_crosshairY	268
7.73.5.19 m_dataMaxY	269
7.73.5.20 m_dataMinY	269
7.73.5.21 m_dragOriginalClimbEndSec	269
7.73.5.22 m_dragOriginalClimbStartSec	269
7.73.5.23 m_effectivePowerSmoothingSeconds	269
7.73.5.24 m_hasData	269
7.73.5.25 m_intervalEndMin	269
7.73.5.26 m_intervals	269
7.73.5.27 m_intervalSelecting	269
7.73.5.28 m_intervalStartMin	269
7.73.5.29 m_isRebuilding	269
7.73.5.30 m_metric	269
7.73.5.31 m_metricOverlayEnabled	270
7.73.5.32 m_metricOverlayMetric	270
7.73.5.33 m_newClimbCreating	270
7.73.5.34 m_newClimbCurrentSec	270
7.73.5.35 m_newClimbDragging	270
7.73.5.36 m_newClimbStartSec	270
7.73.5.37 m_origMaxX	270
7.73.5.38 m_overlaySeries	270
7.73.5.39 m_rawSeries	270
7.73.5.40 m_referenceSeries	270
7.73.5.41 m_requestedPowerSmoothingSeconds	270
7.73.5.42 m_rideData	270
7.73.5.43 m_selectedClimbIndex	271
7.73.5.44 m_series	271
7.73.5.45 m_smoothedSeriesCache	271
7.73.5.46 m_syncing	271
7.73.5.47 m_tooltipDisplayValues	271
7.73.5.48 m_tooltipPoints	271
7.73.5.49 m_tooltipRawValues	271
7.73.5.50 valid	271
7.74 RideData Class Reference	271
7.74.1 Detailed Description	272
7.74.2 Member Data Documentation	272

7.74.2.1 activityEndTimeUtc	272
7.74.2.2 activityStartTimeUtc	272
7.74.2.3 fileTimestampUtc	272
7.74.2.4 records	272
7.75 RideDataLoader Class Reference	272
7.75.1 Detailed Description	272
7.75.2 Member Function Documentation	273
7.75.2.1 loadActivitySamples()	273
7.76 RideDataSerializer Class Reference	273
7.76.1 Detailed Description	273
7.76.2 Member Function Documentation	273
7.76.2.1 loadRideFromDatabase()	273
7.76.2.2 saveRideToDatabase()	273
7.77 RideRecord Struct Reference	274
7.77.1 Detailed Description	274
7.77.2 Member Data Documentation	274
7.77.2.1 altitude	274
7.77.2.2 cadence	275
7.77.2.3 elapsedSeconds	275
7.77.2.4 hasAltitude	275
7.77.2.5 hasCadence	275
7.77.2.6 hasGps	275
7.77.2.7 hasHeartRate	275
7.77.2.8 hasPower	275
7.77.2.9 hasSpeed	275
7.77.2.10 heartRate	276
7.77.2.11 latitude	276
7.77.2.12 longitude	276
7.77.2.13 power	276
7.77.2.14 speed	276
7.78 anonymous_namespace{ClimbMetrics.cpp}::SliceStats Struct Reference	276
7.78.1 Detailed Description	276
7.78.2 Member Data Documentation	277
7.78.2.1 avgCadence	277
7.78.2.2 avgHeartRate	277
7.78.2.3 avgPower	277
7.79 SmoothingEngine Class Reference	277
7.79.1 Detailed Description	277
7.79.2 Member Function Documentation	277
7.79.2.1 smoothPower()	277
7.80 TileCache Class Reference	278
7.80.1 Detailed Description	279

7.80.2 Constructor & Destructor Documentation	279
7.80.2.1 TileCache()	279
7.80.3 Member Function Documentation	280
7.80.3.1 attribution()	280
7.80.3.2 clearMemoryCache()	280
7.80.3.3 diskCacheRoot()	280
7.80.3.4 diskTilePath()	280
7.80.3.5 diskTilePathForRoot() [1/2]	280
7.80.3.6 diskTilePathForRoot() [2/2]	280
7.80.3.7 dispatchNextDownloads()	280
7.80.3.8 isTileCachedOnDisk()	280
7.80.3.9 key()	281
7.80.3.10 keyForStyle()	281
7.80.3.11 mapStyle()	281
7.80.3.12 mapStyleName()	281
7.80.3.13 maxZoom()	281
7.80.3.14 onReplyFinished	281
7.80.3.15 providerForStyle()	281
7.80.3.16 setMapStyle()	281
7.80.3.17 setMemoryCacheSize()	281
7.80.3.18 styleDisplayName()	281
7.80.3.19 styleKey()	282
7.80.3.20 tile()	282
7.80.3.21 tileBlocking()	282
7.80.4 Member Data Documentation	282
7.80.4.1 kMaxConcurrentDownloads	282
7.80.4.2 m_activeDownloads	282
7.80.4.3 m_cache	282
7.80.4.4 m_diskCacheRoot	282
7.80.4.5 m_diskFallbackRoots	282
7.80.4.6 m_downloadQueue	282
7.80.4.7 m_mapStyle	282
7.80.4.8 m_nam	283
7.80.4.9 m_pending	283
7.80.4.10 m_provider	283
7.81 TileCacheConfig Struct Reference	283
7.81.1 Detailed Description	283
7.81.2 Member Data Documentation	283
7.81.2.1 diskCacheRoot	283
7.81.2.2 diskFallbackRoots	283
7.81.2.3 maxTilesInMemory	283
7.82 TileProvider Struct Reference	283

7.82.1 Detailed Description	284
7.82.2 Member Data Documentation	284
7.82.2.1 attribution	284
7.82.2.2 maxZoom	284
7.82.2.3 name	284
7.82.2.4 urlTemplate	284
7.83 TileCache::TileRequest Struct Reference	284
7.83.1 Detailed Description	284
7.83.2 Member Data Documentation	284
7.83.2.1 x	284
7.83.2.2 y	284
7.83.2.3 z	285
7.84 anonymous_namespace{VideoRenderJob.cpp}::TileViewport Struct Reference	285
7.84.1 Detailed Description	285
7.84.2 Member Function Documentation	285
7.84.2.1 operator==()	285
7.84.3 Member Data Documentation	285
7.84.3.1 nX	285
7.84.3.2 nY	285
7.84.3.3 tileX0	285
7.84.3.4 tileY0	285
7.85 TrainingLoad Class Reference	286
7.85.1 Detailed Description	286
7.85.2 Member Function Documentation	286
7.85.2.1 fitnessTimeline()	286
7.85.2.2 intensityFactor()	286
7.85.2.3 normalizedPower()	287
7.85.2.4 trainingStressScore()	287
7.86 TrainingMetrics Class Reference	288
7.86.1 Detailed Description	289
7.86.2 Member Function Documentation	289
7.86.2.1 compute()	289
7.87 TrainingMetricsSummary Struct Reference	289
7.87.1 Detailed Description	290
7.87.2 Member Data Documentation	290
7.87.2.1 averageHeartRate	290
7.87.2.2 averagePower	290
7.87.2.3 efficiencyFactor	290
7.87.2.4 intensityFactor	290
7.87.2.5 normalizedPower	290
7.87.2.6 trainingStressScore	291
7.87.2.7 variabilityIndex	291

7.88 UndoCommand Class Reference	291
7.88.1 Detailed Description	291
7.88.2 Member Function Documentation	291
7.88.2.1 description()	291
7.88.2.2 redo()	292
7.88.2.3 undo()	292
7.89 UndoManager Class Reference	292
7.89.1 Detailed Description	293
7.89.2 Constructor & Destructor Documentation	294
7.89.2.1 UndoManager()	294
7.89.3 Member Function Documentation	294
7.89.3.1 canRedo()	294
7.89.3.2 canUndo()	294
7.89.3.3 clear()	295
7.89.3.4 notify()	295
7.89.3.5 push()	295
7.89.3.6 redo	295
7.89.3.7 redoDescription()	295
7.89.3.8 undo	295
7.89.3.9 undoDescription()	295
7.89.4 Member Data Documentation	295
7.89.4.1 m_maxStackSize	295
7.89.4.2 m_redoStack	296
7.89.4.3 m_undoStack	296
7.90 VideoExportDialog Class Reference	296
7.90.1 Detailed Description	298
7.90.2 Constructor & Destructor Documentation	298
7.90.2.1 VideoExportDialog()	298
7.90.2.2 ~VideoExportDialog()	299
7.90.3 Member Function Documentation	299
7.90.3.1 cancelOrClose	299
7.90.3.2 chooseOutputFile	299
7.90.3.3 collectSettings()	299
7.90.3.4 onFinished	299
7.90.3.5 onFrameProgressChanged	299
7.90.3.6 onProgressChanged	299
7.90.3.7 onStageChanged	299
7.90.3.8 onTileStatsChanged	299
7.90.3.9 setAllOverlayChecks()	300
7.90.3.10 startExport	300
7.90.3.11 updateVideoLengthLabel()	300
7.90.4 Member Data Documentation	300

7.90.4.1 m_altitudeCheck	300
7.90.4.2 m_athleteCheck	300
7.90.4.3 m_cadenceCheck	300
7.90.4.4 m_cancelButton	300
7.90.4.5 m_chartsCheck	300
7.90.4.6 m_defaults	300
7.90.4.7 m_deleteTemporaryTilesCheck	300
7.90.4.8 m_entireActivityRadio	300
7.90.4.9 m_etaLabel	301
7.90.4.10 m_exportButton	301
7.90.4.11 m_fixedZoomSpin	301
7.90.4.12 m_fpsSpin	301
7.90.4.13 m_frameLabel	301
7.90.4.14 m_hrCheck	301
7.90.4.15 m_job	301
7.90.4.16 m_legendCheck	301
7.90.4.17 m_mapStyleCombo	301
7.90.4.18 m_outputEdit	301
7.90.4.19 m_powerCheck	301
7.90.4.20 m_progressBar	301
7.90.4.21 m_resolutionCombo	302
7.90.4.22 m_routeColorCombo	302
7.90.4.23 m_running	302
7.90.4.24 m_selectedSegmentRadio	302
7.90.4.25 m_speedCheck	302
7.90.4.26 m_speedSpin	302
7.90.4.27 m_stageLabel	302
7.90.4.28 m_telemetryPanelCheck	302
7.90.4.29 m_thread	302
7.90.4.30 m_tileStatsLabel	302
7.90.4.31 m_timeCheck	302
7.90.4.32 m_videoLengthLabel	302
7.91 VideoExportSettings Struct Reference	303
7.91.1 Detailed Description	304
7.91.2 Member Data Documentation	304
7.91.2.1 activityName	304
7.91.2.2 athleteName	304
7.91.2.3 autoZoom	305
7.91.2.4 colorContext	305
7.91.2.5 deleteTemporaryTilesAfterExport	305
7.91.2.6 ffmpegPath	305
7.91.2.7 fixedZoom	305

7.91.2.8 followAthlete	305
7.91.2.9 fps	305
7.91.2.10 height	305
7.91.2.11 mapStyle	305
7.91.2.12 outputPath	306
7.91.2.13 overlayAltitude	306
7.91.2.14 overlayAthleteName	306
7.91.2.15 overlayCadence	306
7.91.2.16 overlayCharts	306
7.91.2.17 overlayHeartRate	306
7.91.2.18 overlayPower	306
7.91.2.19 overlayRouteLegend	306
7.91.2.20 overlaySpeed	306
7.91.2.21 overlayTelemetryPanel	307
7.91.2.22 overlayTime	307
7.91.2.23 playbackSpeed	307
7.91.2.24 rideData	307
7.91.2.25 routeColorMetric	307
7.91.2.26 segmentEndSeconds	307
7.91.2.27 segmentStartSeconds	307
7.91.2.28 singleRouteColor	307
7.91.2.29 useSelectedSegment	307
7.91.2.30 width	308
7.92 VideoRenderJob Class Reference	308
7.92.1 Detailed Description	309
7.92.2 Constructor & Destructor Documentation	309
7.92.2.1 VideoRenderJob()	309
7.92.3 Member Function Documentation	309
7.92.3.1 cancel()	309
7.92.3.2 finished	310
7.92.3.3 frameProgressChanged	310
7.92.3.4 progressChanged	311
7.92.3.5 run()	311
7.92.3.6 stageChanged	312
7.92.3.7 tileStatsChanged	312
7.92.4 Member Data Documentation	313
7.92.4.1 m_cancelRequested	313
7.92.4.2 m_settings	313
7.93 VideoTileId Struct Reference	313
7.93.1 Detailed Description	313
7.93.2 Member Function Documentation	314
7.93.2.1 operator==()	314

7.93.3 Member Data Documentation	314
7.93.3.1 x	314
7.93.3.2 y	314
7.93.3.3 z	314
7.94 VideoTileIdHash Struct Reference	314
7.94.1 Detailed Description	314
7.94.2 Member Function Documentation	314
7.94.2.1 operator()()	314
7.95 VideoTileProvider Class Reference	315
7.95.1 Detailed Description	316
7.95.2 Constructor & Destructor Documentation	316
7.95.2.1 VideoTileProvider()	316
7.95.3 Member Function Documentation	316
7.95.3.1 cleanupExportCache()	316
7.95.3.2 clearMemoryCache()	316
7.95.3.3 exportCacheRoot()	317
7.95.3.4 getTile()	317
7.95.3.5 isTileCachedOnDisk()	318
7.95.3.6 requiredTileCount()	318
7.95.3.7 requiredTilesAvailableOnDisk()	318
7.95.3.8 setRequiredTiles()	318
7.95.3.9 shouldDeleteExportCacheAfterExport()	319
7.95.3.10 stats()	319
7.95.4 Member Data Documentation	319
7.95.4.1 m_deleteTemporaryTilesAfterExport	319
7.95.4.2 m_downloadedTiles	319
7.95.4.3 m_requiredTiles	319
7.95.4.4 m_requiredTilesOnDisk	319
7.95.4.5 m_tileCache	319
7.96 VideoTileProviderConfig Struct Reference	320
7.96.1 Detailed Description	320
7.96.2 Member Data Documentation	320
7.96.2.1 deleteTemporaryTilesAfterExport	320
7.96.2.2 exportCacheRoot	320
7.96.2.3 memoryCacheTiles	320
7.96.2.4 style	320
7.97 VideoTileStats Struct Reference	320
7.97.1 Detailed Description	321
7.97.2 Member Data Documentation	321
7.97.2.1 downloadedTiles	321
7.97.2.2 onDiskTiles	321
7.97.2.3 requiredTiles	321

7.98 WeightEntry Struct Reference	321
7.98.1 Detailed Description	322
7.98.2 Member Data Documentation	322
7.98.2.1 athleteId	322
7.98.2.2 effectiveFrom	322
7.98.2.3 id	322
7.98.2.4 weightKg	322
7.99 WelcomeWidget Class Reference	322
7.99.1 Detailed Description	323
7.99.2 Constructor & Destructor Documentation	323
7.99.2.1 WelcomeWidget()	323
7.99.3 Member Function Documentation	324
7.99.3.1 setFirstLaunch()	324
7.99.4 Member Data Documentation	324
7.99.4.1 m_createDbButton	324
7.99.4.2 m_importButton	324
7.99.4.3 m_openDbButton	325
7.99.4.4 m_subtitleLabel	325
7.100 WorkoutAnalyzer Class Reference	325
7.100.1 Detailed Description	325
7.100.2 Member Function Documentation	325
7.100.2.1 analyze()	325
7.101 WorkoutController Class Reference	326
7.101.1 Detailed Description	328
7.101.2 Constructor & Destructor Documentation	328
7.101.2.1 WorkoutController()	328
7.101.3 Member Function Documentation	328
7.101.3.1 climbs()	328
7.101.3.2 currentActivityId()	328
7.101.3.3 currentAthleteId()	328
7.101.3.4 efficiencyFactor()	328
7.101.3.5 ftp()	328
7.101.3.6 importFile()	328
7.101.3.7 intensityFactor()	328
7.101.3.8 intervals()	329
7.101.3.9 loadActivity()	329
7.101.3.10 loadFile()	329
7.101.3.11 loadOrGenerateClimbs()	329
7.101.3.12 loadOrGenerateIntervals()	329
7.101.3.13 normalizedPower()	329
7.101.3.14 pdcPoints()	329
7.101.3.15 powerZones()	329

7.101.3.16 reanalyze()	329
7.101.3.17 reloadClimbsFromDatabase()	329
7.101.3.18 rideData()	329
7.101.3.19 setAnalysisQueue()	330
7.101.3.20 setCurrentAthlete()	330
7.101.3.21 setDatabaseManager()	330
7.101.3.22 setFTP()	330
7.101.3.23 statistics()	330
7.101.3.24 trainingStressScore()	330
7.101.3.25 variabilityIndex()	330
7.101.4 Member Data Documentation	330
7.101.4.1 m_analysisQueue	330
7.101.4.2 m_analyzer	330
7.101.4.3 m_climbs	330
7.101.4.4 m_currentActivityId	330
7.101.4.5 m_currentAthleteId	331
7.101.4.6 m_dbManager	331
7.101.4.7 m_ef	331
7.101.4.8 m ftp	331
7.101.4.9 m_if	331
7.101.4.10 m_intervals	331
7.101.4.11 m_np	331
7.101.4.12 m_pdcPoints	331
7.101.4.13 m_powerZones	331
7.101.4.14 m_rideData	331
7.101.4.15 m_statistics	331
7.101.4.16 m_tss	331
7.101.4.17 m_vi	332
7.102 WorkoutStatistics Struct Reference	332
7.102.1 Detailed Description	332
7.102.2 Member Data Documentation	332
7.102.2.1 averageCadence	332
7.102.2.2 averageHeartRate	333
7.102.2.3 averagePower	333
7.102.2.4 averageSpeed	333
7.102.2.5 durationSeconds	333
7.102.2.6 elevationGainM	333
7.102.2.7 maximumCadence	333
7.102.2.8 maximumHeartRate	333
7.102.2.9 maximumPower	333
7.102.2.10 maximumSpeed	333
7.102.2.11 movingTimeSeconds	334

7.102.2.12 totalDistanceKm	334
7.103 Zone Struct Reference	334
7.103.1 Detailed Description	334
7.103.2 Member Data Documentation	334
7.103.2.1 color	334
7.103.2.2 maxValu	334
7.103.2.3 minValu	335
7.103.2.4 name	335
7.104 ZoneCalculator Class Reference	335
7.104.1 Detailed Description	335
7.104.2 Member Function Documentation	335
7.104.2.1 altitudeZones()	335
7.104.2.2 cadenceZones()	336
7.104.2.3 computeDistribution()	336
7.104.2.4 hasMetricSamples()	337
7.104.2.5 heartRateZones()	337
7.104.2.6 powerZones()	338
7.104.2.7 speedZones()	338
7.104.2.8 zonesForMetric()	339
7.105 ZoneDistribution Struct Reference	340
7.105.1 Detailed Description	341
7.105.2 Member Data Documentation	341
7.105.2.1 percent	341
7.105.2.2 seconds	341
7.105.2.3 zone	341
8 File Documentation	343
8.1 ClimbDetector.cpp	343
8.2 ClimbDetector.h	351
8.3 ClimbMetrics.cpp	352
8.4 ClimbMetrics.h	358
8.5 FTPZones.cpp	359
8.6 FTPZones.h	360
8.7 IntervalDetector.cpp	360
8.8 IntervalDetector.h	363
8.9 PowerCurve.cpp	364
8.10 PowerCurve.h	364
8.11 AnalysisQueue.cpp	365
8.12 AnalysisQueue.h	366
8.13 AnalysisWorker.cpp	366
8.14 AnalysisWorker.h	369
8.15 SmoothingEngine.cpp	370

8.16 SmoothingEngine.h	371
8.17 TrainingLoad.cpp	371
8.18 TrainingLoad.h	372
8.19 TrainingMetrics.cpp	372
8.20 TrainingMetrics.h	373
8.21 WorkoutAnalyzer.cpp	374
8.22 WorkoutAnalyzer.h	375
8.23 FitnessChartWidget.cpp	375
8.24 FitnessChartWidget.h	377
8.25 Metric.h	377
8.26 PowerCurveWidget.cpp	377
8.27 PowerCurveWidget.h	380
8.28 PowerHistogramWidget.cpp	380
8.29 PowerHistogramWidget.h	382
8.30 RideChartWidget.cpp	382
8.31 RideChartWidget.h	406
8.32 ActivityAnalysisFlags.h	408
8.33 AnalysisVersions.h	408
8.34 AppSettings.cpp	408
8.35 AppSettings.h	409
8.36 DateFormatter.cpp	410
8.37 DateFormatter.h	411
8.38 ClimbEditCommand.cpp	411
8.39 ClimbEditCommand.h	412
8.40 UndoCommand.h	412
8.41 UndoManager.cpp	412
8.42 UndoManager.h	413
8.43 ColorProvider.cpp	414
8.44 ColorProvider.h	415
8.45 ZoneCalculator.cpp	416
8.46 ZoneCalculator.h	418
8.47 ZoneDefinition.cpp	419
8.48 ZoneDefinition.h	419
8.49 ActivityAnalysisCacheRepository.cpp	420
8.50 ActivityAnalysisCacheRepository.h	421
8.51 ActivityAnalysisSettingsRepository.cpp	421
8.52 ActivityAnalysisSettingsRepository.h	422
8.53 ActivityRepository.cpp	422
8.54 ActivityRepository.h	426
8.55 AthleteRepository.cpp	427
8.56 AthleteRepository.h	430
8.57 ClimbRepository.cpp	431

8.58 ClimbRepository.h	434
8.59 DatabaseConnectionGuard.cpp	435
8.60 DatabaseConnectionGuard.h	435
8.61 DatabaseManager.cpp	436
8.62 DatabaseManager.h	441
8.63 DatabaseSchema.cpp	441
8.64 DatabaseSchema.h	442
8.65 ImportRepository.cpp	445
8.66 ImportRepository.h	446
8.67 IntervalRepository.cpp	446
8.68 IntervalRepository.h	448
8.69 SqlQueryHelper.h	449
8.70 FitParser.cpp	450
8.71 FitParser.h	452
8.72 FitRecordListener.cpp	452
8.73 FitRecordListener.h	454
8.74 RideData.h	454
8.75 fitlyzerC.cpp	455
8.76 AboutDialog.cpp	455
8.77 AboutDialog.h	456
8.78 ActivityBrowser.cpp	456
8.79 ActivityBrowser.h	461
8.80 AthleteDialog.cpp	461
8.81 AthleteDialog.h	465
8.82 AthleteHeaderWidget.cpp	465
8.83 AthleteHeaderWidget.h	466
8.84 AthleteListDialog.cpp	466
8.85 AthleteListDialog.h	468
8.86 CalendarWidget.cpp	469
8.87 CalendarWidget.h	472
8.88 CreateDatabaseDialog.cpp	473
8.89 CreateDatabaseDialog.h	475
8.90 EditActivityDialog.cpp	475
8.91 EditActivityDialog.h	477
8.92 EditNotesDialog.cpp	477
8.93 EditNotesDialog.h	478
8.94 ImportStatusWidget.cpp	478
8.95 ImportStatusWidget.h	479
8.96 IntervalEditorDialog.cpp	479
8.97 IntervalEditorDialog.h	480
8.98 MainWindow.cpp	481
8.99 MainWindow.h	538

8.100 NavigationSidebar.cpp	542
8.101 NavigationSidebar.h	542
8.102 PlannedWorkoutDialog.cpp	543
8.103 PlannedWorkoutDialog.h	544
8.104 WelcomeWidget.cpp	545
8.105 WelcomeWidget.h	546
8.106 WorkoutController.cpp	546
8.107 WorkoutController.h	552
8.108 ImportJob.cpp	554
8.109 ImportJob.h	554
8.110 ImportProgressModel.cpp	554
8.111 ImportProgressModel.h	555
8.112 ImportQueue.cpp	556
8.113 ImportQueue.h	557
8.114 ImportWorker.cpp	558
8.115 ImportWorker.h	562
8.116 MapFitMath.cpp	562
8.117 MapFitMath.h	564
8.118 MapRenderer.cpp	564
8.119 MapRenderer.h	574
8.120 TileCache.cpp	576
8.121 TileCache.h	581
8.122 RideDataLoader.cpp	582
8.123 RideDataLoader.h	583
8.124 RideDataSerializer.cpp	583
8.125 RideDataSerializer.h	585
8.126 Platform.cpp	586
8.127 Platform.h	586
8.128 Platform_linux.cpp	586
8.129 Platform_macos.cpp	587
8.130 Platform_windows.cpp	587
8.131 FfmpegPath.cpp	587
8.132 FfmpegPath.h	588
8.133 UuidGenerator.h	588
8.134 VideoExportDialog.cpp	589
8.135 VideoExportDialog.h	594
8.136 VideoExportSettings.h	595
8.137 VideoRenderJob.cpp	596
8.138 VideoRenderJob.h	608
8.139 VideoTileProvider.cpp	608
8.140 VideoTileProvider.h	610

Chapter 1

FitlyzerC

Modern Garmin FIT analysis for cyclists, runners, triathletes, coaches, and data enthusiasts.

Warning

FitlyzerC is pre-release software.

Database schemas, internal data structures, and project formats may change between releases. Upgrading may require recreating databases or re-importing activities. Backward compatibility is not guaranteed yet.

1.1 Download

Prebuilt binaries are available from the [Releases](#) page.

Platform	Package
Windows	NSIS Installer
macOS	DMG
Linux	ApplImage

1.2 What is FitlyzerC?

FitlyzerC is a native desktop application for analyzing Garmin FIT activity files.

Unlike cloud-based training platforms, FitlyzerC focuses on local-first analysis, high-performance visualization, and complete ownership of your training data. The application is written in modern C++23 and Qt 6 and runs on Windows, macOS, and Linux.

The project is designed primarily for cycling and currently focuses on detailed ride analysis, interval inspection, power-based training metrics, GPS visualization, and athlete management.

1.3 Features

1.3.1 Activity Analysis

- Import Garmin FIT activities
- Decode FIT files using the Garmin FIT SDK
- Power, heart rate, cadence, speed, elevation and GPS analysis

- Automatic interval detection
- Power curve generation
- Power histograms
- Training load calculations
- Fitness tracking metrics
- Workout review tools
- Ride notes

1.3.2 Interactive Charts

- Zoomable and pannable charts
- Synchronized chart navigation
- Power analysis
- Fitness trend visualization
- Ride timeline inspection
- Smoothed data visualization

1.3.3 Mapping

- Interactive route display
- Multiple map styles
- GPS track visualization
- Segment selection directly on the map
- Draggable selection markers
- Route coloring by:
 - Power
 - Heart Rate
 - Cadence
 - Speed
 - Altitude
 - Gradient

1.3.4 Athlete Management

- Multiple athlete profiles
- Athlete-specific FTP settings
- Training history
- Calendar-based activity browsing
- Planned workouts

1.3.5 Video Export

- Animated ride playback
- Map-based activity visualization
- Chart synchronization
- Power zone overlays
- MP4 export through FFmpeg
- Custom export settings

1.3.6 Cross Platform

- Windows
 - macOS
 - Linux
-

1.4 Screenshots

1.5 Current Development Status

FitlyzerC is under active development.
Recent milestones include:

- Linux Appliance packaging
- Reworked map tile system
- Additional map styles
- Improved route rendering
- Movable interval markers
- Segment selection on maps
- Video export support
- FFmpeg integration
- Multiple video rendering improvements

1.5.1 Planned Areas

- Additional training metrics
 - Reporting features
 - More export formats
 - Better workout planning
 - UI/UX refinements
 - Performance improvements
-

1.6 Technology Stack

Component	Technology
Language	C++23
GUI	Qt 6
Charts	Qt Charts
Database	SQLite
Networking	Qt Network
Build System	CMake
Packaging	CPack
FIT Parsing	Garmin FIT SDK

1.7 Project Structure

```

src/
analysis/      Training metrics, power curves, interval detection
charts/       Interactive chart widgets
core/         Zone calculations and shared logic
database/     SQLite repositories and schema management
fit/          Garmin FIT decoding
gui/          User interface
maps/         Tile engine and route rendering
model/        Serialization helpers
platform/     OS-specific functionality
video/        Video export pipeline
reports/      Reporting infrastructure

```

1.8 Build Requirements

1.8.1 Required Software

Component	Version
CMake	3.21+
Qt	6.x
Compiler	C++23 capable
Git	Current
Ninja	Recommended
FFmpeg (for bundled releases)	Source tree in <code>third_party/ffmpeg</code> or prebuilt binary in <code>resources/ffmpeg/<platform>/</code>

1.9 Building

1.9.1 macOS

1.9.1.1 Dependencies

```

brew install cmake ninja qt
export PATH="/opt/homebrew/opt/qt/bin:$PATH"

```


1.9.1.2 Configure

```
cmake --preset macos-release
```

1.9.1.3 Build

```
cmake --build --preset macos-release
```

1.9.1.4 Create DMG

```
cmake --build --preset macos-dmg
```

1.9.2 Windows

Install:

- Visual Studio 2026
- Qt 6 (MSVC)
- CMake
- Ninja

1.9.2.1 Configure

```
cmake --preset windows-release
```

1.9.2.2 Build

```
cmake --build --preset windows-release
```

1.9.2.3 Create Installer

```
cmake --build --preset windows-nsis
```

1.9.3 Linux

1.9.3.1 Arch Linux

```
sudo pacman -S \  
    base-devel \  
    cmake \  
    ninja \  
    qt6-base \  
    qt6-charts \  
    git
```

1.9.3.2 Configure

```
cmake --preset linux-release
```

1.9.3.3 Build

```
cmake --build --preset linux-release
```

1.9.3.4 AppImage

Install:

```
sudo pacman -S patchelf  
yay -S appimagetool-bin  
cmake --preset linux-appimage  
cmake --build --preset linux-appimage
```

1.9.4 One-Command Packaging

```
cmake -P scripts/package.cmake
```

Supported outputs:

Platform	Output
Windows	Setup.exe
macOS	DMG
Linux	ApplImage

1.9.5 Garmin FIT SDK

By default, CMake automatically downloads the Garmin FIT SDK.

Repository:

<https://github.com/garmin/fit-cpp-sdk>

To use a local SDK copy:

```
cmake \
  -DGARMIN_FIT_SDK_DIR=/path/to/sdk \
  -S . \
  -B build
```

Supported locations:

```
fit-sdk/
fit-sdk/src/
fit-sdk/cpp-sdk/src/
```

1.9.6 FFmpeg Bundling

Release builds are configured to bundle FFmpeg with the app so end users do not need a system FFmpeg installation.

Bundling priority:

1. Build from FFmpeg source in `third_party/ffmpeg`
2. Auto-download FFmpeg from <https://git.ffmpeg.org/ffmpeg.git> when the local source tree is missing
3. Use prebuilt executable in `resources/ffmpeg/<platform>/`
4. Fallback to system `ffmpeg` if neither source nor bundled binary is available

Default prebuilt lookup paths:

```
resources/ffmpeg/macos/ffmpeg
resources/ffmpeg/linux/ffmpeg
resources/ffmpeg/windows/ffmpeg.exe
```

Useful CMake options:

```
-DFITLYZERC_BUNDLE_FFMPEG=ON
-DFITLYZERC_FFMPEG_SOURCE_DIR=third_party/ffmpeg
-DFITLYZERC_AUTO_DOWNLOAD_FFMPEG=ON
-DFITLYZERC_FFMPEG_GIT_REPOSITORY=https://git.ffmpeg.org/ffmpeg.git
-DFITLYZERC_FFMPEG_GIT_TAG=n7.0
-DFITLYZERC_FFMPEG_BINARY_PATH=/absolute/path/to/ffmpeg
```

If you want fully reproducible builds, point `FITLYZERC_FFMPEG_SOURCE_DIR` at a local FFmpeg checkout or pin `FITLYZERC_FFMPEG_GIT_TAG` to a specific release tag/commit.

Disable bundling (not recommended for release artifacts):

```
cmake -DFITLYZERC_BUNDLE_FFMPEG=OFF --preset <your-preset>
```

1.9.7 Running

Typical workflow:

1. Create an athlete
2. Import FIT activities

3. Browse activities in the calendar
 4. Inspect charts and maps
 5. Select intervals
 6. Analyze performance metrics
 7. Export video visualizations
-

1.9.8 Contributing

Issues, bug reports, feature requests and pull requests are welcome.
If you encounter a problem, include:

- Operating system
 - Application version
 - FIT file characteristics
 - Steps to reproduce
-

1.9.9 License

Informal license:

Do whatever you want with it, provide attribution and link back to the repository.
Except dopers. They cannot use it.

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

ActivityAnalysisFlags	
Bitmask flags for analysis completion tracking	23
AnalysisVersions	
Algorithm version tracking for analysis results	24
anonymous_namespace{ActivityBrowser.cpp}	25
anonymous_namespace{AnalysisWorker.cpp}	25
anonymous_namespace{AthleteDialog.cpp}	25
anonymous_namespace{ClimbDetector.cpp}	26
anonymous_namespace{ClimbMetrics.cpp}	27
anonymous_namespace{ClimbRepository.cpp}	28
anonymous_namespace{ColorProvider.cpp}	29
anonymous_namespace{DateFormatter.cpp}	29
anonymous_namespace{FfmpegPath.cpp}	29
anonymous_namespace{FitParser.cpp}	29
anonymous_namespace{ImportWorker.cpp}	30
anonymous_namespace{MapFitMath.cpp}	30
anonymous_namespace{RideChartWidget.cpp}	31
anonymous_namespace{TileCache.cpp}	32
anonymous_namespace{VideoExportDialog.cpp}	32
anonymous_namespace{VideoRenderJob.cpp}	33
anonymous_namespace{VideoTileProvider.cpp}	35
anonymous_namespace{WorkoutController.cpp}	35
anonymous_namespace{ZoneCalculator.cpp}	36
Platform	
Platform-specific abstractions and utilities	36
SqlQueryHelper	
Common SQL patterns and query builders to reduce duplication across repositories	37

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Activity	42
ActivityAnalysisCacheRepository	48
ActivityAnalysisSettings	50
ActivityAnalysisSettingsRepository	52
ActivityRepository	58
AppSettings	73
Athlete	75
AthleteRepository	85
Climb	94
ClimbDetector	99
ClimbDetectorConfig	106
anonymous_namespace{RideChartWidget.cpp}::ClimbHandleHitResult	114
ClimbMetrics	115
ClimbQuarterMetrics	117
ClimbRecord	118
ClimbRepository	122
ColorContext	131
ColorProvider	133
DailyLoadPoint	137
DatabaseConnectionGuard	138
DatabaseManager	139
DateFormatter	143
DuplicateActivityInfo	147
anonymous_namespace{VideoRenderJob.cpp}::ExportTileCleanupGuard	152
fit::FileIdMesgListener	
anonymous_namespace{FitParser.cpp}::FitFileIdListener	153
FitnessMetricsPoint	156
FitParser	157
MapFitMath::FitResult	161
FtpEntry	164
FTPZones	166
MainWindow::ImportBatchSummary	167
ImportJob	168
ImportRecord	179
ImportRepository	180
Interval	187
IntervalDetector	189
IntervalDetectorConfig	191
IntervalRecord	194
IntervalRepository	197

PlannedWorkoutDraft	241
PowerCurve	242
PowerCurvePoint	244
PowerZone	250
QChartView	
FitnessChartWidget	154
PowerCurveWidget	245
PowerHistogramWidget	248
RideChartWidget	251
QDialog	
AboutDialog	41
AthleteDialog	77
AthleteListDialog	83
CreateDatabaseDialog	134
EditActivityDialog	147
EditNotesDialog	150
IntervalEditorDialog	192
PlannedWorkoutDialog	239
VideoExportDialog	296
QMainWindow	
MainWindow	201
QObject	
AnalysisQueue	63
AnalysisWorker	69
ImportProgressModel	170
ImportQueue	174
ImportWorker	184
TileCache	278
UndoManager	292
VideoRenderJob	308
WorkoutController	326
QTableWidgetItem	
ClimbNameTableWidgetItem	116
ClimbSortKeyTableWidgetItem	129
anonymous_namespace{ActivityBrowser.cpp}::ActivityDateTableWidgetItem	57
QWidget	
ActivityBrowser	53
AthleteHeaderWidget	81
CalendarWidget	91
ImportStatusWidget	182
MapRenderer	229
NavigationSidebar	237
WelcomeWidget	322
fit::RecordMsgListener	
FitRecordListener	159
RideData	271
RideDataLoader	272
RideDataSerializer	273
RideRecord	274
fit::SessionMsgListener	
anonymous_namespace{FitParser.cpp}::FitSessionListener	163
anonymous_namespace{ClimbMetrics.cpp}::SliceStats	276
SmoothingEngine	277
TileCacheConfig	283
TileProvider	283
TileCache::TileRequest	284
anonymous_namespace{VideoRenderJob.cpp}::TileViewport	285
TrainingLoad	286

TrainingMetrics	288
TrainingMetricsSummary	289
UndoCommand	291
ClimbEditCommand	112
VideoExportSettings	303
VideoTileId	313
VideoTileIdHash	314
VideoTileProvider	315
VideoTileProviderConfig	320
VideoTileStats	320
WeightEntry	321
WorkoutAnalyzer	325
WorkoutStatistics	332
Zone	334
ZoneCalculator	335
ZoneDistribution	340

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

AboutDialog	Displays application about/splash dialog	41
Activity	Represents a single recorded activity or workout	42
ActivityAnalysisCacheRepository	Caches expensive analysis results per activity	48
ActivityAnalysisSettings	Analysis settings per activity	50
ActivityAnalysisSettingsRepository	Repository for per-activity analysis settings	52
ActivityBrowser	Browse and manage activities	53
anonymous_namespace{ActivityBrowser.cpp}::ActivityDateTableItem		57
ActivityRepository	Repository for activity record persistence	58
AnalysisQueue	Manages sequential background analysis task queue	63
AnalysisWorker	Worker thread for background activity analysis	69
AppSettings	Application-wide settings singleton	73
Athlete	Represents an athlete profile	75
AthleteDialog	Dialog for creating or editing athlete profile	77
AthleteHeaderWidget	Header bar displaying current athlete and ride metrics	81
AthleteListDialog	Dialog for browsing and managing athletes	83
AthleteRepository	Repository for athlete profile data persistence	85
CalendarWidget	Calendar widget for activity scheduling and display	91
Climb	Represents a detected climb with comprehensive metrics	94
ClimbDetector	Detects and categorizes climbs in cycling activities	99
ClimbDetectorConfig	Configuration parameters for climb detection	106

ClimbEditCommand	
Undo/redo command for climb boundary or field edits	112
anonymous_namespace{RideChartWidget.cpp}::ClimbHandleHitResult	114
ClimbMetrics	
Computes comprehensive climb metrics from activity data	115
ClimbNameTableItem	
Table item that sorts climb names alphabetically	116
ClimbQuarterMetrics	
Metrics for a quarter of a climb segment	117
ClimbRecord	
Database representation of a stored climb	118
ClimbRepository	
Repository for climb record persistence	122
ClimbSortKeyTableItem	
Table item that sorts by numeric key, with text fallback	129
ColorContext	
Context parameters for determining zone colors based on athlete metrics	131
ColorProvider	
Provides color mapping for athletic metrics	133
CreateDatabaseDialog	
Dialog for creating a new database	134
DailyLoadPoint	
Daily training stress score value	137
DatabaseConnectionGuard	
RAII guard for temporary SQLite database connections	138
DatabaseManager	
Manages a single database connection and schema for FitlyzerC	139
DateFormatter	
Provides date/time formatting consistent with user settings	143
DuplicateActivityInfo	147
EditActivityDialog	
Dialog for editing activity metadata	147
EditNotesDialog	
Dialog for editing activity notes and weather	150
anonymous_namespace{VideoRenderJob.cpp}::ExportTileCleanupGuard	152
anonymous_namespace{FitParser.cpp}::FitFileIdListener	153
FitnessChartWidget	
Displays fitness metrics timeline (CTL, ATL, TSB)	154
FitnessMetricsPoint	
Fitness metrics for a single day	156
FitParser	
Parses FIT files and extracts activity recording data	157
FitRecordListener	
Listener for FIT record messages during parsing	159
MapFitMath::FitResult	
Result of fitting GPS bounds to map viewport	161
anonymous_namespace{FitParser.cpp}::FitSessionListener	163
FtpEntry	
Represents a historical FTP entry	164
FTPZones	
Analyzes time spent in FTP-based power zones	166
MainWindow::ImportBatchSummary	167
ImportJob	
Represents a single background FIT import request	168
ImportProgressModel	
UI-facing aggregate state for background import queue activity	170
ImportQueue	
Sequential queue that dispatches background FIT import jobs	174

ImportRecord	
Log entry for an activity import	179
ImportRepository	
Repository for import log persistence	180
ImportStatusWidget	
Toolbar widget showing live background import queue progress	182
ImportWorker	
Worker that imports FIT files in a dedicated background thread	184
Interval	
Represents a single work or recovery interval	187
IntervalDetector	
Detects work and recovery intervals in activity data	189
IntervalDetectorConfig	
Configuration parameters for interval detection	191
IntervalEditorDialog	
Dialog for editing interval metadata	192
IntervalRecord	
Database representation of an interval record	194
IntervalRepository	
Repository for interval record persistence	197
MainWindow	
Main application window	201
MapRenderer	
Interactive map widget for displaying and editing activity routes	229
NavigationSidebar	
Main navigation sidebar for app pages	237
PlannedWorkoutDialog	
Dialog for creating or editing planned workouts	239
PlannedWorkoutDraft	
Draft data for a planned workout	241
PowerCurve	
Computes power duration curve (PDC) for an activity	242
PowerCurvePoint	
A single point on a power duration curve	244
PowerCurveWidget	
Displays power output over duration curve	245
PowerHistogramWidget	
Displays power distribution histogram	248
PowerZone	
Represents a single power zone with time spent	250
RideChartWidget	
Interactive chart displaying ride data by selected metric	251
RideData	
Container for all recorded samples and metadata for a single activity	271
RideDataLoader	
Loads activity ride data samples from the database	272
RideDataSerializer	
Persists ride data to database	273
RideRecord	
A single recorded sample from an activity recording	274
anonymous_namespace(ClimbMetrics.cpp)::SliceStats	276
SmoothingEngine	
Provides data smoothing algorithms for activity metrics	277
TileCache	
Caches map tiles in memory and on disk	278
TileCacheConfig	
Configuration for tile caching behavior	283

TileProvider	
Tile provider configuration	283
TileCache::TileRequest	284
anonymous_namespace{VideoRenderJob.cpp}::TileViewport	285
TrainingLoad	
Computes training load metrics from activity power data	286
TrainingMetrics	
Computes training metrics from activity data	288
TrainingMetricsSummary	
Summary of training-related metrics for an activity	289
UndoCommand	
Abstract base class for all undo/redo operations	291
UndoManager	
Central undo/redo stack manager	292
VideoExportDialog	
Dialog for configuring and launching activity video export	296
VideoExportSettings	
Configuration for activity video export	303
VideoRenderJob	
Renders activity video with map overlay and data visualization	308
VideoTileId	
Unique identifier for a map tile (z/x/y coordinates)	313
VideoTileIdHash	
Hash function for VideoTileId	314
VideoTileProvider	
Manages map tiles for video export	315
VideoTileProviderConfig	
Configuration for video tile provider	320
VideoTileStats	
Tile availability statistics for export	320
WeightEntry	
Represents a historical weight entry	321
WelcomeWidget	
Welcome screen for first-time users	322
WorkoutAnalyzer	
Analyzes ride data to compute workout statistics	325
WorkoutController	
Manages activity loading, import, and analysis	326
WorkoutStatistics	
Statistics summary for a workout activity	332
Zone	
Represents a single training or power zone with bounds and color	334
ZoneCalculator	
Calculates training zones for various metrics	335
ZoneDistribution	
Distribution of time or distance spent in a specific zone	340

Chapter 5

File Index

5.1 File List

Here is a list of all documented files with brief descriptions:

src/ fitlyzerC.cpp	455
src/analysis/ ClimbDetector.cpp	343
src/analysis/ ClimbDetector.h	351
src/analysis/ ClimbMetrics.cpp	352
src/analysis/ ClimbMetrics.h	358
src/analysis/ FTPZones.cpp	359
src/analysis/ FTPZones.h	360
src/analysis/ IntervalDetector.cpp	360
src/analysis/ IntervalDetector.h	363
src/analysis/ PowerCurve.cpp	364
src/analysis/ PowerCurve.h	364
src/analysis/ SmoothingEngine.cpp	370
src/analysis/ SmoothingEngine.h	371
src/analysis/ TrainingLoad.cpp	371
src/analysis/ TrainingLoad.h	372
src/analysis/ TrainingMetrics.cpp	372
src/analysis/ TrainingMetrics.h	373
src/analysis/ WorkoutAnalyzer.cpp	374
src/analysis/ WorkoutAnalyzer.h	375
src/analysis/queue/ AnalysisQueue.cpp	365
src/analysis/queue/ AnalysisQueue.h	366
src/analysis/queue/ AnalysisWorker.cpp	366
src/analysis/queue/ AnalysisWorker.h	369
src/charts/ FitnessChartWidget.cpp	375
src/charts/ FitnessChartWidget.h	377
src/charts/ Metric.h	377
src/charts/ PowerCurveWidget.cpp	377
src/charts/ PowerCurveWidget.h	380
src/charts/ PowerHistogramWidget.cpp	380
src/charts/ PowerHistogramWidget.h	382
src/charts/ RideChartWidget.cpp	382
src/charts/ RideChartWidget.h	406
src/core/ ActivityAnalysisFlags.h	408
src/core/ AnalysisVersions.h	408
src/core/settings/ AppSettings.cpp	408
src/core/settings/ AppSettings.h	409
src/core/settings/ DateFormatter.cpp	410
src/core/settings/ DateFormatter.h	411
src/core/undo/ ClimbEditCommand.cpp	411
src/core/undo/ ClimbEditCommand.h	412

src/core/undo/UndoCommand.h	412
src/core/undo/UndoManager.cpp	412
src/core/undo/UndoManager.h	413
src/core/zones/ColorProvider.cpp	414
src/core/zones/ColorProvider.h	415
src/core/zones/ZoneCalculator.cpp	416
src/core/zones/ZoneCalculator.h	418
src/core/zones/ZoneDefinition.cpp	419
src/core/zones/ZoneDefinition.h	419
src/database/ActivityAnalysisCacheRepository.cpp	420
src/database/ActivityAnalysisCacheRepository.h	421
src/database/ActivityAnalysisSettingsRepository.cpp	421
src/database/ActivityAnalysisSettingsRepository.h	422
src/database/ActivityRepository.cpp	422
src/database/ActivityRepository.h	426
src/database/AthleteRepository.cpp	427
src/database/AthleteRepository.h	430
src/database/ClimbRepository.cpp	431
src/database/ClimbRepository.h	434
src/database/DatabaseConnectionGuard.cpp	435
src/database/DatabaseConnectionGuard.h	435
src/database/DatabaseManager.cpp	436
src/database/DatabaseManager.h	441
src/database/DatabaseSchema.cpp	441
src/database/DatabaseSchema.h	442
src/database/ImportRepository.cpp	445
src/database/ImportRepository.h	446
src/database/IntervalRepository.cpp	446
src/database/IntervalRepository.h	448
src/database/SqlQueryHelper.h	449
src/fit/FitParser.cpp	450
src/fit/FitParser.h	452
src/fit/FitRecordListener.cpp	452
src/fit/FitRecordListener.h	454
src/fit/RideData.h	454
src/gui/AboutDialog.cpp	455
src/gui/AboutDialog.h	456
src/gui/ActivityBrowser.cpp	456
src/gui/ActivityBrowser.h	461
src/gui/AthleteDialog.cpp	461
src/gui/AthleteDialog.h	465
src/gui/AthleteHeaderWidget.cpp	465
src/gui/AthleteHeaderWidget.h	466
src/gui/AthleteListDialog.cpp	466
src/gui/AthleteListDialog.h	468
src/gui/CalendarWidget.cpp	469
src/gui/CalendarWidget.h	472
src/gui/CreateDatabaseDialog.cpp	473
src/gui/CreateDatabaseDialog.h	475
src/gui/EditActivityDialog.cpp	475
src/gui/EditActivityDialog.h	477
src/gui/EditNotesDialog.cpp	477
src/gui/EditNotesDialog.h	478
src/gui/ImportStatusWidget.cpp	478
src/gui/ImportStatusWidget.h	479
src/gui/IntervalEditorDialog.cpp	479
src/gui/IntervalEditorDialog.h	480
src/gui/MainWindow.cpp	481

src/gui/MainWindow.h	538
src/gui/NavigationSidebar.cpp	542
src/gui/NavigationSidebar.h	542
src/gui/PlannedWorkoutDialog.cpp	543
src/gui/PlannedWorkoutDialog.h	544
src/gui/WelcomeWidget.cpp	545
src/gui/WelcomeWidget.h	546
src/gui/WorkoutController.cpp	546
src/gui/WorkoutController.h	552
src/import/ImportJob.cpp	554
src/import/ImportJob.h	554
src/import/ImportProgressModel.cpp	554
src/import/ImportProgressModel.h	555
src/import/ImportQueue.cpp	556
src/import/ImportQueue.h	557
src/import/ImportWorker.cpp	558
src/import/ImportWorker.h	562
src/maps/MapFitMath.cpp	562
src/maps/MapFitMath.h	564
src/maps/MapRenderer.cpp	564
src/maps/MapRenderer.h	574
src/maps/TileCache.cpp	576
src/maps/TileCache.h	581
src/model/RideDataLoader.cpp	582
src/model/RideDataLoader.h	583
src/model/RideDataSerializer.cpp	583
src/model/RideDataSerializer.h	585
src/platform/Platform.cpp	586
src/platform/Platform.h	586
src/platform/Platform_linux.cpp	586
src/platform/Platform_macos.cpp	587
src/platform/Platform_windows.cpp	587
src/utls/FfmpegPath.cpp	587
src/utls/FfmpegPath.h	588
src/utls/UuidGenerator.h	588
src/video/VideoExportDialog.cpp	589
src/video/VideoExportDialog.h	594
src/video/VideoExportSettings.h	595
src/video/VideoRenderJob.cpp	596
src/video/VideoRenderJob.h	608
src/video/VideoTileProvider.cpp	608
src/video/VideoTileProvider.h	610

Chapter 6

Namespace Documentation

6.1 ActivityAnalysisFlags Namespace Reference

Bitmask flags for analysis completion tracking.

Variables

- constexpr int [None](#) = 0
No analysis flags set.
- constexpr int [HasClimbs](#) = 1 << 0
Climb detection has been computed.
- constexpr int [HasIntervals](#) = 1 << 1
Interval detection has been computed.
- constexpr int [HasPowerCurve](#) = 1 << 2
Power curve has been computed.
- constexpr int [HasHistogram](#) = 1 << 3
Power histogram has been computed.
- constexpr int [HasVideo](#) = 1 << 4
Video export has been created.

6.1.1 Detailed Description

Bitmask flags for analysis completion tracking.

Flags are stored in `activities.analysis_flags` to track which analysis products have been computed, allowing the UI to warn on outdated results.

6.1.2 Variable Documentation

6.1.2.1 HasClimbs

```
int ActivityAnalysisFlags::HasClimbs = 1 << 0 [constexpr]
```

[Climb](#) detection has been computed.

Definition at line 22 of file [ActivityAnalysisFlags.h](#).

Referenced by [AnalysisWorker::processTask\(\)](#).

6.1.2.2 HasHistogram

```
int ActivityAnalysisFlags::HasHistogram = 1 << 3 [constexpr]
```

Power histogram has been computed.

Definition at line 37 of file [ActivityAnalysisFlags.h](#).

6.1.2.3 HasIntervals

```
int ActivityAnalysisFlags::HasIntervals = 1 << 1 [constexpr]
```

[Interval](#) detection has been computed.

Definition at line 27 of file [ActivityAnalysisFlags.h](#).

Referenced by [AnalysisWorker::processTask\(\)](#).

6.1.2.4 HasPowerCurve

```
int ActivityAnalysisFlags::HasPowerCurve = 1 << 2 [constexpr]
```

Power curve has been computed.

Definition at line 32 of file [ActivityAnalysisFlags.h](#).

6.1.2.5 HasVideo

```
int ActivityAnalysisFlags::HasVideo = 1 << 4 [constexpr]
```

Video export has been created.

Definition at line 42 of file [ActivityAnalysisFlags.h](#).

6.1.2.6 None

```
int ActivityAnalysisFlags::None = 0 [constexpr]
```

No analysis flags set.

Definition at line 17 of file [ActivityAnalysisFlags.h](#).

6.2 AnalysisVersions Namespace Reference

Algorithm version tracking for analysis results.

Variables

- constexpr int [ClimbDetection](#) = 1
Climb detection algorithm version.
- constexpr int [IntervalDetection](#) = 1
Interval detection algorithm version.

6.2.1 Detailed Description

Algorithm version tracking for analysis results.

Version constants are incremented when algorithms change to track data compatibility and warn on outdated results.

6.2.2 Variable Documentation

6.2.2.1 ClimbDetection

```
int AnalysisVersions::ClimbDetection = 1 [constexpr]
```

[Climb](#) detection algorithm version.

Definition at line 17 of file [AnalysisVersions.h](#).

Referenced by [AnalysisWorker::processTask\(\)](#).

6.2.2.2 IntervalDetection

```
int AnalysisVersions::IntervalDetection = 1 [constexpr]
```

[Interval](#) detection algorithm version.

Definition at line 22 of file [AnalysisVersions.h](#).

Referenced by [AnalysisWorker::processTask\(\)](#).

6.3 anonymous_namespace{ActivityBrowser.cpp} Namespace Reference

Classes

- class [ActivityDateTableItem](#)

Variables

- constexpr int [kNotesRole](#) = Qt::UserRole + 1
- constexpr int [kWeatherRole](#) = Qt::UserRole + 2
- constexpr int [kIsoDateRole](#) = Qt::UserRole + 3

6.3.1 Variable Documentation

6.3.1.1 kIsoDateRole

```
int anonymous_namespace{ActivityBrowser.cpp}::kIsoDateRole = Qt::UserRole + 3 [constexpr]
```

Definition at line 27 of file [ActivityBrowser.cpp](#).

6.3.1.2 kNotesRole

```
int anonymous_namespace{ActivityBrowser.cpp}::kNotesRole = Qt::UserRole + 1 [constexpr]
```

Definition at line 25 of file [ActivityBrowser.cpp](#).

6.3.1.3 kWeatherRole

```
int anonymous_namespace{ActivityBrowser.cpp}::kWeatherRole = Qt::UserRole + 2 [constexpr]
```

Definition at line 26 of file [ActivityBrowser.cpp](#).

6.4 anonymous_namespace{AnalysisWorker.cpp} Namespace Reference

Functions

- void [applySqlitePragmas](#) (QSqlDatabase &db)

6.4.1 Function Documentation

6.4.1.1 applySqlitePragmas()

```
void anonymous_namespace{AnalysisWorker.cpp}::applySqlitePragmas (  
    QSqlDatabase & db)
```

Definition at line 14 of file [AnalysisWorker.cpp](#).

6.5 anonymous_namespace{AthleteDialog.cpp} Namespace Reference

Functions

- QString [displayDateOrRaw](#) (const QString &raw)
- QString [tolsoOrRaw](#) (const QString &text)

Variables

- constexpr int [kEntryIdRole](#) = Qt::UserRole

6.5.1 Function Documentation

6.5.1.1 displayDateOrRaw()

```
QString anonymous_namespace{AthleteDialog.cpp}::displayDateOrRaw (
    const QString & raw)
```

Definition at line 27 of file [AthleteDialog.cpp](#).

6.5.1.2 toIsoOrRaw()

```
QString anonymous_namespace{AthleteDialog.cpp}::toIsoOrRaw (
    const QString & text)
```

Definition at line 33 of file [AthleteDialog.cpp](#).

6.5.2 Variable Documentation

6.5.2.1 kEntryIdRole

```
int anonymous_namespace{AthleteDialog.cpp}::kEntryIdRole = Qt::UserRole [constexpr]
```

Definition at line 25 of file [AthleteDialog.cpp](#).

6.6 anonymous_namespace{ClimbDetector.cpp} Namespace Reference

Functions

- bool [isFinite](#) (double value)
- double [clampValue](#) (double value, double lo, double hi)
- double [segmentMeters](#) (double lat1, double lon1, double lat2, double lon2)
- std::vector< double > [buildFilteredAltitudeMeters](#) (const [RideData](#) &rideData, const std::vector< double > &cumulativeDistanceMeters, double maxVerticalSpeedMps)
- double [estimateSpeedKmhAt](#) (const [RideData](#) &rideData, const std::vector< double > &cumulativeDistanceMeters, int index)

6.6.1 Function Documentation

6.6.1.1 buildFilteredAltitudeMeters()

```
std::vector< double > anonymous_namespace{ClimbDetector.cpp}::buildFilteredAltitudeMeters (
    const RideData & rideData,
    const std::vector< double > & cumulativeDistanceMeters,
    double maxVerticalSpeedMps)
```

Definition at line 31 of file [ClimbDetector.cpp](#).

6.6.1.2 clampValue()

```
double anonymous_namespace{ClimbDetector.cpp}::clampValue (
    double value,
    double lo,
    double hi)
```

Definition at line 18 of file [ClimbDetector.cpp](#).

6.6.1.3 estimateSpeedKmhAt()

```
double anonymous_namespace{ClimbDetector.cpp}::estimateSpeedKmhAt (
    const RideData & rideData,
    const std::vector< double > & cumulativeDistanceMeters,
    int index)
```

Definition at line 79 of file [ClimbDetector.cpp](#).

6.6.1.4 isFinite()

```
bool anonymous_namespace{ClimbDetector.cpp}::isFinite (
    double value)
```

Definition at line 13 of file [ClimbDetector.cpp](#).

6.6.1.5 segmentMeters()

```
double anonymous_namespace{ClimbDetector.cpp}::segmentMeters (
    double lat1,
    double lon1,
    double lat2,
    double lon2)
```

Definition at line 23 of file [ClimbDetector.cpp](#).

6.7 anonymous_namespace{ClimbMetrics.cpp} Namespace Reference

Classes

- struct [SliceStats](#)

Functions

- bool [isFinite](#) (double value)
- double [computeLinearRegressionSlope](#) (const std::vector< double > &x, const std::vector< double > &y, double *interceptOut=nullptr)
- QString [classifyClimbCategory](#) (double gainM, double difficultyScore)
- QString [classifyClimbShape](#) (const std::vector< double > &gradients, double lengthKm, double gainM, double avgGradient, double maxGradient)
- [SliceStats computeSliceStats](#) (const [RideData](#) &rideData, int startIdx, int endIdx, const std::vector< double > &basis, double sliceStart, double sliceEnd)
- double [computeNormalizedPower](#) (const std::vector< double > &power)

6.7.1 Function Documentation

6.7.1.1 classifyClimbCategory()

```
QString anonymous_namespace{ClimbMetrics.cpp}::classifyClimbCategory (
    double gainM,
    double difficultyScore)
```

Definition at line 64 of file [ClimbMetrics.cpp](#).

6.7.1.2 classifyClimbShape()

```
QString anonymous_namespace{ClimbMetrics.cpp}::classifyClimbShape (
    const std::vector< double > &gradients,
    double lengthKm,
    double gainM,
    double avgGradient,
    double maxGradient)
```

Definition at line 77 of file [ClimbMetrics.cpp](#).

6.7.1.3 computeLinearRegressionSlope()

```
double anonymous_namespace{ClimbMetrics.cpp}::computeLinearRegressionSlope (
    const std::vector< double > & x,
    const std::vector< double > & y,
    double * interceptOut = nullptr)
```

Definition at line 24 of file [ClimbMetrics.cpp](#).

6.7.1.4 computeNormalizedPower()

```
double anonymous_namespace{ClimbMetrics.cpp}::computeNormalizedPower (
    const std::vector< double > & power)
```

Definition at line 194 of file [ClimbMetrics.cpp](#).

6.7.1.5 computeSliceStats()

```
SliceStats anonymous_namespace{ClimbMetrics.cpp}::computeSliceStats (
    const RideData & rideData,
    int startIdx,
    int endIdx,
    const std::vector< double > & basis,
    double sliceStart,
    double sliceEnd)
```

Definition at line 141 of file [ClimbMetrics.cpp](#).

6.7.1.6 isFinite()

```
bool anonymous_namespace{ClimbMetrics.cpp}::isFinite (
    double value)
```

Definition at line 19 of file [ClimbMetrics.cpp](#).

6.8 anonymous_namespace{ClimbRepository.cpp} Namespace Reference

Functions

- [ClimbRecord](#) [recordFromQuery](#) (QSqlQuery &q)

Variables

- const char * [kSelectCols](#)

6.8.1 Function Documentation

6.8.1.1 recordFromQuery()

```
ClimbRecord anonymous_namespace{ClimbRepository.cpp}::recordFromQuery (
    QSqlQuery & q)
```

Definition at line 24 of file [ClimbRepository.cpp](#).

6.8.2 Variable Documentation

6.8.2.1 kSelectCols

```
const char* anonymous_namespace{ClimbRepository.cpp}::kSelectCols
```

Initial value:

```
=
"id, activity_id, COALESCE(uuid,"), start_seconds, end_seconds,"
" COALESCE(original_start_seconds,-1.0), COALESCE(original_end_seconds,-1.0),"
" COALESCE(name,"), COALESCE(elevation_gain_m,0.0), COALESCE(length_km,0.0),"
" COALESCE(average_gradient,0.0),"
" COALESCE(avg_power,0.0), COALESCE(np,0.0), COALESCE(avg_hr,0.0),"
" COALESCE(avg_cadence,0.0), COALESCE(vam,0.0),"
" COALESCE(source,'auto'), COALESCE(locked,0), COALESCE(deleted,0),"
" COALESCE(algorithm_version,1),"
" COALESCE(notes,"), COALESCE(favorite,0), COALESCE(rating,0)"
```

Definition at line 13 of file [ClimbRepository.cpp](#).

6.9 anonymous_namespace{ColorProvider.cpp} Namespace Reference

Functions

- QColor [interpolateColor](#) (const QColor &a, const QColor &b, double t)

6.9.1 Function Documentation

6.9.1.1 interpolateColor()

```
QColor anonymous_namespace{ColorProvider.cpp}::interpolateColor (  
    const QColor & a,  
    const QColor & b,  
    double t)
```

Definition at line 12 of file [ColorProvider.cpp](#).

6.10 anonymous_namespace{DateFormatter.cpp} Namespace Reference

Functions

- QString [datePatternForFormat](#) (DateFormat format)

6.10.1 Function Documentation

6.10.1.1 datePatternForFormat()

```
QString anonymous_namespace{DateFormatter.cpp}::datePatternForFormat (  
    DateFormat format)
```

Definition at line 12 of file [DateFormatter.cpp](#).

6.11 anonymous_namespace{FfmpegPath.cpp} Namespace Reference

Functions

- QString [absoluteExecutablePath](#) (const QString &candidate)

6.11.1 Function Documentation

6.11.1.1 absoluteExecutablePath()

```
QString anonymous_namespace{FfmpegPath.cpp}::absoluteExecutablePath (  
    const QString & candidate)
```

Definition at line 13 of file [FfmpegPath.cpp](#).

6.12 anonymous_namespace{FitParser.cpp} Namespace Reference

Classes

- class [FitSessionListener](#)
- class [FitFileIdListener](#)

Functions

- QString [fitTimestampToIsoUtc](#) (FIT_DATE_TIME timestamp)

6.12.1 Function Documentation

6.12.1.1 fitTimestampToIsoUtc()

```
QString anonymous_namespace{FitParser.cpp}::fitTimestampToIsoUtc (
    FIT_DATE_TIME timestamp)
```

Definition at line 21 of file [FitParser.cpp](#).

6.13 anonymous_namespace{ImportWorker.cpp} Namespace Reference

Functions

- void [applySqlitePragmas](#) (QSqlDatabase &db)

6.13.1 Function Documentation

6.13.1.1 applySqlitePragmas()

```
void anonymous_namespace{ImportWorker.cpp}::applySqlitePragmas (
    QSqlDatabase & db)
```

Definition at line 24 of file [ImportWorker.cpp](#).

6.14 anonymous_namespace{MapFitMath.cpp} Namespace Reference

Functions

- double [normalizeLongitude](#) (double lon)
- double [latToTileY](#) (double lat)
- double [tileYToLat](#) (double tileY)

Variables

- constexpr double [kPi](#) = 3.14159265358979323846
- constexpr int [kTilePx](#) = 256
- constexpr double [kMinLatSpan](#) = 0.001
- constexpr double [kMinLonSpan](#) = 0.001
- constexpr double [kMaxMercatorLat](#) = 85.051129

6.14.1 Function Documentation

6.14.1.1 latToTileY()

```
double anonymous_namespace{MapFitMath.cpp}::latToTileY (
    double lat)
```

Definition at line 25 of file [MapFitMath.cpp](#).

6.14.1.2 normalizeLongitude()

```
double anonymous_namespace{MapFitMath.cpp}::normalizeLongitude (
    double lon)
```

Definition at line 17 of file [MapFitMath.cpp](#).

6.14.1.3 tileYToLat()

```
double anonymous_namespace{MapFitMath.cpp}::tileYToLat (
    double tileY)
```

Definition at line 32 of file [MapFitMath.cpp](#).

6.14.2 Variable Documentation

6.14.2.1 kMaxMercatorLat

`double anonymous_namespace{MapFitMath.cpp}::kMaxMercatorLat = 85.051129 [constexpr]`

Definition at line 15 of file [MapFitMath.cpp](#).

6.14.2.2 kMinLatSpan

`double anonymous_namespace{MapFitMath.cpp}::kMinLatSpan = 0.001 [constexpr]`

Definition at line 13 of file [MapFitMath.cpp](#).

6.14.2.3 kMinLonSpan

`double anonymous_namespace{MapFitMath.cpp}::kMinLonSpan = 0.001 [constexpr]`

Definition at line 14 of file [MapFitMath.cpp](#).

6.14.2.4 kPi

`double anonymous_namespace{MapFitMath.cpp}::kPi = 3.14159265358979323846 [constexpr]`

Definition at line 11 of file [MapFitMath.cpp](#).

6.14.2.5 kTilePx

`int anonymous_namespace{MapFitMath.cpp}::kTilePx = 256 [constexpr]`

Definition at line 12 of file [MapFitMath.cpp](#).

6.15 anonymous_namespace{RideChartWidget.cpp} Namespace Reference

Classes

- struct [ClimbHandleHitResult](#)

Functions

- [ClimbHandleHitResult hitTestClimbHandles](#) (QChart *chart, QLineSeries *series, const QValueAxis *axisY, const [Climb](#) &climb, const QPointF &scenePos)

Variables

- constexpr double [kClimbHandleCenterOffsetYPx](#) = 10.0
- constexpr double [kClimbHandleRadiusPx](#) = 8.0
- constexpr double [kClimbHandleGrabRadiusPx](#) = 24.0

6.15.1 Function Documentation

6.15.1.1 hitTestClimbHandles()

```
ClimbHandleHitResult anonymous_namespace{RideChartWidget.cpp}::hitTestClimbHandles (
    QChart * chart,
    QLineSeries * series,
    const QValueAxis * axisY,
    const Climb & climb,
    const QPointF & scenePos)
```

Definition at line 43 of file [RideChartWidget.cpp](#).

6.15.2 Variable Documentation

6.15.2.1 kClimbHandleCenterOffsetYPx

`double anonymous_namespace{RideChartWidget.cpp}::kClimbHandleCenterOffsetYPx = 10.0 [constexpr]`
 Definition at line 33 of file [RideChartWidget.cpp](#).

6.15.2.2 kClimbHandleGrabRadiusPx

`double anonymous_namespace{RideChartWidget.cpp}::kClimbHandleGrabRadiusPx = 24.0 [constexpr]`
 Definition at line 35 of file [RideChartWidget.cpp](#).

6.15.2.3 kClimbHandleRadiusPx

`double anonymous_namespace{RideChartWidget.cpp}::kClimbHandleRadiusPx = 8.0 [constexpr]`
 Definition at line 34 of file [RideChartWidget.cpp](#).

6.16 anonymous_namespace{TileCache.cpp} Namespace Reference

Functions

- `QString defaultDiskCacheRoot ()`

6.16.1 Function Documentation

6.16.1.1 defaultDiskCacheRoot()

`QString anonymous_namespace{TileCache.cpp}::defaultDiskCacheRoot ()`
 Definition at line 19 of file [TileCache.cpp](#).

6.17 anonymous_namespace{VideoExportDialog.cpp} Namespace Reference

Functions

- `QString mapStyleName (MapStyle style)`
- `QString makeDefaultFilename (const VideoExportSettings &defaults)`
- `QString formatVideoLength (double seconds)`

6.17.1 Function Documentation

6.17.1.1 formatVideoLength()

`QString anonymous_namespace{VideoExportDialog.cpp}::formatVideoLength (double seconds)`
 Definition at line 73 of file [VideoExportDialog.cpp](#).

6.17.1.2 makeDefaultFilename()

`QString anonymous_namespace{VideoExportDialog.cpp}::makeDefaultFilename (const VideoExportSettings & defaults)`
 Definition at line 50 of file [VideoExportDialog.cpp](#).

6.17.1.3 mapStyleName()

`QString anonymous_namespace{VideoExportDialog.cpp}::mapStyleName (MapStyle style)`
 Definition at line 35 of file [VideoExportDialog.cpp](#).

6.18 anonymous_namespace{VideoRenderJob.cpp} Namespace Reference

Classes

- struct [TileViewport](#)
- class [ExportTileCleanupGuard](#)

Functions

- [TileViewport computeTileViewport](#) (const QPointF ¢erTile, int width, int height)
- std::vector< [VideoTileId](#) > [tilesForViewport](#) (const [TileViewport](#) &viewport, int zoom)
- QPointF [latLonToTile](#) (double lat, double lon, int zoom)
- QPointF [tileToScreen](#) (const QPointF &tilePt, const QPointF ¢erTile, int width, int height)
- int [nearestRecordIndex](#) (const std::vector< [RideRecord](#) > &records, double elapsedSeconds)
- bool [interpolateGpsAtTime](#) (const std::vector< const [RideRecord](#) * > &gpsRecords, double elapsedSeconds, double &outLat, double &outLon)
- QString [fmtDuration](#) (double seconds)
- QString [formatLegendValue](#) (double value, ColorMetric metric)
- QString [zoneLegendText](#) (const [Zone](#) &zone, ColorMetric metric)
- QColor [routeColor](#) (const [RideRecord](#) &rec, ColorMetric metric, const [ColorContext](#) &context, const QColor &singleColor)

Variables

- static constexpr int [kTilePx](#) = 256
- static constexpr double [kPi](#) = 3.14159265358979323846
- static constexpr int [kVideoExportTileCache](#) = 96

6.18.1 Function Documentation

6.18.1.1 computeTileViewport()

```
TileViewport anonymous_namespace{VideoRenderJob.cpp}::computeTileViewport (
    const QPointF & centerTile,
    int width,
    int height)
```

Definition at line 51 of file [VideoRenderJob.cpp](#).

6.18.1.2 fmtDuration()

```
QString anonymous_namespace{VideoRenderJob.cpp}::fmtDuration (
    double seconds)
```

Definition at line 172 of file [VideoRenderJob.cpp](#).

6.18.1.3 formatLegendValue()

```
QString anonymous_namespace{VideoRenderJob.cpp}::formatLegendValue (
    double value,
    ColorMetric metric)
```

Definition at line 183 of file [VideoRenderJob.cpp](#).

6.18.1.4 interpolateGpsAtTime()

```
bool anonymous_namespace{VideoRenderJob.cpp}::interpolateGpsAtTime (
    const std::vector< const RideRecord * > & gpsRecords,
    double elapsedSeconds,
    double & outLat,
    double & outLon)
```

Definition at line 118 of file [VideoRenderJob.cpp](#).

6.18.1.5 latLonToTile()

```
QPointF anonymous_namespace{VideoRenderJob.cpp}::latLonToTile (
    double lat,
    double lon,
    int zoom)
```

Definition at line 82 of file [VideoRenderJob.cpp](#).

6.18.1.6 nearestRecordIndex()

```
int anonymous_namespace{VideoRenderJob.cpp}::nearestRecordIndex (
    const std::vector< RideRecord > & records,
    double elapsedSeconds)
```

Definition at line 99 of file [VideoRenderJob.cpp](#).

6.18.1.7 routeColor()

```
QColor anonymous_namespace{VideoRenderJob.cpp}::routeColor (
    const RideRecord & rec,
    ColorMetric metric,
    const ColorContext & context,
    const QColor & singleColor)
```

Definition at line 221 of file [VideoRenderJob.cpp](#).

6.18.1.8 tilesForViewport()

```
std::vector< VideoTileId > anonymous_namespace{VideoRenderJob.cpp}::tilesForViewport (
    const TileViewport & viewport,
    int zoom)
```

Definition at line 61 of file [VideoRenderJob.cpp](#).

6.18.1.9 tileToScreen()

```
QPointF anonymous_namespace{VideoRenderJob.cpp}::tileToScreen (
    const QPointF & tilePt,
    const QPointF & centerTile,
    int width,
    int height)
```

Definition at line 91 of file [VideoRenderJob.cpp](#).

6.18.1.10 zoneLegendText()

```
QString anonymous_namespace{VideoRenderJob.cpp}::zoneLegendText (
    const Zone & zone,
    ColorMetric metric)
```

Definition at line 202 of file [VideoRenderJob.cpp](#).

6.18.2 Variable Documentation

6.18.2.1 kPi

`double anonymous_namespace{VideoRenderJob.cpp}::kPi = 3.14159265358979323846 [static], [constexpr]`
Definition at line 32 of file [VideoRenderJob.cpp](#).

6.18.2.2 kTilePx

`int anonymous_namespace{VideoRenderJob.cpp}::kTilePx = 256 [static], [constexpr]`
Definition at line 31 of file [VideoRenderJob.cpp](#).

6.18.2.3 kVideoExportTileCache

`int anonymous_namespace{VideoRenderJob.cpp}::kVideoExportTileCache = 96 [static], [constexpr]`
Definition at line 33 of file [VideoRenderJob.cpp](#).

6.19 anonymous_namespace{VideoTileProvider.cpp} Namespace Reference

Functions

- `QString defaultVideoExportCacheRoot ()`
- `QString defaultPrimaryTileCacheRoot ()`

6.19.1 Function Documentation

6.19.1.1 defaultPrimaryTileCacheRoot()

`QString anonymous_namespace{VideoTileProvider.cpp}::defaultPrimaryTileCacheRoot ()`
Definition at line 21 of file [VideoTileProvider.cpp](#).

6.19.1.2 defaultVideoExportCacheRoot()

`QString anonymous_namespace{VideoTileProvider.cpp}::defaultVideoExportCacheRoot ()`
Definition at line 13 of file [VideoTileProvider.cpp](#).

6.20 anonymous_namespace{WorkoutController.cpp} Namespace Reference

Functions

- `QDate activityDateForFtp (const Activity &activity)`
- `int resolveAthleteFtpForDate (AthleteRepository &repo, int athleteId, const QDate &date)`

6.20.1 Function Documentation

6.20.1.1 activityDateForFtp()

`QDate anonymous_namespace{WorkoutController.cpp}::activityDateForFtp (const Activity & activity)`
Definition at line 31 of file [WorkoutController.cpp](#).

6.20.1.2 resolveAthleteFtpForDate()

```
int anonymous_namespace{WorkoutController.cpp}::resolveAthleteFtpForDate (
    AthleteRepository & repo,
    int athleteId,
    const QDate & date)
```

Definition at line 39 of file [WorkoutController.cpp](#).

6.21 anonymous_namespace{ZoneCalculator.cpp} Namespace Reference

Functions

- double [valueForMetric](#) (const [RideRecord](#) &record, ColorMetric metric)
- bool [sampleAvailable](#) (const [RideRecord](#) &record, ColorMetric metric)
- double [sampleDurationSeconds](#) (const std::vector< [RideRecord](#) > &records, int index)

6.21.1 Function Documentation

6.21.1.1 sampleAvailable()

```
bool anonymous_namespace{ZoneCalculator.cpp}::sampleAvailable (
    const RideRecord & record,
    ColorMetric metric)
```

Definition at line 27 of file [ZoneCalculator.cpp](#).

6.21.1.2 sampleDurationSeconds()

```
double anonymous_namespace{ZoneCalculator.cpp}::sampleDurationSeconds (
    const std::vector< RideRecord > & records,
    int index)
```

Definition at line 43 of file [ZoneCalculator.cpp](#).

6.21.1.3 valueForMetric()

```
double anonymous_namespace{ZoneCalculator.cpp}::valueForMetric (
    const RideRecord & record,
    ColorMetric metric)
```

Definition at line 11 of file [ZoneCalculator.cpp](#).

6.22 Platform Namespace Reference

Platform-specific abstractions and utilities.

Functions

- QString [applicationDataDirectory](#) ()
- QString [defaultDatabaseDirectory](#) ()
- QString [defaultDatabasePath](#) ()
- QString [defaultBackupPath](#) (const QString &databasePath)
- void [openInFileManager](#) (const QString &path)

6.22.1 Detailed Description

Platform-specific abstractions and utilities.

Provides OS-agnostic access to system paths and file operations, with implementation specific to macOS, Linux, or Windows.

6.22.2 Function Documentation

6.22.2.1 applicationDataDirectory()

QString Platform::applicationDataDirectory ()

Definition at line 12 of file [Platform.cpp](#).

6.22.2.2 defaultBackupPath()

QString Platform::defaultBackupPath (
const QString & databasePath)

Definition at line 36 of file [Platform.cpp](#).

6.22.2.3 defaultDatabaseDirectory()

QString Platform::defaultDatabaseDirectory ()

Definition at line 22 of file [Platform.cpp](#).

6.22.2.4 defaultDatabasePath()

QString Platform::defaultDatabasePath ()

Definition at line 31 of file [Platform.cpp](#).

6.23 SqlQueryHelper Namespace Reference

Common SQL patterns and query builders to reduce duplication across repositories.

Functions

- void [bindBoolValue](#) (QSqlQuery &query, const QString &placeholder, bool value)
Binds a boolean value to a query placeholder (0 or 1).
- QString [selectExistsNonDeleted](#) (const QString &tableName)
Generates EXISTS query for non-deleted records.
- QString [selectExistsLocked](#) (const QString &tableName)
Generates EXISTS query for locked records.
- QString [selectExistsEver](#) (const QString &tableName)
Generates EXISTS query regardless of deletion state.

Variables

- constexpr std::string_view [kWhereNotDeleted](#) = "COALESCE(deleted,0)=0"
SQL predicate for non-deleted rows.
- constexpr std::string_view [kWhereDeleted](#) = "COALESCE(deleted,0)=1"
SQL predicate for deleted rows.
- constexpr std::string_view [kWhereLocked](#) = "locked=1"
SQL predicate for locked rows.

6.23.1 Detailed Description

Common SQL patterns and query builders to reduce duplication across repositories.
Helper utilities for SQL query generation and binding.

6.23.2 Function Documentation

6.23.2.1 bindBoolValue()

```
void SqlQueryHelper::bindBoolValue (
    QSqlQuery & query,
    const QString & placeholder,
    bool value) [inline]
```

Binds a boolean value to a query placeholder (0 or 1).

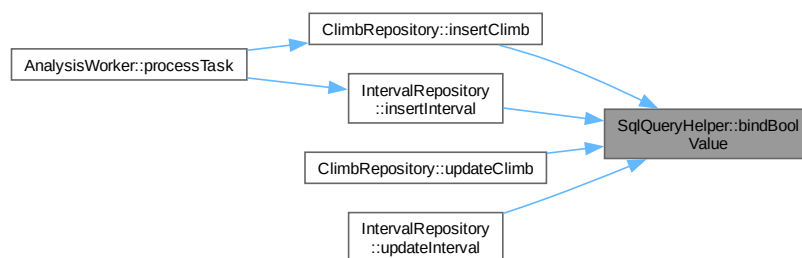
Parameters

<i>query</i>	SQL query to bind to.
<i>placeholder</i>	Placeholder name (e.g., ":locked").
<i>value</i>	Boolean value to bind.

Definition at line 32 of file [SqlQueryHelper.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), [IntervalRepository::insertInterval\(\)](#), [ClimbRepository::updateClimb\(\)](#), and [IntervalRepository::updateInterval\(\)](#).

Here is the caller graph for this function:



6.23.2.2 selectExistsEver()

```
QString SqlQueryHelper::selectExistsEver (
    const QString & tableName) [inline]
```

Generates EXISTS query regardless of deletion state.

Parameters

<i>tableName</i>	Table to query.
------------------	-----------------

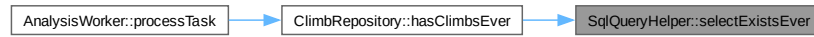
Returns

Query string (requires :id parameter).

Definition at line 65 of file [SqlQueryHelper.h](#).

Referenced by [ClimbRepository::hasClimbsEver\(\)](#).

Here is the caller graph for this function:

**6.23.2.3 selectExistsLocked()**

```
QString SqlQueryHelper::selectExistsLocked (
    const QString & tableName) [inline]
```

Generates EXISTS query for locked records.

Parameters

<i>tableName</i>	Table to query.
------------------	-----------------

Returns

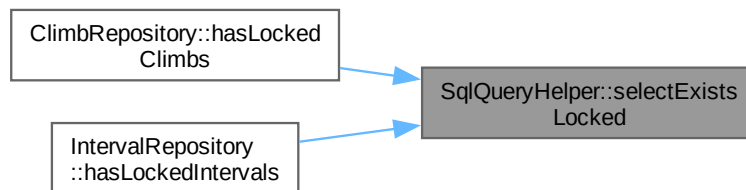
Query string (requires :id parameter).

Definition at line 53 of file [SqlQueryHelper.h](#).

References [kWhereLocked](#), and [kWhereNotDeleted](#).

Referenced by [ClimbRepository::hasLockedClimbs\(\)](#), and [IntervalRepository::hasLockedIntervals\(\)](#).

Here is the caller graph for this function:

**6.23.2.4 selectExistsNonDeleted()**

```
QString SqlQueryHelper::selectExistsNonDeleted (
    const QString & tableName) [inline]
```

Generates EXISTS query for non-deleted records.

Parameters

<i>tableName</i>	Table to query.
------------------	-----------------

Returns

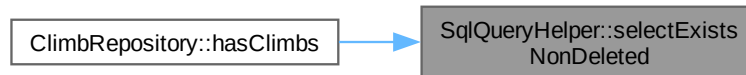
Query string (requires :id parameter).

Definition at line 42 of file [SqlQueryHelper.h](#).

References [kWhereNotDeleted](#).

Referenced by [ClimbRepository::hasClimbs\(\)](#).

Here is the caller graph for this function:



6.23.3 Variable Documentation

6.23.3.1 kWhereDeleted

```
std::string_view SqlQueryHelper::kWhereDeleted = "COALESCE(deleted,0)=1" [inline], [constexpr]
```

SQL predicate for deleted rows.

Definition at line 21 of file [SqlQueryHelper.h](#).

6.23.3.2 kWhereLocked

```
std::string_view SqlQueryHelper::kWhereLocked = "locked=1" [inline], [constexpr]
```

SQL predicate for locked rows.

Definition at line 24 of file [SqlQueryHelper.h](#).

Referenced by [selectExistsLocked\(\)](#).

6.23.3.3 kWhereNotDeleted

```
std::string_view SqlQueryHelper::kWhereNotDeleted = "COALESCE(deleted,0)=0" [inline], [constexpr]
```

SQL predicate for non-deleted rows.

Definition at line 18 of file [SqlQueryHelper.h](#).

Referenced by [selectExistsLocked\(\)](#), and [selectExistsNonDeleted\(\)](#).

Chapter 7

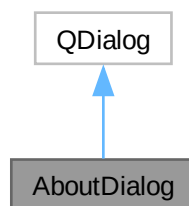
Class Documentation

7.1 AboutDialog Class Reference

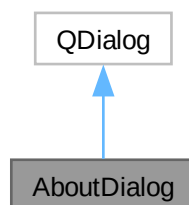
Displays application about/splash dialog.

```
#include <AboutDialog.h>
```

Inheritance diagram for AboutDialog:



Collaboration diagram for AboutDialog:



Public Member Functions

- [AboutDialog](#) (QWidget *parent=nullptr)
Constructs about dialog.

7.1.1 Detailed Description

Displays application about/splash dialog.
Definition at line 11 of file [AboutDialog.h](#).

7.1.2 Constructor & Destructor Documentation

7.1.2.1 AboutDialog()

```
AboutDialog::AboutDialog (
    QWidget * parent = nullptr) [explicit]
```

Constructs about dialog.

Parameters

<i>parent</i>	Parent widget.
---------------	----------------

Definition at line 13 of file [AboutDialog.cpp](#).

The documentation for this class was generated from the following files:

- [src/gui/AboutDialog.h](#)
- [src/gui/AboutDialog.cpp](#)

7.2 Activity Struct Reference

Represents a single recorded activity or workout.
`#include <ActivityRepository.h>`

Public Member Functions

- bool [isValid](#) () const
Checks if activity data is valid.

Public Attributes

- int [id](#) = -1
Unique activity identifier.
- int [athleteId](#) = -1
Associated athlete ID.
- QString [fitHash](#)
SHA-256 hash of original FIT file for duplicate detection.
- QString [fileName](#)
Original filename.
- QString [sport](#)
Sport type (cycling, running, etc.).
- QString [startTime](#)
ISO8601 activity start time.
- QString [endTime](#)
ISO8601 activity end time.
- double [durationSec](#) = 0.0
Total duration in seconds.
- double [distanceM](#) = 0.0
Total distance in meters.
- double [avgPower](#) = 0.0
Average power in watts.

- double `maxPower` = 0.0
Maximum power in watts.
- double `normalizedPower` = 0.0
Normalized power in watts.
- double `avgHr` = 0.0
Average heart rate in bpm.
- double `maxHr` = 0.0
Maximum heart rate in bpm.
- double `avgCadence` = 0.0
Average cadence in rpm.
- double `avgSpeed` = 0.0
Average speed in km/h.
- double `elevationGain` = 0.0
Total elevation gain in meters.
- QString `notes`
User activity notes.
- QString `weatherNotes`
Weather/environmental notes.
- double `temperature` = 0.0
Temperature in Celsius.
- QString `weather`
Weather description.
- QString `wind`
Wind conditions.
- int `rpe` = 0
Rate of perceived exertion (1-10).
- int `fatigue` = 0
Fatigue level (1-10).
- double `sleepHours` = 0.0
Sleep before activity in hours.
- double `weightKg` = 0.0
Athlete weight in kg.
- QString `bike`
Bike used.
- QString `equipment`
Equipment used.
- QString `importedAt`
ISO8601 import timestamp.
- int `analysisVersion` = 1
Analysis algorithm version used.
- QString `fingerprint`
Content hash for detecting meaningful changes.
- int `analysisFlags` = 0
Bitmask of analysis flags (climbs, intervals, etc.).
- QString `importSource`
Source of import (file, web, api, etc.).

7.2.1 Detailed Description

Represents a single recorded activity or workout.

Contains all activity metadata, measurements, and athlete information.

Definition at line 16 of file [ActivityRepository.h](#).

7.2.2 Member Function Documentation

7.2.2.1 isValid()

```
bool Activity::isValid () const [inline]
```

Checks if activity data is valid.

Returns

True if activity has a valid positive ID.

Definition at line 121 of file [ActivityRepository.h](#).

7.2.3 Member Data Documentation

7.2.3.1 analysisFlags

```
int Activity::analysisFlags = 0
```

Bitmask of analysis flags (climbs, intervals, etc.).

Definition at line 112 of file [ActivityRepository.h](#).

7.2.3.2 analysisVersion

```
int Activity::analysisVersion = 1
```

Analysis algorithm version used.

Definition at line 106 of file [ActivityRepository.h](#).

7.2.3.3 athleteId

```
int Activity::athleteId = -1
```

Associated athlete ID.

Definition at line 22 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

7.2.3.4 avgCadence

```
double Activity::avgCadence = 0.0
```

Average cadence in rpm.

Definition at line 61 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

7.2.3.5 avgHr

```
double Activity::avgHr = 0.0
```

Average heart rate in bpm.

Definition at line 55 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

7.2.3.6 avgPower

```
double Activity::avgPower = 0.0
```

Average power in watts.

Definition at line 46 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

7.2.3.7 avgSpeed

```
double Activity::avgSpeed = 0.0
```

Average speed in km/h.

Definition at line 64 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

7.2.3.8 bike

```
QString Activity::bike
```

Bike used.

Definition at line 97 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ActivityRepository::updateActivityProperties\(\)](#).

7.2.3.9 distanceM

```
double Activity::distanceM = 0.0
```

Total distance in meters.

Definition at line 43 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

7.2.3.10 durationSec

```
double Activity::durationSec = 0.0
```

Total duration in seconds.

Definition at line 40 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

7.2.3.11 elevationGain

```
double Activity::elevationGain = 0.0
```

Total elevation gain in meters.

Definition at line 67 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

7.2.3.12 endTime

```
QString Activity::endTime
```

ISO8601 activity end time.

Definition at line 37 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

7.2.3.13 equipment

```
QString Activity::equipment
```

Equipment used.

Definition at line 100 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ActivityRepository::updateActivityProperties\(\)](#).

7.2.3.14 fatigue

```
int Activity::fatigue = 0
```

Fatigue level (1-10).

Definition at line 88 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ActivityRepository::updateActivityProperties\(\)](#).

7.2.3.15 fileName

```
QString Activity::fileName
```

Original filename.

Definition at line 28 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

7.2.3.16 fingerprint

```
QString Activity::fingerprint
```

Content hash for detecting meaningful changes.

Definition at line 109 of file [ActivityRepository.h](#).

7.2.3.17 fitHash

`QString Activity::fitHash`

SHA-256 hash of original FIT file for duplicate detection.

Definition at line 25 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

7.2.3.18 id

`int Activity::id = -1`

Unique activity identifier.

Definition at line 19 of file [ActivityRepository.h](#).

7.2.3.19 importedAt

`QString Activity::importedAt`

ISO8601 import timestamp.

Definition at line 103 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

7.2.3.20 importSource

`QString Activity::importSource`

Source of import (file, web, api, etc.).

Definition at line 115 of file [ActivityRepository.h](#).

7.2.3.21 maxHr

`double Activity::maxHr = 0.0`

Maximum heart rate in bpm.

Definition at line 58 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

7.2.3.22 maxPower

`double Activity::maxPower = 0.0`

Maximum power in watts.

Definition at line 49 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

7.2.3.23 normalizedPower

`double Activity::normalizedPower = 0.0`

Normalized power in watts.

Definition at line 52 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

7.2.3.24 notes

`QString Activity::notes`

User activity notes.

Definition at line 70 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ActivityRepository::updateActivityProperties\(\)](#).

7.2.3.25 rpe

```
int Activity::rpe = 0
```

Rate of perceived exertion (1-10).

Definition at line 85 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ActivityRepository::updateActivityProperties\(\)](#).

7.2.3.26 sleepHours

```
double Activity::sleepHours = 0.0
```

Sleep before activity in hours.

Definition at line 91 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ActivityRepository::updateActivityProperties\(\)](#).

7.2.3.27 sport

```
QString Activity::sport
```

Sport type (cycling, running, etc.).

Definition at line 31 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

7.2.3.28 startTime

```
QString Activity::startTime
```

ISO8601 activity start time.

Definition at line 34 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

7.2.3.29 temperature

```
double Activity::temperature = 0.0
```

Temperature in Celsius.

Definition at line 76 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ActivityRepository::updateActivityProperties\(\)](#).

7.2.3.30 weather

```
QString Activity::weather
```

Weather description.

Definition at line 79 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ActivityRepository::updateActivityProperties\(\)](#).

7.2.3.31 weatherNotes

```
QString Activity::weatherNotes
```

Weather/environmental notes.

Definition at line 73 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ActivityRepository::updateActivityProperties\(\)](#).

7.2.3.32 weightKg

```
double Activity::weightKg = 0.0
```

[Athlete](#) weight in kg.

Definition at line 94 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ActivityRepository::updateActivityProperties\(\)](#).

7.2.3.33 wind

```
QString Activity::wind
```

Wind conditions.

Definition at line 82 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ActivityRepository::updateActivityProperties\(\)](#).

The documentation for this struct was generated from the following file:

- [src/database/ActivityRepository.h](#)

7.3 ActivityAnalysisCacheRepository Class Reference

Caches expensive analysis results per activity.

```
#include <ActivityAnalysisCacheRepository.h>
```

Public Member Functions

- [ActivityAnalysisCacheRepository](#) (QSqlDatabase &db)
Constructs repository with database connection.
- bool [setCache](#) (int activityId, const QString &key, const QByteArray &value)
Stores analysis result cache.
- QByteArray [getCache](#) (int activityId, const QString &key)
Retrieves cached analysis result.
- bool [hasCache](#) (int activityId, const QString &key)
Checks if cache entry exists.
- bool [invalidateCache](#) (int activityId)
Invalidates all cache for an activity.
- bool [invalidateKey](#) (int activityId, const QString &key)
Invalidates specific cache key for an activity.

Private Attributes

- QSqlDatabase & [m_db](#)

7.3.1 Detailed Description

Caches expensive analysis results per activity.

Stores pre-computed power curves, histograms, best efforts, and other analysis data to improve responsiveness on repeated access.

Definition at line 17 of file [ActivityAnalysisCacheRepository.h](#).

7.3.2 Constructor & Destructor Documentation

7.3.2.1 ActivityAnalysisCacheRepository()

```
ActivityAnalysisCacheRepository::ActivityAnalysisCacheRepository (  
    QSqlDatabase & db) [explicit]
```

Constructs repository with database connection.

Parameters

<i>db</i>	Database connection.
-----------	----------------------

Definition at line 9 of file [ActivityAnalysisCacheRepository.cpp](#).

7.3.3 Member Function Documentation

7.3.3.1 getCache()

```
QByteArray ActivityAnalysisCacheRepository::getCache (  
    int activityId,  
    const QString & key)
```

Retrieves cached analysis result.

Parameters

<i>activityId</i>	Activity ID.
<i>key</i>	Cache key name.

Returns

Cached data, or empty if not found.

Definition at line 25 of file [ActivityAnalysisCacheRepository.cpp](#).

7.3.3.2 hasCache()

```
bool ActivityAnalysisCacheRepository::hasCache (  
    int activityId,  
    const QString & key)
```

Checks if cache entry exists.

Parameters

<i>activityId</i>	Activity ID.
<i>key</i>	Cache key name.

Returns

True if entry exists.

Definition at line 36 of file [ActivityAnalysisCacheRepository.cpp](#).

7.3.3.3 invalidateCache()

```
bool ActivityAnalysisCacheRepository::invalidateCache (  
    int activityId)
```

Invalidates all cache for an activity.

Parameters

<i>activityId</i>	Activity ID.
-------------------	------------------------------

Returns

True on success.

Definition at line 45 of file [ActivityAnalysisCacheRepository.cpp](#).

7.3.3.4 invalidateKey()

```
bool ActivityAnalysisCacheRepository::invalidateKey (  
    int activityId,  
    const QString & key)
```

Invalidates specific cache key for an activity.

Parameters

<i>activityId</i>	Activity ID.
<i>key</i>	Cache key name.

Returns

True on success.

Definition at line 53 of file [ActivityAnalysisCacheRepository.cpp](#).

7.3.3.5 setCache()

```
bool ActivityAnalysisCacheRepository::setCache (
    int activityId,
    const QString & key,
    const QByteArray & value)
```

Stores analysis result cache.

Parameters

<i>activityId</i>	Activity ID.
<i>key</i>	Cache key name.
<i>value</i>	Serialized cache data.

Returns

True on success.

Definition at line 13 of file [ActivityAnalysisCacheRepository.cpp](#).

7.3.4 Member Data Documentation**7.3.4.1 m_db**

```
QSqlDatabase& ActivityAnalysisCacheRepository::m_db [private]
```

Definition at line 67 of file [ActivityAnalysisCacheRepository.h](#).

The documentation for this class was generated from the following files:

- src/database/ActivityAnalysisCacheRepository.h
- src/database/ActivityAnalysisCacheRepository.cpp

7.4 ActivityAnalysisSettings Struct Reference

Analysis settings per activity.

```
#include <ActivityAnalysisSettingsRepository.h>
```

Public Member Functions

- bool [isValid](#) () const
Checks if settings are valid.

Public Attributes

- int `activityId` = -1
Associated activity ID.
- double `climbMinGain` = 30.0
Minimum climb elevation gain in meters.
- double `climbMinLength` = 0.5
Minimum climb length in kilometers.
- double `climbMinGradient` = 3.0
Minimum climb gradient percentage.
- double `intervalWorkThreshold` = 0.0
Work interval power threshold (fraction of FTP).
- double `intervalRestThreshold` = 0.0
Recovery interval power threshold (fraction of FTP).

7.4.1 Detailed Description

Analysis settings per activity.

Definition at line 14 of file [ActivityAnalysisSettingsRepository.h](#).

7.4.2 Member Function Documentation

7.4.2.1 isValid()

```
bool ActivityAnalysisSettings::isValid () const [inline]
```

Checks if settings are valid.

Definition at line 37 of file [ActivityAnalysisSettingsRepository.h](#).

References [activityId](#).

7.4.3 Member Data Documentation

7.4.3.1 activityId

```
int ActivityAnalysisSettings::activityId = -1
```

Associated activity ID.

Definition at line 17 of file [ActivityAnalysisSettingsRepository.h](#).

Referenced by [ActivityAnalysisSettingsRepository::insertOrUpdate\(\)](#), [isValid\(\)](#), and [ActivityAnalysisSettingsRepository::settingsForActivity\(\)](#).

7.4.3.2 climbMinGain

```
double ActivityAnalysisSettings::climbMinGain = 30.0
```

Minimum climb elevation gain in meters.

Definition at line 20 of file [ActivityAnalysisSettingsRepository.h](#).

Referenced by [ActivityAnalysisSettingsRepository::insertOrUpdate\(\)](#), and [ActivityAnalysisSettingsRepository::settingsForActivity\(\)](#).

7.4.3.3 climbMinGradient

```
double ActivityAnalysisSettings::climbMinGradient = 3.0
```

Minimum climb gradient percentage.

Definition at line 26 of file [ActivityAnalysisSettingsRepository.h](#).

Referenced by [ActivityAnalysisSettingsRepository::insertOrUpdate\(\)](#), and [ActivityAnalysisSettingsRepository::settingsForActivity\(\)](#).

7.4.3.4 climbMinLength

```
double ActivityAnalysisSettings::climbMinLength = 0.5
```

Minimum climb length in kilometers.

Definition at line 23 of file [ActivityAnalysisSettingsRepository.h](#).

Referenced by [ActivityAnalysisSettingsRepository::insertOrUpdate\(\)](#), and [ActivityAnalysisSettingsRepository::settingsForActivity\(\)](#).

7.4.3.5 intervalRestThreshold

```
double ActivityAnalysisSettings::intervalRestThreshold = 0.0
```

Recovery interval power threshold (fraction of FTP).

Definition at line 32 of file [ActivityAnalysisSettingsRepository.h](#).

Referenced by [ActivityAnalysisSettingsRepository::insertOrUpdate\(\)](#), and [ActivityAnalysisSettingsRepository::settingsForActivity\(\)](#).

7.4.3.6 intervalWorkThreshold

```
double ActivityAnalysisSettings::intervalWorkThreshold = 0.0
```

Work interval power threshold (fraction of FTP).

Definition at line 29 of file [ActivityAnalysisSettingsRepository.h](#).

Referenced by [ActivityAnalysisSettingsRepository::insertOrUpdate\(\)](#), and [ActivityAnalysisSettingsRepository::settingsForActivity\(\)](#).

The documentation for this struct was generated from the following file:

- [src/database/ActivityAnalysisSettingsRepository.h](#)

7.5 ActivityAnalysisSettingsRepository Class Reference

Repository for per-activity analysis settings.

```
#include <ActivityAnalysisSettingsRepository.h>
```

Public Member Functions

- [ActivityAnalysisSettingsRepository](#) (QSqlDatabase &db)
Constructs repository with database connection.
- bool [insertOrUpdate](#) (const [ActivityAnalysisSettings](#) &settings)
Saves or updates activity settings.
- [ActivityAnalysisSettings settingsForActivity](#) (int activityId)
Retrieves settings for an activity.

Private Attributes

- QSqlDatabase & [m_db](#)

7.5.1 Detailed Description

Repository for per-activity analysis settings.

Stores custom climb detection and interval detection parameters per activity.

Definition at line 45 of file [ActivityAnalysisSettingsRepository.h](#).

7.5.2 Constructor & Destructor Documentation

7.5.2.1 ActivityAnalysisSettingsRepository()

```
ActivityAnalysisSettingsRepository::ActivityAnalysisSettingsRepository (  
    QSqlDatabase & db) [explicit]
```

Constructs repository with database connection.

Parameters

<i>db</i>	Database connection.
-----------	----------------------

Definition at line 9 of file [ActivityAnalysisSettingsRepository.cpp](#).

7.5.3 Member Function Documentation

7.5.3.1 insertOrUpdate()

```
bool ActivityAnalysisSettingsRepository::insertOrUpdate (
    const ActivityAnalysisSettings & settings)
```

Saves or updates activity settings.

Parameters

<i>settings</i>	Settings to save.
-----------------	-------------------

Returns

True on success.

Definition at line 13 of file [ActivityAnalysisSettingsRepository.cpp](#).

References [ActivityAnalysisSettings::activityId](#), [ActivityAnalysisSettings::climbMinGain](#), [ActivityAnalysisSettings::climbMinGradient](#), [ActivityAnalysisSettings::climbMinLength](#), [ActivityAnalysisSettings::intervalRestThreshold](#), and [ActivityAnalysisSettings::intervalWork](#).

7.5.3.2 settingsForActivity()

```
ActivityAnalysisSettings ActivityAnalysisSettingsRepository::settingsForActivity (
    int activityId)
```

Retrieves settings for an activity.

Parameters

<i>activityId</i>	Activity ID.
-------------------	------------------------------

Returns

Settings, or invalid if not found.

Definition at line 34 of file [ActivityAnalysisSettingsRepository.cpp](#).

References [ActivityAnalysisSettings::activityId](#), [ActivityAnalysisSettings::climbMinGain](#), [ActivityAnalysisSettings::climbMinGradient](#), [ActivityAnalysisSettings::climbMinLength](#), [ActivityAnalysisSettings::intervalRestThreshold](#), and [ActivityAnalysisSettings::intervalWork](#).

7.5.4 Member Data Documentation

7.5.4.1 m_db

```
QSqlDatabase& ActivityAnalysisSettingsRepository::m_db [private]
```

Definition at line 69 of file [ActivityAnalysisSettingsRepository.h](#).

The documentation for this class was generated from the following files:

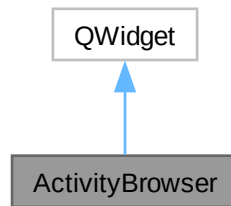
- src/database/ActivityAnalysisSettingsRepository.h
- src/database/ActivityAnalysisSettingsRepository.cpp

7.6 ActivityBrowser Class Reference

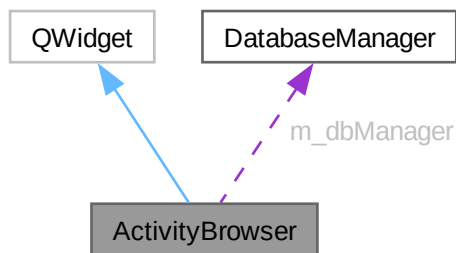
Browse and manage activities.

```
#include <ActivityBrowser.h>
```

Inheritance diagram for ActivityBrowser:



Collaboration diagram for ActivityBrowser:



Signals

- void **activitySelected** (int activityId)
- void **activityDeleted** (int activityId)
- void **fitFilesDropped** (const QStringList &filePaths)

Public Member Functions

- [ActivityBrowser](#) ([DatabaseManager](#) *dbManager, QWidget *parent=nullptr)
- void [refresh](#) (int athleteId=-1)
- void [focusSearch](#) ()

Protected Member Functions

- void [dragEnterEvent](#) (QDragEnterEvent *event) override
- void [dropEvent](#) (QDropEvent *event) override

Private Member Functions

- void [applyFilters](#) ()
- void [showContextMenu](#) (const QPoint &pos)
- void [editNotes](#) (int activityId)
- void [deleteActivity](#) (int activityId)

Private Attributes

- `DatabaseManager * m_dbManager` = nullptr
- `QLabel * m_emptyStateLabel` = nullptr
- `QLineEdit * m_searchEdit` = nullptr
- `QComboBox * m_rangeFilter` = nullptr
- `QTableWidget * m_table` = nullptr
- `int m_athleteId` = -1

7.6.1 Detailed Description

Browse and manage activities.

Displays activities in a filterable table and provides context-menu actions.

The widget accepts local `.fit` drops and emits `fitFilesDropped()` so the main window can enqueue background imports.

Definition at line 26 of file [ActivityBrowser.h](#).

7.6.2 Constructor & Destructor Documentation**7.6.2.1 ActivityBrowser()**

```
ActivityBrowser::ActivityBrowser (
    DatabaseManager * dbManager,
    QWidget * parent = nullptr) [explicit]
```

Definition at line 65 of file [ActivityBrowser.cpp](#).

7.6.3 Member Function Documentation**7.6.3.1 applyFilters()**

```
void ActivityBrowser::applyFilters () [private]
```

Definition at line 229 of file [ActivityBrowser.cpp](#).

7.6.3.2 deleteActivity()

```
void ActivityBrowser::deleteActivity (
    int activityId) [private]
```

Definition at line 354 of file [ActivityBrowser.cpp](#).

7.6.3.3 dragEnterEvent()

```
void ActivityBrowser::dragEnterEvent (
    QDragEnterEvent * event) [override], [protected]
```

Definition at line 139 of file [ActivityBrowser.cpp](#).

7.6.3.4 dropEvent()

```
void ActivityBrowser::dropEvent (
    QDropEvent * event) [override], [protected]
```

Definition at line 156 of file [ActivityBrowser.cpp](#).

7.6.3.5 editNotes()

```
void ActivityBrowser::editNotes (
    int activityId) [private]
```

Definition at line 326 of file [ActivityBrowser.cpp](#).

7.6.3.6 focusSearch()

```
void ActivityBrowser::focusSearch ()
```

Definition at line 130 of file [ActivityBrowser.cpp](#).

7.6.3.7 refresh()

```
void ActivityBrowser::refresh (
    int athleteId = -1)
```

Definition at line 179 of file [ActivityBrowser.cpp](#).

7.6.3.8 showContextMenu()

```
void ActivityBrowser::showContextMenu (
    const QPoint & pos) [private]
```

Definition at line 302 of file [ActivityBrowser.cpp](#).

7.6.4 Member Data Documentation

7.6.4.1 m_athleteId

```
int ActivityBrowser::m_athleteId = -1 [private]
```

Definition at line 62 of file [ActivityBrowser.h](#).

7.6.4.2 m_dbManager

```
DatabaseManager* ActivityBrowser::m_dbManager = nullptr [private]
```

Definition at line 57 of file [ActivityBrowser.h](#).

7.6.4.3 m_emptyStateLabel

```
QLabel* ActivityBrowser::m_emptyStateLabel = nullptr [private]
```

Definition at line 58 of file [ActivityBrowser.h](#).

7.6.4.4 m_rangeFilter

```
QComboBox* ActivityBrowser::m_rangeFilter = nullptr [private]
```

Definition at line 60 of file [ActivityBrowser.h](#).

7.6.4.5 m_searchEdit

```
QLineEdit* ActivityBrowser::m_searchEdit = nullptr [private]
```

Definition at line 59 of file [ActivityBrowser.h](#).

7.6.4.6 m_table

```
QTableWidget* ActivityBrowser::m_table = nullptr [private]
```

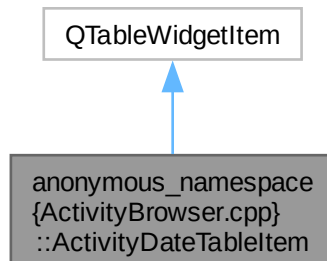
Definition at line 61 of file [ActivityBrowser.h](#).

The documentation for this class was generated from the following files:

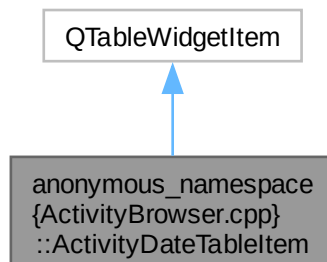
- [src/gui/ActivityBrowser.h](#)
- [src/gui/ActivityBrowser.cpp](#)

7.7 anonymous_namespace{ActivityBrowser.cpp}::ActivityDateTableItem Class Reference

Inheritance diagram for anonymous_namespace{ActivityBrowser.cpp}::ActivityDateTableItem:



Collaboration diagram for anonymous_namespace{ActivityBrowser.cpp}::ActivityDateTableItem:



Public Member Functions

- bool `operator<` (const QTableWidgetItem &other) const override

7.7.1 Detailed Description

Definition at line 29 of file [ActivityBrowser.cpp](#).

7.7.2 Member Function Documentation

7.7.2.1 operator<()

```
bool anonymous_namespace{ActivityBrowser.cpp}::ActivityDateTableItem::operator< (
    const QTableWidgetItem & other) const [inline], [override]
```

Definition at line 34 of file [ActivityBrowser.cpp](#).

The documentation for this class was generated from the following file:

- `src/gui/ActivityBrowser.cpp`

7.8 ActivityRepository Class Reference

Repository for activity record persistence.

```
#include <ActivityRepository.h>
```

Public Member Functions

- [ActivityRepository](#) (QSqlDatabase &db)
Constructs repository with database connection.
- int [insertActivity](#) (const [Activity](#) &a)
Inserts a new activity into the database.
- [Activity](#) [getActivity](#) (int activityId)
Retrieves an activity by ID.
- QList< [Activity](#) > [listActivities](#) (int athleteId)
Lists all activities for an athlete.
- bool [deleteActivity](#) (int activityId)
Deletes an activity.
- bool [hasFitHash](#) (const QString &hash)
Checks if activity exists by FIT file hash.
- bool [updateNotes](#) (int activityId, const QString ¬es, const QString &weatherNotes)
Updates activity notes and weather information.
- bool [updateActivityProperties](#) (int activityId, const [Activity](#) &a)
Updates comprehensive activity properties.
- bool [updateAnalysisFlags](#) (int activityId, int flags)
Updates analysis flags for an activity.
- bool [updateFingerprint](#) (int activityId, const QString &fingerprint)
Updates activity fingerprint for duplicate detection.
- bool [hasFingerprintMatch](#) (const QString &fingerprint)
Checks if fingerprint already exists.
- bool [setImportSource](#) (int activityId, const QString &source)
Sets the import source label for an activity.

Private Attributes

- QSqlDatabase & [m_db](#)

7.8.1 Detailed Description

Repository for activity record persistence.

Provides CRUD operations and queries for activities in the database.

Definition at line 129 of file [ActivityRepository.h](#).

7.8.2 Constructor & Destructor Documentation

7.8.2.1 ActivityRepository()

```
ActivityRepository::ActivityRepository (  
    QSqlDatabase & db) [explicit]
```

Constructs repository with database connection.

Parameters

<i>db</i>	Database connection.
-----------	----------------------

Definition at line 13 of file [ActivityRepository.cpp](#).

7.8.3 Member Function Documentation

7.8.3.1 deleteActivity()

```
bool ActivityRepository::deleteActivity (
    int activityId)
```

Deletes an activity.

Parameters

<i>activityId</i>	Activity ID to delete.
-------------------	--

Returns

True if deletion succeeded.

Definition at line 154 of file [ActivityRepository.cpp](#).

7.8.3.2 getActivity()

```
Activity ActivityRepository::getActivity (
    int activityId)
```

Retrieves an activity by ID.

Parameters

<i>activityId</i>	Activity ID.
-------------------	------------------------------

Returns

[Activity](#) record, or invalid [Activity](#) if not found.

Definition at line 123 of file [ActivityRepository.cpp](#).

7.8.3.3 hasFingerprintMatch()

```
bool ActivityRepository::hasFingerprintMatch (
    const QString & fingerprint)
```

Checks if fingerprint already exists.

Parameters

<i>fingerprint</i>	Fingerprint to check.
--------------------	-----------------------

Returns

True if match found.

Definition at line 189 of file [ActivityRepository.cpp](#).

7.8.3.4 hasFitHash()

```
bool ActivityRepository::hasFitHash (
    const QString & hash)
```

Checks if activity exists by FIT file hash.

Parameters

<i>hash</i>	SHA-256 hash of FIT file.
-------------	---------------------------

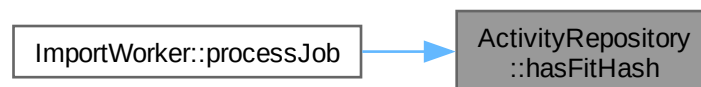
Returns

True if hash already exists in database.

Definition at line 162 of file [ActivityRepository.cpp](#).

Referenced by [ImportWorker::processJob\(\)](#).

Here is the caller graph for this function:

**7.8.3.5 insertActivity()**

```
int ActivityRepository::insertActivity (
    const Activity & a)
```

Inserts a new activity into the database.

Parameters

<i>a</i>	Activity to insert.
----------	-------------------------------------

Returns

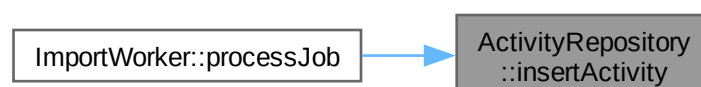
ID of inserted activity, or -1 on error.

Definition at line 70 of file [ActivityRepository.cpp](#).

References [Activity::athleteId](#), [Activity::avgCadence](#), [Activity::avgHr](#), [Activity::avgPower](#), [Activity::avgSpeed](#), [Activity::bike](#), [Activity::distanceM](#), [Activity::durationSec](#), [Activity::elevationGain](#), [Activity::endTime](#), [Activity::equipment](#), [Activity::fatigue](#), [Activity::fileName](#), [Activity::fitHash](#), [Activity::importedAt](#), [Activity::maxHr](#), [Activity::maxPower](#), [Activity::normalizedPower](#), [Activity::notes](#), [Activity::rpe](#), [Activity::sleepHours](#), [Activity::sport](#), [Activity::startTime](#), [Activity::temperature](#), [Activity::weather](#), [Activity::weatherNotes](#), [Activity::weightKg](#), and [Activity::wind](#).

Referenced by [ImportWorker::processJob\(\)](#).

Here is the caller graph for this function:



7.8.3.6 listActivities()

```
QList< Activity > ActivityRepository::listActivities (
    int athleteId)
```

Lists all activities for an athlete.

Parameters

<i>athleteId</i>	Athlete ID.
------------------	-------------

Returns

List of activities.

Definition at line 133 of file [ActivityRepository.cpp](#).

7.8.3.7 setImportSource()

```
bool ActivityRepository::setImportSource (
    int activityId,
    const QString & source)
```

Sets the import source label for an activity.

Parameters

<i>activityId</i>	Activity ID.
<i>source</i>	Import source label (e.g., "manual_import", "directory_import").

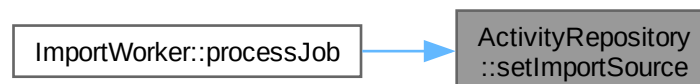
Returns

True if update succeeded.

Definition at line 199 of file [ActivityRepository.cpp](#).

Referenced by [ImportWorker::processJob\(\)](#).

Here is the caller graph for this function:



7.8.3.8 updateActivityProperties()

```
bool ActivityRepository::updateActivityProperties (
    int activityId,
    const Activity & a)
```

Updates comprehensive activity properties.

Parameters

<i>activity↔ Id</i>	Activity ID.
<i>a</i>	Activity with new values.

Returns

True if update succeeded.

Definition at line 220 of file [ActivityRepository.cpp](#).

References [Activity::bike](#), [Activity::equipment](#), [Activity::fatigue](#), [Activity::notes](#), [Activity::rpe](#), [Activity::sleepHours](#), [Activity::temperature](#), [Activity::weather](#), [Activity::weatherNotes](#), [Activity::weightKg](#), and [Activity::wind](#).

7.8.3.9 updateAnalysisFlags()

```
bool ActivityRepository::updateAnalysisFlags (
    int activityId,
    int flags)
```

Updates analysis flags for an activity.

Parameters

<i>activity↔ Id</i>	Activity ID.
<i>flags</i>	Analysis flag bitmask.

Returns

True if update succeeded.

Definition at line 171 of file [ActivityRepository.cpp](#).

7.8.3.10 updateFingerprint()

```
bool ActivityRepository::updateFingerprint (
    int activityId,
    const QString & fingerprint)
```

Updates activity fingerprint for duplicate detection.

Parameters

<i>activityId</i>	Activity ID.
<i>fingerprint</i>	Fingerprint string.

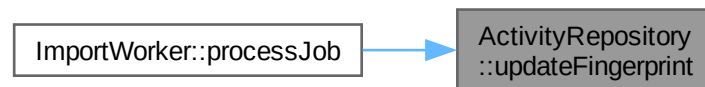
Returns

True if update succeeded.

Definition at line 180 of file [ActivityRepository.cpp](#).

Referenced by [ImportWorker::processJob\(\)](#).

Here is the caller graph for this function:

**7.8.3.11 updateNotes()**

```

bool ActivityRepository::updateNotes (
    int activityId,
    const QString & notes,
    const QString & weatherNotes)
  
```

Updates activity notes and weather information.

Parameters

<i>activityId</i>	Activity ID.
<i>notes</i>	Activity notes.
<i>weatherNotes</i>	Weather observations.

Returns

True if update succeeded.

Definition at line 208 of file [ActivityRepository.cpp](#).

7.8.4 Member Data Documentation**7.8.4.1 m_db**

```

QSqlDatabase& ActivityRepository::m_db [private]
  
```

Definition at line 223 of file [ActivityRepository.h](#).

The documentation for this class was generated from the following files:

- [src/database/ActivityRepository.h](#)
- [src/database/ActivityRepository.cpp](#)

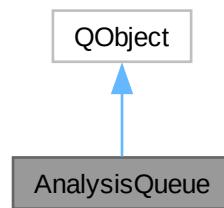
7.9 AnalysisQueue Class Reference

Manages sequential background analysis task queue.

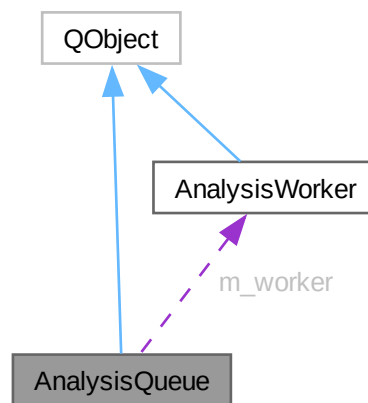
```

#include <AnalysisQueue.h>
  
```

Inheritance diagram for AnalysisQueue:



Collaboration diagram for AnalysisQueue:



Signals

- void `taskCompleted` (int activityId)
signal Emitted when a task completes successfully.
- void `taskFailed` (int activityId, const QString &error)
signal Emitted when a task fails.
- void `pendingCountChanged` (int count)
signal Emitted when pending task count changes.

Public Member Functions

- `AnalysisQueue` (QObject *parent=nullptr)
Constructs analysis queue.
- void `setDatabasePath` (const QString &path)
Sets the database path for analysis operations.
- void `enqueue` (int activityId)
Queues an activity for background analysis.

- int [pendingCount](#) () const
Gets the number of pending analysis tasks.
- bool [isBusy](#) () const
Checks if analysis is currently running.

Private Slots

- void [onTaskCompleted](#) (int activityId)
- void [onTaskFailed](#) (int activityId, const QString &error)

Private Member Functions

- void [processNext](#) ()

Private Attributes

- QThread * [m_thread](#) = nullptr
- [AnalysisWorker](#) * [m_worker](#) = nullptr
- QQueue< int > [m_queue](#)
- bool [m_busy](#) = false
- QString [m_dbPath](#)

7.9.1 Detailed Description

Manages sequential background analysis task queue.

Processes activities one at a time in a background thread, detecting climbs, intervals, and computing analysis metrics.

Definition at line 19 of file [AnalysisQueue.h](#).

7.9.2 Constructor & Destructor Documentation

7.9.2.1 AnalysisQueue()

```
AnalysisQueue::AnalysisQueue (
    QObject * parent = nullptr) [explicit]
```

Constructs analysis queue.

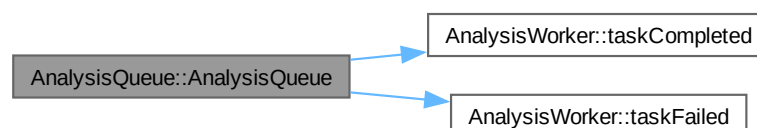
Parameters

<i>parent</i>	Parent object.
---------------	----------------

Definition at line 9 of file [AnalysisQueue.cpp](#).

References [AnalysisWorker::taskCompleted\(\)](#), and [AnalysisWorker::taskFailed\(\)](#).

Here is the call graph for this function:



7.9.2.2 ~AnalysisQueue()

`AnalysisQueue::~AnalysisQueue () [override]`
 Definition at line 24 of file [AnalysisQueue.cpp](#).

7.9.3 Member Function Documentation

7.9.3.1 enqueue()

`void AnalysisQueue::enqueue (`
 `int activityId)`

Queues an activity for background analysis.

Parameters

<i>activityId</i>	Activity ID to analyze.
-------------------	---

Definition at line 40 of file [AnalysisQueue.cpp](#).
 References [pendingCountChanged\(\)](#).
 Here is the call graph for this function:



7.9.3.2 isBusy()

`bool AnalysisQueue::isBusy () const`
 Checks if analysis is currently running.

Returns

True if a task is being processed.

Definition at line 53 of file [AnalysisQueue.cpp](#).

7.9.3.3 onTaskCompleted

`void AnalysisQueue::onTaskCompleted (`
 `int activityId) [private], [slot]`

Definition at line 72 of file [AnalysisQueue.cpp](#).

7.9.3.4 onTaskFailed

`void AnalysisQueue::onTaskFailed (`
 `int activityId,`
 `const QString & error) [private], [slot]`

Definition at line 79 of file [AnalysisQueue.cpp](#).

7.9.3.5 pendingCount()

`int AnalysisQueue::pendingCount () const`
 Gets the number of pending analysis tasks.

Returns

Count of queued activities.

Definition at line 48 of file [AnalysisQueue.cpp](#).

7.9.3.6 pendingCountChanged

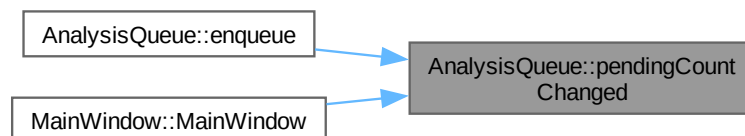
```
void AnalysisQueue::pendingCountChanged (
    int count) [signal]
```

\signal Emitted when pending task count changes.

Parameters

<i>count</i>	Number of pending analysis tasks.
--------------	-----------------------------------

Referenced by [enqueue\(\)](#), and [MainWindow::MainWindow\(\)](#).
Here is the caller graph for this function:

**7.9.3.7 processNext()**

```
void AnalysisQueue::processNext () [private]
```

Definition at line 58 of file [AnalysisQueue.cpp](#).

7.9.3.8 setDatabasePath()

```
void AnalysisQueue::setDatabasePath (
    const QString & path)
```

Sets the database path for analysis operations.

Parameters

<i>path</i>	Database file path.
-------------	---------------------

Definition at line 31 of file [AnalysisQueue.cpp](#).

7.9.3.9 taskCompleted

```
void AnalysisQueue::taskCompleted (
    int activityId) [signal]
```

\signal Emitted when a task completes successfully.

Parameters

<i>activityId</i>	Activity ID that was analyzed.
-------------------	--

Referenced by [MainWindow::MainWindow\(\)](#).
Here is the caller graph for this function:

**7.9.3.10 taskFailed**

```
void AnalysisQueue::taskFailed (
    int activityId,
    const QString & error) [signal]
```

\signal Emitted when a task fails.

Parameters

<i>activityId</i>	Activity ID that failed.
<i>error</i>	Error message describing the failure.

7.9.4 Member Data Documentation**7.9.4.1 m_busy**

```
bool AnalysisQueue::m_busy = false [private]
```

Definition at line 78 of file [AnalysisQueue.h](#).

7.9.4.2 m_dbPath

```
QString AnalysisQueue::m_dbPath [private]
```

Definition at line 79 of file [AnalysisQueue.h](#).

7.9.4.3 m_queue

```
QQueue<int> AnalysisQueue::m_queue [private]
```

Definition at line 77 of file [AnalysisQueue.h](#).

7.9.4.4 m_thread

```
QThread* AnalysisQueue::m_thread = nullptr [private]
```

Definition at line 75 of file [AnalysisQueue.h](#).

7.9.4.5 m_worker

```
AnalysisWorker* AnalysisQueue::m_worker = nullptr [private]
```

Definition at line 76 of file [AnalysisQueue.h](#).

The documentation for this class was generated from the following files:

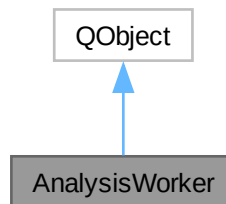
- `src/analysis/queue/AnalysisQueue.h`
- `src/analysis/queue/AnalysisQueue.cpp`

7.10 AnalysisWorker Class Reference

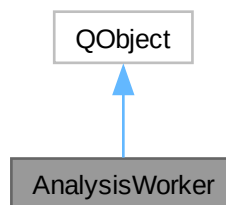
Worker thread for background activity analysis.

```
#include <AnalysisWorker.h>
```

Inheritance diagram for AnalysisWorker:



Collaboration diagram for AnalysisWorker:



Public Slots

- void `processTask` (int activityId)
Processes an activity analysis task.

Signals

- void `taskCompleted` (int activityId)
signal Emitted when a task completes successfully.
- void `taskFailed` (int activityId, const QString &error)
signal Emitted when a task fails.

Public Member Functions

- `AnalysisWorker` (QObject *parent=nullptr)
Constructs analysis worker.

- void [setDatabasePath](#) (const QString &path)
Sets database path for analysis operations.

Private Attributes

- QString [m_dbPath](#)

7.10.1 Detailed Description

Worker thread for background activity analysis.

Processes analysis tasks (detect climbs, intervals, compute metrics) synchronously in a dedicated thread.

Definition at line 15 of file [AnalysisWorker.h](#).

7.10.2 Constructor & Destructor Documentation

7.10.2.1 AnalysisWorker()

```
AnalysisWorker::AnalysisWorker (
    QObject * parent = nullptr) [explicit]
```

Constructs analysis worker.

Parameters

<i>parent</i>	Parent object.
---------------	----------------

Definition at line 37 of file [AnalysisWorker.cpp](#).

7.10.3 Member Function Documentation

7.10.3.1 processTask

```
void AnalysisWorker::processTask (
    int activityId) [slot]
```

Processes an activity analysis task.

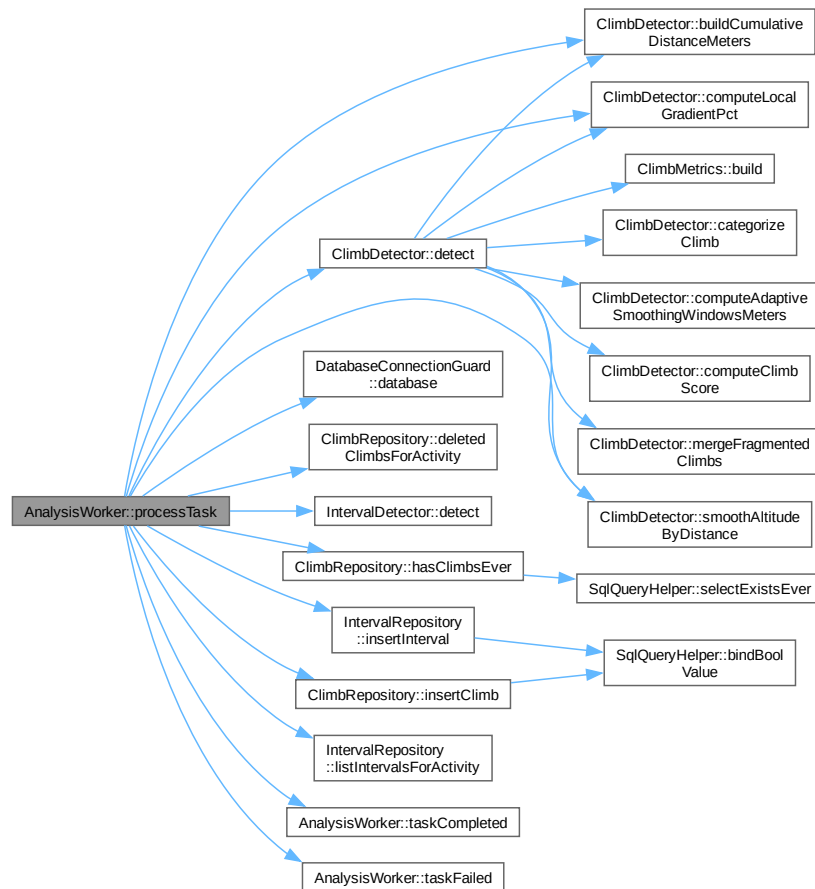
Parameters

<i>activityId</i>	Activity ID to analyze.
-------------------	---

Definition at line 46 of file [AnalysisWorker.cpp](#).

References [ClimbRecord::activityId](#), [IntervalRecord::activityId](#), [ClimbRecord::algorithmVersion](#), [IntervalRecord::algorithmVersion](#), [ClimbRecord::averageGradient](#), [ClimbRecord::avgCadence](#), [IntervalRecord::avgCadence](#), [ClimbRecord::avgHr](#), [IntervalRecord::avgHr](#), [ClimbRecord::avgPower](#), [IntervalRecord::avgPower](#), [ClimbDetector::buildCumulativeDistanceMeters\(\)](#), [AnalysisVersions::ClimbDetection](#), [ClimbDetector::computeLocalGradientPct\(\)](#), [DatabaseConnectionGuard::database\(\)](#), [ClimbRecord::deleted](#), [IntervalRecord::deleted](#), [ClimbRepository::deletedClimbsForActivity\(\)](#), [ClimbDetector::detect\(\)](#), [IntervalDetector::detect\(\)](#), [ClimbRecord::elevationGainM](#), [ClimbDetectorConfig::elevationSmoothingMeters](#), [IntervalRecord::endSample](#), [ClimbRecord::endSeconds](#), [IntervalDetectorConfig::ftp](#), [ClimbDetectorConfig::gradientWindowHalfMeters](#), [ActivityAnalysisFlags::HasClimbs](#), [ClimbRepository::hasClimbsEver\(\)](#), [ActivityAnalysisFlags::HasIntervals](#), [ClimbRepository::insertClimb\(\)](#), [IntervalRepository::insertInterval\(\)](#), [AnalysisVersions::IntervalDetection](#), [ClimbRecord::lengthKm](#), [IntervalRepository::listIntervalsForActivity\(\)](#), [ClimbRecord::locked](#), [IntervalRecord::locked](#), [ClimbDetectorConfig::minAverageGradient](#), [ClimbDetectorConfig::minElevationGainM](#), [ClimbDetectorConfig::minLengthKm](#), [ClimbRecord::name](#), [IntervalRecord::name](#), [ClimbRecord::np](#), [IntervalRecord::np](#), [ClimbRecord::originalEndSeconds](#), [ClimbRecord::originalStartSeconds](#), [RideData::records](#), [ClimbDetector::smoothAltitudeByDistance\(\)](#), [ClimbRecord::source](#), [IntervalRecord::source](#), [IntervalRecord::startSample](#), [ClimbRecord::startSeconds](#), [taskCompleted\(\)](#), [taskFailed\(\)](#), [IntervalRecord::type](#), [ClimbRecord::uuid](#), [IntervalRecord::uuid](#), and [ClimbRecord::vam](#).

Here is the call graph for this function:



7.10.3.2 setDatabasePath()

```
void AnalysisWorker::setDatabasePath (
    const QString & path)
```

Sets database path for analysis operations.

Parameters

<i>path</i>	Database file path.
-------------	---------------------

Definition at line 41 of file [AnalysisWorker.cpp](#).

7.10.3.3 taskCompleted

```
void AnalysisWorker::taskCompleted (
    int activityId) [signal]
```

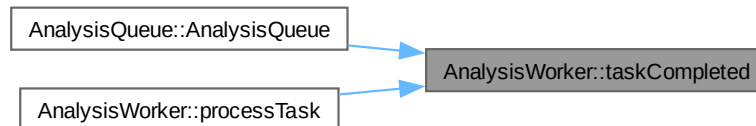
\signal Emitted when a task completes successfully.

Parameters

<i>activityId</i>	Activity ID that was analyzed.
-------------------	--

Referenced by [AnalysisQueue::AnalysisQueue\(\)](#), and [processTask\(\)](#).

Here is the caller graph for this function:

**7.10.3.4 taskFailed**

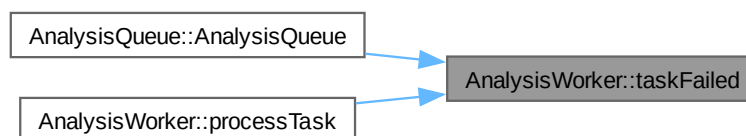
```
void AnalysisWorker::taskFailed (
    int activityId,
    const QString & error) [signal]
\signal Emitted when a task fails.
```

Parameters

<i>activityId</i>	Activity ID that failed.
<i>error</i>	Error message describing the failure.

Referenced by [AnalysisQueue::AnalysisQueue\(\)](#), and [processTask\(\)](#).

Here is the caller graph for this function:

**7.10.4 Member Data Documentation****7.10.4.1 m_dbPath**

```
QString AnalysisWorker::m_dbPath [private]
```

Definition at line 49 of file [AnalysisWorker.h](#).

The documentation for this class was generated from the following files:

- `src/analysis/queue/AnalysisWorker.h`
- `src/analysis/queue/AnalysisWorker.cpp`

7.11 AppSettings Class Reference

Application-wide settings singleton.

```
#include <AppSettings.h>
```

Public Member Functions

- `DateFormat` [dateFormat](#) () const
Gets the current date format preference.
- `void` [setDateFormat](#) (DateFormat format)
Sets the date format preference.
- `int` [tileCacheSize](#) () const
Gets the tile cache size.
- `void` [setTileCacheSize](#) (int tiles)
Sets the tile cache size.

Static Public Member Functions

- `static` [AppSettings](#) & [instance](#) ()
Returns the global settings instance.

Static Private Member Functions

- `static` QString [dateFormatKey](#) ()
- `static` DateFormat [sanitizeDateFormat](#) (int rawValue)
- `static` QString [tileCacheSizeKey](#) ()

7.11.1 Detailed Description

Application-wide settings singleton.

Manages persistent user preferences including date formatting and cache sizes.

Definition at line 26 of file [AppSettings.h](#).

7.11.2 Member Function Documentation

7.11.2.1 dateFormat()

```
DateFormat AppSettings::dateFormat () const
```

Gets the current date format preference.

Returns

Current DateFormat.

Definition at line 14 of file [AppSettings.cpp](#).

7.11.2.2 dateFormatKey()

```
QString AppSettings::dateFormatKey () [static], [private]
```

Definition at line 27 of file [AppSettings.cpp](#).

7.11.2.3 instance()

```
AppSettings & AppSettings::instance () [static]
```

Returns the global settings instance.

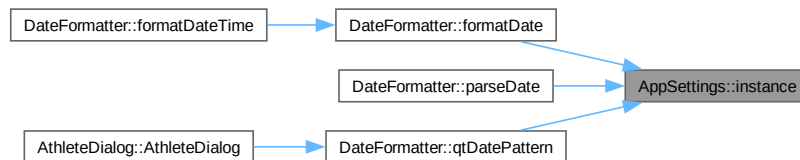
Returns

Reference to the singleton [AppSettings](#).

Definition at line 8 of file [AppSettings.cpp](#).

Referenced by [DateFormatter::formatDate\(\)](#), [DateFormatter::parseDate\(\)](#), and [DateFormatter::qtDatePattern\(\)](#).

Here is the caller graph for this function:

**7.11.2.4 sanitizeDateFormat()**

```
DateFormat AppSettings::sanitizeDateFormat (
    int rawValue) [static], [private]
```

Definition at line 32 of file [AppSettings.cpp](#).

7.11.2.5 setDateFormat()

```
void AppSettings::setDateFormat (
    DateFormat format)
```

Sets the date format preference.

Parameters

<i>format</i>	New DateFormat.
---------------	-----------------

Definition at line 21 of file [AppSettings.cpp](#).

7.11.2.6 setTileCacheSize()

```
void AppSettings::setTileCacheSize (
    int tiles)
```

Sets the tile cache size.

Parameters

<i>tiles</i>	Number of tiles to cache (128-2048).
--------------	--------------------------------------

Definition at line 61 of file [AppSettings.cpp](#).

7.11.2.7 tileCacheSize()

```
int AppSettings::tileCacheSize () const
```

Gets the tile cache size.

Returns

Number of tiles to keep in RAM (128-2048).

Definition at line 49 of file [AppSettings.cpp](#).

7.11.2.8 tileCacheSizeKey()

QString AppSettings::tileCacheSizeKey () [static], [private]
Definition at line 67 of file [AppSettings.cpp](#).

The documentation for this class was generated from the following files:

- src/core/settings/AppSettings.h
- src/core/settings/AppSettings.cpp

7.12 Athlete Struct Reference

Represents an athlete profile.

```
#include <AthleteRepository.h>
```

Public Member Functions

- QString [fullName](#) () const
Returns full name (first + space + last).
- bool [isValid](#) () const
Checks if athlete data is valid.

Public Attributes

- int [id](#) = -1
Unique athlete identifier.
- QString [firstName](#)
Athlete first name.
- QString [lastName](#)
Athlete last name.
- QString [dateOfBirth](#)
Date of birth (ISO 8601 YYYY-MM-DD).
- QString [email](#)
Email address.
- double [heightCm](#) = 0.0
Height in centimeters.
- int [ftpWatts](#) = 250
Functional threshold power in watts.
- QString [createdAt](#)
ISO8601 creation timestamp.

7.12.1 Detailed Description

Represents an athlete profile.

Definition at line 15 of file [AthleteRepository.h](#).

7.12.2 Member Function Documentation

7.12.2.1 fullName()

QString Athlete::fullName () const [inline]

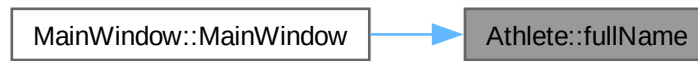
Returns full name (first + space + last).

Definition at line 44 of file [AthleteRepository.h](#).

References [firstName](#), and [lastName](#).

Referenced by [MainWindow::MainWindow\(\)](#).

Here is the caller graph for this function:



7.12.2.2 isValid()

```
bool Athlete::isValid () const [inline]
```

Checks if athlete data is valid.

Definition at line 49 of file [AthleteRepository.h](#).

7.12.3 Member Data Documentation

7.12.3.1 createdAt

```
QString Athlete::createdAt
```

ISO8601 creation timestamp.

Definition at line 39 of file [AthleteRepository.h](#).

Referenced by [AthleteRepository::getAthlete\(\)](#), and [AthleteRepository::listAthletes\(\)](#).

7.12.3.2 dateOfBirth

```
QString Athlete::dateOfBirth
```

Date of birth (ISO 8601 YYYY-MM-DD).

Definition at line 27 of file [AthleteRepository.h](#).

Referenced by [AthleteRepository::getAthlete\(\)](#), [AthleteRepository::insertAthlete\(\)](#), [AthleteRepository::listAthletes\(\)](#), and [AthleteRepository::updateAthlete\(\)](#).

7.12.3.3 email

```
QString Athlete::email
```

Email address.

Definition at line 30 of file [AthleteRepository.h](#).

Referenced by [AthleteRepository::getAthlete\(\)](#), [AthleteRepository::insertAthlete\(\)](#), [AthleteRepository::listAthletes\(\)](#), and [AthleteRepository::updateAthlete\(\)](#).

7.12.3.4 firstName

```
QString Athlete::firstName
```

[Athlete](#) first name.

Definition at line 21 of file [AthleteRepository.h](#).

Referenced by [fullName\(\)](#), [AthleteRepository::getAthlete\(\)](#), [AthleteRepository::insertAthlete\(\)](#), [AthleteRepository::listAthletes\(\)](#), and [AthleteRepository::updateAthlete\(\)](#).

7.12.3.5 ftpWatts

```
int Athlete::ftpWatts = 250
```

Functional threshold power in watts.

Definition at line 36 of file [AthleteRepository.h](#).

Referenced by [AthleteRepository::getAthlete\(\)](#), [AthleteRepository::insertAthlete\(\)](#), [AthleteRepository::listAthletes\(\)](#), and [AthleteRepository::updateAthlete\(\)](#).

7.12.3.6 heightCm

```
double Athlete::heightCm = 0.0
```

Height in centimeters.

Definition at line 33 of file [AthleteRepository.h](#).

Referenced by [AthleteRepository::getAthlete\(\)](#), [AthleteRepository::insertAthlete\(\)](#), [AthleteRepository::listAthletes\(\)](#), and [AthleteRepository::updateAthlete\(\)](#).

7.12.3.7 id

```
int Athlete::id = -1
```

Unique athlete identifier.

Definition at line 18 of file [AthleteRepository.h](#).

Referenced by [AthleteRepository::getAthlete\(\)](#), [AthleteRepository::listAthletes\(\)](#), and [AthleteRepository::updateAthlete\(\)](#).

7.12.3.8 lastName

```
QString Athlete::lastName
```

[Athlete](#) last name.

Definition at line 24 of file [AthleteRepository.h](#).

Referenced by [fullName\(\)](#), [AthleteRepository::getAthlete\(\)](#), [AthleteRepository::insertAthlete\(\)](#), [AthleteRepository::listAthletes\(\)](#), and [AthleteRepository::updateAthlete\(\)](#).

The documentation for this struct was generated from the following file:

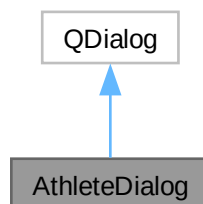
- [src/database/AthleteRepository.h](#)

7.13 AthleteDialog Class Reference

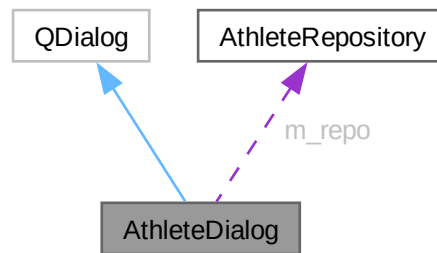
Dialog for creating or editing athlete profile.

```
#include <AthleteDialog.h>
```

Inheritance diagram for AthleteDialog:



Collaboration diagram for AthleteDialog:



Public Member Functions

- [AthleteDialog](#) ([AthleteRepository](#) &repo, int athleteId=-1, QWidget *parent=nullptr)
Constructs athlete edit dialog.
- int [savedAthleteId](#) () const
Gets ID of saved athlete.

Private Slots

- void [onAccepted](#) ()
- void [addFtpRow](#) ()
- void [deleteFtpRow](#) ()
- void [addWeightRow](#) ()
- void [deleteWeightRow](#) ()

Private Member Functions

- void [loadAthlete](#) ()
- void [populateFtpTable](#) (const QList< [FtpEntry](#) > &entries)
- void [populateWeightTable](#) (const QList< [WeightEntry](#) > &entries)

Private Attributes

- [AthleteRepository](#) & [m_repo](#)
- int [m_athleteId](#)
- int [m_savedId](#) = -1
- QLineEdit * [m_firstName](#) = nullptr
- QLineEdit * [m_lastName](#) = nullptr
- QLineEdit * [m_dob](#) = nullptr
- QLineEdit * [m_email](#) = nullptr
- QDoubleSpinBox * [m_height](#) = nullptr
- QSpinBox * [m ftp](#) = nullptr
- QWidget * [m ftpTable](#) = nullptr
- QWidget * [m_weightTable](#) = nullptr

7.13.1 Detailed Description

Dialog for creating or editing athlete profile.
Allows entry of personal info, FTP history, and weight history.
Definition at line 21 of file [AthleteDialog.h](#).

7.13.2 Constructor & Destructor Documentation

7.13.2.1 AthleteDialog()

```
AthleteDialog::AthleteDialog (
    AthleteRepository & repo,
    int athleteId = -1,
    QWidget * parent = nullptr) [explicit]
```

Constructs athlete edit dialog.

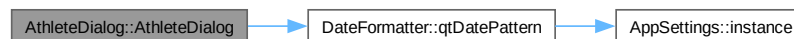
Parameters

<i>repo</i>	Athlete repository.
<i>athleteId</i>	Athlete ID to edit (-1 to create new).
<i>parent</i>	Parent widget.

Definition at line 40 of file [AthleteDialog.cpp](#).

References [DateFormatter::qtDatePattern\(\)](#).

Here is the call graph for this function:



7.13.3 Member Function Documentation

7.13.3.1 addFtpRow

```
void AthleteDialog::addFtpRow () [private], [slot]
```

Definition at line 175 of file [AthleteDialog.cpp](#).

7.13.3.2 addWeightRow

```
void AthleteDialog::addWeightRow () [private], [slot]
```

Definition at line 196 of file [AthleteDialog.cpp](#).

7.13.3.3 deleteFtpRow

```
void AthleteDialog::deleteFtpRow () [private], [slot]
```

Definition at line 186 of file [AthleteDialog.cpp](#).

7.13.3.4 deleteWeightRow

```
void AthleteDialog::deleteWeightRow () [private], [slot]
```

Definition at line 206 of file [AthleteDialog.cpp](#).

7.13.3.5 loadAthlete()

```
void AthleteDialog::loadAthlete () [private]
```

Definition at line 132 of file [AthleteDialog.cpp](#).

7.13.3.6 onAccepted

```
void AthleteDialog::onAccepted () [private], [slot]
```

Definition at line 216 of file [AthleteDialog.cpp](#).

7.13.3.7 populateFtpTable()

```
void AthleteDialog::populateFtpTable (  
    const QList< FtpEntry > & entries) [private]
```

Definition at line 146 of file [AthleteDialog.cpp](#).

7.13.3.8 populateWeightTable()

```
void AthleteDialog::populateWeightTable (  
    const QList< WeightEntry > & entries) [private]
```

Definition at line 161 of file [AthleteDialog.cpp](#).

7.13.3.9 savedAthleteld()

```
int AthleteDialog::savedAthleteId () const [inline]
```

Gets ID of saved athlete.

Returns

[Athlete](#) ID (valid after Accepted).

Definition at line 39 of file [AthleteDialog.h](#).

7.13.4 Member Data Documentation

7.13.4.1 m_athleteld

```
int AthleteDialog::m_athleteId [private]
```

Definition at line 54 of file [AthleteDialog.h](#).

7.13.4.2 m_dob

```
QLineEdit* AthleteDialog::m_dob = nullptr [private]
```

Definition at line 59 of file [AthleteDialog.h](#).

7.13.4.3 m_email

```
QLineEdit* AthleteDialog::m_email = nullptr [private]
```

Definition at line 60 of file [AthleteDialog.h](#).

7.13.4.4 m_firstName

```
QLineEdit* AthleteDialog::m_firstName = nullptr [private]
```

Definition at line 57 of file [AthleteDialog.h](#).

7.13.4.5 m_ftp

```
QSpinBox* AthleteDialog::m_ftp = nullptr [private]
```

Definition at line 62 of file [AthleteDialog.h](#).

7.13.4.6 m_ftpTable

```
QTableWidget* AthleteDialog::m_ftpTable = nullptr [private]
```

Definition at line 63 of file [AthleteDialog.h](#).

7.13.4.7 m_height

`QDoubleSpinBox* AthleteDialog::m_height = nullptr [private]`
Definition at line 61 of file [AthleteDialog.h](#).

7.13.4.8 m_lastName

`QLineEdit* AthleteDialog::m_lastName = nullptr [private]`
Definition at line 58 of file [AthleteDialog.h](#).

7.13.4.9 m_repo

`AthleteRepository& AthleteDialog::m_repo [private]`
Definition at line 53 of file [AthleteDialog.h](#).

7.13.4.10 m_savedId

`int AthleteDialog::m_savedId = -1 [private]`
Definition at line 55 of file [AthleteDialog.h](#).

7.13.4.11 m_weightTable

`QTableWidget* AthleteDialog::m_weightTable = nullptr [private]`
Definition at line 64 of file [AthleteDialog.h](#).

The documentation for this class was generated from the following files:

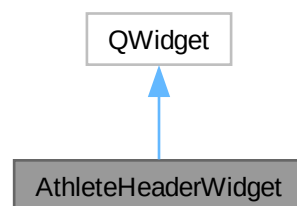
- `src/gui/AthleteDialog.h`
- `src/gui/AthleteDialog.cpp`

7.14 AthleteHeaderWidget Class Reference

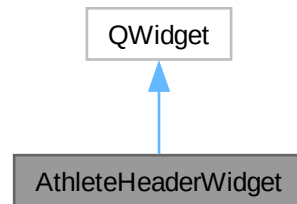
Header bar displaying current athlete and ride metrics.

```
#include <AthleteHeaderWidget.h>
```

Inheritance diagram for AthleteHeaderWidget:



Collaboration diagram for AthleteHeaderWidget:



Public Member Functions

- [AthleteHeaderWidget](#) (`QWidget *parent=nullptr`)
- void [setSummary](#) (`const QString &athleteName`, `int ftpWatts`, `int activityCount`, `const QString &lastActivityDate`)
- void [setRideMetrics](#) (`const QString &metricsText`)

Private Attributes

- `QLabel * m_nameLabel = nullptr`
- `QLabel * m_statsLabel = nullptr`
- `QLabel * m_metricsLabel = nullptr`

7.14.1 Detailed Description

Header bar displaying current athlete and ride metrics.

Shows athlete name, FTP, activity count, and current ride metrics (NP, IF, TSS).

Definition at line 15 of file [AthleteHeaderWidget.h](#).

7.14.2 Constructor & Destructor Documentation

7.14.2.1 AthleteHeaderWidget()

```
AthleteHeaderWidget::AthleteHeaderWidget (
    QWidget * parent = nullptr) [explicit]
```

Definition at line 9 of file [AthleteHeaderWidget.cpp](#).

7.14.3 Member Function Documentation

7.14.3.1 setRideMetrics()

```
void AthleteHeaderWidget::setRideMetrics (
    const QString & metricsText)
```

Definition at line 60 of file [AthleteHeaderWidget.cpp](#).

7.14.3.2 setSummary()

```
void AthleteHeaderWidget::setSummary (
    const QString & athleteName,
    int ftpWatts,
    int activityCount,
    const QString & lastActivityDate)
```

Definition at line 47 of file [AthleteHeaderWidget.cpp](#).

7.14.4 Member Data Documentation

7.14.4.1 m_metricsLabel

`QLabel* AthleteHeaderWidget::m_metricsLabel = nullptr [private]`

Definition at line 32 of file [AthleteHeaderWidget.h](#).

7.14.4.2 m_nameLabel

`QLabel* AthleteHeaderWidget::m_nameLabel = nullptr [private]`

Definition at line 30 of file [AthleteHeaderWidget.h](#).

7.14.4.3 m_statsLabel

`QLabel* AthleteHeaderWidget::m_statsLabel = nullptr [private]`

Definition at line 31 of file [AthleteHeaderWidget.h](#).

The documentation for this class was generated from the following files:

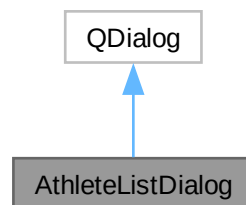
- `src/gui/AthleteHeaderWidget.h`
- `src/gui/AthleteHeaderWidget.cpp`

7.15 AthleteListDialog Class Reference

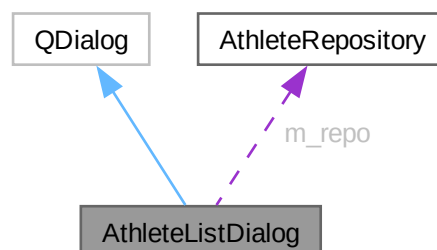
Dialog for browsing and managing athletes.

```
#include <AthleteListDialog.h>
```

Inheritance diagram for AthleteListDialog:



Collaboration diagram for AthleteListDialog:



Public Member Functions

- [AthleteListDialog](#) ([AthleteRepository](#) &repo, [QWidget](#) *parent=nullptr)
- int [selectedAthleteId](#) () const

Private Slots

- void [refresh](#) ()
- void [newAthlete](#) ()
- void [editAthlete](#) ()
- void [deleteAthlete](#) ()
- void [selectAthlete](#) ()

Private Attributes

- [AthleteRepository](#) & [m_repo](#)
- int [m_selectedId](#) = -1
- [QTableWidget](#) * [m_table](#) = nullptr
- [QPushButton](#) * [m_editBtn](#) = nullptr
- [QPushButton](#) * [m_deleteBtn](#) = nullptr
- [QPushButton](#) * [m_selectBtn](#) = nullptr

7.15.1 Detailed Description

Dialog for browsing and managing athletes.

Lists all athletes, allows selection as current athlete, and provides create/edit/delete operations.

Definition at line 19 of file [AthleteListDialog.h](#).

7.15.2 Constructor & Destructor Documentation

7.15.2.1 AthleteListDialog()

```
AthleteListDialog::AthleteListDialog (  
    AthleteRepository & repo,  
    QWidget * parent = nullptr) [explicit]
```

Definition at line 19 of file [AthleteListDialog.cpp](#).

7.15.3 Member Function Documentation

7.15.3.1 deleteAthlete

```
void AthleteListDialog::deleteAthlete () [private], [slot]
```

Definition at line 111 of file [AthleteListDialog.cpp](#).

7.15.3.2 editAthlete

```
void AthleteListDialog::editAthlete () [private], [slot]
```

Definition at line 101 of file [AthleteListDialog.cpp](#).

7.15.3.3 newAthlete

```
void AthleteListDialog::newAthlete () [private], [slot]
```

Definition at line 94 of file [AthleteListDialog.cpp](#).

7.15.3.4 refresh

```
void AthleteListDialog::refresh () [private], [slot]
```

Definition at line 77 of file [AthleteListDialog.cpp](#).

7.15.3.5 selectAthlete

void AthleteListDialog::selectAthlete () [private], [slot]
 Definition at line 128 of file [AthleteListDialog.cpp](#).

7.15.3.6 selectedAthleteId()

int AthleteListDialog::selectedAthleteId () const [inline]
 Definition at line 26 of file [AthleteListDialog.h](#).

7.15.4 Member Data Documentation

7.15.4.1 m_deleteBtn

QPushButton* AthleteListDialog::m_deleteBtn = nullptr [private]
 Definition at line 41 of file [AthleteListDialog.h](#).

7.15.4.2 m_editBtn

QPushButton* AthleteListDialog::m_editBtn = nullptr [private]
 Definition at line 40 of file [AthleteListDialog.h](#).

7.15.4.3 m_repo

[AthleteRepository](#)& AthleteListDialog::m_repo [private]
 Definition at line 36 of file [AthleteListDialog.h](#).

7.15.4.4 m_selectBtn

QPushButton* AthleteListDialog::m_selectBtn = nullptr [private]
 Definition at line 42 of file [AthleteListDialog.h](#).

7.15.4.5 m_selectedId

int AthleteListDialog::m_selectedId = -1 [private]
 Definition at line 37 of file [AthleteListDialog.h](#).

7.15.4.6 m_table

QTableWidget* AthleteListDialog::m_table = nullptr [private]
 Definition at line 39 of file [AthleteListDialog.h](#).

The documentation for this class was generated from the following files:

- [src/gui/AthleteListDialog.h](#)
- [src/gui/AthleteListDialog.cpp](#)

7.16 AthleteRepository Class Reference

Repository for athlete profile data persistence.

#include <AthleteRepository.h>

Public Member Functions

- [AthleteRepository](#) (QSqlDatabase &db)
Constructs repository with database connection.
- int [insertAthlete](#) (const [Athlete](#) &a)
Inserts a new athlete.
- bool [updateAthlete](#) (const [Athlete](#) &a)
Updates an existing athlete.

- bool [deleteAthlete](#) (int athleteId)
Deletes an athlete.
- QList< [Athlete](#) > [listAthletes](#) ()
Lists all athletes.
- [Athlete](#) [getAthlete](#) (int athleteId)
Retrieves an athlete by ID.
- int [addFtpEntry](#) (const [FtpEntry](#) &e)
Adds an FTP history entry.
- bool [deleteFtpEntry](#) (int entryId)
Deletes an FTP history entry.
- QList< [FtpEntry](#) > [getFtpHistory](#) (int athleteId)
Retrieves FTP history for an athlete.
- int [getFtpForDate](#) (int athleteId, const QDate &date)
Gets FTP for an athlete on a specific date.
- int [addWeightEntry](#) (const [WeightEntry](#) &e)
Adds a weight history entry.
- bool [deleteWeightEntry](#) (int entryId)
Deletes a weight history entry.
- QList< [WeightEntry](#) > [getWeightHistory](#) (int athleteId)
Retrieves weight history for an athlete.

Private Attributes

- QSqlDatabase & [m_db](#)

7.16.1 Detailed Description

Repository for athlete profile data persistence.
Provides CRUD operations for athletes and their FTP/weight history.
Definition at line 96 of file [AthleteRepository.h](#).

7.16.2 Constructor & Destructor Documentation

7.16.2.1 AthleteRepository()

```
AthleteRepository::AthleteRepository (
    QSqlDatabase & db) [explicit]
```

Constructs repository with database connection.

Parameters

<i>db</i>	Database connection.
-----------	----------------------

Definition at line 11 of file [AthleteRepository.cpp](#).

7.16.3 Member Function Documentation

7.16.3.1 addFtpEntry()

```
int AthleteRepository::addFtpEntry (
    const FtpEntry & e)
```

Adds an FTP history entry.

Parameters

<i>e</i>	FTP entry to add.
----------	-------------------

Returns

ID of inserted entry, or -1 on error.

Definition at line 106 of file [AthleteRepository.cpp](#).

References [FtpEntry::athleteId](#), [FtpEntry::effectiveFrom](#), [FtpEntry::ftpWatts](#), and [FtpEntry::notes](#).

7.16.3.2 addWeightEntry()

```
int AthleteRepository::addWeightEntry (  
    const WeightEntry & e)
```

Adds a weight history entry.

Parameters

<i>e</i>	Weight entry to add.
----------	----------------------

Returns

ID of inserted entry, or -1 on error.

Definition at line 167 of file [AthleteRepository.cpp](#).

References [WeightEntry::athleteId](#), [WeightEntry::effectiveFrom](#), and [WeightEntry::weightKg](#).

7.16.3.3 deleteAthlete()

```
bool AthleteRepository::deleteAthlete (  
    int athleteId)
```

Deletes an athlete.

Parameters

<i>athleteId</i>	Athlete ID.
------------------	-----------------------------

Returns

True if deletion succeeded.

Definition at line 51 of file [AthleteRepository.cpp](#).

7.16.3.4 deleteFtpEntry()

```
bool AthleteRepository::deleteFtpEntry (  
    int entryId)
```

Deletes an FTP history entry.

Parameters

<i>entryId</i>	Entry ID.
----------------	-----------

Returns

True if deletion succeeded.

Definition at line 120 of file [AthleteRepository.cpp](#).

7.16.3.5 deleteWeightEntry()

```
bool AthleteRepository::deleteWeightEntry (
    int entryId)
```

Deletes a weight history entry.

Parameters

<i>entryId</i>	Entry ID.
----------------	-----------

Returns

True if deletion succeeded.

Definition at line 180 of file [AthleteRepository.cpp](#).

7.16.3.6 getAthlete()

```
Athlete AthleteRepository::getAthlete (
    int athleteId)
```

Retrieves an athlete by ID.

Parameters

<i>athleteId</i>	Athlete ID.
------------------	-----------------------------

Returns

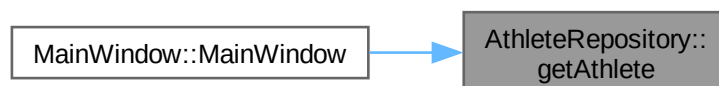
[Athlete](#) record.

Definition at line 81 of file [AthleteRepository.cpp](#).

References [Athlete::createdAt](#), [Athlete::dateOfBirth](#), [Athlete::email](#), [Athlete::firstName](#), [Athlete::ftpWatts](#), [Athlete::heightCm](#), [Athlete::id](#), and [Athlete::lastName](#).

Referenced by [MainWindow::MainWindow\(\)](#).

Here is the caller graph for this function:



7.16.3.7 getFtpForDate()

```
int AthleteRepository::getFtpForDate (
    int athleteId,
    const QDate & date)
```

Gets FTP for an athlete on a specific date.

Parameters

<i>athleteId</i>	Athlete ID.
<i>date</i>	Query date.

Returns

FTP in watts, or 0 if no entry found on or before date.

Definition at line 149 of file [AthleteRepository.cpp](#).

7.16.3.8 getFtpHistory()

```
QList< FtpEntry > AthleteRepository::getFtpHistory (
    int athleteId)
```

Retrieves FTP history for an athlete.

Parameters

<i>athleteId</i>	Athlete ID.
------------------	-------------

Returns

List of FTP entries.

Definition at line 128 of file [AthleteRepository.cpp](#).

References [FtpEntry::athleteId](#), [FtpEntry::effectiveFrom](#), [FtpEntry::ftpWatts](#), [FtpEntry::id](#), and [FtpEntry::notes](#).

7.16.3.9 getWeightHistory()

```
QList< WeightEntry > AthleteRepository::getWeightHistory (
    int athleteId)
```

Retrieves weight history for an athlete.

Parameters

<i>athleteId</i>	Athlete ID.
------------------	-------------

Returns

Weight entries sorted by date.

Definition at line 188 of file [AthleteRepository.cpp](#).

References [WeightEntry::athleteId](#), [WeightEntry::effectiveFrom](#), [WeightEntry::id](#), and [WeightEntry::weightKg](#).

7.16.3.10 insertAthlete()

```
int AthleteRepository::insertAthlete (  
    const Athlete & a)
```

Inserts a new athlete.

Parameters

<i>a</i>	Athlete to insert.
----------	--------------------

Returns

ID of inserted athlete, or -1 on error.

Definition at line 17 of file [AthleteRepository.cpp](#).

References [Athlete::dateOfBirth](#), [Athlete::email](#), [Athlete::firstName](#), [Athlete::ftpWatts](#), [Athlete::heightCm](#), and [Athlete::lastName](#).

7.16.3.11 listAthletes()

```
QList< Athlete > AthleteRepository::listAthletes ()
```

Lists all athletes.

Returns

List of athlete records.

Definition at line 59 of file [AthleteRepository.cpp](#).

References [Athlete::createdAt](#), [Athlete::dateOfBirth](#), [Athlete::email](#), [Athlete::firstName](#), [Athlete::ftpWatts](#), [Athlete::heightCm](#), [Athlete::id](#), and [Athlete::lastName](#).

7.16.3.12 updateAthlete()

```
bool AthleteRepository::updateAthlete (  
    const Athlete & a)
```

Updates an existing athlete.

Parameters

<i>a</i>	Athlete with updated values.
----------	------------------------------

Returns

True if update succeeded.

Definition at line 35 of file [AthleteRepository.cpp](#).

References [Athlete::dateOfBirth](#), [Athlete::email](#), [Athlete::firstName](#), [Athlete::ftpWatts](#), [Athlete::heightCm](#), [Athlete::id](#), and [Athlete::lastName](#).

7.16.4 Member Data Documentation

7.16.4.1 m_db

```
QSqlDatabase& AthleteRepository::m_db [private]
```

Definition at line 190 of file [AthleteRepository.h](#).

The documentation for this class was generated from the following files:

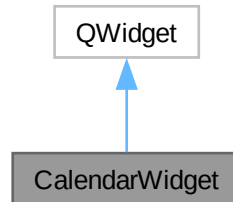
- [src/database/AthleteRepository.h](#)
- [src/database/AthleteRepository.cpp](#)

7.17 CalendarWidget Class Reference

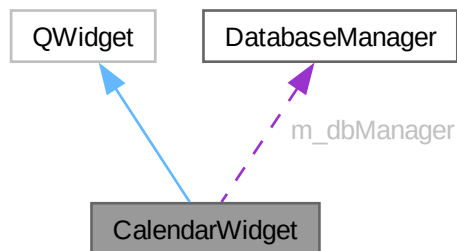
Calendar widget for activity scheduling and display.

```
#include <CalendarWidget.h>
```

Inheritance diagram for CalendarWidget:



Collaboration diagram for CalendarWidget:



Public Member Functions

- [CalendarWidget](#) (QWidget *parent=nullptr)
- void [setDatabaseManager](#) (DatabaseManager *dbManager)
- void [setAthletelId](#) (int athleteId)
- void [refresh](#) ()

Private Slots

- void [onDateChanged](#) ()
- void [addPlannedWorkout](#) ()
- void [editSelectedWorkout](#) ()
- void [deleteSelectedWorkout](#) ()
- void [recolorMonth](#) ()

Private Member Functions

- void [reloadDayList](#) (const QDate &date)
- void [updateMonthFormatting](#) ()

Private Attributes

- `QCalendarWidget * m_calendar = nullptr`
- `QComboBox * m_colorByCombo = nullptr`
- `QListWidget * m_dayList = nullptr`
- `QPushButton * m_addButton = nullptr`
- `QPushButton * m_editButton = nullptr`
- `QPushButton * m_deleteButton = nullptr`
- `DatabaseManager * m_dbManager = nullptr`
- `int m_athleteld = -1`

7.17.1 Detailed Description

Calendar widget for activity scheduling and display.

Shows activities by date with customizable coloring and planned workout support.

Definition at line 22 of file [CalendarWidget.h](#).

7.17.2 Constructor & Destructor Documentation

7.17.2.1 CalendarWidget()

```
CalendarWidget::CalendarWidget (  
    QWidget * parent = nullptr) [explicit]
```

Definition at line 22 of file [CalendarWidget.cpp](#).

7.17.3 Member Function Documentation

7.17.3.1 addPlannedWorkout

```
void CalendarWidget::addPlannedWorkout () [private], [slot]
```

Definition at line 95 of file [CalendarWidget.cpp](#).

7.17.3.2 deleteSelectedWorkout

```
void CalendarWidget::deleteSelectedWorkout () [private], [slot]
```

Definition at line 185 of file [CalendarWidget.cpp](#).

7.17.3.3 editSelectedWorkout

```
void CalendarWidget::editSelectedWorkout () [private], [slot]
```

Definition at line 130 of file [CalendarWidget.cpp](#).

7.17.3.4 onDateChanged

```
void CalendarWidget::onDateChanged () [private], [slot]
```

Definition at line 88 of file [CalendarWidget.cpp](#).

7.17.3.5 recolorMonth

```
void CalendarWidget::recolorMonth () [private], [slot]
```

Definition at line 210 of file [CalendarWidget.cpp](#).

7.17.3.6 refresh()

```
void CalendarWidget::refresh ()
```

Definition at line 80 of file [CalendarWidget.cpp](#).

7.17.3.7 reloadDayList()

```
void CalendarWidget::reloadDayList (
    const QDate & date) [private]
```

Definition at line 215 of file [CalendarWidget.cpp](#).

7.17.3.8 setAthleteld()

```
void CalendarWidget::setAthleteId (
    int athleteId)
```

Definition at line 74 of file [CalendarWidget.cpp](#).

7.17.3.9 setDatabaseManager()

```
void CalendarWidget::setDatabaseManager (
    DatabaseManager * dbManager)
```

Definition at line 68 of file [CalendarWidget.cpp](#).

7.17.3.10 updateMonthFormatting()

```
void CalendarWidget::updateMonthFormatting () [private]
```

Definition at line 257 of file [CalendarWidget.cpp](#).

7.17.4 Member Data Documentation

7.17.4.1 m_addButton

```
QPushButton* CalendarWidget::m_addButton = nullptr [private]
```

Definition at line 50 of file [CalendarWidget.h](#).

7.17.4.2 m_athleteld

```
int CalendarWidget::m_athleteId = -1 [private]
```

Definition at line 55 of file [CalendarWidget.h](#).

7.17.4.3 m_calendar

```
QCalendarWidget* CalendarWidget::m_calendar = nullptr [private]
```

Definition at line 47 of file [CalendarWidget.h](#).

7.17.4.4 m_colorByCombo

```
QComboBox* CalendarWidget::m_colorByCombo = nullptr [private]
```

Definition at line 48 of file [CalendarWidget.h](#).

7.17.4.5 m_dayList

```
QListWidget* CalendarWidget::m_dayList = nullptr [private]
```

Definition at line 49 of file [CalendarWidget.h](#).

7.17.4.6 m_dbManager

```
DatabaseManager* CalendarWidget::m_dbManager = nullptr [private]
```

Definition at line 54 of file [CalendarWidget.h](#).

7.17.4.7 m_deleteButton

```
QPushButton* CalendarWidget::m_deleteButton = nullptr [private]
```

Definition at line 52 of file [CalendarWidget.h](#).

7.17.4.8 m_editButton

QPushButton* CalendarWidget::m_editButton = nullptr [private]

Definition at line 51 of file [CalendarWidget.h](#).

The documentation for this class was generated from the following files:

- src/gui/CalendarWidget.h
- src/gui/CalendarWidget.cpp

7.18 Climb Struct Reference

Represents a detected climb with comprehensive metrics.

#include <ClimbMetrics.h>

Public Attributes

- int [id](#) = -1
Unique climb identifier (database row id).
- double [startSeconds](#) = 0.0
Climb start time in seconds from activity start.
- double [endSeconds](#) = 0.0
Climb end time in seconds from activity start.
- double [startDistanceKm](#) = 0.0
Climb start distance in km from activity start.
- double [endDistanceKm](#) = 0.0
Climb end distance in km from activity start.
- double [durationSeconds](#) = 0.0
Climb duration in seconds.
- double [lengthKm](#) = 0.0
Climb length in km.
- double [elevationGainM](#) = 0.0
Total elevation gain in meters.
- double [startElevationM](#) = 0.0
Starting elevation in meters.
- double [endElevationM](#) = 0.0
Ending elevation in meters.
- double [averageGradient](#) = 0.0
Average gradient percentage.
- double [maximumGradient](#) = 0.0
Maximum gradient percentage.
- double [climbScore](#) = 0.0
Composite climb score.
- double [gradePercentile](#) = 0.0
Percentile of climb with grade above threshold.
- ClimbCategory [climbCategory](#) = ClimbCategory::FalsePositive
Climb category (Major, Medium, Punchy, FalsePositive).
- double [averagePower](#) = 0.0
Average power in watts.
- double [averageWattsPerKg](#) = 0.0
Average power to weight in watts/kg.
- double [normalizedPower](#) = 0.0
Normalized power in watts.
- double [averageHeartRate](#) = 0.0

- Average heart rate in bpm.*
 - double `averageCadence` = 0.0
- Average cadence in rpm.*
 - double `averageSpeed` = 0.0
- Average speed in km/h.*
 - double `var` = 0.0
- VAM (Vertical Ascent Meter) in m/h.*
 - double `powerFirstHalf` = 0.0
- Average power in first half.*
 - double `powerSecondHalf` = 0.0
- Average power in second half.*
 - double `powerFadePct` = 0.0
- Power fade percentage from first to second half.*
 - double `aerobicDecouplingPct` = 0.0
- Aerobic decoupling percentage.*
 - double `hrDriftPct` = 0.0
- Heart rate drift percentage.*
 - double `difficultyScore` = 0.0
- Overall difficulty score.*
 - `std::array< ClimbQuarterMetrics, 4 > quarterMetrics {}`
- Metrics for each quarter of climb.*
 - `QString category`
- Category description string.*
 - `QString shapeClass`
- Shape classification (steep, ramp, var, etc.).*
 - `QString wattsPerKgRank`
- Watts/kg performance ranking.*
 - `QString name`
- Climb name.*

7.18.1 Detailed Description

Represents a detected climb with comprehensive metrics.

Contains temporal, spatial, effort, and performance data for a single detected climb.

Definition at line 44 of file [ClimbMetrics.h](#).

7.18.2 Member Data Documentation

7.18.2.1 aerobicDecouplingPct

```
double Climb::aerobicDecouplingPct = 0.0
```

Aerobic decoupling percentage.

Definition at line 122 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

7.18.2.2 averageCadence

```
double Climb::averageCadence = 0.0
```

Average cadence in rpm.

Definition at line 104 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

7.18.2.3 averageGradient

```
double Climb::averageGradient = 0.0
```

Average gradient percentage.

Definition at line 77 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#), [ClimbDetector::detect\(\)](#), and [ClimbDetector::mergeFragmentedClimbs\(\)](#).

7.18.2.4 averageHeartRate

```
double Climb::averageHeartRate = 0.0
```

Average heart rate in bpm.

Definition at line 101 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

7.18.2.5 averagePower

```
double Climb::averagePower = 0.0
```

Average power in watts.

Definition at line 92 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

7.18.2.6 averageSpeed

```
double Climb::averageSpeed = 0.0
```

Average speed in km/h.

Definition at line 107 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

7.18.2.7 averageWattsPerKg

```
double Climb::averageWattsPerKg = 0.0
```

Average power to weight in watts/kg.

Definition at line 95 of file [ClimbMetrics.h](#).

7.18.2.8 category

```
QString Climb::category
```

Category description string.

Definition at line 134 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

7.18.2.9 climbCategory

```
ClimbCategory Climb::climbCategory = ClimbCategory::FalsePositive
```

[Climb](#) category (Major, Medium, Punchy, FalsePositive).

Definition at line 89 of file [ClimbMetrics.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

7.18.2.10 climbScore

```
double Climb::climbScore = 0.0
```

Composite climb score.

Definition at line 83 of file [ClimbMetrics.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

7.18.2.11 difficultyScore

```
double Climb::difficultyScore = 0.0
```

Overall difficulty score.

Definition at line 128 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

7.18.2.12 durationSeconds

```
double Climb::durationSeconds = 0.0
```

[Climb](#) duration in seconds.

Definition at line 62 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#), and [ClimbDetector::mergeFragmentedClimbs\(\)](#).

7.18.2.13 elevationGainM

```
double Climb::elevationGainM = 0.0
```

Total elevation gain in meters.

Definition at line 68 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#), [ClimbDetector::detect\(\)](#), and [ClimbDetector::mergeFragmentedClimbs\(\)](#).

7.18.2.14 endDistanceKm

```
double Climb::endDistanceKm = 0.0
```

[Climb](#) end distance in km from activity start.

Definition at line 59 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#), and [ClimbDetector::mergeFragmentedClimbs\(\)](#).

7.18.2.15 endElevationM

```
double Climb::endElevationM = 0.0
```

Ending elevation in meters.

Definition at line 74 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#), and [ClimbDetector::mergeFragmentedClimbs\(\)](#).

7.18.2.16 endSeconds

```
double Climb::endSeconds = 0.0
```

[Climb](#) end time in seconds from activity start.

Definition at line 53 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#), [RideChartWidget::drawForeground\(\)](#), [ClimbDetector::mergeFragmentedClimbs\(\)](#), [RideChartWidget::mouseMoveEvent\(\)](#), [RideChartWidget::mousePressEvent\(\)](#), and [RideChartWidget::mouseReleaseEvent\(\)](#).

7.18.2.17 gradePercentile

```
double Climb::gradePercentile = 0.0
```

Percentile of climb with grade above threshold.

Definition at line 86 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#), and [ClimbDetector::detect\(\)](#).

7.18.2.18 hrDriftPct

```
double Climb::hrDriftPct = 0.0
```

Heart rate drift percentage.

Definition at line 125 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

7.18.2.19 id

```
int Climb::id = -1
```

Unique climb identifier (database row id).

Definition at line 47 of file [ClimbMetrics.h](#).

7.18.2.20 lengthKm

```
double Climb::lengthKm = 0.0
```

[Climb](#) length in km.

Definition at line 65 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#), [ClimbDetector::detect\(\)](#), and [ClimbDetector::mergeFragmentedClimbs\(\)](#).

7.18.2.21 maximumGradient

```
double Climb::maximumGradient = 0.0
```

Maximum gradient percentage.

Definition at line 80 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#), and [ClimbDetector::detect\(\)](#).

7.18.2.22 name

```
QString Climb::name
```

[Climb](#) name.

Definition at line 143 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

7.18.2.23 normalizedPower

```
double Climb::normalizedPower = 0.0
```

Normalized power in watts.

Definition at line 98 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

7.18.2.24 powerFadePct

```
double Climb::powerFadePct = 0.0
```

Power fade percentage from first to second half.

Definition at line 119 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

7.18.2.25 powerFirstHalf

```
double Climb::powerFirstHalf = 0.0
```

Average power in first half.

Definition at line 113 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

7.18.2.26 powerSecondHalf

```
double Climb::powerSecondHalf = 0.0
```

Average power in second half.

Definition at line 116 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

7.18.2.27 quarterMetrics

```
std::array<ClimbQuarterMetrics, 4> Climb::quarterMetrics {}
```

Metrics for each quarter of climb.

Definition at line 131 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

7.18.2.28 shapeClass

```
QString Climb::shapeClass
```

Shape classification (steep, ramp, var, etc.).

Definition at line 137 of file [ClimbMetrics.h](#).
 Referenced by [ClimbMetrics::build\(\)](#).

7.18.2.29 startDistanceKm

`double Climb::startDistanceKm = 0.0`
[Climb](#) start distance in km from activity start.
 Definition at line 56 of file [ClimbMetrics.h](#).
 Referenced by [ClimbMetrics::build\(\)](#), and [ClimbDetector::mergeFragmentedClimbs\(\)](#).

7.18.2.30 startElevationM

`double Climb::startElevationM = 0.0`
 Starting elevation in meters.
 Definition at line 71 of file [ClimbMetrics.h](#).
 Referenced by [ClimbMetrics::build\(\)](#), and [ClimbDetector::mergeFragmentedClimbs\(\)](#).

7.18.2.31 startSeconds

`double Climb::startSeconds = 0.0`
[Climb](#) start time in seconds from activity start.
 Definition at line 50 of file [ClimbMetrics.h](#).
 Referenced by [ClimbMetrics::build\(\)](#), [RideChartWidget::drawForeground\(\)](#), [ClimbDetector::mergeFragmentedClimbs\(\)](#), [RideChartWidget::mouseMoveEvent\(\)](#), [RideChartWidget::mousePressEvent\(\)](#), and [RideChartWidget::mouseReleaseEvent\(\)](#).

7.18.2.32 vam

`double Climb::vam = 0.0`
 VAM (Vertical Ascent Meter) in m/h.
 Definition at line 110 of file [ClimbMetrics.h](#).
 Referenced by [ClimbMetrics::build\(\)](#), and [ClimbDetector::detect\(\)](#).

7.18.2.33 wattsPerKgRank

`QString Climb::wattsPerKgRank`
 Watts/kg performance ranking.
 Definition at line 140 of file [ClimbMetrics.h](#).
 The documentation for this struct was generated from the following file:

- `src/analysis/ClimbMetrics.h`

7.19 ClimbDetector Class Reference

Detects and categorizes climbs in cycling activities.
`#include <ClimbDetector.h>`

Public Types

- using [Config](#) = [ClimbDetectorConfig](#)
Configuration typedef for climb detection parameters.

Static Public Member Functions

- static `std::vector< Climb > detect` (const [RideData](#) &rideData, const [Config](#) &config=[Config](#){})
Detects climbs in activity data using multi-phase algorithm.
- static `std::vector< double > buildCumulativeDistanceMeters` (const [RideData](#) &rideData)
Builds cumulative distance array for activity.

- static std::vector< double > [smoothAltitudeByDistance](#) (const [RideData](#) &rideData, const std::vector< double > &cumulativeDistanceMeters, double windowMeters, const std::vector< double > &adaptiveWindowMeters={}, double altitudeSpikeVerticalSpeedMps=8.0)
Smooths altitude data using distance-based rolling window.
- static std::vector< double > [computeAdaptiveSmoothingWindowsMeters](#) (const [RideData](#) &rideData, const std::vector< double > &cumulativeDistanceMeters, const [Config](#) &config)
Computes adaptive smoothing windows for elevation data.
- static std::vector< double > [computeLocalGradientPct](#) (const std::vector< double > &smoothedAltitudeMeters, const std::vector< double > &cumulativeDistanceMeters, double halfWindowMeters)
Computes local gradient at each point.
- static double [computeClimbScore](#) (double gainMeters, double averageGrade, double distanceKm, double continuityRatio, double vam)
Computes scoring-based climb difficulty metric.
- static double [computeVAM](#) (double elevationGainM, double durationSeconds)
Computes vertical ascent metric (VAM).
- static ClimbCategory [categorizeClimb](#) (double gainMeters, double lengthKm, double averageGrade)
Categorizes a climb by difficulty.
- static std::vector< [Climb](#) > [mergeFragmentedClimbs](#) (std::vector< [Climb](#) > &candidates, const [Config](#) &config)
Merges fragmented climb candidates into continuous climbs.

7.19.1 Detailed Description

Detects and categorizes climbs in cycling activities.

Provides static methods for analyzing activity elevation data to identify climbs using multi-phase detection including gradient thresholds, scoring, and merge logic.

Definition at line 100 of file [ClimbDetector.h](#).

7.19.2 Member Typedef Documentation

7.19.2.1 Config

```
using ClimbDetector::Config = ClimbDetectorConfig
```

Configuration typedef for climb detection parameters.

Definition at line 104 of file [ClimbDetector.h](#).

7.19.3 Member Function Documentation

7.19.3.1 buildCumulativeDistanceMeters()

```
std::vector< double > ClimbDetector::buildCumulativeDistanceMeters (
    const RideData & rideData) [static]
```

Builds cumulative distance array for activity.

Parameters

<i>rideData</i>	Activity data.
-----------------	--------------------------------

Returns

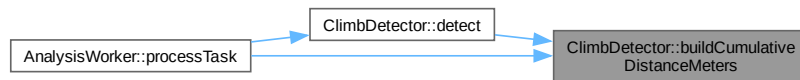
Array of cumulative distances in meters.

Definition at line 108 of file [ClimbDetector.cpp](#).

References [RideRecord::elapsedSeconds](#), [RideRecord::hasGps](#), [RideRecord::hasSpeed](#), [RideRecord::latitude](#), [RideRecord::longitude](#), [RideData::records](#), and [RideRecord::speed](#).

Referenced by [detect\(\)](#), and [AnalysisWorker::processTask\(\)](#).

Here is the caller graph for this function:



7.19.3.2 categorizeClimb()

```

ClimbCategory ClimbDetector::categorizeClimb (
    double gainMeters,
    double lengthKm,
    double averageGrade) [static]
  
```

Categorizes a climb by difficulty.

Parameters

<i>gainMeters</i>	Elevation gain in meters.
<i>lengthKm</i>	Climb length in kilometers.
<i>averageGrade</i>	Average gradient percentage.

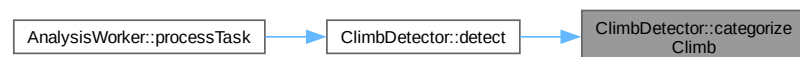
Returns

Category classification.

Definition at line 309 of file [ClimbDetector.cpp](#).

Referenced by [detect\(\)](#).

Here is the caller graph for this function:



7.19.3.3 computeAdaptiveSmoothingWindowsMeters()

```

std::vector< double > ClimbDetector::computeAdaptiveSmoothingWindowsMeters (
    const RideData & rideData,
    const std::vector< double > & cumulativeDistanceMeters,
    const Config & config) [static]
  
```

Computes adaptive smoothing windows for elevation data.

Parameters

<i>rideData</i>	Activity data.
<i>cumulativeDistanceMeters</i>	Cumulative distances.
<i>config</i>	Configuration with adaptive smoothing parameters.

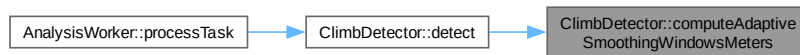
Returns

Array of adaptive window sizes.

Definition at line 207 of file [ClimbDetector.cpp](#).

References [ClimbDetectorConfig::adaptiveSmoothingMaxMeters](#), [ClimbDetectorConfig::adaptiveSmoothingMinMeters](#), [ClimbDetectorConfig::elevationSmoothingMeters](#), [RideData::records](#), and [ClimbDetectorConfig::useAdaptiveSmoothing](#).
Referenced by [detect\(\)](#).

Here is the caller graph for this function:

**7.19.3.4 computeClimbScore()**

```
double ClimbDetector::computeClimbScore (
    double gainMeters,
    double averageGrade,
    double distanceKm,
    double continuityRatio,
    double vam) [static]
```

Computes scoring-based climb difficulty metric.

Parameters

<i>gainMeters</i>	Elevation gain in meters.
<i>averageGrade</i>	Average gradient percentage.
<i>distanceKm</i>	Climb distance in kilometers.
<i>continuityRatio</i>	Ratio of continuous vs interrupted climbing.
<i>vam</i>	Vertical ascent speed in meters/hour.

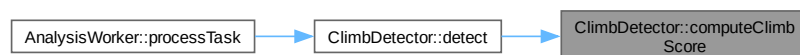
Returns

Computed climb score.

Definition at line 280 of file [ClimbDetector.cpp](#).

Referenced by [detect\(\)](#).

Here is the caller graph for this function:

**7.19.3.5 computeLocalGradientPct()**

```
std::vector< double > ClimbDetector::computeLocalGradientPct (
    const std::vector< double > & smoothedAltitudeMeters,
```

```
const std::vector< double > & cumulativeDistanceMeters,
double halfWindowMeters) [static]
```

Computes local gradient at each point.

Parameters

<i>smoothedAltitudeMeters</i>	Smoothed elevation data.
<i>cumulativeDistanceMeters</i>	Cumulative distances.
<i>halfWindowMeters</i>	Half-width of gradient window in meters.

Returns

Array of gradient percentages.

Definition at line 239 of file [ClimbDetector.cpp](#).

Referenced by [detect\(\)](#), and [AnalysisWorker::processTask\(\)](#).

Here is the caller graph for this function:



7.19.3.6 computeVAM()

```
double ClimbDetector::computeVAM (
    double elevationGainM,
    double durationSeconds) [static]
```

Computes vertical ascent metric (VAM).

Parameters

<i>elevationGainM</i>	Elevation gain in meters.
<i>durationSeconds</i>	Duration in seconds.

Returns

VAM in meters/hour.

Definition at line 296 of file [ClimbDetector.cpp](#).

7.19.3.7 detect()

```
std::vector< Climb > ClimbDetector::detect (
    const RideData & rideData,
    const Config & config = Config{}) [static]
```

Detects climbs in activity data using multi-phase algorithm.

Parameters

<i>rideData</i>	Activity recording data.
-----------------	--

<i>config</i>	Detection configuration parameters.
---------------	-------------------------------------

Returns

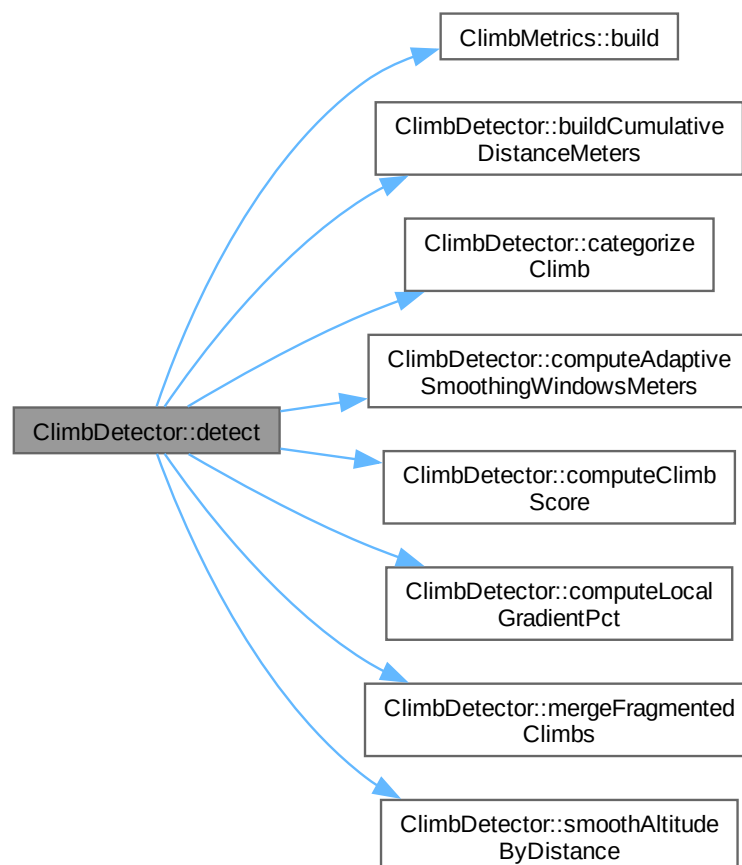
Vector of detected climbs.

Definition at line 396 of file [ClimbDetector.cpp](#).

References [ClimbDetectorConfig::altitudeSpikeVerticalSpeedMps](#), [Climb::averageGradient](#), [ClimbMetrics::build\(\)](#), [buildCumulativeDistanceMeters\(\)](#), [categorizeClimb\(\)](#), [Climb::climbCategory](#), [Climb::climbScore](#), [computeAdaptiveSmoothingWindowsMeters\(\)](#), [computeClimbScore\(\)](#), [computeLocalGradientPct\(\)](#), [ClimbDetectorConfig::continueGradient](#), [Climb::elevationGainM](#), [ClimbDetectorConfig::elevationSmoothingMeters](#), [ClimbDetectorConfig::enableClimbMerging](#), [ClimbDetectorConfig::endGradient](#), [Climb::gradePercentile](#), [RideRecord::hasAltitude](#), [Climb::lengthKm](#), [ClimbDetectorConfig::maxDipDistanceMeters](#), [ClimbDetectorConfig::maxDipMeters](#), [ClimbDetectorConfig::maxDipPctOfGain](#), [Climb::maximumGradient](#), [ClimbDetectorConfig::maxPlateauDistanceMeters](#), [ClimbDetectorConfig::maxSmallDipsCount](#), [ClimbDetectorConfig::maxStartInterrupt](#), [mergeFragmentedClimbs\(\)](#), [ClimbDetectorConfig::minAverageGradient](#), [ClimbDetectorConfig::minClimbScore](#), [ClimbDetectorConfig::minElevationGainM](#), [ClimbDetectorConfig::minLengthKm](#), [ClimbDetectorConfig::minStartContinuousDistanceMeters](#), [ClimbDetectorConfig::plateauMinGradient](#), [RideData::records](#), [ClimbDetectorConfig::smallDipThresholdMeters](#), [smoothAltitudeByDistance\(\)](#), [ClimbDetectorConfig::startGradient](#), [ClimbDetectorConfig::useScoringDetection](#), and [Climb::vam](#).

Referenced by [AnalysisWorker::processTask\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.19.3.8 mergeFragmentedClimbs()

```

std::vector< Climb > ClimbDetector::mergeFragmentedClimbs (
    std::vector< Climb > & candidates,
    const Config & config) [static]
  
```

Merges fragmented climb candidates into continuous climbs.

Parameters

<i>candidates</i>	Vector of initial climb candidates.
<i>config</i>	Merge configuration parameters.

Returns

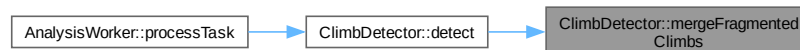
Merged climb results.

Definition at line 330 of file [ClimbDetector.cpp](#).

References [Climb::averageGradient](#), [Climb::durationSeconds](#), [Climb::elevationGainM](#), [ClimbDetectorConfig::enableClimbMerging](#), [Climb::endDistanceKm](#), [Climb::endElevationM](#), [Climb::endSeconds](#), [Climb::lengthKm](#), [ClimbDetectorConfig::mergeMaxGapElevation](#), [ClimbDetectorConfig::mergeMaxGapMeters](#), [Climb::startDistanceKm](#), [Climb::startElevationM](#), and [Climb::startSeconds](#).

Referenced by [detect\(\)](#).

Here is the caller graph for this function:



7.19.3.9 smoothAltitudeByDistance()

```

std::vector< double > ClimbDetector::smoothAltitudeByDistance (
    const RideData & rideData,
    const std::vector< double > & cumulativeDistanceMeters,
    double windowMeters,
    const std::vector< double > & adaptiveWindowMeters = {},
    double altitudeSpikeVerticalSpeedMps = 8.0) [static]
  
```

Smooths altitude data using distance-based rolling window.

Parameters

<i>rideData</i>	Activity data.
-----------------	--------------------------------

<i>cumulativeDistanceMeters</i>	Cumulative distances.
<i>windowMeters</i>	Fixed smoothing window in meters.
<i>adaptiveWindowMeters</i>	Optional adaptive window per sample.
<i>altitudeSpikeVerticalSpeedMps</i>	Spike detection threshold in m/s.

Returns

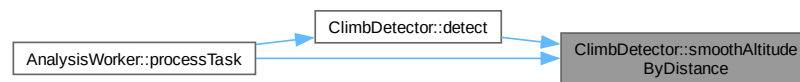
Smoothed altitude array.

Definition at line 147 of file [ClimbDetector.cpp](#).

References [RideData::records](#).

Referenced by [detect\(\)](#), and [AnalysisWorker::processTask\(\)](#).

Here is the caller graph for this function:



The documentation for this class was generated from the following files:

- `src/analysis/ClimbDetector.h`
- `src/analysis/ClimbDetector.cpp`

7.20 ClimbDetectorConfig Struct Reference

Configuration parameters for climb detection.

```
#include <ClimbDetector.h>
```

Public Attributes

- double `minLengthKm` = 0.15
Minimum climb length in km (legacy).
- double `minElevationGainM` = 10.0
Minimum elevation gain in meters (legacy).
- double `minAverageGradient` = 4.0
Minimum average gradient (legacy).
- double `startGradient` = 1.5
Gradient threshold to start climb detection (%).
- double `continueGradient` = 1.0
Gradient threshold to continue climb (%).
- double `endGradient` = 0.5
Gradient threshold to end climb (%).
- double `minStartContinuousDistanceMeters` = 100.0
Minimum continuous distance at start gradient (meters).
- double `maxStartInterruptionMeters` = 30.0
Maximum interruption when starting climb (meters).
- double `maxDipMeters` = 10.0
Maximum dip elevation loss (meters).
- double `maxDipDistanceMeters` = 200.0

- Maximum dip horizontal distance (meters).*
 - double `maxDipPctOfGain` = 10.0
- Maximum dip as percentage of gain (%).*
 - double `maxSmallDipsCount` = 3
- Maximum number of small dips (< 5m) allowed.*
 - double `smallDipThresholdMeters` = 5.0
- Threshold for small dip detection (meters).*
 - double `maxPlateauDistanceMeters` = 500.0
- Maximum distance for plateau detection (meters).*
 - double `plateauMinGradient` = -1.0
- Minimum gradient for plateau (%).*
 - double `elevationSmoothingMeters` = 50.0
- Elevation smoothing window (meters).*
 - bool `useAdaptiveSmoothing` = true
- Enable adaptive smoothing based on grade changes.*
 - double `adaptiveSmoothingMinMeters` = 25.0
- Minimum adaptive smoothing window (meters).*
 - double `adaptiveSmoothingMaxMeters` = 100.0
- Maximum adaptive smoothing window (meters).*
 - double `gradientWindowHalfMeters` = 25.0
- Half-window for gradient computation (meters).*
 - double `altitudeSpikeVerticalSpeedMps` = 8.0
- Altitude spike detection threshold (m/s).*
 - bool `useScoringDetection` = true
- Enable scoring-based climb detection.*
 - double `minClimbScore` = 50.0
- Minimum composite climb score.*
 - double `gainScoreWeight` = 0.5
- Weight for elevation gain in score.*
 - double `gradeScoreWeight` = 4.0
- Weight for average grade in score.*
 - double `durationScoreWeight` = 2.0
- Weight for duration in score.*
 - double `vamScoreBoost` = 800.0
- VAM threshold for score boost.*
 - bool `requireGradePersistence` = true
- Enable grade persistence requirement.*
 - double `gradePersistenceThreshold` = 4.0
- Minimum gradient for persistence (%).*
 - double `gradePersistenceMinMeters` = 50.0
- Minimum distance at grade for persistence (meters).*
 - bool `enableClimbMerging` = true
- Enable merge of fragmented climbs.*
 - double `mergeMaxGapMeters` = 50.0
- Maximum gap between climbs to merge (meters).*
 - double `mergeMaxGapElevationM` = 10.0
- Maximum elevation gap to allow merge (meters).*

7.20.1 Detailed Description

Configuration parameters for climb detection.

Contains tunable thresholds and flags used to control climb detection behavior, including gradient thresholds, distance tolerances, and elevation smoothing parameters.

Definition at line 16 of file [ClimbDetector.h](#).

7.20.2 Member Data Documentation

7.20.2.1 adaptiveSmoothingMaxMeters

```
double ClimbDetectorConfig::adaptiveSmoothingMaxMeters = 100.0
```

Maximum adaptive smoothing window (meters).

Definition at line 59 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::computeAdaptiveSmoothingWindowsMeters\(\)](#).

7.20.2.2 adaptiveSmoothingMinMeters

```
double ClimbDetectorConfig::adaptiveSmoothingMinMeters = 25.0
```

Minimum adaptive smoothing window (meters).

Definition at line 57 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::computeAdaptiveSmoothingWindowsMeters\(\)](#).

7.20.2.3 altitudeSpikeVerticalSpeedMps

```
double ClimbDetectorConfig::altitudeSpikeVerticalSpeedMps = 8.0
```

Altitude spike detection threshold (m/s).

Definition at line 64 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

7.20.2.4 continueGradient

```
double ClimbDetectorConfig::continueGradient = 1.0
```

Gradient threshold to continue climb (%).

Definition at line 28 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

7.20.2.5 durationScoreWeight

```
double ClimbDetectorConfig::durationScoreWeight = 2.0
```

Weight for duration in score.

Definition at line 75 of file [ClimbDetector.h](#).

7.20.2.6 elevationSmoothingMeters

```
double ClimbDetectorConfig::elevationSmoothingMeters = 50.0
```

Elevation smoothing window (meters).

Definition at line 53 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::computeAdaptiveSmoothingWindowsMeters\(\)](#), [ClimbDetector::detect\(\)](#), and [AnalysisWorker::processTask\(\)](#).

7.20.2.7 enableClimbMerging

```
bool ClimbDetectorConfig::enableClimbMerging = true
```

Enable merge of fragmented climbs.

Definition at line 87 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#), and [ClimbDetector::mergeFragmentedClimbs\(\)](#).

7.20.2.8 endGradient

```
double ClimbDetectorConfig::endGradient = 0.5
```

Gradient threshold to end climb (%).

Definition at line 30 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

7.20.2.9 gainScoreWeight

```
double ClimbDetectorConfig::gainScoreWeight = 0.5
```

Weight for elevation gain in score.

Definition at line 71 of file [ClimbDetector.h](#).

7.20.2.10 gradePersistenceMinMeters

```
double ClimbDetectorConfig::gradePersistenceMinMeters = 50.0
```

Minimum distance at grade for persistence (meters).

Definition at line 84 of file [ClimbDetector.h](#).

7.20.2.11 gradePersistenceThreshold

```
double ClimbDetectorConfig::gradePersistenceThreshold = 4.0
```

Minimum gradient for persistence (%).

Definition at line 82 of file [ClimbDetector.h](#).

7.20.2.12 gradeScoreWeight

```
double ClimbDetectorConfig::gradeScoreWeight = 4.0
```

Weight for average grade in score.

Definition at line 73 of file [ClimbDetector.h](#).

7.20.2.13 gradientWindowHalfMeters

```
double ClimbDetectorConfig::gradientWindowHalfMeters = 25.0
```

Half-window for gradient computation (meters).

Definition at line 61 of file [ClimbDetector.h](#).

Referenced by [AnalysisWorker::processTask\(\)](#).

7.20.2.14 maxDipDistanceMeters

```
double ClimbDetectorConfig::maxDipDistanceMeters = 200.0
```

Maximum dip horizontal distance (meters).

Definition at line 39 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

7.20.2.15 maxDipMeters

```
double ClimbDetectorConfig::maxDipMeters = 10.0
```

Maximum dip elevation loss (meters).

Definition at line 37 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

7.20.2.16 maxDipPctOfGain

```
double ClimbDetectorConfig::maxDipPctOfGain = 10.0
```

Maximum dip as percentage of gain (%).

Definition at line 41 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

7.20.2.17 maxPlateauDistanceMeters

```
double ClimbDetectorConfig::maxPlateauDistanceMeters = 500.0
```

Maximum distance for plateau detection (meters).

Definition at line 48 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

7.20.2.18 maxSmallDipsCount

```
double ClimbDetectorConfig::maxSmallDipsCount = 3
```

Maximum number of small dips (< 5m) allowed.

Definition at line 43 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

7.20.2.19 maxStartInterruptionMeters

```
double ClimbDetectorConfig::maxStartInterruptionMeters = 30.0
```

Maximum interruption when starting climb (meters).

Definition at line 34 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

7.20.2.20 mergeMaxGapElevationM

```
double ClimbDetectorConfig::mergeMaxGapElevationM = 10.0
```

Maximum elevation gap to allow merge (meters).

Definition at line 91 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::mergeFragmentedClimbs\(\)](#).

7.20.2.21 mergeMaxGapMeters

```
double ClimbDetectorConfig::mergeMaxGapMeters = 50.0
```

Maximum gap between climbs to merge (meters).

Definition at line 89 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::mergeFragmentedClimbs\(\)](#).

7.20.2.22 minAverageGradient

```
double ClimbDetectorConfig::minAverageGradient = 4.0
```

Minimum average gradient (legacy).

Definition at line 23 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#), and [AnalysisWorker::processTask\(\)](#).

7.20.2.23 minClimbScore

```
double ClimbDetectorConfig::minClimbScore = 50.0
```

Minimum composite climb score.

Definition at line 69 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

7.20.2.24 minElevationGainM

```
double ClimbDetectorConfig::minElevationGainM = 10.0
```

Minimum elevation gain in meters (legacy).

Definition at line 21 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#), and [AnalysisWorker::processTask\(\)](#).

7.20.2.25 minLengthKm

```
double ClimbDetectorConfig::minLengthKm = 0.15
```

Minimum climb length in km (legacy).

Definition at line 19 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#), and [AnalysisWorker::processTask\(\)](#).

7.20.2.26 minStartContinuousDistanceMeters

```
double ClimbDetectorConfig::minStartContinuousDistanceMeters = 100.0
```

Minimum continuous distance at start gradient (meters).

Definition at line 32 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

7.20.2.27 plateauMinGradient

```
double ClimbDetectorConfig::plateauMinGradient = -1.0
```

Minimum gradient for plateau (%).

Definition at line 50 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

7.20.2.28 requireGradePersistence

```
bool ClimbDetectorConfig::requireGradePersistence = true
```

Enable grade persistence requirement.

Definition at line 80 of file [ClimbDetector.h](#).

7.20.2.29 smallDipThresholdMeters

```
double ClimbDetectorConfig::smallDipThresholdMeters = 5.0
```

Threshold for small dip detection (meters).

Definition at line 45 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

7.20.2.30 startGradient

```
double ClimbDetectorConfig::startGradient = 1.5
```

Gradient threshold to start climb detection (%).

Definition at line 26 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

7.20.2.31 useAdaptiveSmoothing

```
bool ClimbDetectorConfig::useAdaptiveSmoothing = true
```

Enable adaptive smoothing based on grade changes.

Definition at line 55 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::computeAdaptiveSmoothingWindowsMeters\(\)](#).

7.20.2.32 useScoringDetection

```
bool ClimbDetectorConfig::useScoringDetection = true
```

Enable scoring-based climb detection.

Definition at line 67 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

7.20.2.33 vamScoreBoost

```
double ClimbDetectorConfig::vamScoreBoost = 800.0
```

VAM threshold for score boost.

Definition at line 77 of file [ClimbDetector.h](#).

The documentation for this struct was generated from the following file:

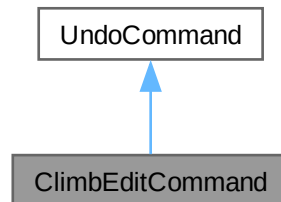
- [src/analysis/ClimbDetector.h](#)

7.21 ClimbEditCommand Class Reference

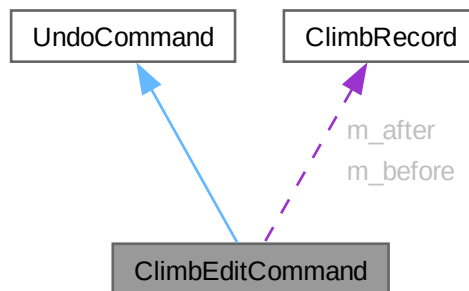
Undo/redo command for climb boundary or field edits.

```
#include <ClimbEditCommand.h>
```

Inheritance diagram for ClimbEditCommand:



Collaboration diagram for ClimbEditCommand:



Public Member Functions

- **ClimbEditCommand** (const **ClimbRecord** &before, const **ClimbRecord** &after, const QString &dbConnection↔ Name, std::function< void()> refreshCallback)
Constructs climb edit command.
- void **undo** () override
Reverses the command (undo operation).
- void **redo** () override
Re-applies the command (redo operation).
- QString **description** () const override
Returns a human-readable description of the operation.

Private Member Functions

- void **apply** (const **ClimbRecord** &record)

Private Attributes

- [ClimbRecord](#) `m_before`
- [ClimbRecord](#) `m_after`
- [QString](#) `m_connName`
- `std::function< void()>` `m_refresh`

7.21.1 Detailed Description

Undo/redo command for climb boundary or field edits.

Stores before/after [ClimbRecord](#) pairs and applies changes through the [ClimbRepository](#) during undo/redo operations.

Definition at line 19 of file [ClimbEditCommand.h](#).

7.21.2 Constructor & Destructor Documentation**7.21.2.1 ClimbEditCommand()**

```
ClimbEditCommand::ClimbEditCommand (
    const ClimbRecord & before,
    const ClimbRecord & after,
    const QString & dbConnectionName,
    std::function< void()> refreshCallback)
```

Constructs climb edit command.

Parameters

<i>before</i>	Previous climb record state.
<i>after</i>	New climb record state.
<i>dbConnectionName</i>	Database connection name.
<i>refreshCallback</i>	Function to call after applying changes.

Definition at line 8 of file [ClimbEditCommand.cpp](#).

7.21.3 Member Function Documentation**7.21.3.1 apply()**

```
void ClimbEditCommand::apply (
    const ClimbRecord & record) [private]
```

Definition at line 33 of file [ClimbEditCommand.cpp](#).

7.21.3.2 description()

```
QString ClimbEditCommand::description () const [override], [virtual]
```

Returns a human-readable description of the operation.

Returns

Short description shown in menus/status bar.

Reimplemented from [UndoCommand](#).

Definition at line 28 of file [ClimbEditCommand.cpp](#).

7.21.3.3 redo()

```
void ClimbEditCommand::redo () [override], [virtual]
```

Re-applies the command (redo operation).

Implements [UndoCommand](#).

Definition at line 23 of file [ClimbEditCommand.cpp](#).

7.21.3.4 undo()

```
void ClimbEditCommand::undo () [override], [virtual]
```

Reverses the command (undo operation).

Implements [UndoCommand](#).

Definition at line 18 of file [ClimbEditCommand.cpp](#).

7.21.4 Member Data Documentation

7.21.4.1 m_after

```
ClimbRecord ClimbEditCommand::m_after [private]
```

Definition at line 42 of file [ClimbEditCommand.h](#).

7.21.4.2 m_before

```
ClimbRecord ClimbEditCommand::m_before [private]
```

Definition at line 41 of file [ClimbEditCommand.h](#).

7.21.4.3 m_connName

```
QString ClimbEditCommand::m_connName [private]
```

Definition at line 43 of file [ClimbEditCommand.h](#).

7.21.4.4 m_refresh

```
std::function<void()> ClimbEditCommand::m_refresh [private]
```

Definition at line 44 of file [ClimbEditCommand.h](#).

The documentation for this class was generated from the following files:

- [src/core/undo/ClimbEditCommand.h](#)
- [src/core/undo/ClimbEditCommand.cpp](#)

7.22 anonymous_namespace{RideChartWidget.cpp}::ClimbHandleHitResult Struct Reference

Public Attributes

- bool [hit](#) = false
- bool [startHandle](#) = true

7.22.1 Detailed Description

Definition at line 37 of file [RideChartWidget.cpp](#).

7.22.2 Member Data Documentation

7.22.2.1 hit

```
bool anonymous_namespace{RideChartWidget.cpp}::ClimbHandleHitResult::hit = false
```

Definition at line 39 of file [RideChartWidget.cpp](#).

7.22.2.2 startHandle

```
bool anonymous_namespace{RideChartWidget.cpp}::ClimbHandleHitResult::startHandle = true
```

Definition at line 40 of file [RideChartWidget.cpp](#).

The documentation for this struct was generated from the following file:

- [src/charts/RideChartWidget.cpp](#)

7.23 ClimbMetrics Class Reference

Computes comprehensive climb metrics from activity data.

```
#include <ClimbMetrics.h>
```

Static Public Member Functions

- static [Climb](#) **build** (const [RideData](#) &rideData, const std::vector< double > &cumulativeDistanceMeters, const std::vector< double > &smoothedAltitudeMeters, const std::vector< double > &localGradientPct, int startIdx, int endIdx, int climbOrdinal)

Builds a complete [Climb](#) object from activity data.

7.23.1 Detailed Description

Computes comprehensive climb metrics from activity data.

Definition at line 149 of file [ClimbMetrics.h](#).

7.23.2 Member Function Documentation

7.23.2.1 build()

```
Climb ClimbMetrics::build (
    const RideData & rideData,
    const std::vector< double > & cumulativeDistanceMeters,
    const std::vector< double > & smoothedAltitudeMeters,
    const std::vector< double > & localGradientPct,
    int startIdx,
    int endIdx,
    int climbOrdinal) [static]
```

Builds a complete [Climb](#) object from activity data.

Parameters

<i>rideData</i>	Activity recording data.
<i>cumulativeDistanceMeters</i>	Distance array.
<i>smoothedAltitudeMeters</i>	Elevation array (smoothed).
<i>localGradientPct</i>	Gradient array.
<i>startIdx</i>	Index of climb start.
<i>endIdx</i>	Index of climb end.
<i>climbOrdinal</i>	Ordinal position of climb in activity.

Returns

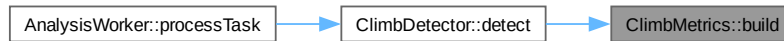
Fully populated [Climb](#) structure with all metrics.

Definition at line 221 of file [ClimbMetrics.cpp](#).

References [Climb::aerobicDecouplingPct](#), [RideRecord::altitude](#), [Climb::averageCadence](#), [Climb::averageGradient](#), [Climb::averageHeartRate](#), [Climb::averagePower](#), [Climb::averageSpeed](#), [RideRecord::cadence](#), [Climb::category](#), [Climb::difficultyScore](#), [Climb::durationSeconds](#), [RideRecord::elapsedSeconds](#), [Climb::elevationGainM](#), [Climb::endDistanceKm](#), [Climb::endElevationM](#), [Climb::endSeconds](#), [Climb::gradePercentile](#), [RideRecord::hasAltitude](#), [RideRecord::hasCadence](#), [RideRecord::hasHeartRate](#), [RideRecord::hasPower](#), [RideRecord::hasSpeed](#), [RideRecord::heartRate](#), [Climb::hrDriftPct](#), [Climb::lengthKm](#), [Climb::maximumGradient](#), [Climb::name](#), [Climb::normalizedPower](#), [RideRecord::power](#), [Climb::powerFadePct](#), [Climb::powerFirstHalf](#), [Climb::powerSecondHalf](#), [Climb::quarterMetrics](#), [RideData::records](#), [Climb::shapeClass](#), [RideRecord::speed](#), [Climb::startDistanceKm](#), [Climb::startElevationM](#), [Climb::startSeconds](#), and [Climb::vam](#).

Referenced by [ClimbDetector::detect\(\)](#).

Here is the caller graph for this function:



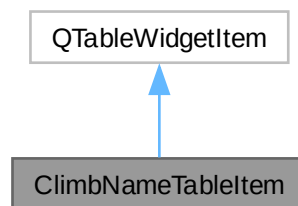
The documentation for this class was generated from the following files:

- src/analysis/ClimbMetrics.h
- src/analysis/ClimbMetrics.cpp

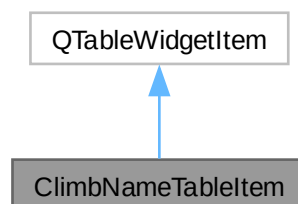
7.24 ClimbNameTableItem Class Reference

Table item that sorts climb names alphabetically.

Inheritance diagram for ClimbNameTableItem:



Collaboration diagram for ClimbNameTableItem:



Public Member Functions

- [ClimbNameTableItem](#) (const QString &text)
Constructs a climb name table item.
- bool [operator<](#) (const QTableWidgetItem &other) const override
Compares items for sorting (alphabetical).

Static Private Member Functions

- static int [trailingNumber](#) (const QString &text, bool *ok)

7.24.1 Detailed Description

Table item that sorts climb names alphabetically.

Definition at line 265 of file [MainWindow.cpp](#).

7.24.2 Constructor & Destructor Documentation

7.24.2.1 ClimbNameTableItem()

```
ClimbNameTableItem::ClimbNameTableItem (
    const QString & text) [inline], [explicit]
```

Constructs a climb name table item.

Parameters

<i>text</i>	Climb name text.
-------------	----------------------------------

Definition at line 272 of file [MainWindow.cpp](#).

7.24.3 Member Function Documentation

7.24.3.1 operator<()

```
bool ClimbNameTableItem::operator< (
    const QTableWidgetItem & other) const [inline], [override]
```

Compares items for sorting (alphabetical).

Parameters

<i>other</i>	Item to compare with.
--------------	-----------------------

Returns

True if this should sort before other.

Definition at line 281 of file [MainWindow.cpp](#).

7.24.3.2 trailingNumber()

```
int ClimbNameTableItem::trailingNumber (
    const QString & text,
    bool * ok) [inline], [static], [private]
```

Definition at line 295 of file [MainWindow.cpp](#).

The documentation for this class was generated from the following file:

- src/gui/MainWindow.cpp

7.25 ClimbQuarterMetrics Struct Reference

Metrics for a quarter of a climb segment.

```
#include <ClimbMetrics.h>
```

Public Attributes

- double `averagePower` = 0.0
Average power during quarter.
- double `averageHeartRate` = 0.0
Average heart rate during quarter.
- double `averageCadence` = 0.0
Average cadence during quarter.

7.25.1 Detailed Description

Metrics for a quarter of a climb segment.
Definition at line 27 of file [ClimbMetrics.h](#).

7.25.2 Member Data Documentation

7.25.2.1 averageCadence

`double ClimbQuarterMetrics::averageCadence = 0.0`
Average cadence during quarter.
Definition at line 36 of file [ClimbMetrics.h](#).

7.25.2.2 averageHeartRate

`double ClimbQuarterMetrics::averageHeartRate = 0.0`
Average heart rate during quarter.
Definition at line 33 of file [ClimbMetrics.h](#).

7.25.2.3 averagePower

`double ClimbQuarterMetrics::averagePower = 0.0`
Average power during quarter.
Definition at line 30 of file [ClimbMetrics.h](#).
The documentation for this struct was generated from the following file:

- `src/analysis/ClimbMetrics.h`

7.26 ClimbRecord Struct Reference

Database representation of a stored climb.
`#include <ClimbRepository.h>`

Public Attributes

- int `id` = -1
Unique climb record ID.
- int `activityId` = -1
Associated activity ID.
- QString `uuid`
Stable UUID, generated once on insert.
- double `startSeconds` = 0.0
Climb start time in seconds from activity start.
- double `endSeconds` = 0.0
Climb end time in seconds from activity start.
- double `originalStartSeconds` = -1.0
Start bounds at first auto-detection (-1 if not set).

- double `originalEndSeconds` = -1.0
End bounds at first auto-detection (-1 if not set).
- QString `name`
Climb name.
- double `elevationGainM` = 0.0
Elevation gain in meters.
- double `lengthKm` = 0.0
Climb length in kilometers.
- double `averageGradient` = 0.0
Average gradient percentage.
- double `avgPower` = 0.0
Average power in watts (cached).
- double `np` = 0.0
Normalized power in watts (cached).
- double `avgHr` = 0.0
Average heart rate in bpm (cached).
- double `avgCadence` = 0.0
Average cadence in rpm (cached).
- double `vam` = 0.0
VAM (Vertical Ascent Meter) in m/h (cached).
- QString `source` = QStringLiteral("auto")
Source of climb (auto, manual, edited).
- bool `locked` = false
Whether climb is locked from editing.
- bool `deleted` = false
Soft-delete flag.
- int `algorithmVersion` = 1
Algorithm version used for detection.
- QString `notes`
User notes about climb.
- bool `favorite` = false
Whether climb is marked as favorite.
- int `rating` = 0
User rating (0-5).

7.26.1 Detailed Description

Database representation of a stored climb.

Contains all climb metrics and lifecycle information persisted in the database.

Definition at line 16 of file [ClimbRepository.h](#).

7.26.2 Member Data Documentation

7.26.2.1 activityId

```
int ClimbRecord::activityId = -1
```

Associated activity ID.

Definition at line 22 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), and [AnalysisWorker::processTask\(\)](#).

7.26.2.2 algorithmVersion

```
int ClimbRecord::algorithmVersion = 1
```

Algorithm version used for detection.

Definition at line 76 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), and [AnalysisWorker::processTask\(\)](#).

7.26.2.3 averageGradient

```
double ClimbRecord::averageGradient = 0.0
```

Average gradient percentage.

Definition at line 49 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), [AnalysisWorker::processTask\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

7.26.2.4 avgCadence

```
double ClimbRecord::avgCadence = 0.0
```

Average cadence in rpm (cached).

Definition at line 61 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), [AnalysisWorker::processTask\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

7.26.2.5 avgHr

```
double ClimbRecord::avgHr = 0.0
```

Average heart rate in bpm (cached).

Definition at line 58 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), [AnalysisWorker::processTask\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

7.26.2.6 avgPower

```
double ClimbRecord::avgPower = 0.0
```

Average power in watts (cached).

Definition at line 52 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), [AnalysisWorker::processTask\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

7.26.2.7 deleted

```
bool ClimbRecord::deleted = false
```

Soft-delete flag.

Definition at line 73 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), [AnalysisWorker::processTask\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

7.26.2.8 elevationGainM

```
double ClimbRecord::elevationGainM = 0.0
```

Elevation gain in meters.

Definition at line 43 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), [AnalysisWorker::processTask\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

7.26.2.9 endSeconds

```
double ClimbRecord::endSeconds = 0.0
```

[Climb](#) end time in seconds from activity start.

Definition at line 31 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), [AnalysisWorker::processTask\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

7.26.2.10 favorite

```
bool ClimbRecord::favorite = false
```

Whether climb is marked as favorite.

Definition at line 82 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

7.26.2.11 id

```
int ClimbRecord::id = -1
```

Unique climb record ID.

Definition at line 19 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::updateClimb\(\)](#).

7.26.2.12 lengthKm

```
double ClimbRecord::lengthKm = 0.0
```

[Climb](#) length in kilometers.

Definition at line 46 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), [AnalysisWorker::processTask\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

7.26.2.13 locked

```
bool ClimbRecord::locked = false
```

Whether climb is locked from editing.

Definition at line 70 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), [AnalysisWorker::processTask\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

7.26.2.14 name

```
QString ClimbRecord::name
```

[Climb](#) name.

Definition at line 40 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), [AnalysisWorker::processTask\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

7.26.2.15 notes

```
QString ClimbRecord::notes
```

User notes about climb.

Definition at line 79 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

7.26.2.16 np

```
double ClimbRecord::np = 0.0
```

Normalized power in watts (cached).

Definition at line 55 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), [AnalysisWorker::processTask\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

7.26.2.17 originalEndSeconds

```
double ClimbRecord::originalEndSeconds = -1.0
```

End bounds at first auto-detection (-1 if not set).

Definition at line 37 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), and [AnalysisWorker::processTask\(\)](#).

7.26.2.18 originalStartSeconds

```
double ClimbRecord::originalStartSeconds = -1.0
```

Start bounds at first auto-detection (-1 if not set).

Definition at line 34 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), and [AnalysisWorker::processTask\(\)](#).

7.26.2.19 rating

```
int ClimbRecord::rating = 0
```

User rating (0-5).

Definition at line 85 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

7.26.2.20 source

```
QString ClimbRecord::source = QStringLiteral("auto")
```

Source of climb (auto, manual, edited).

Definition at line 67 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), [AnalysisWorker::processTask\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

7.26.2.21 startSeconds

```
double ClimbRecord::startSeconds = 0.0
```

[Climb](#) start time in seconds from activity start.

Definition at line 28 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), [AnalysisWorker::processTask\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

7.26.2.22 uuid

```
QString ClimbRecord::uuid
```

Stable UUID, generated once on insert.

Definition at line 25 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), and [AnalysisWorker::processTask\(\)](#).

7.26.2.23 vam

```
double ClimbRecord::vam = 0.0
```

VAM (Vertical Ascent Meter) in m/h (cached).

Definition at line 64 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), [AnalysisWorker::processTask\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

The documentation for this struct was generated from the following file:

- `src/database/ClimbRepository.h`

7.27 ClimbRepository Class Reference

Repository for climb record persistence.

```
#include <ClimbRepository.h>
```

Public Member Functions

- [ClimbRepository](#) (QSqlDatabase &db)
Constructs repository with database connection.
- int [insertClimb](#) (const [ClimbRecord](#) &climb)
Inserts a new climb record into the database.
- bool [updateClimb](#) (const [ClimbRecord](#) &climb)
Updates an existing climb record.
- bool [softDeleteClimb](#) (int climbId)
Soft-deletes a climb (marks deleted=1, locked=1).
- bool [hardDeleteClimb](#) (int climbId)
Hard-deletes a climb row (permanent removal).
- bool [deleteClimbsForActivity](#) (int activityId)
Deletes all climbs for an activity (hard delete).

- [ClimbRecord](#) `getClimb` (int climbId)
Retrieves a single climb by ID.
- `QList< ClimbRecord > climbsForActivity` (int activityId)
Lists all non-deleted climbs for an activity.
- `QList< ClimbRecord > deletedClimbsForActivity` (int activityId)
Lists all deleted climbs for an activity.
- bool `hasClimbs` (int activityId)
Checks if non-deleted climbs exist for an activity.
- bool `hasClimbsEver` (int activityId)
Checks if analysis has been run for an activity.
- bool `hasLockedClimbs` (int activityId)
Checks if any locked non-deleted climbs exist.

Private Attributes

- `QSqlDatabase & m_db`

7.27.1 Detailed Description

Repository for climb record persistence.
Provides CRUD operations and queries for climbs in the database.
Definition at line 93 of file [ClimbRepository.h](#).

7.27.2 Constructor & Destructor Documentation

7.27.2.1 ClimbRepository()

```
ClimbRepository::ClimbRepository (
    QSqlDatabase & db) [explicit]
```

Constructs repository with database connection.

Parameters

<i>db</i>	Database connection.
-----------	----------------------

Definition at line 55 of file [ClimbRepository.cpp](#).

7.27.3 Member Function Documentation

7.27.3.1 climbsForActivity()

```
QList< ClimbRecord > ClimbRepository::climbsForActivity (
    int activityId)
```

Lists all non-deleted climbs for an activity.

Parameters

<i>activityId</i>	Activity ID.
-------------------	------------------------------

Returns

List of climb records.

Definition at line 187 of file [ClimbRepository.cpp](#).

7.27.3.2 deleteClimbsForActivity()

```
bool ClimbRepository::deleteClimbsForActivity (
    int activityId)
```

Deletes all climbs for an activity (hard delete).

Parameters

<i>activityId</i>	Activity ID.
-------------------	--------------

Returns

True if successful.

Definition at line 169 of file [ClimbRepository.cpp](#).

7.27.3.3 deletedClimbsForActivity()

```
QList< ClimbRecord > ClimbRepository::deletedClimbsForActivity (
    int activityId)
```

Lists all deleted climbs for an activity.

Parameters

<i>activityId</i>	Activity ID.
-------------------	--------------

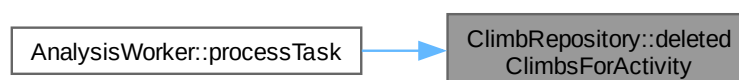
Returns

List of deleted climb records.

Definition at line 201 of file [ClimbRepository.cpp](#).

Referenced by [AnalysisWorker::processTask\(\)](#).

Here is the caller graph for this function:



7.27.3.4 getClimb()

```
ClimbRecord ClimbRepository::getClimb (
    int climbId)
```

Retrieves a single climb by ID.

Parameters

<i>climbId</i>	Climb ID.
----------------	-----------

Returns

[Climb](#) record.

Definition at line 177 of file [ClimbRepository.cpp](#).

7.27.3.5 hardDeleteClimb()

```
bool ClimbRepository::hardDeleteClimb (  
    int climbId)
```

Hard-deletes a climb row (permanent removal).

Parameters

<i>climbId</i>	Climb ID to delete.
----------------	-------------------------------------

Returns

True if successful.

Definition at line 161 of file [ClimbRepository.cpp](#).

7.27.3.6 hasClimbs()

```
bool ClimbRepository::hasClimbs (  
    int activityId)
```

Checks if non-deleted climbs exist for an activity.

Parameters

<i>activityId</i>	Activity ID.
-------------------	------------------------------

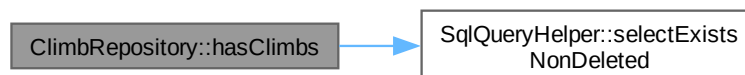
Returns

True if any non-deleted climbs found.

Definition at line 215 of file [ClimbRepository.cpp](#).

References [SqlQueryHelper::selectExistsNonDeleted\(\)](#).

Here is the call graph for this function:

**7.27.3.7 hasClimbsEver()**

```
bool ClimbRepository::hasClimbsEver (  
    int activityId)
```

Checks if analysis has been run for an activity.

Parameters

<i>activityId</i>	Activity ID.
-------------------	------------------------------

Returns

True if any climb records exist (deleted or not).

Definition at line 223 of file [ClimbRepository.cpp](#).

References [SqlQueryHelper::selectExistsEver\(\)](#).

Referenced by [AnalysisWorker::processTask\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:

**7.27.3.8 hasLockedClimbs()**

```
bool ClimbRepository::hasLockedClimbs (  
    int activityId)
```

Checks if any locked non-deleted climbs exist.

Parameters

<i>activityId</i>	Activity ID.
-------------------	------------------------------

Returns

True if locked climbs found.

Definition at line 231 of file [ClimbRepository.cpp](#).

References [SqlQueryHelper::selectExistsLocked\(\)](#).

Here is the call graph for this function:

**7.27.3.9 insertClimb()**

```
int ClimbRepository::insertClimb (
    const ClimbRecord & climb)
```

Inserts a new climb record into the database.

Parameters

<i>climb</i>	Climb record to insert.
--------------	---

Returns

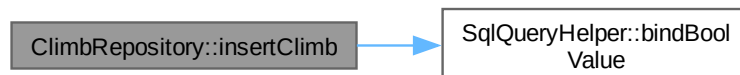
ID of inserted record, or -1 on error.

Definition at line 59 of file [ClimbRepository.cpp](#).

References [ClimbRecord::activityId](#), [ClimbRecord::algorithmVersion](#), [ClimbRecord::averageGradient](#), [ClimbRecord::avgCadence](#), [ClimbRecord::avgHr](#), [ClimbRecord::avgPower](#), [SqlQueryHelper::bindBoolValue\(\)](#), [ClimbRecord::deleted](#), [ClimbRecord::elevationGain](#), [ClimbRecord::endSeconds](#), [ClimbRecord::favorite](#), [ClimbRecord::lengthKm](#), [ClimbRecord::locked](#), [ClimbRecord::name](#), [ClimbRecord::notes](#), [ClimbRecord::np](#), [ClimbRecord::originalEndSeconds](#), [ClimbRecord::originalStartSeconds](#), [ClimbRecord::rating](#), [ClimbRecord::source](#), [ClimbRecord::startSeconds](#), [ClimbRecord::uuid](#), and [ClimbRecord::vam](#).

Referenced by [AnalysisWorker::processTask\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.27.3.10 softDeleteClimb()

```
bool ClimbRepository::softDeleteClimb (
    int climbId)
```

Soft-deletes a climb (marks deleted=1, locked=1).

Parameters

<i>climbId</i>	Climb ID to delete.
----------------	-------------------------------------

Returns

True if successful.

Definition at line 153 of file [ClimbRepository.cpp](#).

7.27.3.11 updateClimb()

```
bool ClimbRepository::updateClimb (
    const ClimbRecord & climb)
```

Updates an existing climb record.

Parameters

<i>climb</i>	Climb record with updated values.
--------------	---

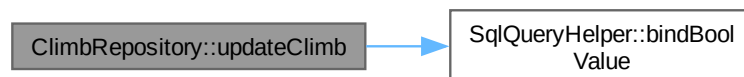
Returns

True if update succeeded.

Definition at line 107 of file [ClimbRepository.cpp](#).

References [ClimbRecord::averageGradient](#), [ClimbRecord::avgCadence](#), [ClimbRecord::avgHr](#), [ClimbRecord::avgPower](#), [SqlQueryHelper::bindBoolValue\(\)](#), [ClimbRecord::deleted](#), [ClimbRecord::elevationGainM](#), [ClimbRecord::endSeconds](#), [ClimbRecord::favorite](#), [ClimbRecord::id](#), [ClimbRecord::lengthKm](#), [ClimbRecord::locked](#), [ClimbRecord::name](#), [ClimbRecord::notes](#), [ClimbRecord::np](#), [ClimbRecord::rating](#), [ClimbRecord::source](#), [ClimbRecord::startSeconds](#), and [ClimbRecord::vam](#).

Here is the call graph for this function:

**7.27.4 Member Data Documentation****7.27.4.1 m_db**

`QSqlDatabase& ClimbRepository::m_db [private]`

Definition at line 180 of file [ClimbRepository.h](#).

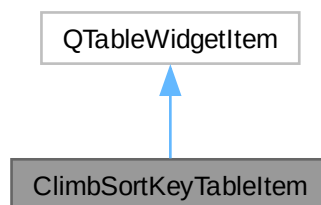
The documentation for this class was generated from the following files:

- `src/database/ClimbRepository.h`
- `src/database/ClimbRepository.cpp`

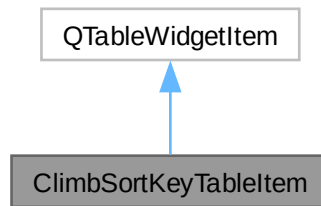
7.28 ClimbSortKeyTableItem Class Reference

Table item that sorts by numeric key, with text fallback.

Inheritance diagram for `ClimbSortKeyTableItem`:



Collaboration diagram for ClimbSortKeyTableItem:



Public Member Functions

- [ClimbSortKeyTableItem](#) (const QString &text, double sortKey)
Constructs a climb sort key table item.
- bool [operator<](#) (const QTableWidgetItem &other) const override
Compares items for sorting (numeric key first, then alphabetical).

Private Attributes

- double [m_sortKey](#) = std::numeric_limits<double>::quiet_NaN()

7.28.1 Detailed Description

Table item that sorts by numeric key, with text fallback.
Definition at line 323 of file [MainWindow.cpp](#).

7.28.2 Constructor & Destructor Documentation

7.28.2.1 ClimbSortKeyTableItem()

```

ClimbSortKeyTableItem::ClimbSortKeyTableItem (
    const QString & text,
    double sortKey) [inline], [explicit]

```

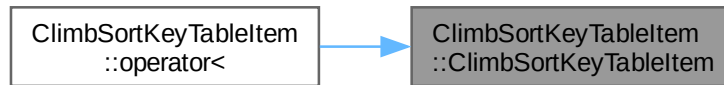
Constructs a climb sort key table item.

Parameters

<i>text</i>	Display text.
<i>sortKey</i>	Numeric sort key (NaN for text-based sorting).

Definition at line 331 of file [MainWindow.cpp](#).
Referenced by [operator<\(\)](#).

Here is the caller graph for this function:



7.28.3 Member Function Documentation

7.28.3.1 operator<()

```
bool ClimbSortKeyTableItem::operator< (
    const QTableWidgetItem & other) const [inline], [override]
```

Compares items for sorting (numeric key first, then alphabetical).

Parameters

<i>other</i>	Item to compare with.
--------------	-----------------------

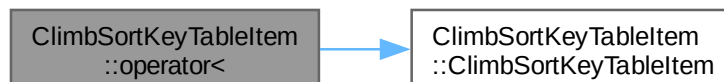
Returns

True if this should sort before other.

Definition at line 341 of file [MainWindow.cpp](#).

References [ClimbSortKeyTableItem\(\)](#).

Here is the call graph for this function:



7.28.4 Member Data Documentation

7.28.4.1 m_sortKey

```
double ClimbSortKeyTableItem::m_sortKey = std::numeric_limits<double>::quiet_NaN() [private]
```

Definition at line 360 of file [MainWindow.cpp](#).

The documentation for this class was generated from the following file:

- [src/gui/MainWindow.cpp](#)

7.29 ColorContext Struct Reference

Context parameters for determining zone colors based on athlete metrics.

```
#include <ZoneDefinition.h>
```

Public Attributes

- int `ftp` = 250
Functional threshold power in watts.
- int `heartRateMax` = 190
Maximum heart rate in bpm.
- int `heartRateRest` = 60
Resting heart rate in bpm.
- int `thresholdHeartRate` = 0
Heart rate threshold in bpm (for HR zones).
- double `altitudeMin` = 0.0
Minimum altitude in meters (for altitude coloring).
- double `altitudeMax` = 0.0
Maximum altitude in meters (for altitude coloring).
- bool `hasAltitudeRange` = false
Whether altitude range is valid for coloring.

7.29.1 Detailed Description

Context parameters for determining zone colors based on athlete metrics.
Definition at line 59 of file [ZoneDefinition.h](#).

7.29.2 Member Data Documentation

7.29.2.1 altitudeMax

```
double ColorContext::altitudeMax = 0.0
```

Maximum altitude in meters (for altitude coloring).
Definition at line 77 of file [ZoneDefinition.h](#).
Referenced by [ZoneCalculator::zonesForMetric\(\)](#).

7.29.2.2 altitudeMin

```
double ColorContext::altitudeMin = 0.0
```

Minimum altitude in meters (for altitude coloring).
Definition at line 74 of file [ZoneDefinition.h](#).
Referenced by [ZoneCalculator::zonesForMetric\(\)](#).

7.29.2.3 ftp

```
int ColorContext::ftp = 250
```

Functional threshold power in watts.
Definition at line 62 of file [ZoneDefinition.h](#).
Referenced by [ZoneCalculator::zonesForMetric\(\)](#).

7.29.2.4 hasAltitudeRange

```
bool ColorContext::hasAltitudeRange = false
```

Whether altitude range is valid for coloring.
Definition at line 80 of file [ZoneDefinition.h](#).

7.29.2.5 heartRateMax

```
int ColorContext::heartRateMax = 190
```

Maximum heart rate in bpm.
Definition at line 65 of file [ZoneDefinition.h](#).
Referenced by [ZoneCalculator::zonesForMetric\(\)](#).

7.29.2.6 heartRateRest

```
int ColorContext::heartRateRest = 60
```

Resting heart rate in bpm.

Definition at line 68 of file [ZoneDefinition.h](#).

Referenced by [ZoneCalculator::zonesForMetric\(\)](#).

7.29.2.7 thresholdHeartRate

```
int ColorContext::thresholdHeartRate = 0
```

Heart rate threshold in bpm (for HR zones).

Definition at line 71 of file [ZoneDefinition.h](#).

Referenced by [ZoneCalculator::zonesForMetric\(\)](#).

The documentation for this struct was generated from the following file:

- [src/core/zones/ZoneDefinition.h](#)

7.30 ColorProvider Class Reference

Provides color mapping for athletic metrics.

```
#include <ColorProvider.h>
```

Static Public Member Functions

- static QColor [colorForMetric](#) (ColorMetric metric, double value, const [ColorContext](#) &context)
- static QColor [colorForPower](#) (double watts, int ftp)
- static QColor [colorForHeartRate](#) (double bpm, int heartRateMax, int heartRateRest=60, int thresholdHeartRate=0)
- static QColor [colorForCadence](#) (double cadence)
- static QColor [colorForSpeed](#) (double speedKmh)
- static QColor [colorForAltitude](#) (double altitude, double minAltitude, double maxAltitude)
- static QColor [colorForValue](#) (double value, const std::vector< [Zone](#) > &zones, const QColor &fallback=QColor("#94a3b8"))

7.30.1 Detailed Description

Provides color mapping for athletic metrics.

Generates colors for power, heart rate, cadence, speed, and altitude based on zones.

Definition at line 17 of file [ColorProvider.h](#).

7.30.2 Member Function Documentation

7.30.2.1 colorForAltitude()

```
QColor ColorProvider::colorForAltitude (
    double altitude,
    double minAltitude,
    double maxAltitude) [static]
```

Definition at line 75 of file [ColorProvider.cpp](#).

7.30.2.2 colorForCadence()

```
QColor ColorProvider::colorForCadence (
    double cadence) [static]
```

Definition at line 65 of file [ColorProvider.cpp](#).

7.30.2.3 colorForHeartRate()

```
QColor ColorProvider::colorForHeartRate (
    double bpm,
    int heartRateMax,
    int heartRateRest = 60,
    int thresholdHeartRate = 0) [static]
```

Definition at line 55 of file [ColorProvider.cpp](#).

7.30.2.4 colorForMetric()

```
QColor ColorProvider::colorForMetric (
    ColorMetric metric,
    double value,
    const QColorContext & context) [static]
```

Definition at line 23 of file [ColorProvider.cpp](#).

7.30.2.5 colorForPower()

```
QColor ColorProvider::colorForPower (
    double watts,
    int ftp) [static]
```

Definition at line 50 of file [ColorProvider.cpp](#).

7.30.2.6 colorForSpeed()

```
QColor ColorProvider::colorForSpeed (
    double speedKmh) [static]
```

Definition at line 70 of file [ColorProvider.cpp](#).

7.30.2.7 colorForValue()

```
QColor ColorProvider::colorForValue (
    double value,
    const std::vector< Zone > & zones,
    const QColor & fallback = QColor("#94a3b8")) [static]
```

Definition at line 90 of file [ColorProvider.cpp](#).

The documentation for this class was generated from the following files:

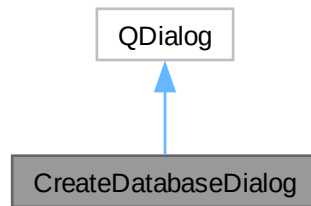
- [src/core/zones/ColorProvider.h](#)
- [src/core/zones/ColorProvider.cpp](#)

7.31 CreateDatabaseDialog Class Reference

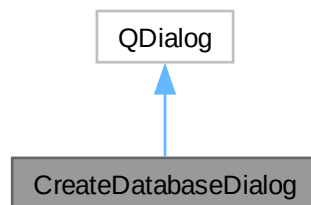
Dialog for creating a new database.

```
#include <CreateDatabaseDialog.h>
```

Inheritance diagram for CreateDatabaseDialog:



Collaboration diagram for CreateDatabaseDialog:



Public Member Functions

- `CreateDatabaseDialog` (const QString &defaultDirectory, QWidget *parent=nullptr)
- QString `databasePath` () const
- bool `shouldCreateDefaultAthlete` () const
- QString `defaultAthleteFirstName` () const
- QString `defaultAthleteLastName` () const

Private Slots

- void `browseLocation` ()
- void `updateAthleteFields` ()
- void `accept` () override

Private Member Functions

- QString `normalizedDatabaseName` () const

Private Attributes

- QLineEdit * `m_nameEdit` = nullptr
- QLineEdit * `m_locationEdit` = nullptr
- QCheckBox * `m_createAthleteCheck` = nullptr
- QLineEdit * `m_firstNameEdit` = nullptr

- `QLineEdit * m_lastNameEdit = nullptr`
- `QPushButton * m_browseButton = nullptr`
- `QDialogButtonBox * m_buttonBox = nullptr`

7.31.1 Detailed Description

Dialog for creating a new database.

Prompts for database name, location, and optionally creates a default athlete.

Definition at line 18 of file [CreateDatabaseDialog.h](#).

7.31.2 Constructor & Destructor Documentation

7.31.2.1 CreateDatabaseDialog()

```
CreateDatabaseDialog::CreateDatabaseDialog (  
    const QString & defaultDirectory,  
    QWidget * parent = nullptr) [explicit]
```

Definition at line 19 of file [CreateDatabaseDialog.cpp](#).

7.31.3 Member Function Documentation

7.31.3.1 accept

```
void CreateDatabaseDialog::accept () [override], [private], [slot]
```

Definition at line 110 of file [CreateDatabaseDialog.cpp](#).

7.31.3.2 browseLocation

```
void CreateDatabaseDialog::browseLocation () [private], [slot]
```

Definition at line 93 of file [CreateDatabaseDialog.cpp](#).

7.31.3.3 databasePath()

```
QString CreateDatabaseDialog::databasePath () const
```

Definition at line 69 of file [CreateDatabaseDialog.cpp](#).

7.31.3.4 defaultAthleteFirstName()

```
QString CreateDatabaseDialog::defaultAthleteFirstName () const
```

Definition at line 83 of file [CreateDatabaseDialog.cpp](#).

7.31.3.5 defaultAthleteLastName()

```
QString CreateDatabaseDialog::defaultAthleteLastName () const
```

Definition at line 88 of file [CreateDatabaseDialog.cpp](#).

7.31.3.6 normalizedDatabaseName()

```
QString CreateDatabaseDialog::normalizedDatabaseName () const [private]
```

Definition at line 145 of file [CreateDatabaseDialog.cpp](#).

7.31.3.7 shouldCreateDefaultAthlete()

```
bool CreateDatabaseDialog::shouldCreateDefaultAthlete () const
```

Definition at line 78 of file [CreateDatabaseDialog.cpp](#).

7.31.3.8 updateAthleteFields

```
void CreateDatabaseDialog::updateAthleteFields () [private], [slot]
```

Definition at line 101 of file [CreateDatabaseDialog.cpp](#).

7.31.4 Member Data Documentation

7.31.4.1 m_browseButton

`QPushButton* CreateDatabaseDialog::m_browseButton = nullptr [private]`

Definition at line 46 of file [CreateDatabaseDialog.h](#).

7.31.4.2 m_buttonBox

`QDialogButtonBox* CreateDatabaseDialog::m_buttonBox = nullptr [private]`

Definition at line 47 of file [CreateDatabaseDialog.h](#).

7.31.4.3 m_createAthleteCheck

`QCheckBox* CreateDatabaseDialog::m_createAthleteCheck = nullptr [private]`

Definition at line 43 of file [CreateDatabaseDialog.h](#).

7.31.4.4 m_firstNameEdit

`QLineEdit* CreateDatabaseDialog::m_firstNameEdit = nullptr [private]`

Definition at line 44 of file [CreateDatabaseDialog.h](#).

7.31.4.5 m_lastNameEdit

`QLineEdit* CreateDatabaseDialog::m_lastNameEdit = nullptr [private]`

Definition at line 45 of file [CreateDatabaseDialog.h](#).

7.31.4.6 m_locationEdit

`QLineEdit* CreateDatabaseDialog::m_locationEdit = nullptr [private]`

Definition at line 42 of file [CreateDatabaseDialog.h](#).

7.31.4.7 m_nameEdit

`QLineEdit* CreateDatabaseDialog::m_nameEdit = nullptr [private]`

Definition at line 41 of file [CreateDatabaseDialog.h](#).

The documentation for this class was generated from the following files:

- [src/gui/CreateDatabaseDialog.h](#)
- [src/gui/CreateDatabaseDialog.cpp](#)

7.32 DailyLoadPoint Struct Reference

Daily training stress score value.

```
#include <TrainingLoad.h>
```

Public Attributes

- double `tss` = 0.0

Training stress score for day.

7.32.1 Detailed Description

Daily training stress score value.

Definition at line 13 of file [TrainingLoad.h](#).

7.32.2 Member Data Documentation

7.32.2.1 tss

```
double DailyLoadPoint::tss = 0.0
```

Training stress score for day.

Definition at line 16 of file [TrainingLoad.h](#).

The documentation for this struct was generated from the following file:

- [src/analysis/TrainingLoad.h](#)

7.33 DatabaseConnectionGuard Class Reference

RAII guard for temporary SQLite database connections.

```
#include <DatabaseConnectionGuard.h>
```

Public Member Functions

- [DatabaseConnectionGuard](#) (const QString &connectionName)
Constructs guard with connection name.
- [DatabaseConnectionGuard](#) (const [DatabaseConnectionGuard](#) &)=delete
- [DatabaseConnectionGuard](#) & **operator=** (const [DatabaseConnectionGuard](#) &)=delete
- QSqlDatabase [database](#) () const
Gets the database connection.

Private Attributes

- QString [m_connectionName](#)

7.33.1 Detailed Description

RAII guard for temporary SQLite database connections.

Ensures connections are properly closed and removed from Qt's connection registry, even if exceptions occur during processing.

Definition at line 15 of file [DatabaseConnectionGuard.h](#).

7.33.2 Constructor & Destructor Documentation

7.33.2.1 DatabaseConnectionGuard()

```
DatabaseConnectionGuard::DatabaseConnectionGuard (
    const QString & connectionName) [explicit]
```

Constructs guard with connection name.

Parameters

<i>connectionName</i>	Unique name for this connection.
-----------------------	----------------------------------

Definition at line 6 of file [DatabaseConnectionGuard.cpp](#).

7.33.2.2 ~DatabaseConnectionGuard()

```
DatabaseConnectionGuard::~DatabaseConnectionGuard ()
```

Definition at line 16 of file [DatabaseConnectionGuard.cpp](#).

7.33.3 Member Function Documentation

7.33.3.1 database()

`QSqlDatabase DatabaseConnectionGuard::database () const`

Gets the database connection.

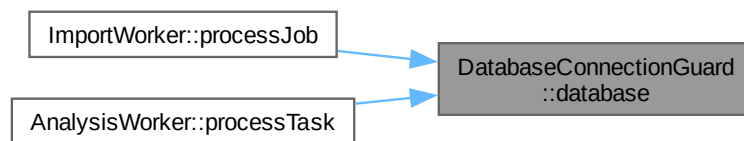
Returns

QSqlDatabase for this guard.

Definition at line 33 of file [DatabaseConnectionGuard.cpp](#).

Referenced by [ImportWorker::processJob\(\)](#), and [AnalysisWorker::processTask\(\)](#).

Here is the caller graph for this function:



7.33.4 Member Data Documentation

7.33.4.1 m_connectionName

`QString DatabaseConnectionGuard::m_connectionName [private]`

Definition at line 36 of file [DatabaseConnectionGuard.h](#).

The documentation for this class was generated from the following files:

- `src/database/DatabaseConnectionGuard.h`
- `src/database/DatabaseConnectionGuard.cpp`

7.34 DatabaseManager Class Reference

Manages a single database connection and schema for FitlyzerC.

```
#include <DatabaseManager.h>
```

Public Member Functions

- bool [open](#) (const QString &path, QString *errorOut=nullptr)
Opens an existing database file.
- bool [create](#) (const QString &path, QString *errorOut=nullptr)
Creates a new database file at the specified path.
- void [close](#) ()
Closes the database connection.
- bool [isOpen](#) () const
Checks if database connection is open.
- QSqlDatabase [database](#) () const
Gets the underlying SQL database connection.
- QString [path](#) () const
Gets the database file path.

Private Member Functions

- bool [applySchema](#) (QString *errorOut)
- bool [upgradeSchema](#) (int fromVersion, QString *errorOut)
- void [cleanupFailedOpen](#) ()

Private Attributes

- QString [m_path](#)
- QString [m_connectionName](#)
- QSqlDatabase [m_db](#)

7.34.1 Detailed Description

Manages a single database connection and schema for FitlyzerC.

Handles opening, creating, and closing SQLite database connections, applying schema initialization and upgrades as needed.

Definition at line 20 of file [DatabaseManager.h](#).

7.34.2 Constructor & Destructor Documentation

7.34.2.1 ~DatabaseManager()

```
DatabaseManager::~DatabaseManager ()
```

Definition at line 14 of file [DatabaseManager.cpp](#).

7.34.3 Member Function Documentation

7.34.3.1 applySchema()

```
bool DatabaseManager::applySchema (
    QString * errorOut) [private]
```

Definition at line 100 of file [DatabaseManager.cpp](#).

7.34.3.2 cleanupFailedOpen()

```
void DatabaseManager::cleanupFailedOpen () [private]
```

Definition at line 411 of file [DatabaseManager.cpp](#).

7.34.3.3 close()

```
void DatabaseManager::close ()
```

Closes the database connection.

Definition at line 431 of file [DatabaseManager.cpp](#).

Referenced by [open\(\)](#).

Here is the caller graph for this function:



7.34.3.4 create()

```
bool DatabaseManager::create (
    const QString & path,
    QString * errorOut = nullptr)
```

Creates a new database file at the specified path.

Parameters

<i>path</i>	Path for new database file.
<i>errorOut</i>	Optional error message output.

Returns

True if successfully created, false otherwise.

Definition at line 425 of file [DatabaseManager.cpp](#).

References [open\(\)](#), and [path\(\)](#).

Here is the call graph for this function:



7.34.3.5 database()

```
QSqlDatabase DatabaseManager::database () const
```

Gets the underlying SQL database connection.

Returns

`QSqlDatabase` handle (valid only when [isOpen\(\)](#) returns true).

Definition at line 451 of file [DatabaseManager.cpp](#).

7.34.3.6 isOpen()

```
bool DatabaseManager::isOpen () const
```

Checks if database connection is open.

Returns

True if connected to a database.

Definition at line 446 of file [DatabaseManager.cpp](#).

7.34.3.7 open()

```
bool DatabaseManager::open (
    const QString & path,
    QString * errorOut = nullptr)
```

Opens an existing database file.

Parameters

<i>path</i>	Path to database file.
<i>errorOut</i>	Optional error message output.

Returns

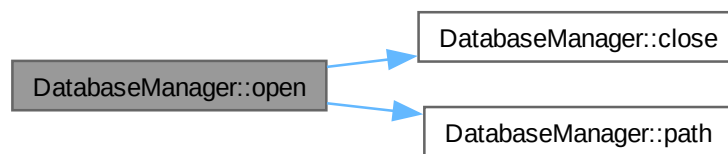
True if successfully opened, false otherwise.

Definition at line 363 of file [DatabaseManager.cpp](#).

References [close\(\)](#), and [path\(\)](#).

Referenced by [create\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:

**7.34.3.8 path()**

```
QString DatabaseManager::path () const [inline]
```

Gets the database file path.

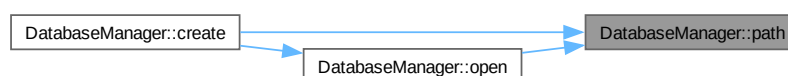
Returns

Path to the database file.

Definition at line 63 of file [DatabaseManager.h](#).

Referenced by [create\(\)](#), and [open\(\)](#).

Here is the caller graph for this function:



7.34.3.9 upgradeSchema()

```
bool DatabaseManager::upgradeSchema (
    int fromVersion,
    QString * errorOut) [private]
```

Definition at line 133 of file [DatabaseManager.cpp](#).

7.34.4 Member Data Documentation

7.34.4.1 m_connectionName

```
QString DatabaseManager::m_connectionName [private]
```

Definition at line 71 of file [DatabaseManager.h](#).

7.34.4.2 m_db

```
QSqlDatabase DatabaseManager::m_db [private]
```

Definition at line 72 of file [DatabaseManager.h](#).

7.34.4.3 m_path

```
QString DatabaseManager::m_path [private]
```

Definition at line 70 of file [DatabaseManager.h](#).

The documentation for this class was generated from the following files:

- src/database/DatabaseManager.h
- src/database/DatabaseManager.cpp

7.35 DateFormatter Class Reference

Provides date/time formatting consistent with user settings.

```
#include <DateFormatter.h>
```

Static Public Member Functions

- static QString [formatDate](#) (const QDate &date)
Formats a date according to current settings.
- static QString [formatDateTime](#) (const QDateTime &dateTime)
Formats a date/time according to current settings.
- static QString [tolsoDate](#) (const QDate &date)
Formats a date as ISO 8601.
- static QString [tolsoDateTime](#) (const QDateTime &dateTime)
Formats a date/time as ISO 8601.
- static QDate [parseDate](#) (const QString &text)
Parses a date string.
- static QString [qtDatePattern](#) ()
Gets the Qt date format pattern.

7.35.1 Detailed Description

Provides date/time formatting consistent with user settings.

Definition at line 13 of file [DateFormatter.h](#).

7.35.2 Member Function Documentation

7.35.2.1 `formatDate()`

```
QString DateFormatter::formatDate (
    const QDate & date) [static]
```

Formats a date according to current settings.

Parameters

<i>date</i>	Date to format.
-------------	-----------------

Returns

Formatted date string.

Definition at line 30 of file [DateFormatter.cpp](#).

References [AppSettings::instance\(\)](#).

Referenced by [formatDateTime\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.35.2.2 `formatDateTime()`

```
QString DateFormatter::formatDateTime (
    const QDateTime & dateTime) [static]
```

Formats a date/time according to current settings.

Parameters

<i>dateTime</i>	Date/time to format.
-----------------	----------------------

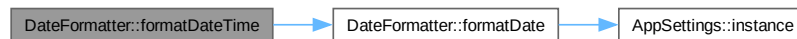
Returns

Formatted date/time string.

Definition at line 38 of file [DateFormatter.cpp](#).

References [formatDate\(\)](#).

Here is the call graph for this function:

**7.35.2.3 parseDate()**

```
QDate DateFormatter::parseDate (
    const QString & text) [static]
```

Parses a date string.

Parameters

<i>text</i>	Date string to parse.
-------------	-----------------------

Returns

Parsed QDate.

Definition at line 57 of file [DateFormatter.cpp](#).

References [AppSettings::instance\(\)](#).

Here is the call graph for this function:

**7.35.2.4 qtDatePattern()**

```
QString DateFormatter::qtDatePattern () [static]
```

Gets the Qt date format pattern.

Returns

Format pattern string.

Definition at line 91 of file [DateFormatter.cpp](#).

References [AppSettings::instance\(\)](#).

Referenced by [AthleteDialog::AthleteDialog\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.35.2.5 toIsoDate()

```
QString DateFormatter::toIsoDate (
    const QDate & date) [static]
```

Formats a date as ISO 8601.

Parameters

<i>date</i>	Date to format.
-------------	-----------------

Returns

ISO date string (YYYY-MM-DD).

Definition at line 47 of file [DateFormatter.cpp](#).

7.35.2.6 toIsoDateTime()

```
QString DateFormatter::toIsoDateTime (
    const QDateTime & dateTime) [static]
```

Formats a date/time as ISO 8601.

Parameters

<i>dateTime</i>	Date/time to format.
-----------------	----------------------

Returns

ISO date/time string.

Definition at line 52 of file [DateFormatter.cpp](#).

The documentation for this class was generated from the following files:

- [src/core/settings/DateFormatter.h](#)
- [src/core/settings/DateFormatter.cpp](#)

7.36 DuplicateActivityInfo Struct Reference

Public Attributes

- int [existingActivityId](#) = -1
- QString [existingStartUtc](#)
- double [existingDurationSeconds](#) = 0.0
- QString [newStartUtc](#)
- double [newDurationSeconds](#) = 0.0

7.36.1 Detailed Description

Definition at line 37 of file [WorkoutController.h](#).

7.36.2 Member Data Documentation

7.36.2.1 existingActivityId

```
int DuplicateActivityInfo::existingActivityId = -1
```

Definition at line 40 of file [WorkoutController.h](#).

7.36.2.2 existingDurationSeconds

```
double DuplicateActivityInfo::existingDurationSeconds = 0.0
```

Definition at line 46 of file [WorkoutController.h](#).

7.36.2.3 existingStartUtc

```
QString DuplicateActivityInfo::existingStartUtc
```

Definition at line 43 of file [WorkoutController.h](#).

7.36.2.4 newDurationSeconds

```
double DuplicateActivityInfo::newDurationSeconds = 0.0
```

Definition at line 52 of file [WorkoutController.h](#).

7.36.2.5 newStartUtc

```
QString DuplicateActivityInfo::newStartUtc
```

Definition at line 49 of file [WorkoutController.h](#).

The documentation for this struct was generated from the following file:

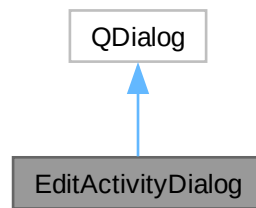
- [src/gui/WorkoutController.h](#)

7.37 EditActivityDialog Class Reference

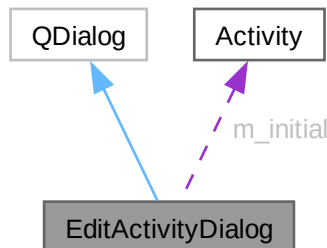
Dialog for editing activity metadata.

```
#include <EditActivityDialog.h>
```

Inheritance diagram for EditActivityDialog:



Collaboration diagram for EditActivityDialog:



Public Member Functions

- `EditActivityDialog` (const [Activity](#) &activity, QWidget *parent=nullptr)
- `Activity updatedActivity ()` const

Private Attributes

- [Activity](#) `m_initial`
- `QDoubleSpinBox * m_temperatureSpin = nullptr`
- `QLineEdit * m_weatherEdit = nullptr`
- `QLineEdit * m_windEdit = nullptr`
- `QSpinBox * m_rpeSpin = nullptr`
- `QSpinBox * m_fatigueSpin = nullptr`
- `QDoubleSpinBox * m_sleepSpin = nullptr`
- `QDoubleSpinBox * m_weightSpin = nullptr`
- `QLineEdit * m_bikeEdit = nullptr`
- `QLineEdit * m_equipmentEdit = nullptr`
- `QTextEdit * m_notesEdit = nullptr`

7.37.1 Detailed Description

Dialog for editing activity metadata.

Allows editing of weather, temperature, RPE, fatigue, sleep, weight, bike, equipment, and notes.

Definition at line 21 of file [EditActivityDialog.h](#).

7.37.2 Constructor & Destructor Documentation

7.37.2.1 EditActivityDialog()

```
EditActivityDialog::EditActivityDialog (  
    const Activity & activity,  
    QWidget * parent = nullptr) [explicit]
```

Definition at line 14 of file [EditActivityDialog.cpp](#).

7.37.3 Member Function Documentation

7.37.3.1 updatedActivity()

```
Activity EditActivityDialog::updatedActivity () const
```

Definition at line 79 of file [EditActivityDialog.cpp](#).

7.37.4 Member Data Documentation

7.37.4.1 m_bikeEdit

```
QLineEdit* EditActivityDialog::m_bikeEdit = nullptr [private]
```

Definition at line 40 of file [EditActivityDialog.h](#).

7.37.4.2 m_equipmentEdit

```
QLineEdit* EditActivityDialog::m_equipmentEdit = nullptr [private]
```

Definition at line 41 of file [EditActivityDialog.h](#).

7.37.4.3 m_fatigueSpin

```
QSpinBox* EditActivityDialog::m_fatigueSpin = nullptr [private]
```

Definition at line 37 of file [EditActivityDialog.h](#).

7.37.4.4 m_initial

```
Activity EditActivityDialog::m_initial [private]
```

Definition at line 31 of file [EditActivityDialog.h](#).

7.37.4.5 m_notesEdit

```
QTextEdit* EditActivityDialog::m_notesEdit = nullptr [private]
```

Definition at line 42 of file [EditActivityDialog.h](#).

7.37.4.6 m_rpeSpin

```
QSpinBox* EditActivityDialog::m_rpeSpin = nullptr [private]
```

Definition at line 36 of file [EditActivityDialog.h](#).

7.37.4.7 m_sleepSpin

```
QDoubleSpinBox* EditActivityDialog::m_sleepSpin = nullptr [private]
```

Definition at line 38 of file [EditActivityDialog.h](#).

7.37.4.8 m_temperatureSpin

```
QDoubleSpinBox* EditActivityDialog::m_temperatureSpin = nullptr [private]
```

Definition at line 33 of file [EditActivityDialog.h](#).

7.37.4.9 m_weatherEdit

```
QLineEdit* EditActivityDialog::m_weatherEdit = nullptr [private]
```

Definition at line 34 of file [EditActivityDialog.h](#).

7.37.4.10 m_weightSpin

`QDoubleSpinBox* EditActivityDialog::m_weightSpin = nullptr [private]`
Definition at line 39 of file [EditActivityDialog.h](#).

7.37.4.11 m_windEdit

`QLineEdit* EditActivityDialog::m_windEdit = nullptr [private]`
Definition at line 35 of file [EditActivityDialog.h](#).

The documentation for this class was generated from the following files:

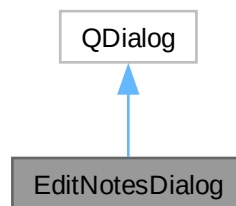
- `src/gui/EditActivityDialog.h`
- `src/gui/EditActivityDialog.cpp`

7.38 EditNotesDialog Class Reference

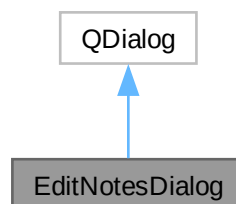
Dialog for editing activity notes and weather.

`#include <EditNotesDialog.h>`

Inheritance diagram for EditNotesDialog:



Collaboration diagram for EditNotesDialog:



Public Member Functions

- [EditNotesDialog](#) (const QString ¬es, const QString &weatherNotes, QWidget *parent=nullptr)
- QString [notes](#) () const
- QString [weatherNotes](#) () const

Private Attributes

- QTextEdit * [m_notes](#) = nullptr
- QLineEdit * [m_weather](#) = nullptr

7.38.1 Detailed Description

Dialog for editing activity notes and weather.

Definition at line 15 of file [EditNotesDialog.h](#).

7.38.2 Constructor & Destructor Documentation

7.38.2.1 EditNotesDialog()

```
EditNotesDialog::EditNotesDialog (  
    const QString & notes,  
    const QString & weatherNotes,  
    QWidget * parent = nullptr) [explicit]
```

Definition at line 13 of file [EditNotesDialog.cpp](#).

7.38.3 Member Function Documentation

7.38.3.1 notes()

```
QString EditNotesDialog::notes () const
```

Definition at line 42 of file [EditNotesDialog.cpp](#).

7.38.3.2 weatherNotes()

```
QString EditNotesDialog::weatherNotes () const
```

Definition at line 47 of file [EditNotesDialog.cpp](#).

7.38.4 Member Data Documentation

7.38.4.1 m_notes

```
QTextEdit* EditNotesDialog::m_notes = nullptr [private]
```

Definition at line 28 of file [EditNotesDialog.h](#).

7.38.4.2 m_weather

```
QLineEdit* EditNotesDialog::m_weather = nullptr [private]
```

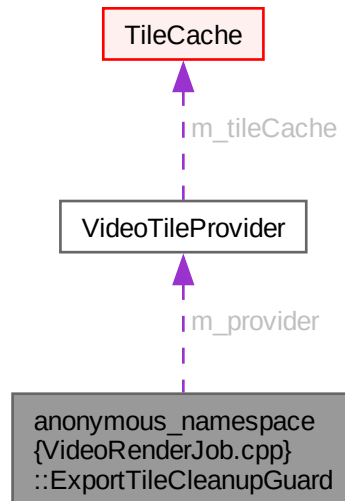
Definition at line 29 of file [EditNotesDialog.h](#).

The documentation for this class was generated from the following files:

- [src/gui/EditNotesDialog.h](#)
- [src/gui/EditNotesDialog.cpp](#)

7.39 anonymous_namespace{VideoRenderJob.cpp}::ExportTileCleanupGuard Class Reference

Collaboration diagram for anonymous_namespace{VideoRenderJob.cpp}::ExportTileCleanupGuard:



Public Member Functions

- [ExportTileCleanupGuard](#) ([VideoTileProvider](#) &provider)

Private Attributes

- [VideoTileProvider](#) & `m_provider`

7.39.1 Detailed Description

Definition at line 237 of file [VideoRenderJob.cpp](#).

7.39.2 Constructor & Destructor Documentation

7.39.2.1 ExportTileCleanupGuard()

```
anonymous_namespace{VideoRenderJob.cpp}::ExportTileCleanupGuard::ExportTileCleanupGuard (
    VideoTileProvider & provider) [inline], [explicit]
```

Definition at line 240 of file [VideoRenderJob.cpp](#).

7.39.2.2 ~ExportTileCleanupGuard()

```
anonymous_namespace{VideoRenderJob.cpp}::ExportTileCleanupGuard::~~ExportTileCleanupGuard ()
[inline]
```

Definition at line 244 of file [VideoRenderJob.cpp](#).

7.39.3 Member Data Documentation

7.39.3.1 m_provider

`VideoTileProvider& anonymous_namespace{VideoRenderJob.cpp}::ExportTileCleanupGuard::m_provider`
[private]

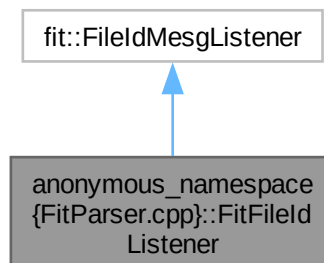
Definition at line 251 of file [VideoRenderJob.cpp](#).

The documentation for this class was generated from the following file:

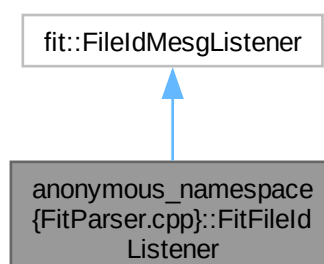
- `src/video/VideoRenderJob.cpp`

7.40 anonymous_namespace{FitParser.cpp}::FitFileIdListener Class Reference

Inheritance diagram for anonymous_namespace{FitParser.cpp}::FitFileIdListener:



Collaboration diagram for anonymous_namespace{FitParser.cpp}::FitFileIdListener:



Public Member Functions

- void [OnMesg](#) (fit::FileIdMesg &mesg) override
- FIT_DATE_TIME [timeCreated](#) () const

Private Attributes

- FIT_DATE_TIME [m_timeCreated](#) = FIT_DATE_TIME_INVALID

7.40.1 Detailed Description

Definition at line 47 of file [FitParser.cpp](#).

7.40.2 Member Function Documentation

7.40.2.1 OnMesg()

```
void anonymous_namespace{FitParser.cpp}::FitFileIdListener::OnMesg (
    fit::FileIdMesg & msg) [inline], [override]
```

Definition at line 50 of file [FitParser.cpp](#).

7.40.2.2 timeCreated()

```
FIT_DATE_TIME anonymous_namespace{FitParser.cpp}::FitFileIdListener::timeCreated () const
[inline]
```

Definition at line 56 of file [FitParser.cpp](#).

7.40.3 Member Data Documentation

7.40.3.1 m_timeCreated

```
FIT_DATE_TIME anonymous_namespace{FitParser.cpp}::FitFileIdListener::m_timeCreated = FIT_↵
DATE_TIME_INVALID [private]
```

Definition at line 59 of file [FitParser.cpp](#).

The documentation for this class was generated from the following file:

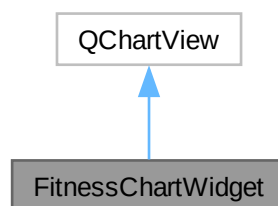
- [src/fit/FitParser.cpp](#)

7.41 FitnessChartWidget Class Reference

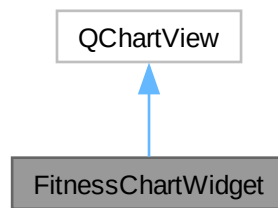
Displays fitness metrics timeline (CTL, ATL, TSB).

```
#include <FitnessChartWidget.h>
```

Inheritance diagram for FitnessChartWidget:



Collaboration diagram for FitnessChartWidget:



Public Member Functions

- [FitnessChartWidget](#) (`QWidget *parent=nullptr`)
Constructs fitness chart widget.
- void [setTimeline](#) (`const std::vector< FitnessMetricsPoint > &timeline`)
Sets fitness metrics timeline to display.
- void [clearChart](#) ()
Clears the chart.

Private Attributes

- `QLineSeries * m\_ctlSeries = nullptr`
- `QLineSeries * m\_atlSeries = nullptr`
- `QLineSeries * m\_tsbSeries = nullptr`
- `QValueAxis * m\_axisX = nullptr`
- `QValueAxis * m\_axisY = nullptr`

7.41.1 Detailed Description

Displays fitness metrics timeline (CTL, ATL, TSB).

Shows cumulative training load, acute training load, and training stress balance over a date range using a line chart.

Definition at line 21 of file [FitnessChartWidget.h](#).

7.41.2 Constructor & Destructor Documentation

7.41.2.1 FitnessChartWidget()

```

FitnessChartWidget::FitnessChartWidget (
    QWidget * parent = nullptr) [explicit]

```

Constructs fitness chart widget.

Parameters

<i>parent</i>	Parent widget.
---------------	----------------

Definition at line 13 of file [FitnessChartWidget.cpp](#).

7.41.3 Member Function Documentation

7.41.3.1 clearChart()

```
void FitnessChartWidget::clearChart ()
```

Clears the chart.

Definition at line 93 of file [FitnessChartWidget.cpp](#).

7.41.3.2 setTimeline()

```
void FitnessChartWidget::setTimeline (
    const std::vector< FitnessMetricsPoint > & timeline)
```

Sets fitness metrics timeline to display.

Parameters

<i>timeline</i>	Timeline of fitness metrics.
-----------------	------------------------------

Definition at line 56 of file [FitnessChartWidget.cpp](#).

7.41.4 Member Data Documentation

7.41.4.1 m_atlSeries

```
QLineSeries* FitnessChartWidget::m_atlSeries = nullptr [private]
```

Definition at line 45 of file [FitnessChartWidget.h](#).

7.41.4.2 m_axisX

```
QValueAxis* FitnessChartWidget::m_axisX = nullptr [private]
```

Definition at line 47 of file [FitnessChartWidget.h](#).

7.41.4.3 m_axisY

```
QValueAxis* FitnessChartWidget::m_axisY = nullptr [private]
```

Definition at line 48 of file [FitnessChartWidget.h](#).

7.41.4.4 m_ctlSeries

```
QLineSeries* FitnessChartWidget::m_ctlSeries = nullptr [private]
```

Definition at line 44 of file [FitnessChartWidget.h](#).

7.41.4.5 m_tsbSeries

```
QLineSeries* FitnessChartWidget::m_tsbSeries = nullptr [private]
```

Definition at line 46 of file [FitnessChartWidget.h](#).

The documentation for this class was generated from the following files:

- [src/charts/FitnessChartWidget.h](#)
- [src/charts/FitnessChartWidget.cpp](#)

7.42 FitnessMetricsPoint Struct Reference

Fitness metrics for a single day.

```
#include <TrainingLoad.h>
```


Public Attributes

- double `atl` = 0.0
Acute Training Load (7-day EWMA).
- double `ctl` = 0.0
Chronic Training Load (42-day EWMA).
- double `tsb` = 0.0
Training Stress Balance (ctl - atl).

7.42.1 Detailed Description

Fitness metrics for a single day.

Represents Acute Training Load (ATL), Chronic Training Load (CTL), and Training Stress Balance (TSB) calculated from Training Stress Score.

Definition at line 25 of file [TrainingLoad.h](#).

7.42.2 Member Data Documentation**7.42.2.1 atl**

```
double FitnessMetricsPoint::atl = 0.0
```

Acute Training Load (7-day EWMA).

Definition at line 28 of file [TrainingLoad.h](#).

Referenced by [TrainingLoad::fitnessTimeline\(\)](#).

7.42.2.2 ctl

```
double FitnessMetricsPoint::ctl = 0.0
```

Chronic Training Load (42-day EWMA).

Definition at line 31 of file [TrainingLoad.h](#).

Referenced by [TrainingLoad::fitnessTimeline\(\)](#).

7.42.2.3 tsb

```
double FitnessMetricsPoint::tsb = 0.0
```

Training Stress Balance (ctl - atl).

Definition at line 34 of file [TrainingLoad.h](#).

Referenced by [TrainingLoad::fitnessTimeline\(\)](#).

The documentation for this struct was generated from the following file:

- `src/analysis/TrainingLoad.h`

7.43 FitParser Class Reference

Parses FIT files and extracts activity recording data.

```
#include <FitParser.h>
```

Public Member Functions

- [RideData load](#) (const QString &fileName)
Loads activity data from a FIT file.

Static Private Member Functions

- static double **semicirclesToDegrees** (long value)

7.43.1 Detailed Description

Parses FIT files and extracts activity recording data.
Definition at line 13 of file [FitParser.h](#).

7.43.2 Member Function Documentation

7.43.2.1 load()

```
RideData FitParser::load (
    const QString & fileName)
```

Loads activity data from a FIT file.

Parameters

<i>fileName</i>	Path to .fit file.
-----------------	--------------------

Returns

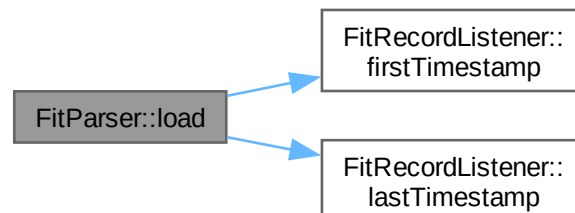
[RideData](#) with parsed activity records.

Definition at line 124 of file [FitParser.cpp](#).

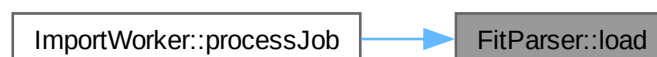
References [RideData::activityEndTimeUtc](#), [RideData::activityStartTimeUtc](#), [RideData::fileTimestampUtc](#), [FitRecordListener::firstTime](#), [FitRecordListener::lastTimestamp\(\)](#), and [RideData::records](#).

Referenced by [ImportWorker::processJob\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



The documentation for this class was generated from the following files:

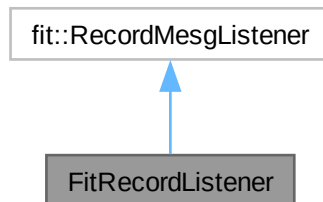
- `src/fit/FitParser.h`
- `src/fit/FitParser.cpp`

7.44 FitRecordListener Class Reference

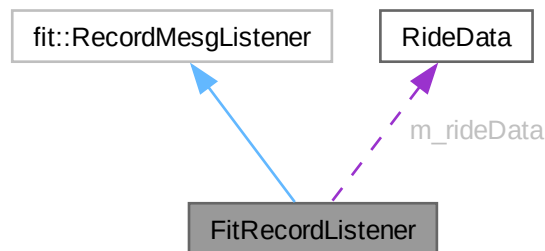
Listener for FIT record messages during parsing.

```
#include <FitRecordListener.h>
```

Inheritance diagram for FitRecordListener:



Collaboration diagram for FitRecordListener:



Public Member Functions

- `FitRecordListener` (`RideData` &`rideData`)
Constructs listener with reference to ride data accumulator.
- `void OnMesg` (`fit::RecordMesg` &`mesg`) override
Processes a single FIT record message.
- `FIT_DATE_TIME firstTimestamp` () const
Gets the first timestamp from records.
- `FIT_DATE_TIME lastTimestamp` () const
Gets the last timestamp from records.

Private Attributes

- `RideData` & `m_rideData`
- `FIT_DATE_TIME m_firstTimestamp` = `FIT_DATE_TIME_INVALID`
- `FIT_DATE_TIME m_lastTimestamp` = `FIT_DATE_TIME_INVALID`

7.44.1 Detailed Description

Listener for FIT record messages during parsing.

Processes individual FIT record messages and accumulates sample data into [RideData](#).

Definition at line 15 of file [FitRecordListener.h](#).

7.44.2 Constructor & Destructor Documentation

7.44.2.1 FitRecordListener()

```
FitRecordListener::FitRecordListener (
    RideData & rideData) [explicit]
```

Constructs listener with reference to ride data accumulator.

Parameters

<i>rideData</i>	Reference to RideData object to populate.
-----------------	---

Definition at line 8 of file [FitRecordListener.cpp](#).

7.44.3 Member Function Documentation

7.44.3.1 firstTimestamp()

```
FIT_DATE_TIME FitRecordListener::firstTimestamp () const [inline]
```

Gets the first timestamp from records.

Returns

First timestamp value.

Definition at line 37 of file [FitRecordListener.h](#).

Referenced by [FitParser::load\(\)](#).

Here is the caller graph for this function:



7.44.3.2 lastTimestamp()

```
FIT_DATE_TIME FitRecordListener::lastTimestamp () const [inline]
```

Gets the last timestamp from records.

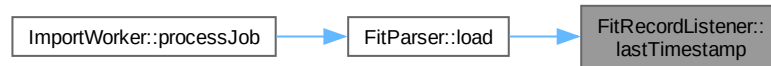
Returns

Last timestamp value.

Definition at line 43 of file [FitRecordListener.h](#).

Referenced by [FitParser::load\(\)](#).

Here is the caller graph for this function:

**7.44.3.3 OnMesg()**

```
void FitRecordListener::OnMesg (
    fit::RecordMesg & mesg) [override]
```

Processes a single FIT record message.

Parameters

<i>mesg</i>	FIT record message.
-------------	---------------------

Definition at line 14 of file [FitRecordListener.cpp](#).

References [RideRecord::altitude](#), [RideRecord::cadence](#), [RideRecord::elapsedSeconds](#), [RideRecord::hasAltitude](#), [RideRecord::hasCadence](#), [RideRecord::hasGps](#), [RideRecord::hasHeartRate](#), [RideRecord::hasPower](#), [RideRecord::hasSpeed](#), [RideRecord::heartRate](#), [RideRecord::latitude](#), [RideRecord::longitude](#), [RideRecord::power](#), and [RideRecord::speed](#).

7.44.4 Member Data Documentation**7.44.4.1 m_firstTimestamp**

```
FIT_DATE_TIME FitRecordListener::m_firstTimestamp = FIT_DATE_TIME_INVALID [private]
```

Definition at line 48 of file [FitRecordListener.h](#).

7.44.4.2 m_lastTimestamp

```
FIT_DATE_TIME FitRecordListener::m_lastTimestamp = FIT_DATE_TIME_INVALID [private]
```

Definition at line 49 of file [FitRecordListener.h](#).

7.44.4.3 m_rideData

```
RideData& FitRecordListener::m_rideData [private]
```

Definition at line 46 of file [FitRecordListener.h](#).

The documentation for this class was generated from the following files:

- [src/fit/FitRecordListener.h](#)
- [src/fit/FitRecordListener.cpp](#)

7.45 MapFitMath::FitResult Struct Reference

Result of fitting GPS bounds to map viewport.

```
#include <MapFitMath.h>
```

Public Attributes

- double `minLat` = 0.0
Minimum latitude of track bounds.
- double `maxLat` = 0.0
Maximum latitude of track bounds.
- double `minLon` = 0.0
Minimum longitude of track bounds.
- double `maxLon` = 0.0
Maximum longitude of track bounds.
- double `centerLat` = 0.0
Center latitude of track bounds.
- double `centerLon` = 0.0
Center longitude of track bounds.
- double `lonSpanDegrees` = 0.0
Longitude span in degrees for centering calculation.
- int `zoom` = 1
Recommended zoom level to fit bounds in viewport.

7.45.1 Detailed Description

Result of fitting GPS bounds to map viewport.
Definition at line 11 of file [MapFitMath.h](#).

7.45.2 Member Data Documentation

7.45.2.1 `centerLat`

```
double MapFitMath::FitResult::centerLat = 0.0
```

Center latitude of track bounds.
Definition at line 26 of file [MapFitMath.h](#).

7.45.2.2 `centerLon`

```
double MapFitMath::FitResult::centerLon = 0.0
```

Center longitude of track bounds.
Definition at line 29 of file [MapFitMath.h](#).

7.45.2.3 `lonSpanDegrees`

```
double MapFitMath::FitResult::lonSpanDegrees = 0.0
```

Longitude span in degrees for centering calculation.
Definition at line 32 of file [MapFitMath.h](#).

7.45.2.4 `maxLat`

```
double MapFitMath::FitResult::maxLat = 0.0
```

Maximum latitude of track bounds.
Definition at line 17 of file [MapFitMath.h](#).

7.45.2.5 `maxLon`

```
double MapFitMath::FitResult::maxLon = 0.0
```

Maximum longitude of track bounds.
Definition at line 23 of file [MapFitMath.h](#).

7.45.2.6 minLat

```
double MapFitMath::FitResult::minLat = 0.0
```

Minimum latitude of track bounds.

Definition at line 14 of file [MapFitMath.h](#).

7.45.2.7 minLon

```
double MapFitMath::FitResult::minLon = 0.0
```

Minimum longitude of track bounds.

Definition at line 20 of file [MapFitMath.h](#).

7.45.2.8 zoom

```
int MapFitMath::FitResult::zoom = 1
```

Recommended zoom level to fit bounds in viewport.

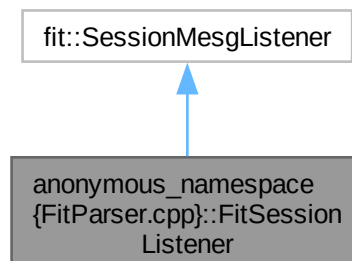
Definition at line 35 of file [MapFitMath.h](#).

The documentation for this struct was generated from the following file:

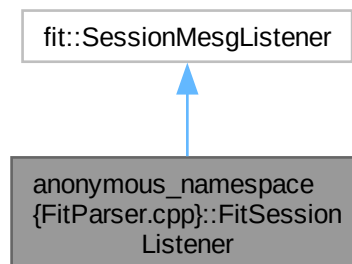
- [src/maps/MapFitMath.h](#)

7.46 anonymous_namespace{FitParser.cpp}::FitSessionListener Class Reference

Inheritance diagram for anonymous_namespace{FitParser.cpp}::FitSessionListener:



Collaboration diagram for `anonymous_namespace{FitParser.cpp}::FitSessionListener`:



Public Member Functions

- void [OnMesg](#) (`fit::SessionMesg &mesg`) override
- `FIT_DATE_TIME` [startTime](#) () const

Private Attributes

- `FIT_DATE_TIME` [m_startTime](#) = `FIT_DATE_TIME_INVALID`

7.46.1 Detailed Description

Definition at line 32 of file [FitParser.cpp](#).

7.46.2 Member Function Documentation

7.46.2.1 OnMesg()

```
void anonymous_namespace{FitParser.cpp}::FitSessionListener::OnMesg (
    fit::SessionMesg & mesg) [inline], [override]
```

Definition at line 35 of file [FitParser.cpp](#).

7.46.2.2 startTime()

```
FIT_DATE_TIME anonymous_namespace{FitParser.cpp}::FitSessionListener::startTime () const [inline]
```

Definition at line 41 of file [FitParser.cpp](#).

7.46.3 Member Data Documentation

7.46.3.1 m_startTime

```
FIT_DATE_TIME anonymous_namespace{FitParser.cpp}::FitSessionListener::m_startTime = FIT_DATE←
_TIME_INVALID [private]
```

Definition at line 44 of file [FitParser.cpp](#).

The documentation for this class was generated from the following file:

- `src/fit/FitParser.cpp`

7.47 FtpEntry Struct Reference

Represents a historical FTP entry.

```
#include <AthleteRepository.h>
```


Public Attributes

- int `id` = -1
Unique FTP entry identifier.
- int `athleteld` = -1
Associated athlete ID.
- int `ftpWatts` = 0
FTP value in watts.
- QString `effectiveFrom`
Date FTP becomes effective (ISO 8601 YYYY-MM-DD).
- QString `notes`
Optional notes about FTP change.

7.47.1 Detailed Description

Represents a historical FTP entry.

Definition at line 55 of file [AthleteRepository.h](#).

7.47.2 Member Data Documentation

7.47.2.1 `athleteld`

```
int FtpEntry::athleteId = -1
```

Associated athlete ID.

Definition at line 61 of file [AthleteRepository.h](#).

Referenced by [AthleteRepository::addFtpEntry\(\)](#), and [AthleteRepository::getFtpHistory\(\)](#).

7.47.2.2 `effectiveFrom`

```
QString FtpEntry::effectiveFrom
```

Date FTP becomes effective (ISO 8601 YYYY-MM-DD).

Definition at line 67 of file [AthleteRepository.h](#).

Referenced by [AthleteRepository::addFtpEntry\(\)](#), and [AthleteRepository::getFtpHistory\(\)](#).

7.47.2.3 `ftpWatts`

```
int FtpEntry::ftpWatts = 0
```

FTP value in watts.

Definition at line 64 of file [AthleteRepository.h](#).

Referenced by [AthleteRepository::addFtpEntry\(\)](#), and [AthleteRepository::getFtpHistory\(\)](#).

7.47.2.4 `id`

```
int FtpEntry::id = -1
```

Unique FTP entry identifier.

Definition at line 58 of file [AthleteRepository.h](#).

Referenced by [AthleteRepository::getFtpHistory\(\)](#).

7.47.2.5 `notes`

```
QString FtpEntry::notes
```

Optional notes about FTP change.

Definition at line 70 of file [AthleteRepository.h](#).

Referenced by [AthleteRepository::addFtpEntry\(\)](#), and [AthleteRepository::getFtpHistory\(\)](#).

The documentation for this struct was generated from the following file:

- `src/database/AthleteRepository.h`

7.48 FTPZones Class Reference

Analyzes time spent in FTP-based power zones.

#include <FTPZones.h>

Public Types

- using [ZoneArray](#) = std::array<[PowerZone](#), [kZoneCount](#)>
Array container type for zone definitions.

Static Public Member Functions

- static [ZoneArray](#) [defaultZones](#) ()
Returns the default power zone definitions.
- static [ZoneArray](#) [compute](#) (const [RideData](#) &rideData, double ftp)
Computes power zones for an activity.

Static Public Attributes

- static constexpr int [kZoneCount](#) = 7
Number of FTP power zones (always 7 for standard zones).

7.48.1 Detailed Description

Analyzes time spent in FTP-based power zones.

Definition at line 35 of file [FTPZones.h](#).

7.48.2 Member Typedef Documentation

7.48.2.1 ZoneArray

```
using FTPZones::ZoneArray = std::array<PowerZone, kZoneCount>
```

Array container type for zone definitions.

Definition at line 42 of file [FTPZones.h](#).

7.48.3 Member Function Documentation

7.48.3.1 compute()

```
FTPZones::ZoneArray FTPZones::compute (
    const RideData & rideData,
    double ftp) [static]
```

Computes power zones for an activity.

Parameters

<i>rideData</i>	Activity data.
<i>ftp</i>	Athlete functional threshold power in watts.

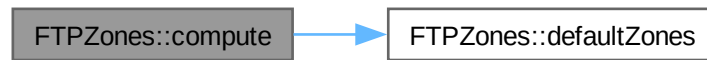
Returns

Array of zones with time spent in each.

Definition at line 22 of file [FTPZones.cpp](#).

References [defaultZones\(\)](#), and [RideData::records](#).

Here is the call graph for this function:



7.48.3.2 defaultZones()

```
FTPZones::ZoneArray FTPZones::defaultZones () [static]
```

Returns the default power zone definitions.

Returns

Array of standard FTP-based zones.

Definition at line 9 of file [FTPZones.cpp](#).

Referenced by [compute\(\)](#).

Here is the caller graph for this function:



7.48.4 Member Data Documentation

7.48.4.1 kZoneCount

```
int FTPZones::kZoneCount = 7 [static], [constexpr]
```

Number of FTP power zones (always 7 for standard zones).

Definition at line 39 of file [FTPZones.h](#).

The documentation for this class was generated from the following files:

- [src/analysis/FTPZones.h](#)
- [src/analysis/FTPZones.cpp](#)

7.49 MainWindow::ImportBatchSummary Struct Reference

Public Attributes

- int [queued](#) = 0
- int [imported](#) = 0
- int [duplicates](#) = 0
- int [failed](#) = 0
- bool [showResultDialog](#) = false
- QString [sourceLabel](#)
- QStringList [errors](#)

7.49.1 Detailed Description

Definition at line 363 of file [MainWindow.h](#).

7.49.2 Member Data Documentation

7.49.2.1 duplicates

```
int MainWindow::ImportBatchSummary::duplicates = 0
```

Definition at line 367 of file [MainWindow.h](#).

7.49.2.2 errors

```
QStringList MainWindow::ImportBatchSummary::errors
```

Definition at line 371 of file [MainWindow.h](#).

7.49.2.3 failed

```
int MainWindow::ImportBatchSummary::failed = 0
```

Definition at line 368 of file [MainWindow.h](#).

7.49.2.4 imported

```
int MainWindow::ImportBatchSummary::imported = 0
```

Definition at line 366 of file [MainWindow.h](#).

7.49.2.5 queued

```
int MainWindow::ImportBatchSummary::queued = 0
```

Definition at line 365 of file [MainWindow.h](#).

7.49.2.6 showResultDialog

```
bool MainWindow::ImportBatchSummary::showResultDialog = false
```

Definition at line 369 of file [MainWindow.h](#).

7.49.2.7 sourceLabel

```
QString MainWindow::ImportBatchSummary::sourceLabel
```

Definition at line 370 of file [MainWindow.h](#).

The documentation for this struct was generated from the following file:

- [src/gui/MainWindow.h](#)

7.50 ImportJob Struct Reference

Represents a single background FIT import request.

```
#include <ImportJob.h>
```

Public Types

- enum class [State](#) {
 [Queued](#) , [Parsing](#) , [Storing](#) , [Completed](#) ,
 [Failed](#) }

Processing stage used for worker/queue state reporting.

Public Attributes

- QString [jobId](#)
Unique import job id.
- QString [batchId](#)
Caller-defined batch id for grouped summaries.
- QString [filePath](#)
Absolute path to FIT file.
- QString [athleteId](#)
Target athlete id.
- QString [importSource](#)
Source tag (e.g. manual, drag_drop, watch_folder).
- [State](#) [state](#) = [State::Queued](#)
- double [progress](#) = 0.0
Best-effort progress percentage [0..100].
- QString [errorMessage](#)
User-facing failure details when state is Failed.

7.50.1 Detailed Description

Represents a single background FIT import request.
Definition at line 10 of file [ImportJob.h](#).

7.50.2 Member Enumeration Documentation

7.50.2.1 State

```
enum class ImportJob::State [strong]
```

Processing stage used for worker/queue state reporting.

Enumerator

Queued	Waiting in queue.
Parsing	FIT file parsing and duplicate checks.
Storing	Activity/sample persistence in database.
Completed	Successfully imported.
Failed	Failed or rejected as duplicate.

Definition at line 13 of file [ImportJob.h](#).

7.50.3 Member Data Documentation

7.50.3.1 athleteId

```
QString ImportJob::athleteId
```

Target athlete id.

Definition at line 25 of file [ImportJob.h](#).

Referenced by [ImportQueue::enqueue\(\)](#), and [ImportWorker::processJob\(\)](#).

7.50.3.2 batchId

```
QString ImportJob::batchId
```

Caller-defined batch id for grouped summaries.

Definition at line 23 of file [ImportJob.h](#).

Referenced by [ImportQueue::enqueue\(\)](#), and [ImportWorker::processJob\(\)](#).

7.50.3.3 errorMessage

`QString ImportJob::errorMessage`

User-facing failure details when state is Failed.

Definition at line 29 of file [ImportJob.h](#).

7.50.3.4 filePath

`QString ImportJob::filePath`

Absolute path to FIT file.

Definition at line 24 of file [ImportJob.h](#).

Referenced by [ImportQueue::enqueue\(\)](#), and [ImportWorker::processJob\(\)](#).

7.50.3.5 importSource

`QString ImportJob::importSource`

Source tag (e.g. manual, drag_drop, watch_folder).

Definition at line 26 of file [ImportJob.h](#).

Referenced by [ImportQueue::enqueue\(\)](#), and [ImportWorker::processJob\(\)](#).

7.50.3.6 jobId

`QString ImportJob::jobId`

Unique import job id.

Definition at line 22 of file [ImportJob.h](#).

Referenced by [ImportQueue::enqueue\(\)](#), and [ImportWorker::processJob\(\)](#).

7.50.3.7 progress

`double ImportJob::progress = 0.0`

Best-effort progress percentage [0..100].

Definition at line 28 of file [ImportJob.h](#).

Referenced by [ImportQueue::enqueue\(\)](#).

7.50.3.8 state

`State ImportJob::state = State::Queued`

Definition at line 27 of file [ImportJob.h](#).

The documentation for this struct was generated from the following file:

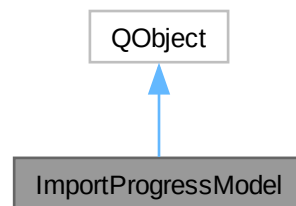
- [src/import/ImportJob.h](#)

7.51 ImportProgressModel Class Reference

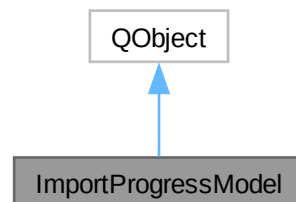
UI-facing aggregate state for background import queue activity.

```
#include <ImportProgressModel.h>
```

Inheritance diagram for ImportProgressModel:



Collaboration diagram for ImportProgressModel:



Signals

- void **changed** ()
Emitted whenever visible model state changes.

Public Member Functions

- [ImportProgressModel](#) (QObject *parent=nullptr)
- void [attachQueue](#) ([ImportQueue](#) *queue)
Connects queue signals and starts mirroring import state.
- int [activeJobs](#) () const
- int [completedJobs](#) () const
- int [failedJobs](#) () const
- double [progress](#) () const
- QString [currentFileName](#) () const
- QString [statusText](#) () const

Private Attributes

- int [m_activeJobs](#) = 0
- int [m_completedJobs](#) = 0
- int [m_failedJobs](#) = 0
- double [m_progress](#) = 0.0
- QString [m_currentFileName](#)
- QString [m_statusText](#)

7.51.1 Detailed Description

UI-facing aggregate state for background import queue activity.
Tracks currently active jobs and latest progress text for status widgets.
Definition at line 15 of file [ImportProgressModel.h](#).

7.51.2 Constructor & Destructor Documentation

7.51.2.1 ImportProgressModel()

```
ImportProgressModel::ImportProgressModel (
    QObject * parent = nullptr) [explicit]
```

Definition at line 9 of file [ImportProgressModel.cpp](#).

7.51.3 Member Function Documentation

7.51.3.1 activeJobs()

```
int ImportProgressModel::activeJobs () const [inline]
```

Definition at line 24 of file [ImportProgressModel.h](#).

7.51.3.2 attachQueue()

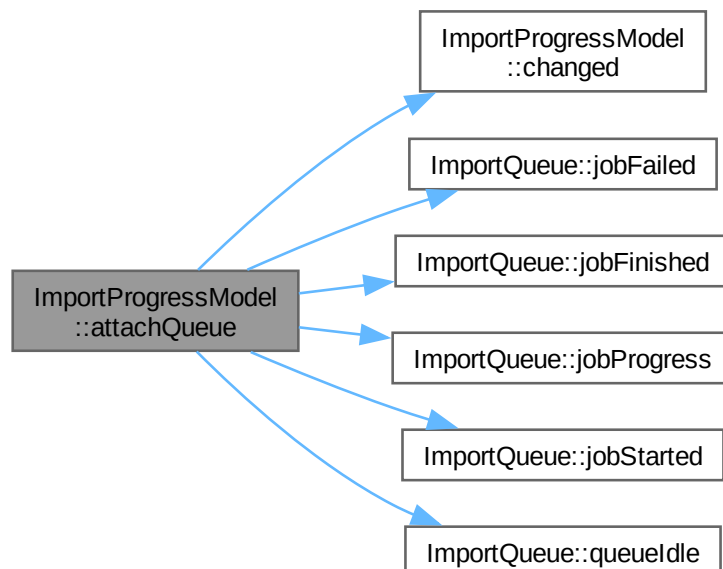
```
void ImportProgressModel::attachQueue (
    ImportQueue * queue)
```

Connects queue signals and starts mirroring import state.

Definition at line 13 of file [ImportProgressModel.cpp](#).

References [changed\(\)](#), [ImportQueue::jobFailed\(\)](#), [ImportQueue::jobFinished\(\)](#), [ImportQueue::jobProgress\(\)](#), [ImportQueue::jobStarted\(\)](#), and [ImportQueue::queueIdle\(\)](#).

Here is the call graph for this function:



7.51.3.3 completedJobs()

```
int ImportProgressModel::completedJobs () const [inline]
```

Definition at line 25 of file [ImportProgressModel.h](#).

7.51.3.4 currentFileName()

```
QString ImportProgressModel::currentFileName () const [inline]
```

Definition at line 28 of file [ImportProgressModel.h](#).

7.51.3.5 failedJobs()

```
int ImportProgressModel::failedJobs () const [inline]
```

Definition at line 26 of file [ImportProgressModel.h](#).

7.51.3.6 progress()

```
double ImportProgressModel::progress () const [inline]
```

Definition at line 27 of file [ImportProgressModel.h](#).

7.51.3.7 statusText()

```
QString ImportProgressModel::statusText () const [inline]
```

Definition at line 29 of file [ImportProgressModel.h](#).

7.51.4 Member Data Documentation

7.51.4.1 m_activeJobs

```
int ImportProgressModel::m_activeJobs = 0 [private]
```

Definition at line 36 of file [ImportProgressModel.h](#).

7.51.4.2 m_completedJobs

```
int ImportProgressModel::m_completedJobs = 0 [private]
```

Definition at line 37 of file [ImportProgressModel.h](#).

7.51.4.3 m_currentFileName

```
QString ImportProgressModel::m_currentFileName [private]
```

Definition at line 40 of file [ImportProgressModel.h](#).

7.51.4.4 m_failedJobs

```
int ImportProgressModel::m_failedJobs = 0 [private]
```

Definition at line 38 of file [ImportProgressModel.h](#).

7.51.4.5 m_progress

```
double ImportProgressModel::m_progress = 0.0 [private]
```

Definition at line 39 of file [ImportProgressModel.h](#).

7.51.4.6 m_statusText

```
QString ImportProgressModel::m_statusText [private]
```

Definition at line 41 of file [ImportProgressModel.h](#).

The documentation for this class was generated from the following files:

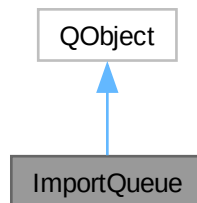
- [src/import/ImportProgressModel.h](#)
- [src/import/ImportProgressModel.cpp](#)

7.52 ImportQueue Class Reference

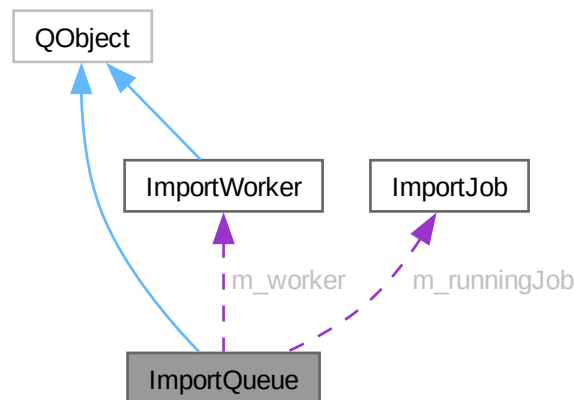
Sequential queue that dispatches background FIT import jobs.

```
#include <ImportQueue.h>
```

Inheritance diagram for ImportQueue:



Collaboration diagram for ImportQueue:



Signals

- void **jobQueued** (const QString &jobId, const QString &batchId, const QString &filePath, int activeCount)
Emitted when a job enters the queue.
- void **jobStarted** (const QString &jobId, const QString &batchId, const QString &filePath, int activeCount)
Emitted when a queued job begins processing.
- void **jobProgress** (const QString &jobId, const QString &batchId, const QString &filePath, double percentage, const QString &statusText, int activeCount)
Emitted with progress updates from the running job.
- void **jobFinished** (const QString &jobId, const QString &batchId, const QString &filePath, int activityId, int activeCount)
Emitted when a job successfully creates an activity.
- void **jobFailed** (const QString &jobId, const QString &batchId, const QString &filePath, const QString &reason, const QString &errorMessage, int activeCount)

Emitted when a job fails or is rejected as duplicate.

- void **queueStateChanged** (int runningCount, int pendingCount)

Emitted whenever running/pending counts change.

- void **queueIdle** ()

Emitted after the last queued/running job is fully processed.

Public Member Functions

- [ImportQueue](#) (QObject *parent=nullptr)
- void [setDatabasePath](#) (const QString &path)
Sets the SQLite database path used by the worker.
- void [enqueue](#) (const QString &filePath, const QString &athletelId, const QString &batchId, const QString &importSource)
Enqueues a file import request.
- int [pendingCount](#) () const
Returns active jobs count (queued + currently running).
- bool [isBusy](#) () const
Returns true when a job is currently running.

Private Slots

- void [onWorkerProgress](#) (const QString &jobId, const QString &batchId, const QString &filePath, double percentage, const QString &statusText)
- void [onWorkerCompleted](#) (const QString &jobId, const QString &batchId, const QString &filePath, int activityId)
- void [onWorkerFailed](#) (const QString &jobId, const QString &batchId, const QString &filePath, const QString &reason, const QString &errorMessage)

Private Member Functions

- void [processNext](#) ()
- int [activeCountUnsafe](#) () const

Private Attributes

- QThread * [m_thread](#) = nullptr
- [ImportWorker](#) * [m_worker](#) = nullptr
- QQueue< [ImportJob](#) > [m_queue](#)
- bool [m_busy](#) = false
- QString [m_dbPath](#)
- [ImportJob](#) [m_runningJob](#)

7.52.1 Detailed Description

Sequential queue that dispatches background FIT import jobs.

Jobs are processed one-at-a-time on a worker thread. The queue emits lifecycle signals for UI updates and batch bookkeeping.

Definition at line 20 of file [ImportQueue.h](#).

7.52.2 Constructor & Destructor Documentation

7.52.2.1 ImportQueue()

```
ImportQueue::ImportQueue (
    QObject * parent = nullptr) [explicit]
```

Definition at line 10 of file [ImportQueue.cpp](#).

7.52.2.2 ~ImportQueue()

`ImportQueue::~~ImportQueue () [override]`

Definition at line 27 of file [ImportQueue.cpp](#).

7.52.3 Member Function Documentation

7.52.3.1 activeCountUnsafe()

`int ImportQueue::activeCountUnsafe () const [private]`

Definition at line 140 of file [ImportQueue.cpp](#).

7.52.3.2 enqueue()

```
void ImportQueue::enqueue (
    const QString & filePath,
    const QString & athleteId,
    const QString & batchId,
    const QString & importSource)
```

Enqueues a file import request.

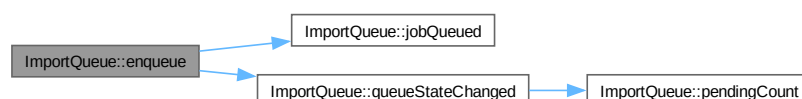
Parameters

<i>filePath</i>	Absolute file path to a FIT file.
<i>athleteId</i>	Target athlete id as string.
<i>batchId</i>	Caller-defined batch id for grouped summaries.
<i>importSource</i>	Source metadata saved with the imported activity.

Definition at line 43 of file [ImportQueue.cpp](#).

References [ImportJob::athleteId](#), [ImportJob::batchId](#), [ImportJob::filePath](#), [ImportJob::importSource](#), [ImportJob::jobId](#), [jobQueued\(\)](#), [ImportJob::progress](#), [ImportJob::Queued](#), and [queueStateChanged\(\)](#).

Here is the call graph for this function:



7.52.3.3 isBusy()

`bool ImportQueue::isBusy () const`

Returns true when a job is currently running.

Definition at line 71 of file [ImportQueue.cpp](#).

7.52.3.4 jobFailed

```
void ImportQueue::jobFailed (
    const QString & jobId,
    const QString & batchId,
    const QString & filePath,
    const QString & reason,
```

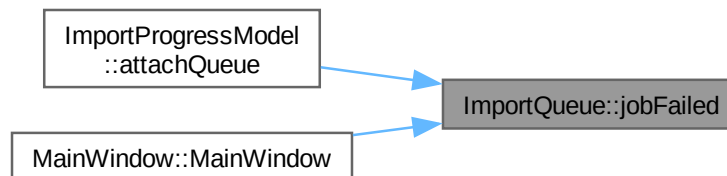
```
const QString & errorMessage,
int activeCount) [signal]
```

Emitted when a job fails or is rejected as duplicate.

`reason` is a stable category such as `duplicate`, `time_overlap`, or `error`; `errorMessage` provides user-facing details.

Referenced by [ImportProgressModel::attachQueue\(\)](#), and [MainWindow::MainWindow\(\)](#).

Here is the caller graph for this function:



7.52.3.5 onWorkerCompleted

```
void ImportQueue::onWorkerCompleted (
    const QString & jobId,
    const QString & batchId,
    const QString & filePath,
    int activityId) [private], [slot]
```

Definition at line 85 of file [ImportQueue.cpp](#).

7.52.3.6 onWorkerFailed

```
void ImportQueue::onWorkerFailed (
    const QString & jobId,
    const QString & batchId,
    const QString & filePath,
    const QString & reason,
    const QString & errorMessage) [private], [slot]
```

Definition at line 103 of file [ImportQueue.cpp](#).

7.52.3.7 onWorkerProgress

```
void ImportQueue::onWorkerProgress (
    const QString & jobId,
    const QString & batchId,
    const QString & filePath,
    double percentage,
    const QString & statusText) [private], [slot]
```

Definition at line 76 of file [ImportQueue.cpp](#).

7.52.3.8 pendingCount()

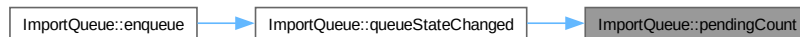
```
int ImportQueue::pendingCount () const
```

Returns active jobs count (queued + currently running).

Definition at line 66 of file [ImportQueue.cpp](#).

Referenced by [queueStateChanged\(\)](#).

Here is the caller graph for this function:



7.52.3.9 processNext()

`void ImportQueue::processNext () [private]`
 Definition at line 122 of file [ImportQueue.cpp](#).

7.52.3.10 setDatabasePath()

`void ImportQueue::setDatabasePath (const QString & path)`
 Sets the SQLite database path used by the worker.
 Definition at line 34 of file [ImportQueue.cpp](#).

7.52.4 Member Data Documentation

7.52.4.1 m_busy

`bool ImportQueue::m_busy = false [private]`
 Definition at line 116 of file [ImportQueue.h](#).

7.52.4.2 m_dbPath

`QString ImportQueue::m_dbPath [private]`
 Definition at line 117 of file [ImportQueue.h](#).

7.52.4.3 m_queue

`QQueue<ImportJob> ImportQueue::m_queue [private]`
 Definition at line 115 of file [ImportQueue.h](#).

7.52.4.4 m_runningJob

`ImportJob ImportQueue::m_runningJob [private]`
 Definition at line 118 of file [ImportQueue.h](#).

7.52.4.5 m_thread

`QThread* ImportQueue::m_thread = nullptr [private]`
 Definition at line 113 of file [ImportQueue.h](#).

7.52.4.6 m_worker

`ImportWorker* ImportQueue::m_worker = nullptr [private]`
 Definition at line 114 of file [ImportQueue.h](#).

The documentation for this class was generated from the following files:

- `src/import/ImportQueue.h`
- `src/import/ImportQueue.cpp`

7.53 ImportRecord Struct Reference

Log entry for an activity import.

```
#include <ImportRepository.h>
```

Public Attributes

- int `id` = -1
Unique import log entry ID.
- QString `fileName`
Name of imported file.
- QString `importedAt`
ISO8601 import timestamp.
- QString `status`
Import status (success, duplicate, error).
- int `activityId` = -1
ID of imported activity (or -1 if failed).
- QString `errorMessage`
Error message if import failed.

7.53.1 Detailed Description

Log entry for an activity import.

Definition at line 14 of file [ImportRepository.h](#).

7.53.2 Member Data Documentation

7.53.2.1 activityId

```
int ImportRecord::activityId = -1
```

ID of imported activity (or -1 if failed).

Definition at line 29 of file [ImportRepository.h](#).

Referenced by [ImportRepository::getRecentImports\(\)](#), [ImportRepository::logImport\(\)](#), and [ImportWorker::processJob\(\)](#).

7.53.2.2 errorMessage

```
QString ImportRecord::errorMessage
```

Error message if import failed.

Definition at line 32 of file [ImportRepository.h](#).

Referenced by [ImportRepository::getRecentImports\(\)](#), [ImportRepository::logImport\(\)](#), and [ImportWorker::processJob\(\)](#).

7.53.2.3 fileName

```
QString ImportRecord::fileName
```

Name of imported file.

Definition at line 20 of file [ImportRepository.h](#).

Referenced by [ImportRepository::getRecentImports\(\)](#), [ImportRepository::logImport\(\)](#), and [ImportWorker::processJob\(\)](#).

7.53.2.4 id

```
int ImportRecord::id = -1
```

Unique import log entry ID.

Definition at line 17 of file [ImportRepository.h](#).

Referenced by [ImportRepository::getRecentImports\(\)](#).

7.53.2.5 importedAt

QString ImportRecord::importedAt

ISO8601 import timestamp.

Definition at line 23 of file [ImportRepository.h](#).

Referenced by [ImportRepository::getRecentImports\(\)](#), and [ImportRepository::logImport\(\)](#).

7.53.2.6 status

QString ImportRecord::status

Import status (success, duplicate, error).

Definition at line 26 of file [ImportRepository.h](#).

Referenced by [ImportRepository::getRecentImports\(\)](#), [ImportRepository::logImport\(\)](#), and [ImportWorker::processJob\(\)](#).

The documentation for this struct was generated from the following file:

- [src/database/ImportRepository.h](#)

7.54 ImportRepository Class Reference

Repository for import log persistence.

```
#include <ImportRepository.h>
```

Public Member Functions

- [ImportRepository](#) (QSqlDatabase &db)
Constructs repository with database connection.
- int [logImport](#) (const [ImportRecord](#) &r)
Logs an import attempt.
- QList< [ImportRecord](#) > [getRecentImports](#) (int limit=50)
Retrieves recent import log entries.

Private Attributes

- QSqlDatabase & [m_db](#)

7.54.1 Detailed Description

Repository for import log persistence.

Tracks all import attempts including successes, duplicates, and errors.

Definition at line 40 of file [ImportRepository.h](#).

7.54.2 Constructor & Destructor Documentation

7.54.2.1 ImportRepository()

```
ImportRepository::ImportRepository (  
    QSqlDatabase & db) [explicit]
```

Constructs repository with database connection.

Parameters

<i>db</i>	Database connection.
-----------	----------------------

Definition at line 10 of file [ImportRepository.cpp](#).

7.54.3 Member Function Documentation

7.54.3.1 getRecentImports()

```
QList< ImportRecord > ImportRepository::getRecentImports (
    int limit = 50)
```

Retrieves recent import log entries.

Parameters

<i>limit</i>	Maximum number of entries to return (default 50).
--------------	---

Returns

List of import records.

Definition at line 31 of file [ImportRepository.cpp](#).

References [ImportRecord::activityId](#), [ImportRecord::errorMessage](#), [ImportRecord::fileName](#), [ImportRecord::id](#), [ImportRecord::importedAt](#), and [ImportRecord::status](#).

7.54.3.2 logImport()

```
int ImportRepository::logImport (
    const ImportRecord & r)
```

Logs an import attempt.

Parameters

<i>r</i>	Import record to log.
----------	-----------------------

Returns

ID of log entry, or -1 on error.

Definition at line 14 of file [ImportRepository.cpp](#).

References [ImportRecord::activityId](#), [ImportRecord::errorMessage](#), [ImportRecord::fileName](#), [ImportRecord::importedAt](#), and [ImportRecord::status](#).

Referenced by [ImportWorker::processJob\(\)](#).

Here is the caller graph for this function:



7.54.4 Member Data Documentation

7.54.4.1 m_db

```
QSqlDatabase& ImportRepository::m_db [private]
```

Definition at line 64 of file [ImportRepository.h](#).

The documentation for this class was generated from the following files:

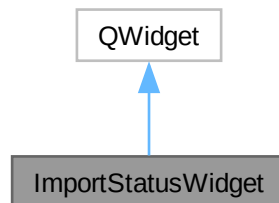
- [src/database/ImportRepository.h](#)
- [src/database/ImportRepository.cpp](#)

7.55 ImportStatusWidget Class Reference

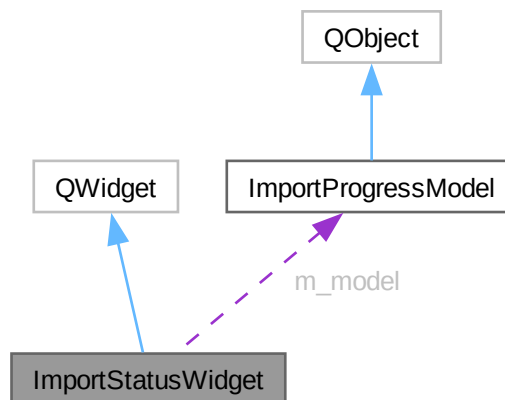
Toolbar widget showing live background import queue progress.

```
#include <ImportStatusWidget.h>
```

Inheritance diagram for ImportStatusWidget:



Collaboration diagram for ImportStatusWidget:



Public Member Functions

- [ImportStatusWidget](#) ([QWidget](#) *parent=nullptr)
- void [setModel](#) ([ImportProgressModel](#) *model)
Binds a progress model; disconnects any previously bound model.

Private Member Functions

- void [updateFromModel](#) ()

Private Attributes

- [ImportProgressModel](#) * [m_model](#) = nullptr
- [QLabel](#) * [m_titleLabel](#) = nullptr
- [QLabel](#) * [m_detailLabel](#) = nullptr

- `QProgressBar * m_progressBar = nullptr`

7.55.1 Detailed Description

Toolbar widget showing live background import queue progress.
Definition at line 14 of file [ImportStatusWidget.h](#).

7.55.2 Constructor & Destructor Documentation

7.55.2.1 ImportStatusWidget()

```
ImportStatusWidget::ImportStatusWidget (
    QWidget * parent = nullptr) [explicit]
```

Definition at line 11 of file [ImportStatusWidget.cpp](#).

7.55.3 Member Function Documentation

7.55.3.1 setModel()

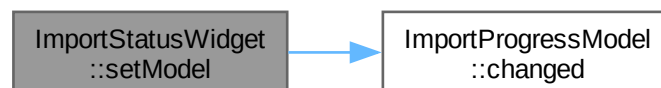
```
void ImportStatusWidget::setModel (
    ImportProgressModel * model)
```

Binds a progress model; disconnects any previously bound model.

Definition at line 36 of file [ImportStatusWidget.cpp](#).

References [ImportProgressModel::changed\(\)](#).

Here is the call graph for this function:



7.55.3.2 updateFromModel()

```
void ImportStatusWidget::updateFromModel () [private]
```

Definition at line 51 of file [ImportStatusWidget.cpp](#).

7.55.4 Member Data Documentation

7.55.4.1 m_detailLabel

```
QLabel* ImportStatusWidget::m_detailLabel = nullptr [private]
```

Definition at line 28 of file [ImportStatusWidget.h](#).

7.55.4.2 m_model

```
ImportProgressModel* ImportStatusWidget::m_model = nullptr [private]
```

Definition at line 26 of file [ImportStatusWidget.h](#).

7.55.4.3 m_progressBar

```
QProgressBar* ImportStatusWidget::m_progressBar = nullptr [private]
```

Definition at line 29 of file [ImportStatusWidget.h](#).

7.55.4.4 m_titleLabel

```
QLabel* ImportStatusWidget::m_titleLabel = nullptr [private]
```

Definition at line 27 of file [ImportStatusWidget.h](#).

The documentation for this class was generated from the following files:

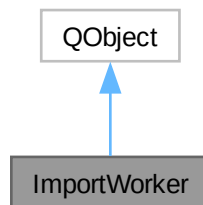
- [src/gui/ImportStatusWidget.h](#)
- [src/gui/ImportStatusWidget.cpp](#)

7.56 ImportWorker Class Reference

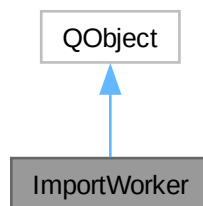
Worker that imports FIT files in a dedicated background thread.

```
#include <ImportWorker.h>
```

Inheritance diagram for ImportWorker:



Collaboration diagram for ImportWorker:



Public Slots

- void [processJob](#) (const [ImportJob](#) &job)
Processes one queued import job.

Signals

- void **progressChanged** (const QString &jobId, const QString &batchId, const QString &filePath, double percentage, const QString &statusText)
Progress update for the currently running import job.
- void **completed** (const QString &jobId, const QString &batchId, const QString &filePath, int activityId)

Emitted when an activity was imported successfully.

- void [failed](#) (const QString &jobId, const QString &batchId, const QString &filePath, const QString &reason, const QString &errorMessage)

Emitted when import fails or is rejected as duplicate.

Public Member Functions

- [ImportWorker](#) (QObject *parent=nullptr)
- void [setDatabasePath](#) (const QString &path)
Sets the SQLite database path used for imports.

Private Attributes

- QString [m_dbPath](#)

7.56.1 Detailed Description

Worker that imports FIT files in a dedicated background thread.

The worker parses FIT payloads, performs duplicate checks, writes activity and sample rows, and emits progress/↔ final status signals.

Definition at line 16 of file [ImportWorker.h](#).

7.56.2 Constructor & Destructor Documentation

7.56.2.1 ImportWorker()

```
ImportWorker::ImportWorker (
    QObject * parent = nullptr) [explicit]
```

Definition at line 35 of file [ImportWorker.cpp](#).

7.56.3 Member Function Documentation

7.56.3.1 failed

```
void ImportWorker::failed (
    const QString & jobId,
    const QString & batchId,
    const QString & filePath,
    const QString & reason,
    const QString & errorMessage) [signal]
```

Emitted when import fails or is rejected as duplicate.

reason is a machine-readable category (duplicate, time_overlap, error) and errorMessage contains details.

Referenced by [processJob\(\)](#).

Here is the caller graph for this function:



7.56.3.2 processJob

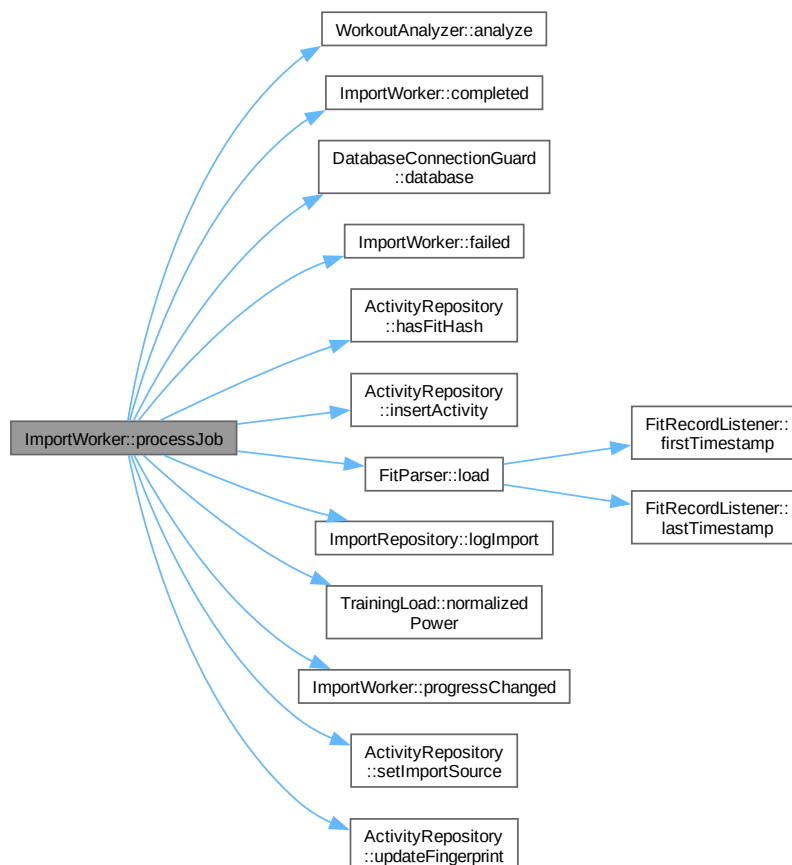
```
void ImportWorker::processJob (
    const ImportJob & job) [slot]
```

Processes one queued import job.

Definition at line 44 of file [ImportWorker.cpp](#).

References [RideData::activityEndTimeUtc](#), [ImportRecord::activityId](#), [RideData::activityStartTimeUtc](#), [RideRecord::altitude](#), [WorkoutAnalyzer::analyze\(\)](#), [Activity::athleteId](#), [ImportJob::athleteId](#), [WorkoutStatistics::averageCadence](#), [WorkoutStatistics::averageHeartRate](#), [WorkoutStatistics::averagePower](#), [WorkoutStatistics::averageSpeed](#), [Activity::avgCadence](#), [Activity::avgHr](#), [Activity::avgPower](#), [Activity::avgSpeed](#), [ImportJob::batchId](#), [RideRecord::cadence](#), [completed\(\)](#), [DatabaseConnectionGuard::database\(\)](#), [Activity::distanceM](#), [Activity::durationSec](#), [WorkoutStatistics::durationSeconds](#), [RideRecord::elapsedSeconds](#), [Activity::elevationGain](#), [WorkoutStatistics::elevationGainM](#), [Activity::endTime](#), [ImportRecord::errorMessage](#), [failed\(\)](#), [Activity::fileName](#), [ImportRecord::fileName](#), [ImportJob::filePath](#), [Activity::fitHash](#), [RideRecord::hasAltitude](#), [RideRecord::hasCadence](#), [ActivityRepository::hasFitHash\(\)](#), [RideRecord::hasGps](#), [RideRecord::hasHeartRate](#), [RideRecord::hasPower](#), [RideRecord::hasSpeed](#), [RideRecord::heartRate](#), [Activity::importedAt](#), [ImportJob::importSource](#), [ActivityRepository::insertActivity\(\)](#), [ImportJob::jobId](#), [RideRecord::latitude](#), [FitParser::load\(\)](#), [ImportRepository::logImport\(\)](#), [RideRecord::longitude](#), [Activity::maxHr](#), [WorkoutStatistics::maximumHeartRate](#), [WorkoutStatistics::maximumPower](#), [Activity::maxPower](#), [Activity::normalizedPower](#), [TrainingLoad::normalizedPower\(\)](#), [RideRecord::power](#), [progressChanged\(\)](#), [RideData::records](#), [ActivityRepository::setImportSource\(\)](#), [RideRecord::speed](#), [Activity::sport](#), [Activity::startTime](#), [ImportRecord::status](#), [WorkoutStatistics::totalDistanceKm](#), and [ActivityRepository::updateFingerprint](#).

Here is the call graph for this function:



7.56.3.3 setDatabasePath()

```
void ImportWorker::setDatabasePath (
```

```
const QString & path)
```

Sets the SQLite database path used for imports.

Definition at line 39 of file [ImportWorker.cpp](#).

7.56.4 Member Data Documentation

7.56.4.1 m_dbPath

```
QString ImportWorker::m_dbPath [private]
```

Definition at line 54 of file [ImportWorker.h](#).

The documentation for this class was generated from the following files:

- [src/import/ImportWorker.h](#)
- [src/import/ImportWorker.cpp](#)

7.57 Interval Struct Reference

Represents a single work or recovery interval.

```
#include <IntervalDetector.h>
```

Public Attributes

- double [startSeconds](#) = 0.0
Interval start time in seconds from activity start.
- double [endSeconds](#) = 0.0
Interval end time in seconds from activity start.
- double [durationSeconds](#) = 0.0
Interval duration in seconds.
- double [averagePower](#) = 0.0
Average power in watts.
- double [maximumPower](#) = 0.0
Maximum power in watts.
- double [normalizedPower](#) = 0.0
Normalized power in watts.
- double [averageHeartRate](#) = 0.0
Average heart rate in bpm.
- double [averageCadence](#) = 0.0
Average cadence in rpm.
- double [averageSpeed](#) = 0.0
Average speed in km/h.
- std::string [label](#)
Interval type label (Work or Recovery).

7.57.1 Detailed Description

Represents a single work or recovery interval.

Definition at line 14 of file [IntervalDetector.h](#).

7.57.2 Member Data Documentation

7.57.2.1 averageCadence

```
double Interval::averageCadence = 0.0
```

Average cadence in rpm.

Definition at line 38 of file [IntervalDetector.h](#).

Referenced by [IntervalDetector::removeOverlappingAndDuplicates\(\)](#).

7.57.2.2 averageHeartRate

```
double Interval::averageHeartRate = 0.0
```

Average heart rate in bpm.

Definition at line 35 of file [IntervalDetector.h](#).

Referenced by [IntervalDetector::removeOverlappingAndDuplicates\(\)](#).

7.57.2.3 averagePower

```
double Interval::averagePower = 0.0
```

Average power in watts.

Definition at line 26 of file [IntervalDetector.h](#).

Referenced by [IntervalDetector::removeOverlappingAndDuplicates\(\)](#).

7.57.2.4 averageSpeed

```
double Interval::averageSpeed = 0.0
```

Average speed in km/h.

Definition at line 41 of file [IntervalDetector.h](#).

Referenced by [IntervalDetector::removeOverlappingAndDuplicates\(\)](#).

7.57.2.5 durationSeconds

```
double Interval::durationSeconds = 0.0
```

[Interval](#) duration in seconds.

Definition at line 23 of file [IntervalDetector.h](#).

Referenced by [IntervalDetector::removeOverlappingAndDuplicates\(\)](#).

7.57.2.6 endSeconds

```
double Interval::endSeconds = 0.0
```

[Interval](#) end time in seconds from activity start.

Definition at line 20 of file [IntervalDetector.h](#).

Referenced by [IntervalDetector::removeOverlappingAndDuplicates\(\)](#).

7.57.2.7 label

```
std::string Interval::label
```

[Interval](#) type label (Work or Recovery).

Definition at line 44 of file [IntervalDetector.h](#).

7.57.2.8 maximumPower

```
double Interval::maximumPower = 0.0
```

Maximum power in watts.

Definition at line 29 of file [IntervalDetector.h](#).

Referenced by [IntervalDetector::removeOverlappingAndDuplicates\(\)](#).

7.57.2.9 normalizedPower

```
double Interval::normalizedPower = 0.0
```

Normalized power in watts.

Definition at line 32 of file [IntervalDetector.h](#).

Referenced by [IntervalDetector::removeOverlappingAndDuplicates\(\)](#).

7.57.2.10 startSeconds

```
double Interval::startSeconds = 0.0
```

[Interval](#) start time in seconds from activity start.

Definition at line 17 of file [IntervalDetector.h](#).

Referenced by [IntervalDetector::removeOverlappingAndDuplicates\(\)](#).

The documentation for this struct was generated from the following file:

- `src/analysis/IntervalDetector.h`

7.58 IntervalDetector Class Reference

Detects work and recovery intervals in activity data.

```
#include <IntervalDetector.h>
```

Public Types

- using [Config](#) = [IntervalDetectorConfig](#)
Interval detection configuration structure.

Static Public Member Functions

- static `std::vector< Interval > detect` (const [RideData](#) &rideData, const [Config](#) &config=[Config](#){})
Detects intervals in activity data.
- static `std::vector< Interval > removeOverlappingAndDuplicates` (std::vector< [Interval](#) > intervals)
Removes duplicate and overlapping intervals.

Static Private Member Functions

- static `std::vector< double > smoothPower` (const [RideData](#) &rideData, int window)
- static [Interval](#) `buildInterval` (const [RideData](#) &rideData, int startIdx, int endIdx, const std::string &label)

7.58.1 Detailed Description

Detects work and recovery intervals in activity data.

Identifies distinct work intervals (above power threshold) and recovery periods based on configurable FTP-derived thresholds.

Definition at line 74 of file [IntervalDetector.h](#).

7.58.2 Member Typedef Documentation

7.58.2.1 Config

using [IntervalDetector::Config](#) = [IntervalDetectorConfig](#)
[Interval](#) detection configuration structure.

Definition at line 78 of file [IntervalDetector.h](#).

7.58.3 Member Function Documentation

7.58.3.1 buildInterval()

```
Interval IntervalDetector::buildInterval (
    const RideData & rideData,
    int startIdx,
    int endIdx,
    const std::string & label) [static], [private]
```

Definition at line 34 of file [IntervalDetector.cpp](#).

7.58.3.2 detect()

```
std::vector< Interval > IntervalDetector::detect (
    const RideData & rideData,
    const Config & config = Config{}) [static]
```

Detects intervals in activity data.

Parameters

<i>rideData</i>	Activity recording data.
<i>config</i>	Detection parameters.

Returns

List of detected intervals.

Definition at line 92 of file [IntervalDetector.cpp](#).

References [IntervalDetectorConfig::ftp](#), [IntervalDetectorConfig::minRecoverySeconds](#), [IntervalDetectorConfig::minWorkSeconds](#), [RideData::records](#), [IntervalDetectorConfig::smoothingWindow](#), and [IntervalDetectorConfig::workThresholdPct](#).

Referenced by [AnalysisWorker::processTask\(\)](#).

Here is the caller graph for this function:



7.58.3.3 removeOverlappingAndDuplicates()

```
std::vector< Interval > IntervalDetector::removeOverlappingAndDuplicates (
    std::vector< Interval > intervals) [static]
```

Removes duplicate and overlapping intervals.

Parameters

<i>intervals</i>	List of intervals to clean.
------------------	-----------------------------

Returns

Cleaned list (within 1 second tolerance).

Definition at line 146 of file [IntervalDetector.cpp](#).

References [Interval::averageCadence](#), [Interval::averageHeartRate](#), [Interval::averagePower](#), [Interval::averageSpeed](#), [Interval::durationSeconds](#), [Interval::endSeconds](#), [Interval::maximumPower](#), [Interval::normalizedPower](#), and [Interval::startSeconds](#).

7.58.3.4 smoothPower()

```
std::vector< double > IntervalDetector::smoothPower (
    const RideData & rideData,
    int window) [static], [private]
```

Definition at line 9 of file [IntervalDetector.cpp](#).

The documentation for this class was generated from the following files:

- [src/analysis/IntervalDetector.h](#)
- [src/analysis/IntervalDetector.cpp](#)

7.59 IntervalDetectorConfig Struct Reference

Configuration parameters for interval detection.

```
#include <IntervalDetector.h>
```

Public Attributes

- double `ftp` = 250.0
Functional threshold power in watts.
- double `workThresholdPct` = 0.76
Work threshold as fraction of FTP.
- double `minWorkSeconds` = 60.0
Minimum work interval duration in seconds.
- double `minRecoverySeconds` = 20.0
Minimum recovery interval duration in seconds.
- int `smoothingWindow` = 30
Rolling average window width in samples.

7.59.1 Detailed Description

Configuration parameters for interval detection.

Definition at line 50 of file [IntervalDetector.h](#).

7.59.2 Member Data Documentation

7.59.2.1 ftp

```
double IntervalDetectorConfig::ftp = 250.0
```

Functional threshold power in watts.

Definition at line 53 of file [IntervalDetector.h](#).

Referenced by [IntervalDetector::detect\(\)](#), and [AnalysisWorker::processTask\(\)](#).

7.59.2.2 minRecoverySeconds

```
double IntervalDetectorConfig::minRecoverySeconds = 20.0
```

Minimum recovery interval duration in seconds.

Definition at line 62 of file [IntervalDetector.h](#).

Referenced by [IntervalDetector::detect\(\)](#).

7.59.2.3 minWorkSeconds

```
double IntervalDetectorConfig::minWorkSeconds = 60.0
```

Minimum work interval duration in seconds.

Definition at line 59 of file [IntervalDetector.h](#).

Referenced by [IntervalDetector::detect\(\)](#).

7.59.2.4 smoothingWindow

```
int IntervalDetectorConfig::smoothingWindow = 30
```

Rolling average window width in samples.

Definition at line 65 of file [IntervalDetector.h](#).

Referenced by [IntervalDetector::detect\(\)](#).

7.59.2.5 workThresholdPct

```
double IntervalDetectorConfig::workThresholdPct = 0.76
```

Work threshold as fraction of FTP.

Definition at line 56 of file [IntervalDetector.h](#).

Referenced by [IntervalDetector::detect\(\)](#).

The documentation for this struct was generated from the following file:

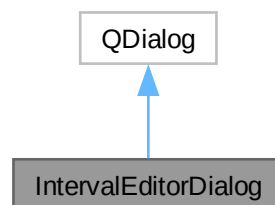
- `src/analysis/IntervalDetector.h`

7.60 IntervalEditorDialog Class Reference

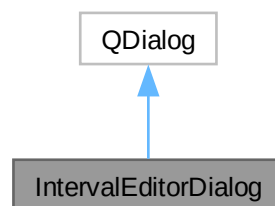
Dialog for editing interval metadata.

```
#include <IntervalEditorDialog.h>
```

Inheritance diagram for IntervalEditorDialog:



Collaboration diagram for IntervalEditorDialog:



Public Member Functions

- [IntervalEditorDialog](#) (double startSeconds, double endSeconds, QWidget *parent=nullptr)
Constructs interval editor dialog.
- QString [intervalName](#) () const
Gets entered interval name.
- QString [intervalType](#) () const
Gets entered interval type.
- QString [intervalNotes](#) () const
Gets entered interval notes.

Private Attributes

- QLineEdit * [m_nameEdit](#) = nullptr
- QLineEdit * [m_typeEdit](#) = nullptr
- QTextEdit * [m_notesEdit](#) = nullptr
- QDoubleSpinBox * [m_startSpin](#) = nullptr
- QDoubleSpinBox * [m_endSpin](#) = nullptr

7.60.1 Detailed Description

Dialog for editing interval metadata.

Allows entry of interval name, type, and notes.

Definition at line 18 of file [IntervalEditorDialog.h](#).

7.60.2 Constructor & Destructor Documentation**7.60.2.1 IntervalEditorDialog()**

```
IntervalEditorDialog::IntervalEditorDialog (
    double startSeconds,
    double endSeconds,
    QWidget * parent = nullptr) [explicit]
```

Constructs interval editor dialog.

Parameters

<i>startSeconds</i>	Start time in seconds.
<i>endSeconds</i>	End time in seconds.
<i>parent</i>	Parent widget.

Definition at line 13 of file [IntervalEditorDialog.cpp](#).

7.60.3 Member Function Documentation**7.60.3.1 intervalName()**

```
QString IntervalEditorDialog::intervalName () const
```

Gets entered interval name.

Returns

[Interval](#) name.

Definition at line 54 of file [IntervalEditorDialog.cpp](#).

7.60.3.2 intervalNotes()

```
QString IntervalEditorDialog::intervalNotes () const
```

Gets entered interval notes.

Returns

Notes text.

Definition at line 64 of file [IntervalEditorDialog.cpp](#).

7.60.3.3 intervalType()

```
QString IntervalEditorDialog::intervalType () const
```

Gets entered interval type.

Returns

[Interval](#) type.

Definition at line 59 of file [IntervalEditorDialog.cpp](#).

7.60.4 Member Data Documentation

7.60.4.1 m_endSpin

```
QDoubleSpinBox* IntervalEditorDialog::m_endSpin = nullptr [private]
```

Definition at line 56 of file [IntervalEditorDialog.h](#).

7.60.4.2 m_nameEdit

```
QLineEdit* IntervalEditorDialog::m_nameEdit = nullptr [private]
```

Definition at line 52 of file [IntervalEditorDialog.h](#).

7.60.4.3 m_notesEdit

```
QTextEdit* IntervalEditorDialog::m_notesEdit = nullptr [private]
```

Definition at line 54 of file [IntervalEditorDialog.h](#).

7.60.4.4 m_startSpin

```
QDoubleSpinBox* IntervalEditorDialog::m_startSpin = nullptr [private]
```

Definition at line 55 of file [IntervalEditorDialog.h](#).

7.60.4.5 m_typeEdit

```
QLineEdit* IntervalEditorDialog::m_typeEdit = nullptr [private]
```

Definition at line 53 of file [IntervalEditorDialog.h](#).

The documentation for this class was generated from the following files:

- [src/gui/IntervalEditorDialog.h](#)
- [src/gui/IntervalEditorDialog.cpp](#)

7.61 IntervalRecord Struct Reference

Database representation of an interval record.

```
#include <IntervalRepository.h>
```

Public Attributes

- int [id](#) = -1
Unique interval record ID.
- int [activityId](#) = -1
Associated activity ID.
- int [startSample](#) = 0
Starting sample index.
- int [endSample](#) = 0
Ending sample index.
- QString [name](#)
Interval name.
- QString [type](#)
Interval type (Work, Recovery, etc.).
- double [avgPower](#) = 0.0
Average power in watts.

- double `np` = 0.0
Normalized power in watts.
- double `avgHr` = 0.0
Average heart rate in bpm.
- double `avgCadence` = 0.0
Average cadence in rpm.
- QString `notes`
User notes about interval.
- QString `source` = `QStringLiteral("auto")`
Source of interval (auto, manual, edited).
- bool `locked` = false
Whether interval is locked from editing.
- bool `deleted` = false
Soft-delete flag.
- int `algorithmVersion` = 1
Algorithm version used for detection.
- QString `uuid`
Stable UUID, generated once on insert.

7.61.1 Detailed Description

Database representation of an interval record.
Definition at line 14 of file [IntervalRepository.h](#).

7.61.2 Member Data Documentation

7.61.2.1 `activityId`

```
int IntervalRecord::activityId = -1
```

Associated activity ID.

Definition at line 20 of file [IntervalRepository.h](#).

Referenced by [IntervalRepository::insertInterval\(\)](#), [IntervalRepository::listIntervalsForActivity\(\)](#), and [AnalysisWorker::processTask\(\)](#).

7.61.2.2 `algorithmVersion`

```
int IntervalRecord::algorithmVersion = 1
```

Algorithm version used for detection.

Definition at line 59 of file [IntervalRepository.h](#).

Referenced by [IntervalRepository::insertInterval\(\)](#), [IntervalRepository::listIntervalsForActivity\(\)](#), and [AnalysisWorker::processTask\(\)](#).

7.61.2.3 `avgCadence`

```
double IntervalRecord::avgCadence = 0.0
```

Average cadence in rpm.

Definition at line 44 of file [IntervalRepository.h](#).

Referenced by [IntervalRepository::insertInterval\(\)](#), [IntervalRepository::listIntervalsForActivity\(\)](#), [AnalysisWorker::processTask\(\)](#), and [IntervalRepository::updateInterval\(\)](#).

7.61.2.4 `avgHr`

```
double IntervalRecord::avgHr = 0.0
```

Average heart rate in bpm.

Definition at line 41 of file [IntervalRepository.h](#).

Referenced by [IntervalRepository::insertInterval\(\)](#), [IntervalRepository::listIntervalsForActivity\(\)](#), [AnalysisWorker::processTask\(\)](#), and [IntervalRepository::updateInterval\(\)](#).

7.61.2.5 avgPower

```
double IntervalRecord::avgPower = 0.0
```

Average power in watts.

Definition at line 35 of file [IntervalRepository.h](#).

Referenced by [IntervalRepository::insertInterval\(\)](#), [IntervalRepository::listIntervalsForActivity\(\)](#), [AnalysisWorker::processTask\(\)](#), and [IntervalRepository::updateInterval\(\)](#).

7.61.2.6 deleted

```
bool IntervalRecord::deleted = false
```

Soft-delete flag.

Definition at line 56 of file [IntervalRepository.h](#).

Referenced by [IntervalRepository::insertInterval\(\)](#), [IntervalRepository::listIntervalsForActivity\(\)](#), [AnalysisWorker::processTask\(\)](#), and [IntervalRepository::updateInterval\(\)](#).

7.61.2.7 endSample

```
int IntervalRecord::endSample = 0
```

Ending sample index.

Definition at line 26 of file [IntervalRepository.h](#).

Referenced by [IntervalRepository::insertInterval\(\)](#), [IntervalRepository::listIntervalsForActivity\(\)](#), [AnalysisWorker::processTask\(\)](#), and [IntervalRepository::updateInterval\(\)](#).

7.61.2.8 id

```
int IntervalRecord::id = -1
```

Unique interval record ID.

Definition at line 17 of file [IntervalRepository.h](#).

Referenced by [IntervalRepository::listIntervalsForActivity\(\)](#), and [IntervalRepository::updateInterval\(\)](#).

7.61.2.9 locked

```
bool IntervalRecord::locked = false
```

Whether interval is locked from editing.

Definition at line 53 of file [IntervalRepository.h](#).

Referenced by [IntervalRepository::insertInterval\(\)](#), [IntervalRepository::listIntervalsForActivity\(\)](#), [AnalysisWorker::processTask\(\)](#), and [IntervalRepository::updateInterval\(\)](#).

7.61.2.10 name

```
QString IntervalRecord::name
```

[Interval](#) name.

Definition at line 29 of file [IntervalRepository.h](#).

Referenced by [IntervalRepository::insertInterval\(\)](#), [IntervalRepository::listIntervalsForActivity\(\)](#), [AnalysisWorker::processTask\(\)](#), and [IntervalRepository::updateInterval\(\)](#).

7.61.2.11 notes

```
QString IntervalRecord::notes
```

User notes about interval.

Definition at line 47 of file [IntervalRepository.h](#).

Referenced by [IntervalRepository::insertInterval\(\)](#), [IntervalRepository::listIntervalsForActivity\(\)](#), and [IntervalRepository::updateInterval\(\)](#).

7.61.2.12 np

```
double IntervalRecord::np = 0.0
```

Normalized power in watts.

Definition at line 38 of file [IntervalRepository.h](#).

Referenced by [IntervalRepository::insertInterval\(\)](#), [IntervalRepository::listIntervalsForActivity\(\)](#), [AnalysisWorker::processTask\(\)](#), and [IntervalRepository::updateInterval\(\)](#).

7.61.2.13 source

```
QString IntervalRecord::source = QStringLiteral("auto")
```

Source of interval (auto, manual, edited).

Definition at line 50 of file [IntervalRepository.h](#).

Referenced by [IntervalRepository::insertInterval\(\)](#), [IntervalRepository::listIntervalsForActivity\(\)](#), [AnalysisWorker::processTask\(\)](#), and [IntervalRepository::updateInterval\(\)](#).

7.61.2.14 startSample

```
int IntervalRecord::startSample = 0
```

Starting sample index.

Definition at line 23 of file [IntervalRepository.h](#).

Referenced by [IntervalRepository::insertInterval\(\)](#), [IntervalRepository::listIntervalsForActivity\(\)](#), [AnalysisWorker::processTask\(\)](#), and [IntervalRepository::updateInterval\(\)](#).

7.61.2.15 type

```
QString IntervalRecord::type
```

[Interval](#) type (Work, Recovery, etc.).

Definition at line 32 of file [IntervalRepository.h](#).

Referenced by [IntervalRepository::insertInterval\(\)](#), [IntervalRepository::listIntervalsForActivity\(\)](#), [AnalysisWorker::processTask\(\)](#), and [IntervalRepository::updateInterval\(\)](#).

7.61.2.16 uuid

```
QString IntervalRecord::uuid
```

Stable UUID, generated once on insert.

Definition at line 62 of file [IntervalRepository.h](#).

Referenced by [IntervalRepository::insertInterval\(\)](#), [IntervalRepository::listIntervalsForActivity\(\)](#), and [AnalysisWorker::processTask\(\)](#).

The documentation for this struct was generated from the following file:

- [src/database/IntervalRepository.h](#)

7.62 IntervalRepository Class Reference

Repository for interval record persistence.

```
#include <IntervalRepository.h>
```

Public Member Functions

- [IntervalRepository](#) (QSqlDatabase &db)
Constructs interval repository.
- int [insertInterval](#) (const [IntervalRecord](#) &interval)
Inserts a new interval record.
- bool [updateInterval](#) (const [IntervalRecord](#) &interval)
Updates an existing interval record.
- bool [softDeleteInterval](#) (int intervalId)
Soft-delete: marks deleted=1.
- bool [deleteInterval](#) (int intervalId)
Hard-delete (for full reset before re-detect).
- bool [deleteIntervalsForActivity](#) (int activityId)
Deletes all intervals for an activity.
- QList< [IntervalRecord](#) > [listIntervalsForActivity](#) (int activityId)

Returns non-deleted intervals only.

- bool [hasLockedIntervals](#) (int activityId)

Checks if activity has locked intervals.

Private Attributes

- QSqlDatabase & [m_db](#)

7.62.1 Detailed Description

Repository for interval record persistence.

Provides CRUD operations and queries for work/recovery intervals in the database.

Definition at line 70 of file [IntervalRepository.h](#).

7.62.2 Constructor & Destructor Documentation

7.62.2.1 IntervalRepository()

```
IntervalRepository::IntervalRepository (
    QSqlDatabase & db) [explicit]
```

Constructs interval repository.

Parameters

<i>db</i>	SQL database reference.
-----------	-------------------------

Definition at line 10 of file [IntervalRepository.cpp](#).

7.62.3 Member Function Documentation

7.62.3.1 deleteInterval()

```
bool IntervalRepository::deleteInterval (
    int intervalId)
```

Hard-delete (for full reset before re-detect).

Definition at line 92 of file [IntervalRepository.cpp](#).

7.62.3.2 deleteIntervalsForActivity()

```
bool IntervalRepository::deleteIntervalsForActivity (
    int activityId)
```

Deletes all intervals for an activity.

Parameters

<i>activityId</i>	Activity ID.
-------------------	------------------------------

Returns

True if deletion succeeded.

Definition at line 101 of file [IntervalRepository.cpp](#).

7.62.3.3 hasLockedIntervals()

```
bool IntervalRepository::hasLockedIntervals (
    int activityId)
```

Checks if activity has locked intervals.

Parameters

<i>activity</i> <i>Id</i>	Activity ID.
------------------------------	------------------------------

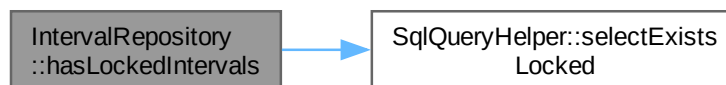
Returns

True if locked intervals exist.

Definition at line 148 of file [IntervalRepository.cpp](#).

References [SqlQueryHelper::selectExistsLocked\(\)](#).

Here is the call graph for this function:



7.62.3.4 insertInterval()

```
int IntervalRepository::insertInterval (
    const IntervalRecord & interval)
```

Inserts a new interval record.

Parameters

<i>interval</i>	Interval to insert.
-----------------	-------------------------------------

Returns

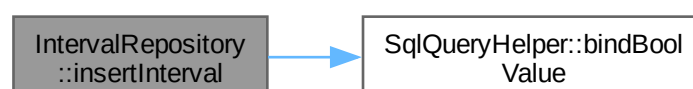
New interval ID, or -1 on error.

Definition at line 14 of file [IntervalRepository.cpp](#).

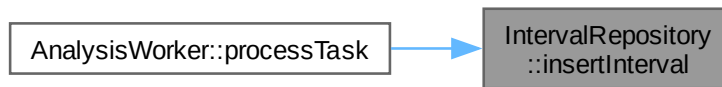
References [IntervalRecord::activityId](#), [IntervalRecord::algorithmVersion](#), [IntervalRecord::avgCadence](#), [IntervalRecord::avgHr](#), [IntervalRecord::avgPower](#), [SqlQueryHelper::bindBoolValue\(\)](#), [IntervalRecord::deleted](#), [IntervalRecord::endSample](#), [IntervalRecord::locked](#), [IntervalRecord::name](#), [IntervalRecord::notes](#), [IntervalRecord::np](#), [IntervalRecord::source](#), [IntervalRecord::startSample](#), [IntervalRecord::type](#), and [IntervalRecord::uuid](#).

Referenced by [AnalysisWorker::processTask\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.62.3.5 listIntervalsForActivity()

```
QList< IntervalRecord > IntervalRepository::listIntervalsForActivity (
    int activityId)
```

Returns non-deleted intervals only.

Definition at line 109 of file [IntervalRepository.cpp](#).

References [IntervalRecord::activityId](#), [IntervalRecord::algorithmVersion](#), [IntervalRecord::avgCadence](#), [IntervalRecord::avgHr](#), [IntervalRecord::avgPower](#), [IntervalRecord::deleted](#), [IntervalRecord::endSample](#), [IntervalRecord::id](#), [IntervalRecord::locked](#), [IntervalRecord::name](#), [IntervalRecord::notes](#), [IntervalRecord::np](#), [IntervalRecord::source](#), [IntervalRecord::startSample](#), [IntervalRecord::type](#), and [IntervalRecord::uuid](#).

Referenced by [AnalysisWorker::processTask\(\)](#).

Here is the caller graph for this function:



7.62.3.6 softDeleteInterval()

```
bool IntervalRepository::softDeleteInterval (
    int intervalId)
```

Soft-delete: marks deleted=1.

Definition at line 84 of file [IntervalRepository.cpp](#).

7.62.3.7 updateInterval()

```
bool IntervalRepository::updateInterval (
    const IntervalRecord & interval)
```

Updates an existing interval record.

Parameters

<i>interval</i>	Interval with updated data.
-----------------	---

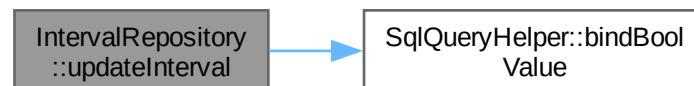
Returns

True if update succeeded.

Definition at line 48 of file [IntervalRepository.cpp](#).

References [IntervalRecord::avgCadence](#), [IntervalRecord::avgHr](#), [IntervalRecord::avgPower](#), [SqlQueryHelper::bindBoolValue\(\)](#), [IntervalRecord::deleted](#), [IntervalRecord::endSample](#), [IntervalRecord::id](#), [IntervalRecord::locked](#), [IntervalRecord::name](#), [IntervalRecord::notes](#), [IntervalRecord::np](#), [IntervalRecord::source](#), [IntervalRecord::startSample](#), and [IntervalRecord::type](#).

Here is the call graph for this function:

**7.62.4 Member Data Documentation****7.62.4.1 m_db**

`QSqlDatabase& IntervalRepository::m_db [private]`

Definition at line 116 of file [IntervalRepository.h](#).

The documentation for this class was generated from the following files:

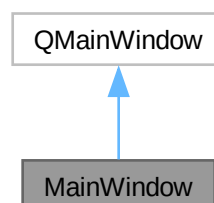
- `src/database/IntervalRepository.h`
- `src/database/IntervalRepository.cpp`

7.63 MainWindow Class Reference

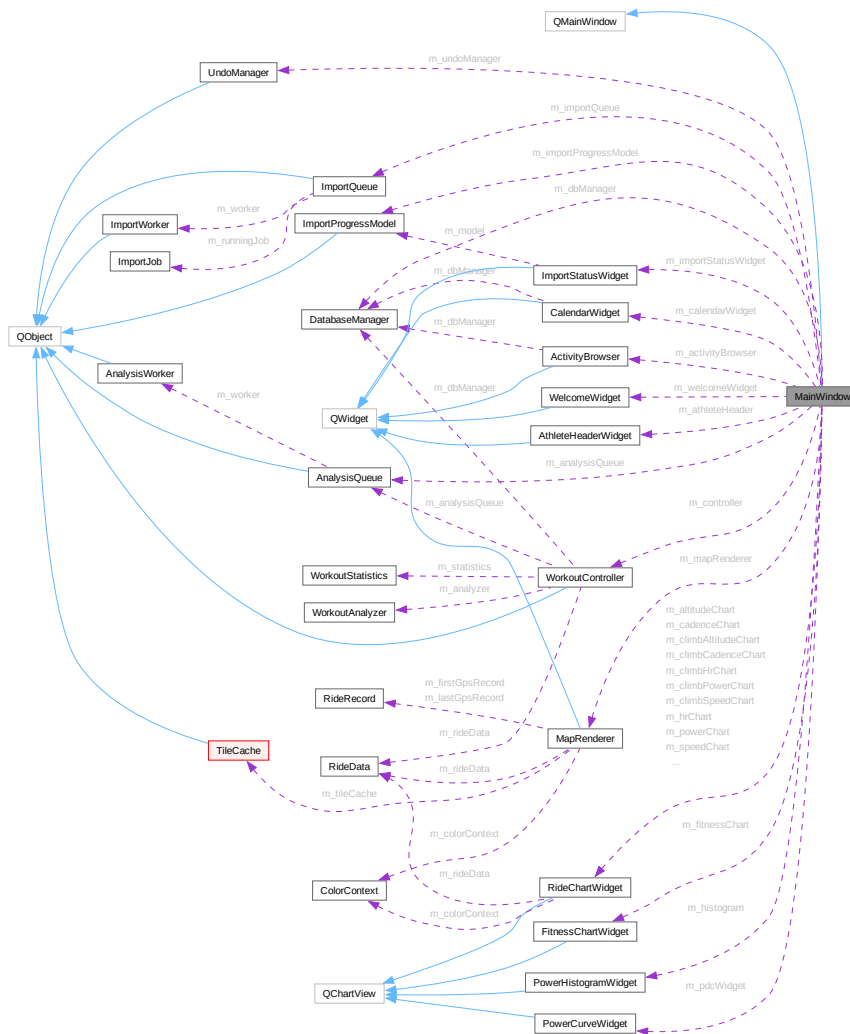
Main application window.

```
#include <MainWindow.h>
```

Inheritance diagram for MainWindow:



Collaboration diagram for MainWindow:



Classes

- struct [ImportBatchSummary](#)

Public Member Functions

- [MainWindow](#) (QWidget *parent=nullptr)
Constructs main window.

Protected Member Functions

- void [closeEvent](#) (QCloseEvent *event) override
Handles application close event.
- bool [eventFilter](#) (QObject *watched, QEvent *event) override
Filters events for custom handling.
- void [dragEnterEvent](#) (QDragEnterEvent *event) override
Accepts drag-enter when the payload contains local .fit files.
- void [dropEvent](#) (QDropEvent *event) override
Extracts dropped .fit files and forwards them to the import queue.

Private Slots

- void [importActivities](#) ()
- void [previewFitFile](#) ()
- void [onWorkoutLoaded](#) ()
- void [onActivityImported](#) (int activityId)
- void [openDatabase](#) ()
- void [createDatabase](#) ()
- void [manageAthletes](#) ()
- void [backupDatabase](#) ()
- void [openSettingsDialog](#) ()
- void [openRecentDatabase](#) ()
- void [onAthleteSelectionChanged](#) (int index)
- void [createVideo](#) ()
- void [showAbout](#) ()
- void [triggerDetectClimbs](#) ()
- void [triggerDetectIntervals](#) ()
- void [revertSelectedClimbToAuto](#) ()

Private Member Functions

- void [buildUI](#) ()
- void [buildToolBar](#) ()
- void [buildStatusBar](#) ()
- void [setupShortcuts](#) ()
- void [saveSettings](#) ()
- void [loadSettings](#) ()
- void [addRecentDatabase](#) (const QString &path)
- void [updateRecentDatabaseMenu](#) ()
- void [openDatabasePath](#) (const QString &path)
- bool [canImportActivities](#) () const
Returns whether importing is currently allowed.
- void [updateImportAvailability](#) ()
Enables/disables import actions based on current readiness.
- void [importFiles](#) (const QStringList &filePaths)
Queues a batch of file paths for background import.
- QStringList [fitFilesFromMimeData](#) (const QMimeData *mimeData) const
Returns deduplicated local .fit files from MIME drop/paste data.
- void [scheduleActivityBrowserRefresh](#) ()
- void [finalizeImportBatches](#) ()
Shows a per-batch import result summary when queue work completes.
- void [refreshAthleteSelector](#) ()
- void [resetAllZoom](#) ()
- void [increaseChartHeight](#) ()
- void [decreaseChartHeight](#) ()
- void [applyChartHeight](#) ()
- void [updateStatsLabel](#) ()
- void [updateCharts](#) ()
- void [applyChartSmoothing](#) ()
- void [updateZonesTab](#) ()
- void [updateHistogram](#) ()
- void [updatePowerCurve](#) ()
- void [updateFitnessChart](#) ()
- void [updateAnalysisEmptyStates](#) ()
- void [updateIntervals](#) ()

- void [updateClimbingTab](#) ()
- void [detectClimbsAndRefresh](#) ()
- void [refreshClimbViews](#) (double preferredStartSeconds=-1.0, double preferredEndSeconds=-1.0)
- void [updateClimbQuarterCharts](#) (const [Climb](#) *climb)
- double [resolveActiveActivityWeightKg](#) () const
- void [assignClimbWeightMetrics](#) ([Climb](#) &climb, double riderWeightKg) const
- void [applyWeightMetricsToClimbs](#) (std::vector< [Climb](#) > &climbs, double riderWeightKg) const
- std::vector< int > [selectedClimbIndicesFromTable](#) () const
- void [onClimbTableContextMenuRequested](#) (const QPoint &pos)
- void [removeSelectedClimbs](#) ()
- void [editSelectedClimbBoundaries](#) ()
- void [joinSelectedClimbs](#) ()
- void [updateClimbRowStyles](#) ()
- void [onClimbSelectionChanged](#) ()
- void [onClimbRowDoubleClicked](#) (QTableWidgetItem *item)
- void [onClimbBoundaryEdited](#) (double oldStartSeconds, double oldEndSeconds, double newStartSeconds, double newEndSeconds)
- void [onNewClimbRequested](#) (double startSeconds, double endSeconds)
- void [onIntervalSelectionChanged](#) ()
- void [onIntervalRowDoubleClicked](#) (QTableWidgetItem *item)
- void [navigateToInterval](#) (double startSeconds, double endSeconds, bool exactZoom)
- void [selectIntervalRowOffset](#) (int delta)
- void [updateIntervalRowStyles](#) ()
- void [updateIntervalSummary](#) (double startSeconds, double endSeconds)
- void [clearIntervalSummary](#) ()
- bool [ensureDatabase](#) ()
- bool [ensureAthlete](#) ()
- bool [ensureImportReady](#) ()
 - *Ensures database path, athlete selection, and queue state are import-ready.*
- bool [createOrOpenDefaultDatabase](#) ()
- QString [defaultDatabasePathForAutoCreate](#) () const
- void [showWelcomeScreen](#) ()
- void [hideWelcomeScreen](#) ()
- void [updateWelcomeScreenVisibility](#) ()
- int [activityCount](#) () const
- void [updateAthleteLabel](#) ()
- void [updateStatusBarInfo](#) ()
- ColorMetric [currentColorMetric](#) () const
- [ColorContext](#) [buildColorContext](#) () const
- void [updateColorLegend](#) ()
- void [updateZoneAvailability](#) ()
- void [applyChartPreset](#) (int presetId)
- void [editCurrentActivityProperties](#) ()
- double [estimatedFtpFromCurrentRide](#) () const
- void [applyEstimatedFtpForCurrentAthlete](#) ()
- void [importFilesInternal](#) (const QStringList &filePaths, bool showResultDialog, const QString &sourceLabel)
 - *Internal import entry point shared by menu action and drag-drop.*
- void [configureFolderWatcher](#) ()
- QStringList [monitoredDirectories](#) () const
- void [scanWatchDirectories](#) (bool initialScan)
- void [onWatchDirectoryChanged](#) (const QString &path)

Private Attributes

- [UndoManager](#) * [m_undoManager](#) = nullptr
- [AnalysisQueue](#) * [m_analysisQueue](#) = nullptr
- [ImportQueue](#) * [m_importQueue](#) = nullptr
- [ImportProgressModel](#) * [m_importProgressModel](#) = nullptr
- [WorkoutController](#) * [m_controller](#) = nullptr
- [DatabaseManager](#) [m_dbManager](#)
- int [m_currentAthleteId](#) = -1
- QString [m_currentAthleteName](#)
- QMenu * [m_athletesMenu](#) = nullptr
- QAction * [m_athleteLabelAct](#) = nullptr
- QAction * [m_importAct](#) = nullptr
- QAction * [m_toolbarImportAct](#) = nullptr
- QAction * [m_toolbarSearchAct](#) = nullptr
- QAction * [m_previewAct](#) = nullptr
- QMenu * [m_recentDbMenu](#) = nullptr
- QString [m_lastOpenDir](#)
- [WelcomeWidget](#) * [m_welcomeWidget](#) = nullptr
- bool [m_firstLaunchCompleted](#) = false
- QFileSystemWatcher * [m_watcher](#) = nullptr
- QTimer * [m_watchRescanTimer](#) = nullptr
- QTimer * [m_importRefreshTimer](#) = nullptr
- bool [m_watchFolderEnabled](#) = true
- QString [m_watchFolderPath](#)
- QSet< QString > [m_knownWatchedFitFiles](#)
- QComboBox * [m_athleteCombo](#) = nullptr
- QComboBox * [m_colorMetricCombo](#) = nullptr
- QComboBox * [m_chartPresetCombo](#) = nullptr
- QComboBox * [m_powerSmoothingCombo](#) = nullptr
- QCheckBox * [m_autoSmoothingCheck](#) = nullptr
- [ImportStatusWidget](#) * [m_importStatusWidget](#) = nullptr
- QLabel * [m_dbStatusLabel](#) = nullptr
- QLabel * [m_athleteStatusLabel](#) = nullptr
- QLabel * [m_activityCountLabel](#) = nullptr
- [AthleteHeaderWidget](#) * [m_athleteHeader](#) = nullptr
- QTabWidget * [m_tabWidget](#) = nullptr
- QTabWidget * [m_analysisTabWidget](#) = nullptr
- QStackedLayout * [m_activityTabStack](#) = nullptr
- QStackedLayout * [m_zonesTabStack](#) = nullptr
- QStackedLayout * [m_histogramTabStack](#) = nullptr
- QStackedLayout * [m_powerCurveTabStack](#) = nullptr
- QStackedLayout * [m_calendarTabStack](#) = nullptr
- QStackedLayout * [m_fitnessTabStack](#) = nullptr
- QStackedLayout * [m_climbingTabStack](#) = nullptr
- QScrollArea * [m_chartScroll](#) = nullptr
- QWidget * [m_chartsContainer](#) = nullptr
- QVBoxLayout * [m_chartsLayout](#) = nullptr
- QSplitter * [m_chartMapSplit](#) = nullptr
- int [m_chartHeight](#) = 220
- [RideChartWidget](#) * [m_powerChart](#) = nullptr
- [RideChartWidget](#) * [m_hrChart](#) = nullptr
- [RideChartWidget](#) * [m_cadenceChart](#) = nullptr
- [RideChartWidget](#) * [m_speedChart](#) = nullptr
- [RideChartWidget](#) * [m_altitudeChart](#) = nullptr

- [RideChartWidget](#) * [m_climbAltitudeChart](#) = nullptr
- [RideChartWidget](#) * [m_climbPowerChart](#) = nullptr
- [RideChartWidget](#) * [m_climbHrChart](#) = nullptr
- [RideChartWidget](#) * [m_climbCadenceChart](#) = nullptr
- [RideChartWidget](#) * [m_climbSpeedChart](#) = nullptr
- [QChartView](#) * [m_climbQuarterPowerChart](#) = nullptr
- [QChartView](#) * [m_climbQuarterHrChart](#) = nullptr
- [QChartView](#) * [m_climbQuarterCadenceChart](#) = nullptr
- [QPushButton](#) * [m_fitChartsButton](#) = nullptr
- [QPushButton](#) * [m_chartHeightIncreaseButton](#) = nullptr
- [QPushButton](#) * [m_chartHeightDecreaseButton](#) = nullptr
- [QPushButton](#) * [m_useEstimatedFtpButton](#) = nullptr
- [QWidget](#) * [m_colorLegendWidget](#) = nullptr
- [QHBoxLayout](#) * [m_colorLegendLayout](#) = nullptr
- [QCheckBox](#) * [m_showPower](#) = nullptr
- [QCheckBox](#) * [m_showHR](#) = nullptr
- [QCheckBox](#) * [m_showCadence](#) = nullptr
- [QCheckBox](#) * [m_showSpeed](#) = nullptr
- [QCheckBox](#) * [m_showAltitude](#) = nullptr
- [QTableWidget](#) * [m_zonesTable](#) = nullptr
- [PowerHistogramWidget](#) * [m_histogram](#) = nullptr
- [PowerCurveWidget](#) * [m_pdcWidget](#) = nullptr
- [FitnessChartWidget](#) * [m_fitnessChart](#) = nullptr
- [QTableWidget](#) * [m_intervalsTable](#) = nullptr
- [QTableWidget](#) * [m_lapsTable](#) = nullptr
- [QTextEdit](#) * [m_activityNotesView](#) = nullptr
- [QCheckBox](#) * [m_followIntervalSelectionCheck](#) = nullptr
- [QLabel](#) * [m_intervalSummaryLabel](#) = nullptr
- [QTableWidget](#) * [m_climbsTable](#) = nullptr
- [QLabel](#) * [m_climbSummaryLabel](#) = nullptr
- [QDoubleSpinBox](#) * [m_climbMinLengthSpin](#) = nullptr
- [QDoubleSpinBox](#) * [m_climbMinGainSpin](#) = nullptr
- [QDoubleSpinBox](#) * [m_climbMinGradientSpin](#) = nullptr
- [QDoubleSpinBox](#) * [m_climbStartGradientSpin](#) = nullptr
- [QDoubleSpinBox](#) * [m_climbDipMetersSpin](#) = nullptr
- [QDoubleSpinBox](#) * [m_climbDipDistanceSpin](#) = nullptr
- [QDoubleSpinBox](#) * [m_climbSmoothingSpin](#) = nullptr
- [QCheckBox](#) * [m_climbOverlayEnabledCheck](#) = nullptr
- [QComboBox](#) * [m_climbOverlayMetricCombo](#) = nullptr
- `std::vector< Climb >` [m_detectedClimbs](#)
- `bool` [m_suppressClimbAutoZoom](#) = false
- [MapRenderer](#) * [m_mapRenderer](#) = nullptr
- [QComboBox](#) * [m_mapStyleCombo](#) = nullptr
- [QCheckBox](#) * [m_mapAutoContrastCheck](#) = nullptr
- [CalendarWidget](#) * [m_calendarWidget](#) = nullptr
- [ActivityBrowser](#) * [m_activityBrowser](#) = nullptr
- [QHash< QString, \[ImportBatchSummary\]\(#\) >](#) [m_importBatchSummaries](#)

Static Private Attributes

- static constexpr int [kTabActivities](#) = 0
- static constexpr int [kTabAnalysis](#) = 1
- static constexpr int [kAnalysisTabCharts](#) = 0
- static constexpr int [kAnalysisTabZones](#) = 1
- static constexpr int [kAnalysisTabHistogram](#) = 2
- static constexpr int [kAnalysisTabPDC](#) = 3
- static constexpr int [kAnalysisTabCalendar](#) = 4
- static constexpr int [kAnalysisTabFitness](#) = 5
- static constexpr int [kAnalysisTabClimbing](#) = 6

7.63.1 Detailed Description

Main application window.

Provides integrated UI for activity browsing, analysis, charting, mapping, and athlete/profile management.

Definition at line 61 of file [MainWindow.h](#).

7.63.2 Constructor & Destructor Documentation

7.63.2.1 MainWindow()

```
MainWindow::MainWindow (
    QWidget * parent = nullptr) [explicit]
```

Constructs main window.

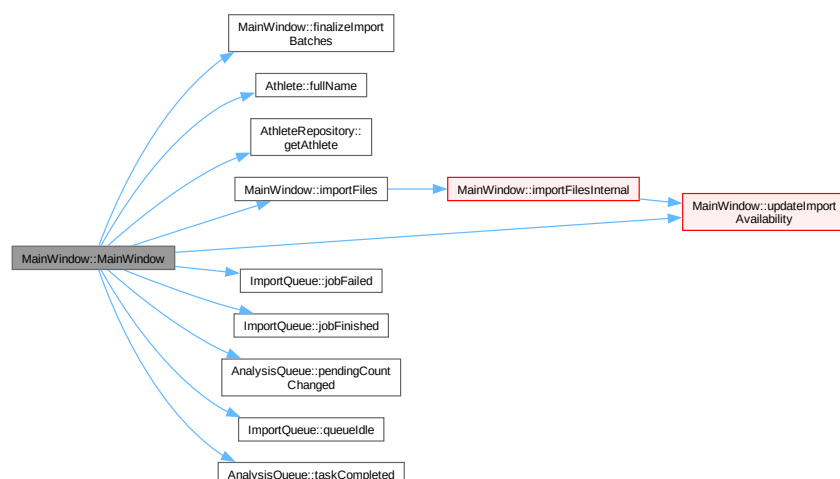
Parameters

<i>parent</i>	Parent widget.
---------------	----------------

Definition at line 365 of file [MainWindow.cpp](#).

References [finalizeImportBatches\(\)](#), [Athlete::fullName\(\)](#), [AthleteRepository::getAthlete\(\)](#), [importFiles\(\)](#), [ImportQueue::jobFailed\(\)](#), [ImportQueue::jobFinished\(\)](#), [AnalysisQueue::pendingCountChanged\(\)](#), [ImportQueue::queueIdle\(\)](#), [AnalysisQueue::taskCompleted\(\)](#), and [updateImportAvailability\(\)](#).

Here is the call graph for this function:



7.63.3 Member Function Documentation

7.63.3.1 activityCount()

```
int MainWindow::activityCount () const [private]
```

Definition at line 2286 of file [MainWindow.cpp](#).

7.63.3.2 addRecentDatabase()

```
void MainWindow::addRecentDatabase (  
    const QString & path) [private]
```

Definition at line 2022 of file [MainWindow.cpp](#).

7.63.3.3 applyChartHeight()

```
void MainWindow::applyChartHeight () [private]
```

Definition at line 1721 of file [MainWindow.cpp](#).

7.63.3.4 applyChartPreset()

```
void MainWindow::applyChartPreset (  
    int presetId) [private]
```

Definition at line 3131 of file [MainWindow.cpp](#).

7.63.3.5 applyChartSmoothing()

```
void MainWindow::applyChartSmoothing () [private]
```

Definition at line 3195 of file [MainWindow.cpp](#).

7.63.3.6 applyEstimatedFtpForCurrentAthlete()

```
void MainWindow::applyEstimatedFtpForCurrentAthlete () [private]
```

Definition at line 3338 of file [MainWindow.cpp](#).

7.63.3.7 applyWeightMetricsToClimbs()

```
void MainWindow::applyWeightMetricsToClimbs (  
    std::vector< Climb > & climbs,  
    double riderWeightKg) const [private]
```

Definition at line 4097 of file [MainWindow.cpp](#).

7.63.3.8 assignClimbWeightMetrics()

```
void MainWindow::assignClimbWeightMetrics (  
    Climb & climb,  
    double riderWeightKg) const [private]
```

Definition at line 4074 of file [MainWindow.cpp](#).

7.63.3.9 backupDatabase

```
void MainWindow::backupDatabase () [private], [slot]
```

Definition at line 2888 of file [MainWindow.cpp](#).

7.63.3.10 buildColorContext()

```
ColorContext MainWindow::buildColorContext () const [private]
```

Definition at line 2316 of file [MainWindow.cpp](#).

7.63.3.11 buildStatusBar()

void MainWindow::buildStatusBar () [private]
 Definition at line 1638 of file [MainWindow.cpp](#).

7.63.3.12 buildToolBar()

void MainWindow::buildToolBar () [private]
 Definition at line 1581 of file [MainWindow.cpp](#).

7.63.3.13 buildUI()

void MainWindow::buildUI () [private]
 Definition at line 534 of file [MainWindow.cpp](#).

7.63.3.14 canImportActivities()

bool MainWindow::canImportActivities () const [private]
 Returns whether importing is currently allowed.
 Requires an open database and a selected athlete.
 Definition at line 2094 of file [MainWindow.cpp](#).
 Referenced by [updateImportAvailability\(\)](#).
 Here is the caller graph for this function:

**7.63.3.15 clearIntervalSummary()**

void MainWindow::clearIntervalSummary () [private]
 Definition at line 3567 of file [MainWindow.cpp](#).

7.63.3.16 closeEvent()

void MainWindow::closeEvent (
 QCloseEvent * event) [override], [protected]
 Handles application close event.
 Definition at line 526 of file [MainWindow.cpp](#).

7.63.3.17 configureFolderWatcher()

void MainWindow::configureFolderWatcher () [private]
 Definition at line 1793 of file [MainWindow.cpp](#).

7.63.3.18 createDatabase

void MainWindow::createDatabase () [private], [slot]
 Definition at line 2735 of file [MainWindow.cpp](#).

7.63.3.19 createOrOpenDefaultDatabase()

bool MainWindow::createOrOpenDefaultDatabase () [private]
 Definition at line 2099 of file [MainWindow.cpp](#).

7.63.3.20 createVideo

```
void MainWindow::createVideo () [private], [slot]
```

Definition at line 2625 of file [MainWindow.cpp](#).

7.63.3.21 currentColorMetric()

```
ColorMetric MainWindow::currentColorMetric () const [private]
```

Definition at line 2308 of file [MainWindow.cpp](#).

7.63.3.22 decreaseChartHeight()

```
void MainWindow::decreaseChartHeight () [private]
```

Definition at line 1715 of file [MainWindow.cpp](#).

7.63.3.23 defaultDatabasePathForAutoCreate()

```
QString MainWindow::defaultDatabasePathForAutoCreate () const [private]
```

Definition at line 2239 of file [MainWindow.cpp](#).

7.63.3.24 detectClimbsAndRefresh()

```
void MainWindow::detectClimbsAndRefresh () [private]
```

Definition at line 4005 of file [MainWindow.cpp](#).

7.63.3.25 dragEnterEvent()

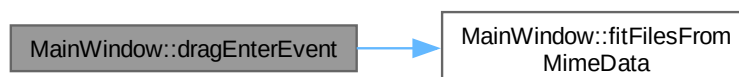
```
void MainWindow::dragEnterEvent (
    QDragEnterEvent * event) [override], [protected]
```

Accepts drag-enter when the payload contains local .fit files.

Definition at line 1771 of file [MainWindow.cpp](#).

References [fitFilesFromMimeData\(\)](#).

Here is the call graph for this function:

**7.63.3.26 dropEvent()**

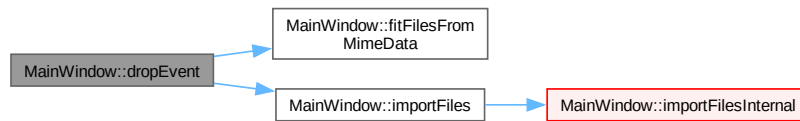
```
void MainWindow::dropEvent (
    QDropEvent * event) [override], [protected]
```

Extracts dropped .fit files and forwards them to the import queue.

Definition at line 1780 of file [MainWindow.cpp](#).

References [fitFilesFromMimeData\(\)](#), and [importFiles\(\)](#).

Here is the call graph for this function:



7.63.3.27 editCurrentActivityProperties()

`void MainWindow::editCurrentActivityProperties () [private]`
 Definition at line 4948 of file [MainWindow.cpp](#).

7.63.3.28 editSelectedClimbBoundaries()

`void MainWindow::editSelectedClimbBoundaries () [private]`
 Definition at line 4209 of file [MainWindow.cpp](#).

7.63.3.29 ensureAthlete()

`bool MainWindow::ensureAthlete () [private]`
 Definition at line 2158 of file [MainWindow.cpp](#).

7.63.3.30 ensureDatabase()

`bool MainWindow::ensureDatabase () [private]`
 Definition at line 2139 of file [MainWindow.cpp](#).

7.63.3.31 ensureImportReady()

`bool MainWindow::ensureImportReady () [private]`
 Ensures database path, athlete selection, and queue state are import-ready.
 Definition at line 2234 of file [MainWindow.cpp](#).
 Referenced by [importFilesInternal\(\)](#).
 Here is the caller graph for this function:



7.63.3.32 estimatedFtpFromCurrentRide()

`double MainWindow::estimatedFtpFromCurrentRide () const [private]`
 Definition at line 3330 of file [MainWindow.cpp](#).

7.63.3.33 eventFilter()

`bool MainWindow::eventFilter (`
 `QObject * watched,`
 `QEvent * event) [override], [protected]`

Filters events for custom handling.

Definition at line 1747 of file [MainWindow.cpp](#).

7.63.3.34 finalizeImportBatches()

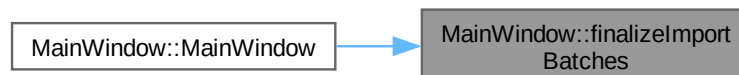
```
void MainWindow::finalizeImportBatches () [private]
```

Shows a per-batch import result summary when queue work completes.

Definition at line 2481 of file [MainWindow.cpp](#).

Referenced by [MainWindow\(\)](#).

Here is the caller graph for this function:



7.63.3.35 fitFilesFromMimeData()

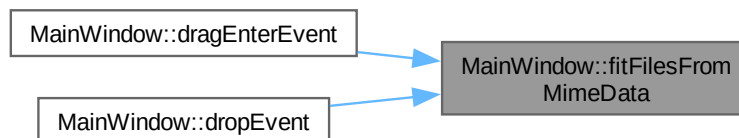
```
QStringList MainWindow::fitFilesFromMimeData (
    const QMimeData * mimeData) const [private]
```

Returns deduplicated local `.fit` files from MIME drop/paste data.

Definition at line 2521 of file [MainWindow.cpp](#).

Referenced by [dragEnterEvent\(\)](#), and [dropEvent\(\)](#).

Here is the caller graph for this function:



7.63.3.36 hideWelcomeScreen()

```
void MainWindow::hideWelcomeScreen () [private]
```

Definition at line 2278 of file [MainWindow.cpp](#).

7.63.3.37 importActivities

```
void MainWindow::importActivities () [private], [slot]
```

Definition at line 2599 of file [MainWindow.cpp](#).

7.63.3.38 importFiles()

```
void MainWindow::importFiles (
    const QStringList & filePaths) [private]
```

Queues a batch of file paths for background import.

Definition at line 2427 of file [MainWindow.cpp](#).

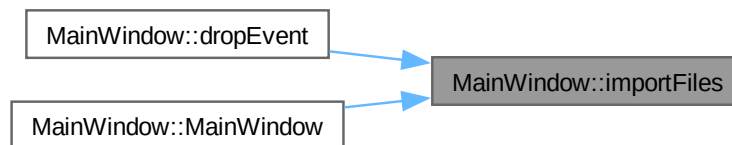
References [importFilesInternal\(\)](#).

Referenced by [dropEvent\(\)](#), and [MainWindow\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.63.3.39 importFilesInternal()

```
void MainWindow::importFilesInternal (
    const QStringList & filePaths,
    bool showResultDialog,
    const QString & sourceLabel) [private]
```

Internal import entry point shared by menu action and drag-drop.

Parameters

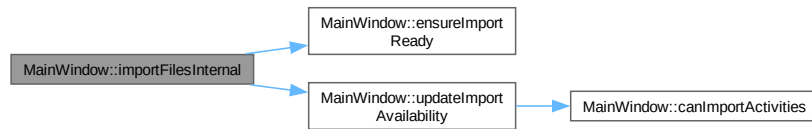
<i>filePaths</i>	Candidate files to import (only <code>.fit</code> files are queued).
<i>showResultDialog</i>	Whether to display completion summary dialogs.
<i>sourceLabel</i>	Source tag stored in import metadata (e.g. "manual", "drop").

Definition at line 2432 of file [MainWindow.cpp](#).

References [ensureImportReady\(\)](#), and [updateImportAvailability\(\)](#).

Referenced by [importFiles\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.63.3.40 increaseChartHeight()

`void MainWindow::increaseChartHeight () [private]`
 Definition at line 1709 of file [MainWindow.cpp](#).

7.63.3.41 joinSelectedClimbs()

`void MainWindow::joinSelectedClimbs () [private]`
 Definition at line 4241 of file [MainWindow.cpp](#).

7.63.3.42 loadSettings()

`void MainWindow::loadSettings () [private]`
 Definition at line 1948 of file [MainWindow.cpp](#).

7.63.3.43 manageAthletes

`void MainWindow::manageAthletes () [private], [slot]`
 Definition at line 2789 of file [MainWindow.cpp](#).

7.63.3.44 monitoredDirectories()

`QStringList MainWindow::monitoredDirectories () const [private]`
 Definition at line 1838 of file [MainWindow.cpp](#).

7.63.3.45 navigateToInterval()

`void MainWindow::navigateToInterval (`
 `double startSeconds,`
 `double endSeconds,`
 `bool exactZoom) [private]`
 Definition at line 3492 of file [MainWindow.cpp](#).

7.63.3.46 onActivityImported

```
void MainWindow::onActivityImported (
    int activityId) [private], [slot]
```

Definition at line 2706 of file [MainWindow.cpp](#).

7.63.3.47 onAthleteSelectionChanged

```
void MainWindow::onAthleteSelectionChanged (
    int index) [private], [slot]
```

Definition at line 2941 of file [MainWindow.cpp](#).

7.63.3.48 onClimbBoundaryEdited()

```
void MainWindow::onClimbBoundaryEdited (
    double oldStartSeconds,
    double oldEndSeconds,
    double newStartSeconds,
    double newEndSeconds) [private]
```

Definition at line 4456 of file [MainWindow.cpp](#).

7.63.3.49 onClimbRowDoubleClicked()

```
void MainWindow::onClimbRowDoubleClicked (
    QTableWidgetItem * item) [private]
```

Definition at line 4404 of file [MainWindow.cpp](#).

7.63.3.50 onClimbSelectionChanged()

```
void MainWindow::onClimbSelectionChanged () [private]
```

Definition at line 4325 of file [MainWindow.cpp](#).

7.63.3.51 onClimbTableContextMenuRequested()

```
void MainWindow::onClimbTableContextMenuRequested (
    const QPoint & pos) [private]
```

Definition at line 4136 of file [MainWindow.cpp](#).

7.63.3.52 onIntervalRowDoubleClicked()

```
void MainWindow::onIntervalRowDoubleClicked (
    QTableWidgetItem * item) [private]
```

Definition at line 3473 of file [MainWindow.cpp](#).

7.63.3.53 onIntervalSelectionChanged()

```
void MainWindow::onIntervalSelectionChanged () [private]
```

Definition at line 3433 of file [MainWindow.cpp](#).

7.63.3.54 onNewClimbRequested()

```
void MainWindow::onNewClimbRequested (
    double startSeconds,
    double endSeconds) [private]
```

Definition at line 4726 of file [MainWindow.cpp](#).

7.63.3.55 onWatchDirectoryChanged()

```
void MainWindow::onWatchDirectoryChanged (
    const QString & path) [private]
```

Definition at line 1902 of file [MainWindow.cpp](#).

7.63.3.56 onWorkoutLoaded

void MainWindow::onWorkoutLoaded () [private], [slot]
Definition at line 2972 of file [MainWindow.cpp](#).

7.63.3.57 openDatabase

void MainWindow::openDatabase () [private], [slot]
Definition at line 2725 of file [MainWindow.cpp](#).

7.63.3.58 openDatabasePath()

void MainWindow::openDatabasePath (
 const QString & path) [private]
Definition at line 2056 of file [MainWindow.cpp](#).

7.63.3.59 openRecentDatabase

void MainWindow::openRecentDatabase () [private], [slot]
Definition at line 2932 of file [MainWindow.cpp](#).

7.63.3.60 openSettingsDialog

void MainWindow::openSettingsDialog () [private], [slot]
Definition at line 2818 of file [MainWindow.cpp](#).

7.63.3.61 previewFitFile

void MainWindow::previewFitFile () [private], [slot]
Definition at line 2611 of file [MainWindow.cpp](#).

7.63.3.62 refreshAthleteSelector()

void MainWindow::refreshAthleteSelector () [private]
Definition at line 2540 of file [MainWindow.cpp](#).

7.63.3.63 refreshClimbViews()

void MainWindow::refreshClimbViews (
 double preferredStartSeconds = -1.0,
 double preferredEndSeconds = -1.0) [private]
Definition at line 3816 of file [MainWindow.cpp](#).

7.63.3.64 removeSelectedClimbs()

void MainWindow::removeSelectedClimbs () [private]
Definition at line 4174 of file [MainWindow.cpp](#).

7.63.3.65 resetAllZoom()

void MainWindow::resetAllZoom () [private]
Definition at line 1701 of file [MainWindow.cpp](#).

7.63.3.66 resolveActiveActivityWeightKg()

double MainWindow::resolveActiveActivityWeightKg () const [private]
Definition at line 4036 of file [MainWindow.cpp](#).

7.63.3.67 revertSelectedClimbToAuto

```
void MainWindow::revertSelectedClimbToAuto () [private], [slot]
```

Definition at line 4686 of file [MainWindow.cpp](#).

7.63.3.68 saveSettings()

```
void MainWindow::saveSettings () [private]
```

Definition at line 1912 of file [MainWindow.cpp](#).

7.63.3.69 scanWatchDirectories()

```
void MainWindow::scanWatchDirectories (
    bool initialScan) [private]
```

Definition at line 1862 of file [MainWindow.cpp](#).

7.63.3.70 scheduleActivityBrowserRefresh()

```
void MainWindow::scheduleActivityBrowserRefresh () [private]
```

Definition at line 2475 of file [MainWindow.cpp](#).

7.63.3.71 selectedClimbIndicesFromTable()

```
std::vector< int > MainWindow::selectedClimbIndicesFromTable () const [private]
```

Definition at line 4103 of file [MainWindow.cpp](#).

7.63.3.72 selectIntervalRowOffset()

```
void MainWindow::selectIntervalRowOffset (
    int delta) [private]
```

Definition at line 3528 of file [MainWindow.cpp](#).

7.63.3.73 setupShortcuts()

```
void MainWindow::setupShortcuts () [private]
```

Definition at line 1651 of file [MainWindow.cpp](#).

7.63.3.74 showAbout

```
void MainWindow::showAbout () [private], [slot]
```

Definition at line 4973 of file [MainWindow.cpp](#).

7.63.3.75 showWelcomeScreen()

```
void MainWindow::showWelcomeScreen () [private]
```

Definition at line 2270 of file [MainWindow.cpp](#).

7.63.3.76 triggerDetectClimbs

```
void MainWindow::triggerDetectClimbs () [private], [slot]
```

Definition at line 4822 of file [MainWindow.cpp](#).

7.63.3.77 triggerDetectIntervals

```
void MainWindow::triggerDetectIntervals () [private], [slot]
```

Definition at line 4889 of file [MainWindow.cpp](#).

7.63.3.78 updateAnalysisEmptyStates()

```
void MainWindow::updateAnalysisEmptyStates () [private]
```

Definition at line 3009 of file [MainWindow.cpp](#).

7.63.3.79 updateAthleteLabel()

void MainWindow::updateAthleteLabel () [private]
 Definition at line 2923 of file [MainWindow.cpp](#).

7.63.3.80 updateCharts()

void MainWindow::updateCharts () [private]
 Definition at line 3082 of file [MainWindow.cpp](#).

7.63.3.81 updateClimbingTab()

void MainWindow::updateClimbingTab () [private]
 Definition at line 3794 of file [MainWindow.cpp](#).

7.63.3.82 updateClimbQuarterCharts()

void MainWindow::updateClimbQuarterCharts (
 const [Climb](#) * climb) [private]
 Definition at line 3922 of file [MainWindow.cpp](#).

7.63.3.83 updateClimbRowStyles()

void MainWindow::updateClimbRowStyles () [private]
 Definition at line 4304 of file [MainWindow.cpp](#).

7.63.3.84 updateColorLegend()

void MainWindow::updateColorLegend () [private]
 Definition at line 2372 of file [MainWindow.cpp](#).

7.63.3.85 updateFitnessChart()

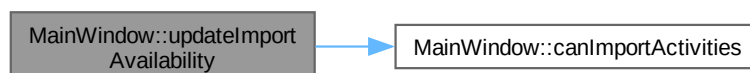
void MainWindow::updateFitnessChart () [private]
 Definition at line 3375 of file [MainWindow.cpp](#).

7.63.3.86 updateHistogram()

void MainWindow::updateHistogram () [private]
 Definition at line 3252 of file [MainWindow.cpp](#).

7.63.3.87 updateImportAvailability()

void MainWindow::updateImportAvailability () [private]
 Enables/disables import actions based on current readiness.
 Definition at line 2248 of file [MainWindow.cpp](#).
 References [canImportActivities\(\)](#).
 Referenced by [importFilesInternal\(\)](#), and [MainWindow\(\)](#).
 Here is the call graph for this function:



Here is the caller graph for this function:



7.63.3.88 updateIntervalRowStyles()

`void MainWindow::updateIntervalRowStyles () [private]`
 Definition at line 3546 of file [MainWindow.cpp](#).

7.63.3.89 updateIntervals()

`void MainWindow::updateIntervals () [private]`
 Definition at line 3660 of file [MainWindow.cpp](#).

7.63.3.90 updateIntervalSummary()

`void MainWindow::updateIntervalSummary (`
 `double startSeconds,`
 `double endSeconds) [private]`
 Definition at line 3573 of file [MainWindow.cpp](#).

7.63.3.91 updatePowerCurve()

`void MainWindow::updatePowerCurve () [private]`
 Definition at line 3257 of file [MainWindow.cpp](#).

7.63.3.92 updateRecentDatabaseMenu()

`void MainWindow::updateRecentDatabaseMenu () [private]`
 Definition at line 2033 of file [MainWindow.cpp](#).

7.63.3.93 updateStatsLabel()

`void MainWindow::updateStatsLabel () [private]`
 Definition at line 3032 of file [MainWindow.cpp](#).

7.63.3.94 updateStatusBarInfo()

`void MainWindow::updateStatusBarInfo () [private]`
 Definition at line 2571 of file [MainWindow.cpp](#).

7.63.3.95 updateWelcomeScreenVisibility()

`void MainWindow::updateWelcomeScreenVisibility () [private]`
 Definition at line 2296 of file [MainWindow.cpp](#).

7.63.3.96 updateZoneAvailability()

`void MainWindow::updateZoneAvailability () [private]`
 Definition at line 2416 of file [MainWindow.cpp](#).

7.63.3.97 updateZonesTab()

`void MainWindow::updateZonesTab () [private]`
 Definition at line 3213 of file [MainWindow.cpp](#).

7.63.4 Member Data Documentation

7.63.4.1 kAnalysisTabCalendar

```
int MainWindow::kAnalysisTabCalendar = 4 [static], [constexpr], [private]
```

Definition at line 277 of file [MainWindow.h](#).

7.63.4.2 kAnalysisTabCharts

```
int MainWindow::kAnalysisTabCharts = 0 [static], [constexpr], [private]
```

Definition at line 273 of file [MainWindow.h](#).

7.63.4.3 kAnalysisTabClimbing

```
int MainWindow::kAnalysisTabClimbing = 6 [static], [constexpr], [private]
```

Definition at line 279 of file [MainWindow.h](#).

7.63.4.4 kAnalysisTabFitness

```
int MainWindow::kAnalysisTabFitness = 5 [static], [constexpr], [private]
```

Definition at line 278 of file [MainWindow.h](#).

7.63.4.5 kAnalysisTabHistogram

```
int MainWindow::kAnalysisTabHistogram = 2 [static], [constexpr], [private]
```

Definition at line 275 of file [MainWindow.h](#).

7.63.4.6 kAnalysisTabPDC

```
int MainWindow::kAnalysisTabPDC = 3 [static], [constexpr], [private]
```

Definition at line 276 of file [MainWindow.h](#).

7.63.4.7 kAnalysisTabZones

```
int MainWindow::kAnalysisTabZones = 1 [static], [constexpr], [private]
```

Definition at line 274 of file [MainWindow.h](#).

7.63.4.8 kTabActivities

```
int MainWindow::kTabActivities = 0 [static], [constexpr], [private]
```

Definition at line 270 of file [MainWindow.h](#).

7.63.4.9 kTabAnalysis

```
int MainWindow::kTabAnalysis = 1 [static], [constexpr], [private]
```

Definition at line 271 of file [MainWindow.h](#).

7.63.4.10 m_activityBrowser

```
ActivityBrowser* MainWindow::m_activityBrowser = nullptr [private]
```

Definition at line 361 of file [MainWindow.h](#).

7.63.4.11 m_activityCountLabel

```
QLabel* MainWindow::m_activityCountLabel = nullptr [private]
```

Definition at line 254 of file [MainWindow.h](#).

7.63.4.12 m_activityNotesView

```
QTextEdit* MainWindow::m_activityNotesView = nullptr [private]
```

Definition at line 333 of file [MainWindow.h](#).

7.63.4.13 m_activityTabStack

`QStackedLayout* MainWindow::m_activityTabStack = nullptr [private]`
Definition at line 262 of file [MainWindow.h](#).

7.63.4.14 m_altitudeChart

`RideChartWidget* MainWindow::m_altitudeChart = nullptr [private]`
Definition at line 291 of file [MainWindow.h](#).

7.63.4.15 m_analysisQueue

`AnalysisQueue* MainWindow::m_analysisQueue = nullptr [private]`
Definition at line 207 of file [MainWindow.h](#).

7.63.4.16 m_analysisTabWidget

`QTabWidget* MainWindow::m_analysisTabWidget = nullptr [private]`
Definition at line 261 of file [MainWindow.h](#).

7.63.4.17 m_athleteCombo

`QComboBox* MainWindow::m_athleteCombo = nullptr [private]`
Definition at line 244 of file [MainWindow.h](#).

7.63.4.18 m_athleteHeader

`AthleteHeaderWidget* MainWindow::m_athleteHeader = nullptr [private]`
Definition at line 257 of file [MainWindow.h](#).

7.63.4.19 m_athleteLabelAct

`QAction* MainWindow::m_athleteLabelAct = nullptr [private]`
Definition at line 223 of file [MainWindow.h](#).

7.63.4.20 m_athletesMenu

`QMenu* MainWindow::m_athletesMenu = nullptr [private]`
Definition at line 222 of file [MainWindow.h](#).

7.63.4.21 m_athleteStatusLabel

`QLabel* MainWindow::m_athleteStatusLabel = nullptr [private]`
Definition at line 253 of file [MainWindow.h](#).

7.63.4.22 m_autoSmoothingCheck

`QCheckBox* MainWindow::m_autoSmoothingCheck = nullptr [private]`
Definition at line 248 of file [MainWindow.h](#).

7.63.4.23 m_cadenceChart

`RideChartWidget* MainWindow::m_cadenceChart = nullptr [private]`
Definition at line 289 of file [MainWindow.h](#).

7.63.4.24 m_calendarTabStack

`QStackedLayout* MainWindow::m_calendarTabStack = nullptr [private]`
Definition at line 266 of file [MainWindow.h](#).

7.63.4.25 m_calendarWidget

`CalendarWidget* MainWindow::m_calendarWidget = nullptr [private]`
Definition at line 358 of file [MainWindow.h](#).

7.63.4.26 m_chartHeight

`int MainWindow::m_chartHeight = 220 [private]`
Definition at line 286 of file [MainWindow.h](#).

7.63.4.27 m_chartHeightDecreaseButton

`QPushButton* MainWindow::m_chartHeightDecreaseButton = nullptr [private]`
Definition at line 306 of file [MainWindow.h](#).

7.63.4.28 m_chartHeightIncreaseButton

`QPushButton* MainWindow::m_chartHeightIncreaseButton = nullptr [private]`
Definition at line 305 of file [MainWindow.h](#).

7.63.4.29 m_chartMapSplit

`QSplitter* MainWindow::m_chartMapSplit = nullptr [private]`
Definition at line 285 of file [MainWindow.h](#).

7.63.4.30 m_chartPresetCombo

`QComboBox* MainWindow::m_chartPresetCombo = nullptr [private]`
Definition at line 246 of file [MainWindow.h](#).

7.63.4.31 m_chartsContainer

`QWidget* MainWindow::m_chartsContainer = nullptr [private]`
Definition at line 283 of file [MainWindow.h](#).

7.63.4.32 m_chartScroll

`QScrollArea* MainWindow::m_chartScroll = nullptr [private]`
Definition at line 282 of file [MainWindow.h](#).

7.63.4.33 m_chartsLayout

`QVBoxLayout* MainWindow::m_chartsLayout = nullptr [private]`
Definition at line 284 of file [MainWindow.h](#).

7.63.4.34 m_climbAltitudeChart

`RideChartWidget* MainWindow::m_climbAltitudeChart = nullptr [private]`
Definition at line 294 of file [MainWindow.h](#).

7.63.4.35 m_climbCadenceChart

`RideChartWidget* MainWindow::m_climbCadenceChart = nullptr [private]`
Definition at line 297 of file [MainWindow.h](#).

7.63.4.36 m_climbDipDistanceSpin

`QDoubleSpinBox* MainWindow::m_climbDipDistanceSpin = nullptr [private]`
Definition at line 345 of file [MainWindow.h](#).

7.63.4.37 m_climbDipMetersSpin

QDoubleSpinBox* MainWindow::m_climbDipMetersSpin = nullptr [private]
Definition at line 344 of file [MainWindow.h](#).

7.63.4.38 m_climbHrChart

RideChartWidget* MainWindow::m_climbHrChart = nullptr [private]
Definition at line 296 of file [MainWindow.h](#).

7.63.4.39 m_climbingTabStack

QStackedLayout* MainWindow::m_climbingTabStack = nullptr [private]
Definition at line 268 of file [MainWindow.h](#).

7.63.4.40 m_climbMinGainSpin

QDoubleSpinBox* MainWindow::m_climbMinGainSpin = nullptr [private]
Definition at line 341 of file [MainWindow.h](#).

7.63.4.41 m_climbMinGradientSpin

QDoubleSpinBox* MainWindow::m_climbMinGradientSpin = nullptr [private]
Definition at line 342 of file [MainWindow.h](#).

7.63.4.42 m_climbMinLengthSpin

QDoubleSpinBox* MainWindow::m_climbMinLengthSpin = nullptr [private]
Definition at line 340 of file [MainWindow.h](#).

7.63.4.43 m_climbOverlayEnabledCheck

QCheckBox* MainWindow::m_climbOverlayEnabledCheck = nullptr [private]
Definition at line 347 of file [MainWindow.h](#).

7.63.4.44 m_climbOverlayMetricCombo

QComboBox* MainWindow::m_climbOverlayMetricCombo = nullptr [private]
Definition at line 348 of file [MainWindow.h](#).

7.63.4.45 m_climbPowerChart

RideChartWidget* MainWindow::m_climbPowerChart = nullptr [private]
Definition at line 295 of file [MainWindow.h](#).

7.63.4.46 m_climbQuarterCadenceChart

QChartView* MainWindow::m_climbQuarterCadenceChart = nullptr [private]
Definition at line 301 of file [MainWindow.h](#).

7.63.4.47 m_climbQuarterHrChart

QChartView* MainWindow::m_climbQuarterHrChart = nullptr [private]
Definition at line 300 of file [MainWindow.h](#).

7.63.4.48 m_climbQuarterPowerChart

QChartView* MainWindow::m_climbQuarterPowerChart = nullptr [private]
Definition at line 299 of file [MainWindow.h](#).

7.63.4.49 m_climbSmoothingSpin

QDoubleSpinBox* MainWindow::m_climbSmoothingSpin = nullptr [private]
Definition at line 346 of file [MainWindow.h](#).

7.63.4.50 m_climbSpeedChart

RideChartWidget* MainWindow::m_climbSpeedChart = nullptr [private]
Definition at line 298 of file [MainWindow.h](#).

7.63.4.51 m_climbsTable

QTableWidget* MainWindow::m_climbsTable = nullptr [private]
Definition at line 338 of file [MainWindow.h](#).

7.63.4.52 m_climbStartGradientSpin

QDoubleSpinBox* MainWindow::m_climbStartGradientSpin = nullptr [private]
Definition at line 343 of file [MainWindow.h](#).

7.63.4.53 m_climbSummaryLabel

QLabel* MainWindow::m_climbSummaryLabel = nullptr [private]
Definition at line 339 of file [MainWindow.h](#).

7.63.4.54 m_colorLegendLayout

QHBoxLayout* MainWindow::m_colorLegendLayout = nullptr [private]
Definition at line 309 of file [MainWindow.h](#).

7.63.4.55 m_colorLegendWidget

QWidget* MainWindow::m_colorLegendWidget = nullptr [private]
Definition at line 308 of file [MainWindow.h](#).

7.63.4.56 m_colorMetricCombo

QComboBox* MainWindow::m_colorMetricCombo = nullptr [private]
Definition at line 245 of file [MainWindow.h](#).

7.63.4.57 m_controller

WorkoutController* MainWindow::m_controller = nullptr [private]
Definition at line 214 of file [MainWindow.h](#).

7.63.4.58 m_currentAthleteId

int MainWindow::m_currentAthleteId = -1 [private]
Definition at line 218 of file [MainWindow.h](#).

7.63.4.59 m_currentAthleteName

QString MainWindow::m_currentAthleteName [private]
Definition at line 219 of file [MainWindow.h](#).

7.63.4.60 m_dbManager

DatabaseManager MainWindow::m_dbManager [private]
Definition at line 217 of file [MainWindow.h](#).

7.63.4.61 m_dbStatusLabel

QLabel* MainWindow::m_dbStatusLabel = nullptr [private]
Definition at line 252 of file [MainWindow.h](#).

7.63.4.62 m_detectedClimbs

std::vector<Climb> MainWindow::m_detectedClimbs [private]
Definition at line 349 of file [MainWindow.h](#).

7.63.4.63 m_firstLaunchCompleted

bool MainWindow::m_firstLaunchCompleted = false [private]
Definition at line 233 of file [MainWindow.h](#).

7.63.4.64 m_fitChartsButton

QPushButton* MainWindow::m_fitChartsButton = nullptr [private]
Definition at line 304 of file [MainWindow.h](#).

7.63.4.65 m_fitnessChart

FitnessChartWidget* MainWindow::m_fitnessChart = nullptr [private]
Definition at line 328 of file [MainWindow.h](#).

7.63.4.66 m_fitnessTabStack

QStackedLayout* MainWindow::m_fitnessTabStack = nullptr [private]
Definition at line 267 of file [MainWindow.h](#).

7.63.4.67 m_followIntervalSelectionCheck

QCheckBox* MainWindow::m_followIntervalSelectionCheck = nullptr [private]
Definition at line 334 of file [MainWindow.h](#).

7.63.4.68 m_histogram

PowerHistogramWidget* MainWindow::m_histogram = nullptr [private]
Definition at line 322 of file [MainWindow.h](#).

7.63.4.69 m_histogramTabStack

QStackedLayout* MainWindow::m_histogramTabStack = nullptr [private]
Definition at line 264 of file [MainWindow.h](#).

7.63.4.70 m_hrChart

RideChartWidget* MainWindow::m_hrChart = nullptr [private]
Definition at line 288 of file [MainWindow.h](#).

7.63.4.71 m_importAct

QAction* MainWindow::m_importAct = nullptr [private]
Definition at line 224 of file [MainWindow.h](#).

7.63.4.72 m_importBatchSummaries

QHash<QString, ImportBatchSummary> MainWindow::m_importBatchSummaries [private]
Definition at line 374 of file [MainWindow.h](#).

7.63.4.73 m_importProgressModel

`ImportProgressModel* MainWindow::m_importProgressModel = nullptr [private]`
Definition at line 211 of file [MainWindow.h](#).

7.63.4.74 m_importQueue

`ImportQueue* MainWindow::m_importQueue = nullptr [private]`
Definition at line 210 of file [MainWindow.h](#).

7.63.4.75 m_importRefreshTimer

`QTimer* MainWindow::m_importRefreshTimer = nullptr [private]`
Definition at line 238 of file [MainWindow.h](#).

7.63.4.76 m_importStatusWidget

`ImportStatusWidget* MainWindow::m_importStatusWidget = nullptr [private]`
Definition at line 249 of file [MainWindow.h](#).

7.63.4.77 m_intervalsTable

`QTableWidget* MainWindow::m_intervalsTable = nullptr [private]`
Definition at line 331 of file [MainWindow.h](#).

7.63.4.78 m_intervalSummaryLabel

`QLabel* MainWindow::m_intervalSummaryLabel = nullptr [private]`
Definition at line 335 of file [MainWindow.h](#).

7.63.4.79 m_knownWatchedFitFiles

`QSet<QString> MainWindow::m_knownWatchedFitFiles [private]`
Definition at line 241 of file [MainWindow.h](#).

7.63.4.80 m_lapsTable

`QTableWidget* MainWindow::m_lapsTable = nullptr [private]`
Definition at line 332 of file [MainWindow.h](#).

7.63.4.81 m_lastOpenDir

`QString MainWindow::m_lastOpenDir [private]`
Definition at line 229 of file [MainWindow.h](#).

7.63.4.82 m_mapAutoContrastCheck

`QCheckBox* MainWindow::m_mapAutoContrastCheck = nullptr [private]`
Definition at line 355 of file [MainWindow.h](#).

7.63.4.83 m_mapRenderer

`MapRenderer* MainWindow::m_mapRenderer = nullptr [private]`
Definition at line 353 of file [MainWindow.h](#).

7.63.4.84 m_mapStyleCombo

`QComboBox* MainWindow::m_mapStyleCombo = nullptr [private]`
Definition at line 354 of file [MainWindow.h](#).

7.63.4.85 m_pdcWidget

`PowerCurveWidget* MainWindow::m_pdcWidget = nullptr [private]`
Definition at line 325 of file [MainWindow.h](#).

7.63.4.86 m_powerChart

`RideChartWidget* MainWindow::m_powerChart = nullptr [private]`
Definition at line 287 of file [MainWindow.h](#).

7.63.4.87 m_powerCurveTabStack

`QStackedLayout* MainWindow::m_powerCurveTabStack = nullptr [private]`
Definition at line 265 of file [MainWindow.h](#).

7.63.4.88 m_powerSmoothingCombo

`QComboBox* MainWindow::m_powerSmoothingCombo = nullptr [private]`
Definition at line 247 of file [MainWindow.h](#).

7.63.4.89 m_previewAct

`QAction* MainWindow::m_previewAct = nullptr [private]`
Definition at line 227 of file [MainWindow.h](#).

7.63.4.90 m_recentDbMenu

`QMenu* MainWindow::m_recentDbMenu = nullptr [private]`
Definition at line 228 of file [MainWindow.h](#).

7.63.4.91 m_showAltitude

`QCheckBox* MainWindow::m_showAltitude = nullptr [private]`
Definition at line 316 of file [MainWindow.h](#).

7.63.4.92 m_showCadence

`QCheckBox* MainWindow::m_showCadence = nullptr [private]`
Definition at line 314 of file [MainWindow.h](#).

7.63.4.93 m_showHR

`QCheckBox* MainWindow::m_showHR = nullptr [private]`
Definition at line 313 of file [MainWindow.h](#).

7.63.4.94 m_showPower

`QCheckBox* MainWindow::m_showPower = nullptr [private]`
Definition at line 312 of file [MainWindow.h](#).

7.63.4.95 m_showSpeed

`QCheckBox* MainWindow::m_showSpeed = nullptr [private]`
Definition at line 315 of file [MainWindow.h](#).

7.63.4.96 m_speedChart

`RideChartWidget* MainWindow::m_speedChart = nullptr [private]`
Definition at line 290 of file [MainWindow.h](#).

7.63.4.97 m_suppressClimbAutoZoom

`bool MainWindow::m_suppressClimbAutoZoom = false [private]`
Definition at line 350 of file [MainWindow.h](#).

7.63.4.98 m_tabWidget

`QTabWidget* MainWindow::m_tabWidget = nullptr [private]`
Definition at line 260 of file [MainWindow.h](#).

7.63.4.99 m_toolbarImportAct

`QAction* MainWindow::m_toolbarImportAct = nullptr [private]`
Definition at line 225 of file [MainWindow.h](#).

7.63.4.100 m_toolbarSearchAct

`QAction* MainWindow::m_toolbarSearchAct = nullptr [private]`
Definition at line 226 of file [MainWindow.h](#).

7.63.4.101 m_undoManager

`UndoManager* MainWindow::m_undoManager = nullptr [private]`
Definition at line 204 of file [MainWindow.h](#).

7.63.4.102 m_useEstimatedFtpButton

`QPushButton* MainWindow::m_useEstimatedFtpButton = nullptr [private]`
Definition at line 307 of file [MainWindow.h](#).

7.63.4.103 m_watcher

`QFileSystemWatcher* MainWindow::m_watcher = nullptr [private]`
Definition at line 236 of file [MainWindow.h](#).

7.63.4.104 m_watchFolderEnabled

`bool MainWindow::m_watchFolderEnabled = true [private]`
Definition at line 239 of file [MainWindow.h](#).

7.63.4.105 m_watchFolderPath

`QString MainWindow::m_watchFolderPath [private]`
Definition at line 240 of file [MainWindow.h](#).

7.63.4.106 m_watchRescanTimer

`QTimer* MainWindow::m_watchRescanTimer = nullptr [private]`
Definition at line 237 of file [MainWindow.h](#).

7.63.4.107 m_welcomeWidget

`WelcomeWidget* MainWindow::m_welcomeWidget = nullptr [private]`
Definition at line 232 of file [MainWindow.h](#).

7.63.4.108 m_zonesTable

`QTableWidget* MainWindow::m_zonesTable = nullptr [private]`
Definition at line 319 of file [MainWindow.h](#).

7.63.4.109 m_zonesTabStack

QStackedLayout* MainWindow::m_zonesTabStack = nullptr [private]

Definition at line 263 of file [MainWindow.h](#).

The documentation for this class was generated from the following files:

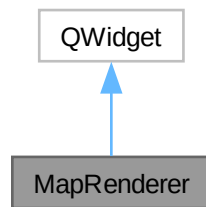
- src/gui/MainWindow.h
- src/gui/MainWindow.cpp

7.64 MapRenderer Class Reference

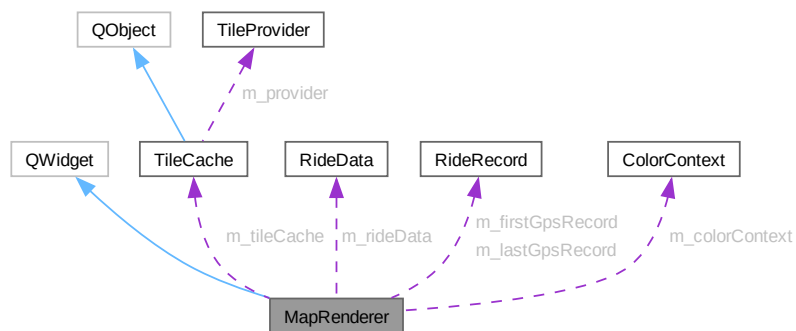
Interactive map widget for displaying and editing activity routes.

```
#include <MapRenderer.h>
```

Inheritance diagram for MapRenderer:



Collaboration diagram for MapRenderer:



Public Slots

- void [fitToTrack](#) ()
- void [fitToVisibleRange](#) ()
- void [zoomIn](#) ()
- void [zoomOut](#) ()
- void [setHighlightedElapsedSeconds](#) (double elapsedSeconds)

Signals

- void **segmentSelectionChanged** (double startSeconds, double endSeconds)
- void **segmentSelectionFinished** (double startSeconds, double endSeconds)

Public Member Functions

- [MapRenderer](#) (QWidget *parent=nullptr)
- void [setRideData](#) (const [RideData](#) &rideData, ColorMetric colorMetric=ColorMetric::None, const [ColorContext](#) &colorContext={})
- void [setVisibleTimeRange](#) (double startSeconds, double endSeconds)
- void [setCurrentTime](#) (double seconds)
- void [setRouteColorMode](#) (RouteColorMode mode, const [ColorContext](#) &colorContext={})
- void [setMapStyle](#) (MapStyle style)
- MapStyle [mapStyle](#) () const
- void [setAutoRouteContrast](#) (bool enabled)
- bool [autoRouteContrast](#) () const
- void [setTileCacheSize](#) (int maxTiles)

Protected Member Functions

- void [paintEvent](#) (QPaintEvent *event) override
- void [wheelEvent](#) (QWheelEvent *event) override
- void [mouseDoubleClickEvent](#) (QMouseEvent *event) override
- void [mousePressEvent](#) (QMouseEvent *event) override
- void [mouseMoveEvent](#) (QMouseEvent *event) override
- void [mouseReleaseEvent](#) (QMouseEvent *event) override

Private Types

- enum class **DragHandle** { **None** , **Start** , **End** }

Private Member Functions

- void [fitToBounds](#) (double minLat, double maxLat, double minLon, double maxLon, bool updateMinZoom)
- QPointF [tileToScreen](#) (const QPointF &tilePt) const
- QColor [adjustRouteColorForStyle](#) (const QColor &color) const
- const [RideRecord](#) * [nearestGpsRecord](#) (const QPointF &screenPos) const
- void [rebuildGpsCache](#) ()
- void [rebuildSegmentColors](#) ()
- const [RideRecord](#) * [gpsRecordAtTime](#) (double elapsedSeconds) const

Static Private Member Functions

- static QPointF [latLonToTile](#) (double lat, double lon, int zoom)

Private Attributes

- [TileCache](#) [m_tileCache](#)
- [RideData](#) [m_rideData](#)
- std::vector< const [RideRecord](#) * > [m_gpsRecords](#)
- std::vector< QColor > [m_segmentColors](#)
- const [RideRecord](#) * [m_firstGpsRecord](#) = nullptr
- const [RideRecord](#) * [m_lastGpsRecord](#) = nullptr
- bool [m_hasGps](#) = false
- RouteColorMode [m_routeColorMode](#) = RouteColorMode::None
- [ColorContext](#) [m_colorContext](#)

- double [m_minLat](#) = 90.0
- double [m_maxLat](#) = -90.0
- double [m_minLon](#) = 180.0
- double [m_maxLon](#) = -180.0
- double [m_centerLat](#) = 51.5
- double [m_centerLon](#) = 0.0
- int [m_zoom](#) = 13
- int [m_minZoom](#) = 1
- QPointF [m_panStart](#)
- bool [m_panning](#) = false
- DragHandle [m_dragHandle](#) = DragHandle::None
- QRectF [m_startMarkerRect](#)
- QRectF [m_endMarkerRect](#)
- bool [m_draggingSelection](#) = false
- bool [m_userMovedMap](#) = false
- bool [m_autoFitVisibleRange](#) = true
- bool [m_autoRouteContrast](#) = true
- double [m_highlightElapsedSeconds](#) = -1.0
- double [m_visibleStartSeconds](#) = -1.0
- double [m_visibleEndSeconds](#) = -1.0
- double [m_currentTimeSeconds](#) = -1.0
- QTimer [m_tileRepaintTimer](#)

7.64.1 Detailed Description

Interactive map widget for displaying and editing activity routes.

Renders map tiles with activity GPS trace, supports color-coding by metrics, time-range selection, and interacts with the main UI.

Definition at line 35 of file [MapRenderer.h](#).

7.64.2 Member Enumeration Documentation

7.64.2.1 DragHandle

```
enum class MapRenderer::DragHandle [strong], [private]
```

Definition at line 85 of file [MapRenderer.h](#).

7.64.3 Constructor & Destructor Documentation

7.64.3.1 MapRenderer()

```
MapRenderer::MapRenderer (
    QWidget * parent = nullptr) [explicit]
```

Definition at line 95 of file [MapRenderer.cpp](#).

7.64.4 Member Function Documentation

7.64.4.1 adjustRouteColorForStyle()

```
QColor MapRenderer::adjustRouteColorForStyle (
    const QColor & color) const [private]
```

Definition at line 310 of file [MapRenderer.cpp](#).

7.64.4.2 autoRouteContrast()

```
bool MapRenderer::autoRouteContrast () const [inline]
```

Definition at line 50 of file [MapRenderer.h](#).

7.64.4.3 fitToBounds()

```
void MapRenderer::fitToBounds (
    double minLat,
    double maxLat,
    double minLon,
    double maxLon,
    bool updateMinZoom) [private]
```

Definition at line 431 of file [MapRenderer.cpp](#).

7.64.4.4 fitToTrack

```
void MapRenderer::fitToTrack () [slot]
```

Definition at line 361 of file [MapRenderer.cpp](#).

7.64.4.5 fitToVisibleRange

```
void MapRenderer::fitToVisibleRange () [slot]
```

Definition at line 385 of file [MapRenderer.cpp](#).

7.64.4.6 gpsRecordAtTime()

```
const RideRecord * MapRenderer::gpsRecordAtTime (
    double elapsedSeconds) const [private]
```

Definition at line 203 of file [MapRenderer.cpp](#).

7.64.4.7 latLonToTile()

```
QPointF MapRenderer::latLonToTile (
    double lat,
    double lon,
    int zoom) [static], [private]
```

Definition at line 76 of file [MapRenderer.cpp](#).

7.64.4.8 mapStyle()

```
MapStyle MapRenderer::mapStyle () const [inline]
```

Definition at line 48 of file [MapRenderer.h](#).

7.64.4.9 mouseDoubleClickEvent()

```
void MapRenderer::mouseDoubleClickEvent (
    QMouseEvent * event) [override], [protected]
```

Definition at line 717 of file [MapRenderer.cpp](#).

7.64.4.10 mouseMoveEvent()

```
void MapRenderer::mouseMoveEvent (
    QMouseEvent * event) [override], [protected]
```

Definition at line 775 of file [MapRenderer.cpp](#).

7.64.4.11 mousePressEvent()

```
void MapRenderer::mousePressEvent (
    QMouseEvent * event) [override], [protected]
```

Definition at line 743 of file [MapRenderer.cpp](#).

7.64.4.12 mouseReleaseEvent()

```
void MapRenderer::mouseReleaseEvent (  
    QMouseEvent * event) [override], [protected]
```

Definition at line 849 of file [MapRenderer.cpp](#).

7.64.4.13 nearestGpsRecord()

```
const RideRecord * MapRenderer::nearestGpsRecord (  
    const QPointF & screenPos) const [private]
```

Definition at line 336 of file [MapRenderer.cpp](#).

7.64.4.14 paintEvent()

```
void MapRenderer::paintEvent (  
    QPaintEvent * event) [override], [protected]
```

Definition at line 479 of file [MapRenderer.cpp](#).

7.64.4.15 rebuildGpsCache()

```
void MapRenderer::rebuildGpsCache () [private]
```

Definition at line 183 of file [MapRenderer.cpp](#).

7.64.4.16 rebuildSegmentColors()

```
void MapRenderer::rebuildSegmentColors () [private]
```

Definition at line 115 of file [MapRenderer.cpp](#).

7.64.4.17 setAutoRouteContrast()

```
void MapRenderer::setAutoRouteContrast (  
    bool enabled)
```

Definition at line 297 of file [MapRenderer.cpp](#).

7.64.4.18 setCurrentTime()

```
void MapRenderer::setCurrentTime (  
    double seconds)
```

Definition at line 273 of file [MapRenderer.cpp](#).

7.64.4.19 setHighlightedElapsedSeconds

```
void MapRenderer::setHighlightedElapsedSeconds (  
    double elapsedSeconds) [slot]
```

Definition at line 472 of file [MapRenderer.cpp](#).

7.64.4.20 setMapStyle()

```
void MapRenderer::setMapStyle (  
    MapStyle style)
```

Definition at line 287 of file [MapRenderer.cpp](#).

7.64.4.21 setRideData()

```
void MapRenderer::setRideData (  
    const RideData & rideData,  
    ColorMetric colorMetric = ColorMetric::None,  
    const ColorContext & colorContext = {})
```

Definition at line 223 of file [MapRenderer.cpp](#).

7.64.4.22 setRouteColorMode()

```
void MapRenderer::setRouteColorMode (
    RouteColorMode mode,
    const ColorContext & colorContext = {})
```

Definition at line 279 of file [MapRenderer.cpp](#).

7.64.4.23 setTileCacheSize()

```
void MapRenderer::setTileCacheSize (
    int maxTiles)
```

Definition at line 305 of file [MapRenderer.cpp](#).

7.64.4.24 setVisibleTimeRange()

```
void MapRenderer::setVisibleTimeRange (
    double startSeconds,
    double endSeconds)
```

Definition at line 262 of file [MapRenderer.cpp](#).

7.64.4.25 tileToScreen()

```
QPointF MapRenderer::tileToScreen (
    const QPointF & tilePt) const [private]
```

Definition at line 86 of file [MapRenderer.cpp](#).

7.64.4.26 wheelEvent()

```
void MapRenderer::wheelEvent (
    QWheelEvent * event) [override], [protected]
```

Definition at line 676 of file [MapRenderer.cpp](#).

7.64.4.27 zoomIn

```
void MapRenderer::zoomIn () [slot]
```

Definition at line 456 of file [MapRenderer.cpp](#).

7.64.4.28 zoomOut

```
void MapRenderer::zoomOut () [slot]
```

Definition at line 464 of file [MapRenderer.cpp](#).

7.64.5 Member Data Documentation

7.64.5.1 m_autoFitVisibleRange

```
bool MapRenderer::m_autoFitVisibleRange = true [private]
```

Definition at line 131 of file [MapRenderer.h](#).

7.64.5.2 m_autoRouteContrast

```
bool MapRenderer::m_autoRouteContrast = true [private]
```

Definition at line 132 of file [MapRenderer.h](#).

7.64.5.3 m_centerLat

```
double MapRenderer::m_centerLat = 51.5 [private]
```

Definition at line 119 of file [MapRenderer.h](#).

7.64.5.4 m_centerLon

double MapRenderer::m_centerLon = 0.0 [private]
Definition at line 120 of file [MapRenderer.h](#).

7.64.5.5 m_colorContext

[ColorContext](#) MapRenderer::m_colorContext [private]
Definition at line 114 of file [MapRenderer.h](#).

7.64.5.6 m_currentTimeSeconds

double MapRenderer::m_currentTimeSeconds = -1.0 [private]
Definition at line 136 of file [MapRenderer.h](#).

7.64.5.7 m_draggingSelection

bool MapRenderer::m_draggingSelection = false [private]
Definition at line 129 of file [MapRenderer.h](#).

7.64.5.8 m_dragHandle

[DragHandle](#) MapRenderer::m_dragHandle = [DragHandle::None](#) [private]
Definition at line 126 of file [MapRenderer.h](#).

7.64.5.9 m_endMarkerRect

[QRectF](#) MapRenderer::m_endMarkerRect [private]
Definition at line 128 of file [MapRenderer.h](#).

7.64.5.10 m_firstGpsRecord

const [RideRecord*](#) MapRenderer::m_firstGpsRecord = nullptr [private]
Definition at line 110 of file [MapRenderer.h](#).

7.64.5.11 m_gpsRecords

[std::vector](#)<const [RideRecord*](#)> MapRenderer::m_gpsRecords [private]
Definition at line 106 of file [MapRenderer.h](#).

7.64.5.12 m_hasGps

bool MapRenderer::m_hasGps = false [private]
Definition at line 112 of file [MapRenderer.h](#).

7.64.5.13 m_highlightElapsedSeconds

double MapRenderer::m_highlightElapsedSeconds = -1.0 [private]
Definition at line 133 of file [MapRenderer.h](#).

7.64.5.14 m_lastGpsRecord

const [RideRecord*](#) MapRenderer::m_lastGpsRecord = nullptr [private]
Definition at line 111 of file [MapRenderer.h](#).

7.64.5.15 m_maxLat

double MapRenderer::m_maxLat = -90.0 [private]
Definition at line 116 of file [MapRenderer.h](#).

7.64.5.16 m_maxLon

double MapRenderer::m_maxLon = -180.0 [private]
Definition at line 117 of file [MapRenderer.h](#).

7.64.5.17 m_minLat

double MapRenderer::m_minLat = 90.0 [private]
Definition at line 116 of file [MapRenderer.h](#).

7.64.5.18 m_minLon

double MapRenderer::m_minLon = 180.0 [private]
Definition at line 117 of file [MapRenderer.h](#).

7.64.5.19 m_minZoom

int MapRenderer::m_minZoom = 1 [private]
Definition at line 122 of file [MapRenderer.h](#).

7.64.5.20 m_panning

bool MapRenderer::m_panning = false [private]
Definition at line 125 of file [MapRenderer.h](#).

7.64.5.21 m_panStart

QPointF MapRenderer::m_panStart [private]
Definition at line 124 of file [MapRenderer.h](#).

7.64.5.22 m_rideData

[RideData](#) MapRenderer::m_rideData [private]
Definition at line 102 of file [MapRenderer.h](#).

7.64.5.23 m_routeColorMode

RouteColorMode MapRenderer::m_routeColorMode = RouteColorMode::None [private]
Definition at line 113 of file [MapRenderer.h](#).

7.64.5.24 m_segmentColors

std::vector<QColor> MapRenderer::m_segmentColors [private]
Definition at line 109 of file [MapRenderer.h](#).

7.64.5.25 m_startMarkerRect

QRectF MapRenderer::m_startMarkerRect [private]
Definition at line 127 of file [MapRenderer.h](#).

7.64.5.26 m_tileCache

[TileCache](#) MapRenderer::m_tileCache [private]
Definition at line 101 of file [MapRenderer.h](#).

7.64.5.27 m_tileRepaintTimer

QTimer MapRenderer::m_tileRepaintTimer [private]
Definition at line 137 of file [MapRenderer.h](#).

7.64.5.28 m_userMovedMap

`bool MapRenderer::m_userMovedMap = false [private]`
Definition at line 130 of file [MapRenderer.h](#).

7.64.5.29 m_visibleEndSeconds

`double MapRenderer::m_visibleEndSeconds = -1.0 [private]`
Definition at line 135 of file [MapRenderer.h](#).

7.64.5.30 m_visibleStartSeconds

`double MapRenderer::m_visibleStartSeconds = -1.0 [private]`
Definition at line 134 of file [MapRenderer.h](#).

7.64.5.31 m_zoom

`int MapRenderer::m_zoom = 13 [private]`
Definition at line 121 of file [MapRenderer.h](#).

The documentation for this class was generated from the following files:

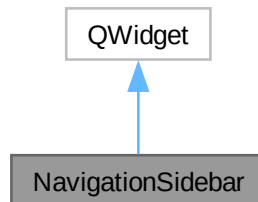
- `src/maps/MapRenderer.h`
- `src/maps/MapRenderer.cpp`

7.65 NavigationSidebar Class Reference

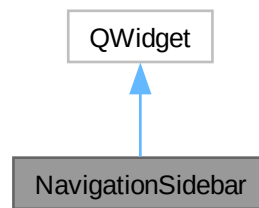
Main navigation sidebar for app pages.

```
#include <NavigationSidebar.h>
```

Inheritance diagram for NavigationSidebar:



Collaboration diagram for NavigationSidebar:



Public Types

- enum class **Page** {
 Dashboard = 0 , **Calendar** , **Activities** , **Analysis** ,
 Athletes , **Settings** }

Signals

- void **pageSelected** (NavigationSidebar::Page page)

Public Member Functions

- [NavigationSidebar](#) (QWidget *parent=nullptr)
- void [setCurrentPage](#) (Page page)
- Page [currentPage](#) () const

Private Attributes

- QListWidget * [m_list](#) = nullptr

7.65.1 Detailed Description

Main navigation sidebar for app pages.

Definition at line 14 of file [NavigationSidebar.h](#).

7.65.2 Member Enumeration Documentation

7.65.2.1 Page

```
enum class NavigationSidebar::Page [strong]
```

Definition at line 19 of file [NavigationSidebar.h](#).

7.65.3 Constructor & Destructor Documentation

7.65.3.1 NavigationSidebar()

```
NavigationSidebar::NavigationSidebar (  
    QWidget * parent = nullptr) [explicit]
```

Definition at line 9 of file [NavigationSidebar.cpp](#).

7.65.4 Member Function Documentation

7.65.4.1 currentPage()

NavigationSidebar::Page NavigationSidebar::currentPage () const
Definition at line 38 of file [NavigationSidebar.cpp](#).

7.65.4.2 setCurrentPage()

```
void NavigationSidebar::setCurrentPage (  
    Page page)
```

Definition at line 32 of file [NavigationSidebar.cpp](#).

7.65.5 Member Data Documentation

7.65.5.1 m_list

```
QListWidget* NavigationSidebar::m_list = nullptr [private]  
Definition at line 40 of file NavigationSidebar.h.
```

The documentation for this class was generated from the following files:

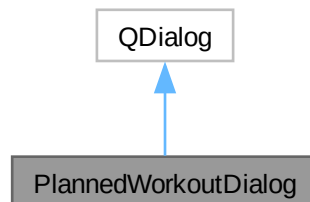
- [src/gui/NavigationSidebar.h](#)
- [src/gui/NavigationSidebar.cpp](#)

7.66 PlannedWorkoutDialog Class Reference

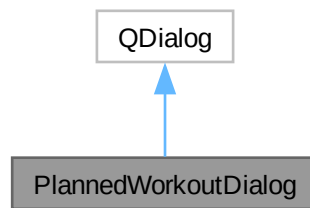
Dialog for creating or editing planned workouts.

```
#include <PlannedWorkoutDialog.h>
```

Inheritance diagram for PlannedWorkoutDialog:



Collaboration diagram for PlannedWorkoutDialog:



Public Member Functions

- [PlannedWorkoutDialog](#) (QWidget *parent=nullptr)
- [PlannedWorkoutDialog](#) (const [PlannedWorkoutDraft](#) &draft, QWidget *parent=nullptr)
- [PlannedWorkoutDraft](#) draft () const

Private Attributes

- QDateEdit * [m_dateEdit](#) = nullptr
- QLineEdit * [m_titleEdit](#) = nullptr
- QLineEdit * [m_typeEdit](#) = nullptr
- QDoubleSpinBox * [m_durationSpin](#) = nullptr
- QDoubleSpinBox * [m_tssSpin](#) = nullptr
- QTextEdit * [m_notesEdit](#) = nullptr
- int [m_id](#) = -1

7.66.1 Detailed Description

Dialog for creating or editing planned workouts.
Definition at line 46 of file [PlannedWorkoutDialog.h](#).

7.66.2 Constructor & Destructor Documentation

7.66.2.1 PlannedWorkoutDialog() [1/2]

```
PlannedWorkoutDialog::PlannedWorkoutDialog (
    QWidget * parent = nullptr) [explicit]
```

Definition at line 16 of file [PlannedWorkoutDialog.cpp](#).

7.66.2.2 PlannedWorkoutDialog() [2/2]

```
PlannedWorkoutDialog::PlannedWorkoutDialog (
    const PlannedWorkoutDraft & draft,
    QWidget * parent = nullptr) [explicit]
```

Definition at line 20 of file [PlannedWorkoutDialog.cpp](#).

7.66.3 Member Function Documentation

7.66.3.1 draft()

```
PlannedWorkoutDraft PlannedWorkoutDialog::draft () const
```

Definition at line 69 of file [PlannedWorkoutDialog.cpp](#).

7.66.4 Member Data Documentation

7.66.4.1 m_dateEdit

`QDateEdit* PlannedWorkoutDialog::m_dateEdit = nullptr [private]`

Definition at line 58 of file [PlannedWorkoutDialog.h](#).

7.66.4.2 m_durationSpin

`QDoubleSpinBox* PlannedWorkoutDialog::m_durationSpin = nullptr [private]`

Definition at line 61 of file [PlannedWorkoutDialog.h](#).

7.66.4.3 m_id

`int PlannedWorkoutDialog::m_id = -1 [private]`

Definition at line 64 of file [PlannedWorkoutDialog.h](#).

7.66.4.4 m_notesEdit

`QTextEdit* PlannedWorkoutDialog::m_notesEdit = nullptr [private]`

Definition at line 63 of file [PlannedWorkoutDialog.h](#).

7.66.4.5 m_titleEdit

`QLineEdit* PlannedWorkoutDialog::m_titleEdit = nullptr [private]`

Definition at line 59 of file [PlannedWorkoutDialog.h](#).

7.66.4.6 m_tssSpin

`QDoubleSpinBox* PlannedWorkoutDialog::m_tssSpin = nullptr [private]`

Definition at line 62 of file [PlannedWorkoutDialog.h](#).

7.66.4.7 m_typeEdit

`QLineEdit* PlannedWorkoutDialog::m_typeEdit = nullptr [private]`

Definition at line 60 of file [PlannedWorkoutDialog.h](#).

The documentation for this class was generated from the following files:

- [src/gui/PlannedWorkoutDialog.h](#)
- [src/gui/PlannedWorkoutDialog.cpp](#)

7.67 PlannedWorkoutDraft Struct Reference

Draft data for a planned workout.

```
#include <PlannedWorkoutDialog.h>
```

Public Attributes

- `int id = -1`
- `QDate date`
- `QString title`
- `QString type`
- `double durationSeconds = 0.0`
- `double targetTss = 0.0`
- `QString notes`

7.67.1 Detailed Description

Draft data for a planned workout.

Definition at line 14 of file [PlannedWorkoutDialog.h](#).

7.67.2 Member Data Documentation

7.67.2.1 date

`QDate PlannedWorkoutDraft::date`

Definition at line 20 of file [PlannedWorkoutDialog.h](#).

7.67.2.2 durationSeconds

`double PlannedWorkoutDraft::durationSeconds = 0.0`

Definition at line 29 of file [PlannedWorkoutDialog.h](#).

7.67.2.3 id

`int PlannedWorkoutDraft::id = -1`

Definition at line 17 of file [PlannedWorkoutDialog.h](#).

7.67.2.4 notes

`QString PlannedWorkoutDraft::notes`

Definition at line 35 of file [PlannedWorkoutDialog.h](#).

7.67.2.5 targetTss

`double PlannedWorkoutDraft::targetTss = 0.0`

Definition at line 32 of file [PlannedWorkoutDialog.h](#).

7.67.2.6 title

`QString PlannedWorkoutDraft::title`

Definition at line 23 of file [PlannedWorkoutDialog.h](#).

7.67.2.7 type

`QString PlannedWorkoutDraft::type`

Definition at line 26 of file [PlannedWorkoutDialog.h](#).

The documentation for this struct was generated from the following file:

- [src/gui/PlannedWorkoutDialog.h](#)

7.68 PowerCurve Class Reference

Computes power duration curve (PDC) for an activity.

`#include <PowerCurve.h>`

Static Public Member Functions

- static `std::vector< double >` [standardDurations](#) ()
Returns standard test durations in seconds.
- static `double` [bestMeanPower](#) (const [RideData](#) &rideData, double durationSeconds)
Calculates best mean power over a duration.
- static `std::vector< PowerCurvePoint >` [computeStandard](#) (const [RideData](#) &rideData)
Computes power duration curve for standard durations.

7.68.1 Detailed Description

Computes power duration curve (PDC) for an activity.

Analyzes best sustained power outputs across a range of durations.

Definition at line 27 of file [PowerCurve.h](#).

7.68.2 Member Function Documentation

7.68.2.1 bestMeanPower()

```
double PowerCurve::bestMeanPower (
    const RideData & rideData,
    double durationSeconds) [static]
```

Calculates best mean power over a duration.

Parameters

<i>rideData</i>	Activity data.
<i>durationSeconds</i>	Duration window in seconds.

Returns

Best mean power in watts.

Definition at line 18 of file [PowerCurve.cpp](#).

References [RideData::records](#).

Referenced by [computeStandard\(\)](#), and [PowerCurveWidget::setRideDataWithComparisons\(\)](#).

Here is the caller graph for this function:



7.68.2.2 computeStandard()

```
std::vector< PowerCurvePoint > PowerCurve::computeStandard (
    const RideData & rideData) [static]
```

Computes power duration curve for standard durations.

Parameters

<i>rideData</i>	Activity data.
-----------------	--------------------------------

Returns

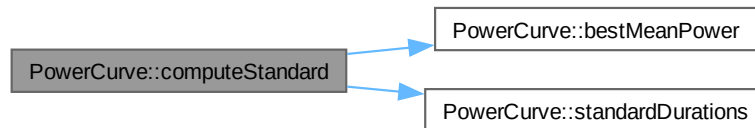
PDC points (omits durations longer than the ride).

Definition at line 47 of file [PowerCurve.cpp](#).

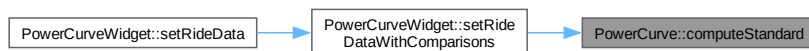
References [bestMeanPower\(\)](#), and [standardDurations\(\)](#).

Referenced by [PowerCurveWidget::setRideDataWithComparisons\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:

**7.68.2.3 standardDurations()**

```
std::vector< double > PowerCurve::standardDurations () [static]
```

Returns standard test durations in seconds.

Returns

Durations from 5 seconds to 2 hours.

Definition at line 9 of file [PowerCurve.cpp](#).

Referenced by [computeStandard\(\)](#), and [PowerCurveWidget::setRideDataWithComparisons\(\)](#).

Here is the caller graph for this function:



The documentation for this class was generated from the following files:

- `src/analysis/PowerCurve.h`
- `src/analysis/PowerCurve.cpp`

7.69 PowerCurvePoint Struct Reference

A single point on a power duration curve.

```
#include <PowerCurve.h>
```


Public Attributes

- double [durationSeconds](#)
Duration in seconds.
- double [bestPower](#)
Best mean power for duration in watts.

7.69.1 Detailed Description

A single point on a power duration curve.
Definition at line 13 of file [PowerCurve.h](#).

7.69.2 Member Data Documentation

7.69.2.1 bestPower

```
double PowerCurvePoint::bestPower
```

Best mean power for duration in watts.
Definition at line 19 of file [PowerCurve.h](#).

7.69.2.2 durationSeconds

```
double PowerCurvePoint::durationSeconds
```

Duration in seconds.
Definition at line 16 of file [PowerCurve.h](#).
The documentation for this struct was generated from the following file:

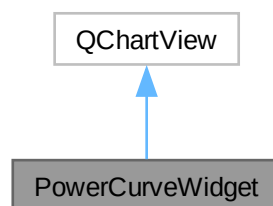
- `src/analysis/PowerCurve.h`

7.70 PowerCurveWidget Class Reference

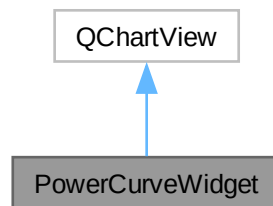
Displays power output over duration curve.

```
#include <PowerCurveWidget.h>
```

Inheritance diagram for PowerCurveWidget:



Collaboration diagram for PowerCurveWidget:



Public Member Functions

- [PowerCurveWidget](#) (QWidget *parent=nullptr)
Constructs power curve chart widget.
- void [setRideData](#) (const [RideData](#) &rideData, double ftp=0.0)
Sets ride data to display.
- void [setRideDataWithComparisons](#) (const [RideData](#) &rideData, const std::vector< [PowerCurvePoint](#) > &last90, const std::vector< [PowerCurvePoint](#) > ¤tYear, const std::vector< [PowerCurvePoint](#) > &allTime, double ftp=0.0)
Sets ride data with comparison curves.
- void [clearChart](#) ()
Clears the chart.

7.70.1 Detailed Description

Displays power output over duration curve.

Shows best mean power at various durations with optional comparison curves (last 90 days, current year, all-time).

Definition at line 17 of file [PowerCurveWidget.h](#).

7.70.2 Constructor & Destructor Documentation

7.70.2.1 PowerCurveWidget()

```
PowerCurveWidget::PowerCurveWidget (
    QWidget * parent = nullptr) [explicit]
```

Constructs power curve chart widget.

Parameters

<i>parent</i>	Parent widget.
---------------	----------------

Definition at line 55 of file [PowerCurveWidget.cpp](#).

7.70.3 Member Function Documentation

7.70.3.1 clearChart()

```
void PowerCurveWidget::clearChart ()
```

Clears the chart.

Definition at line 66 of file [PowerCurveWidget.cpp](#).

Referenced by [setRideDataWithComparisons\(\)](#).

Here is the caller graph for this function:



7.70.3.2 setRideData()

```
void PowerCurveWidget::setRideData (
    const RideData & rideData,
    double ftp = 0.0)
```

Sets ride data to display.

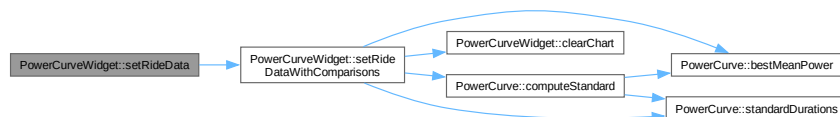
Parameters

<i>rideData</i>	Activity data.
<i>ftp</i>	Functional threshold power (default 0).

Definition at line 74 of file [PowerCurveWidget.cpp](#).

References [setRideDataWithComparisons\(\)](#).

Here is the call graph for this function:



7.70.3.3 setRideDataWithComparisons()

```
void PowerCurveWidget::setRideDataWithComparisons (
    const RideData & rideData,
    const std::vector< PowerCurvePoint > & last90,
    const std::vector< PowerCurvePoint > & currentYear,
    const std::vector< PowerCurvePoint > & allTime,
    double ftp = 0.0)
```

Sets ride data with comparison curves.

Parameters

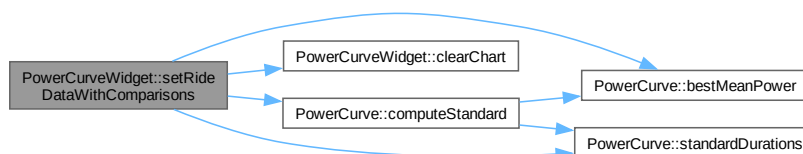
<i>rideData</i>	Current activity data.
<i>last90</i>	Power curve for last 90 days.
<i>currentYear</i>	Power curve for current year.
<i>allTime</i>	Power curve for all time.
<i>ftp</i>	Functional threshold power (default 0).

Definition at line 79 of file [PowerCurveWidget.cpp](#).

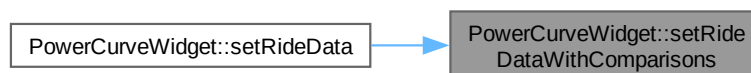
References [PowerCurve::bestMeanPower\(\)](#), [clearChart\(\)](#), [PowerCurve::computeStandard\(\)](#), and [PowerCurve::standardDurations\(\)](#).

Referenced by [setRideData\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



The documentation for this class was generated from the following files:

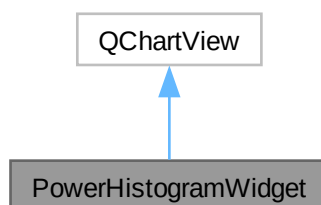
- `src/charts/PowerCurveWidget.h`
- `src/charts/PowerCurveWidget.cpp`

7.71 PowerHistogramWidget Class Reference

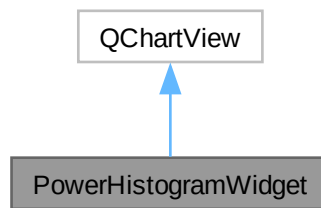
Displays power distribution histogram.

```
#include <PowerHistogramWidget.h>
```

Inheritance diagram for `PowerHistogramWidget`:



Collaboration diagram for PowerHistogramWidget:



Public Member Functions

- [PowerHistogramWidget](#) (`QWidget *parent=nullptr`)
Constructs power histogram chart widget.
- void [setRideData](#) (const [RideData](#) &rideData)
Sets ride data to display.
- void [clearChart](#) ()
Clears the chart.

7.71.1 Detailed Description

Displays power distribution histogram.
Shows frequency distribution of power output across power range.
Definition at line 15 of file [PowerHistogramWidget.h](#).

7.71.2 Constructor & Destructor Documentation

7.71.2.1 PowerHistogramWidget()

```
PowerHistogramWidget::PowerHistogramWidget (
    QWidget * parent = nullptr) [explicit]
```

Constructs power histogram chart widget.

Parameters

<i>parent</i>	Parent widget.
---------------	----------------

Definition at line 17 of file [PowerHistogramWidget.cpp](#).

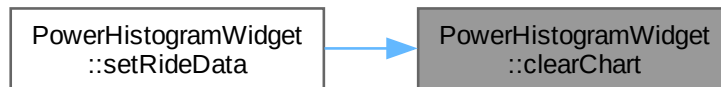
7.71.3 Member Function Documentation

7.71.3.1 clearChart()

```
void PowerHistogramWidget::clearChart ()
```

Clears the chart.
Definition at line 28 of file [PowerHistogramWidget.cpp](#).
Referenced by [setRideData\(\)](#).

Here is the caller graph for this function:



7.71.3.2 setRideData()

```
void PowerHistogramWidget::setRideData (
    const RideData & rideData)
```

Sets ride data to display.

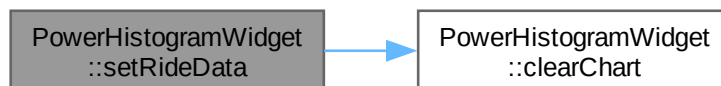
Parameters

<i>rideData</i>	Activity data.
-----------------	----------------

Definition at line 36 of file [PowerHistogramWidget.cpp](#).

References [clearChart\(\)](#), and [RideData::records](#).

Here is the call graph for this function:



The documentation for this class was generated from the following files:

- `src/charts/PowerHistogramWidget.h`
- `src/charts/PowerHistogramWidget.cpp`

7.72 PowerZone Struct Reference

Represents a single power zone with time spent.

```
#include <FTPZones.h>
```

Public Attributes

- int [number](#)
Zone number (1-7).
- std::string [name](#)
Zone name (Z1, Z2, etc.).
- double [lowerPct](#)
Lower bound as fraction of FTP (inclusive).

- double `upperPct`
Upper bound as fraction of FTP (exclusive), -1.0 for unbounded.
- double `seconds` = 0.0
Time spent in zone in seconds.

7.72.1 Detailed Description

Represents a single power zone with time spent.
Definition at line 14 of file [FTPZones.h](#).

7.72.2 Member Data Documentation

7.72.2.1 lowerPct

`double PowerZone::lowerPct`
Lower bound as fraction of FTP (inclusive).
Definition at line 23 of file [FTPZones.h](#).

7.72.2.2 name

`std::string PowerZone::name`
`Zone` name (Z1, Z2, etc.).
Definition at line 20 of file [FTPZones.h](#).

7.72.2.3 number

`int PowerZone::number`
`Zone` number (1-7).
Definition at line 17 of file [FTPZones.h](#).

7.72.2.4 seconds

`double PowerZone::seconds` = 0.0
Time spent in zone in seconds.
Definition at line 29 of file [FTPZones.h](#).

7.72.2.5 upperPct

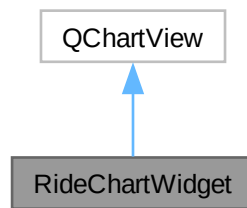
`double PowerZone::upperPct`
Upper bound as fraction of FTP (exclusive), -1.0 for unbounded.
Definition at line 26 of file [FTPZones.h](#).
The documentation for this struct was generated from the following file:

- `src/analysis/FTPZones.h`

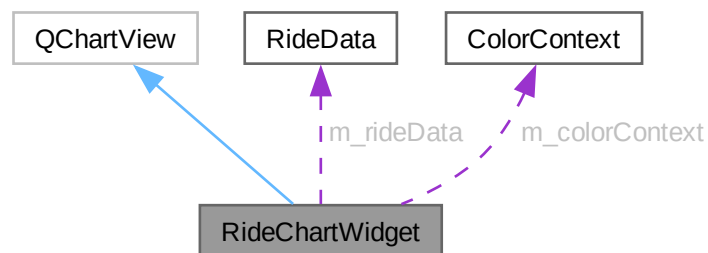
7.73 RideChartWidget Class Reference

Interactive chart displaying ride data by selected metric.
`#include <RideChartWidget.h>`

Inheritance diagram for RideChartWidget:



Collaboration diagram for RideChartWidget:



Public Slots

- void [setXRange](#) (double min, double max)
@slot Sets visible x-axis range.
- void [setCrosshair](#) (double xMinutes)
@slot Sets crosshair position.
- void [resetZoom](#) ()
@slot Resets zoom to full ride extent.
- void [fitToData](#) ()
@slot Alias for [resetZoom\(\)](#).

Signals

- void [xRangeChanged](#) (double min, double max)
@signal Emitted when visible x range changes.
- void [visibleRangeChanged](#) (double startMinutes, double endMinutes)
@signal Emitted when visible time range updates.
- void [crosshairMoved](#) (double xMinutes)
@signal Emitted when crosshair moves.
- void [intervalSelectionRequested](#) (double startSeconds, double endSeconds)
@signal Emitted when user selects time interval.

- void [climbBoundaryEdited](#) (double oldStartSeconds, double oldEndSeconds, double newStartSeconds, double newEndSeconds)
@signal Emitted when climb boundaries are edited.
- void [newClimbRequested](#) (double startSeconds, double endSeconds)
@signal Emitted when user creates new climb.
- void [climbClicked](#) (int climbIndex)
@signal Emitted when climb is clicked.

Public Member Functions

- [RideChartWidget](#) (Metric metric, QWidget *parent=nullptr)
Constructs ride chart for specified metric.
- void [setRideData](#) (const [RideData](#) &rideData, ColorMetric colorMetric=ColorMetric::None, const [ColorContext](#) &colorContext={})
Sets ride data to display.
- void [setPowerSmoothingSeconds](#) (int seconds)
Sets power smoothing window.
- void [setAutoSmoothingEnabled](#) (bool enabled)
Enables/disables automatic smoothing.
- void [setIntervals](#) (const std::vector< [Interval](#) > &intervals)
Sets interval overlays.
- void [setClimbs](#) (const std::vector< [Climb](#) > &climbs)
Sets climb overlays.
- void [setMetricOverlay](#) (ColorMetric metric, bool enabled)
Enables/disables metric overlay.
- void [setSelectedClimbIndex](#) (int index)
Highlights a specific climb.
- void [setClimbEditingEnabled](#) (bool enabled)
Enables/disables climb editing mode.
- void [clearChart](#) ()
Clears the chart.
- bool [hasData](#) () const
Checks if chart has data.
- double [visibleRangeStartMinutes](#) () const
Gets visible range start time.
- double [visibleRangeEndMinutes](#) () const
Gets end time of visible range in minutes from activity start.

Protected Member Functions

- void [wheelEvent](#) (QWheelEvent *event) override
Handles wheel scroll for panning and zooming.
- void [mousePressEvent](#) (QMouseEvent *event) override
Handles mouse press for chart interaction.
- void [mouseReleaseEvent](#) (QMouseEvent *event) override
Handles mouse release for chart interaction.
- void [mouseDoubleClickEvent](#) (QMouseEvent *event) override
Handles double-click for interval creation.
- void [mouseMoveEvent](#) (QMouseEvent *event) override
Handles mouse movement for crosshair and selection.
- void [leaveEvent](#) (QEvent *event) override
Handles mouse leaving widget.

- void [drawForeground](#) (QPainter *painter, const QRectF &rect) override
Renders chart foreground elements.
- void [resizeEvent](#) (QResizeEvent *event) override
Handles widget resize events.

Private Types

- enum class **ClimbHandleDrag** { **None** , **Start** , **End** }

Private Member Functions

- void [rebuildChart](#) (bool preserveXRange=true)
- std::vector< double > [computeSmoothedSeries](#) (Metric metric, int seconds) const
- std::vector< double > [computeSmoothedPowerSeries](#) (int seconds) const
- std::vector< int > [buildSampleIndices](#) (const std::vector< double > &values, int targetPointBudget) const
- void [clearBackgroundSeries](#) ()
- void [addZoneBackgroundBands](#) (double minY, double maxY, double maxXMinutes)
- void [addMetricOverlayBands](#) (double minY, double maxY)
- void [addIntervalBackgroundBands](#) (double minY, double maxY)
- void [addClimbBackgroundBands](#) (double minY, double maxY)
- void [updateXTicks](#) (double minVal, double maxVal)
- void [updateYAxis](#) ()
- void [updateYAxisForVisibleRange](#) (double xMinMin, double xMaxMin)

Private Attributes

- Metric [m_metric](#)
- QLineSeries * [m_series](#) = nullptr
- std::vector< QLineSeries * > [m_rawSeries](#)
- std::vector< QLineSeries * > [m_colorSeries](#)
- std::vector< QLineSeries * > [m_overlaySeries](#)
- std::vector< QLineSeries * > [m_referenceSeries](#)
- std::vector< QAreaSeries * > [m_backgroundSeries](#)
- QValueAxis * [m_axisX](#) = nullptr
- QCategoryAxis * [m_axisXDisplay](#) = nullptr
- QValueAxis * [m_axisY](#) = nullptr
- QValueAxis * [m_axisYOverlay](#) = nullptr
- [RideData](#) [m_rideData](#)
- std::vector< [Interval](#) > [m_intervals](#)
- std::vector< [Climb](#) > [m_climbs](#)
- bool [m_metricOverlayEnabled](#) = false
- ColorMetric [m_metricOverlayMetric](#) = ColorMetric::None
- ColorMetric [m_colorMetric](#) = ColorMetric::None
- [ColorContext](#) [m_colorContext](#)
- std::unordered_map< int, std::vector< double > > [m_smoothedSeriesCache](#)
- std::vector< QPointF > [m_tooltipPoints](#)
- std::vector< double > [m_tooltipRawValues](#)
- std::vector< double > [m_tooltipDisplayValues](#)
- int [m_requestedPowerSmoothingSeconds](#) = 0
- int [m_effectivePowerSmoothingSeconds](#) = 0
- bool [m_autoSmoothing](#) = false
- bool [m_isRebuilding](#) = false
- bool [m_syncing](#) = false
- bool [m_hasData](#) = false

- struct {
 - double [cachedMinX](#) = -1.0
 - double [cachedMaxX](#) = -1.0
 - double [cachedAxisMin](#) = 0.0
 - double [cachedAxisMax](#) = 1.0
 - bool [valid](#) = false
- } **m_yAxisRangeCache**
- double [m_origMaxX](#) = 1.0
- double [m_crosshairX](#) = -1.0
- double [m_crosshairY](#) = std::numeric_limits<double>::quiet_NaN()
- bool [m_intervalSelecting](#) = false
- double [m_intervalStartMin](#) = -1.0
- double [m_intervalEndMin](#) = -1.0
- bool [m_climbEditingEnabled](#) = false
- int [m_selectedClimbIndex](#) = -1
- ClimbHandleDrag [m_climbHandleDrag](#) = ClimbHandleDrag::None
- double [m_dragOriginalClimbStartSec](#) = -1.0
- double [m_dragOriginalClimbEndSec](#) = -1.0
- bool [m_newClimbCreating](#) = false
- bool [m_newClimbDragging](#) = false
- double [m_newClimbStartSec](#) = -1.0
- double [m_newClimbCurrentSec](#) = -1.0
- double [m_dataMinY](#) = 0.0
- double [m_dataMaxY](#) = 1.0

7.73.1 Detailed Description

Interactive chart displaying ride data by selected metric.

Shows power, heart rate, cadence, speed, or altitude over time with overlays for intervals, climbs, and zone visualization.

Definition at line 29 of file [RideChartWidget.h](#).

7.73.2 Member Enumeration Documentation

7.73.2.1 ClimbHandleDrag

```
enum class RideChartWidget::ClimbHandleDrag [strong], [private]
```

Definition at line 257 of file [RideChartWidget.h](#).

7.73.3 Constructor & Destructor Documentation

7.73.3.1 RideChartWidget()

```
RideChartWidget::RideChartWidget (
    Metric metric,
    QWidget * parent = nullptr) [explicit]
```

Constructs ride chart for specified metric.

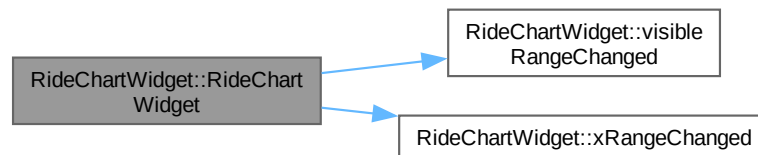
Parameters

<i>metric</i>	Metric to display (Power, HeartRate, etc.).
<i>parent</i>	Parent widget.

Definition at line 232 of file [RideChartWidget.cpp](#).

References [visibleRangeChanged\(\)](#), and [xRangeChanged\(\)](#).

Here is the call graph for this function:



7.73.4 Member Function Documentation

7.73.4.1 addClimbBackgroundBands()

```
void RideChartWidget::addClimbBackgroundBands (
    double minY,
    double maxY) [private]
```

Definition at line 695 of file [RideChartWidget.cpp](#).

7.73.4.2 addIntervalBackgroundBands()

```
void RideChartWidget::addIntervalBackgroundBands (
    double minY,
    double maxY) [private]
```

Definition at line 584 of file [RideChartWidget.cpp](#).

7.73.4.3 addMetricOverlayBands()

```
void RideChartWidget::addMetricOverlayBands (
    double minY,
    double maxY) [private]
```

Definition at line 616 of file [RideChartWidget.cpp](#).

7.73.4.4 addZoneBackgroundBands()

```
void RideChartWidget::addZoneBackgroundBands (
    double minY,
    double maxY,
    double maxXMinutes) [private]
```

Definition at line 548 of file [RideChartWidget.cpp](#).

7.73.4.5 buildSampleIndices()

```
std::vector< int > RideChartWidget::buildSampleIndices (
    const std::vector< double > & values,
    int targetPointBudget) const [private]
```

Definition at line 451 of file [RideChartWidget.cpp](#).

7.73.4.6 clearBackgroundSeries()

```
void RideChartWidget::clearBackgroundSeries () [private]
```

Definition at line 538 of file [RideChartWidget.cpp](#).

7.73.4.7 clearChart()

```
void RideChartWidget::clearChart ()
```

Clears the chart.

Definition at line 1323 of file [RideChartWidget.cpp](#).

7.73.4.8 climbBoundaryEdited

```
void RideChartWidget::climbBoundaryEdited (
    double oldStartSeconds,
    double oldEndSeconds,
    double newStartSeconds,
    double newEndSeconds) [signal]
```

@signal Emitted when climb boundaries are edited.

Parameters

<i>oldStartSeconds</i>	Original start time in seconds.
<i>oldEndSeconds</i>	Original end time in seconds.
<i>newStartSeconds</i>	New start time in seconds.
<i>newEndSeconds</i>	New end time in seconds.

Referenced by [mouseReleaseEvent\(\)](#).

Here is the caller graph for this function:

**7.73.4.9 climbClicked**

```
void RideChartWidget::climbClicked (
    int climbIndex) [signal]
```

@signal Emitted when climb is clicked.

Parameters

<i>climbIndex</i>	Index of clicked climb.
-------------------	-------------------------

Referenced by [mousePressEvent\(\)](#).

Here is the caller graph for this function:



7.73.4.10 computeSmoothedPowerSeries()

```
std::vector< double > RideChartWidget::computeSmoothedPowerSeries (
    int seconds) const [private]
```

Definition at line 376 of file [RideChartWidget.cpp](#).

7.73.4.11 computeSmoothedSeries()

```
std::vector< double > RideChartWidget::computeSmoothedSeries (
    Metric metric,
    int seconds) const [private]
```

Definition at line 381 of file [RideChartWidget.cpp](#).

7.73.4.12 crosshairMoved

```
void RideChartWidget::crosshairMoved (
    double xMinutes) [signal]
```

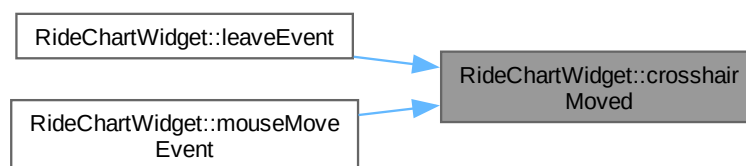
@signal Emitted when crosshair moves.

Parameters

<i>xMinutes</i>	Crosshair time in minutes.
-----------------	----------------------------

Referenced by [leaveEvent\(\)](#), and [mouseMoveEvent\(\)](#).

Here is the caller graph for this function:

**7.73.4.13 drawForeground()**

```
void RideChartWidget::drawForeground (
    QPainter * painter,
    const QRectF & rect) [override], [protected]
```

Renders chart foreground elements.

Definition at line 1898 of file [RideChartWidget.cpp](#).

References [Climb::endSeconds](#), and [Climb::startSeconds](#).

7.73.4.14 fitToData

```
void RideChartWidget::fitToData () [slot]
```

@slot Alias for [resetZoom\(\)](#).

Definition at line 1422 of file [RideChartWidget.cpp](#).

References [resetZoom\(\)](#).

Here is the call graph for this function:



7.73.4.15 hasData()

```
bool RideChartWidget::hasData () const [inline]
```

Checks if chart has data.

Returns

True if data is displayed.

Definition at line 103 of file [RideChartWidget.h](#).

7.73.4.16 intervalSelectionRequested

```
void RideChartWidget::intervalSelectionRequested (
    double startSeconds,
    double endSeconds) [signal]
```

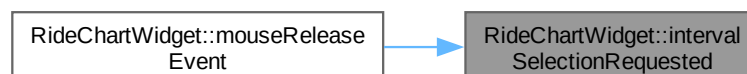
@signal Emitted when user selects time interval.

Parameters

<i>startSeconds</i>	Start time in seconds from activity start.
<i>endSeconds</i>	End time in seconds from activity start.

Referenced by [mouseReleaseEvent\(\)](#).

Here is the caller graph for this function:



7.73.4.17 leaveEvent()

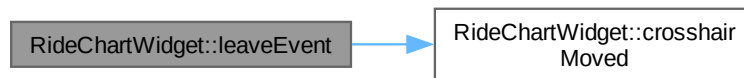
```
void RideChartWidget::leaveEvent (
    QEvent * event) [override], [protected]
```

Handles mouse leaving widget.

Definition at line 1880 of file [RideChartWidget.cpp](#).

References [crosshairMoved\(\)](#).

Here is the call graph for this function:



7.73.4.18 mouseDoubleClickEvent()

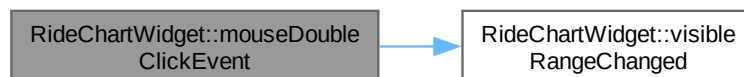
```
void RideChartWidget::mouseDoubleClickEvent (
    QMouseEvent * event) [override], [protected]
```

Handles double-click for interval creation.

Definition at line 1651 of file [RideChartWidget.cpp](#).

References [visibleRangeChanged\(\)](#).

Here is the call graph for this function:



7.73.4.19 mouseMoveEvent()

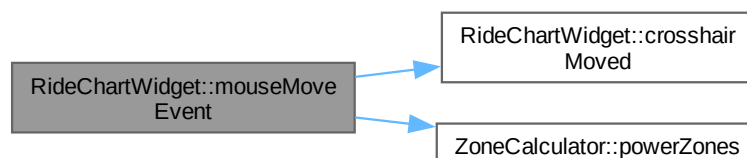
```
void RideChartWidget::mouseMoveEvent (
    QMouseEvent * event) [override], [protected]
```

Handles mouse movement for crosshair and selection.

Definition at line 1686 of file [RideChartWidget.cpp](#).

References [RideRecord::altitude](#), [RideRecord::cadence](#), [crosshairMoved\(\)](#), [Climb::endSeconds](#), [RideRecord::hasAltitude](#), [RideRecord::hasCadence](#), [RideRecord::hasHeartRate](#), [RideRecord::hasPower](#), [RideRecord::hasSpeed](#), [RideRecord::heartRate](#), [Zone::maxValue](#), [Zone::minValue](#), [Zone::name](#), [RideRecord::power](#), [ZoneCalculator::powerZones\(\)](#), [RideRecord::speed](#), and [Climb::startSeconds](#).

Here is the call graph for this function:



7.73.4.20 mousePressEvent()

```
void RideChartWidget::mousePressEvent (
    QMouseEvent * event) [override], [protected]
```

Handles mouse press for chart interaction.

Definition at line 1467 of file [RideChartWidget.cpp](#).

References [climbClicked\(\)](#), [Climb::endSeconds](#), and [Climb::startSeconds](#).

Here is the call graph for this function:

**7.73.4.21 mouseReleaseEvent()**

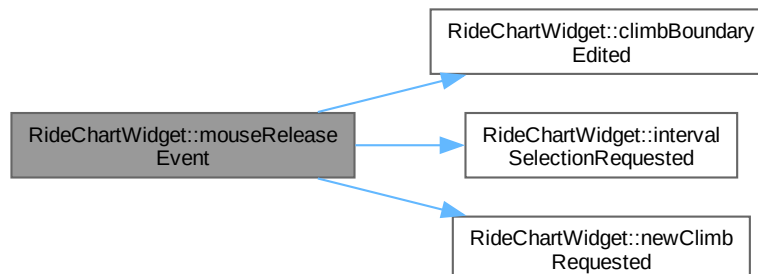
```
void RideChartWidget::mouseReleaseEvent (
    QMouseEvent * event) [override], [protected]
```

Handles mouse release for chart interaction.

Definition at line 1579 of file [RideChartWidget.cpp](#).

References [climbBoundaryEdited\(\)](#), [Climb::endSeconds](#), [intervalSelectionRequested\(\)](#), [newClimbRequested\(\)](#), and [Climb::startSeconds](#).

Here is the call graph for this function:

**7.73.4.22 newClimbRequested**

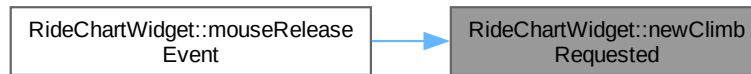
```
void RideChartWidget::newClimbRequested (
    double startSeconds,
    double endSeconds) [signal]
```

@signal Emitted when user creates new climb.

Parameters

<i>startSeconds</i>	Start time in seconds.
<i>endSeconds</i>	End time in seconds.

Referenced by [mouseReleaseEvent\(\)](#).
Here is the caller graph for this function:



7.73.4.23 rebuildChart()

```
void RideChartWidget::rebuildChart (
    bool preserveXRange = true) [private]
```

Definition at line 805 of file [RideChartWidget.cpp](#).

7.73.4.24 resetZoom

```
void RideChartWidget::resetZoom () [slot]
```

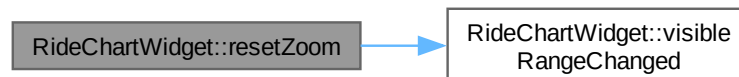
@slot Resets zoom to full ride extent.

Definition at line 1413 of file [RideChartWidget.cpp](#).

References [visibleRangeChanged\(\)](#).

Referenced by [fitToData\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.73.4.25 resizeEvent()

```
void RideChartWidget::resizeEvent (
    QResizeEvent * event) [override], [protected]
```

Handles widget resize events.

Definition at line 1312 of file [RideChartWidget.cpp](#).

7.73.4.26 setAutoSmoothingEnabled()

```
void RideChartWidget::setAutoSmoothingEnabled (
    bool enabled)
```

Enables/disables automatic smoothing.

Parameters

<i>enabled</i>	True to enable auto smoothing.
----------------	--------------------------------

Definition at line 327 of file [RideChartWidget.cpp](#).

7.73.4.27 setClimbEditingEnabled()

```
void RideChartWidget::setClimbEditingEnabled (
    bool enabled)
```

Enables/disables climb editing mode.

Parameters

<i>enabled</i>	True to enable editing.
----------------	-------------------------

Definition at line 368 of file [RideChartWidget.cpp](#).

7.73.4.28 setClimbs()

```
void RideChartWidget::setClimbs (
    const std::vector< Climb > & climbs)
```

Sets climb overlays.

Parameters

<i>climbs</i>	Detected climbs.
---------------	------------------

Definition at line 344 of file [RideChartWidget.cpp](#).

7.73.4.29 setCrosshair

```
void RideChartWidget::setCrosshair (
    double xMinutes) [slot]
```

@slot Sets crosshair position.

Parameters

<i>xMinutes</i>	Time in minutes.
-----------------	------------------

Definition at line 1378 of file [RideChartWidget.cpp](#).

7.73.4.30 setIntervals()

```
void RideChartWidget::setIntervals (
    const std::vector< Interval > & intervals)
```

Sets interval overlays.

Parameters

<i>intervals</i>	Work/recovery intervals.
------------------	--------------------------

Definition at line 337 of file [RideChartWidget.cpp](#).

7.73.4.31 setMetricOverlay()

```
void RideChartWidget::setMetricOverlay (
    ColorMetric metric,
    bool enabled)
```

Enables/disables metric overlay.

Parameters

<i>metric</i>	Metric to overlay.
<i>enabled</i>	True to show overlay.

Definition at line 350 of file [RideChartWidget.cpp](#).

7.73.4.32 setPowerSmoothingSeconds()

```
void RideChartWidget::setPowerSmoothingSeconds (
    int seconds)
```

Sets power smoothing window.

Parameters

<i>seconds</i>	Smoothing window in seconds.
----------------	------------------------------

Definition at line 316 of file [RideChartWidget.cpp](#).

7.73.4.33 setRideData()

```
void RideChartWidget::setRideData (
    const RideData & rideData,
    ColorMetric colorMetric = ColorMetric::None,
    const ColorContext & colorContext = {})
```

Sets ride data to display.

Parameters

<i>rideData</i>	Activity data.
<i>colorMetric</i>	Metric to use for zone coloring.
<i>colorContext</i>	Athlete context for coloring.

Definition at line 794 of file [RideChartWidget.cpp](#).

7.73.4.34 setSelectedClimbIndex()

```
void RideChartWidget::setSelectedClimbIndex (
    int index)
```

Highlights a specific climb.

Parameters

<i>index</i>	Climb index (-1 to clear).
--------------	--

Definition at line 357 of file [RideChartWidget.cpp](#).

7.73.4.35 setXRange

```
void RideChartWidget::setXRange (
    double min,
    double max) [slot]
```

@slot Sets visible x-axis range.

Parameters

<i>min</i>	Minimum time in minutes.
<i>max</i>	Maximum time in minutes.

Definition at line 1370 of file [RideChartWidget.cpp](#).

References [visibleRangeChanged\(\)](#).

Here is the call graph for this function:

**7.73.4.36 updateXTicks()**

```
void RideChartWidget::updateXTicks (
    double minVal,
    double maxVal) [private]
```

Definition at line 744 of file [RideChartWidget.cpp](#).

7.73.4.37 updateYAxis()

```
void RideChartWidget::updateYAxis () [private]
```

Definition at line 1251 of file [RideChartWidget.cpp](#).

7.73.4.38 updateYAxisForVisibleRange()

```
void RideChartWidget::updateYAxisForVisibleRange (
    double xMinMin,
    double xMaxMin) [private]
```

Definition at line 1264 of file [RideChartWidget.cpp](#).

7.73.4.39 visibleRangeChanged

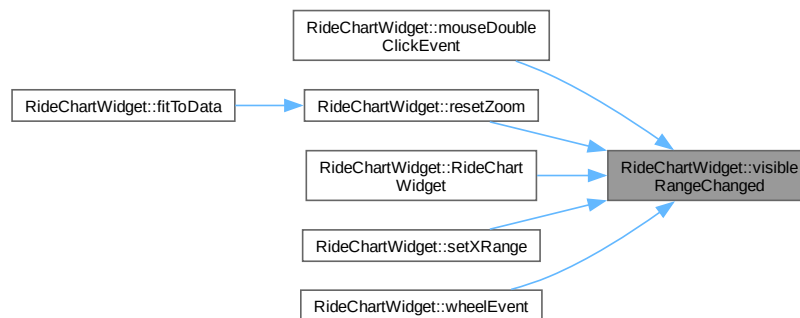
```
void RideChartWidget::visibleRangeChanged (
    double startMinutes,
    double endMinutes) [signal]
```

@signal Emitted when visible time range updates.

Parameters

<i>startMinutes</i>	Start time in minutes.
<i>endMinutes</i>	End time in minutes.

Referenced by [mouseDoubleClickEvent\(\)](#), [resetZoom\(\)](#), [RideChartWidget\(\)](#), [setXRange\(\)](#), and [wheelEvent\(\)](#).
Here is the caller graph for this function:



7.73.4.40 visibleRangeEndMinutes()

```
double RideChartWidget::visibleRangeEndMinutes () const
```

Gets end time of visible range in minutes from activity start.

Definition at line 311 of file [RideChartWidget.cpp](#).

7.73.4.41 visibleRangeStartMinutes()

```
double RideChartWidget::visibleRangeStartMinutes () const
```

Gets visible range start time.

Returns

Start time in minutes.

Definition at line 306 of file [RideChartWidget.cpp](#).

7.73.4.42 wheelEvent()

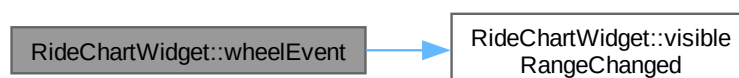
```
void RideChartWidget::wheelEvent (
    QWheelEvent * event) [override], [protected]
```

Handles wheel scroll for panning and zooming.

Definition at line 1427 of file [RideChartWidget.cpp](#).

References [visibleRangeChanged\(\)](#).

Here is the call graph for this function:



7.73.4.43 xRangeChanged

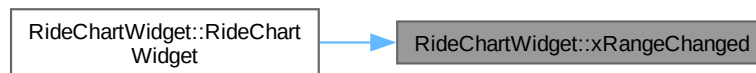
```
void RideChartWidget::xRangeChanged (
    double min,
    double max) [signal]
```

@signal Emitted when visible x range changes.

Parameters

<i>min</i>	Minimum time in minutes.
<i>max</i>	Maximum time in minutes.

Referenced by [RideChartWidget\(\)](#).
Here is the caller graph for this function:



7.73.5 Member Data Documentation

7.73.5.1 cachedAxisMax

```
double RideChartWidget::cachedAxisMax = 1.0
```

Definition at line 246 of file [RideChartWidget.h](#).

7.73.5.2 cachedAxisMin

```
double RideChartWidget::cachedAxisMin = 0.0
```

Definition at line 245 of file [RideChartWidget.h](#).

7.73.5.3 cachedMaxX

```
double RideChartWidget::cachedMaxX = -1.0
```

Definition at line 244 of file [RideChartWidget.h](#).

7.73.5.4 cachedMinX

```
double RideChartWidget::cachedMinX = -1.0
```

Definition at line 243 of file [RideChartWidget.h](#).

7.73.5.5 m_autoSmoothing

```
bool RideChartWidget::m_autoSmoothing = false [private]
```

Definition at line 235 of file [RideChartWidget.h](#).

7.73.5.6 m_axisX

```
QValueAxis* RideChartWidget::m_axisX = nullptr [private]
```

Definition at line 215 of file [RideChartWidget.h](#).

7.73.5.7 m_axisXDisplay

QCategoryAxis* RideChartWidget::m_axisXDisplay = nullptr [private]
Definition at line 216 of file [RideChartWidget.h](#).

7.73.5.8 m_axisY

QValueAxis* RideChartWidget::m_axisY = nullptr [private]
Definition at line 217 of file [RideChartWidget.h](#).

7.73.5.9 m_axisYOverlay

QValueAxis* RideChartWidget::m_axisYOverlay = nullptr [private]
Definition at line 218 of file [RideChartWidget.h](#).

7.73.5.10 m_backgroundSeries

std::vector<QAreaSeries*> RideChartWidget::m_backgroundSeries [private]
Definition at line 214 of file [RideChartWidget.h](#).

7.73.5.11 m_climbEditingEnabled

bool RideChartWidget::m_climbEditingEnabled = false [private]
Definition at line 255 of file [RideChartWidget.h](#).

7.73.5.12 m_climbHandleDrag

ClimbHandleDrag RideChartWidget::m_climbHandleDrag = ClimbHandleDrag::None [private]
Definition at line 263 of file [RideChartWidget.h](#).

7.73.5.13 m_climbs

std::vector<Climb> RideChartWidget::m_climbs [private]
Definition at line 221 of file [RideChartWidget.h](#).

7.73.5.14 m_colorContext

ColorContext RideChartWidget::m_colorContext [private]
Definition at line 225 of file [RideChartWidget.h](#).

7.73.5.15 m_colorMetric

ColorMetric RideChartWidget::m_colorMetric = ColorMetric::None [private]
Definition at line 224 of file [RideChartWidget.h](#).

7.73.5.16 m_colorSeries

std::vector<QLineSeries*> RideChartWidget::m_colorSeries [private]
Definition at line 211 of file [RideChartWidget.h](#).

7.73.5.17 m_crosshairX

double RideChartWidget::m_crosshairX = -1.0 [private]
Definition at line 250 of file [RideChartWidget.h](#).

7.73.5.18 m_crosshairY

double RideChartWidget::m_crosshairY = std::numeric_limits<double>::quiet_NaN() [private]
Definition at line 251 of file [RideChartWidget.h](#).

7.73.5.19 m_dataMaxY

```
double RideChartWidget::m_dataMaxY = 1.0 [private]
```

Definition at line 271 of file [RideChartWidget.h](#).

7.73.5.20 m_dataMinY

```
double RideChartWidget::m_dataMinY = 0.0 [private]
```

Definition at line 270 of file [RideChartWidget.h](#).

7.73.5.21 m_dragOriginalClimbEndSec

```
double RideChartWidget::m_dragOriginalClimbEndSec = -1.0 [private]
```

Definition at line 265 of file [RideChartWidget.h](#).

7.73.5.22 m_dragOriginalClimbStartSec

```
double RideChartWidget::m_dragOriginalClimbStartSec = -1.0 [private]
```

Definition at line 264 of file [RideChartWidget.h](#).

7.73.5.23 m_effectivePowerSmoothingSeconds

```
int RideChartWidget::m_effectivePowerSmoothingSeconds = 0 [private]
```

Definition at line 234 of file [RideChartWidget.h](#).

7.73.5.24 m_hasData

```
bool RideChartWidget::m_hasData = false [private]
```

Definition at line 238 of file [RideChartWidget.h](#).

7.73.5.25 m_intervalEndMin

```
double RideChartWidget::m_intervalEndMin = -1.0 [private]
```

Definition at line 254 of file [RideChartWidget.h](#).

7.73.5.26 m_intervals

```
std::vector<Interval> RideChartWidget::m_intervals [private]
```

Definition at line 220 of file [RideChartWidget.h](#).

7.73.5.27 m_intervalSelecting

```
bool RideChartWidget::m_intervalSelecting = false [private]
```

Definition at line 252 of file [RideChartWidget.h](#).

7.73.5.28 m_intervalStartMin

```
double RideChartWidget::m_intervalStartMin = -1.0 [private]
```

Definition at line 253 of file [RideChartWidget.h](#).

7.73.5.29 m_isRebuilding

```
bool RideChartWidget::m_isRebuilding = false [private]
```

Definition at line 236 of file [RideChartWidget.h](#).

7.73.5.30 m_metric

```
Metric RideChartWidget::m_metric [private]
```

Definition at line 208 of file [RideChartWidget.h](#).

7.73.5.31 m_metricOverlayEnabled

`bool RideChartWidget::m_metricOverlayEnabled = false [private]`
Definition at line 222 of file [RideChartWidget.h](#).

7.73.5.32 m_metricOverlayMetric

`ColorMetric RideChartWidget::m_metricOverlayMetric = ColorMetric::None [private]`
Definition at line 223 of file [RideChartWidget.h](#).

7.73.5.33 m_newClimbCreating

`bool RideChartWidget::m_newClimbCreating = false [private]`
Definition at line 266 of file [RideChartWidget.h](#).

7.73.5.34 m_newClimbCurrentSec

`double RideChartWidget::m_newClimbCurrentSec = -1.0 [private]`
Definition at line 269 of file [RideChartWidget.h](#).

7.73.5.35 m_newClimbDragging

`bool RideChartWidget::m_newClimbDragging = false [private]`
Definition at line 267 of file [RideChartWidget.h](#).

7.73.5.36 m_newClimbStartSec

`double RideChartWidget::m_newClimbStartSec = -1.0 [private]`
Definition at line 268 of file [RideChartWidget.h](#).

7.73.5.37 m_origMaxX

`double RideChartWidget::m_origMaxX = 1.0 [private]`
Definition at line 249 of file [RideChartWidget.h](#).

7.73.5.38 m_overlaySeries

`std::vector<QLineSeries*> RideChartWidget::m_overlaySeries [private]`
Definition at line 212 of file [RideChartWidget.h](#).

7.73.5.39 m_rawSeries

`std::vector<QLineSeries*> RideChartWidget::m_rawSeries [private]`
Definition at line 210 of file [RideChartWidget.h](#).

7.73.5.40 m_referenceSeries

`std::vector<QLineSeries*> RideChartWidget::m_referenceSeries [private]`
Definition at line 213 of file [RideChartWidget.h](#).

7.73.5.41 m_requestedPowerSmoothingSeconds

`int RideChartWidget::m_requestedPowerSmoothingSeconds = 0 [private]`
Definition at line 233 of file [RideChartWidget.h](#).

7.73.5.42 m_rideData

[RideData](#) `RideChartWidget::m_rideData [private]`
Definition at line 219 of file [RideChartWidget.h](#).

7.73.5.43 m_selectedClimbIndex

```
int RideChartWidget::m_selectedClimbIndex = -1 [private]
```

Definition at line 256 of file [RideChartWidget.h](#).

7.73.5.44 m_series

```
QLineSeries* RideChartWidget::m_series = nullptr [private]
```

Definition at line 209 of file [RideChartWidget.h](#).

7.73.5.45 m_smoothedSeriesCache

```
std::unordered_map<int, std::vector<double> > RideChartWidget::m_smoothedSeriesCache [mutable], [private]
```

Definition at line 229 of file [RideChartWidget.h](#).

7.73.5.46 m_syncing

```
bool RideChartWidget::m_syncing = false [private]
```

Definition at line 237 of file [RideChartWidget.h](#).

7.73.5.47 m_tooltipDisplayValues

```
std::vector<double> RideChartWidget::m_tooltipDisplayValues [private]
```

Definition at line 232 of file [RideChartWidget.h](#).

7.73.5.48 m_tooltipPoints

```
std::vector<QPointF> RideChartWidget::m_tooltipPoints [private]
```

Definition at line 230 of file [RideChartWidget.h](#).

7.73.5.49 m_tooltipRawValues

```
std::vector<double> RideChartWidget::m_tooltipRawValues [private]
```

Definition at line 231 of file [RideChartWidget.h](#).

7.73.5.50 valid

```
bool RideChartWidget::valid = false
```

Definition at line 247 of file [RideChartWidget.h](#).

The documentation for this class was generated from the following files:

- [src/charts/RideChartWidget.h](#)
- [src/charts/RideChartWidget.cpp](#)

7.74 RideData Class Reference

Container for all recorded samples and metadata for a single activity.

```
#include <RideData.h>
```

Public Attributes

- `std::vector< RideRecord > records`
Recorded samples.
- `QString activityStartTimeUtc`
Activity start time (UTC, ISO 8601).
- `QString activityEndTimeUtc`
Activity end time (UTC, ISO 8601).
- `QString fileTimestampUtc`
File timestamp (UTC, ISO 8601).

7.74.1 Detailed Description

Container for all recorded samples and metadata for a single activity.
Definition at line 63 of file [RideData.h](#).

7.74.2 Member Data Documentation

7.74.2.1 activityEndTimeUtc

QString RideData::activityEndTimeUtc

[Activity](#) end time (UTC, ISO 8601).

Definition at line 73 of file [RideData.h](#).

Referenced by [FitParser::load\(\)](#), and [ImportWorker::processJob\(\)](#).

7.74.2.2 activityStartTimeUtc

QString RideData::activityStartTimeUtc

[Activity](#) start time (UTC, ISO 8601).

Definition at line 70 of file [RideData.h](#).

Referenced by [FitParser::load\(\)](#), and [ImportWorker::processJob\(\)](#).

7.74.2.3 fileTimestampUtc

QString RideData::fileTimestampUtc

File timestamp (UTC, ISO 8601).

Definition at line 76 of file [RideData.h](#).

Referenced by [FitParser::load\(\)](#).

7.74.2.4 records

std::vector<[RideRecord](#)> RideData::records

Recorded samples.

Definition at line 67 of file [RideData.h](#).

Referenced by [WorkoutAnalyzer::analyze\(\)](#), [PowerCurve::bestMeanPower\(\)](#), [ClimbMetrics::build\(\)](#), [ClimbDetector::buildCumulativeDis](#), [FTPZones::compute\(\)](#), [TrainingMetrics::compute\(\)](#), [ClimbDetector::computeAdaptiveSmoothingWindowsMeters\(\)](#), [ZoneCalculator::computeDistribution\(\)](#), [ClimbDetector::detect\(\)](#), [IntervalDetector::detect\(\)](#), [ZoneCalculator::hasMetricSamples\(\)](#), [FitParser::load\(\)](#), [TrainingLoad::normalizedPower\(\)](#), [ImportWorker::processJob\(\)](#), [AnalysisWorker::processTask\(\)](#), [PowerHistogramWidget::setRideData\(\)](#), [ClimbDetector::smoothAltitudeByDistance\(\)](#), and [SmoothingEngine::smoothPower\(\)](#).

The documentation for this class was generated from the following file:

- [src/fit/RideData.h](#)

7.75 RideDataLoader Class Reference

Loads activity ride data samples from the database.

```
#include <RideDataLoader.h>
```

Static Public Member Functions

- static [RideData loadActivitySamples](#) (int activityId, QSqlDatabase &db)

7.75.1 Detailed Description

Loads activity ride data samples from the database.

Definition at line 12 of file [RideDataLoader.h](#).

7.75.2 Member Function Documentation

7.75.2.1 loadActivitySamples()

```
RideData RideDataLoader::loadActivitySamples (
    int activityId,
    QSqlDatabase & db) [static]
```

Definition at line 8 of file [RideDataLoader.cpp](#).

The documentation for this class was generated from the following files:

- [src/model/RideDataLoader.h](#)
- [src/model/RideDataLoader.cpp](#)

7.76 RideDataSerializer Class Reference

Persists ride data to database.

```
#include <RideDataSerializer.h>
```

Static Public Member Functions

- static int [saveRideToDatabase](#) (const [RideData](#) &rideData, const [WorkoutStatistics](#) &stats, int athleteId, const QString &filePath, const QString &fitHash, double normalizedPower, [DatabaseManager](#) &dbManager, [ActivityRepository](#) &activityRepo, [ImportRepository](#) &importRepo, QString *errorOut=nullptr)
- static [RideData](#) [loadRideFromDatabase](#) (int activityId, [DatabaseManager](#) &dbManager)

7.76.1 Detailed Description

Persists ride data to database.

Saves parsed rides with computed statistics, detects duplicates, and logs all import attempts.

Definition at line 21 of file [RideDataSerializer.h](#).

7.76.2 Member Function Documentation

7.76.2.1 loadRideFromDatabase()

```
RideData RideDataSerializer::loadRideFromDatabase (
    int activityId,
    DatabaseManager & dbManager) [static]
```

Definition at line 171 of file [RideDataSerializer.cpp](#).

7.76.2.2 saveRideToDatabase()

```
int RideDataSerializer::saveRideToDatabase (
    const RideData & rideData,
    const WorkoutStatistics & stats,
    int athleteId,
    const QString & filePath,
    const QString & fitHash,
    double normalizedPower,
    DatabaseManager & dbManager,
    ActivityRepository & activityRepo,
    ImportRepository & importRepo,
    QString * errorOut = nullptr) [static]
```

Definition at line 17 of file [RideDataSerializer.cpp](#).

The documentation for this class was generated from the following files:

- [src/model/RideDataSerializer.h](#)
- [src/model/RideDataSerializer.cpp](#)

7.77 RideRecord Struct Reference

A single recorded sample from an activity recording.

```
#include <RideData.h>
```

Public Attributes

- double [elapsedSeconds](#) = 0.0
Elapsed time in seconds from activity start.
- double [latitude](#) = 0.0
GPS latitude in degrees.
- double [longitude](#) = 0.0
GPS longitude in degrees.
- double [power](#) = 0.0
Power in watts.
- double [heartRate](#) = 0.0
Heart rate in bpm.
- double [cadence](#) = 0.0
Cadence in rpm.
- double [speed](#) = 0.0
Speed in km/h.
- double [altitude](#) = 0.0
Altitude in meters.
- bool [hasGps](#) = false
Whether GPS data is available.
- bool [hasPower](#) = false
Whether power data is available.
- bool [hasHeartRate](#) = false
Whether heart rate data is available.
- bool [hasCadence](#) = false
Whether cadence data is available.
- bool [hasSpeed](#) = false
Whether speed data is available.
- bool [hasAltitude](#) = false
Whether altitude data is available.

7.77.1 Detailed Description

A single recorded sample from an activity recording.

Contains time, position, power, heart rate, and other metrics recorded at one instant during an activity.

Definition at line 15 of file [RideData.h](#).

7.77.2 Member Data Documentation

7.77.2.1 altitude

```
double RideRecord::altitude = 0.0
```

Altitude in meters.

Definition at line 39 of file [RideData.h](#).

Referenced by [ClimbMetrics::build\(\)](#), [RideChartWidget::mouseMoveEvent\(\)](#), [FitRecordListener::OnMesg\(\)](#), [ImportWorker::processJob\(\)](#), and [VideoRenderJob::run\(\)](#).

7.77.2.2 cadence

```
double RideRecord::cadence = 0.0
```

Cadence in rpm.

Definition at line 33 of file [RideData.h](#).

Referenced by [ClimbMetrics::build\(\)](#), [RideChartWidget::mouseMoveEvent\(\)](#), [FitRecordListener::OnMesg\(\)](#), [ImportWorker::processJob\(\)](#), and [VideoRenderJob::run\(\)](#).

7.77.2.3 elapsedSeconds

```
double RideRecord::elapsedSeconds = 0.0
```

Elapsed time in seconds from activity start.

Definition at line 18 of file [RideData.h](#).

Referenced by [ClimbMetrics::build\(\)](#), [ClimbDetector::buildCumulativeDistanceMeters\(\)](#), [FitRecordListener::OnMesg\(\)](#), [ImportWorker::processJob\(\)](#), [VideoRenderJob::run\(\)](#), and [SmoothingEngine::smoothPower\(\)](#).

7.77.2.4 hasAltitude

```
bool RideRecord::hasAltitude = false
```

Whether altitude data is available.

Definition at line 57 of file [RideData.h](#).

Referenced by [ClimbMetrics::build\(\)](#), [ClimbDetector::detect\(\)](#), [RideChartWidget::mouseMoveEvent\(\)](#), [FitRecordListener::OnMesg\(\)](#), [ImportWorker::processJob\(\)](#), and [VideoRenderJob::run\(\)](#).

7.77.2.5 hasCadence

```
bool RideRecord::hasCadence = false
```

Whether cadence data is available.

Definition at line 51 of file [RideData.h](#).

Referenced by [ClimbMetrics::build\(\)](#), [RideChartWidget::mouseMoveEvent\(\)](#), [FitRecordListener::OnMesg\(\)](#), [ImportWorker::processJob\(\)](#), and [VideoRenderJob::run\(\)](#).

7.77.2.6 hasGps

```
bool RideRecord::hasGps = false
```

Whether GPS data is available.

Definition at line 42 of file [RideData.h](#).

Referenced by [ClimbDetector::buildCumulativeDistanceMeters\(\)](#), [FitRecordListener::OnMesg\(\)](#), [ImportWorker::processJob\(\)](#), and [VideoRenderJob::run\(\)](#).

7.77.2.7 hasHeartRate

```
bool RideRecord::hasHeartRate = false
```

Whether heart rate data is available.

Definition at line 48 of file [RideData.h](#).

Referenced by [ClimbMetrics::build\(\)](#), [TrainingMetrics::compute\(\)](#), [RideChartWidget::mouseMoveEvent\(\)](#), [FitRecordListener::OnMesg\(\)](#), [ImportWorker::processJob\(\)](#), and [VideoRenderJob::run\(\)](#).

7.77.2.8 hasPower

```
bool RideRecord::hasPower = false
```

Whether power data is available.

Definition at line 45 of file [RideData.h](#).

Referenced by [ClimbMetrics::build\(\)](#), [TrainingMetrics::compute\(\)](#), [RideChartWidget::mouseMoveEvent\(\)](#), [FitRecordListener::OnMesg\(\)](#), [ImportWorker::processJob\(\)](#), [VideoRenderJob::run\(\)](#), and [SmoothingEngine::smoothPower\(\)](#).

7.77.2.9 hasSpeed

```
bool RideRecord::hasSpeed = false
```

Whether speed data is available.

Definition at line 54 of file [RideData.h](#).

Referenced by [ClimbMetrics::build\(\)](#), [ClimbDetector::buildCumulativeDistanceMeters\(\)](#), [RideChartWidget::mouseMoveEvent\(\)](#), [FitRecordListener::OnMesg\(\)](#), [ImportWorker::processJob\(\)](#), and [VideoRenderJob::run\(\)](#).

7.77.2.10 heartRate

```
double RideRecord::heartRate = 0.0
```

Heart rate in bpm.

Definition at line 30 of file [RideData.h](#).

Referenced by [ClimbMetrics::build\(\)](#), [TrainingMetrics::compute\(\)](#), [RideChartWidget::mouseMoveEvent\(\)](#), [FitRecordListener::OnMesg\(\)](#), [ImportWorker::processJob\(\)](#), and [VideoRenderJob::run\(\)](#).

7.77.2.11 latitude

```
double RideRecord::latitude = 0.0
```

GPS latitude in degrees.

Definition at line 21 of file [RideData.h](#).

Referenced by [ClimbDetector::buildCumulativeDistanceMeters\(\)](#), [FitRecordListener::OnMesg\(\)](#), [ImportWorker::processJob\(\)](#), and [VideoRenderJob::run\(\)](#).

7.77.2.12 longitude

```
double RideRecord::longitude = 0.0
```

GPS longitude in degrees.

Definition at line 24 of file [RideData.h](#).

Referenced by [ClimbDetector::buildCumulativeDistanceMeters\(\)](#), [FitRecordListener::OnMesg\(\)](#), [ImportWorker::processJob\(\)](#), and [VideoRenderJob::run\(\)](#).

7.77.2.13 power

```
double RideRecord::power = 0.0
```

Power in watts.

Definition at line 27 of file [RideData.h](#).

Referenced by [ClimbMetrics::build\(\)](#), [TrainingMetrics::compute\(\)](#), [RideChartWidget::mouseMoveEvent\(\)](#), [FitRecordListener::OnMesg\(\)](#), [ImportWorker::processJob\(\)](#), [VideoRenderJob::run\(\)](#), and [SmoothingEngine::smoothPower\(\)](#).

7.77.2.14 speed

```
double RideRecord::speed = 0.0
```

Speed in km/h.

Definition at line 36 of file [RideData.h](#).

Referenced by [ClimbMetrics::build\(\)](#), [ClimbDetector::buildCumulativeDistanceMeters\(\)](#), [RideChartWidget::mouseMoveEvent\(\)](#), [FitRecordListener::OnMesg\(\)](#), [ImportWorker::processJob\(\)](#), and [VideoRenderJob::run\(\)](#).

The documentation for this struct was generated from the following file:

- [src/fit/RideData.h](#)

7.78 anonymous_namespace{ClimbMetrics.cpp}::SliceStats Struct Reference

Public Attributes

- double [avgPower](#) = 0.0
- double [avgHeartRate](#) = 0.0
- double [avgCadence](#) = 0.0

7.78.1 Detailed Description

Definition at line 12 of file [ClimbMetrics.cpp](#).

7.78.2 Member Data Documentation

7.78.2.1 avgCadence

`double anonymous_namespace{ClimbMetrics.cpp}::SliceStats::avgCadence = 0.0`

Definition at line 16 of file [ClimbMetrics.cpp](#).

7.78.2.2 avgHeartRate

`double anonymous_namespace{ClimbMetrics.cpp}::SliceStats::avgHeartRate = 0.0`

Definition at line 15 of file [ClimbMetrics.cpp](#).

7.78.2.3 avgPower

`double anonymous_namespace{ClimbMetrics.cpp}::SliceStats::avgPower = 0.0`

Definition at line 14 of file [ClimbMetrics.cpp](#).

The documentation for this struct was generated from the following file:

- [src/analysis/ClimbMetrics.cpp](#)

7.79 SmoothingEngine Class Reference

Provides data smoothing algorithms for activity metrics.

`#include <SmoothingEngine.h>`

Static Public Member Functions

- static `std::vector< double > smoothPower` (const [RideData](#) &ride, int seconds)
Computes smoothed power series using rolling average.

7.79.1 Detailed Description

Provides data smoothing algorithms for activity metrics.

Definition at line 13 of file [SmoothingEngine.h](#).

7.79.2 Member Function Documentation

7.79.2.1 smoothPower()

```
std::vector< double > SmoothingEngine::smoothPower (
    const RideData & ride,
    int seconds) [static]
```

Computes smoothed power series using rolling average.

Parameters

<i>ride</i>	Activity data.
<i>seconds</i>	Window width in seconds for rolling average.

Returns

Smoothed power series (watts); NaN for invalid samples.

Definition at line 9 of file [SmoothingEngine.cpp](#).

References [RideRecord::elapsedSeconds](#), [RideRecord::hasPower](#), [RideRecord::power](#), and [RideData::records](#).

The documentation for this class was generated from the following files:

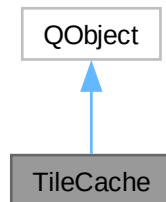
- [src/analysis/SmoothingEngine.h](#)
- [src/analysis/SmoothingEngine.cpp](#)

7.80 TileCache Class Reference

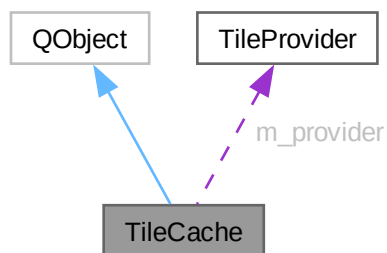
Caches map tiles in memory and on disk.

```
#include <TileCache.h>
```

Inheritance diagram for TileCache:



Collaboration diagram for TileCache:



Classes

- struct [TileRequest](#)

Signals

- void **tileLoaded** (int z, int x, int y)

Public Member Functions

- [TileCache](#) (const [TileCacheConfig](#) &config={}, `QObject` *parent=nullptr)
- `QPixmap` [tile](#) (int z, int x, int y)
- `QPixmap` [tileBlocking](#) (int z, int x, int y, bool allowNetwork=true)
- bool [isTileCachedOnDisk](#) (int z, int x, int y) const
- void [clearMemoryCache](#) ()
- void [setMemoryCacheSize](#) (int maxTiles)
- void [setMapStyle](#) (MapStyle style)
- MapStyle [mapStyle](#) () const
- `QString` [mapStyleName](#) () const

- QString [attribution](#) () const
- int [maxZoom](#) () const
- QString [diskCacheRoot](#) () const

Private Slots

- void [onReplyFinished](#) (QNetworkReply *reply)

Private Member Functions

- QString [key](#) (int z, int x, int y) const
- QString [diskTilePathForRoot](#) (const QString &root, int z, int x, int y) const
- QString [diskTilePath](#) (int z, int x, int y) const
- void [dispatchNextDownloads](#) ()

Static Private Member Functions

- static QString [keyForStyle](#) (const QString &style, int z, int x, int y)
- static QString [diskTilePathForRoot](#) (const QString &root, const QString &style, int z, int x, int y)
- static QString [styleKey](#) (MapStyle style)
- static QString [styleDisplayName](#) (MapStyle style)
- static [TileProvider](#) [providerForStyle](#) (MapStyle style)

Private Attributes

- QNetworkAccessManager [m_nam](#)
- QCache< QString, QPixmap > [m_cache](#)
- QSet< QString > [m_pending](#)
- QQueue< [TileRequest](#) > [m_downloadQueue](#)
- int [m_activeDownloads](#) = 0
- QString [m_diskCacheRoot](#)
- QStringList [m_diskFallbackRoots](#)
- MapStyle [m_mapStyle](#) = MapStyle::Light
- [TileProvider](#) [m_provider](#)

Static Private Attributes

- static constexpr int [kMaxConcurrentDownloads](#) = 6

7.80.1 Detailed Description

Caches map tiles in memory and on disk.

Provides tile lookup, network fetching, and memory/disk cache management with multiple map style support and fallback cache locations.

Definition at line 66 of file [TileCache.h](#).

7.80.2 Constructor & Destructor Documentation

7.80.2.1 TileCache()

```
TileCache::TileCache (
    const TileCacheConfig & config = {},
    QObject * parent = nullptr) [explicit]
```

Definition at line 121 of file [TileCache.cpp](#).

7.80.3 Member Function Documentation

7.80.3.1 attribution()

QString TileCache::attribution () const

Definition at line 225 of file [TileCache.cpp](#).

7.80.3.2 clearMemoryCache()

void TileCache::clearMemoryCache ()

Definition at line 184 of file [TileCache.cpp](#).

7.80.3.3 diskCacheRoot()

QString TileCache::diskCacheRoot () const

Definition at line 235 of file [TileCache.cpp](#).

7.80.3.4 diskTilePath()

```
QString TileCache::diskTilePath (
    int z,
    int x,
    int y) const [private]
```

Definition at line 162 of file [TileCache.cpp](#).

7.80.3.5 diskTilePathForRoot() [1/2]

```
QString TileCache::diskTilePathForRoot (
    const QString & root,
    const QString & style,
    int z,
    int x,
    int y) [static], [private]
```

Definition at line 149 of file [TileCache.cpp](#).

7.80.3.6 diskTilePathForRoot() [2/2]

```
QString TileCache::diskTilePathForRoot (
    const QString & root,
    int z,
    int x,
    int y) const [private]
```

Definition at line 144 of file [TileCache.cpp](#).

7.80.3.7 dispatchNextDownloads()

void TileCache::dispatchNextDownloads () [private]

Definition at line 351 of file [TileCache.cpp](#).

7.80.3.8 isTileCachedOnDisk()

```
bool TileCache::isTileCachedOnDisk (
    int z,
    int x,
    int y) const
```

Definition at line 167 of file [TileCache.cpp](#).

7.80.3.9 key()

```
QString TileCache::key (  
    int z,  
    int x,  
    int y) const [private]
```

Definition at line 134 of file [TileCache.cpp](#).

7.80.3.10 keyForStyle()

```
QString TileCache::keyForStyle (  
    const QString & style,  
    int z,  
    int x,  
    int y) [static], [private]
```

Definition at line 139 of file [TileCache.cpp](#).

7.80.3.11 mapStyle()

```
MapStyle TileCache::mapStyle () const [inline]
```

Definition at line 84 of file [TileCache.h](#).

7.80.3.12 mapStyleName()

```
QString TileCache::mapStyleName () const
```

Definition at line 220 of file [TileCache.cpp](#).

7.80.3.13 maxZoom()

```
int TileCache::maxZoom () const
```

Definition at line 230 of file [TileCache.cpp](#).

7.80.3.14 onReplyFinished

```
void TileCache::onReplyFinished (  
    QNetworkReply * reply) [private], [slot]
```

Definition at line 378 of file [TileCache.cpp](#).

7.80.3.15 providerForStyle()

```
TileProvider TileCache::providerForStyle (  
    MapStyle style) [static], [private]
```

Definition at line 58 of file [TileCache.cpp](#).

7.80.3.16 setMapStyle()

```
void TileCache::setMapStyle (  
    MapStyle style)
```

Definition at line 199 of file [TileCache.cpp](#).

7.80.3.17 setMemoryCacheSize()

```
void TileCache::setMemoryCacheSize (  
    int maxTiles)
```

Definition at line 194 of file [TileCache.cpp](#).

7.80.3.18 styleDisplayName()

```
QString TileCache::styleDisplayName (  
    MapStyle style) [static], [private]
```

Definition at line 43 of file [TileCache.cpp](#).

7.80.3.19 styleKey()

```
QString TileCache::styleKey (  
    MapStyle style) [static], [private]
```

Definition at line 28 of file [TileCache.cpp](#).

7.80.3.20 tile()

```
QPixmap TileCache::tile (  
    int z,  
    int x,  
    int y)
```

Definition at line 240 of file [TileCache.cpp](#).

7.80.3.21 tileBlocking()

```
QPixmap TileCache::tileBlocking (  
    int z,  
    int x,  
    int y,  
    bool allowNetwork = true)
```

Definition at line 282 of file [TileCache.cpp](#).

7.80.4 Member Data Documentation

7.80.4.1 kMaxConcurrentDownloads

```
int TileCache::kMaxConcurrentDownloads = 6 [static], [constexpr], [private]
```

Definition at line 104 of file [TileCache.h](#).

7.80.4.2 m_activeDownloads

```
int TileCache::m_activeDownloads = 0 [private]
```

Definition at line 125 of file [TileCache.h](#).

7.80.4.3 m_cache

```
QCache<QString, QPixmap> TileCache::m_cache [private]
```

Definition at line 122 of file [TileCache.h](#).

7.80.4.4 m_diskCacheRoot

```
QString TileCache::m_diskCacheRoot [private]
```

Definition at line 126 of file [TileCache.h](#).

7.80.4.5 m_diskFallbackRoots

```
QStringList TileCache::m_diskFallbackRoots [private]
```

Definition at line 127 of file [TileCache.h](#).

7.80.4.6 m_downloadQueue

```
QQueue<TileRequest> TileCache::m_downloadQueue [private]
```

Definition at line 124 of file [TileCache.h](#).

7.80.4.7 m_mapStyle

```
MapStyle TileCache::m_mapStyle = MapStyle::Light [private]
```

Definition at line 128 of file [TileCache.h](#).

7.80.4.8 m_nam

QNetworkAccessManager TileCache::m_nam [private]
 Definition at line 121 of file [TileCache.h](#).

7.80.4.9 m_pending

QSet<QString> TileCache::m_pending [private]
 Definition at line 123 of file [TileCache.h](#).

7.80.4.10 m_provider

[TileProvider](#) TileCache::m_provider [private]
 Definition at line 129 of file [TileCache.h](#).

The documentation for this class was generated from the following files:

- [src/maps/TileCache.h](#)
- [src/maps/TileCache.cpp](#)

7.81 TileCacheConfig Struct Reference

Configuration for tile caching behavior.

```
#include <TileCache.h>
```

Public Attributes

- int [maxTilesInMemory](#) = 512
- QString [diskCacheRoot](#)
- QStringList [diskFallbackRoots](#)

7.81.1 Detailed Description

Configuration for tile caching behavior.

Definition at line 48 of file [TileCache.h](#).

7.81.2 Member Data Documentation

7.81.2.1 diskCacheRoot

QString TileCacheConfig::diskCacheRoot
 Definition at line 54 of file [TileCache.h](#).

7.81.2.2 diskFallbackRoots

QStringList TileCacheConfig::diskFallbackRoots
 Definition at line 57 of file [TileCache.h](#).

7.81.2.3 maxTilesInMemory

int TileCacheConfig::maxTilesInMemory = 512
 Definition at line 51 of file [TileCache.h](#).

The documentation for this struct was generated from the following file:

- [src/maps/TileCache.h](#)

7.82 TileProvider Struct Reference

Tile provider configuration.

```
#include <TileCache.h>
```

Public Attributes

- QString [name](#)
- QString [urlTemplate](#)
- int [maxZoom](#) = 18
- QString [attribution](#)

7.82.1 Detailed Description

Tile provider configuration.

Definition at line 28 of file [TileCache.h](#).

7.82.2 Member Data Documentation

7.82.2.1 attribution

```
QString TileProvider::attribution
```

Definition at line 40 of file [TileCache.h](#).

7.82.2.2 maxZoom

```
int TileProvider::maxZoom = 18
```

Definition at line 37 of file [TileCache.h](#).

7.82.2.3 name

```
QString TileProvider::name
```

Definition at line 31 of file [TileCache.h](#).

7.82.2.4 urlTemplate

```
QString TileProvider::urlTemplate
```

Definition at line 34 of file [TileCache.h](#).

The documentation for this struct was generated from the following file:

- [src/maps/TileCache.h](#)

7.83 TileCache::TileRequest Struct Reference

Public Attributes

- int [z](#)
- int [x](#)
- int [y](#)

7.83.1 Detailed Description

Definition at line 102 of file [TileCache.h](#).

7.83.2 Member Data Documentation

7.83.2.1 x

```
int TileCache::TileRequest::x
```

Definition at line 102 of file [TileCache.h](#).

7.83.2.2 y

```
int TileCache::TileRequest::y
```

Definition at line 102 of file [TileCache.h](#).

7.83.2.3 z

```
int TileCache::TileRequest::z
```

Definition at line 102 of file [TileCache.h](#).

The documentation for this struct was generated from the following file:

- [src/maps/TileCache.h](#)

7.84 anonymous_namespace{VideoRenderJob.cpp}::TileViewport Struct Reference

Public Member Functions

- bool [operator==](#) (const [TileViewport](#) &other) const

Public Attributes

- int [tileX0](#) = 0
- int [tileY0](#) = 0
- int [nX](#) = 0
- int [nY](#) = 0

7.84.1 Detailed Description

Definition at line 35 of file [VideoRenderJob.cpp](#).

7.84.2 Member Function Documentation

7.84.2.1 operator==()

```
bool anonymous_namespace{VideoRenderJob.cpp}::TileViewport::operator== (
    const TileViewport & other) const [inline]
```

Definition at line 42 of file [VideoRenderJob.cpp](#).

7.84.3 Member Data Documentation

7.84.3.1 nX

```
int anonymous_namespace{VideoRenderJob.cpp}::TileViewport::nX = 0
```

Definition at line 39 of file [VideoRenderJob.cpp](#).

7.84.3.2 nY

```
int anonymous_namespace{VideoRenderJob.cpp}::TileViewport::nY = 0
```

Definition at line 40 of file [VideoRenderJob.cpp](#).

7.84.3.3 tileX0

```
int anonymous_namespace{VideoRenderJob.cpp}::TileViewport::tileX0 = 0
```

Definition at line 37 of file [VideoRenderJob.cpp](#).

7.84.3.4 tileY0

```
int anonymous_namespace{VideoRenderJob.cpp}::TileViewport::tileY0 = 0
```

Definition at line 38 of file [VideoRenderJob.cpp](#).

The documentation for this struct was generated from the following file:

- [src/video/VideoRenderJob.cpp](#)

7.85 TrainingLoad Class Reference

Computes training load metrics from activity power data.

#include <TrainingLoad.h>

Static Public Member Functions

- static double [normalizedPower](#) (const [RideData](#) &rideData)
Calculates normalized power for an activity.
- static double [intensityFactor](#) (double np, double ftp)
Calculates intensity factor.
- static double [trainingStressScore](#) (double durationSeconds, double np, double ftp)
Calculates training stress score for an activity.
- static std::vector< [FitnessMetricsPoint](#) > [fitnessTimeline](#) (const std::vector< [DailyLoadPoint](#) > &dailyLoads, int atlDays=7, int ctlDays=42)
Computes ATL/CTL/TSB sequence from daily training stress scores.

7.85.1 Detailed Description

Computes training load metrics from activity power data.

Provides static methods to calculate normalized power, intensity factor, training stress score, and fitness timeline metrics (ATL, CTL, TSB).

Definition at line 43 of file [TrainingLoad.h](#).

7.85.2 Member Function Documentation

7.85.2.1 fitnessTimeline()

```
std::vector< FitnessMetricsPoint > TrainingLoad::fitnessTimeline (
    const std::vector< DailyLoadPoint > & dailyLoads,
    int atlDays = 7,
    int ctlDays = 42) [static]
```

Computes ATL/CTL/TSB sequence from daily training stress scores.

Parameters

<i>dailyLoads</i>	Daily TSS values.
<i>atlDays</i>	Acute training load window in days (default 7).
<i>ctlDays</i>	Chronic training load window in days (default 42).

Returns

Timeline of fitness metrics.

Definition at line 54 of file [TrainingLoad.cpp](#).

References [FitnessMetricsPoint::atl](#), [FitnessMetricsPoint::ctl](#), and [FitnessMetricsPoint::tsb](#).

7.85.2.2 intensityFactor()

```
double TrainingLoad::intensityFactor (
    double np,
    double ftp) [static]
```

Calculates intensity factor.

Parameters

<i>np</i>	Normalized power in watts.
<i>ftp</i>	Functional threshold power in watts.

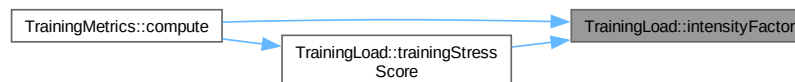
Returns

Intensity factor (NP/FTP ratio).

Definition at line 40 of file [TrainingLoad.cpp](#).

Referenced by [TrainingMetrics::compute\(\)](#), and [trainingStressScore\(\)](#).

Here is the caller graph for this function:

**7.85.2.3 normalizedPower()**

```
double TrainingLoad::normalizedPower (
    const RideData & rideData) [static]
```

Calculates normalized power for an activity.

Parameters

<i>rideData</i>	Activity data to analyze.
-----------------	---

Returns

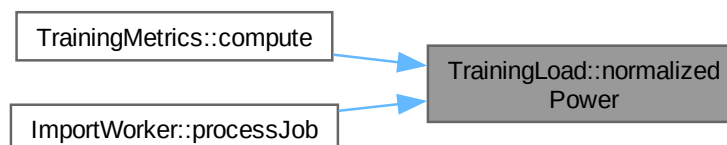
Normalized power in watts.

Definition at line 9 of file [TrainingLoad.cpp](#).

References [RideData::records](#).

Referenced by [TrainingMetrics::compute\(\)](#), and [ImportWorker::processJob\(\)](#).

Here is the caller graph for this function:

**7.85.2.4 trainingStressScore()**

```
double TrainingLoad::trainingStressScore (
    double durationSeconds,
```

```
double np,
double ftp) [static]
```

Calculates training stress score for an activity.

Parameters

<i>durationSeconds</i>	Activity duration in seconds.
<i>np</i>	Normalized power in watts.
<i>ftp</i>	Functional threshold power in watts.

Returns

Training stress score.

Definition at line 46 of file [TrainingLoad.cpp](#).

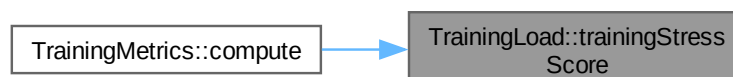
References [intensityFactor\(\)](#).

Referenced by [TrainingMetrics::compute\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



The documentation for this class was generated from the following files:

- `src/analysis/TrainingLoad.h`
- `src/analysis/TrainingLoad.cpp`

7.86 TrainingMetrics Class Reference

Computes training metrics from activity data.

```
#include <TrainingMetrics.h>
```

Static Public Member Functions

- static [TrainingMetricsSummary compute](#) (const [RideData](#) &rideData, double ftp, double duration↵
Seconds=0.0)
Computes training metrics for an activity.

7.86.1 Detailed Description

Computes training metrics from activity data.
Definition at line 38 of file [TrainingMetrics.h](#).

7.86.2 Member Function Documentation

7.86.2.1 compute()

```
TrainingMetricsSummary TrainingMetrics::compute (
    const RideData & rideData,
    double ftp,
    double durationSeconds = 0.0) [static]
```

Computes training metrics for an activity.

Parameters

<i>rideData</i>	Activity data.
<i>ftp</i>	Functional threshold power in watts.
<i>durationSeconds</i>	Optional duration override (0 = use data duration).

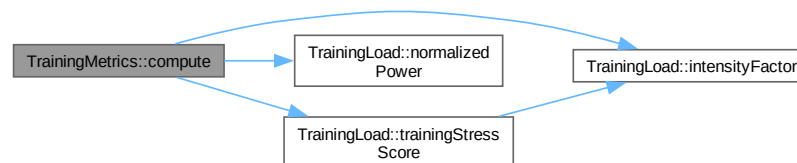
Returns

Metrics summary.

Definition at line 8 of file [TrainingMetrics.cpp](#).

References [TrainingMetricsSummary::averageHeartRate](#), [TrainingMetricsSummary::averagePower](#), [TrainingMetricsSummary::efficiency](#), [RideRecord::hasHeartRate](#), [RideRecord::hasPower](#), [RideRecord::heartRate](#), [TrainingLoad::intensityFactor\(\)](#), [TrainingMetricsSummary::intensityFactor](#), [TrainingLoad::normalizedPower\(\)](#), [TrainingMetricsSummary::normalizedPower](#), [RideRecord::power](#), [RideData::records](#), [TrainingLoad::trainingStressScore\(\)](#), [TrainingMetricsSummary::trainingStressScore](#), and [TrainingMetricsSummary::variabilityIndex](#).

Here is the call graph for this function:



The documentation for this class was generated from the following files:

- `src/analysis/TrainingMetrics.h`
- `src/analysis/TrainingMetrics.cpp`

7.87 TrainingMetricsSummary Struct Reference

Summary of training-related metrics for an activity.
`#include <TrainingMetrics.h>`

Public Attributes

- double `normalizedPower` = 0.0

- Normalized power in watts.*
- double [intensityFactor](#) = 0.0
Intensity factor (normalized power / FTP).
- double [trainingStressScore](#) = 0.0
Training stress score (BikeScore).
- double [variabilityIndex](#) = 0.0
Variability index (normalized power / average power).
- double [efficiencyFactor](#) = 0.0
Efficiency factor (average power / average heart rate).
- double [averagePower](#) = 0.0
Average power in watts.
- double [averageHeartRate](#) = 0.0
Average heart rate in bpm.

7.87.1 Detailed Description

Summary of training-related metrics for an activity.
Definition at line 11 of file [TrainingMetrics.h](#).

7.87.2 Member Data Documentation

7.87.2.1 averageHeartRate

```
double TrainingMetricsSummary::averageHeartRate = 0.0
```

Average heart rate in bpm.
Definition at line 32 of file [TrainingMetrics.h](#).
Referenced by [TrainingMetrics::compute\(\)](#).

7.87.2.2 averagePower

```
double TrainingMetricsSummary::averagePower = 0.0
```

Average power in watts.
Definition at line 29 of file [TrainingMetrics.h](#).
Referenced by [TrainingMetrics::compute\(\)](#).

7.87.2.3 efficiencyFactor

```
double TrainingMetricsSummary::efficiencyFactor = 0.0
```

Efficiency factor (average power / average heart rate).
Definition at line 26 of file [TrainingMetrics.h](#).
Referenced by [TrainingMetrics::compute\(\)](#).

7.87.2.4 intensityFactor

```
double TrainingMetricsSummary::intensityFactor = 0.0
```

Intensity factor (normalized power / FTP).
Definition at line 17 of file [TrainingMetrics.h](#).
Referenced by [TrainingMetrics::compute\(\)](#).

7.87.2.5 normalizedPower

```
double TrainingMetricsSummary::normalizedPower = 0.0
```

Normalized power in watts.
Definition at line 14 of file [TrainingMetrics.h](#).
Referenced by [TrainingMetrics::compute\(\)](#).

7.87.2.6 trainingStressScore

```
double TrainingMetricsSummary::trainingStressScore = 0.0
```

Training stress score (BikeScore).

Definition at line 20 of file [TrainingMetrics.h](#).

Referenced by [TrainingMetrics::compute\(\)](#).

7.87.2.7 variabilityIndex

```
double TrainingMetricsSummary::variabilityIndex = 0.0
```

Variability index (normalized power / average power).

Definition at line 23 of file [TrainingMetrics.h](#).

Referenced by [TrainingMetrics::compute\(\)](#).

The documentation for this struct was generated from the following file:

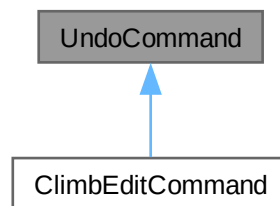
- [src/analysis/TrainingMetrics.h](#)

7.88 UndoCommand Class Reference

Abstract base class for all undo/redo operations.

```
#include <UndoCommand.h>
```

Inheritance diagram for UndoCommand:



Public Member Functions

- virtual void [undo](#) ()=0
Reverses the command (undo operation).
- virtual void [redo](#) ()=0
Re-applies the command (redo operation).
- virtual QString [description](#) () const
Returns a human-readable description of the operation.

7.88.1 Detailed Description

Abstract base class for all undo/redo operations.

Subclasses implement specific reversible edits by providing undo and redo methods.

Definition at line 13 of file [UndoCommand.h](#).

7.88.2 Member Function Documentation

7.88.2.1 description()

```
virtual QString UndoCommand::description () const [inline], [virtual]
```

Returns a human-readable description of the operation.

Returns

Short description shown in menus/status bar.

Reimplemented in [ClimbEditCommand](#).

Definition at line 32 of file [UndoCommand.h](#).

7.88.2.2 redo()

```
virtual void UndoCommand::redo () [pure virtual]
```

Re-applies the command (redo operation).

Implemented in [ClimbEditCommand](#).

7.88.2.3 undo()

```
virtual void UndoCommand::undo () [pure virtual]
```

Reverses the command (undo operation).

Implemented in [ClimbEditCommand](#).

The documentation for this class was generated from the following file:

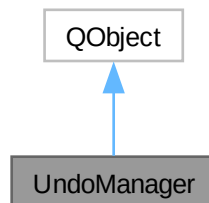
- [src/core/undo/UndoCommand.h](#)

7.89 UndoManager Class Reference

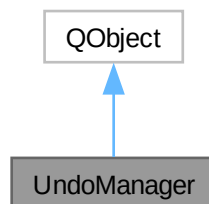
Central undo/redo stack manager.

```
#include <UndoManager.h>
```

Inheritance diagram for UndoManager:



Collaboration diagram for UndoManager:



Public Slots

- void [undo](#) ()
Performs undo operation on the top command.
- void [redo](#) ()
Performs redo operation on the next command.

Signals

- void **canUndoChanged** (bool canUndo)
Emitted when undo availability changes.
- void **canRedoChanged** (bool canRedo)
Emitted when redo availability changes.
- void **undoDescriptionChanged** (const QString &desc)
Emitted when undo description changes.
- void **redoDescriptionChanged** (const QString &desc)
Emitted when redo description changes.

Public Member Functions

- [UndoManager](#) (int maxStackSize=50, QObject *parent=nullptr)
Constructs the undo manager.
- void [push](#) (std::unique_ptr< [UndoCommand](#) > cmd)
Pushes a newly executed command onto the undo stack.
- void [clear](#) ()
Clears all undo and redo history.
- bool [canUndo](#) () const
Checks if undo is available.
- bool [canRedo](#) () const
Checks if redo is available.
- QString [undoDescription](#) () const
Gets description of the next undo operation.
- QString [redoDescription](#) () const
Gets description of the next redo operation.

Private Member Functions

- void [notify](#) ()

Private Attributes

- int [m_maxStackSize](#)
- std::deque< std::unique_ptr< [UndoCommand](#) > > [m_undoStack](#)
- std::deque< std::unique_ptr< [UndoCommand](#) > > [m_redoStack](#)

7.89.1 Detailed Description

Central undo/redo stack manager.

Manages a queue of reversible commands, supporting undo and redo operations with change notifications for UI updates.

Definition at line 20 of file [UndoManager.h](#).

7.89.2 Constructor & Destructor Documentation

7.89.2.1 UndoManager()

```
UndoManager::UndoManager (
    int maxStackSize = 50,
    QObject * parent = nullptr) [explicit]
```

Constructs the undo manager.

Parameters

<i>maxStackSize</i>	Maximum number of commands to retain (default 50).
<i>parent</i>	Qt parent object.

Definition at line 6 of file [UndoManager.cpp](#).

7.89.3 Member Function Documentation

7.89.3.1 canRedo()

```
bool UndoManager::canRedo () const
```

Checks if redo is available.

Returns

True if there are commands to redo.

Definition at line 32 of file [UndoManager.cpp](#).

Referenced by [canRedoChanged\(\)](#).

Here is the caller graph for this function:



7.89.3.2 canUndo()

```
bool UndoManager::canUndo () const
```

Checks if undo is available.

Returns

True if there are commands to undo.

Definition at line 31 of file [UndoManager.cpp](#).

Referenced by [canUndoChanged\(\)](#).

Here is the caller graph for this function:



7.89.3.3 clear()

```
void UndoManager::clear ()
```

Clears all undo and redo history.

Definition at line 24 of file [UndoManager.cpp](#).

7.89.3.4 notify()

```
void UndoManager::notify () [private]
```

Definition at line 68 of file [UndoManager.cpp](#).

7.89.3.5 push()

```
void UndoManager::push (
    std::unique_ptr< UndoCommand > cmd)
```

Pushes a newly executed command onto the undo stack.

Clears the redo stack when a new command is pushed.

Parameters

<i>cmd</i>	Unique pointer to the command to execute.
------------	---

Definition at line 11 of file [UndoManager.cpp](#).

7.89.3.6 redo

```
void UndoManager::redo () [slot]
```

Performs redo operation on the next command.

Definition at line 56 of file [UndoManager.cpp](#).

7.89.3.7 redoDescription()

```
QString UndoManager::redoDescription () const
```

Gets description of the next redo operation.

Returns

Human-readable redo description.

Definition at line 39 of file [UndoManager.cpp](#).

7.89.3.8 undo

```
void UndoManager::undo () [slot]
```

Performs undo operation on the top command.

Definition at line 44 of file [UndoManager.cpp](#).

7.89.3.9 undoDescription()

```
QString UndoManager::undoDescription () const
```

Gets description of the next undo operation.

Returns

Human-readable undo description.

Definition at line 34 of file [UndoManager.cpp](#).

7.89.4 Member Data Documentation**7.89.4.1 m_maxStackSize**

```
int UndoManager::m_maxStackSize [private]
```

Definition at line 103 of file [UndoManager.h](#).

7.89.4.2 m_redoStack

```
std::deque<std::unique_ptr<UndoCommand> > UndoManager::m_redoStack [private]
```

Definition at line 105 of file [UndoManager.h](#).

7.89.4.3 m_undoStack

```
std::deque<std::unique_ptr<UndoCommand> > UndoManager::m_undoStack [private]
```

Definition at line 104 of file [UndoManager.h](#).

The documentation for this class was generated from the following files:

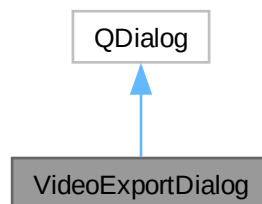
- [src/core/undo/UndoManager.h](#)
- [src/core/undo/UndoManager.cpp](#)

7.90 VideoExportDialog Class Reference

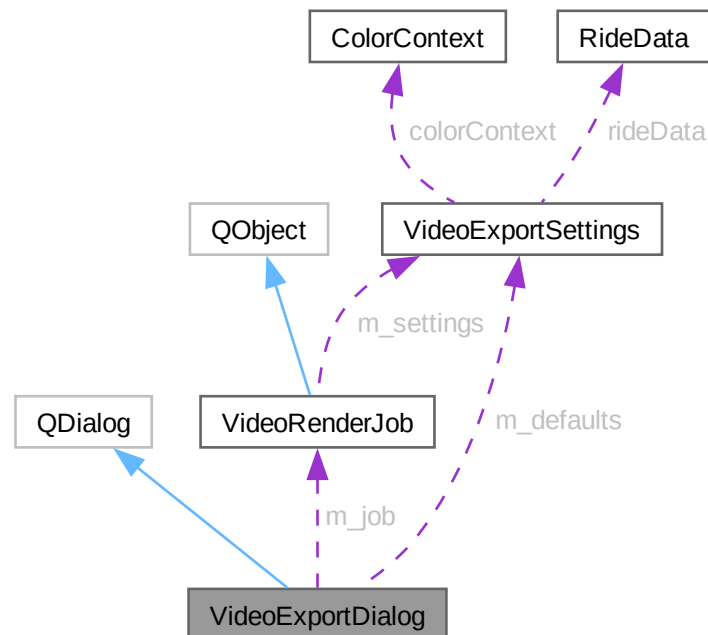
Dialog for configuring and launching activity video export.

```
#include <VideoExportDialog.h>
```

Inheritance diagram for VideoExportDialog:



Collaboration diagram for VideoExportDialog:



Public Member Functions

- [VideoExportDialog](#) (const [VideoExportSettings](#) &defaults, QWidget *parent=nullptr)
Constructs video export dialog.

Private Slots

- void [chooseOutputFile](#) ()
- void [startExport](#) ()
- void [cancelOrClose](#) ()
- void [onStageChanged](#) (const QString &stage)
- void [onProgressChanged](#) (int value, int maximum)
- void [onFrameProgressChanged](#) (int currentFrame, int totalFrames, const QString &etaText)
- void [onTileStatsChanged](#) (const QString &statsText)
- void [onFinished](#) (bool success, const QString &message, bool canceled)

Private Member Functions

- [VideoExportSettings collectSettings](#) () const
- void [setAllOverlayChecks](#) (bool enabled)
- void [updateVideoLengthLabel](#) ()

Private Attributes

- [VideoExportSettings m_defaults](#)
- QLineEdit * [m_outputEdit](#) = nullptr
- QComboBox * [m_resolutionCombo](#) = nullptr

- `QSpinBox * m_fpsSpin = nullptr`
- `QDoubleSpinBox * m_speedSpin = nullptr`
- `QLabel * m_videoLengthLabel = nullptr`
- `QRadioButton * m_selectedSegmentRadio = nullptr`
- `QRadioButton * m_entireActivityRadio = nullptr`
- `QComboBox * m_mapStyleCombo = nullptr`
- `QSpinBox * m_fixedZoomSpin = nullptr`
- `QComboBox * m_routeColorCombo = nullptr`
- `QCheckBox * m_powerCheck = nullptr`
- `QCheckBox * m_hrCheck = nullptr`
- `QCheckBox * m_cadenceCheck = nullptr`
- `QCheckBox * m_speedCheck = nullptr`
- `QCheckBox * m_altitudeCheck = nullptr`
- `QCheckBox * m_timeCheck = nullptr`
- `QCheckBox * m_athleteCheck = nullptr`
- `QCheckBox * m_telemetryPanelCheck = nullptr`
- `QCheckBox * m_legendCheck = nullptr`
- `QCheckBox * m_chartsCheck = nullptr`
- `QCheckBox * m_deleteTemporaryTilesCheck = nullptr`
- `QLabel * m_stageLabel = nullptr`
- `QLabel * m_frameLabel = nullptr`
- `QLabel * m_etaLabel = nullptr`
- `QLabel * m_tileStatsLabel = nullptr`
- `QProgressBar * m_progressBar = nullptr`
- `QPushButton * m_exportButton = nullptr`
- `QPushButton * m_cancelButton = nullptr`
- `QThread * m_thread = nullptr`
- `VideoRenderJob * m_job = nullptr`
- `bool m_running = false`

7.90.1 Detailed Description

Dialog for configuring and launching activity video export.

Provides UI for selecting output file, video format, map style, data overlays, and other video generation settings, then launches rendering job.

Definition at line 31 of file [VideoExportDialog.h](#).

7.90.2 Constructor & Destructor Documentation

7.90.2.1 VideoExportDialog()

```
VideoExportDialog::VideoExportDialog (
    const VideoExportSettings & defaults,
    QWidget * parent = nullptr) [explicit]
```

Constructs video export dialog.

Parameters

<i>defaults</i>	Default export settings.
<i>parent</i>	Parent widget.

Definition at line 85 of file [VideoExportDialog.cpp](#).

References [VideoExportSettings::fixedZoom](#), [VideoExportSettings::fps](#), [VideoExportSettings::height](#), [VideoExportSettings::mapStyle](#), [VideoExportSettings::outputPath](#), [VideoExportSettings::overlayAltitude](#), [VideoExportSettings::overlayAthleteName](#), [VideoExportSettings::overlayCadence](#), [VideoExportSettings::overlayCharts](#), [VideoExportSettings::overlayHeartRate](#), [VideoExportSettings::overlayPower](#), [VideoExportSettings::overlayRouteLegend](#), [VideoExportSettings::overlaySpeed](#), [VideoExportSettings::overlayTelemetryPanel](#), [VideoExportSettings::overlayTime](#), [VideoExportSettings::playbackSpeed](#), [VideoExportSettings::useSelectedSegment](#), and [VideoExportSettings::width](#).

7.90.2.2 ~VideoExportDialog()

VideoExportDialog::~VideoExportDialog () [override]
Definition at line 294 of file [VideoExportDialog.cpp](#).

7.90.3 Member Function Documentation

7.90.3.1 cancelOrClose

void VideoExportDialog::cancelOrClose () [private], [slot]
Definition at line 397 of file [VideoExportDialog.cpp](#).

7.90.3.2 chooseOutputFile

void VideoExportDialog::chooseOutputFile () [private], [slot]
Definition at line 305 of file [VideoExportDialog.cpp](#).

7.90.3.3 collectSettings()

[VideoExportSettings](#) VideoExportDialog::collectSettings () const [private]
Definition at line 316 of file [VideoExportDialog.cpp](#).

7.90.3.4 onFinished

```
void VideoExportDialog::onFinished (
    bool success,
    const QString & message,
    bool canceled) [private], [slot]
```

Definition at line 432 of file [VideoExportDialog.cpp](#).

7.90.3.5 onFrameProgressChanged

```
void VideoExportDialog::onFrameProgressChanged (
    int currentFrame,
    int totalFrames,
    const QString & etaText) [private], [slot]
```

Definition at line 420 of file [VideoExportDialog.cpp](#).

7.90.3.6 onProgressChanged

```
void VideoExportDialog::onProgressChanged (
    int value,
    int maximum) [private], [slot]
```

Definition at line 414 of file [VideoExportDialog.cpp](#).

7.90.3.7 onStageChanged

```
void VideoExportDialog::onStageChanged (
    const QString & stage) [private], [slot]
```

Definition at line 409 of file [VideoExportDialog.cpp](#).

7.90.3.8 onTileStatsChanged

```
void VideoExportDialog::onTileStatsChanged (
    const QString & statsText) [private], [slot]
```

Definition at line 426 of file [VideoExportDialog.cpp](#).

7.90.3.9 setAllOverlayChecks()

```
void VideoExportDialog::setAllOverlayChecks (  
    bool enabled) [private]
```

Definition at line 453 of file [VideoExportDialog.cpp](#).

7.90.3.10 startExport

```
void VideoExportDialog::startExport () [private], [slot]
```

Definition at line 355 of file [VideoExportDialog.cpp](#).

7.90.3.11 updateVideoLengthLabel()

```
void VideoExportDialog::updateVideoLengthLabel () [private]
```

Definition at line 467 of file [VideoExportDialog.cpp](#).

7.90.4 Member Data Documentation

7.90.4.1 m_altitudeCheck

```
QCheckBox* VideoExportDialog::m_altitudeCheck = nullptr [private]
```

Definition at line 79 of file [VideoExportDialog.h](#).

7.90.4.2 m_athleteCheck

```
QCheckBox* VideoExportDialog::m_athleteCheck = nullptr [private]
```

Definition at line 81 of file [VideoExportDialog.h](#).

7.90.4.3 m_cadenceCheck

```
QCheckBox* VideoExportDialog::m_cadenceCheck = nullptr [private]
```

Definition at line 77 of file [VideoExportDialog.h](#).

7.90.4.4 m_cancelButton

```
QPushButton* VideoExportDialog::m_cancelButton = nullptr [private]
```

Definition at line 94 of file [VideoExportDialog.h](#).

7.90.4.5 m_chartsCheck

```
QCheckBox* VideoExportDialog::m_chartsCheck = nullptr [private]
```

Definition at line 84 of file [VideoExportDialog.h](#).

7.90.4.6 m_defaults

```
VideoExportSettings VideoExportDialog::m_defaults [private]
```

Definition at line 59 of file [VideoExportDialog.h](#).

7.90.4.7 m_deleteTemporaryTilesCheck

```
QCheckBox* VideoExportDialog::m_deleteTemporaryTilesCheck = nullptr [private]
```

Definition at line 85 of file [VideoExportDialog.h](#).

7.90.4.8 m_entireActivityRadio

```
QRadioButton* VideoExportDialog::m_entireActivityRadio = nullptr [private]
```

Definition at line 68 of file [VideoExportDialog.h](#).

7.90.4.9 m_etaLabel

QLabel* VideoExportDialog::m_etaLabel = nullptr [private]

Definition at line 89 of file [VideoExportDialog.h](#).

7.90.4.10 m_exportButton

QPushButton* VideoExportDialog::m_exportButton = nullptr [private]

Definition at line 93 of file [VideoExportDialog.h](#).

7.90.4.11 m_fixedZoomSpin

QSpinBox* VideoExportDialog::m_fixedZoomSpin = nullptr [private]

Definition at line 71 of file [VideoExportDialog.h](#).

7.90.4.12 m_fpsSpin

QSpinBox* VideoExportDialog::m_fpsSpin = nullptr [private]

Definition at line 63 of file [VideoExportDialog.h](#).

7.90.4.13 m_frameLabel

QLabel* VideoExportDialog::m_frameLabel = nullptr [private]

Definition at line 88 of file [VideoExportDialog.h](#).

7.90.4.14 m_hrCheck

QCheckBox* VideoExportDialog::m_hrCheck = nullptr [private]

Definition at line 76 of file [VideoExportDialog.h](#).

7.90.4.15 m_job

[VideoRenderJob](#)* VideoExportDialog::m_job = nullptr [private]

Definition at line 97 of file [VideoExportDialog.h](#).

7.90.4.16 m_legendCheck

QCheckBox* VideoExportDialog::m_legendCheck = nullptr [private]

Definition at line 83 of file [VideoExportDialog.h](#).

7.90.4.17 m_mapStyleCombo

QComboBox* VideoExportDialog::m_mapStyleCombo = nullptr [private]

Definition at line 70 of file [VideoExportDialog.h](#).

7.90.4.18 m_outputEdit

QLineEdit* VideoExportDialog::m_outputEdit = nullptr [private]

Definition at line 61 of file [VideoExportDialog.h](#).

7.90.4.19 m_powerCheck

QCheckBox* VideoExportDialog::m_powerCheck = nullptr [private]

Definition at line 75 of file [VideoExportDialog.h](#).

7.90.4.20 m_progressBar

QProgressBar* VideoExportDialog::m_progressBar = nullptr [private]

Definition at line 91 of file [VideoExportDialog.h](#).

7.90.4.21 m_resolutionCombo

`QComboBox* VideoExportDialog::m_resolutionCombo = nullptr [private]`
Definition at line 62 of file [VideoExportDialog.h](#).

7.90.4.22 m_routeColorCombo

`QComboBox* VideoExportDialog::m_routeColorCombo = nullptr [private]`
Definition at line 73 of file [VideoExportDialog.h](#).

7.90.4.23 m_running

`bool VideoExportDialog::m_running = false [private]`
Definition at line 98 of file [VideoExportDialog.h](#).

7.90.4.24 m_selectedSegmentRadio

`QRadioButton* VideoExportDialog::m_selectedSegmentRadio = nullptr [private]`
Definition at line 67 of file [VideoExportDialog.h](#).

7.90.4.25 m_speedCheck

`QCheckBox* VideoExportDialog::m_speedCheck = nullptr [private]`
Definition at line 78 of file [VideoExportDialog.h](#).

7.90.4.26 m_speedSpin

`QDoubleSpinBox* VideoExportDialog::m_speedSpin = nullptr [private]`
Definition at line 64 of file [VideoExportDialog.h](#).

7.90.4.27 m_stageLabel

`QLabel* VideoExportDialog::m_stageLabel = nullptr [private]`
Definition at line 87 of file [VideoExportDialog.h](#).

7.90.4.28 m_telemetryPanelCheck

`QCheckBox* VideoExportDialog::m_telemetryPanelCheck = nullptr [private]`
Definition at line 82 of file [VideoExportDialog.h](#).

7.90.4.29 m_thread

`QThread* VideoExportDialog::m_thread = nullptr [private]`
Definition at line 96 of file [VideoExportDialog.h](#).

7.90.4.30 m_tileStatsLabel

`QLabel* VideoExportDialog::m_tileStatsLabel = nullptr [private]`
Definition at line 90 of file [VideoExportDialog.h](#).

7.90.4.31 m_timeCheck

`QCheckBox* VideoExportDialog::m_timeCheck = nullptr [private]`
Definition at line 80 of file [VideoExportDialog.h](#).

7.90.4.32 m_videoLengthLabel

`QLabel* VideoExportDialog::m_videoLengthLabel = nullptr [private]`
Definition at line 65 of file [VideoExportDialog.h](#).

The documentation for this class was generated from the following files:

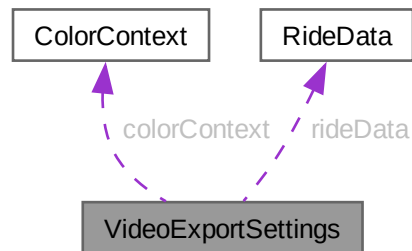
- src/video/VideoExportDialog.h
- src/video/VideoExportDialog.cpp

7.91 VideoExportSettings Struct Reference

Configuration for activity video export.

```
#include <VideoExportSettings.h>
```

Collaboration diagram for VideoExportSettings:



Public Attributes

- QString [outputPath](#)
Output file path.
- QString [ffmpegPath](#)
FFmpeg executable path.
- int [width](#) = 1920
Output video width in pixels.
- int [height](#) = 1080
Output video height in pixels.
- int [fps](#) = 30
Output video frame rate in fps.
- double [playbackSpeed](#) = 3.0
Playback speed multiplier.
- bool [useSelectedSegment](#) = true
Whether to render only selected segment.
- double [segmentStartSeconds](#) = 0.0
Segment start time in seconds.
- double [segmentEndSeconds](#) = 0.0
Segment end time in seconds.
- MapStyle [mapStyle](#) = MapStyle::Light
Map style for rendering.
- bool [autoZoom](#) = true
Whether to enable automatic zoom adjustment.
- int [fixedZoom](#) = 18
Fixed zoom level when autoZoom disabled.
- bool [followAthlete](#) = true
Whether to center map on athlete position.

- ColorMetric [routeColorMetric](#) = ColorMetric::Power
Color metric for route coloring.
- QColor [singleRouteColor](#) = QColor("#2563eb")
Fixed route color when not using metric coloring.
- bool [overlayPower](#) = true
Whether to overlay power data.
- bool [overlayHeartRate](#) = true
Whether to overlay heart rate data.
- bool [overlayCadence](#) = true
Whether to overlay cadence data.
- bool [overlaySpeed](#) = true
Whether to overlay speed data.
- bool [overlayAltitude](#) = true
Whether to overlay altitude data.
- bool [overlayTime](#) = true
Whether to overlay elapsed time.
- bool [overlayAthleteName](#) = true
Whether to overlay athlete name.
- bool [overlayTelemetryPanel](#) = true
Whether to overlay telemetry panel.
- bool [overlayRouteLegend](#) = true
Whether to overlay route legend.
- bool [overlayCharts](#) = true
Whether to overlay metric charts.
- bool [deleteTemporaryTilesAfterExport](#) = true
Whether to delete temporary tiles after export.
- QString [athleteName](#)
Athlete name for display.
- QString [activityName](#)
Activity name for display.
- ColorContext [colorContext](#)
Color context for metric coloring.
- RideData [rideData](#)
Activity ride data for rendering.

7.91.1 Detailed Description

Configuration for activity video export.

Specifies output format, map rendering options, data overlays, and telemetry display settings for video generation.

Definition at line 19 of file [VideoExportSettings.h](#).

7.91.2 Member Data Documentation

7.91.2.1 [activityName](#)

QString VideoExportSettings::activityName

[Activity](#) name for display.

Definition at line 103 of file [VideoExportSettings.h](#).

7.91.2.2 [athleteName](#)

QString VideoExportSettings::athleteName

[Athlete](#) name for display.

Definition at line 100 of file [VideoExportSettings.h](#).

7.91.2.3 autoZoom

```
bool VideoExportSettings::autoZoom = true
```

Whether to enable automatic zoom adjustment.

Definition at line 52 of file [VideoExportSettings.h](#).

7.91.2.4 colorContext

```
ColorContext VideoExportSettings::colorContext
```

Color context for metric coloring.

Definition at line 106 of file [VideoExportSettings.h](#).

7.91.2.5 deleteTemporaryTilesAfterExport

```
bool VideoExportSettings::deleteTemporaryTilesAfterExport = true
```

Whether to delete temporary tiles after export.

Definition at line 97 of file [VideoExportSettings.h](#).

7.91.2.6 ffmpegPath

```
QString VideoExportSettings::ffmpegPath
```

FFmpeg executable path.

Definition at line 25 of file [VideoExportSettings.h](#).

7.91.2.7 fixedZoom

```
int VideoExportSettings::fixedZoom = 18
```

Fixed zoom level when autoZoom disabled.

Definition at line 55 of file [VideoExportSettings.h](#).

Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

7.91.2.8 followAthlete

```
bool VideoExportSettings::followAthlete = true
```

Whether to center map on athlete position.

Definition at line 58 of file [VideoExportSettings.h](#).

7.91.2.9 fps

```
int VideoExportSettings::fps = 30
```

Output video frame rate in fps.

Definition at line 34 of file [VideoExportSettings.h](#).

Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

7.91.2.10 height

```
int VideoExportSettings::height = 1080
```

Output video height in pixels.

Definition at line 31 of file [VideoExportSettings.h](#).

Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

7.91.2.11 mapStyle

```
MapStyle VideoExportSettings::mapStyle = MapStyle::Light
```

Map style for rendering.

Definition at line 49 of file [VideoExportSettings.h](#).

Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

7.91.2.12 outputPath

```
QString VideoExportSettings::outputPath
```

Output file path.

Definition at line 22 of file [VideoExportSettings.h](#).

Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

7.91.2.13 overlayAltitude

```
bool VideoExportSettings::overlayAltitude = true
```

Whether to overlay altitude data.

Definition at line 79 of file [VideoExportSettings.h](#).

Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

7.91.2.14 overlayAthleteName

```
bool VideoExportSettings::overlayAthleteName = true
```

Whether to overlay athlete name.

Definition at line 85 of file [VideoExportSettings.h](#).

Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

7.91.2.15 overlayCadence

```
bool VideoExportSettings::overlayCadence = true
```

Whether to overlay cadence data.

Definition at line 73 of file [VideoExportSettings.h](#).

Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

7.91.2.16 overlayCharts

```
bool VideoExportSettings::overlayCharts = true
```

Whether to overlay metric charts.

Definition at line 94 of file [VideoExportSettings.h](#).

Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

7.91.2.17 overlayHeartRate

```
bool VideoExportSettings::overlayHeartRate = true
```

Whether to overlay heart rate data.

Definition at line 70 of file [VideoExportSettings.h](#).

Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

7.91.2.18 overlayPower

```
bool VideoExportSettings::overlayPower = true
```

Whether to overlay power data.

Definition at line 67 of file [VideoExportSettings.h](#).

Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

7.91.2.19 overlayRouteLegend

```
bool VideoExportSettings::overlayRouteLegend = true
```

Whether to overlay route legend.

Definition at line 91 of file [VideoExportSettings.h](#).

Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

7.91.2.20 overlaySpeed

```
bool VideoExportSettings::overlaySpeed = true
```

Whether to overlay speed data.

Definition at line 76 of file [VideoExportSettings.h](#).
Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

7.91.2.21 overlayTelemetryPanel

```
bool VideoExportSettings::overlayTelemetryPanel = true
```

Whether to overlay telemetry panel.
Definition at line 88 of file [VideoExportSettings.h](#).
Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

7.91.2.22 overlayTime

```
bool VideoExportSettings::overlayTime = true
```

Whether to overlay elapsed time.
Definition at line 82 of file [VideoExportSettings.h](#).
Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

7.91.2.23 playbackSpeed

```
double VideoExportSettings::playbackSpeed = 3.0
```

Playback speed multiplier.
Definition at line 37 of file [VideoExportSettings.h](#).
Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

7.91.2.24 rideData

```
RideData VideoExportSettings::rideData
```

[Activity](#) ride data for rendering.
Definition at line 109 of file [VideoExportSettings.h](#).

7.91.2.25 routeColorMetric

```
ColorMetric VideoExportSettings::routeColorMetric = ColorMetric::Power
```

Color metric for route coloring.
Definition at line 61 of file [VideoExportSettings.h](#).

7.91.2.26 segmentEndSeconds

```
double VideoExportSettings::segmentEndSeconds = 0.0
```

Segment end time in seconds.
Definition at line 46 of file [VideoExportSettings.h](#).

7.91.2.27 segmentStartSeconds

```
double VideoExportSettings::segmentStartSeconds = 0.0
```

Segment start time in seconds.
Definition at line 43 of file [VideoExportSettings.h](#).

7.91.2.28 singleRouteColor

```
QColor VideoExportSettings::singleRouteColor = QColor("#2563eb")
```

Fixed route color when not using metric coloring.
Definition at line 64 of file [VideoExportSettings.h](#).

7.91.2.29 useSelectedSegment

```
bool VideoExportSettings::useSelectedSegment = true
```

Whether to render only selected segment.
Definition at line 40 of file [VideoExportSettings.h](#).
Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

7.91.2.30 width

```
int VideoExportSettings::width = 1920
```

Output video width in pixels.

Definition at line 28 of file [VideoExportSettings.h](#).

Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

The documentation for this struct was generated from the following file:

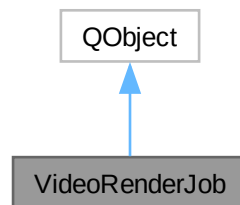
- `src/video/VideoExportSettings.h`

7.92 VideoRenderJob Class Reference

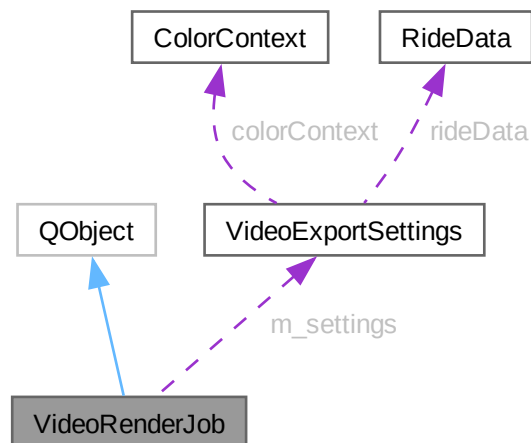
Renders activity video with map overlay and data visualization.

```
#include <VideoRenderJob.h>
```

Inheritance diagram for VideoRenderJob:



Collaboration diagram for VideoRenderJob:



Signals

- void [stageChanged](#) (const QString &stage)

- *@signal Emitted when render stage changes.*
- void [progressChanged](#) (int value, int maximum)
- *@signal Emitted when progress updates.*
- void [frameProgressChanged](#) (int currentFrame, int totalFrames, const QString &etaText)
- *@signal Emitted with frame-level progress.*
- void [tileStatsChanged](#) (const QString &statsText)
- *@signal Emitted when tile cache statistics update.*
- void [finished](#) (bool success, const QString &message, bool canceled)
- *@signal Emitted when rendering completes or fails.*

Public Member Functions

- [VideoRenderJob](#) ([VideoExportSettings](#) settings, QObject *parent=nullptr)
- *Constructs a video render job.*
- void [run](#) ()
- *@slot Runs the video rendering process.*
- void [cancel](#) ()
- *@slot Requests cancellation of rendering.*

Private Attributes

- [VideoExportSettings](#) m_settings
- QAtomicInteger< int > [m_cancelRequested](#) { 0 }

7.92.1 Detailed Description

Renders activity video with map overlay and data visualization.

Generates video file from activity GPS and performance data, handling frame rendering, ffmpeg encoding, and progress reporting.

Definition at line 17 of file [VideoRenderJob.h](#).

7.92.2 Constructor & Destructor Documentation

7.92.2.1 VideoRenderJob()

```
VideoRenderJob::VideoRenderJob (
    VideoExportSettings settings,
    QObject * parent = nullptr) [explicit]
```

Constructs a video render job.

Parameters

<i>settings</i>	Export settings.
<i>parent</i>	Parent object.

Definition at line 255 of file [VideoRenderJob.cpp](#).

7.92.3 Member Function Documentation

7.92.3.1 cancel()

```
void VideoRenderJob::cancel ()
```

@slot Requests cancellation of rendering.

Definition at line 260 of file [VideoRenderJob.cpp](#).

7.92.3.2 finished

```
void VideoRenderJob::finished (
    bool success,
    const QString & message,
    bool canceled) [signal]
```

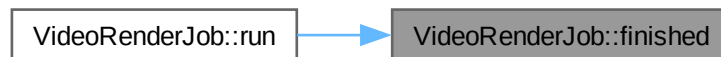
@signal Emitted when rendering completes or fails.

Parameters

<i>success</i>	True if render completed successfully.
<i>message</i>	Status or error message.
<i>canceled</i>	True if render was canceled by user.

Referenced by [run\(\)](#).

Here is the caller graph for this function:



7.92.3.3 frameProgressChanged

```
void VideoRenderJob::frameProgressChanged (
    int currentFrame,
    int totalFrames,
    const QString & etaText) [signal]
```

@signal Emitted with frame-level progress.

Parameters

<i>currentFrame</i>	Current frame number.
<i>totalFrames</i>	Total number of frames.
<i>etaText</i>	Estimated time remaining.

Referenced by [run\(\)](#).

Here is the caller graph for this function:



7.92.3.4 progressChanged

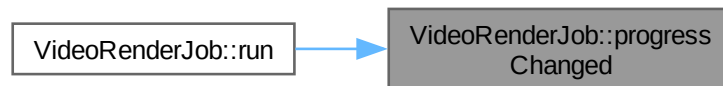
```
void VideoRenderJob::progressChanged (
    int value,
    int maximum) [signal]
@signal Emitted when progress updates.
```

Parameters

<i>value</i>	Current progress value.
<i>maximum</i>	Maximum progress value.

Referenced by [run\(\)](#).

Here is the caller graph for this function:



7.92.3.5 run()

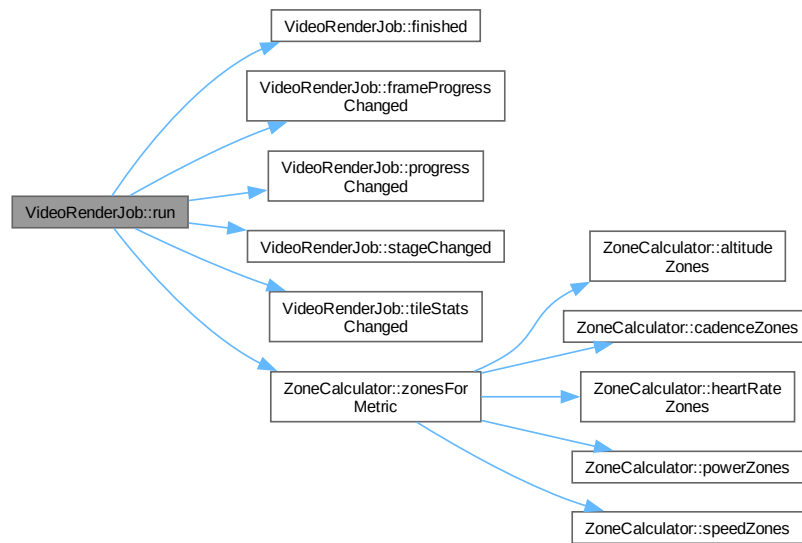
```
void VideoRenderJob::run ()
```

@slot Runs the video rendering process.

Definition at line 265 of file [VideoRenderJob.cpp](#).

References [RideRecord::altitude](#), [RideRecord::cadence](#), [VideoTileStats::downloadedTiles](#), [RideRecord::elapsedSeconds](#), [finished\(\)](#), [frameProgressChanged\(\)](#), [RideRecord::hasAltitude](#), [RideRecord::hasCadence](#), [RideRecord::hasGps](#), [RideRecord::hasHeartRate](#), [RideRecord::hasPower](#), [RideRecord::hasSpeed](#), [RideRecord::heartRate](#), [RideRecord::latitude](#), [RideRecord::longitude](#), [VideoTileStats::onDiskTiles](#), [RideRecord::power](#), [progressChanged\(\)](#), [VideoTileStats::requiredTiles](#), [RideRecord::speed](#), [stageChanged\(\)](#), [tileStatsChanged\(\)](#), and [ZoneCalculator::zonesForMetric\(\)](#).

Here is the call graph for this function:



7.92.3.6 stageChanged

```
void VideoRenderJob::stageChanged (
    const QString & stage) [signal]
```

@signal Emitted when render stage changes.

Parameters

<i>stage</i>	Description of current stage.
--------------	-------------------------------

Referenced by [run\(\)](#).

Here is the caller graph for this function:



7.92.3.7 tileStatsChanged

```
void VideoRenderJob::tileStatsChanged (
    const QString & statsText) [signal]
```

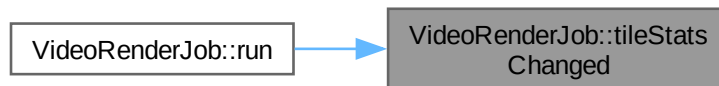
@signal Emitted when tile cache statistics update.

Parameters

<i>statsText</i>	Formatted statistics text.
------------------	----------------------------

Referenced by [run\(\)](#).

Here is the caller graph for this function:

**7.92.4 Member Data Documentation****7.92.4.1 m_cancelRequested**

```
QAtomicInteger<int> VideoRenderJob::m_cancelRequested { 0 } [private]
```

Definition at line 67 of file [VideoRenderJob.h](#).

7.92.4.2 m_settings

```
VideoExportSettings VideoRenderJob::m_settings [private]
```

Definition at line 66 of file [VideoRenderJob.h](#).

The documentation for this class was generated from the following files:

- [src/video/VideoRenderJob.h](#)
- [src/video/VideoRenderJob.cpp](#)

7.93 VideoTileId Struct Reference

Unique identifier for a map tile (z/x/y coordinates).

```
#include <VideoTileProvider.h>
```

Public Member Functions

- bool [operator==](#) (const [VideoTileId](#) &other) const
Equality comparison operator.

Public Attributes

- int [z](#) = 0
Zoom level.
- int [x](#) = 0
X coordinate.
- int [y](#) = 0
Y coordinate.

7.93.1 Detailed Description

Unique identifier for a map tile (z/x/y coordinates).

Definition at line 33 of file [VideoTileProvider.h](#).

7.93.2 Member Function Documentation

7.93.2.1 operator==()

```
bool VideoTileId::operator==(
    const VideoTileId & other) const [inline]
```

Equality comparison operator.

Definition at line 45 of file [VideoTileProvider.h](#).

References [x](#), [y](#), and [z](#).

7.93.3 Member Data Documentation

7.93.3.1 x

```
int VideoTileId::x = 0
```

X coordinate.

Definition at line 39 of file [VideoTileProvider.h](#).

Referenced by [operator==\(\)](#).

7.93.3.2 y

```
int VideoTileId::y = 0
```

Y coordinate.

Definition at line 42 of file [VideoTileProvider.h](#).

Referenced by [operator==\(\)](#).

7.93.3.3 z

```
int VideoTileId::z = 0
```

Zoom level.

Definition at line 36 of file [VideoTileProvider.h](#).

Referenced by [operator==\(\)](#).

The documentation for this struct was generated from the following file:

- [src/video/VideoTileProvider.h](#)

7.94 VideoTileIdHash Struct Reference

Hash function for [VideoTileId](#).

```
#include <VideoTileProvider.h>
```

Public Member Functions

- `std::size_t operator() (const VideoTileId &id) const noexcept`
Computes hash for tile ID.

7.94.1 Detailed Description

Hash function for [VideoTileId](#).

Definition at line 54 of file [VideoTileProvider.h](#).

7.94.2 Member Function Documentation

7.94.2.1 operator()()

```
std::size_t VideoTileIdHash::operator() (
    const VideoTileId & id) const [inline], [noexcept]
```

Computes hash for tile ID.

Definition at line 57 of file [VideoTileProvider.h](#).

The documentation for this struct was generated from the following file:

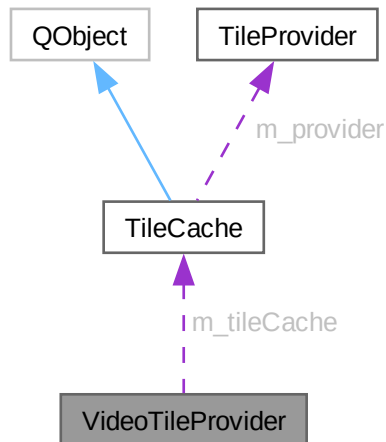
- src/video/VideoTileProvider.h

7.95 VideoTileProvider Class Reference

Manages map tiles for video export.

```
#include <VideoTileProvider.h>
```

Collaboration diagram for VideoTileProvider:



Public Member Functions

- [VideoTileProvider](#) (const [VideoTileProviderConfig](#) &config)
Constructs tile provider with configuration.
- void [setRequiredTiles](#) (const std::unordered_set< [VideoTileId](#), [VideoTileIdHash](#) > &required)
Sets the set of tiles required for video export.
- int [requiredTileCount](#) () const
Gets count of required tiles.
- int [requiredTilesAvailableOnDisk](#) () const
Gets count of required tiles available on disk.
- QPixmap [getTile](#) (const [VideoTileId](#) &id, bool allowNetwork)
Retrieves a tile for rendering.
- bool [isTileCachedOnDisk](#) (const [VideoTileId](#) &id) const
Checks if a tile is cached on disk.
- [VideoTileStats](#) [stats](#) () const
Gets cache statistics.
- void [clearMemoryCache](#) ()
Clears in-memory tile cache.
- void [cleanupExportCache](#) ()
Cleans up export cache directory.
- bool [shouldDeleteExportCacheAfterExport](#) () const
Checks if export cache should be deleted after export.
- QString [exportCacheRoot](#) () const
Gets export cache root directory.

Private Attributes

- [TileCache m_tileCache](#)
- bool [m_deleteTemporaryTilesAfterExport](#) = true
- std::unordered_set< [VideoTileId](#), [VideoTileIdHash](#) > [m_requiredTiles](#)
- int [m_requiredTilesOnDisk](#) = 0
- int [m_downloadedTiles](#) = 0

7.95.1 Detailed Description

Manages map tiles for video export.

Handles tile caching, download coordination, and cleanup for video rendering.

Definition at line 86 of file [VideoTileProvider.h](#).

7.95.2 Constructor & Destructor Documentation

7.95.2.1 VideoTileProvider()

```
VideoTileProvider::VideoTileProvider (
    const VideoTileProviderConfig & config) [explicit]
```

Constructs tile provider with configuration.

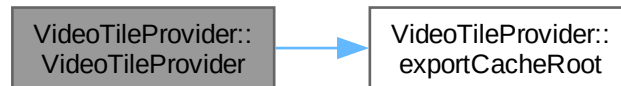
Parameters

<i>config</i>	Provider configuration.
---------------	-------------------------

Definition at line 30 of file [VideoTileProvider.cpp](#).

References [exportCacheRoot\(\)](#), and [VideoTileProviderConfig::style](#).

Here is the call graph for this function:



7.95.3 Member Function Documentation

7.95.3.1 cleanupExportCache()

```
void VideoTileProvider::cleanupExportCache ()
```

Cleans up export cache directory.

Definition at line 95 of file [VideoTileProvider.cpp](#).

7.95.3.2 clearMemoryCache()

```
void VideoTileProvider::clearMemoryCache ()
```

Clears in-memory tile cache.

Definition at line 90 of file [VideoTileProvider.cpp](#).

7.95.3.3 exportCacheRoot()

QString VideoTileProvider::exportCacheRoot () const
 Gets export cache root directory.

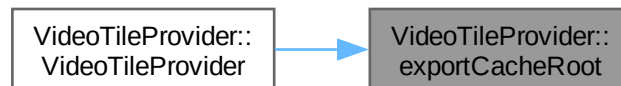
Returns

Cache directory path.

Definition at line 110 of file [VideoTileProvider.cpp](#).

Referenced by [VideoTileProvider\(\)](#).

Here is the caller graph for this function:



7.95.3.4 getTile()

QPixmap VideoTileProvider::getTile (
 const VideoTileId & id,
 bool allowNetwork)

Retrieves a tile for rendering.

Parameters

<i>id</i>	Tile identifier.
<i>allowNetwork</i>	True to fetch from network if not cached.

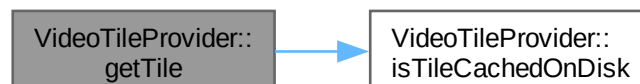
Returns

Tile pixmap, or empty if unavailable.

Definition at line 67 of file [VideoTileProvider.cpp](#).

References [isTileCachedOnDisk\(\)](#).

Here is the call graph for this function:



7.95.3.5 isTileCachedOnDisk()

```
bool VideoTileProvider::isTileCachedOnDisk (
    const VideoTileId & id) const
```

Checks if a tile is cached on disk.

Parameters

<i>id</i>	Tile identifier.
-----------	------------------

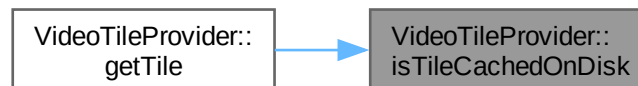
Returns

True if tile exists in disk cache.

Definition at line 85 of file [VideoTileProvider.cpp](#).

Referenced by [getTile\(\)](#).

Here is the caller graph for this function:



7.95.3.6 requiredTileCount()

```
int VideoTileProvider::requiredTileCount () const
```

Gets count of required tiles.

Returns

Number of required tiles.

Definition at line 57 of file [VideoTileProvider.cpp](#).

7.95.3.7 requiredTilesAvailableOnDisk()

```
int VideoTileProvider::requiredTilesAvailableOnDisk () const
```

Gets count of required tiles available on disk.

Returns

Number of tiles on disk cache.

Definition at line 62 of file [VideoTileProvider.cpp](#).

7.95.3.8 setRequiredTiles()

```
void VideoTileProvider::setRequiredTiles (
    const std::unordered_set< VideoTileId, VideoTileIdHash > & required)
```

Sets the set of tiles required for video export.

Parameters

<i>required</i>	Set of required tile IDs.
-----------------	---------------------------

Definition at line 44 of file [VideoTileProvider.cpp](#).

7.95.3.9 shouldDeleteExportCacheAfterExport()

```
bool VideoTileProvider::shouldDeleteExportCacheAfterExport () const
```

Checks if export cache should be deleted after export.

Returns

True if temporary tiles should be cleaned up.

Definition at line 105 of file [VideoTileProvider.cpp](#).

7.95.3.10 stats()

```
VideoTileStats VideoTileProvider::stats () const
```

Gets cache statistics.

Returns

Stats including required, on-disk, and downloaded counts.

Definition at line 76 of file [VideoTileProvider.cpp](#).

References [VideoTileStats::downloadedTiles](#), [VideoTileStats::onDiskTiles](#), and [VideoTileStats::requiredTiles](#).

7.95.4 Member Data Documentation**7.95.4.1 m_deleteTemporaryTilesAfterExport**

```
bool VideoTileProvider::m_deleteTemporaryTilesAfterExport = true [private]
```

Definition at line 158 of file [VideoTileProvider.h](#).

7.95.4.2 m_downloadedTiles

```
int VideoTileProvider::m_downloadedTiles = 0 [private]
```

Definition at line 161 of file [VideoTileProvider.h](#).

7.95.4.3 m_requiredTiles

```
std::unordered_set<VideoTileId, VideoTileIdHash> VideoTileProvider::m_requiredTiles [private]
```

Definition at line 159 of file [VideoTileProvider.h](#).

7.95.4.4 m_requiredTilesOnDisk

```
int VideoTileProvider::m_requiredTilesOnDisk = 0 [private]
```

Definition at line 160 of file [VideoTileProvider.h](#).

7.95.4.5 m_tileCache

```
TileCache VideoTileProvider::m_tileCache [private]
```

Definition at line 157 of file [VideoTileProvider.h](#).

The documentation for this class was generated from the following files:

- [src/video/VideoTileProvider.h](#)
- [src/video/VideoTileProvider.cpp](#)

7.96 VideoTileProviderConfig Struct Reference

Configuration for video tile provider.

```
#include <VideoTileProvider.h>
```

Public Attributes

- MapStyle [style](#) = MapStyle::Light
Map style for video.
- int [memoryCacheTiles](#) = 96
Maximum tiles to keep in memory.
- bool [deleteTemporaryTilesAfterExport](#) = true
Whether to delete temporary tiles after export.
- QString [exportCacheRoot](#)
Root directory for export cache.

7.96.1 Detailed Description

Configuration for video tile provider.

Definition at line 15 of file [VideoTileProvider.h](#).

7.96.2 Member Data Documentation

7.96.2.1 deleteTemporaryTilesAfterExport

```
bool VideoTileProviderConfig::deleteTemporaryTilesAfterExport = true
```

Whether to delete temporary tiles after export.

Definition at line 24 of file [VideoTileProvider.h](#).

7.96.2.2 exportCacheRoot

```
QString VideoTileProviderConfig::exportCacheRoot
```

Root directory for export cache.

Definition at line 27 of file [VideoTileProvider.h](#).

7.96.2.3 memoryCacheTiles

```
int VideoTileProviderConfig::memoryCacheTiles = 96
```

Maximum tiles to keep in memory.

Definition at line 21 of file [VideoTileProvider.h](#).

7.96.2.4 style

```
MapStyle VideoTileProviderConfig::style = MapStyle::Light
```

Map style for video.

Definition at line 18 of file [VideoTileProvider.h](#).

Referenced by [VideoTileProvider::VideoTileProvider\(\)](#).

The documentation for this struct was generated from the following file:

- src/video/VideoTileProvider.h

7.97 VideoTileStats Struct Reference

Tile availability statistics for export.

```
#include <VideoTileProvider.h>
```

Public Attributes

- int [requiredTiles](#) = 0
Number of tiles required for video.
- int [onDiskTiles](#) = 0
Number of tiles available on disk.
- int [downloadedTiles](#) = 0
Number of tiles downloaded during export prep.

7.97.1 Detailed Description

Tile availability statistics for export.

Definition at line 69 of file [VideoTileProvider.h](#).

7.97.2 Member Data Documentation**7.97.2.1 downloadedTiles**

```
int VideoTileStats::downloadedTiles = 0
```

Number of tiles downloaded during export prep.

Definition at line 78 of file [VideoTileProvider.h](#).

Referenced by [VideoRenderJob::run\(\)](#), and [VideoTileProvider::stats\(\)](#).

7.97.2.2 onDiskTiles

```
int VideoTileStats::onDiskTiles = 0
```

Number of tiles available on disk.

Definition at line 75 of file [VideoTileProvider.h](#).

Referenced by [VideoRenderJob::run\(\)](#), and [VideoTileProvider::stats\(\)](#).

7.97.2.3 requiredTiles

```
int VideoTileStats::requiredTiles = 0
```

Number of tiles required for video.

Definition at line 72 of file [VideoTileProvider.h](#).

Referenced by [VideoRenderJob::run\(\)](#), and [VideoTileProvider::stats\(\)](#).

The documentation for this struct was generated from the following file:

- [src/video/VideoTileProvider.h](#)

7.98 WeightEntry Struct Reference

Represents a historical weight entry.

```
#include <AthleteRepository.h>
```

Public Attributes

- int [id](#) = -1
Unique weight entry identifier.
- int [athleteId](#) = -1
Associated athlete ID.
- double [weightKg](#) = 0.0
Body weight in kilograms.
- QString [effectiveFrom](#)
Date weight becomes effective (ISO 8601 YYYY-MM-DD).

7.98.1 Detailed Description

Represents a historical weight entry.

Definition at line 76 of file [AthleteRepository.h](#).

7.98.2 Member Data Documentation

7.98.2.1 athleteId

```
int WeightEntry::athleteId = -1
```

Associated athlete ID.

Definition at line 82 of file [AthleteRepository.h](#).

Referenced by [AthleteRepository::addWeightEntry\(\)](#), and [AthleteRepository::getWeightHistory\(\)](#).

7.98.2.2 effectiveFrom

```
QString WeightEntry::effectiveFrom
```

Date weight becomes effective (ISO 8601 YYYY-MM-DD).

Definition at line 88 of file [AthleteRepository.h](#).

Referenced by [AthleteRepository::addWeightEntry\(\)](#), and [AthleteRepository::getWeightHistory\(\)](#).

7.98.2.3 id

```
int WeightEntry::id = -1
```

Unique weight entry identifier.

Definition at line 79 of file [AthleteRepository.h](#).

Referenced by [AthleteRepository::getWeightHistory\(\)](#).

7.98.2.4 weightKg

```
double WeightEntry::weightKg = 0.0
```

Body weight in kilograms.

Definition at line 85 of file [AthleteRepository.h](#).

Referenced by [AthleteRepository::addWeightEntry\(\)](#), and [AthleteRepository::getWeightHistory\(\)](#).

The documentation for this struct was generated from the following file:

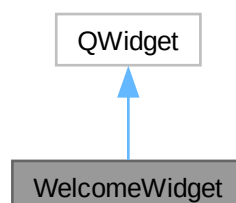
- [src/database/AthleteRepository.h](#)

7.99 WelcomeWidget Class Reference

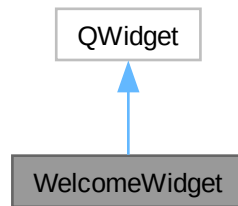
Welcome screen for first-time users.

```
#include <WelcomeWidget.h>
```

Inheritance diagram for WelcomeWidget:



Collaboration diagram for WelcomeWidget:



Signals

- void **importRequested** ()
signal Emitted when user requests activity import.
- void **openDatabaseRequested** ()
signal Emitted when user requests to open existing database.
- void **createDatabaseRequested** ()
signal Emitted when user requests to create new database.

Public Member Functions

- [WelcomeWidget](#) (QWidget *parent=nullptr)
Constructs welcome widget.
- void [setFirstLaunch](#) (bool firstLaunch)
Sets first launch mode.

Private Attributes

- QLabel * [m_subtitleLabel](#) = nullptr
- QPushButton * [m_importButton](#) = nullptr
- QPushButton * [m_openDbButton](#) = nullptr
- QPushButton * [m_createDbButton](#) = nullptr

7.99.1 Detailed Description

Welcome screen for first-time users.

Provides quick action buttons for creating database, opening existing, or importing activities.

Definition at line 17 of file [WelcomeWidget.h](#).

7.99.2 Constructor & Destructor Documentation

7.99.2.1 WelcomeWidget()

```

WelcomeWidget::WelcomeWidget (
    QWidget * parent = nullptr) [explicit]

```

Constructs welcome widget.

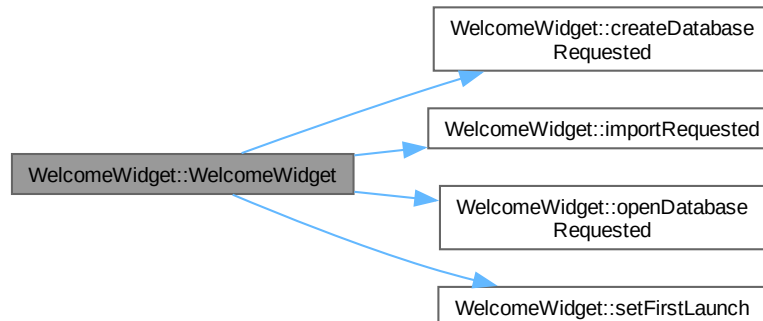
Parameters

<i>parent</i>	Parent widget.
---------------	----------------

Definition at line 12 of file [WelcomeWidget.cpp](#).

References [createDatabaseRequested\(\)](#), [importRequested\(\)](#), [openDatabaseRequested\(\)](#), and [setFirstLaunch\(\)](#).

Here is the call graph for this function:



7.99.3 Member Function Documentation

7.99.3.1 setFirstLaunch()

```
void WelcomeWidget::setFirstLaunch (
    bool firstLaunch)
```

Sets first launch mode.

Parameters

<i>firstLaunch</i>	True to show first-launch UI.
--------------------	-------------------------------

Definition at line 81 of file [WelcomeWidget.cpp](#).

Referenced by [WelcomeWidget\(\)](#).

Here is the caller graph for this function:



7.99.4 Member Data Documentation

7.99.4.1 m_createDbButton

```
QPushButton* WelcomeWidget::m_createDbButton = nullptr [private]
```

Definition at line 48 of file [WelcomeWidget.h](#).

7.99.4.2 m_importButton

```
QPushButton* WelcomeWidget::m_importButton = nullptr [private]
```

Definition at line 46 of file [WelcomeWidget.h](#).

7.99.4.3 m_openDbButton

QPushButton* WelcomeWidget::m_openDbButton = nullptr [private]
 Definition at line 47 of file [WelcomeWidget.h](#).

7.99.4.4 m_subtitleLabel

QLabel* WelcomeWidget::m_subtitleLabel = nullptr [private]
 Definition at line 45 of file [WelcomeWidget.h](#).

The documentation for this class was generated from the following files:

- src/gui/WelcomeWidget.h
- src/gui/WelcomeWidget.cpp

7.100 WorkoutAnalyzer Class Reference

Analyzes ride data to compute workout statistics.

#include <WorkoutAnalyzer.h>

Public Member Functions

- [WorkoutStatistics analyze](#) (const [RideData](#) &rideData)
Analyzes activity data to compute statistics.

7.100.1 Detailed Description

Analyzes ride data to compute workout statistics.

Definition at line 53 of file [WorkoutAnalyzer.h](#).

7.100.2 Member Function Documentation

7.100.2.1 analyze()

```
WorkoutStatistics WorkoutAnalyzer::analyze (
    const RideData & rideData)
```

Analyzes activity data to compute statistics.

Parameters

<i>rideData</i>	Activity recording data.
-----------------	--

Returns

Computed statistics.

Definition at line 19 of file [WorkoutAnalyzer.cpp](#).

References [WorkoutStatistics::averageCadence](#), [WorkoutStatistics::averageHeartRate](#), [WorkoutStatistics::averagePower](#), [WorkoutStatistics::averageSpeed](#), [WorkoutStatistics::durationSeconds](#), [WorkoutStatistics::elevationGainM](#), [WorkoutStatistics::maximumCadence](#), [WorkoutStatistics::maximumHeartRate](#), [WorkoutStatistics::maximumPower](#), [WorkoutStatistics::maximumSpeed](#), [WorkoutStatistics::movingTimeSeconds](#), [RideData::records](#), and [WorkoutStatistics::totalDistance](#).
 Referenced by [ImportWorker::processJob\(\)](#).

Here is the caller graph for this function:



The documentation for this class was generated from the following files:

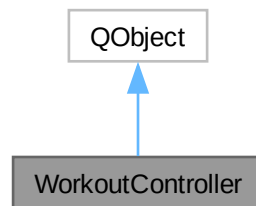
- `src/analysis/WorkoutAnalyzer.h`
- `src/analysis/WorkoutAnalyzer.cpp`

7.101 WorkoutController Class Reference

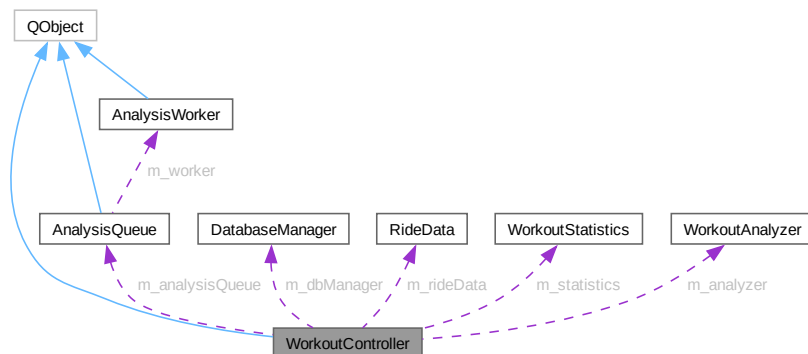
Manages activity loading, import, and analysis.

```
#include <WorkoutController.h>
```

Inheritance diagram for `WorkoutController`:



Collaboration diagram for `WorkoutController`:



Signals

- void **workoutLoaded** ()
- void **activityImported** (int activityId)
- void **ftpChanged** (double ftp)

Public Member Functions

- [WorkoutController](#) (QObject *parent=nullptr)
- void [setFTP](#) (double ftp)
- double [ftp](#) () const
- void [setDatabaseManager](#) ([DatabaseManager](#) *dbManager)
- void [setCurrentAthlete](#) (int athleteId)
- int [currentAthleteId](#) () const
- bool [loadFile](#) (const QString &path, QString &errorOut)
- int [importFile](#) (const QString &path, QString &errorOut, bool allowTimeOverlap=false, bool run↵ Analysis=true, const QString &importSource=QString(), [DuplicateActivityInfo](#) *duplicateInfo=nullptr, Import↵ Result *result=nullptr)
- bool [loadActivity](#) (int activityId, QString &errorOut)
- void [reloadClimbsFromDatabase](#) ()
- void [setAnalysisQueue](#) ([AnalysisQueue](#) *queue)
- const [RideData](#) & [rideData](#) () const
- const [WorkoutStatistics](#) & [statistics](#) () const
- const std::vector< [ZoneDistribution](#) > & [powerZones](#) () const
- const std::vector< [PowerCurvePoint](#) > & [pdcPoints](#) () const
- const std::vector< [Interval](#) > & [intervals](#) () const
- const std::vector< [Climb](#) > & [climbs](#) () const
- double [normalizedPower](#) () const
- double [intensityFactor](#) () const
- double [trainingStressScore](#) () const
- double [variabilityIndex](#) () const
- double [efficiencyFactor](#) () const
- int [currentActivityId](#) () const

Private Member Functions

- void [reanalyze](#) ()
- void [loadOrGenerateIntervals](#) ()
- void [loadOrGenerateClimbs](#) ()

Private Attributes

- [AnalysisQueue](#) * [m_analysisQueue](#) = nullptr
- double [m_ftp](#) = 250.0
- int [m_currentAthleteId](#) = -1
- int [m_currentActivityId](#) = -1
- [DatabaseManager](#) * [m_dbManager](#) = nullptr
- [RideData](#) [m_rideData](#)
- [WorkoutStatistics](#) [m_statistics](#)
- [WorkoutAnalyzer](#) [m_analyzer](#)
- std::vector< [ZoneDistribution](#) > [m_powerZones](#)
- std::vector< [PowerCurvePoint](#) > [m_pdcPoints](#)
- std::vector< [Interval](#) > [m_intervals](#)
- std::vector< [Climb](#) > [m_climbs](#)
- double [m_np](#) = 0.0
- double [m_if](#) = 0.0
- double [m_tss](#) = 0.0
- double [m_vi](#) = 0.0
- double [m_ef](#) = 0.0

7.101.1 Detailed Description

Manages activity loading, import, and analysis.

Central controller for ride data loading, importing FIT files, and coordinating analysis operations including climb detection, interval detection, and metrics computation.

Definition at line 61 of file [WorkoutController.h](#).

7.101.2 Constructor & Destructor Documentation

7.101.2.1 WorkoutController()

```
WorkoutController::WorkoutController (
    QObject * parent = nullptr) [explicit]
```

Definition at line 53 of file [WorkoutController.cpp](#).

7.101.3 Member Function Documentation

7.101.3.1 climbs()

```
const std::vector< Climb > & WorkoutController::climbs () const [inline]
```

Definition at line 102 of file [WorkoutController.h](#).

7.101.3.2 currentActivityId()

```
int WorkoutController::currentActivityId () const [inline]
```

Definition at line 114 of file [WorkoutController.h](#).

7.101.3.3 currentAthleteId()

```
int WorkoutController::currentAthleteId () const [inline]
```

Definition at line 75 of file [WorkoutController.h](#).

7.101.3.4 efficiencyFactor()

```
double WorkoutController::efficiencyFactor () const [inline]
```

Definition at line 112 of file [WorkoutController.h](#).

7.101.3.5 ftp()

```
double WorkoutController::ftp () const [inline]
```

Definition at line 69 of file [WorkoutController.h](#).

7.101.3.6 importFile()

```
int WorkoutController::importFile (
    const QString & path,
    QString & errorOut,
    bool allowTimeOverlap = false,
    bool runAnalysis = true,
    const QString & importSource = QString(),
    DuplicateActivityInfo * duplicateInfo = nullptr,
    ImportResult * result = nullptr)
```

Definition at line 109 of file [WorkoutController.cpp](#).

7.101.3.7 intensityFactor()

```
double WorkoutController::intensityFactor () const [inline]
```

Definition at line 106 of file [WorkoutController.h](#).

7.101.3.8 intervals()

```
const std::vector< Interval > & WorkoutController::intervals () const [inline]
```

Definition at line 100 of file [WorkoutController.h](#).

7.101.3.9 loadActivity()

```
bool WorkoutController::loadActivity (
    int activityId,
    QString & errorOut)
```

Definition at line 224 of file [WorkoutController.cpp](#).

7.101.3.10 loadFile()

```
bool WorkoutController::loadFile (
    const QString & path,
    QString & errorOut)
```

Definition at line 91 of file [WorkoutController.cpp](#).

7.101.3.11 loadOrGenerateClimbs()

```
void WorkoutController::loadOrGenerateClimbs () [private]
```

Definition at line 364 of file [WorkoutController.cpp](#).

7.101.3.12 loadOrGenerateIntervals()

```
void WorkoutController::loadOrGenerateIntervals () [private]
```

Definition at line 292 of file [WorkoutController.cpp](#).

7.101.3.13 normalizedPower()

```
double WorkoutController::normalizedPower () const [inline]
```

Definition at line 104 of file [WorkoutController.h](#).

7.101.3.14 pdcPoints()

```
const std::vector< PowerCurvePoint > & WorkoutController::pdcPoints () const [inline]
```

Definition at line 98 of file [WorkoutController.h](#).

7.101.3.15 powerZones()

```
const std::vector< ZoneDistribution > & WorkoutController::powerZones () const [inline]
```

Definition at line 96 of file [WorkoutController.h](#).

7.101.3.16 reanalyze()

```
void WorkoutController::reanalyze () [private]
```

Definition at line 262 of file [WorkoutController.cpp](#).

7.101.3.17 reloadClimbsFromDatabase()

```
void WorkoutController::reloadClimbsFromDatabase ()
```

Definition at line 355 of file [WorkoutController.cpp](#).

7.101.3.18 rideData()

```
const RideData & WorkoutController::rideData () const [inline]
```

Definition at line 92 of file [WorkoutController.h](#).

7.101.3.19 setAnalysisQueue()

```
void WorkoutController::setAnalysisQueue (  
    AnalysisQueue * queue) [inline]
```

Definition at line 90 of file [WorkoutController.h](#).

7.101.3.20 setCurrentAthlete()

```
void WorkoutController::setCurrentAthlete (  
    int athleteId)
```

Definition at line 70 of file [WorkoutController.cpp](#).

7.101.3.21 setDatabaseManager()

```
void WorkoutController::setDatabaseManager (  
    DatabaseManager * dbManager)
```

Definition at line 65 of file [WorkoutController.cpp](#).

7.101.3.22 setFTP()

```
void WorkoutController::setFTP (  
    double ftp)
```

Definition at line 57 of file [WorkoutController.cpp](#).

7.101.3.23 statistics()

```
const WorkoutStatistics & WorkoutController::statistics () const [inline]
```

Definition at line 94 of file [WorkoutController.h](#).

7.101.3.24 trainingStressScore()

```
double WorkoutController::trainingStressScore () const [inline]
```

Definition at line 108 of file [WorkoutController.h](#).

7.101.3.25 variabilityIndex()

```
double WorkoutController::variabilityIndex () const [inline]
```

Definition at line 110 of file [WorkoutController.h](#).

7.101.4 Member Data Documentation

7.101.4.1 m_analysisQueue

```
AnalysisQueue* WorkoutController::m_analysisQueue = nullptr [private]
```

Definition at line 130 of file [WorkoutController.h](#).

7.101.4.2 m_analyzer

```
WorkoutAnalyzer WorkoutController::m_analyzer [private]
```

Definition at line 140 of file [WorkoutController.h](#).

7.101.4.3 m_climbs

```
std::vector<Climb> WorkoutController::m_climbs [private]
```

Definition at line 144 of file [WorkoutController.h](#).

7.101.4.4 m_currentActivityId

```
int WorkoutController::m_currentActivityId = -1 [private]
```

Definition at line 134 of file [WorkoutController.h](#).

7.101.4.5 m_currentAthleteId

```
int WorkoutController::m_currentAthleteId = -1 [private]
```

Definition at line 133 of file [WorkoutController.h](#).

7.101.4.6 m_dbManager

```
DatabaseManager* WorkoutController::m_dbManager = nullptr [private]
```

Definition at line 136 of file [WorkoutController.h](#).

7.101.4.7 m_ef

```
double WorkoutController::m_ef = 0.0 [private]
```

Definition at line 150 of file [WorkoutController.h](#).

7.101.4.8 m_ftp

```
double WorkoutController::m_ftp = 250.0 [private]
```

Definition at line 132 of file [WorkoutController.h](#).

7.101.4.9 m_if

```
double WorkoutController::m_if = 0.0 [private]
```

Definition at line 147 of file [WorkoutController.h](#).

7.101.4.10 m_intervals

```
std::vector<Interval> WorkoutController::m_intervals [private]
```

Definition at line 143 of file [WorkoutController.h](#).

7.101.4.11 m_np

```
double WorkoutController::m_np = 0.0 [private]
```

Definition at line 146 of file [WorkoutController.h](#).

7.101.4.12 m_pdcPoints

```
std::vector<PowerCurvePoint> WorkoutController::m_pdcPoints [private]
```

Definition at line 142 of file [WorkoutController.h](#).

7.101.4.13 m_powerZones

```
std::vector<ZoneDistribution> WorkoutController::m_powerZones [private]
```

Definition at line 141 of file [WorkoutController.h](#).

7.101.4.14 m_rideData

```
RideData WorkoutController::m_rideData [private]
```

Definition at line 138 of file [WorkoutController.h](#).

7.101.4.15 m_statistics

```
WorkoutStatistics WorkoutController::m_statistics [private]
```

Definition at line 139 of file [WorkoutController.h](#).

7.101.4.16 m_tss

```
double WorkoutController::m_tss = 0.0 [private]
```

Definition at line 148 of file [WorkoutController.h](#).

7.101.4.17 m_vi

```
double WorkoutController::m_vi = 0.0 [private]
```

Definition at line 149 of file [WorkoutController.h](#).

The documentation for this class was generated from the following files:

- [src/gui/WorkoutController.h](#)
- [src/gui/WorkoutController.cpp](#)

7.102 WorkoutStatistics Struct Reference

Statistics summary for a workout activity.

```
#include <WorkoutAnalyzer.h>
```

Public Attributes

- double [durationSeconds](#) = 0.0
Total elapsed time in seconds.
- double [movingTimeSeconds](#) = 0.0
Moving time in seconds (speed > 2 km/h).
- double [totalDistanceKm](#) = 0.0
Total distance in kilometers.
- double [elevationGainM](#) = 0.0
Total elevation gain in meters.
- double [averagePower](#) = 0.0
Average power in watts.
- double [maximumPower](#) = 0.0
Maximum power in watts.
- double [averageHeartRate](#) = 0.0
Average heart rate in bpm.
- double [maximumHeartRate](#) = 0.0
Maximum heart rate in bpm.
- double [averageCadence](#) = 0.0
Average cadence in rpm.
- double [maximumCadence](#) = 0.0
Maximum cadence in rpm.
- double [averageSpeed](#) = 0.0
Average speed in km/h.
- double [maximumSpeed](#) = 0.0
Maximum speed in km/h.

7.102.1 Detailed Description

Statistics summary for a workout activity.

Definition at line 11 of file [WorkoutAnalyzer.h](#).

7.102.2 Member Data Documentation

7.102.2.1 averageCadence

```
double WorkoutStatistics::averageCadence = 0.0
```

Average cadence in rpm.

Definition at line 38 of file [WorkoutAnalyzer.h](#).

Referenced by [WorkoutAnalyzer::analyze\(\)](#), and [ImportWorker::processJob\(\)](#).

7.102.2.2 averageHeartRate

```
double WorkoutStatistics::averageHeartRate = 0.0
```

Average heart rate in bpm.

Definition at line 32 of file [WorkoutAnalyzer.h](#).

Referenced by [WorkoutAnalyzer::analyze\(\)](#), and [ImportWorker::processJob\(\)](#).

7.102.2.3 averagePower

```
double WorkoutStatistics::averagePower = 0.0
```

Average power in watts.

Definition at line 26 of file [WorkoutAnalyzer.h](#).

Referenced by [WorkoutAnalyzer::analyze\(\)](#), and [ImportWorker::processJob\(\)](#).

7.102.2.4 averageSpeed

```
double WorkoutStatistics::averageSpeed = 0.0
```

Average speed in km/h.

Definition at line 44 of file [WorkoutAnalyzer.h](#).

Referenced by [WorkoutAnalyzer::analyze\(\)](#), and [ImportWorker::processJob\(\)](#).

7.102.2.5 durationSeconds

```
double WorkoutStatistics::durationSeconds = 0.0
```

Total elapsed time in seconds.

Definition at line 14 of file [WorkoutAnalyzer.h](#).

Referenced by [WorkoutAnalyzer::analyze\(\)](#), and [ImportWorker::processJob\(\)](#).

7.102.2.6 elevationGainM

```
double WorkoutStatistics::elevationGainM = 0.0
```

Total elevation gain in meters.

Definition at line 23 of file [WorkoutAnalyzer.h](#).

Referenced by [WorkoutAnalyzer::analyze\(\)](#), and [ImportWorker::processJob\(\)](#).

7.102.2.7 maximumCadence

```
double WorkoutStatistics::maximumCadence = 0.0
```

Maximum cadence in rpm.

Definition at line 41 of file [WorkoutAnalyzer.h](#).

Referenced by [WorkoutAnalyzer::analyze\(\)](#).

7.102.2.8 maximumHeartRate

```
double WorkoutStatistics::maximumHeartRate = 0.0
```

Maximum heart rate in bpm.

Definition at line 35 of file [WorkoutAnalyzer.h](#).

Referenced by [WorkoutAnalyzer::analyze\(\)](#), and [ImportWorker::processJob\(\)](#).

7.102.2.9 maximumPower

```
double WorkoutStatistics::maximumPower = 0.0
```

Maximum power in watts.

Definition at line 29 of file [WorkoutAnalyzer.h](#).

Referenced by [WorkoutAnalyzer::analyze\(\)](#), and [ImportWorker::processJob\(\)](#).

7.102.2.10 maximumSpeed

```
double WorkoutStatistics::maximumSpeed = 0.0
```

Maximum speed in km/h.

Definition at line 47 of file [WorkoutAnalyzer.h](#).
 Referenced by [WorkoutAnalyzer::analyze\(\)](#).

7.102.2.11 movingTimeSeconds

`double WorkoutStatistics::movingTimeSeconds = 0.0`
 Moving time in seconds (speed > 2 km/h).
 Definition at line 17 of file [WorkoutAnalyzer.h](#).
 Referenced by [WorkoutAnalyzer::analyze\(\)](#).

7.102.2.12 totalDistanceKm

`double WorkoutStatistics::totalDistanceKm = 0.0`
 Total distance in kilometers.
 Definition at line 20 of file [WorkoutAnalyzer.h](#).
 Referenced by [WorkoutAnalyzer::analyze\(\)](#), and [ImportWorker::processJob\(\)](#).
 The documentation for this struct was generated from the following file:

- `src/analysis/WorkoutAnalyzer.h`

7.103 Zone Struct Reference

Represents a single training or power zone with bounds and color.
`#include <ZoneDefinition.h>`

Public Attributes

- `QString name`
Zone name.
- `double minValue = 0.0`
Minimum zone value (inclusive).
- `double maxValue = 0.0`
Maximum zone value (exclusive).
- `QColor color`
Zone color for display.

7.103.1 Detailed Description

Represents a single training or power zone with bounds and color.
 Definition at line 26 of file [ZoneDefinition.h](#).

7.103.2 Member Data Documentation

7.103.2.1 color

`QColor Zone::color`
Zone color for display.
 Definition at line 38 of file [ZoneDefinition.h](#).

7.103.2.2 maxValue

`double Zone::maxValue = 0.0`
Maximum zone value (exclusive).
 Definition at line 35 of file [ZoneDefinition.h](#).
 Referenced by [RideChartWidget::mouseMoveEvent\(\)](#).

7.103.2.3 minValue

```
double Zone::minValue = 0.0
```

Minimum zone value (inclusive).

Definition at line 32 of file [ZoneDefinition.h](#).

Referenced by [RideChartWidget::mouseMoveEvent\(\)](#).

7.103.2.4 name

```
QString Zone::name
```

[Zone](#) name.

Definition at line 29 of file [ZoneDefinition.h](#).

Referenced by [RideChartWidget::mouseMoveEvent\(\)](#).

The documentation for this struct was generated from the following file:

- [src/core/zones/ZoneDefinition.h](#)

7.104 ZoneCalculator Class Reference

Calculates training zones for various metrics.

```
#include <ZoneCalculator.h>
```

Static Public Member Functions

- static `std::vector< Zone > powerZones` (int ftp)
Computes power zones based on FTP.
- static `std::vector< Zone > heartRateZones` (int heartRateMax, int heartRateRest=60, int thresholdHeartRate=0)
Computes heart rate zones.
- static `std::vector< Zone > cadenceZones` ()
Computes cadence zones.
- static `std::vector< Zone > speedZones` ()
Computes speed zones.
- static `std::vector< Zone > altitudeZones` (double minAltitude, double maxAltitude)
Computes altitude zones.
- static `std::vector< Zone > zonesForMetric` (ColorMetric metric, const [ColorContext](#) &context)
Gets zones for a specific metric.
- static `std::vector< ZoneDistribution > computeDistribution` (const [RideData](#) &rideData, ColorMetric metric, const [ColorContext](#) &context)
Computes time/distance distribution across zones.
- static bool `hasMetricSamples` (const [RideData](#) &rideData, ColorMetric metric)
Checks if activity has samples for a metric.

7.104.1 Detailed Description

Calculates training zones for various metrics.

Provides zone definitions and distributions for power, heart rate, cadence, speed, and altitude.

Definition at line 16 of file [ZoneCalculator.h](#).

7.104.2 Member Function Documentation

7.104.2.1 altitudeZones()

```
std::vector< Zone > ZoneCalculator::altitudeZones (
    double minAltitude,
    double maxAltitude) [static]
```

Computes altitude zones.

Parameters

<i>minAltitude</i>	Minimum altitude in range.
<i>maxAltitude</i>	Maximum altitude in range.

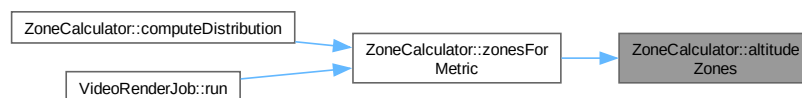
Returns

List of altitude zones.

Definition at line 112 of file [ZoneCalculator.cpp](#).

Referenced by [zonesForMetric\(\)](#).

Here is the caller graph for this function:

**7.104.2.2 cadenceZones()**

```
std::vector< Zone > ZoneCalculator::cadenceZones () [static]
```

Computes cadence zones.

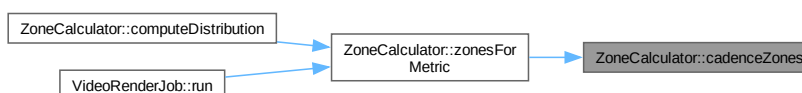
Returns

List of cadence zones.

Definition at line 90 of file [ZoneCalculator.cpp](#).

Referenced by [zonesForMetric\(\)](#).

Here is the caller graph for this function:

**7.104.2.3 computeDistribution()**

```
std::vector< ZoneDistribution > ZoneCalculator::computeDistribution (
    const RideData & rideData,
    ColorMetric metric,
    const ColorContext & context) [static]
```

Computes time/distance distribution across zones.

Parameters

<i>rideData</i>	Activity data.
<i>metric</i>	Metric type.

<i>context</i>	Athlete context.
----------------	----------------------------------

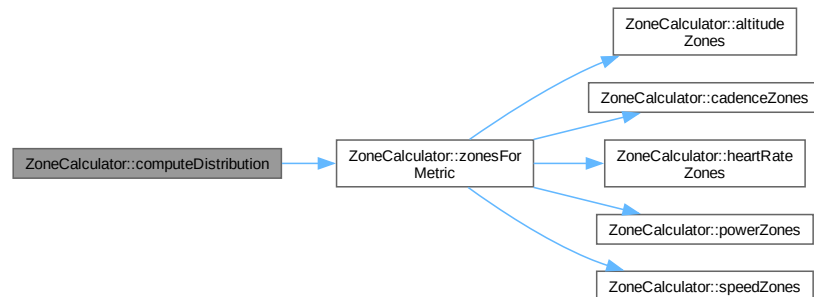
Returns

Distribution of time spent in each zone.

Definition at line 152 of file [ZoneCalculator.cpp](#).

References [RideData::records](#), and [zonesForMetric\(\)](#).

Here is the call graph for this function:

**7.104.2.4 hasMetricSamples()**

```
bool ZoneCalculator::hasMetricSamples (
    const RideData & rideData,
    ColorMetric metric) [static]
```

Checks if activity has samples for a metric.

Parameters

<i>rideData</i>	Activity data.
<i>metric</i>	Metric type.

Returns

True if samples exist.

Definition at line 199 of file [ZoneCalculator.cpp](#).

References [RideData::records](#).

7.104.2.5 heartRateZones()

```
std::vector< Zone > ZoneCalculator::heartRateZones (
    int heartRateMax,
    int heartRateRest = 60,
    int thresholdHeartRate = 0) [static]
```

Computes heart rate zones.

Parameters

<i>heartRateMax</i>	Maximum heart rate.
---------------------	---------------------

<i>heartRateRest</i>	Resting heart rate (default 60).
<i>thresholdHeartRate</i>	Threshold HR (default 0).

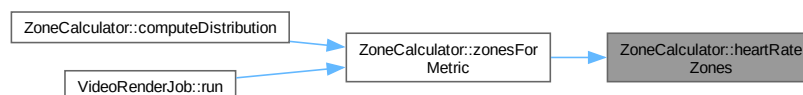
Returns

List of heart rate zones.

Definition at line 76 of file [ZoneCalculator.cpp](#).

Referenced by [zonesForMetric\(\)](#).

Here is the caller graph for this function:



7.104.2.6 powerZones()

```
std::vector< Zone > ZoneCalculator::powerZones (
    int ftp) [static]
```

Computes power zones based on FTP.

Parameters

<i>ftp</i>	Functional threshold power in watts.
------------	--------------------------------------

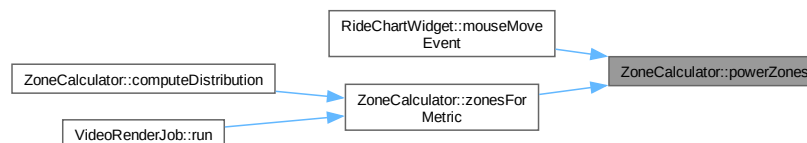
Returns

List of power zones.

Definition at line 62 of file [ZoneCalculator.cpp](#).

Referenced by [RideChartWidget::mouseMoveEvent\(\)](#), and [zonesForMetric\(\)](#).

Here is the caller graph for this function:



7.104.2.7 speedZones()

```
std::vector< Zone > ZoneCalculator::speedZones () [static]
```

Computes speed zones.

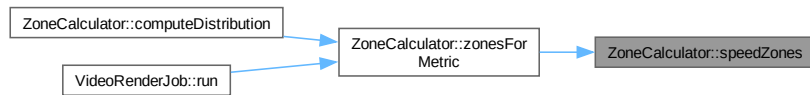
Returns

List of speed zones.

Definition at line 101 of file [ZoneCalculator.cpp](#).

Referenced by [zonesForMetric\(\)](#).

Here is the caller graph for this function:

**7.104.2.8 zonesForMetric()**

```
std::vector< Zone > ZoneCalculator::zonesForMetric (
    ColorMetric metric,
    const ColorContext & context) [static]
```

Gets zones for a specific metric.

Parameters

<i>metric</i>	Metric type.
<i>context</i>	Athlete context (FTP, HR max, etc.).

Returns

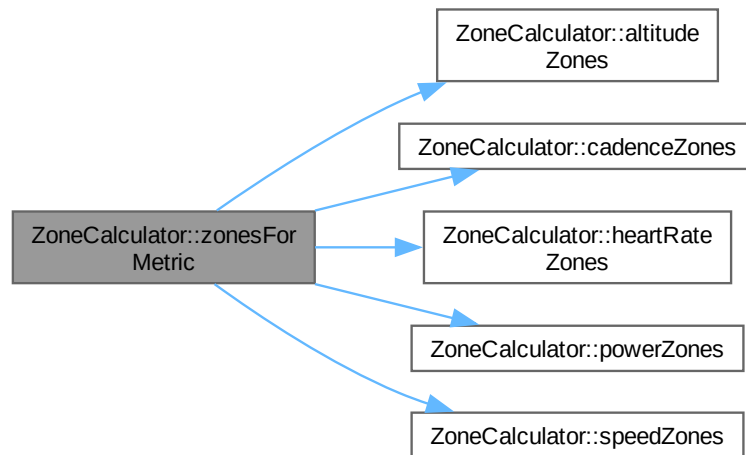
[Zone](#) definitions for the metric.

Definition at line 127 of file [ZoneCalculator.cpp](#).

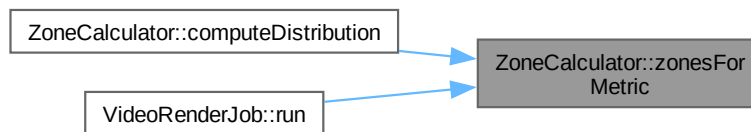
References [ColorContext::altitudeMax](#), [ColorContext::altitudeMin](#), [altitudeZones\(\)](#), [cadenceZones\(\)](#), [ColorContext::ftp](#), [ColorContext::heartRateMax](#), [ColorContext::heartRateRest](#), [heartRateZones\(\)](#), [powerZones\(\)](#), [speedZones\(\)](#), and [ColorContext::thresholdHeartRate](#).

Referenced by [computeDistribution\(\)](#), and [VideoRenderJob::run\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



The documentation for this class was generated from the following files:

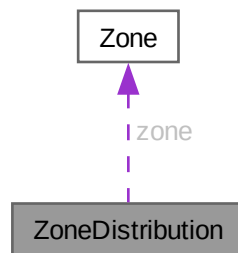
- `src/core/zones/ZoneCalculator.h`
- `src/core/zones/ZoneCalculator.cpp`

7.105 ZoneDistribution Struct Reference

Distribution of time or distance spent in a specific zone.

```
#include <ZoneDefinition.h>
```


Collaboration diagram for ZoneDistribution:



Public Attributes

- [Zone zone](#)
Zone definition.
- double [seconds](#) = 0.0
Time spent in zone (seconds).
- double [percent](#) = 0.0
Percentage of total time in zone.

7.105.1 Detailed Description

Distribution of time or distance spent in a specific zone.
Definition at line 44 of file [ZoneDefinition.h](#).

7.105.2 Member Data Documentation

7.105.2.1 percent

```
double ZoneDistribution::percent = 0.0
```

Percentage of total time in zone.
Definition at line 53 of file [ZoneDefinition.h](#).

7.105.2.2 seconds

```
double ZoneDistribution::seconds = 0.0
```

Time spent in zone (seconds).
Definition at line 50 of file [ZoneDefinition.h](#).

7.105.2.3 zone

```
Zone ZoneDistribution::zone
```

[Zone](#) definition.

Definition at line 47 of file [ZoneDefinition.h](#).

The documentation for this struct was generated from the following file:

- `src/core/zones/ZoneDefinition.h`

Chapter 8

File Documentation

8.1 ClimbDetector.cpp

```
00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "ClimbDetector.h"
00005
00006 #include <algorithm>
00007 #include <cmath>
00008 #include <limits>
00009 #include <numbers>
00010
00011 namespace
00012 {
00013 bool isFinite(double value)
00014 {
00015     return std::isfinite(value);
00016 }
00017
00018 double clampValue(double value, double lo, double hi)
00019 {
00020     return std::max(lo, std::min(hi, value));
00021 }
00022
00023 double segmentMeters(double lat1, double lon1, double lat2, double lon2)
00024 {
00025     const double dLat = (lat2 - lat1) * std::numbers::pi_v<double> / 180.0;
00026     const double dLon = (lon2 - lon1) * std::numbers::pi_v<double> / 180.0;
00027     const double mLat = (lat1 + lat2) / 2.0 * std::numbers::pi_v<double> / 180.0;
00028     return std::hypot(dLat, dLon * std::cos(mLat)) * 6371000.0;
00029 }
00030
00031 std::vector<double> buildFilteredAltitudeMeters(
00032     const RideData& rideData,
00033     const std::vector<double>& cumulativeDistanceMeters,
00034     double maxVerticalSpeedMps)
00035 {
00036     const size_t n = rideData.records.size();
00037     std::vector<double> filtered(n, std::numeric_limits<double>::quiet_NaN());
00038     for (size_t i = 0; i < n; ++i)
00039     {
00040         if (rideData.records[i].hasAltitude)
00041             filtered[i] = rideData.records[i].altitude;
00042     }
00043
00044     if (n < 3 || maxVerticalSpeedMps <= 0.0)
00045         return filtered;
00046
00047     for (size_t i = 1; i + 1 < n; ++i)
00048     {
00049         if (!isFinite(filtered[i - 1]) || !isFinite(filtered[i]) || !isFinite(filtered[i + 1]))
00050             continue;
00051
00052         const RideRecord& prev = rideData.records[i - 1];
00053         const RideRecord& now = rideData.records[i];
00054         const RideRecord& next = rideData.records[i + 1];
00055
00056         const double dtPrev = now.elapsedSeconds - prev.elapsedSeconds;
00057         const double dtNext = next.elapsedSeconds - now.elapsedSeconds;
00058         if (dtPrev <= 0.0 || dtNext <= 0.0)
00059             continue;
00060
00061         const double vsPrev = (filtered[i] - filtered[i - 1]) / dtPrev;
```

```

00062         const double vsNext = (filtered[i + 1] - filtered[i]) / dtNext;
00063         const bool oppositeDirections = (vsPrev > 0.0 && vsNext < 0.0) || (vsPrev < 0.0 && vsNext >
0.0);
00064         if (!oppositeDirections)
00065             continue;
00066
00067         if (std::abs(vsPrev) <= maxVerticalSpeedMps || std::abs(vsNext) <= maxVerticalSpeedMps)
00068             continue;
00069
00070         const double spanDist = cumulativeDistanceMeters[i + 1] - cumulativeDistanceMeters[i - 1];
00071         const double localDist = cumulativeDistanceMeters[i] - cumulativeDistanceMeters[i - 1];
00072         const double ratio = spanDist > 0.0 ? clampValue(localDist / spanDist, 0.0, 1.0) : 0.5;
00073         filtered[i] = filtered[i - 1] + (filtered[i + 1] - filtered[i - 1]) * ratio;
00074     }
00075
00076     return filtered;
00077 }
00078
00079 double estimateSpeedKmhAt (
00080     const RideData& rideData,
00081     const std::vector<double>& cumulativeDistanceMeters,
00082     int index)
00083 {
00084     const int n = static_cast<int>(rideData.records.size());
00085     if (index < 0 || index >= n)
00086         return std::numeric_limits<double>::quiet_NaN();
00087
00088     const RideRecord& r = rideData.records[static_cast<size_t>(index)];
00089     if (r.hasSpeed && isFinite(r.speed) && r.speed >= 0.0)
00090         return r.speed;
00091
00092     const int prev = std::max(0, index - 1);
00093     const int next = std::min(n - 1, index + 1);
00094     if (next == prev)
00095         return std::numeric_limits<double>::quiet_NaN();
00096
00097     const double dt = rideData.records[static_cast<size_t>(next)].elapsedSeconds -
rideData.records[static_cast<size_t>(prev)].elapsedSeconds;
00098     const double dd = cumulativeDistanceMeters[static_cast<size_t>(next)] -
cumulativeDistanceMeters[static_cast<size_t>(prev)];
00099     if (dt <= 0.0 || dd < 0.0)
00100         return std::numeric_limits<double>::quiet_NaN();
00101
00102     return (dd / dt) * 3.6;
00103 }
00104 } // namespace
00105
00106 std::vector<double> ClimbDetector::buildCumulativeDistanceMeters(const RideData& rideData)
00107 {
00108     const size_t n = rideData.records.size();
00109     std::vector<double> cumulative(n, 0.0);
00110     if (n == 0)
00111         return cumulative;
00112
00113     bool prevHasGps = false;
00114     double prevLat = 0.0;
00115     double prevLon = 0.0;
00116
00117     for (size_t i = 1; i < n; ++i)
00118     {
00119         const RideRecord& prev = rideData.records[i - 1];
00120         const RideRecord& now = rideData.records[i];
00121
00122         double segment = 0.0;
00123         if (now.hasGps)
00124         {
00125             if (prevHasGps)
00126                 segment = segmentMeters(prevLat, prevLon, now.latitude, now.longitude);
00127             prevHasGps = true;
00128             prevLat = now.latitude;
00129             prevLon = now.longitude;
00130         }
00131         else if (now.hasSpeed)
00132         {
00133             const double dt = std::max(0.0, now.elapsedSeconds - prev.elapsedSeconds);
00134             const double prevSpeed = prev.hasSpeed ? prev.speed : now.speed;
00135             const double avgSpeedKmh = (now.speed + prevSpeed) / 2.0;
00136             segment = avgSpeedKmh * (dt / 3600.0) * 1000.0;
00137         }
00138         cumulative[i] = cumulative[i - 1] + std::max(0.0, segment);
00139     }
00140     return cumulative;
00141 }
00142
00143 std::vector<double> ClimbDetector::smoothAltitudeByDistance(

```

```

00148     const RideData& rideData,
00149     const std::vector<double>& cumulativeDistanceMeters,
00150     double windowMeters,
00151     const std::vector<double>& adaptiveWindowMeters,
00152     double altitudeSpikeVerticalSpeedMps)
00153 {
00154     const size_t n = rideData.records.size();
00155     std::vector<double> smoothed(n, std::numeric_limits<double>::quiet_NaN());
00156     if (n == 0)
00157         return smoothed;
00158
00159     const std::vector<double> filteredAltitudeMeters = buildFilteredAltitudeMeters(
00160         rideData,
00161         cumulativeDistanceMeters,
00162         altitudeSpikeVerticalSpeedMps);
00163
00164     if (windowMeters <= 0.0)
00165     {
00166         for (size_t i = 0; i < n; ++i)
00167         {
00168             if (isFinite(filteredAltitudeMeters[i]))
00169                 smoothed[i] = filteredAltitudeMeters[i];
00170         }
00171         return smoothed;
00172     }
00173
00174     size_t left = 0;
00175     size_t right = 0;
00176
00177     for (size_t i = 0; i < n; ++i)
00178     {
00179         const double sampleWindow =
00180             (adaptiveWindowMeters.size() == n && isFinite(adaptiveWindowMeters[i]) &&
00181              adaptiveWindowMeters[i] > 0.0)
00182             ? adaptiveWindowMeters[i]
00183             : windowMeters;
00184         const double halfWindow = std::max(1.0, sampleWindow / 2.0);
00185         const double center = cumulativeDistanceMeters[i];
00186         while (left < n && cumulativeDistanceMeters[left] < center - halfWindow)
00187             ++left;
00188         while (right < n && cumulativeDistanceMeters[right] <= center + halfWindow)
00189             ++right;
00190
00191         double sumAlt = 0.0;
00192         int countAlt = 0;
00193         for (size_t j = left; j < right; ++j)
00194         {
00195             if (!isFinite(filteredAltitudeMeters[j]))
00196                 continue;
00197             sumAlt += filteredAltitudeMeters[j];
00198             ++countAlt;
00199         }
00200         if (countAlt > 0)
00201             smoothed[i] = sumAlt / static_cast<double>(countAlt);
00202     }
00203
00204     return smoothed;
00205 }
00206
00207 std::vector<double> ClimbDetector::computeAdaptiveSmoothingWindowsMeters(
00208     const RideData& rideData,
00209     const std::vector<double>& cumulativeDistanceMeters,
00210     const Config& config)
00211 {
00212     const size_t n = rideData.records.size();
00213     std::vector<double> windows(n, std::max(1.0, config.elevationSmoothingMeters));
00214     if (n == 0)
00215         return windows;
00216
00217     if (!config.useAdaptiveSmoothing)
00218         return windows;
00219
00220     const double minWin = std::max(1.0, std::min(config.adaptiveSmoothingMinMeters,
00221         config.adaptiveSmoothingMaxMeters));
00222     const double maxWin = std::max(minWin, std::max(config.adaptiveSmoothingMinMeters,
00223         config.adaptiveSmoothingMaxMeters));
00224
00225     for (int i = 0; i < static_cast<int>(n); ++i)
00226     {
00227         const double speedKmh = estimateSpeedKmhAt(rideData, cumulativeDistanceMeters, i);
00228         if (!isFinite(speedKmh))
00229             continue;
00230         windows[static_cast<size_t>(i)] = clampValue(config.elevationSmoothingMeters, minWin,

```

```

00231
00232     const double targetWindow = 20.0 + speedKmh * 1.8;
00233     windows[static_cast<size_t>(i)] = clampValue(targetWindow, minWin, maxWin);
00234 }
00235
00236     return windows;
00237 }
00238
00239 std::vector<double> ClimbDetector::computeLocalGradientPct(
00240     const std::vector<double>& smoothedAltitudeMeters,
00241     const std::vector<double>& cumulativeDistanceMeters,
00242     double halfWindowMeters)
00243 {
00244     const size_t n = smoothedAltitudeMeters.size();
00245     std::vector<double> gradient(n, std::numeric_limits<double>::quiet_NaN());
00246     if (n == 0)
00247         return gradient;
00248
00249     if (halfWindowMeters <= 0.0)
00250         halfWindowMeters = 1.0;
00251
00252     size_t left = 0;
00253     size_t right = 0;
00254
00255     for (size_t i = 0; i < n; ++i)
00256     {
00257         const double center = cumulativeDistanceMeters[i];
00258         while (left + 1 < n && cumulativeDistanceMeters[left + 1] <= center - halfWindowMeters)
00259             ++left;
00260         while (right < n && cumulativeDistanceMeters[right] < center + halfWindowMeters)
00261             ++right;
00262         if (right >= n)
00263             right = n - 1;
00264
00265         if (!isFinite(smoothedAltitudeMeters[left]) || !isFinite(smoothedAltitudeMeters[right]))
00266             continue;
00267
00268         const double distDiff = cumulativeDistanceMeters[right] - cumulativeDistanceMeters[left];
00269         if (distDiff <= 0.0)
00270             continue;
00271
00272         const double altDiff = smoothedAltitudeMeters[right] - smoothedAltitudeMeters[left];
00273         gradient[i] = (altDiff / distDiff) * 100.0;
00274     }
00275
00276     return gradient;
00277 }
00278
00279 // Phase 2: Scoring-based detection
00280 double ClimbDetector::computeClimbScore(
00281     double gainMeters,
00282     double averageGrade,
00283     double distanceKm,
00284     double continuityRatio,
00285     double vam)
00286 {
00287     double score = gainMeters * 0.8 + averageGrade * 8.0 + distanceKm * 15.0 + continuityRatio * 20.0;
00288
00289     if (vam > 800.0)
00290         score += 10.0;
00291
00292     return score;
00293 }
00294
00295 // Phase 6: VAM computation
00296 double ClimbDetector::computeVAM(double elevationGainM, double durationSeconds)
00297 {
00298     if (durationSeconds <= 0.0)
00299         return 0.0;
00300
00301     const double durationHours = durationSeconds / 3600.0;
00302     if (durationHours <= 0.0)
00303         return 0.0;
00304
00305     return elevationGainM / durationHours;
00306 }
00307
00308 // Phase 1: Categorization
00309 ClimbCategory ClimbDetector::categorizeClimb(
00310     double gainMeters,
00311     double lengthKm,
00312     double averageGrade)
00313 {
00314     // Major: > 100m gain, > 2km length, > 3% avg grade
00315     if (gainMeters > 100.0 && lengthKm > 2.0 && averageGrade > 3.0)
00316         return ClimbCategory::Major;
00317

```

```

00318 // Medium: > 40m gain, > 500m length, > 3% avg grade
00319 if (gainMeters > 40.0 && lengthKm > 0.5 && averageGrade > 3.0)
00320     return ClimbCategory::Medium;
00321
00322 // Punchy: > 10m gain, > 100m length, > 5% avg grade (short steep)
00323 if (gainMeters > 10.0 && lengthKm > 0.1 && averageGrade > 5.0)
00324     return ClimbCategory::Punchy;
00325
00326 return ClimbCategory::FalsePositive;
00327 }
00328
00329 // Phase 7: Merge fragmented climbs
00330 std::vector<Climb> ClimbDetector::mergeFragmentedClimbs(
00331     std::vector<Climb>& candidates,
00332     const Config& config)
00333 {
00334     if (!config.enableClimbMerging || candidates.size() < 2)
00335         return candidates;
00336
00337     std::vector<Climb> merged;
00338     std::vector<bool> processed(candidates.size(), false);
00339
00340     for (size_t i = 0; i < candidates.size(); ++i)
00341     {
00342         if (processed[i])
00343             continue;
00344
00345         Climb combined = candidates[i];
00346         processed[i] = true;
00347
00348         // Try to merge nearby climbs
00349         size_t nextIdx = i + 1;
00350         while (nextIdx < candidates.size())
00351         {
00352             if (processed[nextIdx])
00353             {
00354                 ++nextIdx;
00355                 continue;
00356             }
00357
00358             const Climb& next = candidates[nextIdx];
00359
00360             const double gapMeters = next.startDistanceKm * 1000.0 - combined.endDistanceKm * 1000.0;
00361             const double elevationGapM = std::abs(next.startElevationM - combined.endElevationM);
00362             const bool overlaps = gapMeters <= 0.0;
00363
00364             if (overlaps ||
00365                 gapMeters <= config.mergeMaxGapMeters ||
00366                 elevationGapM <= config.mergeMaxGapElevationM)
00367             {
00368                 const double bridgingGainM = std::max(0.0, next.startElevationM -
00369 combined.endElevationM);
00370 combined.endSeconds = std::max(combined.endSeconds, next.endSeconds);
00371 combined.endDistanceKm = std::max(combined.endDistanceKm,
00372 next.endDistanceKm);
00373 combined.durationSeconds = combined.endSeconds - combined.startSeconds;
00374 combined.lengthKm = combined.endDistanceKm - combined.startDistanceKm;
00375 if (next.endDistanceKm >= combined.endDistanceKm)
00376     combined.endElevationM = next.endElevationM;
00377 combined.elevationGainM += overlaps ? next.elevationGainM : bridgingGainM
00378 + next.elevationGainM;
00379
00380 if (combined.lengthKm > 0.0)
00381     combined.averageGradient = (combined.elevationGainM / (combined.lengthKm *
00382 1000.0)) * 100.0;
00383
00384 processed[nextIdx] = true;
00385 ++nextIdx;
00386 }
00387 else
00388 {
00389     break;
00390 }
00391 }
00392 merged.push_back(combined);
00393 }
00394 return merged;
00395 }
00396 std::vector<Climb> ClimbDetector::detect(const RideData& rideData, const Config& config)
00397 {
00398     const int n = static_cast<int>(rideData.records.size());
00399     if (n < 3)
00400         return {};

```

```

00401
00402     bool hasAltitude = false;
00403     for (const RideRecord& r : rideData.records)
00404     {
00405         if (r.hasAltitude)
00406         {
00407             hasAltitude = true;
00408             break;
00409         }
00410     }
00411     if (!hasAltitude)
00412         return {};
00413
00414     const std::vector<double> cumulativeDistanceMeters = buildCumulativeDistanceMeters(rideData);
00415     const std::vector<double> adaptiveSmoothingWindows = computeAdaptiveSmoothingWindowsMeters(
00416         rideData,
00417         cumulativeDistanceMeters,
00418         config);
00419     const std::vector<double> smoothedAltitudeMeters = smoothAltitudeByDistance(
00420         rideData,
00421         cumulativeDistanceMeters,
00422         config.elevationSmoothingMeters,
00423         adaptiveSmoothingWindows,
00424         config.altitudeSpikeVerticalSpeedMps);
00425     const std::vector<double> gradient50mPct = computeLocalGradientPct(
00426         smoothedAltitudeMeters,
00427         cumulativeDistanceMeters,
00428         25.0);
00429     const std::vector<double> gradient100mPct = computeLocalGradientPct(
00430         smoothedAltitudeMeters,
00431         cumulativeDistanceMeters,
00432         50.0);
00433     const std::vector<double> gradient250mPct = computeLocalGradientPct(
00434         smoothedAltitudeMeters,
00435         cumulativeDistanceMeters,
00436         125.0);
00437
00438     auto isUphillSignal = [&](int idx)
00439     {
00440         const double g50 = gradient50mPct[static_cast<size_t>(idx)];
00441         const double g100 = gradient100mPct[static_cast<size_t>(idx)];
00442         const double g250 = gradient250mPct[static_cast<size_t>(idx)];
00443         return (isFinite(g100) && g100 >= config.startGradient) ||
00444             (isFinite(g250) && g250 >= config.startGradient) ||
00445             (isFinite(g50) && g50 >= config.startGradient + 1.5);
00446     };
00447
00448     auto isContinuationSignal = [&](int idx)
00449     {
00450         const double g50 = gradient50mPct[static_cast<size_t>(idx)];
00451         const double g100 = gradient100mPct[static_cast<size_t>(idx)];
00452         const double g250 = gradient250mPct[static_cast<size_t>(idx)];
00453         return (isFinite(g100) && g100 >= config.continueGradient) ||
00454             (isFinite(g250) && g250 >= config.startGradient) ||
00455             (isFinite(g50) && g50 >= config.startGradient + 2.0);
00456     };
00457
00458     std::vector<Climb> climbs;
00459     int i = 0;
00460
00461     while (i < n)
00462     {
00463         int triggerIdx = -1;
00464         for (; i < n; ++i)
00465         {
00466             if (isUphillSignal(i))
00467             {
00468                 triggerIdx = i;
00469                 break;
00470             }
00471         }
00472
00473         if (triggerIdx < 0)
00474             break;
00475
00476         int localMinIdx = triggerIdx;
00477         double localMinAlt = smoothedAltitudeMeters[static_cast<size_t>(triggerIdx)];
00478         if (!isFinite(localMinAlt))
00479         {
00480             i = triggerIdx + 1;
00481             continue;
00482         }
00483
00484         // Backtrack up to 1 km to anchor the climb start at the local minimum.
00485         for (int b = triggerIdx - 1; b >= 0; --b)
00486         {
00487             const double backtrackDist =

```



```

00488         cumulativeDistanceMeters[static_cast<size_t>(triggerIdx)] -
cumulativeDistanceMeters[static_cast<size_t>(b)];
00489         if (backtrackDist > 1000.0)
00490             break;
00491
00492         const double alt = smoothedAltitudeMeters[static_cast<size_t>(b)];
00493         if (isFinite(alt) && alt <= localMinAlt)
00494         {
00495             localMinAlt = alt;
00496             localMinIdx = b;
00497         }
00498     }
00499
00500     const int climbStart = localMinIdx;
00501
00502     double sustainedDistance = 0.0;
00503     double interruptionDistance = 0.0;
00504     double candidateGainM = 0.0;
00505     int confirmedIdx = -1;
00506     for (int j = std::max(triggerIdx, climbStart + 1); j < n; ++j)
00507     {
00508         const double segmentDist =
00509             cumulativeDistanceMeters[static_cast<size_t>(j)] -
cumulativeDistanceMeters[static_cast<size_t>(j - 1)];
00510         const double prevAlt = smoothedAltitudeMeters[static_cast<size_t>(j - 1)];
00511         const double alt = smoothedAltitudeMeters[static_cast<size_t>(j)];
00512
00513         if (isFinite(prevAlt) && isFinite(alt))
00514             candidateGainM += std::max(0.0, alt - prevAlt);
00515
00516         if (isContinuationSignal(j))
00517         {
00518             sustainedDistance += std::max(0.0, segmentDist);
00519             interruptionDistance = 0.0;
00520         }
00521         else
00522         {
00523             interruptionDistance += std::max(0.0, segmentDist);
00524         }
00525
00526         if (interruptionDistance > config.maxStartInterruptionMeters)
00527             break;
00528
00529         if (sustainedDistance >= config.minStartContinuousDistanceMeters &&
00530             candidateGainM >= std::min(config.minElevationGainM, 5.0))
00531         {
00532             confirmedIdx = j;
00533             break;
00534         }
00535     }
00536
00537     if (confirmedIdx < 0)
00538     {
00539         i = triggerIdx + 1;
00540         continue;
00541     }
00542
00543     int peakIdx = climbStart;
00544     double peakAlt = localMinAlt;
00545     double plateauDistance = 0.0;
00546     int smallDipsCount = 0; // Phase 3: Track small dips
00547     double cumulativeGainM = 0.0;
00548
00549     int k = confirmedIdx;
00550     for (; k < n; ++k)
00551     {
00552         const double alt = smoothedAltitudeMeters[static_cast<size_t>(k)];
00553         if (!isFinite(alt))
00554             continue;
00555
00556         const double gradient100mPct[static_cast<size_t>(k)];
00557         const double segmentDist = (k > 0)
00558             ? std::max(0.0, cumulativeDistanceMeters[static_cast<size_t>(k)] -
cumulativeDistanceMeters[static_cast<size_t>(k - 1)])
00559             : 0.0;
00560         if (k > 0)
00561         {
00562             const double prevAlt = smoothedAltitudeMeters[static_cast<size_t>(k - 1)];
00563             if (isFinite(prevAlt) && isFinite(alt))
00564                 cumulativeGainM += std::max(0.0, alt - prevAlt);
00565         }
00566
00567         if (alt >= peakAlt)
00568         {
00569             peakAlt = alt;
00570             peakIdx = k;
00571             plateauDistance = 0.0;

```

```

00572         smallDipsCount = 0; // Reset dip counter on new peak
00573         continue;
00574     }
00575
00576     const double dipMeters = peakAlt - alt;
00577     const double dipDistance =
00578         cumulativeDistanceMeters[static_cast<size_t>(k)] -
00579         cumulativeDistanceMeters[static_cast<size_t>(peakIdx)];
00580
00581     const double gainSoFar = std::max(0.0, peakAlt - localMinAlt);
00582     const double allowedDipMeters = std::max(
00583         config.maxDipMeters,
00584         std::max(gainSoFar, cumulativeGainM) * (config.maxDipPctOfGain / 100.0));
00585
00586     // Phase 3: Count small dips (< 5m) separately
00587     if (dipMeters <= config.smallDipThresholdMeters)
00588         smallDipsCount++;
00589
00590     if (isContinuationSignal(k))
00591     {
00592         plateauDistance = 0.0;
00593         smallDipsCount = 0; // Reset on uphill gradient
00594         continue;
00595     }
00596
00597     if (isFinite(gradient) && gradient >= config.plateauMinGradient)
00598         plateauDistance += segmentDist;
00599     else
00600         plateauDistance = config.maxPlateauDistanceMeters + 1.0;
00601
00602     // Phase 3: Allow multiple small dips before terminating
00603     const bool exceededDip =
00604         dipMeters > allowedDipMeters &&
00605         dipDistance > config.maxDipDistanceMeters &&
00606         smallDipsCount > config.maxSmallDipsCount;
00607     const bool plateauExpired =
00608         plateauDistance > config.maxPlateauDistanceMeters &&
00609         (!isFinite(gradient) || gradient < config.endGradient);
00610
00611     if (exceededDip || plateauExpired)
00612         break;
00613 }
00614
00615 const int climbEnd = peakIdx;
00616 i = std::max(k, climbEnd + 1);
00617
00618 if (climbEnd <= climbStart)
00619     continue;
00620
00621 Climb climb = ClimbMetrics::build(
00622     rideData,
00623     cumulativeDistanceMeters,
00624     smoothedAltitudeMeters,
00625     gradient100mPct,
00626     climbStart,
00627     climbEnd,
00628     static_cast<int>(climbs.size()) + 1);
00629
00630 // Phase 2: Compute composite climb score
00631 climb.climbScore = computeClimbScore(
00632     climb.elevationGainM,
00633     climb.averageGradient,
00634     climb.lengthKm,
00635     climb.gradePercentile / 100.0,
00636     climb.vam);
00637
00638 // Phase 1: Categorize the climb
00639 climb.climbCategory = categorizeClimb(
00640     climb.elevationGainM,
00641     climb.lengthKm,
00642     climb.averageGradient);
00643
00644 // Phase 2: Use scoring-based detection if enabled
00645 if (config.useScoringDetection)
00646 {
00647     const bool isPunchyClimb =
00648         climb.elevationGainM >= 10.0 &&
00649         climb.lengthKm >= 0.1 &&
00650         climb.maximumGradient >= 5.0;
00651     const bool isRegularClimb =
00652         climb.elevationGainM >= 25.0 &&
00653         climb.lengthKm >= 0.5 &&
00654         climb.averageGradient >= 3.0;
00655     const bool isLongShallowClimb =
00656         climb.elevationGainM >= 40.0 &&
00657         climb.lengthKm >= 1.0 &&
00658         climb.averageGradient >= 1.5;

```

```

00658
00659         if (climb.climbScore < config.minClimbScore &&
00660             !isPunchyClimb &&
00661             !isRegularClimb &&
00662             !isLongShallowClimb)
00663             continue;
00664     }
00665     else
00666     {
00667         // Fallback to legacy threshold-based detection
00668         if (climb.lengthKm < config.minLengthKm)
00669             continue;
00670         if (climb.elevationGainM < config.minElevationGainM)
00671             continue;
00672         if (climb.averageGradient < config.minAverageGradient)
00673             continue;
00674     }
00675     climbs.push_back(climb);
00676 }
00677
00678 // Phase 7: Merge fragmented climbs if enabled
00679 if (config.enableClimbMerging)
00680 {
00681     climbs = mergeFragmentedClimbs(climbs, config);
00682 }
00683
00684 return climbs;
00685 }
00686 }

```

8.2 ClimbDetector.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004 #include <vector>
00005
00006 #include "analysis/ClimbMetrics.h"
00007
00016 struct ClimbDetectorConfig
00017 {
00019     double minLengthKm = 0.15;
00021     double minElevationGainM = 10.0;
00023     double minAverageGradient = 4.0;
00024
00026     double startGradient = 1.5;
00028     double continueGradient = 1.0;
00030     double endGradient = 0.5;
00032     double minStartContinuousDistanceMeters = 100.0;
00034     double maxStartInterruptionMeters = 30.0;
00035
00037     double maxDipMeters = 10.0;
00039     double maxDipDistanceMeters = 200.0;
00041     double maxDipPctOfGain = 10.0;
00043     double maxSmallDipsCount = 3;
00045     double smallDipThresholdMeters = 5.0;
00046
00048     double maxPlateauDistanceMeters = 500.0;
00050     double plateauMinGradient = -1.0;
00051
00053     double elevationSmoothingMeters = 50.0;
00055     bool useAdaptiveSmoothing = true;
00057     double adaptiveSmoothingMinMeters = 25.0;
00059     double adaptiveSmoothingMaxMeters = 100.0;
00061     double gradientWindowHalfMeters = 25.0;
00062
00064     double altitudeSpikeVerticalSpeedMps = 8.0;
00065
00067     bool useScoringDetection = true;
00069     double minClimbScore = 50.0;
00071     double gainScoreWeight = 0.5;
00073     double gradeScoreWeight = 4.0;
00075     double durationScoreWeight = 2.0;
00077     double vamScoreBoost = 800.0;
00078
00080     bool requireGradePersistence = true;
00082     double gradePersistenceThreshold = 4.0;
00084     double gradePersistenceMinMeters = 50.0;
00085
00087     bool enableClimbMerging = true;
00089     double mergeMaxGapMeters = 50.0;
00091     double mergeMaxGapElevationM = 10.0;

```

```

00092 };
00093
00100 class ClimbDetector
00101 {
00102 public:
00104     using Config = ClimbDetectorConfig;
00105
00112     static std::vector<Climb> detect(
00113         const RideData& rideData,
00114         const Config& config = Config{});
00115
00121     static std::vector<double> buildCumulativeDistanceMeters(const RideData& rideData);
00122
00132     static std::vector<double> smoothAltitudeByDistance(
00133         const RideData& rideData,
00134         const std::vector<double>& cumulativeDistanceMeters,
00135         double windowMeters,
00136         const std::vector<double>& adaptiveWindowMeters = {},
00137         double altitudeSpikeVerticalSpeedMps = 8.0);
00138
00146     static std::vector<double> computeAdaptiveSmoothingWindowsMeters(
00147         const RideData& rideData,
00148         const std::vector<double>& cumulativeDistanceMeters,
00149         const Config& config);
00150
00158     static std::vector<double> computeLocalGradientPct(
00159         const std::vector<double>& smoothedAltitudeMeters,
00160         const std::vector<double>& cumulativeDistanceMeters,
00161         double halfWindowMeters);
00162
00172     static double computeClimbScore(
00173         double gainMeters,
00174         double averageGrade,
00175         double distanceKm,
00176         double continuityRatio,
00177         double vam);
00178
00185     static double computeVAM(double elevationGainM, double durationSeconds);
00186
00194     static ClimbCategory categorizeClimb(
00195         double gainMeters,
00196         double lengthKm,
00197         double averageGrade);
00198
00205     static std::vector<Climb> mergeFragmentedClimbs(
00206         std::vector<Climb>& candidates,
00207         const Config& config);
00208 };

```

8.3 ClimbMetrics.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "ClimbMetrics.h"
00005
00006 #include <algorithm>
00007 #include <cmath>
00008 #include <limits>
00009
00010 namespace
00011 {
00012     struct SliceStats
00013     {
00014         double avgPower = 0.0;
00015         double avgHeartRate = 0.0;
00016         double avgCadence = 0.0;
00017     };
00018
00019 bool isFinite(double value)
00020 {
00021     return std::isfinite(value);
00022 }
00023
00024 double computeLinearRegressionSlope(
00025     const std::vector<double>& x,
00026     const std::vector<double>& y,
00027     double* interceptOut = nullptr)
00028 {
00029     if (x.size() != y.size() || x.size() < 2)
00030     {
00031         if (interceptOut)
00032             *interceptOut = 0.0;
00033         return 0.0;
00034     }

```

```

00034     }
00035
00036     const double n = static_cast<double>(x.size());
00037     double sumX = 0.0;
00038     double sumY = 0.0;
00039     double sumXY = 0.0;
00040     double sumXX = 0.0;
00041
00042     for (size_t i = 0; i < x.size(); ++i)
00043     {
00044         sumX += x[i];
00045         sumY += y[i];
00046         sumXY += x[i] * y[i];
00047         sumXX += x[i] * x[i];
00048     }
00049
00050     const double denom = (n * sumXX) - (sumX * sumX);
00051     if (std::abs(denom) < 1e-9)
00052     {
00053         if (interceptOut)
00054             *interceptOut = sumY / n;
00055         return 0.0;
00056     }
00057
00058     const double slope = ((n * sumXY) - (sumX * sumY)) / denom;
00059     if (interceptOut)
00060         *interceptOut = (sumY - slope * sumX) / n;
00061     return slope;
00062 }
00063
00064 QString classifyClimbCategory(double gainM, double difficultyScore)
00065 {
00066     if (gainM >= 800.0 || difficultyScore >= 4500.0)
00067         return QStringLiteral("Mountain Pass");
00068     if (gainM >= 400.0)
00069         return QStringLiteral("Major Climb");
00070     if (gainM >= 150.0)
00071         return QStringLiteral("Climb");
00072     if (gainM >= 50.0)
00073         return QStringLiteral("Hill");
00074     return QStringLiteral("Bump");
00075 }
00076
00077 QString classifyClimbShape(
00078     const std::vector<double>& gradients,
00079     double lengthKm,
00080     double gainM,
00081     double avgGradient,
00082     double maxGradient)
00083 {
00084     if (gradients.size() < 4)
00085         return QStringLiteral("Steady");
00086
00087     double sum = 0.0;
00088     double sumSq = 0.0;
00089     double minG = gradients.front();
00090     double maxG = gradients.front();
00091     int rollingTransitions = 0;
00092     int prevBand = 0;
00093
00094     for (double g : gradients)
00095     {
00096         sum += g;
00097         sumSq += g * g;
00098         minG = std::min(minG, g);
00099         maxG = std::max(maxG, g);
00100
00101         const int band = (g >= 4.0) ? 1 : (g <= 1.0 ? -1 : 0);
00102         if (prevBand != 0 && band != 0 && band != prevBand)
00103             ++rollingTransitions;
00104         if (band != 0)
00105             prevBand = band;
00106     }
00107
00108     const double n = static_cast<double>(gradients.size());
00109     const double mean = sum / n;
00110     const double variance = std::max(0.0, (sumSq / n) - (mean * mean));
00111     const double stdDev = std::sqrt(variance);
00112
00113     const size_t chunk = std::max<size_t>(1, gradients.size() / 3);
00114     auto meanChunk = [&gradients](size_t begin, size_t end)
00115     {
00116         if (begin >= end || end > gradients.size())
00117             return 0.0;
00118         double s = 0.0;
00119         for (size_t i = begin; i < end; ++i)
00120             s += gradients[i];

```

```

00121         return s / static_cast<double>(end - begin);
00122     };
00123
00124     const double firstThird = meanChunk(0, std::min(gradients.size(), chunk));
00125     const double midThird = meanChunk(std::min(gradients.size(), chunk), std::min(gradients.size(), 2
* chunk));
00126     const double lastThird = meanChunk(std::min(gradients.size(), 2 * chunk), gradients.size());
00127
00128     if (lengthKm >= 8.0 && gainM >= 600.0 && avgGradient >= 5.0 && stdDev < 2.2)
00129         return QStringLiteral("Alpine");
00130     if (rollingTransitions >= 3 && stdDev >= 2.5)
00131         return QStringLiteral("Rolling");
00132     if (maxGradient >= 10.0 && (lastThird - firstThird >= 3.0 || maxGradient - mean >= 5.0))
00133         return QStringLiteral("Wall");
00134     if (stdDev >= 2.0 && (midThird + 1.0 < firstThird || midThird + 1.0 < lastThird) && (maxG - minG)
>= 5.0)
00135         return QStringLiteral("Staircase");
00136     if (stdDev <= 1.5)
00137         return QStringLiteral("Steady");
00138     return QStringLiteral("Mixed");
00139 }
00140
00141 SliceStats computeSliceStats(
00142     const RideData& rideData,
00143     int startIdx,
00144     int endIdx,
00145     const std::vector<double>& basis,
00146     double sliceStart,
00147     double sliceEnd)
00148 {
00149     SliceStats stats;
00150     if (startIdx < 0 || endIdx < startIdx || endIdx >= static_cast<int>(rideData.records.size()))
00151         return stats;
00152
00153     double sumPower = 0.0;
00154     int countPower = 0;
00155     double sumHr = 0.0;
00156     int countHr = 0;
00157     double sumCadence = 0.0;
00158     int countCadence = 0;
00159
00160     for (int i = startIdx; i <= endIdx; ++i)
00161     {
00162         const double v = basis[static_cast<size_t>(i)];
00163         if (!isFinite(v) || v < sliceStart || v > sliceEnd)
00164             continue;
00165
00166         const RideRecord& r = rideData.records[static_cast<size_t>(i)];
00167         if (r.hasPower)
00168         {
00169             sumPower += r.power;
00170             ++countPower;
00171         }
00172         if (r.hasHeartRate)
00173         {
00174             sumHr += r.heartRate;
00175             ++countHr;
00176         }
00177         if (r.hasCadence)
00178         {
00179             sumCadence += r.cadence;
00180             ++countCadence;
00181         }
00182     }
00183
00184     if (countPower > 0)
00185         stats.avgPower = sumPower / static_cast<double>(countPower);
00186     if (countHr > 0)
00187         stats.avgHeartRate = sumHr / static_cast<double>(countHr);
00188     if (countCadence > 0)
00189         stats.avgCadence = sumCadence / static_cast<double>(countCadence);
00190
00191     return stats;
00192 }
00193
00194 double computeNormalizedPower(const std::vector<double>& power)
00195 {
00196     if (power.empty())
00197         return 0.0;
00198
00199     const int npWin = std::min(30, static_cast<int>(power.size()));
00200     if (npWin <= 0)
00201         return 0.0;
00202
00203     double rollingSum = 0.0;
00204     for (int i = 0; i < npWin; ++i)
00205         rollingSum += power[static_cast<size_t>(i)];

```

```

00206
00207     double sum4 = std::pow(rollingSum / static_cast<double>(npWin), 4.0);
00208     int count = 1;
00209
00210     for (int i = npWin; i < static_cast<int>(power.size()); ++i)
00211     {
00212         rollingSum += power[static_cast<size_t>(i)] - power[static_cast<size_t>(i - npWin)];
00213         sum4 += std::pow(rollingSum / static_cast<double>(npWin), 4.0);
00214         ++count;
00215     }
00216
00217     return std::pow(sum4 / static_cast<double>(count), 0.25);
00218 }
00219 } // namespace
00220
00221 Climb ClimbMetrics::build(
00222     const RideData& rideData,
00223     const std::vector<double>& cumulativeDistanceMeters,
00224     const std::vector<double>& smoothedAltitudeMeters,
00225     const std::vector<double>& localGradientPct,
00226     int startIdx,
00227     int endIdx,
00228     int climbOrdinal)
00229 {
00230     Climb climb;
00231     if (startIdx < 0 || endIdx < startIdx || endIdx >= static_cast<int>(rideData.records.size()))
00232         return climb;
00233
00234     const RideRecord& first = rideData.records[static_cast<size_t>(startIdx)];
00235     const RideRecord& last = rideData.records[static_cast<size_t>(endIdx)];
00236
00237     climb.startSeconds = first.elapsedSeconds;
00238     climb.endSeconds = last.elapsedSeconds;
00239     climb.durationSeconds = std::max(0.0, climb.endSeconds - climb.startSeconds);
00240
00241     climb.startDistanceKm = cumulativeDistanceMeters[static_cast<size_t>(startIdx)] / 1000.0;
00242     climb.endDistanceKm = cumulativeDistanceMeters[static_cast<size_t>(endIdx)] / 1000.0;
00243     climb.lengthKm = std::max(0.0, climb.endDistanceKm - climb.startDistanceKm);
00244
00245     double startAlt = first.hasAltitude ? first.altitude : std::numeric_limits<double>::quiet_NaN();
00246     double endAlt = last.hasAltitude ? last.altitude : std::numeric_limits<double>::quiet_NaN();
00247     if (isFinite(smoothedAltitudeMeters[static_cast<size_t>(startIdx)]))
00248         startAlt = smoothedAltitudeMeters[static_cast<size_t>(startIdx)];
00249     if (isFinite(smoothedAltitudeMeters[static_cast<size_t>(endIdx)]))
00250         endAlt = smoothedAltitudeMeters[static_cast<size_t>(endIdx)];
00251
00252     climb.startElevationM = isFinite(startAlt) ? startAlt : 0.0;
00253     climb.endElevationM = isFinite(endAlt) ? endAlt : climb.startElevationM;
00254
00255     double cumulativeGainM = 0.0;
00256     for (int i = startIdx + 1; i <= endIdx; ++i)
00257     {
00258         const double prevAlt = smoothedAltitudeMeters[static_cast<size_t>(i - 1)];
00259         const double currAlt = smoothedAltitudeMeters[static_cast<size_t>(i)];
00260         if (!isFinite(prevAlt) || !isFinite(currAlt))
00261             continue;
00262
00263         cumulativeGainM += std::max(0.0, currAlt - prevAlt);
00264     }
00265     climb.elevationGainM = cumulativeGainM;
00266
00267     const double lengthMeters = climb.lengthKm * 1000.0;
00268     if (lengthMeters > 0.0)
00269         climb.averageGradient = (climb.elevationGainM / lengthMeters) * 100.0;
00270
00271     double maxGradient = 0.0;
00272     int validGradientCount = 0;
00273     int climbingGradientCount = 0;
00274     for (int i = startIdx; i <= endIdx; ++i)
00275     {
00276         const double g = localGradientPct[static_cast<size_t>(i)];
00277         if (isFinite(g))
00278         {
00279             maxGradient = std::max(maxGradient, g);
00280             ++validGradientCount;
00281             if (g >= 1.5)
00282                 ++climbingGradientCount;
00283         }
00284     }
00285     climb.maximumGradient = maxGradient;
00286     if (validGradientCount > 0)
00287     {
00288         climb.gradePercentile =
00289             (static_cast<double>(climbingGradientCount) / static_cast<double>(validGradientCount)) *
00290             100.0;
00291     }

```

```

00292     std::vector<double> powerSeries;
00293     powerSeries.reserve(static_cast<size_t>(endIdx - startIdx + 1));
00294
00295     double powerSum = 0.0;
00296     int powerCount = 0;
00297     double hrSum = 0.0;
00298     int hrCount = 0;
00299     double cadenceSum = 0.0;
00300     int cadenceCount = 0;
00301     double speedSum = 0.0;
00302     int speedCount = 0;
00303
00304     for (int i = startIdx; i <= endIdx; ++i)
00305     {
00306         const RideRecord& r = rideData.records[static_cast<size_t>(i)];
00307         powerSeries.push_back(r.hasPower ? r.power : 0.0);
00308
00309         if (r.hasPower)
00310         {
00311             powerSum += r.power;
00312             ++powerCount;
00313         }
00314         if (r.hasHeartRate)
00315         {
00316             hrSum += r.heartRate;
00317             ++hrCount;
00318         }
00319         if (r.hasCadence)
00320         {
00321             cadenceSum += r.cadence;
00322             ++cadenceCount;
00323         }
00324         if (r.hasSpeed)
00325         {
00326             speedSum += r.speed;
00327             ++speedCount;
00328         }
00329     }
00330
00331     if (powerCount > 0)
00332         climb.averagePower = powerSum / static_cast<double>(powerCount);
00333     if (hrCount > 0)
00334         climb.averageHeartRate = hrSum / static_cast<double>(hrCount);
00335     if (cadenceCount > 0)
00336         climb.averageCadence = cadenceSum / static_cast<double>(cadenceCount);
00337     if (speedCount > 0)
00338         climb.averageSpeed = speedSum / static_cast<double>(speedCount);
00339
00340     climb.normalizedPower = computeNormalizedPower(powerSeries);
00341
00342     const double durationHours = climb.durationSeconds / 3600.0;
00343     if (durationHours > 0.0)
00344         climb.vam = climb.elevationGainM / durationHours;
00345
00346     const bool canUseDistanceSlices =
00347         cumulativeDistanceMeters[static_cast<size_t>(endIdx)] >
00348         cumulativeDistanceMeters[static_cast<size_t>(startIdx)];
00349
00350     std::vector<double> basis(rideData.records.size(), 0.0);
00351     if (canUseDistanceSlices)
00352     {
00353         basis = cumulativeDistanceMeters;
00354     }
00355     else
00356     {
00357         for (size_t i = 0; i < rideData.records.size(); ++i)
00358             basis[i] = rideData.records[i].elapsedSeconds;
00359     }
00360
00361     const double spanStart = basis[static_cast<size_t>(startIdx)];
00362     const double spanEnd = basis[static_cast<size_t>(endIdx)];
00363     const double span = std::max(0.0, spanEnd - spanStart);
00364
00365     if (span > 0.0)
00366     {
00367         const double q0 = spanStart;
00368         const double q1 = spanStart + span * 0.25;
00369         const double q2 = spanStart + span * 0.50;
00370         const double q3 = spanStart + span * 0.75;
00371         const double q4 = spanEnd;
00372
00373         const SliceStats firstHalf = computeSliceStats(rideData, startIdx, endIdx, basis, q0, q2);
00374         const SliceStats secondHalf = computeSliceStats(rideData, startIdx, endIdx, basis, q2, q4);
00375         climb.powerFirstHalf = firstHalf.avgPower;
00376         climb.powerSecondHalf = secondHalf.avgPower;
00377
00378         const SliceStats firstQuarter = computeSliceStats(rideData, startIdx, endIdx, basis, q0, q1);

```



```

00378     const SliceStats secondQuarter = computeSliceStats(rideData, startIdx, endIdx, basis, q1, q2);
00379     const SliceStats thirdQuarter = computeSliceStats(rideData, startIdx, endIdx, basis, q2, q3);
00380     const SliceStats lastQuarter = computeSliceStats(rideData, startIdx, endIdx, basis, q3, q4);
00381
00382     climb.quarterMetrics[0] = { firstQuarter.avgPower, firstQuarter.avgHeartRate,
firstQuarter.avgCadence };
00383     climb.quarterMetrics[1] = { secondQuarter.avgPower, secondQuarter.avgHeartRate,
secondQuarter.avgCadence };
00384     climb.quarterMetrics[2] = { thirdQuarter.avgPower, thirdQuarter.avgHeartRate,
thirdQuarter.avgCadence };
00385     climb.quarterMetrics[3] = { lastQuarter.avgPower, lastQuarter.avgHeartRate,
lastQuarter.avgCadence };
00386
00387     if (firstQuarter.avgPower > 0.0)
00388     {
00389         climb.powerFadePct =
00390             ((lastQuarter.avgPower - firstQuarter.avgPower) / firstQuarter.avgPower) * 100.0;
00391     }
00392
00393     if (firstQuarter.avgPower > 0.0 && lastQuarter.avgPower > 0.0 &&
00394         firstQuarter.avgHeartRate > 0.0 && lastQuarter.avgHeartRate > 0.0)
00395     {
00396         const double startRatio = firstQuarter.avgHeartRate / firstQuarter.avgPower;
00397         const double endRatio = lastQuarter.avgHeartRate / lastQuarter.avgPower;
00398         if (startRatio > 0.0)
00399             climb.hrDriftPct = ((endRatio - startRatio) / startRatio) * 100.0;
00400     }
00401 }
00402
00403 std::vector<double> decoupleX;
00404 std::vector<double> decoupleY;
00405 decoupleX.reserve(static_cast<size_t>(endIdx - startIdx + 1));
00406 decoupleY.reserve(static_cast<size_t>(endIdx - startIdx + 1));
00407
00408 for (int i = startIdx; i <= endIdx; ++i)
00409 {
00410     const RideRecord& r = rideData.records[static_cast<size_t>(i)];
00411     if (!r.hasHeartRate || !r.hasPower || r.power <= 0.0 || r.heartRate <= 0.0)
00412         continue;
00413
00414     const double progress = (climb.durationSeconds > 0.0)
00415         ? (r.elapsedSeconds - climb.startSeconds) / climb.durationSeconds
00416         : 0.0;
00417     decoupleX.push_back(std::clamp(progress, 0.0, 1.0));
00418     decoupleY.push_back(r.heartRate / r.power);
00419 }
00420
00421 if (decoupleX.size() >= 6)
00422 {
00423     double intercept = 0.0;
00424     const double slope = computeLinearRegressionSlope(decoupleX, decoupleY, &intercept);
00425     const double startRatio = intercept;
00426     const double endRatio = intercept + slope;
00427     if (startRatio > 0.0)
00428         climb.aerobicDecouplingPct = ((endRatio - startRatio) / startRatio) * 100.0;
00429 }
00430 else
00431 {
00432     climb.aerobicDecouplingPct = climb.hrDriftPct;
00433 }
00434
00435 climb.hrDriftPct = climb.aerobicDecouplingPct;
00436
00437 climb.difficultyScore = climb.elevationGainM * climb.averageGradient;
00438
00439 std::vector<double> climbGradients;
00440 climbGradients.reserve(static_cast<size_t>(endIdx - startIdx + 1));
00441 for (int i = startIdx; i <= endIdx; ++i)
00442 {
00443     const double g = localGradientPct[static_cast<size_t>(i)];
00444     if (isFinite(g))
00445         climbGradients.push_back(g);
00446 }
00447
00448 climb.category = classifyClimbCategory(climb.elevationGainM, climb.difficultyScore);
00449 climb.shapeClass = classifyClimbShape(
00450     climbGradients,
00451     climb.lengthKm,
00452     climb.elevationGainM,
00453     climb.averageGradient,
00454     climb.maximumGradient);
00455 climb.name = QString("Climb %1").arg(climbOrdinal);
00456
00457 return climb;
00458 }

```

8.4 ClimbMetrics.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <array>
00007 #include <vector>
00008
00009 #include <QString>
00010
00011 #include "fit/RideData.h"
00012
00016 enum class ClimbCategory
00017 {
00018     Major,          // >100m gain, >2km length, >3% avg grade
00019     Medium,         // >40m gain, >500m length, >3% avg grade
00020     Punchy,         // >10m gain, >100m length, >5% avg grade (short steep)
00021     FalsePositive // failed quality checks
00022 };
00023
00027 struct ClimbQuarterMetrics
00028 {
00030     double averagePower = 0.0;
00031
00033     double averageHeartRate = 0.0;
00034
00036     double averageCadence = 0.0;
00037 };
00038
00044 struct Climb
00045 {
00047     int id = -1;
00048
00050     double startSeconds = 0.0;
00051
00053     double endSeconds = 0.0;
00054
00056     double startDistanceKm = 0.0;
00057
00059     double endDistanceKm = 0.0;
00060
00062     double durationSeconds = 0.0;
00063
00065     double lengthKm = 0.0;
00066
00068     double elevationGainM = 0.0;
00069
00071     double startElevationM = 0.0;
00072
00074     double endElevationM = 0.0;
00075
00077     double averageGradient = 0.0;
00078
00080     double maximumGradient = 0.0;
00081
00083     double climbScore = 0.0;
00084
00086     double gradePercentile = 0.0;
00087
00089     ClimbCategory climbCategory = ClimbCategory::FalsePositive;
00090
00092     double averagePower = 0.0;
00093
00095     double averageWattsPerKg = 0.0;
00096
00098     double normalizedPower = 0.0;
00099
00101     double averageHeartRate = 0.0;
00102
00104     double averageCadence = 0.0;
00105
00107     double averageSpeed = 0.0;
00108
00110     double vam = 0.0;
00111
00113     double powerFirstHalf = 0.0;
00114
00116     double powerSecondHalf = 0.0;
00117
00119     double powerFadePct = 0.0;
00120
00122     double aerobicDecouplingPct = 0.0;
00123
00125     double hrDriftPct = 0.0;

```

```

00126
00128     double difficultyScore = 0.0;
00129
00131     std::array<ClimbQuarterMetrics, 4> quarterMetrics{};
00132
00134     QString category;
00135
00137     QString shapeClass;
00138
00140     QString wattsPerKgRank;
00141
00143     QString name;
00144 };
00145
00149 class ClimbMetrics
00150 {
00151 public:
00163     static Climb build(
00164         const RideData& rideData,
00165         const std::vector<double>& cumulativeDistanceMeters,
00166         const std::vector<double>& smoothedAltitudeMeters,
00167         const std::vector<double>& localGradientPct,
00168         int startIdx,
00169         int endIdx,
00170         int climbOrdinal);
00171 };

```

8.5 FTPZones.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "FTPZones.h"
00005
00006 #include <algorithm>
00007 #include <cmath>
00008
00009 FTPZones::ZoneArray FTPZones::defaultZones()
00010 {
00011     return {{
00012         { 1, "Active Recovery", 0.00, 0.55, 0.0 },
00013         { 2, "Endurance", 0.55, 0.75, 0.0 },
00014         { 3, "Tempo", 0.75, 0.90, 0.0 },
00015         { 4, "Threshold", 0.90, 1.05, 0.0 },
00016         { 5, "VO2 Max", 1.05, 1.20, 0.0 },
00017         { 6, "Anaerobic", 1.20, 1.50, 0.0 },
00018         { 7, "Neuromuscular", 1.50, -1.00, 0.0 },
00019     }};
00020 }
00021
00022 FTPZones::ZoneArray FTPZones::compute(
00023     const RideData& rideData, double ftp)
00024 {
00025     auto zones = defaultZones();
00026     if (ftp <= 0.0 || rideData.records.empty())
00027         return zones;
00028
00029     const auto& records = rideData.records;
00030     const int n = static_cast<int>(records.size());
00031
00032     for (int i = 0; i < n; ++i)
00033     {
00034         const auto& r = records[i];
00035         if (!r.hasPower) continue;
00036
00037         // Midpoint time weight
00038         double dt = 1.0;
00039         if (n > 1)
00040         {
00041             if (i == 0)
00042                 dt = records[1].elapsedSeconds - records[0].elapsedSeconds;
00043             else if (i == n - 1)
00044                 dt = records[n - 1].elapsedSeconds - records[n - 2].elapsedSeconds;
00045             else
00046                 dt = (records[i + 1].elapsedSeconds -
00047                     records[i - 1].elapsedSeconds) / 2.0;
00048         }
00049         dt = std::max(0.0, dt);
00050
00051         const double pct = r.power / ftp;
00052         for (auto& zone : zones)
00053         {
00054             if (pct >= zone.lowerPct &&
00055                 (zone.upperPct < 0.0 || pct < zone.upperPct))

```

```

00056         {
00057             zone.seconds += dt;
00058             break;
00059         }
00060     }
00061 }
00062
00063 return zones;
00064 }

```

8.6 FTPZones.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <array>
00007 #include <string>
00008
00009 #include "fit/RideData.h"
00010
00014 struct PowerZone
00015 {
00017     int         number;
00018
00020     std::string name;
00021
00023     double      lowerPct;
00024
00026     double      upperPct;
00027
00029     double      seconds = 0.0;
00030 };
00031
00035 class FTPZones
00036 {
00037 public:
00039     static constexpr int kZoneCount = 7;
00040
00042     using ZoneArray = std::array<PowerZone, kZoneCount>;
00043
00048     static ZoneArray defaultZones();
00049
00056     static ZoneArray compute(const RideData& rideData, double ftp);
00057 };

```

8.7 IntervalDetector.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "IntervalDetector.h"
00005
00006 #include <algorithm>
00007 #include <cmath>
00008
00009 std::vector<double> IntervalDetector::smoothPower(
00010     const RideData& rideData, int window)
00011 {
00012     const int n = static_cast<int>(rideData.records.size());
00013     std::vector<double> raw(n);
00014     for (int i = 0; i < n; ++i)
00015     {
00016         const auto& r = rideData.records[i];
00017         raw[i] = r.hasPower ? r.power : 0.0;
00018     }
00019
00020     const int half = window / 2;
00021     std::vector<double> out(n);
00022     for (int i = 0; i < n; ++i)
00023     {
00024         const int lo = std::max(0, i - half);
00025         const int hi = std::min(n - 1, i + half);
00026         double s = 0.0;
00027         for (int j = lo; j <= hi; ++j) s += raw[j];
00028         out[i] = s / (hi - lo + 1);
00029     }
00030
00031     return out;
00032 }

```

```

00033
00034 Interval IntervalDetector::buildInterval(
00035     const RideData& rideData,
00036     int startIdx, int endIdx,
00037     const std::string& label)
00038 {
00039     Interval iv;
00040     iv.label = label;
00041     iv.startSeconds = rideData.records[startIdx].elapsedSeconds;
00042     iv.endSeconds = rideData.records[endIdx].elapsedSeconds;
00043     iv.durationSeconds = iv.endSeconds - iv.startSeconds;
00044
00045     std::vector<double> pArr;
00046     double pSum = 0; int pN = 0;
00047     double hrSum = 0; int hrN = 0;
00048     double cadSum = 0; int cadN = 0;
00049     double spdSum = 0; int spdN = 0;
00050
00051     for (int i = startIdx; i <= endIdx; ++i)
00052     {
00053         const auto& r = rideData.records[i];
00054         pArr.push_back(r.hasPower ? r.power : 0.0);
00055
00056         if (r.hasPower)
00057         {
00058             pSum += r.power;
00059             iv.maximumPower = std::max(iv.maximumPower, r.power);
00060             ++pN;
00061         }
00062         if (r.hasHeartRate) { hrSum += r.heartRate; ++hrN; }
00063         if (r.hasCadence) { cadSum += r.cadence; ++cadN; }
00064         if (r.hasSpeed) { spdSum += r.speed; ++spdN; }
00065     }
00066
00067     if (pN > 0) iv.averagePower = pSum / pN;
00068     if (hrN > 0) iv.averageHeartRate = hrSum / hrN;
00069     if (cadN > 0) iv.averageCadence = cadSum / cadN;
00070     if (spdN > 0) iv.averageSpeed = spdSum / spdN;
00071
00072     // Normalised Power (30 s rolling)
00073     const int npWin = std::min(30, static_cast<int>(pArr.size()));
00074     if (npWin > 0)
00075     {
00076         double wSum = 0.0;
00077         for (int i = 0; i < npWin; ++i) wSum += pArr[i];
00078         double s4 = std::pow(wSum / npWin, 4.0);
00079         int cnt = 1;
00080         for (int i = npWin; i < static_cast<int>(pArr.size()); ++i)
00081         {
00082             wSum += pArr[i] - pArr[i - npWin];
00083             s4 += std::pow(wSum / npWin, 4.0);
00084             ++cnt;
00085         }
00086         iv.normalizedPower = std::pow(s4 / cnt, 0.25);
00087     }
00088
00089     return iv;
00090 }
00091
00092 std::vector<Interval> IntervalDetector::detect(
00093     const RideData& rideData, const Config& config)
00094 {
00095     const auto& records = rideData.records;
00096     if (records.empty()) return {};
00097
00098     bool hasPower = false;
00099     for (const auto& r : records)
00100         if (r.hasPower) { hasPower = true; break; }
00101     if (!hasPower) return {};
00102
00103     const double threshold =
00104         config.ftp * config.workThresholdPct;
00105     const auto smoothed =
00106         smoothPower(rideData, config.smoothingWindow);
00107
00108     const int n = static_cast<int>(records.size());
00109     std::vector<Interval> result;
00110
00111     int segStart = 0;
00112     bool inWork = smoothed[0] >= threshold;
00113
00114     auto flush = [&](int endIdx)
00115     {
00116         if (endIdx < segStart) return;
00117         const double dur =
00118             records[endIdx].elapsedSeconds -
00119             records[segStart].elapsedSeconds;

```

```

00120
00121     if (inWork && dur >= config.minWorkSeconds)
00122         result.push_back(
00123             buildInterval(rideData, segStart, endIdx, "Work"));
00124     else if (!inWork &&
00125             dur >= config.minRecoverySeconds &&
00126             !result.empty())
00127         result.push_back(
00128             buildInterval(rideData, segStart, endIdx, "Recovery"));
00129 };
00130
00131 for (int i = 1; i < n; ++i)
00132 {
00133     const bool nowWork = smoothed[i] >= threshold;
00134     if (nowWork != inWork)
00135     {
00136         flush(i - 1);
00137         segStart = i;
00138         inWork = nowWork;
00139     }
00140 }
00141 flush(n - 1);
00142
00143 return result;
00144 }
00145
00146 std::vector<Interval> IntervalDetector::removeOverlappingAndDuplicates(
00147     std::vector<Interval> intervals)
00148 {
00149     if (intervals.size() <= 1)
00150         return intervals;
00151
00152     // Sort by start time
00153     std::sort(intervals.begin(), intervals.end(),
00154         [](const Interval& a, const Interval& b) {
00155             return a.startSeconds < b.startSeconds;
00156         });
00157
00158     std::vector<Interval> result;
00159     result.reserve(intervals.size());
00160
00161     // Merge overlapping/adjacent intervals and remove duplicates
00162     const double tolerance = 1.0; // 1 second boundary tolerance for duplicates
00163
00164     for (const Interval& current : intervals)
00165     {
00166         // Skip if this interval is a duplicate of the last one
00167         // (same start/end within tolerance)
00168         if (!result.empty())
00169         {
00170             const Interval& last = result.back();
00171             if (std::abs(last.startSeconds - current.startSeconds) <= tolerance &&
00172                 std::abs(last.endSeconds - current.endSeconds) <= tolerance)
00173             {
00174                 // Duplicate detected: keep the existing one
00175                 continue;
00176             }
00177
00178             // Check for overlap: if current starts before last ends, merge them
00179             if (current.startSeconds < last.endSeconds + tolerance)
00180             {
00181                 // Merge by extending the end time and averaging metrics
00182                 Interval& merged = result.back();
00183                 merged.endSeconds = std::max(merged.endSeconds, current.endSeconds);
00184                 merged.durationSeconds = merged.endSeconds - merged.startSeconds;
00185
00186                 // Average the metrics proportionally (weighted by interval duration)
00187                 const double lastDur = last.durationSeconds;
00188                 const double currDur = current.durationSeconds;
00189                 const double totalDur = lastDur + currDur;
00190
00191                 if (totalDur > 0.0)
00192                 {
00193                     merged.averagePower = (last.averagePower * lastDur + current.averagePower *
00194 currDur) / totalDur;
00195                     merged.maximumPower = std::max(last.maximumPower, current.maximumPower);
00196                     merged.normalizedPower = (last.normalizedPower * lastDur +
00197 current.normalizedPower * currDur) / totalDur;
00198                     merged.averageHeartRate = (last.averageHeartRate * lastDur +
00199 current.averageHeartRate * currDur) / totalDur;
00200                     merged.averageCadence = (last.averageCadence * lastDur + current.averageCadence
00201 * currDur) / totalDur;
00202                     merged.averageSpeed = (last.averageSpeed * lastDur + current.averageSpeed *
00203 currDur) / totalDur;
00204                 }
00205                 continue;
00206             }
00207             result.push_back(current);
00208         }
00209     }
00210 }

```

```

00202     }
00203
00204     // No overlap/duplicate: add as new interval
00205     result.push_back(current);
00206 }
00207
00208 return result;
00209 }

```

8.8 IntervalDetector.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <string>
00007 #include <vector>
00008
00009 #include "fit/RideData.h"
00010
00014 struct Interval
00015 {
00017     double startSeconds    = 0.0;
00018
00020     double endSeconds      = 0.0;
00021
00023     double durationSeconds = 0.0;
00024
00026     double averagePower    = 0.0;
00027
00029     double maximumPower    = 0.0;
00030
00032     double normalizedPower = 0.0;
00033
00035     double averageHeartRate = 0.0;
00036
00038     double averageCadence  = 0.0;
00039
00041     double averageSpeed    = 0.0;
00042
00044     std::string label;
00045 };
00046
00050 struct IntervalDetectorConfig
00051 {
00053     double ftp              = 250.0;
00054
00056     double workThresholdPct = 0.76;
00057
00059     double minWorkSeconds   = 60.0;
00060
00062     double minRecoverySeconds = 20.0;
00063
00065     int smoothingWindow    = 30;
00066 };
00067
00074 class IntervalDetector
00075 {
00076 public:
00078     using Config = IntervalDetectorConfig;
00079
00086     static std::vector<Interval> detect(
00087         const RideData& rideData,
00088         const Config& config = Config{});
00089
00095     static std::vector<Interval> removeOverlappingAndDuplicates(
00096         std::vector<Interval> intervals);
00097
00098 private:
00099     static std::vector<double> smoothPower(
00100         const RideData& rideData, int window);
00101
00102     static Interval buildInterval(
00103         const RideData& rideData,
00104         int startIdx, int endIdx,
00105         const std::string& label);
00106 };

```

8.9 PowerCurve.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "PowerCurve.h"
00005
00006 #include <cmath>
00007 #include <vector>
00008
00009 std::vector<double> PowerCurve::standardDurations()
00010 {
00011     return {
00012         5, 10, 15, 30, 60,
00013         120, 180, 300, 600,
00014         1200, 1800, 3600, 5400, 7200
00015     };
00016 }
00017
00018 double PowerCurve::bestMeanPower(
00019     const RideData& rideData, double durationSeconds)
00020 {
00021     const auto& records = rideData.records;
00022     if (records.empty()) return 0.0;
00023
00024     std::vector<double> power;
00025     power.reserve(records.size());
00026     for (const auto& r : records)
00027         power.push_back(r.hasPower ? r.power : 0.0);
00028
00029     const int n = static_cast<int>(power.size());
00030     const int window = static_cast<int>(std::round(durationSeconds));
00031     if (window <= 0 || window > n) return 0.0;
00032
00033     double sum = 0.0;
00034     for (int i = 0; i < window; ++i)
00035         sum += power[i];
00036
00037     double best = sum;
00038     for (int i = window; i < n; ++i)
00039     {
00040         sum += power[i] - power[i - window];
00041         if (sum > best) best = sum;
00042     }
00043
00044     return best / window;
00045 }
00046
00047 std::vector<PowerCurvePoint> PowerCurve::computeStandard(
00048     const RideData& rideData)
00049 {
00050     const auto durations = standardDurations();
00051     std::vector<PowerCurvePoint> result;
00052     result.reserve(durations.size());
00053
00054     for (double d : durations)
00055     {
00056         const double p = bestMeanPower(rideData, d);
00057         if (p > 0.0)
00058             result.push_back({ d, p });
00059     }
00060
00061     return result;
00062 }

```

8.10 PowerCurve.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <vector>
00007
00008 #include "fit/RideData.h"
00009
00010 struct PowerCurvePoint
00011 {
00012     double durationSeconds;
00013     double bestPower;
00014 };
00015
00016

```



```

00027 class PowerCurve
00028 {
00029 public:
00034     static std::vector<double> standardDurations();
00035
00042     static double bestMeanPower(
00043         const RideData& rideData, double durationSeconds);
00044
00050     static std::vector<PowerCurvePoint> computeStandard(
00051         const RideData& rideData);
00052 };

```

8.11 AnalysisQueue.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "AnalysisQueue.h"
00005 #include "AnalysisWorker.h"
00006
00007 #include <QThread>
00008
00009 AnalysisQueue::AnalysisQueue(QObject* parent)
00010     : QObject(parent)
00011 {
00012     m_thread = new QThread(this);
00013     m_worker = new AnalysisWorker;
00014     m_worker->moveToThread(m_thread);
00015
00016     connect(m_worker, &AnalysisWorker::taskCompleted,
00017         this, &AnalysisQueue::onTaskCompleted);
00018     connect(m_worker, &AnalysisWorker::taskFailed,
00019         this, &AnalysisQueue::onTaskFailed);
00020
00021     m_thread->start();
00022 }
00023
00024 AnalysisQueue::~AnalysisQueue()
00025 {
00026     m_thread->quit();
00027     m_thread->wait(3000);
00028     delete m_worker;
00029 }
00030
00031 void AnalysisQueue::setDatabasePath(const QString& path)
00032 {
00033     m_dbPath = path;
00034     // Forward to worker on its own thread.
00035     QMetaObject::invokeMethod(m_worker, [this, path]() {
00036         m_worker->setDatabasePath(path);
00037     }, Qt::QueuedConnection);
00038 }
00039
00040 void AnalysisQueue::enqueue(int activityId)
00041 {
00042     m_queue.enqueue(activityId);
00043     emit pendingCountChanged(m_queue.size());
00044     if (!m_busy)
00045         processNext();
00046 }
00047
00048 int AnalysisQueue::pendingCount() const
00049 {
00050     return m_queue.size() + (m_busy ? 1 : 0);
00051 }
00052
00053 bool AnalysisQueue::isBusy() const
00054 {
00055     return m_busy;
00056 }
00057
00058 void AnalysisQueue::processNext()
00059 {
00060     if (m_queue.isEmpty() || m_busy)
00061         return;
00062
00063     m_busy = true;
00064     const int activityId = m_queue.dequeue();
00065     emit pendingCountChanged(m_queue.size());
00066
00067     QMetaObject::invokeMethod(m_worker, [this, activityId]() {
00068         m_worker->processTask(activityId);
00069     }, Qt::QueuedConnection);
00070 }

```

```

00071
00072 void AnalysisQueue::onTaskCompleted(int activityId)
00073 {
00074     m_busy = false;
00075     emit taskCompleted(activityId);
00076     processNext();
00077 }
00078
00079 void AnalysisQueue::onTaskFailed(int activityId, const QString& error)
00080 {
00081     m_busy = false;
00082     emit taskFailed(activityId, error);
00083     processNext();
00084 }

```

8.12 AnalysisQueue.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QObject>
00007 #include <QQueue>
00008 #include <QString>
00009
00010 class QThread;
00011 class AnalysisWorker;
00012
00019 class AnalysisQueue : public QObject
00020 {
00021     Q_OBJECT
00022 public:
00027     explicit AnalysisQueue(QObject* parent = nullptr);
00028     ~AnalysisQueue() override;
00029
00034     void setDatabasePath(const QString& path);
00035
00040     void enqueue(int activityId);
00041
00046     int pendingCount() const;
00047
00052     bool isBusy() const;
00053
00054 signals:
00057     void taskCompleted(int activityId);
00058
00062     void taskFailed(int activityId, const QString& error);
00063
00066     void pendingCountChanged(int count);
00067
00068 private slots:
00069     void onTaskCompleted(int activityId);
00070     void onTaskFailed(int activityId, const QString& error);
00071
00072 private:
00073     void processNext();
00074
00075     QThread* m_thread = nullptr;
00076     AnalysisWorker* m_worker = nullptr;
00077     QQueue<int> m_queue;
00078     bool m_busy = false;
00079     QString m_dbPath;
00080 };

```

8.13 AnalysisWorker.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "AnalysisWorker.h"
00005
00006 #include <QSqlDatabase>
00007 #include <QSqlError>
00008 #include <QSqlQuery>
00009 #include <QThread>
00010 #include <QUuid>
00011
00012 namespace
00013 {
00014 void applySqlitePragmas(QSqlDatabase& db)

```

```

00015 {
00016     QSqlQuery q(db);
00017     q.exec("PRAGMA journal_mode=WAL");
00018     q.exec("PRAGMA synchronous=NORMAL");
00019     q.exec("PRAGMA temp_store=MEMORY");
00020     q.exec("PRAGMA foreign_keys=ON");
00021     q.exec("PRAGMA cache_size=-20000");
00022 }
00023 }
00024
00025 #include "analysis/ClimbDetector.h"
00026 #include "analysis/ClimbMetrics.h"
00027 #include "analysis/IntervalDetector.h"
00028 #include "core/AnalysisVersions.h"
00029 #include "core/ActivityAnalysisFlags.h"
00030 #include "database/ActivityRepository.h"
00031 #include "database/ClimbRepository.h"
00032 #include "database/DatabaseConnectionGuard.h"
00033 #include "database/IntervalRepository.h"
00034 #include "model/RideDataLoader.h"
00035 #include "utils/UuidGenerator.h"
00036
00037 AnalysisWorker::AnalysisWorker(QObject* parent)
00038     : QObject(parent)
00039 {}
00040
00041 void AnalysisWorker::setDatabasePath(const QString& path)
00042 {
00043     m_dbPath = path;
00044 }
00045
00046 void AnalysisWorker::processTask(int activityId)
00047 {
00048     if (m_dbPath.isEmpty())
00049     {
00050         emit taskFailed(activityId, QStringLiteral("No database path set.));
00051         return;
00052     }
00053
00054     // Use RAII guard to ensure connection cleanup even on exceptions
00055     const QString connName = QStringLiteral("AnalysisWorker_%1_%2_%3")
00056         .arg(activityId)
00057         .arg(reinterpret_cast<quintptr>(QThread::currentThread()))
00058         .arg(QUuid::createUuid().toString(QUuid::WithoutBraces));
00059
00060     try
00061     {
00062         DatabaseConnectionGuard guard(connName);
00063         QSqlDatabase db = guard.database();
00064         db.setDatabaseName(m_dbPath);
00065
00066         if (!db.open())
00067         {
00068             emit taskFailed(activityId, db.lastError().text());
00069             return;
00070         }
00071
00072         applySqlitePragmas(db);
00073
00074         // Load ride data
00075         RideData ride = RideDataLoader::loadActivitySamples(activityId, db);
00076
00077         if (ride.records.empty())
00078         {
00079             emit taskFailed(activityId, QStringLiteral("Activity has no samples.));
00080             return;
00081         }
00082
00083         ClimbRepository climbRepo(db);
00084         IntervalRepository intervalRepo(db);
00085
00086         // Skip if already analyzed
00087         {
00088             QSqlQuery flagQ(db);
00089             flagQ.prepare("SELECT COALESCE(analysis_flags,0) FROM activities WHERE id=:id");
00090             flagQ.bindValue(":id", activityId);
00091             if (flagQ.exec() && flagQ.next())
00092             {
00093                 const int flags = flagQ.value(0).toInt();
00094                 const bool hasClimbs = (flags & ActivityAnalysisFlags::HasClimbs) != 0;
00095                 const bool hasIntervals = (flags & ActivityAnalysisFlags::HasIntervals) != 0;
00096                 if (hasClimbs && hasIntervals)
00097                 {
00098                     emit taskCompleted(activityId);
00099                     return;
00100                 }
00101             }
00102         }
00103     }

```

```

00102     }
00103
00104     // Detect and store climbs
00105     if (!climbRepo.hasClimbsEver(activityId))
00106     {
00107         ClimbDetector::Config climbCfg;
00108
00109         // Load per-activity settings if available
00110         {
00111             QSqlQuery settQ(db);
00112             settQ.prepare(
00113                 "SELECT climb_min_gain, climb_min_length, climb_min_gradient"
00114                 " FROM activity_analysis_settings WHERE activity_id=:id");
00115             settQ.bindValue(":id", activityId);
00116             if (settQ.exec() && settQ.next())
00117             {
00118                 if (!settQ.value(0).isNull()) climbCfg.minElevationGainM =
00119                     settQ.value(0).toDouble();
00120                 if (!settQ.value(1).isNull()) climbCfg.minLengthKm =
00121                     settQ.value(1).toDouble();
00122                 if (!settQ.value(2).isNull()) climbCfg.minAverageGradient =
00123                     settQ.value(2).toDouble();
00124             }
00125         }
00126
00127         const std::vector<Climb> climbs = ClimbDetector::detect(ride, climbCfg);
00128
00129         // Load deleted climbs for resurrection prevention
00130         const QList<ClimbRecord> deleted = climbRepo.deletedClimbsForActivity(activityId);
00131
00132         const std::vector<double> cumulative = ClimbDetector::buildCumulativeDistanceMeters(ride);
00133         const std::vector<double> smoothed = ClimbDetector::smoothAltitudeByDistance(
00134             ride, cumulative, climbCfg.elevationSmoothingMeters);
00135         const std::vector<double> gradient = ClimbDetector::computeLocalGradientPct(
00136             smoothed, cumulative, climbCfg.gradientWindowHalfMeters);
00137
00138         int ordinal = 1;
00139         for (const Climb& c : climbs)
00140         {
00141             // Resurrection prevention
00142             bool overlapsDeleted = false;
00143             for (const ClimbRecord& d : deleted)
00144             {
00145                 if (c.startSeconds < d.endSeconds && c.endSeconds > d.startSeconds)
00146                 {
00147                     overlapsDeleted = true;
00148                     break;
00149                 }
00150             }
00151             if (overlapsDeleted) continue;
00152
00153             ClimbRecord record;
00154             record.activityId = activityId;
00155             record.uuid = UuidGenerator::create();
00156             record.startSeconds = c.startSeconds;
00157             record.endSeconds = c.endSeconds;
00158             record.originalStartSeconds = c.startSeconds;
00159             record.originalEndSeconds = c.endSeconds;
00160             record.name = c.name;
00161             record.elevationGainM = c.elevationGainM;
00162             record.lengthKm = c.lengthKm;
00163             record.averageGradient = c.averageGradient;
00164             record.avgPower = c.averagePower;
00165             record.np = c.normalizedPower;
00166             record.avgHr = c.averageHeartRate;
00167             record.avgCadence = c.averageCadence;
00168             record.vam = c.vam;
00169             record.source = QStringLiteral("auto");
00170             record.locked = false;
00171             record.deleted = false;
00172             record.algorithmVersion = AnalysisVersions::ClimbDetection;
00173             climbRepo.insertClimb(record);
00174             ++ordinal;
00175         }
00176
00177         // Mark HasClimbs flag
00178         QSqlQuery upFlags(db);
00179         upFlags.prepare(
00180             "UPDATE activities SET analysis_flags = COALESCE(analysis_flags,0) | :f WHERE
00181             id=:id");
00182         upFlags.bindValue(":f", ActivityAnalysisFlags::HasClimbs);
00183         upFlags.bindValue(":id", activityId);
00184         upFlags.exec();
00185     }
00186
00187     // Detect and store intervals
00188     {

```

```

00185         const QList<IntervalRecord> existing = intervalRepo.listIntervalsForActivity(activityId);
00186         if (existing.isEmpty())
00187         {
00188             // Resolve FTP for this activity's date
00189             double ftp = 250.0;
00190             {
00191                 QSqlQuery ftpQ(db);
00192                 ftpQ.prepare(
00193                     "SELECT ah.ftp_watts FROM ftp_history ah"
00194                     " JOIN activities a ON a.athlete_id = ah.athlete_id"
00195                     " WHERE a.id=:id AND ah.effective_from <= SUBSTR(a.activity_start_time,1,10)"
00196                     " ORDER BY ah.effective_from DESC LIMIT 1");
00197                 ftpQ.bindValue(":id", activityId);
00198                 if (ftpQ.exec() && ftpQ.next())
00199                     ftp = ftpQ.value(0).toDouble();
00200             }
00201
00202             IntervalDetector::Config ivCfg;
00203             ivCfg.ftp = ftp;
00204             const std::vector<Interval> intervals = IntervalDetector::detect(ride, ivCfg);
00205
00206             for (const Interval& iv : intervals)
00207             {
00208                 IntervalRecord record;
00209                 record.activityId = activityId;
00210                 record.uuid = UuidGenerator::create();
00211                 record.startSample = static_cast<int>(std::round(iv.startSeconds));
00212                 record.endSample = static_cast<int>(std::round(iv.endSeconds));
00213                 record.name = QString::fromStdString(iv.label);
00214                 record.type = QString::fromStdString(iv.label);
00215                 record.avgPower = iv.averagePower;
00216                 record.np = iv.normalizedPower;
00217                 record.avgHr = iv.averageHeartRate;
00218                 record.avgCadence = iv.averageCadence;
00219                 record.source = QStringLiteral("auto");
00220                 record.locked = false;
00221                 record.deleted = false;
00222                 record.algorithmVersion = AnalysisVersions::IntervalDetection;
00223                 intervalRepo.insertInterval(record);
00224             }
00225
00226             QSqlQuery upFlags(db);
00227             upFlags.prepare(
00228                 "UPDATE activities SET analysis_flags = COALESCE(analysis_flags,0) | :f WHERE
id=:id");
00229             upFlags.bindValue(":f", ActivityAnalysisFlags::HasIntervals);
00230             upFlags.bindValue(":id", activityId);
00231             upFlags.exec();
00232         }
00233     }
00234
00235     emit taskCompleted(activityId);
00236 }
00237 catch (const std::exception& ex)
00238 {
00239     // Guard destructor will clean up connection automatically
00240     emit taskFailed(activityId, QString::fromStdString(ex.what()));
00241 }
00242 catch (...)
00243 {
00244     // Guard destructor will clean up connection automatically
00245     emit taskFailed(activityId, QStringLiteral("Unknown error during analysis."));
00246 }
00247 }

```

8.14 AnalysisWorker.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QObject>
00007 #include <QString>
00008
00015 class AnalysisWorker : public QObject
00016 {
00017     Q_OBJECT
00018 public:
00023     explicit AnalysisWorker(QObject* parent = nullptr);
00024
00029     void setDatabasePath(const QString& path);
00030
00031 public slots:

```

```

00036     void processTask(int activityId);
00037
00038 signals:
00041     void taskCompleted(int activityId);
00042
00046     void taskFailed(int activityId, const QString& error);
00047
00048 private:
00049     QString m_dbPath;
00050 };

```

8.15 SmoothingEngine.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "SmoothingEngine.h"
00005
00006 #include <cmath>
00007 #include <limits>
00008
00009 std::vector<double> SmoothingEngine::smoothPower(const RideData& ride, int seconds)
00010 {
00011     const int n = static_cast<int>(ride.records.size());
00012     std::vector<double> out(static_cast<size_t>(n), std::numeric_limits<double>::quiet_NaN());
00013     if (n == 0)
00014         return out;
00015
00016     if (seconds <= 0)
00017     {
00018         for (int i = 0; i < n; ++i)
00019         {
00020             const RideRecord& r = ride.records[static_cast<size_t>(i)];
00021             if (r.hasPower && r.power > 0.0)
00022                 out[static_cast<size_t>(i)] = r.power;
00023         }
00024         return out;
00025     }
00026
00027     const double halfWindow = seconds / 2.0;
00028     std::vector<double> prefixSum(static_cast<size_t>(n) + 1, 0.0);
00029     std::vector<int> prefixCount(static_cast<size_t>(n) + 1, 0);
00030
00031     for (int i = 0; i < n; ++i)
00032     {
00033         const RideRecord& r = ride.records[static_cast<size_t>(i)];
00034         const bool valid = r.hasPower && r.power > 0.0;
00035         prefixSum[static_cast<size_t>(i) + 1] = prefixSum[static_cast<size_t>(i)] + (valid ? r.power :
00036 0.0);
00037         prefixCount[static_cast<size_t>(i) + 1] = prefixCount[static_cast<size_t>(i)] + (valid ? 1 :
00038 0);
00039     }
00040
00041     int left = 0;
00042     int right = 0;
00043     for (int i = 0; i < n; ++i)
00044     {
00045         const RideRecord& r = ride.records[static_cast<size_t>(i)];
00046         const double t = r.elapsedSeconds;
00047
00048         while (left < n && ride.records[static_cast<size_t>(left)].elapsedSeconds < t - halfWindow)
00049             ++left;
00050         while (right < n && ride.records[static_cast<size_t>(right)].elapsedSeconds <= t + halfWindow)
00051             ++right;
00052
00053         if (!(r.hasPower && r.power > 0.0))
00054             continue;
00055
00056         const double sum = prefixSum[static_cast<size_t>(right)] -
00057 prefixSum[static_cast<size_t>(left)];
00058         const int count = prefixCount[static_cast<size_t>(right)] -
00059 prefixCount[static_cast<size_t>(left)];
00060         if (count > 0)
00061             out[static_cast<size_t>(i)] = sum / static_cast<double>(count);
00062     }
00063
00064     return out;
00065 }

```

8.16 SmoothingEngine.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <vector>
00007
00008 #include "fit/RideData.h"
00009
00013 class SmoothingEngine
00014 {
00015 public:
00022     static std::vector<double> smoothPower(const RideData& ride, int seconds);
00023 };

```

8.17 TrainingLoad.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "TrainingLoad.h"
00005
00006 #include <cmath>
00007 #include <vector>
00008
00009 double TrainingLoad::normalizedPower(const RideData& rideData)
00010 {
00011     const auto& records = rideData.records;
00012     if (records.empty()) return 0.0;
00013
00014     std::vector<double> power;
00015     power.reserve(records.size());
00016     for (const auto& r : records)
00017         power.push_back(r.hasPower ? r.power : 0.0);
00018
00019     constexpr int kWindow = 30;
00020     const int n = static_cast<int>(power.size());
00021     if (n < kWindow) return 0.0;
00022
00023     double windowSum = 0.0;
00024     for (int i = 0; i < kWindow; ++i)
00025         windowSum += power[i];
00026
00027     double sum4 = std::pow(windowSum / kWindow, 4.0);
00028     int count = 1;
00029
00030     for (int i = kWindow; i < n; ++i)
00031     {
00032         windowSum += power[i] - power[i - kWindow];
00033         sum4 += std::pow(windowSum / kWindow, 4.0);
00034         ++count;
00035     }
00036
00037     return std::pow(sum4 / count, 0.25);
00038 }
00039
00040 double TrainingLoad::intensityFactor(double np, double ftp)
00041 {
00042     if (ftp <= 0.0) return 0.0;
00043     return np / ftp;
00044 }
00045
00046 double TrainingLoad::trainingStressScore(
00047     double durationSeconds, double np, double ftp)
00048 {
00049     if (ftp <= 0.0) return 0.0;
00050     const double IF = intensityFactor(np, ftp);
00051     return (durationSeconds * np * IF) / (ftp * 3600.0) * 100.0;
00052 }
00053
00054 std::vector<FitnessMetricsPoint> TrainingLoad::fitnessTimeline(
00055     const std::vector<DailyLoadPoint>& dailyLoads,
00056     int atlDays,
00057     int ctlDays)
00058 {
00059     std::vector<FitnessMetricsPoint> out;
00060     out.reserve(dailyLoads.size());
00061
00062     if (dailyLoads.empty())
00063         return out;
00064 }

```

```

00065     const double atlAlpha = atlDays > 0 ? (2.0 / (atlDays + 1.0)) : 1.0;
00066     const double ctlAlpha = ctlDays > 0 ? (2.0 / (ctlDays + 1.0)) : 1.0;
00067
00068     double atl = 0.0;
00069     double ctl = 0.0;
00070     bool initialized = false;
00071
00072     for (const DailyLoadPoint& p : dailyLoads)
00073     {
00074         if (!initialized)
00075         {
00076             atl = p.tss;
00077             ctl = p.tss;
00078             initialized = true;
00079         }
00080         else
00081         {
00082             atl += atlAlpha * (p.tss - atl);
00083             ctl += ctlAlpha * (p.tss - ctl);
00084         }
00085
00086         FitnessMetricsPoint point;
00087         point.atl = atl;
00088         point.ctl = ctl;
00089         point.tsb = ctl - atl;
00090         out.push_back(point);
00091     }
00092
00093     return out;
00094 }

```

8.18 TrainingLoad.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <vector>
00007
00008 #include "fit/RideData.h"
00009
00013 struct DailyLoadPoint
00014 {
00016     double tss = 0.0;
00017 };
00018
00025 struct FitnessMetricsPoint
00026 {
00028     double atl = 0.0;
00029
00031     double ctl = 0.0;
00032
00034     double tsb = 0.0;
00035 };
00036
00043 class TrainingLoad
00044 {
00045 public:
00051     static double normalizedPower(const RideData& rideData);
00052
00059     static double intensityFactor(double np, double ftp);
00060
00068     static double trainingStressScore(
00069         double durationSeconds, double np, double ftp);
00070
00078     static std::vector<FitnessMetricsPoint> fitnessTimeline(
00079         const std::vector<DailyLoadPoint>& dailyLoads,
00080         int atlDays = 7,
00081         int ctlDays = 42);
00082 };

```

8.19 TrainingMetrics.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "TrainingMetrics.h"
00005
00006 #include "analysis/TrainingLoad.h"
00007

```



```

00008 TrainingMetricsSummary TrainingMetrics::compute(const RideData& rideData,
00009                                                    double ftp,
00010                                                    double durationSeconds)
00011 {
00012     TrainingMetricsSummary out;
00013
00014     if (durationSeconds <= 0.0 && !rideData.records.empty())
00015         durationSeconds = rideData.records.back().elapsedSeconds;
00016
00017     double powerSum = 0.0;
00018     int powerCount = 0;
00019     double hrSum = 0.0;
00020     int hrCount = 0;
00021
00022     for (const RideRecord& r : rideData.records)
00023     {
00024         if (r.hasPower && r.power > 0.0)
00025         {
00026             powerSum += r.power;
00027             ++powerCount;
00028         }
00029         if (r.hasHeartRate && r.heartRate > 0.0)
00030         {
00031             hrSum += r.heartRate;
00032             ++hrCount;
00033         }
00034     }
00035
00036     out.averagePower = powerCount > 0 ? (powerSum / powerCount) : 0.0;
00037     out.averageHeartRate = hrCount > 0 ? (hrSum / hrCount) : 0.0;
00038
00039     out.normalizedPower = TrainingLoad::normalizedPower(rideData);
00040     out.intensityFactor = TrainingLoad::intensityFactor(out.normalizedPower, ftp);
00041     out.trainingStressScore = TrainingLoad::trainingStressScore(durationSeconds, out.normalizedPower,
00042 ftp);
00043     if (out.averagePower > 0.0)
00044         out.variabilityIndex = out.normalizedPower / out.averagePower;
00045
00046     if (out.averageHeartRate > 0.0)
00047         out.efficiencyFactor = out.normalizedPower / out.averageHeartRate;
00048
00049     return out;
00050 }

```

8.20 TrainingMetrics.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include "fit/RideData.h"
00007
00011 struct TrainingMetricsSummary
00012 {
00014     double normalizedPower = 0.0;
00015
00017     double intensityFactor = 0.0;
00018
00020     double trainingStressScore = 0.0;
00021
00023     double variabilityIndex = 0.0;
00024
00026     double efficiencyFactor = 0.0;
00027
00029     double averagePower = 0.0;
00030
00032     double averageHeartRate = 0.0;
00033 };
00034
00038 class TrainingMetrics
00039 {
00040 public:
00048     static TrainingMetricsSummary compute(const RideData& rideData,
00049                                           double ftp,
00050                                           double durationSeconds = 0.0);
00051 };

```

8.21 WorkoutAnalyzer.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "WorkoutAnalyzer.h"
00005
00006 #include <algorithm>
00007 #include <cmath>
00008 #include <numbers>
00009
00010 static double segmentKm(double lat1, double lon1, double lat2, double lon2)
00011 {
00012     constexpr double R = 6371.0;
00013     const double dLat = (lat2 - lat1) * std::numbers::pi_v<double> / 180.0;
00014     const double dLon = (lon2 - lon1) * std::numbers::pi_v<double> / 180.0;
00015     const double mLat = (lat1 + lat2) / 2.0 * std::numbers::pi_v<double> / 180.0;
00016     return std::hypot(dLat, dLon * std::cos(mLat)) * R;
00017 }
00018
00019 WorkoutStatistics WorkoutAnalyzer::analyze(const RideData& rideData)
00020 {
00021     WorkoutStatistics stats;
00022     if (rideData.records.empty()) return stats;
00023
00024     const auto& records = rideData.records;
00025     const int n = static_cast<int>(records.size());
00026
00027     double powerSum = 0, hrSum = 0, cadSum = 0, spdSum = 0;
00028     int powerN = 0, hrN = 0, cadN = 0, spdN = 0;
00029
00030     bool prevHasGps = false;
00031     double prevLat = 0, prevLon = 0;
00032     bool usedGps = false;
00033
00034     bool prevHasAlt = false;
00035     double prevAlt = 0;
00036
00037     for (int i = 0; i < n; ++i)
00038     {
00039         const auto& r = records[i];
00040         stats.durationSeconds = r.elapsedSeconds;
00041
00042         if (r.hasPower)
00043         {
00044             powerSum += r.power;
00045             stats.maximumPower = std::max(stats.maximumPower, r.power);
00046             ++powerN;
00047         }
00048
00049         if (r.hasHeartRate)
00050         {
00051             hrSum += r.heartRate;
00052             stats.maximumHeartRate = std::max(stats.maximumHeartRate, r.heartRate);
00053             ++hrN;
00054         }
00055
00056         if (r.hasCadence)
00057         {
00058             cadSum += r.cadence;
00059             stats.maximumCadence = std::max(stats.maximumCadence, r.cadence);
00060             ++cadN;
00061         }
00062
00063         if (r.hasSpeed)
00064         {
00065             spdSum += r.speed;
00066             stats.maximumSpeed = std::max(stats.maximumSpeed, r.speed);
00067             ++spdN;
00068
00069             if (r.speed > 2.0 && i > 0)
00070             {
00071                 const double dt = r.elapsedSeconds - records[i-1].elapsedSeconds;
00072                 stats.movingTimeSeconds += std::max(0.0, dt);
00073             }
00074         }
00075
00076         if (r.hasGps)
00077         {
00078             if (prevHasGps)
00079             {
00080                 stats.totalDistanceKm +=
00081                     segmentKm(prevLat, prevLon, r.latitude, r.longitude);
00082                 usedGps = true;
00083             }
00084             prevHasGps = true;

```

```

00085         prevLat    = r.latitude;
00086         prevLon    = r.longitude;
00087     }
00088
00089     if (r.hasAltitude)
00090     {
00091         if (prevHasAlt && r.altitude > prevAlt)
00092             stats.elevationGainM += r.altitude - prevAlt;
00093         prevHasAlt = true;
00094         prevAlt    = r.altitude;
00095     }
00096 }
00097
00098 if (!usedGps && spdN > 0)
00099 {
00100     for (int i = 1; i < n; ++i)
00101     {
00102         const auto& r      = records[i];
00103         const auto& prev   = records[i-1];
00104         if (!r.hasSpeed) continue;
00105         const double dt    = r.elapsedSeconds - prev.elapsedSeconds;
00106         const double spd   = (r.speed + (prev.hasSpeed ? prev.speed : r.speed)) / 2.0;
00107         stats.totalDistanceKm += spd * std::max(0.0, dt) / 3600.0;
00108     }
00109 }
00110
00111 if (powerN) stats.averagePower    = powerSum / powerN;
00112 if (hrN)    stats.averageHeartRate = hrSum    / hrN;
00113 if (cadN)   stats.averageCadence  = cadSum   / cadN;
00114 if (spdN)   stats.averageSpeed    = spdSum   / spdN;
00115
00116 return stats;
00117 }

```

8.22 WorkoutAnalyzer.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include "fit/RideData.h"
00007
00011 struct WorkoutStatistics
00012 {
00014     double durationSeconds    = 0.0;
00015
00017     double movingTimeSeconds  = 0.0;
00018
00020     double totalDistanceKm    = 0.0;
00021
00023     double elevationGainM     = 0.0;
00024
00026     double averagePower       = 0.0;
00027
00029     double maximumPower       = 0.0;
00030
00032     double averageHeartRate   = 0.0;
00033
00035     double maximumHeartRate   = 0.0;
00036
00038     double averageCadence     = 0.0;
00039
00041     double maximumCadence     = 0.0;
00042
00044     double averageSpeed       = 0.0;
00045
00047     double maximumSpeed       = 0.0;
00048 };
00049
00053 class WorkoutAnalyzer
00054 {
00055 public:
00061     WorkoutStatistics analyze(const RideData& rideData);
00062 };

```

8.23 FitnessChartWidget.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003

```

```

00004 #include "FitnessChartWidget.h"
00005
00006 #include <QPen>
00007 #include <QtCharts/QChart>
00008 #include <QtCharts/QLineSeries>
00009 #include <QtCharts/QValueAxis>
00010
00011 #include <algorithm>
00012
00013 FitnessChartWidget::FitnessChartWidget(QWidget* parent)
00014     : QChartView(new QChart, parent)
00015 {
00016     m_ctlSeries = new QLineSeries;
00017     m_atlSeries = new QLineSeries;
00018     m_tsbSeries = new QLineSeries;
00019
00020     m_ctlSeries->setName("Fitness (CTL)");
00021     m_atlSeries->setName("Fatigue (ATL)");
00022     m_tsbSeries->setName("Form (TSB)");
00023
00024     m_ctlSeries->setPen(QPen(QColor("#16a34a"), 2.0));
00025     m_atlSeries->setPen(QPen(QColor("#dc2626"), 2.0));
00026     m_tsbSeries->setPen(QPen(QColor("#2563eb"), 1.8, Qt::DashLine));
00027
00028     m_axisX = new QValueAxis;
00029     m_axisY = new QValueAxis;
00030     m_axisX->setLabelFormat("%.0f");
00031     m_axisX->setTitleText("Day Index");
00032     m_axisY->setLabelFormat("%.0f");
00033     m_axisY->setTitleText("Load");
00034
00035     chart()->addSeries(m_ctlSeries);
00036     chart()->addSeries(m_atlSeries);
00037     chart()->addSeries(m_tsbSeries);
00038     chart()->addAxis(m_axisX, Qt::AlignBottom);
00039     chart()->addAxis(m_axisY, Qt::AlignLeft);
00040
00041     m_ctlSeries->attachAxis(m_axisX);
00042     m_ctlSeries->attachAxis(m_axisY);
00043     m_atlSeries->attachAxis(m_axisX);
00044     m_atlSeries->attachAxis(m_axisY);
00045     m_tsbSeries->attachAxis(m_axisX);
00046     m_tsbSeries->attachAxis(m_axisY);
00047
00048     chart()->setTitle("Fitness / Fatigue / Form");
00049     chart()->legend()->setVisible(true);
00050     chart()->setAnimationOptions(QChart::NoAnimation);
00051
00052     setRenderHint(QPainter::Antialiasing);
00053     setMinimumHeight(280);
00054 }
00055
00056 void FitnessChartWidget::setTimeline(const std::vector<FitnessMetricsPoint>& timeline)
00057 {
00058     m_ctlSeries->clear();
00059     m_atlSeries->clear();
00060     m_tsbSeries->clear();
00061
00062     if (timeline.empty())
00063     {
00064         m_axisX->setRange(0.0, 1.0);
00065         m_axisY->setRange(-10.0, 10.0);
00066         return;
00067     }
00068
00069     double minY = 1e9;
00070     double maxY = -1e9;
00071
00072     for (size_t i = 0; i < timeline.size(); ++i)
00073     {
00074         const double x = static_cast<double>(i);
00075         const auto& p = timeline[i];
00076
00077         m_ctlSeries->append(x, p.ctl);
00078         m_atlSeries->append(x, p.atl);
00079         m_tsbSeries->append(x, p.tsb);
00080
00081         minY = std::min(minY, std::min(p.tsb, std::min(p.ctl, p.atl)));
00082         maxY = std::max(maxY, std::max(p.tsb, std::max(p.ctl, p.atl)));
00083     }
00084
00085     if (maxY <= minY)
00086         maxY = minY + 1.0;
00087
00088     const double pad = std::max(5.0, (maxY - minY) * 0.1);
00089     m_axisX->setRange(0.0, static_cast<double>(timeline.size() - 1));
00090     m_axisY->setRange(minY - pad, maxY + pad);

```

```

00091 }
00092
00093 void FitnessChartWidget::clearChart()
00094 {
00095     m_ctlSeries->clear();
00096     m_atlSeries->clear();
00097     m_tsbSeries->clear();
00098     m_axisX->setRange(0.0, 1.0);
00099     m_axisY->setRange(-10.0, 10.0);
00100 }

```

8.24 FitnessChartWidget.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QtCharts/QChartView>
00007
00008 #include <vector>
00009
00010 #include "analysis/TrainingLoad.h"
00011
00012 class QLineSeries;
00013 class QValueAxis;
00014
00021 class FitnessChartWidget : public QChartView
00022 {
00023     Q_OBJECT
00024
00025 public:
00030     explicit FitnessChartWidget(QWidget* parent = nullptr);
00031
00036     void setTimeline(const std::vector<FitnessMetricsPoint>& timeline);
00037
00041     void clearChart();
00042
00043 private:
00044     QLineSeries* m_ctlSeries = nullptr;
00045     QLineSeries* m_atlSeries = nullptr;
00046     QLineSeries* m_tsbSeries = nullptr;
00047     QValueAxis* m_axisX = nullptr;
00048     QValueAxis* m_axisY = nullptr;
00049 };

```

8.25 Metric.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00009 enum class Metric
00010 {
00011     Power,
00012     HeartRate,
00013     Cadence,
00014     Speed,
00015     Altitude
00016 };

```

8.26 PowerCurveWidget.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "PowerCurveWidget.h"
00005
00006 #include <QtCharts/QCategoryAxis>
00007 #include <QtCharts/QAbstractSeries>
00008 #include <QtCharts/QChart>
00009 #include <QtCharts/QLineSeries>
00010 #include <QtCharts/QScatterSeries>
00011 #include <QtCharts/QValueAxis>
00012 #include <QCursor>
00013 #include <QPainter>
00014 #include <QToolTip>

```

```

00015
00016 #include <algorithm>
00017 #include <cmath>
00018
00019 static QString formatDurLabel(double seconds)
00020 {
00021     const int s = static_cast<int>(seconds);
00022     if (s < 60)
00023         return QString("%1s").arg(s);
00024     if (s < 3600)
00025         return QString("%1m").arg(s / 60);
00026     const int h = s / 3600;
00027     const int m = (s % 3600) / 60;
00028     return m > 0
00029         ? QString("%1h%2m").arg(h).arg(m, 2, 10, QChar('0'))
00030         : QString("%1h").arg(h);
00031 }
00032
00033 static std::vector<PowerCurvePoint> mergeWithDurations(const std::vector<double>& durations,
00034                                                         const std::vector<PowerCurvePoint>& points)
00035 {
00036     std::vector<PowerCurvePoint> out;
00037     out.reserve(durations.size());
00038     for (double d : durations)
00039     {
00040         double best = 0.0;
00041         for (const PowerCurvePoint& p : points)
00042         {
00043             if (std::abs(p.durationSeconds - d) < 0.5)
00044             {
00045                 best = p.bestPower;
00046                 break;
00047             }
00048         }
00049         if (best > 0.0)
00050             out.push_back({d, best});
00051     }
00052     return out;
00053 }
00054
00055 PowerCurveWidget::PowerCurveWidget(QWidget* parent)
00056     : QChartView(new QChart, parent)
00057 {
00058     chart()->setTitle("Power Duration Curve");
00059     chart()->legend()->show();
00060     chart()->setAnimationOptions(QChart::NoAnimation);
00061     chart()->setMargins(QMargins(4, 4, 4, 4));
00062     setRenderHint(QPainter::Antialiasing);
00063     setMinimumHeight(240);
00064 }
00065
00066 void PowerCurveWidget::clearChart()
00067 {
00068     chart()->removeAllSeries();
00069     const auto axes = chart()->axes();
00070     for (auto* ax : axes) chart()->removeAxis(ax);
00071     chart()->setTitle("Power Duration Curve");
00072 }
00073
00074 void PowerCurveWidget::setRideData(const RideData& rideData, double ftp)
00075 {
00076     setRideDataWithComparisons(rideData, {}, {}, {}, ftp);
00077 }
00078
00079 void PowerCurveWidget::setRideDataWithComparisons(const RideData& rideData,
00080                                                     const std::vector<PowerCurvePoint>& last90,
00081                                                     const std::vector<PowerCurvePoint>& currentYear,
00082                                                     const std::vector<PowerCurvePoint>& allTime,
00083                                                     double ftp)
00084 {
00085     clearChart();
00086
00087     const std::vector<double> durations = PowerCurve::standardDurations();
00088     auto pdcPts = mergeWithDurations(durations, PowerCurve::computeStandard(rideData));
00089     if (pdcPts.empty())
00090     {
00091         chart()->setTitle("Power Duration Curve (no power data)");
00092         return;
00093     }
00094
00095     // Use integer indices on X so QCategoryAxis gives clean duration labels
00096     auto* lineSeries = new QLineSeries;
00097     lineSeries->setName("Best Power");
00098
00099     auto* dotSeries = new QScatterSeries;
00100     dotSeries->setName("");
00101     dotSeries->setMarkerSize(9.0);

```

```

00102     dotSeries->setColor(QColor(255, 140, 0));
00103
00104     double maxPower = 0.0;
00105     int idx = 0;
00106     for (const auto& pt : pdcPts)
00107     {
00108         lineSeries->append(idx, pt.bestPower);
00109         dotSeries->append(idx, pt.bestPower);
00110         maxPower = std::max(maxPower, pt.bestPower);
00111         ++idx;
00112     }
00113
00114     chart()->addSeries(lineSeries);
00115     chart()->addSeries(dotSeries);
00116
00117     auto addComparisonSeries = [this, & durations, & maxPower](const QString& name,
00118                                                                const QColor& color,
00119                                                                Qt::PenStyle style,
00120                                                                const std::vector<PowerCurvePoint>& pts)
00121     {
00122         if (pts.empty())
00123             return;
00124
00125         const std::vector<PowerCurvePoint> merged = mergeWithDurations(durations, pts);
00126         if (merged.empty())
00127             return;
00128
00129         auto* s = new QLineSeries;
00130         s->setName(name);
00131         QPen pen(color);
00132         pen.setWidthF(1.4);
00133         pen.setStyle(style);
00134         s->setPen(pen);
00135
00136         int idx = 0;
00137         for (double d : durations)
00138         {
00139             for (const PowerCurvePoint& p : merged)
00140             {
00141                 if (std::abs(p.durationSeconds - d) < 0.5)
00142                 {
00143                     s->append(idx, p.bestPower);
00144                     maxPower = std::max(maxPower, p.bestPower);
00145                 }
00146             }
00147             ++idx;
00148         }
00149         chart()->addSeries(s);
00150     };
00151
00152     addComparisonSeries("Last 90 Days", QColor("#0ea5e9"), Qt::DashLine, last90);
00153     addComparisonSeries("Current Year", QColor("#14b8a6"), Qt::DotLine, currentYear);
00154     addComparisonSeries("All Time", QColor("#64748b"), Qt::DashDotLine, allTime);
00155
00156     // X axis: category axis with formatted duration labels
00157     auto* axisX = new QCategoryAxis;
00158     axisX->setTitleText("Duration");
00159     axisX->setGridLineVisible(true);
00160     axisX->setLabelsPosition(QCategoryAxis::AxisLabelsPositionOnValue);
00161     for (int i = 0; i < static_cast<int>(pdcPts.size()); ++i)
00162         axisX->append(formatDurLabel(pdcPts[i].durationSeconds), i);
00163     axisX->setRange(-0.5, idx - 0.5);
00164     chart()->addAxis(axisX, Qt::AlignBottom);
00165     for (QAbstractSeries* series : chart()->series())
00166         series->attachAxis(axisX);
00167
00168     // Y axis
00169     auto* axisY = new QValueAxis;
00170     axisY->setTitleText("Power (W)");
00171     axisY->setLabelFormat("%.0f");
00172     axisY->setGridLineVisible(true);
00173     axisY->setRange(0, maxPower * 1.1);
00174     chart()->addAxis(axisY, Qt::AlignLeft);
00175     for (QAbstractSeries* series : chart()->series())
00176         series->attachAxis(axisY);
00177
00178     // Optional FTP reference line
00179     if (ftp > 0.0 && ftp <= maxPower * 1.2)
00180     {
00181         auto* ftpLine = new QLineSeries;
00182         ftpLine->setName(QString("FTP (%1 W)").arg(ftp, 0, 'f', 0));
00183         ftpLine->setPen(QPen(Qt::red, 1.5, Qt::DashLine));
00184         ftpLine->append(-0.5, ftp);
00185         ftpLine->append(idx - 0.5, ftp);
00186         chart()->addSeries(ftpLine);
00187         ftpLine->attachAxis(axisX);
00188         ftpLine->attachAxis(axisY);
00189     }

```

```

00189     }
00190
00191     // Tooltip for scatter points
00192     connect(dotSeries, &QScatterSeries::hovered,
00193            this, [pdcPts](const QPointF& point, bool state)
00194            {
00195                if (state)
00196                {
00197                    const int i =
00198                        static_cast<int>(std::round(point.x()));
00199                    if (i >= 0 && i < static_cast<int>(pdcPts.size()))
00200                        QToolTip::showText(
00201                            QCursor::pos(),
00202                            formatDurLabel(pdcPts[i].durationSeconds) +
00203                                "\n" +
00204                                QString("%1 W").arg(
00205                                    pdcPts[i].bestPower, 0, 'f', 0));
00206                }
00207                else
00208                {
00209                    QToolTip::hideText();
00210                }
00211            });
00212
00213     const double p3m = PowerCurve::bestMeanPower(rideData, 180.0);
00214     const double p12m = PowerCurve::bestMeanPower(rideData, 720.0);
00215     double cp = 0.0;
00216     double wPrime = 0.0;
00217     if (p3m > 0.0 && p12m > 0.0)
00218     {
00219         cp = ((p12m * 720.0) - (p3m * 180.0)) / (720.0 - 180.0);
00220         wPrime = std::max(0.0, (p3m - cp) * 180.0);
00221     }
00222
00223     chart()->setTitle(
00224         QString("Power Duration Curve   CP %1 W   W' %2 kJ")
00225             .arg(cp, 0, 'f', 0)
00226             .arg(wPrime / 1000.0, 0, 'f', 1));
00227 }

```

8.27 PowerCurveWidget.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QtCharts/QChartView>
00007
00008 #include "analysis/PowerCurve.h"
00009 #include "fit/RideData.h"
00010
00017 class PowerCurveWidget : public QChartView
00018 {
00019     Q_OBJECT
00020 public:
00025     explicit PowerCurveWidget(QWidget* parent = nullptr);
00026
00032     void setRideData(const RideData& rideData, double ftp = 0.0);
00033
00042     void setRideDataWithComparisons(const RideData& rideData,
00043                                     const std::vector<PowerCurvePoint>& last90,
00044                                     const std::vector<PowerCurvePoint>& currentYear,
00045                                     const std::vector<PowerCurvePoint>& allTime,
00046                                     double ftp = 0.0);
00047
00051     void clearChart();
00052 };

```

8.28 PowerHistogramWidget.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "PowerHistogramWidget.h"
00005
00006 #include <QtCharts/QBarCategoryAxis>
00007 #include <QtCharts/QBarSeries>
00008 #include <QtCharts/QBarSet>
00009 #include <QtCharts/QChart>
00010 #include <QtCharts/QValueAxis>

```



```

00011 #include <QPainter>
00012
00013 #include <algorithm>
00014 #include <cmath>
00015 #include <map>
00016
00017 PowerHistogramWidget::PowerHistogramWidget(QWidget* parent)
00018     : QChartView(new QChart, parent)
00019 {
00020     chart()->setTitle("Power Distribution");
00021     chart()->legend()->hide();
00022     chart()->setAnimationOptions(QChart::NoAnimation);
00023     chart()->setMargins(QMargins(4, 4, 4, 4));
00024     setRenderHint(QPainter::Antialiasing);
00025     setMinimumHeight(200);
00026 }
00027
00028 void PowerHistogramWidget::clearChart()
00029 {
00030     chart()->removeAllSeries();
00031     const auto axes = chart()->axes();
00032     for (auto* ax : axes) chart()->removeAxis(ax);
00033     chart()->setTitle("Power Distribution");
00034 }
00035
00036 void PowerHistogramWidget::setRideData(const RideData& rideData)
00037 {
00038     clearChart();
00039
00040     constexpr double kBucketW = 25.0;    // watts per bucket
00041
00042     std::map<int, double> buckets;        // bucket index → seconds
00043
00044     const int n = static_cast<int>(rideData.records.size());
00045     for (int i = 0; i < n; ++i)
00046     {
00047         const auto& r = rideData.records[i];
00048         if (!r.hasPower || r.power <= 0.0) continue;
00049
00050         double dt = 1.0;
00051         if (n > 1)
00052         {
00053             if (i == 0)
00054                 dt = rideData.records[1].elapsedSeconds -
00055                     rideData.records[0].elapsedSeconds;
00056             else if (i == n - 1)
00057                 dt = rideData.records[n - 1].elapsedSeconds -
00058                     rideData.records[n - 2].elapsedSeconds;
00059             else
00060                 dt = (rideData.records[i + 1].elapsedSeconds -
00061                     rideData.records[i - 1].elapsedSeconds) / 2.0;
00062         }
00063         dt = std::max(0.0, dt);
00064
00065         const int bucket = static_cast<int>(r.power / kBucketW);
00066         buckets[bucket] += dt;
00067     }
00068
00069     if (buckets.empty())
00070     {
00071         chart()->setTitle("Power Distribution (no data)");
00072         return;
00073     }
00074
00075     const int minBucket = buckets.begin()->first;
00076     const int maxBucket = buckets.rbegin()->first;
00077     const int numBuckets = maxBucket - minBucket + 1;
00078
00079     auto* barSet = new QBarSet("Time (s)");
00080     QStringList categories;
00081
00082     // Show axis labels every N buckets to avoid crowding
00083     const int labelStep = std::max(1, numBuckets / 10);
00084
00085     for (int b = minBucket; b <= maxBucket; ++b)
00086     {
00087         const double sec = buckets.count(b) ? buckets.at(b) : 0.0;
00088         *barSet << sec;
00089
00090         const int relIdx = b - minBucket;
00091         categories <<
00092             (relIdx % labelStep == 0
00093              ? QString::number(static_cast<int>(b * kBucketW))
00094              : QString(""));
00095     }
00096
00097     auto* series = new QBarSeries;

```

```

00098     series->append(barSet);
00099     chart()->addSeries(series);
00100
00101     auto* axisX = new QBarCategoryAxis;
00102     axisX->append(categories);
00103     axisX->setTitleText("Power (W)");
00104     axisX->setLabelsAngle(-45);
00105     chart()->addAxis(axisX, Qt::AlignBottom);
00106     series->attachAxis(axisX);
00107
00108     auto* axisY = new QValueAxis;
00109     axisY->setTitleText("Time (s)");
00110     axisY->setLabelFormat("%.0f");
00111     axisY->setGridLineVisible(true);
00112     chart()->addAxis(axisY, Qt::AlignLeft);
00113     series->attachAxis(axisY);
00114
00115     chart()->setTitle("Power Distribution");
00116 }

```

8.29 PowerHistogramWidget.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QtCharts/QChartView>
00007
00008 #include "fit/RideData.h"
00009
00015 class PowerHistogramWidget : public QChartView
00016 {
00017     Q_OBJECT
00018 public:
00023     explicit PowerHistogramWidget(QWidget* parent = nullptr);
00024
00029     void setRideData(const RideData& rideData);
00030
00034     void clearChart();
00035 };

```

8.30 RideChartWidget.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "RideChartWidget.h"
00005
00006 #include <QColor>
00007 #include <QCursor>
00008 #include <QAction>
00009 #include <QMenu>
00010 #include <QPen>
00011 #include <QMouseEvent>
00012 #include <QPainter>
00013 #include <QResizeEvent>
00014 #include <QToolTip>
00015 #include <QWheelEvent>
00016 #include <QtCharts/QCategoryAxis>
00017 #include <QtCharts/QAreaSeries>
00018 #include <QtCharts/QChart>
00019 #include <QtCharts/QLineSeries>
00020 #include <QtCharts/QValueAxis>
00021
00022 #include "analysis/TrainingLoad.h"
00023 #include "core/zones/ColorProvider.h"
00024 #include "core/zones/ZoneCalculator.h"
00025
00026 #include <algorithm>
00027 #include <array>
00028 #include <cmath>
00029 #include <limits>
00030
00031 namespace
00032 {
00033     constexpr double kClimbHandleCenterOffsetYPx = 10.0;
00034     constexpr double kClimbHandleRadiusPx = 8.0;
00035     constexpr double kClimbHandleGrabRadiusPx = 24.0;
00036
00037     struct ClimbHandleHitResult

```

```

00038 {
00039     bool hit = false;
00040     bool startHandle = true;
00041 };
00042
00043 ClimbingHandleHitResult hitTestClimbingHandles(
00044     QChart* chart,
00045     QLineSeries* series,
00046     const QValueAxis* axisY,
00047     const Climbing& climb,
00048     const QPointF& scenePos)
00049 {
00050     if (!chart || !series || !axisY)
00051         return {};
00052
00053     const QRectF plotArea = chart->plotArea();
00054     if (!plotArea.contains(scenePos))
00055         return {};
00056
00057     QPointF startPos = chart->mapToPosition(QPointF(climb.startSeconds / 60.0, axisY->min()), series);
00058     QPointF endPos = chart->mapToPosition(QPointF(climb.endSeconds / 60.0, axisY->min()), series);
00059     startPos.setY(plotArea.top() + kClimbingHandleCenterOffsetYPx);
00060     endPos.setY(plotArea.top() + kClimbingHandleCenterOffsetYPx);
00061
00062     const double dxStart = scenePos.x() - startPos.x();
00063     const double dyStart = scenePos.y() - startPos.y();
00064     const double dxEnd = scenePos.x() - endPos.x();
00065     const double dyEnd = scenePos.y() - endPos.y();
00066     const double distStartSq = dxStart * dxStart + dyStart * dyStart;
00067     const double distEndSq = dxEnd * dxEnd + dyEnd * dyEnd;
00068     const double radiusSq = kClimbingHandleGrabRadiusPx * kClimbingHandleGrabRadiusPx;
00069
00070     if (distStartSq <= radiusSq || distEndSq <= radiusSq)
00071         return { true, distStartSq <= distEndSq };
00072
00073     return {};
00074 }
00075
00076
00077 static QString labelForMetric(Metric metric)
00078 {
00079     switch (metric)
00080     {
00081         case Metric::Power: return "Power (W)";
00082         case Metric::HeartRate: return "Heart Rate (bpm)";
00083         case Metric::Cadence: return "Cadence (rpm)";
00084         case Metric::Speed: return "Speed (km/h)";
00085         case Metric::Altitude: return "Altitude (m)";
00086     }
00087     return {};
00088 }
00089
00090 static QString labelForColorMetric(ColorMetric metric)
00091 {
00092     switch (metric)
00093     {
00094         case ColorMetric::Power: return "Power (W)";
00095         case ColorMetric::HeartRate: return "Heart Rate (bpm)";
00096         case ColorMetric::Cadence: return "Cadence (rpm)";
00097         case ColorMetric::Speed: return "Speed (km/h)";
00098         case ColorMetric::Altitude: return "Altitude (m)";
00099         case ColorMetric::Gradient: return "Gradient (%)";
00100         case ColorMetric::None: return "";
00101     }
00102     return "";
00103 }
00104
00105 static Metric metricFromColorMetric(ColorMetric metric)
00106 {
00107     switch (metric)
00108     {
00109         case ColorMetric::Power: return Metric::Power;
00110         case ColorMetric::HeartRate: return Metric::HeartRate;
00111         case ColorMetric::Cadence: return Metric::Cadence;
00112         case ColorMetric::Speed: return Metric::Speed;
00113         case ColorMetric::Altitude: return Metric::Altitude;
00114         case ColorMetric::Gradient:
00115         case ColorMetric::None:
00116             break;
00117     }
00118     return Metric::Power;
00119 }
00120
00121 static double valueForMetric(const RideRecord& record, Metric metric)
00122 {
00123     switch (metric)
00124     {

```

```

00125         case Metric::Power:      return record.power;
00126         case Metric::HeartRate:   return record.heartRate;
00127         case Metric::Cadence:     return record.cadence;
00128         case Metric::Speed:       return record.speed;
00129         case Metric::Altitude:    return record.altitude;
00130     }
00131     return 0.0;
00132 }
00133
00134 static bool isValidSample(const RideRecord& record, Metric metric)
00135 {
00136     switch (metric)
00137     {
00138         case Metric::Power:      return record.hasPower && record.power > 0.0;
00139         case Metric::HeartRate:  return record.hasHeartRate;
00140         case Metric::Cadence:    return record.hasCadence;
00141         case Metric::Speed:      return record.hasSpeed;
00142         case Metric::Altitude:   return record.hasAltitude;
00143     }
00144     return false;
00145 }
00146
00147 static int autoSmoothingSecondsForSpan(double spanMinutes)
00148 {
00149     if (spanMinutes < 5.0)    return 0;
00150     if (spanMinutes < 30.0)   return 5;
00151     if (spanMinutes < 120.0)  return 10;
00152     if (spanMinutes < 360.0)  return 30;
00153     return 60;
00154 }
00155
00156 static QColor colorForMetricValue(ColorMetric metric,
00157                                   const ColorContext& context,
00158                                   const RideRecord& record)
00159 {
00160     switch (metric)
00161     {
00162         case ColorMetric::Power:
00163             return ColorProvider::colorForPower(record.power, context.ftp);
00164         case ColorMetric::HeartRate:
00165             return ColorProvider::colorForHeartRate(record.heartRate,
00166                                                       context.heartRateMax,
00167                                                       context.heartRateRest,
00168                                                       context.thresholdHeartRate);
00169         case ColorMetric::Cadence:
00170             return ColorProvider::colorForCadence(record.cadence);
00171         case ColorMetric::Speed:
00172             return ColorProvider::colorForSpeed(record.speed);
00173         case ColorMetric::Altitude:
00174             return ColorProvider::colorForAltitude(record.altitude,
00175                                                       context.altitudeMin,
00176                                                       context.altitudeMax);
00177         case ColorMetric::Gradient:
00178         case ColorMetric::None:
00179             return QColor("#2563eb");
00180     }
00181
00182     return QColor("#2563eb");
00183 }
00184
00185 static bool hasColorMetricValue(ColorMetric metric, const RideRecord& record)
00186 {
00187     switch (metric)
00188     {
00189         case ColorMetric::Power:      return record.hasPower && record.power > 0.0;
00190         case ColorMetric::HeartRate:  return record.hasHeartRate;
00191         case ColorMetric::Cadence:    return record.hasCadence;
00192         case ColorMetric::Speed:      return record.hasSpeed;
00193         case ColorMetric::Altitude:   return record.hasAltitude;
00194         case ColorMetric::Gradient:
00195         case ColorMetric::None:      return false;
00196     }
00197
00198     return false;
00199 }
00200
00201 // Nice axis helpers
00202
00203 // Return a "nice" step size for approximately 8 ticks over `range`.
00204 static double niceStep(double range)
00205 {
00206     if (range <= 0.0) return 1.0;
00207     const double rough = range / 8.0;
00208     const double magnitude = std::pow(10.0, std::floor(std::log10(rough)));
00209     const double residual = rough / magnitude;
00210     double step;
00211     if (residual < 1.5) step = 1.0;

```

```

00212     else if (residual < 3.5) step = 2.0;
00213     else if (residual < 7.5) step = 5.0;
00214     else step = 10.0;
00215     return step * magnitude;
00216 }
00217
00218 // Expand [dataMin, dataMax] outward to "nice" multiples of the step,
00219 // keeping the axis floor at 0 when data starts near there.
00220 static void niceRange(double dataMin, double dataMax,
00221                     double& axisMin, double& axisMax)
00222 {
00223     const double range = dataMax - dataMin;
00224     const double step = niceStep(range > 0.0 ? range : dataMax);
00225     // Floor: use 0 when data starts within one step of 0, else snap down
00226     axisMin = (dataMin < step) ? 0.0 : std::floor(dataMin / step) * step;
00227     axisMax = std::ceil (dataMax / step) * step;
00228     // Guarantee at least one step of headroom
00229     if (axisMax <= axisMin) axisMax = axisMin + step;
00230 }
00231
00232 RideChartWidget::RideChartWidget(Metric metric, QWidget* parent)
00233 : QChartView(new QChart, parent)
00234 , m_metric(metric)
00235 {
00236     m_series = new QLineSeries;
00237     m_series->setUseOpenGL(false);
00238     m_series->setVisible(false);
00239
00240     m_axisX = new QValueAxis;
00241     m_axisX->setVisible(false);
00242
00243     m_axisXDisplay = new QCategoryAxis;
00244     m_axisXDisplay->setTitleText("Time");
00245     m_axisXDisplay->setGridLineVisible(true);
00246     m_axisXDisplay->setLabelsPosition(QCategoryAxis::AxisLabelsPositionOnValue);
00247
00248     m_axisY = new QValueAxis;
00249     m_axisY->setTitleText(labelForMetric(metric));
00250     m_axisY->setLabelFormat("%.0f");
00251     m_axisY->setGridLineVisible(true);
00252
00253     m_axisYOverlay = new QValueAxis;
00254     m_axisYOverlay->setVisible(false);
00255     m_axisYOverlay->setGridLineVisible(false);
00256     m_axisYOverlay->setLabelsColor(QColor("#475569"));
00257
00258     chart()->addSeries(m_series);
00259     chart()->addAxis(m_axisX, Qt::AlignBottom);
00260     chart()->addAxis(m_axisXDisplay, Qt::AlignBottom);
00261     chart()->addAxis(m_axisY, Qt::AlignLeft);
00262     chart()->addAxis(m_axisYOverlay, Qt::AlignRight);
00263     m_series->attachAxis(m_axisX);
00264     m_series->attachAxis(m_axisY);
00265
00266     chart()->setTitle(labelForMetric(metric));
00267     chart()->legend()->hide();
00268     chart()->setMargins(QMargins(4, 4, 4, 4));
00269     chart()->setAnimationOptions(QChart::NoAnimation);
00270
00271     // Horizontal-only rubber-band keeps Y-axis scale fixed
00272     setRubberBand(QChartView::HorizontalRubberBand);
00273     setRenderHint(QPainter::Antialiasing);
00274
00275     // Mouse tracking for crosshair (no button press required)
00276     setMouseTracking(true);
00277     viewport()->setMouseTracking(true);
00278
00279     // rangeChanged: clamp to [0, m_origMaxX], rebuild labels, emit for sync, rescale Y
00280     connect(m_axisX, &QValueAxis::rangeChanged,
00281         this, [this](double min, double max)
00282         {
00283             const double finalMin = std::max(0.0, min);
00284             const double finalMax = std::min(max, m_origMaxX);
00285
00286             if (finalMin != min || finalMax != max)
00287             {
00288                 QSignalBlocker blocker(m_axisX);
00289                 m_axisX->setRange(finalMin, finalMax);
00290             }
00291
00292             updateXTicks(finalMin, finalMax);
00293             updateYAxisForVisibleRange(finalMin, finalMax);
00294
00295             emit visibleRangeChanged(finalMin, finalMax);
00296
00297             if (!m_syncing)
00298                 emit xRangeChanged(finalMin, finalMax);
00299

```

```

00299         });
00300
00301         setMinimumHeight(220);
00302         setSizePolicy(QSizePolicy::Expanding, QSizePolicy::Fixed);
00303
00304     }
00305
00306     double RideChartWidget::visibleRangeStartMinutes() const
00307     {
00308         return m_axisX ? m_axisX->min() : 0.0;
00309     }
00310
00311     double RideChartWidget::visibleRangeEndMinutes() const
00312     {
00313         return m_axisX ? m_axisX->max() : 0.0;
00314     }
00315
00316     void RideChartWidget::setPowerSmoothingSeconds(int seconds)
00317     {
00318         const int normalized = std::max(0, seconds);
00319         if (m_requestedPowerSmoothingSeconds == normalized)
00320             return;
00321
00322         m_requestedPowerSmoothingSeconds = normalized;
00323         if (m_metric == Metric::Power)
00324             rebuildChart();
00325     }
00326
00327     void RideChartWidget::setAutoSmoothingEnabled(bool enabled)
00328     {
00329         if (m_autoSmoothing == enabled)
00330             return;
00331
00332         m_autoSmoothing = enabled;
00333         if (m_metric == Metric::Power)
00334             rebuildChart();
00335     }
00336
00337     void RideChartWidget::setIntervals(const std::vector<Interval>& intervals)
00338     {
00339         m_intervals = intervals;
00340         if (m_metric == Metric::Power)
00341             rebuildChart();
00342     }
00343
00344     void RideChartWidget::setClimbs(const std::vector<Climb>& climbs)
00345     {
00346         m_climbs = climbs;
00347         rebuildChart();
00348     }
00349
00350     void RideChartWidget::setMetricOverlay(ColorMetric metric, bool enabled)
00351     {
00352         m_metricOverlayMetric = metric;
00353         m_metricOverlayEnabled = enabled && metric != ColorMetric::None;
00354         rebuildChart();
00355     }
00356
00357     void RideChartWidget::setSelectedClimbIndex(int index)
00358     {
00359         if (m_selectedClimbIndex == index)
00360             return;
00361
00362         m_selectedClimbIndex = index;
00363         // Selected climb fill is rendered as a background band, so rebuild is needed
00364         // to reflect table single-click selection changes.
00365         rebuildChart();
00366     }
00367
00368     void RideChartWidget::setClimbEditingEnabled(bool enabled)
00369     {
00370         m_climbEditingEnabled = enabled;
00371         if (!enabled)
00372             m_climbHandleDrag = ClimbHandleDrag::None;
00373         scene()->invalidate(chart()->plotArea(), QGraphicsScene::ForegroundLayer);
00374     }
00375
00376     std::vector<double> RideChartWidget::computeSmoothedPowerSeries(int seconds) const
00377     {
00378         return computeSmoothedSeries(Metric::Power, seconds);
00379     }
00380
00381     std::vector<double> RideChartWidget::computeSmoothedSeries(Metric metric, int seconds) const
00382     {
00383         const int n = static_cast<int>(m_rideData.records.size());
00384         std::vector<double> out(static_cast<size_t>(n), std::numeric_limits<double>::quiet_NaN());
00385         if (n == 0)

```

```

00386         return out;
00387
00388         // Check cache: key = (metric_id * 1000 + seconds)
00389         // Only cache if seconds > 0 (raw data doesn't need caching)
00390         if (seconds > 0)
00391         {
00392             const int metricId = static_cast<int>(metric);
00393             const int cacheKey = metricId * 1000 + seconds;
00394             auto cached = m_smoothedSeriesCache.find(cacheKey);
00395             if (cached != m_smoothedSeriesCache.end())
00396                 return cached->second;
00397         }
00398
00399         if (seconds <= 0)
00400         {
00401             for (int i = 0; i < n; ++i)
00402             {
00403                 const RideRecord& record = m_rideData.records[static_cast<size_t>(i)];
00404                 if (isValidSample(record, metric))
00405                     out[static_cast<size_t>(i)] = valueForMetric(record, metric);
00406             }
00407             return out;
00408         }
00409
00410         const double halfWindow = seconds / 2.0;
00411         std::vector<double> prefixSum(static_cast<size_t>(n) + 1, 0.0);
00412         std::vector<int> prefixCount(static_cast<size_t>(n) + 1, 0);
00413
00414         for (int i = 0; i < n; ++i)
00415         {
00416             const RideRecord& record = m_rideData.records[static_cast<size_t>(i)];
00417             const bool valid = isValidSample(record, metric);
00418             prefixSum[static_cast<size_t>(i) + 1] = prefixSum[static_cast<size_t>(i)] + (valid ?
valueForMetric(record, metric) : 0.0);
00419             prefixCount[static_cast<size_t>(i) + 1] = prefixCount[static_cast<size_t>(i)] + (valid ? 1 :
0);
00420         }
00421
00422         int left = 0;
00423         int right = 0;
00424         for (int i = 0; i < n; ++i)
00425         {
00426             const RideRecord& record = m_rideData.records[static_cast<size_t>(i)];
00427             const double t = record.elapsedSeconds;
00428
00429             while (left < n && m_rideData.records[static_cast<size_t>(left)].elapsedSeconds < t -
halfWindow)
00430                 ++left;
00431             while (right < n && m_rideData.records[static_cast<size_t>(right)].elapsedSeconds <= t +
halfWindow)
00432                 ++right;
00433
00434             if (!isValidSample(record, metric))
00435                 continue;
00436
00437             const double sum = prefixSum[static_cast<size_t>(right)] -
prefixSum[static_cast<size_t>(left)];
00438             const int count = prefixCount[static_cast<size_t>(right)] -
prefixCount[static_cast<size_t>(left)];
00439             if (count > 0)
00440                 out[static_cast<size_t>(i)] = sum / static_cast<double>(count);
00441         }
00442
00443         // Store in cache
00444         const int metricId = static_cast<int>(metric);
00445         const int cacheKey = metricId * 1000 + seconds;
00446         m_smoothedSeriesCache[cacheKey] = out;
00447
00448         return out;
00449     }
00450
00451     std::vector<int> RideChartWidget::buildSampleIndices(const std::vector<double>& values,
00452                                                         int targetPointBudget) const
00453     {
00454         std::vector<int> indices;
00455         const int total = static_cast<int>(values.size());
00456         if (total == 0)
00457             return indices;
00458
00459         if (total <= targetPointBudget || targetPointBudget <= 8)
00460         {
00461             indices.reserve(static_cast<size_t>(total));
00462             for (int i = 0; i < total; ++i)
00463                 indices.push_back(i);
00464             return indices;
00465         }
00466     }

```

```

00467     const int bucketCount = std::max(1, targetPointBudget / 4);
00468     const double bucketSize = static_cast<double>(total) / static_cast<double>(bucketCount);
00469     indices.reserve(static_cast<size_t>(targetPointBudget));
00470
00471     for (int bucket = 0; bucket < bucketCount; ++bucket)
00472     {
00473         const int begin = static_cast<int>(std::floor(bucket * bucketSize));
00474         const int end = std::min(total, static_cast<int>(std::floor((bucket + 1) * bucketSize)));
00475         if (begin >= end)
00476             continue;
00477
00478         int first = -1;
00479         int last = -1;
00480         int minIdx = -1;
00481         int maxIdx = -1;
00482         double minVal = std::numeric_limits<double>::max();
00483         double maxVal = std::numeric_limits<double>::lowest();
00484
00485         for (int i = begin; i < end; ++i)
00486         {
00487             const double v = values[static_cast<size_t>(i)];
00488             if (std::isnan(v))
00489                 continue;
00490
00491             if (first < 0)
00492                 first = i;
00493             last = i;
00494             if (v < minVal)
00495             {
00496                 minVal = v;
00497                 minIdx = i;
00498             }
00499             if (v > maxVal)
00500             {
00501                 maxVal = v;
00502                 maxIdx = i;
00503             }
00504         }
00505
00506         if (first < 0)
00507             continue;
00508
00509         std::vector<int> bucketIndices = { first };
00510         if (minIdx >= 0 && minIdx != first)
00511             bucketIndices.push_back(minIdx);
00512         if (maxIdx >= 0 && maxIdx != first && maxIdx != minIdx)
00513             bucketIndices.push_back(maxIdx);
00514         if (last >= 0 && last != first && last != minIdx && last != maxIdx)
00515             bucketIndices.push_back(last);
00516
00517         std::sort(bucketIndices.begin(), bucketIndices.end());
00518         for (int idx : bucketIndices)
00519         {
00520             if (indices.empty() || indices.back() != idx)
00521                 indices.push_back(idx);
00522         }
00523     }
00524
00525     if (!indices.empty() && indices.front() != 0)
00526         indices.insert(indices.begin(), 0);
00527     if (!indices.empty() && indices.back() != total - 1)
00528         indices.push_back(total - 1);
00529     if (indices.empty())
00530     {
00531         for (int i = 0; i < total; ++i)
00532             indices.push_back(i);
00533     }
00534
00535     return indices;
00536 }
00537
00538 void RideChartWidget::clearBackgroundSeries()
00539 {
00540     for (QAreaSeries* series : m_backgroundSeries)
00541     {
00542         chart()->removeSeries(series);
00543         delete series;
00544     }
00545     m_backgroundSeries.clear();
00546 }
00547
00548 void RideChartWidget::addZoneBackgroundBands(double minY, double maxY, double maxXMinutes)
00549 {
00550     if (m_metric != Metric::Power)
00551         return;
00552
00553     const std::vector<Zone> zones = ZoneCalculator::powerZones(m_colorContext.ftp);

```



```

00554     for (const Zone& zone : zones)
00555     {
00556         const double y0 = std::clamp(zone.minValue, minY, maxY);
00557         const double y1 = std::clamp(
00558             zone.maxValue >= std::numeric_limits<double>::max() / 2.0 ? maxY : zone.maxValue,
00559             minY,
00560             maxY);
00561         if (y1 <= y0)
00562             continue;
00563
00564         auto* upper = new QLineSeries;
00565         auto* lower = new QLineSeries;
00566         upper->append(0.0, y1);
00567         upper->append(maxXMinutes, y1);
00568         lower->append(0.0, y0);
00569         lower->append(maxXMinutes, y0);
00570
00571         auto* area = new QAreaSeries(upper, lower);
00572         QColor fill = zone.color;
00573         fill.setAlpha(24);
00574         area->setColor(fill);
00575         area->setPen(Qt::NoPen);
00576
00577         chart()->addSeries(area);
00578         area->attachAxis(m_axisX);
00579         area->attachAxis(m_axisY);
00580         m_backgroundSeries.push_back(area);
00581     }
00582 }
00583
00584 void RideChartWidget::addIntervalBackgroundBands(double minY, double maxY)
00585 {
00586     if (m_metric != Metric::Power || m_intervals.empty())
00587         return;
00588
00589     for (const Interval& iv : m_intervals)
00590     {
00591         const bool isWork = iv.label == "Work";
00592         const QColor base = isWork ? QColor("#dc2626") : QColor("#0284c7");
00593
00594         auto* upper = new QLineSeries;
00595         auto* lower = new QLineSeries;
00596         const double x0 = iv.startSeconds / 60.0;
00597         const double x1 = iv.endSeconds / 60.0;
00598         upper->append(x0, maxY);
00599         upper->append(x1, maxY);
00600         lower->append(x0, minY);
00601         lower->append(x1, minY);
00602
00603         auto* area = new QAreaSeries(upper, lower);
00604         QColor fill = base;
00605         fill.setAlpha(isWork ? 18 : 10);
00606         area->setColor(fill);
00607         area->setPen(Qt::NoPen);
00608
00609         chart()->addSeries(area);
00610         area->attachAxis(m_axisX);
00611         area->attachAxis(m_axisY);
00612         m_backgroundSeries.push_back(area);
00613     }
00614 }
00615
00616 void RideChartWidget::addMetricOverlayBands(double minY, double maxY)
00617 {
00618     if (!m_metricOverlayEnabled || m_metricOverlayMetric == ColorMetric::None)
00619         return;
00620
00621     const auto& records = m_rideData.records;
00622     if (records.size() < 2)
00623         return;
00624
00625     auto quantizeColor = [](const QColor& in)
00626     {
00627         auto q = [](int c) { return (c / 32) * 32; };
00628         return QColor(q(in.red()), q(in.green()), q(in.blue()));
00629     };
00630
00631     int segmentStart = -1;
00632     QColor current;
00633
00634     auto flush = [&](int startIndex, int endIndex, const QColor& color)
00635     {
00636         if (startIndex < 0 || endIndex <= startIndex)
00637             return;
00638
00639         const double x0 = records[static_cast<size_t>(startIndex)].elapsedSeconds / 60.0;
00640         const double x1 = records[static_cast<size_t>(endIndex)].elapsedSeconds / 60.0;

```

```

00641         if (x1 <= x0)
00642             return;
00643
00644         auto* upper = new QLineSeries;
00645         auto* lower = new QLineSeries;
00646         upper->append(x0, maxY);
00647         upper->append(x1, maxY);
00648         lower->append(x0, minY);
00649         lower->append(x1, minY);
00650
00651         auto* area = new QAreaSeries(upper, lower);
00652         QColor fill = color;
00653         fill.setAlpha(26);
00654         area->setColor(fill);
00655         area->setPen(Qt::NoPen);
00656         chart()->addSeries(area);
00657         area->attachAxis(m_axisX);
00658         area->attachAxis(m_axisY);
00659         m_backgroundSeries.push_back(area);
00660     };
00661
00662     for (int i = 0; i < static_cast<int>(records.size()); ++i)
00663     {
00664         const RideRecord& r = records[static_cast<size_t>(i)];
00665         if (!hasColorMetricValue(m_metricOverlayMetric, r))
00666         {
00667             if (segmentStart >= 0)
00668             {
00669                 flush(segmentStart, i, current);
00670                 segmentStart = -1;
00671             }
00672             continue;
00673         }
00674
00675         QColor c = quantizeColor(colorForMetricValue(m_metricOverlayMetric, m_colorContext, r));
00676         if (segmentStart < 0)
00677         {
00678             segmentStart = i;
00679             current = c;
00680             continue;
00681         }
00682
00683         if (c != current)
00684         {
00685             flush(segmentStart, i, current);
00686             segmentStart = i;
00687             current = c;
00688         }
00689     }
00690
00691     if (segmentStart >= 0)
00692         flush(segmentStart, static_cast<int>(records.size()) - 1, current);
00693 }
00694
00695 void RideChartWidget::addClimbBackgroundBands(double minY, double maxY)
00696 {
00697     if (m_climbs.empty())
00698         return;
00699
00700     auto addBand = [this, minY, maxY](const Climb& climb, bool selected)
00701     {
00702         auto* upper = new QLineSeries;
00703         auto* lower = new QLineSeries;
00704         const double x0 = climb.startSeconds / 60.0;
00705         const double x1 = climb.endSeconds / 60.0;
00706         upper->append(x0, maxY);
00707         upper->append(x1, maxY);
00708         lower->append(x0, minY);
00709         lower->append(x1, minY);
00710
00711         auto* area = new QAreaSeries(upper, lower);
00712         QColor fill("#16a34a");
00713         fill.setAlpha(selected ? 54 : 20);
00714         area->setColor(fill);
00715         if (selected)
00716         {
00717             QPen border(QColor("#15803d"));
00718             border.setWidthF(1.5);
00719             area->setPen(border);
00720         }
00721         else
00722         {
00723             area->setPen(Qt::NoPen);
00724         }
00725
00726         chart()->addSeries(area);
00727         area->attachAxis(m_axisX);

```

```

00728         area->attachAxis(m_axisY);
00729         m_backgroundSeries.push_back(area);
00730     };
00731
00732     // Draw non-selected climbs first, then selected climb on top.
00733     for (int idx = 0; idx < static_cast<int>(m_climbs.size()); ++idx)
00734     {
00735         if (idx == m_selectedClimbIndex)
00736             continue;
00737         addBand(m_climbs[static_cast<size_t>(idx)], false);
00738     }
00739
00740     if (m_selectedClimbIndex >= 0 && m_selectedClimbIndex < static_cast<int>(m_climbs.size()))
00741         addBand(m_climbs[static_cast<size_t>(m_selectedClimbIndex)], true);
00742 }
00743
00744 void RideChartWidget::updateXTicks(double minVal, double maxVal)
00745 {
00746     const QStringList existing = m_axisXDisplay->categoriesLabels();
00747     for (const auto& lbl : existing)
00748         m_axisXDisplay->remove(lbl);
00749
00750     const double span = maxVal - minVal;
00751     double interval;
00752     bool showSeconds = false;
00753
00754     if (span <= 1.0) { interval = 1.0 / 6.0; showSeconds = true; }
00755     else if (span <= 3.0) { interval = 0.5; showSeconds = true; }
00756     else if (span <= 8.0) { interval = 1.0; }
00757     else if (span <= 20.0) { interval = 2.0; }
00758     else if (span <= 40.0) { interval = 5.0; }
00759     else if (span <= 90.0) { interval = 10.0; }
00760     else if (span <= 180.0) { interval = 15.0; }
00761     else if (span <= 360.0) { interval = 30.0; }
00762     else { interval = 60.0; }
00763
00764     auto formatTime = [showSeconds](double minutes) -> QString
00765     {
00766         const int totalSec = static_cast<int>(std::round(minutes * 60.0));
00767         const int h = totalSec / 3600;
00768         const int m = (totalSec % 3600) / 60;
00769         const int s = totalSec % 60;
00770         if (h > 0)
00771             return showSeconds
00772                 ? QString("%1:%2:%3").arg(h).arg(m, 2, 10, QChar('0')).arg(s, 2, 10, QChar('0'))
00773                 : QString("%1:%2").arg(h).arg(m, 2, 10, QChar('0'));
00774         else
00775             return showSeconds
00776                 ? QString("%1:%2").arg(m).arg(s, 2, 10, QChar('0'))
00777                 : QString("%1").arg(m);
00778     };
00779
00780     double firstTick = std::ceil(minVal / interval) * interval;
00781     if (firstTick < 0.0) firstTick = 0.0;
00782
00783     for (double t = firstTick; t <= maxVal + interval * 1e-6; t += interval)
00784     {
00785         if (t < 0.0) continue;
00786         const QString label = formatTime(t);
00787         if (!m_axisXDisplay->categoriesLabels().contains(label))
00788             m_axisXDisplay->append(label, t);
00789     }
00790
00791     m_axisXDisplay->setRange(minVal, maxVal);
00792 }
00793
00794 void RideChartWidget::setRideData(const RideData& rideData,
00795                                   ColorMetric colorMetric,
00796                                   const ColorContext& colorContext)
00797 {
00798     m_rideData = rideData;
00799     m_colorMetric = colorMetric;
00800     m_colorContext = colorContext;
00801     m_smoothedSeriesCache.clear();
00802     rebuildChart(false);
00803 }
00804
00805 void RideChartWidget::rebuildChart(bool preserveXRange)
00806 {
00807     if (m_isRebuilding)
00808         return;
00809
00810     m_isRebuilding = true;
00811
00812     // Invalidate Y-axis range cache since data is changing
00813     m_yAxisRangeCache.valid = false;
00814

```

```

00815     const bool hadData = m_hasData;
00816     const double previousMinX = m_axisX->min();
00817     const double previousMaxX = m_axisX->max();
00818
00819     for (QLineSeries* series : m_rawSeries)
00820     {
00821         chart()->removeSeries(series);
00822         delete series;
00823     }
00824     m_rawSeries.clear();
00825
00826     for (QLineSeries* series : m_colorSeries)
00827     {
00828         chart()->removeSeries(series);
00829         delete series;
00830     }
00831     m_colorSeries.clear();
00832
00833     for (QLineSeries* series : m_overlaySeries)
00834     {
00835         chart()->removeSeries(series);
00836         delete series;
00837     }
00838     m_overlaySeries.clear();
00839
00840     for (QLineSeries* series : m_referenceSeries)
00841     {
00842         chart()->removeSeries(series);
00843         delete series;
00844     }
00845     m_referenceSeries.clear();
00846     clearBackgroundSeries();
00847
00848     m_axisYOverlay->setVisible(false);
00849     m_axisYOverlay->setTitleText("");
00850
00851     m_tooltipPoints.clear();
00852     m_tooltipRawValues.clear();
00853     m_tooltipDisplayValues.clear();
00854
00855     if (m_rideData.records.empty())
00856     {
00857         clearChart();
00858         m_isRebuilding = false;
00859         return;
00860     }
00861
00862     constexpr int kTargetPoints = 2600;
00863     const int total = static_cast<int>(m_rideData.records.size());
00864     const double visibleSpanMinutes = hadData ? std::max(0.0, previousMaxX - previousMinX) :
m_rideData.records.back().elapsedSeconds / 60.0;
00865     m_effectivePowerSmoothingSeconds = m_autoSmoothing
? autoSmoothingSecondsForSpan(visibleSpanMinutes)
00866     : m_requestedPowerSmoothingSeconds;
00867
00868     std::vector<double> primarySeries(total, std::numeric_limits<double>::quiet_NaN());
00869     if (m_effectivePowerSmoothingSeconds > 0)
00870     {
00871         // computeSmoothedSeries handles caching internally for all metrics
00872         primarySeries = computeSmoothedSeries(m_metric, m_effectivePowerSmoothingSeconds);
00873     }
00874     else
00875     {
00876         primarySeries = computeSmoothedSeries(m_metric, 0);
00877     }
00878
00879     const std::vector<int> sampleIndices = buildSampleIndices(primarySeries, kTargetPoints);
00880
00881     QList<QPointF> probePoints;
00882     probePoints.reserve(sampleIndices.size() + 2);
00883
00884     double sum = 0.0;
00885     double minY = std::numeric_limits<double>::max();
00886     double maxY = std::numeric_limits<double>::lowest();
00887     int validCount = 0;
00888
00889     QLineSeries* rawSeries = nullptr;
00890     QLineSeries* colorSeries = nullptr;
00891     QColor currentColor;
00892     bool hadValidPrimary = false;
00893     QPointF previousPrimaryPoint;
00894
00895     auto makeRawSeries = [this]()
00896     {
00897         auto* series = new QLineSeries;
00898         QPen pen(QColor(37, 99, 235, 60));
00899         pen.setWidthF(1.0);
00900

```

```

00901         series->setPen(pen);
00902         chart()->addSeries(series);
00903         series->attachAxis(m_axisX);
00904         series->attachAxis(m_axisY);
00905         m_rawSeries.push_back(series);
00906         return series;
00907     };
00908
00909     auto makeColorSeries = [this](const QColor& color)
00910     {
00911         auto* series = new QLineSeries;
00912         QPen pen(color);
00913         pen.setWidthF(2.4);
00914         pen.setCapStyle(Qt::RoundCap);
00915         pen.setJoinStyle(Qt::RoundJoin);
00916         series->setPen(pen);
00917         chart()->addSeries(series);
00918         series->attachAxis(m_axisX);
00919         series->attachAxis(m_axisY);
00920         m_colorSeries.push_back(series);
00921         return series;
00922     };
00923
00924     const bool showRawOverlay = (m_metric == Metric::Power && m_effectivePowerSmoothingSeconds > 0);
00925
00926     for (int index : sampleIndices)
00927     {
00928         const RideRecord& record = m_rideData.records[static_cast<size_t>(index)];
00929         const bool rawValid = isValidSample(record, m_metric);
00930         const double rawValue = rawValid ? valueForMetric(record, m_metric) :
std::numeric_limits<double>::quiet_NaN();
00931
00932         const double primaryValue = primarySeries[static_cast<size_t>(index)];
00933         const bool primaryValid = !std::isnan(primaryValue);
00934
00935         if (showRawOverlay)
00936         {
00937             if (rawValid)
00938             {
00939                 if (!rawSeries)
00940                     rawSeries = makeRawSeries();
00941                 rawSeries->append(record.elapsedSeconds / 60.0, rawValue);
00942             }
00943             else
00944             {
00945                 rawSeries = nullptr;
00946             }
00947         }
00948
00949         if (!primaryValid)
00950         {
00951             colorSeries = nullptr;
00952             hadValidPrimary = false;
00953             continue;
00954         }
00955
00956         const QPointF point(record.elapsedSeconds / 60.0, primaryValue);
00957         probePoints.append(point);
00958         m_tooltipPoints.push_back(point);
00959         m_tooltipRawValues.push_back(rawValue);
00960         m_tooltipDisplayValues.push_back(primaryValue);
00961         sum += primaryValue;
00962         minY = std::min(minY, primaryValue);
00963         maxY = std::max(maxY, primaryValue);
00964         ++validCount;
00965
00966         QColor edgeColor = QColor("#2563eb");
00967         if (m_colorMetric != ColorMetric::None && hasColorMetricValue(m_colorMetric, record))
00968             edgeColor = colorForMetricValue(m_colorMetric, m_colorContext, record);
00969
00970         if (!hadValidPrimary)
00971         {
00972             colorSeries = makeColorSeries(edgeColor);
00973             colorSeries->append(point);
00974             currentColor = edgeColor;
00975         }
00976         else if (!colorSeries || currentColor != edgeColor)
00977         {
00978             colorSeries = makeColorSeries(edgeColor);
00979             colorSeries->append(previousPrimaryPoint);
00980             colorSeries->append(point);
00981             currentColor = edgeColor;
00982         }
00983         else
00984         {
00985             colorSeries->append(point);
00986         }

```

```

00987
00988     previousPrimaryPoint = point;
00989     hadValidPrimary = true;
00990 }
00991
00992 m_series->replace(probePoints);
00993
00994 const double maxX = m_rideData.records.back().elapsedSeconds / 60.0;
00995 m_origMaxX = maxX;
00996 m_hasData = validCount > 0;
00997
00998 if (m_hasData)
00999 {
01000     m_dataMinY = minY;
01001     m_dataMaxY = maxY;
01002     updateYAxis();
01003
01004     addZoneBackgroundBands(m_dataMinY, m_dataMaxY, maxX);
01005     addMetricOverlayBands(m_dataMinY, m_dataMaxY);
01006     addIntervalBackgroundBands(m_dataMinY, m_dataMaxY);
01007     addClimbBackgroundBands(m_dataMinY, m_dataMaxY);
01008
01009     if (m_metricOverlayEnabled && m_metricOverlayMetric != ColorMetric::None)
01010     {
01011         const Metric overlayMetric = metricFromColorMetric(m_metricOverlayMetric);
01012         std::vector<double> overlayValues = computeSmoothedSeries(overlayMetric, 0);
01013         const std::vector<int> overlayIndices = buildSampleIndices(overlayValues, kTargetPoints);
01014
01015         double overlayMin = std::numeric_limits<double>::max();
01016         double overlayMax = std::numeric_limits<double>::lowest();
01017         QLineSeries* currentOverlay = nullptr;
01018         QColor currentOverlayColor;
01019         QPointF previousOverlayPoint;
01020         bool hadOverlayPoint = false;
01021
01022         auto makeOverlaySeries = [this](const QColor& color)
01023         {
01024             auto* series = new QLineSeries;
01025             QPen pen(color);
01026             pen.setWidthF(1.8);
01027             pen.setCapStyle(Qt::RoundCap);
01028             pen.setJoinStyle(Qt::RoundJoin);
01029             series->setPen(pen);
01030             chart()->addSeries(series);
01031             series->attachAxis(m_axisX);
01032             series->attachAxis(m_axisYOverlay);
01033             m_overlaySeries.push_back(series);
01034             return series;
01035         };
01036
01037         for (int idx : overlayIndices)
01038         {
01039             const RideRecord& r = m_rideData.records[static_cast<size_t>(idx)];
01040             if (!isValidSample(r, overlayMetric))
01041             {
01042                 currentOverlay = nullptr;
01043                 hadOverlayPoint = false;
01044                 continue;
01045             }
01046
01047             const double y = overlayValues[static_cast<size_t>(idx)];
01048             if (std::isnan(y))
01049             {
01050                 currentOverlay = nullptr;
01051                 hadOverlayPoint = false;
01052                 continue;
01053             }
01054
01055             overlayMin = std::min(overlayMin, y);
01056             overlayMax = std::max(overlayMax, y);
01057
01058             const QPointF point(r.elapsedSeconds / 60.0, y);
01059             const QColor segColor = colorForMetricValue(m_metricOverlayMetric, m_colorContext, r);
01060
01061             if (!currentOverlay)
01062             {
01063                 currentOverlay = makeOverlaySeries(segColor);
01064                 currentOverlayColor = segColor;
01065                 currentOverlay->append(point);
01066             }
01067             else if (currentOverlayColor != segColor)
01068             {
01069                 currentOverlay = makeOverlaySeries(segColor);
01070                 currentOverlayColor = segColor;
01071                 if (hadOverlayPoint)
01072                     currentOverlay->append(previousOverlayPoint);
01073                 currentOverlay->append(point);
01074             }
01075         }
01076     }
01077 }

```

```

01074         }
01075         else
01076         {
01077             currentOverlay->append(point);
01078         }
01079
01080         previousOverlayPoint = point;
01081         hadOverlayPoint = true;
01082     }
01083
01084     if (!m_overlaySeries.empty())
01085     {
01086         // Keep metric overlay confined to the bottom third of the plot so
01087         // the altitude profile remains visually dominant.
01088         const double rawMin = overlayMin;
01089         const double rawMax = overlayMax;
01090         const double span = std::max(0.001, rawMax - rawMin);
01091
01092         // Extend the axis maximum by 2× the data span so the overlay line
01093         // sits in the lower third, but anchor ticks at the actual data
01094         // range so labels don't appear in the empty upper portion.
01095         m_axisYOverlay->setRange(rawMin, rawMax + 2.0 * span);
01096
01097         double niceMin = rawMin;
01098         double niceMax = rawMax;
01099         niceRange(rawMin, rawMax, niceMin, niceMax);
01100         const int desiredTicks = std::clamp(height() / 60, 3, 6);
01101         const double niceInterval = std::max(1.0, (niceMax - niceMin) / desiredTicks);
01102         m_axisYOverlay->setTickType(QValueAxis::TicksDynamic);
01103         m_axisYOverlay->setTickAnchor(niceMin);
01104         m_axisYOverlay->setTickInterval(niceInterval);
01105
01106         m_axisYOverlay->setTitleText(labelForColorMetric(m_metricOverlayMetric));
01107         m_axisYOverlay->setVisible(true);
01108     }
01109 }
01110
01111 if (m_metric == Metric::Power)
01112 {
01113     const double avg = sum / validCount;
01114     const double np = TrainingLoad::normalizedPower(m_rideData);
01115     auto addReferenceLine = [this, maxX](double y, const QColor& color, Qt::PenStyle style)
01116     {
01117         auto* series = new QLineSeries;
01118         QPen pen(color);
01119         pen.setWidthF(1.0);
01120         pen.setStyle(style);
01121         series->setPen(pen);
01122         series->append(0.0, y);
01123         series->append(maxX, y);
01124         chart()->addSeries(series);
01125         series->attachAxis(m_axisX);
01126         series->attachAxis(m_axisY);
01127         m_referenceSeries.push_back(series);
01128     };
01129
01130     addReferenceLine(m_colorContext.ftp, QColor("#7c3aed"), Qt::DashLine);
01131     addReferenceLine(avg, QColor("#0f766e"), Qt::DotLine);
01132     if (np > 0.0)
01133         addReferenceLine(np, QColor("#b45309"), Qt::DashDotLine);
01134
01135     // Mark best efforts by duration as vertical guide lines.
01136     const std::array<int, 9> bestDurations = { 1, 5, 15, 30, 60, 300, 600, 1200, 3600 };
01137     std::vector<double> rawPower(static_cast<size_t>(total), 0.0);
01138     std::vector<int> validMask(static_cast<size_t>(total), 0);
01139     for (int i = 0; i < total; ++i)
01140     {
01141         const RideRecord& rec = m_rideData.records[static_cast<size_t>(i)];
01142         if (rec.hasPower && rec.power > 0.0)
01143         {
01144             rawPower[static_cast<size_t>(i)] = rec.power;
01145             validMask[static_cast<size_t>(i)] = 1;
01146         }
01147     }
01148
01149     std::vector<double> prefixSum(static_cast<size_t>(total) + 1, 0.0);
01150     std::vector<int> prefixCount(static_cast<size_t>(total) + 1, 0);
01151     for (int i = 0; i < total; ++i)
01152     {
01153         prefixSum[static_cast<size_t>(i) + 1] = prefixSum[static_cast<size_t>(i)] +
01154         rawPower[static_cast<size_t>(i)];
01155         prefixCount[static_cast<size_t>(i) + 1] = prefixCount[static_cast<size_t>(i)] +
01156         validMask[static_cast<size_t>(i)];
01157     }
01158
01159     auto addBestMarker = [this](double xMinutes, double minV, double maxV, const QColor&
01160     color)

```

```

01158         {
01159             auto* series = new QLineSeries;
01160             QPen pen(color);
01161             pen.setWidthF(1.0);
01162             pen.setStyle(Qt::DashDotDotLine);
01163             series->setPen(pen);
01164             series->append(xMinutes, minV);
01165             series->append(xMinutes, maxV);
01166             chart()->addSeries(series);
01167             series->attachAxis(m_axisX);
01168             series->attachAxis(m_axisY);
01169             m_referenceSeries.push_back(series);
01170         };
01171
01172         for (int duration : bestDurations)
01173         {
01174             if (duration <= 0 || duration >= total)
01175                 continue;
01176
01177             double bestAvg = -1.0;
01178             int bestEndIndex = -1;
01179             for (int end = duration - 1; end < total; ++end)
01180             {
01181                 const int begin = end - duration + 1;
01182                 const double winSum = prefixSum[static_cast<size_t>(end) + 1] -
01183                 prefixSum[static_cast<size_t>(begin)];
01184                 const int winCount = prefixCount[static_cast<size_t>(end) + 1] -
01185                 prefixCount[static_cast<size_t>(begin)];
01186                 if (winCount <= 0)
01187                     continue;
01188
01189                 const double avgPower = winSum / winCount;
01190                 if (avgPower > bestAvg)
01191                 {
01192                     bestAvg = avgPower;
01193                     bestEndIndex = end;
01194                 }
01195             }
01196             if (bestEndIndex < 0)
01197                 continue;
01198
01199             const double xMinutes =
01200             m_rideData.records[static_cast<size_t>(bestEndIndex)].elapsedSeconds / 60.0;
01201             QColor markerColor("#1d4ed8");
01202             markerColor.setAlpha(170);
01203             addBestMarker(xMinutes, m_dataMinY, m_dataMaxY, markerColor);
01204
01205             const QString smoothingLabel = m_effectivePowerSmoothingSeconds <= 0
01206             ? QString("raw")
01207             : QString("%1s").arg(m_effectivePowerSmoothingSeconds);
01208
01209             chart()->setTitle(
01210             QString("%1 min %2 avg %3 max %4 FTP %5 NP %6 smoothing %7")
01211             .arg(labelForMetric(m_metric))
01212             .arg(minY, 0, 'f', 0)
01213             .arg(avg, 0, 'f', 0)
01214             .arg(maxY, 0, 'f', 0)
01215             .arg(m_colorContext.ftp)
01216             .arg(np, 0, 'f', 0)
01217             .arg(smoothingLabel));
01218         }
01219         else
01220         {
01221             const double avg = sum / validCount;
01222             chart()->setTitle(
01223             labelForMetric(m_metric) +
01224             QString(" min %1 avg %2 max %3 smoothing %4")
01225             .arg(minY, 0, 'f', 0)
01226             .arg(avg, 0, 'f', 0)
01227             .arg(maxY, 0, 'f', 0)
01228             .arg(m_effectivePowerSmoothingSeconds <= 0 ? QString("raw") :
01229             QString("%1s").arg(m_effectivePowerSmoothingSeconds)));
01230         }
01231         else
01232         {
01233             m_axisY->setRange(0.0, 1.0);
01234             chart()->setTitle(labelForMetric(m_metric) + " (no data)");
01235         }
01236         if (m_hasData)
01237         {
01238             if (preserveXRange && hadData && previousMaxX > previousMinX)
01239                 m_axisX->setRange(std::max(0.0, previousMinX), std::min(previousMaxX, maxX));
01240             else

```



```

01241         m_axisX->setRange(0.0, maxX);
01242     }
01243     else
01244     {
01245         m_axisX->setRange(0.0, 1.0);
01246     }
01247
01248     m_isRebuilding = false;
01249 }
01250
01251 void RideChartWidget::updateYAxis()
01252 {
01253     if (!m_hasData && m_dataMaxY <= m_dataMinY) return;
01254
01255     double axisMin, axisMax;
01256     niceRange(m_dataMinY, m_dataMaxY, axisMin, axisMax);
01257     m_axisY->setRange(axisMin, axisMax);
01258
01259     // Tick count: ~one label per 40 px of chart height, clamped 4-15.
01260     const int ticks = std::clamp(height() / 40, 4, 15);
01261     m_axisY->setTickCount(ticks);
01262 }
01263
01264 void RideChartWidget::updateYAxisForVisibleRange(double xMinMin, double xMaxMin)
01265 {
01266     if (!m_hasData || m_tooltipPoints.empty()) return;
01267
01268     // Check cache: if same range, reuse calculation
01269     if (m_yAxisRangeCache.valid &&
01270         xMinMin == m_yAxisRangeCache.cachedMinX &&
01271         xMaxMin == m_yAxisRangeCache.cachedMaxX)
01272     {
01273         m_axisY->setRange(m_yAxisRangeCache.cachedAxisMin, m_yAxisRangeCache.cachedAxisMax);
01274         return;
01275     }
01276
01277     double visMin = std::numeric_limits<double>::max();
01278     double visMax = std::numeric_limits<double>::lowest();
01279
01280     for (const QPointF& pt : m_tooltipPoints)
01281     {
01282         if (pt.x() >= xMinMin && pt.x() <= xMaxMin)
01283         {
01284             visMin = std::min(visMin, pt.y());
01285             visMax = std::max(visMax, pt.y());
01286         }
01287     }
01288
01289     if (visMin > visMax)
01290     {
01291         // No points in view -- fall back to full range
01292         updateYAxis();
01293         return;
01294     }
01295
01296     double axisMin, axisMax;
01297     niceRange(visMin, visMax, axisMin, axisMax);
01298
01299     // Cache this result for rapid zoom operations
01300     m_yAxisRangeCache.cachedMinX = xMinMin;
01301     m_yAxisRangeCache.cachedMaxX = xMaxMin;
01302     m_yAxisRangeCache.cachedAxisMin = axisMin;
01303     m_yAxisRangeCache.cachedAxisMax = axisMax;
01304     m_yAxisRangeCache.valid = true;
01305
01306     m_axisY->setRange(axisMin, axisMax);
01307
01308     const int ticks = std::clamp(height() / 40, 4, 15);
01309     m_axisY->setTickCount(ticks);
01310 }
01311
01312 void RideChartWidget::resizeEvent(QResizeEvent* event)
01313 {
01314     QChartView::resizeEvent(event);
01315     // Only update tick count when we have data; don't change range on resize.
01316     if (m_hasData)
01317     {
01318         const int ticks = std::clamp(height() / 40, 4, 15);
01319         m_axisY->setTickCount(ticks);
01320     }
01321 }
01322
01323 void RideChartWidget::clearChart()
01324 {
01325     for (QLineSeries* series : m_rawSeries)
01326     {
01327         chart()->removeSeries(series);

```

```

01328         delete series;
01329     }
01330     m_rawSeries.clear();
01331
01332     for (QLineSeries* series : m_colorSeries)
01333     {
01334         chart()->removeSeries(series);
01335         delete series;
01336     }
01337     m_colorSeries.clear();
01338
01339     for (QLineSeries* series : m_overlaySeries)
01340     {
01341         chart()->removeSeries(series);
01342         delete series;
01343     }
01344     m_overlaySeries.clear();
01345
01346     for (QLineSeries* series : m_referenceSeries)
01347     {
01348         chart()->removeSeries(series);
01349         delete series;
01350     }
01351     m_referenceSeries.clear();
01352     clearBackgroundSeries();
01353     m_series->clear();
01354     m_smoothedSeriesCache.clear();
01355     m_yAxisRangeCache.valid = false;
01356     m_tooltipPoints.clear();
01357     m_tooltipRawValues.clear();
01358     m_tooltipDisplayValues.clear();
01359     m_hasData = false;
01360     m_origMaxX = 1.0;
01361     m_crosshairX = -1.0;
01362     updateXTicks(0.0, 1.0);
01363     m_axisX->setRange(0.0, 1.0);
01364     m_axisY->setRange(0.0, 1.0);
01365     m_axisYOverlay->setVisible(false);
01366     m_axisYOverlay->setTitleText("");
01367     chart()->setTitle(labelForMetric(m_metric));
01368 }
01369
01370 void RideChartWidget::setXRange(double min, double max)
01371 {
01372     m_syncing = true;
01373     m_axisX->setRange(min, max);
01374     m_syncing = false;
01375     emit visibleRangeChanged(m_axisX->min(), m_axisX->max());
01376 }
01377
01378 void RideChartWidget::setCrosshair(double xMinutes)
01379 {
01380     m_crosshairX = xMinutes;
01381
01382     if (m_crosshairX < 0.0 || m_tooltipPoints.empty())
01383     {
01384         m_crosshairY = std::numeric_limits<double>::quiet_NaN();
01385         scene()->invalidate(chart()->plotArea(), QGraphicsScene::ForegroundLayer);
01386         return;
01387     }
01388
01389     const auto it = std::lower_bound(
01390         m_tooltipPoints.begin(),
01391         m_tooltipPoints.end(),
01392         m_crosshairX,
01393         [](const QPointF& point, double x) { return point.x() < x; });
01394
01395     size_t index = 0;
01396     if (it == m_tooltipPoints.end())
01397         index = m_tooltipPoints.size() - 1;
01398     else
01399         index = static_cast<size_t>(std::distance(m_tooltipPoints.begin(), it));
01400
01401     if (index > 0 && index < m_tooltipPoints.size())
01402     {
01403         const double currDist = std::abs(m_tooltipPoints[index].x() - m_crosshairX);
01404         const double prevDist = std::abs(m_tooltipPoints[index - 1].x() - m_crosshairX);
01405         if (prevDist < currDist)
01406             --index;
01407     }
01408
01409     m_crosshairY = m_tooltipPoints[index].y();
01410     scene()->invalidate(chart()->plotArea(), QGraphicsScene::ForegroundLayer);
01411 }
01412
01413 void RideChartWidget::resetZoom()
01414 {

```

```

01415     if (m_hasData)
01416     {
01417         m_axisX->setRange(0.0, m_origMaxX);
01418         emit visibleRangeChanged(m_axisX->min(), m_axisX->max());
01419     }
01420 }
01421
01422 void RideChartWidget::fitToData()
01423 {
01424     resetZoom();
01425 }
01426
01427 void RideChartWidget::wheelEvent(QWheelEvent* event)
01428 {
01429     if (m_newClimbCreating)
01430     {
01431         event->accept();
01432         return;
01433     }
01434
01435     if (m_climbEditingEnabled &&
01436         m_selectedClimbIndex >= 0 &&
01437         m_selectedClimbIndex < static_cast<int>(m_climbs.size()))
01438     {
01439         const Climb& climb = m_climbs[static_cast<size_t>(m_selectedClimbIndex)];
01440         const QPointF scenePos = mapToScene(event->position()).toPoint();
01441         if (hitTestClimbHandles(chart(), m_series, m_axisY, climb, scenePos).hit)
01442         {
01443             event->accept();
01444             return;
01445         }
01446     }
01447
01448     const double factor = event->angleDelta().y() > 0 ? 1.15 : 1.0 / 1.15;
01449
01450     const double lo = m_axisX->min();
01451     const double hi = m_axisX->max();
01452     const double center = (lo + hi) / 2.0;
01453     const double half = (hi - lo) / 2.0 / factor;
01454
01455     const double newMin = std::max(0.0, center - half);
01456     const double newMax = std::min(m_origMaxX, center + half);
01457
01458     if (newMax > newMin)
01459     {
01460         m_axisX->setRange(newMin, newMax);
01461         emit visibleRangeChanged(m_axisX->min(), m_axisX->max());
01462     }
01463
01464     event->accept();
01465 }
01466
01467 void RideChartWidget::mousePressEvent(QMouseEvent* event)
01468 {
01469     if (m_newClimbCreating && m_hasData && event->button() == Qt::LeftButton)
01470     {
01471         // Update current end position on press; mouseRelease will commit.
01472         const QPointF scenePos = mapToScene(event->pos());
01473         const QRectF plotArea = chart()->plotArea();
01474         const QPointF clampedPos(
01475             std::clamp(scenePos.x(), plotArea.left(), plotArea.right()),
01476             std::clamp(scenePos.y(), plotArea.top(), plotArea.bottom()));
01477         const double xMinutes = chart()->mapToValue(clampedPos, m_series).x();
01478         m_newClimbCurrentSec = std::clamp(xMinutes * 60.0, 0.0, m_origMaxX * 60.0);
01479         scene()->invalidate(chart()->plotArea(), QGraphicsScene::ForegroundLayer);
01480         event->accept();
01481         return;
01482     }
01483
01484     if (m_climbEditingEnabled &&
01485         m_hasData &&
01486         event->button() == Qt::RightButton)
01487     {
01488         const QPointF scenePos = mapToScene(event->pos());
01489         if (chart()->plotArea().contains(scenePos))
01490         {
01491             const double xMinutes = chart()->mapToValue(scenePos, m_series).x();
01492             const double anchorSec = std::clamp(xMinutes * 60.0, 0.0, m_origMaxX * 60.0);
01493
01494             QMenu menu(this);
01495             QAction* newClimbAction = menu.addAction("New Climb");
01496             QAction* selected = menu.exec(event->globalPosition().toPoint());
01497             if (selected == newClimbAction)
01498             {
01499                 m_newClimbCreating = true;
01500                 m_newClimbDragging = true; // box visible immediately after menu close
01501                 m_newClimbStartSec = anchorSec;

```

```

01502         m_newClimbCurrentSec = anchorSec;
01503         viewport()->setCursor(Qt::SizeHorCursor);
01504         setMouseTracking(true);
01505         scene()->invalidate(chart()->plotArea(), QGraphicsScene::ForegroundLayer);
01506         event->accept();
01507         return;
01508     }
01509 }
01510 }
01511
01512 if (m_climbEditingEnabled &&
01513     m_hasData &&
01514     event->button() == Qt::LeftButton &&
01515     m_selectedClimbIndex >= 0 &&
01516     m_selectedClimbIndex < static_cast<int>(m_climbs.size()))
01517 {
01518     const Climb& climb = m_climbs[static_cast<size_t>(m_selectedClimbIndex)];
01519     const QPointF scenePos = mapToScene(event->pos());
01520     const ClimbHandleHitResult hit = hitTestClimbHandles(chart(), m_series, m_axisY, climb,
scenePos);
01521     if (hit.hit)
01522     {
01523         m_climbHandleDrag = hit.startHandle ? ClimbHandleDrag::Start : ClimbHandleDrag::End;
01524         m_dragOriginalClimbStartSec = climb.startSeconds;
01525         m_dragOriginalClimbEndSec = climb.endSeconds;
01526         viewport()->setCursor(Qt::SizeHorCursor);
01527         event->accept();
01528         return;
01529     }
01530 }
01531
01532 // Left-click inside a climb band → select it
01533 if (m_hasData && !m_climbs.empty() && event->button() == Qt::LeftButton)
01534 {
01535     const QPointF scenePos = mapToScene(event->pos());
01536     if (chart()->plotArea().contains(scenePos))
01537     {
01538         const double xSec = chart()->mapToValue(scenePos, m_series).x() * 60.0;
01539         for (int i = 0; i < static_cast<int>(m_climbs.size()); ++i)
01540         {
01541             const Climb& c = m_climbs[static_cast<size_t>(i)];
01542             if (xSec >= c.startSeconds && xSec <= c.endSeconds)
01543             {
01544                 emit climbClicked(i);
01545                 event->accept();
01546                 return;
01547             }
01548         }
01549     }
01550 }
01551
01552 if (m_metric == Metric::Power &&
01553     m_hasData &&
01554     event->button() == Qt::RightButton &&
01555     (event->modifiers() & Qt::ShiftModifier))
01556 {
01557     const QPointF scenePos = mapToScene(event->pos());
01558     if (chart()->plotArea().contains(scenePos))
01559     {
01560         const double x = chart()->mapToValue(scenePos, m_series).x();
01561         m_intervalSelecting = true;
01562         m_intervalStartMin = std::clamp(x, 0.0, m_origMaxX);
01563         m_intervalEndMin = m_intervalStartMin;
01564         scene()->invalidate(chart()->plotArea(), QGraphicsScene::ForegroundLayer);
01565         event->accept();
01566         return;
01567     }
01568 }
01569
01570 if (event->button() == Qt::RightButton)
01571 {
01572     event->accept();
01573     return;
01574 }
01575
01576 QChartView::mousePressEvent(event);
01577 }
01578
01579 void RideChartWidget::mouseReleaseEvent(QMouseEvent* event)
01580 {
01581     if (m_newClimbCreating && event->button() == Qt::LeftButton)
01582     {
01583         if (m_newClimbDragging)
01584         {
01585             const double startSec = std::min(m_newClimbStartSec, m_newClimbCurrentSec);
01586             const double endSec = std::max(m_newClimbStartSec, m_newClimbCurrentSec);
01587             if (endSec - startSec >= 1.0)

```

```

01588         emit newClimbRequested(startSec, endSec);
01589     }
01590
01591     m_newClimbCreating = false;
01592     m_newClimbDragging = false;
01593     m_newClimbStartSec = -1.0;
01594     m_newClimbCurrentSec = -1.0;
01595     viewport()->unsetCursor();
01596     scene()->invalidate(chart()->plotArea(), QGraphicsScene::ForegroundLayer);
01597     event->accept();
01598     return;
01599 }
01600
01601 if (event->button() == Qt::LeftButton && m_climbHandleDrag != ClimbHandleDrag::None)
01602 {
01603     if (m_selectedClimbIndex >= 0 &&
01604         m_selectedClimbIndex < static_cast<int>(m_climbs.size()))
01605     {
01606         const Climb& climb = m_climbs[static_cast<size_t>(m_selectedClimbIndex)];
01607         if (std::abs(climb.startSeconds - m_dragOriginalClimbStartSec) > 1e-3 ||
01608             std::abs(climb.endSeconds - m_dragOriginalClimbEndSec) > 1e-3)
01609         {
01610             emit climbBoundaryEdited(
01611                 m_dragOriginalClimbStartSec,
01612                 m_dragOriginalClimbEndSec,
01613                 climb.startSeconds,
01614                 climb.endSeconds);
01615         }
01616     }
01617
01618     m_climbHandleDrag = ClimbHandleDrag::None;
01619     viewport()->unsetCursor();
01620     event->accept();
01621     return;
01622 }
01623
01624 if (m_intervalSelecting && event->button() == Qt::RightButton)
01625 {
01626     m_intervalSelecting = false;
01627     const double startMin = std::min(m_intervalStartMin, m_intervalEndMin);
01628     const double endMin = std::max(m_intervalStartMin, m_intervalEndMin);
01629     m_intervalStartMin = -1.0;
01630     m_intervalEndMin = -1.0;
01631     scene()->invalidate(chart()->plotArea(), QGraphicsScene::ForegroundLayer);
01632
01633     const double startSec = startMin * 60.0;
01634     const double endSec = endMin * 60.0;
01635     if (endSec - startSec >= 1.0)
01636         emit intervalSelectionRequested(startSec, endSec);
01637
01638     event->accept();
01639     return;
01640 }
01641
01642 if (event->button() == Qt::RightButton)
01643 {
01644     event->accept();
01645     return;
01646 }
01647
01648 QChartView::mouseReleaseEvent(event);
01649 }
01650
01651 void RideChartWidget::mouseDoubleClickEvent(QMouseEvent* event)
01652 {
01653     if (event->button() == Qt::RightButton)
01654     {
01655         event->accept();
01656         return;
01657     }
01658
01659     if (m_newClimbCreating)
01660     {
01661         event->accept();
01662         return;
01663     }
01664
01665     if (m_climbEditingEnabled &&
01666         m_selectedClimbIndex >= 0 &&
01667         m_selectedClimbIndex < static_cast<int>(m_climbs.size()))
01668     {
01669         const Climb& climb = m_climbs[static_cast<size_t>(m_selectedClimbIndex)];
01670         const QPointF scenePos = mapToScene(event->pos());
01671         if (hitTestClimbHandles(chart(), m_series, m_axisY, climb, scenePos).hit)
01672         {
01673             event->accept();
01674             return;
01675         }
01676     }

```

```

01675     }
01676 }
01677
01678 if (m_hasData)
01679 {
01680     m_axisX->setRange(0.0, m_origMaxX);
01681     emit visibleRangeChanged(m_axisX->min(), m_axisX->max());
01682 }
01683 QChartView::mouseDoubleClickEvent(event);
01684 }
01685
01686 void RideChartWidget::mouseMoveEvent(QMouseEvent* event)
01687 {
01688     if (m_newClimbCreating)
01689     {
01690         const QPointF scenePos = mapToScene(event->pos());
01691         const QRectF plotArea = chart()->plotArea();
01692         const QPointF clampedChartPos(
01693             std::clamp(scenePos.x(), plotArea.left(), plotArea.right()),
01694             std::clamp(scenePos.y(), plotArea.top(), plotArea.bottom()));
01695         const double xMinutes = chart()->mapToValue(clampedChartPos, m_series).x();
01696         m_newClimbCurrentSec = std::clamp(xMinutes * 60.0, 0.0, m_origMaxX * 60.0);
01697         scene()->invalidate(chart()->plotArea(), QGraphicsScene::ForegroundLayer);
01698         event->accept();
01699         return;
01700     }
01701
01702     if (m_climbHandleDrag != ClimbHandleDrag::None &&
01703         m_selectedClimbIndex >= 0 &&
01704         m_selectedClimbIndex < static_cast<int>(m_climbs.size()))
01705     {
01706         const QPointF chartPos = mapToScene(event->pos());
01707         const QRectF plotArea = chart()->plotArea();
01708         const QPointF clampedChartPos(
01709             std::clamp(chartPos.x(), plotArea.left(), plotArea.right()),
01710             std::clamp(chartPos.y(), plotArea.top(), plotArea.bottom()));
01711         const double xMinutes = chart()->mapToValue(clampedChartPos, m_series).x();
01712         const double xSec = std::clamp(xMinutes * 60.0, 0.0, m_origMaxX * 60.0);
01713
01714         Climb& climb = m_climbs[static_cast<size_t>(m_selectedClimbIndex)];
01715         const double oldStart = climb.startSeconds;
01716         const double oldEnd = climb.endSeconds;
01717         constexpr double kMinClimbDurationSec = 1.0;
01718
01719         if (m_climbHandleDrag == ClimbHandleDrag::Start)
01720             climb.startSeconds = std::min(xSec, climb.endSeconds - kMinClimbDurationSec);
01721         else
01722             climb.endSeconds = std::max(xSec, climb.startSeconds + kMinClimbDurationSec);
01723
01724         (void)oldStart;
01725         (void)oldEnd;
01726         scene()->invalidate(chart()->plotArea(), QGraphicsScene::ForegroundLayer);
01727         event->accept();
01728         return;
01729     }
01730
01731     if (m_climbEditingEnabled &&
01732         m_selectedClimbIndex >= 0 &&
01733         m_selectedClimbIndex < static_cast<int>(m_climbs.size()))
01734     {
01735         const Climb& climb = m_climbs[static_cast<size_t>(m_selectedClimbIndex)];
01736         const QPointF scenePos = mapToScene(event->pos());
01737         const bool hoverHandle = hitTestClimbHandles(chart(), m_series, m_axisY, climb, scenePos).hit();
01738         if (hoverHandle)
01739         {
01740             viewport()->setCursor(Qt::SizeHorCursor);
01741             event->accept();
01742             return;
01743         }
01744         viewport()->unsetCursor();
01745     }
01746
01747     if (m_hasData)
01748     {
01749         const QPointF scenePos = mapToScene(event->pos());
01750         if (chart()->plotArea().contains(scenePos))
01751         {
01752             const double dataX = chart()->mapToValue(scenePos, m_series).x();
01753             m_crosshairX = std::clamp(dataX, 0.0, m_origMaxX);
01754             m_crosshairY = chart()->mapToValue(scenePos, m_series).y();
01755
01756             if (m_intervalSelecting)
01757             {
01758                 m_intervalEndMin = m_crosshairX;
01759                 scene()->invalidate(chart()->plotArea(), QGraphicsScene::ForegroundLayer);
01760                 event->accept();
01761                 return;

```

```

01762     }
01763
01764     emit crosshairMoved(m_crosshairX);
01765     scene()->invalidate(chart()->plotArea(), QGraphicsScene::ForegroundLayer);
01766
01767     if (!m_tooltipPoints.empty())
01768     {
01769         const auto it = std::lower_bound(
01770             m_tooltipPoints.begin(),
01771             m_tooltipPoints.end(),
01772             m_crosshairX,
01773             [](const QPointF& point, double x) { return point.x() < x; });
01774
01775         size_t index = 0;
01776         if (it == m_tooltipPoints.end())
01777             index = m_tooltipPoints.size() - 1;
01778         else
01779             index = static_cast<size_t>(std::distance(m_tooltipPoints.begin(), it));
01780
01781         if (index > 0 && index < m_tooltipPoints.size())
01782         {
01783             const double currDist = std::abs(m_tooltipPoints[index].x() - m_crosshairX);
01784             const double prevDist = std::abs(m_tooltipPoints[index - 1].x() - m_crosshairX);
01785             if (prevDist < currDist)
01786                 --index;
01787         }
01788
01789         const QPointF& point = m_tooltipPoints[index];
01790         m_crosshairY = point.y();
01791         const int totalSec = static_cast<int>(point.x() * 60.0);
01792         const int h = totalSec / 3600;
01793         const int m = (totalSec % 3600) / 60;
01794         const int s = totalSec % 60;
01795         const double visibleMinutes = m_axisX->max() - m_axisX->min();
01796         const bool showSeconds = visibleMinutes <= 5.0;
01797
01798         QString timeStr;
01799         if (h > 0)
01800             timeStr = showSeconds
01801                 ? QString("%1:%2:%3").arg(h).arg(m, 2, 10, QChar('0')).arg(s, 2, 10,
01802                     QChar('0'))
01803                 : QString("%1:%2").arg(h).arg(m, 2, 10, QChar('0'));
01804         else
01805             timeStr = showSeconds
01806                 ? QString("%1:%2").arg(m).arg(s, 2, 10, QChar('0'))
01807                 : QString("%1 min").arg(m);
01808
01809         size_t nearestRecordIndex = 0;
01810         if (!m_rideData.records.empty())
01811         {
01812             const auto rit = std::lower_bound(
01813                 m_rideData.records.begin(),
01814                 m_rideData.records.end(),
01815                 point.x() * 60.0,
01816                 [](const RideRecord& rec, double sec) { return rec.elapsedSeconds < sec; });
01817             if (rit == m_rideData.records.end())
01818                 nearestRecordIndex = m_rideData.records.size() - 1;
01819             else
01820                 nearestRecordIndex =
01821                     static_cast<size_t>(std::distance(m_rideData.records.begin(), rit));
01822             if (nearestRecordIndex > 0 && nearestRecordIndex < m_rideData.records.size())
01823             {
01824                 const double currDist =
01825                     std::abs(m_rideData.records[nearestRecordIndex].elapsedSeconds - point.x() * 60.0);
01826                 const double prevDist = std::abs(m_rideData.records[nearestRecordIndex -
01827                     1].elapsedSeconds - point.x() * 60.0);
01828                 if (prevDist < currDist)
01829                     --nearestRecordIndex;
01830             }
01831
01832             const RideRecord& rec = m_rideData.records[nearestRecordIndex];
01833
01834             double rolling30Power = 0.0;
01835             int rollingCount = 0;
01836             const size_t startIdx = nearestRecordIndex > 30 ? (nearestRecordIndex - 30) : 0;
01837             for (size_t i = startIdx; i <= nearestRecordIndex; ++i)
01838             {
01839                 const RideRecord& rp = m_rideData.records[i];
01840                 if (rp.hasPower && rp.power > 0.0)
01841                 {
01842                     rolling30Power += rp.power;
01843                     ++rollingCount;
01844                 }
01845             }
01846             if (rollingCount > 0)
01847                 rolling30Power /= static_cast<double>(rollingCount);

```

```

01845
01846         QColor zoneColor = ColorProvider::colorForPower(rec.power, m_colorContext.ftp);
01847         const std::vector<Zone> powerZones = ZoneCalculator::powerZones(m_colorContext.ftp);
01848         QString zoneLabel = "-";
01849         for (size_t zi = 0; zi < powerZones.size(); ++zi)
01850         {
01851             const Zone& z = powerZones[zi];
01852             if (rec.power >= z.minValue && rec.power < z.maxValue)
01853             {
01854                 zoneLabel = QString("%1 %2").arg(static_cast<int>(zi) + 1).arg(z.name);
01855                 break;
01856             }
01857         }
01858
01859         QString tooltip;
01860         tooltip += QString("Time: %1").arg(timeStr);
01861         tooltip += QString("\nPower: %1 W").arg(rec.hasPower ? QString::number(rec.power, 'f',
01862         0) : "-");
01863         tooltip += QString("\n30s Power: %1 W").arg(rollingCount > 0 ?
01864         QString::number(rolling30Power, 'f', 0) : "-");
01865         tooltip += QString("\nHR: %1 bpm").arg(rec.hasHeartRate ?
01866         QString::number(rec.heartRate, 'f', 0) : "-");
01867         tooltip += QString("\nCadence: %1 rpm").arg(rec.hasCadence ?
01868         QString::number(rec.cadence, 'f', 0) : "-");
01869         tooltip += QString("\nSpeed: %1 km/h").arg(rec.hasSpeed ? QString::number(rec.speed,
01870         'f', 1) : "-");
01871         tooltip += QString("\nAltitude: %1 m").arg(rec.hasAltitude ?
01872         QString::number(rec.altitude, 'f', 0) : "-");
01873         tooltip += QString("\nZone: %1").arg(zoneLabel);
01874         tooltip += QString("\nColor: %1").arg(zoneColor.name());
01875
01876         if (m_metric == Metric::Power && m_effectivePowerSmoothingSeconds > 0)
01877             tooltip += QString("\nSmoothed: %1 W").arg(m_tooltipDisplayValues[index], 0, 'f',
01878         0);
01879
01880         QToolTip::showText(QCursor::pos(), tooltip, this);
01881     }
01882 }
01883
01884 void RideChartWidget::mouseMoveEvent(QMouseEvent* event);
01885
01886 void RideChartWidget::leaveEvent(QEvent* event)
01887 {
01888     if (m_newClimbCreating && !m_newClimbDragging)
01889     {
01890         m_newClimbCreating = false;
01891         m_newClimbStartSec = -1.0;
01892         m_newClimbCurrentSec = -1.0;
01893     }
01894
01895     viewport()->unsetCursor();
01896     m_crosshairX = -1.0;
01897     m_crosshairY = std::numeric_limits<double>::quiet_NaN();
01898     QToolTip::hideText();
01899     emit crosshairMoved(-1.0);
01900     scene()->invalidate(chart()->plotArea(), QGraphicsScene::ForegroundLayer);
01901     RideChartWidget::leaveEvent(event);
01902 }
01903
01904 void RideChartWidget::drawForeground(QPainter* painter, const QRectF&)
01905 {
01906     if (!m_hasData) return;
01907
01908     const QRectF plotArea = chart()->plotArea();
01909     painter->save();
01910
01911     // --- New-climb creation overlay (line + box) -----
01912     if (m_newClimbCreating && m_newClimbStartSec >= 0.0)
01913     {
01914         const double x0sec = std::min(m_newClimbStartSec, m_newClimbCurrentSec);
01915         const double x1sec = std::max(m_newClimbStartSec, m_newClimbCurrentSec);
01916         const QPointF p0 = chart()->mapToPosition(QPointF(x0sec / 60.0, m_axisY->min()), m_series);
01917         const QPointF p1 = chart()->mapToPosition(QPointF(x1sec / 60.0, m_axisY->min()), m_series);
01918         const double px0 = std::clamp(p0.x(), plotArea.left(), plotArea.right());
01919         const double px1 = std::clamp(p1.x(), plotArea.left(), plotArea.right());
01920
01921         painter->setPen(QPen(QColor("#16a34a"), 2.0, Qt::SolidLine));
01922
01923         constexpr double kBoxThreshold = 4.0; // px -- show box once wider than this
01924         if (px1 - px0 < kBoxThreshold)
01925         {
01926             // Just a vertical line at anchor
01927             painter->drawLine(QLineF(px0, plotArea.top(), px0, plotArea.bottom()));
01928         }
01929         else
01930         {
01931

```



```

01925         QColor fill("#16a34a");
01926         fill.setAlpha(45);
01927         const QRectF box(QPointF(px0, plotArea.top()), QPointF(px1, plotArea.bottom()));
01928         painter->fillRect(box, fill);
01929         painter->drawRect(box);
01930     }
01931
01932     painter->restore();
01933     return; // skip crosshair while creating
01934 }
01935 // -----
01936
01937 if (m_crosshairX < 0.0)
01938 {
01939     painter->restore();
01940     return;
01941 }
01942
01943 const QPointF chartPos =
01944     chart()->mapToPosition(QPointF(m_crosshairX, m_axisY->min()), m_series);
01945
01946 if (chartPos.x() < plotArea.left() || chartPos.x() > plotArea.right())
01947 {
01948     painter->restore();
01949     return;
01950 }
01951
01952 painter->setPen(QPen(QColor(220, 220, 220, 200), 1, Qt::DashLine));
01953 painter->drawLine(QLineF(chartPos.x(), plotArea.top(),
01954                         chartPos.x(), plotArea.bottom()));
01955
01956 if (!std::isnan(m_crosshairY))
01957 {
01958     const QPointF yPos = chart()->mapToPosition(QPointF(m_axisX->min(), m_crosshairY), m_series);
01959     if (yPos.y() >= plotArea.top() && yPos.y() <= plotArea.bottom())
01960         painter->drawLine(QLineF(plotArea.left(), yPos.y(), plotArea.right(), yPos.y()));
01961 }
01962
01963 if (m_intervalSelecting && m_intervalStartMin >= 0.0 && m_intervalEndMin >= 0.0)
01964 {
01965     const double x0 = std::min(m_intervalStartMin, m_intervalEndMin);
01966     const double x1 = std::max(m_intervalStartMin, m_intervalEndMin);
01967     const QPointF p0 = chart()->mapToPosition(QPointF(x0, m_axisY->min()), m_series);
01968     const QPointF p1 = chart()->mapToPosition(QPointF(x1, m_axisY->min()), m_series);
01969     const QRectF selectionRect(
01970         QPointF(std::max(plotArea.left(), p0.x()), plotArea.top()),
01971         QPointF(std::min(plotArea.right(), p1.x()), plotArea.bottom()));
01972
01973     QColor fill("#0284c7");
01974     fill.setAlpha(30);
01975     painter->fillRect(selectionRect.normalized(), fill);
01976     painter->setPen(QPen(QColor("#0284c7"), 1.0, Qt::DashLine));
01977     painter->drawRect(selectionRect.normalized());
01978 }
01979
01980 if (m_selectedClimbIndex >= 0 && m_selectedClimbIndex < static_cast<int>(m_climbs.size()))
01981 {
01982     const Climbs& climb = m_climbs[static_cast<size_t>(m_selectedClimbIndex)];
01983     const QPointF p0 = chart()->mapToPosition(QPointF(climb.startSeconds / 60.0, m_axisY->min()),
01984 m_series);
01985     const QPointF p1 = chart()->mapToPosition(QPointF(climb.endSeconds / 60.0, m_axisY->min()),
01986 m_series);
01987     if (p1.x() >= plotArea.left() && p0.x() <= plotArea.right())
01988     {
01989         painter->setPen(QPen(QColor("#16a34a"), 1.5, Qt::SolidLine));
01990         painter->setBrush(Qt::NoBrush);
01991         painter->drawLine(QLineF(p0.x(), plotArea.top(), p0.x(), plotArea.bottom()));
01992         painter->drawLine(QLineF(p1.x(), plotArea.top(), p1.x(), plotArea.bottom()));
01993
01994         if (m_climbEditingEnabled)
01995         {
01996             painter->setBrush(QColor("#15803d"));
01997             painter->setPen(QPen(Qt::white, 1.0));
01998             painter->drawEllipse(QPointF(p0.x(), plotArea.top() + kClimbHandleCenterOffsetYPx),
01999 kClimbHandleRadiusPx, kClimbHandleRadiusPx);
02000             painter->drawEllipse(QPointF(p1.x(), plotArea.top() + kClimbHandleCenterOffsetYPx),
02001 kClimbHandleRadiusPx, kClimbHandleRadiusPx);
02002         }
02003     }
02004 }
02005
02006 painter->restore();
02007 }

```

8.31 RideChartWidget.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QtCharts/QChartView>
00007
00008 #include <limits>
00009 #include <unordered_map>
00010 #include <vector>
00011
00012 #include "analysis/IntervalDetector.h"
00013 #include "analysis/ClimbMetrics.h"
00014 #include "charts/Metric.h"
00015 #include "core/zones/ZoneDefinition.h"
00016 #include "fit/RideData.h"
00017
00018 class QLineSeries;
00019 class QAreaSeries;
00020 class QValueAxis;
00021 class QCategoryAxis;
00022
00029 class RideChartWidget : public QChartView
00030 {
00031     Q_OBJECT
00032
00033 public:
00039     explicit RideChartWidget(Metric metric, QWidget* parent = nullptr);
00040
00047     void setRideData(const RideData& rideData,
00048                     ColorMetric colorMetric = ColorMetric::None,
00049                     const ColorContext& colorContext = {});
00050
00055     void setPowerSmoothingSeconds(int seconds);
00056
00061     void setAutoSmoothingEnabled(bool enabled);
00062
00067     void setIntervals(const std::vector<Interval>& intervals);
00068
00073     void setClimbs(const std::vector<Climb>& climbs);
00074
00080     void setMetricOverlay(ColorMetric metric, bool enabled);
00081
00086     void setSelectedClimbIndex(int index);
00087
00092     void setClimbEditingEnabled(bool enabled);
00093
00097     void clearChart();
00098
00103     bool hasData() const { return m_hasData; }
00104
00109     double visibleRangeStartMinutes() const;
00111     double visibleRangeEndMinutes() const;
00112
00113 public slots:
00117     void setXRange(double min, double max);
00118
00121     void setCrosshair(double xMinutes);
00122
00124     void resetZoom();
00125
00127     void fitToData();
00128
00129 signals:
00133     void xRangeChanged(double min, double max);
00134
00138     void visibleRangeChanged(double startMinutes, double endMinutes);
00139
00142     void crosshairMoved(double xMinutes);
00143
00147     void intervalSelectionRequested(double startSeconds, double endSeconds);
00148
00154     void climbBoundaryEdited(
00155         double oldStartSeconds,
00156         double oldEndSeconds,
00157         double newStartSeconds,
00158         double newEndSeconds);
00159
00163     void newClimbRequested(double startSeconds, double endSeconds);
00164
00167     void climbClicked(int climbIndex);
00168
00169 protected:
00171     void wheelEvent(QWheelEvent* event) override;

```

```

00172
00174 void mousePressEvent(QMouseEvent* event) override;
00175
00177 void mouseReleaseEvent(QMouseEvent* event) override;
00178
00180 void mouseDoubleClickEvent(QMouseEvent* event) override;
00181
00183 void mouseMoveEvent(QMouseEvent* event) override;
00184
00186 void leaveEvent(QEvent* event) override;
00187
00189 void drawForeground(QPainter* painter, const QRectF& rect) override;
00190
00192 void resizeEvent(QResizeEvent* event) override;
00193
00194 private:
00195 void rebuildChart(bool preserveXRange = true);
00196 std::vector<double> computeSmoothedSeries(Metric metric, int seconds) const;
00197 std::vector<double> computeSmoothedPowerSeries(int seconds) const;
00198 std::vector<int> buildSampleIndices(const std::vector<double>& values, int targetPointBudget)
00199 const;
00200 void clearBackgroundSeries();
00201 void addZoneBackgroundBands(double minY, double maxY, double maxXMinutes);
00202 void addMetricOverlayBands(double minY, double maxY);
00203 void addIntervalBackgroundBands(double minY, double maxY);
00204 void addClimbBackgroundBands(double minY, double maxY);
00205 void updateXTicks(double minVal, double maxVal);
00206 void updateYAxis(); // recalculates nice range + tick count from full data
00207 void updateYAxisForVisibleRange(double xMinMin, double xMaxMin); // rescale to visible window
00208
00209 Metric m_metric;
00210 QLineSeries* m_series = nullptr;
00211 std::vector<QLineSeries*> m_rawSeries;
00212 std::vector<QLineSeries*> m_colorSeries;
00213 std::vector<QLineSeries*> m_overlaySeries;
00214 std::vector<QLineSeries*> m_referenceSeries;
00215 std::vector<QAreaSeries*> m_backgroundSeries;
00216 QValueAxis* m_axisX = nullptr;
00217 QCategoryAxis* m_axisXDisplay = nullptr;
00218 QValueAxis* m_axisY = nullptr;
00219 QValueAxis* m_axisYOverlay = nullptr;
00220 RideData m_rideData;
00221 std::vector<Interval> m_intervals;
00222 std::vector<Climb> m_climbs;
00223 bool m_metricOverlayEnabled = false;
00224 ColorMetric m_metricOverlayMetric = ColorMetric::None;
00225 ColorMetric m_colorMetric = ColorMetric::None;
00226 ColorContext m_colorContext;
00227
00228 // Cache for smoothed metric series: keyed by (metric * 1000 + smoothing_seconds)
00229 // Avoids recomputation when chart is redrawn or multiple overlays use same smoothing
00230 mutable std::unordered_map<int, std::vector<double>> m_smoothedSeriesCache;
00231 std::vector<QPointF> m_tooltipPoints;
00232 std::vector<double> m_tooltipRawValues;
00233 std::vector<double> m_tooltipDisplayValues;
00234 int m_requestedPowerSmoothingSeconds = 0;
00235 int m_effectivePowerSmoothingSeconds = 0;
00236 bool m_autoSmoothing = false;
00237 bool m_isRebuilding = false;
00238 bool m_syncing = false;
00239 bool m_hasData = false;
00240
00241 // Cache for Y-axis range calculations during zoom to avoid redundant recalculation
00242 mutable struct
00243 {
00244     double cachedMinX = -1.0;
00245     double cachedMaxX = -1.0;
00246     double cachedAxisMin = 0.0;
00247     double cachedAxisMax = 1.0;
00248     bool valid = false;
00249 } m_yAxisRangeCache;
00250 double m_origMaxX = 1.0;
00251 double m_crosshairX = -1.0;
00252 double m_crosshairY = std::numeric_limits<double>::quiet_NaN();
00253 bool m_intervalSelecting = false;
00254 double m_intervalStartMin = -1.0;
00255 double m_intervalEndMin = -1.0;
00256 bool m_climbEditingEnabled = false;
00257 int m_selectedClimbIndex = -1;
00258 enum class ClimbHandleDrag
00259 {
00260     None,
00261     Start,
00262     End
00263 };
00264 ClimbHandleDrag m_climbHandleDrag = ClimbHandleDrag::None;
00265 double m_dragOriginalClimbStartSec = -1.0;

```

```

00265     double      m_dragOriginalClimbEndSec = -1.0;
00266     bool         m_newClimbCreating = false;
00267     bool         m_newClimbDragging = false;
00268     double      m_newClimbStartSec = -1.0;
00269     double      m_newClimbCurrentSec = -1.0;
00270     double      m_dataMinY          = 0.0;    // raw min from ride data
00271     double      m_dataMaxY          = 1.0;    // raw max from ride data
00272 };

```

8.32 ActivityAnalysisFlags.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00012 namespace ActivityAnalysisFlags
00013 {
00017     constexpr int None          = 0;
00018
00022     constexpr int HasClimbs     = 1 << 0;
00023
00027     constexpr int HasIntervals = 1 << 1;
00028
00032     constexpr int HasPowerCurve = 1 << 2;
00033
00037     constexpr int HasHistogram  = 1 << 3;
00038
00042     constexpr int HasVideo      = 1 << 4;
00043 }

```

8.33 AnalysisVersions.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00012 namespace AnalysisVersions
00013 {
00017     constexpr int ClimbDetection = 1;
00018
00022     constexpr int IntervalDetection = 1;
00023 }

```

8.34 AppSettings.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "AppSettings.h"
00005
00006 #include <QSettings>
00007
00008 AppSettings& AppSettings::instance()
00009 {
00010     static AppSettings settings;
00011     return settings;
00012 }
00013
00014 DateFormat AppSettings::dateFormat() const
00015 {
00016     QSettings settings("Fitlyzer", "FitlyzerC");
00017     const int rawValue = settings.value(dateFormatKey(),
00018         static_cast<int>(DateFormat::DD_MM_YYYY)).toInt();
00018     return sanitizeDateFormat(rawValue);
00019 }
00020
00021 void AppSettings::setDateFormat(DateFormat format)
00022 {
00023     QSettings settings("Fitlyzer", "FitlyzerC");
00024     settings.setValue(dateFormatKey(), static_cast<int>(format));
00025 }
00026
00027 QString AppSettings::dateFormatKey()
00028 {
00029     return QStringLiteral("general/dateFormat");
00030 }

```

```

00031
00032 DateFormat AppSettings::sanitizeDateFormat(int rawValue)
00033 {
00034     switch (rawValue)
00035     {
00036         case static_cast<int>(DateFormat::DD_MM_YYYY):
00037             return DateFormat::DD_MM_YYYY;
00038         case static_cast<int>(DateFormat::YYYY_MM_DD):
00039             return DateFormat::YYYY_MM_DD;
00040         case static_cast<int>(DateFormat::DD_DOT_MM_DOT_YYYY):
00041             return DateFormat::DD_DOT_MM_DOT_YYYY;
00042         case static_cast<int>(DateFormat::MM_DD_YYYY):
00043             return DateFormat::MM_DD_YYYY;
00044         default:
00045             return DateFormat::DD_MM_YYYY;
00046     }
00047 }
00048
00049 int AppSettings::tileCacheSize() const
00050 {
00051     QSettings settings("Fitlyzer", "FitlyzerC");
00052     const int stored = settings.value(tileCacheSizeKey(), 512).toInt();
00053     // Clamp to known-good values in case of corruption.
00054     const int valid[] = {128, 256, 512, 1024, 2048};
00055     for (int v : valid)
00056         if (stored == v)
00057             return v;
00058     return 512;
00059 }
00060
00061 void AppSettings::setTileCacheSize(int tiles)
00062 {
00063     QSettings settings("Fitlyzer", "FitlyzerC");
00064     settings.setValue(tileCacheSizeKey(), tiles);
00065 }
00066
00067 QString AppSettings::tileCacheSizeKey()
00068 {
00069     return QStringLiteral("maps/tileCacheSize");
00070 }

```

8.35 AppSettings.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QString>
00007
00008 class QSettings;
00009
00013 enum class DateFormat
00014 {
00015     DD_MM_YYYY = 0,
00016     YYYY_MM_DD,
00017     DD_DOT_MM_DOT_YYYY,
00018     MM_DD_YYYY
00019 };
00020
00026 class AppSettings
00027 {
00028 public:
00033     static AppSettings& instance();
00034
00039     DateFormat dateFormat() const;
00040
00045     void setDateFormat(DateFormat format);
00046
00051     int tileCacheSize() const;
00052
00057     void setTileCacheSize(int tiles);
00058
00059 private:
00060     AppSettings() = default;
00061
00062     static QString dateFormatKey();
00063     static DateFormat sanitizeDateFormat(int rawValue);
00064     static QString tileCacheSizeKey();
00065 };

```

8.36 DateFormatter.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "DateFormatter.h"
00005
00006 #include "AppSettings.h"
00007
00008 #include <QStringList>
00009
00010 namespace
00011 {
00012     QString datePatternForFormat(DateFormat format)
00013     {
00014         switch (format)
00015         {
00016             case DateFormat::DD_MM_YYYY:
00017                 return QStringLiteral("dd-MM-yyyy");
00018             case DateFormat::YYYY_MM_DD:
00019                 return QStringLiteral("yyyy-MM-dd");
00020             case DateFormat::DD_DOT_MM_DOT_YYYY:
00021                 return QStringLiteral("dd.MM.yyyy");
00022             case DateFormat::MM_DD_YYYY:
00023                 return QStringLiteral("MM/dd/yyyy");
00024         }
00025
00026         return QStringLiteral("dd-MM-yyyy");
00027     }
00028 }
00029
00030 QString DateFormatter::formatDate(const QDate& date)
00031 {
00032     if (!date.isValid())
00033         return QStringLiteral("-");
00034
00035     return date.toString(datePatternForFormat(AppSettings::instance().dateFormat()));
00036 }
00037
00038 QString DateFormatter::formatDateTime(const QDateTime& dateTime)
00039 {
00040     if (!dateTime.isValid())
00041         return QStringLiteral("-");
00042
00043     return QStringLiteral("%1 %2")
00044         .arg(formatDate(dateTime.date()), dateTime.time().toString(QStringLiteral("HH:mm")));
00045 }
00046
00047 QString DateFormatter::toIsoDate(const QDate& date)
00048 {
00049     return date.toString(Qt::ISODate);
00050 }
00051
00052 QString DateFormatter::toIsoDateTime(const QDateTime& dateTime)
00053 {
00054     return dateTime.toString(Qt::ISODate);
00055 }
00056
00057 QDate DateFormatter::parseDate(const QString& text)
00058 {
00059     const QString trimmed = text.trimmed();
00060     if (trimmed.isEmpty())
00061         return QDate();
00062
00063     const QString activePattern = datePatternForFormat(AppSettings::instance().dateFormat());
00064
00065     QDate parsed = QDate::fromString(trimmed, activePattern);
00066     if (parsed.isValid())
00067         return parsed;
00068
00069     parsed = QDate::fromString(trimmed, Qt::ISODate);
00070     if (parsed.isValid())
00071         return parsed;
00072
00073     // Accept dates that may have been entered using any supported format.
00074     static const QStringList allPatterns = {
00075         QStringLiteral("dd-MM-yyyy"),
00076         QStringLiteral("yyyy-MM-dd"),
00077         QStringLiteral("dd.MM.yyyy"),
00078         QStringLiteral("MM/dd/yyyy")
00079     };
00080
00081     for (const QString& pattern : allPatterns)
00082     {
00083         parsed = QDate::fromString(trimmed, pattern);
00084         if (parsed.isValid())

```

```

00085         return parsed;
00086     }
00087
00088     return QDate();
00089 }
00090
00091 QString DateFormatter::qtDatePattern()
00092 {
00093     return datePatternForFormat(AppSettings::instance().dateFormat());
00094 }

```

8.37 DateFormatter.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QDate>
00007 #include <QDateTime>
00008 #include <QString>
00009
00013 class DateFormatter
00014 {
00015 public:
00021     static QString formatDate(const QDate& date);
00022
00028     static QString formatDateTime(const QDateTime& dateTime);
00029
00035     static QString toIsoDate(const QDate& date);
00036
00042     static QString toIsoDateTime(const QDateTime& dateTime);
00043
00049     static QDate parseDate(const QString& text);
00050
00055     static QString qtDatePattern();
00056 };

```

8.38 ClimbEditCommand.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "ClimbEditCommand.h"
00005
00006 #include <QSqlDatabase>
00007
00008 ClimbEditCommand::ClimbEditCommand(const ClimbRecord& before,
00009                                     const ClimbRecord& after,
00010                                     const QString& dbConnectionName,
00011                                     std::function<void()> refreshCallback)
00012     : m_before(before)
00013     , m_after(after)
00014     , m_connName(dbConnectionName)
00015     , m_refresh(std::move(refreshCallback))
00016 {}
00017
00018 void ClimbEditCommand::undo()
00019 {
00020     apply(m_before);
00021 }
00022
00023 void ClimbEditCommand::redo()
00024 {
00025     apply(m_after);
00026 }
00027
00028 QString ClimbEditCommand::description() const
00029 {
00030     return QStringLiteral("Edit Climb Boundary");
00031 }
00032
00033 void ClimbEditCommand::apply(const ClimbRecord& record)
00034 {
00035     auto db = QSqlDatabase::database(m_connName);
00036     if (db.isValid() && db.isOpen())
00037     {
00038         ClimbRepository repo(db);
00039         repo.updateClimb(record);
00040     }
00041     if (m_refresh)

```

```
00042         m_refresh();
00043     }
```

8.39 ClimbEditCommand.h

```
00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <functional>
00007
00008 #include <QSqlDatabase>
00009
00010 #include "UndoCommand.h"
00011 #include "database/ClimbRepository.h"
00012
00019 class ClimbEditCommand final : public UndoCommand
00020 {
00021 public:
00029     ClimbEditCommand(const ClimbRecord& before,
00030                     const ClimbRecord& after,
00031                     const QString& dbConnectionName,
00032                     std::function<void()> refreshCallback);
00033
00034     void undo() override;
00035     void redo() override;
00036     QString description() const override;
00037 private:
00039     void apply(const ClimbRecord& record);
00040
00041     ClimbRecord m_before;
00042     ClimbRecord m_after;
00043     QString m_connName;
00044     std::function<void()> m_refresh;
00045 };
```

8.40 UndoCommand.h

```
00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QString>
00007
00013 class UndoCommand
00014 {
00015 public:
00016     virtual ~UndoCommand() = default;
00017
00021     virtual void undo() = 0;
00022
00026     virtual void redo() = 0;
00027
00032     virtual QString description() const { return QStringLiteral("Edit"); }
00033 };
```

8.41 UndoManager.cpp

```
00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "UndoManager.h"
00005
00006 UndoManager::UndoManager(int maxStackSize, QObject* parent)
00007     : QObject(parent)
00008     , m_maxStackSize(maxStackSize)
00009 {}
00010
00011 void UndoManager::push(std::unique_ptr<UndoCommand> cmd)
00012 {
00013     m_redoStack.clear();
00014     m_undoStack.push_back(std::move(cmd));
00015
00016     // Trim oldest entries when the stack exceeds the limit.
00017     // std::deque::pop_front() is O(1), whereas vector::erase(begin()) is O(n).
```



```

00018     while (static_cast<int>(m_undoStack.size()) > m_maxStackSize)
00019         m_undoStack.pop_front();
00020
00021     notify();
00022 }
00023
00024 void UndoManager::clear()
00025 {
00026     m_undoStack.clear();
00027     m_redoStack.clear();
00028     notify();
00029 }
00030
00031 bool UndoManager::canUndo() const { return !m_undoStack.empty(); }
00032 bool UndoManager::canRedo() const { return !m_redoStack.empty(); }
00033
00034 QString UndoManager::undoDescription() const
00035 {
00036     return m_undoStack.empty() ? QString() : m_undoStack.back()->description();
00037 }
00038
00039 QString UndoManager::redoDescription() const
00040 {
00041     return m_redoStack.empty() ? QString() : m_redoStack.back()->description();
00042 }
00043
00044 void UndoManager::undo()
00045 {
00046     if (m_undoStack.empty())
00047         return;
00048
00049     auto cmd = std::move(m_undoStack.back());
00050     m_undoStack.pop_back();
00051     cmd->undo();
00052     m_redoStack.push_back(std::move(cmd));
00053     notify();
00054 }
00055
00056 void UndoManager::redo()
00057 {
00058     if (m_redoStack.empty())
00059         return;
00060
00061     auto cmd = std::move(m_redoStack.back());
00062     m_redoStack.pop_back();
00063     cmd->redo();
00064     m_undoStack.push_back(std::move(cmd));
00065     notify();
00066 }
00067
00068 void UndoManager::notify()
00069 {
00070     emit canUndoChanged(canUndo());
00071     emit canRedoChanged(canRedo());
00072     emit undoDescriptionChanged(undoDescription());
00073     emit redoDescriptionChanged(redoDescription());
00074 }

```

8.42 UndoManager.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <deque>
00007 #include <memory>
00008
00009 #include <QObject>
00010 #include <QString>
00011
00012 #include "UndoCommand.h"
00013
00020 class UndoManager : public QObject
00021 {
00022     Q_OBJECT
00023 public:
00029     explicit UndoManager(int maxStackSize = 50, QObject* parent = nullptr);
00030
00037     void push(std::unique_ptr<UndoCommand> cmd);
00038
00042     void clear();
00043
00048     bool canUndo() const;

```

```

00049
00054     bool    canRedo() const;
00055
00060     QString undoDescription() const;
00061
00066     QString redoDescription() const;
00067
00068 public slots:
00072     void undo();
00073
00077     void redo();
00078
00079 signals:
00083     void canUndoChanged(bool canUndo);
00084
00088     void canRedoChanged(bool canRedo);
00089
00093     void undoDescriptionChanged(const QString& desc);
00094
00098     void redoDescriptionChanged(const QString& desc);
00099
00100 private:
00101     void notify();
00102
00103     int m_maxStackSize;
00104     std::deque<std::unique_ptr<UndoCommand> m_undoStack;
00105     std::deque<std::unique_ptr<UndoCommand> m_redoStack;
00106 };

```

8.43 ColorProvider.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "ColorProvider.h"
00005
00006 #include "ZoneCalculator.h"
00007
00008 #include <algorithm>
00009
00010 namespace {
00011
00012 QColor interpolateColor(const QColor& a, const QColor& b, double t)
00013 {
00014     const double clamped = std::clamp(t, 0.0, 1.0);
00015     return QColor(
00016         static_cast<int>(a.red() + (b.red() - a.red()) * clamped),
00017         static_cast<int>(a.green() + (b.green() - a.green()) * clamped),
00018         static_cast<int>(a.blue() + (b.blue() - a.blue()) * clamped));
00019 }
00020
00021 }
00022
00023 QColor ColorProvider::colorForMetric(ColorMetric metric,
00024                                     double value,
00025                                     const ColorContext& context)
00026 {
00027     switch (metric)
00028     {
00029     case ColorMetric::Power:
00030         return colorForPower(value, context.ftp);
00031     case ColorMetric::HeartRate:
00032         return colorForHeartRate(value,
00033                                 context.heartRateMax,
00034                                 context.heartRateRest,
00035                                 context.thresholdHeartRate);
00036     case ColorMetric::Cadence:
00037         return colorForCadence(value);
00038     case ColorMetric::Speed:
00039         return colorForSpeed(value);
00040     case ColorMetric::Altitude:
00041         return colorForAltitude(value, context.altitudeMin, context.altitudeMax);
00042     case ColorMetric::Gradient:
00043     case ColorMetric::None:
00044         return QColor("#94a3b8");
00045     }
00046
00047     return QColor("#94a3b8");
00048 }
00049
00050 QColor ColorProvider::colorForPower(double watts, int ftp)
00051 {
00052     return colorForValue(watts, ZoneCalculator::powerZones(ftp));
00053 }

```

```

00054
00055 QColor ColorProvider::colorForHeartRate(double bpm,
00056                                         int heartRateMax,
00057                                         int heartRateRest,
00058                                         int thresholdHeartRate)
00059 {
00060     return colorForValue(
00061         bpm,
00062         ZoneCalculator::heartRateZones(heartRateMax, heartRateRest, thresholdHeartRate));
00063 }
00064
00065 QColor ColorProvider::colorForCadence(double cadence)
00066 {
00067     return colorForValue(cadence, ZoneCalculator::cadenceZones());
00068 }
00069
00070 QColor ColorProvider::colorForSpeed(double speedKmh)
00071 {
00072     return colorForValue(speedKmh, ZoneCalculator::speedZones());
00073 }
00074
00075 QColor ColorProvider::colorForAltitude(double altitude,
00076                                       double minAltitude,
00077                                       double maxAltitude)
00078 {
00079     if (maxAltitude <= minAltitude)
00080         return QColor("#6fa8dc");
00081
00082     const double t = std::clamp((altitude - minAltitude) / (maxAltitude - minAltitude), 0.0, 1.0);
00083     if (t < 0.33)
00084         return interpolateColor(QColor("#6fa8dc"), QColor("#6aa84f"), t / 0.33);
00085     if (t < 0.66)
00086         return interpolateColor(QColor("#6aa84f"), QColor("#8d6e63"), (t - 0.33) / 0.33);
00087     return interpolateColor(QColor("#8d6e63"), QColor("#f5f5f5"), (t - 0.66) / 0.34);
00088 }
00089
00090 QColor ColorProvider::colorForValue(double value,
00091                                     const std::vector<Zone>& zones,
00092                                     const QColor& fallback)
00093 {
00094     for (const Zone& zone : zones)
00095     {
00096         if (value >= zone.minValue && value < zone.maxValue)
00097             return zone.color;
00098     }
00099
00100     return fallback;
00101 }

```

8.44 ColorProvider.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QColor>
00007
00008 #include <vector>
00009
00010 #include "ZoneDefinition.h"
00011
00012 class ColorProvider
00013 {
00014 public:
00015     static QColor colorForMetric(ColorMetric metric,
00016                                 double value,
00017                                 const ColorContext& context);
00018
00019     static QColor colorForPower(double watts, int ftp);
00020
00021     static QColor colorForHeartRate(double bpm,
00022                                     int heartRateMax,
00023                                     int heartRateRest = 60,
00024                                     int thresholdHeartRate = 0);
00025
00026     static QColor colorForCadence(double cadence);
00027
00028     static QColor colorForSpeed(double speedKmh);
00029
00030     static QColor colorForAltitude(double altitude,
00031                                    double minAltitude,
00032                                    double maxAltitude);
00033 }

```

```

00039     static QColor colorForValue(double value,
00040                                 const std::vector<Zone>& zones,
00041                                 const QColor& fallback = QColor("#94a3b8"));
00042 };

```

8.45 ZoneCalculator.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "ZoneCalculator.h"
00005
00006 #include <algorithm>
00007 #include <cmath>
00008
00009 namespace {
00010
00011 double valueForMetric(const RideRecord& record, ColorMetric metric)
00012 {
00013     switch (metric)
00014     {
00015         case ColorMetric::Power:      return record.power;
00016         case ColorMetric::HeartRate:  return record.heartRate;
00017         case ColorMetric::Cadence:    return record.cadence;
00018         case ColorMetric::Speed:      return record.speed;
00019         case ColorMetric::Altitude:   return record.altitude;
00020         case ColorMetric::Gradient:   return 0.0;
00021         case ColorMetric::None:       return 0.0;
00022     }
00023
00024     return 0.0;
00025 }
00026
00027 bool sampleAvailable(const RideRecord& record, ColorMetric metric)
00028 {
00029     switch (metric)
00030     {
00031         case ColorMetric::Power:      return record.hasPower;
00032         case ColorMetric::HeartRate:  return record.hasHeartRate;
00033         case ColorMetric::Cadence:    return record.hasCadence;
00034         case ColorMetric::Speed:      return record.hasSpeed;
00035         case ColorMetric::Altitude:   return record.hasAltitude;
00036         case ColorMetric::Gradient:   return false;
00037         case ColorMetric::None:       return false;
00038     }
00039
00040     return false;
00041 }
00042
00043 double sampleDurationSeconds(const std::vector<RideRecord>& records, int index)
00044 {
00045     const int size = static_cast<int>(records.size());
00046     if (size <= 1)
00047         return 1.0;
00048
00049     double dt = 1.0;
00050     if (index == 0)
00051         dt = records[1].elapsedSeconds - records[0].elapsedSeconds;
00052     else if (index == size - 1)
00053         dt = records[size - 1].elapsedSeconds - records[size - 2].elapsedSeconds;
00054     else
00055         dt = (records[index + 1].elapsedSeconds - records[index - 1].elapsedSeconds) / 2.0;
00056
00057     return std::max(0.0, dt);
00058 }
00059
00060 }
00061
00062 std::vector<Zone> ZoneCalculator::powerZones(int ftp)
00063 {
00064     const double threshold = std::max(ftp, 1);
00065     return {
00066         {"Z1 Recovery",      0.0,          threshold * 0.55, QColor("#b0b0b0")},
00067         {"Z2 Endurance",    threshold * 0.55, threshold * 0.75, QColor("#6fa8dc")},
00068         {"Z3 Tempo",        threshold * 0.75, threshold * 0.87, QColor("#6aa84f")},
00069         {"Z4 Threshold",    threshold * 0.87, threshold * 0.95, QColor("#ffd966")},
00070         {"Z5 VO2 Max",      threshold * 0.95, threshold * 1.05, QColor("#e69138")},
00071         {"Z6 Anaerobic",    threshold * 1.05, threshold * 1.20, QColor("#cc0000")},
00072         {"Z7 Neuromuscular", threshold * 1.20, DBL_MAX,      QColor("#741b47")},
00073     };
00074 }
00075
00076 std::vector<Zone> ZoneCalculator::heartRateZones(int heartRateMax,
00077

```

```

00078                                     int thresholdHeartRate)
00079 {
00080     const double reference = std::max(thresholdHeartRate > 0 ? thresholdHeartRate : heartRateMax,
00081     120);
00082     return {
00083         {"Z1 Recovery", 0.0, reference * 0.68, QColor("#b0b0b0")},
00084         {"Z2 Aerobic", reference * 0.68, reference * 0.83, QColor("#6fa8dc")},
00085         {"Z3 Tempo", reference * 0.83, reference * 0.94, QColor("#6aa84f")},
00086         {"Z4 Threshold", reference * 0.94, reference * 1.03, QColor("#ffd966")},
00087         {"Z5 Max", reference * 1.03, DBL_MAX, QColor("#cc0000")},
00088     };
00089 }
00090 std::vector<Zone> ZoneCalculator::cadenceZones()
00091 {
00092     return {
00093         {"Low", 0.0, 70.0, QColor("#b0b0b0")},
00094         {"Steady", 70.0, 85.0, QColor("#6fa8dc")},
00095         {"Endurance", 85.0, 95.0, QColor("#6aa84f")},
00096         {"Fast", 95.0, 105.0, QColor("#ffd966")},
00097         {"Spin", 105.0, DBL_MAX, QColor("#cc0000")},
00098     };
00099 }
00100
00101 std::vector<Zone> ZoneCalculator::speedZones()
00102 {
00103     return {
00104         {"Easy", 0.0, 15.0, QColor("#6fa8dc")},
00105         {"Rolling", 15.0, 25.0, QColor("#6aa84f")},
00106         {"Tempo", 25.0, 35.0, QColor("#ffd966")},
00107         {"Fast", 35.0, 45.0, QColor("#e69138")},
00108         {"Descent", 45.0, DBL_MAX, QColor("#cc0000")},
00109     };
00110 }
00111
00112 std::vector<Zone> ZoneCalculator::altitudeZones(double minAltitude,
00113     double maxAltitude)
00114 {
00115     if (maxAltitude <= minAltitude)
00116         maxAltitude = minAltitude + 1.0;
00117
00118     const double span = maxAltitude - minAltitude;
00119     return {
00120         {"Low", minAltitude, minAltitude + span * 0.25, QColor("#6fa8dc")},
00121         {"Rolling", minAltitude + span * 0.25, minAltitude + span * 0.55, QColor("#6aa84f")},
00122         {"High", minAltitude + span * 0.55, minAltitude + span * 0.80, QColor("#8d6e63")},
00123         {"Alpine", minAltitude + span * 0.80, DBL_MAX, QColor("#f5f5f5")},
00124     };
00125 }
00126
00127 std::vector<Zone> ZoneCalculator::zonesForMetric(ColorMetric metric,
00128     const ColorContext& context)
00129 {
00130     switch (metric)
00131     {
00132     case ColorMetric::Power:
00133         return powerZones(context ftp);
00134     case ColorMetric::HeartRate:
00135         return heartRateZones(context.heartRateMax,
00136             context.heartRateRest,
00137             context.thresholdHeartRate);
00138     case ColorMetric::Cadence:
00139         return cadenceZones();
00140     case ColorMetric::Speed:
00141         return speedZones();
00142     case ColorMetric::Altitude:
00143         return altitudeZones(context.altitudeMin, context.altitudeMax);
00144     case ColorMetric::Gradient:
00145     case ColorMetric::None:
00146         return {};
00147     }
00148
00149     return {};
00150 }
00151
00152 std::vector<ZoneDistribution> ZoneCalculator::computeDistribution(
00153     const RideData& rideData,
00154     ColorMetric metric,
00155     const ColorContext& context)
00156 {
00157     std::vector<ZoneDistribution> result;
00158     const std::vector<Zone> zones = zonesForMetric(metric, context);
00159     result.reserve(zones.size());
00160     for (const Zone& zone : zones)
00161         result.push_back({zone, 0.0, 0.0});
00162
00163     if (metric == ColorMetric::None || rideData.records.empty() || result.empty())

```

```

00164         return result;
00165
00166     if (metric == ColorMetric::Gradient)
00167         return result;
00168     double totalSeconds = 0.0;
00169     for (int index = 0; index < static_cast<int>(rideData.records.size()); ++index)
00170     {
00171         const RideRecord& record = rideData.records[static_cast<size_t>(index)];
00172         if (!sampleAvailable(record, metric))
00173             continue;
00174
00175         const double value = valueForMetric(record, metric);
00176         const double dt = sampleDurationSeconds(rideData.records, index);
00177         totalSeconds += dt;
00178
00179         for (ZoneDistribution& distribution : result)
00180         {
00181             if (value >= distribution.zone.minValue &&
00182                 value < distribution.zone.maxValue)
00183             {
00184                 distribution.seconds += dt;
00185                 break;
00186             }
00187         }
00188     }
00189
00190     if (totalSeconds > 0.0)
00191     {
00192         for (ZoneDistribution& distribution : result)
00193             distribution.percent = distribution.seconds / totalSeconds;
00194     }
00195     return result;
00196 }
00197
00198
00199 bool ZoneCalculator::hasMetricSamples(const RideData& rideData, ColorMetric metric)
00200 {
00201     if (metric == ColorMetric::None)
00202         return false;
00203
00204     for (const RideRecord& record : rideData.records)
00205     {
00206         if (sampleAvailable(record, metric))
00207             return true;
00208     }
00209
00210     return false;
00211 }

```

8.46 ZoneCalculator.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <vector>
00007
00008 #include "ZoneDefinition.h"
00009 #include "fit/RideData.h"
00010
00016 class ZoneCalculator
00017 {
00018 public:
00024     static std::vector<Zone> powerZones(int ftp);
00025
00033     static std::vector<Zone> heartRateZones(int heartRateMax,
00034                                             int heartRateRest = 60,
00035                                             int thresholdHeartRate = 0);
00036
00041     static std::vector<Zone> cadenceZones();
00042
00047     static std::vector<Zone> speedZones();
00048
00055     static std::vector<Zone> altitudeZones(double minAltitude,
00056                                           double maxAltitude);
00057
00064     static std::vector<Zone> zonesForMetric(ColorMetric metric,
00065                                             const ColorContext& context);
00066
00074     static std::vector<ZoneDistribution> computeDistribution(
00075         const RideData& rideData,
00076         ColorMetric metric,
00077         const ColorContext& context);

```

```

00078
00085     static bool hasMetricSamples(const RideData& rideData, ColorMetric metric);
00086 };

```

8.47 ZoneDefinition.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "ZoneDefinition.h"
00005
00006 QString colorMetricDisplayName(ColorMetric metric)
00007 {
00008     switch (metric)
00009     {
00010         case ColorMetric::None:      return "None";
00011         case ColorMetric::Power:     return "Power";
00012         case ColorMetric::HeartRate: return "Heart Rate";
00013         case ColorMetric::Cadence:   return "Cadence";
00014         case ColorMetric::Speed:     return "Speed";
00015         case ColorMetric::Altitude:  return "Altitude";
00016         case ColorMetric::Gradient:  return "Gradient";
00017     }
00018
00019     return "None";
00020 }
00021
00022 QString colorMetricUnit(ColorMetric metric)
00023 {
00024     switch (metric)
00025     {
00026         case ColorMetric::Power:      return "w";
00027         case ColorMetric::HeartRate:  return "bpm";
00028         case ColorMetric::Cadence:    return "rpm";
00029         case ColorMetric::Speed:      return "km/h";
00030         case ColorMetric::Altitude:   return "m";
00031         case ColorMetric::Gradient:   return {};
00032         case ColorMetric::None:       return {};
00033     }
00034
00035     return {};
00036 }

```

8.48 ZoneDefinition.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QColor>
00007 #include <QString>
00008
00012 enum class ColorMetric
00013 {
00014     None,
00015     Power,
00016     HeartRate,
00017     Cadence,
00018     Speed,
00019     Altitude,
00020     Gradient
00021 };
00022
00026 struct Zone
00027 {
00029     QString name;
00030
00032     double minValue = 0.0;
00033
00035     double maxValue = 0.0;
00036
00038     QColor color;
00039 };
00040
00044 struct ZoneDistribution
00045 {
00047     Zone zone;
00048
00050     double seconds = 0.0;
00051

```

```

00053     double percent = 0.0;
00054 };
00055
00059 struct ColorContext
00060 {
00062     int ftp = 250;
00063
00065     int heartRateMax = 190;
00066
00068     int heartRateRest = 60;
00069
00071     int thresholdHeartRate = 0;
00072
00074     double altitudeMin = 0.0;
00075
00077     double altitudeMax = 0.0;
00078
00080     bool hasAltitudeRange = false;
00081 };
00082
00088 QString colorMetricDisplayName(ColorMetric metric);
00089
00095 QString colorMetricUnit(ColorMetric metric);

```

8.49 ActivityAnalysisCacheRepository.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "ActivityAnalysisCacheRepository.h"
00005
00006 #include <QSqlDatabase>
00007 #include <QSqlQuery>
00008
00009 ActivityAnalysisCacheRepository::ActivityAnalysisCacheRepository(QSqlDatabase& db)
00010     : m_db(db)
00011 {}
00012
00013 bool ActivityAnalysisCacheRepository::setCache(int activityId, const QString& key, const QByteArray&
value)
00014 {
00015     QSqlQuery q(m_db);
00016     q.prepare(
00017         "INSERT OR REPLACE INTO activity_analysis_cache(activity_id, cache_key, cache_value)"
00018         " VALUES(:id, :k, :v)");
00019     q.bindValue(":id", activityId);
00020     q.bindValue(":k", key);
00021     q.bindValue(":v", value);
00022     return q.exec();
00023 }
00024
00025 QByteArray ActivityAnalysisCacheRepository::getCache(int activityId, const QString& key)
00026 {
00027     QSqlQuery q(m_db);
00028     q.prepare("SELECT cache_value FROM activity_analysis_cache WHERE activity_id=:id AND
cache_key=:k");
00029     q.bindValue(":id", activityId);
00030     q.bindValue(":k", key);
00031     if (q.exec() && q.next())
00032         return q.value(0).toByteArray();
00033     return {};
00034 }
00035
00036 bool ActivityAnalysisCacheRepository::hasCache(int activityId, const QString& key)
00037 {
00038     QSqlQuery q(m_db);
00039     q.prepare("SELECT 1 FROM activity_analysis_cache WHERE activity_id=:id AND cache_key=:k LIMIT 1");
00040     q.bindValue(":id", activityId);
00041     q.bindValue(":k", key);
00042     return q.exec() && q.next();
00043 }
00044
00045 bool ActivityAnalysisCacheRepository::invalidateCache(int activityId)
00046 {
00047     QSqlQuery q(m_db);
00048     q.prepare("DELETE FROM activity_analysis_cache WHERE activity_id=:id");
00049     q.bindValue(":id", activityId);
00050     return q.exec();
00051 }
00052
00053 bool ActivityAnalysisCacheRepository::invalidateKey(int activityId, const QString& key)
00054 {
00055     QSqlQuery q(m_db);
00056     q.prepare("DELETE FROM activity_analysis_cache WHERE activity_id=:id AND cache_key=:k");

```



```

00057     q.bindValue(":id", activityId);
00058     q.bindValue(":k", key);
00059     return q.exec();
00060 }

```

8.50 ActivityAnalysisCacheRepository.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QByteArray>
00007 #include <QString>
00008
00009 class QSqlDatabase;
00010
00017 class ActivityAnalysisCacheRepository
00018 {
00019 public:
00024     explicit ActivityAnalysisCacheRepository(QSqlDatabase& db);
00025
00033     bool        setCache(int activityId, const QString& key, const QByteArray& value);
00034
00041     QByteArray getCache(int activityId, const QString& key);
00042
00049     bool        hasCache(int activityId, const QString& key);
00050
00056     bool        invalidateCache(int activityId);
00057
00064     bool        invalidateKey(int activityId, const QString& key);
00065
00066 private:
00067     QSqlDatabase& m_db;
00068 };

```

8.51 ActivityAnalysisSettingsRepository.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "ActivityAnalysisSettingsRepository.h"
00005
00006 #include <QSqlDatabase>
00007 #include <QSqlQuery>
00008
00009 ActivityAnalysisSettingsRepository::ActivityAnalysisSettingsRepository(QSqlDatabase& db)
00010     : m_db(db)
00011 {}
00012
00013 bool ActivityAnalysisSettingsRepository::insertOrUpdate(const ActivityAnalysisSettings& settings)
00014 {
00015     QSqlQuery q(m_db);
00016     q.prepare(
00017         "INSERT OR REPLACE INTO activity_analysis_settings("
00018         "    activity_id, climb_min_gain, climb_min_length, climb_min_gradient,"
00019         "    interval_work_threshold, interval_rest_threshold"
00020         ") VALUES ("
00021         "    :activity_id, :climb_min_gain, :climb_min_length, :climb_min_gradient,"
00022         "    :interval_work_threshold, :interval_rest_threshold"
00023         ")");
00024
00025     q.bindValue(":activity_id", settings.activityId);
00026     q.bindValue(":climb_min_gain", settings.climbMinGain);
00027     q.bindValue(":climb_min_length", settings.climbMinLength);
00028     q.bindValue(":climb_min_gradient", settings.climbMinGradient);
00029     q.bindValue(":interval_work_threshold", settings.intervalWorkThreshold);
00030     q.bindValue(":interval_rest_threshold", settings.intervalRestThreshold);
00031     return q.exec();
00032 }
00033
00034 ActivityAnalysisSettings ActivityAnalysisSettingsRepository::settingsForActivity(int activityId)
00035 {
00036     ActivityAnalysisSettings s;
00037     QSqlQuery q(m_db);
00038     q.prepare(
00039         "SELECT activity_id, climb_min_gain, climb_min_length, climb_min_gradient,"
00040         "    interval_work_threshold, interval_rest_threshold"
00041         " FROM activity_analysis_settings WHERE activity_id=:id");
00042     q.bindValue(":id", activityId);
00043     if (!q.exec() || !q.next())

```

```

00044         return s;
00045
00046         s.activityId           = q.value(0).toInt();
00047         s.climbMinGain         = q.value(1).toDouble();
00048         s.climbMinLength       = q.value(2).toDouble();
00049         s.climbMinGradient     = q.value(3).toDouble();
00050         s.intervalWorkThreshold = q.value(4).toDouble();
00051         s.intervalRestThreshold = q.value(5).toDouble();
00052         return s;
00053     }

```

8.52 ActivityAnalysisSettingsRepository.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QList>
00007 #include <QString>
00008
00009 class QSqlDatabase;
00010
00014 struct ActivityAnalysisSettings
00015 {
00017     int     activityId           = -1;
00018
00020     double  climbMinGain         = 30.0;
00021
00023     double  climbMinLength       = 0.5;
00024
00026     double  climbMinGradient     = 3.0;
00027
00029     double  intervalWorkThreshold = 0.0;
00030
00032     double  intervalRestThreshold = 0.0;
00033
00037     bool    isValid() const { return activityId > 0; }
00038 };
00039
00045 class ActivityAnalysisSettingsRepository
00046 {
00047 public:
00052     explicit ActivityAnalysisSettingsRepository(QSqlDatabase& db);
00053
00059     bool    insertOrUpdate(const ActivityAnalysisSettings& settings);
00060
00066     ActivityAnalysisSettings settingsForActivity(int activityId);
00067
00068 private:
00069     QSqlDatabase& m_db;
00070 };

```

8.53 ActivityRepository.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "ActivityRepository.h"
00005
00006 #include <QDateTime>
00007 #include <QSqlDatabase>
00008 #include <QSqlError>
00009 #include <QSqlQuery>
00010 #include <QVariant>
00011 #include <QStringList>
00012
00013 ActivityRepository::ActivityRepository(QSqlDatabase& db)
00014     : m_db(db)
00015 {}
00016
00017 // Helpers
00018
00019 static Activity activityFromQuery(QSqlQuery& q)
00020 {
00021     Activity a;
00022     int col = 0;
00023     a.id           = q.value(col++).toInt();
00024     a.athleteId    = q.value(col++).toInt();
00025     a.fitHash       = q.value(col++).toString();
00026     a.fileName      = q.value(col++).toString();

```

```

00027     a.sport           = q.value(col++).toString();
00028     a.startTime       = q.value(col++).toString();
00029     a.endTime         = q.value(col++).toString();
00030     a.durationSec     = q.value(col++).toDouble();
00031     a.distanceM       = q.value(col++).toDouble();
00032     a.avgPower        = q.value(col++).toDouble();
00033     a.maxPower        = q.value(col++).toDouble();
00034     a.normalizedPower = q.value(col++).toDouble();
00035     a.avgHr           = q.value(col++).toDouble();
00036     a.maxHr           = q.value(col++).toDouble();
00037     a.avgCadence      = q.value(col++).toDouble();
00038     a.avgSpeed        = q.value(col++).toDouble();
00039     a.elevationGain   = q.value(col++).toDouble();
00040     a.notes           = q.value(col++).toString();
00041     a.weatherNotes    = q.value(col++).toString();
00042     a.temperature     = q.value(col++).toDouble();
00043     a.weather         = q.value(col++).toString();
00044     a.wind            = q.value(col++).toString();
00045     a.rpe             = q.value(col++).toInt();
00046     a.fatigue         = q.value(col++).toInt();
00047     a.sleepHours      = q.value(col++).toDouble();
00048     a.weightKg        = q.value(col++).toDouble();
00049     a.bike            = q.value(col++).toString();
00050     a.equipment       = q.value(col++).toString();
00051     a.importedAt      = q.value(col++).toString();
00052     a.analysisVersion = q.value(col++).toInt();
00053     a.fingerprint     = q.value(col++).toString();
00054     a.analysisFlags   = q.value(col++).toInt();
00055     a.importSource    = q.value(col++).toString();
00056     return a;
00057 }
00058
00059 static const char* kSelectCols =
00060     "id, athlete_id, fit_hash, file_name, sport, activity_start_time, end_time,"
00061     " duration_sec, distance_m, avg_power, max_power, normalized_power,"
00062     " avg_hr, max_hr, avg_cadence, avg_speed, elevation_gain,"
00063     " notes, weather_notes, temperature, weather, wind, rpe, fatigue, sleep,"
00064     " weight, bike, equipment, import_time,"
00065     " COALESCE(analysis_version,1), COALESCE(fingerprint,''),"
00066     " COALESCE(analysis_flags,0), COALESCE(import_source,')";
00067
00068 // Public API
00069
00070 int ActivityRepository::insertActivity(const Activity& a)
00071 {
00072     QSqlQuery q(m_db);
00073     q.prepare(
00074         "INSERT INTO activities("
00075         " athlete_id, fit_hash, file_name, sport, activity_start_time, end_time,"
00076         " duration_sec, distance_m, avg_power, max_power, normalized_power,"
00077         " avg_hr, max_hr, avg_cadence, avg_speed, elevation_gain,"
00078         " notes, weather_notes, temperature, weather, wind, rpe, fatigue, sleep,"
00079         " weight, bike, equipment, import_time"
00080         ") VALUES("
00081         " :aid, :fh, :fn, :sport, :st, :et,"
00082         " :dur, :dist, :ap, :mp, :np,"
00083         " :ah, :mh, :ac, :as_, :eg,"
00084         " :notes, :wn, :temp, :weather, :wind, :rpe, :fatigue, :sleep,"
00085         " :weight, :bike, :equipment, :import_time"
00086         ")");
00087
00088     q.bindValue(":aid", a.athleteId);
00089     q.bindValue(":fh", a.fitHash.isEmpty() ? QVariant() : QVariant(a.fitHash));
00090     q.bindValue(":fn", a.fileName);
00091     q.bindValue(":sport", a.sport.isEmpty() ? QVariant() : QVariant(a.sport));
00092     q.bindValue(":st", a.startTime.isEmpty() ? QVariant() : QVariant(a.startTime));
00093     q.bindValue(":et", a.endTime.isEmpty() ? QVariant() : QVariant(a.endTime));
00094     q.bindValue(":dur", a.durationSec);
00095     q.bindValue(":dist", a.distanceM);
00096     q.bindValue(":ap", a.avgPower);
00097     q.bindValue(":mp", a.maxPower);
00098     q.bindValue(":np", a.normalizedPower);
00099     q.bindValue(":ah", a.avgHr);
00100     q.bindValue(":mh", a.maxHr);
00101     q.bindValue(":ac", a.avgCadence);
00102     q.bindValue(":as_", a.avgSpeed);
00103     q.bindValue(":eg", a.elevationGain);
00104     q.bindValue(":notes", a.notes);
00105     q.bindValue(":wn", a.weatherNotes);
00106     q.bindValue(":temp", a.temperature);
00107     q.bindValue(":weather", a.weather.isEmpty() ? QVariant() : QVariant(a.weather));
00108     q.bindValue(":wind", a.wind.isEmpty() ? QVariant() : QVariant(a.wind));
00109     q.bindValue(":rpe", a.rpe > 0 ? QVariant(a.rpe) : QVariant());
00110     q.bindValue(":fatigue", a.fatigue > 0 ? QVariant(a.fatigue) : QVariant());
00111     q.bindValue(":sleep", a.sleepHours > 0.0 ? QVariant(a.sleepHours) : QVariant());
00112     q.bindValue(":weight", a.weightKg > 0.0 ? QVariant(a.weightKg) : QVariant());
00113     q.bindValue(":bike", a.bike.isEmpty() ? QVariant() : QVariant(a.bike));

```

```

00114     q.bindValue(":equipment", a.equipment.isEmpty() ? QVariant() : QVariant(a.equipment));
00115     q.bindValue(":import_time", a.importedAt.isEmpty()
00116         ? QDateTime::currentDateTimeUtc().toString(Qt::ISODate)
00117         : a.importedAt);
00118
00119     if (!q.exec()) return -1;
00120     return q.lastInsertId().toInt();
00121 }
00122
00123 Activity ActivityRepository::getActivity(int activityId)
00124 {
00125     QSqlQuery q(m_db);
00126     q.prepare(QString("SELECT %1 FROM activities WHERE id=:id").arg(kSelectCols));
00127     q.bindValue(":id", activityId);
00128     q.exec();
00129     if (q.next()) return activityFromQuery(q);
00130     return {};
00131 }
00132
00133 QList<Activity> ActivityRepository::listActivities(int athleteId)
00134 {
00135     QList<Activity> result;
00136     QSqlQuery q(m_db);
00137     if (athleteId > 0)
00138     {
00139         q.prepare(QString("SELECT %1 FROM activities WHERE athlete_id=:aid"
00140             " ORDER BY activity_start_time DESC").arg(kSelectCols));
00141         q.bindValue(":aid", athleteId);
00142     }
00143     else
00144     {
00145         q.prepare(QString("SELECT %1 FROM activities ORDER BY activity_start_time DESC")
00146             .arg(kSelectCols));
00147     }
00148     q.exec();
00149     while (q.next())
00150         result.append(activityFromQuery(q));
00151     return result;
00152 }
00153
00154 bool ActivityRepository::deleteActivity(int activityId)
00155 {
00156     QSqlQuery q(m_db);
00157     q.prepare("DELETE FROM activities WHERE id=:id");
00158     q.bindValue(":id", activityId);
00159     return q.exec();
00160 }
00161
00162 bool ActivityRepository::hasFitHash(const QString& hash)
00163 {
00164     QSqlQuery q(m_db);
00165     q.prepare("SELECT 1 FROM activities WHERE fit_hash=:h LIMIT 1");
00166     q.bindValue(":h", hash);
00167     q.exec();
00168     return q.next();
00169 }
00170
00171 bool ActivityRepository::updateAnalysisFlags(int activityId, int flags)
00172 {
00173     QSqlQuery q(m_db);
00174     q.prepare("UPDATE activities SET analysis_flags=:f WHERE id=:id");
00175     q.bindValue(":f", flags);
00176     q.bindValue(":id", activityId);
00177     return q.exec();
00178 }
00179
00180 bool ActivityRepository::updateFingerprint(int activityId, const QString& fingerprint)
00181 {
00182     QSqlQuery q(m_db);
00183     q.prepare("UPDATE activities SET fingerprint=:fp WHERE id=:id");
00184     q.bindValue(":fp", fingerprint);
00185     q.bindValue(":id", activityId);
00186     return q.exec();
00187 }
00188
00189 bool ActivityRepository::hasFingerprintMatch(const QString& fingerprint)
00190 {
00191     if (fingerprint.isEmpty()) return false;
00192     QSqlQuery q(m_db);
00193     q.prepare("SELECT 1 FROM activities WHERE fingerprint=:fp LIMIT 1");
00194     q.bindValue(":fp", fingerprint);
00195     q.exec();
00196     return q.next();
00197 }
00198
00199 bool ActivityRepository::setImportSource(int activityId, const QString& source)
00200 {

```

```

00201     QSqlQuery q(m_db);
00202     q.prepare("UPDATE activities SET import_source=:s WHERE id=:id");
00203     q.bindValue(":s", source);
00204     q.bindValue(":id", activityId);
00205     return q.exec();
00206 }
00207
00208 bool ActivityRepository::updateNotes(int activityId,
00209                                     const QString& notes,
00210                                     const QString& weatherNotes)
00211 {
00212     QSqlQuery q(m_db);
00213     q.prepare("UPDATE activities SET notes=:n, weather_notes=:wn WHERE id=:id");
00214     q.bindValue(":n", notes);
00215     q.bindValue(":wn", weatherNotes);
00216     q.bindValue(":id", activityId);
00217     return q.exec();
00218 }
00219
00220 bool ActivityRepository::updateActivityProperties(int activityId, const Activity& a)
00221 {
00222     QSqlQuery q(m_db);
00223     q.prepare(
00224         "UPDATE activities SET "
00225         "notes=:notes, weather_notes=:wn, temperature=:temp, weather=:weather, wind=:wind, "
00226         "rpe=:rpe, fatigue=:fatigue, sleep=:sleep, weight=:weight, bike=:bike, equipment=:equipment "
00227         "WHERE id=:id");
00228     q.bindValue(":notes", a.notes);
00229     q.bindValue(":wn", a.weatherNotes);
00230     q.bindValue(":temp", a.temperature > 0.0 ? QVariant(a.temperature) : QVariant());
00231     q.bindValue(":weather", a.weather.isEmpty() ? QVariant() : QVariant(a.weather));
00232     q.bindValue(":wind", a.wind.isEmpty() ? QVariant() : QVariant(a.wind));
00233     q.bindValue(":rpe", a.rpe > 0 ? QVariant(a.rpe) : QVariant());
00234     q.bindValue(":fatigue", a.fatigue > 0 ? QVariant(a.fatigue) : QVariant());
00235     q.bindValue(":sleep", a.sleepHours > 0.0 ? QVariant(a.sleepHours) : QVariant());
00236     q.bindValue(":weight", a.weightKg > 0.0 ? QVariant(a.weightKg) : QVariant());
00237     q.bindValue(":bike", a.bike.isEmpty() ? QVariant() : QVariant(a.bike));
00238     q.bindValue(":equipment", a.equipment.isEmpty() ? QVariant() : QVariant(a.equipment));
00239     q.bindValue(":id", activityId);
00240     if (!q.exec())
00241         return false;
00242
00243     // Keep equipment usage in sync for service interval tracking.
00244     QSqlQuery details(m_db);
00245     details.prepare("SELECT athlete_id, distance_m, duration_sec FROM activities WHERE id=:id");
00246     details.bindValue(":id", activityId);
00247     if (!details.exec() || !details.next())
00248         return true;
00249
00250     const int athleteId = details.value(0).toInt();
00251     const double distanceKm = details.value(1).toDouble() / 1000.0;
00252     const double durationSec = details.value(2).toDouble();
00253
00254     QSqlQuery clearUsage(m_db);
00255     clearUsage.prepare("DELETE FROM equipment_usage WHERE activity_id=:id");
00256     clearUsage.bindValue(":id", activityId);
00257     clearUsage.exec();
00258
00259     QStringList equipmentNames;
00260     if (!a.bike.trimmed().isEmpty())
00261         equipmentNames << a.bike.trimmed();
00262     if (!a.equipment.trimmed().isEmpty())
00263     {
00264         const QStringList split = a.equipment.split(',', Qt::SkipEmptyParts);
00265         for (const QString& entry : split)
00266         {
00267             const QString trimmed = entry.trimmed();
00268             if (!trimmed.isEmpty())
00269                 equipmentNames << trimmed;
00270         }
00271     }
00272     equipmentNames.removeDuplicates();
00273
00274     for (const QString& name : equipmentNames)
00275     {
00276         int equipmentId = -1;
00277         QSqlQuery findEq(m_db);
00278         findEq.prepare("SELECT id FROM equipment WHERE athlete_id=:athlete_id AND name=:name LIMIT
00279 1");
00280         findEq.bindValue(":athlete_id", athleteId);
00281         findEq.bindValue(":name", name);
00282         if (findEq.exec() && findEq.next())
00283         {
00284             equipmentId = findEq.value(0).toInt();
00285         }
00286         else
00287         {

```

```

00287         QSqlQuery insertEq(m_db);
00288         insertEq.prepare(
00289             "INSERT INTO equipment(athlete_id, name, category, notes) "
00290             "VALUES(:athlete_id, :name, :category, :notes)");
00291         insertEq.bindValue(":athlete_id", athleteId);
00292         insertEq.bindValue(":name", name);
00293         insertEq.bindValue(":category", name.compare(a.bike, Qt::CaseInsensitive) == 0 ? "Bike" :
"Component");
00294         insertEq.bindValue(":notes", QString());
00295         if (insertEq.exec())
00296             equipmentId = insertEq.lastInsertId().toInt();
00297     }
00298
00299     if (equipmentId <= 0)
00300         continue;
00301
00302     QSqlQuery usage(m_db);
00303     usage.prepare(
00304         "INSERT INTO equipment_usage(equipment_id, activity_id, distance_km, duration_sec) "
00305         "VALUES(:equipment_id, :activity_id, :distance_km, :duration_sec)");
00306     usage.bindValue(":equipment_id", equipmentId);
00307     usage.bindValue(":activity_id", activityId);
00308     usage.bindValue(":distance_km", distanceKm);
00309     usage.bindValue(":duration_sec", durationSec);
00310     usage.exec();
00311 }
00312
00313 return true;
00314 }

```

8.54 ActivityRepository.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QList>
00007 #include <QString>
00008
00009 class QSqlDatabase;
00010
00016 struct Activity
00017 {
00019     int id = -1;
00020
00022     int athleteId = -1;
00023
00025     QString fitHash;
00026
00028     QString fileName;
00029
00031     QString sport;
00032
00034     QString startTime;
00035
00037     QString endTime;
00038
00040     double durationSec = 0.0;
00041
00043     double distanceM = 0.0;
00044
00046     double avgPower = 0.0;
00047
00049     double maxPower = 0.0;
00050
00052     double normalizedPower = 0.0;
00053
00055     double avgHr = 0.0;
00056
00058     double maxHr = 0.0;
00059
00061     double avgCadence = 0.0;
00062
00064     double avgSpeed = 0.0;
00065
00067     double elevationGain = 0.0;
00068
00070     QString notes;
00071
00073     QString weatherNotes;
00074
00076     double temperature = 0.0;
00077

```

```

00079     QString weather;
00080
00082     QString wind;
00083
00085     int     rpe           = 0;
00086
00088     int     fatigue       = 0;
00089
00091     double  sleepHours    = 0.0;
00092
00094     double  weightKg      = 0.0;
00095
00097     QString bike;
00098
00100     QString equipment;
00101
00103     QString importedAt;
00104
00106     int     analysisVersion = 1;
00107
00109     QString fingerprint;
00110
00112     int     analysisFlags  = 0;
00113
00115     QString importSource;
00116
00121     bool isValid() const { return id > 0; }
00122 };
00123
00129 class ActivityRepository
00130 {
00131 public:
00136     explicit ActivityRepository(QSqlDatabase& db);
00137
00143     int         insertActivity(const Activity& a);
00144
00150     Activity    getActivity(int activityId);
00151
00157     QList<Activity> listActivities(int athleteId);
00158
00164     bool        deleteActivity(int activityId);
00165
00171     bool        hasFitHash(const QString& hash);
00172
00180     bool        updateNotes(int activityId, const QString& notes,
00181                             const QString& weatherNotes);
00182
00189     bool        updateActivityProperties(int activityId, const Activity& a);
00190
00197     bool        updateAnalysisFlags(int activityId, int flags);
00198
00205     bool        updateFingerprint(int activityId, const QString& fingerprint);
00206
00212     bool        hasFingerprintMatch(const QString& fingerprint);
00213
00220     bool        setImportSource(int activityId, const QString& source);
00221
00222 private:
00223     QSqlDatabase& m_db;
00224 };

```

8.55 AthleteRepository.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "AthleteRepository.h"
00005
00006 #include <QDateTime>
00007 #include <QSqlDatabase>
00008 #include <QSqlError>
00009 #include <QSqlQuery>
00010
00011 AthleteRepository::AthleteRepository(QSqlDatabase& db)
00012     : m_db(db)
00013 {}
00014
00015 // Athletes
00016
00017 int AthleteRepository::insertAthlete(const Athlete& a)
00018 {
00019     QSqlQuery q(m_db);
00020     q.prepare(
00021         "INSERT INTO athletes(first_name, last_name, date_of_birth, email, height_cm, ftp,

```

```

        created_at)"
00022     " VALUES(:fn, :ln, :dob, :email, :h, :ftp, :ts)");
00023     q.bindValue(":fn", a.firstName);
00024     q.bindValue(":ln", a.lastName);
00025     q.bindValue(":dob", a.dateOfBirth);
00026     q.bindValue(":email", a.email);
00027     q.bindValue(":h", a.heightCm > 0.0 ? QVariant(a.heightCm) : QVariant());
00028     q.bindValue(":ftp", a.ftpWatts > 0 ? a.ftpWatts : 250);
00029     q.bindValue(":ts", QDateTime::currentDateTimeUtc().toString(Qt::ISODate));
00030
00031     if (!q.exec()) return -1;
00032     return q.lastInsertId().toInt();
00033 }
00034
00035 bool AthleteRepository::updateAthlete(const Athlete& a)
00036 {
00037     QSqlQuery q(m_db);
00038     q.prepare(
00039         "UPDATE athletes SET first_name=:fn, last_name=:ln, date_of_birth=:dob,"
00040         " email=:email, height_cm=:h, ftp=:ftp WHERE id=:id");
00041     q.bindValue(":fn", a.firstName);
00042     q.bindValue(":ln", a.lastName);
00043     q.bindValue(":dob", a.dateOfBirth);
00044     q.bindValue(":email", a.email);
00045     q.bindValue(":h", a.heightCm > 0.0 ? QVariant(a.heightCm) : QVariant());
00046     q.bindValue(":ftp", a.ftpWatts > 0 ? a.ftpWatts : 250);
00047     q.bindValue(":id", a.id);
00048     return q.exec();
00049 }
00050
00051 bool AthleteRepository::deleteAthlete(int athleteId)
00052 {
00053     QSqlQuery q(m_db);
00054     q.prepare("DELETE FROM athletes WHERE id=:id");
00055     q.bindValue(":id", athleteId);
00056     return q.exec();
00057 }
00058
00059 QList<Athlete> AthleteRepository::listAthletes()
00060 {
00061     QList<Athlete> result;
00062     QSqlQuery q(m_db);
00063     q.exec("SELECT id, first_name, last_name, date_of_birth, email, height_cm, ftp, created_at"
00064         " FROM athletes ORDER BY last_name, first_name");
00065     while (q.next())
00066     {
00067         Athlete a;
00068         a.id = q.value(0).toInt();
00069         a.firstName = q.value(1).toString();
00070         a.lastName = q.value(2).toString();
00071         a.dateOfBirth = q.value(3).toString();
00072         a.email = q.value(4).toString();
00073         a.heightCm = q.value(5).toDouble();
00074         a.ftpWatts = q.value(6).toInt() > 0 ? q.value(6).toInt() : 250;
00075         a.created_at = q.value(7).toString();
00076         result.append(a);
00077     }
00078     return result;
00079 }
00080
00081 Athlete AthleteRepository::getAthlete(int athleteId)
00082 {
00083     QSqlQuery q(m_db);
00084     q.prepare("SELECT id, first_name, last_name, date_of_birth, email, height_cm, ftp, created_at"
00085         " FROM athletes WHERE id=:id");
00086     q.bindValue(":id", athleteId);
00087     q.exec();
00088     if (q.next())
00089     {
00090         Athlete a;
00091         a.id = q.value(0).toInt();
00092         a.firstName = q.value(1).toString();
00093         a.lastName = q.value(2).toString();
00094         a.dateOfBirth = q.value(3).toString();
00095         a.email = q.value(4).toString();
00096         a.heightCm = q.value(5).toDouble();
00097         a.ftpWatts = q.value(6).toInt() > 0 ? q.value(6).toInt() : 250;
00098         a.created_at = q.value(7).toString();
00099         return a;
00100     }
00101     return {};
00102 }
00103
00104 // FTP history
00105
00106 int AthleteRepository::addFtpEntry(const FtpEntry& e)
00107 {

```



```

00108     QSqlQuery q(m_db);
00109     q.prepare(
00110         "INSERT INTO ftp_history(athlete_id, ftp_watts, effective_from, notes)"
00111         " VALUES(:aid, :w, :from, :notes)");
00112     q.bindValue(":aid", e.athleteId);
00113     q.bindValue(":w", e.ftpWatts);
00114     q.bindValue(":from", e.effectiveFrom);
00115     q.bindValue(":notes", e.notes);
00116     if (!q.exec()) return -1;
00117     return q.lastInsertId().toInt();
00118 }
00119
00120 bool AthleteRepository::deleteFtpEntry(int entryId)
00121 {
00122     QSqlQuery q(m_db);
00123     q.prepare("DELETE FROM ftp_history WHERE id=:id");
00124     q.bindValue(":id", entryId);
00125     return q.exec();
00126 }
00127
00128 QList<FtpEntry> AthleteRepository::getFtpHistory(int athleteId)
00129 {
00130     QList<FtpEntry> result;
00131     QSqlQuery q(m_db);
00132     q.prepare("SELECT id, athlete_id, ftp_watts, effective_from, notes"
00133         " FROM ftp_history WHERE athlete_id=:aid ORDER BY effective_from DESC");
00134     q.bindValue(":aid", athleteId);
00135     q.exec();
00136     while (q.next())
00137     {
00138         FtpEntry e;
00139         e.id = q.value(0).toInt();
00140         e.athleteId = q.value(1).toInt();
00141         e.ftpWatts = q.value(2).toInt();
00142         e.effectiveFrom = q.value(3).toString();
00143         e.notes = q.value(4).toString();
00144         result.append(e);
00145     }
00146     return result;
00147 }
00148
00149 int AthleteRepository::getFtpForDate(int athleteId, const QDate& date)
00150 {
00151     // Find the most recent FTP entry on or before the given date.
00152     QSqlQuery q(m_db);
00153     q.prepare(
00154         "SELECT ftp_watts FROM ftp_history"
00155         " WHERE athlete_id=:aid AND effective_from <= :date"
00156         " ORDER BY effective_from DESC LIMIT 1");
00157     q.bindValue(":aid", athleteId);
00158     q.bindValue(":date", date.toString(Qt::ISODate));
00159     q.exec();
00160     if (q.next())
00161         return q.value(0).toInt();
00162     return 0;
00163 }
00164
00165 // Weight history
00166
00167 int AthleteRepository::addWeightEntry(const WeightEntry& e)
00168 {
00169     QSqlQuery q(m_db);
00170     q.prepare(
00171         "INSERT INTO weight_history(athlete_id, weight_kg, effective_from)"
00172         " VALUES(:aid, :w, :from)");
00173     q.bindValue(":aid", e.athleteId);
00174     q.bindValue(":w", e.weightKg);
00175     q.bindValue(":from", e.effectiveFrom);
00176     if (!q.exec()) return -1;
00177     return q.lastInsertId().toInt();
00178 }
00179
00180 bool AthleteRepository::deleteWeightEntry(int entryId)
00181 {
00182     QSqlQuery q(m_db);
00183     q.prepare("DELETE FROM weight_history WHERE id=:id");
00184     q.bindValue(":id", entryId);
00185     return q.exec();
00186 }
00187
00188 QList<WeightEntry> AthleteRepository::getWeightHistory(int athleteId)
00189 {
00190     QList<WeightEntry> result;
00191     QSqlQuery q(m_db);
00192     q.prepare("SELECT id, athlete_id, weight_kg, effective_from"
00193         " FROM weight_history WHERE athlete_id=:aid ORDER BY effective_from DESC");
00194     q.bindValue(":aid", athleteId);

```

```

00195     q.exec();
00196     while (q.next())
00197     {
00198         WeightEntry e;
00199         e.id          = q.value(0).toInt();
00200         e.athleteId   = q.value(1).toInt();
00201         e.weightKg     = q.value(2).toDouble();
00202         e.effectiveFrom = q.value(3).toString();
00203         result.append(e);
00204     }
00205     return result;
00206 }

```

8.56 AthleteRepository.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QDate>
00007 #include <QList>
00008 #include <QString>
00009
00010 class QSqlDatabase;
00011
00015 struct Athlete
00016 {
00018     int      id          = -1;
00019
00021     QString  firstName;
00022
00024     QString  lastName;
00025
00027     QString  dateOfBirth;
00028
00030     QString  email;
00031
00033     double   heightCm    = 0.0;
00034
00036     int      ftpWatts    = 250;
00037
00039     QString  createdAt;
00040
00044     QString  fullName() const { return firstName + " " + lastName; }
00045
00049     bool     isValid()  const { return id > 0; }
00050 };
00051
00055 struct FtpEntry
00056 {
00058     int      id          = -1;
00059
00061     int      athleteId   = -1;
00062
00064     int      ftpWatts     = 0;
00065
00067     QString  effectiveFrom;
00068
00070     QString  notes;
00071 };
00072
00076 struct WeightEntry
00077 {
00079     int      id          = -1;
00080
00082     int      athleteId   = -1;
00083
00085     double   weightKg     = 0.0;
00086
00088     QString  effectiveFrom;
00089 };
00090
00096 class AthleteRepository
00097 {
00098 public:
00103     explicit AthleteRepository(QSqlDatabase& db);
00104
00110     int      insertAthlete(const Athlete& a);
00111
00117     bool     updateAthlete(const Athlete& a);
00118
00124     bool     deleteAthlete(int athleteId);
00125

```

```

00130     QList<Athlete> listAthletes();
00131
00137     Athlete         getAthlete(int athleteId);
00138
00144     int             addFtpEntry(const FtpEntry& e);
00145
00151     bool            deleteFtpEntry(int entryId);
00152
00158     QList<FtpEntry> getFtpHistory(int athleteId);
00159
00166     int             getFtpForDate(int athleteId, const QDate& date);
00167
00173     int             addWeightEntry(const WeightEntry& e);
00174
00180     bool            deleteWeightEntry(int entryId);
00181
00187     QList<WeightEntry> getWeightHistory(int athleteId);
00188
00189 private:
00190     QSqlDatabase& m_db;
00191 };

```

8.57 ClimbRepository.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "ClimbRepository.h"
00005 #include "SqlQueryHelper.h"
00006
00007 #include <QSqlDatabase>
00008 #include <QSqlQuery>
00009 #include <QVariant>
00010
00011 namespace
00012 {
00013     const char* kSelectCols =
00014         "id, activity_id, COALESCE(uuid,''), start_seconds, end_seconds,"
00015         " COALESCE(original_start_seconds,-1.0), COALESCE(original_end_seconds,-1.0),"
00016         " COALESCE(name,''), COALESCE(elevation_gain_m,0.0), COALESCE(length_km,0.0),"
00017         " COALESCE(average_gradient,0.0),"
00018         " COALESCE(avg_power,0.0), COALESCE(np,0.0), COALESCE(avg_hr,0.0),"
00019         " COALESCE(avg_cadence,0.0), COALESCE(vam,0.0),"
00020         " COALESCE(source,'auto'), COALESCE(locked,0), COALESCE(deleted,0),"
00021         " COALESCE(algorithm_version,1),"
00022         " COALESCE(notes,''), COALESCE(favorite,0), COALESCE(rating,0)";
00023
00024     ClimbRecord recordFromQuery(QSqlQuery& q)
00025     {
00026         ClimbRecord r;
00027         int col = 0;
00028         r.id = q.value(col++).toInt();
00029         r.activityId = q.value(col++).toInt();
00030         r.uuid = q.value(col++).toString();
00031         r.startSeconds = q.value(col++).toDouble();
00032         r.endSeconds = q.value(col++).toDouble();
00033         r.originalStartSeconds = q.value(col++).toDouble();
00034         r.originalEndSeconds = q.value(col++).toDouble();
00035         r.name = q.value(col++).toString();
00036         r.elevationGainM = q.value(col++).toDouble();
00037         r.lengthKm = q.value(col++).toDouble();
00038         r.averageGradient = q.value(col++).toDouble();
00039         r.avgPower = q.value(col++).toDouble();
00040         r.np = q.value(col++).toDouble();
00041         r.avgHr = q.value(col++).toDouble();
00042         r.avgCadence = q.value(col++).toDouble();
00043         r.vam = q.value(col++).toDouble();
00044         r.source = q.value(col++).toString();
00045         r.locked = q.value(col++).toInt() != 0;
00046         r.deleted = q.value(col++).toInt() != 0;
00047         r.algorithmVersion = q.value(col++).toInt();
00048         r.notes = q.value(col++).toString();
00049         r.favorite = q.value(col++).toInt() != 0;
00050         r.rating = q.value(col++).toInt();
00051         return r;
00052     }
00053 } // namespace
00054
00055 ClimbRepository::ClimbRepository(QSqlDatabase& db)
00056 : m_db(db)
00057 {}
00058
00059 int ClimbRepository::insertClimb(const ClimbRecord& climb)
00060 {

```

```

00061     QSqlQuery q(m_db);
00062     q.prepare(
00063         "INSERT INTO climbs("
00064         " activity_id, uuid, start_seconds, end_seconds,"
00065         " original_start_seconds, original_end_seconds,"
00066         " name, elevation_gain_m, length_km, average_gradient,"
00067         " avg_power, np, avg_hr, avg_cadence, vam,"
00068         " source, locked, deleted, algorithm_version,"
00069         " notes, favorite, rating"
00070         ") VALUES ("
00071         " :activity_id, :uuid, :start_seconds, :end_seconds,"
00072         " :orig_start, :orig_end,"
00073         " :name, :elevation_gain_m, :length_km, :average_gradient,"
00074         " :avg_power, :np, :avg_hr, :avg_cadence, :vam,"
00075         " :source, :locked, :deleted, :algorithm_version,"
00076         " :notes, :favorite, :rating"
00077         ")");
00078
00079     q.bindValue(":activity_id", climb.activityId);
00080     q.bindValue(":uuid", climb.uuid.isEmpty() ? QVariant() : QVariant(climb.uuid));
00081     q.bindValue(":start_seconds", climb.startSeconds);
00082     q.bindValue(":end_seconds", climb.endSeconds);
00083     q.bindValue(":orig_start", climb.originalStartSeconds < 0 ? QVariant() :
00084         QVariant(climb.originalStartSeconds));
00084     q.bindValue(":orig_end", climb.originalEndSeconds < 0 ? QVariant() :
00085         QVariant(climb.originalEndSeconds));
00085     q.bindValue(":name", climb.name);
00086     q.bindValue(":elevation_gain_m", climb.elevationGainM);
00087     q.bindValue(":length_km", climb.lengthKm);
00088     q.bindValue(":average_gradient", climb.averageGradient);
00089     q.bindValue(":avg_power", climb.avgPower);
00090     q.bindValue(":np", climb.np);
00091     q.bindValue(":avg_hr", climb.avgHr);
00092     q.bindValue(":avg_cadence", climb.avgCadence);
00093     q.bindValue(":vam", climb.vam);
00094     q.bindValue(":source", climb.source);
00095     SqlQueryHelper::bindBoolValue(q, ":locked", climb.locked);
00096     SqlQueryHelper::bindBoolValue(q, ":deleted", climb.deleted);
00097     q.bindValue(":algorithm_version", climb.algorithmVersion);
00098     q.bindValue(":notes", climb.notes);
00099     q.bindValue(":favorite", climb.favorite ? 1 : 0);
00100     q.bindValue(":rating", climb.rating);
00101
00102     if (!q.exec())
00103         return -1;
00104     return q.lastInsertId().toInt();
00105 }
00106
00107 bool ClimbRepository::updateClimb(const ClimbRecord& climb)
00108 {
00109     QSqlQuery q(m_db);
00110     q.prepare(
00111         "UPDATE climbs SET"
00112         " start_seconds = :start_seconds,"
00113         " end_seconds = :end_seconds,"
00114         " name = :name,"
00115         " elevation_gain_m = :elevation_gain_m,"
00116         " length_km = :length_km,"
00117         " average_gradient = :average_gradient,"
00118         " avg_power = :avg_power,"
00119         " np = :np,"
00120         " avg_hr = :avg_hr,"
00121         " avg_cadence = :avg_cadence,"
00122         " vam = :vam,"
00123         " source = :source,"
00124         " locked = :locked,"
00125         " deleted = :deleted,"
00126         " notes = :notes,"
00127         " favorite = :favorite,"
00128         " rating = :rating,"
00129         " updated_at = CURRENT_TIMESTAMP"
00130         " WHERE id = :id");
00131
00132     q.bindValue(":start_seconds", climb.startSeconds);
00133     q.bindValue(":end_seconds", climb.endSeconds);
00134     q.bindValue(":name", climb.name);
00135     q.bindValue(":elevation_gain_m", climb.elevationGainM);
00136     q.bindValue(":length_km", climb.lengthKm);
00137     q.bindValue(":average_gradient", climb.averageGradient);
00138     q.bindValue(":avg_power", climb.avgPower);
00139     q.bindValue(":np", climb.np);
00140     q.bindValue(":avg_hr", climb.avgHr);
00141     q.bindValue(":avg_cadence", climb.avgCadence);
00142     q.bindValue(":vam", climb.vam);
00143     q.bindValue(":source", climb.source);
00144     SqlQueryHelper::bindBoolValue(q, ":locked", climb.locked);
00145     SqlQueryHelper::bindBoolValue(q, ":deleted", climb.deleted);

```

```

00146     q.bindValue(":notes", climb.notes);
00147     q.bindValue(":favorite", climb.favorite ? 1 : 0);
00148     q.bindValue(":rating", climb.rating);
00149     q.bindValue(":id", climb.id);
00150     return q.exec();
00151 }
00152
00153 bool ClimbRepository::softDeleteClimb(int climbId)
00154 {
00155     QSqlQuery q(m_db);
00156     q.prepare("UPDATE climbs SET deleted=1, locked=1, updated_at=CURRENT_TIMESTAMP WHERE id=:id");
00157     q.bindValue(":id", climbId);
00158     return q.exec();
00159 }
00160
00161 bool ClimbRepository::hardDeleteClimb(int climbId)
00162 {
00163     QSqlQuery q(m_db);
00164     q.prepare("DELETE FROM climbs WHERE id=:id");
00165     q.bindValue(":id", climbId);
00166     return q.exec();
00167 }
00168
00169 bool ClimbRepository::deleteClimbsForActivity(int activityId)
00170 {
00171     QSqlQuery q(m_db);
00172     q.prepare("DELETE FROM climbs WHERE activity_id=:id");
00173     q.bindValue(":id", activityId);
00174     return q.exec();
00175 }
00176
00177 ClimbRecord ClimbRepository::getClimb(int climbId)
00178 {
00179     QSqlQuery q(m_db);
00180     q.prepare(QString("SELECT %1 FROM climbs WHERE id=:id").arg(kSelectCols));
00181     q.bindValue(":id", climbId);
00182     if (q.exec() && q.next())
00183         return recordFromQuery(q);
00184     return {};
00185 }
00186
00187 QList<ClimbRecord> ClimbRepository::climbsForActivity(int activityId)
00188 {
00189     QList<ClimbRecord> out;
00190     QSqlQuery q(m_db);
00191     q.prepare(QString("SELECT %1 FROM climbs WHERE activity_id=:id AND COALESCE(deleted,0)=0"
00192         " ORDER BY start_seconds ASC").arg(kSelectCols));
00193     q.bindValue(":id", activityId);
00194     if (!q.exec())
00195         return out;
00196     while (q.next())
00197         out.push_back(recordFromQuery(q));
00198     return out;
00199 }
00200
00201 QList<ClimbRecord> ClimbRepository::deletedClimbsForActivity(int activityId)
00202 {
00203     QList<ClimbRecord> out;
00204     QSqlQuery q(m_db);
00205     q.prepare(QString("SELECT %1 FROM climbs WHERE activity_id=:id AND COALESCE(deleted,0)=1"
00206         " ORDER BY start_seconds ASC").arg(kSelectCols));
00207     q.bindValue(":id", activityId);
00208     if (!q.exec())
00209         return out;
00210     while (q.next())
00211         out.push_back(recordFromQuery(q));
00212     return out;
00213 }
00214
00215 bool ClimbRepository::hasClimbs(int activityId)
00216 {
00217     QSqlQuery q(m_db);
00218     q.prepare(SqlQueryHelper::selectExistsNonDeleted("climbs"));
00219     q.bindValue(":id", activityId);
00220     return q.exec() && q.next();
00221 }
00222
00223 bool ClimbRepository::hasClimbsEver(int activityId)
00224 {
00225     QSqlQuery q(m_db);
00226     q.prepare(SqlQueryHelper::selectExistsEver("climbs"));
00227     q.bindValue(":id", activityId);
00228     return q.exec() && q.next();
00229 }
00230
00231 bool ClimbRepository::hasLockedClimbs(int activityId)
00232 {

```

```

00233     QSqlQuery q(m_db);
00234     q.prepare(SqlQueryHelper::selectExistsLocked("climbs"));
00235     q.bindValue(":id", activityId);
00236     return q.exec() && q.next();
00237 }

```

8.58 ClimbRepository.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QList>
00007 #include <QString>
00008
00009 class QSqlDatabase;
00010
00016 struct ClimbRecord
00017 {
00019     int id = -1;
00022     int activityId = -1;
00023
00025     QString uuid;
00026
00028     double startSeconds = 0.0;
00029
00031     double endSeconds = 0.0;
00032
00034     double originalStartSeconds = -1.0;
00035
00037     double originalEndSeconds = -1.0;
00038
00040     QString name;
00041
00043     double elevationGainM = 0.0;
00044
00046     double lengthKm = 0.0;
00047
00049     double averageGradient = 0.0;
00050
00052     double avgPower = 0.0;
00053
00055     double np = 0.0;
00056
00058     double avgHr = 0.0;
00059
00061     double avgCadence = 0.0;
00062
00064     double vam = 0.0;
00065
00067     QString source = QStringLiteral("auto");
00068
00070     bool locked = false;
00071
00073     bool deleted = false;
00074
00076     int algorithmVersion = 1;
00077
00079     QString notes;
00080
00082     bool favorite = false;
00083
00085     int rating = 0;
00086 };
00087
00093 class ClimbRepository
00094 {
00095 public:
00100     explicit ClimbRepository(QSqlDatabase& db);
00101
00107     int insertClimb(const ClimbRecord& climb);
00108
00114     bool updateClimb(const ClimbRecord& climb);
00115
00121     bool softDeleteClimb(int climbId);
00122
00128     bool hardDeleteClimb(int climbId);
00129
00135     bool deleteClimbsForActivity(int activityId);
00136
00142     ClimbRecord getClimb(int climbId);
00143

```

```

00149     QList<ClimbRecord> climbsForActivity(int activityId);
00150
00156     QList<ClimbRecord> deletedClimbsForActivity(int activityId);
00157
00163     bool hasClimbs(int activityId);
00164
00170     bool hasClimbsEver(int activityId);
00171
00177     bool hasLockedClimbs(int activityId);
00178
00179 private:
00180     QSqlDatabase& m_db;
00181 };

```

8.59 DatabaseConnectionGuard.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "DatabaseConnectionGuard.h"
00005
00006 DatabaseConnectionGuard::DatabaseConnectionGuard(const QString& connectionName)
00007     : m_connectionName(connectionName)
00008 {
00009     // Create the connection
00010     if (!QSqlDatabase::contains(connectionName))
00011     {
00012         QSqlDatabase::addDatabase(QStringLiteral("QSQLITE"), connectionName);
00013     }
00014 }
00015
00016 DatabaseConnectionGuard::~DatabaseConnectionGuard()
00017 {
00018     // Ensure connection is closed before removal
00019     if (QSqlDatabase::contains(m_connectionName))
00020     {
00021         {
00022             QSqlDatabase db = QSqlDatabase::database(m_connectionName, false);
00023             if (db.isOpen())
00024             {
00025                 db.close();
00026             }
00027         }
00028         // Remove the connection from Qt's registry
00029         QSqlDatabase::removeDatabase(m_connectionName);
00030     }
00031 }
00032
00033 QSqlDatabase DatabaseConnectionGuard::database() const
00034 {
00035     return QSqlDatabase::database(m_connectionName, false);
00036 }

```

8.60 DatabaseConnectionGuard.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QSqlDatabase>
00007 #include <QString>
00008
00015 class DatabaseConnectionGuard
00016 {
00017 public:
00022     explicit DatabaseConnectionGuard(const QString& connectionName);
00023     ~DatabaseConnectionGuard();
00024
00025     // Non-copyable
00026     DatabaseConnectionGuard(const DatabaseConnectionGuard&) = delete;
00027     DatabaseConnectionGuard& operator=(const DatabaseConnectionGuard&) = delete;
00028
00033     QSqlDatabase database() const;
00034
00035 private:
00036     QString m_connectionName;
00037 };

```

8.61 DatabaseManager.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "DatabaseManager.h"
00005 #include "DatabaseSchema.h"
00006
00007 #include <QDateTime>
00008 #include <QSqlError>
00009 #include <QSqlQuery>
00010 #include <QUuid>
00011
00012 DatabaseManager::DatabaseManager() = default;
00013
00014 DatabaseManager::~DatabaseManager()
00015 {
00016     close();
00017 }
00018
00019 // Private helpers
00020
00021 static void applySqlitePragmas(QSqlDatabase& db)
00022 {
00023     QSqlQuery q(db);
00024     q.exec("PRAGMA journal_mode=WAL");
00025     q.exec("PRAGMA synchronous=NORMAL");
00026     q.exec("PRAGMA temp_store=MEMORY");
00027     q.exec("PRAGMA foreign_keys=ON");
00028     q.exec("PRAGMA cache_size=-20000");
00029 }
00030
00031 static bool columnExists(QSqlDatabase& db, const QString& tableName, const QString& columnName)
00032 {
00033     QSqlQuery q(db);
00034     q.exec(QString("PRAGMA table_info(%1)").arg(tableName));
00035     while (q.next())
00036     {
00037         if (q.value(1).toString().compare(columnName, Qt::CaseInsensitive) == 0)
00038             return true;
00039     }
00040     return false;
00041 }
00042
00043 static bool ensureActivityTimeColumns(QSqlDatabase& db, QString* errorOut)
00044 {
00045     if (!columnExists(db, "activities", "activity_start_time"))
00046     {
00047         QSqlQuery q(db);
00048         if (!q.exec("ALTER TABLE activities ADD COLUMN activity_start_time TEXT"))
00049         {
00050             if (errorOut)
00051                 *errorOut = q.lastError().text();
00052             return false;
00053         }
00054     }
00055
00056     if (!columnExists(db, "activities", "import_time"))
00057     {
00058         QSqlQuery q(db);
00059         if (!q.exec("ALTER TABLE activities ADD COLUMN import_time TEXT"))
00060         {
00061             if (errorOut)
00062                 *errorOut = q.lastError().text();
00063             return false;
00064         }
00065     }
00066
00067     const bool hasLegacyStart = columnExists(db, "activities", "start_time");
00068     const bool hasLegacyImported = columnExists(db, "activities", "imported_at");
00069
00070     QString startExpr = QStringLiteral("activity_start_time = COALESCE(activity_start_time");
00071     if (hasLegacyStart)
00072         startExpr += QStringLiteral(", start_time");
00073     if (hasLegacyImported)
00074         startExpr += QStringLiteral(", imported_at");
00075     startExpr += QStringLiteral(", datetime('now')");
00076
00077     QString importExpr = QStringLiteral("import_time = COALESCE(import_time");
00078     if (hasLegacyImported)
00079         importExpr += QStringLiteral(", imported_at");
00080     importExpr += QStringLiteral(", datetime('now')");
00081
00082     {
00083         QSqlQuery q(db);
00084         if (!q.exec(QString("UPDATE activities SET %1, %2").arg(startExpr, importExpr)))

```



```

00085     {
00086         if (errorOut)
00087             *errorOut = q.lastError().text();
00088         return false;
00089     }
00090 }
00091
00092 {
00093     QSqlQuery q(db);
00094     q.exec("CREATE INDEX IF NOT EXISTS idx_activities_activity_start_time ON
activities(activity_start_time DESC)");
00095 }
00096
00097     return true;
00098 }
00099
00100 bool DatabaseManager::applySchema(QString* errorOut)
00101 {
00102     QSqlQuery q(m_db);
00103
00104     // Run every DDL statement from the schema
00105     for (const auto& stmt : DatabaseSchema::kStatements)
00106     {
00107         const QString sql = QString::fromUtf8(stmt.data(), static_cast<qsize_t>(stmt.size()));
00108         if (!q.exec(sql))
00109         {
00110             if (errorOut)
00111                 *errorOut = q.lastError().text();
00112             return false;
00113         }
00114     }
00115
00116     if (!ensureActivityTimeColumns(m_db, errorOut))
00117         return false;
00118
00119     // Write schema version
00120     q.prepare("INSERT OR REPLACE INTO schema_info(version, created_at) VALUES(:v, :ts)");
00121     q.bindValue(":v", DatabaseSchema::kCurrentVersion);
00122     q.bindValue(":ts", QDateTime::currentDateTimeUtc().toString(Qt::ISODate));
00123     if (!q.exec())
00124     {
00125         if (errorOut)
00126             *errorOut = q.lastError().text();
00127         return false;
00128     }
00129
00130     return true;
00131 }
00132
00133 bool DatabaseManager::upgradeSchema(int fromVersion, QString* errorOut)
00134 {
00135     if (fromVersion < 2 && !columnExists(m_db, "athletes", "ftp"))
00136     {
00137         QSqlQuery q(m_db);
00138         if (!q.exec("ALTER TABLE athletes ADD COLUMN ftp INTEGER NOT NULL DEFAULT 250"))
00139         {
00140             if (errorOut)
00141                 *errorOut = q.lastError().text();
00142             return false;
00143         }
00144     }
00145
00146     if (fromVersion < 3)
00147     {
00148         struct ActivityColumnDef
00149         {
00150             const char* name;
00151             const char* definition;
00152         };
00153
00154         static const ActivityColumnDef kActivityColumns[] = {
00155             {"temperature", "REAL"},
00156             {"weather", "TEXT"},
00157             {"wind", "TEXT"},
00158             {"rpe", "INTEGER"},
00159             {"fatigue", "INTEGER"},
00160             {"sleep", "REAL"},
00161             {"weight", "REAL"},
00162             {"bike", "TEXT"},
00163             {"equipment", "TEXT"},
00164         };
00165
00166         for (const ActivityColumnDef& col : kActivityColumns)
00167         {
00168             if (columnExists(m_db, "activities", col.name))
00169                 continue;
00170

```

```

00171         QSqlQuery q(m_db);
00172         const QString sql = QString("ALTER TABLE activities ADD COLUMN %1 %2")
00173             .arg(col.name)
00174             .arg(col.definition);
00175         if (!q.exec(sql))
00176         {
00177             if (errorOut)
00178                 *errorOut = q.lastError().text();
00179             return false;
00180         }
00181     }
00182 }
00183
00184 if (fromVersion < 4)
00185 {
00186     if (!ensureActivityTimeColumns(m_db, errorOut))
00187         return false;
00188 }
00189
00190 if (fromVersion < 5)
00191 {
00192     QSqlQuery q(m_db);
00193     q.exec(
00194         "CREATE INDEX IF NOT EXISTS idx_activities_athlete_start_time"
00195         " ON activities(athlete_id, activity_start_time DESC)");
00196 }
00197
00198 if (fromVersion < 6)
00199 {
00200     // Add climbs table
00201     {
00202         QSqlQuery q(m_db);
00203         q.exec(
00204             "CREATE TABLE IF NOT EXISTS climbs ("
00205             "     id                INTEGER PRIMARY KEY AUTOINCREMENT,"
00206             "     activity_id       INTEGER NOT NULL REFERENCES activities(id) ON DELETE CASCADE,"
00207             "     start_seconds     REAL    NOT NULL,"
00208             "     end_seconds       REAL    NOT NULL,"
00209             "     name              TEXT,"
00210             "     elevation_gain_m  REAL,"
00211             "     length_km         REAL,"
00212             "     average_gradient  REAL,"
00213             "     source            TEXT    NOT NULL DEFAULT 'auto',"
00214             "     locked            INTEGER NOT NULL DEFAULT 0,"
00215             "     created_at        TEXT    DEFAULT CURRENT_TIMESTAMP,"
00216             "     updated_at        TEXT    DEFAULT CURRENT_TIMESTAMP"
00217             " );");
00218     }
00219     {
00220         QSqlQuery q(m_db);
00221         q.exec("CREATE INDEX IF NOT EXISTS idx_climbs_activity_id ON climbs(activity_id)");
00222     }
00223
00224     // Add source, locked, updated_at to intervals
00225     struct ColDef { const char* name; const char* definition; };
00226     static const ColDef kIntervalCols[] = {
00227         { "source", "TEXT NOT NULL DEFAULT 'auto' " },
00228         { "locked", "INTEGER NOT NULL DEFAULT 0 " },
00229         { "updated_at", "TEXT " },
00230     };
00231     for (const ColDef& col : kIntervalCols)
00232     {
00233         if (columnExists(m_db, "intervals", col.name))
00234             continue;
00235         QSqlQuery q(m_db);
00236         q.exec(QString("ALTER TABLE intervals ADD COLUMN %1 %2")
00237             .arg(col.name).arg(col.definition));
00238     }
00239
00240     // Indexes on activities for fast overlap / hash lookup
00241     {
00242         QSqlQuery q(m_db);
00243         q.exec(
00244             "CREATE UNIQUE INDEX IF NOT EXISTS idx_activity_filehash"
00245             " ON activities(fit_hash) WHERE fit_hash IS NOT NULL");
00246     }
00247     {
00248         QSqlQuery q(m_db);
00249         q.exec("CREATE INDEX IF NOT EXISTS idx_activity_start_time ON
activities(activity_start_time)");
00250     }
00251     {
00252         QSqlQuery q(m_db);
00253         q.exec("CREATE INDEX IF NOT EXISTS idx_activity_end_time ON activities(end_time)");
00254     }
00255 }
00256

```

```

00257     if (fromVersion < 7)
00258     {
00259         // activities: analysis_version, fingerprint, analysis_flags, import_source
00260         struct ColDef7a { const char* name; const char* def; };
00261         static const ColDef7a kActivityCols[] = {
00262             { "analysis_version", "INTEGER NOT NULL DEFAULT 1" },
00263             { "fingerprint", "TEXT" },
00264             { "analysis_flags", "INTEGER NOT NULL DEFAULT 0" },
00265             { "import_source", "TEXT" },
00266         };
00267         for (const ColDef7a& col : kActivityCols)
00268         {
00269             if (columnExists(m_db, "activities", col.name)) continue;
00270             QSqlQuery q(m_db);
00271             q.exec(QString("ALTER TABLE activities ADD COLUMN %1 %2").arg(col.name).arg(col.def));
00272         }
00273
00274         // intervals: algorithm_version, deleted, uuid
00275         struct ColDef7b { const char* name; const char* def; };
00276         static const ColDef7b kIntervalCols[] = {
00277             { "algorithm_version", "INTEGER NOT NULL DEFAULT 1" },
00278             { "deleted", "INTEGER NOT NULL DEFAULT 0" },
00279             { "uuid", "TEXT" },
00280         };
00281         for (const ColDef7b& col : kIntervalCols)
00282         {
00283             if (columnExists(m_db, "intervals", col.name)) continue;
00284             QSqlQuery q(m_db);
00285             q.exec(QString("ALTER TABLE intervals ADD COLUMN %1 %2").arg(col.name).arg(col.def));
00286         }
00287
00288         // climbs: algorithm_version, deleted, uuid, original bounds, statistics, notes, favorite,
00289         rating
00290         struct ColDef7c { const char* name; const char* def; };
00291         static const ColDef7c kClimbCols[] = {
00292             { "algorithm_version", "INTEGER NOT NULL DEFAULT 1" },
00293             { "deleted", "INTEGER NOT NULL DEFAULT 0" },
00294             { "uuid", "TEXT" },
00295             { "original_start_seconds", "REAL" },
00296             { "original_end_seconds", "REAL" },
00297             { "avg_power", "REAL" },
00298             { "np", "REAL" },
00299             { "avg_hr", "REAL" },
00300             { "avg_cadence", "REAL" },
00301             { "vam", "REAL" },
00302             { "notes", "TEXT" },
00303             { "favorite", "INTEGER NOT NULL DEFAULT 0" },
00304             { "rating", "INTEGER NOT NULL DEFAULT 0" },
00305         };
00306         for (const ColDef7c& col : kClimbCols)
00307         {
00308             if (columnExists(m_db, "climbs", col.name)) continue;
00309             QSqlQuery q(m_db);
00310             q.exec(QString("ALTER TABLE climbs ADD COLUMN %1 %2").arg(col.name).arg(col.def));
00311         }
00312
00313         // new tables
00314         {
00315             QSqlQuery q(m_db);
00316             q.exec(
00317                 "CREATE TABLE IF NOT EXISTS activity_analysis_settings ("
00318                 "    activity_id          INTEGER PRIMARY KEY,"
00319                 "    climb_min_gain      REAL,"
00320                 "    climb_min_length   REAL,"
00321                 "    climb_min_gradient REAL,"
00322                 "    interval_work_threshold REAL,"
00323                 "    interval_rest_threshold REAL,"
00324                 "    FOREIGN KEY(activity_id) REFERENCES activities(id) ON DELETE CASCADE"
00325                 ")");
00326         }
00327         {
00328             QSqlQuery q(m_db);
00329             q.exec(
00330                 "CREATE TABLE IF NOT EXISTS activity_analysis_cache ("
00331                 "    activity_id INTEGER NOT NULL,"
00332                 "    cache_key   TEXT    NOT NULL,"
00333                 "    cache_value BLOB,"
00334                 "    PRIMARY KEY (activity_id, cache_key)"
00335                 ")");
00336         }
00337
00338     if (fromVersion < 8)
00339     {
00340         // Version 8: Add performance-critical indexes
00341         {
00342             QSqlQuery q(m_db);

```

```

00343         q.exec("CREATE INDEX IF NOT EXISTS idx_activity_samples_time"
00344             " ON activity_samples(activity_id, elapsed_seconds)");
00345     }
00346     {
00347         QSqlQuery q(m_db);
00348         q.exec("CREATE INDEX IF NOT EXISTS idx_imports_date"
00349             " ON imports(imported_at)");
00350     }
00351     {
00352         QSqlQuery q(m_db);
00353         q.exec("CREATE INDEX IF NOT EXISTS idx_workouts_athlete_date"
00354             " ON planned_workouts(athlete_id, workout_date)");
00355     }
00356 }
00357
00358 return applySchema(errorOut);
00359 }
00360
00361 // Public API
00362
00363 bool DatabaseManager::open(const QString& path, QString* errorOut)
00364 {
00365     close();
00366
00367     m_connectionName = QUuid::createUuid().toString(QUuid::WithoutBraces);
00368     m_db = QSqlDatabase::addDatabase("QSQLITE", m_connectionName);
00369     m_db.setDatabaseName(path);
00370
00371     if (!m_db.open())
00372     {
00373         if (errorOut)
00374             *errorOut = m_db.lastError().text();
00375         QSqlDatabase::removeDatabase(m_connectionName);
00376         m_connectionName.clear();
00377         return false;
00378     }
00379
00380     applySqlitePragmas(m_db);
00381
00382     // Check schema version
00383     QSqlQuery q(m_db);
00384     q.exec("SELECT version FROM schema_info ORDER BY rowid DESC LIMIT 1");
00385     if (q.next())
00386     {
00387         const int version = q.value(0).toInt();
00388         if (version < DatabaseSchema::kCurrentVersion)
00389         {
00390             if (!upgradeSchema(version, errorOut))
00391             {
00392                 cleanupFailedOpen();
00393                 return false;
00394             }
00395         }
00396     }
00397     else
00398     {
00399         // schema_info missing: fresh file or legacy -- apply schema
00400         if (!applySchema(errorOut))
00401         {
00402             cleanupFailedOpen();
00403             return false;
00404         }
00405     }
00406
00407     m_path = path;
00408     return true;
00409 }
00410
00411 void DatabaseManager::cleanupFailedOpen()
00412 {
00413     if (m_db.isOpen())
00414         m_db.close();
00415
00416     if (!m_connectionName.isEmpty())
00417     {
00418         QString conn = m_connectionName;
00419         m_db = QSqlDatabase();
00420         QSqlDatabase::removeDatabase(conn);
00421         m_connectionName.clear();
00422     }
00423 }
00424
00425 bool DatabaseManager::create(const QString& path, QString* errorOut)
00426 {
00427     // Sqlite creates the file if it doesn't exist -- same as open
00428     return open(path, errorOut);
00429 }

```

```

00430
00431 void DatabaseManager::close()
00432 {
00433     if (m_db.isOpen())
00434         m_db.close();
00435
00436     if (!m_connectionName.isEmpty())
00437     {
00438         QSqlDatabase::removeDatabase(m_connectionName);
00439         m_connectionName.clear();
00440     }
00441
00442     m_path.clear();
00443     m_db = QSqlDatabase(); // reset to default-constructed
00444 }
00445
00446 bool DatabaseManager::isOpen() const
00447 {
00448     return m_db.isValid() && m_db.isOpen();
00449 }
00450
00451 QSqlDatabase DatabaseManager::database() const
00452 {
00453     return m_db;
00454 }

```

8.62 DatabaseManager.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QSqlDatabase>
00007 #include <QString>
00008
00009 // DatabaseManager owns a single named QSqlDatabase connection.
00010 // Usage:
00011 //     DatabaseManager db;
00012 //     if (!db.open("/path/to/file.fitlyzerdb", &errorMsg)) { ... }
00013 //     db.database(); // use for queries
00020 class DatabaseManager
00021 {
00022 public:
00023     DatabaseManager();
00024     ~DatabaseManager();
00025
00032     bool open(const QString& path, QString* errorOut = nullptr);
00033
00040     bool create(const QString& path, QString* errorOut = nullptr);
00041
00045     void close();
00046
00051     bool isOpen() const;
00052
00057     QSqlDatabase database() const;
00058
00063     QString path() const { return m_path; }
00064
00065 private:
00066     bool applySchema(QString* errorOut);
00067     bool upgradeSchema(int fromVersion, QString* errorOut);
00068     void cleanupFailedOpen();
00069
00070     QString m_path;
00071     QString m_connectionName;
00072     QSqlDatabase m_db;
00073 };

```

8.63 DatabaseSchema.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "DatabaseSchema.h"
00005 // All definitions are header-only (constexpr / inline).

```

8.64 DatabaseSchema.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <array>
00007 #include <string_view>
00008
00009 namespace DatabaseSchema {
00010
00011     static constexpr int kCurrentVersion = 8;
00012
00013     inline constexpr std::array kStatements = {
00014         std::string_view{
00015             "CREATE TABLE IF NOT EXISTS schema_info ("
00016             "    version    INTEGER NOT NULL,"
00017             "    created_at TEXT    NOT NULL"
00018             ")"
00019         },
00020         std::string_view{
00021             "CREATE TABLE IF NOT EXISTS athletes ("
00022             "    id          INTEGER PRIMARY KEY AUTOINCREMENT,"
00023             "    first_name TEXT    NOT NULL,"
00024             "    last_name  TEXT    NOT NULL,"
00025             "    date_of_birth TEXT,"
00026             "    email      TEXT,"
00027             "    height_cm  REAL,"
00028             "    ftp        INTEGER NOT NULL DEFAULT 250,"
00029             "    created_at TEXT    NOT NULL"
00030             ")"
00031         },
00032         std::string_view{
00033             "CREATE TABLE IF NOT EXISTS ftp_history ("
00034             "    id          INTEGER PRIMARY KEY AUTOINCREMENT,"
00035             "    athlete_id  INTEGER NOT NULL REFERENCES athletes(id) ON DELETE CASCADE,"
00036             "    ftp_watts   INTEGER NOT NULL,"
00037             "    effective_from TEXT    NOT NULL,"
00038             "    notes      TEXT"
00039             ")"
00040         },
00041         std::string_view{
00042             "CREATE TABLE IF NOT EXISTS weight_history ("
00043             "    id          INTEGER PRIMARY KEY AUTOINCREMENT,"
00044             "    athlete_id  INTEGER NOT NULL REFERENCES athletes(id) ON DELETE CASCADE,"
00045             "    weight_kg    REAL    NOT NULL,"
00046             "    effective_from TEXT    NOT NULL"
00047             ")"
00048         },
00049         std::string_view{
00050             "CREATE TABLE IF NOT EXISTS activities ("
00051             "    id          INTEGER PRIMARY KEY AUTOINCREMENT,"
00052             "    athlete_id  INTEGER NOT NULL REFERENCES athletes(id) ON DELETE CASCADE,"
00053             "    fit_hash     TEXT    UNIQUE,"
00054             "    file_name    TEXT,"
00055             "    sport        TEXT,"
00056             "    activity_start_time TEXT NOT NULL,"
00057             "    end_time     TEXT,"
00058             "    duration_sec REAL,"
00059             "    distance_m   REAL,"
00060             "    avg_power    REAL,"
00061             "    max_power    REAL,"
00062             "    normalized_power REAL,"
00063             "    avg_hr       REAL,"
00064             "    max_hr       REAL,"
00065             "    avg_cadence  REAL,"
00066             "    avg_speed    REAL,"
00067             "    elevation_gain REAL,"
00068             "    notes        TEXT,"
00069             "    weather_notes TEXT,"
00070             "    temperature  REAL,"
00071             "    weather      TEXT,"
00072             "    wind         TEXT,"
00073             "    rpe         INTEGER,"
00074             "    fatigue      INTEGER,"
00075             "    sleep        REAL,"
00076             "    weight        REAL,"
00077             "    bike          TEXT,"
00078             "    equipment     TEXT,"
00079             "    import_time   TEXT    NOT NULL,"
00080             "    analysis_version INTEGER NOT NULL DEFAULT 1,"
00081             "    fingerprint  TEXT,"
00082             "    analysis_flags INTEGER NOT NULL DEFAULT 0,"
00083             "    import_source TEXT"
00084             ")"
00085         }
00086     };
00087
00088 }

```

```

00094     },
00095     std::string_view{
00096         "CREATE INDEX IF NOT EXISTS idx_activities_activity_start_time"
00097         " ON activities(activity_start_time DESC)"
00098     },
00099     std::string_view{
00100         "CREATE INDEX IF NOT EXISTS idx_activities_athlete_start_time"
00101         " ON activities(athlete_id, activity_start_time DESC)"
00102     },
00103     std::string_view{
00104         "CREATE TABLE IF NOT EXISTS activity_samples ("
00105         " id INTEGER PRIMARY KEY AUTOINCREMENT,"
00106         " activity_id INTEGER NOT NULL REFERENCES activities(id) ON DELETE CASCADE,"
00107         " elapsed_seconds REAL NOT NULL,"
00108         " power_total REAL,"
00109         " heart_rate REAL,"
00110         " cadence REAL,"
00111         " speed REAL,"
00112         " latitude REAL,"
00113         " longitude REAL,"
00114         " altitude REAL,"
00115         " has_gps INTEGER NOT NULL DEFAULT 0,"
00116         " has_power INTEGER NOT NULL DEFAULT 0,"
00117         " has_heart_rate INTEGER NOT NULL DEFAULT 0,"
00118         " has_cadence INTEGER NOT NULL DEFAULT 0,"
00119         " has_speed INTEGER NOT NULL DEFAULT 0,"
00120         " has_altitude INTEGER NOT NULL DEFAULT 0"
00121         ") "
00122     },
00123     std::string_view{
00124         "CREATE INDEX IF NOT EXISTS idx_activity_samples_activity_id"
00125         " ON activity_samples(activity_id)"
00126     },
00127     std::string_view{
00128         "CREATE INDEX IF NOT EXISTS idx_activity_samples_time"
00129         " ON activity_samples(activity_id, elapsed_seconds)"
00130     },
00131     std::string_view{
00132         "CREATE TABLE IF NOT EXISTS laps ("
00133         " id INTEGER PRIMARY KEY AUTOINCREMENT,"
00134         " activity_id INTEGER NOT NULL REFERENCES activities(id) ON DELETE CASCADE,"
00135         " lap_number INTEGER NOT NULL,"
00136         " start_time TEXT,"
00137         " duration_sec REAL,"
00138         " distance_m REAL,"
00139         " avg_power REAL,"
00140         " avg_hr REAL"
00141         ") "
00142     },
00143     std::string_view{
00144         "CREATE TABLE IF NOT EXISTS tags ("
00145         " id INTEGER PRIMARY KEY AUTOINCREMENT,"
00146         " name TEXT NOT NULL UNIQUE"
00147         ") "
00148     },
00149     std::string_view{
00150         "CREATE TABLE IF NOT EXISTS activity_tags ("
00151         " activity_id INTEGER NOT NULL REFERENCES activities(id) ON DELETE CASCADE,"
00152         " tag_id INTEGER NOT NULL REFERENCES tags(id) ON DELETE CASCADE,"
00153         " PRIMARY KEY (activity_id, tag_id)"
00154         ") "
00155     },
00156     std::string_view{
00157         "CREATE TABLE IF NOT EXISTS intervals ("
00158         " id INTEGER PRIMARY KEY AUTOINCREMENT,"
00159         " activity_id INTEGER NOT NULL REFERENCES activities(id) ON DELETE CASCADE,"
00160         " start_sample INTEGER NOT NULL,"
00161         " end_sample INTEGER NOT NULL,"
00162         " name TEXT,"
00163         " type TEXT,"
00164         " avg_power REAL,"
00165         " np REAL,"
00166         " avg_hr REAL,"
00167         " avg_cadence REAL,"
00168         " notes TEXT,"
00169         " source TEXT NOT NULL DEFAULT 'auto',"
00170         " locked INTEGER NOT NULL DEFAULT 0,"
00171         " updated_at TEXT,"
00172         " algorithm_version INTEGER NOT NULL DEFAULT 1,"
00173         " deleted INTEGER NOT NULL DEFAULT 0,"
00174         " uuid TEXT"
00175         ") "
00176     },
00177     std::string_view{
00178         "CREATE TABLE IF NOT EXISTS climbs ("
00179         " id INTEGER PRIMARY KEY AUTOINCREMENT,"
00180         " activity_id INTEGER NOT NULL REFERENCES activities(id) ON DELETE CASCADE,"

```

```

00181         " start_seconds REAL NOT NULL,"
00182         " end_seconds REAL NOT NULL,"
00183         " name TEXT,"
00184         " elevation_gain_m REAL,"
00185         " length_km REAL,"
00186         " average_gradient REAL,"
00187         " source TEXT NOT NULL DEFAULT 'auto',"
00188         " locked INTEGER NOT NULL DEFAULT 0,"
00189         " created_at TEXT DEFAULT CURRENT_TIMESTAMP,"
00190         " updated_at TEXT DEFAULT CURRENT_TIMESTAMP,"
00191         " algorithm_version INTEGER NOT NULL DEFAULT 1,"
00192         " deleted INTEGER NOT NULL DEFAULT 0,"
00193         " uuid TEXT,"
00194         " original_start_seconds REAL,"
00195         " original_end_seconds REAL,"
00196         " avg_power REAL,"
00197         " np REAL,"
00198         " avg_hr REAL,"
00199         " avg_cadence REAL,"
00200         " vam REAL,"
00201         " notes TEXT,"
00202         " favorite INTEGER NOT NULL DEFAULT 0,"
00203         " rating INTEGER NOT NULL DEFAULT 0"
00204     )"
00205 },
00206 std::string_view{
00207     "CREATE INDEX IF NOT EXISTS idx_climbs_activity_id"
00208     " ON climbs(activity_id)"
00209 },
00210 std::string_view{
00211     "CREATE UNIQUE INDEX IF NOT EXISTS idx_activity_filehash"
00212     " ON activities(fit_hash) WHERE fit_hash IS NOT NULL"
00213 },
00214 std::string_view{
00215     "CREATE INDEX IF NOT EXISTS idx_activity_start_time"
00216     " ON activities(activity_start_time)"
00217 },
00218 std::string_view{
00219     "CREATE INDEX IF NOT EXISTS idx_activity_end_time"
00220     " ON activities(end_time)"
00221 },
00222 std::string_view{
00223     "CREATE INDEX IF NOT EXISTS idx_intervals_activity"
00224     " ON intervals(activity_id)"
00225 },
00226 std::string_view{
00227     "CREATE TABLE IF NOT EXISTS planned_workouts ("
00228     " id INTEGER PRIMARY KEY AUTOINCREMENT,"
00229     " athlete_id INTEGER NOT NULL REFERENCES athletes(id) ON DELETE CASCADE,"
00230     " workout_date TEXT NOT NULL,"
00231     " title TEXT NOT NULL,"
00232     " type TEXT,"
00233     " duration_sec REAL,"
00234     " target_tss REAL,"
00235     " notes TEXT"
00236     ")"
00237 },
00238 std::string_view{
00239     "CREATE INDEX IF NOT EXISTS idx_workouts_athlete_date"
00240     " ON planned_workouts(athlete_id, workout_date)"
00241 },
00242 std::string_view{
00243     "CREATE TABLE IF NOT EXISTS daily_metrics ("
00244     " id INTEGER PRIMARY KEY AUTOINCREMENT,"
00245     " athlete_id INTEGER NOT NULL REFERENCES athletes(id) ON DELETE CASCADE,"
00246     " metric_date TEXT NOT NULL,"
00247     " atl REAL NOT NULL DEFAULT 0,"
00248     " ctl REAL NOT NULL DEFAULT 0,"
00249     " tsb REAL NOT NULL DEFAULT 0"
00250     ")"
00251 },
00252 std::string_view{
00253     "CREATE UNIQUE INDEX IF NOT EXISTS idx_daily_metrics_athlete_date"
00254     " ON daily_metrics(athlete_id, metric_date)"
00255 },
00256 std::string_view{
00257     "CREATE TABLE IF NOT EXISTS equipment ("
00258     " id INTEGER PRIMARY KEY AUTOINCREMENT,"
00259     " athlete_id INTEGER NOT NULL REFERENCES athletes(id) ON DELETE CASCADE,"
00260     " name TEXT NOT NULL,"
00261     " category TEXT NOT NULL,"
00262     " service_interval_km REAL,"
00263     " service_interval_hours REAL,"
00264     " notes TEXT"
00265     ")"
00266 },
00267 std::string_view{

```



```

00268         "CREATE TABLE IF NOT EXISTS equipment_usage ("
00269         "     id             INTEGER PRIMARY KEY AUTOINCREMENT,"
00270         "     equipment_id  INTEGER NOT NULL REFERENCES equipment(id) ON DELETE CASCADE,"
00271         "     activity_id   INTEGER NOT NULL REFERENCES activities(id) ON DELETE CASCADE,"
00272         "     distance_km   REAL,"
00273         "     duration_sec  REAL"
00274         ") "
00275     },
00276     std::string_view{
00277         "CREATE TABLE IF NOT EXISTS imports ("
00278         "     id             INTEGER PRIMARY KEY AUTOINCREMENT,"
00279         "     file_name      TEXT     NOT NULL,"
00280         "     imported_at    TEXT     NOT NULL,"
00281         "     status         TEXT     NOT NULL,"
00282         "     activity_id    INTEGER,"
00283         "     error_message  TEXT"
00284         ") "
00285     },
00286     std::string_view{
00287         "CREATE INDEX IF NOT EXISTS idx_imports_date"
00288         "     ON imports(imported_at)"
00289     },
00290     std::string_view{
00291         "CREATE TABLE IF NOT EXISTS activity_analysis_settings ("
00292         "     activity_id    INTEGER PRIMARY KEY,"
00293         "     climb_min_gain  REAL,"
00294         "     climb_min_length REAL,"
00295         "     climb_min_gradient REAL,"
00296         "     interval_work_threshold REAL,"
00297         "     interval_rest_threshold REAL,"
00298         "     FOREIGN KEY(activity_id) REFERENCES activities(id) ON DELETE CASCADE"
00299         ") "
00300     },
00301     std::string_view{
00302         "CREATE TABLE IF NOT EXISTS activity_analysis_cache ("
00303         "     activity_id    INTEGER NOT NULL,"
00304         "     cache_key       TEXT     NOT NULL,"
00305         "     cache_value     BLOB,"
00306         "     PRIMARY KEY (activity_id, cache_key)"
00307         ") "
00308     },
00309     // Enable foreign-key enforcement (must be per-connection, set here via a dummy table)
00310     std::string_view{
00311         "PRAGMA foreign_keys = ON"
00312     },
00313 };
00314
00315 } // namespace DatabaseSchema

```

8.65 ImportRepository.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "ImportRepository.h"
00005
00006 #include <QDateTime>
00007 #include <QSqlDatabase>
00008 #include <QSqlQuery>
00009
00010 ImportRepository::ImportRepository(QSqlDatabase& db)
00011     : m_db(db)
00012 {}
00013
00014 int ImportRepository::logImport(const ImportRecord& r)
00015 {
00016     QSqlQuery q(m_db);
00017     q.prepare(
00018         "INSERT INTO imports(file_name, imported_at, status, activity_id, error_message)"
00019         " VALUES(:fn, :ts, :st, :aid, :err)");
00020     q.bindValue(":fn", r.fileName());
00021     q.bindValue(":ts", r.importedAt().isEmpty()
00022         ? QDateTime::currentDateTimeUtc().toString(Qt::ISODate)
00023         : r.importedAt());
00024     q.bindValue(":st", r.status());
00025     q.bindValue(":aid", r.activityId() > 0 ? QVariant(r.activityId()) : QVariant());
00026     q.bindValue(":err", r.errorMessage().isEmpty() ? QVariant() : QVariant(r.errorMessage()));
00027     if (!q.exec()) return -1;
00028     return q.lastInsertId().toInt();
00029 }
00030
00031 QList<ImportRecord> ImportRepository::getRecentImports(int limit)
00032 {
00033     QList<ImportRecord> result;

```

```

00034     QSqlQuery q(m_db);
00035     q.prepare(
00036         "SELECT id, file_name, imported_at, status, activity_id, error_message"
00037         " FROM imports ORDER BY imported_at DESC LIMIT :lim");
00038     q.bindValue(":lim", limit);
00039     q.exec();
00040     while (q.next())
00041     {
00042         ImportRecord r;
00043         r.id = q.value(0).toInt();
00044         r.fileName = q.value(1).toString();
00045         r.importedAt = q.value(2).toString();
00046         r.status = q.value(3).toString();
00047         r.activityId = q.value(4).toInt();
00048         r.errorMessage = q.value(5).toString();
00049         result.append(r);
00050     }
00051     return result;
00052 }

```

8.66 ImportRepository.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QList>
00007 #include <QString>
00008
00009 class QSqlDatabase;
00010
00014 struct ImportRecord
00015 {
00017     int id = -1;
00018
00020     QString fileName;
00021
00023     QString importedAt;
00024
00026     QString status;
00027
00029     int activityId = -1;
00030
00032     QString errorMessage;
00033 };
00034
00040 class ImportRepository
00041 {
00042 public:
00047     explicit ImportRepository(QSqlDatabase& db);
00048
00054     int logImport(const ImportRecord& r);
00055
00061     QList<ImportRecord> getRecentImports(int limit = 50);
00062
00063 private:
00064     QSqlDatabase& m_db;
00065 };

```

8.67 IntervalRepository.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "IntervalRepository.h"
00005 #include "SqlQueryHelper.h"
00006
00007 #include <QSqlDatabase>
00008 #include <QSqlQuery>
00009
00010 IntervalRepository::IntervalRepository(QSqlDatabase& db)
00011     : m_db(db)
00012 {}
00013
00014 int IntervalRepository::insertInterval(const IntervalRecord& interval)
00015 {
00016     QSqlQuery q(m_db);
00017     q.prepare(
00018         "INSERT INTO intervals("
00019         " activity_id, start_sample, end_sample, name, type,"

```

```

00020         " avg_power, np, avg_hr, avg_cadence, notes,"
00021         " source, locked, deleted, algorithm_version, uuid"
00022         ") VALUES ("
00023         " :activity_id, :start_sample, :end_sample, :name, :type,"
00024         " :avg_power, :np, :avg_hr, :avg_cadence, :notes,"
00025         " :source, :locked, :deleted, :algorithm_version, :uuid"
00026         ")");
00027
00028         q.bindValue(":activity_id",          interval.activityId);
00029         q.bindValue(":start_sample",         interval.startSample);
00030         q.bindValue(":end_sample",           interval.endSample);
00031         q.bindValue(":name",                 interval.name);
00032         q.bindValue(":type",                 interval.type);
00033         q.bindValue(":avg_power",             interval.avgPower);
00034         q.bindValue(":np",                   interval.np);
00035         q.bindValue(":avg_hr",                interval.avgHr);
00036         q.bindValue(":avg_cadence",           interval.avgCadence);
00037         q.bindValue(":notes",                 interval.notes);
00038         q.bindValue(":source",                interval.source);
00039         SqlQueryHelper::bindBoolValue(q, ":locked",          interval.locked);
00040         SqlQueryHelper::bindBoolValue(q, ":deleted",         interval.deleted);
00041         q.bindValue(":algorithm_version",      interval.algorithmVersion);
00042         q.bindValue(":uuid",                   interval.uuid.isEmpty() ? QVariant() : QVariant(interval.uuid));
00043
00044         if (!q.exec()) return -1;
00045         return q.lastInsertId().toInt();
00046     }
00047
00048     bool IntervalRepository::updateInterval(const IntervalRecord& interval)
00049     {
00050         QSqlQuery q(m_db);
00051         q.prepare(
00052             "UPDATE intervals SET"
00053             " start_sample = :start_sample,"
00054             " end_sample   = :end_sample,"
00055             " name         = :name,"
00056             " type         = :type,"
00057             " avg_power    = :avg_power,"
00058             " np           = :np,"
00059             " avg_hr       = :avg_hr,"
00060             " avg_cadence  = :avg_cadence,"
00061             " notes        = :notes,"
00062             " source       = :source,"
00063             " locked       = :locked,"
00064             " deleted      = :deleted,"
00065             " updated_at   = CURRENT_TIMESTAMP"
00066             " WHERE id = :id");
00067
00068         q.bindValue(":start_sample", interval.startSample);
00069         q.bindValue(":end_sample",   interval.endSample);
00070         q.bindValue(":name",         interval.name);
00071         q.bindValue(":type",         interval.type);
00072         q.bindValue(":avg_power",    interval.avgPower);
00073         q.bindValue(":np",           interval.np);
00074         q.bindValue(":avg_hr",       interval.avgHr);
00075         q.bindValue(":avg_cadence",  interval.avgCadence);
00076         q.bindValue(":notes",        interval.notes);
00077         q.bindValue(":source",       interval.source);
00078         SqlQueryHelper::bindBoolValue(q, ":locked",          interval.locked);
00079         SqlQueryHelper::bindBoolValue(q, ":deleted",         interval.deleted);
00080         q.bindValue(":id",           interval.id);
00081         return q.exec();
00082     }
00083
00084     bool IntervalRepository::softDeleteInterval(int intervalId)
00085     {
00086         QSqlQuery q(m_db);
00087         q.prepare("UPDATE intervals SET deleted=1, locked=1, updated_at=CURRENT_TIMESTAMP WHERE id=:id");
00088         q.bindValue(":id", intervalId);
00089         return q.exec();
00090     }
00091
00092     bool IntervalRepository::deleteInterval(int intervalId)
00093     {
00094         QSqlQuery q(m_db);
00095         q.prepare("DELETE FROM intervals WHERE id=:id");
00096         q.bindValue(":id", intervalId);
00097         return q.exec();
00098     }
00099     // deleteInterval remains a hard delete (used from UI when permanent removal is wanted)
00100
00101     bool IntervalRepository::deleteIntervalsForActivity(int activityId)
00102     {
00103         QSqlQuery q(m_db);
00104         q.prepare("DELETE FROM intervals WHERE activity_id=:id");
00105         q.bindValue(":id", activityId);
00106         return q.exec();

```

```

00107 }
00108
00109 QList<IntervalRecord> IntervalRepository::listIntervalsForActivity(int activityId)
00110 {
00111     QList<IntervalRecord> out;
00112     QSqlQuery q(m_db);
00113     q.prepare(
00114         "SELECT id, activity_id, start_sample, end_sample, name, type, "
00115         " avg_power, np, avg_hr, avg_cadence, notes, "
00116         " COALESCE(source,'auto'), COALESCE(locked,0), COALESCE(deleted,0), "
00117         " COALESCE(algorithm_version,1), COALESCE(uuid,")
00118         " FROM intervals WHERE activity_id=:id AND COALESCE(deleted,0)=0"
00119         " ORDER BY start_sample ASC");
00120     q.bindValue(":id", activityId);
00121     q.exec();
00122
00123     while (q.next())
00124     {
00125         IntervalRecord record;
00126         int col = 0;
00127         record.id = q.value(col++).toInt();
00128         record.activityId = q.value(col++).toInt();
00129         record.startSample = q.value(col++).toInt();
00130         record.endSample = q.value(col++).toInt();
00131         record.name = q.value(col++).toString();
00132         record.type = q.value(col++).toString();
00133         record.avgPower = q.value(col++).toDouble();
00134         record.np = q.value(col++).toDouble();
00135         record.avgHr = q.value(col++).toDouble();
00136         record.avgCadence = q.value(col++).toDouble();
00137         record.notes = q.value(col++).toString();
00138         record.source = q.value(col++).toString();
00139         record.locked = q.value(col++).toInt() != 0;
00140         record.deleted = q.value(col++).toInt() != 0;
00141         record.algorithmVersion = q.value(col++).toInt();
00142         record.uuid = q.value(col++).toString();
00143         out.push_back(record);
00144     }
00145     return out;
00146 }
00147
00148 bool IntervalRepository::hasLockedIntervals(int activityId)
00149 {
00150     QSqlQuery q(m_db);
00151     q.prepare(SqlQueryHelper::selectExistsLocked("intervals"));
00152     q.bindValue(":id", activityId);
00153     return q.exec() && q.next();
00154 }

```

8.68 IntervalRepository.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QList>
00007 #include <QString>
00008
00009 class QSqlDatabase;
00010
00014 struct IntervalRecord
00015 {
00017     int id = -1;
00018
00020     int activityId = -1;
00021
00023     int startSample = 0;
00024
00026     int endSample = 0;
00027
00029     QString name;
00030
00032     QString type;
00033
00035     double avgPower = 0.0;
00036
00038     double np = 0.0;
00039
00041     double avgHr = 0.0;
00042
00044     double avgCadence = 0.0;
00045
00047     QString notes;

```

```

00048
00050     QString source          = QStringLiteral("auto");
00051
00053     bool    locked         = false;
00054
00056     bool    deleted        = false;
00057
00059     int     algorithmVersion = 1;
00060
00062     QString uuid;
00063 };
00064
00070 class IntervalRepository
00071 {
00072 public:
00077     explicit IntervalRepository(QSqlDatabase& db);
00078
00084     int  insertInterval(const IntervalRecord& interval);
00085
00091     bool updateInterval(const IntervalRecord& interval);
00092
00094     bool softDeleteInterval(int intervalId);
00096     bool deleteInterval(int intervalId);
00097
00103     bool deleteIntervalsForActivity(int activityId);
00104
00106     QList<IntervalRecord> listIntervalsForActivity(int activityId);
00107
00113     bool hasLockedIntervals(int activityId);
00114
00115 private:
00116     QSqlDatabase& m_db;
00117 };

```

8.69 SqlQueryHelper.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <string_view>
00007
00008 #include <QSqlQuery>
00009 #include <QString>
00010
00012
00015 namespace SqlQueryHelper {
00016
00018 inline constexpr std::string_view kWhereNotDeleted = "COALESCE(deleted,0)=0";
00019
00021 inline constexpr std::string_view kWhereDeleted = "COALESCE(deleted,0)=1";
00022
00024 inline constexpr std::string_view kWhereLocked = "locked=1";
00025
00032 inline void bindBoolValue(QSqlQuery& query, const QString& placeholder, bool value)
00033 {
00034     query.bindValue(placeholder, value ? 1 : 0);
00035 }
00036
00042 inline QString selectExistsNonDeleted(const QString& tableName)
00043 {
00044     return QString("SELECT 1 FROM %1 WHERE activity_id=:id AND %2 LIMIT 1")
00045         .arg(tableName, QString::fromUtf8(kWhereNotDeleted.data(), kWhereNotDeleted.size()));
00046 }
00047
00053 inline QString selectExistsLocked(const QString& tableName)
00054 {
00055     return QString("SELECT 1 FROM %1 WHERE activity_id=:id AND %2 AND %3 LIMIT 1")
00056         .arg(tableName, QString::fromUtf8(kWhereLocked.data(), kWhereLocked.size()),
00057             QString::fromUtf8(kWhereNotDeleted.data(), kWhereNotDeleted.size()));
00058 }
00059
00065 inline QString selectExistsEver(const QString& tableName)
00066 {
00067     return QString("SELECT 1 FROM %1 WHERE activity_id=:id LIMIT 1").arg(tableName);
00068 }
00069
00070 } // namespace SqlQueryHelper

```

8.70 FitParser.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "FitParser.h"
00005 #include "FitRecordListener.h"
00006
00007 #include <QDateTime>
00008 #include <QTimeZone>
00009
00010 #include <fstream>
00011 #include <cmath>
00012 #include <numbers>
00013
00014 #include <fit_decode.hpp>
00015 #include <fit_file_id_mesg_listener.hpp>
00016 #include <fit_mesg_broadcaster.hpp>
00017 #include <fit_session_mesg_listener.hpp>
00018
00019 namespace
00020 {
00021 QString fitTimestampToIsoUtc(FIT_DATE_TIME timestamp)
00022 {
00023     if (timestamp == FIT_DATE_TIME_INVALID)
00024         return {};
00025
00026     // FIT epoch: 1989-12-31T00:00:00Z.
00027     static constexpr quint64 kFitEpochToUnixOffset = 631065600;
00028     const quint64 unixSeconds = kFitEpochToUnixOffset + static_cast<quint64>(timestamp);
00029     return QDateTime::fromSecsSinceEpoch(unixSeconds, QTimeZone::UTC).toString(Qt::ISODate);
00030 }
00031
00032 class FitSessionListener final : public fit::SessionMesgListener
00033 {
00034 public:
00035     void OnMesg(fit::SessionMesg& mesg) override
00036     {
00037         if (mesg.IsStartTimeValid())
00038             m_startTime = mesg.GetStartTime();
00039     }
00040
00041     FIT_DATE_TIME startTime() const { return m_startTime; }
00042
00043 private:
00044     FIT_DATE_TIME m_startTime = FIT_DATE_TIME_INVALID;
00045 };
00046
00047 class FitFileIdListener final : public fit::FileIdMesgListener
00048 {
00049 public:
00050     void OnMesg(fit::FileIdMesg& mesg) override
00051     {
00052         if (mesg.IsTimeCreatedValid())
00053             m_timeCreated = mesg.GetTimeCreated();
00054     }
00055
00056     FIT_DATE_TIME timeCreated() const { return m_timeCreated; }
00057
00058 private:
00059     FIT_DATE_TIME m_timeCreated = FIT_DATE_TIME_INVALID;
00060 };
00061 }
00062
00063 // Remove GPS points that imply speed > 150 km/h (equirectangular approximation)
00064 static void filterGpsOutliers(RideData& rideData)
00065 {
00066     constexpr double kMaxSpeedKmh = 150.0;
00067     constexpr double kMinSpeedKmh = 0.0;
00068
00069     bool prevValid = false;
00070     double prevLat = 0, prevLon = 0, prevSec = 0;
00071
00072     for (auto& r : rideData.records)
00073     {
00074         if (!r.hasGps)
00075         {
00076             prevValid = false;
00077             continue;
00078         }
00079
00080         // Additional sanity check on GPS coordinates
00081         if (r.latitude < -90.0 || r.latitude > 90.0 ||
00082             r.longitude < -180.0 || r.longitude > 180.0)
00083         {
00084             r.hasGps = false;

```

```

00085         prevValid = false;
00086         continue;
00087     }
00088     if (prevValid)
00089     {
00090         const double dt = r.elapsedSeconds - prevSec;
00091         if (dt > 0.0 && dt < 3600.0) // Sanity: max 1 hour between samples
00092         {
00093             const double dLat = (r.latitude - prevLat) * std::numbers::pi_v<double> / 180.0;
00094             const double dLon = (r.longitude - prevLon) * std::numbers::pi_v<double> / 180.0;
00095             const double mLat = (r.latitude + prevLat) / 2.0 * std::numbers::pi_v<double> /
180.0;
00097             const double distKm =
00098                 std::hypot(dLat, dLon * std::cos(mLat)) * 6371.0;
00099             const double speedKmh = distKm / (dt / 3600.0);
00100
00101             if (speedKmh > kMaxSpeedKmh || speedKmh < kMinSpeedKmh)
00102             {
00103                 r.hasGps = false;
00104                 prevValid = false;
00105                 continue;
00106             }
00107         }
00108         else if (dt <= 0.0 || dt >= 3600.0)
00109         {
00110             // Invalid time jump; don't trust this GPS point
00111             r.hasGps = false;
00112             prevValid = false;
00113             continue;
00114         }
00115     }
00116     prevValid = true;
00117     prevLat = r.latitude;
00118     prevLon = r.longitude;
00119     prevSec = r.elapsedSeconds;
00120 }
00121 }
00122 }
00123
00124 RideData FitParser::load(const QString& fileName)
00125 {
00126     RideData rideData;
00127
00128     std::ifstream fitFile(fileName.toStdString(), std::ios::binary);
00129     if (!fitFile.is_open())
00130         throw std::runtime_error("Cannot open FIT file: " +
00131             fileName.toStdString());
00132
00133     fit::Decode decode;
00134     fit::MsgBroadcaster broadcaster;
00135
00136     FitRecordListener listener(rideData);
00137     FitSessionListener sessionListener;
00138     FitFileIdListener fileIdListener;
00139     broadcaster.AddListener((fit::RecordMsgListener&)listener);
00140     broadcaster.AddListener((fit::SessionMsgListener&)sessionListener);
00141     broadcaster.AddListener((fit::FileIdMsgListener&)fileIdListener);
00142
00143     if (!decode.CheckIntegrity(fitFile))
00144         throw std::runtime_error(
00145             "FIT integrity check failed.\n"
00146             "The file may be corrupted or was not fully written.");
00147
00148     fitFile.clear();
00149     fitFile.seekg(0);
00150
00151     decode.Read(fitFile, broadcaster, broadcaster);
00152
00153     if (rideData.records.empty())
00154         throw std::runtime_error(
00155             "No activity records found in this FIT file.\n"
00156             "The file may be a device configuration or course file, not an activity.");
00157
00158     // Count records with power or heart rate data (key fields for cycling analysis)
00159     int recordsWithData = 0;
00160     for (const auto& r : rideData.records)
00161     {
00162         if (r.hasPower || r.hasHeartRate)
00163             ++recordsWithData;
00164     }
00165
00166     // If less than 10% of records have key data, file is likely invalid
00167     if (recordsWithData == 0 || recordsWithData < static_cast<int>(rideData.records.size() * 0.1))
00168         throw std::runtime_error(
00169             "Activity file contains insufficient data.\n"
00170             "Expected power or heart rate data in most samples.");

```

```

00171
00172     filterGpsOutliers(rideData);
00173
00174     const FIT_DATE_TIME sessionStart = sessionListener.startTime();
00175     const FIT_DATE_TIME firstRecordTime = listener.firstTimestamp();
00176     const FIT_DATE_TIME lastRecordTime = listener.lastTimestamp();
00177     const FIT_DATE_TIME fileTime = fileIdListener.timeCreated();
00178
00179     FIT_DATE_TIME chosenStart = sessionStart;
00180     if (chosenStart == FIT_DATE_TIME_INVALID)
00181         chosenStart = firstRecordTime;
00182     if (chosenStart == FIT_DATE_TIME_INVALID)
00183         chosenStart = fileTime;
00184
00185     rideData.activityStartTimeUtc = fitTimestampToIsoUtc(chosenStart);
00186     rideData.activityEndTimeUtc = fitTimestampToIsoUtc(lastRecordTime != FIT_DATE_TIME_INVALID ?
lastRecordTime : chosenStart);
00187     rideData.fileTimestampUtc = fitTimestampToIsoUtc(fileTime);
00188
00189     if (rideData.activityStartTimeUtc.isEmpty())
00190         rideData.activityStartTimeUtc = QDateTime::currentDateTimeUtc().toString(Qt::ISODate);
00191     if (rideData.activityEndTimeUtc.isEmpty())
00192         rideData.activityEndTimeUtc = rideData.activityStartTimeUtc;
00193
00194     return rideData;
00195 }

```

8.71 FitParser.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QString>
00007
00008 #include "RideData.h"
00009
00013 class FitParser
00014 {
00015 public:
00021     RideData load(const QString& fileName);
00022
00023 private:
00024     static double semicirclesToDegrees(long value);
00025 };

```

8.72 FitRecordListener.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "FitRecordListener.h"
00005
00006 #include <algorithm>
00007
00008 FitRecordListener::FitRecordListener(
00009     RideData& rideData)
00010     : m_rideData(rideData)
00011 {
00012 }
00013
00014 void FitRecordListener::OnMesg(
00015     fit::RecordMesg& mesg)
00016 {
00017     RideRecord record;
00018
00019     // GPS: validate coordinates are within valid range
00020     if (mesg.IsPositionLatValid() &&
00021         mesg.IsPositionLongValid())
00022     {
00023         const double lat = mesg.GetPositionLat() * (180.0 / 2147483648.0);
00024         const double lon = mesg.GetPositionLong() * (180.0 / 2147483648.0);
00025
00026         if (lat >= -90.0 && lat <= 90.0 && lon >= -180.0 && lon <= 180.0)
00027         {
00028             record.latitude = lat;
00029             record.longitude = lon;
00030             record.hasGps = true;
00031         }
00032     }

```



```

00033
00034 // Heart Rate: valid range 0-255 bpm
00035 if (mesg.IsHeartRateValid())
00036 {
00037     const auto hr = mesg.GetHeartRate();
00038     if (hr >= 0 && hr <= 255)
00039     {
00040         record.heartRate = hr;
00041         record.hasHeartRate = true;
00042     }
00043 }
00044
00045 // Cadence: valid range 0-255 rpm
00046 if (mesg.IsCadenceValid())
00047 {
00048     const auto cad = mesg.GetCadence();
00049     if (cad >= 0 && cad <= 255)
00050     {
00051         record.cadence = cad;
00052         record.hasCadence = true;
00053     }
00054 }
00055
00056 // Power: valid range 0-5000 watts
00057 if (mesg.IsPowerValid())
00058 {
00059     const auto pow = mesg.GetPower();
00060     if (pow >= 0 && pow <= 5000)
00061     {
00062         record.power = pow;
00063         record.hasPower = true;
00064     }
00065 }
00066
00067 // Altitude: valid range -500 to 10000 meters
00068 if (mesg.IsAltitudeValid())
00069 {
00070     const auto alt = mesg.GetAltitude();
00071     if (alt >= -500.0 && alt <= 10000.0)
00072     {
00073         record.altitude = alt;
00074         record.hasAltitude = true;
00075     }
00076 }
00077
00078 // Speed: valid range 0-150 km/h (converted from m/s)
00079 if (mesg.IsSpeedValid())
00080 {
00081     const auto spd = mesg.GetSpeed();
00082     const double spdKmh = spd * 3.6;
00083     if (spdKmh >= 0.0 && spdKmh <= 150.0)
00084     {
00085         record.speed = spdKmh;
00086         record.hasSpeed = true;
00087     }
00088 }
00089
00090 // Timestamp: validate monotonicity
00091 if (mesg.IsTimestampValid())
00092 {
00093     FIT_DATE_TIME ts = mesg.GetTimestamp();
00094
00095     if (m_firstTimestamp == FIT_DATE_TIME_INVALID)
00096         m_firstTimestamp = ts;
00097
00098     // Ensure monotonic timestamps; clamp to at least the last recorded elapsed time
00099     if (ts >= m_lastTimestamp)
00100     {
00101         m_lastTimestamp = ts;
00102         record.elapsedSeconds = static_cast<double>(ts - m_firstTimestamp);
00103     }
00104     else if (!m_rideData.records.empty())
00105     {
00106         // Backward timestamp: use previous elapsed + 1 second
00107         record.elapsedSeconds = m_rideData.records.back().elapsedSeconds + 1.0;
00108     }
00109     else
00110     {
00111         record.elapsedSeconds = 0.0;
00112     }
00113 }
00114 else if (!m_rideData.records.empty())
00115 {
00116     // No timestamp: continue sequence from last record
00117     record.elapsedSeconds =
00118         m_rideData.records.back().elapsedSeconds + 1.0;
00119 }

```

```

00120
00121 // Only add records that have at least one valid field
00122 if (record.hasGps || record.hasPower || record.hasHeartRate ||
00123     record.hasCadence || record.hasSpeed || record.hasAltitude)
00124 {
00125     m_rideData.records.push_back(record);
00126 }
00127 }

```

8.73 FitRecordListener.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include "RideData.h"
00007
00008 #include <fit_record_mesg_listener.hpp>
00009
00015 class FitRecordListener :
00016     public fit::RecordMesgListener
00017 {
00018 public:
00023     explicit FitRecordListener(
00024         RideData& rideData);
00025
00030     void OnMesg(
00031         fit::RecordMesg& mesg) override;
00032
00037     FIT_DATE_TIME firstTimestamp() const { return m_firstTimestamp; }
00038
00043     FIT_DATE_TIME lastTimestamp() const { return m_lastTimestamp; }
00044
00045 private:
00046     RideData& m_rideData;
00047
00048     FIT_DATE_TIME m_firstTimestamp = FIT_DATE_TIME_INVALID;
00049     FIT_DATE_TIME m_lastTimestamp = FIT_DATE_TIME_INVALID;
00050 };

```

8.74 RideData.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QString>
00007 #include <vector>
00008
00015 struct RideRecord
00016 {
00018     double elapsedSeconds = 0.0;
00019
00021     double latitude = 0.0;
00022
00024     double longitude = 0.0;
00025
00027     double power = 0.0;
00028
00030     double heartRate = 0.0;
00031
00033     double cadence = 0.0;
00034
00036     double speed = 0.0;
00037
00039     double altitude = 0.0;
00040
00042     bool hasGps = false;
00043
00045     bool hasPower = false;
00046
00048     bool hasHeartRate = false;
00049
00051     bool hasCadence = false;
00052
00054     bool hasSpeed = false;
00055
00057     bool hasAltitude = false;
00058 };

```

```

00059
00063 class RideData
00064 {
00065     public:
00067         std::vector<RideRecord> records;
00068
00070         QString activityStartTimeUtc;
00071
00073         QString activityEndTimeUtc;
00074
00076         QString fileTimestampUtc;
00077 };

```

8.75 fitlyzerC.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include <QApplication>
00005 #include <QIcon>
00006 #include <QPalette>
00007 #include <QStyleFactory>
00008
00009 #include "gui/MainWindow.h"
00010
00017 int main(int argc, char *argv[])
00018 {
00019     QApplication app(argc, argv);
00020     app.setOrganizationName("Fitlyzer");
00021     app.setApplicationName("FitlyzerC");
00022     app.setStyle(QStyleFactory::create("Fusion"));
00023
00024     QPalette lightPalette;
00025     lightPalette.setColor(QPalette::Window, QColor(245, 247, 250));
00026     lightPalette.setColor(QPalette::WindowText, QColor(31, 41, 55));
00027     lightPalette.setColor(QPalette::Base, QColor(255, 255, 255));
00028     lightPalette.setColor(QPalette::AlternateBase, QColor(241, 245, 249));
00029     lightPalette.setColor(QPalette::ToolTipBase, QColor(255, 255, 255));
00030     lightPalette.setColor(QPalette::ToolTipText, QColor(31, 41, 55));
00031     lightPalette.setColor(QPalette::Text, QColor(31, 41, 55));
00032     lightPalette.setColor(QPalette::Button, QColor(239, 243, 248));
00033     lightPalette.setColor(QPalette::ButtonText, QColor(31, 41, 55));
00034     lightPalette.setColor(QPalette::BrightText, Qt::white);
00035     lightPalette.setColor(QPalette::Highlight, QColor(37, 99, 235));
00036     lightPalette.setColor(QPalette::HighlightedText, Qt::white);
00037     app.setPalette(lightPalette);
00038
00039     app.setWindowIcon(QIcon(":/resources/icons/fitlyzer_logo.png"));
00040
00041     MainWindow window;
00042     window.show();
00043
00044     return app.exec();
00045 }

```

8.76 AboutDialog.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "AboutDialog.h"
00005
00006 #include <QDesktopServices>
00007 #include <QDialogButtonBox>
00008 #include <QLabel>
00009 #include <QPushButton>
00010 #include <QUrl>
00011 #include <QVBoxLayout>
00012
00013 AboutDialog::AboutDialog(QWidget* parent)
00014     : QDialog(parent)
00015 {
00016     setWindowTitle("About FitlyzerC");
00017     setMinimumWidth(450);
00018
00019     auto* layout = new QVBoxLayout(this);
00020
00021     // Title
00022     auto* titleLabel = new QLabel("FitlyzerC", this);
00023     titleLabel->setStyleSheet("font-size: 18px; font-weight: bold; margin-bottom: 8px;");
00024     layout->addWidget(titleLabel);

```

```

00025
00026 // Description
00027 auto* descLabel = new QLabel(
00028     "FitlyzerC is a desktop application for analyzing cycling workouts and fitness data from "
00029     "FIT format files. It provides comprehensive training analytics, power curve analysis, "
00030     "and performance tracking for cyclists.",
00031     this);
00032 descLabel->setWordWrap(true);
00033 layout->addWidget(descLabel);
00034 layout->addSpacing(12);
00035
00036 // Contact & Links
00037 auto* contactLabel = new QLabel("Contact & Links:", this);
00038 contactLabel->setStyleSheet("font-weight: bold; margin-top: 8px;");
00039 layout->addWidget(contactLabel);
00040
00041 auto* emailLabel = new QLabel(
00042     "Email: <a href=\"mailto:email@larseberhart.com\">email@larseberhart.com</a>",
00043     this);
00044 emailLabel->setOpenExternalLinks(true);
00045 layout->addWidget(emailLabel);
00046
00047 auto* gitHubLabel = new QLabel(
00048     "GitHub: <a
href=\"https://github.com/larseberhart/fitlyzerC\">https://github.com/larseberhart/fitlyzerC</a>",
00049     this);
00050 gitHubLabel->setOpenExternalLinks(true);
00051 layout->addWidget(gitHubLabel);
00052
00053 layout->addSpacing(8);
00054 layout->addStretch();
00055
00056 // Close button
00057 auto* btnBox = new QDialogButtonBox(QDialogButtonBox::Close, this);
00058 connect(btnBox, &QDialogButtonBox::rejected, this, &QDialog::accept);
00059 layout->addWidget(btnBox);
00060 }

```

8.77 AboutDialog.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QDialog>
00007
00011 class AboutDialog : public QDialog
00012 {
00013     Q_OBJECT
00014
00015 public:
00020     explicit AboutDialog(QWidget* parent = nullptr);
00021 };

```

8.78 ActivityBrowser.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "ActivityBrowser.h"
00005 #include "EditNotesDialog.h"
00006 #include "core/settings/DateFormatter.h"
00007
00008 #include <QHeaderView>
00009 #include <QHBoxLayout>
00010 #include <QComboBox>
00011 #include <QLabel>
00012 #include <QLineEdit>
00013 #include <QMimeData>
00014 #include <QMenu>
00015 #include <QMessageBox>
00016 #include <QTableWidget>
00017 #include <QTableWidgetItem>
00018 #include <QUrl>
00019 #include <QVBoxLayout>
00020 #include <QFileInfo>
00021 #include <QDate>
00022
00023 namespace
00024 {

```

```

00025 constexpr int kNotesRole = Qt::UserRole + 1;
00026 constexpr int kWeatherRole = Qt::UserRole + 2;
00027 constexpr int kIsoDateRole = Qt::UserRole + 3;
00028
00029 class ActivityDateTableItem : public QTableWidgetItem
00030 {
00031 public:
00032     using QTableWidgetItem::QTableWidgetItem;
00033
00034     bool operator<(const QTableWidgetItem& other) const override
00035     {
00036         const QString lhsIso = data(kIsoDateRole).toString();
00037         const QString rhsIso = other.data(kIsoDateRole).toString();
00038
00039         const QDate lhsDate = QDate::fromString(lhsIso, Qt::ISODate);
00040         const QDate rhsDate = QDate::fromString(rhsIso, Qt::ISODate);
00041         if (lhsDate.isValid() && rhsDate.isValid())
00042             return lhsDate < rhsDate;
00043
00044         // Fall back to ISO lexical compare for partial values and then display text.
00045         const int isoCmp = QString::compare(lhsIso, rhsIso, Qt::CaseInsensitive);
00046         if (isoCmp != 0)
00047             return isoCmp < 0;
00048
00049         return QTableWidgetItem::operator<(other);
00050     }
00051 };
00052
00053 static QString fmtDurAB(double seconds)
00054 {
00055     const int total = static_cast<int>(seconds);
00056     const int h = total / 3600;
00057     const int m = (total % 3600) / 60;
00058     const int s = total % 60;
00059     return h > 0
00060         ? QString("%1:%2:%3").arg(h).arg(m, 2, 10, QChar('0')).arg(s, 2, 10, QChar('0'))
00061         : QString("%1:%2").arg(m).arg(s, 2, 10, QChar('0'));
00062 }
00063
00064 ActivityBrowser::ActivityBrowser(DatabaseManager* dbManager, QWidget* parent)
00065 : QWidget(parent)
00066 , m_dbManager(dbManager)
00067 {
00068     setAcceptDrops(true);
00069
00070     auto* layout = new QVBoxLayout(this);
00071     layout->setContentsMargins(0, 0, 0, 0);
00072
00073     auto* filters = new QHBoxLayout;
00074     m_searchEdit = new QLineEdit(this);
00075     m_searchEdit->setPlaceholderText("Search activities, notes, or dates");
00076     m_searchEdit->setClearButtonEnabled(true);
00077     m_rangeFilter = new QComboBox(this);
00078     m_rangeFilter->addItem("All");
00079     m_rangeFilter->addItem("Last 7 Days");
00080     m_rangeFilter->addItem("Last 30 Days");
00081     m_rangeFilter->addItem("This Year");
00082     filters->addWidget(m_searchEdit, 1);
00083     filters->addWidget(m_rangeFilter);
00084     layout->addLayout(filters);
00085
00086     m_emptyStateLabel = new QLabel(this);
00087     m_emptyStateLabel->setWordWrap(true);
00088     m_emptyStateLabel->setAlignment(Qt::AlignCenter);
00089     m_emptyStateLabel->setText(
00090         "No activities yet.\n\nDrop .fit files here or use Import Activities to add rides.");
00091     m_emptyStateLabel->setStyleSheet(
00092         "QLabel { background: #f8f8fc; color: #475569; border: 1px dashed #cbd5e1;"
00093         " padding: 20px; border-radius: 8px; font-size: 14px; }");
00094     layout->addWidget(m_emptyStateLabel);
00095
00096     m_table = new QTableWidgetItem(0, 8);
00097     m_table->setHorizontalHeaderLabels({
00098         "Date", "File", "Duration", "Distance (km)",
00099         "Avg W", "NP", "Avg HR", "Elevation (m)"
00100     });
00101     m_table->horizontalHeader()->setStretchLastSection(true);
00102     m_table->setSelectionBehavior(QAbstractItemView::SelectRows);
00103     m_table->setSelectionMode(QAbstractItemView::SingleSelection);
00104     m_table->setEditTriggers(QAbstractItemView::NoEditTriggers);
00105     m_table->setAlternatingRowColors(true);
00106     m_table->setSortingEnabled(true);
00107     m_table->setContextMenuPolicy(Qt::CustomContextMenu);
00108
00109     layout->addWidget(m_table);
00110
00111     connect(m_searchEdit, &QLineEdit::textChanged,
00112         this, [this] { applyFilters(); });
00113     connect(m_rangeFilter, qOverload<int>(&QComboBox::currentIndexChanged),

```

```

00112         this, [this](int) { applyFilters(); });
00113
00114     connect(m_table, &QTableWidget::cellDoubleClicked,
00115            this, [this](int row, int)
00116            {
00117                auto* item = m_table->item(row, 0);
00118                if (!item) return;
00119                const int actId = item->data(Qt::UserRole).toInt();
00120                if (actId > 0)
00121                    emit activitySelected(actId);
00122            });
00123
00124     connect(m_table, &QWidget::customContextMenuRequested,
00125            this, &ActivityBrowser::showContextMenu);
00126
00127     applyFilters();
00128 }
00129
00130 void ActivityBrowser::focusSearch()
00131 {
00132     if (!m_searchEdit)
00133         return;
00134
00135     m_searchEdit->setFocus();
00136     m_searchEdit->selectAll();
00137 }
00138
00139 void ActivityBrowser::dragEnterEvent(QDragEnterEvent* event)
00140 {
00141     if (!event->mimeTypeData()->hasUrls())
00142         return;
00143
00144     for (const QUrl& url : event->mimeTypeData()->urls())
00145     {
00146         if (!url.isLocalFile())
00147             continue;
00148         if (url.toLocalFile().toLower().endsWith(".fit"))
00149         {
00150             event->acceptProposedAction();
00151             return;
00152         }
00153     }
00154 }
00155
00156 void ActivityBrowser::dropEvent(QDropEvent* event)
00157 {
00158     if (!event->mimeTypeData()->hasUrls())
00159         return;
00160
00161     QStringList fitFiles;
00162     for (const QUrl& url : event->mimeTypeData()->urls())
00163     {
00164         if (!url.isLocalFile())
00165             continue;
00166         const QString path = url.toLocalFile();
00167         if (QFileInfo(path).suffix().compare("fit", Qt::CaseInsensitive) == 0)
00168             fitFiles << path;
00169     }
00170
00171     fitFiles.removeDuplicates();
00172     if (fitFiles.isEmpty())
00173         return;
00174
00175     emit fitFilesDropped(fitFiles);
00176     event->acceptProposedAction();
00177 }
00178
00179 void ActivityBrowser::refresh(int athleteId)
00180 {
00181     m_athleteId = athleteId;
00182     m_table->setSortingEnabled(false);
00183     m_table->setRowCount(0);
00184
00185     if (!m_dbManager || !m_dbManager->isOpen())
00186     {
00187         applyFilters();
00188         return;
00189     }
00190
00191     auto db = m_dbManager->database();
00192     ActivityRepository repo(db);
00193
00194     for (const Activity& a : repo.listActivities(athleteId))
00195     {
00196         const int row = m_table->rowCount();
00197         m_table->insertRow(row);
00198     }

```

```

00199         const QString isoDate = !a.startTime.isEmpty() ? a.startTime.left(10) : a.importedAt.left(10);
00200         const QDate parsedDate = QDate::fromString(isoDate, Qt::ISODate);
00201         const QString displayDate = parsedDate.isValid() ? DateFormatter::formatDate(parsedDate) :
isoDate;
00202
00203         auto* dateItem = new ActivityDateTableItem(displayDate);
00204         dateItem->setData(Qt::UserRole, a.id);
00205         dateItem->setData(kNotesRole, a.notes);
00206         dateItem->setData(kWeatherRole, a.weatherNotes);
00207         dateItem->setData(kIsoDateRole, isoDate);
00208         m_table->setItem(row, 0, dateItem);
00209
00210         m_table->setItem(row, 1, new QTableWidgetItem(a.fileName));
00211         m_table->setItem(row, 2, new QTableWidgetItem(fmtDurAB(a.durationSec)));
00212         m_table->setItem(row, 3, new QTableWidgetItem(
00213             a.distanceM > 0 ? QString::number(a.distanceM / 1000.0, 'f', 1) : ""));
00214         m_table->setItem(row, 4, new QTableWidgetItem(
00215             a.avgPower > 0 ? QString::number(a.avgPower, 'f', 0) : ""));
00216         m_table->setItem(row, 5, new QTableWidgetItem(
00217             a.normalizedPower > 0 ? QString::number(a.normalizedPower, 'f', 0) : ""));
00218         m_table->setItem(row, 6, new QTableWidgetItem(
00219             a.avgHr > 0 ? QString::number(a.avgHr, 'f', 0) : ""));
00220         m_table->setItem(row, 7, new QTableWidgetItem(
00221             a.elevationGain > 0 ? QString::number(a.elevationGain, 'f', 0) : ""));
00222     }
00223
00224     m_table->setSortingEnabled(true);
00225     m_table->resizeColumnsToContents();
00226     applyFilters();
00227 }
00228
00229 void ActivityBrowser::applyFilters()
00230 {
00231     if (!m_table)
00232         return;
00233
00234     const QString term = m_searchEdit ? m_searchEdit->text().trimmed().toLower() : QString();
00235     const QString range = m_rangeFilter ? m_rangeFilter->currentText() : QString("All");
00236     const QDate today = QDate::currentDate();
00237
00238     int visibleRows = 0;
00239     for (int row = 0; row < m_table->rowCount(); ++row)
00240     {
00241         bool matchesText = term.isEmpty();
00242         if (!matchesText)
00243         {
00244             QString rowText;
00245             for (int col = 0; col < m_table->columnCount(); ++col)
00246             {
00247                 auto* item = m_table->item(row, col);
00248                 if (item)
00249                     rowText += item->text().toLower() + " ";
00250             }
00251             auto* dateItem = m_table->item(row, 0);
00252             if (dateItem)
00253                 rowText += dateItem->data(kNotesRole).toString().toLower() + " ";
00254             matchesText = rowText.contains(term);
00255         }
00256
00257         bool matchesDate = true;
00258         auto* dateItem = m_table->item(row, 0);
00259         const QDate rowDate = dateItem
00260             ? QDate::fromString(dateItem->data(kIsoDateRole).toString(), Qt::ISODate)
00261             : QDate();
00262
00263         if (rowDate.isValid() && range != "All")
00264         {
00265             if (range == "Last 7 Days")
00266                 matchesDate = rowDate >= today.addDays(-7);
00267             else if (range == "Last 30 Days")
00268                 matchesDate = rowDate >= today.addDays(-30);
00269             else if (range == "This Year")
00270                 matchesDate = rowDate.year() == today.year();
00271         }
00272
00273         const bool visible = matchesText && matchesDate;
00274         m_table->setRowHidden(row, !visible);
00275         if (visible)
00276             ++visibleRows;
00277     }
00278
00279     if (m_emptyStateLabel)
00280     {
00281         const bool hasDatabase = m_dbManager && m_dbManager->isOpen();
00282         if (!hasDatabase)
00283         {
00284             m_emptyStateLabel->setText("Open or create a database to browse activities.");

```

```

00285     }
00286     else if (m_table->rowCount() == 0)
00287     {
00288         m_emptyStateLabel->setText(
00289             "No activities yet.\n\nDrop .fit files here or use Import Activities to add rides.");
00290     }
00291     else if (visibleRows == 0)
00292     {
00293         m_emptyStateLabel->setText("No activities match the current search or date filter.");
00294     }
00295
00296     m_emptyStateLabel->setVisible(!hasDatabase || m_table->rowCount() == 0 || visibleRows == 0);
00297 }
00298
00299 m_table->setVisible(m_table->rowCount() > 0);
00300 }
00301
00302 void ActivityBrowser::showContextMenu(const QPoint& pos)
00303 {
00304     const int row = m_table->rowAt(pos.y());
00305     if (row < 0) return;
00306     auto* item = m_table->item(row, 0);
00307     if (!item) return;
00308     const int actId = item->data(Qt::UserRole).toInt();
00309     if (actId <= 0) return;
00310
00311     QMenu menu(this);
00312     auto* loadAct = menu.addAction("Load Activity");
00313     auto* notesAct = menu.addAction("Edit Notes...");
00314     menu.addSeparator();
00315     auto* deleteAct = menu.addAction("Delete Activity");
00316
00317     const QAction* chosen = menu.exec(m_table->viewport()->mapToGlobal(pos));
00318     if (chosen == loadAct)
00319         emit activitySelected(actId);
00320     else if (chosen == notesAct)
00321         editNotes(actId);
00322     else if (chosen == deleteAct)
00323         deleteActivity(actId);
00324 }
00325
00326 void ActivityBrowser::editNotes(int activityId)
00327 {
00328     if (!m_dbManager || !m_dbManager->isOpen()) return;
00329
00330     // Retrieve current notes from the row's stored data
00331     QString notes, weather;
00332     for (int row = 0; row < m_table->rowCount(); ++row)
00333     {
00334         auto* item = m_table->item(row, 0);
00335         if (item && item->data(Qt::UserRole).toInt() == activityId)
00336         {
00337             notes = item->data(kNotesRole).toString();
00338             weather = item->data(kWeatherRole).toString();
00339             break;
00340         }
00341     }
00342
00343     EditNotesDialog dlg(notes, weather, this);
00344     if (dlg.exec() != QDialog::Accepted) return;
00345
00346     auto db = m_dbManager->database();
00347     ActivityRepository repo(db);
00348     if (repo.updateNotes(activityId, dlg.notes(), dlg.weatherNotes()))
00349         refresh(m_athleteId); // reload to show updated data
00350     else
00351         QMessageBox::critical(this, "Error", "Failed to save notes.");
00352 }
00353
00354 void ActivityBrowser::deleteActivity(int activityId)
00355 {
00356     const auto ans = QMessageBox::question(
00357         this, "Delete Activity",
00358         "Delete this activity and all its samples?\nThis cannot be undone.",
00359         QMessageBox::Yes | QMessageBox::No);
00360     if (ans != QMessageBox::Yes) return;
00361
00362     if (!m_dbManager || !m_dbManager->isOpen()) return;
00363
00364     auto db = m_dbManager->database();
00365     ActivityRepository repo(db);
00366     if (repo.deleteActivity(activityId))
00367     {
00368         refresh(m_athleteId);
00369         emit activityDeleted(activityId);
00370     }
00371     else

```



```

00372         QMessageBox::critical(this, "Error", "Failed to delete activity.");
00373     }

```

8.79 ActivityBrowser.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QDragEnterEvent>
00007 #include <QDropEvent>
00008 #include <QWidget>
00009
00010 #include "database/DatabaseManager.h"
00011 #include "database/ActivityRepository.h"
00012
00013 class QTableWidgetItem;
00014 class QComboBox;
00015 class QLineEdit;
00016 class QLabel;
00017
00026 class ActivityBrowser : public QWidget
00027 {
00028     Q_OBJECT
00029 public:
00030     explicit ActivityBrowser(DatabaseManager* dbManager, QWidget* parent = nullptr);
00031
00032     void refresh(int athleteId = -1);
00033
00034     void focusSearch();
00035
00036 signals:
00037     // Emitted when an activity is selected in the table.
00038     void activitySelected(int activityId);
00039
00040     // Emitted when an activity is deleted.
00041     void activityDeleted(int activityId);
00042
00043     // Emitted when one or more local .fit files are dropped onto the widget.
00044     void fitFilesDropped(const QStringList& filePaths);
00045
00046 protected:
00047     void dragEnterEvent(QDragEnterEvent* event) override;
00048
00049     void dropEvent(QDropEvent* event) override;
00050
00051 private:
00052     void applyFilters();
00053     void showContextMenu(const QPoint& pos);
00054     void editNotes(int activityId);
00055     void deleteActivity(int activityId);
00056
00057     DatabaseManager* m_dbManager = nullptr;
00058     QLabel* m_emptyStateLabel = nullptr;
00059     QLineEdit* m_searchEdit = nullptr;
00060     QComboBox* m_rangeFilter = nullptr;
00061     QTableWidgetItem* m_table = nullptr;
00062     int m_athleteId = -1;
00063 };

```

8.80 AthleteDialog.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "AthleteDialog.h"
00005 #include "core/settings/DateFormatter.h"
00006
00007 #include <QDate>
00008 #include <QDialogButtonBox>
00009 #include <QDoubleSpinBox>
00010 #include <QFormLayout>
00011 #include <QGroupBox>
00012 #include <QHBoxLayout>
00013 #include <QHeaderView>
00014 #include <QLabel>
00015 #include <QLineEdit>
00016 #include <QMessageBox>
00017 #include <QPushButton>
00018 #include <QSpinBox>

```

```

00019 #include <QTableWidget>
00020 #include <QTableWidgetItem>
00021 #include <QVBoxLayout>
00022
00023 namespace
00024 {
00025 constexpr int kEntryIdRole = Qt::UserRole;
00026
00027 QString displayDateOrRaw(const QString& raw)
00028 {
00029     const QDate parsed = QDate::fromString(raw, Qt::ISODate);
00030     return parsed.isValid() ? DateFormatter::formatDate(parsed) : raw;
00031 }
00032
00033 QString toIsoOrRaw(const QString& text)
00034 {
00035     const QDate parsed = DateFormatter::parseDate(text);
00036     return parsed.isValid() ? DateFormatter::toIsoDate(parsed) : text.trimmed();
00037 }
00038 }
00039
00040 AthleteDialog::AthleteDialog(AthleteRepository& repo, int athleteId, QWidget* parent)
00041     : QDialog(parent)
00042     , m_repo(repo)
00043     , m_athleteId(athleteId)
00044 {
00045     setWindowTitle(athleteId < 0 ? "New Athlete" : "Edit Athlete");
00046     setMinimumWidth(520);
00047
00048     auto* mainLayout = new QVBoxLayout(this);
00049
00050     // Basic info
00051     auto* form = new QFormLayout;
00052     m_firstName = new QLineEdit;
00053     m_lastName = new QLineEdit;
00054     m_dob = new QLineEdit;
00055     m_dob->setPlaceholderText(QString("Date format: %1").arg(DateFormatter::qtDatePattern()));
00056     m_email = new QLineEdit;
00057     m_height = new QDoubleSpinBox;
00058     m_ftp = new QSpinBox;
00059     m_height->setRange(0.0, 250.0);
00060     m_height->setSuffix(" cm");
00061     m_height->setDecimals(1);
00062     m_height->setSpecialValueText("(not set)");
00063     m_ftp->setRange(100, 600);
00064     m_ftp->setValue(250);
00065     m_ftp->setSuffix(" W");
00066
00067     form->addRow("First name:", m_firstName);
00068     form->addRow("Last name:", m_lastName);
00069     form->addRow("Date of birth:", m_dob);
00070     form->addRow("Email:", m_email);
00071     form->addRow("Height:", m_height);
00072     form->addRow("FTP:", m_ftp);
00073     mainLayout->addLayout(form);
00074
00075     // FTP history
00076     auto* ftpGroup = new QGroupBox("FTP History");
00077     auto* ftpLayout = new QVBoxLayout(ftpGroup);
00078     m_ftpTable = new QTableWidget(0, 3);
00079     m_ftpTable->setHorizontalHeaderLabels({"Date", "FTP (W)", "Notes"});
00080     m_ftpTable->horizontalHeader()->setStretchLastSection(true);
00081     m_ftpTable->setSelectionBehavior(QAbstractItemView::SelectRows);
00082     m_ftpTable->setMinimumHeight(120);
00083
00084     auto* ftpBtnBar = new QHBoxLayout;
00085     auto* addFtp = new QPushButton("Add");
00086     auto* delFtp = new QPushButton("Delete");
00087     ftpBtnBar->addWidget(addFtp);
00088     ftpBtnBar->addWidget(delFtp);
00089     ftpBtnBar->addStretch();
00090
00091     ftpLayout->addWidget(m_ftpTable);
00092     ftpLayout->addLayout(ftpBtnBar);
00093     mainLayout->addWidget(ftpGroup);
00094
00095     connect(addFtp, &QPushButton::clicked, this, &AthleteDialog::addFtpRow);
00096     connect(delFtp, &QPushButton::clicked, this, &AthleteDialog::deleteFtpRow);
00097
00098     // Weight history
00099     auto* wGroup = new QGroupBox("Weight History");
00100     auto* wLayout = new QVBoxLayout(wGroup);
00101     m_weightTable = new QTableWidget(0, 2);
00102     m_weightTable->setHorizontalHeaderLabels({"Date", "Weight (kg)"});
00103     m_weightTable->horizontalHeader()->setStretchLastSection(true);
00104     m_weightTable->setSelectionBehavior(QAbstractItemView::SelectRows);
00105     m_weightTable->setMinimumHeight(100);

```

```

00106
00107     auto* wBtnBar = new QHBoxLayout;
00108     auto* addW    = new QPushButton("Add");
00109     auto* delW    = new QPushButton("Delete");
00110     wBtnBar->addWidget(addW);
00111     wBtnBar->addWidget(delW);
00112     wBtnBar->addStretch();
00113
00114     wLayout->addWidget(m_weightTable);
00115     wLayout->addLayout(wBtnBar);
00116     mainLayout->addWidget(wGroup);
00117
00118     connect(addW, &QPushButton::clicked, this, &AthleteDialog::addWeightRow);
00119     connect(delW, &QPushButton::clicked, this, &AthleteDialog::deleteWeightRow);
00120
00121     // Buttons
00122     auto* bbox = new QDialogButtonBox(QDialogButtonBox::Ok | QDialogButtonBox::Cancel);
00123     mainLayout->addWidget(bbox);
00124
00125     connect(bbox, &QDialogButtonBox::accepted, this, &AthleteDialog::onAccepted);
00126     connect(bbox, &QDialogButtonBox::rejected, this, &QDialog::reject);
00127
00128     if (m_athleteId > 0)
00129         loadAthlete();
00130 }
00131
00132 void AthleteDialog::loadAthlete()
00133 {
00134     Athlete a = m_repo.getAthlete(m_athleteId);
00135     m_firstName->setText(a.firstName);
00136     m_lastName->setText(a.lastName);
00137     m_dob->setText(displayDateOrRaw(a.dateOfBirth));
00138     m_email->setText(a.email);
00139     m_height->setValue(a.heightCm);
00140     m_ftp->setValue(a.ftpWatts > 0 ? a.ftpWatts : 250);
00141
00142     populateFtpTable(m_repo.getFtpHistory(m_athleteId));
00143     populateWeightTable(m_repo.getWeightHistory(m_athleteId));
00144 }
00145
00146 void AthleteDialog::populateFtpTable(const QList<FtpEntry>& entries)
00147 {
00148     m_ftpTable->setRowCount(0);
00149     for (const auto& e : entries)
00150     {
00151         const int row = m_ftpTable->rowCount();
00152         m_ftpTable->insertRow(row);
00153         m_ftpTable->setItem(row, 0, new QTableWidgetItem(displayDateOrRaw(e.effectiveFrom)));
00154         m_ftpTable->setItem(row, 1, new QTableWidgetItem(QString::number(e.ftpWatts)));
00155         m_ftpTable->setItem(row, 2, new QTableWidgetItem(e.notes));
00156         // Store the entry id in the row's first item
00157         m_ftpTable->item(row, 0)->setData(kEntryIdRole, e.id);
00158     }
00159 }
00160
00161 void AthleteDialog::populateWeightTable(const QList<WeightEntry>& entries)
00162 {
00163     m_weightTable->setRowCount(0);
00164     for (const auto& e : entries)
00165     {
00166         const int row = m_weightTable->rowCount();
00167         m_weightTable->insertRow(row);
00168         m_weightTable->setItem(row, 0, new QTableWidgetItem(displayDateOrRaw(e.effectiveFrom)));
00169         m_weightTable->setItem(row, 1, new QTableWidgetItem(
00170             QString::number(e.weightKg, 'f', 1)));
00171         m_weightTable->item(row, 0)->setData(kEntryIdRole, e.id);
00172     }
00173 }
00174
00175 void AthleteDialog::addFtpRow()
00176 {
00177     const int row = m_ftpTable->rowCount();
00178     m_ftpTable->insertRow(row);
00179     m_ftpTable->setItem(row, 0, new
00180     QTableWidgetItem(DateFormatter::formatDate(QDate::currentDate())));
00181     m_ftpTable->setItem(row, 1, new QTableWidgetItem("250"));
00182     m_ftpTable->setItem(row, 2, new QTableWidgetItem(""));
00183     m_ftpTable->item(row, 0)->setData(kEntryIdRole, -1); // new, no db id yet
00184     m_ftpTable->editItem(m_ftpTable->item(row, 0));
00185 }
00186 void AthleteDialog::deleteFtpRow()
00187 {
00188     const int row = m_ftpTable->currentRow();
00189     if (row < 0) return;
00190     const int entryId = m_ftpTable->item(row, 0)->data(kEntryIdRole).toInt();
00191     if (entryId > 0 && m_athleteId > 0)

```

```

00192         m_repo.deleteFtpEntry(entryId);
00193         m_ftpTable->removeRow(row);
00194     }
00195
00196 void AthleteDialog::addWeightRow()
00197 {
00198     const int row = m_weightTable->rowCount();
00199     m_weightTable->insertRow(row);
00200     m_weightTable->setItem(row, 0, new
    QTableWidgetItem(DateFormatter::formatDate(QDate::currentDate())));
00201     m_weightTable->setItem(row, 1, new QTableWidgetItem("70.0"));
00202     m_weightTable->item(row, 0)->setData(kEntryIdRole, -1);
00203     m_weightTable->editItem(m_weightTable->item(row, 0));
00204 }
00205
00206 void AthleteDialog::deleteWeightRow()
00207 {
00208     const int row = m_weightTable->currentRow();
00209     if (row < 0) return;
00210     const int entryId = m_weightTable->item(row, 0)->data(kEntryIdRole).toInt();
00211     if (entryId > 0 && m_athleteId > 0)
00212         m_repo.deleteWeightEntry(entryId);
00213     m_weightTable->removeRow(row);
00214 }
00215
00216 void AthleteDialog::onAccepted()
00217 {
00218     if (m_firstName->text().trimmed().isEmpty() ||
00219         m_lastName->text().trimmed().isEmpty())
00220     {
00221         QMessageBox::warning(this, "Missing data",
00222             "First name and last name are required.");
00223         return;
00224     }
00225
00226     Athlete a;
00227     a.id = m_athleteId;
00228     a.firstName = m_firstName->text().trimmed();
00229     a.lastName = m_lastName->text().trimmed();
00230     a.dateOfBirth = toIsoOrRaw(m_dob->text());
00231     a.email = m_email->text().trimmed();
00232     a.heightCm = m_height->value();
00233     a.ftpWatts = m_ftp->value();
00234
00235     if (m_athleteId < 0)
00236         m_savedId = m_repo.insertAthlete(a);
00237     else
00238     {
00239         m_repo.updateAthlete(a);
00240         m_savedId = m_athleteId;
00241     }
00242
00243     if (m_savedId < 0)
00244     {
00245         QMessageBox::critical(this, "Error", "Failed to save athlete.");
00246         return;
00247     }
00248
00249     // Save new FTP entries (those with id == -1)
00250     for (int row = 0; row < m_ftpTable->rowCount(); ++row)
00251     {
00252         const int entryId = m_ftpTable->item(row, 0)->data(kEntryIdRole).toInt();
00253         if (entryId < 0)
00254         {
00255             FtpEntry e;
00256             e.athleteId = m_savedId;
00257             e.effectiveFrom = toIsoOrRaw(m_ftpTable->item(row, 0)->text());
00258             e.ftpWatts = m_ftpTable->item(row, 1)->text().toInt();
00259             e.notes = m_ftpTable->item(row, 2)->text();
00260             m_repo.addFtpEntry(e);
00261         }
00262     }
00263
00264     // Save new weight entries
00265     for (int row = 0; row < m_weightTable->rowCount(); ++row)
00266     {
00267         const int entryId = m_weightTable->item(row, 0)->data(kEntryIdRole).toInt();
00268         if (entryId < 0)
00269         {
00270             WeightEntry e;
00271             e.athleteId = m_savedId;
00272             e.effectiveFrom = toIsoOrRaw(m_weightTable->item(row, 0)->text());
00273             e.weightKg = m_weightTable->item(row, 1)->text().toDouble();
00274             m_repo.addWeightEntry(e);
00275         }
00276     }
00277

```

```
00278     accept();
00279 }
```

8.81 AthleteDialog.h

```
00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QDialog>
00007 #include <QList>
00008
00009 #include "database/AthleteRepository.h"
00010
00011 class QLineEdit;
00012 class QDoubleSpinBox;
00013 class QSpinBox;
00014 class QTableWidget;
00015
00021 class AthleteDialog : public QDialog
00022 {
00023     Q_OBJECT
00024 public:
00031     explicit AthleteDialog(AthleteRepository& repo,
00032                             int athleteId = -1,
00033                             QWidget* parent = nullptr);
00034
00039     int savedAthleteId() const { return m_savedId; }
00040
00041 private slots:
00042     void onAccepted();
00043     void addFtpRow();
00044     void deleteFtpRow();
00045     void addWeightRow();
00046     void deleteWeightRow();
00047
00048 private:
00049     void loadAthlete();
00050     void populateFtpTable(const QList<FtpEntry>& entries);
00051     void populateWeightTable(const QList<WeightEntry>& entries);
00052
00053     AthleteRepository& m_repo;
00054     int m_athleteId;
00055     int m_savedId = -1;
00056
00057     QLineEdit* m_firstName = nullptr;
00058     QLineEdit* m_lastName = nullptr;
00059     QLineEdit* m_dob = nullptr;
00060     QLineEdit* m_email = nullptr;
00061     QDoubleSpinBox* m_height = nullptr;
00062     QSpinBox* m ftp = nullptr;
00063     QTableWidget* m ftpTable = nullptr;
00064     QTableWidget* m_weightTable = nullptr;
00065 };
```

8.82 AthleteHeaderWidget.cpp

```
00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "AthleteHeaderWidget.h"
00005
00006 #include <QHBoxLayout>
00007 #include <QLabel>
00008
00009 AthleteHeaderWidget::AthleteHeaderWidget(QWidget* parent)
00010     : QWidget(parent)
00011 {
00012     setFixedHeight(40);
00013     setStyleSheet(
00014         "AthleteHeaderWidget {"
00015         "    background: #1e3a5f;"
00016         "    border-bottom: 1px solid #14274a;"
00017         "});");
00018
00019     auto* row = new QHBoxLayout(this);
00020     row->setContentsMargins(12, 0, 12, 0);
00021     row->setSpacing(0);
00022
00023     // Left: athlete name
```

```

00024     m_nameLabel = new QLabel("No athlete selected", this);
00025     m_nameLabel->setStyleSheet(
00026         "font-size: 14px; font-weight: 700; color: #0c4a6e;"
00027         " min-width: 160px;");
00028
00029     // Middle: FTP / activities / last ride
00030     m_statsLabel = new QLabel("FTP 0 W   Activities 0   Last -", this);
00031     m_statsLabel->setAlignment(Qt::AlignCenter);
00032     m_statsLabel->setStyleSheet(
00033         "font-size: 12px; color: #374151;");
00034
00035     // Right: per-ride metrics
00036     m_metricsLabel = new QLabel("", this);
00037     m_metricsLabel->setAlignment(Qt::AlignRight | Qt::AlignVCenter);
00038     m_metricsLabel->setStyleSheet(
00039         "font-size: 12px; font-weight: 600; color: #1f2937;"
00040         " min-width: 160px;");
00041
00042     row->addWidget(m_nameLabel, 0);
00043     row->addWidget(m_statsLabel, 1);
00044     row->addWidget(m_metricsLabel, 0);
00045 }
00046
00047 void AthleteHeaderWidget::setSummary(const QString& athleteName,
00048                                     int ftpWatts,
00049                                     int activityCount,
00050                                     const QString& lastActivityDate)
00051 {
00052     m_nameLabel->setText(athleteName.isEmpty() ? QString("No athlete selected") : athleteName);
00053     m_statsLabel->setText(
00054         QString("FTP %1 W   \u00b7 %2 activities   \u00b7 Last %3")
00055             .arg(ftpWatts)
00056             .arg(activityCount)
00057             .arg(lastActivityDate));
00058 }
00059
00060 void AthleteHeaderWidget::setRideMetrics(const QString& metricsText)
00061 {
00062     m_metricsLabel->setText(metricsText);
00063 }

```

8.83 AthleteHeaderWidget.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QWidget>
00007
00008 class QLabel;
00009
00015 class AthleteHeaderWidget : public QWidget
00016 {
00017     Q_OBJECT
00018
00019 public:
00020     explicit AthleteHeaderWidget(QWidget* parent = nullptr);
00021
00022     void setSummary(const QString& athleteName,
00023                   int ftpWatts,
00024                   int activityCount,
00025                   const QString& lastActivityDate);
00026
00027     void setRideMetrics(const QString& metricsText);
00028
00029 private:
00030     QLabel* m_nameLabel      = nullptr;    // left: athlete name
00031     QLabel* m_statsLabel     = nullptr;    // middle: FTP / activities / last ride
00032     QLabel* m_metricsLabel   = nullptr;    // right: NP / IF / TSS ...
00033 };

```

8.84 AthleteListDialog.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "AthleteListDialog.h"
00005 #include "AthleteDialog.h"
00006 #include "core/settings/DateFormatter.h"
00007

```

```

00008 #include <QDate>
00009
00010 #include <QDialogButtonBox>
00011 #include <QHBoxLayout>
00012 #include <QHeaderView>
00013 #include <QMessageBox>
00014 #include <QPushButton>
00015 #include <QTableWidget>
00016 #include <QTableWidgetItem>
00017 #include <QVBoxLayout>
00018
00019 AthleteListDialog::AthleteListDialog(AthleteRepository& repo, QWidget* parent)
00020     : QDialog(parent)
00021     , m_repo(repo)
00022 {
00023     setWindowTitle("Athletes");
00024     setMinimumSize(460, 300);
00025
00026     auto* mainLayout = new QVBoxLayout(this);
00027
00028     m_table = new QTableWidget(0, 3);
00029     m_table->setHorizontalHeaderLabels({"Name", "Date of Birth", "Email"});
00030     m_table->horizontalHeader()->setStretchLastSection(true);
00031     m_table->setSelectionBehavior(QAbstractItemView::SelectRows);
00032     m_table->setSelectionMode(QAbstractItemView::SingleSelection);
00033     m_table->setEditTriggers(QAbstractItemView::NoEditTriggers);
00034     m_table->setAlternatingRowColors(true);
00035     mainLayout->addWidget(m_table);
00036
00037     // Buttons
00038     auto* btnBar = new QHBoxLayout;
00039     auto* newBtn = new QPushButton("New");
00040     m_editBtn = new QPushButton("Edit");
00041     m_deleteBtn = new QPushButton("Delete");
00042     m_selectBtn = new QPushButton("Select");
00043     m_selectBtn->setDefault(true);
00044
00045     btnBar->addWidget(newBtn);
00046     btnBar->addWidget(m_editBtn);
00047     btnBar->addWidget(m_deleteBtn);
00048     btnBar->addStretch();
00049     btnBar->addWidget(m_selectBtn);
00050     mainLayout->addLayout(btnBar);
00051
00052     auto* closeBtn = new QDialogButtonBox(QDialogButtonBox::Close);
00053     mainLayout->addWidget(closeBtn);
00054
00055     connect(newBtn, &QPushButton::clicked, this, &AthleteListDialog::newAthlete);
00056     connect(m_editBtn, &QPushButton::clicked, this, &AthleteListDialog::editAthlete);
00057     connect(m_deleteBtn, &QPushButton::clicked, this, &AthleteListDialog::deleteAthlete);
00058     connect(m_selectBtn, &QPushButton::clicked, this, &AthleteListDialog::selectAthlete);
00059     connect(closeBtn, &QDialogButtonBox::rejected, this, &QDialog::reject);
00060
00061     connect(m_table, &QTableWidget::cellDoubleClicked,
00062             this, [this](int, int){ editAthlete(); });
00063     connect(m_table, &QTableWidget::itemSelectionChanged, this, [this]{
00064         const bool hasSel = m_table->currentRow() >= 0;
00065         m_editBtn->setEnabled(hasSel);
00066         m_deleteBtn->setEnabled(hasSel);
00067         m_selectBtn->setEnabled(hasSel);
00068     });
00069
00070     m_editBtn->setEnabled(false);
00071     m_deleteBtn->setEnabled(false);
00072     m_selectBtn->setEnabled(false);
00073
00074     refresh();
00075 }
00076
00077 void AthleteListDialog::refresh()
00078 {
00079     m_table->setRowCount(0);
00080     for (const Athlete& a : m_repo.listAthletes())
00081     {
00082         const int row = m_table->rowCount();
00083         m_table->insertRow(row);
00084         auto* nameItem = new QTableWidgetItem(a.fullName());
00085         nameItem->setData(Qt::UserRole, a.id);
00086         m_table->setItem(row, 0, nameItem);
00087         const QDate dob = QDate::fromString(a.dateOfBirth, Qt::ISODate);
00088         const QString dobText = dob.isValid() ? DateFormatter::formatDate(dob) : a.dateOfBirth;
00089         m_table->setItem(row, 1, new QTableWidgetItem(dobText));
00090         m_table->setItem(row, 2, new QTableWidgetItem(a.email));
00091     }
00092 }
00093
00094 void AthleteListDialog::newAthlete()

```

```

00095 {
00096     AthleteDialog dlg(m_repo, -1, this);
00097     if (dlg.exec() == QDialog::Accepted)
00098         refresh();
00099 }
00100
00101 void AthleteListDialog::editAthlete()
00102 {
00103     const int row = m_table->currentRow();
00104     if (row < 0) return;
00105     const int athleteId = m_table->item(row, 0)->data(Qt::UserRole).toInt();
00106     AthleteDialog dlg(m_repo, athleteId, this);
00107     if (dlg.exec() == QDialog::Accepted)
00108         refresh();
00109 }
00110
00111 void AthleteListDialog::deleteAthlete()
00112 {
00113     const int row = m_table->currentRow();
00114     if (row < 0) return;
00115     const int athleteId = m_table->item(row, 0)->data(Qt::UserRole).toInt();
00116     const QString name = m_table->item(row, 0)->text();
00117
00118     const auto ans = QMessageBox::question(
00119         this, "Delete Athlete",
00120         QString("Delete \"%1\" and all their activities?").arg(name),
00121         QMessageBox::Yes | QMessageBox::No);
00122     if (ans != QMessageBox::Yes) return;
00123
00124     m_repo.deleteAthlete(athleteId);
00125     refresh();
00126 }
00127
00128 void AthleteListDialog::selectAthlete()
00129 {
00130     const int row = m_table->currentRow();
00131     if (row < 0) return;
00132     m_selectedId = m_table->item(row, 0)->data(Qt::UserRole).toInt();
00133     accept();
00134 }

```

8.85 AthleteListDialog.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QDialog>
00007
00008 #include "database/AthleteRepository.h"
00009
00010 class QTableWidget;
00011 class QPushButton;
00012
00019 class AthleteListDialog : public QDialog
00020 {
00021     Q_OBJECT
00022 public:
00023     explicit AthleteListDialog(AthleteRepository& repo,
00024                               QWidget* parent = nullptr);
00025
00026     int selectedAthleteId() const { return m_selectedId; }
00027
00028 private slots:
00029     void refresh();
00030     void newAthlete();
00031     void editAthlete();
00032     void deleteAthlete();
00033     void selectAthlete();
00034
00035 private:
00036     AthleteRepository& m_repo;
00037     int m_selectedId = -1;
00038
00039     QTableWidget* m_table = nullptr;
00040     QPushButton* m_editBtn = nullptr;
00041     QPushButton* m_deleteBtn = nullptr;
00042     QPushButton* m_selectBtn = nullptr;
00043 };

```


8.86 CalendarWidget.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "CalendarWidget.h"
00005
00006 #include "PlannedWorkoutDialog.h"
00007 #include "core/settings/DateFormatter.h"
00008 #include "database/DatabaseManager.h"
00009
00010 #include <QCalendarWidget>
00011 #include <QComboBox>
00012 #include <QDateEdit>
00013 #include <QHBoxLayout>
00014 #include <QLabel>
00015 #include <QListWidget>
00016 #include <QMessageBox>
00017 #include <QPushButton>
00018 #include <QSqlQuery>
00019 #include <QTextCharFormat>
00020 #include <QVBoxLayout>
00021
00022 CalendarWidget::CalendarWidget(QWidget* parent)
00023     : QWidget(parent)
00024 {
00025     auto* layout = new QVBoxLayout(this);
00026
00027     auto* top = new QHBoxLayout;
00028     top->addWidget(new QLabel("Color by:", this));
00029
00030     m_colorByCombo = new QComboBox(this);
00031     m_colorByCombo->addItem("TSS");
00032     m_colorByCombo->addItem("Duration");
00033     m_colorByCombo->addItem("Workout Type");
00034     top->addWidget(m_colorByCombo);
00035
00036     m_addButton = new QPushButton("Add", this);
00037     m_editButton = new QPushButton("Edit", this);
00038     m_deleteButton = new QPushButton("Delete", this);
00039     top->addWidget(m_addButton);
00040     top->addWidget(m_editButton);
00041     top->addWidget(m_deleteButton);
00042     top->addStretch(1);
00043
00044     m_calendar = new QCalendarWidget(this);
00045     m_calendar->setGridVisible(true);
00046
00047     m_dayList = new QListWidget(this);
00048     m_dayList->addItem("Planned workouts and completed activities will appear here.");
00049
00050     layout->addLayout(top);
00051     layout->addWidget(m_calendar);
00052     layout->addWidget(m_dayList);
00053
00054     connect(m_calendar, &QCalendarWidget::selectionChanged,
00055             this, &CalendarWidget::onDateChanged);
00056     connect(m_calendar, &QCalendarWidget::currentPageChanged,
00057             this, [this](int, int) { updateMonthFormatting(); });
00058     connect(m_colorByCombo, qOverload<int>(&QComboBox::currentIndexChanged),
00059             this, &CalendarWidget::recolorMonth);
00060     connect(m_addButton, &QPushButton::clicked,
00061             this, &CalendarWidget::addPlannedWorkout);
00062     connect(m_editButton, &QPushButton::clicked,
00063             this, &CalendarWidget::editSelectedWorkout);
00064     connect(m_deleteButton, &QPushButton::clicked,
00065             this, &CalendarWidget::deleteSelectedWorkout);
00066 }
00067
00068 void CalendarWidget::setDatabaseManager(DatabaseManager* dbManager)
00069 {
00070     m_dbManager = dbManager;
00071     refresh();
00072 }
00073
00074 void CalendarWidget::setAthleteId(int athleteId)
00075 {
00076     m_athleteId = athleteId;
00077     refresh();
00078 }
00079
00080 void CalendarWidget::refresh()
00081 {
00082     if (!m_calendar)
00083         return;
00084     reloadDayList(m_calendar->selectedDate());

```

```

00085     updateMonthFormatting();
00086 }
00087
00088 void CalendarWidget::onDateChanged()
00089 {
00090     if (!m_calendar)
00091         return;
00092     reloadDayList(m_calendar->selectedDate());
00093 }
00094
00095 void CalendarWidget::addPlannedWorkout()
00096 {
00097     if (!m_dbManager || !m_dbManager->isOpen() || m_athleteId <= 0)
00098         return;
00099
00100     PlannedWorkoutDraft draft;
00101     draft.date = m_calendar ? m_calendar->selectedDate() : QDate::currentDate();
00102     PlannedWorkoutDialog dlg(draft, this);
00103     if (dlg.exec() != QDialog::Accepted)
00104         return;
00105
00106     const PlannedWorkoutDraft saved = dlg.draft();
00107     if (!saved.date.isValid() || saved.title.isEmpty())
00108         return;
00109
00110     QSqlQuery q(m_dbManager->database());
00111     q.prepare(
00112         "INSERT INTO planned_workouts(athlete_id, workout_date, title, type, duration_sec, target_tss,
notes) "
00113         "VALUES(:athlete_id, :workout_date, :title, :type, :duration_sec, :target_tss, :notes)");
00114     q.bindValue(":athlete_id", m_athleteId);
00115     q.bindValue(":workout_date", DateFormatter::toIsoDate(saved.date));
00116     q.bindValue(":title", saved.title);
00117     q.bindValue(":type", saved.type);
00118     q.bindValue(":duration_sec", saved.durationSeconds);
00119     q.bindValue(":target_tss", saved.targetTss);
00120     q.bindValue(":notes", saved.notes);
00121     if (!q.exec())
00122     {
00123         QMessageBox::critical(this, "Calendar", "Failed to add planned workout.");
00124         return;
00125     }
00126
00127     refresh();
00128 }
00129
00130 void CalendarWidget::editSelectedWorkout()
00131 {
00132     if (!m_dbManager || !m_dbManager->isOpen() || !m_dayList)
00133         return;
00134
00135     QListWidgetItem* item = m_dayList->currentItem();
00136     if (!item)
00137         return;
00138
00139     const int id = item->data(Qt::UserRole).toInt();
00140     if (id <= 0)
00141         return;
00142
00143     QSqlQuery q(m_dbManager->database());
00144     q.prepare(
00145         "SELECT workout_date, title, type, duration_sec, target_tss, notes "
00146         "FROM planned_workouts WHERE id=:id");
00147     q.bindValue(":id", id);
00148     if (!q.exec() || !q.next())
00149         return;
00150
00151     PlannedWorkoutDraft draft;
00152     draft.id = id;
00153     draft.date = QDate::fromString(q.value(0).toString(), Qt::ISODate);
00154     draft.title = q.value(1).toString();
00155     draft.type = q.value(2).toString();
00156     draft.durationSeconds = q.value(3).toDouble();
00157     draft.targetTss = q.value(4).toDouble();
00158     draft.notes = q.value(5).toString();
00159
00160     PlannedWorkoutDialog dlg(draft, this);
00161     if (dlg.exec() != QDialog::Accepted)
00162         return;
00163
00164     const PlannedWorkoutDraft saved = dlg.draft();
00165     QSqlQuery update(m_dbManager->database());
00166     update.prepare(
00167         "UPDATE planned_workouts SET workout_date=:workout_date, title=:title, type=:type, "
00168         "duration_sec=:duration_sec, target_tss=:target_tss, notes=:notes WHERE id=:id");
00169     update.bindValue(":workout_date", DateFormatter::toIsoDate(saved.date));
00170     update.bindValue(":title", saved.title);

```

```

00171     update.bindValue(":type", saved.type);
00172     update.bindValue(":duration_sec", saved.durationSeconds);
00173     update.bindValue(":target_tss", saved.targetTss);
00174     update.bindValue(":notes", saved.notes);
00175     update.bindValue(":id", id);
00176     if (!update.exec())
00177     {
00178         QMessageBox::critical(this, "Calendar", "Failed to update planned workout.");
00179         return;
00180     }
00181     refresh();
00182 void CalendarWidget::deleteSelectedWorkout()
00183 {
00184     if (!m_dbManager || !m_dbManager->isOpen() || !m_dayList)
00185         return;
00186     QListWidgetItem* item = m_dayList->currentItem();
00187     if (!item)
00188         return;
00189     const int id = item->data(Qt::UserRole).toInt();
00190     if (id <= 0)
00191         return;
00192     QSqlQuery q(m_dbManager->database());
00193     q.prepare("DELETE FROM planned_workouts WHERE id=:id");
00194     q.bindValue(":id", id);
00195     if (!q.exec())
00196     {
00197         QMessageBox::critical(this, "Calendar", "Failed to delete planned workout.");
00198         return;
00199     }
00200     refresh();
00201 void CalendarWidget::recolorMonth()
00202 {
00203     updateMonthFormatting();
00204 void CalendarWidget::reloadDayList(const QDate& date)
00205 {
00206     if (!m_dayList)
00207         return;
00208     m_dayList->clear();
00209     if (!m_dbManager || !m_dbManager->isOpen() || m_athleteId <= 0)
00210     {
00211         m_dayList->addItem("Open a database and select an athlete.");
00212         return;
00213     }
00214     QSqlQuery q(m_dbManager->database());
00215     q.prepare(
00216         "SELECT id, title, type, duration_sec, target_tss FROM planned_workouts "
00217         "WHERE athlete_id=:athlete_id AND workout_date=:workout_date ORDER BY id ASC");
00218     q.bindValue(":athlete_id", m_athleteId);
00219     q.bindValue(":workout_date", DateFormatter::toIsoDate(date));
00220     q.exec();
00221     bool hasRows = false;
00222     while (q.next())
00223     {
00224         hasRows = true;
00225         const int id = q.value(0).toInt();
00226         const QString title = q.value(1).toString();
00227         const QString type = q.value(2).toString();
00228         const double durationSec = q.value(3).toDouble();
00229         const double tss = q.value(4).toDouble();
00230         const QString text = QString("%1 (%2, %3 h, TSS %4)")
00231             .arg(title)
00232             .arg(type.isEmpty() ? QStringLiteral("Planned") : type)
00233             .arg(durationSec / 3600.0, 0, 'f', 2)
00234             .arg(tss, 0, 'f', 0);
00235         auto* item = new QListWidgetItem(text, m_dayList);
00236         item->setData(Qt::UserRole, id);
00237     }
00238     if (!hasRows)
00239         m_dayList->addItem("No planned workouts for this day.");
00240 void CalendarWidget::updateMonthFormatting()

```

```

00258 {
00259     if (!m_calendar)
00260         return;
00261
00262     const QTextCharFormat defaultFormat;
00263     const QDate shown = m_calendar->selectedDate().isValid() ? m_calendar->selectedDate() :
QDate::currentDate();
00264     const QDate monthStart(shown.year(), shown.month(), 1);
00265     const QDate monthEnd = monthStart.addMonths(1).addDays(-1);
00266
00267     for (QDate d = monthStart; d <= monthEnd; d = d.addDays(1))
00268         m_calendar->setDateTextFormat(d, defaultFormat);
00269
00270     if (!m_dbManager || !m_dbManager->isOpen() || m_athleteId <= 0)
00271         return;
00272
00273     QSqlQuery q(m_dbManager->database());
00274     q.prepare(
00275         "SELECT workout_date, SUM(target_tss), SUM(duration_sec), COUNT(*), MAX(type) "
00276         "FROM planned_workouts "
00277         "WHERE athlete_id=:athlete_id AND workout_date>=:start_date AND workout_date<=:end_date "
00278         "GROUP BY workout_date");
00279     q.bindValue(":athlete_id", m_athleteId);
00280     q.bindValue(":start_date", DateFormatter::toIsoDate(monthStart));
00281     q.bindValue(":end_date", DateFormatter::toIsoDate(monthEnd));
00282     q.exec();
00283
00284     while (q.next())
00285     {
00286         const QDate date = QDate::fromString(q.value(0).toString(), Qt::ISODate);
00287         const double totalTss = q.value(1).toDouble();
00288         const double totalSec = q.value(2).toDouble();
00289         const int count = q.value(3).toInt();
00290         const QString type = q.value(4).toString();
00291
00292         QColor bg("#dbeafe");
00293         const QString colorBy = m_colorByCombo ? m_colorByCombo->currentText() :
QStringLiteral("TSS");
00294         if (colorBy == "TSS")
00295         {
00296             if (totalTss >= 200.0) bg = QColor("#991b1b");
00297             else if (totalTss >= 120.0) bg = QColor("#dc2626");
00298             else if (totalTss >= 70.0) bg = QColor("#f97316");
00299             else if (totalTss >= 30.0) bg = QColor("#facc15");
00300             else bg = QColor("#86efac");
00301         }
00302         else if (colorBy == "Duration")
00303         {
00304             const double hours = totalSec / 3600.0;
00305             if (hours >= 4.0) bg = QColor("#1d4ed8");
00306             else if (hours >= 2.0) bg = QColor("#2563eb");
00307             else if (hours >= 1.0) bg = QColor("#38bdf8");
00308             else bg = QColor("#bae6fd");
00309         }
00310         else
00311         {
00312             if (type.contains("V02", Qt::CaseInsensitive)) bg = QColor("#ef4444");
00313             else if (type.contains("Threshold", Qt::CaseInsensitive)) bg = QColor("#f59e0b");
00314             else if (type.contains("Endurance", Qt::CaseInsensitive)) bg = QColor("#22c55e");
00315             else bg = QColor("#a3a3a3");
00316         }
00317
00318         QTextCharFormat fmt;
00319         fmt.setBackground(bg);
00320         fmt.setForeground((bg.lightness() < 128) ? QColor("#f8fafc") : QColor("#111827"));
00321         fmt.setToolTip(QString("%1 planned workouts").arg(count));
00322         m_calendar->setDateTextFormat(date, fmt);
00323     }
00324 }

```

8.87 CalendarWidget.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QDate>
00007 #include <QString>
00008 #include <QWidget>
00009
00010 class QCalendarWidget;
00011 class QComboBox;
00012 class QListWidget;

```

```

00013 class QPushButton;
00014 class QListWidgetItem;
00015 class DatabaseManager;
00016
00022 class CalendarWidget : public QWidget
00023 {
00024     Q_OBJECT
00025
00026 public:
00027     explicit CalendarWidget(QWidget* parent = nullptr);
00028
00029     void setDatabaseManager(DatabaseManager* dbManager);
00030
00031     void setAthleteId(int athleteId);
00032
00033     void refresh();
00034
00035 private slots:
00036     void onDateChanged();
00037     void addPlannedWorkout();
00038     void editSelectedWorkout();
00039     void deleteSelectedWorkout();
00040     void recolorMonth();
00041
00042 private:
00043     void reloadDayList(const QDate& date);
00044     void updateMonthFormatting();
00045
00046 private:
00047     QCalendarWidget* m_calendar = nullptr;
00048     QComboBox* m_colorByCombo = nullptr;
00049     QListWidget* m_dayList = nullptr;
00050     QPushButton* m_addButton = nullptr;
00051     QPushButton* m_editButton = nullptr;
00052     QPushButton* m_deleteButton = nullptr;
00053
00054     DatabaseManager* m_dbManager = nullptr;
00055     int m_athleteId = -1;
00056 };

```

8.88 CreateDatabaseDialog.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "CreateDatabaseDialog.h"
00005
00006 #include <QCheckBox>
00007 #include <QDialogButtonBox>
00008 #include <QFileDialog>
00009 #include <QFileInfo>
00010 #include <QFormLayout>
00011 #include <QHBoxLayout>
00012 #include <QLabel>
00013 #include <QLineEdit>
00014 #include <QMessageBox>
00015 #include <QPushButton>
00016 #include <QVBoxLayout>
00017 #include <QDir>
00018
00019 CreateDatabaseDialog::CreateDatabaseDialog(const QString& defaultDirectory, QWidget* parent)
00020     : QDialog(parent)
00021 {
00022     setWindowTitle("Create Database");
00023     setMinimumWidth(520);
00024
00025     auto* mainLayout = new QVBoxLayout(this);
00026
00027     auto* intro = new QLabel(
00028         "Create a new training database and optionally add a default athlete.", this);
00029     intro->setWordWrap(true);
00030     mainLayout->addWidget(intro);
00031
00032     auto* form = new QFormLayout;
00033     m_nameEdit = new QLineEdit("fitlyzer", this);
00034     m_locationEdit = new QLineEdit(defaultDirectory, this);
00035     m_browseButton = new QPushButton("Browse...", this);
00036
00037     auto* locationRow = new QHBoxLayout;
00038     locationRow->addWidget(m_locationEdit, 1);
00039     locationRow->addWidget(m_browseButton);
00040
00041     form->addRow("Database name:", m_nameEdit);
00042     form->addRow("Location:", locationRow);

```

```

00043     mainLayout->addLayout(form);
00044
00045     m_createAthleteCheck = new QCheckBox("Create default athlete", this);
00046     m_createAthleteCheck->setChecked(true);
00047     mainLayout->addWidget(m_createAthleteCheck);
00048
00049     auto* athleteForm = new QFormLayout;
00050     m_firstNameEdit = new QLineEdit(this);
00051     m_lastNameEdit = new QLineEdit(this);
00052     m_firstNameEdit->setPlaceholderText("First name");
00053     m_lastNameEdit->setPlaceholderText("Last name");
00054     athleteForm->addRow("First name:", m_firstNameEdit);
00055     athleteForm->addRow("Last name:", m_lastNameEdit);
00056     mainLayout->addLayout(athleteForm);
00057
00058     m_buttonBox = new QDialogButtonBox(QDialogButtonBox::Ok | QDialogButtonBox::Cancel, this);
00059     mainLayout->addWidget(m_buttonBox);
00060
00061     connect(m_browseButton, &QPushButton::clicked, this, &CreateDatabaseDialog::browseLocation);
00062     connect(m_createAthleteCheck, &QCheckBox::toggled, this,
&CreateDatabaseDialog::updateAthleteFields);
00063     connect(m_buttonBox, &QDialogButtonBox::accepted, this, &CreateDatabaseDialog::accept);
00064     connect(m_buttonBox, &QDialogButtonBox::rejected, this, &QDialog::reject);
00065
00066     updateAthleteFields();
00067 }
00068
00069 QString CreateDatabaseDialog::databasePath() const
00070 {
00071     const QString dirPath = m_locationEdit ? m_locationEdit->text().trimmed() : QString();
00072     if (dirPath.isEmpty())
00073         return QString();
00074
00075     return QDir(dirPath).filePath(normalizedDatabaseName());
00076 }
00077
00078 bool CreateDatabaseDialog::shouldCreateDefaultAthlete() const
00079 {
00080     return m_createAthleteCheck && m_createAthleteCheck->isChecked();
00081 }
00082
00083 QString CreateDatabaseDialog::defaultAthleteFirstName() const
00084 {
00085     return m_firstNameEdit ? m_firstNameEdit->text().trimmed() : QString();
00086 }
00087
00088 QString CreateDatabaseDialog::defaultAthleteLastName() const
00089 {
00090     return m_lastNameEdit ? m_lastNameEdit->text().trimmed() : QString();
00091 }
00092
00093 void CreateDatabaseDialog::browseLocation()
00094 {
00095     const QString current = m_locationEdit ? m_locationEdit->text().trimmed() : QString();
00096     const QString dir = QFileDialog::getExistingDirectory(this, "Choose Database Location", current);
00097     if (!dir.isEmpty() && m_locationEdit)
00098         m_locationEdit->setText(dir);
00099 }
00100
00101 void CreateDatabaseDialog::updateAthleteFields()
00102 {
00103     const bool enabled = shouldCreateDefaultAthlete();
00104     if (m_firstNameEdit)
00105         m_firstNameEdit->setEnabled(enabled);
00106     if (m_lastNameEdit)
00107         m_lastNameEdit->setEnabled(enabled);
00108 }
00109
00110 void CreateDatabaseDialog::accept()
00111 {
00112     const QString dirPath = m_locationEdit ? m_locationEdit->text().trimmed() : QString();
00113     if (dirPath.isEmpty())
00114     {
00115         QMessageBox::warning(this, "Missing Location", "Choose a folder for the new database.");
00116         return;
00117     }
00118
00119     const QFileInfo dirInfo(dirPath);
00120     if (!dirInfo.exists() || !dirInfo.isDir())
00121     {
00122         QMessageBox::warning(this, "Invalid Location", "Choose an existing folder for the new
database.");
00123         return;
00124     }
00125
00126     if (normalizedDatabaseName().isEmpty())
00127     {

```

```

00128         QMessageBox::warning(this, "Missing Name", "Enter a database name.");
00129         return;
00130     }
00131
00132     if (shouldCreateDefaultAthlete())
00133     {
00134         if (defaultAthleteFirstName().isEmpty() || defaultAthleteLastName().isEmpty())
00135         {
00136             QMessageBox::warning(this, "Missing Athlete Name",
00137                 "Enter a first and last name for the default athlete, or disable that option.");
00138             return;
00139         }
00140     }
00141
00142     QDialog::accept();
00143 }
00144
00145 QString CreateDatabaseDialog::normalizedDatabaseName() const
00146 {
00147     QString name = m_nameEdit ? m_nameEdit->text().trimmed() : QString();
00148     if (name.endsWith(".fitlyzerdb", Qt::CaseInsensitive))
00149         return name;
00150     if (!name.isEmpty())
00151         name += ".fitlyzerdb";
00152     return name;
00153 }

```

8.89 CreateDatabaseDialog.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QDialog>
00007
00008 class QCheckBox;
00009 class QDialogButtonBox;
00010 class QLineEdit;
00011 class QPushButton;
00012
00018 class CreateDatabaseDialog : public QDialog
00019 {
00020     Q_OBJECT
00021
00022 public:
00023     explicit CreateDatabaseDialog(const QString& defaultDirectory, QWidget* parent = nullptr);
00024
00025     QString databasePath() const;
00026
00027     bool shouldCreateDefaultAthlete() const;
00028
00029     QString defaultAthleteFirstName() const;
00030
00031     QString defaultAthleteLastName() const;
00032
00033 private slots:
00034     void browseLocation();
00035     void updateAthleteFields();
00036     void accept() override;
00037
00038 private:
00039     QString normalizedDatabaseName() const;
00040
00041     QLineEdit* m_nameEdit = nullptr;
00042     QLineEdit* m_locationEdit = nullptr;
00043     QCheckBox* m_createAthleteCheck = nullptr;
00044     QLineEdit* m_firstNameEdit = nullptr;
00045     QLineEdit* m_lastNameEdit = nullptr;
00046     QPushButton* m_browseButton = nullptr;
00047     QDialogButtonBox* m_buttonBox = nullptr;
00048 };

```

8.90 EditActivityDialog.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "EditActivityDialog.h"
00005
00006 #include <QDialogButtonBox>

```

```

00007 #include <QDoubleSpinBox>
00008 #include <QFormLayout>
00009 #include <QLineEdit>
00010 #include <QSpinBox>
00011 #include <QTextEdit>
00012 #include <QVBoxLayout>
00013
00014 EditActivityDialog::EditActivityDialog(const Activity& activity, QWidget* parent)
00015     : QDialog(parent)
00016     , m_initial(activity)
00017 {
00018     setWindowTitle("Activity Properties");
00019     setMinimumWidth(460);
00020
00021     auto* layout = new QVBoxLayout(this);
00022     auto* form = new QFormLayout;
00023
00024     m_temperatureSpin = new QDoubleSpinBox(this);
00025     m_temperatureSpin->setRange(-40.0, 60.0);
00026     m_temperatureSpin->setDecimals(1);
00027     m_temperatureSpin->setSuffix(" C");
00028     m_temperatureSpin->setValue(activity.temperature);
00029
00030     m_weatherEdit = new QLineEdit(activity.weather, this);
00031     m_windEdit = new QLineEdit(activity.wind, this);
00032
00033     m_rpeSpin = new QSpinBox(this);
00034     m_rpeSpin->setRange(0, 10);
00035     m_rpeSpin->setValue(activity.rpe);
00036
00037     m_fatigueSpin = new QSpinBox(this);
00038     m_fatigueSpin->setRange(0, 10);
00039     m_fatigueSpin->setValue(activity.fatigue);
00040
00041     m_sleepSpin = new QDoubleSpinBox(this);
00042     m_sleepSpin->setRange(0.0, 24.0);
00043     m_sleepSpin->setDecimals(1);
00044     m_sleepSpin->setSuffix(" h");
00045     m_sleepSpin->setValue(activity.sleepHours);
00046
00047     m_weightSpin = new QDoubleSpinBox(this);
00048     m_weightSpin->setRange(0.0, 200.0);
00049     m_weightSpin->setDecimals(1);
00050     m_weightSpin->setSuffix(" kg");
00051     m_weightSpin->setValue(activity.weightKg);
00052
00053     m_bikeEdit = new QLineEdit(activity.bike, this);
00054     m_equipmentEdit = new QLineEdit(activity.equipment, this);
00055
00056     m_notesEdit = new QTextEdit(this);
00057     m_notesEdit->setMinimumHeight(120);
00058     m_notesEdit->setPlainText(activity.notes);
00059
00060     form->addRow("Temperature:", m_temperatureSpin);
00061     form->addRow("Weather:", m_weatherEdit);
00062     form->addRow("Wind:", m_windEdit);
00063     form->addRow("RPE:", m_rpeSpin);
00064     form->addRow("Fatigue:", m_fatigueSpin);
00065     form->addRow("Sleep:", m_sleepSpin);
00066     form->addRow("Weight:", m_weightSpin);
00067     form->addRow("Bike:", m_bikeEdit);
00068     form->addRow("Equipment:", m_equipmentEdit);
00069     form->addRow("Notes:", m_notesEdit);
00070
00071     layout->addLayout(form);
00072
00073     auto* buttons = new QDialogButtonBox(QDialogButtonBox::Ok | QDialogButtonBox::Cancel, this);
00074     connect(buttons, &QDialogButtonBox::accepted, this, &QDialog::accept);
00075     connect(buttons, &QDialogButtonBox::rejected, this, &QDialog::reject);
00076     layout->addWidget(buttons);
00077 }
00078
00079 Activity EditActivityDialog::updatedActivity() const
00080 {
00081     Activity out = m_initial;
00082     out.temperature = m_temperatureSpin ? m_temperatureSpin->value() : m_initial.temperature;
00083     out.weather = m_weatherEdit ? m_weatherEdit->text().trimmed() : m_initial.weather;
00084     out.wind = m_windEdit ? m_windEdit->text().trimmed() : m_initial.wind;
00085     out.rpe = m_rpeSpin ? m_rpeSpin->value() : m_initial.rpe;
00086     out.fatigue = m_fatigueSpin ? m_fatigueSpin->value() : m_initial.fatigue;
00087     out.sleepHours = m_sleepSpin ? m_sleepSpin->value() : m_initial.sleepHours;
00088     out.weightKg = m_weightSpin ? m_weightSpin->value() : m_initial.weightKg;
00089     out.bike = m_bikeEdit ? m_bikeEdit->text().trimmed() : m_initial.bike;
00090     out.equipment = m_equipmentEdit ? m_equipmentEdit->text().trimmed() : m_initial.equipment;
00091     out.notes = m_notesEdit ? m_notesEdit->toPlainText().trimmed() : m_initial.notes;
00092     return out;
00093 }

```


8.91 EditActivityDialog.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QDialog>
00007
00008 #include "database/ActivityRepository.h"
00009
00010 class QDoubleSpinBox;
00011 class QLineEdit;
00012 class QTextEdit;
00013 class QSpinBox;
00014
00021 class EditActivityDialog : public QDialog
00022 {
00023     Q_OBJECT
00024
00025 public:
00026     explicit EditActivityDialog(const Activity& activity, QWidget* parent = nullptr);
00027
00028     Activity updatedActivity() const;
00029
00030 private:
00031     Activity m_initial;
00032
00033     QDoubleSpinBox* m_temperatureSpin = nullptr;
00034     QLineEdit* m_weatherEdit = nullptr;
00035     QLineEdit* m_windEdit = nullptr;
00036     QSpinBox* m_rpeSpin = nullptr;
00037     QSpinBox* m_fatigueSpin = nullptr;
00038     QDoubleSpinBox* m_sleepSpin = nullptr;
00039     QDoubleSpinBox* m_weightSpin = nullptr;
00040     QLineEdit* m_bikeEdit = nullptr;
00041     QLineEdit* m_equipmentEdit = nullptr;
00042     QTextEdit* m_notesEdit = nullptr;
00043 };

```

8.92 EditNotesDialog.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "EditNotesDialog.h"
00005
00006 #include <QDialogButtonBox>
00007 #include <QFormLayout>
00008 #include <QLabel>
00009 #include <QLineEdit>
00010 #include <QTextEdit>
00011 #include <QVBoxLayout>
00012
00013 EditNotesDialog::EditNotesDialog(const QString& notes,
00014                                   const QString& weatherNotes,
00015                                   QWidget* parent)
00016     : QDialog(parent)
00017 {
00018     setWindowTitle("Edit Activity Notes");
00019     setMinimumWidth(420);
00020
00021     auto* layout = new QVBoxLayout(this);
00022
00023     auto* form = new QFormLayout;
00024     m_notes = new QTextEdit;
00025     m_notes->setPlainText(notes);
00026     m_notes->setMinimumHeight(120);
00027     form->addRow("Notes:", m_notes);
00028
00029     m_weather = new QLineEdit(weatherNotes);
00030     form->addRow("Weather:", m_weather);
00031
00032     layout->addLayout(form);
00033
00034     auto* bbox = new QDialogButtonBox(
00035         QDialogButtonBox::Ok | QDialogButtonBox::Cancel);
00036     layout->addWidget(bbox);
00037
00038     connect(bbox, &QDialogButtonBox::accepted, this, &QDialog::accept);
00039     connect(bbox, &QDialogButtonBox::rejected, this, &QDialog::reject);
00040 }
00041

```

```

00042 QString EditNotesDialog::notes() const
00043 {
00044     return m_notes->toPlainText();
00045 }
00046
00047 QString EditNotesDialog::weatherNotes() const
00048 {
00049     return m_weather->text();
00050 }

```

8.93 EditNotesDialog.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QDialog>
00007 #include <QString>
00008
00009 class QLineEdit;
00010 class QTextEdit;
00011
00015 class EditNotesDialog : public QDialog
00016 {
00017     Q_OBJECT
00018 public:
00019     explicit EditNotesDialog(const QString& notes,
00020                             const QString& weatherNotes,
00021                             QWidget* parent = nullptr);
00022
00023     QString notes() const;
00024
00025     QString weatherNotes() const;
00026
00027 private:
00028     QTextEdit* m_notes = nullptr;
00029     QLineEdit* m_weather = nullptr;
00030 };

```

8.94 ImportStatusWidget.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "ImportStatusWidget.h"
00004
00005 #include <QHBoxLayout>
00006 #include <QLabel>
00007 #include <QProgressBar>
00008
00009 #include "import/ImportProgressModel.h"
00010
00011 ImportStatusWidget::ImportStatusWidget(QWidget* parent)
00012     : QWidget(parent)
00013 {
00014     auto* row = new QHBoxLayout(this);
00015     row->setContentsMargins(0, 0, 0, 0);
00016     row->setSpacing(6);
00017
00018     m_titleLabel = new QLabel(this);
00019     m_titleLabel->setMinimumWidth(120);
00020
00021     m_progressBar = new QProgressBar(this);
00022     m_progressBar->setRange(0, 100);
00023     m_progressBar->setMinimumWidth(150);
00024     m_progressBar->setTextVisible(true);
00025
00026     m_detailLabel = new QLabel(this);
00027     m_detailLabel->setMinimumWidth(220);
00028
00029     row->addWidget(m_titleLabel);
00030     row->addWidget(m_progressBar);
00031     row->addWidget(m_detailLabel);
00032
00033     hide();
00034 }
00035
00036 void ImportStatusWidget::setModel(ImportProgressModel* model)
00037 {
00038     if (m_model == model)
00039         return;

```

```

00040
00041     if (m_model)
00042         disconnect(m_model, nullptr, this, nullptr);
00043
00044     m_model = model;
00045     if (m_model)
00046         connect(m_model, &ImportProgressModel::changed, this, &ImportStatusWidget::updateFromModel);
00047
00048     updateFromModel();
00049 }
00050
00051 void ImportStatusWidget::updateFromModel()
00052 {
00053     if (!m_model || m_model->activeJobs() <= 0)
00054     {
00055         hide();
00056         return;
00057     }
00058
00059     show();
00060
00061     const int remaining = m_model->activeJobs();
00062     const QString plural = remaining == 1 ? QStringLiteral("file") : QStringLiteral("files");
00063     m_titleLabel->setText(QString("Importing %1 %2...").arg(remaining).arg(plural));
00064     m_progressBar->setValue(static_cast<int>(m_model->progress()));
00065     m_detailLabel->setText(
00066         QString("Current: %1 - %2")
00067             .arg(m_model->currentFileName().isEmpty() ? QStringLiteral("-") :
00068                 m_model->currentFileName())
00069             .arg(m_model->statusText().isEmpty() ? QStringLiteral("Working") :
00070                 m_model->statusText()));
00069 }

```

8.95 ImportStatusWidget.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <QWidget>
00006
00007 class QLabel;
00008 class QProgressBar;
00009 class ImportProgressModel;
00010
00014 class ImportStatusWidget : public QWidget
00015 {
00016     Q_OBJECT
00017 public:
00018     explicit ImportStatusWidget(QWidget* parent = nullptr);
00019
00021     void setModel(ImportProgressModel* model);
00022
00023 private:
00024     void updateFromModel();
00025
00026     ImportProgressModel* m_model = nullptr;
00027     QLabel* m_titleLabel = nullptr;
00028     QLabel* m_detailLabel = nullptr;
00029     QProgressBar* m_progressBar = nullptr;
00030 };

```

8.96 IntervalEditorDialog.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "IntervalEditorDialog.h"
00005
00006 #include <QDialogButtonBox>
00007 #include <QDoubleSpinBox>
00008 #include <QFormLayout>
00009 #include <QLineEdit>
00010 #include <QTextEdit>
00011 #include <QVBoxLayout>
00012
00013 IntervalEditorDialog::IntervalEditorDialog(double startSeconds,
00014     double endSeconds,
00015     QWidget* parent)
00016 : QDialog(parent)
00017 {

```

```

00018     setWindowTitle("Create Interval");
00019     setMinimumWidth(420);
00020
00021     auto* layout = new QVBoxLayout(this);
00022     auto* form = new QFormLayout;
00023
00024     m_nameEdit = new QLineEdit("Manual Interval", this);
00025     m_typeEdit = new QLineEdit("Manual", this);
00026
00027     m_startSpin = new QDoubleSpinBox(this);
00028     m_startSpin->setRange(0.0, 86400.0);
00029     m_startSpin->setSuffix(" s");
00030     m_startSpin->setValue(startSeconds);
00031
00032     m_endSpin = new QDoubleSpinBox(this);
00033     m_endSpin->setRange(0.0, 86400.0);
00034     m_endSpin->setSuffix(" s");
00035     m_endSpin->setValue(endSeconds);
00036
00037     m_notesEdit = new QTextEdit(this);
00038     m_notesEdit->setMinimumHeight(100);
00039
00040     form->addRow("Name:", m_nameEdit);
00041     form->addRow("Type:", m_typeEdit);
00042     form->addRow("Start:", m_startSpin);
00043     form->addRow("End:", m_endSpin);
00044     form->addRow("Notes:", m_notesEdit);
00045
00046     layout->addLayout(form);
00047
00048     auto* buttons = new QDialogButtonBox(QDialogButtonBox::Ok | QDialogButtonBox::Cancel, this);
00049     connect(buttons, &QDialogButtonBox::accepted, this, &QDialog::accept);
00050     connect(buttons, &QDialogButtonBox::rejected, this, &QDialog::reject);
00051     layout->addWidget(buttons);
00052 }
00053
00054 QString IntervalEditorDialog::intervalName() const
00055 {
00056     return m_nameEdit ? m_nameEdit->text().trimmed() : QString();
00057 }
00058
00059 QString IntervalEditorDialog::intervalType() const
00060 {
00061     return m_typeEdit ? m_typeEdit->text().trimmed() : QString();
00062 }
00063
00064 QString IntervalEditorDialog::intervalNotes() const
00065 {
00066     return m_notesEdit ? m_notesEdit->toPlainText().trimmed() : QString();
00067 }

```

8.97 IntervalEditorDialog.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QDialog>
00007 #include <QString>
00008
00009 class QDoubleSpinBox;
00010 class QLineEdit;
00011 class QTextEdit;
00012
00013 class IntervalEditorDialog : public QDialog
00014 {
00015     Q_OBJECT
00016
00017 public:
00018     explicit IntervalEditorDialog(double startSeconds,
00019                                   double endSeconds,
00020                                   QWidget* parent = nullptr);
00021
00022     QString intervalName() const;
00023
00024     QString intervalType() const;
00025
00026     QString intervalNotes() const;
00027
00028 private:
00029     QLineEdit* m_nameEdit = nullptr;
00030     QLineEdit* m_typeEdit = nullptr;
00031     QTextEdit* m_notesEdit = nullptr;

```

```
00055     QDoubleSpinBox* m_startSpin = nullptr;
00056     QDoubleSpinBox* m_endSpin = nullptr;
00057 };
```

8.98 MainWindow.cpp

```
00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "MainWindow.h"
00005
00006 #include <QAction>
00007 #include <QApplication>
00008 #include <QBrush>
00009 #include <QCheckBox>
00010 #include <QCloseEvent>
00011 #include <QColor>
00012 #include <QComboBox>
00013 #include <QCoreApplication>
00014 #include <QDate>
00015 #include <QDialog>
00016 #include <QDialogButtonBox>
00017 #include <QDoubleSpinBox>
00018 #include <QDragEnterEvent>
00019 #include <QDropEvent>
00020 #include <QDir>
00021 #include <QDirIterator>
00022 #include <QFileDialog>
00023 #include <QFileInfo>
00024 #include <QFile>
00025 #include <QFileSystemWatcher>
00026 #include <QFormLayout>
00027 #include <QGroupBox>
00028 #include <QHBoxLayout>
00029 #include <QHeaderView>
00030 #include <QIcon>
00031 #include <QMenuBar>
00032 #include <QMessageBox>
00033 #include <QMimeData>
00034 #include <QKeySequence>
00035 #include <QScrollArea>
00036 #include <QScrollBar>
00037 #include <QSettings>
00038 #include <QSignalBlocker>
00039 #include <QSizePolicy>
00040 #include <QStandardPaths>
00041 #include <QSplitter>
00042 #include <QSqlQuery>
00043 #include <QStatusBar>
00044 #include <QStackedLayout>
00045 #include <QStyle>
00046 #include <QTableWidgetItem>
00047 #include <QToolBar>
00048 #include <QToolButton>
00049 #include <QTimer>
00050 #include <QUrl>
00051 #include <QUuid>
00052 #include <QVBoxLayout>
00053 #include <QWidget>
00054 #include <QPainter>
00055
00056 #include <QtCharts/QAbstractAxis>
00057 #include <QtCharts/QBarCategoryAxis>
00058 #include <QtCharts/QBarSeries>
00059 #include <QtCharts/QBarSet>
00060 #include <QtCharts/QChart>
00061 #include <QtCharts/QChartView>
00062 #include <QtCharts/QValueAxis>
00063
00064 #include "ActivityBrowser.h"
00065 #include "AboutDialog.h"
00066 #include "AthleteHeaderWidget.h"
00067 #include "AthleteDialog.h"
00068 #include "AthleteListDialog.h"
00069 #include "CalendarWidget.h"
00070 #include "CreateDatabaseDialog.h"
00071 #include "EditActivityDialog.h"
00072 #include "ImportStatusWidget.h"
00073 #include "IntervalEditorDialog.h"
00074 #include "WelcomeWidget.h"
00075 #include "charts/FitnessChartWidget.h"
00076 #include "charts/PowerCurveWidget.h"
00077 #include "charts/PowerHistogramWidget.h"
00078 #include "charts/RideChartWidget.h"
```

```

00079 #include "core/undo/ClimbEditCommand.h"
00080 #include "analysis/ClimbDetector.h"
00081 #include "analysis/ClimbMetrics.h"
00082 #include "analysis/IntervalDetector.h"
00083 #include "core/settings/AppSettings.h"
00084 #include "core/settings/DateFormatter.h"
00085 #include "core/zones/ZoneCalculator.h"
00086 #include "database/AthleteRepository.h"
00087 #include "database/ClimbRepository.h"
00088 #include "database/IntervalRepository.h"
00089 #include "maps/MapRenderer.h"
00090 #include "model/RideDataSerializer.h"
00091 #include "platform/Platform.h"
00092 #include "import/ImportProgressModel.h"
00093 #include "video/VideoExportDialog.h"
00094 #include "video/VideoExportSettings.h"
00095 #include "utils/FfmpegPath.h"
00096
00097 #include <cmath>
00098 #include <functional>
00099 #include <map>
00100 #include <numeric>
00101
00102 // Helpers
00103
00110 static QString fmtDur(double seconds)
00111 {
00112     const int total = static_cast<int>(seconds);
00113     const int h = total / 3600;
00114     const int m = (total % 3600) / 60;
00115     const int s = total % 60;
00116     return h > 0
00117         ? QString("%1:%2:%3")
00118           .arg(h).arg(m, 2, 10, QChar('0')).arg(s, 2, 10, QChar('0'))
00119         : QString("%1:%2")
00120           .arg(m).arg(s, 2, 10, QChar('0'));
00121 }
00122
00128 static QString formatActivityDateLabel(const QString& isoDate)
00129 {
00130     const QDate date = QDate::fromString(isoDate, Qt::ISODate);
00131     if (!date.isValid())
00132         return QStringLiteral("-");
00133
00134     const QDate today = QDate::currentDate();
00135     if (date == today)
00136         return QStringLiteral("Today");
00137     if (date == today.addDays(-1))
00138         return QStringLiteral("Yesterday");
00139
00140     return DateFormatter::formatDate(date);
00141 }
00142
00148 static QDate activityDateForFtpContext(const Activity& activity)
00149 {
00150     const QString rawDate = !activity.startTime.isEmpty()
00151         ? activity.startTime.left(10)
00152         : activity.importedAt.left(10);
00153     return QDate::fromString(rawDate, Qt::ISODate);
00154 }
00155
00161 static QDate activityDateForWeightContext(const Activity& activity)
00162 {
00163     const QString rawDate = !activity.startTime.isEmpty()
00164         ? activity.startTime.left(10)
00165         : activity.importedAt.left(10);
00166     return QDate::fromString(rawDate, Qt::ISODate);
00167 }
00168
00170 static constexpr int kMaxRecentDatabases = 8;
00171
00175 enum ChartPresetId
00176 {
00177     ChartPresetCustom = 0,
00178     ChartPresetPowerAnalysis,
00179     ChartPresetHeartRateFocus,
00180     ChartPresetRaceReview
00181 };
00182
00184 static constexpr int kIntervalStartRole = Qt::UserRole + 1;
00185
00187 static constexpr int kIntervalEndRole = Qt::UserRole + 2;
00188
00190 static constexpr int kClimbStartRole = Qt::UserRole + 3;
00191
00193 static constexpr int kClimbEndRole = Qt::UserRole + 4;

```

```

00194
00201 static QString rangeLabelForZone(const Zone& zone, ColorMetric metric)
00202 {
00203     const QString unit = colorMetricUnit(metric);
00204     const QString minText = QString::number(zone.minValue, 'f', metric == ColorMetric::Speed ? 1 : 0);
00205     const QString maxText = QString::number(zone.maxValue, 'f', metric == ColorMetric::Speed ? 1 : 0);
00206
00207     if (zone.maxValue >= DBL_MAX / 2.0)
00208         return QString("> %1 %2").arg(minText, unit).trimmed();
00209
00210     return QString("%1 - %2 %3").arg(minText, maxText, unit).trimmed();
00211 }
00212
00218 static QString sanitizeVideoFilePart(QString text)
00219 {
00220     static const QString invalid = QStringLiteral("\\/:*?\"<>|");
00221     for (const QChar c : invalid)
00222         text.replace(c, '-');
00223     text = text.simplified().replace(' ', '-');
00224     while (text.contains("--"))
00225         text.replace("--", "-");
00226     return text.trimmed();
00227 }
00228
00236 static QWidget* wrapWithNoActivityState(
00237     QWidget* content,
00238     QStackedLayout** outStack,
00239     const QString& message = QStringLiteral("No Activity Selected"))
00240 {
00241     auto* root = new QWidget;
00242     auto* stack = new QStackedLayout(root);
00243     stack->setContentsMargins(0, 0, 0, 0);
00244
00245     auto* emptyPage = new QWidget(root);
00246     auto* emptyLayout = new QVBoxLayout(emptyPage);
00247     emptyLayout->setContentsMargins(0, 0, 0, 0);
00248     auto* label = new QLabel(message, emptyPage);
00249     label->setAlignment(Qt::AlignCenter);
00250     label->setStyleSheet("color: #64748b; font-size: 16px; font-weight: 600;");
00251     emptyLayout->addWidget(label, 1);
00252
00253     stack->addWidget(emptyPage);
00254     stack->addWidget(content);
00255     stack->setCurrentIndex(0);
00256
00257     if (*outStack)
00258         *outStack = stack;
00259     return root;
00260 }
00261
00265 class ClimbNameTableWidgetItem final : public QTableWidgetItem
00266 {
00267 public:
00272     explicit ClimbNameTableWidgetItem(const QString& text)
00273         : QTableWidgetItem(text)
00274     {}
00275
00281     bool operator<(const QTableWidgetItem& other) const override
00282     {
00283         bool leftOk = false;
00284         bool rightOk = false;
00285         const int leftNumber = trailingNumber(text(), &leftOk);
00286         const int rightNumber = trailingNumber(other.text(), &rightOk);
00287
00288         if (leftOk && rightOk)
00289             return leftNumber < rightNumber;
00290
00291         return QString::localeAwareCompare(text(), other.text()) < 0;
00292     }
00293
00294 private:
00295     static int trailingNumber(const QString& text, bool* ok)
00296     {
00297         if (ok)
00298             *ok = false;
00299
00300         int i = text.size() - 1;
00301         while (i >= 0 && text[i].isSpace())
00302             --i;
00303
00304         if (i < 0 || !text[i].isDigit())
00305             return 0;
00306
00307         int end = i;
00308         while (i >= 0 && text[i].isDigit())
00309             --i;
00310

```

```

00311         const QString number = text.mid(i + 1, end - i);
00312         bool converted = false;
00313         const int value = number.toInt(&converted);
00314         if (ok)
00315             *ok = converted;
00316         return value;
00317     }
00318 };
00319
00323 class ClimbSortKeyTableWidgetItem final : public QTableWidgetItem
00324 {
00325 public:
00331     explicit ClimbSortKeyTableWidgetItem(const QString& text, double sortKey)
00332         : QTableWidgetItem(text)
00333         , m_sortKey(sortKey)
00334     {}
00335
00341     bool operator<(const QTableWidgetItem& other) const override
00342     {
00343         const auto* rhs = dynamic_cast<const ClimbSortKeyTableWidgetItem*>(&other);
00344         if (!rhs)
00345             return QString::localeAwareCompare(text(), other.text()) < 0;
00346
00347         const bool leftNaN = std::isnan(m_sortKey);
00348         const bool rightNaN = std::isnan(rhs->m_sortKey);
00349         if (leftNaN != rightNaN)
00350             return !leftNaN;
00351         if (leftNaN && rightNaN)
00352             return QString::localeAwareCompare(text(), rhs->text()) < 0;
00353
00354         if (m_sortKey == rhs->m_sortKey)
00355             return QString::localeAwareCompare(text(), rhs->text()) < 0;
00356         return m_sortKey < rhs->m_sortKey;
00357     }
00358
00359 private:
00360     double m_sortKey = std::numeric_limits<double>::quiet_NaN();
00361 };
00362
00363 // Constructor
00364
00365 MainWindow::MainWindow(QWidget* parent)
00366     : QMainWindow(parent)
00367 {
00368     setAcceptDrops(true);
00369
00370     m_controller = new WorkoutController(this);
00371     m_controller->setDatabaseManager(&m_dbManager);
00372
00373     buildUI();
00374
00375     connect(m_controller, &WorkoutController::workoutLoaded,
00376             this, &MainWindow::onWorkoutLoaded);
00377     connect(m_controller, &WorkoutController::activityImported,
00378             this, &MainWindow::onActivityImported);
00379
00380     m_undoManager = new UndoManager(50, this);
00381     m_analysisQueue = new AnalysisQueue(this);
00382     m_controller->setAnalysisQueue(m_analysisQueue);
00383     m_importQueue = new ImportQueue(this);
00384     m_importProgressModel = new ImportProgressModel(this);
00385     m_importProgressModel->attachQueue(m_importQueue);
00386
00387     m_importRefreshTimer = new QTimer(this);
00388     m_importRefreshTimer->setSingleShot(true);
00389     m_importRefreshTimer->setInterval(500);
00390     connect(m_importRefreshTimer, &QTimer::timeout, this, [this]()
00391     {
00392         if (m_activityBrowser)
00393             m_activityBrowser->refresh(m_currentAthleteId);
00394         updateStatsLabel();
00395         updateStatusBarInfo();
00396         updateWelcomeScreenVisibility();
00397     });
00398
00399     // Status bar feedback from the analysis queue
00400     connect(m_analysisQueue, &AnalysisQueue::pendingCountChanged, this, [this](int count)
00401     {
00402         if (count > 0)
00403             statusBar()->showMessage(QString("Analyzing %1 activit%2 in background...")
00404                                     .arg(count).arg(count == 1 ? "y" : "ies"), 0);
00405         else
00406             statusBar()->clearMessage();
00407     });
00408     connect(m_analysisQueue, &AnalysisQueue::taskCompleted, this, [this](int activityId)
00409     {
00410         // If the completed task is the currently loaded activity, refresh climbing/intervals

```



```

00411         if (m_controller && m_controller->currentActivityId() == activityId)
00412         {
00413             m_controller->reloadClimbsFromDatabase();
00414             m_detectedClimbs = m_controller->climbs();
00415             applyWeightMetricsToClimbs(m_detectedClimbs, resolveActiveActivityWeightKg());
00416             refreshClimbViews();
00417             updateIntervals();
00418         }
00419     });
00420
00421     connect(m_importQueue, &ImportQueue::jobFinished, this,
00422            [this](const QString&, const QString& batchId, const QString&, int activityId, int)
00423            {
00424                if (activityId > 0 && m_analysisQueue)
00425                    m_analysisQueue->enqueue(activityId);
00426
00427                auto it = m_importBatchSummaries.find(batchId);
00428                if (it != m_importBatchSummaries.end())
00429                    ++it->imported;
00430
00431                scheduleActivityBrowserRefresh();
00432                hideWelcomeScreen();
00433            });
00434
00435     connect(m_importQueue, &ImportQueue::jobFailed, this,
00436            [this](const QString&, const QString& batchId, const QString& filePath,
00437                const QString& reason, const QString& errorMessage, int)
00438            {
00439                auto it = m_importBatchSummaries.find(batchId);
00440                if (it != m_importBatchSummaries.end())
00441                {
00442                    if (reason == QStringLiteral("duplicate") || reason == QStringLiteral("time_overlap"))
00443                        ++it->duplicates;
00444                    else
00445                        ++it->failed;
00446
00447                    if (!errorMessage.isEmpty())
00448                        it->errors « QString("%1: %2").arg(QFileInfo(filePath).fileName(), errorMessage);
00449                }
00450            });
00451
00452     connect(m_importQueue, &ImportQueue::queueIdle, this, [this]()
00453            {
00454                scheduleActivityBrowserRefresh();
00455                finalizeImportBatches();
00456            });
00457
00458     if (m_activityBrowser)
00459     {
00460         connect(m_activityBrowser, &ActivityBrowser::fitFilesDropped,
00461                this, &MainWindow::importFiles);
00462     }
00463
00464     setupShortcuts();
00465     loadSettings();
00466     applyChartHeight();
00467
00468     QSettings s("Fitlyzer", "FitlyzerC");
00469     if (!s.contains("firstLaunchCompleted"))
00470         s.setValue("firstLaunchCompleted", false);
00471     m_firstLaunchCompleted = s.value("firstLaunchCompleted", false).toBool();
00472
00473     // Re-open last database and athlete
00474     const QString lastDb = s.value("lastDatabase").toString();
00475     if (!lastDb.isEmpty())
00476     {
00477         QString err;
00478         if (!m_dbManager.open(lastDb, &err))
00479         {
00480             QMessageBox::warning(this, "Database",
00481                QString("Could not open last database:\n%1\n\n%2")
00482                    .arg(lastDb).arg(err));
00483         }
00484         else
00485         {
00486             m_controller->setDatabaseManager(&m_dbManager);
00487             if (m_analysisQueue)
00488                 m_analysisQueue->setDatabasePath(lastDb);
00489             if (m_importQueue)
00490                 m_importQueue->setDatabasePath(lastDb);
00491             const int aid = s.value("currentAthleteId", -1).toInt();
00492             if (aid > 0)
00493             {
00494                 m_currentAthleteId = aid;
00495                 m_controller->setCurrentAthlete(aid);
00496                 auto db = m_dbManager.database();
00497                 AthleteRepository repo(db);

```

```

00498         m_currentAthleteName = repo.getAthlete(aid).fullName();
00499     }
00500     updateAthleteLabel();
00501     refreshAthleteSelector();
00502     if (m_activityBrowser)
00503         m_activityBrowser->refresh(m_currentAthleteId);
00504
00505     const int lastActivityId = s.value("lastActivityId", -1).toInt();
00506     if (lastActivityId > 0)
00507     {
00508         QString loadErr;
00509         if (!m_controller->loadActivity(lastActivityId, loadErr))
00510             s.remove("lastActivityId");
00511     }
00512 }
00513 }
00514
00515 updateRecentDatabaseMenu();
00516 refreshAthleteSelector();
00517 updateImportAvailability();
00518 updateStatsLabel();
00519 updateStatusBarInfo();
00520 updateWelcomeScreenVisibility();
00521 configureFolderWatcher();
00522 }
00523
00524 // Close event
00525
00526 void MainWindow::closeEvent(QCloseEvent* event)
00527 {
00528     saveSettings();
00529     QMainWindow::closeEvent(event);
00530 }
00531
00532 // UI Construction
00533
00534 void MainWindow::buildUI()
00535 {
00536     // -- Menu bar -----
00537     {
00538         // File menu (database-oriented workflow)
00539         auto* fileMenu = menuBar()->addMenu("&File");
00540
00541         auto* openDbAct = new QAction("Open Database...", this);
00542         connect(openDbAct, &QAction::triggered, this, &MainWindow::openDatabase);
00543         fileMenu->addAction(openDbAct);
00544
00545         auto* newDbAct = new QAction("Create Database...", this);
00546         connect(newDbAct, &QAction::triggered, this, &MainWindow::createDatabase);
00547         fileMenu->addAction(newDbAct);
00548
00549         auto* backupAct = new QAction("Backup Database...", this);
00550         connect(backupAct, &QAction::triggered, this, &MainWindow::backupDatabase);
00551         fileMenu->addAction(backupAct);
00552
00553         fileMenu->addSeparator();
00554         m_recentDbMenu = fileMenu->addMenu("Recent Databases");
00555
00556         fileMenu->addSeparator();
00557
00558         auto* exitAct = new QAction("E&xit", this);
00559         exitAct->setShortcut(QKeySequence::Quit);
00560         connect(exitAct, &QAction::triggered, qApp, &QApplication::quit);
00561         fileMenu->addAction(exitAct);
00562
00563         // Activities menu
00564         auto* actsMenu = menuBar()->addMenu("&Activities");
00565
00566         m_importAct = new QAction("Import Activities...", this);
00567         m_importAct->setShortcut(QKeySequence::Open);
00568         m_importAct->setEnabled(false);
00569         connect(m_importAct, &QAction::triggered, this, &MainWindow::importActivities);
00570         actsMenu->addAction(m_importAct);
00571
00572         auto* createVideoAct = new QAction("Create Video...", this);
00573         connect(createVideoAct, &QAction::triggered, this, &MainWindow::createVideo);
00574         actsMenu->addAction(createVideoAct);
00575
00576         actsMenu->addSeparator();
00577
00578         auto* detectClimbsAct = new QAction("Detect Climbs...", this);
00579         connect(detectClimbsAct, &QAction::triggered, this, &MainWindow::triggerDetectClimbs);
00580         actsMenu->addAction(detectClimbsAct);
00581
00582         auto* detectIntervalsAct = new QAction("Detect Intervals...", this);
00583         connect(detectIntervalsAct, &QAction::triggered, this, &MainWindow::triggerDetectIntervals);
00584         actsMenu->addAction(detectIntervalsAct);

```

```

00585
00586     // Athletes menu
00587     m_athletesMenu = menuBar()->addMenu("&Athletes");
00588
00589     auto* manageAct = new QAction("Manage Athletes...", this);
00590     connect(manageAct, &QAction::triggered, this, &MainWindow::manageAthletes);
00591     m_athletesMenu->addAction(manageAct);
00592
00593     // Tools menu for power-user actions
00594     auto* toolsMenu = menuBar()->addMenu("&Tools");
00595     m_previewAct = new QAction("Preview FIT File...", this);
00596     connect(m_previewAct, &QAction::triggered, this, &MainWindow::previewFitFile);
00597     toolsMenu->addAction(m_previewAct);
00598     toolsMenu->addSeparator();
00599     auto* settingsAct = new QAction("Settings...", this);
00600     settingsAct->setShortcut(QKeySequence::Preferences);
00601     connect(settingsAct, &QAction::triggered, this, &MainWindow::openSettingsDialog);
00602     toolsMenu->addAction(settingsAct);
00603
00604     // View and Help placeholders for expected desktop structure
00605     auto* viewMenu = menuBar()->addMenu("&View");
00606     viewMenu->addAction("Reset Zoom", QKeySequence("Ctrl+0"), this, &MainWindow::resetAllZoom);
00607     viewMenu->addAction("Increase Chart Height", this, &MainWindow::increaseChartHeight);
00608     viewMenu->addAction("Decrease Chart Height", this, &MainWindow::decreaseChartHeight);
00609     auto* helpMenu = menuBar()->addMenu("&Help");
00610     helpMenu->addAction("&About", this, &MainWindow::showAbout);
00611
00612 }
00613
00614 // Central widget
00615 auto* central = new QWidget(this);
00616 auto* mainLayout = new QVBoxLayout(central);
00617 mainLayout->setSpacing(6);
00618 mainLayout->setContentsMargins(8, 8, 8, 8);
00619
00620 // Header panel
00621 m_athleteHeader = new AthleteHeaderWidget(this);
00622 mainLayout->addWidget(m_athleteHeader);
00623
00624 m_welcomeWidget = new WelcomeWidget(this);
00625 mainLayout->addWidget(m_welcomeWidget, 1);
00626 m_welcomeWidget->setVisible(false);
00627 connect(m_welcomeWidget, &WelcomeWidget::importRequested, this, &MainWindow::importActivities);
00628 connect(m_welcomeWidget, &WelcomeWidget::openDatabaseRequested, this, &MainWindow::openDatabase);
00629 connect(m_welcomeWidget, &WelcomeWidget::createDatabaseRequested, this,
&MainWindow::createDatabase);
00630
00631 // Tab widget
00632 m_tabWidget = new QTabWidget;
00633 m_analysisTabWidget = new QTabWidget;
00634
00635 auto* analysisContainer = new QWidget(this);
00636 auto* analysisLayout = new QVBoxLayout(analysisContainer);
00637 analysisLayout->setContentsMargins(0, 0, 0, 0);
00638 analysisLayout->setSpacing(6);
00639
00640 auto* colorBar = new QHBoxLayout;
00641 colorBar->setContentsMargins(0, 0, 0, 0);
00642 colorBar->addWidget(new QLabel("Color By:"));
00643
00644 m_colorMetricCombo = new QComboBox(this);
00645 auto addColorMetric = [this](ColorMetric metric)
00646 {
00647     m_colorMetricCombo->addItem(colorMetricDisplayName(metric), static_cast<int>(metric));
00648 };
00649 addColorMetric(ColorMetric::None);
00650 addColorMetric(ColorMetric::Power);
00651 addColorMetric(ColorMetric::HeartRate);
00652 addColorMetric(ColorMetric::Cadence);
00653 addColorMetric(ColorMetric::Speed);
00654 addColorMetric(ColorMetric::Altitude);
00655 m_colorMetricCombo->setCurrentIndex(
00656     m_colorMetricCombo->findData(static_cast<int>(ColorMetric::Power)));
00657 colorBar->addWidget(m_colorMetricCombo);
00658 colorBar->addStretch(1);
00659 analysisLayout->addLayout(colorBar);
00660
00661 // == Tab 0: Activities Browser ==
00662 {
00663     m_activityBrowser = new ActivityBrowser(&m_dbManager, this);
00664     m_tabWidget->addTab(m_activityBrowser, "Activities");
00665
00666     connect(m_activityBrowser, &ActivityBrowser::activitySelected,
00667         this, [this](int activityId)
00668     {
00669         QString err;
00670         if (!m_controller->loadActivity(activityId, err))

```

```

00671         QMessageBox::critical(this, "Load Error", err);
00672     else
00673     {
00674         QSettings("Fitlyzer", "FitlyzerC").setValue("lastActivityId", activityId);
00675         m_tabWidget->setCurrentIndex(kTabAnalysis);
00676         if (m_analysisTabWidget)
00677             m_analysisTabWidget->setCurrentIndex(kAnalysisTabCharts);
00678     }
00679 });
00680
00681 connect(m_activityBrowser, &ActivityBrowser::activityDeleted,
00682         this, [this](int activityId)
00683 {
00684     // If the currently loaded activity was deleted, clear the view.
00685     (void)activityId;
00686     updateStatsLabel();
00687     updateStatusBarInfo();
00688     updateWelcomeScreenVisibility();
00689 });
00690 }
00691
00692 // == Tab 1: Charts ==
00693 {
00694     // -- Metrics dropdown -----
00695     m_showPower = new QCheckBox("Power");      m_showPower->setChecked(true);
00696     m_showHR = new QCheckBox("Heart Rate");    m_showHR->setChecked(true);
00697     m_showCadence = new QCheckBox("Cadence");  m_showCadence->setChecked(true);
00698     m_showSpeed = new QCheckBox("Speed");      m_showSpeed->setChecked(true);
00699     m_showAltitude = new QCheckBox("Altitude"); m_showAltitude->setChecked(true);
00700
00701     auto* metricsBtn = new QToolButton;
00702     metricsBtn->setText("Metrics");
00703     metricsBtn->setPopupMode(QToolButton::InstantPopup);
00704     auto* metricsMenu = new QMenu(metricsBtn);
00705     auto addMetricAction = [metricsMenu](const QString& title, QCheckBox* box)
00706     {
00707         auto* act = metricsMenu->addAction(title);
00708         act->setCheckable(true);
00709         act->setChecked(box->isChecked());
00710         QObject::connect(act, &QAction::toggled, box, &QCheckBox::setChecked);
00711         QObject::connect(box, &QCheckBox::toggled, act, &QAction::setChecked);
00712     };
00713     addMetricAction("Power", m_showPower);
00714     addMetricAction("Heart Rate", m_showHR);
00715     addMetricAction("Cadence", m_showCadence);
00716     addMetricAction("Speed", m_showSpeed);
00717     addMetricAction("Altitude", m_showAltitude);
00718     metricsBtn->setMenu(metricsMenu);
00719
00720     m_powerSmoothingCombo = new QComboBox(this);
00721     m_powerSmoothingCombo->addItem("Raw", 0);
00722     m_powerSmoothingCombo->addItem("3 sec", 3);
00723     m_powerSmoothingCombo->addItem("5 sec", 5);
00724     m_powerSmoothingCombo->addItem("10 sec", 10);
00725     m_powerSmoothingCombo->addItem("30 sec", 30);
00726     m_powerSmoothingCombo->addItem("60 sec", 60);
00727
00728     m_autoSmoothingCheck = new QCheckBox("Auto Smoothing", this);
00729     m_autoSmoothingCheck->setChecked(false);
00730
00731     // -- Chart height / fit controls -----
00732     m_fitChartsButton = new QPushButton("Reset Zoom");
00733     m_chartHeightIncreaseButton = new QPushButton("Increase Height");
00734     m_chartHeightDecreaseButton = new QPushButton("Decrease Height");
00735     m_useEstimatedFtpButton = new QPushButton("Use Estimated FTP");
00736     auto* editActivityButton = new QPushButton("Activity Properties");
00737     m_fitChartsButton->setFixedWidth(110);
00738     m_chartHeightIncreaseButton->setFixedWidth(130);
00739     m_chartHeightDecreaseButton->setFixedWidth(130);
00740     m_useEstimatedFtpButton->setFixedWidth(150);
00741     editActivityButton->setFixedWidth(150);
00742     m_fitChartsButton->setToolTip("Reset all chart zoom to full ride (Ctrl+0)");
00743     m_chartHeightIncreaseButton->setToolTip("Increase chart height");
00744     m_chartHeightDecreaseButton->setToolTip("Decrease chart height");
00745     m_useEstimatedFtpButton->setToolTip("Estimate FTP as 95% of best 20-minute power");
00746     m_useEstimatedFtpButton->setEnabled(false);
00747
00748     connect(m_fitChartsButton, &QPushButton::clicked,
00749             this, &MainWindow::resetAllZoom);
00750     connect(m_chartHeightIncreaseButton, &QPushButton::clicked,
00751             this, &MainWindow::increaseChartHeight);
00752     connect(m_chartHeightDecreaseButton, &QPushButton::clicked,
00753             this, &MainWindow::decreaseChartHeight);
00754     connect(editActivityButton, &QPushButton::clicked,
00755             this, &MainWindow::editCurrentActivityProperties);
00756     connect(m_useEstimatedFtpButton, &QPushButton::clicked,
00757             this, &MainWindow::applyEstimatedFtpForCurrentAthlete);

```

```

00758
00759     auto* ctrlBar = new QHBoxLayout;
00760     ctrlBar->setContentsMargins(0, 0, 0, 0);
00761     ctrlBar->addWidget(metricsBtn);
00762     ctrlBar->addSpacing(10);
00763     ctrlBar->addWidget(new QLabel("Preset:"));
00764
00765     m_chartPresetCombo = new QComboBox(this);
00766     m_chartPresetCombo->addItem("Custom", ChartPresetCustom);
00767     m_chartPresetCombo->addItem("Power Analysis", ChartPresetPowerAnalysis);
00768     m_chartPresetCombo->addItem("Heart Rate Focus", ChartPresetHeartRateFocus);
00769     m_chartPresetCombo->addItem("Race Review", ChartPresetRaceReview);
00770     m_chartPresetCombo->setCurrentIndex(0);
00771     ctrlBar->addWidget(m_chartPresetCombo);
00772
00773     ctrlBar->addSpacing(10);
00774     ctrlBar->addWidget(new QLabel("Power Smoothing:"));
00775     ctrlBar->addWidget(m_powerSmoothingCombo);
00776     ctrlBar->addWidget(m_autoSmoothingCheck);
00777     ctrlBar->addStretch(1);
00778     ctrlBar->addSpacing(8);
00779     ctrlBar->addWidget(m_fitChartsButton);
00780     ctrlBar->addWidget(m_chartHeightIncreaseButton);
00781     ctrlBar->addWidget(m_chartHeightDecreaseButton);
00782     ctrlBar->addWidget(m_useEstimatedFtpButton);
00783     ctrlBar->addWidget(editActivityButton);
00784
00785     // -- Charts -----
00786     m_powerChart = new RideChartWidget(Metric::Power);
00787     m_hrChart = new RideChartWidget(Metric::HeartRate);
00788     m_cadenceChart = new RideChartWidget(Metric::Cadence);
00789     m_speedChart = new RideChartWidget(Metric::Speed);
00790     m_altitudeChart = new RideChartWidget(Metric::Altitude);
00791
00792     connect(m_powerSmoothingCombo,
00793             qOverload<int>(&QComboBox::currentIndexChanged),
00794             this,
00795             [this](int)
00796     {
00797         if (!m_powerSmoothingCombo)
00798             return;
00799         applyChartSmoothing();
00800     });
00801
00802     connect(m_autoSmoothingCheck, &QCheckBox::toggled,
00803             this, [this](bool checked)
00804     {
00805         if (!m_powerSmoothingCombo)
00806             return;
00807         m_powerSmoothingCombo->setEnabled(!checked);
00808         applyChartSmoothing();
00809     });
00810
00811     connect(m_chartPresetCombo,
00812             qOverload<int>(&QComboBox::currentIndexChanged),
00813             this,
00814             [this](int)
00815     {
00816         if (!m_chartPresetCombo)
00817             return;
00818         applyChartPreset(m_chartPresetCombo->currentData().toInt());
00819     });
00820
00821     // Container inside scroll area.
00822     // setWidgetResizable(true) fills the viewport width automatically.
00823     // We set a minimumHeight so the vertical scrollbar appears when the
00824     // window is shorter than the total chart area.
00825     m_chartsContainer = new QWidget;
00826     m_chartsLayout = new QVBoxLayout(m_chartsContainer);
00827     m_chartsLayout->setSpacing(4);
00828     m_chartsLayout->setContentsMargins(0, 0, 0, 0);
00829     m_chartsLayout->addWidget(m_powerChart);
00830     m_chartsLayout->addWidget(m_hrChart);
00831     m_chartsLayout->addWidget(m_cadenceChart);
00832     m_chartsLayout->addWidget(m_speedChart);
00833     m_chartsLayout->addWidget(m_altitudeChart);
00834     m_chartsLayout->addStretch();
00835
00836     m_chartScroll = new QScrollArea;
00837     m_chartScroll->setWidget(m_chartsContainer);
00838     m_chartScroll->setWidgetResizable(true);
00839     m_chartScroll->setHorizontalScrollBarPolicy(Qt::ScrollBarAlwaysOff);
00840     m_chartScroll->setVerticalScrollBarPolicy(Qt::ScrollBarAlwaysOn);
00841     m_chartScroll->viewport()->installEventFilter(this);
00842
00843     // Tab container: controls bar + scroll area
00844     auto* chartsTab = new QWidget;

```

```

00845     auto* chartsVL = new QVBoxLayout(chartsTab);
00846     chartsVL->setContentsMargins(0, 0, 0, 0);
00847     chartsVL->setSpacing(4);
00848     chartsVL->addLayout(ctrlBar);
00849
00850     m_colorLegendWidget = new QWidget(chartsTab);
00851     m_colorLegendLayout = new QHBoxLayout(m_colorLegendWidget);
00852     m_colorLegendLayout->setContentsMargins(0, 0, 0, 0);
00853     m_colorLegendLayout->setSpacing(8);
00854     chartsVL->addWidget(m_colorLegendWidget);
00855     // Integrated analysis workspace: charts + map + intervals/laps/notes.
00856     m_mapRenderer = new MapRenderer;
00857
00858     auto* mapPanel = new QWidget(chartsTab);
00859     auto* mapVL = new QVBoxLayout(mapPanel);
00860     mapVL->setContentsMargins(0, 0, 0, 0);
00861     mapVL->setSpacing(4);
00862     auto* mapBtnBar = new QHBoxLayout;
00863     auto* mapFitBtn = new QPushButton("Fit to Track", mapPanel);
00864     auto* mapFitSelBtn = new QPushButton("Fit Map To Selection", mapPanel);
00865     auto* mapZoomIn = new QPushButton("+", mapPanel);
00866     auto* mapZoomOut = new QPushButton("-", mapPanel);
00867     m_mapStyleCombo = new QComboBox(mapPanel);
00868     m_mapStyleCombo->addItem("Light", static_cast<int>(MapStyle::Light));
00869     m_mapStyleCombo->addItem("Street Map", static_cast<int>(MapStyle::Street));
00870     m_mapStyleCombo->addItem("Satellite", static_cast<int>(MapStyle::Satellite));
00871     m_mapStyleCombo->addItem("Dark", static_cast<int>(MapStyle::Dark));
00872     m_mapStyleCombo->addItem("Terrain", static_cast<int>(MapStyle::Terrain));
00873     m_mapStyleCombo->addItem("Topographic", static_cast<int>(MapStyle::Topographic));
00874     m_mapStyleCombo->addItem("Minimal", static_cast<int>(MapStyle::Minimal));
00875     m_mapAutoContrastCheck = new QCheckBox("Auto route contrast", mapPanel);
00876     m_mapAutoContrastCheck->setChecked(true);
00877     mapZoomIn->setFixedWidth(32);
00878     mapZoomOut->setFixedWidth(32);
00879     connect(mapFitBtn, &QPushButton::clicked, m_mapRenderer, &MapRenderer::fitToTrack);
00880     connect(mapFitSelBtn, &QPushButton::clicked, m_mapRenderer, &MapRenderer::fitToVisibleRange);
00881     connect(mapZoomIn, &QPushButton::clicked, m_mapRenderer, &MapRenderer::zoomIn);
00882     connect(mapZoomOut, &QPushButton::clicked, m_mapRenderer, &MapRenderer::zoomOut);
00883     connect(m_mapStyleCombo, qOverload<int>(&QComboBox::currentIndexChanged), this, [this](int)
00884     {
00885         if (!m_mapRenderer || !m_mapStyleCombo)
00886             return;
00887         const MapStyle style = static_cast<MapStyle>(m_mapStyleCombo->currentData().toInt());
00888         m_mapRenderer->setMapStyle(style);
00889     });
00890     connect(m_mapAutoContrastCheck, &QCheckBox::toggled, this, [this](bool checked)
00891     {
00892         if (m_mapRenderer)
00893             m_mapRenderer->setAutoRouteContrast(checked);
00894     });
00895     mapBtnBar->addWidget(mapFitBtn);
00896     mapBtnBar->addWidget(mapFitSelBtn);
00897     mapBtnBar->addWidget(mapZoomIn);
00898     mapBtnBar->addWidget(mapZoomOut);
00899     mapBtnBar->addSpacing(8);
00900     mapBtnBar->addWidget(new QLabel("Map Style:", mapPanel));
00901     mapBtnBar->addWidget(m_mapStyleCombo);
00902     mapBtnBar->addWidget(m_mapAutoContrastCheck);
00903     mapBtnBar->addStretch(1);
00904     mapVL->addLayout(mapBtnBar);
00905     mapVL->addWidget(m_mapRenderer, 1);
00906
00907     m_intervalsTable = new QTableWidgetItem(0, 8, chartsTab);
00908     m_intervalsTable->setHorizontalHeaderLabels(
00909     { "Type", "Start", "Duration", "Avg W", "NP", "IF", "Max W", "Avg HR" });
00910     m_intervalsTable->horizontalHeader()->setStretchLastSection(true);
00911     m_intervalsTable->setEditTriggers(QAbstractItemView::NoEditTriggers);
00912     m_intervalsTable->setAlternatingRowColors(true);
00913     m_intervalsTable->setSelectionBehavior(QAbstractItemView::SelectRows);
00914     m_intervalsTable->setSelectionMode(QAbstractItemView::SingleSelection);
00915     m_intervalsTable->setStyleSheet(
00916     "QTableWidgetItem::item:selected { background-color: #bfdbfe; color: #0f172a; }");
00917     connect(m_intervalsTable, &QTableWidgetItem::itemSelectionChanged,
00918     this, &MainWindow::onIntervalSelectionChanged);
00919     connect(m_intervalsTable, &QTableWidgetItem::itemDoubleClicked,
00920     this, &MainWindow::onIntervalRowDoubleClicked);
00921
00922     m_lapsTable = new QTableWidgetItem(0, 3, chartsTab);
00923     m_lapsTable->setHorizontalHeaderLabels({ "Lap", "Start", "Duration" });
00924     m_lapsTable->horizontalHeader()->setStretchLastSection(true);
00925     m_lapsTable->setEditTriggers(QAbstractItemView::NoEditTriggers);
00926
00927     m_activityNotesView = new QTextEdit(chartsTab);
00928     m_activityNotesView->setReadOnly(true);
00929     m_activityNotesView->setPlaceholderText("Activity notes and conditions");
00930
00931     auto* intervalsPanel = new QWidget(chartsTab);

```

```

00932     auto* intervalsVL = new QVBoxLayout(intervalsPanel);
00933     intervalsVL->setContentsMargins(0, 0, 0, 0);
00934     intervalsVL->setSpacing(4);
00935     auto* intervalsLabel = new QLabel("Intervals", intervalsPanel);
00936     intervalsLabel->setStyleSheet("font-weight: 600;");
00937     intervalsVL->addWidget(intervalsLabel);
00938     auto* intervalsToolBar = new QHBoxLayout;
00939     auto* saveAutoBtn = new QPushButton("Save Auto Intervals", intervalsPanel);
00940     auto* addManualBtn = new QPushButton("Add Manual Interval", intervalsPanel);
00941     m_followIntervalSelectionCheck = new QCheckBox("Follow interval selection", intervalsPanel);
00942     m_followIntervalSelectionCheck->setChecked(true);
00943     intervalsToolBar->addWidget(saveAutoBtn);
00944     intervalsToolBar->addWidget(addManualBtn);
00945     intervalsToolBar->addWidget(m_followIntervalSelectionCheck);
00946     intervalsToolBar->addStretch(1);
00947     intervalsVL->addLayout(intervalsToolBar);
00948     intervalsVL->addWidget(m_intervalsTable, 1);
00949     m_intervalSummaryLabel = new QLabel("Interval summary: select an interval", intervalsPanel);
00950     m_intervalSummaryLabel->setStyleSheet("color: #334155;");
00951     intervalsVL->addWidget(m_intervalSummaryLabel);
00952
00953     auto* lapsPanel = new QWidget(chartsTab);
00954     auto* lapsVL = new QVBoxLayout(lapsPanel);
00955     lapsVL->setContentsMargins(0, 0, 0, 0);
00956     lapsVL->setSpacing(4);
00957     auto* lapsLabel = new QLabel("Laps", lapsPanel);
00958     lapsLabel->setStyleSheet("font-weight: 600;");
00959     lapsVL->addWidget(lapsLabel);
00960     lapsVL->addWidget(m_lapsTable, 1);
00961
00962     auto* notesPanel = new QWidget(chartsTab);
00963     auto* notesVL = new QVBoxLayout(notesPanel);
00964     notesVL->setContentsMargins(0, 0, 0, 0);
00965     notesVL->setSpacing(4);
00966     auto* notesLabel = new QLabel("Notes", notesPanel);
00967     notesLabel->setStyleSheet("font-weight: 600;");
00968     notesVL->addWidget(notesLabel);
00969     notesVL->addWidget(m_activityNotesView, 1);
00970
00971     auto* topSplit = new QSplitter(Qt::Horizontal, chartsTab);
00972     m_chartMapSplit = topSplit;
00973     topSplit->addWidget(m_chartScroll);
00974     topSplit->addWidget(mapPanel);
00975     topSplit->setStretchFactor(0, 1);
00976     topSplit->setStretchFactor(1, 1);
00977     topSplit->setCollapsible(0, false);
00978     topSplit->setCollapsible(1, false);
00979
00980     // Initialize 50/50 split after layout is complete and keep it balanced
00981     // when the available width changes.
00982     QTimer::singleShot(0, topSplit, [topSplit]()
00983     {
00984         const QList<int> sizes = topSplit->sizes();
00985         const int totalWidth = std::accumulate(sizes.begin(), sizes.end(), 0);
00986         if (totalWidth > 0)
00987             topSplit->setSizes({ totalWidth / 2, totalWidth / 2 });
00988     });
00989
00990     auto* bottomSplit = new QSplitter(Qt::Horizontal, chartsTab);
00991     bottomSplit->addWidget(intervalsPanel);
00992     bottomSplit->addWidget(lapsPanel);
00993     bottomSplit->addWidget(notesPanel);
00994     bottomSplit->setStretchFactor(0, 2);
00995     bottomSplit->setStretchFactor(1, 1);
00996     bottomSplit->setStretchFactor(2, 2);
00997
00998     auto* activitySplit = new QSplitter(Qt::Vertical, chartsTab);
00999     activitySplit->addWidget(topSplit);
01000     activitySplit->addWidget(bottomSplit);
01001     activitySplit->setStretchFactor(0, 3);
01002     activitySplit->setStretchFactor(1, 2);
01003
01004     chartsVL->addWidget(activitySplit, 1);
01005
01006     m_analysisTabWidget->addTab(
01007         wrapWithNoActivityState(chartsTab, &m_activityTabStack),
01008         "Activity");
01009
01010     // Checkbox <-> chart visibility; adjustSize keeps container height in sync
01011     auto makeVisToggle = [this](QCheckBox* cb, RideChartWidget* chart) {
01012         connect(cb, &QCheckBox::toggled, this, [this, chart](bool v) {
01013             chart->setVisible(v);
01014             if (m_chartsContainer) m_chartsContainer->adjustSize();
01015         });
01016     };
01017     makeVisToggle(m_showPower, m_powerChart);
01018     makeVisToggle(m_showHR, m_hrChart);

```



```

01019     makeVisToggle(m_showCadence, m_cadenceChart);
01020     makeVisToggle(m_showSpeed, m_speedChart);
01021     makeVisToggle(m_showAltitude, m_altitudeChart);
01022
01023     // X-zoom sync
01024     const std::initializer_list<RideChartWidget*> all =
01025     { m_powerChart, m_hrChart, m_cadenceChart, m_speedChart, m_altitudeChart };
01026     for (auto* src : all)
01027         for (auto* dst : all)
01028             if (src != dst)
01029                 connect(src, &RideChartWidget::xRangeChanged,
01030                         dst, &RideChartWidget::setXRange);
01031
01032     // Crosshair sync
01033     for (auto* src : all)
01034         for (auto* dst : all)
01035             if (src != dst)
01036                 connect(src, &RideChartWidget::crosshairMoved,
01037                         dst, &RideChartWidget::setCrosshair);
01038
01039     for (auto* src : all)
01040     {
01041         connect(src, &RideChartWidget::crosshairMoved,
01042                 this, [this](double xMinutes)
01043                 {
01044                     if (!m_mapRenderer)
01045                         return;
01046                     m_mapRenderer->setCurrentTime(xMinutes < 0.0 ? -1.0 : xMinutes * 60.0);
01047                 });
01048     }
01049
01050     connect(m_powerChart, &RideChartWidget::visibleRangeChanged,
01051            this, [this](double startMinutes, double endMinutes)
01052            {
01053                if (!m_mapRenderer)
01054                    return;
01055                m_mapRenderer->setVisibleTimeRange(startMinutes * 60.0, endMinutes * 60.0);
01056            });
01057
01058     auto syncChartsToMapSelection = [this](double startSeconds, double endSeconds)
01059     {
01060         const double startMinutes = std::min(startSeconds, endSeconds) / 60.0;
01061         const double endMinutes = std::max(startSeconds, endSeconds) / 60.0;
01062         m_powerChart->setXRange(startMinutes, endMinutes);
01063         m_hrChart->setXRange(startMinutes, endMinutes);
01064         m_cadenceChart->setXRange(startMinutes, endMinutes);
01065         m_speedChart->setXRange(startMinutes, endMinutes);
01066         m_altitudeChart->setXRange(startMinutes, endMinutes);
01067     };
01068
01069     connect(m_mapRenderer, &MapRenderer::segmentSelectionChanged,
01070            this, [syncChartsToMapSelection](double startSeconds, double endSeconds)
01071            {
01072                syncChartsToMapSelection(startSeconds, endSeconds);
01073            });
01074
01075     connect(m_mapRenderer, &MapRenderer::segmentSelectionFinished,
01076            this, [syncChartsToMapSelection](double startSeconds, double endSeconds)
01077            {
01078                syncChartsToMapSelection(startSeconds, endSeconds);
01079            });
01080
01081     connect(m_powerChart, &RideChartWidget::intervalSelectionRequested,
01082            this, [this](double startSec, double endSec)
01083            {
01084                if (!m_dbManager.isOpen() || m_controller->currentActivityId() <= 0)
01085                    return;
01086
01087                IntervalEditorDialog dlg(startSec, endSec, this);
01088                if (dlg.exec() != QDialog::Accepted)
01089                    return;
01090
01091                auto db = m_dbManager.database();
01092                IntervalRepository repo(db);
01093
01094                const auto& records = m_controller->rideData().records;
01095                double sumPower = 0.0;
01096                int countPower = 0;
01097                double sumHr = 0.0;
01098                int countHr = 0;
01099                double sumCadence = 0.0;
01100                int countCadence = 0;
01101                std::vector<double> powerValues;
01102                for (const RideRecord& r : records)
01103                {
01104                    if (r.elapsedSeconds < startSec || r.elapsedSeconds > endSec)
01105                        continue;

```



```

01106         if (r.hasPower)
01107         {
01108             sumPower += r.power;
01109             ++countPower;
01110             powerValues.push_back(r.power);
01111         }
01112         if (r.hasHeartRate)
01113         {
01114             sumHr += r.heartRate;
01115             ++countHr;
01116         }
01117         if (r.hasCadence)
01118         {
01119             sumCadence += r.cadence;
01120             ++countCadence;
01121         }
01122     }
01123
01124     double np = 0.0;
01125     if (!powerValues.empty())
01126     {
01127         double sum4 = 0.0;
01128         for (double p : powerValues)
01129             sum4 += std::pow(p, 4.0);
01130         np = std::pow(sum4 / static_cast<double>(powerValues.size()), 0.25);
01131     }
01132
01133     IntervalRecord record;
01134     record.activityId = m_controller->currentActivityId();
01135     record.startSample = static_cast<int>(std::round(startSec));
01136     record.endSample = static_cast<int>(std::round(endSec));
01137     record.name = dlg.intervalName().isEmpty() ? QStringLiteral("Manual Interval") :
01138     dlg.intervalName();
01139     record.type = dlg.intervalType().isEmpty() ? QStringLiteral("Manual") :
01140     dlg.intervalType();
01141     record.avgPower = countPower > 0 ? (sumPower / static_cast<double>(countPower)) : 0.0;
01142     record.np = np;
01143     record.avgHr = countHr > 0 ? (sumHr / static_cast<double>(countHr)) : 0.0;
01144     record.avgCadence = countCadence > 0 ? (sumCadence / static_cast<double>(countCadence)) :
01145     0.0;
01146     record.notes = dlg.intervalNotes();
01147     record.source = QStringLiteral("manual");
01148     record.locked = true;
01149     repo.insertInterval(record);
01150     updateIntervals();
01151     statusBar()->showMessage("Manual interval saved from chart selection.", 2200);
01152 });
01153
01154 connect(saveAutoBtn, &QPushButton::clicked, this, [this]
01155 {
01156     if (!m_dbManager.isOpen() || m_controller->currentActivityId() <= 0)
01157         return;
01158
01159     auto db = m_dbManager.database();
01160     IntervalRepository repo(db);
01161     repo.deleteIntervalsForActivity(m_controller->currentActivityId());
01162     for (const Interval& iv : m_controller->intervals())
01163     {
01164         IntervalRecord record;
01165         record.activityId = m_controller->currentActivityId();
01166         record.startSample = static_cast<int>(std::round(iv.startSeconds));
01167         record.endSample = static_cast<int>(std::round(iv.endSeconds));
01168         record.name = QString::fromStdString(iv.label);
01169         record.type = QString::fromStdString(iv.label);
01170         record.avgPower = iv.averagePower;
01171         record.np = iv.normalizedPower;
01172         record.avgHr = iv.averageHeartRate;
01173         record.avgCadence = iv.averageCadence;
01174         repo.insertInterval(record);
01175     }
01176     updateIntervals();
01177     statusBar()->showMessage("Auto intervals saved.", 2000);
01178 });
01179
01180 connect(addManualBtn, &QPushButton::clicked, this, [this]
01181 {
01182     if (!m_dbManager.isOpen() || m_controller->currentActivityId() <= 0)
01183         return;
01184     const auto& records = m_controller->rideData().records;
01185     if (records.size() < 2)
01186         return;
01187
01188     const double startSec = records[records.size() / 4].elapsedSeconds;
01189     const double endSec = records[records.size() / 4 + records.size() / 10].elapsedSeconds;
01190
01191     IntervalEditorDialog dlg(startSec, endSec, this);
01192     if (dlg.exec() != QDialog::Accepted)

```

```

01190         return;
01191
01192         auto db = m_dbManager.database();
01193         IntervalRepository repo(db);
01194         IntervalRecord record;
01195         record.activityId = m_controller->currentActivityId();
01196         record.startSample = static_cast<int>(std::round(startSec));
01197         record.endSample = static_cast<int>(std::round(endSec));
01198         record.name = dlg.intervalName();
01199         record.type = dlg.intervalType().isEmpty() ? QStringLiteral("Manual") :
dlg.intervalType();
01200         record.notes = dlg.intervalNotes();
01201         record.source = QStringLiteral("manual");
01202         record.locked = true;
01203         repo.insertInterval(record);
01204         updateIntervals();
01205         statusBar()->showMessage("Manual interval saved.", 2000);
01206     });
01207
01208 }
01209 // == Tab 1: Power Zones ==
01210 {
01211     m_zonesTable = new QTableWidget(0, 5);
01212     m_zonesTable->setHorizontalHeaderLabels(
01213         { "Zone", "Name", "Range", "Time in Zone", "%" });
01214     m_zonesTable->horizontalHeader()->setStretchLastSection(true);
01215     m_zonesTable->setEditTriggers(QAbstractItemView::NoEditTriggers);
01216     m_zonesTable->setAlternatingRowColors(true);
01217     m_zonesTable->setSelectionMode(QAbstractItemView::SingleSelection);
01218
01219     auto* w = new QWidget;
01220     auto* vl = new QVBoxLayout(w);
01221     vl->addWidget(m_zonesTable);
01222     m_analysisTabWidget->addTab(
01223         wrapWithNoActivityState(w, &m_zonesTabStack),
01224         "Zones");
01225 }
01226
01227 // == Tab 2: Power Histogram ==
01228 {
01229     m_histogram = new PowerHistogramWidget;
01230     m_analysisTabWidget->addTab(
01231         wrapWithNoActivityState(m_histogram, &m_histogramTabStack),
01232         "Histogram");
01233 }
01234
01235 // == Tab 3: Power Duration Curve ==
01236 {
01237     m_pdcWidget = new PowerCurveWidget;
01238     m_analysisTabWidget->addTab(
01239         wrapWithNoActivityState(m_pdcWidget, &m_powerCurveTabStack),
01240         "Power Curve");
01241 }
01242
01243 // == Tab 4: Calendar ==
01244 {
01245     m_calendarWidget = new CalendarWidget(this);
01246     m_calendarWidget->setDatabaseManager(&m_dbManager);
01247     m_calendarWidget->setAthleteId(m_currentAthleteId);
01248     m_analysisTabWidget->addTab(
01249         wrapWithNoActivityState(m_calendarWidget, &m_calendarTabStack),
01250         "Calendar");
01251 }
01252
01253 // == Tab 5: Fitness/Fatigue/Form ==
01254 {
01255     m_fitnessChart = new FitnessChartWidget(this);
01256     m_analysisTabWidget->addTab(
01257         wrapWithNoActivityState(m_fitnessChart, &m_fitnessTabStack),
01258         "Fitness");
01259 }
01260
01261 // == Tab 6: Climbing ==
01262 {
01263     auto* climbingTab = new QWidget(this);
01264     auto* climbingVL = new QVBoxLayout(climbingTab);
01265     climbingVL->setContentsMargins(0, 0, 0, 0);
01266     climbingVL->setSpacing(6);
01267
01268     m_climbAltitudeChart = new RideChartWidget(Metric::Altitude, climbingTab);
01269     m_climbPowerChart = new RideChartWidget(Metric::Power, climbingTab);
01270     m_climbHrChart = new RideChartWidget(Metric::HeartRate, climbingTab);
01271     m_climbCadenceChart = new RideChartWidget(Metric::Cadence, climbingTab);
01272     m_climbSpeedChart = new RideChartWidget(Metric::Speed, climbingTab);
01273     m_climbAltitudeChart->setClimbEditingEnabled(true);
01274     m_climbAltitudeChart->setMinimumHeight(320);
01275     m_climbAltitudeChart->setSizePolicy(QSizePolicy::Expanding, QSizePolicy::Expanding);

```

```

01276         m_climbPowerChart->hide();
01277         m_climbHrChart->hide();
01278         m_climbCadenceChart->hide();
01279         m_climbSpeedChart->hide();
01280
01281         auto* climbCharts = new QWidget(climbingTab);
01282         auto* climbChartsVL = new QVBoxLayout(climbCharts);
01283         climbChartsVL->setContentsMargins(0, 0, 0, 0);
01284         climbChartsVL->setSpacing(4);
01285         climbChartsVL->addWidget(m_climbAltitudeChart);
01286
01287         auto createQuarterChart = [climbingTab](const QString& title, const QColor& color)
01288         {
01289             auto* view = new QChartView(new QChart, climbingTab);
01290             view->setRenderHint(QPainter::Antialiasing);
01291             view->setMinimumHeight(160);
01292             view->setSizePolicy(QSizePolicy::Expanding, QSizePolicy::Preferred);
01293             view->chart()->setTitle(title);
01294             view->chart()->setAnimationOptions(QChart::NoAnimation);
01295             view->chart()->legend()->hide();
01296             view->chart()->setBackgroundVisible(false);
01297             view->setStyleSheet("background: #ffffff; border: 1px solid #e2e8f0; border-radius:
01298         6px;");
01299
01300             auto* set = new QBarSet(title);
01301             set->setColor(color);
01302             *set << 0.0 << 0.0 << 0.0 << 0.0;
01303
01304             auto* series = new QBarSeries(view->chart());
01305             series->append(set);
01306             view->chart()->addSeries(series);
01307
01308             auto* axisX = new QBarCategoryAxis(view->chart());
01309             axisX->append({ "Q1", "Q2", "Q3", "Q4" });
01310             view->chart()->addAxis(axisX, Qt::AlignBottom);
01311             series->attachAxis(axisX);
01312
01313             auto* axisY = new QValueAxis(view->chart());
01314             axisY->setRange(0.0, 1.0);
01315             view->chart()->addAxis(axisY, Qt::AlignLeft);
01316             series->attachAxis(axisY);
01317
01318             return view;
01319         };
01320
01321         auto* quarterCharts = new QWidget(climbingTab);
01322         auto* quarterChartsHL = new QHBoxLayout(quarterCharts);
01323         quarterChartsHL->setContentsMargins(0, 0, 0, 0);
01324         quarterChartsHL->setSpacing(6);
01325
01326         m_climbQuarterPowerChart = createQuarterChart(QStringLiteral("Power by Quarter"), QColor(37,
01327         99, 235));
01328         m_climbQuarterHrChart = createQuarterChart(QStringLiteral("Heart Rate by Quarter"),
01329         QColor(220, 38, 38));
01330         m_climbQuarterCadenceChart = createQuarterChart(QStringLiteral("Cadence by Quarter"),
01331         QColor(5, 150, 105));
01332
01333         quarterChartsHL->addWidget(m_climbQuarterPowerChart, 1);
01334         quarterChartsHL->addWidget(m_climbQuarterHrChart, 1);
01335         quarterChartsHL->addWidget(m_climbQuarterCadenceChart, 1);
01336         climbChartsVL->addWidget(quarterCharts);
01337
01338         m_climbMinLengthSpin = new QDoubleSpinBox(climbingTab);
01339         m_climbMinLengthSpin->setRange(0.1, 20.0);
01340         m_climbMinLengthSpin->setDecimals(2);
01341         m_climbMinLengthSpin->setSingleStep(0.1);
01342         m_climbMinLengthSpin->setValue(0.15);
01343
01344         m_climbMinGainSpin = new QDoubleSpinBox(climbingTab);
01345         m_climbMinGainSpin->setRange(1.0, 2000.0);
01346         m_climbMinGainSpin->setDecimals(0);
01347         m_climbMinGainSpin->setSingleStep(5.0);
01348         m_climbMinGainSpin->setValue(10.0);
01349
01350         m_climbMinGradientSpin = new QDoubleSpinBox(climbingTab);
01351         m_climbMinGradientSpin->setRange(0.5, 20.0);
01352         m_climbMinGradientSpin->setDecimals(1);
01353         m_climbMinGradientSpin->setSingleStep(0.5);
01354         m_climbMinGradientSpin->setValue(4.0);
01355
01356         m_climbStartGradientSpin = new QDoubleSpinBox(climbingTab);
01357         m_climbStartGradientSpin->setRange(0.5, 20.0);
01358         m_climbStartGradientSpin->setDecimals(1);
01359         m_climbStartGradientSpin->setSingleStep(0.5);
01360         m_climbStartGradientSpin->setValue(1.5);
01361
01362         m_climbDipMetersSpin = new QDoubleSpinBox(climbingTab);

```

```

01359     m_climbDipMetersSpin->setRange(0.0, 200.0);
01360     m_climbDipMetersSpin->setDecimals(1);
01361     m_climbDipMetersSpin->setSingleStep(1.0);
01362     m_climbDipMetersSpin->setValue(10.0);
01363
01364     m_climbDipDistanceSpin = new QDoubleSpinBox(climbingTab);
01365     m_climbDipDistanceSpin->setRange(10.0, 2000.0);
01366     m_climbDipDistanceSpin->setDecimals(0);
01367     m_climbDipDistanceSpin->setSingleStep(10.0);
01368     m_climbDipDistanceSpin->setValue(200.0);
01369
01370     m_climbSmoothingSpin = new QDoubleSpinBox(climbingTab);
01371     m_climbSmoothingSpin->setRange(1.0, 500.0);
01372     m_climbSmoothingSpin->setDecimals(0);
01373     m_climbSmoothingSpin->setSingleStep(5.0);
01374     m_climbSmoothingSpin->setValue(50.0);
01375
01376     m_climbOverlayEnabledCheck = new QCheckBox("Overlay", climbingTab);
01377     m_climbOverlayEnabledCheck->setChecked(false);
01378     m_climbOverlayEnabledCheck->setStyleSheet(
01379         "QCheckBox { color: #0f172a; font-weight: 600; }"
01380         "QCheckBox::indicator { width: 16px; height: 16px; border: 2px solid #334155; background:
01381         #ffffff; }"
01382         "QCheckBox::indicator:checked { background: #0f172a; border-color: #0f172a; }");
01383     m_climbOverlayMetricCombo = new QComboBox(climbingTab);
01384     m_climbOverlayMetricCombo->addItem("Power", static_cast<int>(ColorMetric::Power));
01385     m_climbOverlayMetricCombo->addItem("Heart Rate", static_cast<int>(ColorMetric::HeartRate));
01386     m_climbOverlayMetricCombo->addItem("Cadence", static_cast<int>(ColorMetric::Cadence));
01387     m_climbOverlayMetricCombo->addItem("Speed", static_cast<int>(ColorMetric::Speed));
01388     m_climbOverlayMetricCombo->setCurrentIndex(0);
01389     m_climbOverlayMetricCombo->setEnabled(false);
01390
01391     auto* climbParamsInline = new QWidget(analysisContainer);
01392     auto* climbParamsHL = new QHBoxLayout(climbParamsInline);
01393     climbParamsHL->setContentsMargins(0, 0, 0, 0);
01394     climbParamsHL->setSpacing(4);
01395
01396     auto addParam = [climbParamsHL](const QString& label, QDoubleSpinBox* spin)
01397     {
01398         auto* tag = new QLabel(label);
01399         tag->setStyleSheet("color: #334155;");
01400         spin->setButtonSymbols(QAbstractSpinBox::NoButtons);
01401         spin->setFixedWidth(62);
01402         climbParamsHL->addWidget(tag);
01403         climbParamsHL->addWidget(spin);
01404     };
01405
01406     climbParamsHL->addWidget(new QLabel("Climb:"));
01407     addParam("Len", m_climbMinLengthSpin);
01408     addParam("Gain", m_climbMinGainSpin);
01409     addParam("AvgGrad", m_climbMinGradientSpin);
01410     addParam("StartGrad", m_climbStartGradientSpin);
01411     addParam("DipM", m_climbDipMetersSpin);
01412     addParam("DipDist", m_climbDipDistanceSpin);
01413     addParam("Smooth", m_climbSmoothingSpin);
01414
01415     colorBar->insertSpacing(2, 12);
01416     colorBar->insertWidget(3, climbParamsInline);
01417
01418     m_climbsTable = new QTableWidget(0, 19, climbingTab);
01419     m_climbsTable->setHorizontalHeaderLabels(
01420         { "Name", "Length (km)", "Gain (m)", "Avg Gradient %", "Max Gradient %",
01421           "Duration", "Avg Power", "W/kg", "W/kg Rank", "NP", "Avg HR", "Avg
01422           Cadence", "Avg Speed", "VAM", "Fade %", "Decouple %", "Difficulty", "Category", "Shape" });
01423     m_climbsTable->horizontalHeader()->setStretchLastSection(true);
01424     m_climbsTable->setEditTriggers(QAbstractItemView::NoEditTriggers);
01425     m_climbsTable->setSelectionBehavior(QAbstractItemView::SelectRows);
01426     m_climbsTable->setSelectionMode(QAbstractItemView::ExtendedSelection);
01427     m_climbsTable->setAlternatingRowColors(true);
01428     m_climbsTable->setSortingEnabled(true);
01429     m_climbsTable->setSortItems(0, Qt::AscendingOrder);
01430     m_climbsTable->setContextMenuPolicy(Qt::CustomContextMenu);
01431
01432     m_climbSummaryLabel = new QLabel("Climb summary: select a climb", climbingTab);
01433     m_climbSummaryLabel->setStyleSheet("color: #334155;");
01434
01435     auto* climbOverlayBar = new QWidget(climbingTab);
01436     auto* climbOverlayHL = new QHBoxLayout(climbOverlayBar);
01437     climbOverlayHL->setContentsMargins(0, 0, 0, 0);
01438     climbOverlayHL->setSpacing(6);
01439     climbOverlayHL->addWidget(new QLabel("Chart Overlay:", climbOverlayBar));
01440     climbOverlayHL->addWidget(m_climbOverlayEnabledCheck);
01441     climbOverlayHL->addWidget(m_climbOverlayMetricCombo);
01442     climbOverlayHL->addStretch(1);
01443
01444     auto* climbsBottom = new QWidget(climbingTab);
01445     auto* climbsBottomVL = new QVBoxLayout(climbsBottom);

```

```

01444     climbsBottomVL->setContentsMargins(0, 0, 0, 0);
01445     climbsBottomVL->setSpacing(4);
01446     climbsBottomVL->addWidget(new QLabel("Recognized Climbs", climbsBottom));
01447     climbsBottomVL->addWidget(m_climbsTable, 1);
01448     climbsBottomVL->addWidget(m_climbSummaryLabel);
01449
01450     auto* climbingSplit = new QSplitter(Qt::Vertical, climbingTab);
01451     climbingSplit->addWidget(climbCharts);
01452     climbingSplit->addWidget(climbsBottom);
01453     climbingSplit->setStretchFactor(0, 3);
01454     climbingSplit->setStretchFactor(1, 2);
01455     climbingSplit->setCollapsible(0, false);
01456     climbingSplit->setCollapsible(1, false);
01457
01458     QTimer::singleShot(0, climbingSplit, [climbingSplit]()
01459     {
01460         const QList<int> sizes = climbingSplit->sizes();
01461         const int total = std::accumulate(sizes.begin(), sizes.end(), 0);
01462         if (total > 0)
01463             climbingSplit->setSizes({ (total * 60) / 100, (total * 40) / 100 });
01464     });
01465
01466     climbingVL->addWidget(climbOverlayBar);
01467     climbingVL->addWidget(climbingSplit, 1);
01468
01469     auto wireDetectionChange = [this](QDoubleSpinBox* spin)
01470     {
01471         connect(spin, qOverload<double>(&QDoubleSpinBox::valueChanged),
01472             this, [this](double)
01473             {
01474                 detectClimbsAndRefresh();
01475             });
01476     };
01477
01478     wireDetectionChange(m_climbMinLengthSpin);
01479     wireDetectionChange(m_climbMinGainSpin);
01480     wireDetectionChange(m_climbMinGradientSpin);
01481     wireDetectionChange(m_climbStartGradientSpin);
01482     wireDetectionChange(m_climbDipMetersSpin);
01483     wireDetectionChange(m_climbDipDistanceSpin);
01484     wireDetectionChange(m_climbSmoothingSpin);
01485
01486     connect(m_climbOverlayEnabledCheck, &QCheckBox::toggled, this, [this](bool checked)
01487     {
01488         if (!m_climbOverlayMetricCombo || !m_climbAltitudeChart)
01489             return;
01490         m_climbOverlayMetricCombo->setEnabled(checked);
01491         const ColorMetric metric =
01492             static_cast<ColorMetric>(m_climbOverlayMetricCombo->currentData().toInt());
01493         m_climbAltitudeChart->setMetricOverlay(metric, checked);
01494     });
01495     connect(m_climbOverlayMetricCombo, qOverload<int>(&QComboBox::currentIndexChanged), this,
01496         [this](int)
01497         {
01498             if (!m_climbOverlayEnabledCheck || !m_climbOverlayEnabledCheck->isChecked() ||
01499                 !m_climbAltitudeChart || !m_climbOverlayMetricCombo)
01500                 return;
01501             const ColorMetric metric =
01502                 static_cast<ColorMetric>(m_climbOverlayMetricCombo->currentData().toInt());
01503             m_climbAltitudeChart->setMetricOverlay(metric, true);
01504         });
01505
01506     connect(m_climbsTable, &QTableWidget::itemSelectionChanged,
01507         this, &MainWindow::onClimbSelectionChanged);
01508     connect(m_climbsTable, &QTableWidget::itemDoubleClicked,
01509         this, &MainWindow::onClimbRowDoubleClicked);
01510     connect(m_climbsTable, &QWidget::customContextMenuRequested,
01511         this, &MainWindow::onClimbTableContextMenuRequested);
01512     connect(m_climbAltitudeChart, &RideChartWidget::climbBoundaryEdited,
01513         this, &MainWindow::onClimbBoundaryEdited);
01514     connect(m_climbAltitudeChart, &RideChartWidget::newClimbRequested,
01515         this, &MainWindow::onNewClimbRequested);
01516     connect(m_climbAltitudeChart, &RideChartWidget::climbClicked,
01517         this, [this](int climbIndex)
01518         {
01519             if (!m_climbsTable || climbIndex < 0 || climbIndex >=
01520                 static_cast<int>(m_detectedClimbs.size()))
01521                 return;
01522             const Climbs& c = m_detectedClimbs[static_cast<size_t>(climbIndex)];
01523             for (int row = 0; row < m_climbsTable->rowCount(); ++row)
01524             {
01525                 auto* item = m_climbsTable->item(row, 0);
01526                 if (!item) continue;
01527                 if (std::abs(item->data(kClimbStartRole).toDouble() - c.startSeconds) < 0.75 &&
01528                     std::abs(item->data(kClimbEndRole).toDouble() - c.endSeconds) < 0.75)
01529                     {

```

```

01526         m_climbsTable->setCurrentCell(row, 0);
01527         m_climbsTable->scrollToItem(item);
01528         break;
01529     }
01530 }
01531 });
01532
01533     const std::initializer_list<RideChartWidget*> climbChartsAll =
01534     { m_climbAltitudeChart, m_climbPowerChart, m_climbHrChart, m_climbCadenceChart,
m_climbSpeedChart };
01535     for (auto* src : climbChartsAll)
01536         for (auto* dst : climbChartsAll)
01537             if (src != dst)
01538                 connect(src, &RideChartWidget::xRangeChanged, dst, &RideChartWidget::setXRange);
01539
01540     for (auto* src : climbChartsAll)
01541         for (auto* dst : climbChartsAll)
01542             if (src != dst)
01543                 connect(src, &RideChartWidget::crosshairMoved, dst,
&RideChartWidget::setCrosshair);
01544
01545     updateClimbQuarterCharts(nullptr);
01546
01547     m_analysisTabWidget->addTab(
01548         wrapWithNoActivityState(climbingTab, &m_climbingTabStack),
01549         "Climbing");
01550 }
01551
01552     updateAnalysisEmptyStates();
01553
01554     analysisLayout->addWidget(m_analysisTabWidget, 1);
01555     m_tabWidget->addTab(analysisContainer, "Analysis");
01556
01557     mainLayout->addWidget(m_tabWidget, 1);
01558     setCentralWidget(central);
01559
01560     buildToolBar();
01561     buildStatusBar();
01562
01563     connect(m_colorMetricCombo,
01564         qOverload<int>(&QComboBox::currentIndexChanged),
01565         this,
01566         [this](int)
01567         {
01568             updateColorLegend();
01569             updateZoneAvailability();
01570             updateCharts();
01571             updateZonesTab();
01572             if (m_mapRenderer)
01573                 m_mapRenderer->setRideData(m_controller->rideData(), currentColorMetric(),
buildColorContext());
01574         });
01575
01576     resize(1050, 860);
01577     setWindowTitle("Fitlyzer C++");
01578     setWindowIcon(QIcon(":/resources/icons/fitlyzer_logo.png"));
01579 }
01580
01581 void MainWindow::buildToolBar()
01582 {
01583     auto* tb = addToolBar("Main");
01584     tb->setMovable(false);
01585     tb->setToolButtonStyle(Qt::ToolButtonTextBesideIcon);
01586
01587     m_toolbarImportAct = new QAction("Import", this);
01588     m_toolbarImportAct->setIcon(style()->standardIcon(QStyle::SP_DialogOpenButton));
01589     m_toolbarImportAct->setEnabled(false);
01590     connect(m_toolbarImportAct, &QAction::triggered, this, &MainWindow::importActivities);
01591     tb->addAction(m_toolbarImportAct);
01592
01593     auto* athleteManageAct = new QAction("Athletes", this);
01594     athleteManageAct->setIcon(style()->standardIcon(QStyle::SP_FileDialogDetailedView));
01595     connect(athleteManageAct, &QAction::triggered, this, &MainWindow::manageAthletes);
01596     tb->addAction(athleteManageAct);
01597
01598     auto* calendarAct = new QAction("Calendar", this);
01599     calendarAct->setIcon(style()->standardIcon(QStyle::SP_FileDialogContentsView));
01600     connect(calendarAct, &QAction::triggered, this, [this]
01601     {
01602         if (!m_tabWidget || !m_analysisTabWidget)
01603             return;
01604         m_tabWidget->setCurrentIndex(kTabAnalysis);
01605         m_analysisTabWidget->setCurrentIndex(kAnalysisTabCalendar);
01606     });
01607     tb->addAction(calendarAct);
01608
01609     m_toolbarSearchAct = new QAction("Search", this);

```

```

01610     m_toolbarSearchAct->setIcon(style()->standardIcon(QStyle::SP_FileDialogContentsView));
01611     connect(m_toolbarSearchAct, &QAction::triggered, this, [this]
01612     {
01613         if (m_tabWidget)
01614             m_tabWidget->setCurrentIndex(kTabActivities);
01615         if (m_activityBrowser)
01616             m_activityBrowser->focusSearch();
01617     });
01618     tb->addAction(m_toolbarSearchAct);
01619
01620     tb->addSeparator();
01621     tb->addWidget(new QLabel("Current Athlete:"));
01622
01623     m_athleteCombo = new QComboBox(tb);
01624     m_athleteCombo->setMinimumWidth(220);
01625     connect(m_athleteCombo, &QComboBox::currentIndexChanged,
01626             this, &MainWindow::onAthleteSelectionChanged);
01627     tb->addWidget(m_athleteCombo);
01628
01629     tb->addSeparator();
01630     tb->addWidget(new QLabel(" "));
01631
01632     m_importStatusWidget = new ImportStatusWidget(tb);
01633     if (m_importProgressModel)
01634         m_importStatusWidget->setModel(m_importProgressModel);
01635     tb->addWidget(m_importStatusWidget);
01636 }
01637
01638 void MainWindow::buildStatusBar()
01639 {
01640     m_dbStatusLabel = new QLabel("Database: none");
01641     m_athleteStatusLabel = new QLabel("Athlete: none");
01642     m_activityCountLabel = new QLabel("Activities: 0");
01643
01644     statusBar()->addPermanentWidget(m_dbStatusLabel);
01645     statusBar()->addPermanentWidget(m_athleteStatusLabel);
01646     statusBar()->addPermanentWidget(m_activityCountLabel);
01647 }
01648
01649 // Keyboard shortcuts
01650
01651 void MainWindow::setupShortcuts()
01652 {
01653     // F11: toggle full screen
01654     auto* fsAct = new QAction(this);
01655     fsAct->setShortcut(Qt::Key_F11);
01656     connect(fsAct, &QAction::triggered, this, [this]{
01657         isFullScreen() ? showNormal() : showFullScreen();
01658     });
01659     addAction(fsAct);
01660
01661     // Ctrl+0: reset zoom on all charts
01662     auto* resetAct = new QAction(this);
01663     resetAct->setShortcut(QKeySequence("Ctrl+0"));
01664     connect(resetAct, &QAction::triggered, this, &MainWindow::resetAllZoom);
01665     addAction(resetAct);
01666
01667     // Ctrl+Z: undo
01668     auto* undoAct = new QAction(this);
01669     undoAct->setShortcut(QKeySequence::Undo);
01670     connect(undoAct, &QAction::triggered, m_undoManager, &UndoManager::undo);
01671     addAction(undoAct);
01672
01673     // Ctrl+Y / Ctrl+Shift+Z: redo
01674     auto* redoAct = new QAction(this);
01675     redoAct->setShortcut(QKeySequence::Redo);
01676     connect(redoAct, &QAction::triggered, m_undoManager, &UndoManager::redo);
01677     addAction(redoAct);
01678
01679     if (m_intervalsTable)
01680     {
01681         auto* prevIntervalAct = new QAction(m_intervalsTable);
01682         prevIntervalAct->setShortcut(Qt::Key_Up);
01683         prevIntervalAct->setShortcutContext(Qt::WidgetShortcut);
01684         connect(prevIntervalAct, &QAction::triggered, this, [this]
01685         {
01686             selectIntervalRowOffset(-1);
01687         });
01688         m_intervalsTable->addAction(prevIntervalAct);
01689
01690         auto* nextIntervalAct = new QAction(m_intervalsTable);
01691         nextIntervalAct->setShortcut(Qt::Key_Down);
01692         nextIntervalAct->setShortcutContext(Qt::WidgetShortcut);
01693         connect(nextIntervalAct, &QAction::triggered, this, [this]
01694         {
01695             selectIntervalRowOffset(1);
01696         });
01697     }

```

```

01697         m_intervalsTable->addAction(nextIntervalAct);
01698     }
01699 }
01700
01701 void MainWindow::resetAllZoom()
01702 {
01703     const std::initializer_list<RideChartWidget*> all =
01704         { m_powerChart, m_hrChart, m_cadenceChart, m_speedChart, m_altitudeChart };
01705     for (auto* c : all)
01706         c->fitToData();
01707 }
01708
01709 void MainWindow::increaseChartHeight()
01710 {
01711     m_chartHeight += 40;
01712     applyChartHeight();
01713 }
01714
01715 void MainWindow::decreaseChartHeight()
01716 {
01717     m_chartHeight = std::max(120, m_chartHeight - 40);
01718     applyChartHeight();
01719 }
01720
01721 void MainWindow::applyChartHeight()
01722 {
01723     const std::initializer_list<RideChartWidget*> all =
01724         { m_powerChart, m_hrChart, m_cadenceChart, m_speedChart, m_altitudeChart };
01725
01726     int visibleCount = 0;
01727     for (auto* c : all) {
01728         c->setFixedHeight(m_chartHeight);
01729         if (c->isVisible()) ++visibleCount;
01730         // resizeEvent on the chart will fire and update tick count automatically
01731     }
01732
01733     if (!m_chartsContainer) return;
01734
01735     // Compute the exact pixel height the container needs.
01736     const int spacing = m_chartsLayout ? m_chartsLayout->spacing() : 4;
01737     const int totalH = visibleCount * m_chartHeight
01738         + std::max(0, visibleCount - 1) * spacing;
01739     m_chartsContainer->setFixedHeight(std::max(totalH, 1));
01740
01741     // Don't set a fixed width - let the splitter handle horizontal sizing.
01742     // The charts will naturally fill their allocated space in the splitter.
01743     if (m_chartsContainer)
01744         m_chartsContainer->setMaximumWidth(QWIDGETSIZE_MAX);
01745 }
01746
01747 bool MainWindow::eventFilter(QObject* watched, QEvent* event)
01748 {
01749     // When the viewport is resized (including on first show), sync the
01750     // container width so charts fill the full horizontal space. Calling
01751     // applyChartHeight() also refreshes the fixed height at that point.
01752     if (m_chartScroll && watched == m_chartScroll->viewport()
01753         && event->type() == QEvent::Resize) {
01754         applyChartHeight();
01755     }
01756
01757     if (m_chartMapSplit && watched == m_chartMapSplit && event->type() == QEvent::Resize)
01758     {
01759         const QList<int> sizes = m_chartMapSplit->sizes();
01760         if (sizes.size() == 2 && std::abs(sizes[0] - sizes[1]) > 1)
01761         {
01762             const int totalWidth = sizes[0] + sizes[1];
01763             if (totalWidth > 0)
01764                 m_chartMapSplit->setSizes({ totalWidth / 2, totalWidth / 2 });
01765         }
01766     }
01767
01768     return QMainWindow::eventFilter(watched, event);
01769 }
01770
01771 void MainWindow::dragEnterEvent(QDragEnterEvent* event)
01772 {
01773     if (!event)
01774         return;
01775
01776     if (!fitFilesFromMimeData(event->mimeType()).isEmpty())
01777         event->acceptProposedAction();
01778 }
01779
01780 void MainWindow::dropEvent(QDropEvent* event)
01781 {
01782     if (!event)
01783         return;

```



```

01784
01785     const QStringList files = fitFilesFromMimeData(event->mimeType());
01786     if (files.isEmpty())
01787         return;
01788
01789     importFiles(files);
01790     event->acceptProposedAction();
01791 }
01792
01793 void MainWindow::configureFolderWatcher()
01794 {
01795     if (!m_watcher)
01796     {
01797         m_watcher = new QFileSystemWatcher(this);
01798         connect(m_watcher, &QFileSystemWatcher::directoryChanged,
01799             this, &MainWindow::onWatchDirectoryChanged);
01800     }
01801
01802     if (!m_watchRescanTimer)
01803     {
01804         m_watchRescanTimer = new QTimer(this);
01805         m_watchRescanTimer->setSingleShot(true);
01806         m_watchRescanTimer->setInterval(700);
01807         connect(m_watchRescanTimer, &QTimer::timeout, this, [this]
01808             {
01809                 scanWatchDirectories(false);
01810             });
01811     }
01812
01813     const QStringList current = m_watcher->directories();
01814     if (!current.isEmpty())
01815         m_watcher->removePaths(current);
01816
01817     m_knownWatchedFitFiles.clear();
01818     if (!m_watchFolderEnabled)
01819         return;
01820
01821     const QStringList dirs = monitoredDirectories();
01822     QStringList validDirs;
01823     for (const QString& dirPath : dirs)
01824     {
01825         const QFileInfo info(dirPath);
01826         if (info.exists() && info.isDir())
01827             validDirs << info.absoluteFilePath();
01828     }
01829
01830     validDirs.removeDuplicates();
01831     if (!validDirs.isEmpty())
01832     {
01833         m_watcher->addPaths(validDirs);
01834         scanWatchDirectories(true);
01835     }
01836 }
01837
01838 QStringList MainWindow::monitoredDirectories() const
01839 {
01840     QStringList dirs;
01841     const QString downloads = QStandardPaths::writableLocation(QStandardPaths::DownloadLocation);
01842     const QString primary = m_watchFolderPath.trimmed().isEmpty() ? downloads : m_watchFolderPath;
01843     if (!primary.isEmpty())
01844         dirs << QDir::cleanPath(primary);
01845
01846     const QString home = QDir::homePath();
01847     const QStringList candidates = {
01848         QDir(home).filePath("Downloads/Garmin"),
01849         QDir(home).filePath("Downloads/Garmin Export"),
01850         QDir(home).filePath("Garmin/Exports")
01851     };
01852     for (const QString& candidate : candidates)
01853     {
01854         if (QFileInfo::exists(candidate))
01855             dirs << QDir::cleanPath(candidate);
01856     }
01857
01858     dirs.removeDuplicates();
01859     return dirs;
01860 }
01861
01862 void MainWindow::scanWatchDirectories(bool initialScan)
01863 {
01864     if (!m_watchFolderEnabled)
01865         return;
01866
01867     const QStringList dirs = monitoredDirectories();
01868     QSet<QString> currentSet;
01869
01870     for (const QString& dirPath : dirs)

```

```

01871     {
01872         QDirIterator it(
01873             dirPath,
01874             QStringList() « "*.fit" « "*.FIT",
01875             QDir::Files,
01876             QDirIterator::Subdirectories);
01877         while (it.hasNext())
01878             currentSet.insert(QDir::cleanPath(it.next()));
01879     }
01880
01881     if (initialScan)
01882     {
01883         m_knownWatchedFitFiles = currentSet;
01884         return;
01885     }
01886
01887     QStringList newFiles;
01888     for (const QString& path : currentSet)
01889     {
01890         if (!m_knownWatchedFitFiles.contains(path))
01891             newFiles « path;
01892     }
01893
01894     m_knownWatchedFitFiles = currentSet;
01895     if (newFiles.isEmpty())
01896         return;
01897
01898     std::sort(newFiles.begin(), newFiles.end());
01899     importFilesInternal(newFiles, false, QStringLiteral("Folder watcher"));
01900 }
01901
01902 void MainWindow::onWatchDirectoryChanged(const QString& path)
01903 {
01904     (void)path;
01905     if (!m_watchRescanTimer)
01906         return;
01907     m_watchRescanTimer->start();
01908 }
01909
01910 // Settings
01911
01912 void MainWindow::saveSettings()
01913 {
01914     QSettings s("Fitlyzer", "FitlyzerC");
01915     s.setValue("geometry", saveGeometry());
01916     s.setValue("activeTab", m_tabWidget->currentIndex());
01917     if (m_analysisTabWidget)
01918         s.setValue("analysisTab", m_analysisTabWidget->currentIndex());
01919     if (m_colorMetricCombo)
01920         s.setValue("colorMetric", m_colorMetricCombo->currentData().toInt());
01921     if (m_powerSmoothingCombo)
01922         s.setValue("powerSmoothing", m_powerSmoothingCombo->currentData().toInt());
01923     if (m_chartPresetCombo)
01924         s.setValue("chartPreset", m_chartPresetCombo->currentData().toInt());
01925     if (m_autoSmoothingCheck)
01926         s.setValue("autoSmoothing", m_autoSmoothingCheck->isChecked());
01927     if (m_followIntervalSelectionCheck)
01928         s.setValue("followIntervalSelection", m_followIntervalSelectionCheck->isChecked());
01929     if (m_mapStyleCombo)
01930         s.setValue("mapStyle", m_mapStyleCombo->currentData().toInt());
01931     if (m_mapAutoContrastCheck)
01932         s.setValue("mapAutoRouteContrast", m_mapAutoContrastCheck->isChecked());
01933     s.setValue("lastOpenDir", m_lastOpenDir);
01934     s.setValue("showPower", m_showPower->isChecked());
01935     s.setValue("showHR", m_showHR->isChecked());
01936     s.setValue("showCadence", m_showCadence->isChecked());
01937     s.setValue("showSpeed", m_showSpeed->isChecked());
01938     s.setValue("showAltitude", m_showAltitude->isChecked());
01939     s.setValue("watchFolderEnabled", m_watchFolderEnabled);
01940     s.setValue("watchFolderPath", m_watchFolderPath);
01941     s.setValue("chartHeight", m_chartHeight);
01942     s.setValue("currentAthleteId", m_currentAthleteId);
01943     s.setValue("lastActivityId", m_controller ? m_controller->currentActivityId() : -1);
01944     if (m_dbManager.isOpen())
01945         s.setValue("lastDatabase", m_dbManager.path());
01946 }
01947
01948 void MainWindow::loadSettings()
01949 {
01950     QSettings s("Fitlyzer", "FitlyzerC");
01951
01952     if (s.contains("geometry"))
01953         restoreGeometry(s.value("geometry").toByteArray());
01954
01955     m_lastOpenDir = s.value("lastOpenDir", QString()).toString();
01956
01957     m_showPower->setChecked(s.value("showPower", true).toBool());

```

```

01958     m_showHR->setChecked( s.value("showHR", true).toBool());
01959     m_showCadence->setChecked(s.value("showCadence",true).toBool());
01960     m_showSpeed->setChecked( s.value("showSpeed", true).toBool());
01961     m_showAltitude->setChecked(s.value("showAltitude", true).toBool());
01962     m_watchFolderEnabled = s.value("watchFolderEnabled", true).toBool();
01963     m_watchFolderPath = s.value(
01964         "watchFolderPath",
01965         QStandardPaths::writableLocation(QStandardPaths::DownloadLocation)).toString();
01966
01967     m_chartHeight = std::clamp(s.value("chartHeight", 220).toInt(), 120, 600);
01968
01969     if (m_powerSmoothingCombo)
01970     {
01971         const int smoothing = s.value("powerSmoothing", 0).toInt();
01972         const int comboIndex = m_powerSmoothingCombo->findData(smoothing);
01973         if (comboIndex >= 0)
01974             m_powerSmoothingCombo->setCurrentIndex(comboIndex);
01975     }
01976     if (m_autoSmoothingCheck)
01977     {
01978         const bool autoSmoothing = s.value("autoSmoothing", false).toBool();
01979         m_autoSmoothingCheck->setChecked(autoSmoothing);
01980         if (m_powerSmoothingCombo)
01981             m_powerSmoothingCombo->setEnabled(!autoSmoothing);
01982     }
01983     if (m_followIntervalSelectionCheck)
01984     {
01985         m_followIntervalSelectionCheck->setChecked(
01986             s.value("followIntervalSelection", true).toBool());
01987     }
01988     if (m_mapStyleCombo)
01989     {
01990         const int styleValue = s.value("mapStyle", static_cast<int>(MapStyle::Light)).toInt();
01991         const int comboIndex = m_mapStyleCombo->findData(styleValue);
01992         if (comboIndex >= 0)
01993             m_mapStyleCombo->setCurrentIndex(comboIndex);
01994     }
01995     if (m_mapAutoContrastCheck)
01996     {
01997         m_mapAutoContrastCheck->setChecked(s.value("mapAutoRouteContrast", true).toBool());
01998     }
01999
02000     if (m_chartPresetCombo)
02001     {
02002         const int presetValue = s.value("chartPreset", ChartPresetCustom).toInt();
02003         const int comboIndex = m_chartPresetCombo->findData(presetValue);
02004         if (comboIndex >= 0)
02005             m_chartPresetCombo->setCurrentIndex(comboIndex);
02006     }
02007
02008     if (m_colorMetricCombo)
02009     {
02010         const int colorMetricValue = s.value("colorMetric",
static_cast<int>(ColorMetric::Power)).toInt();
02011         const int comboIndex = m_colorMetricCombo->findData(colorMetricValue);
02012         if (comboIndex >= 0)
02013             m_colorMetricCombo->setCurrentIndex(comboIndex);
02014     }
02015
02016     if (s.contains("activeTab"))
02017         m_tabWidget->setCurrentIndex(s.value("activeTab", 0).toInt());
02018     if (m_analysisTabWidget && s.contains("analysisTab"))
02019         m_analysisTabWidget->setCurrentIndex(s.value("analysisTab", 0).toInt());
02020 }
02021
02022 void MainWindow::addRecentDatabase(const QString& path)
02023 {
02024     QSettings s("Fitlyzer", "FitlyzerC");
02025     QStringList recent = s.value("recentDatabases").toStringList();
02026     recent.removeAll(path);
02027     recent.prepend(path);
02028     while (recent.size() > kMaxRecentDatabases)
02029         recent.removeLast();
02030     s.setValue("recentDatabases", recent);
02031 }
02032
02033 void MainWindow::updateRecentDatabaseMenu()
02034 {
02035     if (!m_recentDbMenu) return;
02036
02037     m_recentDbMenu->clear();
02038     QSettings s("Fitlyzer", "FitlyzerC");
02039     const QStringList recent = s.value("recentDatabases").toStringList();
02040
02041     for (const QString& path : recent)
02042     {
02043         auto* act = m_recentDbMenu->addAction(QFileInfo(path).fileName());

```

```

02044         act->setData(path);
02045         act->setStatusTip(path);
02046         connect(act, &QAction::triggered, this, &MainWindow::openRecentDatabase);
02047     }
02048
02049     if (recent.isEmpty())
02050     {
02051         auto* emptyAct = m_recentDbMenu->addAction("(none)");
02052         emptyAct->setEnabled(false);
02053     }
02054 }
02055
02056 void MainWindow::openDatabasePath(const QString& path)
02057 {
02058     QString err;
02059     if (!m_dbManager.open(path, &err))
02060     {
02061         QMessageBox::critical(this, "Database Error",
02062             "Could not open database:\n" + err);
02063         return;
02064     }
02065
02066     m_controller->setDatabaseManager(&m_dbManager);
02067     QSettings settings("Fitlyzer", "FitlyzerC");
02068     settings.setValue("lastDatabase", path);
02069     addRecentDatabase(path);
02070     updateRecentDatabaseMenu();
02071     if (m_analysisQueue)
02072         m_analysisQueue->setDatabasePath(path);
02073     if (m_importQueue)
02074         m_importQueue->setDatabasePath(path);
02075
02076     if (m_activityBrowser)
02077         m_activityBrowser->refresh(m_currentAthleteId);
02078
02079     if (m_calendarWidget)
02080     {
02081         m_calendarWidget->setDatabaseManager(&m_dbManager);
02082         m_calendarWidget->setAthleteId(m_currentAthleteId);
02083     }
02084
02085     refreshAthleteSelector();
02086     updateAthleteLabel();
02087     updateImportAvailability();
02088     updateStatsLabel();
02089     updateStatusBarInfo();
02090     updateWelcomeScreenVisibility();
02091     statusBar()->showMessage("Database opened.", 2500);
02092 }
02093
02094 bool MainWindow::canImportActivities() const
02095 {
02096     return m_dbManager.isOpen() && m_currentAthleteId > 0;
02097 }
02098
02099 bool MainWindow::createOrOpenDefaultDatabase()
02100 {
02101     const QString path = defaultDatabasePathForAutoCreate();
02102     if (path.isEmpty())
02103         return false;
02104
02105     const QFileInfo info(path);
02106     QDir dir = info.dir();
02107     if (!dir.exists() && !dir.mkpath("."))
02108     {
02109         QMessageBox::critical(this, "Database Error",
02110             "Could not create default database folder:\n" + dir.absolutePath());
02111         return false;
02112     }
02113
02114     QString err;
02115     const bool ok = info.exists()
02116         ? m_dbManager.open(path, &err)
02117         : m_dbManager.create(path, &err);
02118     if (!ok)
02119     {
02120         QMessageBox::critical(this, "Database Error",
02121             QString("Could not open default database:\n%1\n%2").arg(path, err));
02122         return false;
02123     }
02124
02125     m_controller->setDatabaseManager(&m_dbManager);
02126     m_lastOpenDir = info.absolutePath();
02127     if (m_analysisQueue)
02128         m_analysisQueue->setDatabasePath(path);
02129     if (m_importQueue)
02130         m_importQueue->setDatabasePath(path);

```

```

02131
02132     QSettings settings("Fitlyzer", "FitlyzerC");
02133     settings.setValue("lastDatabase", path);
02134     addRecentDatabase(path);
02135     updateRecentDatabaseMenu();
02136     return true;
02137 }
02138
02139 bool MainWindow::ensureDatabase()
02140 {
02141     if (m_dbManager.isOpen())
02142         return true;
02143
02144     if (!createOrOpenDefaultDatabase())
02145         return false;
02146
02147     refreshAthleteSelector();
02148     updateAthleteLabel();
02149     updateImportAvailability();
02150     updateStatsLabel();
02151     updateStatusBarInfo();
02152     updateWelcomeScreenVisibility();
02153
02154     statusBar()->showMessage("Created default database for import.", 3500);
02155     return true;
02156 }
02157
02158 bool MainWindow::ensureAthlete()
02159 {
02160     if (!m_dbManager.isOpen())
02161         return false;
02162
02163     auto db = m_dbManager.database();
02164     AthleteRepository repo(db);
02165     QList<Athlete> athletes = repo.listAthletes();
02166
02167     bool createdDefaultAthlete = false;
02168     if (athletes.isEmpty())
02169     {
02170         Athlete athlete;
02171         athlete.firstName = "Default";
02172         athlete.lastName = "Athlete";
02173         const int createdId = repo.insertAthlete(athlete);
02174         if (createdId <= 0)
02175         {
02176             QMessageBox::critical(this, "Athlete Error",
02177                 "Could not create a default athlete for import.");
02178             return false;
02179         }
02180         athletes = repo.listAthletes();
02181         createdDefaultAthlete = true;
02182     }
02183
02184     bool hasCurrent = false;
02185     for (const Athlete& athlete : athletes)
02186     {
02187         if (athlete.id == m_currentAthleteId)
02188         {
02189             hasCurrent = true;
02190             break;
02191         }
02192     }
02193
02194     if (!hasCurrent)
02195     {
02196         m_currentAthleteId = athletes.first().id;
02197         m_currentAthleteName = athletes.first().fullName();
02198         m_controller->setCurrentAthlete(m_currentAthleteId);
02199         QSettings("Fitlyzer", "FitlyzerC").setValue("currentAthleteId", m_currentAthleteId);
02200     }
02201
02202     refreshAthleteSelector();
02203     updateAthleteLabel();
02204     updateImportAvailability();
02205     updateStatsLabel();
02206     updateStatusBarInfo();
02207
02208     if (createdDefaultAthlete)
02209     {
02210         const auto ans = QMessageBox::question(
02211             this,
02212             "Default Athlete Created",
02213             "A default athlete was created. Rename athlete now?",
02214             QMessageBox::Yes | QMessageBox::No,
02215             QMessageBox::Yes);
02216
02217         if (ans == QMessageBox::Yes)

```

```

02218     {
02219         AthleteDialog dlg(repo, m_currentAthleteId, this);
02220         if (dlg.exec() == QDialog::Accepted)
02221         {
02222             const Athlete athlete = repo.getAthlete(m_currentAthleteId);
02223             m_currentAthleteName = athlete.fullName();
02224             refreshAthleteSelector();
02225             updateAthleteLabel();
02226             updateStatusBarInfo();
02227         }
02228     }
02229 }
02230
02231 return m_currentAthleteId > 0;
02232 }
02233
02234 bool MainWindow::ensureImportReady()
02235 {
02236     return ensureDatabase() && ensureAthlete();
02237 }
02238
02239 QString MainWindow::defaultDatabasePathForAutoCreate() const
02240 {
02241     const QString documentsDir = QStandardPaths::writableLocation(QStandardPaths::DocumentsLocation);
02242     const QString baseDir = documentsDir.isEmpty()
02243         ? Platform::defaultDatabaseDirectory()
02244         : documentsDir;
02245     return QDir(baseDir).filePath("Fitlyzer/default.fitlyzerdb");
02246 }
02247
02248 void MainWindow::updateImportAvailability()
02249 {
02250     const bool enabled = canImportActivities();
02251     if (m_importAct)
02252         m_importAct->setEnabled(enabled);
02253     if (m_toolbarImportAct)
02254         m_toolbarImportAct->setEnabled(enabled);
02255
02256     if (enabled)
02257     {
02258         statusBar()->showMessage("Ready to import activities.", 2000);
02259     }
02260     else if (!m_dbManager.isOpen())
02261     {
02262         statusBar()->showMessage("Open or create a database to import activities.", 3000);
02263     }
02264     else
02265     {
02266         statusBar()->showMessage("Select an athlete to import activities.", 3000);
02267     }
02268 }
02269
02270 void MainWindow::showWelcomeScreen()
02271 {
02272     if (m_welcomeWidget)
02273         m_welcomeWidget->setVisible(true);
02274     if (m_tabWidget)
02275         m_tabWidget->setVisible(false);
02276 }
02277
02278 void MainWindow::hideWelcomeScreen()
02279 {
02280     if (m_welcomeWidget)
02281         m_welcomeWidget->setVisible(false);
02282     if (m_tabWidget)
02283         m_tabWidget->setVisible(true);
02284 }
02285
02286 int MainWindow::activityCount() const
02287 {
02288     if (!m_dbManager.isOpen())
02289         return 0;
02290
02291     auto db = m_dbManager.database();
02292     ActivityRepository repo(db);
02293     return repo.listActivities(-1).size();
02294 }
02295
02296 void MainWindow::updateWelcomeScreenVisibility()
02297 {
02298     if (!m_welcomeWidget)
02299         return;
02300
02301     m_welcomeWidget->setFirstLaunch(!m_firstLaunchCompleted);
02302     if (activityCount() == 0)
02303         showWelcomeScreen();
02304     else

```

```

02305         hideWelcomeScreen();
02306     }
02307
02308     ColorMetric MainWindow::currentColorMetric() const
02309     {
02310         if (!m_colorMetricCombo)
02311             return ColorMetric::None;
02312         return static_cast<ColorMetric>(m_colorMetricCombo->currentData().toInt());
02313     }
02314
02315     ColorContext MainWindow::buildColorContext() const
02316     {
02317         ColorContext context;
02318         context.ftp = static_cast<int>(m_controller->ftp());
02319
02320         if (m_dbManager.isOpen() && m_currentAthleteId > 0)
02321         {
02322             auto db = m_dbManager.database();
02323             AthleteRepository repo(db);
02324
02325             int resolvedFtp = 0;
02326             if (m_controller->currentActivityId() > 0)
02327             {
02328                 ActivityRepository actRepo(db);
02329                 const Activity activity = actRepo.getActivity(m_controller->currentActivityId());
02330                 const QDate ftpDate = activityDateForFtpContext(activity);
02331                 if (ftpDate.isValid())
02332                     resolvedFtp = repo.getFtpForDate(m_currentAthleteId, ftpDate);
02333             }
02334
02335             if (resolvedFtp <= 0)
02336                 resolvedFtp = repo.getAthlete(m_currentAthleteId).ftpWatts;
02337
02338             if (resolvedFtp > 0)
02339                 context.ftp = resolvedFtp;
02340         }
02341
02342         int maxHeartRate = 0;
02343         double minAltitude = std::numeric_limits<double>::max();
02344         double maxAltitude = std::numeric_limits<double>::lowest();
02345         bool hasAltitude = false;
02346
02347         for (const RideRecord& record : m_controller->rideData().records)
02348         {
02349             if (record.hasHeartRate)
02350                 maxHeartRate = std::max(maxHeartRate, static_cast<int>(record.heartRate));
02351             if (record.hasAltitude)
02352             {
02353                 minAltitude = std::min(minAltitude, record.altitude);
02354                 maxAltitude = std::max(maxAltitude, record.altitude);
02355                 hasAltitude = true;
02356             }
02357         }
02358
02359         if (maxHeartRate > 0)
02360             context.heartRateMax = maxHeartRate;
02361         if (hasAltitude)
02362         {
02363             context.altitudeMin = minAltitude;
02364             context.altitudeMax = maxAltitude;
02365             context.hasAltitudeRange = true;
02366         }
02367         return context;
02368     }
02369
02370 void MainWindow::updateColorLegend()
02371 {
02372     if (!m_colorLegendLayout || !m_colorLegendWidget)
02373         return;
02374
02375     while (QLayoutItem* item = m_colorLegendLayout->takeAt(0))
02376     {
02377         if (QWidget* widget = item->widget())
02378             widget->deleteLater();
02379         delete item;
02380     }
02381
02382     const ColorMetric metric = currentColorMetric();
02383     if (metric == ColorMetric::None)
02384     {
02385         auto* label = new QLabel("Coloring disabled", m_colorLegendWidget);
02386         label->setStyleSheet("color: #64748b;");
02387         m_colorLegendLayout->addWidget(label);
02388         m_colorLegendLayout->addStretch(1);
02389         return;
02390     }
02391

```

```

02392     }
02393
02394     const std::vector<Zone> zones = ZoneCalculator::zonesForMetric(metric, buildColorContext());
02395     for (const Zone& zone : zones)
02396     {
02397         auto* swatch = new QLabel(m_colorLegendWidget);
02398         swatch->setFixedSize(12, 12);
02399         swatch->setStyleSheet(QString("background: %1; border: 1px solid rgba(15,23,42,0.15);")
02400             .arg(zone.color.name()));
02401
02402         auto* text = new QLabel(zone.name, m_colorLegendWidget);
02403
02404         auto* chip = new QWidget(m_colorLegendWidget);
02405         auto* chipLayout = new QHBoxLayout(chip);
02406         chipLayout->setContentsMargins(0, 0, 0, 0);
02407         chipLayout->setSpacing(4);
02408         chipLayout->addWidget(swatch);
02409         chipLayout->addWidget(text);
02410         m_colorLegendLayout->addWidget(chip);
02411     }
02412
02413     m_colorLegendLayout->addStretch(1);
02414 }
02415
02416 void MainWindow::updateZoneAvailability()
02417 {
02418     if (!m_analysisTabWidget)
02419         return;
02420
02421     const ColorMetric metric = currentColorMetric();
02422     const bool hasSelectedMetric = ZoneCalculator::hasMetricSamples(m_controller->rideData(), metric);
02423     m_analysisTabWidget->setTabEnabled(kAnalysisTabZones,
02424         metric != ColorMetric::None && hasSelectedMetric);
02425 }
02426
02427 void MainWindow::importFiles(const QStringList& filePaths)
02428 {
02429     importFilesInternal(filePaths, true, QStringLiteral("Manual import"));
02430 }
02431
02432 void MainWindow::importFilesInternal(const QStringList& filePaths,
02433     bool showResultDialog,
02434     const QString& sourceLabel)
02435 {
02436     QStringList deduped = filePaths;
02437     deduped.removeDuplicates();
02438     if (deduped.isEmpty())
02439         return;
02440
02441     if (!ensureImportReady())
02442     {
02443         updateImportAvailability();
02444         return;
02445     }
02446
02447     if (!m_importQueue)
02448         return;
02449
02450     const QString batchId = QUuid::createUuid().toString(QUuid::WithoutBraces);
02451     ImportBatchSummary summary;
02452     summary.queued = deduped.size();
02453     summary.showResultDialog = showResultDialog;
02454     summary.sourceLabel = sourceLabel;
02455     m_importBatchSummaries.insert(batchId, summary);
02456
02457     const QString importSource = showResultDialog
02458         ? QStringLiteral("manual_import")
02459         : QStringLiteral("directory_import");
02460
02461     for (const QString& filePath : deduped)
02462     {
02463         m_importQueue->enqueue(filePath,
02464             QString::number(m_currentAthleteId),
02465             batchId,
02466             importSource);
02467     }
02468
02469     const QString submitMessage = QString("Queued %1 FIT file%2 for background import.")
02470         .arg(deduped.size())
02471         .arg(deduped.size() == 1 ? QString() : QStringLiteral("s"));
02472     statusBar()->showMessage(submitMessage, 2500);
02473 }
02474
02475 void MainWindow::scheduleActivityBrowserRefresh()
02476 {
02477     if (m_importRefreshTimer)
02478         m_importRefreshTimer->start();

```



```

02479 }
02480
02481 void MainWindow::finalizeImportBatches()
02482 {
02483     if (m_importBatchSummaries.isEmpty())
02484         return;
02485
02486     for (auto it = m_importBatchSummaries.begin(); it != m_importBatchSummaries.end(); ++it)
02487     {
02488         const ImportBatchSummary& summary = it.value();
02489         const QString label = summary.sourceLabel.isEmpty()
02490             ? QStringLiteral("Import")
02491             : summary.sourceLabel;
02492
02493         if (summary.showResultDialog)
02494         {
02495             QString text = QString("%1 files selected\n\nImported: %2\nDuplicates: %3\nFailed: %4")
02496                 .arg(summary.queued)
02497                 .arg(summary.imported)
02498                 .arg(summary.duplicates)
02499                 .arg(summary.failed);
02500
02501             if (!summary.errors.isEmpty())
02502                 text += QString("\n\nErrors:\n%1").arg(summary.errors.join('\n'));
02503
02504             QMessageBox::information(this, "Import Results", text);
02505         }
02506         else
02507         {
02508             statusBar()->showMessage(
02509                 QString("%1: imported %2, duplicates %3, failed %4")
02510                     .arg(label)
02511                     .arg(summary.imported)
02512                     .arg(summary.duplicates)
02513                     .arg(summary.failed),
02514                 5000);
02515         }
02516     }
02517
02518     m_importBatchSummaries.clear();
02519 }
02520
02521 QStringList MainWindow::fitFilesFromMimeData(const QMimeData* mimeData) const
02522 {
02523     QStringList files;
02524     if (!mimeData || !mimeData->hasUrls())
02525         return files;
02526
02527     for (const QUrl& url : mimeData->urls())
02528     {
02529         if (!url.isLocalFile())
02530             continue;
02531         const QString path = url.toLocalFile();
02532         if (QFileInfo(path).suffix().compare("fit", Qt::CaseInsensitive) == 0)
02533             files « path;
02534     }
02535
02536     files.removeDuplicates();
02537     return files;
02538 }
02539
02540 void MainWindow::refreshAthleteSelector()
02541 {
02542     if (!m_athleteCombo)
02543         return;
02544
02545     const QSignalBlocker blocker(m_athleteCombo);
02546     m_athleteCombo->clear();
02547     m_athleteCombo->addItem("Select athlete...", -1);
02548
02549     if (m_dbManager.isOpen())
02550     {
02551         auto db = m_dbManager.database();
02552         AthleteRepository repo(db);
02553         for (const Athlete& athlete : repo.listAthletes())
02554             m_athleteCombo->addItem(athlete.fullName(), athlete.id);
02555     }
02556
02557     const int idx = m_athleteCombo->findData(m_currentAthleteId);
02558     if (idx >= 0)
02559     {
02560         m_athleteCombo->setCurrentIndex(idx);
02561     }
02562     else
02563     {
02564         m_athleteCombo->setCurrentIndex(0);
02565         m_currentAthleteId = -1;

```

```

02566         m_currentAthleteName.clear();
02567         m_controller->setCurrentAthlete(-1);
02568     }
02569 }
02570
02571 void MainWindow::updateStatusBarInfo()
02572 {
02573     if (m_dbStatusLabel)
02574         m_dbStatusLabel->setText(
02575             m_dbManager.isOpen()
02576             ? QString("Database: %1").arg(QFileInfo(m_dbManager.path()).fileName())
02577             : "Database: none");
02578
02579     if (m_athleteStatusLabel)
02580         m_athleteStatusLabel->setText(
02581             m_currentAthleteName.isEmpty()
02582             ? "Athlete: none"
02583             : QString("Athlete: %1").arg(m_currentAthleteName));
02584
02585     int activityCount = 0;
02586     if (m_dbManager.isOpen())
02587     {
02588         auto db = m_dbManager.database();
02589         ActivityRepository repo(db);
02590         activityCount = repo.listActivities(m_currentAthleteId).size();
02591     }
02592
02593     if (m_activityCountLabel)
02594         m_activityCountLabel->setText(QString("Activities: %1").arg(activityCount));
02595 }
02596
02597 // Slots
02598
02599 void MainWindow::importActivities()
02600 {
02601     const QStringList files = QFileDialog::getOpenFileNames(
02602         this, "Import Activities",
02603         m_lastOpenDir,
02604         "FIT Files (*.fit);;All Files (*)");
02605
02606     if (files.isEmpty()) return;
02607     m_lastOpenDir = QFileInfo(files.front()).absolutePath();
02608     importFiles(files);
02609 }
02610
02611 void MainWindow::previewFitFile()
02612 {
02613     const QString fileName = QFileDialog::getOpenFileName(
02614         this, "Preview FIT File",
02615         m_lastOpenDir,
02616         "FIT Files (*.fit);;All Files (*)");
02617     if (fileName.isEmpty()) return;
02618     m_lastOpenDir = QFileInfo(fileName).absolutePath();
02619
02620     QString err;
02621     if (!m_controller->loadFile(fileName, err))
02622         QMessageBox::critical(this, "Preview Failed", err);
02623 }
02624
02625 void MainWindow::createVideo()
02626 {
02627     const RideData& rideData = m_controller->rideData();
02628     if (rideData.records.empty())
02629     {
02630         QMessageBox::information(this, "Create Video", "Load an activity before creating a video.");
02631         return;
02632     }
02633
02634     if (m_controller->currentActivityId() <= 0 || !m_dbManager.isOpen())
02635     {
02636         QMessageBox::information(this, "Create Video", "Video export is available for saved
activities.");
02637         return;
02638     }
02639
02640     auto db = m_dbManager.database();
02641     ActivityRepository actRepo(db);
02642     const Activity activity = actRepo.getActivity(m_controller->currentActivityId());
02643
02644     QString activityName = QFileInfo(activity.fileName).completeBaseName();
02645     if (activityName.isEmpty())
02646         activityName = activity.sport;
02647     if (activityName.isEmpty())
02648         activityName = QStringLiteral("Ride");
02649
02650     QString athleteName = m_currentAthleteName.trimmed();
02651     if (athleteName.isEmpty() && m_currentAthleteId > 0)

```

```

02652     {
02653         AthleteRepository athleteRepo(db);
02654         athleteName = athleteRepo.getAthlete(m_currentAthleteId).fullName().trimmed();
02655     }
02656     if (athleteName.isEmpty())
02657         athleteName = QStringLiteral("Athlete");
02658
02659     const QDate date = !activity.startTime.isEmpty()
02660         ? QDate::fromString(activity.startTime.left(10), Qt::ISODate)
02661         : QDate::fromString(activity.importedAt.left(10), Qt::ISODate);
02662     const QString dateText = DateFormatter::toIsoDate(date.isValid() ? date : QDate::currentDate());
02663
02664     const QString defaultName = QString("%1-%2-%3.mp4")
02665         .arg(dateText,
02666             sanitizeVideoFilePart(activityName),
02667             sanitizeVideoFilePart(athleteName));
02668     const QString defaultOutput = QDir(Platform::defaultDatabaseDirectory()).filePath(defaultName);
02669
02670     const double visibleStartMin = m_powerChart ? m_powerChart->visibleRangeStartMinutes() : 0.0;
02671     const double visibleEndMin = m_powerChart ? m_powerChart->visibleRangeEndMinutes() : 0.0;
02672
02673     VideoExportSettings defaults;
02674     defaults.outputPath = defaultOutput;
02675     defaults.ffmpegPath = resolveFfmpegExecutablePath();
02676     defaults.width = 1920;
02677     defaults.height = 1080;
02678     defaults.fps = 30;
02679     defaults.playbackSpeed = 3.0;
02680     defaults.useSelectedSegment = true;
02681     defaults.segmentStartSeconds = std::max(0.0, std::min(visibleStartMin, visibleEndMin) * 60.0);
02682     defaults.segmentEndSeconds = std::max(defaults.segmentStartSeconds,
02683         std::max(visibleStartMin, visibleEndMin) * 60.0);
02684     defaults.mapStyle = m_mapRenderer ? m_mapRenderer->mapStyle() : MapStyle::Light;
02685     defaults.autoZoom = true;
02686     defaults.fixedZoom = 18;
02687     defaults.followAthlete = true;
02688     defaults.routeColorMetric = currentColorMetric() == ColorMetric::None
02689         ? ColorMetric::Power
02690         : currentColorMetric();
02691     defaults.athleteName = athleteName;
02692     defaults.activityName = activityName;
02693     defaults.colorContext = buildColorContext();
02694     defaults.rideData = rideData;
02695
02696     if (defaults.segmentEndSeconds - defaults.segmentStartSeconds < 1.0)
02697     {
02698         defaults.segmentStartSeconds = 0.0;
02699         defaults.segmentEndSeconds = rideData.records.back().elapsedSeconds;
02700     }
02701
02702     VideoExportDialog dialog(defaults, this);
02703     dialog.exec();
02704 }
02705
02706 void MainWindow::onActivityImported(int activityId)
02707 {
02708     QSettings("Fitlyzer", "FitlyzerC").setValue("lastActivityId", activityId);
02709     if (!m_firstLaunchCompleted)
02710     {
02711         m_firstLaunchCompleted = true;
02712         QSettings("Fitlyzer", "FitlyzerC").setValue("firstLaunchCompleted", true);
02713     }
02714
02715     if (m_activityBrowser)
02716     {
02717         m_activityBrowser->refresh(m_currentAthleteId);
02718         m_tabWidget->setCurrentIndex(kTabActivities);
02719     }
02720
02721     updateStatusBarInfo();
02722     hideWelcomeScreen();
02723 }
02724
02725 void MainWindow::openDatabase()
02726 {
02727     const QString startDir = Platform::defaultDatabaseDirectory();
02728     const QString path = QFileDialog::getOpenFileName(
02729         this, "Open Database", startDir,
02730         "Fitlyzer Database (*.fitlyzerdb);;All Files (*)");
02731     if (path.isEmpty()) return;
02732     openDatabasePath(path);
02733 }
02734
02735 void MainWindow::createDatabase()
02736 {
02737     CreateDatabaseDialog dlg(Platform::defaultDatabaseDirectory(), this);
02738     if (dlg.exec() != QDialog::Accepted)

```

```

02739         return;
02740
02741         const QString path = dlg.databasePath();
02742         if (path.isEmpty())
02743             return;
02744
02745         QString err;
02746         if (!m_dbManager.create(path, &err))
02747         {
02748             QMessageBox::critical(this, "Database Error",
02749                 "Could not create database:\n" + err);
02750             return;
02751         }
02752
02753         m_controller->setDatabaseManager(&m_dbManager);
02754         QSettings settings("Fitlyzer", "FitlyzerC");
02755         settings.setValue("lastDatabase", path);
02756         m_lastOpenDir = QFile::info(path).absolutePath();
02757
02758         if (dlg.shouldCreateDefaultAthlete())
02759         {
02760             auto db = m_dbManager.database();
02761             AthleteRepository repo(db);
02762
02763             Athlete athlete;
02764             athlete.firstName = dlg.defaultAthleteFirstName();
02765             athlete.lastName = dlg.defaultAthleteLastName();
02766             const int athleteId = repo.insertAthlete(athlete);
02767             if (athleteId > 0)
02768             {
02769                 m_currentAthleteId = athleteId;
02770                 m_currentAthleteName = repo.getAthlete(athleteId).fullName();
02771                 m_controller->setCurrentAthlete(athleteId);
02772                 settings.setValue("currentAthleteId", athleteId);
02773             }
02774         }
02775
02776         addRecentDatabase(path);
02777         updateRecentDatabaseMenu();
02778         refreshAthleteSelector();
02779         updateAthleteLabel();
02780         updateImportAvailability();
02781         updateStatsLabel();
02782         updateStatusBarInfo();
02783         updateWelcomeScreenVisibility();
02784
02785         QMessageBox::information(this, "Database Created",
02786             "Database created successfully.\n" + path);
02787     }
02788
02789     void MainWindow::manageAthletes()
02790     {
02791         if (!m_dbManager.isOpen())
02792         {
02793             QMessageBox::information(this, "No Database",
02794                 "Please open or create a database first.");
02795             return;
02796         }
02797
02798         auto db = m_dbManager.database();
02799         AthleteRepository repo(db);
02800         AthleteListDialog dlg(repo, this);
02801         if (dlg.exec() == QDialog::Accepted)
02802         {
02803             m_currentAthleteId = dlg.selectedAthleteId();
02804             m_controller->setCurrentAthlete(m_currentAthleteId);
02805             m_currentAthleteName = repo.getAthlete(m_currentAthleteId).fullName();
02806             QSettings("Fitlyzer", "FitlyzerC")
02807                 .setValue("currentAthleteId", m_currentAthleteId);
02808             updateAthleteLabel();
02809             refreshAthleteSelector();
02810             if (m_activityBrowser)
02811                 m_activityBrowser->refresh(m_currentAthleteId);
02812             updateImportAvailability();
02813             updateStatusBarInfo();
02814             updateWelcomeScreenVisibility();
02815         }
02816     }
02817
02818     void MainWindow::openSettingsDialog()
02819     {
02820         QDialog dialog(this);
02821         dialog.setWindowTitle("Settings");
02822         dialog.setModal(true);
02823
02824         auto* layout = new QVBoxLayout(&dialog);
02825         auto* generalGroup = new QGroupBox("General", &dialog);

```

```

02826     auto* form = new QFormLayout(generalGroup);
02827
02828     auto* dateFormatCombo = new QComboBox(generalGroup);
02829     dateFormatCombo->addItem("DD-MM-YYYY (14-06-2026)", static_cast<int>(DateFormat::DD_MM_YYYY));
02830     dateFormatCombo->addItem("YYYY-MM-DD (2026-06-14)", static_cast<int>(DateFormat::YYYY_MM_DD));
02831     dateFormatCombo->addItem("DD.MM.YYYY (14.06.2026)",
static_cast<int>(DateFormat::DD_DOT_MM_DOT_YYYY));
02832     dateFormatCombo->addItem("MM/DD/YYYY (06/14/2026)", static_cast<int>(DateFormat::MM_DD_YYYY));
02833
02834     const DateFormat currentFormat = AppSettings::instance().dateFormat();
02835     const int currentIndex = dateFormatCombo->findData(static_cast<int>(currentFormat));
02836     dateFormatCombo->setCurrentIndex(currentIndex >= 0 ? currentIndex : 0);
02837
02838     form->addRow("Date Format:", dateFormatCombo);
02839     layout->addWidget(generalGroup);
02840
02841     // Maps group
02842     auto* mapsGroup = new QGroupBox("Maps", &dialog);
02843     auto* mapsForm = new QFormLayout(mapsGroup);
02844     auto* tileCacheCombo = new QComboBox(mapsGroup);
02845     tileCacheCombo->addItem("128 tiles (~32 MB)", 128);
02846     tileCacheCombo->addItem("256 tiles (~64 MB)", 256);
02847     tileCacheCombo->addItem("512 tiles (~128 MB)", 512);
02848     tileCacheCombo->addItem("1024 tiles (~256 MB)", 1024);
02849     tileCacheCombo->addItem("2048 tiles (~512 MB)", 2048);
02850     const int currentCacheSize = AppSettings::instance().tileCacheSize();
02851     const int cacheSizeIndex = tileCacheCombo->findData(currentCacheSize);
02852     tileCacheCombo->setCurrentIndex(cacheSizeIndex >= 0 ? cacheSizeIndex : 2);
02853     mapsForm->addRow("Tile Memory Cache:", tileCacheCombo);
02854     layout->addWidget(mapsGroup);
02855
02856     auto* buttons = new QDialogButtonBox(QDialogButtonBox::Ok | QDialogButtonBox::Cancel, &dialog);
02857     layout->addWidget(buttons);
02858
02859     connect(buttons, &QDialogButtonBox::accepted, &dialog, &QDialog::accept);
02860     connect(buttons, &QDialogButtonBox::rejected, &dialog, &QDialog::reject);
02861
02862     if (dialog.exec() != QDialog::Accepted)
02863         return;
02864
02865     const DateFormat selectedFormat = static_cast<DateFormat>(dateFormatCombo->currentData().toInt());
02866     if (selectedFormat == currentFormat)
02867         return;
02868
02869     AppSettings::instance().setDateFormat(selectedFormat);
02870
02871     const int selectedCacheSize = tileCacheCombo->currentData().toInt();
02872     if (selectedCacheSize != currentCacheSize)
02873     {
02874         AppSettings::instance().setTileCacheSize(selectedCacheSize);
02875         if (m_mapRenderer)
02876             m_mapRenderer->setTileCacheSize(selectedCacheSize);
02877     }
02878
02879     if (selectedFormat == currentFormat)
02880         return;
02881
02882     if (m_activityBrowser)
02883         m_activityBrowser->refresh(m_currentAthleteId);
02884
02885     updateStatsLabel();
02886 }
02887
02888 void MainWindow::backupDatabase()
02889 {
02890     if (!m_dbManager.isOpen())
02891     {
02892         QMessageBox::information(this, "No Database",
02893             "No database is open.");
02894         return;
02895     }
02896     const QString startPath = Platform::defaultBackupPath(m_dbManager.path());
02897     const QString dest = QFileDialog::getSaveFileName(
02898         this, "Backup Database", startPath,
02899         "Fitlyzer Database (*.fitlyzerdb)");
02900     if (dest.isEmpty()) return;
02901
02902     // Checkpoint the WAL before copying so the backup file is self-contained.
02903     {
02904         auto db = m_dbManager.database();
02905         QSqlQuery q(db);
02906         q.exec("PRAGMA wal_checkpoint(FULL)");
02907     }
02908
02909     if (QFile::exists(dest) && !QFile::remove(dest))
02910     {
02911         QMessageBox::critical(this, "Backup Failed",

```

```

02912         "Cannot overwrite existing file:\n" + dest);
02913         return;
02914     }
02915
02916     if (QFile::copy(m_dbManager.path(), dest))
02917         QMessageBox::information(this, "Backup", "Backup saved to:\n" + dest);
02918     else
02919         QMessageBox::critical(this, "Backup Failed",
02920             "Could not copy database to:\n" + dest);
02921 }
02922
02923 void MainWindow::updateAthleteLabel()
02924 {
02925     if (m_athleteLabelAct)
02926         m_athleteLabelAct->setText(
02927             m_currentAthleteName.isEmpty()
02928             ? "(no athlete selected)"
02929             : "Current: " + m_currentAthleteName);
02930 }
02931
02932 void MainWindow::openRecentDatabase()
02933 {
02934     auto* action = qobject_cast<QAction*>(sender());
02935     if (!action) return;
02936     const QString path = action->data().toString();
02937     if (path.isEmpty()) return;
02938     openDatabasePath(path);
02939 }
02940
02941 void MainWindow::onAthleteSelectionChanged(int index)
02942 {
02943     if (!m_athleteCombo || index < 0)
02944         return;
02945
02946     const int athleteId = m_athleteCombo->currentData().toInt();
02947     if (athleteId <= 0)
02948     {
02949         m_currentAthleteId = -1;
02950         m_currentAthleteName.clear();
02951     }
02952     else
02953     {
02954         m_currentAthleteId = athleteId;
02955         m_currentAthleteName = m_athleteCombo->currentText();
02956         m_controller->setCurrentAthlete(athleteId);
02957     }
02958
02959     QSettings("Fitlyzer", "FitlyzerC").setValue("currentAthleteId", m_currentAthleteId);
02960     updateAthleteLabel();
02961     if (m_activityBrowser)
02962         m_activityBrowser->refresh(m_currentAthleteId);
02963     if (m_calendarWidget)
02964         m_calendarWidget->setAthleteId(m_currentAthleteId);
02965     updateAnalysisEmptyStates();
02966     updateImportAvailability();
02967     updateStatsLabel();
02968     updateStatusBarInfo();
02969     updateWelcomeScreenVisibility();
02970 }
02971
02972 void MainWindow::onWorkoutLoaded()
02973 {
02974     const bool hasPower =
02975         m_controller->statistics().maximumPower > 0.0;
02976
02977     updateStatsLabel();
02978     updateAnalysisEmptyStates();
02979     updateCharts();
02980     updateColorLegend();
02981     updateZoneAvailability();
02982
02983     if (m_analysisTabWidget)
02984     {
02985         m_analysisTabWidget->setTabEnabled(kAnalysisTabHistogram, hasPower);
02986         m_analysisTabWidget->setTabEnabled(kAnalysisTabPDC, hasPower);
02987         m_analysisTabWidget->setTabEnabled(kAnalysisTabCalendar, true);
02988         m_analysisTabWidget->setTabEnabled(kAnalysisTabFitness, true);
02989     }
02990
02991     if (hasPower)
02992     {
02993         updateZonesTab();
02994         updateHistogram();
02995         updatePowerCurve();
02996         updateIntervals();
02997         updateFitnessChart();
02998     }

```

```

02999
03000     updateClimbingTab();
03001
03002     if (m_mapRenderer)
03003         m_mapRenderer->setRideData(m_controller->rideData(), currentColorMetric(),
buildColorContext());
03004
03005     // Activity selection should always start from the full-ride chart extent.
03006     resetAllZoom();
03007 }
03008
03009 void MainWindow::updateAnalysisEmptyStates()
03010 {
03011     const bool hasActivity = !m_controller->rideData().records.empty();
03012     const int page = hasActivity ? 1 : 0;
03013
03014     const std::initializer_list<QStackedLayout*> stacks =
03015     { m_activityTabStack,
03016       m_zonesTabStack,
03017       m_histogramTabStack,
03018       m_powerCurveTabStack,
03019       m_calendarTabStack,
03020       m_fitnessTabStack,
03021       m_climbingTabStack };
03022
03023     for (auto* stack : stacks)
03024     {
03025         if (stack)
03026             stack->setCurrentIndex(page);
03027     }
03028 }
03029
03030 // Update helpers
03031
03032 void MainWindow::updateStatsLabel()
03033 {
03034     if (!m_athleteHeader)
03035         return;
03036
03037     QString athleteName = m_currentAthleteName.isEmpty() ? "No athlete selected" :
m_currentAthleteName;
03038     int activityCount = 0;
03039     QString lastActivity = "-";
03040
03041     if (m_dbManager.isOpen() && m_currentAthleteId > 0)
03042     {
03043         auto db = m_dbManager.database();
03044         ActivityRepository actRepo(db);
03045         const QList<Activity> activities = actRepo.listActivities(m_currentAthleteId);
03046         activityCount = activities.size();
03047         if (!activities.isEmpty())
03048         {
03049             const QString rawDate = !activities.first().startTime.isEmpty()
? activities.first().startTime.left(10)
: activities.first().importedAt.left(10);
03050             lastActivity = formatActivityDateLabel(rawDate);
03051         }
03052     }
03053
03054     const int ftpWatts = static_cast<int>(m_controller->ftp());
03055     const double estimatedFtp = estimatedFtpFromCurrentRide();
03056     const int estimatedFtpRounded = estimatedFtp > 0.0 ? static_cast<int>(std::round(estimatedFtp)) :
0;
03057
03058     const int ftpDelta = estimatedFtpRounded - ftpWatts;
03059     m_athleteHeader->setSummary(athleteName, ftpWatts, activityCount, lastActivity);
03060
03061     if (m_useEstimatedFtpButton)
03062         m_useEstimatedFtpButton->setEnabled(m_currentAthleteId > 0 && estimatedFtpRounded > 0);
03063
03064     const QString ftpSummary = estimatedFtpRounded > 0
? QString("FTP %1 Est %2 \u0394%3")
    .arg(ftpWatts)
    .arg(estimatedFtpRounded)
    .arg(ftpDelta >= 0 ? QString("+%1").arg(ftpDelta) : QString::number(ftpDelta))
: QString();
03065
03066     const QString rideText = QString("NP %1 W IF %2 TSS %3 VI %4 EF %5%6")
    .arg(m_controller->normalizedPower(), 0, 'f', 0)
    .arg(m_controller->intensityFactor(), 0, 'f', 2)
    .arg(m_controller->trainingStressScore(), 0, 'f', 0)
    .arg(m_controller->variabilityIndex(), 0, 'f', 2)
    .arg(m_controller->efficiencyFactor(), 0, 'f', 2)
    .arg(ftpSummary.isEmpty() ? QString() : QString(" ") + ftpSummary);
03067
03068     m_athleteHeader->setRideMetrics(rideText);
03069 }
03070
03071 void MainWindow::updateCharts()

```

```

03083 {
03084     const ColorMetric colorMetric = currentColorMetric();
03085     const ColorContext colorContext = buildColorContext();
03086     const std::vector<Climb>& climbsForView =
03087         m_climbMinLengthSpin ? m_detectedClimbs : m_controller->climbs();
03088
03089     setUpdatesEnabled(false);
03090     applyChartSmoothing();
03091     if (m_powerChart)
03092     {
03093         m_powerChart->setIntervals(m_controller->intervals());
03094         m_powerChart->setClimbs(climbsForView);
03095     }
03096     if (m_hrChart) m_hrChart->setClimbs(climbsForView);
03097     if (m_cadenceChart) m_cadenceChart->setClimbs(climbsForView);
03098     if (m_speedChart) m_speedChart->setClimbs(climbsForView);
03099     if (m_altitudeChart) m_altitudeChart->setClimbs(climbsForView);
03100     m_powerChart->setRideData(m_controller->rideData(), colorMetric, colorContext);
03101     m_hrChart->setRideData(m_controller->rideData(), colorMetric, colorContext);
03102     m_cadenceChart->setRideData(m_controller->rideData(), colorMetric, colorContext);
03103     m_speedChart->setRideData(m_controller->rideData(), colorMetric, colorContext);
03104     m_altitudeChart->setRideData(m_controller->rideData(), colorMetric, colorContext);
03105
03106     if (m_climbPowerChart)
03107     {
03108         m_climbPowerChart->setClimbs(climbsForView);
03109         m_climbHrChart->setClimbs(climbsForView);
03110         m_climbCadenceChart->setClimbs(climbsForView);
03111         m_climbSpeedChart->setClimbs(climbsForView);
03112         m_climbAltitudeChart->setClimbs(climbsForView);
03113
03114         m_climbPowerChart->setRideData(m_controller->rideData(), colorMetric, colorContext);
03115         m_climbHrChart->setRideData(m_controller->rideData(), colorMetric, colorContext);
03116         m_climbCadenceChart->setRideData(m_controller->rideData(), colorMetric, colorContext);
03117         m_climbSpeedChart->setRideData(m_controller->rideData(), colorMetric, colorContext);
03118         m_climbAltitudeChart->setRideData(m_controller->rideData(), colorMetric, colorContext);
03119
03120         if (m_climbAltitudeChart && m_climbOverlayEnabledCheck && m_climbOverlayMetricCombo)
03121         {
03122             const ColorMetric overlayMetric =
03123                 static_cast<ColorMetric>(m_climbOverlayMetricCombo->currentData().toInt());
03124             m_climbAltitudeChart->setMetricOverlay(
03125                 overlayMetric,
03126                 m_climbOverlayEnabledCheck->isChecked());
03127         }
03128         setUpdatesEnabled(true);
03129     }
03130
03131 void MainWindow::applyChartPreset(int presetId)
03132 {
03133     if (!m_colorMetricCombo || !m_powerSmoothingCombo || !m_autoSmoothingCheck)
03134         return;
03135
03136     if (presetId == ChartPresetCustom)
03137         return;
03138
03139     QSignalBlocker colorBlock(m_colorMetricCombo);
03140     QSignalBlocker smoothingBlock(m_powerSmoothingCombo);
03141     QSignalBlocker autoBlock(m_autoSmoothingCheck);
03142
03143     switch (presetId)
03144     {
03145     case ChartPresetPowerAnalysis:
03146         m_colorMetricCombo->setCurrentIndex(
03147             m_colorMetricCombo->findData(static_cast<int>(ColorMetric::Power)));
03148         m_powerSmoothingCombo->setCurrentIndex(m_powerSmoothingCombo->findData(5));
03149         m_autoSmoothingCheck->setChecked(false);
03150         m_showPower->setChecked(true);
03151         m_showHR->setChecked(true);
03152         m_showCadence->setChecked(true);
03153         m_showSpeed->setChecked(false);
03154         m_showAltitude->setChecked(false);
03155         break;
03156
03157     case ChartPresetHeartRateFocus:
03158         m_colorMetricCombo->setCurrentIndex(
03159             m_colorMetricCombo->findData(static_cast<int>(ColorMetric::HeartRate)));
03160         m_powerSmoothingCombo->setCurrentIndex(m_powerSmoothingCombo->findData(10));
03161         m_autoSmoothingCheck->setChecked(true);
03162         m_showPower->setChecked(true);
03163         m_showHR->setChecked(true);
03164         m_showCadence->setChecked(false);
03165         m_showSpeed->setChecked(false);
03166         m_showAltitude->setChecked(false);
03167         break;
03168

```



```

03169         case ChartPresetRaceReview:
03170             m_colorMetricCombo->setCurrentIndex(
03171                 m_colorMetricCombo->findData(static_cast<int>(ColorMetric::Power)));
03172             m_powerSmoothingCombo->setCurrentIndex(m_powerSmoothingCombo->findData(30));
03173             m_autoSmoothingCheck->setChecked(true);
03174             m_showPower->setChecked(true);
03175             m_showHR->setChecked(true);
03176             m_showCadence->setChecked(true);
03177             m_showSpeed->setChecked(true);
03178             m_showAltitude->setChecked(true);
03179             break;
03180
03181         default:
03182             return;
03183     }
03184
03185     m_powerSmoothingCombo->setEnabled(!m_autoSmoothingCheck->isChecked());
03186     applyChartSmoothing();
03187     updateColorLegend();
03188     updateZoneAvailability();
03189     updateCharts();
03190     updateZonesTab();
03191     if (m_mapRenderer)
03192         m_mapRenderer->setRideData(m_controller->rideData(), currentColorMetric(),
03193             buildColorContext());
03194
03195 void MainWindow::applyChartSmoothing()
03196 {
03197     const std::initializer_list<RideChartWidget*> all =
03198         { m_powerChart, m_hrChart, m_cadenceChart, m_speedChart, m_altitudeChart,
03199           m_climbPowerChart, m_climbHrChart, m_climbCadenceChart, m_climbSpeedChart,
03200           m_climbAltitudeChart };
03201
03202     const int smoothingSeconds = m_powerSmoothingCombo ? m_powerSmoothingCombo->currentData().toInt()
03203 : 0;
03204     const bool autoSmoothing = m_autoSmoothingCheck ? m_autoSmoothingCheck->isChecked() : false;
03205     for (auto* chart : all)
03206     {
03207         if (!chart)
03208             continue;
03209         chart->setPowerSmoothingSeconds(smoothingSeconds);
03210         chart->setAutoSmoothingEnabled(autoSmoothing);
03211     }
03212
03213 void MainWindow::updateZonesTab()
03214 {
03215     const ColorMetric metric = currentColorMetric();
03216     const std::vector<ZoneDistribution> distributions = ZoneCalculator::computeDistribution(
03217         m_controller->rideData(),
03218         metric,
03219         buildColorContext());
03220
03221     m_zonesTable->setRowCount(static_cast<int>(distributions.size()));
03222
03223     for (int row = 0; row < static_cast<int>(distributions.size()); ++row)
03224     {
03225         const ZoneDistribution& distribution = distributions[static_cast<size_t>(row)];
03226
03227         auto mkItem = [](const QString& text)
03228         {
03229             auto* item = new QTableWidgetItem(text);
03230             item->setTextAlignment(Qt::AlignCenter);
03231             return item;
03232         };
03233
03234         auto* zoneItem = mkItem(QString("%1").arg(row + 1));
03235         zoneItem->setBackground(distribution.zone.color);
03236         zoneItem->setForeground(QColor("#111827"));
03237         m_zonesTable->setItem(row, 0, zoneItem);
03238
03239         auto* nameItem = new QTableWidgetItem(distribution.zone.name);
03240         nameItem->setTextAlignment(Qt::AlignLeft | Qt::AlignVCenter);
03241         m_zonesTable->setItem(row, 1, nameItem);
03242
03243         m_zonesTable->setItem(row, 2, mkItem(rangeLabelForZone(distribution.zone, metric)));
03244
03245         m_zonesTable->setItem(row, 3, mkItem(fmtDur(distribution.seconds)));
03246         m_zonesTable->setItem(row, 4, mkItem(QString::number(distribution.percent * 100.0, 'f', 0) +
03247             "%"));
03248     }
03249     m_zonesTable->resizeColumnsToContents();
03250 }
03251

```

```

03252 void MainWindow::updateHistogram()
03253 {
03254     m_histogram->setRideData(m_controller->rideData());
03255 }
03256
03257 void MainWindow::updatePowerCurve()
03258 {
03259     if (!m_pdcWidget)
03260         return;
03261
03262     std::vector<PowerCurvePoint> last90;
03263     std::vector<PowerCurvePoint> currentYear;
03264     std::vector<PowerCurvePoint> allTime;
03265
03266     if (m_dbManager.isOpen() && m_currentAthleteId > 0)
03267     {
03268         auto db = m_dbManager.database();
03269         ActivityRepository repo(db);
03270         const QList<Activity> activities = repo.listActivities(m_currentAthleteId);
03271         const std::vector<double> durations = PowerCurve::standardDurations();
03272
03273         auto aggregate = [&](auto predicate)
03274         {
03275             std::vector<double> best(durations.size(), 0.0);
03276             for (const Activity& activity : activities)
03277             {
03278                 QString raw = !activity.startTime.isEmpty() ? activity.startTime.left(10) :
activity.importedAt.left(10);
03279                 const QDate day = QDate::fromString(raw, Qt::ISODate);
03280                 if (!predicate(day))
03281                     continue;
03282
03283                 const RideData ride = RideDataSerializer::loadRideFromDatabase(activity.id,
m_dbManager);
03284                 if (ride.records.empty())
03285                     continue;
03286
03287                 for (size_t i = 0; i < durations.size(); ++i)
03288                 {
03289                     const double p = PowerCurve::bestMeanPower(ride, durations[i]);
03290                     if (p > best[i])
03291                         best[i] = p;
03292                 }
03293             }
03294
03295             std::vector<PowerCurvePoint> out;
03296             for (size_t i = 0; i < durations.size(); ++i)
03297             {
03298                 if (best[i] > 0.0)
03299                     out.push_back({durations[i], best[i]});
03300             }
03301             return out;
03302         };
03303
03304         const QDate today = QDate::currentDate();
03305         const QDate ninetyAgo = today.addDays(-90);
03306         const int year = today.year();
03307
03308         last90 = aggregate([&](const QDate& day)
03309         {
03310             return day.isValid() && day >= ninetyAgo;
03311         });
03312         currentYear = aggregate([&](const QDate& day)
03313         {
03314             return day.isValid() && day.year() == year;
03315         });
03316         allTime = aggregate([&](const QDate& day)
03317         {
03318             return day.isValid();
03319         });
03320     }
03321
03322     m_pdcWidget->setRideDataWithComparisons(
03323         m_controller->rideData(),
03324         last90,
03325         currentYear,
03326         allTime,
03327         m_controller->ftp());
03328 }
03329
03330 double MainWindow::estimatedFtpFromCurrentRide() const
03331 {
03332     const double best20m = PowerCurve::bestMeanPower(m_controller->rideData(), 20.0 * 60.0);
03333     if (best20m <= 0.0)
03334         return 0.0;
03335     return best20m * 0.95;
03336 }

```

```

03337
03338 void MainWindow::applyEstimatedFtpForCurrentAthlete()
03339 {
03340     if (!m_dbManager.isOpen() || m_currentAthleteId <= 0)
03341         return;
03342
03343     const int estimated = static_cast<int>(std::round(estimatedFtpFromCurrentRide()));
03344     if (estimated <= 0)
03345     {
03346         QMessageBox::information(this, "Estimated FTP", "Insufficient power data to estimate FTP.");
03347         return;
03348     }
03349
03350     auto db = m_dbManager.database();
03351     AthleteRepository repo(db);
03352     Athlete athlete = repo.getAthlete(m_currentAthleteId);
03353     if (!athlete.isValid())
03354         return;
03355
03356     athlete.ftpWatts = estimated;
03357     if (!repo.updateAthlete(athlete))
03358     {
03359         QMessageBox::critical(this, "FTP Update", "Failed to update athlete FTP.");
03360         return;
03361     }
03362
03363     FtpEntry entry;
03364     entry.athleteId = m_currentAthleteId;
03365     entry.ftpWatts = estimated;
03366     entry.effectiveFrom = DateFormatter::toIsoDate(QDate::currentDate());
03367     entry.notes = "Estimated from best 20-minute power (95%).";
03368     repo.addFtpEntry(entry);
03369
03370     m_controller->setCurrentAthlete(m_currentAthleteId);
03371     updateStatsLabel();
03372     statusBar()->showMessage(QString("Estimated FTP applied: %1 W").arg(estimated), 3500);
03373 }
03374
03375 void MainWindow::updateFitnessChart()
03376 {
03377     if (!m_fitnessChart)
03378         return;
03379
03380     if (!m_dbManager.isOpen() || m_currentAthleteId <= 0)
03381     {
03382         m_fitnessChart->clearChart();
03383         return;
03384     }
03385
03386     auto db = m_dbManager.database();
03387     ActivityRepository repo(db);
03388     const QList<Activity> activities = repo.listActivities(m_currentAthleteId);
03389     if (activities.isEmpty())
03390     {
03391         m_fitnessChart->clearChart();
03392         return;
03393     }
03394
03395     std::map<QDate, double> tssPerDay;
03396     for (const Activity& activity : activities)
03397     {
03398         QString raw = !activity.startTime.isEmpty() ? activity.startTime.left(10) :
activity.importedAt.left(10);
03399         QDate day = QDate::fromString(raw, Qt::ISODate);
03400         if (!day.isValid())
03401             continue;
03402
03403         const double tss = TrainingLoad::trainingStressScore(
03404             activity.durationSec,
03405             activity.normalizedPower,
03406             m_controller->ftp());
03407         tssPerDay[day] += tss;
03408     }
03409
03410     if (tssPerDay.empty())
03411     {
03412         m_fitnessChart->clearChart();
03413         return;
03414     }
03415
03416     const QDate first = tssPerDay.begin()->first;
03417     const QDate last = tssPerDay.rbegin()->first;
03418     std::vector<DailyLoadPoint> daily;
03419     daily.reserve(static_cast<size_t>(first.daysTo(last) + 1));
03420     for (QDate day = first; day <= last; day = day.addDays(1))
03421     {
03422         DailyLoadPoint p;

```

```
03423         auto it = tssPerDay.find(day);
03424         if (it != tssPerDay.end())
03425             p.tss = it->second;
03426         daily.push_back(p);
03427     }
03428
03429     const std::vector<FitnessMetricsPoint> timeline = TrainingLoad::fitnessTimeline(daily);
03430     m_fitnessChart->setTimeline(timeline);
03431 }
03432
03433 void MainWindow::onIntervalSelectionChanged()
03434 {
03435     if (!m_intervalsTable)
03436         return;
03437
03438     const int row = m_intervalsTable->currentRow();
03439     if (row < 0)
03440     {
03441         updateIntervalRowStyles();
03442         clearIntervalSummary();
03443         return;
03444     }
03445
03446     auto* firstItem = m_intervalsTable->item(row, 0);
03447     if (!firstItem)
03448     {
03449         updateIntervalRowStyles();
03450         clearIntervalSummary();
03451         return;
03452     }
03453
03454     const QVariant startVar = firstItem->data(kIntervalStartRole);
03455     const QVariant endVar = firstItem->data(kIntervalEndRole);
03456     if (!startVar.isValid() || !endVar.isValid())
03457     {
03458         updateIntervalRowStyles();
03459         clearIntervalSummary();
03460         return;
03461     }
03462
03463     const double startSeconds = startVar.toDouble();
03464     const double endSeconds = endVar.toDouble();
03465
03466     updateIntervalRowStyles();
03467     updateIntervalSummary(startSeconds, endSeconds);
03468
03469     if (m_followIntervalSelectionCheck && m_followIntervalSelectionCheck->isChecked())
03470         navigateToInterval(startSeconds, endSeconds, false);
03471 }
03472
03473 void MainWindow::onIntervalRowDoubleClicked(QTableWidgetItem* item)
03474 {
03475     if (!item || !m_intervalsTable)
03476         return;
03477
03478     const int row = item->row();
03479     auto* firstItem = m_intervalsTable->item(row, 0);
03480     if (!firstItem)
03481         return;
03482
03483     const QVariant startVar = firstItem->data(kIntervalStartRole);
03484     const QVariant endVar = firstItem->data(kIntervalEndRole);
03485     if (!startVar.isValid() || !endVar.isValid())
03486         return;
03487
03488     if (m_followIntervalSelectionCheck && m_followIntervalSelectionCheck->isChecked())
03489         navigateToInterval(startVar.toDouble(), endVar.toDouble(), true);
03490 }
03491
03492 void MainWindow::navigateToInterval(double startSeconds, double endSeconds, bool exactZoom)
03493 {
03494     if (endSeconds <= startSeconds)
03495         return;
03496
03497     const double duration = endSeconds - startSeconds;
03498     const double padding = exactZoom ? 0.0 : std::max(duration * 0.10, 5.0);
03499     const double rangeStart = std::max(0.0, startSeconds - padding);
03500     const double rangeEnd = endSeconds + padding;
03501
03502     const std::initializer_list<RideChartWidget*> all =
03503     { m_powerChart, m_hrChart, m_cadenceChart, m_speedChart, m_altitudeChart };
03504     for (auto* chart : all)
03505     {
03506         if (!chart)
03507             continue;
03508         chart->setXRange(rangeStart / 60.0, rangeEnd / 60.0);
03509         chart->setCrosshair(startSeconds / 60.0);
03510     }
```

```

03510     }
03511
03512     if (m_mapRenderer)
03513     {
03514         m_mapRenderer->setVisibleTimeRange(startSeconds, endSeconds);
03515         m_mapRenderer->fitToVisibleRange();
03516         m_mapRenderer->setCurrentTime(startSeconds);
03517     }
03518
03519     const QString modeLabel = exactZoom ? "exact" : "padded";
03520     statusBar()->showMessage(
03521         QString("Interval inspection: %1 zoom (%2-%3)")
03522             .arg(modeLabel)
03523             .arg(fmtDur(rangeStart))
03524             .arg(fmtDur(rangeEnd)),
03525         2500);
03526 }
03527
03528 void MainWindow::selectIntervalRowOffset(int delta)
03529 {
03530     if (!m_intervalsTable)
03531         return;
03532
03533     const int rows = m_intervalsTable->rowCount();
03534     if (rows <= 0)
03535         return;
03536
03537     int row = m_intervalsTable->currentRow();
03538     if (row < 0)
03539         row = 0;
03540     else
03541         row = std::clamp(row + delta, 0, rows - 1);
03542
03543     m_intervalsTable->setCurrentCell(row, 0);
03544 }
03545
03546 void MainWindow::updateIntervalRowStyles()
03547 {
03548     if (!m_intervalsTable)
03549         return;
03550
03551     const int selectedRow = m_intervalsTable->currentRow();
03552     for (int row = 0; row < m_intervalsTable->rowCount(); ++row)
03553     {
03554         for (int col = 0; col < m_intervalsTable->columnCount(); ++col)
03555         {
03556             auto* item = m_intervalsTable->item(row, col);
03557             if (!item)
03558                 continue;
03559
03560             QFont font = item->font();
03561             font.setBold(row == selectedRow);
03562             item->setFont(font);
03563         }
03564     }
03565 }
03566
03567 void MainWindow::clearIntervalSummary()
03568 {
03569     if (m_intervalSummaryLabel)
03570         m_intervalSummaryLabel->setText("Interval summary: select an interval");
03571 }
03572
03573 void MainWindow::updateIntervalSummary(double startSeconds, double endSeconds)
03574 {
03575     if (!m_intervalSummaryLabel || endSeconds <= startSeconds)
03576     {
03577         clearIntervalSummary();
03578         return;
03579     }
03580
03581     const auto& records = m_controller->rideData().records;
03582     double sumPower = 0.0;
03583     int countPower = 0;
03584     double sumHr = 0.0;
03585     int countHr = 0;
03586     double sumCadence = 0.0;
03587     int countCadence = 0;
03588     double sumSpeed = 0.0;
03589     int countSpeed = 0;
03590     double elevationGain = 0.0;
03591     std::vector<double> powerValues;
03592
03593     bool havePreviousAltitude = false;
03594     double previousAltitude = 0.0;
03595
03596     for (const RideRecord& record : records)

```

```

03597     {
03598         if (record.elapsedSeconds < startSeconds || record.elapsedSeconds > endSeconds)
03599             continue;
03600
03601         if (record.hasPower)
03602         {
03603             sumPower += record.power;
03604             ++countPower;
03605             powerValues.push_back(record.power);
03606         }
03607         if (record.hasHeartRate)
03608         {
03609             sumHr += record.heartRate;
03610             ++countHr;
03611         }
03612         if (record.hasCadence)
03613         {
03614             sumCadence += record.cadence;
03615             ++countCadence;
03616         }
03617         if (record.hasSpeed)
03618         {
03619             sumSpeed += record.speed;
03620             ++countSpeed;
03621         }
03622         if (record.hasAltitude)
03623         {
03624             if (havePreviousAltitude)
03625             {
03626                 const double delta = record.altitude - previousAltitude;
03627                 if (delta > 0.0)
03628                     elevationGain += delta;
03629             }
03630             previousAltitude = record.altitude;
03631             havePreviousAltitude = true;
03632         }
03633     }
03634
03635     double np = 0.0;
03636     if (!powerValues.empty())
03637     {
03638         double sum4 = 0.0;
03639         for (double p : powerValues)
03640             sum4 += std::pow(p, 4.0);
03641         np = std::pow(sum4 / static_cast<double>(powerValues.size()), 0.25);
03642     }
03643
03644     const double avgPower = countPower > 0 ? (sumPower / static_cast<double>(countPower)) : 0.0;
03645     const double avgHr = countHr > 0 ? (sumHr / static_cast<double>(countHr)) : 0.0;
03646     const double avgCadence = countCadence > 0 ? (sumCadence / static_cast<double>(countCadence)) :
0.0;
03647     const double avgSpeed = countSpeed > 0 ? (sumSpeed / static_cast<double>(countSpeed)) : 0.0;
03648
03649     m_intervalSummaryLabel->setText(
03650         QString("Interval %1 | Avg P %2 | NP %3 | HR %4 | Cad %5 | Speed %6 | Elev +%7")
03651             .arg(fmtDur(std::max(0.0, endSeconds - startSeconds)))
03652             .arg(avgPower > 0.0 ? QString("%1 W").arg(avgPower, 0, 'f', 0) : QString("--"))
03653             .arg(np > 0.0 ? QString("%1 W").arg(np, 0, 'f', 0) : QString("--"))
03654             .arg(avgHr > 0.0 ? QString("%1 bpm").arg(avgHr, 0, 'f', 0) : QString("--"))
03655             .arg(avgCadence > 0.0 ? QString("%1 rpm").arg(avgCadence, 0, 'f', 0) : QString("--"))
03656             .arg(avgSpeed > 0.0 ? QString("%1 km/h").arg(avgSpeed, 0, 'f', 1) : QString("--"))
03657             .arg(QString("%1 m").arg(elevationGain, 0, 'f', 0)));
03658 }
03659
03660 void MainWindow::updateIntervals()
03661 {
03662     if (!m_intervalsTable)
03663         return;
03664
03665     const int previousRow = m_intervalsTable->currentRow();
03666     QSignalBlocker selectionBlocker(m_intervalsTable);
03667
03668     QList<IntervalRecord> records;
03669     if (m_dbManager.isOpen() && m_controller->currentActivityId() > 0)
03670     {
03671         auto db = m_dbManager.database();
03672         IntervalRepository repo(db);
03673         records = repo.listIntervalsForActivity(m_controller->currentActivityId());
03674     }
03675
03676     if (records.isEmpty())
03677     {
03678         const auto& ivs = m_controller->intervals();
03679         m_intervalsTable->setRowCount(static_cast<int>(ivs.size()));
03680
03681         for (int row = 0; row < static_cast<int>(ivs.size()); ++row)
03682         {

```

```

03683         const auto& iv = ivs[static_cast<size_t>(row)];
03684
03685         auto mkItem = [](const QString& text, Qt::Alignment align = Qt::AlignCenter)
03686         {
03687             auto* item = new QTableWidgetItem(text);
03688             item->setTextAlignment(align);
03689             return item;
03690         };
03691
03692         auto* typeItem = mkItem(QString::fromStdString(iv.label));
03693         typeItem->setBackground(
03694             iv.label == "Work"
03695                 ? QBrush(QColor(60, 110, 200, 100))
03696                 : QBrush(QColor(60, 170, 60, 100));
03697         typeItem->setData(kIntervalStartRole, iv.startSeconds);
03698         typeItem->setData(kIntervalEndRole, iv.endSeconds);
03699         m_intervalsTable->setItem(row, 0, typeItem);
03700
03701         m_intervalsTable->setItem(row, 1, mkItem(fmtDur(iv.startSeconds)));
03702         m_intervalsTable->setItem(row, 2, mkItem(fmtDur(iv.durationSeconds)));
03703         m_intervalsTable->setItem(row, 3, mkItem(iv.averagePower > 0
03704             ? QString("%1 W").arg(iv.averagePower, 0, 'f', 0) : "\xe2\x80\x94");
03705         m_intervalsTable->setItem(row, 4, mkItem(iv.normalizedPower > 0
03706             ? QString("%1 W").arg(iv.normalizedPower, 0, 'f', 0) : "\xe2\x80\x94");
03707         const double intervalIf = (m_controller->ftp() > 0.0 && iv.normalizedPower > 0.0)
03708             ? (iv.normalizedPower / m_controller->ftp()) : 0.0;
03709         m_intervalsTable->setItem(row, 5, mkItem(intervalIf > 0.0
03710             ? QString::number(intervalIf, 'f', 2) : "\xe2\x80\x94");
03711         m_intervalsTable->setItem(row, 6, mkItem(iv.maximumPower > 0
03712             ? QString("%1 W").arg(iv.maximumPower, 0, 'f', 0) : "\xe2\x80\x94");
03713         m_intervalsTable->setItem(row, 7, mkItem(iv.averageHeartRate > 0
03714             ? QString("%1 bpm").arg(iv.averageHeartRate, 0, 'f', 0) : "\xe2\x80\x94");
03715     }
03716 }
03717 else
03718 {
03719     m_intervalsTable->setRowCount(records.size());
03720     for (int row = 0; row < records.size(); ++row)
03721     {
03722         const IntervalRecord& r = records[row];
03723         auto* typeItem = new QTableWidgetItem(r.type);
03724         typeItem->setTextAlignment(Qt::AlignCenter);
03725         typeItem->setData(kIntervalStartRole, static_cast<double>(r.startSample));
03726         typeItem->setData(kIntervalEndRole, static_cast<double>(r.endSample));
03727         m_intervalsTable->setItem(row, 0, typeItem);
03728         m_intervalsTable->setItem(row, 1, new QTableWidgetItem(fmtDur(r.startSample)));
03729         m_intervalsTable->setItem(row, 2, new QTableWidgetItem(fmtDur(std::max(0, r.endSample -
03730             r.startSample))));
03731         m_intervalsTable->setItem(row, 3, new QTableWidgetItem(r.avgPower > 0 ? QString("%1
03732             W").arg(r.avgPower, 0, 'f', 0) : "\xe2\x80\x94");
03733         m_intervalsTable->setItem(row, 4, new QTableWidgetItem(r.np > 0 ? QString("%1
03734             W").arg(r.np, 0, 'f', 0) : "\xe2\x80\x94");
03735         const double intervalIf = (m_controller->ftp() > 0.0 && r.np > 0.0)
03736             ? (r.np / m_controller->ftp()) : 0.0;
03737         m_intervalsTable->setItem(row, 5, new QTableWidgetItem(intervalIf > 0.0 ?
03738             QString::number(intervalIf, 'f', 2) : "\xe2\x80\x94");
03739         m_intervalsTable->setItem(row, 6, new QTableWidgetItem("\xe2\x80\x94");
03740         m_intervalsTable->setItem(row, 7, new QTableWidgetItem(r.avgHr > 0 ? QString("%1
03741             bpm").arg(r.avgHr, 0, 'f', 0) : "\xe2\x80\x94");
03742     }
03743 }
03744 m_intervalsTable->resizeColumnsToContents();
03745
03746 if (m_intervalsTable->rowCount() > 0)
03747 {
03748     const int targetRow = std::clamp(previousRow >= 0 ? previousRow : 0, 0,
03749         m_intervalsTable->rowCount() - 1);
03750     m_intervalsTable->setCurrentCell(targetRow, 0);
03751     selectionBlocker.unblock();
03752     updateIntervalRowStyles();
03753     if (m_intervalsTable->currentRow() >= 0)
03754         onIntervalSelectionChanged();
03755     else
03756         clearIntervalSummary();
03757 }
03758 if (m_lapsTable)
03759 {
03760     m_lapsTable->setRowCount(1);
03761     m_lapsTable->setItem(0, 0, new QTableWidgetItem("1"));
03762     m_lapsTable->setItem(0, 1, new QTableWidgetItem("0:00"));
03763     m_lapsTable->setItem(0, 2, new
03764         QTableWidgetItem(fmtDur(m_controller->statistics().durationSeconds)));
03765     m_lapsTable->resizeColumnsToContents();
03766 }

```

```

03763
03764     if (m_activityNotesView && m_dbManager.isOpen() && m_controller->currentActivityId() > 0)
03765     {
03766         auto db = m_dbManager.database();
03767         ActivityRepository repo(db);
03768         const Activity activity = repo.getActivity(m_controller->currentActivityId());
03769
03770         const QDateTime activityDateUtc = QDateTime::fromString(activity.startTime, Qt::ISODate);
03771         const QDateTime importDateUtc = QDateTime::fromString(activity.importedAt, Qt::ISODate);
03772         const QString activityDateText = activityDateUtc.isValid()
03773             ? DateFormatter::formatDateTime(activityDateUtc.toLocalTime())
03774             : QStringLiteral("-");
03775         const QString importDateText = importDateUtc.isValid()
03776             ? DateFormatter::formatDateTime(importDateUtc.toLocalTime())
03777             : QStringLiteral("-");
03778
03779         m_activityNotesView->setPlainText(
03780             QString("Activity Date: %1\nImported: %2\nNotes: %3\nWeather: %4\nTemperature: %5
C\nRPE: %6\nFatigue: %7\nSleep: %8 h\nBike: %9\nEquipment: %10")
03781                 .arg(activityDateText)
03782                 .arg(importDateText)
03783                 .arg(activity.notes)
03784                 .arg(activity.weather)
03785                 .arg(activity.temperature, 0, 'f', 1)
03786                 .arg(activity.rpe)
03787                 .arg(activity.fatigue)
03788                 .arg(activity.sleepHours, 0, 'f', 1)
03789                 .arg(activity.bike)
03790                 .arg(activity.equipment));
03791     }
03792 }
03793
03794 void MainWindow::updateClimbingTab()
03795 {
03796     if (!m_climbsTable)
03797         return;
03798
03799     const auto& ride = m_controller->rideData();
03800     if (ride.records.empty())
03801     {
03802         m_detectedClimbs.clear();
03803         m_climbsTable->setRowCount(0);
03804         updateClimbQuarterCharts(nullptr);
03805         if (m_climbSummaryLabel)
03806             m_climbSummaryLabel->setText("Climb summary: select a climb");
03807         return;
03808     }
03809
03810     // Load stored climbs from controller (which loaded from DB, or detected+stored on first import)
03811     m_detectedClimbs = m_controller->climbs();
03812     applyWeightMetricsToClimbs(m_detectedClimbs, resolveActiveActivityWeightKg());
03813     refreshClimbViews();
03814 }
03815
03816 void MainWindow::refreshClimbViews(double preferredStartSeconds, double preferredEndSeconds)
03817 {
03818     if (!m_climbsTable)
03819         return;
03820
03821     const auto applyClimbOverlays = [this](RideChartWidget* chart)
03822     {
03823         if (chart)
03824             chart->setClimbs(m_detectedClimbs);
03825     };
03826
03827     applyClimbOverlays(m_powerChart);
03828     applyClimbOverlays(m_hrChart);
03829     applyClimbOverlays(m_cadenceChart);
03830     applyClimbOverlays(m_speedChart);
03831     applyClimbOverlays(m_altitudeChart);
03832     applyClimbOverlays(m_climbPowerChart);
03833     applyClimbOverlays(m_climbHrChart);
03834     applyClimbOverlays(m_climbCadenceChart);
03835     applyClimbOverlays(m_climbSpeedChart);
03836     applyClimbOverlays(m_climbAltitudeChart);
03837
03838     const int previousRow = m_climbsTable->currentRow();
03839     QSignalBlocker blocker(m_climbsTable);
03840     m_climbsTable->setSortingEnabled(false);
03841     m_climbsTable->setRowCount(static_cast<int>(m_detectedClimbs.size()));
03842
03843     auto mkSortKeyItem = [](const QString& text, double sortKey)
03844     {
03845         auto* item = new ClimbSortKeyTableItem(text, sortKey);
03846         item->setTextAlignment(Qt::AlignCenter);
03847         return item;
03848     };

```



```

03849
03850     auto mkNumericItem = [mkSortKeyItem](double value, int decimals, const QString& fallback =
    QStringLiteral("--"))
03851     {
03852         if (std::isnan(value))
03853             return mkSortKeyItem(fallback, std::numeric_limits<double>::quiet_NaN());
03854         return mkSortKeyItem(QString::number(value, 'f', decimals), value);
03855     };
03856
03857     for (int row = 0; row < static_cast<int>(m_detectedClimbs.size()); ++row)
03858     {
03859         const Climbe climbe = m_detectedClimbs[static_cast<size_t>(row)];
03860         auto* nameItem = new ClimbeNameTableItem(climbe.name.isEmpty() ? QString("Climb %1").arg(row +
1) : climbe.name);
03861         nameItem->setData(kClimbStartRole, climbe.startSeconds);
03862         nameItem->setData(kClimbEndRole, climbe.endSeconds);
03863         nameItem->setBackground(QBrush(QColor(34, 197, 94, 45)));
03864         m_climbsTable->setItem(row, 0, nameItem);
03865         m_climbsTable->setItem(row, 1, mkNumericItem(climbe.lengthKm, 2));
03866         m_climbsTable->setItem(row, 2, mkNumericItem(climbe.elevationGainM, 0));
03867         m_climbsTable->setItem(row, 3, mkNumericItem(climbe.averageGradient, 1));
03868         m_climbsTable->setItem(row, 4, mkNumericItem(climbe.maximumGradient, 1));
03869         m_climbsTable->setItem(row, 5, mkSortKeyItem(fmtDur(climbe.durationSeconds),
    climbe.durationSeconds));
03870         m_climbsTable->setItem(row, 6, mkNumericItem(climbe.averagePower > 0.0 ? climbe.averagePower :
    std::numeric_limits<double>::quiet_NaN(), 0));
03871         m_climbsTable->setItem(row, 7, mkNumericItem(climbe.averageWattsPerKg > 0.0 ?
    climbe.averageWattsPerKg : std::numeric_limits<double>::quiet_NaN(), 2));
03872         m_climbsTable->setItem(row, 8, mkSortKeyItem(climbe.wattsPerKgRank.isEmpty() ?
    QStringLiteral("--") : climbe.wattsPerKgRank, row));
03873         m_climbsTable->setItem(row, 9, mkNumericItem(climbe.normalizedPower > 0.0 ?
    climbe.normalizedPower : std::numeric_limits<double>::quiet_NaN(), 0));
03874         m_climbsTable->setItem(row, 10, mkNumericItem(climbe.averageHeartRate > 0.0 ?
    climbe.averageHeartRate : std::numeric_limits<double>::quiet_NaN(), 0));
03875         m_climbsTable->setItem(row, 11, mkNumericItem(climbe.averageCadence > 0.0 ?
    climbe.averageCadence : std::numeric_limits<double>::quiet_NaN(), 0));
03876         m_climbsTable->setItem(row, 12, mkNumericItem(climbe.averageSpeed > 0.0 ? climbe.averageSpeed :
    std::numeric_limits<double>::quiet_NaN(), 1));
03877         m_climbsTable->setItem(row, 13, mkNumericItem(climbe.vam > 0.0 ? climbe.vam :
    std::numeric_limits<double>::quiet_NaN(), 0));
03878         m_climbsTable->setItem(row, 14, mkNumericItem(climbe.powerFadePct, 1));
03879         m_climbsTable->setItem(row, 15, mkNumericItem(climbe.aerobicDecouplingPct, 1));
03880         m_climbsTable->setItem(row, 16, mkNumericItem(climbe.difficultyScore, 0));
03881         m_climbsTable->setItem(row, 17, mkSortKeyItem(climbe.category.isEmpty() ? QStringLiteral("--")
    : climbe.category, row));
03882         m_climbsTable->setItem(row, 18, mkSortKeyItem(climbe.shapeClass.isEmpty() ?
    QStringLiteral("--") : climbe.shapeClass, row));
03883     }
03884
03885     m_climbsTable->resizeColumnsToContents();
03886
03887     m_climbsTable->setSortingEnabled(true);
03888     m_climbsTable->sortItems(0, Qt::AscendingOrder);
03889
03890     int targetRow = -1;
03891     if (preferredStartSeconds >= 0.0 && preferredEndSeconds >= 0.0)
03892     {
03893         for (int row = 0; row < m_climbsTable->rowCount(); ++row)
03894         {
03895             auto* item = m_climbsTable->item(row, 0);
03896             if (!item)
03897                 continue;
03898             const double s = item->data(kClimbStartRole).toDouble();
03899             const double e = item->data(kClimbEndRole).toDouble();
03900             if (std::abs(s - preferredStartSeconds) < 0.75 &&
03901                 std::abs(e - preferredEndSeconds) < 0.75)
03902             {
03903                 targetRow = row;
03904                 break;
03905             }
03906         }
03907     }
03908
03909     if (targetRow < 0 && m_climbsTable->rowCount() > 0)
03910         targetRow = std::clamp(previousRow >= 0 ? previousRow : 0, 0, m_climbsTable->rowCount() - 1);
03911
03912     if (targetRow >= 0)
03913         m_climbsTable->setCurrentCell(targetRow, 0);
03914
03915     blocker.unblock();
03916     updateClimbRowStyles();
03917     m_suppressClimbAutoZoom = true;
03918     onClimbSelectionChanged();
03919     m_suppressClimbAutoZoom = false;
03920 }
03921
03922 void MainWindow::updateClimbQuarterCharts(const Climbe climbe)

```

```

03923 {
03924     auto updateChart = [climb](QChartView* view,
03925                               const QString& title,
03926                               const QColor& color,
03927                               const std::function<double(const ClimbQuarterMetrics*)>& metric,
03928                               int decimals,
03929                               const QString& axisLabel)
03930     {
03931         if (!view || !view->chart())
03932             return;
03933
03934         QChart* chart = view->chart();
03935         chart->setTitle(title);
03936         chart->removeAllSeries();
03937
03938         const auto oldAxes = chart->axes();
03939         for (QAbstractAxis* axis : oldAxes)
03940         {
03941             chart->removeAxis(axis);
03942             delete axis;
03943         }
03944
03945         auto* set = new QBarSet(title);
03946         set->setColor(color);
03947
03948         double maxValue = 1.0;
03949         for (int q = 0; q < 4; ++q)
03950         {
03951             double value = 0.0;
03952             if (climb)
03953             {
03954                 value = metric(climb->quarterMetrics[static_cast<size_t>(q)]);
03955                 if (!std::isfinite(value) || value < 0.0)
03956                     value = 0.0;
03957             }
03958
03959             *set « value;
03960             maxValue = std::max(maxValue, value);
03961         }
03962
03963         auto* series = new QBarSeries(chart);
03964         series->append(set);
03965         chart->addSeries(series);
03966
03967         auto* axisX = new QBarCategoryAxis(chart);
03968         axisX->append({ QStringLiteral("Q1"), QStringLiteral("Q2"), QStringLiteral("Q3"),
03969                                QStringLiteral("Q4") });
03970         chart->addAxis(axisX, Qt::AlignBottom);
03971         series->attachAxis(axisX);
03972
03973         auto* axisY = new QValueAxis(chart);
03974         axisY->setRange(0.0, maxValue * 1.15);
03975         axisY->setLabelFormat(decimals > 0 ? QString("%%.1f").arg(decimals) : QString("%.0f"));
03976         axisY->setTitleText(axisLabel);
03977         chart->addAxis(axisY, Qt::AlignLeft);
03978         series->attachAxis(axisY);
03979     };
03980
03981     updateChart (
03982         m_climbQuarterPowerChart,
03983         QStringLiteral("Power by Quarter"),
03984         QColor(37, 99, 235),
03985         [](const ClimbQuarterMetrics& m) { return m.averagePower; },
03986         0,
03987         QStringLiteral("W"));
03988
03989     updateChart (
03990         m_climbQuarterHrChart,
03991         QStringLiteral("Heart Rate by Quarter"),
03992         QColor(220, 38, 38),
03993         [](const ClimbQuarterMetrics& m) { return m.averageHeartRate; },
03994         0,
03995         QStringLiteral("bpm"));
03996
03997     updateChart (
03998         m_climbQuarterCadenceChart,
03999         QStringLiteral("Cadence by Quarter"),
04000         QColor(5, 150, 105),
04001         [](const ClimbQuarterMetrics& m) { return m.averageCadence; },
04002         0,
04003         QStringLiteral("rpm"));
04004 }
04005 void MainWindow::detectClimbsAndRefresh()
04006 {
04007     if (!m_climbsTable)
04008         return;

```

```

04009
04010     const auto& ride = m_controller->rideData();
04011     if (ride.records.empty())
04012     {
04013         m_detectedClimbs.clear();
04014         m_climbsTable->setRowCount(0);
04015         updateClimbQuarterCharts(nullptr);
04016         if (m_climbSummaryLabel)
04017             m_climbSummaryLabel->setText("Climb summary: select a climb");
04018         return;
04019     }
04020
04021     ClimbDetector::Config cfg;
04022     cfg.minLengthKm = m_climbMinLengthSpin ? m_climbMinLengthSpin->value() : 0.15;
04023     cfg.minElevationGainM = m_climbMinGainSpin ? m_climbMinGainSpin->value() : 10.0;
04024     cfg.minAverageGradient = m_climbMinGradientSpin ? m_climbMinGradientSpin->value() : 4.0;
04025     cfg.startGradient = m_climbStartGradientSpin ? m_climbStartGradientSpin->value() : 1.5;
04026     cfg.maxDipMeters = m_climbDipMetersSpin ? m_climbDipMetersSpin->value() : 10.0;
04027     cfg.maxDipDistanceMeters = m_climbDipDistanceSpin ? m_climbDipDistanceSpin->value() : 200.0;
04028     cfg.elevationSmoothingMeters = m_climbSmoothingSpin ? m_climbSmoothingSpin->value() : 50.0;
04029
04030     m_detectedClimbs = ClimbDetector::detect(ride, cfg);
04031     applyWeightMetricsToClimbs(m_detectedClimbs, resolveActiveActivityWeightKg());
04032
04033     refreshClimbViews();
04034 }
04035
04036 double MainWindow::resolveActiveActivityWeightKg() const
04037 {
04038     if (!m_dbManager.isOpen() || !m_controller || m_controller->currentActivityId() <= 0)
04039         return 0.0;
04040
04041     auto db = m_dbManager.database();
04042     ActivityRepository activityRepo(db);
04043     const Activity activity = activityRepo.getActivity(m_controller->currentActivityId());
04044
04045     if (activity.weightKg > 0.0)
04046         return activity.weightKg;
04047
04048     if (m_currentAthleteId <= 0)
04049         return 0.0;
04050
04051     const QDate activityDate = activityDateForWeightContext(activity);
04052     AthleteRepository athleteRepo(db);
04053     const QList<WeightEntry> weightHistory = athleteRepo.getWeightHistory(m_currentAthleteId);
04054
04055     if (weightHistory.isEmpty())
04056         return 0.0;
04057
04058     if (activityDate.isValid())
04059     {
04060         for (const WeightEntry& entry : weightHistory)
04061         {
04062             const QDate entryDate = QDate::fromString(entry.effectiveFrom, Qt::ISODate);
04063             if (!entryDate.isValid() || entryDate <= activityDate)
04064             {
04065                 if (entry.weightKg > 0.0)
04066                     return entry.weightKg;
04067             }
04068         }
04069     }
04070
04071     return weightHistory.front().weightKg > 0.0 ? weightHistory.front().weightKg : 0.0;
04072 }
04073
04074 void MainWindow::assignClimbWeightMetrics(Climb& climb, double riderWeightKg) const
04075 {
04076     climb.averageWattsPerKg = 0.0;
04077     climb.wattsPerKgRank.clear();
04078
04079     if (riderWeightKg <= 0.0 || climb.averagePower <= 0.0)
04080         return;
04081
04082     climb.averageWattsPerKg = climb.averagePower / riderWeightKg;
04083     const double wkg = climb.averageWattsPerKg;
04084
04085     if (wkg >= 4.5)
04086         climb.wattsPerKgRank = QStringLiteral("Elite");
04087     else if (wkg >= 3.8)
04088         climb.wattsPerKgRank = QStringLiteral("Very Strong");
04089     else if (wkg >= 3.2)
04090         climb.wattsPerKgRank = QStringLiteral("Strong");
04091     else if (wkg >= 2.5)
04092         climb.wattsPerKgRank = QStringLiteral("Moderate");
04093     else
04094         climb.wattsPerKgRank = QStringLiteral("Developing");
04095 }

```

```

04096
04097 void MainWindow::applyWeightMetricsToClimbs(std::vector<Climb>& climbs, double riderWeightKg) const
04098 {
04099     for (Climb& climb : climbs)
04100         assignClimbWeightMetrics(climb, riderWeightKg);
04101 }
04102
04103 std::vector<int> MainWindow::selectedClimbIndicesFromTable() const
04104 {
04105     std::vector<int> indices;
04106     if (!m_climbsTable || !m_climbsTable->selectionModel())
04107         return indices;
04108
04109     const QModelIndexList rows = m_climbsTable->selectionModel()->selectedRows(0);
04110     for (const QModelIndex& modelIndex : rows)
04111     {
04112         auto* item = m_climbsTable->item(modelIndex.row(), 0);
04113         if (!item)
04114             continue;
04115
04116         const double selectedStart = item->data(kClimbStartRole).toDouble();
04117         const double selectedEnd = item->data(kClimbEndRole).toDouble();
04118
04119         for (int i = 0; i < static_cast<int>(m_detectedClimbs.size()); ++i)
04120         {
04121             const Climb& c = m_detectedClimbs[static_cast<size_t>(i)];
04122             if (std::abs(c.startSeconds - selectedStart) < 0.75 &&
04123                 std::abs(c.endSeconds - selectedEnd) < 0.75)
04124             {
04125                 indices.push_back(i);
04126                 break;
04127             }
04128         }
04129     }
04130
04131     std::sort(indices.begin(), indices.end());
04132     indices.erase(std::unique(indices.begin(), indices.end()), indices.end());
04133     return indices;
04134 }
04135
04136 void MainWindow::onClimbTableContextMenuRequested(const QPoint& pos)
04137 {
04138     if (!m_climbsTable)
04139         return;
04140
04141     const int clickedRow = m_climbsTable->rowAt(pos.y());
04142     if (clickedRow >= 0 && !m_climbsTable->item(clickedRow, 0)->isSelected())
04143     {
04144         m_climbsTable->clearSelection();
04145         m_climbsTable->setCurrentCell(clickedRow, 0);
04146         m_climbsTable->selectRow(clickedRow);
04147     }
04148
04149     const std::vector<int> selected = selectedClimbIndicesFromTable();
04150     if (selected.empty())
04151         return;
04152
04153     QMenu menu(m_climbsTable);
04154     QAction* editAct = menu.addAction("Edit");
04155     QAction* removeAct = menu.addAction("Remove");
04156     QAction* joinAct = menu.addAction("Join");
04157
04158     editAct->setEnabled(selected.size() == 1);
04159     joinAct->setEnabled(selected.size() >= 2);
04160     removeAct->setEnabled(!selected.empty());
04161
04162     QAction* chosen = menu.exec(m_climbsTable->viewport()->mapToGlobal(pos));
04163     if (!chosen)
04164         return;
04165
04166     if (chosen == editAct)
04167         editSelectedClimbBoundaries();
04168     else if (chosen == removeAct)
04169         removeSelectedClimbs();
04170     else if (chosen == joinAct)
04171         joinSelectedClimbs();
04172 }
04173
04174 void MainWindow::removeSelectedClimbs()
04175 {
04176     std::vector<int> selected = selectedClimbIndicesFromTable();
04177     if (selected.empty())
04178         return;
04179
04180     // Soft-delete: keep the row in DB with deleted=1 to prevent resurrection on re-detect.
04181     if (m_dbManager.isOpen() && m_controller->currentActivityId() > 0)
04182     {

```

```

04183     auto db = m_dbManager.database();
04184     ClimbRepository repo(db);
04185     for (int idx : selected)
04186     {
04187         if (idx >= 0 && idx < static_cast<int>(m_detectedClimbs.size()))
04188         {
04189             const int climbId = m_detectedClimbs[static_cast<size_t>(idx)].id;
04190             if (climbId > 0)
04191                 repo.softDeleteClimb(climbId);
04192         }
04193     }
04194 }
04195
04196 std::sort(selected.rbegin(), selected.rend());
04197 for (int idx : selected)
04198 {
04199     if (idx >= 0 && idx < static_cast<int>(m_detectedClimbs.size()))
04200         m_detectedClimbs.erase(m_detectedClimbs.begin() + idx);
04201 }
04202
04203 for (int i = 0; i < static_cast<int>(m_detectedClimbs.size()); ++i)
04204     m_detectedClimbs[static_cast<size_t>(i)].name = QString("Climb %1").arg(i + 1);
04205
04206 refreshClimbViews();
04207 }
04208
04209 void MainWindow::editSelectedClimbBoundaries()
04210 {
04211     std::vector<int> selected = selectedClimbIndicesFromTable();
04212     if (selected.size() != 1)
04213         return;
04214
04215     const int idx = selected.front();
04216     if (idx < 0 || idx >= static_cast<int>(m_detectedClimbs.size()))
04217         return;
04218
04219     const Climb& climb = m_detectedClimbs[static_cast<size_t>(idx)];
04220
04221     // Graphical edit mode: focus the selected climb and let the user drag
04222     // the vertical start/end handles directly in the climb altitude chart.
04223     if (m_climbAltitudeChart)
04224     {
04225         m_climbAltitudeChart->setClimbEditingEnabled(true);
04226         m_climbAltitudeChart->setSelectedClimbIndex(idx);
04227
04228         const double padding = std::max(5.0, climb.durationSeconds * 0.08);
04229         const double rangeStart = std::max(0.0, climb.startSeconds - padding);
04230         const double rangeEnd = climb.endSeconds + padding;
04231         m_climbAltitudeChart->setXRange(rangeStart / 60.0, rangeEnd / 60.0);
04232         m_climbAltitudeChart->setCrosshair(climb.startSeconds / 60.0);
04233         m_climbAltitudeChart->setFocus();
04234     }
04235
04236     statusBar()->showMessage(
04237         "Graphical edit enabled: drag the climb start/end handles in the elevation chart.",
04238         5000);
04239 }
04240
04241 void MainWindow::joinSelectedClimbs()
04242 {
04243     std::vector<int> selected = selectedClimbIndicesFromTable();
04244     if (selected.size() < 2)
04245         return;
04246
04247     std::sort(selected.begin(), selected.end());
04248     const int keepIdx = selected.front();
04249     if (keepIdx < 0 || keepIdx >= static_cast<int>(m_detectedClimbs.size()))
04250         return;
04251
04252     double newStart = m_detectedClimbs[static_cast<size_t>(keepIdx)].startSeconds;
04253     double newEnd = m_detectedClimbs[static_cast<size_t>(keepIdx)].endSeconds;
04254     for (int idx : selected)
04255     {
04256         if (idx < 0 || idx >= static_cast<int>(m_detectedClimbs.size()))
04257             continue;
04258         const Climb& c = m_detectedClimbs[static_cast<size_t>(idx)];
04259         newStart = std::min(newStart, c.startSeconds);
04260         newEnd = std::max(newEnd, c.endSeconds);
04261     }
04262
04263     // Update the kept climb boundaries (persists to DB with locked=1, source='edited')
04264     const Climb keep = m_detectedClimbs[static_cast<size_t>(keepIdx)];
04265     onClimbBoundaryEdited(keep.startSeconds, keep.endSeconds, newStart, newEnd);
04266
04267     // Soft-delete the merged-away climbs
04268     if (m_dbManager.isOpen() && m_controller->currentActivityId() > 0)
04269     {

```

```

04270         auto db = m_dbManager.database();
04271         ClimbRepository repo(db);
04272         std::sort(selected.rbegin(), selected.rend());
04273         for (int idx : selected)
04274         {
04275             if (idx == keepIdx)
04276                 continue;
04277             if (idx >= 0 && idx < static_cast<int>(m_detectedClimbs.size()))
04278             {
04279                 const int climbId = m_detectedClimbs[static_cast<size_t>(idx)].id;
04280                 if (climbId > 0)
04281                     repo.softDeleteClimb(climbId);
04282                 m_detectedClimbs.erase(m_detectedClimbs.begin() + idx);
04283             }
04284         }
04285     }
04286     else
04287     {
04288         std::sort(selected.rbegin(), selected.rend());
04289         for (int idx : selected)
04290         {
04291             if (idx == keepIdx)
04292                 continue;
04293             if (idx >= 0 && idx < static_cast<int>(m_detectedClimbs.size()))
04294                 m_detectedClimbs.erase(m_detectedClimbs.begin() + idx);
04295         }
04296     }
04297
04298     for (int i = 0; i < static_cast<int>(m_detectedClimbs.size()); ++i)
04299         m_detectedClimbs[static_cast<size_t>(i)].name = QString("Climb %1").arg(i + 1);
04300
04301     refreshClimbViews(newStart, newEnd);
04302 }
04303
04304 void MainWindow::updateClimbRowStyles()
04305 {
04306     if (!m_climbsTable)
04307         return;
04308
04309     const int selectedRow = m_climbsTable->currentRow();
04310     for (int row = 0; row < m_climbsTable->rowCount(); ++row)
04311     {
04312         for (int col = 0; col < m_climbsTable->columnCount(); ++col)
04313         {
04314             auto* item = m_climbsTable->item(row, col);
04315             if (!item)
04316                 continue;
04317
04318             QFont font = item->font();
04319             font.setBold(row == selectedRow);
04320             item->setFont(font);
04321         }
04322     }
04323 }
04324
04325 void MainWindow::onClimbSelectionChanged()
04326 {
04327     if (!m_climbsTable)
04328         return;
04329
04330     const int row = m_climbsTable->currentRow();
04331     if (row < 0)
04332     {
04333         if (m_climbAltitudeChart)
04334             m_climbAltitudeChart->setSelectedClimbIndex(-1);
04335         updateClimbQuarterCharts(nullptr);
04336         updateClimbRowStyles();
04337         if (m_climbSummaryLabel)
04338             m_climbSummaryLabel->setText("Climb summary: select a climb");
04339         return;
04340     }
04341
04342     auto* firstItem = m_climbsTable->item(row, 0);
04343     if (!firstItem)
04344         return;
04345
04346     const double selectedStart = firstItem->data(kClimbStartRole).toDouble();
04347     const double selectedEnd = firstItem->data(kClimbEndRole).toDouble();
04348
04349     int climbIndex = -1;
04350     for (int i = 0; i < static_cast<int>(m_detectedClimbs.size()); ++i)
04351     {
04352         const Climb& c = m_detectedClimbs[static_cast<size_t>(i)];
04353         if (std::abs(c.startSeconds - selectedStart) < 0.75 &&
04354             std::abs(c.endSeconds - selectedEnd) < 0.75)
04355         {
04356             climbIndex = i;

```

```

04357         break;
04358     }
04359 }
04360
04361 if (climbIndex < 0)
04362 {
04363     if (m_climbAltitudeChart)
04364         m_climbAltitudeChart->setSelectedClimbIndex(-1);
04365     updateClimbQuarterCharts(nullptr);
04366     return;
04367 }
04368
04369 if (m_climbAltitudeChart)
04370     m_climbAltitudeChart->setSelectedClimbIndex(climbIndex);
04371
04372 const Climb& climb = m_detectedClimbs[static_cast<size_t>(climbIndex)];
04373 updateClimbQuarterCharts(&climb);
04374 const double padding = std::max(5.0, climb.durationSeconds * 0.08);
04375 const double startSeconds = std::max(0.0, climb.startSeconds - padding);
04376 const double endSeconds = climb.endSeconds + padding;
04377
04378 (void)startSeconds;
04379 (void)endSeconds;
04380
04381 if (m_climbSummaryLabel)
04382 {
04383     const QString wkgText = climb.averageWattsPerKg > 0.0
04384         ? QString::number(climb.averageWattsPerKg, 'f', 2)
04385         : QStringLiteral("--");
04386     m_climbSummaryLabel->setText(
04387         QString("%1 | %2 | Gain %3 m | Avg %4%% | W/kg %5 (%6) | VAM %7 m/h | Fade %8%% | Decouple
04388 %9%% | %10 | %11")
04389         .arg(climb.name)
04390         .arg(fmtDur(climb.durationSeconds))
04391         .arg(climb.elevationGainM, 0, 'f', 0)
04392         .arg(climb.averageGradient, 0, 'f', 1)
04393         .arg(wkgText)
04394         .arg(climb.wattsPerKgRank.isEmpty() ? QStringLiteral("--") : climb.wattsPerKgRank)
04395         .arg(climb.vam, 0, 'f', 0)
04396         .arg(climb.powerFadePct, 0, 'f', 1)
04397         .arg(climb.aerobicDecouplingPct, 0, 'f', 1)
04398         .arg(climb.category)
04399         .arg(climb.shapeClass));
04400 }
04401 updateClimbRowStyles();
04402 }
04403
04404 void MainWindow::onClimbRowDoubleClicked(QTableWidgetItem* item)
04405 {
04406     if (!item || !m_climbsTable)
04407         return;
04408
04409     const int row = item->row();
04410     auto* firstItem = m_climbsTable->item(row, 0);
04411     if (!firstItem)
04412         return;
04413
04414     const double selectedStart = firstItem->data(kClimbStartRole).toDouble();
04415     const double selectedEnd = firstItem->data(kClimbEndRole).toDouble();
04416
04417     int climbIndex = -1;
04418     for (int i = 0; i < static_cast<int>(m_detectedClimbs.size()); ++i)
04419     {
04420         const Climb& c = m_detectedClimbs[static_cast<size_t>(i)];
04421         if (std::abs(c.startSeconds - selectedStart) < 0.75 &&
04422             std::abs(c.endSeconds - selectedEnd) < 0.75)
04423         {
04424             climbIndex = i;
04425             break;
04426         }
04427     }
04428
04429     if (climbIndex < 0)
04430         return;
04431
04432     const Climb& climb = m_detectedClimbs[static_cast<size_t>(climbIndex)];
04433     const double padding = std::max(5.0, climb.durationSeconds * 0.08);
04434     const double rangeStart = std::max(0.0, climb.startSeconds - padding);
04435     const double rangeEnd = climb.endSeconds + padding;
04436
04437     const std::initializer_list<RideChartWidget*> all =
04438     { m_powerChart, m_hrChart, m_cadenceChart, m_speedChart, m_altitudeChart,
04439       m_climbPowerChart, m_climbHrChart, m_climbCadenceChart, m_climbSpeedChart,
04440       m_climbAltitudeChart };
04441     for (auto* chart : all)

```

```

04442         if (!chart)
04443             continue;
04444         chart->setXRange(rangeStart / 60.0, rangeEnd / 60.0);
04445         chart->setCrosshair(climb.startSeconds / 60.0);
04446     }
04447
04448     if (m_mapRenderer)
04449     {
04450         m_mapRenderer->setVisibleTimeRange(climb.startSeconds, climb.endSeconds);
04451         m_mapRenderer->fitToVisibleRange();
04452         m_mapRenderer->setCurrentTime(climb.startSeconds);
04453     }
04454 }
04455
04456 void MainWindow::onClimbBoundaryEdited(
04457     double oldStartSeconds,
04458     double oldEndSeconds,
04459     double newStartSeconds,
04460     double newEndSeconds)
04461 {
04462     if (m_detectedClimbs.empty())
04463         return;
04464
04465     int idx = -1;
04466     for (int i = 0; i < static_cast<int>(m_detectedClimbs.size()); ++i)
04467     {
04468         const Climb& c = m_detectedClimbs[static_cast<size_t>(i)];
04469         if (std::abs(c.startSeconds - oldStartSeconds) < 0.75 &&
04470             std::abs(c.endSeconds - oldEndSeconds) < 0.75)
04471         {
04472             idx = i;
04473             break;
04474         }
04475     }
04476
04477     if (idx < 0)
04478         idx = m_climbsTable ? m_climbsTable->currentRow() : -1;
04479     if (idx < 0 || idx >= static_cast<int>(m_detectedClimbs.size()))
04480         return;
04481
04482     const auto& ride = m_controller->rideData();
04483     if (ride.records.empty())
04484         return;
04485
04486     auto nearestIndexForSecond = [&ride](double sec)
04487     {
04488         auto it = std::lower_bound(
04489             ride.records.begin(),
04490             ride.records.end(),
04491             sec,
04492             [](const RideRecord& r, double s) { return r.elapsedSeconds < s; });
04493
04494         if (it == ride.records.end())
04495             return static_cast<int>(ride.records.size()) - 1;
04496
04497         int index = static_cast<int>(std::distance(ride.records.begin(), it));
04498         if (index > 0)
04499         {
04500             const double currDist = std::abs(ride.records[static_cast<size_t>(index)].elapsedSeconds -
04501 sec);
04502             const double prevDist = std::abs(ride.records[static_cast<size_t>(index -
04503 1)].elapsedSeconds - sec);
04504             if (prevDist < currDist)
04505                 --index;
04506         }
04507         return index;
04508     };
04509
04510     int startIdx = nearestIndexForSecond(newStartSeconds);
04511     int endIdx = nearestIndexForSecond(newEndSeconds);
04512     if (endIdx < startIdx)
04513         std::swap(startIdx, endIdx);
04514     if (endIdx <= startIdx)
04515         endIdx = std::min(static_cast<int>(ride.records.size()) - 1, startIdx + 1);
04516
04517     const std::vector<double> cumulative = ClimbDetector::buildCumulativeDistanceMeters(ride);
04518     const std::vector<double> smoothed = ClimbDetector::smoothAltitudeByDistance(
04519         ride,
04520         cumulative,
04521         m_climbSmoothingSpin ? m_climbSmoothingSpin->value() : 50.0);
04522     const std::vector<double> gradient = ClimbDetector::computeLocalGradientPct(smoothed, cumulative,
04523 25.0);
04524
04525     Climb rebuilt = ClimbMetrics::build(
04526         ride,
04527         cumulative,
04528         smoothed,

```



```

04526         gradient,
04527         startIdx,
04528         endIdx,
04529         idx + 1);
04530
04531     assignClimbWeightMetrics(rebuilt, resolveActiveActivityWeightKg());
04532
04533     rebuilt.name = m_detectedClimbs[static_cast<size_t>(idx)].name;
04534     rebuilt.id = m_detectedClimbs[static_cast<size_t>(idx)].id;
04535
04536     // Push undo command BEFORE applying the change
04537     if (m_undoManager && m_dbManager.isOpen() && rebuilt.id > 0)
04538     {
04539         auto db = m_dbManager.database();
04540         ClimbRepository repo(db);
04541         const ClimbRecord beforeRecord = repo.getClimb(rebuilt.id);
04542         auto refresh = [this]() {
04543             m_controller->reloadClimbsFromDatabase();
04544             m_detectedClimbs = m_controller->climbs();
04545             applyWeightMetricsToClimbs(m_detectedClimbs, resolveActiveActivityWeightKg());
04546             refreshClimbViews();
04547         };
04548         // Build the "after" record for the command (will be populated after DB update below)
04549         // We push before update so we capture before; redo will replay the update.
04550         // The command is created with before = current DB state; after is applied immediately.
04551         // On undo: DB restored to beforeRecord, UI refreshed.
04552         // The "after" state is what we're about to store -- we construct it now from rebuilt.
04553         ClimbRecord afterRecord = beforeRecord;
04554         afterRecord.startSeconds = rebuilt.startSeconds;
04555         afterRecord.endSeconds = rebuilt.endSeconds;
04556         afterRecord.elevationGainM = rebuilt.elevationGainM;
04557         afterRecord.lengthKm = rebuilt.lengthKm;
04558         afterRecord.averageGradient = rebuilt.averageGradient;
04559         afterRecord.avgPower = rebuilt.averagePower;
04560         afterRecord.np = rebuilt.normalizedPower;
04561         afterRecord.avgHr = rebuilt.averageHeartRate;
04562         afterRecord.avgCadence = rebuilt.averageCadence;
04563         afterRecord.vam = rebuilt.vam;
04564         afterRecord.source = QStringLiteral("edited");
04565         afterRecord.locked = true;
04566         m_undoManager->push(std::make_unique<ClimbEditCommand>(
04567             beforeRecord, afterRecord,
04568             m_dbManager.database().connectionName(),
04569             refresh));
04570     }
04571
04572     m_detectedClimbs[static_cast<size_t>(idx)] = rebuilt;
04573
04574     // Persist boundary change to DB
04575     if (m_dbManager.isOpen() && m_controller->currentActivityId() > 0)
04576     {
04577         auto db = m_dbManager.database();
04578         ClimbRepository repo(db);
04579         const Climb& c = m_detectedClimbs[static_cast<size_t>(idx)];
04580         if (c.id > 0)
04581         {
04582             ClimbRecord record;
04583             record.id = c.id;
04584             record.activityId = m_controller->currentActivityId();
04585             record.startSeconds = c.startSeconds;
04586             record.endSeconds = c.endSeconds;
04587             record.name = c.name;
04588             record.elevationGainM = c.elevationGainM;
04589             record.lengthKm = c.lengthKm;
04590             record.averageGradient = c.averageGradient;
04591             record.source = QStringLiteral("edited");
04592             record.locked = true;
04593             repo.updateClimb(record);
04594         }
04595         else
04596         {
04597             // Not yet in DB (e.g., live preview detection), insert it
04598             ClimbRecord record;
04599             record.activityId = m_controller->currentActivityId();
04600             record.startSeconds = c.startSeconds;
04601             record.endSeconds = c.endSeconds;
04602             record.name = c.name;
04603             record.elevationGainM = c.elevationGainM;
04604             record.lengthKm = c.lengthKm;
04605             record.averageGradient = c.averageGradient;
04606             record.source = QStringLiteral("edited");
04607             record.locked = true;
04608             m_detectedClimbs[static_cast<size_t>(idx)].id = repo.insertClimb(record);
04609         }
04610     }
04611
04612     const auto applyClimbOverlays = [this](RideChartWidget* chart)

```

```

04613     {
04614         if (chart)
04615             chart->setClimbs(m_detectedClimbs);
04616     };
04617     applyClimbOverlays(m_powerChart);
04618     applyClimbOverlays(m_hrChart);
04619     applyClimbOverlays(m_cadenceChart);
04620     applyClimbOverlays(m_speedChart);
04621     applyClimbOverlays(m_altitudeChart);
04622     applyClimbOverlays(m_climbPowerChart);
04623     applyClimbOverlays(m_climbHrChart);
04624     applyClimbOverlays(m_climbCadenceChart);
04625     applyClimbOverlays(m_climbSpeedChart);
04626     applyClimbOverlays(m_climbAltitudeChart);
04627
04628     if (m_climbsTable)
04629     {
04630         auto setCell = [this, idx](int col, const QString& text)
04631         {
04632             auto* item = m_climbsTable->item(idx, col);
04633             if (!item)
04634             {
04635                 item = new QTableWidgetItem;
04636                 m_climbsTable->setItem(idx, col, item);
04637             }
04638             item->setText(text);
04639             item->setTextAlignment(Qt::AlignCenter);
04640         };
04641
04642         auto setNumericCell = [this, idx](int col, double value, int decimals, const QString& fallback
= QStringLiteral("--"), bool alwaysNumeric = false)
04643         {
04644             QTableWidgetItem* item = nullptr;
04645             if (alwaysNumeric || value > 0.0)
04646                 item = new ClimbsSortKeyTableItem(QString::number(value, 'f', decimals), value);
04647             else
04648                 item = new ClimbsSortKeyTableItem(fallback, std::numeric_limits<double>::quiet_NaN());
04649             item->setTextAlignment(Qt::AlignCenter);
04650             m_climbsTable->setItem(idx, col, item);
04651         };
04652
04653         auto* nameItem = m_climbsTable->item(idx, 0);
04654         if (!nameItem)
04655         {
04656             nameItem = new ClimbsNameTableItem(rebuilt.name);
04657             m_climbsTable->setItem(idx, 0, nameItem);
04658         }
04659         nameItem->setData(kClimbStartRole, rebuilt.startSeconds);
04660         nameItem->setData(kClimbEndRole, rebuilt.endSeconds);
04661
04662         setNumericCell(1, rebuilt.lengthKm, 2, QStringLiteral("0.00"), true);
04663         setNumericCell(2, rebuilt.elevationGainM, 0, QStringLiteral("0"), true);
04664         setNumericCell(3, rebuilt.averageGradient, 1, QStringLiteral("0.0"), true);
04665         setNumericCell(4, rebuilt.maximumGradient, 1, QStringLiteral("0.0"), true);
04666         m_climbsTable->setItem(idx, 5, new ClimbsSortKeyTableItem(fmtDur(rebuilt.durationSeconds),
rebuilt.durationSeconds));
04667         m_climbsTable->item(idx, 5)->setTextAlignment(Qt::AlignCenter);
04668         setNumericCell(6, rebuilt.averagePower, 0);
04669         setNumericCell(7, rebuilt.averageWattsPerKg, 2);
04670         setCell(8, rebuilt.wattsPerKgRank.isEmpty() ? QStringLiteral("--") : rebuilt.wattsPerKgRank);
04671         setNumericCell(9, rebuilt.normalizedPower, 0);
04672         setNumericCell(10, rebuilt.averageHeartRate, 0);
04673         setNumericCell(11, rebuilt.averageCadence, 0);
04674         setNumericCell(12, rebuilt.averageSpeed, 1);
04675         setNumericCell(13, rebuilt.vam, 0);
04676         setNumericCell(14, rebuilt.powerFadePct, 1, QStringLiteral("0.0"), true);
04677         setNumericCell(15, rebuilt.aerobicDecouplingPct, 1, QStringLiteral("0.0"), true);
04678         setNumericCell(16, rebuilt.difficultyScore, 0, QStringLiteral("0"), true);
04679         setCell(17, rebuilt.category.isEmpty() ? QStringLiteral("--") : rebuilt.category);
04680         setCell(18, rebuilt.shapeClass.isEmpty() ? QStringLiteral("--") : rebuilt.shapeClass);
04681     }
04682     onClimbSelectionChanged();
04683 }
04684
04685 void MainWindow::revertSelectedClimbToAuto()
04686 {
04687     std::vector<int> selected = selectedClimbIndicesFromTable();
04688     if (selected.size() != 1)
04689         return;
04690
04691     const int idx = selected.front();
04692     if (idx < 0 || idx >= static_cast<int>(m_detectedClimbs.size()))
04693         return;
04694
04695     if (!m_dbManager.isOpen() || m_controller->currentActivityId() <= 0)
04696         return;

```

```

04698
04699     const int climbId = m_detectedClimbs[static_cast<size_t>(idx)].id;
04700     if (climbId <= 0)
04701         return;
04702
04703     auto db = m_dbManager.database();
04704     ClimbRepository repo(db);
04705     ClimbRecord record = repo.getClimb(climbId);
04706     if (record.id <= 0)
04707         return;
04708
04709     if (record.originalStartSeconds >= 0.0)
04710         record.startSeconds = record.originalStartSeconds;
04711     if (record.originalEndSeconds >= 0.0)
04712         record.endSeconds = record.originalEndSeconds;
04713     record.source = QStringLiteral("auto");
04714     record.locked = false;
04715     record.deleted = false;
04716
04717     if (!repo.updateClimb(record))
04718         return;
04719
04720     m_controller->reloadClimbsFromDatabase();
04721     m_detectedClimbs = m_controller->climbs();
04722     applyWeightMetricsToClimbs(m_detectedClimbs, resolveActiveActivityWeightKg());
04723     refreshClimbViews(record.startSeconds, record.endSeconds);
04724 }
04725
04726 void MainWindow::onNewClimbRequested(double startSeconds, double endSeconds)
04727 {
04728     if (!m_controller)
04729         return;
04730
04731     const auto& ride = m_controller->rideData();
04732     if (ride.records.empty())
04733         return;
04734
04735     const double orderedStart = std::min(startSeconds, endSeconds);
04736     const double orderedEnd = std::max(startSeconds, endSeconds);
04737     if (orderedEnd - orderedStart < 1.0)
04738         return;
04739
04740     auto nearestIndexForSecond = [&ride](double sec)
04741     {
04742         auto it = std::lower_bound(
04743             ride.records.begin(),
04744             ride.records.end(),
04745             sec,
04746             [](const RideRecord& r, double s) { return r.elapsedSeconds < s; });
04747
04748         if (it == ride.records.end())
04749             return static_cast<int>(ride.records.size()) - 1;
04750
04751         int index = static_cast<int>(std::distance(ride.records.begin(), it));
04752         if (index > 0)
04753         {
04754             const double currDist = std::abs(ride.records[static_cast<size_t>(index)].elapsedSeconds -
sec);
04755             const double prevDist = std::abs(ride.records[static_cast<size_t>(index -
1)].elapsedSeconds - sec);
04756             if (prevDist < currDist)
04757                 --index;
04758         }
04759         return index;
04760     };
04761
04762     int startIdx = nearestIndexForSecond(orderedStart);
04763     int endIdx = nearestIndexForSecond(orderedEnd);
04764     if (endIdx < startIdx)
04765         std::swap(startIdx, endIdx);
04766     if (endIdx <= startIdx)
04767         endIdx = std::min(static_cast<int>(ride.records.size()) - 1, startIdx + 1);
04768
04769     const std::vector<double> cumulative = ClimbDetector::buildCumulativeDistanceMeters(ride);
04770     const std::vector<double> smoothed = ClimbDetector::smoothAltitudeByDistance(
04771         ride,
04772         cumulative,
04773         m_climbSmoothingSpin ? m_climbSmoothingSpin->value() : 50.0);
04774     const std::vector<double> gradient = ClimbDetector::computeLocalGradientPct(smoothed, cumulative,
25.0);
04775
04776     Climb newClimb = ClimbMetrics::build(
04777         ride,
04778         cumulative,
04779         smoothed,
04780         gradient,
04781         startIdx,

```

```

04782         endIdx,
04783         static_cast<int>(m_detectedClimbs.size() + 1);
04784
04785     assignClimbWeightMetrics(newClimb, resolveActiveActivityWeightKg());
04786
04787     // Persist new manual climb to DB
04788     if (m_dbManager.isOpen() && m_controller->currentActivityId() > 0)
04789     {
04790         auto db = m_dbManager.database();
04791         ClimbRepository repo(db);
04792         ClimbRecord record;
04793         record.activityId      = m_controller->currentActivityId();
04794         record.startSeconds    = newClimb.startSeconds;
04795         record.endSeconds      = newClimb.endSeconds;
04796         record.name            = newClimb.name;
04797         record.elevationGainM  = newClimb.elevationGainM;
04798         record.lengthKm        = newClimb.lengthKm;
04799         record.averageGradient = newClimb.averageGradient;
04800         record.source          = QStringLiteral("manual");
04801         record.locked          = true;
04802         newClimb.id = repo.insertClimb(record);
04803     }
04804
04805     m_detectedClimbs.push_back(newClimb);
04806     std::sort(
04807         m_detectedClimbs.begin(),
04808         m_detectedClimbs.end(),
04809         [](const Climb& a, const Climb& b)
04810         {
04811             if (std::abs(a.startSeconds - b.startSeconds) > 0.1)
04812                 return a.startSeconds < b.startSeconds;
04813             return a.endSeconds < b.endSeconds;
04814         });
04815
04816     for (int i = 0; i < static_cast<int>(m_detectedClimbs.size()); ++i)
04817         m_detectedClimbs[static_cast<size_t>(i)].name = QString("Climb %1").arg(i + 1);
04818
04819     refreshClimbViews(newClimb.startSeconds, newClimb.endSeconds);
04820 }
04821
04822 void MainWindow::triggerDetectClimbs()
04823 {
04824     if (!m_dbManager.isOpen() || m_controller->currentActivityId() <= 0)
04825     {
04826         QMessageBox::information(this, "Detect Climbs", "No activity loaded.");
04827         return;
04828     }
04829
04830     const auto& ride = m_controller->rideData();
04831     if (ride.records.empty())
04832         return;
04833
04834     // Check for locked (user-edited) climbs
04835     auto db = m_dbManager.database();
04836     ClimbRepository repo(db);
04837     if (repo.hasLockedClimbs(m_controller->currentActivityId()))
04838     {
04839         const int answer = QMessageBox::warning(
04840             this,
04841             "Detect Climbs",
04842             "This activity contains manually edited climbs.\n\n"
04843             "Re-detecting climbs will overwrite all existing climbs,\n"
04844             "including manual changes.\n\nContinue?",
04845             QMessageBox::Cancel | QMessageBox::Yes,
04846             QMessageBox::Cancel);
04847         if (answer != QMessageBox::Yes)
04848             return;
04849     }
04850
04851     // Delete all existing climbs for this activity
04852     repo.deleteClimbsForActivity(m_controller->currentActivityId());
04853
04854     // Run detection with current spin box params
04855     ClimbDetector::Config cfg;
04856     cfg.minLengthKm      = m_climbMinLengthSpin->value() : 0.15;
04857     cfg.minElevationGainM = m_climbMinGainSpin->value() : 10.0;
04858     cfg.minAverageGradient = m_climbMinGradientSpin->value() : 4.0;
04859     cfg.startGradient     = m_climbStartGradientSpin->value() : 1.5;
04860     cfg.maxDipMeters      = m_climbDipMetersSpin->value() : 10.0;
04861     cfg.maxDipDistanceMeters = m_climbDipDistanceSpin->value() : 200.0;
04862     cfg.elevationSmoothingMeters = m_climbSmoothingSpin->value() : 50.0;
04863
04864     m_detectedClimbs = ClimbDetector::detect(ride, cfg);
04865     applyWeightMetricsToClimbs(m_detectedClimbs, resolveActiveActivityWeightKg());
04866
04867     // Store to DB
04868     for (Climb& c : m_detectedClimbs)

```

```

04869     {
04870         ClimbRecord record;
04871         record.activityId      = m_controller->currentActivityId();
04872         record.startSeconds    = c.startSeconds;
04873         record.endSeconds      = c.endSeconds;
04874         record.name            = c.name;
04875         record.elevationGainM  = c.elevationGainM;
04876         record.lengthKm        = c.lengthKm;
04877         record.averageGradient = c.averageGradient;
04878         record.source          = QStringLiteral("auto");
04879         record.locked          = false;
04880         c.id = repo.insertClimb(record);
04881     }
04882
04883     refreshClimbViews();
04884     statusBar()->showMessage(
04885         QString("Detected %1 climb(s) and saved to database.").arg(m_detectedClimbs.size()),
04886         3000);
04887 }
04888
04889 void MainWindow::triggerDetectIntervals()
04890 {
04891     if (!m_dbManager.isOpen() || m_controller->currentActivityId() <= 0)
04892     {
04893         QMessageBox::information(this, "Detect Intervals", "No activity loaded.");
04894         return;
04895     }
04896
04897     auto db = m_dbManager.database();
04898     IntervalRepository repo(db);
04899
04900     // Check for locked (user-edited) intervals
04901     if (repo.hasLockedIntervals(m_controller->currentActivityId()))
04902     {
04903         const int answer = QMessageBox::warning(
04904             this,
04905             "Detect Intervals",
04906             "This activity contains manually edited intervals.\n\n"
04907             "Re-detecting intervals will overwrite all existing intervals, \n"
04908             "including manual changes.\n\nContinue?",
04909             QMessageBox::Cancel | QMessageBox::Yes,
04910             QMessageBox::Cancel);
04911         if (answer != QMessageBox::Yes)
04912             return;
04913     }
04914
04915     repo.deleteIntervalsForActivity(m_controller->currentActivityId());
04916
04917     // Run detection
04918     IntervalDetector::Config cfg;
04919     cfg.ftp = m_controller->ftp();
04920     const std::vector<Interval> detected = IntervalDetector::detect(m_controller->rideData(), cfg);
04921
04922     // Remove overlapping and duplicate intervals before saving
04923     const auto deduplicated = IntervalDetector::removeOverlappingAndDuplicates(detected);
04924
04925     for (const Interval& iv : deduplicated)
04926     {
04927         IntervalRecord record;
04928         record.activityId      = m_controller->currentActivityId();
04929         record.startSample     = static_cast<int>(std::round(iv.startSeconds));
04930         record.endSample       = static_cast<int>(std::round(iv.endSeconds));
04931         record.name            = QString::fromStdString(iv.label);
04932         record.type            = QString::fromStdString(iv.label);
04933         record.avgPower        = iv.averagePower;
04934         record.nvp             = iv.normalizedPower;
04935         record.avgHr           = iv.averageHeartRate;
04936         record.avgCadence      = iv.averageCadence;
04937         record.source          = QStringLiteral("auto");
04938         record.locked          = false;
04939         repo.insertInterval(record);
04940     }
04941
04942     updateIntervals();
04943     statusBar()->showMessage(
04944         QString("Detected %1 interval(s) (after deduplication) and saved to
04945         database.").arg(deduplicated.size()),
04946         3000);
04947 }
04948
04949 void MainWindow::editCurrentActivityProperties()
04950 {
04951     if (!m_dbManager.isOpen() || m_controller->currentActivityId() <= 0)
04952         return;
04953
04954     auto db = m_dbManager.database();
04955     ActivityRepository repo(db);

```

```

04955     Activity activity = repo.getActivity(m_controller->currentActivityId());
04956     if (!activity.isValid())
04957         return;
04958
04959     EditActivityDialog dlg(activity, this);
04960     if (dlg.exec() != QDialog::Accepted)
04961         return;
04962
04963     const Activity updated = dlg.updatedActivity();
04964     if (!repo.updateActivityProperties(activity.id, updated))
04965     {
04966         QMessageBox::critical(this, "Error", "Failed to save activity properties.");
04967         return;
04968     }
04969
04970     statusBar()->showMessage("Activity properties updated.", 2500);
04971 }
04972
04973 void MainWindow::showAbout()
04974 {
04975     AboutDialog dlg(this);
04976     dlg.exec();
04977 }

```

8.99 MainWindow.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QCheckBox>
00007 #include <QComboBox>
00008 #include <QDoubleSpinBox>
00009 #include <QDragEnterEvent>
00010 #include <QDropEvent>
00011 #include <QLabel>
00012 #include <QMainWindow>
00013 #include <QMenu>
00014 #include <QSplitter>
00015 #include <QStackedLayout>
00016 #include <QPushButton>
00017 #include <QScrollArea>
00018 #include <QSet>
00019 #include <QHash>
00020 #include <QStringList>
00021 #include <QTabWidget>
00022 #include <QTableWidget>
00023 #include <QTextEdit>
00024 #include <QVBoxLayout>
00025
00026 #include "WorkoutController.h"
00027 #include "core/zones/ZoneDefinition.h"
00028 #include "analysis/queue/AnalysisQueue.h"
00029 #include "import/ImportQueue.h"
00030 #include "core/undo/UndoManager.h"
00031 #include "database/DatabaseManager.h"
00032 #include "database/ActivityRepository.h"
00033 #include "database/AthleteRepository.h"
00034 #include "database/ClimbRepository.h"
00035 #include "database/ImportRepository.h"
00036
00037 class RideChartWidget;
00038 class FitnessChartWidget;
00039 class PowerHistogramWidget;
00040 class PowerCurveWidget;
00041 class MapRenderer;
00042 class ActivityBrowser;
00043 class AthleteHeaderWidget;
00044 class CalendarWidget;
00045 class WelcomeWidget;
00046 class ImportStatusWidget;
00047 class ImportProgressModel;
00048 class QCloseEvent;
00049 class QFileSystemWatcher;
00050 class QMimeData;
00051 class QPoint;
00052 class QTimer;
00053 class QChartView;
00054
00061 class MainWindow : public QMainWindow
00062 {
00063     Q_OBJECT
00064

```

```

00065 public:
00070     explicit MainWindow(QWidget* parent = nullptr);
00071
00072 protected:
00074     void closeEvent(QCloseEvent* event) override;
00075
00077     bool eventFilter(QObject* watched, QEvent* event) override;
00078
00080     void dragEnterEvent(QDragEnterEvent* event) override;
00081
00083     void dropEvent(QDropEvent* event) override;
00084
00085 private slots:
00086     void importActivities();
00087     void previewFitFile();
00088     void onWorkoutLoaded();
00089     void onActivityImported(int activityId);
00090     void openDatabase();
00091     void createDatabase();
00092     void manageAthletes();
00093     void backupDatabase();
00094     void openSettingsDialog();
00095     void openRecentDatabase();
00096     void onAthleteSelectionChanged(int index);
00097     void createVideo();
00098     void showAbout();
00099     void triggerDetectClimbs();
00100     void triggerDetectIntervals();
00101     void revertSelectedClimbToAuto();
00102
00103 private:
00104     void buildUI();
00105     void buildToolBar();
00106     void buildStatusBar();
00107     void setupShortcuts();
00108     void saveSettings();
00109     void loadSettings();
00110     void addRecentDatabase(const QString& path);
00111     void updateRecentDatabaseMenu();
00112     void openDatabasePath(const QString& path);
00113
00117     bool canImportActivities() const;
00118
00120     void updateImportAvailability();
00121
00123     void importFiles(const QStringList& filePaths);
00124
00126     QStringList fitFilesFromMimeData(const QMimeData* mimeData) const;
00127     void scheduleActivityBrowserRefresh();
00128
00130     void finalizeImportBatches();
00131     void refreshAthleteSelector();
00132     void resetAllZoom();
00133     void increaseChartHeight();
00134     void decreaseChartHeight();
00135     void applyChartHeight();
00136     void updateStatsLabel();
00137     void updateCharts();
00138     void applyChartSmoothing();
00139     void updateZonesTab();
00140     void updateHistogram();
00141     void updatePowerCurve();
00142     void updateFitnessChart();
00143     void updateAnalysisEmptyStates();
00144     void updateIntervals();
00145     void updateClimbingTab();
00146     void detectClimbsAndRefresh();
00147     void refreshClimbViews(double preferredStartSeconds = -1.0, double preferredEndSeconds = -1.0);
00148     void updateClimbQuarterCharts(const Climb* climb);
00149     double resolveActiveActivityWeightKg() const;
00150     void assignClimbWeightMetrics(Climb& climb, double riderWeightKg) const;
00151     void applyWeightMetricsToClimbs(std::vector<Climb>& climbs, double riderWeightKg) const;
00152     std::vector<int> selectedClimbIndicesFromTable() const;
00153     void onClimbTableContextMenuRequested(const QPoint& pos);
00154     void removeSelectedClimbs();
00155     void editSelectedClimbBoundaries();
00156     void joinSelectedClimbs();
00157     void updateClimbRowStyles();
00158     void onClimbSelectionChanged();
00159     void onClimbRowDoubleClicked(QTableWidgetItem* item);
00160     void onClimbBoundaryEdited(double oldStartSeconds, double oldEndSeconds, double newStartSeconds,
00161                                double newEndSeconds);
00161     void onNewClimbRequested(double startSeconds, double endSeconds);
00162     void onIntervalSelectionChanged();
00163     void onIntervalRowDoubleClicked(QTableWidgetItem* item);
00164     void navigateToInterval(double startSeconds, double endSeconds, bool exactZoom);
00165     void selectIntervalRowOffset(int delta);

```

```

00166     void updateIntervalRowStyles();
00167     void updateIntervalSummary(double startSeconds, double endSeconds);
00168     void clearIntervalSummary();
00169     bool ensureDatabase();
00170     bool ensureAthlete();
00171
00173     bool ensureImportReady();
00174     bool createOrOpenDefaultDatabase();
00175     QString defaultDatabasePathForAutoCreate() const;
00176     void showWelcomeScreen();
00177     void hideWelcomeScreen();
00178     void updateWelcomeScreenVisibility();
00179     int activityCount() const;
00180     void updateAthleteLabel();
00181     void updateStatusBarInfo();
00182     ColorMetric currentColorMetric() const;
00183     ColorContext buildColorContext() const;
00184     void updateColorLegend();
00185     void updateZoneAvailability();
00186     void applyChartPreset(int presetId);
00187     void editCurrentActivityProperties();
00188     double estimatedFtpFromCurrentRide() const;
00189     void applyEstimatedFtpForCurrentAthlete();
00190
00195     void importFilesInternal(const QStringList& filePaths,
00196                             bool showResultDialog,
00197                             const QString& sourceLabel);
00198     void configureFolderWatcher();
00199     QStringList monitoredDirectories() const;
00200     void scanWatchDirectories(bool initialScan);
00201     void onWatchDirectoryChanged(const QString& path);
00202
00203     // -- Undo/Redo -----
00204     UndoManager* m_undoManager = nullptr;
00205
00206     // -- Background analysis queue -----
00207     AnalysisQueue* m_analysisQueue = nullptr;
00208
00209     // -- Background import queue -----
00210     ImportQueue* m_importQueue = nullptr;
00211     ImportProgressModel* m_importProgressModel = nullptr;
00212
00213     // -- Controller -----
00214     WorkoutController* m_controller = nullptr;
00215
00216     // -- Database -----
00217     DatabaseManager m_dbManager;
00218     int m_currentAthleteId = -1;
00219     QString m_currentAthleteName;
00220
00221     // -- Menu -----
00222     QMenu* m_athletesMenu = nullptr;
00223     QAction* m_athleteLabelAct = nullptr;
00224     QAction* m_importAct = nullptr;
00225     QAction* m_toolbarImportAct = nullptr;
00226     QAction* m_toolbarSearchAct = nullptr;
00227     QAction* m_previewAct = nullptr;
00228     QMenu* m_recentDbMenu = nullptr;
00229     QString m_lastOpenDir;
00230
00231     // -- Welcome / onboarding -----
00232     WelcomeWidget* m_welcomeWidget = nullptr;
00233     bool m_firstLaunchCompleted = false;
00234
00235     // -- Folder monitoring -----
00236     QFileSystemWatcher* m_watcher = nullptr;
00237     QTimer* m_watchRescanTimer = nullptr;
00238     QTimer* m_importRefreshTimer = nullptr;
00239     bool m_watchFolderEnabled = true;
00240     QString m_watchFolderPath;
00241     QSet<QString> m_knownWatchedFitFiles;
00242
00243     // -- Toolbar/Header -----
00244     QComboBox* m_athleteCombo = nullptr;
00245     QComboBox* m_colorMetricCombo = nullptr;
00246     QComboBox* m_chartPresetCombo = nullptr;
00247     QComboBox* m_powerSmoothingCombo = nullptr;
00248     QCheckBox* m_autoSmoothingCheck = nullptr;
00249     ImportStatusWidget* m_importStatusWidget = nullptr;
00250
00251     // -- Status bar -----
00252     QLabel* m_dbStatusLabel = nullptr;
00253     QLabel* m_athleteStatusLabel = nullptr;
00254     QLabel* m_activityCountLabel = nullptr;
00255
00256     // -- Stats panel -----
00257     AthleteHeaderWidget* m_athleteHeader = nullptr;

```



```

00258
00259 // -- Tab widget -----
00260 QTabWidget* m_tabWidget = nullptr;
00261 QTabWidget* m_analysisTabWidget = nullptr;
00262 QStackedLayout* m_activityTabStack = nullptr;
00263 QStackedLayout* m_zonesTabStack = nullptr;
00264 QStackedLayout* m_histogramTabStack = nullptr;
00265 QStackedLayout* m_powerCurveTabStack = nullptr;
00266 QStackedLayout* m_calendarTabStack = nullptr;
00267 QStackedLayout* m_fitnessTabStack = nullptr;
00268 QStackedLayout* m_climbingTabStack = nullptr;
00269
00270 static constexpr int kTabActivities = 0;
00271 static constexpr int kTabAnalysis = 1;
00272
00273 static constexpr int kAnalysisTabCharts = 0;
00274 static constexpr int kAnalysisTabZones = 1;
00275 static constexpr int kAnalysisTabHistogram = 2;
00276 static constexpr int kAnalysisTabPDC = 3;
00277 static constexpr int kAnalysisTabCalendar = 4;
00278 static constexpr int kAnalysisTabFitness = 5;
00279 static constexpr int kAnalysisTabClimbing = 6;
00280
00281 // Charts tab
00282 QScrollArea* m_chartScroll = nullptr;
00283 QWidget* m_chartsContainer = nullptr;
00284 QVBoxLayout* m_chartsLayout = nullptr;
00285 QSplitter* m_chartMapSplit = nullptr;
00286 int m_chartHeight = 220;
00287 RideChartWidget* m_powerChart = nullptr;
00288 RideChartWidget* m_hrChart = nullptr;
00289 RideChartWidget* m_cadenceChart = nullptr;
00290 RideChartWidget* m_speedChart = nullptr;
00291 RideChartWidget* m_altitudeChart = nullptr;
00292
00293 // Climbing tab charts
00294 RideChartWidget* m_climbAltitudeChart = nullptr;
00295 RideChartWidget* m_climbPowerChart = nullptr;
00296 RideChartWidget* m_climbHrChart = nullptr;
00297 RideChartWidget* m_climbCadenceChart = nullptr;
00298 RideChartWidget* m_climbSpeedChart = nullptr;
00299 QChartView* m_climbQuarterPowerChart = nullptr;
00300 QChartView* m_climbQuarterHrChart = nullptr;
00301 QChartView* m_climbQuarterCadenceChart = nullptr;
00302
00303 // Chart controls
00304 QPushButton* m_fitChartsButton = nullptr;
00305 QPushButton* m_chartHeightIncreaseButton = nullptr;
00306 QPushButton* m_chartHeightDecreaseButton = nullptr;
00307 QPushButton* m_useEstimatedFtpButton = nullptr;
00308 QWidget* m_colorLegendWidget = nullptr;
00309 QHBoxLayout* m_colorLegendLayout = nullptr;
00310
00311 // Metric visibility checkboxes
00312 QCheckBox* m_showPower = nullptr;
00313 QCheckBox* m_showHR = nullptr;
00314 QCheckBox* m_showCadence = nullptr;
00315 QCheckBox* m_showSpeed = nullptr;
00316 QCheckBox* m_showAltitude = nullptr;
00317
00318 // Zones tab
00319 QTableWidgetItem* m_zonesTable = nullptr;
00320
00321 // Histogram tab
00322 PowerHistogramWidget* m_histogram = nullptr;
00323
00324 // Power curve tab
00325 PowerCurveWidget* m_pdcWidget = nullptr;
00326
00327 // Fitness tab
00328 FitnessChartWidget* m_fitnessChart = nullptr;
00329
00330 // Integrated activity panels
00331 QTableWidgetItem* m_intervalsTable = nullptr;
00332 QTableWidgetItem* m_lapsTable = nullptr;
00333 QTextEdit* m_activityNotesView = nullptr;
00334 QCheckBox* m_followIntervalSelectionCheck = nullptr;
00335 QLabel* m_intervalSummaryLabel = nullptr;
00336
00337 // Climbing tab
00338 QTableWidgetItem* m_climbsTable = nullptr;
00339 QLabel* m_climbSummaryLabel = nullptr;
00340 QDoubleSpinBox* m_climbMinLengthSpin = nullptr;
00341 QDoubleSpinBox* m_climbMinGainSpin = nullptr;
00342 QDoubleSpinBox* m_climbMinGradientSpin = nullptr;
00343 QDoubleSpinBox* m_climbStartGradientSpin = nullptr;
00344 QDoubleSpinBox* m_climbDipMetersSpin = nullptr;

```

```

00345     QDoubleSpinBox* m_climbDipDistanceSpin = nullptr;
00346     QDoubleSpinBox* m_climbSmoothingSpin = nullptr;
00347     QCheckBox* m_climbOverlayEnabledCheck = nullptr;
00348     QComboBox* m_climbOverlayMetricCombo = nullptr;
00349     std::vector<Climb> m_detectedClimbs;
00350     bool m_suppressClimbAutoZoom = false;
00351
00352     // Integrated map panel
00353     MapRenderer* m_mapRenderer = nullptr;
00354     QComboBox* m_mapStyleCombo = nullptr;
00355     QCheckBox* m_mapAutoContrastCheck = nullptr;
00356
00357     // Calendar tab
00358     CalendarWidget* m_calendarWidget = nullptr;
00359
00360     // Activities tab
00361     ActivityBrowser* m_activityBrowser = nullptr;
00362
00363     struct ImportBatchSummary
00364     {
00365         int queued = 0;
00366         int imported = 0;
00367         int duplicates = 0;
00368         int failed = 0;
00369         bool showResultDialog = false;
00370         QString sourceLabel;
00371         QStringList errors;
00372     };
00373
00374     QHash<QString, ImportBatchSummary> m_importBatchSummaries;
00375 };

```

8.100 NavigationSidebar.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "NavigationSidebar.h"
00005
00006 #include <QListWidget>
00007 #include <QVBoxLayout>
00008
00009 NavigationSidebar::NavigationSidebar(QWidget* parent)
00010     : QWidget(parent)
00011 {
00012     auto* layout = new QVBoxLayout(this);
00013     layout->setContentsMargins(0, 0, 0, 0);
00014
00015     m_list = new QListWidget(this);
00016     m_list->addItem("Dashboard");
00017     m_list->addItem("Calendar");
00018     m_list->addItem("Activities");
00019     m_list->addItem("Analysis");
00020     m_list->addItem("Athletes");
00021     m_list->addItem("Settings");
00022     m_list->setCurrentRow(static_cast<int>(Page::Activities));
00023     m_list->setFixedWidth(170);
00024     layout->addWidget(m_list);
00025
00026     connect(m_list, &QListWidget::currentRowChanged, this, [this](int row) {
00027         if (row < 0) return;
00028         emit pageSelected(static_cast<Page>(row));
00029     });
00030 }
00031
00032 void NavigationSidebar::setCurrentPage(Page page)
00033 {
00034     if (!m_list) return;
00035     m_list->setCurrentRow(static_cast<int>(page));
00036 }
00037
00038 NavigationSidebar::Page NavigationSidebar::currentPage() const
00039 {
00040     if (!m_list) return Page::Activities;
00041     const int row = m_list->currentRow();
00042     return row < 0 ? Page::Activities : static_cast<Page>(row);
00043 }

```

8.101 NavigationSidebar.h

```

00001 // SPDX-License-Identifier: GPL-3

```

```

00002
00003
00004 #pragma once
00005
00006 #include <QWidget>
00007
00008 class QListWidget;
00009 class QListWidgetItem;
00010
00014 class NavigationSidebar : public QWidget
00015 {
00016     Q_OBJECT
00017
00018 public:
00019     enum class Page
00020     {
00021         Dashboard = 0,
00022         Calendar,
00023         Activities,
00024         Analysis,
00025         Athletes,
00026         Settings
00027     };
00028
00029     explicit NavigationSidebar(QWidget* parent = nullptr);
00030
00031     void setCurrentPage(Page page);
00032
00033     Page currentPage() const;
00034
00035 signals:
00036     // Emitted when navigation page selection changes.
00037     void pageSelected(NavigationSidebar::Page page);
00038
00039 private:
00040     QListWidget* m_list = nullptr;
00041 };

```

8.102 PlannedWorkoutDialog.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "PlannedWorkoutDialog.h"
00005 #include "core/settings/DateFormatter.h"
00006
00007 #include <QDate>
00008 #include <QDateEdit>
00009 #include <QDialogButtonBox>
00010 #include <QDoubleSpinBox>
00011 #include <QFormLayout>
00012 #include <QLineEdit>
00013 #include <QTextEdit>
00014 #include <QVBoxLayout>
00015
00016 PlannedWorkoutDialog::PlannedWorkoutDialog(QWidget* parent)
00017     : PlannedWorkoutDialog(PlannedWorkoutDraft{}, parent)
00018 {}
00019
00020 PlannedWorkoutDialog::PlannedWorkoutDialog(const PlannedWorkoutDraft& draft, QWidget* parent)
00021     : QDialog(parent)
00022 {
00023     setWindowTitle("Planned Workout");
00024     setMinimumWidth(420);
00025
00026     auto* layout = new QVBoxLayout(this);
00027     auto* form = new QFormLayout;
00028
00029     m_dateEdit = new QDateEdit(this);
00030     m_dateEdit->setCalendarPopup(true);
00031     m_dateEdit->setDisplayFormat(DateFormatter::qtDatePattern());
00032     m_dateEdit->setDate(draft.date.isValid() ? draft.date : QDate::currentDate());
00033
00034     m_titleEdit = new QLineEdit(draft.title, this);
00035     m_typeEdit = new QLineEdit(draft.type, this);
00036
00037     m_durationSpin = new QDoubleSpinBox(this);
00038     m_durationSpin->setRange(0.0, 12.0);
00039     m_durationSpin->setDecimals(2);
00040     m_durationSpin->setSuffix(" h");
00041     m_durationSpin->setValue(draft.durationSeconds / 3600.0);
00042
00043     m_tssSpin = new QDoubleSpinBox(this);
00044     m_tssSpin->setRange(0.0, 1000.0);

```

```

00045     m_tssSpin->setDecimals(1);
00046     m_tssSpin->setValue(draft.targetTss);
00047
00048     m_notesEdit = new QTextEdit(this);
00049     m_notesEdit->setMinimumHeight(90);
00050     m_notesEdit->setPlainText(draft.notes);
00051
00052     m_id = draft.id;
00053
00054     form->addRow("Date:", m_dateEdit);
00055     form->addRow("Title:", m_titleEdit);
00056     form->addRow("Type:", m_typeEdit);
00057     form->addRow("Duration:", m_durationSpin);
00058     form->addRow("Target TSS:", m_tssSpin);
00059     form->addRow("Notes:", m_notesEdit);
00060
00061     layout->addLayout(form);
00062
00063     auto* buttons = new QDialogButtonBox(QDialogButtonBox::Ok | QDialogButtonBox::Cancel, this);
00064     connect(buttons, &QDialogButtonBox::accepted, this, &QDialog::accept);
00065     connect(buttons, &QDialogButtonBox::rejected, this, &QDialog::reject);
00066     layout->addWidget(buttons);
00067 }
00068
00069 PlannedWorkoutDraft PlannedWorkoutDialog::draft() const
00070 {
00071     PlannedWorkoutDraft out;
00072     out.id = m_id;
00073     out.date = m_dateEdit ? m_dateEdit->date() : QDate();
00074     out.title = m_titleEdit ? m_titleEdit->text().trimmed() : QString();
00075     out.type = m_typeEdit ? m_typeEdit->text().trimmed() : QString();
00076     out.durationSeconds = (m_durationSpin ? m_durationSpin->value() : 0.0) * 3600.0;
00077     out.targetTss = m_tssSpin ? m_tssSpin->value() : 0.0;
00078     out.notes = m_notesEdit ? m_notesEdit->toPlainText().trimmed() : QString();
00079     return out;
00080 }

```

8.103 PlannedWorkoutDialog.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QDialog>
00007
00008 #include <QDate>
00009 #include <QString>
00010
00014 struct PlannedWorkoutDraft
00015 {
00016     // Unique identifier (-1 if new).
00017     int id = -1;
00018
00019     // Planned workout date.
00020     QDate date;
00021
00022     // Workout title.
00023     QString title;
00024
00025     // Workout type (for example Recovery, Threshold).
00026     QString type;
00027
00028     // Planned duration in seconds.
00029     double durationSeconds = 0.0;
00030
00031     // Target Training Stress Score.
00032     double targetTss = 0.0;
00033
00034     // Additional notes.
00035     QString notes;
00036 };
00037
00038 class QDateEdit;
00039 class QDoubleSpinBox;
00040 class QLineEdit;
00041 class QTextEdit;
00042
00046 class PlannedWorkoutDialog : public QDialog
00047 {
00048     Q_OBJECT
00049
00050 public:
00051     explicit PlannedWorkoutDialog(QWidget* parent = nullptr);

```

```

00052
00053     explicit PlannedWorkoutDialog(const PlannedWorkoutDraft& draft, QWidget* parent = nullptr);
00054
00055     PlannedWorkoutDraft draft() const;
00056
00057 private:
00058     QDateEdit* m_dateEdit = nullptr;
00059     QLineEdit* m_titleEdit = nullptr;
00060     QLineEdit* m_typeEdit = nullptr;
00061     QDoubleSpinBox* m_durationSpin = nullptr;
00062     QDoubleSpinBox* m_tssSpin = nullptr;
00063     QTextEdit* m_notesEdit = nullptr;
00064     int m_id = -1;
00065 };

```

8.104 WelcomeWidget.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "WelcomeWidget.h"
00005
00006 #include <QFrame>
00007 #include <QHBoxLayout>
00008 #include <QLabel>
00009 #include <QPushButton>
00010 #include <QVBoxLayout>
00011
00012 WelcomeWidget::WelcomeWidget(QWidget* parent)
00013     : QWidget(parent)
00014 {
00015     auto* rootLayout = new QVBoxLayout(this);
00016     rootLayout->setContentsMargins(0, 12, 0, 12);
00017
00018     auto* card = new QFrame(this);
00019     card->setObjectName("welcomeCard");
00020     card->setStyleSheet (
00021         "QFrame#welcomeCard {
00022             background: qlineargradient(x1:0, y1:0, x2:1, y2:1, stop:0 #f8fafc, stop:1 #eef2ff);
00023             border: 1px solid #cbd5e1;
00024             border-radius: 12px;
00025         }"
00026     );
00027
00028     auto* cardLayout = new QVBoxLayout(card);
00029     cardLayout->setContentsMargins(28, 24, 28, 24);
00030     cardLayout->setSpacing(12);
00031
00032     auto* title = new QLabel("Drop FIT Files Here", card);
00033     title->setStyleSheet("font-size: 28px; font-weight: 700; color: #0f172a;");
00034     title->setAlignment(Qt::AlignCenter);
00035     cardLayout->addWidget(title);
00036
00037     m_subtitleLabel = new QLabel(card);
00038     m_subtitleLabel->setWordWrap(true);
00039     m_subtitleLabel->setAlignment(Qt::AlignCenter);
00040     m_subtitleLabel->setStyleSheet("font-size: 14px; color: #334155;");
00041     cardLayout->addWidget(m_subtitleLabel);
00042
00043     auto* buttonRow = new QHBoxLayout;
00044     buttonRow->setSpacing(10);
00045
00046     m_importButton = new QPushButton("Import Activity", card);
00047     m_openDbButton = new QPushButton("Open Database", card);
00048     m_createDbButton = new QPushButton("Create Database", card);
00049
00050     m_importButton->setMinimumHeight(36);
00051     m_openDbButton->setMinimumHeight(36);
00052     m_createDbButton->setMinimumHeight(36);
00053
00054     m_importButton->setStyleSheet (
00055         "QPushButton {
00056             background: #1d4ed8; color: white; border: none; border-radius: 8px;
00057             padding: 8px 14px; font-weight: 600;
00058         }"
00059         "QPushButton:hover { background: #1e40af; }"
00060     );
00061
00062     buttonRow->addStretch(1);
00063     buttonRow->addWidget(m_importButton);
00064     buttonRow->addWidget(m_openDbButton);
00065     buttonRow->addWidget(m_createDbButton);
00066     buttonRow->addStretch(1);
00067

```

```

00068     cardLayout->addLayout(buttonRow);
00069
00070     rootLayout->addStretch(1);
00071     rootLayout->addWidget(card);
00072     rootLayout->addStretch(1);
00073
00074     connect(m_importButton, &QPushButton::clicked, this, &WelcomeWidget::importRequested);
00075     connect(m_openDbButton, &QPushButton::clicked, this, &WelcomeWidget::openDatabaseRequested);
00076     connect(m_createDbButton, &QPushButton::clicked, this, &WelcomeWidget::createDatabaseRequested);
00077
00078     setFirstLaunch(true);
00079 }
00080
00081 void WelcomeWidget::setFirstLaunch(bool firstLaunch)
00082 {
00083     if (!m_subtitleLabel)
00084         return;
00085
00086     if (firstLaunch)
00087     {
00088         m_subtitleLabel->setText(
00089             "Get started in seconds: drop one or more .fit files to create your first activity
00090             automatically.");
00091     }
00092     else
00093     {
00094         m_subtitleLabel->setText(
00095             "No activities yet for this database. Import a .fit file to begin analysis.");
00096     }
00097 }

```

8.105 WelcomeWidget.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QWidget>
00007
00008 class QLabel;
00009 class QPushButton;
00010
00017 class WelcomeWidget : public QWidget
00018 {
00019     Q_OBJECT
00020
00021 public:
00026     explicit WelcomeWidget(QWidget* parent = nullptr);
00027
00032     void setFirstLaunch(bool firstLaunch);
00033
00034 signals:
00036     void importRequested();
00037
00039     void openDatabaseRequested();
00040
00042     void createDatabaseRequested();
00043
00044 private:
00045     QLabel* m_subtitleLabel = nullptr;
00046     QPushButton* m_importButton = nullptr;
00047     QPushButton* m_openDbButton = nullptr;
00048     QPushButton* m_createDbButton = nullptr;
00049 };

```

8.106 WorkoutController.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "WorkoutController.h"
00005
00006 #include <QCryptographicHash>
00007 #include <QFile>
00008 #include <QFileInfo>
00009 #include <QSqlQuery>
00010
00011 #include <algorithm>
00012 #include <cmath>
00013

```

```

00014 #include "core/ActivityAnalysisFlags.h"
00015 #include "core/AnalysisVersions.h"
00016 #include "database/ActivityAnalysisSettingsRepository.h"
00017 #include "utils/UuidGenerator.h"
00018 #include "analysis/ClimbDetector.h"
00019 #include "analysis/ClimbMetrics.h"
00020 #include "analysis/TrainingLoad.h"
00021 #include "analysis/TrainingMetrics.h"
00022 #include "core/zones/ZoneCalculator.h"
00023 #include "database/ClimbRepository.h"
00024 #include "database/DatabaseManager.h"
00025 #include "database/IntervalRepository.h"
00026 #include "fit/FitParser.h"
00027 #include "model/RideDataSerializer.h"
00028
00029 namespace
00030 {
00031     QDate activityDateForFtp(const Activity& activity)
00032     {
00033         const QString rawDate = !activity.startTime.isEmpty()
00034             ? activity.startTime.left(10)
00035             : activity.importedAt.left(10);
00036         return QDate::fromString(rawDate, Qt::ISODate);
00037     }
00038
00039     int resolveAthleteFtpForDate(AthleteRepository& repo, int athleteId, const QDate& date)
00040     {
00041         if (date.isValid())
00042         {
00043             const int historicalFtp = repo.getFtpForDate(athleteId, date);
00044             if (historicalFtp > 0)
00045                 return historicalFtp;
00046         }
00047
00048         const Athlete athlete = repo.getAthlete(athleteId);
00049         return athlete.ftpWatts > 0 ? athlete.ftpWatts : 250;
00050     }
00051 }
00052
00053 WorkoutController::WorkoutController(QObject* parent)
00054     : QObject(parent)
00055 {}
00056
00057 void WorkoutController::setFTP(double ftp)
00058 {
00059     m_ftp = ftp;
00060     if (!m_rideData.records.empty())
00061         reanalyze();
00062     emit ftpChanged(ftp);
00063 }
00064
00065 void WorkoutController::setDatabaseManager(DatabaseManager* dbManager)
00066 {
00067     m_dbManager = dbManager;
00068 }
00069
00070 void WorkoutController::setCurrentAthlete(int athleteId)
00071 {
00072     m_currentAthleteId = athleteId;
00073     if (m_dbManager && m_dbManager->isOpen() && athleteId > 0)
00074     {
00075         auto db = m_dbManager->database();
00076         QDate ftpDate = QDate::currentDate();
00077         if (m_currentActivityId > 0)
00078         {
00079             ActivityRepository actRepo(db);
00080             const Activity activity = actRepo.getActivity(m_currentActivityId);
00081             const QDate activityDate = activityDateForFtp(activity);
00082             if (activityDate.isValid())
00083                 ftpDate = activityDate;
00084         }
00085
00086         AthleteRepository repo(db);
00087         setFTP(static_cast<double>(resolveAthleteFtpForDate(repo, athleteId, ftpDate)));
00088     }
00089 }
00090
00091 bool WorkoutController::loadFile(const QString& path, QString& errorOut)
00092 {
00093     try
00094     {
00095         FitParser parser;
00096         m_rideData = parser.load(path);
00097         m_currentActivityId = -1;
00098         reanalyze();
00099         emit workoutLoaded();
00100         return true;

```

```

00101     }
00102     catch (const std::exception& ex)
00103     {
00104         errorOut = QString::fromLocal8Bit(ex.what());
00105         return false;
00106     }
00107 }
00108
00109 int WorkoutController::importFile(const QString& path, QString& errorOut,
00110                                   bool allowTimeOverlap,
00111                                   bool runAnalysis,
00112                                   const QString& importSource,
00113                                   DuplicateActivityInfo* duplicateInfo,
00114                                   ImportResult* result)
00115 {
00116     if (!m_dbManager || !m_dbManager->isOpen())
00117     {
00118         errorOut = "No database is open.";
00119         if (result) *result = ImportResult::Failed;
00120         return -1;
00121     }
00122     if (m_currentAthleteId < 0)
00123     {
00124         errorOut = "No athlete selected.";
00125         if (result) *result = ImportResult::Failed;
00126         return -1;
00127     }
00128
00129     RideData ride;
00130     try
00131     {
00132         FitParser parser;
00133         ride = parser.load(path);
00134     }
00135     catch (const std::exception& ex)
00136     {
00137         errorOut = QString::fromLocal8Bit(ex.what());
00138         if (result) *result = ImportResult::Failed;
00139         return -1;
00140     }
00141
00142     QFile f(path);
00143     QString fitHash;
00144     if (f.open(QIODevice::ReadOnly))
00145     {
00146         fitHash = QCryptographicHash::hash(f.readAll(), QCryptographicHash::Sha256)
00147                 .toHex();
00148         f.close();
00149     }
00150
00151     // Level 2: Time-overlap detection
00152     if (!allowTimeOverlap && !ride.activityStartTimeUtc.isEmpty())
00153     {
00154         auto db = m_dbManager->database();
00155         QSqlQuery q(db);
00156         q.prepare(
00157             "SELECT id, activity_start_time, end_time, duration_sec "
00158             "FROM activities WHERE athlete_id=:aid AND activity_start_time=:st LIMIT 1");
00159         q.bindValue(":aid", m_currentAthleteId);
00160         q.bindValue(":st", ride.activityStartTimeUtc);
00161         if (q.exec() && q.next())
00162         {
00163             if (duplicateInfo)
00164             {
00165                 duplicateInfo->existingActivityId = q.value(0).toInt();
00166                 duplicateInfo->existingStartUtc = q.value(1).toString();
00167                 duplicateInfo->existingDurationSeconds = q.value(3).toDouble();
00168                 duplicateInfo->newStartUtc = ride.activityStartTimeUtc;
00169                 duplicateInfo->newDurationSeconds =
00170                     ride.records.empty() ? 0.0 : ride.records.back().elapsedSeconds;
00171             }
00172
00173             // Return TimeOverlap result with descriptive error message
00174             errorOut = QString("A duplicate activity with the same start time already exists");
00175             if (result) *result = ImportResult::TimeOverlap;
00176             return -1;
00177         }
00178     }
00179
00180     WorkoutAnalyzer analyzer;
00181     WorkoutStatistics stats = analyzer.analyze(ride);
00182     const double np = TrainingLoad::normalizedPower(ride);
00183
00184     auto db = m_dbManager->database();
00185     ActivityRepository actRepo(db);
00186     ImportRepository impRepo(db);
00187

```



```

00188     const int activityId = RideDataSerializer::saveRideToDatabase(
00189         ride, stats, m_currentAthleteId, path, fitHash,
00190         np, *m_dbManager, actRepo, impRepo, &errorOut);
00191
00192     if (activityId < 0)
00193     {
00194         if (result) *result = ImportResult::Failed;
00195         return -1;
00196     }
00197
00198     // Store fingerprint (sport + start_time + rounded duration + rounded distance)
00199     {
00200         QCryptographicHash fpHasher(QCryptographicHash::Sha256);
00201         fpHasher.addData(QStringLiteral("Cycling").toUtf8());
00202         fpHasher.addData(ride.activityStartTimeUtc.toUtf8());
00203         fpHasher.addData(QString::number(static_cast<int>(stats.durationSeconds)).toUtf8());
00204         fpHasher.addData(QString::number(static_cast<int>(stats.totalDistanceKm * 1000.0)).toUtf8());
00205         const QString fingerprint = fpHasher.result().toHex();
00206         actRepo.updateFingerprint(activityId, fingerprint);
00207     }
00208
00209     // Store import source
00210     if (!importSource.isEmpty())
00211         actRepo.setImportSource(activityId, importSource);
00212
00213     emit activityImported(activityId);
00214
00215     if (runAnalysis)
00216         loadActivity(activityId, errorOut);
00217     else if (m_analysisQueue)
00218         m_analysisQueue->enqueue(activityId);
00219
00220     if (result) *result = ImportResult::Success;
00221     return activityId;
00222 }
00223
00224 bool WorkoutController::loadActivity(int activityId, QString& errorOut)
00225 {
00226     if (!m_dbManager || !m_dbManager->isOpen())
00227     {
00228         errorOut = "No database is open.";
00229         return false;
00230     }
00231
00232     m_rideData = RideDataSerializer::loadRideFromDatabase(activityId, *m_dbManager);
00233     if (m_rideData.records.empty())
00234     {
00235         errorOut = "Activity has no samples in the database.";
00236         return false;
00237     }
00238
00239     m_currentActivityId = activityId;
00240
00241     if (m_currentAthleteId > 0)
00242     {
00243         auto db = m_dbManager->database();
00244         ActivityRepository actRepo(db);
00245         const Activity activity = actRepo.getActivity(activityId);
00246         const QDate ftpDate = activityDateForFtp(activity);
00247
00248         AthleteRepository repo(db);
00249         m_ftp = resolveAthleteFtpForDate(
00250             repo,
00251             m_currentAthleteId,
00252             ftpDate.isValid() ? ftpDate : QDate::currentDate());
00253     }
00254
00255     reanalyze();
00256     loadOrGenerateIntervals();
00257     loadOrGenerateClimbs();
00258     emit workoutLoaded();
00259     return true;
00260 }
00261
00262 void WorkoutController::reanalyze()
00263 {
00264     m_statistics = m_analyzer.analyze(m_rideData);
00265     const TrainingMetricsSummary metrics = TrainingMetrics::compute(
00266         m_rideData,
00267         m_ftp,
00268         m_statistics.durationSeconds);
00269
00270     m_np = metrics.normalizedPower;
00271     m_if = metrics.intensityFactor;
00272     m_tss = metrics.trainingStressScore;
00273     m_vi = metrics.variabilityIndex;
00274     m_ef = metrics.efficiencyFactor;

```

```

00275
00276     ColorContext colorContext;
00277     colorContext.ftp = static_cast<int>(m_ftp);
00278     m_powerZones = ZoneCalculator::computeDistribution(
00279         m_rideData,
00280         ColorMetric::Power,
00281         colorContext);
00282     m_pdcPoints = PowerCurve::computeStandard(m_rideData);
00283
00284     IntervalDetector::Config cfg;
00285     cfg.ftp = m_ftp;
00286     m_intervals = IntervalDetector::detect(m_rideData, cfg);
00287
00288     ClimbDetector::Config climbCfg;
00289     m_climbs = ClimbDetector::detect(m_rideData, climbCfg);
00290 }
00291
00292 void WorkoutController::loadOrGenerateIntervals()
00293 {
00294     if (!m_dbManager || !m_dbManager->isOpen() || m_currentActivityId <= 0)
00295         return;
00296
00297     auto db = m_dbManager->database();
00298     IntervalRepository repo(db);
00299     const QList<IntervalRecord> stored = repo.listIntervalsForActivity(m_currentActivityId);
00300
00301     if (!stored.isEmpty())
00302     {
00303         m_intervals.clear();
00304         m_intervals.reserve(static_cast<size_t>(stored.size()));
00305
00306         for (const IntervalRecord& record : stored)
00307         {
00308             Interval iv;
00309             iv.startSeconds = record.startSample;
00310             iv.endSeconds = record.endSample;
00311             iv.durationSeconds = std::max(0.0, iv.endSeconds - iv.startSeconds);
00312             iv.averagePower = record.avgPower;
00313             iv.normalizedPower = record.np;
00314             iv.averageHeartRate = record.avgHr;
00315             iv.averageCadence = record.avgCadence;
00316             iv.label = record.type.isEmpty() ? "Interval" : record.type.toStdString();
00317             m_intervals.push_back(iv);
00318         }
00319         return;
00320     }
00321
00322     if (m_intervals.empty())
00323         return;
00324
00325     for (const Interval& iv : m_intervals)
00326     {
00327         IntervalRecord record;
00328         record.activityId = m_currentActivityId;
00329         record.startSample = static_cast<int>(std::round(iv.startSeconds));
00330         record.endSample = static_cast<int>(std::round(iv.endSeconds));
00331         record.name = QString::fromStdString(iv.label);
00332         record.type = QString::fromStdString(iv.label);
00333         record.avgPower = iv.averagePower;
00334         record.np = iv.normalizedPower;
00335         record.avgHr = iv.averageHeartRate;
00336         record.avgCadence = iv.averageCadence;
00337         record.source = QStringLiteral("auto");
00338         record.locked = false;
00339         record.deleted = false;
00340         record.algorithmVersion = AnalysisVersions::IntervalDetection;
00341         record.uuid = UuidGenerator::create();
00342         repo.insertInterval(record);
00343     }
00344
00345     // Mark HasIntervals flag
00346     {
00347         auto db2 = m_dbManager->database();
00348         ActivityRepository actRepo2(db2);
00349         const Activity actForFlags = actRepo2.getActivity(m_currentActivityId);
00350         actRepo2.updateAnalysisFlags(m_currentActivityId,
00351             actForFlags.analysisFlags |
00352             ActivityAnalysisFlags::HasIntervals);
00353     }
00354 }
00355
00356 void WorkoutController::reloadClimbsFromDatabase()
00357 {
00358     if (m_dbManager && m_dbManager->isOpen() && m_currentActivityId > 0)
00359     {
00360         m_climbs.clear();
00361         loadOrGenerateClimbs();
00362     }
00363 }

```

```

00361     }
00362 }
00363
00364 void WorkoutController::loadOrGenerateClimbs()
00365 {
00366     if (!m_dbManager || !m_dbManager->isOpen() || m_currentActivityId <= 0)
00367         return;
00368
00369     auto db = m_dbManager->database();
00370     ClimbRepository repo(db);
00371     ActivityRepository actRepo(db);
00372
00373     // Use HasClimbs flag as an authoritative "analysis was ever run" sentinel.
00374     // This prevents auto re-detect when the user has deliberately deleted all climbs.
00375     const Activity activity = actRepo.getActivity(m_currentActivityId);
00376     const bool analysisEverRun = (activity.analysisFlags & ActivityAnalysisFlags::HasClimbs) != 0
00377         || repo.hasClimbsEver(m_currentActivityId);
00378
00379     if (analysisEverRun)
00380     {
00381         // Load non-deleted climbs from DB; rebuild full metrics from ride data.
00382         const QList<ClimbRecord> stored = repo.climbsForActivity(m_currentActivityId);
00383         m_climbs.clear();
00384         m_climbs.reserve(static_cast<size_t>(stored.size()));
00385
00386         if (m_rideData.records.empty())
00387             return;
00388
00389         // Build analysis vectors once with default config
00390         ClimbDetector::Config cfg;
00391         const std::vector<double> cumulative =
00392             ClimbDetector::buildCumulativeDistanceMeters(m_rideData);
00393         const std::vector<double> smoothed = ClimbDetector::smoothAltitudeByDistance(
00394             m_rideData, cumulative, cfg.elevationSmoothingMeters);
00395         const std::vector<double> gradient = ClimbDetector::computeLocalGradientPct(
00396             smoothed, cumulative, cfg.gradientWindowHalfMeters);
00397
00398         auto nearestIdx = [this](double sec) -> int
00399         {
00400             auto it = std::lower_bound(
00401                 m_rideData.records.begin(), m_rideData.records.end(), sec,
00402                 [](const RideRecord& r, double s) { return r.elapsedSeconds < s; });
00403             if (it == m_rideData.records.end())
00404                 return static_cast<int>(m_rideData.records.size() - 1);
00405             int idx = static_cast<int>(std::distance(m_rideData.records.begin(), it));
00406             if (idx > 0)
00407             {
00408                 const double curr =
00409                     std::abs(m_rideData.records[static_cast<size_t>(idx)].elapsedSeconds - sec);
00410                 const double prev = std::abs(m_rideData.records[static_cast<size_t>(idx -
00411                     1)].elapsedSeconds - sec);
00412                 if (prev < curr) --idx;
00413             }
00414             return idx;
00415         };
00416
00417         for (int ordinal = 0; ordinal < stored.size(); ++ordinal)
00418         {
00419             const ClimbRecord& record = stored[ordinal];
00420             int startIdx = nearestIdx(record.startSeconds);
00421             int endIdx = nearestIdx(record.endSeconds);
00422             if (endIdx < startIdx) std::swap(startIdx, endIdx);
00423             if (endIdx <= startIdx)
00424                 endIdx = std::min(static_cast<int>(m_rideData.records.size() - 1, startIdx + 1);
00425
00426             Climb c = ClimbMetrics::build(m_rideData, cumulative, smoothed, gradient,
00427                 startIdx, endIdx, ordinal + 1);
00428             c.id = record.id;
00429             if (!record.name.isEmpty())
00430                 c.name = record.name;
00431             m_climbs.push_back(c);
00432         }
00433         return;
00434     }
00435
00436     // First time: m_climbs was populated by reanalyze(). Check for resurrection, then persist.
00437     const QList<ClimbRecord> deletedClimbs = repo.deletedClimbsForActivity(m_currentActivityId);
00438
00439     for (Climb& c : m_climbs)
00440     {
00441         // Resurrection prevention: skip if overlaps a previously deleted climb
00442         bool overlapsDeleted = false;
00443         for (const ClimbRecord& d : deletedClimbs)
00444         {
00445             if (c.startSeconds < d.endSeconds && c.endSeconds > d.startSeconds)
00446             {
00447                 overlapsDeleted = true;
00448             }
00449         }
00450     }

```

```

00445         break;
00446     }
00447 }
00448 if (overlapsDeleted) continue;
00449
00450 ClimbRecord record;
00451 record.activityId      = m_currentActivityId;
00452 record.uuid           = UuidGenerator::create();
00453 record.startSeconds    = c.startSeconds;
00454 record.endSeconds      = c.endSeconds;
00455 record.originalStartSeconds = c.startSeconds;
00456 record.originalEndSeconds = c.endSeconds;
00457 record.name           = c.name;
00458 record.elevationGainM  = c.elevationGainM;
00459 record.lengthKm        = c.lengthKm;
00460 record.averageGradient = c.averageGradient;
00461 record.avgPower         = c.averagePower;
00462 record.np               = c.normalizedPower;
00463 record.avgHr            = c.averageHeartRate;
00464 record.avgCadence       = c.averageCadence;
00465 record.vam              = c.vam;
00466 record.source           = QStringLiteral("auto");
00467 record.locked           = false;
00468 record.algorithmVersion = AnalysisVersions::ClimbDetection;
00469 c.id = repo.insertClimb(record);
00470 }
00471
00472 // Mark HasClimbs flag
00473 if (!m_climbs.empty())
00474     actRepo.updateAnalysisFlags(m_currentActivityId,
00475                                 activity.analysisFlags | ActivityAnalysisFlags::HasClimbs);
00476 }

```

8.107 WorkoutController.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QDate>
00007 #include <QObject>
00008 #include <QString>
00009
00010 #include "analysis/IntervalDetector.h"
00011 #include "analysis/ClimbDetector.h"
00012 #include "analysis/PowerCurve.h"
00013 #include "analysis/TrainingLoad.h"
00014 #include "analysis/TrainingMetrics.h"
00015 #include "analysis/WorkoutAnalyzer.h"
00016 #include "core/zones/ZoneDefinition.h"
00017 #include "analysis/queue/AnalysisQueue.h"
00018 #include "core/AnalysisVersions.h"
00019 #include "core/ActivityAnalysisFlags.h"
00020 #include "database/ActivityRepository.h"
00021 #include "database/AthleteRepository.h"
00022 #include "database/ClimbRepository.h"
00023 #include "database/ImportRepository.h"
00024 #include "database/IntervalRepository.h"
00025 #include "fit/RideData.h"
00026
00027 class DatabaseManager;
00028
00029 enum class ImportResult
00030 {
00031     Success,
00032     Duplicate,
00033     TimeOverlap,
00034     Failed
00035 };
00036
00037 struct DuplicateActivityInfo
00038 {
00039     // Existing conflicting activity ID.
00040     int existingActivityId = -1;
00041
00042     // Existing activity start time in UTC ISO format.
00043     QString existingStartUtc;
00044
00045     // Existing activity duration in seconds.
00046     double existingDurationSeconds = 0.0;
00047
00048     // Incoming activity start time in UTC ISO format.
00049     QString newStartUtc;

```

```

00050
00051     // Incoming activity duration in seconds.
00052     double newDurationSeconds = 0.0;
00053 };
00054
00061 class WorkoutController : public QObject
00062 {
00063     Q_OBJECT
00064 public:
00065     explicit WorkoutController(QObject* parent = nullptr);
00066
00067     void setFTP(double ftp);
00068
00069     double ftp() const { return m_ftp; }
00070
00071     void setDatabaseManager(DatabaseManager* dbManager);
00072
00073     void setCurrentAthlete(int athleteId);
00074
00075     int currentAthleteId() const { return m_currentAthleteId; }
00076
00077     bool loadFile(const QString& path, QString& errorOut);
00078
00079     int importFile(const QString& path, QString& errorOut,
00080                  bool allowTimeOverlap = false,
00081                  bool runAnalysis = true,
00082                  const QString& importSource = QString(),
00083                  DuplicateActivityInfo* duplicateInfo = nullptr,
00084                  ImportResult* result = nullptr);
00085
00086     bool loadActivity(int activityId, QString& errorOut);
00087
00088     void reloadClimbsFromDatabase();
00089
00090     void setAnalysisQueue(AnalysisQueue* queue) { m_analysisQueue = queue; }
00091
00092     const RideData& rideData() const { return m_rideData; }
00093
00094     const WorkoutStatistics& statistics() const { return m_statistics; }
00095
00096     const std::vector<ZoneDistribution>& powerZones() const { return m_powerZones; }
00097
00098     const std::vector<PowerCurvePoint>& pdcPoints() const { return m_pdcPoints; }
00099
00100     const std::vector<Interval>& intervals() const { return m_intervals; }
00101
00102     const std::vector<Climb>& climbs() const { return m_climbs; }
00103
00104     double normalizedPower() const { return m_np; }
00105
00106     double intensityFactor() const { return m_if; }
00107
00108     double trainingStressScore() const { return m_tss; }
00109
00110     double variabilityIndex() const { return m_vi; }
00111
00112     double efficiencyFactor() const { return m_ef; }
00113
00114     int currentActivityId() const { return m_currentActivityId; }
00115
00116 signals:
00117     // Emitted when activity finishes loading.
00118     void workoutLoaded();
00119
00120     // Emitted when activity is imported.
00121     void activityImported(int activityId);
00122
00123     // Emitted when FTP changes.
00124     void ftpChanged(double ftp);
00125
00126 private:
00127     void reanalyze();
00128     void loadOrGenerateIntervals();
00129     void loadOrGenerateClimbs();
00130     AnalysisQueue* m_analysisQueue = nullptr;
00131
00132     double m_ftp = 250.0;
00133     int m_currentAthleteId = -1;
00134     int m_currentActivityId = -1;
00135
00136     DatabaseManager* m_dbManager = nullptr;
00137
00138     RideData m_rideData;
00139     WorkoutStatistics m_statistics;
00140     WorkoutAnalyzer m_analyzer;
00141     std::vector<ZoneDistribution> m_powerZones;
00142     std::vector<PowerCurvePoint> m_pdcPoints;

```

```

00143     std::vector<Interval>      m_intervals;
00144     std::vector<Climb>        m_climbs;
00145
00146     double m_np  = 0.0;
00147     double m_if  = 0.0;
00148     double m_tss = 0.0;
00149     double m_vi  = 0.0;
00150     double m_ef  = 0.0;
00151 };

```

8.108 ImportJob.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "ImportJob.h"

```

8.109 ImportJob.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <QString>
00006
00010 struct ImportJob
00011 {
00013     enum class State
00014     {
00015         Queued,
00016         Parsing,
00017         Storing,
00018         Completed,
00019         Failed
00020     };
00021
00022     QString jobId;
00023     QString batchId;
00024     QString filePath;
00025     QString athleteId;
00026     QString importSource;
00027     State state = State::Queued;
00028     double progress = 0.0;
00029     QString errorMessage;
00030 };

```

8.110 ImportProgressModel.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "ImportProgressModel.h"
00004
00005 #include <QFileInfo>
00006
00007 #include "ImportQueue.h"
00008
00009 ImportProgressModel::ImportProgressModel(QObject* parent)
00010     : QObject(parent)
00011 {}
00012
00013 void ImportProgressModel::attachQueue(ImportQueue* queue)
00014 {
00015     if (!queue)
00016         return;
00017
00018     connect(queue, &ImportQueue::jobStarted, this,
00019             [this](const QString&, const QString&, const QString& filePath, int activeCount)
00020             {
00021                 m_activeJobs = activeCount;
00022                 m_progress = 0.0;
00023                 m_currentFileName = QFileInfo(filePath).fileName();
00024                 m_statusText = QStringLiteral("Starting");
00025                 emit changed();
00026             });
00027
00028     connect(queue, &ImportQueue::jobProgress, this,
00029             [this](const QString&, const QString&, const QString& filePath,
00030                 double percentage, const QString& statusText, int activeCount)
00031             {

```

```

00032         m_activeJobs = activeCount;
00033         m_progress = percentage;
00034         m_currentFileName = QFileInfo(filePath).fileName();
00035         m_statusText = statusText;
00036         emit changed();
00037     });
00038
00039     connect(queue, &ImportQueue::jobFinished, this,
00040             [this](const QString&, const QString&, const QString&, int, int activeCount)
00041     {
00042         m_activeJobs = activeCount;
00043         ++m_completedJobs;
00044         if (m_activeJobs <= 0)
00045         {
00046             m_progress = 100.0;
00047             m_statusText = QStringLiteral("Done");
00048         }
00049         emit changed();
00050     });
00051
00052     connect(queue, &ImportQueue::jobFailed, this,
00053             [this](const QString&, const QString&, const QString&, const QString&, const QString&, int
activeCount)
00054     {
00055         m_activeJobs = activeCount;
00056         ++m_failedJobs;
00057         if (m_activeJobs <= 0)
00058             m_statusText = QStringLiteral("Finished with errors");
00059         emit changed();
00060     });
00061
00062     connect(queue, &ImportQueue::queueIdle, this, [this]()
00063     {
00064         m_activeJobs = 0;
00065         m_currentFileName.clear();
00066         m_statusText.clear();
00067         m_progress = 0.0;
00068         emit changed();
00069     });
00070 }

```

8.111 ImportProgressModel.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <QObject>
00006 #include <QString>
00007
00008 class ImportQueue;
00009
00015 class ImportProgressModel : public QObject
00016 {
00017     Q_OBJECT
00018 public:
00019     explicit ImportProgressModel(QObject* parent = nullptr);
00020
00022     void attachQueue(ImportQueue* queue);
00023
00024     int activeJobs() const { return m_activeJobs; }
00025     int completedJobs() const { return m_completedJobs; }
00026     int failedJobs() const { return m_failedJobs; }
00027     double progress() const { return m_progress; }
00028     QString currentFileName() const { return m_currentFileName; }
00029     QString statusText() const { return m_statusText; }
00030
00031 signals:
00033     void changed();
00034
00035 private:
00036     int m_activeJobs = 0;
00037     int m_completedJobs = 0;
00038     int m_failedJobs = 0;
00039     double m_progress = 0.0;
00040     QString m_currentFileName;
00041     QString m_statusText;
00042 };

```

8.112 ImportQueue.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "ImportQueue.h"
00004
00005 #include <QThread>
00006 #include <QUuid>
00007
00008 #include "ImportWorker.h"
00009
00010 ImportQueue::ImportQueue(QObject* parent)
00011     : QObject(parent)
00012 {
00013     m_thread = new QThread(this);
00014     m_worker = new ImportWorker;
00015     m_worker->moveToThread(m_thread);
00016
00017     connect(m_worker, &ImportWorker::progressChanged,
00018             this, &ImportQueue::onWorkerProgress);
00019     connect(m_worker, &ImportWorker::completed,
00020             this, &ImportQueue::onWorkerCompleted);
00021     connect(m_worker, &ImportWorker::failed,
00022             this, &ImportQueue::onWorkerFailed);
00023
00024     m_thread->start();
00025 }
00026
00027 ImportQueue::~ImportQueue()
00028 {
00029     m_thread->quit();
00030     m_thread->wait(3000);
00031     delete m_worker;
00032 }
00033
00034 void ImportQueue::setDatabasePath(const QString& path)
00035 {
00036     m_dbPath = path;
00037     QMetaObject::invokeMethod(m_worker, [this, path]()
00038     {
00039         m_worker->setDatabasePath(path);
00040     }, Qt::QueuedConnection);
00041 }
00042
00043 void ImportQueue::enqueue(const QString& filePath,
00044                           const QString& athleteId,
00045                           const QString& batchId,
00046                           const QString& importSource)
00047 {
00048     ImportJob job;
00049     job.jobId = QUuid::createUuid().toString(QUuid::WithoutBraces);
00050     job.batchId = batchId;
00051     job.filePath = filePath;
00052     job.athleteId = athleteId;
00053     job.importSource = importSource;
00054     job.state = ImportJob::State::Queued;
00055     job.progress = 0.0;
00056
00057     m_queue.enqueue(job);
00058
00059     emit jobQueued(job.jobId, job.batchId, job.filePath, activeCountUnsafe());
00060     emit queueStateChanged(m_busy ? 1 : 0, m_queue.size());
00061
00062     if (!m_busy)
00063         processNext();
00064 }
00065
00066 int ImportQueue::pendingCount() const
00067 {
00068     return activeCountUnsafe();
00069 }
00070
00071 bool ImportQueue::isBusy() const
00072 {
00073     return m_busy;
00074 }
00075
00076 void ImportQueue::onWorkerProgress(const QString& jobId,
00077                                   const QString& batchId,
00078                                   const QString& filePath,
00079                                   double percentage,
00080                                   const QString& statusText)
00081 {
00082     emit jobProgress(jobId, batchId, filePath, percentage, statusText, activeCountUnsafe());
00083 }
00084

```



```

00085 void ImportQueue::onWorkerCompleted(const QString& jobId,
00086                                     const QString& batchId,
00087                                     const QString& filePath,
00088                                     int activityId)
00089 {
00090     m_busy = false;
00091     emit jobFinished(jobId, batchId, filePath, activityId, activeCountUnsafe());
00092     emit queueStateChanged(0, m_queue.size());
00093
00094     if (!m_queue.isEmpty())
00095     {
00096         processNext();
00097         return;
00098     }
00099
00100     emit queueIdle();
00101 }
00102
00103 void ImportQueue::onWorkerFailed(const QString& jobId,
00104                                 const QString& batchId,
00105                                 const QString& filePath,
00106                                 const QString& reason,
00107                                 const QString& errorMessage)
00108 {
00109     m_busy = false;
00110     emit jobFailed(jobId, batchId, filePath, reason, errorMessage, activeCountUnsafe());
00111     emit queueStateChanged(0, m_queue.size());
00112
00113     if (!m_queue.isEmpty())
00114     {
00115         processNext();
00116         return;
00117     }
00118
00119     emit queueIdle();
00120 }
00121
00122 void ImportQueue::processNext()
00123 {
00124     if (m_busy || m_queue.isEmpty())
00125         return;
00126
00127     m_busy = true;
00128     m_runningJob = m_queue.dequeue();
00129
00130     emit jobStarted(m_runningJob.jobId, m_runningJob.batchId, m_runningJob.filePath,
00131                    activeCountUnsafe());
00132     emit queueStateChanged(1, m_queue.size());
00133
00134     const ImportJob job = m_runningJob;
00135     QMetaObject::invokeMethod(m_worker, [this, job]()
00136     {
00137         m_worker->processJob(job);
00138     }, Qt::QueuedConnection);
00139 }
00140 int ImportQueue::activeCountUnsafe() const
00141 {
00142     return m_queue.size() + (m_busy ? 1 : 0);
00143 }

```

8.113 ImportQueue.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <QObject>
00006 #include <QQueue>
00007 #include <QString>
00008
00009 #include "ImportJob.h"
00010
00011 class ImportWorker;
00012 class QThread;
00013
00020 class ImportQueue : public QObject
00021 {
00022     Q_OBJECT
00023 public:
00024     explicit ImportQueue(QObject* parent = nullptr);
00025     ~ImportQueue() override;
00026
00028     void setDatabasePath(const QString& path);

```

```

00029
00035     void enqueue(const QString& filePath,
00036                 const QString& athleteId,
00037                 const QString& batchId,
00038                 const QString& importSource);
00039
00041     int pendingCount() const;
00042
00044     bool isBusy() const;
00045
00046 signals:
00048     void jobQueued(const QString& jobId,
00049                  const QString& batchId,
00050                  const QString& filePath,
00051                  int activeCount);
00052
00054     void jobStarted(const QString& jobId,
00055                    const QString& batchId,
00056                    const QString& filePath,
00057                    int activeCount);
00058
00060     void jobProgress(const QString& jobId,
00061                     const QString& batchId,
00062                     const QString& filePath,
00063                     double percentage,
00064                     const QString& statusText,
00065                     int activeCount);
00066
00068     void jobFinished(const QString& jobId,
00069                     const QString& batchId,
00070                     const QString& filePath,
00071                     int activityId,
00072                     int activeCount);
00073
00078     void jobFailed(const QString& jobId,
00079                   const QString& batchId,
00080                   const QString& filePath,
00081                   const QString& reason,
00082                   const QString& errorMessage,
00083                   int activeCount);
00084
00086     void queueStateChanged(int runningCount, int pendingCount);
00087
00089     void queueIdle();
00090
00091 private slots:
00092     void onWorkerProgress(const QString& jobId,
00093                          const QString& batchId,
00094                          const QString& filePath,
00095                          double percentage,
00096                          const QString& statusText);
00097
00098     void onWorkerCompleted(const QString& jobId,
00099                           const QString& batchId,
00100                           const QString& filePath,
00101                           int activityId);
00102
00103     void onWorkerFailed(const QString& jobId,
00104                        const QString& batchId,
00105                        const QString& filePath,
00106                        const QString& reason,
00107                        const QString& errorMessage);
00108
00109 private:
00110     void processNext();
00111     int activeCountUnsafe() const;
00112
00113     QThread* m_thread = nullptr;
00114     ImportWorker* m_worker = nullptr;
00115     QQueue<ImportJob> m_queue;
00116     bool m_busy = false;
00117     QString m_dbPath;
00118     ImportJob m_runningJob;
00119 };

```

8.114 ImportWorker.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "ImportWorker.h"
00004
00005 #include <QCryptographicHash>
00006 #include <QDateTime>
00007 #include <QFile>

```

```

00008 #include <QFileInfo>
00009 #include <QIODevice>
00010 #include <QSqlDatabase>
00011 #include <QSqlError>
00012 #include <QSqlQuery>
00013 #include <QUuid>
00014
00015 #include "analysis/TrainingLoad.h"
00016 #include "analysis/WorkoutAnalyzer.h"
00017 #include "database/ActivityRepository.h"
00018 #include "database/DatabaseConnectionGuard.h"
00019 #include "database/ImportRepository.h"
00020 #include "fit/FitParser.h"
00021
00022 namespace
00023 {
00024 void applySqlitePragmas(QSqlDatabase& db)
00025 {
00026     QSqlQuery q(db);
00027     q.exec("PRAGMA journal_mode=WAL");
00028     q.exec("PRAGMA synchronous=NORMAL");
00029     q.exec("PRAGMA temp_store=MEMORY");
00030     q.exec("PRAGMA foreign_keys=ON");
00031     q.exec("PRAGMA cache_size=-20000");
00032 }
00033 }
00034
00035 ImportWorker::ImportWorker(QObject* parent)
00036     : QObject(parent)
00037 {}
00038
00039 void ImportWorker::setDatabasePath(const QString& path)
00040 {
00041     m_dbPath = path;
00042 }
00043
00044 void ImportWorker::processJob(const ImportJob& job)
00045 {
00046     if (m_dbPath.isEmpty())
00047     {
00048         emit failed(job.jobId, job.batchId, job.filePath,
00049                     QStringLiteral("error"), QStringLiteral("No database path set.));
00050         return;
00051     }
00052
00053     const int athleteId = job.athleteId.toInt();
00054     if (athleteId <= 0)
00055     {
00056         emit failed(job.jobId, job.batchId, job.filePath,
00057                     QStringLiteral("error"), QStringLiteral("No athlete selected.));
00058         return;
00059     }
00060
00061     const QString connName =
00062         QStringLiteral("ImportWorker_%1").arg(QUuid::createUuid().toString(QUuid::WithoutBraces));
00063
00064     try
00065     {
00066         DatabaseConnectionGuard guard(connName);
00067         QSqlDatabase db = guard.database();
00068         db.setDatabaseName(m_dbPath);
00069         if (!db.open())
00070         {
00071             emit failed(job.jobId, job.batchId, job.filePath,
00072                         QStringLiteral("error"), db.lastError().text());
00073             return;
00074         }
00075
00076         applySqlitePragmas(db);
00077         emit progressChanged(job.jobId, job.batchId, job.filePath, 5.0, QStringLiteral("Opening
00078 file"));
00079
00080         QString fitHash;
00081         QFile file(job.filePath);
00082         if (file.open(QIODevice::ReadOnly))
00083         {
00084             fitHash = QCryptographicHash::hash(file.readAll(), QCryptographicHash::Sha256).toHex();
00085             file.close();
00086         }
00087
00088         ActivityRepository activityRepo(db);
00089         ImportRepository importRepo(db);
00090
00091         const auto emitLoggedFailure = [&](const QString& reason, const QString& message)
00092         {
00093             ImportRecord rec;
00094             rec.fileName = QFileInfo(job.filePath).fileName();

```

```

00093         rec.status = reason == QStringLiteral("duplicate") || reason ==
QStringLiteral("time_overlap")
00094         ? QStringLiteral("duplicate")
00095         : QStringLiteral("error");
00096         rec.errorMessage = message;
00097         importRepo.logImport(rec);
00098
00099         emit failed(job.jobId, job.batchId, job.filePath, reason, message);
00100     };
00101
00102     if (!fitHash.isEmpty() && activityRepo.hasFitHash(fitHash))
00103     {
00104         emitLoggedFailure(QStringLiteral("duplicate"),
00105                             QStringLiteral("Already imported (duplicate file)."));
00106         return;
00107     }
00108
00109     emit progressChanged(job.jobId, job.batchId, job.filePath, 15.0, QStringLiteral("Parsing
FIT"));
00110
00111     FitParser parser;
00112     RideData ride = parser.load(job.filePath);
00113
00114     if (!ride.activityStartTimeUtc.isEmpty())
00115     {
00116         QSqlQuery overlapQ(db);
00117         overlapQ.prepare(
00118             "SELECT id FROM activities "
00119             "WHERE athlete_id=:aid AND activity_start_time=:st LIMIT 1");
00120         overlapQ.bindValue(":aid", athleteId);
00121         overlapQ.bindValue(":st", ride.activityStartTimeUtc);
00122         if (overlapQ.exec() && overlapQ.next())
00123         {
00124             emitLoggedFailure(QStringLiteral("time_overlap"),
00125                                 QStringLiteral("Potential duplicate activity (matching start
time)."));
00126             return;
00127         }
00128     }
00129
00130     emit progressChanged(job.jobId, job.batchId, job.filePath, 72.0, QStringLiteral("Storing
activity"));
00131
00132     WorkoutAnalyzer analyzer;
00133     const WorkoutStatistics stats = analyzer.analyze(ride);
00134     const double np = TrainingLoad::normalizedPower(ride);
00135
00136     const QString now = QDateTime::currentDateTimeUtc().toString(Qt::ISODate);
00137
00138     Activity act;
00139     act.athleteId = athleteId;
00140     act.fitHash = fitHash;
00141     act.fileName = QFile::fileName(job.filePath);
00142     act.sport = QStringLiteral("Cycling");
00143     act.startTime = ride.activityStartTimeUtc.isEmpty() ? now : ride.activityStartTimeUtc;
00144     act.endTime = ride.activityEndTimeUtc.isEmpty() ? act.startTime : ride.activityEndTimeUtc;
00145     act.durationSec = stats.durationSeconds;
00146     act.distanceM = stats.totalDistanceKm * 1000.0;
00147     act.avgPower = stats.averagePower;
00148     act.maxPower = stats.maximumPower;
00149     act.normalizedPower = np;
00150     act.avgHr = stats.averageHeartRate;
00151     act.maxHr = stats.maximumHeartRate;
00152     act.avgCadence = stats.averageCadence;
00153     act.avgSpeed = stats.averageSpeed;
00154     act.elevationGain = stats.elevationGainM;
00155     act.importedAt = now;
00156
00157     if (!db.transaction())
00158     {
00159         emitLoggedFailure(QStringLiteral("error"), db.lastError().text());
00160         return;
00161     }
00162
00163     const int activityId = activityRepo.insertActivity(act);
00164     if (activityId <= 0)
00165     {
00166         db.rollback();
00167         emitLoggedFailure(QStringLiteral("error"), QStringLiteral("Failed to insert activity."));
00168         return;
00169     }
00170
00171     QCryptographicHash fingerprintHasher(QCryptographicHash::Sha256);
00172     fingerprintHasher.addData(QStringLiteral("Cycling").toUtf8());
00173     fingerprintHasher.addData(ride.activityStartTimeUtc.toUtf8());
00174     fingerprintHasher.addData(QString::number(static_cast<int>(stats.durationSeconds)).toUtf8());
00175     fingerprintHasher.addData(QString::number(static_cast<int>(stats.totalDistanceKm *

```

```

1000.0)).toUtf8());
00176     activityRepo.updateFingerprint(activityId, fingerprintHasher.result().toHex());
00177
00178     QSqlQuery sampleQ(db);
00179     sampleQ.prepare(
00180         "INSERT INTO activity_samples("
00181         " activity_id, elapsed_seconds,"
00182         " power_total, heart_rate, cadence, speed,"
00183         " latitude, longitude, altitude,"
00184         " has_gps, has_power, has_heart_rate, has_cadence, has_speed, has_altitude"
00185         ") VALUES("
00186         " :aid, :es,"
00187         " :pw, :hr, :cad, :spd,"
00188         " :lat, :lon, :alt,"
00189         " :hgps, :hpw, :hhr, :hcad, :hspd, :halt"
00190         ")");
00191
00192     QVariantList aids, es_, pw_, hr_, cad_, spd_;
00193     QVariantList lat_, lon_, alt_;
00194     QVariantList hgps, hpw, hhr, hcad, hspd, halt;
00195
00196     for (const RideRecord& r : ride.records)
00197     {
00198         aids << activityId;
00199         es_ << r.elapsedSeconds;
00200         pw_ << (r.hasPower ? QVariant(r.power) : QVariant());
00201         hr_ << (r.hasHeartRate ? QVariant(r.heartRate) : QVariant());
00202         cad_ << (r.hasCadence ? QVariant(r.cadence) : QVariant());
00203         spd_ << (r.hasSpeed ? QVariant(r.speed) : QVariant());
00204         lat_ << (r.hasGps ? QVariant(r.latitude) : QVariant());
00205         lon_ << (r.hasGps ? QVariant(r.longitude) : QVariant());
00206         alt_ << (r.hasAltitude ? QVariant(r.altitude) : QVariant());
00207         hgps << (r.hasGps ? 1 : 0);
00208         hpw << (r.hasPower ? 1 : 0);
00209         hhr << (r.hasHeartRate ? 1 : 0);
00210         hcad << (r.hasCadence ? 1 : 0);
00211         hspd << (r.hasSpeed ? 1 : 0);
00212         halt << (r.hasAltitude ? 1 : 0);
00213     }
00214
00215     sampleQ.bindValue(":aid", aids);
00216     sampleQ.bindValue(":es", es_);
00217     sampleQ.bindValue(":pw", pw_);
00218     sampleQ.bindValue(":hr", hr_);
00219     sampleQ.bindValue(":cad", cad_);
00220     sampleQ.bindValue(":spd", spd_);
00221     sampleQ.bindValue(":lat", lat_);
00222     sampleQ.bindValue(":lon", lon_);
00223     sampleQ.bindValue(":alt", alt_);
00224     sampleQ.bindValue(":hgps", hgps);
00225     sampleQ.bindValue(":hpw", hpw);
00226     sampleQ.bindValue(":hhr", hhr);
00227     sampleQ.bindValue(":hcad", hcad);
00228     sampleQ.bindValue(":hspd", hspd);
00229     sampleQ.bindValue(":halt", halt);
00230
00231     if (!sampleQ.execBatch())
00232     {
00233         db.rollback();
00234         emitLoggedFailure(QStringLiteral("error"),
00235             QStringLiteral("Failed to insert samples:
00236 %1").arg(sampleQ.lastError().text()));
00237         return;
00238     }
00239
00240     // Store optional import source metadata.
00241     if (!job.importSource.isEmpty())
00242         activityRepo.setImportSource(activityId, job.importSource);
00243
00244     emit progressChanged(job.jobId, job.batchId, job.filePath, 96.0,
00245         QStringLiteral("Committing"));
00246
00247     if (!db.commit())
00248     {
00249         db.rollback();
00250         emitLoggedFailure(QStringLiteral("error"), db.lastError().text());
00251         return;
00252     }
00253
00254     ImportRecord rec;
00255     rec.fileName = act.fileName;
00256     rec.status = QStringLiteral("success");
00257     rec.activityId = activityId;
00258     importRepo.logImport(rec);
00259
00260     emit progressChanged(job.jobId, job.batchId, job.filePath, 100.0, QStringLiteral("Done"));
00261     emit completed(job.jobId, job.batchId, job.filePath, activityId);

```

```

00260     }
00261     catch (const std::exception& ex)
00262     {
00263         emit failed(job.jobId, job.batchId, job.filePath,
00264                     QStringLiteral("error"), QString::fromLocal8Bit(ex.what()));
00265     }
00266     catch (...)
00267     {
00268         emit failed(job.jobId, job.batchId, job.filePath,
00269                     QStringLiteral("error"), QStringLiteral("Unknown import error.));
00270     }
00271 }

```

8.115 ImportWorker.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <QObject>
00006 #include <QString>
00007
00008 #include "ImportJob.h"
00009
00016 class ImportWorker : public QObject
00017 {
00018     Q_OBJECT
00019 public:
00020     explicit ImportWorker(QObject* parent = nullptr);
00021
00023     void setDatabasePath(const QString& path);
00024
00025 public slots:
00027     void processJob(const ImportJob& job);
00028
00029 signals:
00031     void progressChanged(const QString& jobId,
00032                          const QString& batchId,
00033                          const QString& filePath,
00034                          double percentage,
00035                          const QString& statusText);
00036
00038     void completed(const QString& jobId,
00039                   const QString& batchId,
00040                   const QString& filePath,
00041                   int activityId);
00042
00047     void failed(const QString& jobId,
00048                 const QString& batchId,
00049                 const QString& filePath,
00050                 const QString& reason,
00051                 const QString& errorMessage);
00052
00053 private:
00054     QString m_dbPath;
00055 };

```

8.116 MapFitMath.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "MapFitMath.h"
00005
00006 #include <algorithm>
00007 #include <cmath>
00008
00009 namespace
00010 {
00011     constexpr double kPi = 3.14159265358979323846;
00012     constexpr int kTilePx = 256;
00013     constexpr double kMinLatSpan = 0.001;
00014     constexpr double kMinLonSpan = 0.001;
00015     constexpr double kMaxMercatorLat = 85.051129;
00016
00017     double normalizeLongitude(double lon)
00018     {
00019         double normalized = std::fmod(lon + 180.0, 360.0);
00020         if (normalized < 0.0)
00021             normalized += 360.0;
00022         return normalized - 180.0;
00023     }
00024 }

```

```

00023 }
00024
00025 double latToTileY(double lat)
00026 {
00027     const double clamped = std::clamp(lat, -kMaxMercatorLat, kMaxMercatorLat);
00028     const double rad = clamped * kPi / 180.0;
00029     return (1.0 - std::log(std::tan(rad) + 1.0 / std::cos(rad)) / kPi) / 2.0;
00030 }
00031
00032 double tileYToLat(double tileY)
00033 {
00034     const double mercN = kPi * (1.0 - 2.0 * tileY);
00035     return std::atan(std::sinh(mercN)) * 180.0 / kPi;
00036 }
00037 }
00038
00039 namespace MapFitMath
00040 {
00041 FitResult computeFitToBounds(double minLat,
00042                             double maxLat,
00043                             double minLon,
00044                             double maxLon,
00045                             int viewportWidth,
00046                             int viewportHeight,
00047                             int maxProviderZoom)
00048 {
00049     if (maxLat < minLat)
00050         std::swap(minLat, maxLat);
00051     if (maxLon < minLon)
00052         std::swap(minLon, maxLon);
00053
00054     if (maxLat - minLat < kMinLatSpan)
00055     {
00056         const double center = (minLat + maxLat) / 2.0;
00057         minLat = center - kMinLatSpan / 2.0;
00058         maxLat = center + kMinLatSpan / 2.0;
00059     }
00060     if (maxLon - minLon < kMinLonSpan)
00061     {
00062         const double center = (minLon + maxLon) / 2.0;
00063         minLon = center - kMinLonSpan / 2.0;
00064         maxLon = center + kMinLonSpan / 2.0;
00065     }
00066
00067     minLat = std::clamp(minLat, -kMaxMercatorLat, kMaxMercatorLat);
00068     maxLat = std::clamp(maxLat, -kMaxMercatorLat, kMaxMercatorLat);
00069     minLon = normalizeLongitude(minLon);
00070     maxLon = normalizeLongitude(maxLon);
00071
00072     FitResult result;
00073     result.minLat = minLat;
00074     result.maxLat = maxLat;
00075     result.minLon = minLon;
00076     result.maxLon = maxLon;
00077
00078     double lonSpan = result.maxLon - result.minLon;
00079     if (lonSpan < 0.0)
00080         lonSpan += 360.0;
00081     const bool wrapsDateline = lonSpan > 180.0;
00082     if (wrapsDateline)
00083         lonSpan = 360.0 - lonSpan;
00084     result.lonSpanDegrees = lonSpan;
00085
00086     const double northTileY = latToTileY(result.maxLat);
00087     const double southTileY = latToTileY(result.minLat);
00088     const double centerTileY = (northTileY + southTileY) / 2.0;
00089     result.centerLat = std::clamp(tileYToLat(centerTileY), -kMaxMercatorLat, kMaxMercatorLat);
00090
00091     if (wrapsDateline)
00092     {
00093         double shiftedMin = result.minLon;
00094         double shiftedMax = result.maxLon;
00095         if (shiftedMin < 0.0)
00096             shiftedMin += 360.0;
00097         if (shiftedMax < 0.0)
00098             shiftedMax += 360.0;
00099         if (shiftedMin > shiftedMax)
00100             std::swap(shiftedMin, shiftedMax);
00101         result.centerLon = normalizeLongitude((shiftedMin + shiftedMax) / 2.0);
00102     }
00103     else
00104     {
00105         result.centerLon = normalizeLongitude((result.minLon + result.maxLon) / 2.0);
00106     }
00107
00108     const int w = std::max(viewportWidth, 800);
00109     const int h = std::max(viewportHeight, 400);

```

```

00110
00111     if (result.maxLat <= result.minLat || result.lonSpanDegrees <= 0.0)
00112     {
00113         result.zoom = std::min(15, maxProviderZoom);
00114         return result;
00115     }
00116
00117     const int cappedMaxZoom = std::clamp(maxProviderZoom, 1, 18);
00118     result.zoom = 1;
00119     for (int z = cappedMaxZoom; z >= 1; --z)
00120     {
00121         const double n = std::pow(2.0, z);
00122         const double pixW = (result.lonSpanDegrees / 360.0) * n * kTilePx;
00123         const double pixH = std::abs(southTileY - northTileY) * n * kTilePx;
00124         if (pixW <= w * 0.85 && pixH <= h * 0.85)
00125         {
00126             result.zoom = z;
00127             break;
00128         }
00129     }
00130
00131     return result;
00132 }
00133 }

```

8.117 MapFitMath.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 namespace MapFitMath
00007 {
00011 struct FitResult
00012 {
00014     double minLat = 0.0;
00015
00017     double maxLat = 0.0;
00018
00020     double minLon = 0.0;
00021
00023     double maxLon = 0.0;
00024
00026     double centerLat = 0.0;
00027
00029     double centerLon = 0.0;
00030
00032     double lonSpanDegrees = 0.0;
00033
00035     int zoom = 1;
00036 };
00037
00049 FitResult computeFitToBounds(double minLat,
00050                             double maxLat,
00051                             double minLon,
00052                             double maxLon,
00053                             int viewportWidth,
00054                             int viewportHeight,
00055                             int maxProviderZoom);
00056 }

```

8.118 MapRenderer.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "MapRenderer.h"
00005
00006 #include <QLineF>
00007 #include <QMouseEvent>
00008 #include <QPainter>
00009 #include <QPainterPath>
00010 #include <QWheelEvent>
00011
00012 #include "core/settings/AppSettings.h"
00013 #include "core/zones/ColorProvider.h"
00014 #include "maps/MapFitMath.h"
00015
00016 #include <algorithm>
00017 #include <cmath>

```



```

00018 #include <limits>
00019
00025 static QColor gradientColorForSlope(double percent)
00026 {
00027     if (percent <= -8.0) return QColor("#7c2d12");
00028     if (percent <= -4.0) return QColor("#ea580c");
00029     if (percent <= -1.0) return QColor("#f59e0b");
00030     if (percent < 1.0) return QColor("#94a3b8");
00031     if (percent < 4.0) return QColor("#22c55e");
00032     if (percent < 8.0) return QColor("#16a34a");
00033     return QColor("#15803d");
00034 }
00035
00041 static ColorMetric routeModeToMetric(RouteColorMode mode)
00042 {
00043     switch (mode)
00044     {
00045         case RouteColorMode::Power: return ColorMetric::Power;
00046         case RouteColorMode::HeartRate: return ColorMetric::HeartRate;
00047         case RouteColorMode::Cadence: return ColorMetric::Cadence;
00048         case RouteColorMode::Speed: return ColorMetric::Speed;
00049         case RouteColorMode::Altitude: return ColorMetric::Altitude;
00050         case RouteColorMode::None:
00051         case RouteColorMode::Gradient:
00052             return ColorMetric::None;
00053     }
00054
00055     return ColorMetric::None;
00056 }
00057
00059 static constexpr double kPi = 3.14159265358979323846;
00060
00062 static constexpr int kTilePx = 256;
00063
00065 static constexpr qreal kMarkerHitSizePx = 24.0;
00066
00068 static constexpr qreal kMarkerRadiusPx = 6.5;
00069
00071 static constexpr qreal kNearestRecordMaxDistancePx = 40.0;
00072
00074 static constexpr int kEdgePanMargin = 60;
00075
00076 QPointF MapRenderer::latLonToTile(double lat, double lon, int zoom)
00077 {
00078     const double n = std::pow(2.0, zoom);
00079     const double x = (lon + 180.0) / 360.0 * n;
00080     const double rad = lat * kPi / 180.0;
00081     const double y = (1.0 - std::log(
00082         std::tan(rad) + 1.0 / std::cos(rad)) / kPi) / 2.0 * n;
00083     return { x, y };
00084 }
00085
00086 QPointF MapRenderer::tileToScreen(const QPointF& tilePt) const
00087 {
00088     const auto centre = latLonToTile(m_centerLat, m_centerLon, m_zoom);
00089     return {
00090         width() / 2.0 + (tilePt.x() - centre.x()) * kTilePx,
00091         height() / 2.0 + (tilePt.y() - centre.y()) * kTilePx
00092     };
00093 }
00094
00095 MapRenderer::MapRenderer(QWidget* parent)
00096 : QWidget(parent)
00097 {
00098     setMinimumHeight(300);
00099
00100     // Apply persisted tile cache size.
00101     m_tileCache.setMemoryCacheSize(AppSettings::instance().tileCacheSize());
00102
00103     // Coalesce tile-download repaints: many tiles arriving at once
00104     // produce one repaint 50 ms after the last arrival, not one per tile.
00105     m_tileRepaintTimer.setSingleShot(true);
00106     m_tileRepaintTimer.setInterval(50);
00107     connect(&m_tileRepaintTimer, &QTimer::timeout, this, [this] { update(); });
00108
00109     connect(&m_tileCache, &TileCache::tileLoaded,
00110         this, [this](int, int, int) { m_tileRepaintTimer.start(); });
00111 }
00112
00113 // Precompute per-segment base colors (excluding map-style adjustment which changes
00114 // per-repaint). Called once on setRideData() and whenever the color mode changes.
00115 void MapRenderer::rebuildSegmentColors()
00116 {
00117     if (m_gpsRecords.size() < 2 || m_routeColorMode == RouteColorMode::None)
00118     {
00119         m_segmentColors.clear();
00120         return;
00121     }

```

```

00121     }
00122
00123     m_segmentColors.resize(m_gpsRecords.size() - 1);
00124
00125     for (size_t i = 0; i + 1 < m_gpsRecords.size(); ++i)
00126     {
00127         const RideRecord& cur = *m_gpsRecords[i];
00128         const RideRecord& next = *m_gpsRecords[i + 1];
00129
00130         double value = 0.0;
00131         bool hasValue = false;
00132
00133         switch (m_routeColorMode)
00134         {
00135             case RouteColorMode::None: break;
00136             case RouteColorMode::Power:
00137                 if (cur.hasPower) { value = cur.power; hasValue = true; }
00138                 else if (next.hasPower) { value = next.power; hasValue = true; }
00139                 break;
00140             case RouteColorMode::HeartRate:
00141                 if (cur.hasHeartRate) { value = cur.heartRate; hasValue = true; }
00142                 else if (next.hasHeartRate) { value = next.heartRate; hasValue = true; }
00143                 break;
00144             case RouteColorMode::Cadence:
00145                 if (cur.hasCadence) { value = cur.cadence; hasValue = true; }
00146                 else if (next.hasCadence) { value = next.cadence; hasValue = true; }
00147                 break;
00148             case RouteColorMode::Speed:
00149                 if (cur.hasSpeed) { value = cur.speed; hasValue = true; }
00150                 else if (next.hasSpeed) { value = next.speed; hasValue = true; }
00151                 break;
00152             case RouteColorMode::Altitude:
00153                 if (cur.hasAltitude) { value = cur.altitude; hasValue = true; }
00154                 else if (next.hasAltitude) { value = next.altitude; hasValue = true; }
00155                 break;
00156             case RouteColorMode::Gradient:
00157                 if (cur.hasGps && next.hasGps && cur.hasAltitude && next.hasAltitude)
00158                 {
00159                     const double lat1 = cur.latitude * kPi / 180.0;
00160                     const double lat2 = next.latitude * kPi / 180.0;
00161                     const double dLat = lat2 - lat1;
00162                     const double dLon = (next.longitude - cur.longitude) * kPi / 180.0;
00163                     const double a = std::sin(dLat / 2.0) * std::sin(dLat / 2.0)
00164                                     + std::cos(lat1) * std::cos(lat2)
00165                                     * std::sin(dLon / 2.0) * std::sin(dLon / 2.0);
00166                     const double dist = 2.0 * 6371000.0
00167                                     * std::atan2(std::sqrt(a), std::sqrt(std::max(0.0, 1.0 - a)));
00168                     const double elev = next.altitude - cur.altitude;
00169                     if (dist > 0.1) { value = (elev / dist) * 100.0; hasValue = true; }
00170                 }
00171                 break;
00172         }
00173
00174         m_segmentColors[i] = hasValue
00175             ? (m_routeColorMode == RouteColorMode::Gradient
00176                ? gradientColorForSlope(value)
00177                : ColorProvider::colorForMetric(
00178                    routeModeToMetric(m_routeColorMode), value, m_colorContext))
00179             : QColor(); // invalid = no data
00180     }
00181 }
00182
00183 void MapRenderer::rebuildGpsCache()
00184 {
00185     m_gpsRecords.clear();
00186     m_gpsRecords.reserve(m_rideData.records.size());
00187     m_firstGpsRecord = nullptr;
00188     m_lastGpsRecord = nullptr;
00189
00190     for (const auto& r : m_rideData.records)
00191     {
00192         if (!r.hasGps)
00193             continue;
00194         m_gpsRecords.push_back(&r);
00195         if (!m_firstGpsRecord)
00196             m_firstGpsRecord = &r;
00197         m_lastGpsRecord = &r;
00198     }
00199     // Records are already in elapsedSeconds order from FIT parsing.
00200 }
00201
00202 // Nearest GPS record to a given elapsed time: O(log N).
00203 const RideRecord* MapRenderer::gpsRecordAtTime(double elapsedSeconds) const
00204 {
00205     if (m_gpsRecords.empty())
00206         return nullptr;
00207

```

```

00208     auto it = std::lower_bound(
00209         m_gpsRecords.begin(), m_gpsRecords.end(), elapsedSeconds,
00210         [](const RideRecord* r, double t) { return r->elapsedSeconds < t; });
00211
00212     if (it == m_gpsRecords.end())
00213         return m_gpsRecords.back();
00214     if (it == m_gpsRecords.begin())
00215         return m_gpsRecords.front();
00216
00217     auto prev = std::prev(it);
00218     return (std::abs((*prev)->elapsedSeconds - elapsedSeconds) <=
00219         std::abs(*it->elapsedSeconds - elapsedSeconds))
00220         ? *prev : *it;
00221 }
00222
00223 void MapRenderer::setRideData(const RideData& rideData,
00224                               ColorMetric colorMetric,
00225                               const ColorContext& colorContext)
00226 {
00227     m_rideData = rideData;
00228     m_hasGps = false;
00229     m_userMovedMap = false;
00230     m_autoFitVisibleRange = true;
00231     m_visibleStartSeconds = -1.0;
00232     m_visibleEndSeconds = -1.0;
00233     m_currentTimeSeconds = -1.0;
00234     m_highlightElapsedSeconds = -1.0;
00235     m_dragHandle = DragHandle::None;
00236     m_startMarkerRect = QRectF();
00237     m_endMarkerRect = QRectF();
00238     m_draggingSelection = false;
00239     m_routeColorMode = colorMetric == ColorMetric::None
00240         ? RouteColorMode::None
00241         : colorMetric == ColorMetric::Power
00242             ? RouteColorMode::Power
00243             : colorMetric == ColorMetric::HeartRate
00244                 ? RouteColorMode::HeartRate
00245                 : colorMetric == ColorMetric::Cadence
00246                     ? RouteColorMode::Cadence
00247                     : colorMetric == ColorMetric::Speed
00248                         ? RouteColorMode::Speed
00249                         : RouteColorMode::Altitude;
00250     m_colorContext = colorContext;
00251
00252     rebuildGpsCache();
00253     m_hasGps = !m_gpsRecords.empty();
00254     rebuildSegmentColors();
00255
00256     if (m_hasGps)
00257         fitToTrack();
00258
00259     update();
00260 }
00261
00262 void MapRenderer::setVisibleTimeRange(double startSeconds, double endSeconds)
00263 {
00264     m_visibleStartSeconds = std::min(startSeconds, endSeconds);
00265     m_visibleEndSeconds = std::max(startSeconds, endSeconds);
00266
00267     if (m_autoFitVisibleRange && !m_userMovedMap)
00268         fitToVisibleRange();
00269
00270     update();
00271 }
00272
00273 void MapRenderer::setCurrentTime(double seconds)
00274 {
00275     m_currentTimeSeconds = seconds;
00276     update();
00277 }
00278
00279 void MapRenderer::setRouteColorMode(RouteColorMode mode, const ColorContext& colorContext)
00280 {
00281     m_routeColorMode = mode;
00282     m_colorContext = colorContext;
00283     rebuildSegmentColors();
00284     update();
00285 }
00286
00287 void MapRenderer::setMapStyle(MapStyle style)
00288 {
00289     if (m_tileCache.mapStyle() == style)
00290         return;
00291
00292     m_tileCache.setMapStyle(style);
00293     m_zoom = std::clamp(m_zoom, m_minZoom, std::min(18, m_tileCache.maxZoom()));
00294     update();

```

```

00295 }
00296
00297 void MapRenderer::setAutoRouteContrast(bool enabled)
00298 {
00299     if (m_autoRouteContrast == enabled)
00300         return;
00301     m_autoRouteContrast = enabled;
00302     update();
00303 }
00304
00305 void MapRenderer::setTileCacheSize(int maxTiles)
00306 {
00307     m_tileCache.setMemoryCacheSize(maxTiles);
00308 }
00309
00310 QColor MapRenderer::adjustRouteColorForStyle(const QColor& color) const
00311 {
00312     if (!m_autoRouteContrast)
00313         return color;
00314
00315     QColor adjusted = color;
00316     switch (m_tileCache.mapStyle())
00317     {
00318     case MapStyle::Dark:
00319     case MapStyle::Satellite:
00320         adjusted = adjusted.lighter(145);
00321         break;
00322     case MapStyle::Light:
00323     case MapStyle::Street:
00324     case MapStyle::Minimal:
00325         adjusted = adjusted.darker(120);
00326         break;
00327     case MapStyle::Terrain:
00328     case MapStyle::Topographic:
00329         adjusted = adjusted.lighter(120);
00330         break;
00331     }
00332
00333     return adjusted;
00334 }
00335
00336 const RideRecord* MapRenderer::nearestGpsRecord(const QPointF& screenPos) const
00337 {
00338     if (m_gpsRecords.empty())
00339         return nullptr;
00340
00341     const RideRecord* nearest = nullptr;
00342     double bestDistance = std::numeric_limits<double>::max();
00343
00344     for (const RideRecord* record : m_gpsRecords)
00345     {
00346         const QPointF point = tileToScreen(latLonToTile(record->latitude, record->longitude, m_zoom));
00347         const double distance = QLineF(screenPos, point).length();
00348         if (distance < bestDistance)
00349         {
00350             bestDistance = distance;
00351             nearest = record;
00352         }
00353     }
00354
00355     if (bestDistance > kNearestRecordMaxDistancePx)
00356         return nullptr;
00357
00358     return nearest;
00359 }
00360
00361 void MapRenderer::fitToTrack()
00362 {
00363     if (!m_hasGps) return;
00364
00365     double minLat = 90.0;
00366     double maxLat = -90.0;
00367     double minLon = 180.0;
00368     double maxLon = -180.0;
00369
00370     for (const RideRecord* record : m_gpsRecords)
00371     {
00372         minLat = std::min(minLat, record->latitude);
00373         maxLat = std::max(maxLat, record->latitude);
00374         minLon = std::min(minLon, record->longitude);
00375         maxLon = std::max(maxLon, record->longitude);
00376     }
00377
00378     m_userMovedMap = false;
00379     m_autoFitVisibleRange = true;
00380
00381     fitToBounds(minLat, maxLat, minLon, maxLon, true);

```

```

00382     update();
00383 }
00384
00385 void MapRenderer::fitToVisibleRange()
00386 {
00387     if (!m_hasGps)
00388         return;
00389
00390     double minLat = 90.0;
00391     double maxLat = -90.0;
00392     double minLon = 180.0;
00393     double maxLon = -180.0;
00394     int count = 0;
00395
00396     // Use lower_bound to skip records before the visible range start.
00397     auto rangeBegin = m_gpsRecords.begin();
00398     if (m_visibleStartSeconds >= 0.0)
00399     {
00400         rangeBegin = std::lower_bound(
00401             m_gpsRecords.begin(), m_gpsRecords.end(), m_visibleStartSeconds,
00402             [](const RideRecord* r, double t) { return r->elapsedSeconds < t; });
00403     }
00404
00405     for (auto it = rangeBegin; it != m_gpsRecords.end(); ++it)
00406     {
00407         const RideRecord* record = *it;
00408         if (m_visibleEndSeconds >= 0.0 && record->elapsedSeconds > m_visibleEndSeconds)
00409             break;
00410
00411         minLat = std::min(minLat, record->latitude);
00412         maxLat = std::max(maxLat, record->latitude);
00413         minLon = std::min(minLon, record->longitude);
00414         maxLon = std::max(maxLon, record->longitude);
00415         ++count;
00416     }
00417
00418     if (count < 1)
00419     {
00420         fitToTrack();
00421         return;
00422     }
00423
00424     m_userMovedMap = false;
00425     m_autoFitVisibleRange = true;
00426
00427     fitToBounds(minLat, maxLat, minLon, maxLon, false);
00428     update();
00429 }
00430
00431 void MapRenderer::fitToBounds(double minLat, double maxLat, double minLon, double maxLon, bool
updateMinZoom)
00432 {
00433     if (!m_hasGps)
00434         return;
00435
00436     const auto fit = MapFitMath::computeFitToBounds(minLat,
00437                                                       maxLat,
00438                                                       minLon,
00439                                                       maxLon,
00440                                                       width(),
00441                                                       height(),
00442                                                       std::min(18, m_tileCache.maxZoom()));
00443
00444     m_minLat = fit.minLat;
00445     m_maxLat = fit.maxLat;
00446     m_minLon = fit.minLon;
00447     m_maxLon = fit.maxLon;
00448     m_centerLat = fit.centerLat;
00449     m_centerLon = fit.centerLon;
00450     m_zoom = fit.zoom;
00451
00452     if (updateMinZoom)
00453         m_minZoom = m_zoom; // track-level minimum zoom; selection fit must not tighten this floor
00454 }
00455
00456 void MapRenderer::zoomIn()
00457 {
00458     m_zoom = std::clamp(m_zoom + 1, m_minZoom, std::min(18, m_tileCache.maxZoom()));
00459     m_userMovedMap = true;
00460     m_autoFitVisibleRange = false;
00461     update();
00462 }
00463
00464 void MapRenderer::zoomOut()
00465 {
00466     m_zoom = std::clamp(m_zoom - 1, m_minZoom, std::min(18, m_tileCache.maxZoom()));
00467     m_userMovedMap = true;

```

```

00468     m_autoFitVisibleRange = false;
00469     update();
00470 }
00471
00472 void MapRenderer::setHighlightedElapsedSeconds(double elapsedSeconds)
00473 {
00474     m_highlightElapsedSeconds = elapsedSeconds;
00475     m_currentTimeSeconds = elapsedSeconds;
00476     update();
00477 }
00478
00479 void MapRenderer::paintEvent(QPaintEvent*)
00480 {
00481     QPainter p(this);
00482     p.setRenderHint(QPainter::Antialiasing);
00483     p.fillRect(rect(), QColor(200, 200, 200));
00484
00485     if (!m_hasGps)
00486     {
00487         p.drawText(rect(), Qt::AlignCenter, "No GPS data in this file");
00488         return;
00489     }
00490
00491     const auto centreTile = latLonToTile(m_centerLat, m_centerLon, m_zoom);
00492     const int tileX0 = static_cast<int>(std::floor(
00493         centreTile.x() - static_cast<double>(width()) / (2.0 * kTilePx))) - 1;
00494     const int tileY0 = static_cast<int>(std::floor(
00495         centreTile.y() - static_cast<double>(height()) / (2.0 * kTilePx))) - 1;
00496     const int nX = static_cast<int>(std::ceil(
00497         static_cast<double>(width()) / kTilePx)) + 3;
00498     const int nY = static_cast<int>(std::ceil(
00499         static_cast<double>(height()) / kTilePx)) + 3;
00500     const int maxTile = (1 « m_zoom) - 1;
00501
00502     for (int ty = 0; ty <= nY; ++ty)
00503     {
00504         for (int tx = 0; tx <= nX; ++tx)
00505         {
00506             const int tileX = tileX0 + tx;
00507             const int tileY = tileY0 + ty;
00508             if (tileX < 0 || tileX > maxTile || tileY < 0 || tileY > maxTile)
00509                 continue;
00510
00511             const QPointF topLeft = tileToScreen({ (double)tileX, (double)tileY });
00512             const QPixmap px = m_tileCache.tile(m_zoom, tileX, tileY);
00513
00514             if (px.isNull())
00515                 p.fillRect(QRectF(topLeft, QSizeF(kTilePx, kTilePx)), QColor(210,210,210));
00516             else
00517                 p.drawPixmap(static_cast<int>(topLeft.x()),
00518                     static_cast<int>(topLeft.y()),
00519                     kTilePx, kTilePx, px);
00520         }
00521     }
00522
00523     // Use precomputed GPS record list -- no per-frame allocation.
00524     const auto& gpsRecords = m_gpsRecords;
00525
00526     auto segmentVisible = [this](double startSec, double endSec)
00527     {
00528         if (m_visibleStartSeconds < 0.0 || m_visibleEndSeconds < 0.0)
00529             return true;
00530         const double lo = std::min(m_visibleStartSeconds, m_visibleEndSeconds);
00531         const double hi = std::max(m_visibleStartSeconds, m_visibleEndSeconds);
00532         return endSec >= lo && startSec <= hi;
00533     };
00534
00535     auto drawSegment = [&](const QPointF& a, const QPointF& b, const QColor& color, qreal width)
00536     {
00537         p.setPen(QPen(color, width));
00538         p.drawLine(a, b);
00539     };
00540
00541     // O(log N) time lookup using precomputed sorted GPS records.
00542     auto gpsPointAtTime = [this](double elapsedSeconds) -> const RideRecord*
00543     {
00544         return gpsRecordAtTime(elapsedSeconds);
00545     };
00546
00547     const bool hasVisibleRange = m_visibleStartSeconds >= 0.0 && m_visibleEndSeconds >= 0.0;
00548     const QColor mutedGrey = [&]()
00549     {
00550         QColor c("#d1d5db");
00551         c.setAlpha(80);
00552         return c;
00553     }();
00554

```

```

00555     if (m_routeColorMode == RouteColorMode::None)
00556     {
00557         QPainterPath path;
00558         bool first = true;
00559         for (const RideRecord* record : gpsRecords)
00560         {
00561             const auto sp = tileToScreen(latLonToTile(record->latitude, record->longitude, m_zoom));
00562             if (first) { path.moveTo(sp); first = false; }
00563             else { path.lineTo(sp); }
00564         }
00565         p.setPen(QPen(mutedGrey, 1.5));
00566         p.drawPath(path);
00567
00568         QPainterPath visiblePath;
00569         first = true;
00570         for (size_t index = 1; index < gpsRecords.size(); ++index)
00571         {
00572             const RideRecord& prev = *gpsRecords[index - 1];
00573             const RideRecord& curr = *gpsRecords[index];
00574             if (!segmentVisible(prev.elapsedSeconds, curr.elapsedSeconds))
00575                 continue;
00576
00577             const auto sp = tileToScreen(latLonToTile(prev.latitude, prev.longitude, m_zoom));
00578             const auto ep = tileToScreen(latLonToTile(curr.latitude, curr.longitude, m_zoom));
00579             if (first) { visiblePath.moveTo(sp); first = false; }
00580             visiblePath.lineTo(ep);
00581         }
00582         p.setPen(QPen(adjustRouteColorForStyle(QColor(37, 99, 235)), 4.5));
00583         p.drawPath(visiblePath);
00584     }
00585     else
00586     {
00587         for (size_t index = 1; index < gpsRecords.size(); ++index)
00588         {
00589             const RideRecord& prev = *gpsRecords[index - 1];
00590             const RideRecord& curr = *gpsRecords[index];
00591             const QPointF prevPoint = tileToScreen(latLonToTile(prev.latitude, prev.longitude,
00592 m_zoom));
00593             const QPointF currPoint = tileToScreen(latLonToTile(curr.latitude, curr.longitude,
00594 m_zoom));
00595
00596             // Look up precomputed base color; apply style adjustment per-paint.
00597             const QColor& baseColor = m_segmentColors[index - 1];
00598             const QColor color = baseColor.isValid()
00599                 ? adjustRouteColorForStyle(baseColor)
00600                 : QColor("#cbd5e1");
00601
00602             if (!segmentVisible(prev.elapsedSeconds, curr.elapsedSeconds))
00603             {
00604                 drawSegment(prevPoint, currPoint, mutedGrey, 1.5);
00605             }
00606             else
00607             {
00608                 drawSegment(prevPoint, currPoint, mutedGrey, 1.5);
00609                 drawSegment(prevPoint, currPoint, color, 4.5);
00610             }
00611         }
00612
00613         m_startMarkerRect = QRectF();
00614         m_endMarkerRect = QRectF();
00615
00616         if (hasVisibleRange())
00617         {
00618             const RideRecord* startRecord = gpsPointAtTime(m_visibleStartSeconds);
00619             const RideRecord* endRecord = gpsPointAtTime(m_visibleEndSeconds);
00620
00621             auto drawMarker = [&](const RideRecord* record, const QColor& color, QRectF* markerRect)
00622             {
00623                 if (!record)
00624                     return;
00625                 const QPointF point = tileToScreen(latLonToTile(record->latitude, record->longitude,
00626 m_zoom));
00627                 if (markerRect)
00628                 {
00629                     *markerRect = QRectF(point.x() - kMarkerHitSizePx / 2.0,
00630 point.y() - kMarkerHitSizePx / 2.0,
00631 kMarkerHitSizePx,
00632 kMarkerHitSizePx);
00633                 }
00634                 p.setPen(QPen(QColor("#ffffff"), 2.0));
00635                 p.setBrush(color);
00636                 p.drawEllipse(point, kMarkerRadiusPx, kMarkerRadiusPx);
00637             };
00638
00639             drawMarker(startRecord, QColor("#22c55e"), &m_startMarkerRect);
00640             drawMarker(endRecord, QColor("#ef4444"), &m_endMarkerRect);

```

```

00639     }
00640
00641     auto drawDot = [&](double lat, double lon, QColor c)
00642     {
00643         const auto sp = tileToScreen(latLonToTile(lat, lon, m_zoom));
00644         p.setPen(QPen(Qt::white, 2.0));
00645         p.setBrush(c);
00646         p.drawEllipse(sp, 6.0, 6.0);
00647     };
00648     if (m_firstGpsRecord)
00649         drawDot(m_firstGpsRecord->latitude, m_firstGpsRecord->longitude, QColor(50,180,50));
00650     if (m_lastGpsRecord)
00651         drawDot(m_lastGpsRecord->latitude, m_lastGpsRecord->longitude, QColor(220,50,50));
00652
00653     if (m_currentTimeSeconds >= 0.0)
00654     {
00655         const RideRecord* nearest = gpsRecordAtTime(m_currentTimeSeconds);
00656
00657         if (nearest)
00658         {
00659             const QPointF hp = tileToScreen(latLonToTile(nearest->latitude, nearest->longitude,
00660 m_zoom));
00661             p.setPen(QPen(QColor("#f8f8fc"), 2.0));
00662             p.setBrush(QColor("#111827"));
00663             p.drawEllipse(hp, 7.0, 7.0);
00664             p.setPen(QPen(QColor("#ef4444"), 2.0));
00665             p.setBrush(Qt::NoBrush);
00666             p.drawEllipse(hp, 11.0, 11.0);
00667         }
00668
00669         p.setPen(QColor(60, 60, 60));
00670         p.setFont(QFont("Arial", 8));
00671         p.drawText(rect().adjusted(0,0,-4,-4),
00672             Qt::AlignBottom | Qt::AlignRight,
00673             m_tileCache.attribution());
00674     }
00675
00676 void MapRenderer::wheelEvent(QWheelEvent* event)
00677 {
00678     const int delta = event->angleDelta().y() > 0 ? 1 : -1;
00679     const int oldZoom = m_zoom;
00680     const int newZoom = std::clamp(m_zoom + delta, m_minZoom, std::min(18, m_tileCache.maxZoom()));
00681     if (newZoom == oldZoom)
00682     {
00683         event->accept();
00684         return;
00685     }
00686
00687     const QPointF cursor = event->position();
00688     const QPointF oldCenterTile = latLonToTile(m_centerLat, m_centerLon, oldZoom);
00689     const QPointF screenOffset(
00690         (cursor.x() - width() / 2.0) / kTilePx,
00691         (cursor.y() - height() / 2.0) / kTilePx
00692     );
00693
00694     // World position under the cursor before zoom.
00695     const QPointF anchorTileOld = oldCenterTile + screenOffset;
00696     const double nOld = std::pow(2.0, oldZoom);
00697     const double anchorLon = anchorTileOld.x() / nOld * 360.0 - 180.0;
00698     const double anchorMercN = kPi * (1.0 - 2.0 * anchorTileOld.y() / nOld);
00699     const double anchorLat = std::atan(std::sinh(anchorMercN)) * 180.0 / kPi;
00700
00701     // Recompute center at new zoom so the same world point stays under cursor.
00702     const QPointF anchorTileNew = latLonToTile(anchorLat, anchorLon, newZoom);
00703     const QPointF newCenterTile = anchorTileNew - screenOffset;
00704     const double nNew = std::pow(2.0, newZoom);
00705     m_centerLon = newCenterTile.x() / nNew * 360.0 - 180.0;
00706     const double centerMercN = kPi * (1.0 - 2.0 * newCenterTile.y() / nNew);
00707     m_centerLat = std::atan(std::sinh(centerMercN)) * 180.0 / kPi;
00708     m_centerLat = std::clamp(m_centerLat, -85.051129, 85.051129);
00709     m_zoom = newZoom;
00710     m_userMovedMap = true;
00711     m_autoFitVisibleRange = false;
00712
00713     update();
00714     event->accept();
00715 }
00716
00717 void MapRenderer::mouseDoubleClickEvent(QMouseEvent* event)
00718 {
00719     if (!m_hasGps || event->button() != Qt::LeftButton)
00720     {
00721         QWidget::mouseDoubleClickEvent(event);
00722         return;
00723     }
00724

```



```

00725     const QPointF centreTile = latLonToTile(m_centerLat, m_centerLon, m_zoom);
00726     const double clickedTileX = centreTile.x() + (event->position().x() - width() / 2.0) / kTilePx;
00727     const double clickedTileY = centreTile.y() + (event->position().y() - height() / 2.0) / kTilePx;
00728
00729     const double n = std::pow(2.0, m_zoom);
00730     m_centerLon = clickedTileX / n * 360.0 - 180.0;
00731     const double mercN = kPi * (1.0 - 2.0 * clickedTileY / n);
00732     m_centerLat = std::atan(std::sinh(mercN)) * 180.0 / kPi;
00733     m_centerLat = std::clamp(m_centerLat, -85.051129, 85.051129);
00734
00735     m_zoom = std::clamp(m_zoom + 1, m_minZoom, std::min(18, m_tileCache.maxZoom()));
00736     m_panning = false;
00737     m_userMovedMap = true;
00738     m_autoFitVisibleRange = false;
00739     update();
00740     event->accept();
00741 }
00742
00743 void MapRenderer::mousePressEvent(QMouseEvent* event)
00744 {
00745     if (event->button() == Qt::LeftButton)
00746     {
00747         const QPointF pos = event->position();
00748         if (m_startMarkerRect.contains(pos))
00749         {
00750             m_dragHandle = DragHandle::Start;
00751             m_draggingSelection = true;
00752             m_panning = false;
00753             event->accept();
00754             return;
00755         }
00756
00757         if (m_endMarkerRect.contains(pos))
00758         {
00759             m_dragHandle = DragHandle::End;
00760             m_draggingSelection = true;
00761             m_panning = false;
00762             event->accept();
00763             return;
00764         }
00765
00766         m_panning = true;
00767         m_panStart = event->position();
00768         event->accept();
00769         return;
00770     }
00771
00772     QWidget::mousePressEvent(event);
00773 }
00774
00775 void MapRenderer::mouseMoveEvent(QMouseEvent* event)
00776 {
00777     if (m_draggingSelection)
00778     {
00779         const QPointF pos = event->position();
00780
00781         double panX = 0.0;
00782         double panY = 0.0;
00783         if (pos.x() < kEdgePanMargin)
00784             panX = -0.05;
00785         else if (pos.x() > width() - kEdgePanMargin)
00786             panX = 0.05;
00787
00788         if (pos.y() < kEdgePanMargin)
00789             panY = -0.05;
00790         else if (pos.y() > height() - kEdgePanMargin)
00791             panY = 0.05;
00792
00793         if (panX != 0.0 || panY != 0.0)
00794         {
00795             const QPointF prevTile = latLonToTile(m_centerLat, m_centerLon, m_zoom);
00796             const QPointF newTile = prevTile + QPointF(panX, panY);
00797             const double n = std::pow(2.0, m_zoom);
00798             m_centerLon = newTile.x() / n * 360.0 - 180.0;
00799             const double mercN = kPi * (1.0 - 2.0 * newTile.y() / n);
00800             m_centerLat = std::atan(std::sinh(mercN)) * 180.0 / kPi;
00801             m_centerLat = std::clamp(m_centerLat, -85.051129, 85.051129);
00802             m_userMovedMap = true;
00803             m_autoFitVisibleRange = false;
00804         }
00805
00806         const RideRecord* record = nearestGpsRecord(pos);
00807         if (!record)
00808         {
00809             update();
00810             event->accept();
00811             return;

```

```

00812     }
00813
00814     if (m_dragHandle == DragHandle::Start)
00815         m_visibleStartSeconds = std::min(record->elapsedSeconds, m_visibleEndSeconds);
00816     else if (m_dragHandle == DragHandle::End)
00817         m_visibleEndSeconds = std::max(record->elapsedSeconds, m_visibleStartSeconds);
00818
00819     emit segmentSelectionChanged(m_visibleStartSeconds, m_visibleEndSeconds);
00820     update();
00821     event->accept();
00822     return;
00823 }
00824
00825 if (!m_panning)
00826 {
00827     QWidget::mouseMoveEvent(event);
00828     return;
00829 }
00830
00831 const QPointF delta = event->position() - m_panStart;
00832 m_panStart = event->position();
00833
00834 const auto prevTile = latLonToTile(m_centerLat, m_centerLon, m_zoom);
00835 const double newTileX = prevTile.x() - delta.x() / kTilePx;
00836 const double newTileY = prevTile.y() - delta.y() / kTilePx;
00837
00838 const double n = std::pow(2.0, m_zoom);
00839 m_centerLon = newTileX / n * 360.0 - 180.0;
00840 const double mercN = kPi * (1.0 - 2.0 * newTileY / n);
00841 m_centerLat = std::atan(std::sinh(mercN)) * 180.0 / kPi;
00842 m_centerLat = std::clamp(m_centerLat, -85.051129, 85.051129);
00843 m_userMovedMap = true;
00844 m_autoFitVisibleRange = false;
00845 update();
00846 event->accept();
00847 }
00848
00849 void MapRenderer::mouseReleaseEvent(QMouseEvent* event)
00850 {
00851     if (event->button() == Qt::LeftButton)
00852     {
00853         if (m_draggingSelection)
00854         {
00855             emit segmentSelectionFinished(m_visibleStartSeconds, m_visibleEndSeconds);
00856             m_draggingSelection = false;
00857             m_dragHandle = DragHandle::None;
00858             event->accept();
00859             return;
00860         }
00861
00862         m_panning = false;
00863         event->accept();
00864         return;
00865     }
00866
00867     QWidget::mouseReleaseEvent(event);
00868 }

```

8.119 MapRenderer.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QPointF>
00007 #include <QRectF>
00008 #include <QTimer>
00009 #include <QWidget>
00010
00011 #include "core/zones/ZoneDefinition.h"
00012 #include "fit/RideData.h"
00013 #include "maps/TileCache.h"
00014
00015 enum class RouteColorMode
00016 {
00017     None,
00018     Power,
00019     HeartRate,
00020     Cadence,
00021     Speed,
00022     Altitude,
00023     Gradient
00024 };

```

```

00028
00035 class MapRenderer : public QWidget
00036 {
00037     Q_OBJECT
00038 public:
00039     explicit MapRenderer(QWidget* parent = nullptr);
00040
00041     void setRideData(const RideData& rideData,
00042                     ColorMetric colorMetric = ColorMetric::None,
00043                     const ColorContext& colorContext = {});
00044     void setVisibleTimeRange(double startSeconds, double endSeconds);
00045     void setCurrentTime(double seconds);
00046     void setRouteColorMode(RouteColorMode mode, const ColorContext& colorContext = {});
00047     void setMapStyle(MapStyle style);
00048     MapStyle mapStyle() const { return m_tileCache.mapStyle(); }
00049     void setAutoRouteContrast(bool enabled);
00050     bool autoRouteContrast() const { return m_autoRouteContrast; }
00051     void setTileCacheSize(int maxTiles);
00052
00053 signals:
00054     // Emitted when segment selection changes while dragging.
00055     void segmentSelectionChanged(double startSeconds, double endSeconds);
00056
00057     // Emitted when segment selection completes.
00058     void segmentSelectionFinished(double startSeconds, double endSeconds);
00059
00060 public slots:
00061     void fitToTrack();
00062
00063     void fitToVisibleRange();
00064
00065     void zoomIn();
00066
00067     void zoomOut();
00068
00069     void setHighlightedElapsedSeconds(double elapsedSeconds);
00070
00071 protected:
00072     void paintEvent(QPaintEvent* event) override;
00073
00074     void wheelEvent(QWheelEvent* event) override;
00075
00076     void mouseDoubleClickEvent(QMouseEvent* event) override;
00077
00078     void mousePressEvent(QMouseEvent* event) override;
00079
00080     void mouseMoveEvent(QMouseEvent* event) override;
00081
00082     void mouseReleaseEvent(QMouseEvent* event) override;
00083
00084 private:
00085     enum class DragHandle
00086     {
00087         None,
00088         Start,
00089         End
00090     };
00091
00092     static QPointF latLonToTile(double lat, double lon, int zoom);
00093     void fitToBounds(double minLat, double maxLat, double minLon, double maxLon, bool updateMinZoom);
00094     QPointF tileToScreen(const QPointF& tilePt) const;
00095     QColor adjustRouteColorForStyle(const QColor& color) const;
00096     const RideRecord* nearestGpsRecord(const QPointF& screenPos) const;
00097     void rebuildGpsCache();
00098     void rebuildSegmentColors();
00099     const RideRecord* gpsRecordAtTime(double elapsedSeconds) const;
00100
00101     TileCache m_tileCache;
00102     RideData m_rideData;
00103     // WARNING: m_gpsRecords contains raw pointers into m_rideData.records.
00104     // Any mutation of m_rideData.records requires calling rebuildGpsCache().
00105     // Pre-filtered, time-ordered GPS-only records. Rebuilt once in setRideData().
00106     std::vector<const RideRecord*> m_gpsRecords;
00107     // Per-segment base colors (before map-style adjustment). One per GPS segment.
00108     // Invalid QColor means no data for that segment.
00109     std::vector<QColor> m_segmentColors;
00110     const RideRecord* m_firstGpsRecord = nullptr;
00111     const RideRecord* m_lastGpsRecord = nullptr;
00112     bool m_hasGps = false;
00113     RouteColorMode m_routeColorMode = RouteColorMode::None;
00114     ColorContext m_colorContext;
00115
00116     double m_minLat = 90.0, m_maxLat = -90.0;
00117     double m_minLon = 180.0, m_maxLon = -180.0;
00118
00119     double m_centerLat = 51.5;
00120     double m_centerLon = 0.0;

```

```

00121     int      m_zoom      = 13;
00122     int      m_minZoom   = 1;    // set by fitToTrack(); prevents zooming out past track extent
00123
00124     QPointF  m_panStart;
00125     bool      m_panning   = false;
00126     DragHandle m_dragHandle = DragHandle::None;
00127     QRectF    m_startMarkerRect;
00128     QRectF    m_endMarkerRect;
00129     bool      m_draggingSelection = false;
00130     bool      m_userMovedMap = false;
00131     bool      m_autoFitVisibleRange = true;
00132     bool      m_autoRouteContrast = true;
00133     double    m_highlightElapsedSeconds = -1.0;
00134     double    m_visibleStartSeconds = -1.0;
00135     double    m_visibleEndSeconds = -1.0;
00136     double    m_currentTimeSeconds = -1.0;
00137     QTimer    m_tileRepaintTimer;
00138 };

```

8.120 TileCache.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "TileCache.h"
00005
00006 #include <QDir>
00007 #include <QEventLoop>
00008 #include <QFileInfo>
00009 #include <QImage>
00010 #include <QNetworkReply>
00011 #include <QNetworkRequest>
00012 #include <QStandardPaths>
00013 #include <QUrl>
00014
00015 #include <algorithm>
00016
00017 namespace
00018 {
00019 QString defaultDiskCacheRoot()
00020 {
00021     const QString appData = QStandardPaths::writableLocation(QStandardPaths::AppDataLocation);
00022     return appData.isEmpty()
00023         ? QDir::home().filePath(".fitlyzerc/tiles")
00024         : QDir(appData).filePath("tiles");
00025 }
00026 }
00027
00028 QString TileCache::styleKey(MapStyle style)
00029 {
00030     switch (style)
00031     {
00032     case MapStyle::Street:      return "street";
00033     case MapStyle::Light:      return "light";
00034     case MapStyle::Dark:       return "dark";
00035     case MapStyle::Satellite:  return "satellite";
00036     case MapStyle::Terrain:    return "terrain";
00037     case MapStyle::Topographic: return "topographic";
00038     case MapStyle::Minimal:    return "minimal";
00039     }
00040     return "street";
00041 }
00042
00043 QString TileCache::styleDisplayName(MapStyle style)
00044 {
00045     switch (style)
00046     {
00047     case MapStyle::Street:      return "Street Map";
00048     case MapStyle::Light:      return "Light";
00049     case MapStyle::Dark:       return "Dark";
00050     case MapStyle::Satellite:  return "Satellite";
00051     case MapStyle::Terrain:    return "Terrain";
00052     case MapStyle::Topographic: return "Topographic";
00053     case MapStyle::Minimal:    return "Minimal";
00054     }
00055     return "Street Map";
00056 }
00057
00058 TileProvider TileCache::providerForStyle(MapStyle style)
00059 {
00060     switch (style)
00061     {
00062     case MapStyle::Street:
00063         return {

```

```

00064         "OpenStreetMap Standard",
00065         "https://tile.openstreetmap.org/{z}/{x}/{y}.png",
00066         19,
00067         "© OpenStreetMap contributors"
00068     };
00069     case MapStyle::Light:
00070         return {
00071             "Carto Positron",
00072             "https://a.basemaps.cartocdn.com/light_all/{z}/{x}/{y}.png",
00073             20,
00074             "© OpenStreetMap contributors © CARTO"
00075         };
00076     case MapStyle::Dark:
00077         return {
00078             "Carto Dark Matter",
00079             "https://a.basemaps.cartocdn.com/dark_all/{z}/{x}/{y}.png",
00080             20,
00081             "© OpenStreetMap contributors © CARTO"
00082         };
00083     case MapStyle::Satellite:
00084         return {
00085             "Esri World Imagery",
00086             "https://server.arcgisonline.com/ArcGIS/rest/services/World_Imagery/MapServer/tile/{z}/{y}/{x}",
00087             18,
00088             "Tiles © Esri"
00089         };
00090     case MapStyle::Terrain:
00091         return {
00092             "OpenTopoMap",
00093             "https://a.tile.opentopomap.org/{z}/{x}/{y}.png",
00094             17,
00095             "© OpenStreetMap contributors, SRTM | © OpenTopoMap"
00096         };
00097     case MapStyle::Topographic:
00098         return {
00099             "OpenTopoMap",
00100             "https://a.tile.opentopomap.org/{z}/{x}/{y}.png",
00101             17,
00102             "© OpenStreetMap contributors, SRTM | © OpenTopoMap"
00103         };
00104     case MapStyle::Minimal:
00105         return {
00106             "Carto Positron No Labels",
00107             "https://a.basemaps.cartocdn.com/light_nolabels/{z}/{x}/{y}.png",
00108             20,
00109             "© OpenStreetMap contributors © CARTO"
00110         };
00111     }
00112     return {
00113         "OpenStreetMap Standard",
00114         "https://tile.openstreetmap.org/{z}/{x}/{y}.png",
00115         19,
00116         "© OpenStreetMap contributors"
00117     };
00118 }
00119 }
00120
00121 TileCache::TileCache(const TileCacheConfig& config, QObject* parent)
00122     : QObject(parent)
00123     , m_cache(std::max(1, config.maxTilesInMemory))
00124     , m_diskCacheRoot(config.diskCacheRoot.trimmed().isEmpty()
00125         ? defaultDiskCacheRoot()
00126         : config.diskCacheRoot)
00127     , m_diskFallbackRoots(config.diskFallbackRoots)
00128 {
00129     m_provider = providerForStyle(m_mapStyle);
00130     connect(&m_nam, &QNetworkAccessManager::finished,
00131         this, &TileCache::onReplyFinished);
00132 }
00133
00134 QString TileCache::key(int z, int x, int y) const
00135 {
00136     return keyForStyle(styleKey(m_mapStyle), z, x, y);
00137 }
00138
00139 QString TileCache::keyForStyle(const QString& style, int z, int x, int y)
00140 {
00141     return QString("%1/%2/%3/%4").arg(style).arg(z).arg(x).arg(y);
00142 }
00143
00144 QString TileCache::diskTilePathForRoot(const QString& root, int z, int x, int y) const
00145 {
00146     return diskTilePathForRoot(root, styleKey(m_mapStyle), z, x, y);
00147 }
00148
00149 QString TileCache::diskTilePathForRoot(const QString& root,

```

```

00150                                     const QString& style,
00151                                     int z,
00152                                     int x,
00153                                     int y)
00154 {
00155     return QDir(root).filePath(QString("%1/%2/%3/%4.png")
00156                                .arg(style)
00157                                .arg(z)
00158                                .arg(x)
00159                                .arg(y));
00160 }
00161
00162 QString TileCache::diskTilePath(int z, int x, int y) const
00163 {
00164     return diskTilePathForRoot(m_diskCacheRoot, z, x, y);
00165 }
00166
00167 bool TileCache::isTileCachedOnDisk(int z, int x, int y) const
00168 {
00169     z = std::clamp(z, 1, m_provider.maxZoom);
00170     if (QFileInfo::exists(diskTilePath(z, x, y)))
00171         return true;
00172
00173     for (const QString& fallbackRoot : m_diskFallbackRoots)
00174     {
00175         if (fallbackRoot.trimmed().isEmpty())
00176             continue;
00177         if (QFileInfo::exists(diskTilePathForRoot(fallbackRoot, z, x, y)))
00178             return true;
00179     }
00180
00181     return false;
00182 }
00183
00184 void TileCache::clearMemoryCache()
00185 {
00186     m_cache.clear();
00187     m_pending.clear();
00188     m_downloadQueue.clear();
00189     // m_activeDownloads is not reset here: in-flight replies will still arrive
00190     // and decrement it correctly. dispatchNextDownloads() will be a no-op since
00191     // the queue is now empty.
00192 }
00193
00194 void TileCache::setMemoryCacheSize(int maxTiles)
00195 {
00196     m_cache.setMaxCost(std::max(1, maxTiles));
00197 }
00198
00199 void TileCache::setMapStyle(MapStyle style)
00200 {
00201     if (m_mapStyle == style)
00202         return;
00203
00204     // Cancel in-flight downloads from the previous style to reduce stale work.
00205     const auto activeReplies = m_nam.findChildren<QNetworkReply*>();
00206     for (QNetworkReply* reply : activeReplies)
00207     {
00208         if (reply)
00209         {
00210             reply->setProperty("styleChangeAbort", true);
00211             reply->abort();
00212         }
00213     }
00214
00215     m_mapStyle = style;
00216     m_provider = providerForStyle(style);
00217     clearMemoryCache();
00218 }
00219
00220 QString TileCache::mapStyleName() const
00221 {
00222     return styleDisplayName(m_mapStyle);
00223 }
00224
00225 QString TileCache::attribution() const
00226 {
00227     return m_provider.attribution();
00228 }
00229
00230 int TileCache::maxZoom() const
00231 {
00232     return m_provider.maxZoom();
00233 }
00234
00235 QString TileCache::diskCacheRoot() const
00236 {

```

```

00237     return m_diskCacheRoot;
00238 }
00239
00240 QPixmap TileCache::tile(int z, int x, int y)
00241 {
00242     z = std::clamp(z, 1, m_provider.maxZoom);
00243     const QString k = key(z, x, y);
00244
00245     if (auto* px = m_cache.object(k))
00246         return *px;
00247
00248     const QString diskPath = diskTilePath(z, x, y);
00249     QPixmap diskPixmap;
00250     if (diskPixmap.load(diskPath))
00251     {
00252         m_cache.insert(k, new QPixmap(diskPixmap));
00253         return diskPixmap;
00254     }
00255
00256     for (const QString& fallbackRoot : m_diskFallbackRoots)
00257     {
00258         if (fallbackRoot.trimmed().isEmpty())
00259             continue;
00260
00261         const QString fallbackPath = diskTilePathForRoot(fallbackRoot, z, x, y);
00262         if (!diskPixmap.load(fallbackPath))
00263             continue;
00264
00265         const QFileInfo fi(diskPath);
00266         QDir().mkpath(fi.path());
00267         diskPixmap.save(diskPath);
00268         m_cache.insert(k, new QPixmap(diskPixmap));
00269         return diskPixmap;
00270     }
00271
00272     if (!m_pending.contains(k))
00273     {
00274         m_pending.insert(k);
00275         m_downloadQueue.enqueue({z, x, y});
00276         dispatchNextDownloads();
00277     }
00278
00279     return {}; // empty until the download completes
00280 }
00281
00282 QPixmap TileCache::tileBlocking(int z, int x, int y, bool allowNetwork)
00283 {
00284     z = std::clamp(z, 1, m_provider.maxZoom);
00285     const QString k = key(z, x, y);
00286
00287     if (auto* px = m_cache.object(k))
00288         return *px;
00289
00290     const QString diskPath = diskTilePath(z, x, y);
00291     QPixmap diskPixmap;
00292     if (diskPixmap.load(diskPath))
00293     {
00294         m_cache.insert(k, new QPixmap(diskPixmap));
00295         return diskPixmap;
00296     }
00297
00298     for (const QString& fallbackRoot : m_diskFallbackRoots)
00299     {
00300         if (fallbackRoot.trimmed().isEmpty())
00301             continue;
00302
00303         const QString fallbackPath = diskTilePathForRoot(fallbackRoot, z, x, y);
00304         if (!diskPixmap.load(fallbackPath))
00305             continue;
00306
00307         const QFileInfo fi(diskPath);
00308         QDir().mkpath(fi.path());
00309         diskPixmap.save(diskPath);
00310         m_cache.insert(k, new QPixmap(diskPixmap));
00311         return diskPixmap;
00312     }
00313
00314     if (!allowNetwork)
00315         return {};
00316
00317     QString urlStr = m_provider.urlTemplate;
00318     urlStr.replace("{z}", QString::number(z));
00319     urlStr.replace("{x}", QString::number(x));
00320     urlStr.replace("{y}", QString::number(y));
00321
00322     QNetworkAccessManager nam;
00323     QNetworkRequest req{ QUrl(urlStr) };

```

```

00324     req.setHeader(QNetworkRequest::UserAgentHeader, "FitlyzerC/1.0");
00325     req.setAttribute(
00326         QNetworkRequest::RedirectPolicyAttribute,
00327         QNetworkRequest::NoLessSafeRedirectPolicy);
00328
00329     QNetworkReply* reply = nam.get(req);
00330     QEventLoop loop;
00331     connect(reply, &QNetworkReply::finished, &loop, &QEventLoop::quit);
00332     loop.exec();
00333
00334     QPixmap px;
00335     if (reply->error() == QNetworkReply::NoError)
00336     {
00337         const QByteArray bytes = reply->readAll();
00338         if (px.loadFromData(bytes))
00339         {
00340             const QFileInfo fi(diskPath);
00341             QDir().mkpath(fi.path());
00342             px.save(diskPath);
00343             m_cache.insert(k, new QPixmap(px));
00344         }
00345     }
00346
00347     reply->deleteLater();
00348     return px;
00349 }
00350
00351 void TileCache::dispatchNextDownloads()
00352 {
00353     while (m_activeDownloads < kMaxConcurrentDownloads && !m_downloadQueue.isEmpty())
00354     {
00355         const TileRequest req = m_downloadQueue.dequeue();
00356
00357         QString urlStr = m_provider.urlTemplate;
00358         urlStr.replace("{z}", QString::number(req.z));
00359         urlStr.replace("{x}", QString::number(req.x));
00360         urlStr.replace("{y}", QString::number(req.y));
00361
00362         QNetworkRequest netReq{QUrl(urlStr)};
00363         netReq.setHeader(QNetworkRequest::UserAgentHeader, "FitlyzerC/1.0");
00364         netReq.setAttribute(
00365             QNetworkRequest::RedirectPolicyAttribute,
00366             QNetworkRequest::NoLessSafeRedirectPolicy);
00367
00368         auto* reply = m_nam.get(netReq);
00369         reply->setProperty("z", req.z);
00370         reply->setProperty("x", req.x);
00371         reply->setProperty("y", req.y);
00372         reply->setProperty("style", styleKey(m_mapStyle));
00373
00374         ++m_activeDownloads;
00375     }
00376 }
00377
00378 void TileCache::onReplyFinished(QNetworkReply* reply)
00379 {
00380     reply->deleteLater();
00381
00382     // Safely decrement active downloads counter. Aborted replies or any other
00383     // completion will trigger this, so guard against going negative.
00384     if (m_activeDownloads > 0)
00385         --m_activeDownloads;
00386
00387     const int z = reply->property("z").toInt();
00388     const int x = reply->property("x").toInt();
00389     const int y = reply->property("y").toInt();
00390     const QString replyStyle = reply->property("style").toString();
00391     const bool wasAbortedForStyleChange = reply->property("styleChangeAbort").toBool();
00392     const QString k = keyForStyle(replyStyle, z, x, y);
00393     m_pending.remove(k);
00394
00395     // Always redispach so queued tiles are sent as slots free up.
00396     dispatchNextDownloads();
00397
00398     // Skip processing aborted or stale-style replies
00399     if (wasAbortedForStyleChange || replyStyle != styleKey(m_mapStyle))
00400         return;
00401
00402     if (reply->error() != QNetworkReply::NoError)
00403         return;
00404
00405     QPixmap px;
00406     if (!px.loadFromData(reply->readAll()))
00407         return;
00408
00409     const QString diskPath = diskTilePathForRoot(m_diskCacheRoot, replyStyle, z, x, y);
00410     const QFileInfo fi(diskPath);

```



```

00411     QDir().mkpath(fi.path());
00412     px.save(diskPath);
00413
00414     m_cache.insert(k, new QPixmap(px));
00415     emit tileLoaded(z, x, y);
00416 }

```

8.121 TileCache.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QCache>
00007 #include <QNetworkAccessManager>
00008 #include <QObject>
00009 #include <QPixmap>
00010 #include <QQueue>
00011 #include <QSet>
00012 #include <QString>
00013 #include <QStringList>
00014 enum class MapStyle
00015 {
00016     Street = 0,
00017     Light = 1,
00018     Dark = 4,
00019     Satellite = 5,
00020     Terrain = 6,
00021     Topographic = 7,
00022     Minimal = 8
00023 };
00024
00028 struct TileProvider
00029 {
00030     // Provider name.
00031     QString name;
00032
00033     // URL template with {z}/{x}/{y} placeholders.
00034     QString urlTemplate;
00035
00036     // Maximum zoom level.
00037     int maxZoom = 18;
00038
00039     // Attribution text for provider.
00040     QString attribution;
00041 };
00042
00043 class QNetworkReply;
00044
00048 struct TileCacheConfig
00049 {
00050     // Maximum tiles to hold in memory.
00051     int maxTilesInMemory = 512;
00052
00053     // Primary disk cache directory.
00054     QString diskCacheRoot;
00055
00056     // Additional fallback cache directories.
00057     QStringList diskFallbackRoots;
00058 };
00059
00066 class TileCache : public QObject
00067 {
00068     Q_OBJECT
00069 public:
00070     explicit TileCache(const TileCacheConfig& config = {}, QObject* parent = nullptr);
00071
00072     QPixmap tile(int z, int x, int y);
00073
00074     QPixmap tileBlocking(int z, int x, int y, bool allowNetwork = true);
00075
00076     bool isTileCachedOnDisk(int z, int x, int y) const;
00077
00078     void clearMemoryCache();
00079
00080     void setMemoryCacheSize(int maxTiles);
00081
00082     void setMapStyle(MapStyle style);
00083
00084     MapStyle mapStyle() const { return m_mapStyle; }
00085
00086     QString mapStyleName() const;
00087

```

```

00088     QString attribution() const;
00089
00090     int maxZoom() const;
00091
00092     QString diskCacheRoot() const;
00093
00094 signals:
00095     // Emitted when a tile finishes loading.
00096     void tileLoaded(int z, int x, int y);
00097
00098 private slots:
00099     void onReplyFinished(QNetworkReply* reply);
00100
00101 private:
00102     struct TileRequest { int z, x, y; };
00103
00104     static constexpr int kMaxConcurrentDownloads = 6;
00105
00106     QString key(int z, int x, int y) const;
00107     static QString keyForStyle(const QString& style, int z, int x, int y);
00108     QString diskTilePathForRoot(const QString& root, int z, int x, int y) const;
00109     static QString diskTilePathForRoot(const QString& root,
00110                                       const QString& style,
00111                                       int z,
00112                                       int x,
00113                                       int y);
00114     QString diskTilePath(int z, int x, int y) const;
00115     static QString styleKey(MapStyle style);
00116     static QString styleDisplayName(MapStyle style);
00117     static TileProvider providerForStyle(MapStyle style);
00118
00119     void dispatchNextDownloads();
00120
00121     QNetworkAccessManager    m_nam;
00122     QCache<QString, QPixmap> m_cache;
00123     QSet<QString>            m_pending;
00124     QQueue<TileRequest>      m_downloadQueue;
00125     int                      m_activeDownloads = 0;
00126     QString                  m_diskCacheRoot;
00127     QStringList              m_diskFallbackRoots;
00128     MapStyle                 m_mapStyle = MapStyle::Light;
00129     TileProvider             m_provider;
00130 };

```

8.122 RideDataLoader.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "RideDataLoader.h"
00005
00006 #include <QSqlQuery>
00007
00008 RideData RideDataLoader::loadActivitySamples(int activityId, QSqlDatabase& db)
00009 {
00010     RideData ride;
00011
00012     // Pre-reserve vector to avoid repeated reallocations during large sample loads
00013     {
00014         QSqlQuery countQ(db);
00015         countQ.prepare("SELECT COUNT(*) FROM activity_samples WHERE activity_id=:id");
00016         countQ.bindValue(":id", activityId);
00017         if (countQ.exec() && countQ.next())
00018         {
00019             const int sampleCount = countQ.value(0).toInt();
00020             if (sampleCount > 0)
00021                 ride.records.reserve(sampleCount);
00022         }
00023     }
00024
00025     // Load all samples with forward-only mode for memory efficiency
00026     QSqlQuery samplesQuery(db);
00027     samplesQuery.prepare(
00028         "SELECT elapsed_seconds, power_total, heart_rate, cadence, speed,"
00029         " latitude, longitude, altitude,"
00030         " has_gps, has_power, has_heart_rate, has_cadence, has_speed, has_altitude"
00031         " FROM activity_samples WHERE activity_id=:id ORDER BY elapsed_seconds ASC");
00032     samplesQuery.bindValue(":id", activityId);
00033     samplesQuery.setForwardOnly(true);
00034
00035     if (samplesQuery.exec())
00036     {
00037         while (samplesQuery.next())
00038         {

```

```

00039         RideRecord rec;
00040         rec.elapsedSeconds = samplesQuery.value(0).toDouble();
00041         rec.power          = samplesQuery.value(1).toDouble();
00042         rec.heartRate      = samplesQuery.value(2).toDouble();
00043         rec.cadence        = samplesQuery.value(3).toDouble();
00044         rec.speed          = samplesQuery.value(4).toDouble();
00045         rec.latitude       = samplesQuery.value(5).toDouble();
00046         rec.longitude      = samplesQuery.value(6).toDouble();
00047         rec.altitude       = samplesQuery.value(7).toDouble();
00048         rec.hasGps         = samplesQuery.value(8).toInt() != 0;
00049         rec.hasPower       = samplesQuery.value(9).toInt() != 0;
00050         rec.hasHeartRate   = samplesQuery.value(10).toInt() != 0;
00051         rec.hasCadence     = samplesQuery.value(11).toInt() != 0;
00052         rec.hasSpeed       = samplesQuery.value(12).toInt() != 0;
00053         rec.hasAltitude    = samplesQuery.value(13).toInt() != 0;
00054         ride.records.push_back(rec);
00055     }
00056 }
00057
00058 return ride;
00059 }

```

8.123 RideDataLoader.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include "fit/RideData.h"
00007 #include <QSqlDatabase>
00008
00012 class RideDataLoader
00013 {
00014 public:
00015     static RideData loadActivitySamples(int activityId, QSqlDatabase& db);
00016 };

```

8.124 RideDataSerializer.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "RideDataSerializer.h"
00005 #include "database/DatabaseManager.h"
00006 #include "model/RideDataLoader.h"
00007
00008 #include <QDateTime>
00009 #include <QFileInfo>
00010 #include <QSqlDatabase>
00011 #include <QSqlError>
00012 #include <QSqlQuery>
00013 #include <QVariantList>
00014
00015 // Save
00016
00017 int RideDataSerializer::saveRideToDatabase(
00018     const RideData& rideData,
00019     const WorkoutStatistics& stats,
00020     int athleteId,
00021     const QString& filePath,
00022     const QString& fitHash,
00023     double normalizedPower,
00024     DatabaseManager& dbManager,
00025     ActivityRepository& activityRepo,
00026     ImportRepository& importRepo,
00027     QString* errorOut)
00028 {
00029     QSqlDatabase db = dbManager.database();
00030
00031     // Duplicate check
00032     if (!fitHash.isEmpty() && activityRepo.hasFitHash(fitHash))
00033     {
00034         ImportRecord rec;
00035         rec.fileName = QFileInfo(filePath).fileName();
00036         rec.status = "duplicate";
00037         importRepo.logImport(rec);
00038
00039         if (errorOut)
00040             *errorOut = "Already imported (duplicate file).";
00041         return -1;
00042     }

```

```

00042     }
00043
00044     // Build Activity record
00045     const QString now = QDateTime::currentDateTimeUtc().toString(Qt::ISODate);
00046
00047     Activity act;
00048     act.athleteId      = athleteId;
00049     act.fitHash        = fitHash;
00050     act.fileName       = QFile::fileName(filePath);
00051     act.sport          = "Cycling";
00052     act.startTime      = rideData.activityStartTimeUtc.isEmpty() ? now :
rideData.activityStartTimeUtc;
00053     act.endTime        = rideData.activityEndTimeUtc.isEmpty() ? act.startTime :
rideData.activityEndTimeUtc;
00054     act.durationSec    = stats.durationSeconds;
00055     act.distanceM      = stats.totalDistanceKm * 1000.0;
00056     act.avgPower       = stats.averagePower;
00057     act.maxPower       = stats.maximumPower;
00058     act.normalizedPower = normalizedPower;
00059     act.avgHr          = stats.averageHeartRate;
00060     act.maxHr          = stats.maximumHeartRate;
00061     act.avgCadence     = stats.averageCadence;
00062     act.avgSpeed       = stats.averageSpeed;
00063     act.elevationGain  = stats.elevationGainM;
00064     act.importedAt     = now;
00065
00066     // Transaction: activity row + all samples
00067     if (!db.transaction())
00068     {
00069         if (errorOut) *errorOut = db.lastError().text();
00070         return -1;
00071     }
00072
00073     const int activityId = activityRepo.insertActivity(act);
00074     if (activityId < 0)
00075     {
00076         db.rollback();
00077         ImportRecord rec;
00078         rec.fileName    = act.fileName;
00079         rec.status      = "error";
00080         rec.errorMessage = "Failed to insert activity row.";
00081         importRepo.logImport(rec);
00082         if (errorOut) *errorOut = rec.errorMessage;
00083         return -1;
00084     }
00085
00086     // Bulk-insert samples via execBatch
00087     QSqlQuery q(db);
00088     q.prepare(
00089         "INSERT INTO activity_samples("
00090         "  activity_id, elapsed_seconds,"
00091         "  power_total, heart_rate, cadence, speed,"
00092         "  latitude, longitude, altitude,"
00093         "  has_gps, has_power, has_heart_rate, has_cadence, has_speed, has_altitude"
00094         ") VALUES ("
00095         "  :aid, :es,"
00096         "  :pw, :hr, :cad, :spd,"
00097         "  :lat, :lon, :alt,"
00098         "  :hgps, :hpw, :hhr, :hcad, :hspd, :halt"
00099         ")");
00100
00101     QVariantList aids, es_, pw_, hr_, cad_, spd_;
00102     QVariantList lat_, lon_, alt_;
00103     QVariantList hgps, hpw, hhr, hcad, hspd, halt;
00104
00105     for (const auto& r : rideData.records)
00106     {
00107         aids << activityId;
00108         es_ << r.elapsedSeconds;
00109         pw_ << (r.hasPower ? QVariant(r.power) : QVariant());
00110         hr_ << (r.hasHeartRate ? QVariant(r.heartRate) : QVariant());
00111         cad_ << (r.hasCadence ? QVariant(r.cadence) : QVariant());
00112         spd_ << (r.hasSpeed ? QVariant(r.speed) : QVariant());
00113         lat_ << (r.hasGps ? QVariant(r.latitude) : QVariant());
00114         lon_ << (r.hasGps ? QVariant(r.longitude) : QVariant());
00115         alt_ << (r.hasAltitude ? QVariant(r.altitude) : QVariant());
00116         hgps << (r.hasGps ? 1 : 0);
00117         hpw << (r.hasPower ? 1 : 0);
00118         hhr << (r.hasHeartRate ? 1 : 0);
00119         hcad << (r.hasCadence ? 1 : 0);
00120         hspd << (r.hasSpeed ? 1 : 0);
00121         halt << (r.hasAltitude ? 1 : 0);
00122     }
00123
00124     q.bindValue(":aid", aids);
00125     q.bindValue(":es", es_);
00126     q.bindValue(":pw", pw_);

```

```

00127     q.bindValue(":hr",    hr_);
00128     q.bindValue(":cad",   cad_);
00129     q.bindValue(":spd",   spd_);
00130     q.bindValue(":lat",   lat_);
00131     q.bindValue(":lon",   lon_);
00132     q.bindValue(":alt",   alt_);
00133     q.bindValue(":hgps",  hgps);
00134     q.bindValue(":hpw",   hpw);
00135     q.bindValue(":hhr",   hhr);
00136     q.bindValue(":hcad",  hcad);
00137     q.bindValue(":hspd",  hspd);
00138     q.bindValue(":halt",  halt);
00139
00140     if (!q.execBatch())
00141     {
00142         db.rollback();
00143         ImportRecord rec;
00144         rec.fileName = act.fileName;
00145         rec.status = "error";
00146         rec.errorMessage = "Failed to insert samples: " + q.lastError().text();
00147         importRepo.logImport(rec);
00148         if (errorOut) *errorOut = rec.errorMessage;
00149         return -1;
00150     }
00151
00152     if (!db.commit())
00153     {
00154         db.rollback();
00155         if (errorOut) *errorOut = db.lastError().text();
00156         return -1;
00157     }
00158
00159     // Log success
00160     ImportRecord rec;
00161     rec.fileName = act.fileName;
00162     rec.status = "success";
00163     rec.activityId = activityId;
00164     importRepo.logImport(rec);
00165
00166     return activityId;
00167 }
00168
00169 // Load
00170
00171 RideData RideDataSerializer::loadRideFromDatabase(int activityId,
00172                                                    DatabaseManager& dbManager)
00173 {
00174     QSqlDatabase db = dbManager.database();
00175     return RideDataLoader::loadActivitySamples(activityId, db);
00176 }

```

8.125 RideDataSerializer.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QString>
00007
00008 #include "database/ActivityRepository.h"
00009 #include "database/ImportRepository.h"
00010 #include "analysis/WorkoutAnalyzer.h"
00011 #include "fit/RideData.h"
00012
00013 class DatabaseManager;
00014
00021 class RideDataSerializer
00022 {
00023 public:
00024     static int saveRideToDatabase(
00025         const RideData& rideData,
00026         const WorkoutStatistics& stats,
00027         int athleteId,
00028         const QString& filePath,
00029         const QString& fitHash, // SHA-256 hex of the .fit file bytes
00030         double normalizedPower,
00031         DatabaseManager& dbManager,
00032         ActivityRepository& activityRepo,
00033         ImportRepository& importRepo,
00034         QString* errorOut = nullptr);
00035
00036     static RideData loadRideFromDatabase(int activityId, DatabaseManager& dbManager);
00037 };

```

8.126 Platform.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "Platform.h"
00005
00006 #include <QDir>
00007 #include <QFileInfo>
00008 #include <QStandardPaths>
00009
00010 namespace Platform
00011 {
00012 QString applicationDataDirectory()
00013 {
00014     QString dir = QStandardPaths::writableLocation(QStandardPaths::AppDataLocation);
00015     if (dir.isEmpty())
00016         dir = QDir::homePath() + "/FitlyzerC";
00017
00018     QDir().mkpath(dir);
00019     return dir;
00020 }
00021
00022 QString defaultDatabaseDirectory()
00023 {
00024     const QString documentsDir = QStandardPaths::writableLocation(QStandardPaths::DocumentsLocation);
00025     if (!documentsDir.isEmpty())
00026         return documentsDir;
00027
00028     return applicationDataDirectory();
00029 }
00030
00031 QString defaultDatabasePath()
00032 {
00033     return QDir(defaultDatabaseDirectory()).filePath("fitlyzer.fitlyzerdb");
00034 }
00035
00036 QString defaultBackupPath(const QString& databasePath)
00037 {
00038     if (databasePath.isEmpty())
00039         return QDir(defaultDatabaseDirectory()).filePath("fitlyzer-backup.fitlyzerdb");
00040
00041     const QFileInfo info(databasePath);
00042     const QString baseName = info.completeBaseName().isEmpty()
00043         ? QStringLiteral("fitlyzer")
00044         : info.completeBaseName();
00045     return info.dir().filePath(baseName + "-backup.fitlyzerdb");
00046 }
00047 }

```

8.127 Platform.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QString>
00007
00008 namespace Platform
00009 {
00010 QString applicationDataDirectory();
00011
00012 QString defaultDatabaseDirectory();
00013
00014 QString defaultDatabasePath();
00015
00016 QString defaultBackupPath(const QString& databasePath);
00017
00018 void openInFileManager(const QString& path);
00019 }

```

8.128 Platform_linux.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "Platform.h"
00005
00006 #include <QDesktopServices>

```

```
00007 #include <QFileInfo>
00008 #include <QUrl>
00009
00010 #if defined(Q_OS_LINUX)
00011
00012 namespace Platform
00013 {
00014 void openInFileManager(const QString& path)
00015 {
00016     const QFileInfo info(path);
00017     const QString target = info.isDir() ? info.absoluteFilePath() : info.absolutePath();
00018     QDesktopServices::openUrl(QUrl::fromLocalFile(target));
00019 }
00020 }
00021
00022 #endif
```

8.129 Platform_macos.cpp

```
00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "Platform.h"
00005
00006 #include <QDesktopServices>
00007 #include <QFileInfo>
00008 #include <QUrl>
00009
00010 #if defined(Q_OS_MACOS)
00011
00012 namespace Platform
00013 {
00014 void openInFileManager(const QString& path)
00015 {
00016     const QFileInfo info(path);
00017     const QString target = info.isDir() ? info.absoluteFilePath() : info.absolutePath();
00018     QDesktopServices::openUrl(QUrl::fromLocalFile(target));
00019 }
00020 }
00021
00022 #endif
```

8.130 Platform_windows.cpp

```
00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "Platform.h"
00005
00006 #include <QDesktopServices>
00007 #include <QFileInfo>
00008 #include <QUrl>
00009
00010 #if defined(Q_OS_WIN)
00011
00012 namespace Platform
00013 {
00014 void openInFileManager(const QString& path)
00015 {
00016     const QFileInfo info(path);
00017     const QString target = info.isDir() ? info.absoluteFilePath() : info.absolutePath();
00018     QDesktopServices::openUrl(QUrl::fromLocalFile(target));
00019 }
00020 }
00021
00022 #endif
```

8.131 FfmpegPath.cpp

```
00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "FfmpegPath.h"
00004
00005 #include <QCoreApplication>
00006 #include <QDir>
00007 #include <QFileInfo>
00008 #include <QStringList>
00009 #include <QStandardPaths>
```

```

00010
00011 namespace
00012 {
00013 QString absoluteExecutablePath(const QString& candidate)
00014 {
00015     if (candidate.isEmpty())
00016         return {};
00017
00018     const QFileInfo info(candidate);
00019     if (info.exists() && info.isFile())
00020         return info.absoluteFilePath();
00021
00022     return {};
00023 }
00024 }
00025
00026 QString resolveFfmpegExecutablePath()
00027 {
00028 #ifdef Q_OS_WIN
00029     const QString bundledWindowsPath = absoluteExecutablePath(
00030         QDir(QCoreApplication::applicationDirPath()).filePath("ffmpeg.exe"));
00031     if (!bundledWindowsPath.isEmpty())
00032         return bundledWindowsPath;
00033 #endif
00034
00035 #ifdef FITLYZER_FFmpeg_PATH
00036     const QString configured = QString::fromUtf8(FITLYZER_FFmpeg_PATH);
00037     const QStringList candidates = {
00038         configured,
00039         QDir(QCoreApplication::applicationDirPath()).filePath(configured),
00040         QDir(QCoreApplication::applicationDirPath()).filePath("ffmpeg.exe"),
00041         QDir(QCoreApplication::applicationDirPath()).filePath("ffmpeg"),
00042         QDir(QCoreApplication::applicationDirPath()).filePath("../Resources/ffmpeg"),
00043         QDir(QCoreApplication::applicationDirPath()).filePath("../Resources/ffmpeg.exe")
00044     };
00045
00046     for (const QString& candidate : candidates)
00047     {
00048         const QString resolved = absoluteExecutablePath(candidate);
00049         if (!resolved.isEmpty())
00050             return resolved;
00051     }
00052 #endif
00053
00054     const QString systemFfmpeg = QStandardPaths::findExecutable(QStringLiteral("ffmpeg"));
00055     if (!systemFfmpeg.isEmpty())
00056         return systemFfmpeg;
00057
00058     return QStringLiteral("ffmpeg");
00059 }

```

8.132 FfmpegPath.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <QString>
00006
00007 QString resolveFfmpegExecutablePath();

```

8.133 UuidGenerator.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QString>
00007 #include <QUuid>
00008
00009 namespace UuidGenerator
00010 {
00011     inline QString create()
00012     {
00013         return QUuid::createUuid().toString(QUuid::WithoutBraces);
00014     }
00015 }
00016

```


8.134 VideoExportDialog.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "VideoExportDialog.h"
00005
00006 #include "core/settings/DateFormatter.h"
00007
00008 #include <QCheckBox>
00009 #include <QComboBox>
00010 #include <QDate>
00011 #include <QDialogButtonBox>
00012 #include <QDoubleSpinBox>
00013 #include <QFileDialog>
00014 #include <QFileInfo>
00015 #include <QFormLayout>
00016 #include <QGridLayout>
00017 #include <QGroupBox>
00018 #include <QHBoxLayout>
00019 #include <QLabel>
00020 #include <QLineEdit>
00021 #include <QMessageBox>
00022 #include <QProgressBar>
00023 #include <QPushButton>
00024 #include <QRadioButton>
00025 #include <QScrollArea>
00026 #include <QSettings>
00027 #include <QSpinBox>
00028 #include <QThread>
00029 #include <QVBoxLayout>
00030
00031 #include "video/VideoRenderJob.h"
00032
00033 namespace
00034 {
00035 QString mapStyleName(MapStyle style)
00036 {
00037     switch (style)
00038     {
00039         case MapStyle::Street: return "Street";
00040         case MapStyle::Light: return "Light";
00041         case MapStyle::Dark: return "Dark";
00042         case MapStyle::Satellite: return "Satellite";
00043         case MapStyle::Terrain: return "Terrain";
00044         case MapStyle::Topographic: return "Topographic";
00045         case MapStyle::Minimal: return "Minimal";
00046     }
00047     return "Street";
00048 }
00049
00050 QString makeDefaultFilename(const VideoExportSettings& defaults)
00051 {
00052     QString activity = defaults.activityName.trimmed();
00053     if (activity.isEmpty())
00054         activity = "Ride";
00055
00056     QString athlete = defaults.athleteName.trimmed();
00057     if (athlete.isEmpty())
00058         athlete = "Athlete";
00059
00060     const QString date = DateFormatter::toIsoDate(QDate::currentDate());
00061     QString name = QString("%1-%2-%3.mp4").arg(date, activity, athlete);
00062
00063     static const QString invalid = QStringLiteral("\\\\/:*?\"<>|");
00064     for (const QChar c : invalid)
00065         name.replace(c, '-');
00066
00067     while (name.contains("--"))
00068         name.replace("--", "-");
00069
00070     return name;
00071 }
00072
00073 QString formatVideoLength(double seconds)
00074 {
00075     const int total = static_cast<int>(std::round(std::max(0.0, seconds)));
00076     const int h = total / 3600;
00077     const int m = (total % 3600) / 60;
00078     const int s = total % 60;
00079     if (h > 0)
00080         return QString("%1:%2:%3").arg(h).arg(m, 2, 10, QChar('0')).arg(s, 2, 10, QChar('0'));
00081     return QString("%1:%2").arg(m, 2, 10, QChar('0')).arg(s, 2, 10, QChar('0'));
00082 }
00083 }
00084

```

```

00085 VideoExportDialog::VideoExportDialog(const VideoExportSettings& defaults, QWidget* parent)
00086     : QDialog(parent)
00087     , m_defaults(defaults)
00088 {
00089     setWindowTitle("Create Video");
00090     setModal(true);
00091     resize(620, 720);
00092
00093     auto* root = new QVBoxLayout(this);
00094     auto* scrollArea = new QScrollArea(this);
00095     scrollArea->setWidgetResizable(true);
00096     scrollArea->setHorizontalScrollBarPolicy(Qt::ScrollBarAlwaysOff);
00097
00098     auto* contentWidget = new QWidget(scrollArea);
00099     auto* contentLayout = new QVBoxLayout(contentWidget);
00100     contentLayout->setContentsMargins(0, 0, 0, 0);
00101     contentLayout->setSpacing(root->spacing());
00102
00103     scrollArea->setWidget(contentWidget);
00104
00105     auto* outputGroup = new QGroupBox("General", contentWidget);
00106     auto* outputLayout = new QGridLayout(outputGroup);
00107     m_outputEdit = new QLineEdit(outputGroup);
00108     if (defaults.outputPath.isEmpty())
00109         m_outputEdit->setText(makeDefaultFilename(defaults));
00110     else
00111         m_outputEdit->setText(defaults.outputPath);
00112
00113     auto* browseButton = new QPushButton("Browse...", outputGroup);
00114     connect(browseButton, &QPushButton::clicked, this, &VideoExportDialog::chooseOutputFile);
00115
00116     outputLayout->addWidget(new QLabel("Output file:", 0, 0);
00117     outputLayout->addWidget(m_outputEdit, 0, 1);
00118     outputLayout->addWidget(browseButton, 0, 2);
00119
00120     m_selectedSegmentRadio = new QRadioButton("Current selected segment", outputGroup);
00121     m_entireActivityRadio = new QRadioButton("Entire activity", outputGroup);
00122     m_selectedSegmentRadio->setChecked(defaults.useSelectedSegment);
00123     m_entireActivityRadio->setChecked(!defaults.useSelectedSegment);
00124     outputLayout->addWidget(m_selectedSegmentRadio, 1, 0, 1, 3);
00125     outputLayout->addWidget(m_entireActivityRadio, 2, 0, 1, 3);
00126
00127     auto* videoGroup = new QGroupBox("Video", contentWidget);
00128     auto* videoLayout = new QFormLayout(videoGroup);
00129
00130     m_resolutionCombo = new QComboBox(videoGroup);
00131     m_resolutionCombo->addItem("1280x720", QSize(1280, 720));
00132     m_resolutionCombo->addItem("1920x1080", QSize(1920, 1080));
00133     m_resolutionCombo->addItem("2560x1440", QSize(2560, 1440));
00134     m_resolutionCombo->addItem("3840x2160", QSize(3840, 2160));
00135
00136     const int defaultResIndex = m_resolutionCombo->findData(QSize(defaults.width, defaults.height));
00137     m_resolutionCombo->setCurrentIndex(defaultResIndex >= 0 ? defaultResIndex : 1);
00138
00139     m_fpsSpin = new QSpinBox(videoGroup);
00140     m_fpsSpin->setRange(15, 60);
00141     m_fpsSpin->setValue(defaults.fps <= 0 ? 30 : defaults.fps);
00142
00143     m_speedSpin = new QDoubleSpinBox(videoGroup);
00144     m_speedSpin->setRange(1.0, 20.0);
00145     m_speedSpin->setSingleStep(0.5);
00146     m_speedSpin->setValue(defaults.playbackSpeed < 1.0 ? 3.0 : defaults.playbackSpeed);
00147     m_speedSpin->setSuffix("x");
00148     m_speedSpin->setToolTip("Acceleration factor: 1x = real-time, 5x = 5 seconds FIT per 1 second video.");
00149
00150     videoLayout->addRow("Resolution:", m_resolutionCombo);
00151     videoLayout->addRow("FPS:", m_fpsSpin);
00152     videoLayout->addRow("Playback speed:", m_speedSpin);
00153     m_videoLengthLabel = new QLabel(videoGroup);
00154     videoLayout->addRow("Video Length:", m_videoLengthLabel);
00155     videoLayout->addRow("Format:", new QLabel("MP4 (H.264)", videoGroup));
00156
00157     connect(m_selectedSegmentRadio, &QRadioButton::toggled, this,
00158         &VideoExportDialog::updateVideoLengthLabel);
00159     connect(m_entireActivityRadio, &QRadioButton::toggled, this,
00160         &VideoExportDialog::updateVideoLengthLabel);
00161     connect(m_speedSpin, qOverload<double>(&QDoubleSpinBox::valueChanged), this,
00162         &VideoExportDialog::updateVideoLengthLabel);
00163
00164     auto* mapGroup = new QGroupBox("Map", contentWidget);
00165     auto* mapLayout = new QGridLayout(mapGroup);
00166
00167     m_mapStyleCombo = new QComboBox(mapGroup);
00168     m_mapStyleCombo->addItem(mapStyleName(MapStyle::Street), static_cast<int>(MapStyle::Street));
00169     m_mapStyleCombo->addItem(mapStyleName(MapStyle::Light), static_cast<int>(MapStyle::Light));
00170     m_mapStyleCombo->addItem(mapStyleName(MapStyle::Dark), static_cast<int>(MapStyle::Dark));

```

```

00168     m_mapStyleCombo->addItem(mapStyleName(MapStyle::Satellite),
static_cast<int>(MapStyle::Satellite));
00169     m_mapStyleCombo->addItem(mapStyleName(MapStyle::Terrain), static_cast<int>(MapStyle::Terrain));
00170     m_mapStyleCombo->addItem(mapStyleName(MapStyle::Topographic),
static_cast<int>(MapStyle::Topographic));
00171     m_mapStyleCombo->addItem(mapStyleName(MapStyle::Minimal), static_cast<int>(MapStyle::Minimal));
00172     const int styleIndex = m_mapStyleCombo->findData(static_cast<int>(defaults.mapStyle));
00173     m_mapStyleCombo->setCurrentIndex(styleIndex >= 0 ? styleIndex : 0);
00174
00175     m_fixedZoomSpin = new QSpinBox(mapGroup);
00176     m_fixedZoomSpin->setRange(1, 18);
00177     m_fixedZoomSpin->setValue(defaults.fixedZoom <= 0 ? 18 : defaults.fixedZoom);
00178
00179     mapLayout->addWidget(new QLabel("Style:", 0, 0);
00180     mapLayout->addWidget(m_mapStyleCombo, 0, 1, 1, 2);
00181     mapLayout->addWidget(new QLabel("Zoom:", 1, 0);
00182     mapLayout->addWidget(m_fixedZoomSpin, 1, 1);
00183     auto* followLabel = new QLabel("Follow athlete: enabled", mapGroup);
00184     mapLayout->addWidget(followLabel, 2, 0, 1, 3);
00185
00186     auto* routeGroup = new QGroupBox("Route", contentWidget);
00187     auto* routeLayout = new QFormLayout(routeGroup);
00188     m_routeColorCombo = new QComboBox(routeGroup);
00189     m_routeColorCombo->addItem("FTP Zone", static_cast<int>(ColorMetric::Power));
00190     m_routeColorCombo->addItem("Power", static_cast<int>(ColorMetric::Power));
00191     m_routeColorCombo->addItem("Heart Rate", static_cast<int>(ColorMetric::HeartRate));
00192     m_routeColorCombo->addItem("Cadence", static_cast<int>(ColorMetric::Cadence));
00193     m_routeColorCombo->addItem("Speed", static_cast<int>(ColorMetric::Speed));
00194     m_routeColorCombo->addItem("Altitude", static_cast<int>(ColorMetric::Altitude));
00195     m_routeColorCombo->addItem("Single Color", static_cast<int>(ColorMetric::None));
00196     routeLayout->addRow("Color route by:", m_routeColorCombo);
00197
00198     auto* telemetryGroup = new QGroupBox("Telemetry Overlays", contentWidget);
00199     auto* telemetryLayout = new QGridLayout(telemetryGroup);
00200
00201     m_powerCheck = new QCheckBox("Power", telemetryGroup);
00202     m_hrCheck = new QCheckBox("Heart Rate", telemetryGroup);
00203     m_cadenceCheck = new QCheckBox("Cadence", telemetryGroup);
00204     m_speedCheck = new QCheckBox("Speed", telemetryGroup);
00205     m_altitudeCheck = new QCheckBox("Altitude", telemetryGroup);
00206     m_timeCheck = new QCheckBox("Time", telemetryGroup);
00207     m_athleteCheck = new QCheckBox("Athlete Name", telemetryGroup);
00208     m_telemetryPanelCheck = new QCheckBox("Telemetry Box", telemetryGroup);
00209     m_legendCheck = new QCheckBox("Route Legend", telemetryGroup);
00210     m_chartsCheck = new QCheckBox("Metric Charts", telemetryGroup);
00211
00212     m_powerCheck->setChecked(defaults.overlayPower);
00213     m_hrCheck->setChecked(defaults.overlayHeartRate);
00214     m_cadenceCheck->setChecked(defaults.overlayCadence);
00215     m_speedCheck->setChecked(defaults.overlaySpeed);
00216     m_altitudeCheck->setChecked(defaults.overlayAltitude);
00217     m_timeCheck->setChecked(defaults.overlayTime);
00218     m_athleteCheck->setChecked(defaults.overlayAthleteName);
00219     m_telemetryPanelCheck->setChecked(defaults.overlayTelemetryPanel);
00220     m_legendCheck->setChecked(defaults.overlayRouteLegend);
00221     m_chartsCheck->setChecked(defaults.overlayCharts);
00222
00223     telemetryLayout->addWidget(m_powerCheck, 0, 0);
00224     telemetryLayout->addWidget(m_hrCheck, 0, 1);
00225     telemetryLayout->addWidget(m_cadenceCheck, 0, 2);
00226     telemetryLayout->addWidget(m_speedCheck, 1, 0);
00227     telemetryLayout->addWidget(m_altitudeCheck, 1, 1);
00228     telemetryLayout->addWidget(m_timeCheck, 1, 2);
00229     telemetryLayout->addWidget(m_athleteCheck, 2, 0);
00230     telemetryLayout->addWidget(m_telemetryPanelCheck, 2, 1);
00231     telemetryLayout->addWidget(m_legendCheck, 2, 2);
00232     telemetryLayout->addWidget(m_chartsCheck, 3, 0);
00233
00234     auto* overlayButtons = new QHBoxLayout;
00235     auto* enableAllOverlaysButton = new QPushButton("Enable All", telemetryGroup);
00236     auto* disableAllOverlaysButton = new QPushButton("Disable All", telemetryGroup);
00237     connect(enableAllOverlaysButton, &QPushButton::clicked, this, [this]() {
setAllOverlayChecks(true); });
00238     connect(disableAllOverlaysButton, &QPushButton::clicked, this, [this]() {
setAllOverlayChecks(false); });
00239     overlayButtons->addWidget(enableAllOverlaysButton);
00240     overlayButtons->addWidget(disableAllOverlaysButton);
00241     overlayButtons->addStretch(1);
00242     telemetryLayout->addLayout(overlayButtons, 4, 0, 1, 3);
00243
00244     auto* tileGroup = new QGroupBox("Tiles", contentWidget);
00245     auto* tileLayout = new QVBoxLayout(tileGroup);
00246     m_deleteTemporaryTilesCheck = new QCheckBox("Delete temporary video tiles after export",
tileGroup);
00247     m_deleteTemporaryTilesCheck->setToolTip(
00248         "When enabled, tiles downloaded during export are removed after export finishes or is
canceled.");

```

```

00249     QSettings settings("Fitlyzer", "FitlyzerC");
00250     m_deleteTemporaryTilesCheck->setChecked(
00251         settings.value("video/deleteTemporaryTilesAfterExport", true).toBool());
00252     tileLayout->addWidget(m_deleteTemporaryTilesCheck);
00253
00254     auto* progressGroup = new QGroupBox("Export Progress", contentWidget);
00255     auto* progressLayout = new QVBoxLayout(progressGroup);
00256     m_stageLabel = new QLabel("Ready", progressGroup);
00257     m_frameLabel = new QLabel("Frame: -", progressGroup);
00258     m_etaLabel = new QLabel("ETA: -", progressGroup);
00259     m_tileStatsLabel = new QLabel("Tiles: required - | on-disk - | downloaded -", progressGroup);
00260     m_progressBar = new QProgressBar(progressGroup);
00261     m_progressBar->setRange(0, 100);
00262     m_progressBar->setValue(0);
00263
00264     progressLayout->addWidget(m_stageLabel);
00265     progressLayout->addWidget(m_frameLabel);
00266     progressLayout->addWidget(m_etaLabel);
00267     progressLayout->addWidget(m_tileStatsLabel);
00268     progressLayout->addWidget(m_progressBar);
00269
00270     auto* buttons = new QHBoxLayout;
00271     buttons->addStretch(1);
00272     m_exportButton = new QPushButton("Export", this);
00273     m_cancelButton = new QPushButton("Cancel", this);
00274     buttons->addWidget(m_exportButton);
00275     buttons->addWidget(m_cancelButton);
00276
00277     connect(m_exportButton, &QPushButton::clicked, this, &VideoExportDialog::startExport);
00278     connect(m_cancelButton, &QPushButton::clicked, this, &VideoExportDialog::cancelOrClose);
00279
00280     contentLayout->addWidget(outputGroup);
00281     contentLayout->addWidget(videoGroup);
00282     contentLayout->addWidget(mapGroup);
00283     contentLayout->addWidget(routeGroup);
00284     contentLayout->addWidget(telemetryGroup);
00285     contentLayout->addWidget(tileGroup);
00286     contentLayout->addWidget(progressGroup);
00287     contentLayout->addStretch(1);
00288     root->addWidget(scrollArea);
00289     root->addLayout(buttons);
00290
00291     updateVideoLengthLabel();
00292 }
00293
00294 VideoExportDialog::~VideoExportDialog()
00295 {
00296     if (m_job)
00297         m_job->cancel();
00298     if (m_thread)
00299     {
00300         m_thread->quit();
00301         m_thread->wait();
00302     }
00303 }
00304
00305 void VideoExportDialog::chooseOutputFile()
00306 {
00307     const QString path = QFileDialog::getSaveFileName(
00308         this,
00309         "Export Video",
00310         m_outputEdit->text().trimmed(),
00311         "MP4 Video (*.mp4)");
00312     if (!path.isEmpty())
00313         m_outputEdit->setText(path);
00314 }
00315
00316 VideoExportSettings VideoExportDialog::collectSettings() const
00317 {
00318     VideoExportSettings s = m_defaults;
00319     s.outputPath = m_outputEdit->text().trimmed();
00320
00321     const QSize res = m_resolutionCombo->currentData().toSize();
00322     s.width = res.width();
00323     s.height = res.height();
00324     s.fps = m_fpsSpin->value();
00325     s.playbackSpeed = m_speedSpin->value();
00326
00327     s.useSelectedSegment = m_selectedSegmentRadio->isChecked();
00328
00329     s.mapStyle = static_cast<MapStyle>(m_mapStyleCombo->currentData().toInt());
00330     s.autoZoom = false;
00331     s.fixedZoom = m_fixedZoomSpin->value();
00332
00333     s.routeColorMetric = static_cast<ColorMetric>(m_routeColorCombo->currentData().toInt());
00334
00335     s.overlayPower = m_powerCheck->isChecked();

```

```

00336     s.overlayHeartRate = m_hrCheck->isChecked();
00337     s.overlayCadence = m_cadenceCheck->isChecked();
00338     s.overlaySpeed = m_speedCheck->isChecked();
00339     s.overlayAltitude = m_altitudeCheck->isChecked();
00340     s.overlayTime = m_timeCheck->isChecked();
00341     s.overlayAthleteName = m_athleteCheck->isChecked();
00342     s.overlayTelemetryPanel = m_telemetryPanelCheck->isChecked();
00343     s.overlayRouteLegend = m_legendCheck->isChecked();
00344     s.overlayCharts = m_chartsCheck->isChecked();
00345     s.deleteTemporaryTilesAfterExport = m_deleteTemporaryTilesCheck
00346         ? m_deleteTemporaryTilesCheck->isChecked()
00347         : true;
00348
00349     QSettings appSettings("Fitlyzer", "FitlyzerC");
00350     appSettings.setValue("video/deleteTemporaryTilesAfterExport", s.deleteTemporaryTilesAfterExport);
00351
00352     return s;
00353 }
00354
00355 void VideoExportDialog::startExport()
00356 {
00357     if (m_running)
00358         return;
00359
00360     VideoExportSettings settings = collectSettings();
00361     if (settings.outputPath.isEmpty())
00362     {
00363         QMessageBox::warning(this, "Create Video", "Choose an output file.");
00364         return;
00365     }
00366
00367     if (!settings.outputPath.endsWith(".mp4", Qt::CaseInsensitive))
00368         settings.outputPath += ".mp4";
00369
00370     m_thread = new QThread(this);
00371     m_job = new VideoRenderJob(settings);
00372     m_job->moveToThread(m_thread);
00373
00374     connect(m_thread, &QThread::started, m_job, &VideoRenderJob::run);
00375     connect(m_job, &VideoRenderJob::stageChanged, this, &VideoExportDialog::onStageChanged);
00376     connect(m_job, &VideoRenderJob::progressChanged, this, &VideoExportDialog::onProgressChanged);
00377     connect(m_job, &VideoRenderJob::frameProgressChanged, this,
00378         &VideoExportDialog::onFrameProgressChanged);
00379     connect(m_job, &VideoRenderJob::tileStatsChanged, this, &VideoExportDialog::onTileStatsChanged);
00380     connect(m_job, &VideoRenderJob::finished, this, &VideoExportDialog::onFinished);
00381     connect(m_job, &VideoRenderJob::finished, m_thread, &QThread::quit);
00382     connect(m_thread, &QThread::finished, m_job, &QObject::deleteLater);
00383     connect(m_thread, &QThread::finished, m_thread, &QObject::deleteLater);
00384
00385     m_running = true;
00386     m_exportButton->setEnabled(false);
00387     m_cancelButton->setText("Cancel Export");
00388
00389     m_stageLabel->setText("Preparing data...");
00390     m_frameLabel->setText("Frame: 0 / 0");
00391     m_etaLabel->setText("ETA: --:--");
00392     m_tileStatsLabel->setText("Tiles: required - | on-disk - | downloaded -");
00393     m_progressBar->setValue(0);
00394
00395     m_thread->start();
00396 }
00397
00398 void VideoExportDialog::cancelOrClose()
00399 {
00400     if (m_running && m_job)
00401     {
00402         m_job->cancel();
00403         m_cancelButton->setEnabled(false);
00404         return;
00405     }
00406     reject();
00407 }
00408
00409 void VideoExportDialog::onStageChanged(const QString& stage)
00410 {
00411     m_stageLabel->setText(stage);
00412 }
00413
00414 void VideoExportDialog::onProgressChanged(int value, int maximum)
00415 {
00416     m_progressBar->setRange(0, std::max(1, maximum));
00417     m_progressBar->setValue(value);
00418 }
00419
00420 void VideoExportDialog::onFrameProgressChanged(int currentFrame, int totalFrames, const QString&
    etaText)

```

```

00421 {
00422     m_frameLabel->setText(QString("Frame: %1 / %2").arg(currentFrame).arg(totalFrames));
00423     m_etaLabel->setText(QString("ETA: %1").arg(etaText));
00424 }
00425
00426 void VideoExportDialog::onTileStatsChanged(const QString& statsText)
00427 {
00428     if (m_tileStatsLabel)
00429         m_tileStatsLabel->setText(statsText);
00430 }
00431
00432 void VideoExportDialog::onFinished(bool success, const QString& message, bool canceled)
00433 {
00434     m_running = false;
00435     m_exportButton->setEnabled(true);
00436     m_cancelButton->setEnabled(true);
00437     m_cancelButton->setText("Close");
00438
00439     m_thread = nullptr;
00440     m_job = nullptr;
00441
00442     if (success)
00443     {
00444         QMessageBox::information(this, "Create Video", message);
00445         accept();
00446         return;
00447     }
00448
00449     if (!canceled)
00450         QMessageBox::critical(this, "Create Video", message);
00451 }
00452
00453 void VideoExportDialog::setAllOverlayChecks(bool enabled)
00454 {
00455     m_powerCheck->setChecked(enabled);
00456     m_hrCheck->setChecked(enabled);
00457     m_cadenceCheck->setChecked(enabled);
00458     m_speedCheck->setChecked(enabled);
00459     m_altitudeCheck->setChecked(enabled);
00460     m_timeCheck->setChecked(enabled);
00461     m_athleteCheck->setChecked(enabled);
00462     m_telemetryPanelCheck->setChecked(enabled);
00463     m_legendCheck->setChecked(enabled);
00464     m_chartsCheck->setChecked(enabled);
00465 }
00466
00467 void VideoExportDialog::updateVideoLengthLabel()
00468 {
00469     double realSeconds = 0.0;
00470     if (m_selectedSegmentRadio->isChecked())
00471     {
00472         realSeconds = std::max(0.0, m_defaults.segmentEndSeconds - m_defaults.segmentStartSeconds);
00473     }
00474     else if (!m_defaults.rideData.records.empty())
00475     {
00476         realSeconds = std::max(0.0, m_defaults.rideData.records.back().elapsedSeconds);
00477     }
00478
00479     const double speed = std::max(1.0, m_speedSpin->value());
00480     const double videoSeconds = realSeconds / speed;
00481     m_videoLengthLabel->setText(
00482         QString("Real %1 / Video %2")
00483         .arg(formatVideoLength(realSeconds), formatVideoLength(videoSeconds)));
00484 }

```

8.135 VideoExportDialog.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QDialog>
00007
00008 #include "video/VideoExportSettings.h"
00009
00010 class QButtonGroup;
00011 class QCheckBox;
00012 class QComboBox;
00013 class QDoubleSpinBox;
00014 class QGroupBox;
00015 class QLabel;
00016 class QLineEdit;
00017 class QProgressBar;

```

```

00018 class QPushButton;
00019 class QRadioButton;
00020 class QSpinBox;
00021 class QThread;
00022
00023 class VideoRenderJob;
00024
00031 class VideoExportDialog : public QDialog
00032 {
00033     Q_OBJECT
00034
00035 public:
00041     explicit VideoExportDialog(const VideoExportSettings& defaults, QWidget* parent = nullptr);
00042     ~VideoExportDialog() override;
00043
00044 private slots:
00045     void chooseOutputFile();
00046     void startExport();
00047     void cancelOrClose();
00048     void onStageChanged(const QString& stage);
00049     void onProgressChanged(int value, int maximum);
00050     void onFrameProgressChanged(int currentFrame, int totalFrames, const QString& etaText);
00051     void onTileStatsChanged(const QString& statsText);
00052     void onFinished(bool success, const QString& message, bool canceled);
00053
00054 private:
00055     VideoExportSettings collectSettings() const;
00056     void setAllOverlayChecks(bool enabled);
00057     void updateVideoLengthLabel();
00058
00059     VideoExportSettings m_defaults;
00060
00061     QLineEdit* m_outputEdit = nullptr;
00062     QComboBox* m_resolutionCombo = nullptr;
00063     QSpinBox* m_fpsSpin = nullptr;
00064     QDoubleSpinBox* m_speedSpin = nullptr;
00065     QLabel* m_videoLengthLabel = nullptr;
00066
00067     QRadioButton* m_selectedSegmentRadio = nullptr;
00068     QRadioButton* m_entireActivityRadio = nullptr;
00069
00070     QComboBox* m_mapStyleCombo = nullptr;
00071     QSpinBox* m_fixedZoomSpin = nullptr;
00072
00073     QComboBox* m_routeColorCombo = nullptr;
00074
00075     QCheckBox* m_powerCheck = nullptr;
00076     QCheckBox* m_hrCheck = nullptr;
00077     QCheckBox* m_cadenceCheck = nullptr;
00078     QCheckBox* m_speedCheck = nullptr;
00079     QCheckBox* m_altitudeCheck = nullptr;
00080     QCheckBox* m_timeCheck = nullptr;
00081     QCheckBox* m_athleteCheck = nullptr;
00082     QCheckBox* m_telemetryPanelCheck = nullptr;
00083     QCheckBox* m_legendCheck = nullptr;
00084     QCheckBox* m_chartsCheck = nullptr;
00085     QCheckBox* m_deleteTemporaryTilesCheck = nullptr;
00086
00087     QLabel* m_stageLabel = nullptr;
00088     QLabel* m_frameLabel = nullptr;
00089     QLabel* m_etaLabel = nullptr;
00090     QLabel* m_tileStatsLabel = nullptr;
00091     QProgressBar* m_progressBar = nullptr;
00092
00093     QPushButton* m_exportButton = nullptr;
00094     QPushButton* m_cancelButton = nullptr;
00095
00096     QThread* m_thread = nullptr;
00097     VideoRenderJob* m_job = nullptr;
00098     bool m_running = false;
00099 };

```

8.136 VideoExportSettings.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QColor>
00007 #include <QString>
00008
00009 #include "core/zones/ZoneDefinition.h"
00010 #include "fit/RideData.h"

```

```

00011 #include "maps/TileCache.h"
00012
00019 struct VideoExportSettings
00020 {
00022     QString outputPath;
00023
00025     QString ffmpegPath;
00026
00028     int width = 1920;
00029
00031     int height = 1080;
00032
00034     int fps = 30;
00035
00037     double playbackSpeed = 3.0;
00038
00040     bool useSelectedSegment = true;
00041
00043     double segmentStartSeconds = 0.0;
00044
00046     double segmentEndSeconds = 0.0;
00047
00049     MapStyle mapStyle = MapStyle::Light;
00050
00052     bool autoZoom = true;
00053
00055     int fixedZoom = 18;
00056
00058     bool followAthlete = true;
00059
00061     ColorMetric routeColorMetric = ColorMetric::Power;
00062
00064     QColor singleRouteColor = QColor("#2563eb");
00065
00067     bool overlayPower = true;
00068
00070     bool overlayHeartRate = true;
00071
00073     bool overlayCadence = true;
00074
00076     bool overlaySpeed = true;
00077
00079     bool overlayAltitude = true;
00080
00082     bool overlayTime = true;
00083
00085     bool overlayAthleteName = true;
00086
00088     bool overlayTelemetryPanel = true;
00089
00091     bool overlayRouteLegend = true;
00092
00094     bool overlayCharts = true;
00095
00097     bool deleteTemporaryTilesAfterExport = true;
00098
00100     QString athleteName;
00101
00103     QString activityName;
00104
00106     ColorContext colorContext;
00107
00109     RideData rideData;
00110 };

```

8.137 VideoRenderJob.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "VideoRenderJob.h"
00005
00006 #include <QDateTime>
00007 #include <QDir>
00008 #include <QElapsedTimer>
00009 #include <QFileInfo>
00010 #include <QImage>
00011 #include <QPainter>
00012 #include <QPainterPath>
00013 #include <QPen>
00014 #include <QProcess>
00015 #include <QSet>
00016 #include <QStringList>
00017

```



```

00018 #include "core/zones/ColorProvider.h"
00019 #include "core/zones/ZoneCalculator.h"
00020 #include "maps/MapFitMath.h"
00021 #include "video/VideoTileProvider.h"
00022 #include "utils/FfmpegPath.h"
00023
00024 #include <algorithm>
00025 #include <cmath>
00026 #include <limits>
00027 #include <unordered_set>
00028
00029 namespace
00030 {
00031     static constexpr int kTilePx = 256;
00032     static constexpr double kPi = 3.14159265358979323846;
00033     static constexpr int kVideoExportTileCache = 96;
00034
00035     struct TileViewport
00036     {
00037         int tileX0 = 0;
00038         int tileY0 = 0;
00039         int nX = 0;
00040         int nY = 0;
00041
00042         bool operator==(const TileViewport& other) const
00043         {
00044             return tileX0 == other.tileX0 &&
00045                    tileY0 == other.tileY0 &&
00046                    nX == other.nX &&
00047                    nY == other.nY;
00048         }
00049     };
00050
00051     TileViewport computeTileViewport(const QPointF& centerTile, int width, int height)
00052     {
00053         TileViewport viewport;
00054         viewport.tileX0 = static_cast<int>(std::floor(centerTile.x() - static_cast<double>(width) / (2.0 *
00055 kTilePx))) - 1;
00056         viewport.tileY0 = static_cast<int>(std::floor(centerTile.y() - static_cast<double>(height) / (2.0
00057 * kTilePx))) - 1;
00058         viewport.nX = static_cast<int>(std::ceil(static_cast<double>(width) / kTilePx)) + 3;
00059         viewport.nY = static_cast<int>(std::ceil(static_cast<double>(height) / kTilePx)) + 3;
00060         return viewport;
00061     }
00062
00063     std::vector<VideoTileId> tilesForViewport(const TileViewport& viewport, int zoom)
00064     {
00065         std::vector<VideoTileId> tiles;
00066         const int maxTile = (1 << zoom) - 1;
00067         tiles.reserve(static_cast<size_t>((viewport.nX + 1) * (viewport.nY + 1)));
00068
00069         for (int ty = 0; ty <= viewport.nY; ++ty)
00070         {
00071             for (int tx = 0; tx <= viewport.nX; ++tx)
00072             {
00073                 const int tileX = viewport.tileX0 + tx;
00074                 const int tileY = viewport.tileY0 + ty;
00075                 if (tileX < 0 || tileY < 0 || tileX > maxTile || tileY > maxTile)
00076                     continue;
00077                 tiles.push_back({ zoom, tileX, tileY });
00078             }
00079         }
00080         return tiles;
00081     }
00082
00083     QPointF latLonToTile(double lat, double lon, int zoom)
00084     {
00085         const double n = std::pow(2.0, zoom);
00086         const double x = (lon + 180.0) / 360.0 * n;
00087         const double rad = lat * kPi / 180.0;
00088         const double y = (1.0 - std::log(std::tan(rad) + 1.0 / std::cos(rad)) / kPi) / 2.0 * n;
00089         return { x, y };
00090     }
00091
00092     QPointF tileToScreen(const QPointF& tilePt, const QPointF& centerTile, int width, int height)
00093     {
00094         return {
00095             width / 2.0 + (tilePt.x() - centerTile.x()) * kTilePx,
00096             height / 2.0 + (tilePt.y() - centerTile.y()) * kTilePx
00097         };
00098     }
00099
00100     int nearestRecordIndex(const std::vector<RideRecord>& records, double elapsedSeconds)
00101     {
00102         if (records.empty())
00103             return -1;
00104     }

```

```

00103
00104     int best = 0;
00105     double bestDist = std::numeric_limits<double>::max();
00106     for (int i = 0; i < static_cast<int>(records.size()); ++i)
00107     {
00108         const double dist = std::abs(records[static_cast<size_t>(i)].elapsedSeconds - elapsedSeconds);
00109         if (dist < bestDist)
00110         {
00111             bestDist = dist;
00112             best = i;
00113         }
00114     }
00115     return best;
00116 }
00117
00118 bool interpolateGpsAtTime(const std::vector<const RideRecord*>& gpsRecords,
00119                          double elapsedSeconds,
00120                          double& outLat,
00121                          double& outLon)
00122 {
00123     if (gpsRecords.empty())
00124         return false;
00125
00126     if (elapsedSeconds <= gpsRecords.front()->elapsedSeconds)
00127     {
00128         outLat = gpsRecords.front()->latitude;
00129         outLon = gpsRecords.front()->longitude;
00130         return true;
00131     }
00132
00133     if (elapsedSeconds >= gpsRecords.back()->elapsedSeconds)
00134     {
00135         outLat = gpsRecords.back()->latitude;
00136         outLon = gpsRecords.back()->longitude;
00137         return true;
00138     }
00139
00140     auto it = std::lower_bound(
00141         gpsRecords.begin(),
00142         gpsRecords.end(),
00143         elapsedSeconds,
00144         [](const RideRecord* rec, double t)
00145         {
00146             return rec->elapsedSeconds < t;
00147         });
00148
00149     if (it == gpsRecords.begin())
00150     {
00151         outLat = (*it)->latitude;
00152         outLon = (*it)->longitude;
00153         return true;
00154     }
00155
00156     const RideRecord* b = *it;
00157     const RideRecord* a = *(it - 1);
00158     const double dt = b->elapsedSeconds - a->elapsedSeconds;
00159     if (dt <= 1e-6)
00160     {
00161         outLat = a->latitude;
00162         outLon = a->longitude;
00163         return true;
00164     }
00165
00166     const double alpha = std::clamp((elapsedSeconds - a->elapsedSeconds) / dt, 0.0, 1.0);
00167     outLat = a->latitude + (b->latitude - a->latitude) * alpha;
00168     outLon = a->longitude + (b->longitude - a->longitude) * alpha;
00169     return true;
00170 }
00171
00172 QString fmtDuration(double seconds)
00173 {
00174     const int total = static_cast<int>(std::round(std::max(0.0, seconds)));
00175     const int h = total / 3600;
00176     const int m = (total % 3600) / 60;
00177     const int s = total % 60;
00178     if (h > 0)
00179         return QString("%1:%2:%3").arg(h).arg(m, 2, 10, QChar('0')).arg(s, 2, 10, QChar('0'));
00180     return QString("%1:%2").arg(m, 2, 10, QChar('0')).arg(s, 2, 10, QChar('0'));
00181 }
00182
00183 QString formatLegendValue(double value, ColorMetric metric)
00184 {
00185     switch (metric)
00186     {
00187     case ColorMetric::Speed:
00188         return QString::number(value, 'f', 1);
00189     case ColorMetric::Power:

```

```

00190         case ColorMetric::HeartRate:
00191         case ColorMetric::Cadence:
00192         case ColorMetric::Altitude:
00193             return QString::number(static_cast<int>(std::round(value)));
00194         case ColorMetric::Gradient:
00195         case ColorMetric::None:
00196             return QString::number(value, 'f', 1);
00197     }
00198
00199     return QString::number(value, 'f', 1);
00200 }
00201
00202 QString zoneLegendText(const Zone& zone, ColorMetric metric)
00203 {
00204     const QString unit = colorMetricUnit(metric);
00205     const bool hasUnit = !unit.isEmpty();
00206     const QString valueSuffix = hasUnit ? QString("%1").arg(unit) : QString();
00207     const double hugeThreshold = std::numeric_limits<double>::max() * 0.5;
00208     const bool openEnded = !std::isfinite(zone.maxValue) || zone.maxValue >= hugeThreshold;
00209
00210     if (openEnded)
00211         return QString("%1 > %2%3").arg(zone.name, formatLegendValue(zone.minValue, metric),
00212 valueSuffix);
00213     if (zone.minValue <= 0.0)
00214         return QString("%1 < %2%3").arg(zone.name, formatLegendValue(zone.maxValue, metric),
00215 valueSuffix);
00216     return QString("%1 %2-%3%4")
00217         .arg(zone.name,
00218             formatLegendValue(zone.minValue, metric),
00219             formatLegendValue(zone.maxValue, metric),
00220             valueSuffix);
00221 }
00222
00223 QColor routeColor(const RideRecord& rec,
00224                  ColorMetric metric,
00225                  const ColorContext& context,
00226                  const QColor& singleColor)
00227 {
00228     if (metric == ColorMetric::None)
00229         return singleColor;
00230     return ColorProvider::colorForMetric(metric,
00231                                         metric == ColorMetric::Power ? rec.power :
00232                                         metric == ColorMetric::HeartRate ? rec.heartRate :
00233                                         metric == ColorMetric::Cadence ? rec.cadence :
00234                                         metric == ColorMetric::Speed ? rec.speed :
00235                                         metric == ColorMetric::Altitude ? rec.altitude : 0.0,
00236                                         context);
00237 }
00238
00239 class ExportTileCleanupGuard
00240 {
00241 public:
00242     explicit ExportTileCleanupGuard(VideoTileProvider& provider)
00243         : m_provider(provider)
00244     {}
00245     ~ExportTileCleanupGuard()
00246     {
00247         m_provider.clearMemoryCache();
00248         m_provider.cleanupExportCache();
00249     }
00250 private:
00251     VideoTileProvider& m_provider;
00252 };
00253
00254 VideoRenderJob::VideoRenderJob(VideoExportSettings settings, QObject* parent)
00255     : QObject(parent)
00256     , m_settings(std::move(settings))
00257 {}
00258
00259 void VideoRenderJob::cancel()
00260 {
00261     m_cancelRequested.storeRelease(1);
00262 }
00263
00264 void VideoRenderJob::run()
00265 {
00266     const auto canceled = [this]() { return m_cancelRequested.loadAcquire() != 0; };
00267     if (m_settings.rideData.records.empty())
00268     {
00269         emit finished(false, "No ride data to export.", false);
00270         return;
00271     }
00272 }
00273
00274

```

```

00275     QString ffmpegPath = m_settings.ffmpegPath.trimmed();
00276     if (ffmpegPath.isEmpty())
00277         ffmpegPath = resolveFfmpegExecutablePath();
00278
00279     const QFileInfo outInfo(m_settings.outputPath);
00280     QDir().mkpath(outInfo.absolutePath());
00281
00282     double startSec = std::max(0.0, m_settings.segmentStartSeconds);
00283     double endSec = std::max(startSec, m_settings.segmentEndSeconds);
00284     if (!m_settings.useSelectedSegment)
00285     {
00286         startSec = 0.0;
00287         endSec = m_settings.rideData.records.back().elapsedSeconds;
00288     }
00289
00290     if (endSec <= startSec)
00291     {
00292         emit finished(false, "Selected segment is empty.", false);
00293         return;
00294     }
00295
00296     const int fps = std::max(1, m_settings.fps);
00297     const double playbackSpeed = std::max(1.0, m_settings.playbackSpeed);
00298
00299     std::vector<const RideRecord*> gpsRecords;
00300     gpsRecords.reserve(m_settings.rideData.records.size());
00301     double minLat = 90.0;
00302     double maxLat = -90.0;
00303     double minLon = 180.0;
00304     double maxLon = -180.0;
00305
00306     for (const RideRecord& rec : m_settings.rideData.records)
00307     {
00308         if (!rec.hasGps)
00309             continue;
00310         if (rec.elapsedSeconds < startSec || rec.elapsedSeconds > endSec)
00311             continue;
00312
00313         gpsRecords.push_back(&rec);
00314         minLat = std::min(minLat, rec.latitude);
00315         maxLat = std::max(maxLat, rec.latitude);
00316         minLon = std::min(minLon, rec.longitude);
00317         maxLon = std::max(maxLon, rec.longitude);
00318     }
00319
00320     if (gpsRecords.size() < 2)
00321     {
00322         emit finished(false, "Video export requires GPS data in the selected segment.", false);
00323         return;
00324     }
00325
00326     // If the selected segment begins before the first GPS fix, start rendering at
00327     // the first GPS timestamp so the route animation starts immediately.
00328     startSec = std::max(startSec, gpsRecords.front()->elapsedSeconds);
00329     if (endSec <= startSec)
00330     {
00331         emit finished(false, "Selected segment has no duration after GPS alignment.", false);
00332         return;
00333     }
00334
00335     const double sourceDuration = endSec - startSec;
00336     const double outputDuration = sourceDuration / playbackSpeed;
00337     const int totalFrames = std::max(1, static_cast<int>(std::ceil(outputDuration * fps)));
00338
00339     emit stageChanged("Preparing data...");
00340     emit progressChanged(0, totalFrames);
00341
00342     const bool showAnyMetricChart = m_settings.overlayCharts &&
00343         (m_settings.overlayPower ||
00344          m_settings.overlayHeartRate ||
00345          m_settings.overlayCadence ||
00346          m_settings.overlaySpeed ||
00347          m_settings.overlayAltitude);
00348     const int mapHeight = showAnyMetricChart
00349         ? static_cast<int>(std::round(m_settings.height * 0.72))
00350         : m_settings.height;
00351     const int chartHeight = showAnyMetricChart
00352         ? std::max(120, m_settings.height - mapHeight)
00353         : 0;
00354     const double uiScale = std::clamp(static_cast<double>(m_settings.height) / 720.0, 0.85, 2.6);
00355
00356     const int telemetryMargin = static_cast<int>(std::round(18.0 * uiScale));
00357     const int telemetryWidth = static_cast<int>(std::round(320.0 * uiScale));
00358     const int telemetryHeight = static_cast<int>(std::round(180.0 * uiScale));
00359     const int telemetryLeftPadding = static_cast<int>(std::round(12.0 * uiScale));
00360     const int telemetryTopBaseline = static_cast<int>(std::round(26.0 * uiScale));
00361     const int telemetryLineStep = static_cast<int>(std::round(20.0 * uiScale));

```

```

00362     const int telemetryFontPx = static_cast<int>(std::round(12.0 * uiScale));
00363
00364     const int athleteWidth = static_cast<int>(std::round(344.0 * uiScale));
00365     const int athleteHeight = static_cast<int>(std::round(42.0 * uiScale));
00366     const int athleteInset = static_cast<int>(std::round(12.0 * uiScale));
00367     const int athleteFontPx = static_cast<int>(std::round(13.0 * uiScale));
00368
00369     const int chartSidePadding = static_cast<int>(std::round(12.0 * uiScale));
00370     const int chartTopPadding = static_cast<int>(std::round(4.0 * uiScale));
00371     const int chartBottomPadding = static_cast<int>(std::round(8.0 * uiScale));
00372     const int chartLabelFontPx = static_cast<int>(std::round(9.0 * uiScale));
00373     const int chartLabelBaseline = static_cast<int>(std::round(14.0 * uiScale));
00374
00375     const int legendInset = static_cast<int>(std::round(8.0 * uiScale));
00376     const int legendGap = static_cast<int>(std::round(8.0 * uiScale));
00377     const int legendSwatchSize = static_cast<int>(std::round(12.0 * uiScale));
00378     const int legendTitleFontPx = static_cast<int>(std::round(9.0 * uiScale));
00379     const int legendLabelFontPx = static_cast<int>(std::round(8.5 * uiScale));
00380
00381     const std::vector<Zone> routeLegendZones =
ZoneCalculator::zonesForMetric(m_settings.routeColorMetric,
00382 m_settings.colorContext);
00383
00384     const bool showRouteLegend = m_settings.overlayRouteLegend &&
00385         m_settings.routeColorMetric != ColorMetric::None &&
00386         !routeLegendZones.empty();
00387
00388     VideoTileProvider tileProvider(VideoTileProviderConfig{
00389         m_settings.mapStyle,
00390         kVideoExportTileCache,
00391         m_settings.deleteTemporaryTilesAfterExport,
00392         QString()
00393     });
00394     ExportTileCleanupGuard tileCleanupGuard(tileProvider);
00395
00396     int zoom = std::clamp(m_settings.fixedZoom, 1, 18);
00397     if (m_settings.autoZoom)
00398     {
00399         const auto fit = MapFitMath::computeFitToBounds(minLat,
00400                                                         maxLat,
00401                                                         minLon,
00402                                                         maxLon,
00403                                                         m_settings.width,
00404                                                         mapHeight,
00405                                                         18);
00406         zoom = fit.zoom;
00407     }
00408
00409     std::vector<QPointF> gpsTilePoints;
00410     gpsTilePoints.reserve(gpsRecords.size());
00411     for (const RideRecord* rec : gpsRecords)
00412         gpsTilePoints.push_back(latLonToTile(rec->latitude, rec->longitude, zoom));
00413
00414     emit stageChanged("Analyzing required tiles...");
00415
00416     std::unordered_set<VideoTileId, VideoTileIdHash> requiredTiles;
00417     requiredTiles.reserve(static_cast<size_t>(std::max(256, totalFrames / 3)));
00418     for (int frame = 0; frame < totalFrames; ++frame)
00419     {
00420         if (canceled())
00421         {
00422             emit finished(false, "Export canceled.", true);
00423             return;
00424         }
00425
00426         const double sourceSeconds = std::min(
00427             endSec,
00428             startSec + (static_cast<double>(frame) / fps) * playbackSpeed);
00429         double centerLat = 0.0;
00430         double centerLon = 0.0;
00431         if (!interpolateGpsAtTime(gpsRecords, sourceSeconds, centerLat, centerLon))
00432             continue;
00433
00434         const QPointF centerTile = latLonToTile(centerLat, centerLon, zoom);
00435         const TileViewport viewport = computeTileViewport(centerTile, m_settings.width, mapHeight);
00436         const std::vector<VideoTileId> tiles = tilesForViewport(viewport, zoom);
00437         requiredTiles.insert(tiles.begin(), tiles.end());
00438
00439         if (frame % 120 == 0)
00440             emit progressChanged(frame, totalFrames);
00441     }
00442
00443     tileProvider.setRequiredTiles(requiredTiles);
00444     {
00445         const VideoTileStats s = tileProvider.stats();
00446         emit tileStatsChanged(QString("Tiles: required %1 | on-disk %2 | downloaded 0")
00447             .arg(s.requiredTiles)

```

```

00447         .arg(s.onDiskTiles));
00448     }
00449     emit stageChanged(QString("Analyzed %1 required tiles (%2 available on disk)")
00450         .arg(tileProvider.requiredTileCount())
00451         .arg(tileProvider.requiredTilesAvailableOnDisk()));
00452
00453     emit stageChanged("Encoding video...");
00454
00455     QProcess ffmpeg;
00456     ffmpeg.setProgram(ffmpegPath);
00457     ffmpeg.setArguments({
00458         "-y",
00459         "-f", "rawvideo",
00460         "-pixel_format", "rgba",
00461         "-video_size", QString("%1x%2").arg(m_settings.width).arg(m_settings.height),
00462         "-framerate", QString::number(fps),
00463         "-i", "-",
00464         "-an",
00465         "-c:v", "libx264",
00466         "-preset", "medium",
00467         "-crf", "20",
00468         "-pix_fmt", "yuv420p",
00469         m_settings.outputPath
00470     });
00471     ffmpeg.setProcessChannelMode(QProcess::MergedChannels);
00472     ffmpeg.start();
00473
00474     if (!ffmpeg.waitForStarted())
00475     {
00476         const QString detail = ffmpeg.errorString().trimmed();
00477         if (detail.isEmpty())
00478         {
00479             emit finished(false, QString("Failed to start FFmpeg at '%1'.").arg(ffmpegPath), false);
00480         }
00481         else
00482         {
00483             emit finished(false, QString("Failed to start FFmpeg at '%1': %2").arg(ffmpegPath,
00484 detail), false);
00485         }
00486         return;
00487     }
00488
00489     static constexpr double kChartWindowBackSeconds = 180.0;
00490     static constexpr double kChartWindowForwardSeconds = 30.0;
00491
00492     QElapsedTimer timer;
00493     timer.start();
00494
00495     TileViewport lastViewport;
00496     bool hasLastViewport = false;
00497     std::vector<VideoTileId> viewportTiles;
00498     int currentGpsIndex = 0;
00499
00500     for (int frame = 0; frame < totalFrames; ++frame)
00501     {
00502         if (canceled())
00503         {
00504             ffmpeg.kill();
00505             ffmpeg.waitForFinished();
00506             emit finished(false, "Export canceled.", true);
00507             return;
00508         }
00509
00510         const double sourceSeconds = std::min(
00511             endSec,
00512             startSec + (static_cast<double>(frame) / fps) * playbackSpeed);
00513         const int recIndex = nearestRecordIndex(m_settings.rideData.records, sourceSeconds);
00514         if (recIndex < 0)
00515             continue;
00516
00517         const RideRecord& now = m_settings.rideData.records[static_cast<size_t>(recIndex)];
00518         double centerLat = 0.0;
00519         double centerLon = 0.0;
00520         if (!interpolateGpsAtTime(gpsRecords, sourceSeconds, centerLat, centerLon))
00521             continue;
00522         const QPointF centerTile = latLonToTile(centerLat, centerLon, zoom);
00523
00524         const TileViewport viewport = computeTileViewport(centerTile, m_settings.width, mapHeight);
00525         if (!hasLastViewport || !viewport == lastViewport)
00526         {
00527             viewportTiles = tilesForViewport(viewport, zoom);
00528             lastViewport = viewport;
00529             hasLastViewport = true;
00530         }
00531
00532         while (currentGpsIndex + 1 < static_cast<int>(gpsRecords.size()) &&
00533             gpsRecords[static_cast<size_t>(currentGpsIndex + 1)]->elapsedSeconds <= sourceSeconds)

```

```

00533     {
00534         ++currentGpsIndex;
00535     }
00536
00537     QImage image(m_settings.width, m_settings.height, QImage::Format_RGBA8888);
00538     image.fill(QColor("#0f172a"));
00539
00540     QPainter p(&image);
00541     p.setRenderHint(QPainter::Antialiasing);
00542
00543     const QRect mapRect(0, 0, m_settings.width, mapHeight);
00544     p.fillRect(mapRect, QColor("#111827"));
00545
00546     for (const VideoTileId& tileId : viewportTiles)
00547     {
00548         const QPointF topLeft = tileToScreen({ static_cast<double>(tileId.x),
00549 static_cast<double>(tileId.y) },
00549                                     centerTile,
00550                                     m_settings.width,
00551                                     mapHeight);
00552         const QPixmap px = tileProvider.getTile(tileId, true);
00553         if (px.isNull())
00554             p.fillRect(QRectF(topLeft, QSizeF(kTilePx, kTilePx)), QColor("#334155"));
00555         else
00556             p.drawPixmap(static_cast<int>(topLeft.x()), static_cast<int>(topLeft.y()), kTilePx,
00557 kTilePx, px);
00558     }
00559     p.setPen(QPen(QColor(220, 220, 220, 110), 1.5));
00560     for (size_t i = 1; i < gpsTilePoints.size(); ++i)
00561     {
00562         const QPointF ap = tileToScreen(gpsTilePoints[i - 1], centerTile, m_settings.width,
00563 mapHeight);
00564         const QPointF bp = tileToScreen(gpsTilePoints[i], centerTile, m_settings.width,
00565 mapHeight);
00566         p.drawLine(ap, bp);
00567     }
00568     const size_t maxColoredSegment = static_cast<size_t>(std::clamp(currentGpsIndex + 1, 1,
00569 static_cast<int>(gpsRecords.size()) - 1));
00570     for (size_t i = 1; i <= maxColoredSegment; ++i)
00571     {
00572         const RideRecord& a = *gpsRecords[i - 1];
00573         const QPointF ap = tileToScreen(gpsTilePoints[i - 1], centerTile, m_settings.width,
00574 mapHeight);
00575         const QPointF bp = tileToScreen(gpsTilePoints[i], centerTile, m_settings.width,
00576 mapHeight);
00577         const QColor c = routeColor(a,
00578                                     m_settings.routeColorMetric,
00579                                     m_settings.colorContext,
00580                                     m_settings.singleRouteColor);
00581         p.setPen(QPen(c, 4.0, Qt::SolidLine, Qt::RoundCap, Qt::RoundJoin));
00582         p.drawLine(ap, bp);
00583     }
00584     if (currentGpsIndex + 1 < static_cast<int>(gpsRecords.size()))
00585     {
00586         const RideRecord& a = *gpsRecords[static_cast<size_t>(currentGpsIndex)];
00587         const RideRecord& b = *gpsRecords[static_cast<size_t>(currentGpsIndex + 1)];
00588         if (sourceSeconds > a.elapsedSeconds && sourceSeconds < b.elapsedSeconds)
00589         {
00590             const double dt = b.elapsedSeconds - a.elapsedSeconds;
00591             const double alpha = dt > 1e-6
00592                 ? std::clamp((sourceSeconds - a.elapsedSeconds) / dt, 0.0, 1.0)
00593                 : 0.0;
00594             const QPointF ap = tileToScreen(gpsTilePoints[static_cast<size_t>(currentGpsIndex)],
00595                                     centerTile,
00596                                     m_settings.width,
00597                                     mapHeight);
00598             const QPointF bp = tileToScreen(gpsTilePoints[static_cast<size_t>(currentGpsIndex +
00599 1)],
00600                                     centerTile,
00601                                     m_settings.width,
00602                                     mapHeight);
00603             const QPointF ip = ap + (bp - ap) * alpha;
00604             const QColor c = routeColor(a,
00605                                     m_settings.routeColorMetric,
00606                                     m_settings.colorContext,
00607                                     m_settings.singleRouteColor);
00608             p.setPen(QPen(c, 4.0, Qt::SolidLine, Qt::RoundCap, Qt::RoundJoin));
00609             p.drawLine(ap, ip);
00610         }
00611     }
00612     const QPointF currentPos = tileToScreen(latLonToTile(centerLat, centerLon, zoom),
00613                                     centerTile,

```

```

00612                                     m_settings.width,
00613                                     mapHeight);
00614     p.setPen(QPen(QColor("#ffffff"), 2.0));
00615     p.setBrush(QColor("#ef4444"));
00616     p.drawEllipse(currentPos, 7.0, 7.0);
00617
00618     if (showRouteLegend)
00619     {
00620         const bool hasAthleteBox = m_settings.overlayAthleteName;
00621         const int reservedLeft = telemetryMargin + telemetryWidth + telemetryMargin;
00622         const int reservedRight = hasAthleteBox
00623             ? (athleteWidth + telemetryMargin + telemetryMargin)
00624             : telemetryMargin;
00625
00626         int legendLeft = reservedLeft;
00627         int legendRight = m_settings.width - reservedRight;
00628         int legendY = telemetryMargin;
00629
00630         if (legendRight - legendLeft < static_cast<int>(std::round(260.0 * uiScale)))
00631         {
00632             legendLeft = telemetryMargin;
00633             legendRight = m_settings.width - telemetryMargin;
00634             legendY = telemetryMargin + std::max(telemetryHeight, hasAthleteBox ? athleteHeight :
00635                 0) + legendGap;
00636         }
00637         const QString legendTitle = QString("Track Color:
00638 %1").arg(colorMetricDisplayName(m_settings.routeColorMetric));
00639         p.setFont(QFont("Helvetica", legendTitleFontPx, QFont::DemiBold));
00640         const QFontMetrics titleFm = p.fontMetrics();
00641
00642         p.setFont(QFont("Helvetica", legendLabelFontPx, QFont::Normal));
00643         const QFontMetrics labelFm = p.fontMetrics();
00644
00645         const int legendWidth = std::max(120, legendRight - legendLeft);
00646         const int contentWidth = std::max(80, legendWidth - (2 * legendInset));
00647         const int rowHeight = std::max(legendSwatchSize + 2, labelFm.height() + 2);
00648
00649         struct LegendLayoutItem
00650         {
00651             const Zone* zone = nullptr;
00652             QString label;
00653             int row = 0;
00654             int x = 0;
00655             int textWidth = 0;
00656         };
00657         std::vector<LegendLayoutItem> layoutItems;
00658         layoutItems.reserve(routeLegendZones.size());
00659         int currentRow = 0;
00660         int currentX = 0;
00661         for (const Zone& zone : routeLegendZones)
00662         {
00663             const QString label = zoneLegendText(zone, m_settings.routeColorMetric);
00664             const int textW = labelFm.horizontalAdvance(label);
00665             const int itemW = legendSwatchSize + legendGap + textW + legendInset;
00666
00667             if (currentX > 0 && currentX + itemW > contentWidth)
00668             {
00669                 ++currentRow;
00670                 currentX = 0;
00671             }
00672
00673             layoutItems.push_back({ &zone, label, currentRow, currentX, textW });
00674             currentX += itemW;
00675         }
00676
00677         const int rows = std::max(1, currentRow + 1);
00678         const int titleHeight = titleFm.height();
00679         const int legendHeight = (2 * legendInset) + titleHeight + legendGap + (rows * rowHeight);
00680
00681         QRect legendRect(legendLeft,
00682             legendY,
00683             legendWidth,
00684             legendHeight);
00685
00686         p.setPen(Qt::NoPen);
00687         p.setBrush(QColor(15, 23, 42, 170));
00688         p.drawRoundedRect(legendRect, 8.0, 8.0);
00689
00690         p.setPen(QColor("#cbd5e1"));
00691         p.setFont(QFont("Helvetica", legendTitleFontPx, QFont::DemiBold));
00692         const QRect titleRect(legendRect.left() + legendInset,
00693             legendRect.top() + legendInset,
00694             legendRect.width() - (2 * legendInset),
00695             titleHeight);
00696         p.drawText(titleRect, Qt::AlignVCenter | Qt::AlignLeft, legendTitle);

```



```

00697
00698     p.setFont(QFont("Helvetica", legendLabelFontPx, QFont::Normal));
00699     for (const LegendLayoutItem& item : layoutItems)
00700     {
00701         const int rowTop = titleRect.bottom() + legendGap + (item.row * rowHeight);
00702         QRect swatchRect(legendRect.left() + legendInset + item.x,
00703                         rowTop + (rowHeight - legendSwatchSize) / 2,
00704                         legendSwatchSize,
00705                         legendSwatchSize);
00706         p.setPen(QPen(QColor(255, 255, 255, 90), 1.0));
00707         p.setBrush(item.zone->color());
00708         p.drawRoundedRect(swatchRect, 2.0, 2.0);
00709
00710         p.setPen(QColor("#f1f5f9"));
00711         p.drawText(QRect(swatchRect.right() + legendGap,
00712                         rowTop,
00713                         item.textWidth,
00714                         rowHeight),
00715                 Qt::AlignVCenter | Qt::AlignLeft,
00716                 item.label);
00717     }
00718 }
00719
00720 const bool hasTelemetryValue = m_settings.overlayTime ||
00721 (m_settings.overlayPower && now.hasPower) ||
00722 (m_settings.overlayHeartRate && now.hasHeartRate) ||
00723 (m_settings.overlayCadence && now.hasCadence) ||
00724 (m_settings.overlaySpeed && now.hasSpeed) ||
00725 (m_settings.overlayAltitude && now.hasAltitude);
00726
00727 if (m_settings.overlayTelemetryPanel && hasTelemetryValue)
00728 {
00729     QRect telemetryRect(telemetryMargin, telemetryMargin, telemetryWidth, telemetryHeight);
00730     p.setPen(Qt::NoPen);
00731     p.setBrush(QColor(15, 23, 42, 165));
00732     p.drawRoundedRect(telemetryRect, 8.0, 8.0);
00733
00734     p.setPen(QColor("#e2e8f0"));
00735     p.setFont(QFont("Helvetica", telemetryFontPx, QFont::DemiBold));
00736     int y = telemetryRect.top() + telemetryTopBaseline;
00737     auto drawLine = [&](const QString& line)
00738     {
00739         p.drawText(telemetryRect.left() + telemetryLeftPadding, y, line);
00740         y += telemetryLineStep;
00741     };
00742
00743     if (m_settings.overlayTime)
00744         drawLine(QString("Time %1").arg(fmtDuration(sourceSeconds - startSec)));
00745     if (m_settings.overlayPower && now.hasPower)
00746         drawLine(QString("Power %1 W").arg(static_cast<int>(std::round(now.power))));
00747     if (m_settings.overlayHeartRate && now.hasHeartRate)
00748         drawLine(QString("HR %1 bpm").arg(static_cast<int>(std::round(now.heartRate))));
00749     if (m_settings.overlayCadence && now.hasCadence)
00750         drawLine(QString("Cad %1 rpm").arg(static_cast<int>(std::round(now.cadence))));
00751     if (m_settings.overlaySpeed && now.hasSpeed)
00752         drawLine(QString("Speed %1 km/h").arg(now.speed, 0, 'f', 1));
00753     if (m_settings.overlayAltitude && now.hasAltitude)
00754         drawLine(QString("Alt %1 m").arg(now.altitude, 0, 'f', 0));
00755 }
00756
00757 if (m_settings.overlayAthleteName)
00758 {
00759     p.setPen(Qt::NoPen);
00760     p.setBrush(QColor(15, 23, 42, 150));
00761     QRect athleteRect(m_settings.width - athleteWidth - telemetryMargin,
00762                     telemetryMargin,
00763                     athleteWidth,
00764                     athleteHeight);
00765     p.drawRoundedRect(athleteRect, 8.0, 8.0);
00766     p.setPen(QColor("#f8f9fa"));
00767     p.setFont(QFont("Helvetica", athleteFontPx, QFont::Bold));
00768     p.drawText(athleteRect.adjusted(athleteInset, 0, -athleteInset, 0), Qt::AlignVCenter |
00769 Qt::AlignRight, m_settings.athleteName);
00770 }
00771
00772 if (showAnyMetricChart)
00773 {
00774     const QRect chartRect(0, mapHeight, m_settings.width, chartHeight);
00775     p.fillRect(chartRect, QColor("#020617"));
00776
00777     auto drawMetricChart = [&](int row,
00778                                int rows,
00779                                const QString& label,
00780                                auto hasMetric,
00781                                auto metricValue,
00782                                const QColor& color)
00783     {

```

```

00783         const int rowHeight = chartRect.height() / std::max(1, rows);
00784     const QRect rowRect(chartRect.left() + chartSidePadding,
00785         chartRect.top() + row * rowHeight + chartTopPadding,
00786         chartRect.width() - (2 * chartSidePadding),
00787         rowHeight - chartBottomPadding);
00788
00789     p.setPen(QColor("#94a3b8"));
00790     p.setFont(QFont("Helvetica", chartLabelFontPx, QFont::DemiBold));
00791     p.drawText(rowRect.adjusted(0, 0, 0, -rowRect.height() + chartLabelBaseline), label);
00792
00793     const double windowStart = sourceSeconds - kChartWindowBackSeconds;
00794     const double windowEnd = sourceSeconds + kChartWindowForwardSeconds;
00795     const double visibleDataStart = std::max(startSec, windowStart);
00796     const double visibleDataEnd = std::min(endSec, windowEnd);
00797
00798     double minV = std::numeric_limits<double>::max();
00799     double maxV = std::numeric_limits<double>::lowest();
00800     std::vector<QPointF> pts;
00801     for (const RideRecord& rec : m_settings.rideData.records)
00802     {
00803         if (rec.elapsedSeconds < visibleDataStart || rec.elapsedSeconds > visibleDataEnd)
00804             continue;
00805         if (!hasMetric(rec))
00806             continue;
00807         const double v = metricValue(rec);
00808         minV = std::min(minV, v);
00809         maxV = std::max(maxV, v);
00810         pts.emplace_back(rec.elapsedSeconds, v);
00811     }
00812
00813     if (pts.size() < 2)
00814         return;
00815
00816     if (std::abs(maxV - minV) < 0.001)
00817         maxV = minV + 1.0;
00818
00819     QPainterPath path;
00820     bool first = true;
00821     for (const QPointF& pt : pts)
00822     {
00823         const double nx = (pt.x() - windowStart) / std::max(0.001, windowEnd - windowStart);
00824         const double ny = (pt.y() - minV) / (maxV - minV);
00825         const QPointF sp(rowRect.left() + nx * rowRect.width(),
00826             rowRect.bottom() - ny * rowRect.height());
00827         if (first)
00828         {
00829             path.moveTo(sp);
00830             first = false;
00831         }
00832         else
00833         {
00834             path.lineTo(sp);
00835         }
00836     }
00837
00838     p.setPen(QPen(color, 1.8));
00839     p.drawPath(path);
00840
00841     const double cursorX = rowRect.left() + ((sourceSeconds - windowStart) / std::max(0.001,
windowEnd - windowStart)) * rowRect.width();
00842     p.setPen(QPen(QColor("#e2e8f0"), 1.0, Qt::DashLine));
00843     p.drawLine(QPointF(cursorX, rowRect.top()), QPointF(cursorX, rowRect.bottom()));
00844 };
00845
00846     struct ChartSpec
00847     {
00848         bool enabled;
00849         QString label;
00850         std::function<bool(const RideRecord&)> hasMetric;
00851         std::function<double(const RideRecord&)> metricValue;
00852         QColor color;
00853     };
00854
00855     const std::vector<ChartSpec> chartSpecs = {
00856         { m_settings.overlayPower,
00857             "Power",
00858             [](const RideRecord& r) { return r.hasPower; },
00859             [](const RideRecord& r) { return r.power; },
00860             QColor("#ef4444") },
00861         { m_settings.overlayHeartRate,
00862             "Heart Rate",
00863             [](const RideRecord& r) { return r.hasHeartRate; },
00864             [](const RideRecord& r) { return r.heartRate; },
00865             QColor("#f97316") },
00866         { m_settings.overlayCadence,
00867             "Cadence",
00868             [](const RideRecord& r) { return r.hasCadence; },

```

```

00869         [](const RideRecord& r) { return r.cadence; },
00870         QColor("#22c55e") },
00871     { m_settings.overlaySpeed,
00872       "Speed",
00873       [](const RideRecord& r) { return r.hasSpeed; },
00874       [](const RideRecord& r) { return r.speed; },
00875       QColor("#38bdf8") },
00876     { m_settings.overlayAltitude,
00877       "Altitude",
00878       [](const RideRecord& r) { return r.hasAltitude; },
00879       [](const RideRecord& r) { return r.altitude; },
00880       QColor("#c084fc") }
00881     };
00882
00883     int enabledCount = 0;
00884     for (const ChartSpec& spec : chartSpecs)
00885     {
00886         if (spec.enabled)
00887             ++enabledCount;
00888     }
00889
00890     int row = 0;
00891     for (const ChartSpec& spec : chartSpecs)
00892     {
00893         if (!spec.enabled)
00894             continue;
00895         drawMetricChart(row,
00896                        enabledCount,
00897                        spec.label,
00898                        spec.hasMetric,
00899                        spec.metricValue,
00900                        spec.color);
00901         ++row;
00902     }
00903 }
00904
00905 p.end();
00906
00907 if (ffmpeg.write(reinterpret_cast<const char*>(image.constBits()), image.sizeInBytes()) < 0)
00908 {
00909     ffmpeg.kill();
00910     ffmpeg.waitForFinished();
00911     emit finished(false, "FFmpeg pipe write failed.", false);
00912     return;
00913 }
00914
00915 const qint64 elapsedMs = timer.elapsed();
00916 const int done = frame + 1;
00917 const double avgPerFrameMs = done > 0 ? static_cast<double>(elapsedMs) /
static_cast<double>(done) : 0.0;
00918 const double remainingMs = avgPerFrameMs * static_cast<double>(totalFrames - done);
00919 const QString eta = fmtDuration(remainingMs / 1000.0);
00920
00921 if (done == 1 || done % 60 == 0 || done == totalFrames)
00922 {
00923     const VideoTileStats s = tileProvider.stats();
00924     emit tileStatsChanged(QString("Tiles: required %1 | on-disk %2 | downloaded %3")
00925                          .arg(s.requiredTiles)
00926                          .arg(s.onDiskTiles)
00927                          .arg(s.downloadedTiles));
00928 }
00929
00930 emit progressChanged(done, totalFrames);
00931 emit frameProgressChanged(done, totalFrames, eta);
00932 }
00933
00934 ffmpeg.closeWriteChannel();
00935 if (!ffmpeg.waitForFinished(-1))
00936 {
00937     emit finished(false, "FFmpeg did not finish cleanly.", false);
00938     return;
00939 }
00940
00941 if (ffmpeg.exitStatus() != QProcess::NormalExit || ffmpeg.exitCode() != 0)
00942 {
00943     const QString err = QString::fromLocal8Bit(ffmpeg.readAllStandardOutput());
00944     emit finished(false, QString("FFmpeg failed: %1").arg(err.left(500)), false);
00945     return;
00946 }
00947
00948 emit stageChanged("Finalizing...");
00949 emit progressChanged(totalFrames, totalFrames);
00950 {
00951     const VideoTileStats s = tileProvider.stats();
00952     emit tileStatsChanged(QString("Tiles: required %1 | on-disk %2 | downloaded %3")
00953                          .arg(s.requiredTiles)
00954                          .arg(s.onDiskTiles)

```

```

00955                                     .arg(s.downloadedTiles));
00956         emit finished(
00957             true,
00958             QString("Video exported to %1\n\nTile stats: required %2, on-disk %3, downloaded %4")
00959                 .arg(m_settings.outputPath)
00960                 .arg(s.requiredTiles)
00961                 .arg(s.onDiskTiles)
00962                 .arg(s.downloadedTiles),
00963             false);
00964     }
00965 }

```

8.138 VideoRenderJob.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QAtomicInteger>
00007 #include <QObject>
00008
00009 #include "video/VideoExportSettings.h"
00010
00017 class VideoRenderJob : public QObject
00018 {
00019     Q_OBJECT
00020
00021 public:
00027     explicit VideoRenderJob(VideoExportSettings settings, QObject* parent = nullptr);
00028
00032     void run();
00033
00037     void cancel();
00038
00039 signals:
00042     void stageChanged(const QString& stage);
00043
00047     void progressChanged(int value, int maximum);
00048
00053     void frameProgressChanged(int currentFrame, int totalFrames, const QString& etaText);
00054
00057     void tileStatsChanged(const QString& statsText);
00058
00063     void finished(bool success, const QString& message, bool canceled);
00064
00065 private:
00066     VideoExportSettings m_settings;
00067     QAtomicInteger<int> m_cancelRequested { 0 };
00068 };

```

8.139 VideoTileProvider.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "VideoTileProvider.h"
00005
00006 #include <QDir>
00007 #include <QStandardPaths>
00008
00009 #include <algorithm>
00010
00011 namespace
00012 {
00013     QString defaultVideoExportCacheRoot()
00014     {
00015         const QString appData = QStandardPaths::writableLocation(QStandardPaths::AppDataLocation);
00016         return appData.isEmpty()
00017             ? QDir::home().filePath(".fitlyzerc/video-export-cache")
00018             : QDir(appData).filePath("video-export-cache");
00019     }
00020
00021     QString defaultPrimaryTileCacheRoot()
00022     {
00023         const QString appData = QStandardPaths::writableLocation(QStandardPaths::AppDataLocation);
00024         return appData.isEmpty()
00025             ? QDir::home().filePath(".fitlyzerc/tiles")
00026             : QDir(appData).filePath("tiles");
00027     }
00028 }

```

```

00029
00030 VideoTileProvider::VideoTileProvider(const VideoTileProviderConfig& config)
00031     : m_tileCache(
00032         TileCacheConfig{
00033             std::max(1, config.memoryCacheTiles),
00034             config.exportCacheRoot.trimmed().isEmpty()
00035                 ? defaultVideoExportCacheRoot()
00036                 : config.exportCacheRoot,
00037             QStringList{ defaultPrimaryTileCacheRoot() }
00038         })
00039     , m_deleteTemporaryTilesAfterExport(config.deleteTemporaryTilesAfterExport)
00040 {
00041     m_tileCache.setMapStyle(config.style);
00042 }
00043
00044 void VideoTileProvider::setRequiredTiles(const std::unordered_set<VideoTileId, VideoTileIdHash>&
    required)
00045 {
00046     m_requiredTiles = required;
00047     m_requiredTilesOnDisk = 0;
00048     m_downloadedTiles = 0;
00049
00050     for (const VideoTileId& tileId : m_requiredTiles)
00051     {
00052         if (m_tileCache.isTileCachedOnDisk(tileId.z, tileId.x, tileId.y))
00053             ++m_requiredTilesOnDisk;
00054     }
00055 }
00056
00057 int VideoTileProvider::requiredTileCount() const
00058 {
00059     return static_cast<int>(m_requiredTiles.size());
00060 }
00061
00062 int VideoTileProvider::requiredTilesAvailableOnDisk() const
00063 {
00064     return m_requiredTilesOnDisk;
00065 }
00066
00067 QPixmap VideoTileProvider::getTile(const VideoTileId& id, bool allowNetwork)
00068 {
00069     const bool cachedBefore = isTileCachedOnDisk(id);
00070     const QPixmap px = m_tileCache.tileBlocking(id.z, id.x, id.y, allowNetwork);
00071     if (allowNetwork && !cachedBefore && !px.isNull() && isTileCachedOnDisk(id))
00072         ++m_downloadedTiles;
00073     return px;
00074 }
00075
00076 VideoTileStats VideoTileProvider::stats() const
00077 {
00078     VideoTileStats s;
00079     s.requiredTiles = static_cast<int>(m_requiredTiles.size());
00080     s.onDiskTiles = m_requiredTilesOnDisk;
00081     s.downloadedTiles = m_downloadedTiles;
00082     return s;
00083 }
00084
00085 bool VideoTileProvider::isTileCachedOnDisk(const VideoTileId& id) const
00086 {
00087     return m_tileCache.isTileCachedOnDisk(id.z, id.x, id.y);
00088 }
00089
00090 void VideoTileProvider::clearMemoryCache()
00091 {
00092     m_tileCache.clearMemoryCache();
00093 }
00094
00095 void VideoTileProvider::cleanupExportCache()
00096 {
00097     if (!m_deleteTemporaryTilesAfterExport)
00098         return;
00099
00100     QDir cacheRoot(m_tileCache.diskCacheRoot());
00101     if (cacheRoot.exists())
00102         cacheRoot.removeRecursively();
00103 }
00104
00105 bool VideoTileProvider::shouldDeleteExportCacheAfterExport() const
00106 {
00107     return m_deleteTemporaryTilesAfterExport;
00108 }
00109
00110 QString VideoTileProvider::exportCacheRoot() const
00111 {
00112     return m_tileCache.diskCacheRoot();
00113 }

```

8.140 VideoTileProvider.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QString>
00007 #include <unordered_set>
00008 #include <vector>
00009
00010 #include "maps/TileCache.h"
00011
00015 struct VideoTileProviderConfig
00016 {
00018     MapStyle style = MapStyle::Light;
00019
00021     int memoryCacheTiles = 96;
00022
00024     bool deleteTemporaryTilesAfterExport = true;
00025
00027     QString exportCacheRoot;
00028 };
00029
00033 struct VideoTileId
00034 {
00036     int z = 0;
00037
00039     int x = 0;
00040
00042     int y = 0;
00043
00045     bool operator==(const VideoTileId& other) const
00046     {
00047         return z == other.z && x == other.x && y == other.y;
00048     }
00049 };
00050
00054 struct VideoTileIdHash
00055 {
00057     std::size_t operator()(const VideoTileId& id) const noexcept
00058     {
00059         const std::size_t hz = static_cast<std::size_t>(id.z);
00060         const std::size_t hx = static_cast<std::size_t>(id.x);
00061         const std::size_t hy = static_cast<std::size_t>(id.y);
00062         return (hz * 73856093u) ^ (hx * 19349663u) ^ (hy * 83492791u);
00063     }
00064 };
00065
00069 struct VideoTileStats
00070 {
00072     int requiredTiles = 0;
00073
00075     int onDiskTiles = 0;
00076
00078     int downloadedTiles = 0;
00079 };
00080
00086 class VideoTileProvider
00087 {
00088 public:
00093     explicit VideoTileProvider(const VideoTileProviderConfig& config);
00094
00099     void setRequiredTiles(const std::unordered_set<VideoTileId, VideoTileIdHash>& required);
00100
00105     int requiredTileCount() const;
00106
00111     int requiredTilesAvailableOnDisk() const;
00112
00119     QPixmap getTile(const VideoTileId& id, bool allowNetwork);
00120
00126     bool isTileCachedOnDisk(const VideoTileId& id) const;
00127
00132     VideoTileStats stats() const;
00133
00137     void clearMemoryCache();
00138
00142     void cleanupExportCache();
00143
00148     bool shouldDeleteExportCacheAfterExport() const;
00149
00154     QString exportCacheRoot() const;
00155
00156 private:
00157     TileCache m_tileCache;
00158     bool m_deleteTemporaryTilesAfterExport = true;

```

```
00159     std::unordered_set<VideoTileId, VideoTileIdHash> m_requiredTiles;
00160     int m_requiredTilesOnDisk = 0;
00161     int m_downloadedTiles = 0;
00162 };
```


Index

- ~AnalysisQueue
 - AnalysisQueue, [65](#)
- ~DatabaseConnectionGuard
 - DatabaseConnectionGuard, [138](#)
- ~DatabaseManager
 - DatabaseManager, [140](#)
- ~ExportTileCleanupGuard
 - anonymous_namespace{VideoRenderJob.cpp}::ExportTileCleanupGuard, [152](#)
- ~ImportQueue
 - ImportQueue, [175](#)
- ~VideoExportDialog
 - VideoExportDialog, [298](#)
- AboutDialog, [41](#)
 - AboutDialog, [42](#)
- absoluteExecutablePath
 - anonymous_namespace{FfmpegPath.cpp}, [29](#)
- accept
 - CreateDatabaseDialog, [136](#)
- activeCountUnsafe
 - ImportQueue, [176](#)
- activeJobs
 - ImportProgressModel, [172](#)
- Activity, [42](#)
 - analysisFlags, [44](#)
 - analysisVersion, [44](#)
 - athleteId, [44](#)
 - avgCadence, [44](#)
 - avgHr, [44](#)
 - avgPower, [44](#)
 - avgSpeed, [44](#)
 - bike, [44](#)
 - distanceM, [45](#)
 - durationSec, [45](#)
 - elevationGain, [45](#)
 - endTime, [45](#)
 - equipment, [45](#)
 - fatigue, [45](#)
 - fileName, [45](#)
 - fingerprint, [45](#)
 - fitHash, [46](#)
 - id, [46](#)
 - importedAt, [46](#)
 - importSource, [46](#)
 - isValid, [44](#)
 - maxHr, [46](#)
 - maxPower, [46](#)
 - normalizedPower, [46](#)
 - notes, [46](#)
 - rpe, [46](#)
 - sleepHours, [47](#)
 - sport, [47](#)
 - startTime, [47](#)
 - temperature, [47](#)
 - weather, [47](#)
 - weatherNotes, [47](#)
 - WeightingGuard, [47](#)
 - wind, [47](#)
- ActivityAnalysisCacheRepository, [48](#)
 - ActivityAnalysisCacheRepository, [48](#)
 - getCache, [48](#)
 - hasCache, [49](#)
 - invalidateCache, [49](#)
 - invalidateKey, [49](#)
 - m_db, [50](#)
 - setCache, [50](#)
- ActivityAnalysisFlags, [23](#)
 - HasClimbs, [23](#)
 - HasHistogram, [23](#)
 - HasIntervals, [23](#)
 - HasPowerCurve, [24](#)
 - HasVideo, [24](#)
 - None, [24](#)
- ActivityAnalysisSettings, [50](#)
 - activityId, [51](#)
 - climbMinGain, [51](#)
 - climbMinGradient, [51](#)
 - climbMinLength, [51](#)
 - intervalRestThreshold, [51](#)
 - intervalWorkThreshold, [52](#)
 - isValid, [51](#)
- ActivityAnalysisSettingsRepository, [52](#)
 - ActivityAnalysisSettingsRepository, [52](#)
 - insertOrUpdate, [53](#)
 - m_db, [53](#)
 - settingsForActivity, [53](#)
- ActivityBrowser, [53](#)
 - ActivityBrowser, [55](#)
 - applyFilters, [55](#)
 - deleteActivity, [55](#)
 - dragEnterEvent, [55](#)
 - dropEvent, [55](#)
 - editNotes, [55](#)
 - focusSearch, [55](#)
 - m_athleteId, [56](#)
 - m_dbManager, [56](#)
 - m_emptyStateLabel, [56](#)
 - m_rangeFilter, [56](#)

- m_searchEdit, [56](#)
 - m_table, [56](#)
 - refresh, [55](#)
 - showContextMenu, [56](#)
- activityCount
 - MainWindow, [208](#)
- activityDateForFtp
 - anonymous_namespace{WorkoutController.cpp}, [35](#)
- activityEndTimeUtc
 - RideData, [272](#)
- activityId
 - ActivityAnalysisSettings, [51](#)
 - ClimbRecord, [119](#)
 - ImportRecord, [179](#)
 - IntervalRecord, [195](#)
- activityName
 - VideoExportSettings, [304](#)
- ActivityRepository, [58](#)
 - ActivityRepository, [58](#)
 - deleteActivity, [59](#)
 - getActivity, [59](#)
 - hasFingerprintMatch, [59](#)
 - hasFitHash, [59](#)
 - insertActivity, [60](#)
 - listActivities, [60](#)
 - m_db, [63](#)
 - setImportSource, [61](#)
 - updateActivityProperties, [61](#)
 - updateAnalysisFlags, [62](#)
 - updateFingerprint, [62](#)
 - updateNotes, [63](#)
- activityStartTimeUtc
 - RideData, [272](#)
- adaptiveSmoothingMaxMeters
 - ClimbDetectorConfig, [108](#)
- adaptiveSmoothingMinMeters
 - ClimbDetectorConfig, [108](#)
- addClimbBackgroundBands
 - RideChartWidget, [256](#)
- addFtpEntry
 - AthleteRepository, [86](#)
- addFtpRow
 - AthleteDialog, [79](#)
- addIntervalBackgroundBands
 - RideChartWidget, [256](#)
- addMetricOverlayBands
 - RideChartWidget, [256](#)
- addPlannedWorkout
 - CalendarWidget, [92](#)
- addRecentDatabase
 - MainWindow, [208](#)
- addWeightEntry
 - AthleteRepository, [87](#)
- addWeightRow
 - AthleteDialog, [79](#)
- addZoneBackgroundBands
 - RideChartWidget, [256](#)
- adjustRouteColorForStyle
 - MapRenderer, [231](#)
- aerobicDecouplingPct
 - Climb, [95](#)
- algorithmVersion
 - ClimbRecord, [119](#)
 - IntervalRecord, [195](#)
- altitude
 - RideRecord, [274](#)
- altitudeMax
 - ColorContext, [132](#)
- altitudeMin
 - ColorContext, [132](#)
- altitudeSpikeVerticalSpeedMps
 - ClimbDetectorConfig, [108](#)
- altitudeZones
 - ZoneCalculator, [335](#)
- analysisFlags
 - Activity, [44](#)
- AnalysisQueue, [63](#)
 - ~AnalysisQueue, [65](#)
 - AnalysisQueue, [65](#)
 - enqueue, [66](#)
 - isBusy, [66](#)
 - m_busy, [68](#)
 - m_dbPath, [68](#)
 - m_queue, [68](#)
 - m_thread, [68](#)
 - m_worker, [68](#)
 - onTaskCompleted, [66](#)
 - onTaskFailed, [66](#)
 - pendingCount, [66](#)
 - pendingCountChanged, [67](#)
 - processNext, [67](#)
 - setDatabasePath, [67](#)
 - taskCompleted, [67](#)
 - taskFailed, [68](#)
- analysisVersion
 - Activity, [44](#)
- AnalysisVersions, [24](#)
 - ClimbDetection, [24](#)
 - IntervalDetection, [24](#)
- AnalysisWorker, [69](#)
 - AnalysisWorker, [70](#)
 - m_dbPath, [72](#)
 - processTask, [70](#)
 - setDatabasePath, [71](#)
 - taskCompleted, [71](#)
 - taskFailed, [72](#)
- analyze
 - WorkoutAnalyzer, [325](#)
- anonymous_namespace{ActivityBrowser.cpp}, [25](#)
 - kIsoDateRole, [25](#)
 - kNotesRole, [25](#)
 - kWeatherRole, [25](#)
- anonymous_namespace{ActivityBrowser.cpp}::ActivityDateTableItem, [57](#)
 - operator<, [57](#)

- anonymous_namespace{AnalysisWorker.cpp}, 25
 - applySqlitePragmas, 25
- anonymous_namespace{AthleteDialog.cpp}, 25
 - displayDateOrRaw, 26
 - kEntryIdRole, 26
 - tolsoOrRaw, 26
- anonymous_namespace{ClimbDetector.cpp}, 26
 - buildFilteredAltitudeMeters, 26
 - clampValue, 26
 - estimateSpeedKmhAt, 26
 - isFinite, 26
 - segmentMeters, 27
- anonymous_namespace{ClimbMetrics.cpp}, 27
 - classifyClimbCategory, 27
 - classifyClimbShape, 27
 - computeLinearRegressionSlope, 27
 - computeNormalizedPower, 27
 - computeSliceStats, 28
 - isFinite, 28
- anonymous_namespace{ClimbMetrics.cpp}::SliceStats, 276
 - avgCadence, 277
 - avgHeartRate, 277
 - avgPower, 277
- anonymous_namespace{ClimbRepository.cpp}, 28
 - kSelectCols, 28
 - recordFromQuery, 28
- anonymous_namespace{ColorProvider.cpp}, 29
 - interpolateColor, 29
- anonymous_namespace{DateFormatter.cpp}, 29
 - datePatternForFormat, 29
- anonymous_namespace{FfmpegPath.cpp}, 29
 - absoluteExecutablePath, 29
- anonymous_namespace{FitParser.cpp}, 29
 - fitTimestampToIsoUtc, 30
- anonymous_namespace{FitParser.cpp}::FitFileIdListener, 153
 - m_timeCreated, 154
 - OnMesg, 154
 - timeCreated, 154
- anonymous_namespace{FitParser.cpp}::FitSessionListener, 163
 - m_startTime, 164
 - OnMesg, 164
 - startTime, 164
- anonymous_namespace{ImportWorker.cpp}, 30
 - applySqlitePragmas, 30
- anonymous_namespace{MapFitMath.cpp}, 30
 - kMaxMercatorLat, 31
 - kMinLatSpan, 31
 - kMinLonSpan, 31
 - kPi, 31
 - kTilePx, 31
 - latToTileY, 30
 - normalizeLongitude, 30
 - tileYToLat, 30
- anonymous_namespace{RideChartWidget.cpp}, 31
 - hitTestClimbHandles, 31
 - kClimbHandleCenterOffsetYPx, 32
 - kClimbHandleGrabRadiusPx, 32
 - kClimbHandleRadiusPx, 32
- anonymous_namespace{RideChartWidget.cpp}::ClimbHandleHitResult, 114
 - hit, 114
 - startHandle, 114
- anonymous_namespace{TileCache.cpp}, 32
 - defaultDiskCacheRoot, 32
- anonymous_namespace{VideoExportDialog.cpp}, 32
 - formatVideoLength, 32
 - makeDefaultFilename, 32
 - mapStyleName, 32
- anonymous_namespace{VideoRenderJob.cpp}, 33
 - computeTileViewport, 33
 - fmtDuration, 33
 - formatLegendValue, 33
 - interpolateGpsAtTime, 33
 - kPi, 35
 - kTilePx, 35
 - kVideoExportTileCache, 35
 - latLonToTile, 34
 - nearestRecordIndex, 34
 - routeColor, 34
 - tilesForViewport, 34
 - tileToScreen, 34
 - zoneLegendText, 34
- anonymous_namespace{VideoRenderJob.cpp}::ExportTileCleanupGuard, 152
 - ~ExportTileCleanupGuard, 152
 - ExportTileCleanupGuard, 152
 - m_provider, 153
- anonymous_namespace{VideoRenderJob.cpp}::TileViewport, 285
 - nX, 285
 - nY, 285
 - operator==, 285
 - tileX0, 285
 - tileY0, 285
- anonymous_namespace{VideoTileProvider.cpp}, 35
 - defaultPrimaryTileCacheRoot, 35
 - defaultVideoExportCacheRoot, 35
- anonymous_namespace{WorkoutController.cpp}, 35
 - activityDateForFtp, 35
 - resolveAthleteFtpForDate, 35
- anonymous_namespace{ZoneCalculator.cpp}, 36
 - sampleAvailable, 36
 - sampleDurationSeconds, 36
 - valueForMetric, 36
- applicationDataDirectory
 - Platform, 37
- apply
 - ClimbEditCommand, 113
- applyChartHeight
 - MainWindow, 208
- applyChartPreset
 - MainWindow, 208
- applyChartSmoothing

- MainWindow, 208
- applyEstimatedFtpForCurrentAthlete
 - MainWindow, 208
- applyFilters
 - ActivityBrowser, 55
- applySchema
 - DatabaseManager, 140
- applySqlitePragmas
 - anonymous_namespace{AnalysisWorker.cpp}, 25
 - anonymous_namespace{ImportWorker.cpp}, 30
- applyWeightMetricsToClimbs
 - MainWindow, 208
- AppSettings, 73
 - dateFormat, 73
 - dateFormatKey, 73
 - instance, 73
 - sanitizeDateFormat, 74
 - setDateFormat, 74
 - setTileCacheSize, 74
 - tileCacheSize, 74
 - tileCacheSizeKey, 74
- assignClimbWeightMetrics
 - MainWindow, 208
- Athlete, 75
 - createdAt, 76
 - dateOfBirth, 76
 - email, 76
 - firstName, 76
 - ftpWatts, 76
 - fullName, 75
 - heightCm, 76
 - id, 77
 - isValid, 76
 - lastName, 77
- AthleteDialog, 77
 - addFtpRow, 79
 - addWeightRow, 79
 - AthleteDialog, 79
 - deleteFtpRow, 79
 - deleteWeightRow, 79
 - loadAthlete, 79
 - m_athleteId, 80
 - m_dob, 80
 - m_email, 80
 - m_firstName, 80
 - m_ftp, 80
 - m_ftpTable, 80
 - m_height, 80
 - m_lastName, 81
 - m_repo, 81
 - m_savedId, 81
 - m_weightTable, 81
 - onAccepted, 79
 - populateFtpTable, 80
 - populateWeightTable, 80
 - savedAthleteId, 80
- AthleteHeaderWidget, 81
 - AthleteHeaderWidget, 82
- m_metricsLabel, 83
- m_nameLabel, 83
- m_statsLabel, 83
- setRideMetrics, 82
- setSummary, 82
- athleteId
 - Activity, 44
 - FtpEntry, 165
 - ImportJob, 169
 - WeightEntry, 322
- AthleteListDialog, 83
 - AthleteListDialog, 84
 - deleteAthlete, 84
 - editAthlete, 84
 - m_deleteBtn, 85
 - m_editBtn, 85
 - m_repo, 85
 - m_selectBtn, 85
 - m_selectedId, 85
 - m_table, 85
 - newAthlete, 84
 - refresh, 84
 - selectAthlete, 84
 - selectedAthleteId, 85
- athleteName
 - VideoExportSettings, 304
- AthleteRepository, 85
 - addFtpEntry, 86
 - addWeightEntry, 87
 - AthleteRepository, 86
 - deleteAthlete, 87
 - deleteFtpEntry, 87
 - deleteWeightEntry, 88
 - getAthlete, 88
 - getFtpForDate, 88
 - getFtpHistory, 89
 - getWeightHistory, 89
 - insertAthlete, 89
 - listAthletes, 90
 - m_db, 90
 - updateAthlete, 90
- atl
 - FitnessMetricsPoint, 157
- attachQueue
 - ImportProgressModel, 172
- attribution
 - TileCache, 280
 - TileProvider, 284
- autoRouteContrast
 - MapRenderer, 231
- autoZoom
 - VideoExportSettings, 304
- averageCadence
 - Climb, 95
 - ClimbQuarterMetrics, 118
 - Interval, 187
 - WorkoutStatistics, 332
- averageGradient

- Climb, 95
- ClimbRecord, 120
- averageHeartRate
 - Climb, 96
 - ClimbQuarterMetrics, 118
 - Interval, 187
 - TrainingMetricsSummary, 290
 - WorkoutStatistics, 332
- averagePower
 - Climb, 96
 - ClimbQuarterMetrics, 118
 - Interval, 188
 - TrainingMetricsSummary, 290
 - WorkoutStatistics, 333
- averageSpeed
 - Climb, 96
 - Interval, 188
 - WorkoutStatistics, 333
- averageWattsPerKg
 - Climb, 96
- avgCadence
 - Activity, 44
 - anonymous_namespace{ClimbMetrics.cpp}::SliceStats, 277
 - ClimbRecord, 120
 - IntervalRecord, 195
- avgHeartRate
 - anonymous_namespace{ClimbMetrics.cpp}::SliceStats, 277
- avgHr
 - Activity, 44
 - ClimbRecord, 120
 - IntervalRecord, 195
- avgPower
 - Activity, 44
 - anonymous_namespace{ClimbMetrics.cpp}::SliceStats, 277
 - ClimbRecord, 120
 - IntervalRecord, 195
- avgSpeed
 - Activity, 44
- backupDatabase
 - MainWindow, 208
- batchId
 - ImportJob, 169
- bestMeanPower
 - PowerCurve, 243
- bestPower
 - PowerCurvePoint, 245
- bike
 - Activity, 44
- bindBoolValue
 - SqlQueryHelper, 38
- browseLocation
 - CreateDatabaseDialog, 136
- build
 - ClimbMetrics, 115
- buildColorContext
 - MainWindow, 208
- buildCumulativeDistanceMeters
 - ClimbDetector, 100
- buildFilteredAltitudeMeters
 - anonymous_namespace{ClimbDetector.cpp}, 26
- buildInterval
 - IntervalDetector, 189
- buildSampleIndices
 - RideChartWidget, 256
- buildStatusBar
 - MainWindow, 208
- buildToolBar
 - MainWindow, 209
- buildUI
 - MainWindow, 209
- cachedAxisMax
 - RideChartWidget, 267
- cachedAxisMin
 - RideChartWidget, 267
- cachedMaxX
 - RideChartWidget, 267
- cachedMinX
 - RideChartWidget, 267
- cadence
 - RideRecord, 274
- cadenceZones
 - ZoneCalculator, 336
- CalendarWidget, 91
 - addPlannedWorkout, 92
 - CalendarWidget, 92
 - deleteSelectedWorkout, 92
 - editSelectedWorkout, 92
 - m_addButton, 93
 - m_athleteId, 93
 - m_calendar, 93
 - m_colorByCombo, 93
 - m_dayList, 93
 - m_dbManager, 93
 - m_deleteButton, 93
 - m_editButton, 93
 - onDateChanged, 92
 - recolorMonth, 92
 - refresh, 92
 - reloadDayList, 92
 - setAthleteId, 93
 - setDatabaseManager, 93
 - updateMonthFormatting, 93
- cancel
 - VideoRenderJob, 309
- cancelOrClose
 - VideoExportDialog, 299
- canImportActivities
 - MainWindow, 209
- canRedo
 - UndoManager, 294
- canUndo
 - UndoManager, 294
- categorizeClimb

- ClimbDetector, 101
- category
 - Climb, 96
- centerLat
 - MapFitMath::FitResult, 162
- centerLon
 - MapFitMath::FitResult, 162
- chooseOutputFile
 - VideoExportDialog, 299
- clampValue
 - anonymous_namespace{ClimbDetector.cpp}, 26
- classifyClimbCategory
 - anonymous_namespace{ClimbMetrics.cpp}, 27
- classifyClimbShape
 - anonymous_namespace{ClimbMetrics.cpp}, 27
- cleanupExportCache
 - VideoTileProvider, 316
- cleanupFailedOpen
 - DatabaseManager, 140
- clear
 - UndoManager, 294
- clearBackgroundSeries
 - RideChartWidget, 256
- clearChart
 - FitnessChartWidget, 156
 - PowerCurveWidget, 246
 - PowerHistogramWidget, 249
 - RideChartWidget, 256
- clearIntervalSummary
 - MainWindow, 209
- clearMemoryCache
 - TileCache, 280
 - VideoTileProvider, 316
- Climb, 94
 - aerobicDecouplingPct, 95
 - averageCadence, 95
 - averageGradient, 95
 - averageHeartRate, 96
 - averagePower, 96
 - averageSpeed, 96
 - averageWattsPerKg, 96
 - category, 96
 - climbCategory, 96
 - climbScore, 96
 - difficultyScore, 96
 - durationSeconds, 97
 - elevationGainM, 97
 - endDistanceKm, 97
 - endElevationM, 97
 - endSeconds, 97
 - gradePercentile, 97
 - hrDriftPct, 97
 - id, 97
 - lengthKm, 97
 - maximumGradient, 98
 - name, 98
 - normalizedPower, 98
 - powerFadePct, 98
 - powerFirstHalf, 98
 - powerSecondHalf, 98
 - quarterMetrics, 98
 - shapeClass, 98
 - startDistanceKm, 99
 - startElevationM, 99
 - startSeconds, 99
 - vam, 99
 - wattsPerKgRank, 99
- climbBoundaryEdited
 - RideChartWidget, 257
- climbCategory
 - Climb, 96
- climbClicked
 - RideChartWidget, 257
- ClimbDetection
 - AnalysisVersions, 24
- ClimbDetector, 99
 - buildCumulativeDistanceMeters, 100
 - categorizeClimb, 101
 - computeAdaptiveSmoothingWindowsMeters, 101
 - computeClimbScore, 102
 - computeLocalGradientPct, 102
 - computeVAM, 103
 - Config, 100
 - detect, 103
 - mergeFragmentedClimbs, 105
 - smoothAltitudeByDistance, 105
- ClimbDetectorConfig, 106
 - adaptiveSmoothingMaxMeters, 108
 - adaptiveSmoothingMinMeters, 108
 - altitudeSpikeVerticalSpeedMps, 108
 - continueGradient, 108
 - durationScoreWeight, 108
 - elevationSmoothingMeters, 108
 - enableClimbMerging, 108
 - endGradient, 108
 - gainScoreWeight, 109
 - gradePersistenceMinMeters, 109
 - gradePersistenceThreshold, 109
 - gradeScoreWeight, 109
 - gradientWindowHalfMeters, 109
 - maxDipDistanceMeters, 109
 - maxDipMeters, 109
 - maxDipPctOfGain, 109
 - maxPlateauDistanceMeters, 109
 - maxSmallDipsCount, 110
 - maxStartInterruptionMeters, 110
 - mergeMaxGapElevationM, 110
 - mergeMaxGapMeters, 110
 - minAverageGradient, 110
 - minClimbScore, 110
 - minElevationGainM, 110
 - minLengthKm, 110
 - minStartContinuousDistanceMeters, 111
 - plateauMinGradient, 111
 - requireGradePersistence, 111
 - smallDipThresholdMeters, 111

- startGradient, 111
- useAdaptiveSmoothing, 111
- useScoringDetection, 111
- vamScoreBoost, 111
- ClimbEditCommand, 112
 - apply, 113
 - ClimbEditCommand, 113
 - description, 113
 - m_after, 114
 - m_before, 114
 - m_connName, 114
 - m_refresh, 114
 - redo, 113
 - undo, 113
- ClimbHandleDrag
 - RideChartWidget, 255
- ClimbMetrics, 115
 - build, 115
- climbMinGain
 - ActivityAnalysisSettings, 51
- climbMinGradient
 - ActivityAnalysisSettings, 51
- climbMinLength
 - ActivityAnalysisSettings, 51
- ClimbNameTableItem, 116
 - ClimbNameTableItem, 117
 - operator<, 117
 - trailingNumber, 117
- ClimbQuarterMetrics, 117
 - averageCadence, 118
 - averageHeartRate, 118
 - averagePower, 118
- ClimbRecord, 118
 - activityId, 119
 - algorithmVersion, 119
 - averageGradient, 120
 - avgCadence, 120
 - avgHr, 120
 - avgPower, 120
 - deleted, 120
 - elevationGainM, 120
 - endSeconds, 120
 - favorite, 120
 - id, 121
 - lengthKm, 121
 - locked, 121
 - name, 121
 - notes, 121
 - np, 121
 - originalEndSeconds, 121
 - originalStartSeconds, 121
 - rating, 121
 - source, 122
 - startSeconds, 122
 - uuid, 122
 - vam, 122
- ClimbRepository, 122
 - ClimbRepository, 123
 - climbsForActivity, 123
 - deleteClimbsForActivity, 123
 - deletedClimbsForActivity, 124
 - getClimb, 124
 - hardDeleteClimb, 125
 - hasClimbs, 125
 - hasClimbsEver, 125
 - hasLockedClimbs, 126
 - insertClimb, 127
 - m_db, 129
 - softDeleteClimb, 128
 - updateClimb, 128
- climbs
 - WorkoutController, 328
- climbScore
 - Climb, 96
- climbsForActivity
 - ClimbRepository, 123
- ClimbSortKeyTableItem, 129
 - ClimbSortKeyTableItem, 130
 - m_sortKey, 131
 - operator<, 131
- close
 - DatabaseManager, 140
- closeEvent
 - MainWindow, 209
- collectSettings
 - VideoExportDialog, 299
- color
 - Zone, 334
- ColorContext, 131
 - altitudeMax, 132
 - altitudeMin, 132
 - ftp, 132
 - hasAltitudeRange, 132
 - heartRateMax, 132
 - heartRateRest, 132
 - thresholdHeartRate, 133
- colorContext
 - VideoExportSettings, 305
- colorForAltitude
 - ColorProvider, 133
- colorForCadence
 - ColorProvider, 133
- colorForHeartRate
 - ColorProvider, 133
- colorForMetric
 - ColorProvider, 134
- colorForPower
 - ColorProvider, 134
- colorForSpeed
 - ColorProvider, 134
- colorForValue
 - ColorProvider, 134
- ColorProvider, 133
 - colorForAltitude, 133
 - colorForCadence, 133
 - colorForHeartRate, 133

- colorForMetric, [134](#)
 - colorForPower, [134](#)
 - colorForSpeed, [134](#)
 - colorForValue, [134](#)
 - Completed
 - ImportJob, [169](#)
 - completedJobs
 - ImportProgressModel, [172](#)
 - compute
 - FTPZones, [166](#)
 - TrainingMetrics, [289](#)
 - computeAdaptiveSmoothingWindowsMeters
 - ClimbDetector, [101](#)
 - computeClimbScore
 - ClimbDetector, [102](#)
 - computeDistribution
 - ZoneCalculator, [336](#)
 - computeLinearRegressionSlope
 - anonymous_namespace{ClimbMetrics.cpp}, [27](#)
 - computeLocalGradientPct
 - ClimbDetector, [102](#)
 - computeNormalizedPower
 - anonymous_namespace{ClimbMetrics.cpp}, [27](#)
 - computeSliceStats
 - anonymous_namespace{ClimbMetrics.cpp}, [28](#)
 - computeSmoothedPowerSeries
 - RideChartWidget, [257](#)
 - computeSmoothedSeries
 - RideChartWidget, [258](#)
 - computeStandard
 - PowerCurve, [243](#)
 - computeTileViewport
 - anonymous_namespace{VideoRenderJob.cpp}, [33](#)
 - computeVAM
 - ClimbDetector, [103](#)
 - Config
 - ClimbDetector, [100](#)
 - IntervalDetector, [189](#)
 - configureFolderWatcher
 - MainWindow, [209](#)
 - continueGradient
 - ClimbDetectorConfig, [108](#)
 - create
 - DatabaseManager, [140](#)
 - createdAt
 - Athlete, [76](#)
 - createDatabase
 - MainWindow, [209](#)
 - CreateDatabaseDialog, [134](#)
 - accept, [136](#)
 - browseLocation, [136](#)
 - CreateDatabaseDialog, [136](#)
 - databasePath, [136](#)
 - defaultAthleteFirstName, [136](#)
 - defaultAthleteLastName, [136](#)
 - m_browseButton, [137](#)
 - m_buttonBox, [137](#)
 - m_createAthleteCheck, [137](#)
 - m_firstNameEdit, [137](#)
 - m_lastNameEdit, [137](#)
 - m_locationEdit, [137](#)
 - m_nameEdit, [137](#)
 - normalizedDatabaseName, [136](#)
 - shouldCreateDefaultAthlete, [136](#)
 - updateAthleteFields, [136](#)
 - createOrOpenDefaultDatabase
 - MainWindow, [209](#)
 - createVideo
 - MainWindow, [209](#)
 - crosshairMoved
 - RideChartWidget, [258](#)
 - ctl
 - FitnessMetricsPoint, [157](#)
 - currentActivityId
 - WorkoutController, [328](#)
 - currentAthleteId
 - WorkoutController, [328](#)
 - currentColorMetric
 - MainWindow, [210](#)
 - currentFileName
 - ImportProgressModel, [173](#)
 - currentPage
 - NavigationSidebar, [239](#)
 - DailyLoadPoint, [137](#)
 - tss, [138](#)
 - database
 - DatabaseConnectionGuard, [139](#)
 - DatabaseManager, [141](#)
 - DatabaseConnectionGuard, [138](#)
 - ~DatabaseConnectionGuard, [138](#)
 - database, [139](#)
 - DatabaseConnectionGuard, [138](#)
 - m_connectionName, [139](#)
 - DatabaseManager, [139](#)
 - ~DatabaseManager, [140](#)
 - applySchema, [140](#)
 - cleanupFailedOpen, [140](#)
 - close, [140](#)
 - create, [140](#)
 - database, [141](#)
 - isOpen, [141](#)
 - m_connectionName, [143](#)
 - m_db, [143](#)
 - m_path, [143](#)
 - open, [141](#)
 - path, [142](#)
 - upgradeSchema, [142](#)
- databasePath
 - CreateDatabaseDialog, [136](#)
- date
 - PlannedWorkoutDraft, [242](#)
- dateFormat
 - AppSettings, [73](#)
- dateFormatKey
 - AppSettings, [73](#)
- DateFormatter, [143](#)

- formatDate, [144](#)
- formatDateTime, [144](#)
- parseDate, [145](#)
- qtDatePattern, [145](#)
- tolsoDate, [146](#)
- tolsoDateTime, [146](#)
- dateOfBirth
 - Athlete, [76](#)
- datePatternForFormat
 - anonymous_namespace{DateFormatter.cpp}, [29](#)
- decreaseChartHeight
 - MainWindow, [210](#)
- defaultAthleteFirstName
 - CreateDatabaseDialog, [136](#)
- defaultAthleteLastName
 - CreateDatabaseDialog, [136](#)
- defaultBackupPath
 - Platform, [37](#)
- defaultDatabaseDirectory
 - Platform, [37](#)
- defaultDatabasePath
 - Platform, [37](#)
- defaultDatabasePathForAutoCreate
 - MainWindow, [210](#)
- defaultDiskCacheRoot
 - anonymous_namespace{TileCache.cpp}, [32](#)
- defaultPrimaryTileCacheRoot
 - anonymous_namespace{VideoTileProvider.cpp}, [35](#)
- defaultVideoExportCacheRoot
 - anonymous_namespace{VideoTileProvider.cpp}, [35](#)
- defaultZones
 - FTPZones, [167](#)
- deleteActivity
 - ActivityBrowser, [55](#)
 - ActivityRepository, [59](#)
- deleteAthlete
 - AthleteListDialog, [84](#)
 - AthleteRepository, [87](#)
- deleteClimbsForActivity
 - ClimbRepository, [123](#)
- deleted
 - ClimbRecord, [120](#)
 - IntervalRecord, [196](#)
- deletedClimbsForActivity
 - ClimbRepository, [124](#)
- deleteFtpEntry
 - AthleteRepository, [87](#)
- deleteFtpRow
 - AthleteDialog, [79](#)
- deleteInterval
 - IntervalRepository, [198](#)
- deleteIntervalsForActivity
 - IntervalRepository, [198](#)
- deleteSelectedWorkout
 - CalendarWidget, [92](#)
- deleteTemporaryTilesAfterExport
 - VideoExportSettings, [305](#)
 - VideoTileProviderConfig, [320](#)
- deleteWeightEntry
 - AthleteRepository, [88](#)
- deleteWeightRow
 - AthleteDialog, [79](#)
- description
 - ClimbEditCommand, [113](#)
 - UndoCommand, [291](#)
- detect
 - ClimbDetector, [103](#)
 - IntervalDetector, [189](#)
- detectClimbsAndRefresh
 - MainWindow, [210](#)
- difficultyScore
 - Climb, [96](#)
- diskCacheRoot
 - TileCache, [280](#)
 - TileCacheConfig, [283](#)
- diskFallbackRoots
 - TileCacheConfig, [283](#)
- diskTilePath
 - TileCache, [280](#)
- diskTilePathForRoot
 - TileCache, [280](#)
- dispatchNextDownloads
 - TileCache, [280](#)
- displayDateOrRaw
 - anonymous_namespace{AthleteDialog.cpp}, [26](#)
- distanceM
 - Activity, [45](#)
- downloadedTiles
 - VideoTileStats, [321](#)
- draft
 - PlannedWorkoutDialog, [240](#)
- dragEnterEvent
 - ActivityBrowser, [55](#)
 - MainWindow, [210](#)
- DragHandle
 - MapRenderer, [231](#)
- drawForeground
 - RideChartWidget, [258](#)
- dropEvent
 - ActivityBrowser, [55](#)
 - MainWindow, [210](#)
- DuplicateActivityInfo, [147](#)
 - existingActivityId, [147](#)
 - existingDurationSeconds, [147](#)
 - existingStartUtc, [147](#)
 - newDurationSeconds, [147](#)
 - newStartUtc, [147](#)
- duplicates
 - MainWindow::ImportBatchSummary, [168](#)
- durationScoreWeight
 - ClimbDetectorConfig, [108](#)
- durationSec
 - Activity, [45](#)
- durationSeconds

- Climb, 97
- Interval, 188
- PlannedWorkoutDraft, 242
- PowerCurvePoint, 245
- WorkoutStatistics, 333
- EditActivityDialog, 147
 - EditActivityDialog, 149
 - m_bikeEdit, 149
 - m_equipmentEdit, 149
 - m_fatigueSpin, 149
 - m_initial, 149
 - m_notesEdit, 149
 - m_rpeSpin, 149
 - m_sleepSpin, 149
 - m_temperatureSpin, 149
 - m_weatherEdit, 149
 - m_weightSpin, 149
 - m_windEdit, 150
 - updatedActivity, 149
- editAthlete
 - AthleteListDialog, 84
- editCurrentActivityProperties
 - MainWindow, 211
- editNotes
 - ActivityBrowser, 55
- EditNotesDialog, 150
 - EditNotesDialog, 151
 - m_notes, 151
 - m_weather, 151
 - notes, 151
 - weatherNotes, 151
- editSelectedClimbBoundaries
 - MainWindow, 211
- editSelectedWorkout
 - CalendarWidget, 92
- effectiveFrom
 - FtpEntry, 165
 - WeightEntry, 322
- efficiencyFactor
 - TrainingMetricsSummary, 290
 - WorkoutController, 328
- elapsedSeconds
 - RideRecord, 275
- elevationGain
 - Activity, 45
- elevationGainM
 - Climb, 97
 - ClimbRecord, 120
 - WorkoutStatistics, 333
- elevationSmoothingMeters
 - ClimbDetectorConfig, 108
- email
 - Athlete, 76
- enableClimbMerging
 - ClimbDetectorConfig, 108
- endDistanceKm
 - Climb, 97
- endElevationM
 - Climb, 97
- endGradient
 - ClimbDetectorConfig, 108
- endSample
 - IntervalRecord, 196
- endSeconds
 - Climb, 97
 - ClimbRecord, 120
 - Interval, 188
- endTime
 - Activity, 45
- enqueue
 - AnalysisQueue, 66
 - ImportQueue, 176
- ensureAthlete
 - MainWindow, 211
- ensureDatabase
 - MainWindow, 211
- ensureImportReady
 - MainWindow, 211
- equipment
 - Activity, 45
- errorMessage
 - ImportJob, 169
 - ImportRecord, 179
- errors
 - MainWindow::ImportBatchSummary, 168
- estimatedFtpFromCurrentRide
 - MainWindow, 211
- estimateSpeedKmhAt
 - anonymous_namespace{ClimbDetector.cpp}, 26
- eventFilter
 - MainWindow, 211
- existingActivityId
 - DuplicateActivityInfo, 147
- existingDurationSeconds
 - DuplicateActivityInfo, 147
- existingStartUtc
 - DuplicateActivityInfo, 147
- exportCacheRoot
 - VideoTileProvider, 316
 - VideoTileProviderConfig, 320
- ExportTileCleanupGuard
 - anonymous_namespace{VideoRenderJob.cpp}::ExportTileCleanupG
 - 152
- Failed
 - ImportJob, 169
- failed
 - ImportWorker, 185
 - MainWindow::ImportBatchSummary, 168
- failedJobs
 - ImportProgressModel, 173
- fatigue
 - Activity, 45
- favorite
 - ClimbRecord, 120
- ffmpegPath
 - VideoExportSettings, 305

- fileName
 - Activity, [45](#)
 - ImportRecord, [179](#)
- filePath
 - ImportJob, [170](#)
- fileTimestampUtc
 - RideData, [272](#)
- finalizeImportBatches
 - MainWindow, [212](#)
- fingerprint
 - Activity, [45](#)
- finished
 - VideoRenderJob, [309](#)
- firstName
 - Athlete, [76](#)
- firstTimestamp
 - FitRecordListener, [160](#)
- fitFilesFromMimeData
 - MainWindow, [212](#)
- fitHash
 - Activity, [46](#)
- FitlyzerC, [1](#)
- FitnessChartWidget, [154](#)
 - clearChart, [156](#)
 - FitnessChartWidget, [155](#)
 - m_atlSeries, [156](#)
 - m_axisX, [156](#)
 - m_axisY, [156](#)
 - m_ctlSeries, [156](#)
 - m_tsbSeries, [156](#)
 - setTimeline, [156](#)
- FitnessMetricsPoint, [156](#)
 - atl, [157](#)
 - ctl, [157](#)
 - tsb, [157](#)
- fitnessTimeline
 - TrainingLoad, [286](#)
- FitParser, [157](#)
 - load, [158](#)
- FitRecordListener, [159](#)
 - firstTimestamp, [160](#)
 - FitRecordListener, [160](#)
 - lastTimestamp, [160](#)
 - m_firstTimestamp, [161](#)
 - m_lastTimestamp, [161](#)
 - m_rideData, [161](#)
 - OnMsg, [161](#)
- fitTimestampToIsoUtc
 - anonymous_namespace{FitParser.cpp}, [30](#)
- fitToBounds
 - MapRenderer, [231](#)
- fitToData
 - RideChartWidget, [258](#)
- fitToTrack
 - MapRenderer, [232](#)
- fitToVisibleRange
 - MapRenderer, [232](#)
- fixedZoom
 - VideoExportSettings, [305](#)
- fmtDuration
 - anonymous_namespace{VideoRenderJob.cpp}, [33](#)
- focusSearch
 - ActivityBrowser, [55](#)
- followAthlete
 - VideoExportSettings, [305](#)
- formatDate
 - DateFormatter, [144](#)
- formatDateTime
 - DateFormatter, [144](#)
- formatLegendValue
 - anonymous_namespace{VideoRenderJob.cpp}, [33](#)
- formatVideoLength
 - anonymous_namespace{VideoExportDialog.cpp}, [32](#)
- fps
 - VideoExportSettings, [305](#)
- frameProgressChanged
 - VideoRenderJob, [310](#)
- ftp
 - ColorContext, [132](#)
 - IntervalDetectorConfig, [191](#)
 - WorkoutController, [328](#)
- FtpEntry, [164](#)
 - athleteId, [165](#)
 - effectiveFrom, [165](#)
 - ftpWatts, [165](#)
 - id, [165](#)
 - notes, [165](#)
- ftpWatts
 - Athlete, [76](#)
 - FtpEntry, [165](#)
- FTPZones, [166](#)
 - compute, [166](#)
 - defaultZones, [167](#)
 - kZoneCount, [167](#)
 - ZoneArray, [166](#)
- fullName
 - Athlete, [75](#)
- gainScoreWeight
 - ClimbDetectorConfig, [109](#)
- getActivity
 - ActivityRepository, [59](#)
- getAthlete
 - AthleteRepository, [88](#)
- getCache
 - ActivityAnalysisCacheRepository, [48](#)
- getClimb
 - ClimbRepository, [124](#)
- getFtpForDate
 - AthleteRepository, [88](#)
- getFtpHistory
 - AthleteRepository, [89](#)
- getRecentImports
 - ImportRepository, [181](#)
- getTile
 - VideoTileProvider, [317](#)

- getWeightHistory
 - AthleteRepository, 89
- gpsRecordAtTime
 - MapRenderer, 232
- gradePercentile
 - Climb, 97
- gradePersistenceMinMeters
 - ClimbDetectorConfig, 109
- gradePersistenceThreshold
 - ClimbDetectorConfig, 109
- gradeScoreWeight
 - ClimbDetectorConfig, 109
- gradientWindowHalfMeters
 - ClimbDetectorConfig, 109
- hardDeleteClimb
 - ClimbRepository, 125
- hasAltitude
 - RideRecord, 275
- hasAltitudeRange
 - ColorContext, 132
- hasCache
 - ActivityAnalysisCacheRepository, 49
- hasCadence
 - RideRecord, 275
- HasClimbs
 - ActivityAnalysisFlags, 23
- hasClimbs
 - ClimbRepository, 125
- hasClimbsEver
 - ClimbRepository, 125
- hasData
 - RideChartWidget, 259
- hasFingerprintMatch
 - ActivityRepository, 59
- hasFitHash
 - ActivityRepository, 59
- hasGps
 - RideRecord, 275
- hasHeartRate
 - RideRecord, 275
- HasHistogram
 - ActivityAnalysisFlags, 23
- HasIntervals
 - ActivityAnalysisFlags, 23
- hasLockedClimbs
 - ClimbRepository, 126
- hasLockedIntervals
 - IntervalRepository, 198
- hasMetricSamples
 - ZoneCalculator, 337
- hasPower
 - RideRecord, 275
- HasPowerCurve
 - ActivityAnalysisFlags, 24
- hasSpeed
 - RideRecord, 275
- HasVideo
 - ActivityAnalysisFlags, 24

- heartRate
 - RideRecord, 276
- heartRateMax
 - ColorContext, 132
- heartRateRest
 - ColorContext, 132
- heartRateZones
 - ZoneCalculator, 337
- height
 - VideoExportSettings, 305
- heightCm
 - Athlete, 76
- hideWelcomeScreen
 - MainWindow, 212
- hit
 - anonymous_namespace{RideChartWidget.cpp}::ClimbHandleHitRes
114
- hitTestClimbHandles
 - anonymous_namespace{RideChartWidget.cpp},
31
- hrDriftPct
 - Climb, 97
- id
 - Activity, 46
 - Athlete, 77
 - Climb, 97
 - ClimbRecord, 121
 - FtpEntry, 165
 - ImportRecord, 179
 - IntervalRecord, 196
 - PlannedWorkoutDraft, 242
 - WeightEntry, 322
- importActivities
 - MainWindow, 212
- imported
 - MainWindow::ImportBatchSummary, 168
- importedAt
 - Activity, 46
 - ImportRecord, 179
- importFile
 - WorkoutController, 328
- importFiles
 - MainWindow, 212
- importFilesInternal
 - MainWindow, 213
- ImportJob, 168
 - athleteId, 169
 - batchId, 169
 - Completed, 169
 - errorMessage, 169
 - Failed, 169
 - filePath, 170
 - importSource, 170
 - jobId, 170
 - Parsing, 169
 - progress, 170
 - Queued, 169
 - State, 169

- state, 170
- Storing, 169
- ImportProgressModel, 170
 - activeJobs, 172
 - attachQueue, 172
 - completedJobs, 172
 - currentFileName, 173
 - failedJobs, 173
 - ImportProgressModel, 172
 - m_activeJobs, 173
 - m_completedJobs, 173
 - m_currentFileName, 173
 - m_failedJobs, 173
 - m_progress, 173
 - m_statusText, 173
 - progress, 173
 - statusText, 173
- ImportQueue, 174
 - ~ImportQueue, 175
 - activeCountUnsafe, 176
 - enqueue, 176
 - ImportQueue, 175
 - isBusy, 176
 - jobFailed, 176
 - m_busy, 178
 - m_dbPath, 178
 - m_queue, 178
 - m_runningJob, 178
 - m_thread, 178
 - m_worker, 178
 - onWorkerCompleted, 177
 - onWorkerFailed, 177
 - onWorkerProgress, 177
 - pendingCount, 177
 - processNext, 178
 - setDatabasePath, 178
- ImportRecord, 179
 - activityId, 179
 - errorMessage, 179
 - fileName, 179
 - id, 179
 - importedAt, 179
 - status, 180
- ImportRepository, 180
 - getRecentImports, 181
 - ImportRepository, 180
 - logImport, 181
 - m_db, 181
- importSource
 - Activity, 46
 - ImportJob, 170
- ImportStatusWidget, 182
 - ImportStatusWidget, 183
 - m_detailLabel, 183
 - m_model, 183
 - m_progressBar, 183
 - m_titleLabel, 183
 - setModel, 183
 - updateFromModel, 183
- ImportWorker, 184
 - failed, 185
 - ImportWorker, 185
 - m_dbPath, 187
 - processJob, 185
 - setDatabasePath, 186
- increaseChartHeight
 - MainWindow, 214
- insertActivity
 - ActivityRepository, 60
- insertAthlete
 - AthleteRepository, 89
- insertClimb
 - ClimbRepository, 127
- insertInterval
 - IntervalRepository, 199
- insertOrUpdate
 - ActivityAnalysisSettingsRepository, 53
- instance
 - AppSettings, 73
- intensityFactor
 - TrainingLoad, 286
 - TrainingMetricsSummary, 290
 - WorkoutController, 328
- interpolateColor
 - anonymous_namespace{ColorProvider.cpp}, 29
- interpolateGpsAtTime
 - anonymous_namespace{VideoRenderJob.cpp}, 33
- Interval, 187
 - averageCadence, 187
 - averageHeartRate, 187
 - averagePower, 188
 - averageSpeed, 188
 - durationSeconds, 188
 - endSeconds, 188
 - label, 188
 - maximumPower, 188
 - normalizedPower, 188
 - startSeconds, 188
- IntervalDetection
 - AnalysisVersions, 24
- IntervalDetector, 189
 - buildInterval, 189
 - Config, 189
 - detect, 189
 - removeOverlappingAndDuplicates, 190
 - smoothPower, 190
- IntervalDetectorConfig, 191
 - ftp, 191
 - minRecoverySeconds, 191
 - minWorkSeconds, 191
 - smoothingWindow, 191
 - workThresholdPct, 191
- IntervalEditorDialog, 192
 - IntervalEditorDialog, 193
 - intervalName, 193
 - intervalNotes, 193

- intervalType, 193
- m_endSpin, 194
- m_nameEdit, 194
- m_notesEdit, 194
- m_startSpin, 194
- m_typeEdit, 194
- intervalName
 - IntervalEditorDialog, 193
- intervalNotes
 - IntervalEditorDialog, 193
- IntervalRecord, 194
 - activityId, 195
 - algorithmVersion, 195
 - avgCadence, 195
 - avgHr, 195
 - avgPower, 195
 - deleted, 196
 - endSample, 196
 - id, 196
 - locked, 196
 - name, 196
 - notes, 196
 - np, 196
 - source, 197
 - startSample, 197
 - type, 197
 - uuid, 197
- IntervalRepository, 197
 - deleteInterval, 198
 - deleteIntervalsForActivity, 198
 - hasLockedIntervals, 198
 - insertInterval, 199
 - IntervalRepository, 198
 - listIntervalsForActivity, 200
 - m_db, 201
 - softDeleteInterval, 200
 - updateInterval, 200
- intervalRestThreshold
 - ActivityAnalysisSettings, 51
- intervals
 - WorkoutController, 328
- intervalSelectionRequested
 - RideChartWidget, 259
- intervalType
 - IntervalEditorDialog, 193
- intervalWorkThreshold
 - ActivityAnalysisSettings, 52
- invalidateCache
 - ActivityAnalysisCacheRepository, 49
- invalidateKey
 - ActivityAnalysisCacheRepository, 49
- isBusy
 - AnalysisQueue, 66
 - ImportQueue, 176
- isFinite
 - anonymous_namespace{ClimbDetector.cpp}, 26
 - anonymous_namespace{ClimbMetrics.cpp}, 28
- isOpen
 - DatabaseManager, 141
- isTileCachedOnDisk
 - TileCache, 280
 - VideoTileProvider, 317
- isValid
 - Activity, 44
 - ActivityAnalysisSettings, 51
 - Athlete, 76
- jobFailed
 - ImportQueue, 176
- jobId
 - ImportJob, 170
- joinSelectedClimbs
 - MainWindow, 214
- kAnalysisTabCalendar
 - MainWindow, 220
- kAnalysisTabCharts
 - MainWindow, 220
- kAnalysisTabClimbing
 - MainWindow, 220
- kAnalysisTabFitness
 - MainWindow, 220
- kAnalysisTabHistogram
 - MainWindow, 220
- kAnalysisTabPDC
 - MainWindow, 220
- kAnalysisTabZones
 - MainWindow, 220
- kClimbHandleCenterOffsetYPx
 - anonymous_namespace{RideChartWidget.cpp}, 32
- kClimbHandleGrabRadiusPx
 - anonymous_namespace{RideChartWidget.cpp}, 32
- kClimbHandleRadiusPx
 - anonymous_namespace{RideChartWidget.cpp}, 32
- kEntryIdRole
 - anonymous_namespace{AthleteDialog.cpp}, 26
- key
 - TileCache, 280
- keyForStyle
 - TileCache, 281
- kIsoDateRole
 - anonymous_namespace{ActivityBrowser.cpp}, 25
- kMaxConcurrentDownloads
 - TileCache, 282
- kMaxMercatorLat
 - anonymous_namespace{MapFitMath.cpp}, 31
- kMinLatSpan
 - anonymous_namespace{MapFitMath.cpp}, 31
- kMinLonSpan
 - anonymous_namespace{MapFitMath.cpp}, 31
- kNotesRole
 - anonymous_namespace{ActivityBrowser.cpp}, 25
- kPi
 - anonymous_namespace{MapFitMath.cpp}, 31

- anonymous_namespace{VideoRenderJob.cpp}, 35
- kSelectCols
 - anonymous_namespace{ClimbRepository.cpp}, 28
- kTabActivities
 - MainWindow, 220
- kTabAnalysis
 - MainWindow, 220
- kTilePx
 - anonymous_namespace{MapFitMath.cpp}, 31
 - anonymous_namespace{VideoRenderJob.cpp}, 35
- kVideoExportTileCache
 - anonymous_namespace{VideoRenderJob.cpp}, 35
- kWeatherRole
 - anonymous_namespace{ActivityBrowser.cpp}, 25
- kWhereDeleted
 - SqlQueryHelper, 40
- kWhereLocked
 - SqlQueryHelper, 40
- kWhereNotDeleted
 - SqlQueryHelper, 40
- kZoneCount
 - FTPZones, 167
- label
 - Interval, 188
- lastName
 - Athlete, 77
- lastTimestamp
 - FitRecordListener, 160
- latitude
 - RideRecord, 276
- latLonToTile
 - anonymous_namespace{VideoRenderJob.cpp}, 34
 - MapRenderer, 232
- latToTileY
 - anonymous_namespace{MapFitMath.cpp}, 30
- leaveEvent
 - RideChartWidget, 259
- lengthKm
 - Climb, 97
 - ClimbRecord, 121
- listActivities
 - ActivityRepository, 60
- listAthletes
 - AthleteRepository, 90
- listIntervalsForActivity
 - IntervalRepository, 200
- load
 - FitParser, 158
- loadActivity
 - WorkoutController, 329
- loadActivitySamples
 - RideDataLoader, 273
- loadAthlete
 - AthleteDialog, 79
- loadFile
 - WorkoutController, 329
- loadOrGenerateClimbs
 - WorkoutController, 329
- loadOrGenerateIntervals
 - WorkoutController, 329
- loadRideFromDatabase
 - RideDataSerializer, 273
- loadSettings
 - MainWindow, 214
- locked
 - ClimbRecord, 121
 - IntervalRecord, 196
- logImport
 - ImportRepository, 181
- longitude
 - RideRecord, 276
- lonSpanDegrees
 - MapFitMath::FitResult, 162
- lowerPct
 - PowerZone, 251
- m_activeDownloads
 - TileCache, 282
- m_activeJobs
 - ImportProgressModel, 173
- m_activityBrowser
 - MainWindow, 220
- m_activityCountLabel
 - MainWindow, 220
- m_activityNotesView
 - MainWindow, 220
- m_activityTabStack
 - MainWindow, 220
- m_addButton
 - CalendarWidget, 93
- m_after
 - ClimbEditCommand, 114
- m_altitudeChart
 - MainWindow, 221
- m_altitudeCheck
 - VideoExportDialog, 300
- m_analysisQueue
 - MainWindow, 221
 - WorkoutController, 330
- m_analysisTabWidget
 - MainWindow, 221
- m_analyzer
 - WorkoutController, 330
- m_athleteCheck
 - VideoExportDialog, 300
- m_athleteCombo
 - MainWindow, 221
- m_athleteHeader
 - MainWindow, 221
- m_athleteId
 - ActivityBrowser, 56
 - AthleteDialog, 80
 - CalendarWidget, 93
- m_athleteLabelAct
 - MainWindow, 221
- m_athletesMenu
 - MainWindow, 221

- m_athleteStatusLabel
 - MainWindow, [221](#)
- m_atlSeries
 - FitnessChartWidget, [156](#)
- m_autoFitVisibleRange
 - MapRenderer, [234](#)
- m_autoRouteContrast
 - MapRenderer, [234](#)
- m_autoSmoothing
 - RideChartWidget, [267](#)
- m_autoSmoothingCheck
 - MainWindow, [221](#)
- m_axisX
 - FitnessChartWidget, [156](#)
 - RideChartWidget, [267](#)
- m_axisXDisplay
 - RideChartWidget, [267](#)
- m_axisY
 - FitnessChartWidget, [156](#)
 - RideChartWidget, [268](#)
- m_axisYOverlay
 - RideChartWidget, [268](#)
- m_backgroundSeries
 - RideChartWidget, [268](#)
- m_before
 - ClimbEditCommand, [114](#)
- m_bikeEdit
 - EditActivityDialog, [149](#)
- m_browseButton
 - CreateDatabaseDialog, [137](#)
- m_busy
 - AnalysisQueue, [68](#)
 - ImportQueue, [178](#)
- m_buttonBox
 - CreateDatabaseDialog, [137](#)
- m_cache
 - TileCache, [282](#)
- m_cadenceChart
 - MainWindow, [221](#)
- m_cadenceCheck
 - VideoExportDialog, [300](#)
- m_calendar
 - CalendarWidget, [93](#)
- m_calendarTabStack
 - MainWindow, [221](#)
- m_calendarWidget
 - MainWindow, [221](#)
- m_cancelButton
 - VideoExportDialog, [300](#)
- m_cancelRequested
 - VideoRenderJob, [313](#)
- m_centerLat
 - MapRenderer, [234](#)
- m_centerLon
 - MapRenderer, [234](#)
- m_chartHeight
 - MainWindow, [222](#)
- m_chartHeightDecreaseButton
 - MainWindow, [222](#)
- m_chartHeightIncreaseButton
 - MainWindow, [222](#)
- m_chartMapSplit
 - MainWindow, [222](#)
- m_chartPresetCombo
 - MainWindow, [222](#)
- m_chartsCheck
 - VideoExportDialog, [300](#)
- m_chartsContainer
 - MainWindow, [222](#)
- m_chartScroll
 - MainWindow, [222](#)
- m_chartsLayout
 - MainWindow, [222](#)
- m_climbAltitudeChart
 - MainWindow, [222](#)
- m_climbCadenceChart
 - MainWindow, [222](#)
- m_climbDipDistanceSpin
 - MainWindow, [222](#)
- m_climbDipMetersSpin
 - MainWindow, [222](#)
- m_climbEditingEnabled
 - RideChartWidget, [268](#)
- m_climbHandleDrag
 - RideChartWidget, [268](#)
- m_climbHrChart
 - MainWindow, [223](#)
- m_climbingTabStack
 - MainWindow, [223](#)
- m_climbMinGainSpin
 - MainWindow, [223](#)
- m_climbMinGradientSpin
 - MainWindow, [223](#)
- m_climbMinLengthSpin
 - MainWindow, [223](#)
- m_climbOverlayEnabledCheck
 - MainWindow, [223](#)
- m_climbOverlayMetricCombo
 - MainWindow, [223](#)
- m_climbPowerChart
 - MainWindow, [223](#)
- m_climbQuarterCadenceChart
 - MainWindow, [223](#)
- m_climbQuarterHrChart
 - MainWindow, [223](#)
- m_climbQuarterPowerChart
 - MainWindow, [223](#)
- m_climbs
 - RideChartWidget, [268](#)
 - WorkoutController, [330](#)
- m_climbSmoothingSpin
 - MainWindow, [223](#)
- m_climbSpeedChart
 - MainWindow, [224](#)
- m_climbsTable
 - MainWindow, [224](#)

- m_climbStartGradientSpin
 - MainWindow, 224
- m_climbSummaryLabel
 - MainWindow, 224
- m_colorByCombo
 - CalendarWidget, 93
- m_colorContext
 - MapRenderer, 235
 - RideChartWidget, 268
- m_colorLegendLayout
 - MainWindow, 224
- m_colorLegendWidget
 - MainWindow, 224
- m_colorMetric
 - RideChartWidget, 268
- m_colorMetricCombo
 - MainWindow, 224
- m_colorSeries
 - RideChartWidget, 268
- m_completedJobs
 - ImportProgressModel, 173
- m_connectionName
 - DatabaseConnectionGuard, 139
 - DatabaseManager, 143
- m_connName
 - ClimbEditCommand, 114
- m_controller
 - MainWindow, 224
- m_createAthleteCheck
 - CreateDatabaseDialog, 137
- m_createDbButton
 - WelcomeWidget, 324
- m_crosshairX
 - RideChartWidget, 268
- m_crosshairY
 - RideChartWidget, 268
- m_ctlSeries
 - FitnessChartWidget, 156
- m_currentActivityId
 - WorkoutController, 330
- m_currentAthleteId
 - MainWindow, 224
 - WorkoutController, 330
- m_currentAthleteName
 - MainWindow, 224
- m_currentFileName
 - ImportProgressModel, 173
- m_currentTimeSeconds
 - MapRenderer, 235
- m_dataMaxY
 - RideChartWidget, 268
- m_dataMinY
 - RideChartWidget, 269
- m_dateEdit
 - PlannedWorkoutDialog, 241
- m_dayList
 - CalendarWidget, 93
- m_db
 - ActivityAnalysisCacheRepository, 50
 - ActivityAnalysisSettingsRepository, 53
 - ActivityRepository, 63
 - AthleteRepository, 90
 - ClimbRepository, 129
 - DatabaseManager, 143
 - ImportRepository, 181
 - IntervalRepository, 201
- m_dbManager
 - ActivityBrowser, 56
 - CalendarWidget, 93
 - MainWindow, 224
 - WorkoutController, 331
- m_dbPath
 - AnalysisQueue, 68
 - AnalysisWorker, 72
 - ImportQueue, 178
 - ImportWorker, 187
- m_dbStatusLabel
 - MainWindow, 224
- m_defaults
 - VideoExportDialog, 300
- m_deleteBtn
 - AthleteListDialog, 85
- m_deleteButton
 - CalendarWidget, 93
- m_deleteTemporaryTilesAfterExport
 - VideoTileProvider, 319
- m_deleteTemporaryTilesCheck
 - VideoExportDialog, 300
- m_detailLabel
 - ImportStatusWidget, 183
- m_detectedClimbs
 - MainWindow, 225
- m_diskCacheRoot
 - TileCache, 282
- m_diskFallbackRoots
 - TileCache, 282
- m_dob
 - AthleteDialog, 80
- m_downloadedTiles
 - VideoTileProvider, 319
- m_downloadQueue
 - TileCache, 282
- m_draggingSelection
 - MapRenderer, 235
- m_dragHandle
 - MapRenderer, 235
- m_dragOriginalClimbEndSec
 - RideChartWidget, 269
- m_dragOriginalClimbStartSec
 - RideChartWidget, 269
- m_durationSpin
 - PlannedWorkoutDialog, 241
- m_editBtn
 - AthleteListDialog, 85
- m_editButton
 - CalendarWidget, 93

- m_ef
 - WorkoutController, 331
- m_effectivePowerSmoothingSeconds
 - RideChartWidget, 269
- m_email
 - AthleteDialog, 80
- m_emptyStateLabel
 - ActivityBrowser, 56
- m_endMarkerRect
 - MapRenderer, 235
- m_endSpin
 - IntervalEditorDialog, 194
- m_entireActivityRadio
 - VideoExportDialog, 300
- m_equipementEdit
 - EditActivityDialog, 149
- m_etaLabel
 - VideoExportDialog, 300
- m_exportButton
 - VideoExportDialog, 301
- m_failedJobs
 - ImportProgressModel, 173
- m_fatigueSpin
 - EditActivityDialog, 149
- m_firstGpsRecord
 - MapRenderer, 235
- m_firstLaunchCompleted
 - MainWindow, 225
- m_firstName
 - AthleteDialog, 80
- m_firstNameEdit
 - CreateDatabaseDialog, 137
- m_firstTimestamp
 - FitRecordListener, 161
- m_fitChartsButton
 - MainWindow, 225
- m_fitnessChart
 - MainWindow, 225
- m_fitnessTabStack
 - MainWindow, 225
- m_fixedZoomSpin
 - VideoExportDialog, 301
- m_followIntervalSelectionCheck
 - MainWindow, 225
- m_fpsSpin
 - VideoExportDialog, 301
- m_frameLabel
 - VideoExportDialog, 301
- m_ftp
 - AthleteDialog, 80
 - WorkoutController, 331
- m_ftpTable
 - AthleteDialog, 80
- m_gpsRecords
 - MapRenderer, 235
- m_hasData
 - RideChartWidget, 269
- m_hasGps
 - MapRenderer, 235
- m_height
 - AthleteDialog, 80
- m_highlightElapsedSeconds
 - MapRenderer, 235
- m_histogram
 - MainWindow, 225
- m_histogramTabStack
 - MainWindow, 225
- m_hrChart
 - MainWindow, 225
- m_hrCheck
 - VideoExportDialog, 301
- m_id
 - PlannedWorkoutDialog, 241
- m_if
 - WorkoutController, 331
- m_importAct
 - MainWindow, 225
- m_importBatchSummaries
 - MainWindow, 225
- m_importButton
 - WelcomeWidget, 324
- m_importProgressModel
 - MainWindow, 225
- m_importQueue
 - MainWindow, 226
- m_importRefreshTimer
 - MainWindow, 226
- m_importStatusWidget
 - MainWindow, 226
- m_initial
 - EditActivityDialog, 149
- m_intervalEndMin
 - RideChartWidget, 269
- m_intervals
 - RideChartWidget, 269
 - WorkoutController, 331
- m_intervalSelecting
 - RideChartWidget, 269
- m_intervalsTable
 - MainWindow, 226
- m_intervalStartMin
 - RideChartWidget, 269
- m_intervalSummaryLabel
 - MainWindow, 226
- m_isRebuilding
 - RideChartWidget, 269
- m_job
 - VideoExportDialog, 301
- m_knownWatchedFitFiles
 - MainWindow, 226
- m_lapsTable
 - MainWindow, 226
- m_lastGpsRecord
 - MapRenderer, 235
- m_lastName
 - AthleteDialog, 81

- m_lastNameEdit
 - CreateDatabaseDialog, 137
- m_lastOpenDir
 - MainWindow, 226
- m_lastTimestamp
 - FitRecordListener, 161
- m_legendCheck
 - VideoExportDialog, 301
- m_list
 - NavigationSidebar, 239
- m_locationEdit
 - CreateDatabaseDialog, 137
- m_mapAutoContrastCheck
 - MainWindow, 226
- m_mapRenderer
 - MainWindow, 226
- m_mapStyle
 - TileCache, 282
- m_mapStyleCombo
 - MainWindow, 226
 - VideoExportDialog, 301
- m_maxLat
 - MapRenderer, 235
- m_maxLon
 - MapRenderer, 235
- m_maxStackSize
 - UndoManager, 295
- m_metric
 - RideChartWidget, 269
- m_metricOverlayEnabled
 - RideChartWidget, 269
- m_metricOverlayMetric
 - RideChartWidget, 270
- m_metricsLabel
 - AthleteHeaderWidget, 83
- m_minLat
 - MapRenderer, 236
- m_minLon
 - MapRenderer, 236
- m_minZoom
 - MapRenderer, 236
- m_model
 - ImportStatusWidget, 183
- m_nam
 - TileCache, 282
- m_nameEdit
 - CreateDatabaseDialog, 137
 - IntervalEditorDialog, 194
- m_nameLabel
 - AthleteHeaderWidget, 83
- m_newClimbCreating
 - RideChartWidget, 270
- m_newClimbCurrentSec
 - RideChartWidget, 270
- m_newClimbDragging
 - RideChartWidget, 270
- m_newClimbStartSec
 - RideChartWidget, 270
- m_notes
 - EditNotesDialog, 151
- m_notesEdit
 - EditActivityDialog, 149
 - IntervalEditorDialog, 194
 - PlannedWorkoutDialog, 241
- m_np
 - WorkoutController, 331
- m_openDbButton
 - WelcomeWidget, 324
- m_origMaxX
 - RideChartWidget, 270
- m_outputEdit
 - VideoExportDialog, 301
- m_overlaySeries
 - RideChartWidget, 270
- m_panning
 - MapRenderer, 236
- m_panStart
 - MapRenderer, 236
- m_path
 - DatabaseManager, 143
- m_pdcPoints
 - WorkoutController, 331
- m_pdcWidget
 - MainWindow, 226
- m_pending
 - TileCache, 283
- m_powerChart
 - MainWindow, 227
- m_powerCheck
 - VideoExportDialog, 301
- m_powerCurveTabStack
 - MainWindow, 227
- m_powerSmoothingCombo
 - MainWindow, 227
- m_powerZones
 - WorkoutController, 331
- m_previewAct
 - MainWindow, 227
- m_progress
 - ImportProgressModel, 173
- m_progressBar
 - ImportStatusWidget, 183
 - VideoExportDialog, 301
- m_provider
 - anonymous_namespace{VideoRenderJob.cpp}::ExportTileCleanupG
153
 - TileCache, 283
- m_queue
 - AnalysisQueue, 68
 - ImportQueue, 178
- m_rangeFilter
 - ActivityBrowser, 56
- m_rawSeries
 - RideChartWidget, 270
- m_recentDbMenu
 - MainWindow, 227

- m_redoStack
 - UndoManager, 295
- m_referenceSeries
 - RideChartWidget, 270
- m_refresh
 - ClimbEditCommand, 114
- m_repo
 - AthleteDialog, 81
 - AthleteListDialog, 85
- m_requestedPowerSmoothingSeconds
 - RideChartWidget, 270
- m_requiredTiles
 - VideoTileProvider, 319
- m_requiredTilesOnDisk
 - VideoTileProvider, 319
- m_resolutionCombo
 - VideoExportDialog, 301
- m_rideData
 - FitRecordListener, 161
 - MapRenderer, 236
 - RideChartWidget, 270
 - WorkoutController, 331
- m_routeColorCombo
 - VideoExportDialog, 302
- m_routeColorMode
 - MapRenderer, 236
- m_rpeSpin
 - EditActivityDialog, 149
- m_running
 - VideoExportDialog, 302
- m_runningJob
 - ImportQueue, 178
- m_savedId
 - AthleteDialog, 81
- m_searchEdit
 - ActivityBrowser, 56
- m_segmentColors
 - MapRenderer, 236
- m_selectBtn
 - AthleteListDialog, 85
- m_selectedClimbIndex
 - RideChartWidget, 270
- m_selectedId
 - AthleteListDialog, 85
- m_selectedSegmentRadio
 - VideoExportDialog, 302
- m_series
 - RideChartWidget, 271
- m_settings
 - VideoRenderJob, 313
- m_showAltitude
 - MainWindow, 227
- m_showCadence
 - MainWindow, 227
- m_showHR
 - MainWindow, 227
- m_showPower
 - MainWindow, 227
- m_showSpeed
 - MainWindow, 227
- m_sleepSpin
 - EditActivityDialog, 149
- m_smoothedSeriesCache
 - RideChartWidget, 271
- m_sortKey
 - ClimbSortKeyTableItem, 131
- m_speedChart
 - MainWindow, 227
- m_speedCheck
 - VideoExportDialog, 302
- m_speedSpin
 - VideoExportDialog, 302
- m_stageLabel
 - VideoExportDialog, 302
- m_startMarkerRect
 - MapRenderer, 236
- m_startSpin
 - IntervalEditorDialog, 194
- m_startTime
 - anonymous_namespace{FitParser.cpp}::FitSessionListener, 164
- m_statistics
 - WorkoutController, 331
- m_statsLabel
 - AthleteHeaderWidget, 83
- m_statusText
 - ImportProgressModel, 173
- m_subtitleLabel
 - WelcomeWidget, 325
- m_suppressClimbAutoZoom
 - MainWindow, 227
- m_syncing
 - RideChartWidget, 271
- m_table
 - ActivityBrowser, 56
 - AthleteListDialog, 85
- m_tabWidget
 - MainWindow, 228
- m_telemetryPanelCheck
 - VideoExportDialog, 302
- m_temperatureSpin
 - EditActivityDialog, 149
- m_thread
 - AnalysisQueue, 68
 - ImportQueue, 178
 - VideoExportDialog, 302
- m_tileCache
 - MapRenderer, 236
 - VideoTileProvider, 319
- m_tileRepaintTimer
 - MapRenderer, 236
- m_tileStatsLabel
 - VideoExportDialog, 302
- m_timeCheck
 - VideoExportDialog, 302
- m_timeCreated

- anonymous_namespace{FitParser.cpp}::FitFileIdListener, MainWindow, 228
- 154
- m_titleEdit
 - PlannedWorkoutDialog, 241
- m_titleLabel
 - ImportStatusWidget, 183
- m_toolbarImportAct
 - MainWindow, 228
- m_toolbarSearchAct
 - MainWindow, 228
- m_tooltipDisplayValues
 - RideChartWidget, 271
- m_tooltipPoints
 - RideChartWidget, 271
- m_tooltipRawValues
 - RideChartWidget, 271
- m_tsbSeries
 - FitnessChartWidget, 156
- m_tss
 - WorkoutController, 331
- m_tssSpin
 - PlannedWorkoutDialog, 241
- m_typeEdit
 - IntervalEditorDialog, 194
 - PlannedWorkoutDialog, 241
- m_undoManager
 - MainWindow, 228
- m_undoStack
 - UndoManager, 296
- m_useEstimatedFtpButton
 - MainWindow, 228
- m_userMovedMap
 - MapRenderer, 236
- m_vi
 - WorkoutController, 331
- m_videoLengthLabel
 - VideoExportDialog, 302
- m_visibleEndSeconds
 - MapRenderer, 237
- m_visibleStartSeconds
 - MapRenderer, 237
- m_watcher
 - MainWindow, 228
- m_watchFolderEnabled
 - MainWindow, 228
- m_watchFolderPath
 - MainWindow, 228
- m_watchRescanTimer
 - MainWindow, 228
- m_weather
 - EditNotesDialog, 151
- m_weatherEdit
 - EditActivityDialog, 149
- m_weightSpin
 - EditActivityDialog, 149
- m_weightTable
 - AthleteDialog, 81
- m_welcomeWidget
 - m_windEdit
 - EditActivityDialog, 150
 - m_worker
 - AnalysisQueue, 68
 - ImportQueue, 178
 - m_zonesTable
 - MainWindow, 228
 - m_zonesTabStack
 - MainWindow, 228
 - m_zoom
 - MapRenderer, 237
- MainWindow, 201
 - activityCount, 208
 - addRecentDatabase, 208
 - applyChartHeight, 208
 - applyChartPreset, 208
 - applyChartSmoothing, 208
 - applyEstimatedFtpForCurrentAthlete, 208
 - applyWeightMetricsToClimbs, 208
 - assignClimbWeightMetrics, 208
 - backupDatabase, 208
 - buildColorContext, 208
 - buildStatusBar, 208
 - buildToolBar, 209
 - buildUI, 209
 - canImportActivities, 209
 - clearIntervalSummary, 209
 - closeEvent, 209
 - configureFolderWatcher, 209
 - createDatabase, 209
 - createOrOpenDefaultDatabase, 209
 - createVideo, 209
 - currentColorMetric, 210
 - decreaseChartHeight, 210
 - defaultDatabasePathForAutoCreate, 210
 - detectClimbsAndRefresh, 210
 - dragEnterEvent, 210
 - dropEvent, 210
 - editCurrentActivityProperties, 211
 - editSelectedClimbBoundaries, 211
 - ensureAthlete, 211
 - ensureDatabase, 211
 - ensureImportReady, 211
 - estimatedFtpFromCurrentRide, 211
 - eventFilter, 211
 - finalizeImportBatches, 212
 - fitFilesFromMimeData, 212
 - hideWelcomeScreen, 212
 - importActivities, 212
 - importFiles, 212
 - importFilesInternal, 213
 - increaseChartHeight, 214
 - joinSelectedClimbs, 214
 - kAnalysisTabCalendar, 220
 - kAnalysisTabCharts, 220
 - kAnalysisTabClimbing, 220
 - kAnalysisTabFitness, 220

kAnalysisTabHistogram, 220
 kAnalysisTabPDC, 220
 kAnalysisTabZones, 220
 kTabActivities, 220
 kTabAnalysis, 220
 loadSettings, 214
 m_activityBrowser, 220
 m_activityCountLabel, 220
 m_activityNotesView, 220
 m_activityTabStack, 220
 m_altitudeChart, 221
 m_analysisQueue, 221
 m_analysisTabWidget, 221
 m_athleteCombo, 221
 m_athleteHeader, 221
 m_athleteLabelAct, 221
 m_athletesMenu, 221
 m_athleteStatusLabel, 221
 m_autoSmoothingCheck, 221
 m_cadenceChart, 221
 m_calendarTabStack, 221
 m_calendarWidget, 221
 m_chartHeight, 222
 m_chartHeightDecreaseButton, 222
 m_chartHeightIncreaseButton, 222
 m_chartMapSplit, 222
 m_chartPresetCombo, 222
 m_chartsContainer, 222
 m_chartScroll, 222
 m_chartsLayout, 222
 m_climbAltitudeChart, 222
 m_climbCadenceChart, 222
 m_climbDipDistanceSpin, 222
 m_climbDipMetersSpin, 222
 m_climbHrChart, 223
 m_climbingTabStack, 223
 m_climbMinGainSpin, 223
 m_climbMinGradientSpin, 223
 m_climbMinLengthSpin, 223
 m_climbOverlayEnabledCheck, 223
 m_climbOverlayMetricCombo, 223
 m_climbPowerChart, 223
 m_climbQuarterCadenceChart, 223
 m_climbQuarterHrChart, 223
 m_climbQuarterPowerChart, 223
 m_climbSmoothingSpin, 223
 m_climbSpeedChart, 224
 m_climbsTable, 224
 m_climbStartGradientSpin, 224
 m_climbSummaryLabel, 224
 m_colorLegendLayout, 224
 m_colorLegendWidget, 224
 m_colorMetricCombo, 224
 m_controller, 224
 m_currentAthletId, 224
 m_currentAthleteName, 224
 m_dbManager, 224
 m_dbStatusLabel, 224
 m_detectedClimbs, 225
 m_firstLaunchCompleted, 225
 m_fitChartsButton, 225
 m_fitnessChart, 225
 m_fitnessTabStack, 225
 m_followIntervalSelectionCheck, 225
 m_histogram, 225
 m_histogramTabStack, 225
 m_hrChart, 225
 m_importAct, 225
 m_importBatchSummaries, 225
 m_importProgressModel, 225
 m_importQueue, 226
 m_importRefreshTimer, 226
 m_importStatusWidget, 226
 m_intervalsTable, 226
 m_intervalSummaryLabel, 226
 m_knownWatchedFitFiles, 226
 m_lapsTable, 226
 m_lastOpenDir, 226
 m_mapAutoContrastCheck, 226
 m_mapRenderer, 226
 m_mapStyleCombo, 226
 m_pdcWidget, 226
 m_powerChart, 227
 m_powerCurveTabStack, 227
 m_powerSmoothingCombo, 227
 m_previewAct, 227
 m_recentDbMenu, 227
 m_showAltitude, 227
 m_showCadence, 227
 m_showHR, 227
 m_showPower, 227
 m_showSpeed, 227
 m_speedChart, 227
 m_suppressClimbAutoZoom, 227
 m_tabWidget, 228
 m_toolbarImportAct, 228
 m_toolbarSearchAct, 228
 m_undoManager, 228
 m_useEstimatedFtpButton, 228
 m_watcher, 228
 m_watchFolderEnabled, 228
 m_watchFolderPath, 228
 m_watchRescanTimer, 228
 m_welcomeWidget, 228
 m_zonesTable, 228
 m_zonesTabStack, 228
 MainWindow, 207
 manageAthletes, 214
 monitoredDirectories, 214
 navigateToInterval, 214
 onActivityImported, 214
 onAthleteSelectionChanged, 215
 onClimbBoundaryEdited, 215
 onClimbRowDoubleClicked, 215
 onClimbSelectionChanged, 215
 onClimbTableContextMenuRequested, 215

- onIntervalRowDoubleClicked, 215
- onIntervalSelectionChanged, 215
- onNewClimbRequested, 215
- onWatchDirectoryChanged, 215
- onWorkoutLoaded, 215
- openDatabase, 216
- openDatabasePath, 216
- openRecentDatabase, 216
- openSettingsDialog, 216
- previewFitFile, 216
- refreshAthleteSelector, 216
- refreshClimbViews, 216
- removeSelectedClimbs, 216
- resetAllZoom, 216
- resolveActiveActivityWeightKg, 216
- revertSelectedClimbToAuto, 216
- saveSettings, 217
- scanWatchDirectories, 217
- scheduleActivityBrowserRefresh, 217
- selectedClimbIndicesFromTable, 217
- selectIntervalRowOffset, 217
- setupShortcuts, 217
- showAbout, 217
- showWelcomeScreen, 217
- triggerDetectClimbs, 217
- triggerDetectIntervals, 217
- updateAnalysisEmptyStates, 217
- updateAthleteLabel, 217
- updateCharts, 218
- updateClimbingTab, 218
- updateClimbQuarterCharts, 218
- updateClimbRowStyles, 218
- updateColorLegend, 218
- updateFitnessChart, 218
- updateHistogram, 218
- updateImportAvailability, 218
- updateIntervalRowStyles, 219
- updateIntervals, 219
- updateIntervalSummary, 219
- updatePowerCurve, 219
- updateRecentDatabaseMenu, 219
- updateStatsLabel, 219
- updateStatusBarInfo, 219
- updateWelcomeScreenVisibility, 219
- updateZoneAvailability, 219
- updateZonesTab, 219
- MainWindow::ImportBatchSummary, 167
 - duplicates, 168
 - errors, 168
 - failed, 168
 - imported, 168
 - queued, 168
 - showResultDialog, 168
 - sourceLabel, 168
- makeDefaultFilename
 - anonymous_namespace{VideoExportDialog.cpp}, 32
- manageAthletes
 - MainWindow, 214
- MapFitMath::FitResult, 161
 - centerLat, 162
 - centerLon, 162
 - lonSpanDegrees, 162
 - maxLat, 162
 - maxLon, 162
 - minLat, 162
 - minLon, 163
 - zoom, 163
- MapRenderer, 229
 - adjustRouteColorForStyle, 231
 - autoRouteContrast, 231
 - DragHandle, 231
 - fitToBounds, 231
 - fitToTrack, 232
 - fitToVisibleRange, 232
 - gpsRecordAtTime, 232
 - latLonToTile, 232
 - m_autoFitVisibleRange, 234
 - m_autoRouteContrast, 234
 - m_centerLat, 234
 - m_centerLon, 234
 - m_colorContext, 235
 - m_currentTimeSeconds, 235
 - m_draggingSelection, 235
 - m_dragHandle, 235
 - m_endMarkerRect, 235
 - m_firstGpsRecord, 235
 - m_gpsRecords, 235
 - m_hasGps, 235
 - m_highlightElapsedSeconds, 235
 - m_lastGpsRecord, 235
 - m_maxLat, 235
 - m_maxLon, 235
 - m_minLat, 236
 - m_minLon, 236
 - m_minZoom, 236
 - m_panning, 236
 - m_panStart, 236
 - m RideData, 236
 - m_routeColorMode, 236
 - m_segmentColors, 236
 - m_startMarkerRect, 236
 - m_tileCache, 236
 - m_tileRepaintTimer, 236
 - m_userMovedMap, 236
 - m_visibleEndSeconds, 237
 - m_visibleStartSeconds, 237
 - m_zoom, 237
 - MapRenderer, 231
 - mapStyle, 232
 - mouseDoubleClickEvent, 232
 - mouseMoveEvent, 232
 - mousePressEvent, 232
 - mouseReleaseEvent, 232
 - nearestGpsRecord, 233
 - paintEvent, 233

- rebuildGpsCache, [233](#)
- rebuildSegmentColors, [233](#)
- setAutoRouteContrast, [233](#)
- setCurrentTime, [233](#)
- setHighlightedElapsedSeconds, [233](#)
- setMapStyle, [233](#)
- setRideData, [233](#)
- setRouteColorMode, [233](#)
- setTileCacheSize, [234](#)
- setVisibleTimeRange, [234](#)
- tileToScreen, [234](#)
- wheelEvent, [234](#)
- zoomIn, [234](#)
- zoomOut, [234](#)
- mapStyle
 - MapRenderer, [232](#)
 - TileCache, [281](#)
 - VideoExportSettings, [305](#)
- mapStyleName
 - anonymous_namespace{VideoExportDialog.cpp}, [32](#)
 - TileCache, [281](#)
- maxDipDistanceMeters
 - ClimbDetectorConfig, [109](#)
- maxDipMeters
 - ClimbDetectorConfig, [109](#)
- maxDipPctOfGain
 - ClimbDetectorConfig, [109](#)
- maxHr
 - Activity, [46](#)
- maximumCadence
 - WorkoutStatistics, [333](#)
- maximumGradient
 - Climb, [98](#)
- maximumHeartRate
 - WorkoutStatistics, [333](#)
- maximumPower
 - Interval, [188](#)
 - WorkoutStatistics, [333](#)
- maximumSpeed
 - WorkoutStatistics, [333](#)
- maxLat
 - MapFitMath::FitResult, [162](#)
- maxLon
 - MapFitMath::FitResult, [162](#)
- maxPlateauDistanceMeters
 - ClimbDetectorConfig, [109](#)
- maxPower
 - Activity, [46](#)
- maxSmallDipsCount
 - ClimbDetectorConfig, [110](#)
- maxStartInterruptionMeters
 - ClimbDetectorConfig, [110](#)
- maxTilesInMemory
 - TileCacheConfig, [283](#)
- maxValue
 - Zone, [334](#)
- maxZoom
 - TileCache, [281](#)
 - TileProvider, [284](#)
- memoryCacheTiles
 - VideoTileProviderConfig, [320](#)
- mergeFragmentedClimbs
 - ClimbDetector, [105](#)
- mergeMaxGapElevationM
 - ClimbDetectorConfig, [110](#)
- mergeMaxGapMeters
 - ClimbDetectorConfig, [110](#)
- minAverageGradient
 - ClimbDetectorConfig, [110](#)
- minClimbScore
 - ClimbDetectorConfig, [110](#)
- minElevationGainM
 - ClimbDetectorConfig, [110](#)
- minLat
 - MapFitMath::FitResult, [162](#)
- minLengthKm
 - ClimbDetectorConfig, [110](#)
- minLon
 - MapFitMath::FitResult, [163](#)
- minRecoverySeconds
 - IntervalDetectorConfig, [191](#)
- minStartContinuousDistanceMeters
 - ClimbDetectorConfig, [111](#)
- minValue
 - Zone, [334](#)
- minWorkSeconds
 - IntervalDetectorConfig, [191](#)
- monitoredDirectories
 - MainWindow, [214](#)
- mouseDoubleClickEvent
 - MapRenderer, [232](#)
 - RideChartWidget, [260](#)
- mouseMoveEvent
 - MapRenderer, [232](#)
 - RideChartWidget, [260](#)
- mousePressEvent
 - MapRenderer, [232](#)
 - RideChartWidget, [260](#)
- mouseReleaseEvent
 - MapRenderer, [232](#)
 - RideChartWidget, [261](#)
- movingTimeSeconds
 - WorkoutStatistics, [334](#)
- name
 - Climb, [98](#)
 - ClimbRecord, [121](#)
 - IntervalRecord, [196](#)
 - PowerZone, [251](#)
 - TileProvider, [284](#)
 - Zone, [335](#)
- navigateToInterval
 - MainWindow, [214](#)
- NavigationSidebar, [237](#)
 - currentPage, [239](#)
 - m_list, [239](#)

- NavigationSidebar, 238
- Page, 238
- setCurrentPage, 239
- nearestGpsRecord
 - MapRenderer, 233
- nearestRecordIndex
 - anonymous_namespace{VideoRenderJob.cpp}, 34
- newAthlete
 - AthleteListDialog, 84
- newClimbRequested
 - RideChartWidget, 261
- newDurationSeconds
 - DuplicateActivityInfo, 147
- newStartUtc
 - DuplicateActivityInfo, 147
- None
 - ActivityAnalysisFlags, 24
- normalizedDatabaseName
 - CreateDatabaseDialog, 136
- normalizedPower
 - Activity, 46
 - Climb, 98
 - Interval, 188
 - TrainingLoad, 287
 - TrainingMetricsSummary, 290
 - WorkoutController, 329
- normalizeLongitude
 - anonymous_namespace{MapFitMath.cpp}, 30
- notes
 - Activity, 46
 - ClimbRecord, 121
 - EditNotesDialog, 151
 - FtpEntry, 165
 - IntervalRecord, 196
 - PlannedWorkoutDraft, 242
- notify
 - UndoManager, 295
- np
 - ClimbRecord, 121
 - IntervalRecord, 196
- number
 - PowerZone, 251
- nX
 - anonymous_namespace{VideoRenderJob.cpp}::TileViewpoint, 285
- nY
 - anonymous_namespace{VideoRenderJob.cpp}::TileViewpoint, 285
- onAccepted
 - AthleteDialog, 79
- onActivityImported
 - MainWindow, 214
- onAthleteSelectionChanged
 - MainWindow, 215
- onClimbBoundaryEdited
 - MainWindow, 215
- onClimbRowDoubleClicked
 - MainWindow, 215
- onClimbSelectionChanged
 - MainWindow, 215
- onClimbTableContextMenuRequested
 - MainWindow, 215
- onDateChanged
 - CalendarWidget, 92
- onDiskTiles
 - VideoTileStats, 321
- onFinished
 - VideoExportDialog, 299
- onFrameProgressChanged
 - VideoExportDialog, 299
- onIntervalRowDoubleClicked
 - MainWindow, 215
- onIntervalSelectionChanged
 - MainWindow, 215
- OnMsg
 - anonymous_namespace{FitParser.cpp}::FitFileIdListener, 154
 - anonymous_namespace{FitParser.cpp}::FitSessionListener, 164
 - FitRecordListener, 161
- onNewClimbRequested
 - MainWindow, 215
- onProgressChanged
 - VideoExportDialog, 299
- onReplyFinished
 - TileCache, 281
- onStageChanged
 - VideoExportDialog, 299
- onTaskCompleted
 - AnalysisQueue, 66
- onTaskFailed
 - AnalysisQueue, 66
- onTileStatsChanged
 - VideoExportDialog, 299
- onWatchDirectoryChanged
 - MainWindow, 215
- onWorkerCompleted
 - ImportQueue, 177
- onWorkerFailed
 - ImportQueue, 177
- onWorkerProgress
 - ImportQueue, 177
- onWorkoutLoaded
 - MainWindow, 215
- open
 - DatabaseManager, 141
- openDatabase
 - MainWindow, 216
- openDatabasePath
 - MainWindow, 216
- openRecentDatabase
 - MainWindow, 216
- openSettingsDialog
 - MainWindow, 216
- operator<

- anonymous_namespace(ActivityBrowser.cpp)::ActivityDateTableItem, 241
- 57
- ClimbNameTableItem, 117
- ClimbSortKeyTableItem, 131
- operator()
 - VideoTileIdHash, 314
- operator==
 - anonymous_namespace(VideoRenderJob.cpp)::TileVideoReport, 241
 - 285
 - VideoTileId, 314
- originalEndSeconds
 - ClimbRecord, 121
- originalStartSeconds
 - ClimbRecord, 121
- outputPath
 - VideoExportSettings, 305
- overlayAltitude
 - VideoExportSettings, 306
- overlayAthleteName
 - VideoExportSettings, 306
- overlayCadence
 - VideoExportSettings, 306
- overlayCharts
 - VideoExportSettings, 306
- overlayHeartRate
 - VideoExportSettings, 306
- overlayPower
 - VideoExportSettings, 306
- overlayRouteLegend
 - VideoExportSettings, 306
- overlaySpeed
 - VideoExportSettings, 306
- overlayTelemetryPanel
 - VideoExportSettings, 307
- overlayTime
 - VideoExportSettings, 307
- Page
 - NavigationSidebar, 238
- paintEvent
 - MapRenderer, 233
- parseDate
 - DateFormatter, 145
- Parsing
 - ImportJob, 169
- path
 - DatabaseManager, 142
- pdccPoints
 - WorkoutController, 329
- pendingCount
 - AnalysisQueue, 66
 - ImportQueue, 177
- pendingCountChanged
 - AnalysisQueue, 67
- percent
 - ZoneDistribution, 341
- PlannedWorkoutDialog, 239
 - draft, 240
 - m_dateEdit, 241
 - m_deleteSpin, 241
 - m_id, 241
 - m_notesEdit, 241
 - m_titleEdit, 241
 - m_tssSpin, 241
 - m_typeEdit, 241
 - PlannedWorkoutDialog, 240
 - PlannedWorkoutDraft, 241
 - date, 242
 - durationSeconds, 242
 - id, 242
 - notes, 242
 - targetTss, 242
 - title, 242
 - type, 242
- plateauMinGradient
 - ClimbDetectorConfig, 111
- Platform, 36
 - applicationDataDirectory, 37
 - defaultBackupPath, 37
 - defaultDatabaseDirectory, 37
 - defaultDatabasePath, 37
- playbackSpeed
 - VideoExportSettings, 307
- populateFtpTable
 - AthleteDialog, 80
- populateWeightTable
 - AthleteDialog, 80
- power
 - RideRecord, 276
- PowerCurve, 242
 - bestMeanPower, 243
 - computeStandard, 243
 - standardDurations, 244
- PowerCurvePoint, 244
 - bestPower, 245
 - durationSeconds, 245
- PowerCurveWidget, 245
 - clearChart, 246
 - PowerCurveWidget, 246
 - setRideData, 247
 - setRideDataWithComparisons, 247
- powerFadePct
 - Climb, 98
- powerFirstHalf
 - Climb, 98
- PowerHistogramWidget, 248
 - clearChart, 249
 - PowerHistogramWidget, 249
 - setRideData, 250
- powerSecondHalf
 - Climb, 98
- PowerZone, 250
 - lowerPct, 251
 - name, 251
 - number, 251
 - seconds, 251
 - upperPct, 251

- powerZones
 - WorkoutController, 329
 - ZoneCalculator, 338
- previewFitFile
 - MainWindow, 216
- processJob
 - ImportWorker, 185
- processNext
 - AnalysisQueue, 67
 - ImportQueue, 178
- processTask
 - AnalysisWorker, 70
- progress
 - ImportJob, 170
 - ImportProgressModel, 173
- progressChanged
 - VideoRenderJob, 310
- providerForStyle
 - TileCache, 281
- push
 - UndoManager, 295
- qtDatePattern
 - DateFormatter, 145
- quarterMetrics
 - Climb, 98
- Queued
 - ImportJob, 169
- queued
 - MainWindow::ImportBatchSummary, 168
- rating
 - ClimbRecord, 121
- reanalyze
 - WorkoutController, 329
- rebuildChart
 - RideChartWidget, 262
- rebuildGpsCache
 - MapRenderer, 233
- rebuildSegmentColors
 - MapRenderer, 233
- recolorMonth
 - CalendarWidget, 92
- recordFromQuery
 - anonymous_namespace{ClimbRepository.cpp}, 28
- records
 - RideData, 272
- redo
 - ClimbEditCommand, 113
 - UndoCommand, 292
 - UndoManager, 295
- redoDescription
 - UndoManager, 295
- refresh
 - ActivityBrowser, 55
 - AthleteListDialog, 84
 - CalendarWidget, 92
- refreshAthleteSelector
 - MainWindow, 216
- refreshClimbViews
 - MainWindow, 216
- reloadClimbsFromDatabase
 - WorkoutController, 329
- reloadDayList
 - CalendarWidget, 92
- removeOverlappingAndDuplicates
 - IntervalDetector, 190
- removeSelectedClimbs
 - MainWindow, 216
- requiredTileCount
 - VideoTileProvider, 318
- requiredTiles
 - VideoTileStats, 321
- requiredTilesAvailableOnDisk
 - VideoTileProvider, 318
- requireGradePersistence
 - ClimbDetectorConfig, 111
- resetAllZoom
 - MainWindow, 216
- resetZoom
 - RideChartWidget, 262
- resizeEvent
 - RideChartWidget, 262
- resolveActiveActivityWeightKg
 - MainWindow, 216
- resolveAthleteFtpForDate
 - anonymous_namespace{WorkoutController.cpp}, 35
- revertSelectedClimbToAuto
 - MainWindow, 216
- RideChartWidget, 251
 - addClimbBackgroundBands, 256
 - addIntervalBackgroundBands, 256
 - addMetricOverlayBands, 256
 - addZoneBackgroundBands, 256
 - buildSampleIndices, 256
 - cachedAxisMax, 267
 - cachedAxisMin, 267
 - cachedMaxX, 267
 - cachedMinX, 267
 - clearBackgroundSeries, 256
 - clearChart, 256
 - climbBoundaryEdited, 257
 - climbClicked, 257
 - ClimbHandleDrag, 255
 - computeSmoothedPowerSeries, 257
 - computeSmoothedSeries, 258
 - crosshairMoved, 258
 - drawForeground, 258
 - fitToData, 258
 - hasData, 259
 - intervalSelectionRequested, 259
 - leaveEvent, 259
 - m_autoSmoothing, 267
 - m_axisX, 267
 - m_axisXDisplay, 267
 - m_axisY, 268

- m_axisYOverlay, 268
- m_backgroundSeries, 268
- m_climbEditingEnabled, 268
- m_climbHandleDrag, 268
- m_climbs, 268
- m_colorContext, 268
- m_colorMetric, 268
- m_colorSeries, 268
- m_crosshairX, 268
- m_crosshairY, 268
- m_dataMaxY, 268
- m_dataMinY, 269
- m_dragOriginalClimbEndSec, 269
- m_dragOriginalClimbStartSec, 269
- m_effectivePowerSmoothingSeconds, 269
- m_hasData, 269
- m_intervalEndMin, 269
- m_intervals, 269
- m_intervalSelecting, 269
- m_intervalStartMin, 269
- m_isRebuilding, 269
- m_metric, 269
- m_metricOverlayEnabled, 269
- m_metricOverlayMetric, 270
- m_newClimbCreating, 270
- m_newClimbCurrentSec, 270
- m_newClimbDragging, 270
- m_newClimbStartSec, 270
- m_origMaxX, 270
- m_overlaySeries, 270
- m_rawSeries, 270
- m_referenceSeries, 270
- m_requestedPowerSmoothingSeconds, 270
- m_rideData, 270
- m_selectedClimbIndex, 270
- m_series, 271
- m_smoothedSeriesCache, 271
- m_syncing, 271
- m_tooltipDisplayValues, 271
- m_tooltipPoints, 271
- m_tooltipRawValues, 271
- mouseDoubleClickEvent, 260
- mouseMoveEvent, 260
- mousePressEvent, 260
- mouseReleaseEvent, 261
- newClimbRequested, 261
- rebuildChart, 262
- resetZoom, 262
- resizeEvent, 262
- RideChartWidget, 255
- setAutoSmoothingEnabled, 262
- setClimbEditingEnabled, 263
- setClimbs, 263
- setCrosshair, 263
- setIntervals, 263
- setMetricOverlay, 264
- setPowerSmoothingSeconds, 264
- setRideData, 264
- setSelectedClimbIndex, 264
- setXRange, 264
- updateXTicks, 265
- updateYAxis, 265
- updateYAxisForVisibleRange, 265
- valid, 271
- visibleRangeChanged, 265
- visibleRangeEndMinutes, 266
- visibleRangeStartMinutes, 266
- wheelEvent, 266
- xRangeChanged, 266
- RideData, 271
 - activityEndTimeUtc, 272
 - activityStartTimeUtc, 272
 - fileTimestampUtc, 272
 - records, 272
- rideData
 - VideoExportSettings, 307
 - WorkoutController, 329
- RideDataLoader, 272
 - loadActivitySamples, 273
- RideDataSerializer, 273
 - loadRideFromDatabase, 273
 - saveRideToDatabase, 273
- RideRecord, 274
 - altitude, 274
 - cadence, 274
 - elapsedSeconds, 275
 - hasAltitude, 275
 - hasCadence, 275
 - hasGps, 275
 - hasHeartRate, 275
 - hasPower, 275
 - hasSpeed, 275
 - heartRate, 276
 - latitude, 276
 - longitude, 276
 - power, 276
 - speed, 276
- routeColor
 - anonymous_namespace{VideoRenderJob.cpp}, 34
- routeColorMetric
 - VideoExportSettings, 307
- rpe
 - Activity, 46
- run
 - VideoRenderJob, 311
- sampleAvailable
 - anonymous_namespace{ZoneCalculator.cpp}, 36
- sampleDurationSeconds
 - anonymous_namespace{ZoneCalculator.cpp}, 36
- sanitizeDateFormat
 - AppSettings, 74
- savedAthleteld
 - AthleteDialog, 80
- saveRideToDatabase
 - RideDataSerializer, 273
- saveSettings

- MainWindow, 217
- scanWatchDirectories
 - MainWindow, 217
- scheduleActivityBrowserRefresh
 - MainWindow, 217
- seconds
 - PowerZone, 251
 - ZoneDistribution, 341
- segmentEndSeconds
 - VideoExportSettings, 307
- segmentMeters
 - anonymous_namespace{ClimbDetector.cpp}, 27
- segmentStartSeconds
 - VideoExportSettings, 307
- selectAthlete
 - AthleteListDialog, 84
- selectedAthleteld
 - AthleteListDialog, 85
- selectedClimbIndicesFromTable
 - MainWindow, 217
- selectExistsEver
 - SqlQueryHelper, 38
- selectExistsLocked
 - SqlQueryHelper, 39
- selectExistsNonDeleted
 - SqlQueryHelper, 39
- selectIntervalRowOffset
 - MainWindow, 217
- setAllOverlayChecks
 - VideoExportDialog, 299
- setAnalysisQueue
 - WorkoutController, 329
- setAthleteld
 - CalendarWidget, 93
- setAutoRouteContrast
 - MapRenderer, 233
- setAutoSmoothingEnabled
 - RideChartWidget, 262
- setCache
 - ActivityAnalysisCacheRepository, 50
- setClimbEditingEnabled
 - RideChartWidget, 263
- setClimbs
 - RideChartWidget, 263
- setCrosshair
 - RideChartWidget, 263
- setCurrentAthlete
 - WorkoutController, 330
- setCurrentPage
 - NavigationSidebar, 239
- setCurrentTime
 - MapRenderer, 233
- setDatabaseManager
 - CalendarWidget, 93
 - WorkoutController, 330
- setDatabasePath
 - AnalysisQueue, 67
 - AnalysisWorker, 71
 - ImportQueue, 178
 - ImportWorker, 186
- setDateFormat
 - AppSettings, 74
- setFirstLaunch
 - WelcomeWidget, 324
- setFTP
 - WorkoutController, 330
- setHighlightedElapsedSeconds
 - MapRenderer, 233
- setImportSource
 - ActivityRepository, 61
- setIntervals
 - RideChartWidget, 263
- setMapStyle
 - MapRenderer, 233
 - TileCache, 281
- setMemoryCacheSize
 - TileCache, 281
- setMetricOverlay
 - RideChartWidget, 264
- setModel
 - ImportStatusWidget, 183
- setPowerSmoothingSeconds
 - RideChartWidget, 264
- setRequiredTiles
 - VideoTileProvider, 318
- setRideData
 - MapRenderer, 233
 - PowerCurveWidget, 247
 - PowerHistogramWidget, 250
 - RideChartWidget, 264
- setRideDataWithComparisons
 - PowerCurveWidget, 247
- setRideMetrics
 - AthleteHeaderWidget, 82
- setRouteColorMode
 - MapRenderer, 233
- setSelectedClimbIndex
 - RideChartWidget, 264
- setSummary
 - AthleteHeaderWidget, 82
- setTileCacheSize
 - AppSettings, 74
 - MapRenderer, 234
- setTimeline
 - FitnessChartWidget, 156
- settingsForActivity
 - ActivityAnalysisSettingsRepository, 53
- setupShortcuts
 - MainWindow, 217
- setVisibleTimeRange
 - MapRenderer, 234
- setXRange
 - RideChartWidget, 264
- shapeClass
 - Climb, 98
- shouldCreateDefaultAthlete

- CreateDatabaseDialog, 136
- shouldDeleteExportCacheAfterExport
 - VideoTileProvider, 319
- showAbout
 - MainWindow, 217
- showContextMenu
 - ActivityBrowser, 56
- showResultDialog
 - MainWindow::ImportBatchSummary, 168
- showWelcomeScreen
 - MainWindow, 217
- singleRouteColor
 - VideoExportSettings, 307
- sleepHours
 - Activity, 47
- smallDipThresholdMeters
 - ClimbDetectorConfig, 111
- smoothAltitudeByDistance
 - ClimbDetector, 105
- SmoothingEngine, 277
 - smoothPower, 277
- smoothingWindow
 - IntervalDetectorConfig, 191
- smoothPower
 - IntervalDetector, 190
 - SmoothingEngine, 277
- softDeleteClimb
 - ClimbRepository, 128
- softDeleteInterval
 - IntervalRepository, 200
- source
 - ClimbRecord, 122
 - IntervalRecord, 197
- sourceLabel
 - MainWindow::ImportBatchSummary, 168
- speed
 - RideRecord, 276
- speedZones
 - ZoneCalculator, 338
- sport
 - Activity, 47
- SqlQueryHelper, 37
 - bindBoolValue, 38
 - kWhereDeleted, 40
 - kWhereLocked, 40
 - kWhereNotDeleted, 40
 - selectExistsEver, 38
 - selectExistsLocked, 39
 - selectExistsNonDeleted, 39
- src/analysis/ClimbDetector.cpp, 343
- src/analysis/ClimbDetector.h, 351
- src/analysis/ClimbMetrics.cpp, 352
- src/analysis/ClimbMetrics.h, 358
- src/analysis/FTPZones.cpp, 359
- src/analysis/FTPZones.h, 360
- src/analysis/IntervalDetector.cpp, 360
- src/analysis/IntervalDetector.h, 363
- src/analysis/PowerCurve.cpp, 364
- src/analysis/PowerCurve.h, 364
- src/analysis/queue/AnalysisQueue.cpp, 365
- src/analysis/queue/AnalysisQueue.h, 366
- src/analysis/queue/AnalysisWorker.cpp, 366
- src/analysis/queue/AnalysisWorker.h, 369
- src/analysis/SmoothingEngine.cpp, 370
- src/analysis/SmoothingEngine.h, 371
- src/analysis/TrainingLoad.cpp, 371
- src/analysis/TrainingLoad.h, 372
- src/analysis/TrainingMetrics.cpp, 372
- src/analysis/TrainingMetrics.h, 373
- src/analysis/WorkoutAnalyzer.cpp, 374
- src/analysis/WorkoutAnalyzer.h, 375
- src/charts/FitnessChartWidget.cpp, 375
- src/charts/FitnessChartWidget.h, 377
- src/charts/Metric.h, 377
- src/charts/PowerCurveWidget.cpp, 377
- src/charts/PowerCurveWidget.h, 380
- src/charts/PowerHistogramWidget.cpp, 380
- src/charts/PowerHistogramWidget.h, 382
- src/charts/RideChartWidget.cpp, 382
- src/charts/RideChartWidget.h, 406
- src/core/ActivityAnalysisFlags.h, 408
- src/core/AnalysisVersions.h, 408
- src/core/settings/AppSettings.cpp, 408
- src/core/settings/AppSettings.h, 409
- src/core/settings/DateFormatter.cpp, 410
- src/core/settings/DateFormatter.h, 411
- src/core/undo/ClimbEditCommand.cpp, 411
- src/core/undo/ClimbEditCommand.h, 412
- src/core/undo/UndoCommand.h, 412
- src/core/undo/UndoManager.cpp, 412
- src/core/undo/UndoManager.h, 413
- src/core/zones/ColorProvider.cpp, 414
- src/core/zones/ColorProvider.h, 415
- src/core/zones/ZoneCalculator.cpp, 416
- src/core/zones/ZoneCalculator.h, 418
- src/core/zones/ZoneDefinition.cpp, 419
- src/core/zones/ZoneDefinition.h, 419
- src/database/ActivityAnalysisCacheRepository.cpp, 420
- src/database/ActivityAnalysisCacheRepository.h, 421
- src/database/ActivityAnalysisSettingsRepository.cpp, 421
- src/database/ActivityAnalysisSettingsRepository.h, 422
- src/database/ActivityRepository.cpp, 422
- src/database/ActivityRepository.h, 426
- src/database/AthleteRepository.cpp, 427
- src/database/AthleteRepository.h, 430
- src/database/ClimbRepository.cpp, 431
- src/database/ClimbRepository.h, 434
- src/database/DatabaseConnectionGuard.cpp, 435
- src/database/DatabaseConnectionGuard.h, 435
- src/database/DatabaseManager.cpp, 436
- src/database/DatabaseManager.h, 441
- src/database/DatabaseSchema.cpp, 441
- src/database/DatabaseSchema.h, 442
- src/database/ImportRepository.cpp, 445
- src/database/ImportRepository.h, 446

- src/database/IntervalRepository.cpp, 446
- src/database/IntervalRepository.h, 448
- src/database/SqlQueryHelper.h, 449
- src/fit/FitParser.cpp, 450
- src/fit/FitParser.h, 452
- src/fit/FitRecordListener.cpp, 452
- src/fit/FitRecordListener.h, 454
- src/fit/RideData.h, 454
- src/fitlyzerC.cpp, 455
- src/gui/AboutDialog.cpp, 455
- src/gui/AboutDialog.h, 456
- src/gui/ActivityBrowser.cpp, 456
- src/gui/ActivityBrowser.h, 461
- src/gui/AthleteDialog.cpp, 461
- src/gui/AthleteDialog.h, 465
- src/gui/AthleteHeaderWidget.cpp, 465
- src/gui/AthleteHeaderWidget.h, 466
- src/gui/AthleteListDialog.cpp, 466
- src/gui/AthleteListDialog.h, 468
- src/gui/CalendarWidget.cpp, 469
- src/gui/CalendarWidget.h, 472
- src/gui/CreateDatabaseDialog.cpp, 473
- src/gui/CreateDatabaseDialog.h, 475
- src/gui/EditActivityDialog.cpp, 475
- src/gui/EditActivityDialog.h, 477
- src/gui/EditNotesDialog.cpp, 477
- src/gui/EditNotesDialog.h, 478
- src/gui/ImportStatusWidget.cpp, 478
- src/gui/ImportStatusWidget.h, 479
- src/gui/IntervalEditorDialog.cpp, 479
- src/gui/IntervalEditorDialog.h, 480
- src/gui/MainWindow.cpp, 481
- src/gui/MainWindow.h, 538
- src/gui/NavigationSidebar.cpp, 542
- src/gui/NavigationSidebar.h, 542
- src/gui/PlannedWorkoutDialog.cpp, 543
- src/gui/PlannedWorkoutDialog.h, 544
- src/gui/WelcomeWidget.cpp, 545
- src/gui/WelcomeWidget.h, 546
- src/gui/WorkoutController.cpp, 546
- src/gui/WorkoutController.h, 552
- src/import/ImportJob.cpp, 554
- src/import/ImportJob.h, 554
- src/import/ImportProgressModel.cpp, 554
- src/import/ImportProgressModel.h, 555
- src/import/ImportQueue.cpp, 556
- src/import/ImportQueue.h, 557
- src/import/ImportWorker.cpp, 558
- src/import/ImportWorker.h, 562
- src/maps/MapFitMath.cpp, 562
- src/maps/MapFitMath.h, 564
- src/maps/MapRenderer.cpp, 564
- src/maps/MapRenderer.h, 574
- src/maps/TileCache.cpp, 576
- src/maps/TileCache.h, 581
- src/model/RideDataLoader.cpp, 582
- src/model/RideDataLoader.h, 583
- src/model/RideDataSerializer.cpp, 583

- src/model/RideDataSerializer.h, 585
- src/platform/Platform.cpp, 586
- src/platform/Platform.h, 586
- src/platform/Platform_linux.cpp, 586
- src/platform/Platform_macos.cpp, 587
- src/platform/Platform_windows.cpp, 587
- src/utils/FfmpegPath.cpp, 587
- src/utils/FfmpegPath.h, 588
- src/utils/UuidGenerator.h, 588
- src/video/VideoExportDialog.cpp, 589
- src/video/VideoExportDialog.h, 594
- src/video/VideoExportSettings.h, 595
- src/video/VideoRenderJob.cpp, 596
- src/video/VideoRenderJob.h, 608
- src/video/VideoTileProvider.cpp, 608
- src/video/VideoTileProvider.h, 610
- stageChanged
 - VideoRenderJob, 312
- standardDurations
 - PowerCurve, 244
- startDistanceKm
 - Climb, 99
- startElevationM
 - Climb, 99
- startExport
 - VideoExportDialog, 300
- startGradient
 - ClimbDetectorConfig, 111
- startHandle
 - anonymous_namespace{RideChartWidget.cpp}::ClimbHandleHitRes, 114
- startSample
 - IntervalRecord, 197
- startSeconds
 - Climb, 99
 - ClimbRecord, 122
 - Interval, 188
- startTime
 - Activity, 47
 - anonymous_namespace{FitParser.cpp}::FitSessionListener, 164
- State
 - ImportJob, 169
- state
 - ImportJob, 170
- statistics
 - WorkoutController, 330
- stats
 - VideoTileProvider, 319
- status
 - ImportRecord, 180
- statusText
 - ImportProgressModel, 173
- Storing
 - ImportJob, 169
- style
 - VideoTileProviderConfig, 320
- styleDisplayName

- TileCache, 281
- styleKey
 - TileCache, 281
- targetTss
 - PlannedWorkoutDraft, 242
- taskCompleted
 - AnalysisQueue, 67
 - AnalysisWorker, 71
- taskFailed
 - AnalysisQueue, 68
 - AnalysisWorker, 72
- temperature
 - Activity, 47
- thresholdHeartRate
 - ColorContext, 133
- tile
 - TileCache, 282
- tileBlocking
 - TileCache, 282
- TileCache, 278
 - attribution, 280
 - clearMemoryCache, 280
 - diskCacheRoot, 280
 - diskTilePath, 280
 - diskTilePathForRoot, 280
 - dispatchNextDownloads, 280
 - isTileCachedOnDisk, 280
 - key, 280
 - keyForStyle, 281
 - kMaxConcurrentDownloads, 282
 - m_activeDownloads, 282
 - m_cache, 282
 - m_diskCacheRoot, 282
 - m_diskFallbackRoots, 282
 - m_downloadQueue, 282
 - m_mapStyle, 282
 - m_nam, 282
 - m_pending, 283
 - m_provider, 283
 - mapStyle, 281
 - mapStyleName, 281
 - maxZoom, 281
 - onReplyFinished, 281
 - providerForStyle, 281
 - setMapStyle, 281
 - setMemoryCacheSize, 281
 - styleDisplayName, 281
 - styleKey, 281
 - tile, 282
 - tileBlocking, 282
 - TileCache, 279
- TileCache::TileRequest, 284
 - x, 284
 - y, 284
 - z, 284
- TileCacheConfig, 283
 - diskCacheRoot, 283
 - diskFallbackRoots, 283
 - maxTilesInMemory, 283
- tileCacheSize
 - AppSettings, 74
- tileCacheSizeKey
 - AppSettings, 74
- TileProvider, 283
 - attribution, 284
 - maxZoom, 284
 - name, 284
 - urlTemplate, 284
- tilesForViewport
 - anonymous_namespace{VideoRenderJob.cpp}, 34
- tileStatsChanged
 - VideoRenderJob, 312
- tileToScreen
 - anonymous_namespace{VideoRenderJob.cpp}, 34
 - MapRenderer, 234
- tileX0
 - anonymous_namespace{VideoRenderJob.cpp}::TileViewport, 285
- tileY0
 - anonymous_namespace{VideoRenderJob.cpp}::TileViewport, 285
- tileYToLat
 - anonymous_namespace{MapFitMath.cpp}, 30
- timeCreated
 - anonymous_namespace{FitParser.cpp}::FitFileIdListener, 154
- title
 - PlannedWorkoutDraft, 242
- tolsoDate
 - DateFormatter, 146
- tolsoDateTime
 - DateFormatter, 146
- tolsoOrRaw
 - anonymous_namespace{AthleteDialog.cpp}, 26
- totalDistanceKm
 - WorkoutStatistics, 334
- trailingNumber
 - ClimbNameTableItem, 117
- TrainingLoad, 286
 - fitnessTimeline, 286
 - intensityFactor, 286
 - normalizedPower, 287
 - trainingStressScore, 287
- TrainingMetrics, 288
 - compute, 289
- TrainingMetricsSummary, 289
 - averageHeartRate, 290
 - averagePower, 290
 - efficiencyFactor, 290
 - intensityFactor, 290
 - normalizedPower, 290
 - trainingStressScore, 290
 - variabilityIndex, 291
- trainingStressScore
 - TrainingLoad, 287
 - TrainingMetricsSummary, 290

- WorkoutController, 330
- triggerDetectClimbs
 - MainWindow, 217
- triggerDetectIntervals
 - MainWindow, 217
- tsb
 - FitnessMetricsPoint, 157
- tss
 - DailyLoadPoint, 138
- type
 - IntervalRecord, 197
 - PlannedWorkoutDraft, 242
- undo
 - ClimbEditCommand, 113
 - UndoCommand, 292
 - UndoManager, 295
- UndoCommand, 291
 - description, 291
 - redo, 292
 - undo, 292
- undoDescription
 - UndoManager, 295
- UndoManager, 292
 - canRedo, 294
 - canUndo, 294
 - clear, 294
 - m_maxStackSize, 295
 - m_redoStack, 295
 - m_undoStack, 296
 - notify, 295
 - push, 295
 - redo, 295
 - redoDescription, 295
 - undo, 295
 - undoDescription, 295
 - UndoManager, 294
- updateActivityProperties
 - ActivityRepository, 61
- updateAnalysisEmptyStates
 - MainWindow, 217
- updateAnalysisFlags
 - ActivityRepository, 62
- updateAthlete
 - AthleteRepository, 90
- updateAthleteFields
 - CreateDatabaseDialog, 136
- updateAthleteLabel
 - MainWindow, 217
- updateCharts
 - MainWindow, 218
- updateClimb
 - ClimbRepository, 128
- updateClimbingTab
 - MainWindow, 218
- updateClimbQuarterCharts
 - MainWindow, 218
- updateClimbRowStyles
 - MainWindow, 218
- updateColorLegend
 - MainWindow, 218
- updatedActivity
 - EditActivityDialog, 149
- updateFingerprint
 - ActivityRepository, 62
- updateFitnessChart
 - MainWindow, 218
- updateFromModel
 - ImportStatusWidget, 183
- updateHistogram
 - MainWindow, 218
- updateImportAvailability
 - MainWindow, 218
- updateInterval
 - IntervalRepository, 200
- updateIntervalRowStyles
 - MainWindow, 219
- updateIntervals
 - MainWindow, 219
- updateIntervalSummary
 - MainWindow, 219
- updateMonthFormatting
 - CalendarWidget, 93
- updateNotes
 - ActivityRepository, 63
- updatePowerCurve
 - MainWindow, 219
- updateRecentDatabaseMenu
 - MainWindow, 219
- updateStatsLabel
 - MainWindow, 219
- updateStatusBarInfo
 - MainWindow, 219
- updateVideoLengthLabel
 - VideoExportDialog, 300
- updateWelcomeScreenVisibility
 - MainWindow, 219
- updateXTicks
 - RideChartWidget, 265
- updateYAxis
 - RideChartWidget, 265
- updateYAxisForVisibleRange
 - RideChartWidget, 265
- updateZoneAvailability
 - MainWindow, 219
- updateZonesTab
 - MainWindow, 219
- upgradeSchema
 - DatabaseManager, 142
- upperPct
 - PowerZone, 251
- urlTemplate
 - TileProvider, 284
- useAdaptiveSmoothing
 - ClimbDetectorConfig, 111
- useScoringDetection
 - ClimbDetectorConfig, 111

- useSelectedSegment
 - VideoExportSettings, 307
- uuid
 - ClimbRecord, 122
 - IntervalRecord, 197
- valid
 - RideChartWidget, 271
- valueForMetric
 - anonymous_namespace{ZoneCalculator.cpp}, 36
- vam
 - Climb, 99
 - ClimbRecord, 122
- vamScoreBoost
 - ClimbDetectorConfig, 111
- variabilityIndex
 - TrainingMetricsSummary, 291
 - WorkoutController, 330
- VideoExportDialog, 296
 - ~VideoExportDialog, 298
 - cancelOrClose, 299
 - chooseOutputFile, 299
 - collectSettings, 299
 - m_altitudeCheck, 300
 - m_athleteCheck, 300
 - m_cadenceCheck, 300
 - m_cancelButton, 300
 - m_chartsCheck, 300
 - m_defaults, 300
 - m_deleteTemporaryTilesCheck, 300
 - m_entireActivityRadio, 300
 - m_etaLabel, 300
 - m_exportButton, 301
 - m_fixedZoomSpin, 301
 - m_fpsSpin, 301
 - m_frameLabel, 301
 - m_hrCheck, 301
 - m_job, 301
 - m_legendCheck, 301
 - m_mapStyleCombo, 301
 - m_outputEdit, 301
 - m_powerCheck, 301
 - m_progressBar, 301
 - m_resolutionCombo, 301
 - m_routeColorCombo, 302
 - m_running, 302
 - m_selectedSegmentRadio, 302
 - m_speedCheck, 302
 - m_speedSpin, 302
 - m_stageLabel, 302
 - m_telemetryPanelCheck, 302
 - m_thread, 302
 - m_tileStatsLabel, 302
 - m_timeCheck, 302
 - m_videoLengthLabel, 302
 - onFinished, 299
 - onFrameProgressChanged, 299
 - onProgressChanged, 299
 - onStageChanged, 299
 - onTileStatsChanged, 299
 - setAllOverlayChecks, 299
 - startExport, 300
 - updateVideoLengthLabel, 300
 - VideoExportDialog, 298
- VideoExportSettings, 303
 - activityName, 304
 - athleteName, 304
 - autoZoom, 304
 - colorContext, 305
 - deleteTemporaryTilesAfterExport, 305
 - ffmpegPath, 305
 - fixedZoom, 305
 - followAthlete, 305
 - fps, 305
 - height, 305
 - mapStyle, 305
 - outputPath, 305
 - overlayAltitude, 306
 - overlayAthleteName, 306
 - overlayCadence, 306
 - overlayCharts, 306
 - overlayHeartRate, 306
 - overlayPower, 306
 - overlayRouteLegend, 306
 - overlaySpeed, 306
 - overlayTelemetryPanel, 307
 - overlayTime, 307
 - playbackSpeed, 307
 - rideData, 307
 - routeColorMetric, 307
 - segmentEndSeconds, 307
 - segmentStartSeconds, 307
 - singleRouteColor, 307
 - useSelectedSegment, 307
 - width, 307
- VideoRenderJob, 308
 - cancel, 309
 - finished, 309
 - frameProgressChanged, 310
 - m_cancelRequested, 313
 - m_settings, 313
 - progressChanged, 310
 - run, 311
 - stageChanged, 312
 - tileStatsChanged, 312
 - VideoRenderJob, 309
- VideoTileId, 313
 - operator==, 314
 - x, 314
 - y, 314
 - z, 314
- VideoTileIdHash, 314
 - operator(), 314
- VideoTileProvider, 315
 - cleanupExportCache, 316
 - clearMemoryCache, 316
 - exportCacheRoot, 316

- getTile, [317](#)
- isTileCachedOnDisk, [317](#)
- m_deleteTemporaryTilesAfterExport, [319](#)
- m_downloadedTiles, [319](#)
- m_requiredTiles, [319](#)
- m_requiredTilesOnDisk, [319](#)
- m_tileCache, [319](#)
- requiredTileCount, [318](#)
- requiredTilesAvailableOnDisk, [318](#)
- setRequiredTiles, [318](#)
- shouldDeleteExportCacheAfterExport, [319](#)
- stats, [319](#)
- VideoTileProvider, [316](#)
- VideoTileProviderConfig, [320](#)
 - deleteTemporaryTilesAfterExport, [320](#)
 - exportCacheRoot, [320](#)
 - memoryCacheTiles, [320](#)
 - style, [320](#)
- VideoTileStats, [320](#)
 - downloadedTiles, [321](#)
 - onDiskTiles, [321](#)
 - requiredTiles, [321](#)
- visibleRangeChanged
 - RideChartWidget, [265](#)
- visibleRangeEndMinutes
 - RideChartWidget, [266](#)
- visibleRangeStartMinutes
 - RideChartWidget, [266](#)
- wattsPerKgRank
 - Climb, [99](#)
- weather
 - Activity, [47](#)
- weatherNotes
 - Activity, [47](#)
 - EditNotesDialog, [151](#)
- WeightEntry, [321](#)
 - athleteId, [322](#)
 - effectiveFrom, [322](#)
 - id, [322](#)
 - weightKg, [322](#)
- weightKg
 - Activity, [47](#)
 - WeightEntry, [322](#)
- WelcomeWidget, [322](#)
 - m_createDbButton, [324](#)
 - m_importButton, [324](#)
 - m_openDbButton, [324](#)
 - m_subtitleLabel, [325](#)
 - setFirstLaunch, [324](#)
 - WelcomeWidget, [323](#)
- wheelEvent
 - MapRenderer, [234](#)
 - RideChartWidget, [266](#)
- width
 - VideoExportSettings, [307](#)
- wind
 - Activity, [47](#)
- WorkoutAnalyzer, [325](#)
 - analyze, [325](#)
- WorkoutController, [326](#)
 - climbs, [328](#)
 - currentActivityId, [328](#)
 - currentAthleteId, [328](#)
 - efficiencyFactor, [328](#)
 - ftp, [328](#)
 - importFile, [328](#)
 - intensityFactor, [328](#)
 - intervals, [328](#)
 - loadActivity, [329](#)
 - loadFile, [329](#)
 - loadOrGenerateClimbs, [329](#)
 - loadOrGenerateIntervals, [329](#)
 - m_analysisQueue, [330](#)
 - m_analyzer, [330](#)
 - m_climbs, [330](#)
 - m_currentActivityId, [330](#)
 - m_currentAthleteId, [330](#)
 - m_dbManager, [331](#)
 - m_ef, [331](#)
 - m_ftp, [331](#)
 - m_if, [331](#)
 - m_intervals, [331](#)
 - m_np, [331](#)
 - m_pdcPoints, [331](#)
 - m_powerZones, [331](#)
 - m_rideData, [331](#)
 - m_statistics, [331](#)
 - m_tss, [331](#)
 - m_vi, [331](#)
 - normalizedPower, [329](#)
 - pdcPoints, [329](#)
 - powerZones, [329](#)
 - reanalyze, [329](#)
 - reloadClimbsFromDatabase, [329](#)
 - rideData, [329](#)
 - setAnalysisQueue, [329](#)
 - setCurrentAthlete, [330](#)
 - setDatabaseManager, [330](#)
 - setFTP, [330](#)
 - statistics, [330](#)
 - trainingStressScore, [330](#)
 - variabilityIndex, [330](#)
 - WorkoutController, [328](#)
- WorkoutStatistics, [332](#)
 - averageCadence, [332](#)
 - averageHeartRate, [332](#)
 - averagePower, [333](#)
 - averageSpeed, [333](#)
 - durationSeconds, [333](#)
 - elevationGainM, [333](#)
 - maximumCadence, [333](#)
 - maximumHeartRate, [333](#)
 - maximumPower, [333](#)
 - maximumSpeed, [333](#)
 - movingTimeSeconds, [334](#)
 - totalDistanceKm, [334](#)

- workThresholdPct
 - IntervalDetectorConfig, [191](#)
- x
 - TileCache::TileRequest, [284](#)
 - VideoTileId, [314](#)
- xRangeChanged
 - RideChartWidget, [266](#)
- y
 - TileCache::TileRequest, [284](#)
 - VideoTileId, [314](#)
- z
 - TileCache::TileRequest, [284](#)
 - VideoTileId, [314](#)
- Zone, [334](#)
 - color, [334](#)
 - maxValue, [334](#)
 - minValue, [334](#)
 - name, [335](#)
- zone
 - ZoneDistribution, [341](#)
- ZoneArray
 - FTPZones, [166](#)
- ZoneCalculator, [335](#)
 - altitudeZones, [335](#)
 - cadenceZones, [336](#)
 - computeDistribution, [336](#)
 - hasMetricSamples, [337](#)
 - heartRateZones, [337](#)
 - powerZones, [338](#)
 - speedZones, [338](#)
 - zonesForMetric, [339](#)
- ZoneDistribution, [340](#)
 - percent, [341](#)
 - seconds, [341](#)
 - zone, [341](#)
- zoneLegendText
 - anonymous_namespace{VideoRenderJob.cpp}, [34](#)
- zonesForMetric
 - ZoneCalculator, [339](#)
- zoom
 - MapFitMath::FitResult, [163](#)
- zoomIn
 - MapRenderer, [234](#)
- zoomOut
 - MapRenderer, [234](#)